

DNA十六元基索引系统_V5.0_完整源码 - 又名 -华瑞集商业测试版 15万行完整源码

对照-DNA元基催化与肽计算第6修订版_V00095 的源码论据 备份

写给我美好的一家人.

罗瑶光编著

目录

前言	42
第一部分-元基索引花函数	44
1-1-/Trif_DNA-master/C_L_A/A-分析-肽计算-欧拉环路-六元催化接口	44
A_I.MPE.EDVQMSAXQCTI	44
/Trif_DNA-master/C_L_A/A_I.MPE/EDVQMSAXQCTI/AOPM_VECS_IDUQ.java	44
A_I.MPE.MAQPVDCOISIV.full	44
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/full/	
FullDNATokenPDI_do_PDE_RNA_FullFormular_FullBack.java	44
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/full/FullDNATokenPDI.java	44
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/full/FullTokenPDI.java	64
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/full/FullTokenPDITest.java	77
A_I.MPE.MAQPVDCOISIV.initon	77
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/Initon.java	77
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonLinkDNA.java	84
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonLinkDNATest.java	85
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonsPDI.java	85
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonsPDITest.java	86
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonsPDM.java	86
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonsPDMTest.java	95
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonsPDS.java	95
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonsPDSTest.java	101
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/initon/InitonTest.java	101
A_I.MPE.MAQPVDCOISIV.pdc	101
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pdc/PDC.lyg	101
A_I.MPE.MAQPVDCOISIV.pde	101
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Decrement_Formular.java	101
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Decrement_FormularTest.java	109
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Decrement_FullDNAFormular.java	109
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Decrement_FullDNAFormularTest.java	116
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Decrement_FullFormular.java	116
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Decrement_FullFormularTest.java	122
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Formular.java	122
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_FormularTest.java	136
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_FormularWithKey.java	136
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_FormularWithKeyPress.java	150
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_FormularWithKeyTest.java	165
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Increment_Formular.java	165
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Increment_FormularTest.java	174
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Increment_FullDNAFormular.java	174
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Increment_FullDNAFormularTest.java	180
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Increment_FullFormular.java	180
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_Increment_FullFormularTest.java	188
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_RNA_Formular.java	188
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pde/PDE_RNA_FormularTest.java	202
A_I.MPE.MAQPVDCOISIV.pdi	202
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pdi/PowerPDI.java	202
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pdi/PowerPDITest.java	203
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pdi/RangePDI.java	203
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pdi/TokenPDI.java	219
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pdi/TokenPDITest.java	233
A_I.MPE.MAQPVDCOISIV.pdw	233
/Trif_DNA-master/C_L_A/A_I.MPE/MAQPVDCOISIV/pdw/PDW.txt	233
A_I.MPE.MSU.ouler	233
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/ouler/Q_OulerRing.java	233
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/ouler/Q_OulerRing16.java	241
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/ouler/Q_OulerRing16Test.java	250
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/ouler/Q_OulerRingTest.java	250
A_I.MPE.MSU.pde	250
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/pde/DecadeToPDS.java	250
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/pde/DecadeToPDS16.java	255
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/pde/DecadeToPDS16Test.java	260
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/pde/DecadeToPDSTest.java	260
A_I.MPE.MSU.pds	260
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/pds/PDE_PDS_DL.java	260
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/pds/PDE_PDS_DL16.java	273
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/pds/PDE_PDS_DL16Test.java	287
/Trif_DNA-master/C_L_A/A_I.MPE/MSU/pds/PDE_PDS_DLTest.java	287

A_I.MPE.P	287
/Trif_DNA-master/C_L_A/A_I/MPE/P/CopyLaw.java	287
/Trif_DNA-master/C_L_A/A_I/MPE/P/DatingLaw.java	289
/Trif_DNA-master/C_L_A/A_I/MPE/P/DNA.java	289
/Trif_DNA-master/C_L_A/A_I/MPE/P/Family.java	290
/Trif_DNA-master/C_L_A/A_I/MPE/P/InheritanceLaw.java	291
/Trif_DNA-master/C_L_A/A_I/MPE/P/Life.java	291
/Trif_DNA-master/C_L_A/A_I/MPE/P/MutationLaw.java	292
/Trif_DNA-master/C_L_A/A_I/MPE/P/PDN_CODE.java	293
/Trif_DNA-master/C_L_A/A_I/MPE/P/PDN.java	293
/Trif_DNA-master/C_L_A/A_I/MPE/P/Race.java	293
/Trif_DNA-master/C_L_A/A_I/MPE/P/RST.java	294
/Trif_DNA-master/C_L_A/A_I/MPE/P/SeparaterLaw.java	295
/Trif_DNA-master/C_L_A/A_I/MPE/P/SwapLaw.java	295
A_V.ASQ.PSU.test	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoCogs.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoEX2.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoEXTest.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoPOS.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoPOSforSpecial_UTF8.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoPOSMedicine.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/EffectDemo.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/MakeImage.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/S_Count.java	296
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/TimeCheck.java	296
A_V.ATIPDAXQDTIV	296
/Trif_DNA-master/C_L_A/A_V/ATIPDAXQDTIV/TaskMakeCoWorker.java	296
/Trif_DNA-master/C_L_A/A_V/ATIPDAXQDTIV/TaskMakeCoWorkerTest.java	297
A_V.C	297
/Trif_DNA-master/C_L_A/A_V/C/RatioMap.java	297
/Trif_DNA-master/C_L_A/A_V/C/RatioMapTest.java	298
A_V.cap	298
/Trif_DNA-master/C_L_A/A_V/cap/Cordination.java	298
/Trif_DNA-master/C_L_A/A_V/cap/test.java	298
/Trif_DNA-master/C_L_A/A_V/cap/test1.java	298
A_V.E	298
/Trif_DNA-master/C_L_A/A_V/E/RatioMap_E.java	298
/Trif_DNA-master/C_L_A/A_V/E/RatioMap_ETest.java	305
A_V.IAVPDAXQDT	305
/Trif_DNA-master/C_L_A/A_V/IAVPDAXQDT/PaserException.java	305
A_V.ica	305
/Trif_DNA-master/C_L_A/A_V/ica/EducationRatio.java	305
/Trif_DNA-master/C_L_A/A_V/ica/EducationRatioTest.java	306
/Trif_DNA-master/C_L_A/A_V/ica/InitBehaviorICAKernel.java	306
/Trif_DNA-master/C_L_A/A_V/ica/InitBehaviorICAKernelTest.java	312
A_V.IVDTIXQCTIV	312
/Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/EmotionInit.java	312
/Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/EmotionInitTest.java	313
/Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/EnvironmentInit.java	313
/Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/EnvironmentInitTest.java	316
/Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/LenovoInit.java	316
/Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/LenovoInitTest.java	317
1-2-/Trif_DNA-master/C_L_C-控制-分词-VPCS	317
C	317
/Trif_DNA-master/C_L_C/C/Verbal_C.java	317
/Trif_DNA-master/C_L_C/C/VerbalMap_C.java	317
C_A.ME.analysis.C	317
/Trif_DNA-master/C_L_C/C_A/ME/analysis/C/A.java	317
/Trif_DNA-master/C_L_C/C_A/ME/analysis/C/AC.java	318
/Trif_DNA-master/C_L_C/C_A/ME/analysis/C/BinaryForest_A.java	318
/Trif_DNA-master/C_L_C/C_A/ME/analysis/C/CogsBinaryForest_A.java	319
C_A.ME.euclid.C	319
/Trif_DNA-master/C_L_C/C_A/ME/euclid/C/Euclid_C.java	319
/Trif_DNA-master/C_L_C/C_A/ME/euclid/C/EuclidCTest.java	320
C_A.ME.liner.C	320
/Trif_DNA-master/C_L_C/C_A/ME/liner/C/Quick6DLuoYaoguangSort.java	320
/Trif_DNA-master/C_L_C/C_A/ME/liner/C/Quick6DLuoYaoguangSortTest.java	320
C_A.ME.nero.C	320
/Trif_DNA-master/C_L_C/C_A/ME/nero/C/NERO_C_OneTime.java	320
/Trif_DNA-master/C_L_C/C_A/ME/nero/C/NERO_C_OneTimeTest.java	321

/Trif_DNA-master/C_L_C/C_A/ME/nero/C/NERO_C.java	321
/Trif_DNA-master/C_L_C/C_A/ME/nero/C/NEROCTest.java	321
C_A.OCI.AMV.ECS.SVQ.MPC.fhmm.C	321
/Trif_DNA-master/C_L_C/C_A/OCI/AMV/ECS/SVQ/MPC/fhmm/C/SensingMap.java	321
C_A.OCI.AVC.SUQ.SVQ.MPC.fhmm.C	322
/Trif_DNA-master/C_L_C/C_A/OCI/AVC/SUQ/SVQ/MPC/fhmm/C/EmotionMap.java	322
/Trif_DNA-master/C_L_C/C_A/OCI/AVC/SUQ/SVQ/MPC/fhmm/C/EmotionMapTest.java	323
C_A.OCI.OPM.VEC.SVQ.MPC.fhmm.C	323
/Trif_DNA-master/C_L_C/C_A/OCI/OPM/VEC/SVQ/MPC/fhmm/C/BehaviorMap.java	323
/Trif_DNA-master/C_L_C/C_A/OCI/OPM/VEC/SVQ/MPC/fhmm/C/BehaviorMapTest.java	323
C_A.OCI.SVQ.MPC.fhmm.C	323
/Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FHMMList_indexAll_X.java	323
/Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FHMMList_indexAll.java	323
/Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FHMMList.java	324
/Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FHMMListTest.java	326
/Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FHMMList.java	326
C_A.OSI.AOP.MS.VPC.S.hall	327
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/S/hall/DatabaseLog_H.java	327
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/S/hall/DatabaseLog_HTest.java	329
C_A.OSI.AOP.MS.VPC.server	329
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/BootVPCSBackEnd.java	329
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/BootVPCSBackEndTest.java	333
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestFilter_C.java	333
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestFilterCTest.java	335
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestFix_C.java	335
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestFixCTest.java	335
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestRecord_C.java	335
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestRecordCTest.java	337
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/Response_C.java	337
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ResponseCTest.java	338
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerForward_Standard.java	338
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerForward_StandardTest.java	343
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerInit_C_VPCSFrontEnd.java	343
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerInit_C_VPCSFrontEndTest.java	345
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerInit_C.java	345
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerInitCTest.java	349
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerInit_Standard.java	349
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerInit_StandardTest.java	351
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerRestMap_Standard.java	351
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerRestMap_StandardTest.java	358
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerSleeper_Standard.java	358
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerSleeper_StandardTest.java	360
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerVPC_Standard.java	360
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerVPC_StandardTest.java	364
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/VPCSRequest.java	364
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/VPCSRequestTest.java	364
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/VPCSResponse.java	364
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/VPCSResponseTest.java	364
1-3-/Trif_DNA-master/C_L_D	364
DSU.constants	364
/Trif_DNA-master/C_L_D/DSU/constants/DetaDBConstant.java	364
DSU.exceptions	365
/Trif_DNA-master/C_L_D/DSU/exceptions/DetaDBException.java	365
/Trif_DNA-master/C_L_D/DSU/exceptions/DetaDBExceptionTest.java	365
/Trif_DNA-master/C_L_D/DSU/exceptions/ErrorCodeEnum.java	365
/Trif_DNA-master/C_L_D/DSU/exceptions/RequestLimitException.java	366
DSU.V	366
**/Trif_DNA-master/C_L_D/DSU/V/VtoV.java	366
/Trif_DNA-master/C_L_D/DSU/V/VtoVTest.java	367
exceptions	367
/Trif_DNA-master/C_L_D/exceptions/DetaDBException.java	367
/Trif_DNA-master/C_L_D/exceptions/ErrorCodeEnum.java	367
/Trif_DNA-master/C_L_D/exceptions/RequestLimitException.java	367
1-4-/Trif_DNA-master/C_L_E-执行-分词-数据库-ETL-界面UI-卷积-网络传输	368
E_A.ME.analysis.E	368
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE_M.java	368
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE_MTest.java	372
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE_X.java	372
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE_XTest.java	378
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE.java	378

/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AETest.java.....	381
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AS.java.....	381
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/BinaryForest_AE.java.....	382
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/BinaryForest_AETest.java.....	384
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/CogsBinaryForest_AE.java.....	384
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/CogsBinaryForest_AETest.java.....	391
E_A.ME.euclid.E.....	391
/Trif_DNA-master/C_L_E/E_A/ME/euclid/E/Euclid_CE.java.....	391
/Trif_DNA-master/C_L_E/E_A/ME/euclid/E/Euclid_CETest.java.....	393
E_A.ME.liner.E.....	393
/Trif_DNA-master/C_L_E/E_A/ME/liner/E/Quick6DLuoYaoguangSort_E.java.....	393
/Trif_DNA-master/C_L_E/E_A/ME/liner/E/Quick6DLuoYaoguangSort_ETest.java.....	395
/Trif_DNA-master/C_L_E/E_A/ME/liner/E/Quick6DLuoYaoguangSort3DMap_E.java.....	395
/Trif_DNA-master/C_L_E/E_A/ME/liner/E/Quick6DLuoYaoguangSort3DMap_ETest.java.....	397
E_A.ME.nero.E.....	397
/Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_C_OneTime_E.java.....	397
/Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_C_OneTime_ES.java.....	398
/Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_C_OneTime_ETest.java.....	399
/Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_CE.java.....	399
/Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_CETest.java.....	400
E_A.OEI.AMV.ECS.SVQ.MPC.SQ.OEM.E.....	400
/Trif_DNA-master/C_L_E/E_A/OEI/AMV/ECS/SVQ/MPC/SQ/OEM/E/SensingMap_E.java.....	400
/Trif_DNA-master/C_L_E/E_A/OEI/AMV/ECS/SVQ/MPC/SQ/OEM/E/SensingMap_ETest.java.....	401
E_A.OEI.AVC.SUQ.SVQ.MPC.fhmm.E.....	401
/Trif_DNA-master/C_L_E/E_A/OEI/AVC/SUQ/SVQ/MPC/fhmm/E/EmotionMap_E.java.....	401
/Trif_DNA-master/C_L_E/E_A/OEI/AVC/SUQ/SVQ/MPC/fhmm/E/EmotionMap_ETest.java.....	404
E_A.OEI.OPM.VEC.SVQ.MPC.fhmm.E.....	404
/Trif_DNA-master/C_L_E/E_A/OEI/OPM/VEC/SVQ/MPC/fhmm/E/BehaviorMap_E.java.....	404
/Trif_DNA-master/C_L_E/E_A/OEI/OPM/VEC/SVQ/MPC/fhmm/E/BehaviorMap_ETest.java.....	404
E_A.OEI.OSQ.SVQ.DU.CM.E.....	405
/Trif_DNA-master/C_L_E/E_A/OEI/OSQ/SVQ/DU/CM/E/Verbal_CE.java.....	405
/Trif_DNA-master/C_L_E/E_A/OEI/OSQ/SVQ/DU/CM/E/VerbalMap_CE.java.....	405
E_A.OEI.SVQ.MPC.fhmm.E.....	405
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FastReadProjectFile.java.....	405
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FastReadProjectFileTest.java.....	407
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMList_E.java.....	407
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMList_ETest.java.....	413
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMListOneTime_E_maps.java.....	413
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMListOneTime_E_mapsTest.java.....	430
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMListOneTime_E.java.....	430
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMListOneTime_EX.java.....	461
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/PillowsSet.java.....	491
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/PillowsSetTest.java.....	495
E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.company.E.....	495
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/company/E/LoginService_E.java.....	495
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/company/E/LoginService_ETest.java.....	495
E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.factory.E.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/factory/E/LoginDAO_E.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/factory/E/LoginDAO_ETest.java.....	496
E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.port.E.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/Rest_C_Port_E.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/Rest_C_Port_ETest.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_DE.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_DETest.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_IUE.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_IUETest.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_QE.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_QETest.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_UE.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_UETest.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDBConfig_E.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDBConfig_ETest.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDBPLSQL_E.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDBPLSQL_ETest.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestLoginPort_E.java.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestLoginPort_ETest.java.....	496
E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.project.E.....	496
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/project/E/LoginProject_E.java.....	496
E_A.org.tinos.deta.hooks.main.....	496
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/ExceptInformation.java.....	496

/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/ExceptInformationTest.java	497
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Hook_C.java	497
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Hook_CTest.java	497
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Hook_E.java	497
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Hook_ETest.java	500
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Test.java	500
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/TestTest.java	500
E_A.OSI.AOP.PCS.PP.company_E	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/company_E/LoginServiceImpl.java	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/company_E/LoginServiceImplTest.java	500
E_A.OSI.AOP.PCS.PP.factory_E	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/factory_E/LoginDAOImpl.java	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/factory_E/LoginDAOImplTest.java	500
E_A.OSI.AOP.PCS.PP.port_E	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/port_E/RestLoginPortImpl.java	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/port_E/RestLoginPortImplTest.java	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/port_E/RestNLPPortImpl.java	500
E_A.OSI.AOP.VPC.PP.company_E	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/company_E/LoginService_E.java	500
E_A.OSI.AOP.VPC.PP.factory_E	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/factory_E/LoginDAO_E.java	500
E_A.OSI.AOP.VPC.PP.port_E	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/port_E/RestAskPort_E.java	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/port_E/RestAskPort_ETest.java	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/port_E/RestLoginPort_E.java	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/port_E/RestLoginPort_ETest.java	500
E_A.OSI.AOP.VPC.PP.project_E	500
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/project_E/LoginProject_E.java	500
OPE.MSQ.OVU.PQE.nodeInfo	500
/Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeInfo/NodeInfo.java	500
/Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeInfo/NodeInfoTest.java	501
OPE.MSQ.OVU.PQE.nodeProject	501
/Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeProject/NodeProject.java	501
/Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeProject/NodeProjectTest.java	502
/Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeProject/etl.png	502
OPE.MVU.OVU.OVU.PQE.nodeEdit.C	502
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/CheckRange.java	502
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/ChooseCheck.java	503
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/DrawArrow.java	503
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/DynamicLine_U.java	503
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/Sort.java	504
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/U_RelatedLine.java	504
OPE.OEQ.MCQ.OVU.PQE.osgi	504
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGI.java	504
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGITest.java	505
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/NodeOSGI.java	505
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/NodeOSGITest.java	506
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/OSU_AOU_AOQ_VES.java	506
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/OSU_MCI_MCU_MSI.java	506
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/OSU_OPE_OPC_ECI.java	508
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/OSU_OVQ_OSQ_VSQ.java	508
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGI.pde	509
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGI.pds	509
/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGI.tvm	509
OPE.OVQ.MSQ.OVU.PQE.platForm	509
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornJSplitPane.java	509
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornJSplitPaneTest.java	510
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornScrollBarUI.java	510
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornScrollBarUITest.java	513
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornSplitPaneUI.java	513
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornSplitPaneUITest.java	519
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornTreeCellRenderer.java	519
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornTreeCellRendererTest.java	520
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornTreeUI.java	520
/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornTreeUITest.java	524
OPE.OVU.MVQ.OVU.PQE.nodeView	524
/Trif_DNA-master/C_L_E/OPE/OVU/MVQ/OVU/PQE/nodeView/CacuString.java	524
/Trif_DNA-master/C_L_E/OPE/OVU/MVQ/OVU/PQE/nodeView/CacuStringTest.java	525
/Trif_DNA-master/C_L_E/OPE/OVU/MVQ/OVU/PQE/nodeView/NodeShow.java	525
/Trif_DNA-master/C_L_E/OPE/OVU/MVQ/OVU/PQE/nodeView/NodeShowTest.java	527

OPE.OVU.MVU.OVU.PQE.nodeEdit	528
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/CheckRangeVPS.java	528
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/CheckRangeVPSTest.java	529
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/ChooseQ_VPS.java	529
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/ChooseQ_VPSTest.java	530
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawArrowVPS.java	530
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawArrowVPSTest.java	531
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawFlashSide.java	531
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawFlashSideTest.java	532
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawSinLineVPS.java	532
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawSinLineVPSTest.java	534
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DynamicLineU_VPS.java	534
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DynamicLineU_VPSTest.java	535
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/LinkList.java	535
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/LinkListTest.java	538
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/LinkNode.java	538
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/LinkNodeTest.java	539
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/Sort.java	539
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/SortTest.java	540
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/U_RelatedLineVPS.java	540
/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/U_RelatedLineVPSTest.java	541
OPE.SI.MCI.OEI.OVU.PQE.extOSGI	541
/Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/PQE/extOSGI/OSGI_chansfer.java	541
/Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/PQE/extOSGI/OSGI_findNode.java	542
/Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/PQE/extOSGI/OSGI_findNodeTest.java	543
/Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/PQE/extOSGI/	
OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register.java	543
/Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/PQE/extOSGI/	
OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_registerTest.java	550
/Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/PQE/extOSGI/	
OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_OSI_PCI.pde	550
/Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/PQE/extOSGI/	
OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_OSI_PCI.tvm	550
resources	550
/Trif_DNA-master/C_L_E/resources/ipBlock ips	550
1-5-/Trif_DNA-master/C_L_I-增加-三维词汇花-卷积-VPCS-网络-编辑	550
DVIAQVIMIV	550
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering_17.java	550
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering_17Test.java	552
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering_AC_17.java	552
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering_AC_17Test.java	554
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering.java	554
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDecipheringTest.java	556
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDesiphline_17.java	556
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDesiphline_17Test.java	558
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDesiphline.java	558
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDesiphlineTest.java	559
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoPDEofSkin.java	559
/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoPDEofSkinTest.java	561
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetColorRatioScore.java	561
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetColorRatioScoreTest.java	564
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetICAStatisticRatio.java	564
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetICAStatisticRatioTest.java	567
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetRatioScoreSort.java	567
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetShapeRatioScore.java	567
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetSkinBy.java	567
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetSkinByTest.java	567
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetStatisticCount.java	567
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetStatisticCountTest.java	569
/Trif_DNA-master/C_L_I/DVIAQVIMIV/GetStatisticRatio.java	569
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinDetect.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinDetectTest.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetect.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectDis.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectDisTest.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectString.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectStringTest.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTest.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTrip_E.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTrip_ETest.java	572

/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTrip.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTripTest.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTrip.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2D_E.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2D_ETest.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2D.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2DTest.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2Test.java	572
/Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripTest.java	572
I_A.IMO.EVU.bi_P	572
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/BigWord.java	572
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/BigWordTest.java	572
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/Classify.java	572
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/ClassifyTest.java	572
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthor.java	572
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWord_XCDX_M.java	580
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWord_XCDX_MTest.java	596
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWord_XCDX_S.java	596
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWord_XCDX.java	599
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWord_XCDXTest.java	604
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWord.java	604
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWordTest.java	616
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorTest.java	616
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoDNAAuthorForWord.java	616
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoDNAAuthorForWordTest.java	627
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/Histogram.java	627
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/HistogramTest.java	627
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/LineMed.java	627
/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/LineMedTest.java	627
I_A.IMS.EUQ.recoder	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/Jpv.java	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/JpvTest.java	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/Recoder.java	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/RecoderTest.java	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/VoicejPanel.java	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/VoicejPanelTest.java	628
I_A.IMS.EUQ.SP	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/BootLine.java	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/BootLineTest.java	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ComputingLine.java	628
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ComputingLineTest.java	630
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/LYGFileIO.java	630
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/LYGFileIOTest.java	640
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ReadingLine.java	640
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ReadingLineTest.java	643
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/Recoder.java	643
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ShowingLine.java	643
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ShowingLineTest.java	643
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundPlay.java	643
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundPlaying.java	654
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundPlayingTest.java	664
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundPlayTest.java	664
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWave.java	664
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveJPanel.java	666
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveJPanelNew.java	666
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveJPanelNewTest.java	672
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveJPanelTest.java	672
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveJPanelTime.java	672
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveJPanelTimeTest.java	678
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWavePanelVector.java	678
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWavePanelVectorEraser.java	679
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWavePanelVectorNew.java	679
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWavePanelVectorNewTest.java	684
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveVector.java	684
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveVectorNew.java	684
/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveVectorNewTest.java	687
I_Q.hvpcs	687
/Trif_DNA-master/C_L_I/I_Q/hvpcs/说明.txt	687
I_Q.hvpcs.boot.controller	687

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/ConfigController.java	687
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/DBCategoryController.java	687
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/DeleteController.java	687
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/InsertController.java	687
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/SelectController.java	687
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/UpdateController.java	687
I_Q.hvpcs.boot.rest	687
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/rest/LocalRestCall.java	687
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/rest/VPC.java	692
I_Q.hvpcs.boot.server	695
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/server/BootVPCS.java	695
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/server/BootVPCSTest.java	696
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/server/BootVPCSTest2.java	696
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/server/dsd.java	696
I_Q.hvpcs.boot.sets	696
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/BlobLimit.java	696
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/Request_S.java	697
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/Response_S.java	699
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/Sleeper_S.java	702
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/Stable.java	703
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/StableBlob.java	703
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/StableSets.java	704
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/VPCSRequest.java	705
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/VPCSResponse.java	706
I_Q.hvpcs.boot.vpc.controller	709
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/RequestFilterController.java	709
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/RequestFixController.java	709
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/RequestRecordController.java	710
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/RequestRecordControllerBUp.java	713
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/ResponseController.java	713
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/ServerInitController.java	713
I_Q.hvpcs.boot.vpc.process	715
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/process/CacheProcess.java	715
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/process/JsonProcess.java	715
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/process/ThirdPartyProcess.java	715
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/process/TimeProcess.java	715
I_Q.hvpcs.boot.vpc.sleeper	715
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/Sleeper_O.java	715
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/Sleeper_X.java	716
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/Sleeper.java	718
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/SleeperHall.java	722
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/TimeHall.java	724
I_Q.hvpcs.boot.vpc.vision	725
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/ForwardVision.java	725
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_buffer.java	730
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_bufferBytes.java	732
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_bytes.java	734
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_bytesZip.java	736
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_rest.java	739
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision.java	741
I_Q.hvpcs.cache	749
/Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/Cache.java	749
**JDK-- /Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/CacheManager.java	750
/Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/DetaCache.java	754
/Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/DetaCacheManager.java	755
/Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/test.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/TimeCheck.java	756
I_Q.hvpcs.DetaSrc	756
I_Q.hvpcs.html5.upload	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/code.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/DoubleListBytes.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploader.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimple.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleBackUp.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleDNA.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocket.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocketPDE.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocketSound.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocketSoundImage.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocketSoundImageMpeg.java	756

/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderUnwrite.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/SoundWaveVectorMp3.java	756
I_Q.hvpcs.lyg.common	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/KeyStore.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/LYGPasswordDecode.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/NeedLogin.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/POSTREST.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/RequestLimit.java	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/WriteData.java	756
I_Q.hvpcs.lyg.common.constants	756
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/constants/DetaDBConstant.java	756
I_Q.hvpcs.lyg.common.exceptions	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/exceptions/DetaDBException.java	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/exceptions/ErrorCodeEnum.java	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/exceptions/RequestLimitException.java	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/exceptions/RestExceptionHandler.java	757
I_Q.hvpcs.lyg.common.maps	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/maps/VtoV.java	757
I_Q.hvpcs.lyg.common.utils	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/utils/DateUtil.java	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/utils/DetaDBUtil.java	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/utils/DetaUtil.java	757
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/utils/GzipUtil.java	757
I_Q.hvpcs.lyg.vpc.process.companyImpl	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/companyImpl/LoginServiceImpl.java	758
I_Q.hvpcs.lyg.vpc.process.factoryImpl	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/factoryImpl/LoginDAOImpl.java	758
I_Q.hvpcs.lyg.vpc.process.portImpl	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/portImpl/RestAskPortImpl.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/portImpl/RestLoginPortImpl.java	758
I_Q.hvpcs.lyg.vpc.process.projectImpl	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/projectImpl/LoginProjectImpl.java	758
I_Q.hvpcs.lyg.vpc.transaction	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/transaction/TransactionDelegate.java	758
I_Q.hvpcs.lyg.vpc.view	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/Branch.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/Talk.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/Token.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/Usr.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/UsrFull.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/UsrToken.java	758
I_Q.hvpcs.pixes.process	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/pixes/process/ColorProcess.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/pixes/process/DistributeProcess.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/pixes/process/RangeProcess.java	758
/Trif_DNA-master/C_L_I/I_Q/hvpcs/pixes/process/RatioProcess.java	758
I_Q.hvpcs.src.java.org.lyg	759
1-6-/Trif_DNA-master/C_L_M_管理-报表-打印-网络-UI	759
M_V.AVQ.PVU.vedio	759
/Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/PlayerControl.java	759
/Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/PlayerControlTest.java	765
/Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/PlayerShow.java	765
/Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/PlayerShowTest.java	766
/Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/ThreadPanel.java	766
/Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/ThreadPanelTest.java	769
M_V.ME.APM.VSQ.editPane	769
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane_X.java	769
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane_X_OS.java	792
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane_X_OSTest.java	799
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane_X_Test.java	799
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane.java	799
** /Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/PrintTest4.java	806
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/PrintTest4Test.java	808
M_V.ME.APM.VSQ.ESI.SVQ.search	808
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/ESI/SVQ/search/CaiJPanel.java	808
M_V.ME.APM.VSQ.xlsx	809
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlsx/CoAuthorForWord_A.java	809
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlsx/CoAuthorForWord_ATest.java	811
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlsx/CoAuthorForWord_Q.java	811
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlsx/CoAuthorForWord_QTest.java	815

/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_V.java	815
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_V1.java	822
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_VTest.java	822
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_X_maps.java	822
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_X_mapsTest.java	839
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_X_Slider.java	839
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_X_SliderTest.java	864
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord.java	864
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWordTest.java	866
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CommonRotClass.java	866
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/GetCompassList.java	866
/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/GetCompassListTest.java	867
M_V.ME.OVQ.OSQ.VSQ.obj	867
/Trif_DNA-master/C_L_M/M_V/ME/OVQ/OSQ/VSQ/obj/WordInfo.java	867
M_V.ME.SM.OP.SM.AOP.MEC.SIQ.E	867
/Trif_DNA-master/C_L_M/M_V/ME/SM/OP/SM/AOP/MEC/SIQ/E/E_PLSQL_E.java	867
/Trif_DNA-master/C_L_M/M_V/ME/SM/OP/SM/AOP/MEC/SIQ/E/E_PLSQL_ETest.java	869
/Trif_DNA-master/C_L_M/M_V/ME/SM/OP/SM/AOP/MEC/SIQ/E/P_AggregationPLSQL.java	869
/Trif_DNA-master/C_L_M/M_V/ME/SM/OP/SM/AOP/MEC/SIQ/E/P_AggregationPLSQLTest.java	870
M_V.MPI.xls.commonXLS	870
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXls.java	870
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXLSASC.java	874
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXLSASCTest.java	874
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXLSDDNN.java	874
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXLSDDNNTest.java	876
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXlsPage_X_S_s.java	876
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXlsPage_X_S_sListeners.java	876
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXlsPage_X_S_sListenersTest.java	881
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXlsTest.java	881
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/MakeContainerXLS.java	881
/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/MakeContainerXLSTest.java	884
M_V.MPI.xls.unixUtil	884
/Trif_DNA-master/C_L_M/M_V/MPI/xls/unixUtil/FileEncoding.java	884
M_V.MS.OP.SM.AOP.MEC.SIQ.cache	884
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaCache.java	884
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaCacheTest.java	884
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaDBBufferCache.java	884
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaDBBufferCacheTest.java	884
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/TimeCheck.java	884
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/TimeCheckTest.java	884
M_V.MS.VPC.PP	884
/Trif_DNA-master/C_L_M/M_V/MS/VPC/PP/Time_P.java	884
M_V.MS.VPC.SH	884
/Trif_DNA-master/C_L_M/M_V/MS/VPC/SH/Sleeper_H.java	884
/Trif_DNA-master/C_L_M/M_V/MS/VPC/SH/Sleeper_HTest.java	884
/Trif_DNA-master/C_L_M/M_V/MS/VPC/SH/Sleeper.java	884
/Trif_DNA-master/C_L_M/M_V/MS/VPC/SH/SleeperTest.java	885
M_V.MSI.OP.SM.AOP.MEC.SIQ.SM.OSI.E	885
/Trif_DNA-master/C_L_M/M_V/MSI/OP/SM/AOP/MEC/SIQ/SM/OSI/E/I_Tables_E.java	885
/Trif_DNA-master/C_L_M/M_V/MSI/OP/SM/AOP/MEC/SIQ/SM/OSI/E/I_Tables_ETest.java	886
M_V.MSU.OCI.ME.SMS.translator.C	886
/Trif_DNA-master/C_L_M/M_V/MSU/OCI/ME/SMS/translator/C/Translator.java	886
/Trif_DNA-master/C_L_M/M_V/MSU/OCI/ME/SMS/translator/C/TranslatorTest.java	887
M_V.MSU.OEI.ME.SMS.SEU.OSD.OSI.E	887
/Trif_DNA-master/C_L_M/M_V/MSU/OEI/ME/SMS/SEU/OSD/OSI/E/Translator_E.java	887
/Trif_DNA-master/C_L_M/M_V/MSU/OEI/ME/SMS/SEU/OSD/OSI/E/Translator_ETest.java	894
M_V.MSV.OSQ.sets	894
/Trif_DNA-master/C_L_M/M_V/MSV/OSQ/sets/DetaDicimal.java	894
/Trif_DNA-master/C_L_M/M_V/MSV/OSQ/sets/DetaDouble.java	894
/Trif_DNA-master/C_L_M/M_V/MSV/OSQ/sets/DetaFloat.java	894
/Trif_DNA-master/C_L_M/M_V/MSV/OSQ/sets/DetaLong.java	894
M_V.MSV.OSU.array	894
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/array/ArrayValidation.java	894
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/array/ArrayValidationTest.java	896
M_V.MSV.OSU.date	896
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/date/DateValidation.java	896
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/date/DateValidationTest.java	897
M_V.MSV.OSU.list	897
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/list/ListValidation.java	897
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/list/ListValidationTest.java	900

M_V.MSV.OSU.matrix.....	900
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/matrix/MatrixValidation.java	900
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/matrix/MatrixValidationTest.java.....	903
M_V.MSV.OSU.string	903
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/string/NullObject.java	903
/Trif_DNA-master/C_L_M/M_V/MSV/OSU/string/NullObjectTest.java.....	904
M_V.MVQ.button.....	904
/Trif_DNA-master/C_L_M/M_V/MVQ/button/CfxButton.java	904
/Trif_DNA-master/C_L_M/M_V/MVQ/button/CfxButtonTest.java.....	907
/Trif_DNA-master/C_L_M/M_V/MVQ/button/DetaButton.java	907
/Trif_DNA-master/C_L_M/M_V/MVQ/button/DetaButtonTest.java	910
/Trif_DNA-master/C_L_M/M_V/MVQ/button/HrjButton.java	910
/Trif_DNA-master/C_L_M/M_V/MVQ/button/HrjButtonTest.java	913
/Trif_DNA-master/C_L_M/M_V/MVQ/button/YLJShowButton.java	913
/Trif_DNA-master/C_L_M/M_V/MVQ/button/YLJShowButtonTest.java.....	917
M_V.MVQ.filenameFilter	917
/Trif_DNA-master/C_L_M/M_V/MVQ/filenameFilter/TXTFilter.java.....	917
M_V.MVQ.label	917
/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaIndexLabel.java	917
/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaLabel.java	926
/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaLabelConjunction.java	929
/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaLabelConjunctionTest.java.....	931
/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaLabelTest.java	931
M_V.MVQ.slider.....	931
/Trif_DNA-master/C_L_M/M_V/MVQ/slider/DetaSlider.java	931
M_V.MVQ.tabbedPane.....	931
/Trif_DNA-master/C_L_M/M_V/MVQ/tabbedPane/DetabbedPane.java	931
/Trif_DNA-master/C_L_M/M_V/MVQ/tabbedPane/DetabbedPaneTest.java.....	933
M_V.MVQ.tableRender.....	933
/Trif_DNA-master/C_L_M/M_V/MVQ/tableRender/ColorTableRender.java.....	933
/Trif_DNA-master/C_L_M/M_V/MVQ/tableRender/ColorTableRenderTest.java	934
M_V.MVQ.textarea.....	934
/Trif_DNA-master/C_L_M/M_V/MVQ/textarea/CfxTextArea.java	934
/Trif_DNA-master/C_L_M/M_V/MVQ/textarea/CfxTextAreaTest.java	936
M_V.MVQ.textfield.....	937
/Trif_DNA-master/C_L_M/M_V/MVQ/textfield/CfxTextField.java	937
/Trif_DNA-master/C_L_M/M_V/MVQ/textfield/CfxTextFieldTest.java	939
M_V.MVQ.textpane	940
/Trif_DNA-master/C_L_M/M_V/MVQ/textpane/CfxTextPane.java	940
1-7-/Trif_DNA-master/C_L_O-操作-tinshell-数据库-VPCS-文档	940
O_V.OM.config	940
/Trif_DNA-master/C_L_O/O_V/OM/config/Config.java.....	940
O_V.OP.SM.AOP.MEC.SIQ.SM.reflection.....	940
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Base.java.....	940
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/BaseTest.java	941
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Cell.java	941
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/DB.java.....	941
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/DBTest.java	942
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Row.java	942
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/RowTest.java	943
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Spec.java.....	943
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/SpecTest.java	944
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Table.java.....	944
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/TableTest.java	945
O_V.OP.SM.AOP.MEC.SIQ.VPC.view	945
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/VPC/view/Branch.java.....	945
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/VPC/view/UsrFull.java.....	947
O_V.OPE.SVU.OSU.dct	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/Demo.java.....	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/DemoTest.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/Jpm.java.....	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/JpmTest.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/YaoGuangLuoFDT.java.....	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/YaoGuangLuoFDTTest.java	949
O_V.OPE.SVU.OSU.dft.....	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Demo.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/DemoTest.java.....	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Fftframe.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/FftframeTest.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp.java	949

/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp1.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp1Test.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp2.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp2Test.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp3.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp3Test.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp4.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp4Test.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/JpTest.java	949
O_V.OPE.SVU.OSU.fft	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/fft/Demo.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/fft/DemoTest.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/fft/Jpm.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/fft/JpmTest.java	949
O_V.OPE.SVU.OSU.frft	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/frft/Demo.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/frft/DemoTest.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/frft/Jpm.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/frft/JpmTest.java	949
O_V.OPE.SVU.OSU.ifft	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/ifft/Demo.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/ifft/DemoTest.java	949
/Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/ifft/Jpm.java	949
O_V.OPM.ESU.admin	949
/Trif_DNA-master/C_L_O/O_V/OPM/ESU/admin/DBReadMode.java	949
/Trif_DNA-master/C_L_O/O_V/OPM/ESU/admin/DBReadModeTest.java	952
/Trif_DNA-master/C_L_O/O_V/OPM/ESU/admin/PLSQLSectionPanel.java	952
/Trif_DNA-master/C_L_O/O_V/OPM/ESU/admin/VPCSRestPanel.java	961
/Trif_DNA-master/C_L_O/O_V/OPM/ESU/admin/YouBiaoSectionPanel.java	966
O_V.org.tinos.listen	991
/Trif_DNA-master/C_L_O/O_V/org/tinos/listen/Hear.java	991
/Trif_DNA-master/C_L_O/O_V/org/tinos/listen/HearTest.java	994
/Trif_DNA-master/C_L_O/O_V/org/tinos/listen/ReadToWav.java	994
/Trif_DNA-master/C_L_O/O_V/org/tinos/listen/ReadToWavTest.java	997
O_V.OSA.shell	997
/Trif_DNA-master/C_L_O/O_V/OSA/shell/Const.java	997
/Trif_DNA-master/C_L_O/O_V/OSA/shell/Create.java	998
/Trif_DNA-master/C_L_O/O_V/OSA/shell/CreateTest.java	999
/Trif_DNA-master/C_L_O/O_V/OSA/shell/PL_XA_C.java	999
/Trif_DNA-master/C_L_O/O_V/OSA/shell/PL_XA_E.java	1001
/Trif_DNA-master/C_L_O/O_V/OSA/shell/PL_XA_ETest.java	1008
/Trif_DNA-master/C_L_O/O_V/OSA/shell/TestDemo.java	1008
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellQ_JoinRows_E.java	1008
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellQ_JoinRows_ETest.java	1011
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellQ_NestRows_E.java	1011
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellQ_NestRows_ETest.java	1013
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellQ_Rows_E.java	1013
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellQ_Rows_ETest.java	1019
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellTable.java	1019
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellTables.java	1020
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellTablesTest.java	1024
/Trif_DNA-master/C_L_O/O_V/OSA/shell/XA_ShellTableTest.java	1024
O_V.OSI.AOP.freetts.thread.read	1024
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/freetts/thread/read/ReadEnglish.java	1024
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/freetts/thread/read/ReadEnglishTest.java	1024
O_V.OSI.AOP.MEC.SIQ.pletl	1024
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/pletl/Const.java	1024
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/pletl/PLETL_C.java	1024
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/pletl/PLETL_E.java	1024
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/pletl/PLETL_ETest.java	1025
O_V.OSI.AOP.MEC.SIQ.plorm	1025
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/plorm/Const.java	1025
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/plorm/Create.java	1027
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/plorm/CreateTest.java	1028
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/plorm/PLORM_C.java	1028
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/plorm/PLORM_E.java	1029
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/MEC/SIQ/plorm/PLORM_ETest.java	1035
O_V.OSI.AOP.neo.tts	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/neo/tts/ReadChinese.java	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/neo/tts/ReadChineseTest.java	1035

O_V.OSI.AOP.PCS.view	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/Branch.java	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/MrTin.java	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/Token.java	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/Usr.java	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/UsrFull.java	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/PCS/view/UsrToken.java	1035
O_V.OSI.AOP.rest.medicine	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/rest/medicine/RestMedicinePort_E.java	1035
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/rest/medicine/RestMedicinePort_ETest.java	1051
O_V.OSI.AOP.VPC.rest	1051
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPC.java	1051
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPCBackEnd.java	1056
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPCBackEndTest.java	1058
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPCTest.java	1058
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPCYangliaojing.java	1058
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/rest/VPCYangliaojingTest.java	1061
O_V.OSI.AOP.VPC.server	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/BootVPCS.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/BootVPCSFrontEnd.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/BootVPCSFrontEndTest.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/BootVPCSTest.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/RestServer.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/server/RestServerTest.java	1061
O_V.OSI.AOP.VPC.view	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/Branch.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/Talk.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/Token.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/Usr.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/UsrFull.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/UsrFullTest.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/UsrToken.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/AOP/VPC/view/UsrTokenTest.java	1061
O_V.OSI.MSQ.OPE.SIU.read	1061
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadNodeInterface.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadOSU_MSQ_OPE_OPC_ECI.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/config.xml	1061
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/LUO.jpg	1061
/Trif_DNA-master/C_L_O/O_V/OSI/MSQ/OPE/SIU/read/ReadNode.py	1061
O_V.OSI.OSU.AVU.OSQ.wavePlay	1061
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/Jp.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/WavePlayNodeInterface.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/ WavePlayOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/ WavePlayOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/WavePlayOSU_MSQ_OPE_OPC_ECI.java	1061
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/config.xml	1061
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/play.jpg	1061
/Trif_DNA-master/C_L_O/O_V/OSI/OSU/AVU/OSQ/wavePlay/wavePlayNode.py	1061
O_V.OSI.PVI.PEI.PCI.PSI.sikname	1061
/Trif_DNA-master/C_L_O/O_V/OSI/PVI/PEI/PCI/PSI/sikname/SikMap.java	1061
O_V.OSM.shell	1061
/Trif_DNA-master/C_L_O/O_V/OSM/shell/CommandClass.java	1061
/Trif_DNA-master/C_L_O/O_V/OSM/shell/CommandClassBackUp.java	1079
/Trif_DNA-master/C_L_O/O_V/OSM/shell/E_pl_XA_E_2025.java	1079
/Trif_DNA-master/C_L_O/O_V/OSM/shell/E_pl_XA_E_2025Test.java	1079
/Trif_DNA-master/C_L_O/O_V/OSM/shell/E_pl_XA_E.java	1079
/Trif_DNA-master/C_L_O/O_V/OSM/shell/E_pl_XA_ETest.java	1091
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_AO_pl_XA.java	1091
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_AO_pl_XATest.java	1102
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_AO_PLETL.java	1102
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_AO_PLETLTest.java	1110
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_AO_PLTCP.java	1110
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_AO_PLTCPTest.java	1115
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_I_CulumnsPL_XA.java	1115
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_I_CulumnsPL_XATest.java	1116
/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_RelationPL_XA.java	1116

/Trif_DNA-master/C_L_O/O_V/OSM/shell/P_RelationPL_XATest.java	1120
/Trif_DNA-master/C_L_O/O_V/OSM/shell/Pl_XA_Command_E.java	1120
/Trif_DNA-master/C_L_O/O_V/OSM/shell/Pl_XA_Command_ETest.java	1136
/Trif_DNA-master/C_L_O/O_V/OSM/shell/ShellReplace.java	1136
/Trif_DNA-master/C_L_O/O_V/OSM/shell/ShellReplaceTest.java	1144
/Trif_DNA-master/C_L_O/O_V/OSM/shell/SortStringDemo.java	1144
/Trif_DNA-master/C_L_O/O_V/OSM/shell/SortStringDemoTest.java	1147
/Trif_DNA-master/C_L_O/O_V/OSM/shell/rep.txt	1147
OPE.MSI.OEI.SOI.SMQ.save	1147
/Trif_DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/S_java	1147
/Trif_DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/SaveAnd_U_File.java	1147
/Trif_DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/SaveAnd_U_FileTest.java	1150
/Trif_DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/SaveAs_I_File.java	1150
/Trif_DNA-master/C_L_O/OPE/MSI/OEI/SOI/SMQ/save/SaveAs_I_FileTest.java	1158
OPE.PSQ.OEU.SOI.SMQ.neroCell	1158
/Trif_DNA-master/C_L_O/OPE/PSQ/OEU/SOI/SMQ/neroCell/BootNeroCell.java	1158
/Trif_DNA-master/C_L_O/OPE/PSQ/OEU/SOI/SMQ/neroCell/BootNeroCellTest.java	1161
/Trif_DNA-master/C_L_O/OPE/PSQ/OEU/SOI/SMQ/neroCell/BootNeroDoc.java	1161
/Trif_DNA-master/C_L_O/OPE/PSQ/OEU/SOI/SMQ/neroCell/BootNeroDocTest.java	1162
OPE.VPC.S.H_Keeper	1162
/Trif_DNA-master/C_L_O/OPE/VPC/S/H_Keeper/H_Keeper.java	1162
/Trif_DNA-master/C_L_O/OPE/VPC/S/H_Keeper/H_KeeperTest.java	1163
OPE.VPC.S.sets	1163
/Trif_DNA-master/C_L_O/OPE/VPC/S/sets/Sets.java	1163
/Trif_DNA-master/C_L_O/OPE/VPC/S/sets/SetsTest.java	1163
OPE.VPC.S.skivvy	1163
/Trif_DNA-master/C_L_O/OPE/VPC/S/skivvy/Skivvy.java	1163
/Trif_DNA-master/C_L_O/OPE/VPC/S/skivvy/SkivvyTest.java	1163
OPE.VPC.S.V	1163
/Trif_DNA-master/C_L_O/OPE/VPC/S/V/Pillow.java	1163
/Trif_DNA-master/C_L_O/OPE/VPC/S/V/PillowTest.java	1163
/Trif_DNA-master/C_L_O/OPE/VPC/S/V/V.java	1163
1-8-/Trif_DNA-master/C_L_P-处理-坐标-概率-催化-深度计算等,	1164
P_V.PCI.ASQ.basic	1164
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/Compute_S.java	1164
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/Compute_STest.java	1164
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/Ratio.java	1164
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/RatioMatrix.java	1164
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/RatioMatrixTest.java	1165
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/basic/RatioTest.java	1165
P_V.PCI.ASQ.demension	1165
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/AMV_MVS_VSQ_2D.java	1165
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/AMV_MVS_VSQ_2DTest.java	1166
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/AMV_MVS_VSQ_3D.java	1166
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/AMV_MVS_VSQ_3DTest.java	1167
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/Line2D.java	1167
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/Line2DTest.java	1168
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/Line3D.java	1168
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/Line3DTest.java	1169
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/QuarSide2D.java	1169
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/QuarSide2DTest.java	1170
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/QuarSide3D.java	1170
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/demension/QuarSide3DTest.java	1170
P_V.PCI.ASQ.desolation	1171
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/desolation/ErrorAsserts.java	1171
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/desolation/ErrorAssertsTest.java	1172
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/desolation/ErrorCaculations.java	1172
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/desolation/ErrorCaculationsTest.java	1172
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/desolation/ErrorRatio.java	1172
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/desolation/ErrorRatioTest.java	1172
P_V.PCI.ASQ.image	1172
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/image/ImagePixClassification.java	1172
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/image/ImagePixClassificationTest.java	1173
P_V.PCI.ASQ.issueDate	1173
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/issueDate/IssueDate.java	1173
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/issueDate/IssueDateTest.java	1174
P_V.PCI.ASQ.pca	1174
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/pca/PCAMeanOfFuzzPC.java	1174
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/pca/PCAMeanOfFuzzPCTest.java	1175
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/pca/PCAUtils.java	1175

P_V.PCI.ASQ.probability	1175
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/probability/GetListOfArrayRatio.java	1175
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/probability/GetListOfArrayRatioTest.java	1176
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/probability/Percentage.java	1176
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/probability/PercentageTest.java	1177
P_V.PCI.ASQ.ratio	1177
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/ratio/DistanceRatio.java	1177
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/ratio/DistanceRatioTest.java	1181
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/ratio/IncrementRatio.java	1181
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/ratio/IncrementRatioTest.java	1182
P_V.PCI.ASQ.score	1182
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/score/ProbabilityScore.java	1182
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/score/ProbabilityScoreTest.java	1184
P_V.PCI.ASQ.statistic	1184
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindIncrement.java	1184
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindIncrementTest.java	1184
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindMean.java	1184
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindMeanTest.java	1187
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindSum.java	1187
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/FindSumTest.java	1189
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/LYG9DWithDoubleQuickSort4D.java	1189
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/LYG9DWithDoubleQuickSort4DTest.java	1190
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/ShareholdCount.java	1190
/Trif_DNA-master/C_L_P/P_V/PCI/ASQ/statistic/ShareholdCountTest.java	1191
P_V.PCS.common	1191
/Trif_DNA-master/C_L_P/P_V/PCS/common/KeyStore.java	1191
/Trif_DNA-master/C_L_P/P_V/PCS/common/LYGPasswordDecode.java	1191
/Trif_DNA-master/C_L_P/P_V/PCS/common/LYGPasswordDecodeTest.java	1191
/Trif_DNA-master/C_L_P/P_V/PCS/common/NeedLogin.java	1191
/Trif_DNA-master/C_L_P/P_V/PCS/common/POSTREST.java	1191
/Trif_DNA-master/C_L_P/P_V/PCS/common/POSTRESTTest.java	1191
/Trif_DNA-master/C_L_P/P_V/PCS/common/WriteData.java	1191
P_V.PCS.OEC.SIU.test	1191
/Trif_DNA-master/C_L_P/P_V/PCS/OEC/SIU/test/RestPressTest.java	1191
/Trif_DNA-master/C_L_P/P_V/PCS/OEC/SIU/test/RestPressTestTest.java	1191
P_V.PCS.thread	1191
/Trif_DNA-master/C_L_P/P_V/PCS/thread/SocketThread.java	1191
/Trif_DNA-master/C_L_P/P_V/PCS/thread/SocketThreadPool.java	1195
/Trif_DNA-master/C_L_P/P_V/PCS/thread/SocketThreadPoolTest.java	1196
/Trif_DNA-master/C_L_P/P_V/PCS/thread/SocketThreadTest.java	1196
P_V.PCS.transaction	1196
/Trif_DNA-master/C_L_P/P_V/PCS/transaction/DelegateLogin_X_transactionLogin.java	1196
/Trif_DNA-master/C_L_P/P_V/PCS/transaction/DelegateLogin_X_transactionLoginTest.java	1197
/Trif_DNA-master/C_L_P/P_V/PCS/transaction/DelegateLogin_X_transactionRegisterDB_DNA.java	1197
/Trif_DNA-master/C_L_P/P_V/PCS/transaction/DelegateLogin_X_transactionRegisterDB_DNATest.java	1198
P_V.PEQ.AMV.ECS.test	1198
/Trif_DNA-master/C_L_P/P_V/PEQ/AMV/ECS/test/ANNTTest.java	1198
/Trif_DNA-master/C_L_P/P_V/PEQ/AMV/ECS/test/DNNTTest.java	1201
/Trif_DNA-master/C_L_P/P_V/PEQ/AMV/ECS/test/RNN_IDETest.java	1203
/Trif_DNA-master/C_L_P/P_V/PEQ/AMV/ECS/test/SensingTest.java	1208
P_V.PEQ.OPM.VEC.test	1212
/Trif_DNA-master/C_L_P/P_V/PEQ/OPM/VEC/test/EducationLevelTest.java	1212
/Trif_DNA-master/C_L_P/P_V/PEQ/OPM/VEC/test/LiterarinessLevelTest.java	1213
/Trif_DNA-master/C_L_P/P_V/PEQ/OPM/VEC/test/SuccessICATest.java	1214
P_V.PEQ.test	1216
/Trif_DNA-master/C_L_P/P_V/PEQ/test/DemoTSLT.java	1216
1-9-/Trif_DNA-master/C_L_Q-查看-催化-搜索-数据库等,	1218
Q_A.analysis	1218
/Trif_DNA-master/C_L_Q/Q_A/analysis/Creative_A_Issues.java	1218
/Trif_DNA-master/C_L_Q/Q_A/analysis/Help_A_Issues.java	1218
/Trif_DNA-master/C_L_Q/Q_A/analysis/Love_A_Issues.java	1219
/Trif_DNA-master/C_L_Q/Q_A/analysis/Safe_A_Issues.java	1220
/Trif_DNA-master/C_L_Q/Q_A/analysis/Study_A_Issues.java	1220
/Trif_DNA-master/C_L_Q/Q_A/analysis/Work_A_Issues.java	1221
Q_A.life	1222
/Trif_DNA-master/C_L_Q/Q_A/life/Application.java	1222
/Trif_DNA-master/C_L_Q/Q_A/life/Application24.java	1222
/Trif_DNA-master/C_L_Q/Q_A/life/CreativeIssues.java	1228
/Trif_DNA-master/C_L_Q/Q_A/life/HelpIssues.java	1229
/Trif_DNA-master/C_L_Q/Q_A/life/LoveIssues.java	1229

/Trif_DNA-master/C_L_Q/Q_A/life/PDNTTest.java	1230
/Trif_DNA-master/C_L_Q/Q_A/life/SafeIssues.java	1231
/Trif_DNA-master/C_L_Q/Q_A/life/StudyIssues.java	1232
/Trif_DNA-master/C_L_Q/Q_A/life/WorkIssues.java	1232
Q_A.management	1233
/Trif_DNA-master/C_L_Q/Q_A/management/Creative_M_Issues.java	1233
/Trif_DNA-master/C_L_Q/Q_A/management/Help_M_Issues.java	1234
/Trif_DNA-master/C_L_Q/Q_A/management/Love_M_Issues.java	1235
/Trif_DNA-master/C_L_Q/Q_A/management/Safe_M_Issues.java	1235
/Trif_DNA-master/C_L_Q/Q_A/management/Study_M_Issues.java	1236
/Trif_DNA-master/C_L_Q/Q_A/management/Work_M_Issues.java	1236
Q_A.operation	1237
/Trif_DNA-master/C_L_Q/Q_A/operation/Creative_O_Issues.java	1237
/Trif_DNA-master/C_L_Q/Q_A/operation/Help_O_Issues.java	1238
/Trif_DNA-master/C_L_Q/Q_A/operation/Love_O_Issues.java	1238
/Trif_DNA-master/C_L_Q/Q_A/operation/Safe_O_Issues.java	1239
/Trif_DNA-master/C_L_Q/Q_A/operation/Study_O_Issues.java	1240
/Trif_DNA-master/C_L_Q/Q_A/operation/Work_O_Issues.java	1240
Q_A.OSQ.disk	1241
/Trif_DNA-master/C_L_Q/Q_A/OSQ/disk/GetDisk.java	1241
/Trif_DNA-master/C_L_Q/Q_A/OSQ/disk/GetDiskTest.java	1241
Q_A.pdeWords	1241
/Trif_DNA-master/C_L_Q/Q_A/pdeWords/Iden.java	1241
/Trif_DNA-master/C_L_Q/Q_A/pdeWords/IdenTest.java	1241
/Trif_DNA-master/C_L_Q/Q_A/pdeWords/PDCtoPDW.java	1241
Q_A.PP	1241
/Trif_DNA-master/C_L_Q/Q_A/PP/Creative_P_Issues.java	1241
/Trif_DNA-master/C_L_Q/Q_A/PP/Help_P_Issues.java	1241
/Trif_DNA-master/C_L_Q/Q_A/PP/Love_P_Issues.java	1242
/Trif_DNA-master/C_L_Q/Q_A/PP/Safe_P_Issues.java	1242
/Trif_DNA-master/C_L_Q/Q_A/PP/Study_P_Issues.java	1243
/Trif_DNA-master/C_L_Q/Q_A/PP/Work_P_Issues.java	1243
Q_V.OSI.AOP.MEC.SIQ.search	1244
/Trif_DNA-master/C_L_Q/Q_V/OSI/AOP/MEC/SIQ/search/ZhongYaoSearch.java	1244
/Trif_DNA-master/C_L_Q/Q_V/OSI/AOP/MEC/SIQ/search/ZhongYaoSearchBackUp.java	1254
/Trif_DNA-master/C_L_Q/Q_V/OSI/AOP/MEC/SIQ/search/ZhongYaoSearchTest.java	1254
Q_V.OSU.PEQ.AVC.SUQ.test	1254
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/EmotionTest.java	1254
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/EmotionTestTest.java	1254
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/EnvironmentTest.java	1254
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/EnvironmentTestTest.java	1254
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/LenovoTest.java	1254
/Trif_DNA-master/C_L_Q/Q_V/OSU/PEQ/AVC/SUQ/test/LenovoTestTest.java	1254
Q_V.OSV.VCQ.standard	1255
/Trif_DNA-master/C_L_Q/Q_V/OSV/VCQ/standard/DictionaryPLSQLStandardDB.java	1255
/Trif_DNA-master/C_L_Q/Q_V/OSV/VCQ/standard/DictionaryPLSQLStandardDBTest.java	1256
/Trif_DNA-master/C_L_Q/Q_V/OSV/VCQ/standard/DictionaryStandardDB.java	1256
/Trif_DNA-master/C_L_Q/Q_V/OSV/VCQ/standard/DictionaryStandardDBTest.java	1259
Q_V.QSD.VQS.MSO	1259
/Trif_DNA-master/C_L_Q/Q_V/QSD/VQS/MSO/Q_parser.java	1259
1-10-/Trif_DNA-master/C_L_S/S_综合标记-分词--元基花-变量归纳等,	1259
S_A.AVQ.OVQ.OSQ.VSQ.obj	1259
/Trif_DNA-master/C_L_S/S_A/AVQ/OVQ/OSQ/VSQ/obj/FMHMMNode.java	1259
/Trif_DNA-master/C_L_S/S_A/AVQ/OVQ/OSQ/VSQ/obj/FMHMMPOS.java	1260
/Trif_DNA-master/C_L_S/S_A/AVQ/OVQ/OSQ/VSQ/obj/HMMNode.java	1260
/Trif_DNA-master/C_L_S/S_A/AVQ/OVQ/OSQ/VSQ/obj/StringOBJ.java	1260
/Trif_DNA-master/C_L_S/S_A/AVQ/OVQ/OSQ/VSQ/obj/Verbal.java	1260
/Trif_DNA-master/C_L_S/S_A/AVQ/OVQ/OSQ/VSQ/obj/WordFrequency.java	1262
S_A.ESU.code.stable	1263
/Trif_DNA-master/C_L_S/S_A/ESU/code/stable/S_.java	1263
S_A.ESU.MVS.view.stable	1265
/Trif_DNA-master/C_L_S/S_A/ESU/MVS/view/stable/S_Data_ESU.java	1265
S_A.golgi	1266
/Trif_DNA-master/C_L_S/S_A/golgi/SimilarC.java	1266
S_A.linePage	1266
/Trif_DNA-master/C_L_S/S_A/linePage/BalanceTree.java	1266
/Trif_DNA-master/C_L_S/S_A/linePage/CP.java	1266
/Trif_DNA-master/C_L_S/S_A/linePage/FFdit.java	1266
/Trif_DNA-master/C_L_S/S_A/linePage/Line2025.java	1266
/Trif_DNA-master/C_L_S/S_A/linePage/PageP_M.java	1273

/Trif_DNA-master/C_L_S/S_A/linePage/PageQ_E.java.....	1280
/Trif_DNA-master/C_L_S/S_A/linePage/PageQ_S.java.....	1288
/Trif_DNA-master/C_L_S/S_A/linePage/PageQ.java.....	1289
/Trif_DNA-master/C_L_S/S_A/linePage/PageS.java.....	1292
S_A.OSI.SQI.....	1292
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/EPS.java.....	1292
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/2021年4月26日微信公开发布的关于肽株笔记描述.jpg.....	1292
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/20241009.txt.....	1292
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/README.txt.....	1292
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/S.initon.....	1292
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/Screenshot_2021-05-26-18-30-24-50.jpg.....	1292
/Trif_DNA-master/C_L_S/S_A/OSI/SQI/第三卷的心得1.lyg.....	1292
S_A.OSV.ESD.standard.....	1292
/Trif_DNA-master/C_L_S/S_A/OSV/ESD/standard/D_CellStandard.java.....	1292
/Trif_DNA-master/C_L_S/S_A/OSV/ESD/standard/D_Standard.java.....	1294
S_A.OSV.ESI.standard.....	1295
/Trif_DNA-master/C_L_S/S_A/OSV/ESI/standard/IU_CellStandard.java.....	1295
S_A.OSV.ESU.standard.....	1297
/Trif_DNA-master/C_L_S/S_A/OSV/ESU/standard/U_CellStandard.java.....	1297
S_A.pheromone.....	1300
/Trif_DNA-master/C_L_S/S_A/pheromone/AES_QMS_XSD_TIH.java.....	1300
/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQI.java.....	1300
/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQI_S.java.....	1301
/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQI_SA.java.....	1301
/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQI_SS.java.....	1301
/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQI_utils.java.....	1302
/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQI_X.java.....	1305
/Trif_DNA-master/C_L_S/S_A/pheromone/IMV_SQI.java.....	1306
/Trif_DNA-master/C_L_S/S_A/pheromone/MSI_EVD_SVX_TVS_DAH.java.....	1313
S_A.SEM.bloom.....	1314
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/CallFunctionKey.java.....	1314
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticClassMap.java.....	1318
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMap.java.....	1323
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapA_IDUQ_C.java.....	1332
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapA_IDUQ_E.java.....	1335
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapA_VECS_C.java.....	1336
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapA_VECS_E.java.....	1338
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapC_AOPM_C.java.....	1342
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapC_AOPM_E.java.....	1344
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapC_IDUQ_C.java.....	1349
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapC_IDUQ_E.java.....	1349
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapCASE.java.....	1349
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapD_AOPM_C.java.....	1350
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapD_AOPM_E.java.....	1350
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapD_VECS_C.java.....	1350
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapD_VECS_E.java.....	1350
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapE_AOPM_C.java.....	1350
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapE_AOPM_E.java.....	1351
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapE_IDUQ_C.java.....	1354
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapE_IDUQ_E.java.....	1356
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapI_AOPM_C.java.....	1360
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapI_AOPM_E.java.....	1361
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapI_VECS_C.java.....	1363
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapI_VECS_E.java.....	1364
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapM_IDUQ_C.java.....	1368
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapM_IDUQ_E.java.....	1369
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapM_VECS_C.java.....	1370
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapM_VECS_E.java.....	1373
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapO_IDUQ_C.java.....	1388
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapO_IDUQ_E.java.....	1388
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapO_VECS_C.java.....	1390
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapO_VECS_E.java.....	1394
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapP_IDUQ_C.java.....	1410
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapP_IDUQ_E.java.....	1411
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapP_VECS_C.java.....	1413
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapP_VECS_E.java.....	1419
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapQ_AOPM_C.java.....	1445
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapQ_AOPM_E.java.....	1445
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapQ_VECS_C.java.....	1445
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapQ_VECS_E.java.....	1446

/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapS_AOPM_C.java	1455
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapS_AOPM_E.java	1457
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapS_IDUQ_C.java	1460
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapS_IDUQ_E.java	1460
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapU_AOPM_C.java	1461
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapU_AOPM_E.java	1466
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapU_VECS_C.java	1485
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapU_VECS_E.java	1490
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapV_AOPM_C.java	1508
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapV_AOPM_E.java	1511
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapV_IDUQ_C.java	1518
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticFunctionMapV_IDUQ_E.java	1518
/Trif_DNA-master/C_L_S/S_A/SEM/bloom/StaticRootMap.java	1523
S_A.SEM.reflection	1544
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapA_IDUQ_C.java	1544
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapA_VECS_C.java	1546
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapC_AOPM_C.java	1548
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapC_IDUQ_C.java	1555
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapCASE.java	1555
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapD_AOPM_C.java	1556
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapD_VECS_C.java	1556
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapE_AOPM_C.java	1557
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapE_IDUQ_C.java	1562
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapI_AOPM_C.java	1569
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapI_VECS_C.java	1572
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapM_IDUQ_C.java	1576
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapM_VECS_C.java	1578
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapO_IDUQ_C.java	1596
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapO_VECS_C.java	1599
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapP_IDUQ_C.java	1618
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapP_VECS_C.java	1621
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapQ_AOPM_C.java	1650
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapQ_VECS_C.java	1650
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapS_AOPM_C.java	1651
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapS_IDUQ_C.java	1654
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapU_AOPM_C.java	1655
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapU_VECS_C.java	1677
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapV_AOPM_C.java	1706
/Trif_DNA-master/C_L_S/S_A/SEM/reflection/StaticReflectionMapV_IDUQ_C.java	1712
S_A.SEM.register	1714
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterA_IDUQ_C.java	1714
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterA_VECS_C.java	1715
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterC_AOPM_C.java	1716
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterC_IDUQ_C.java	1718
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterD_AOPM_C.java	1719
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterD_VECS_C.java	1719
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterE_AOPM_C.java	1720
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterE_IDUQ_C.java	1721
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterI_AOPM_C.java	1723
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterI_VECS_C.java	1724
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterM_IDUQ_C.java	1725
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterM_VECS_C.java	1726
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterO_IDUQ_C.java	1731
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterO_VECS_C.java	1732
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterP_IDUQ_C.java	1736
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterP_VECS_C.java	1737
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterQ_AOPM_C.java	1741
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterQ_VECS_C.java	1742
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterS_AOPM_C.java	1743
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterS_IDUQ_C.java	1744
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterU_AOPM_C.java	1745
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterU_VECS_C.java	1750
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterV_AOPM_C.java	1755
/Trif_DNA-master/C_L_S/S_A/SEM/register/StaticRegisterV_IDUQ_C.java	1757
S_A.SixActionMap	1758
/Trif_DNA-master/C_L_S/S_A/SixActionMap/CreativeVerbalMap.java	1758
/Trif_DNA-master/C_L_S/S_A/SixActionMap/CreativeVerbalMapTest.java	1764
/Trif_DNA-master/C_L_S/S_A/SixActionMap/FlowerAction.java	1764
/Trif_DNA-master/C_L_S/S_A/SixActionMap/FlowerActionTest.java	1769
/Trif_DNA-master/C_L_S/S_A/SixActionMap/HelpVerbalMap.java	1769

/Trif_DNA-master/C_L_S/S_A/SixActionMap/LoveVerbalMap.java	1770
/Trif_DNA-master/C_L_S/S_A/SixActionMap/SafeVerbalMap.java	1770
/Trif_DNA-master/C_L_S/S_A/SixActionMap/StudyVerbalMap_Q.java	1770
/Trif_DNA-master/C_L_S/S_A/SixActionMap/StudyVerbalMap.java	1795
/Trif_DNA-master/C_L_S/S_A/SixActionMap/StudyVerbalMapBU.java	1806
/Trif_DNA-master/C_L_S/S_A/SixActionMap/StudyVerbalMapTest.java	1806
/Trif_DNA-master/C_L_S/S_A/SixActionMap/WorkVerbalMap_X_S.java	1806
/Trif_DNA-master/C_L_S/S_A/SixActionMap/WorkVerbalMap_X.java	1814
/Trif_DNA-master/C_L_S/S_A/SixActionMap/WorkVerbalMap.java	1823
/Trif_DNA-master/C_L_S/S_A/SixActionMap/WorkVerbalMapTest.java	1830
/Trif_DNA-master/C_L_S/S_A/SixActionMap/studyVerbalMap.outp	1830
/Trif_DNA-master/C_L_S/S_A/SixActionMap/test.txt	1830
S_A.Stable.bloom	1830
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_A_IDUQ.java	1830
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_A_VECS.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_C_AOPM.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_C_IDUQ.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_D_AOPM.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_D_VECS.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_E_AOPM.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_E_IDUQ.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_I_AOPM.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_I_VECS.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_M_IDUQ.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_M_VECS.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_O_IDUQ.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_O_VECS.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_P_IDUQ.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_P_VECS.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_Q_AOPM.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_Q_VECS.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_S_AOPM.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_S_IDUQ.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_U_AOPM.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_U_VECS.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_V_AOPM.java	1831
/Trif_DNA-master/C_L_S/S_A/Stable/bloom/S_V_IDUQ.java	1831
S_A.SVQ.stable	1831
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Annotation.java	1831
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Colors.java	1832
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Common.java	1833
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_File.java	1833
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_HTTP.java	1839
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Maps.java	1840
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_MapsInitons.java	1840
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_ShellETL.java	1847
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_String.java	1848
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Tag.java	1852
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S-Token.java	1853
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Vision.java	1854
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/S_Web.java	1855
/Trif_DNA-master/C_L_S/S_A/SVQ/stable/DNA.pde	1858
S_A.TRNA.RRNA	1858
/Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/DmBolism.java	1858
/Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/I_U.java	1859
/Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/MtBolism.java	1859
/Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/Q_I.java	1863
/Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/S_Q.java	1867
/Trif_DNA-master/C_L_S/S_A/TRNA/RRNA/U_Q.java	1871
S_A.util	1874
/Trif_DNA-master/C_L_S/S_A/util/CanvasPane.java	1874
/Trif_DNA-master/C_L_S/S_A/util/MovingPane.java	1876
/Trif_DNA-master/C_L_S/S_A/util/OptionalPane.java	1878
S_A.VSQ.etl	1880
/Trif_DNA-master/C_L_S/S_A/VSQ/etl/NodeOSGI_S.java	1880
/Trif_DNA-master/C_L_S/S_A/VSQ/etl/OSU_AVQ_ASQ_AOI_AOD_AOU_AOQ_VES_S.java	1880
/Trif_DNA-master/C_L_S/S_A/VSQ/etl/OSU_AVQ_ASQ_OVQ_OSQ_VSQ_S.java	1881
/Trif_DNA-master/C_L_S/S_A/VSQ/etl/OSU_AVQ_ASQ_OPE_OPC_ECI_S.java	1882
/Trif_DNA-master/C_L_S/S_A/VSQ/etl/OSU_MCI_MCU_MSI_S.java	1882
S_A.VSQ.parser	1884

/Trif_DNA-master/C_L_S/S_A/VSQ/parser/EmotionMap_ES.java	1884
/Trif_DNA-master/C_L_S/S_A/VSQ/parser/EmotionSample.java	1885
S_I.OSI.OSU.VSQ.OSU.MSQ.VQ.SQ.lygPlayer	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/BufferReader.java.....	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/Jp.java	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LYGPlayerNodeInterface.java	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/ LYGPlayerOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java.....	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/ LYGPlayerOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java.....	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LYGPlayerOSU_MSQ_OPE_OPC_ECI.java 1887	
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/Pm.java.....	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/Ps.java	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/config.xml	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LUO.jpg	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygPlayer/LYGReadNode.py	1887
S_I.OSI.OSU.VSQ.OSU.MSQ.VQ.SQ.lygRead	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LYGReadNodeInterface.java	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/ LYGReadOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java.....	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/ LYGReadOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java.....	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LYGReadOSU_MSQ_OPE_OPC_ECI.java.... 1887	
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/config.xml	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LUO.jpg	1887
/Trif_DNA-master/C_L_S/S_I/OSI/OSU/VSQ/OSU/MSQ/VQ/SQ/lygRead/LYGReadNode.py	1887
S_I.OSI.OVI.OSU.MSQ.MV.SQ.imageRead	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/ImageReadNodeInterface.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/ ImageReadOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/ ImageReadOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/ImageReadOSU_MSQ_OPE_OPC_ECI.java..... 1888	
/Trif_DNA-master/C_L_S/S_I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/imageread.jpg.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVI/OSU/MSQ/MV/SQ/imageRead/imageReadNode.py	1888
S_I.OSI.OVQ.OSU.MSQ.MV.SQ.findColorB.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorB/FindColorBInterface.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorB/FindColorBPanel.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorB/FindColorBRun.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorB/FindColorBView.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorB/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorB/emboss.jpg	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorB/EmbossFilterNode.py.....	1888
S_I.OSI.OVQ.OSU.MSQ.MV.SQ.findColorG	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorG/FindColorGInterface.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorG/FindColorGPanel.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorG/FindColorGRun.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorG/FindColorGView.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorG/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorG/emboss.jpg	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorG/EmbossFilterNode.py	1888
S_I.OSI.OVQ.OSU.MSQ.MV.SQ.findColorR.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorR/FindColorRInterface.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorR/FindColorRPanel.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorR/FindColorRRun.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorR/FindColorRView.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorR/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorR/emboss.jpg	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/findColorR/EmbossFilterNode.py.....	1888
S_I.OSI.OVQ.OSU.MSQ.MV.SQ.show3D	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/show3D/Show3DInterface.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/show3D/Show3DPanel.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/show3D/Show3DRun.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/show3D/Show3DView.java.....	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/show3D/test.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/show3D/3Dshow.jpg	1888

/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/show3D/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVQ/OSU/MSQ/MV/SQ/show3D/Show3DNode.py	1888
S_I.OSI.OVU.OSU.MSQ.MV.SQ.embossFilter	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterInterface.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterPanel.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterRun.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterView.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/emboss.jpg	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/embossFilter/EmbossFilterNode.py	1888
S_I.OSI.OVU.OSU.MSQ.MV.SQ.grayFilter	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/GrayFilterNodeInterface.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/ GrayFilterOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/ GrayFilterOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/GrayFilterOSU_MSQ_OPE_OPC_ECI.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/GrayFilterNode.py	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/grayFilter/strench.jpg	1888
S_I.OSI.OVU.OSU.MSQ.MV.SQ.guassianFilter	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/guassianFilter/GuassianFilterInterface.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/guassianFilter/GuassianFilterPanel.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/guassianFilter/GuassianFilterRun.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/guassianFilter/GuassianFilterView.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/guassianFilter/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/guassianFilter/gaosi.jpg	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/guassianFilter/GuassianFilterNode.py	1888
S_I.OSI.OVU.OSU.MSQ.MV.SQ.houghTransform	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformInterface.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformPanel.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformRun.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformView.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/HoughTransformNode.py	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/houghTransform/hutans.jpg	1888
S_I.OSI.OVU.OSU.MSQ.MV.SQ.imageStrech	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/imageStrech/ImageStrechNodeInterface.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/imageStrech/ ImageStrechOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/imageStrech/ ImageStrechOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/imageStrech/ImageStrechOSU_MSQ_OPE_OPC_ECI.java	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/imageStrech/config.xml	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/imageStrech/imageStrechNode.py	1888
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/imageStrech/strench.jpg	1888
S_I.OSI.OVU.OSU.MSQ.MV.SQ.laplacianFilter	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterInterface.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterPanel.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterRun.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterView.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/config.xml	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/LaplacianFilterNode.py	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/laplacianFilter/laplas.jpg	1889
S_I.OSI.OVU.OSU.MSQ.MV.SQ.medianImageFilter	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/MedianImageNodeInterface.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/ MedianImageOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/ MedianImageOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/ MedianImageOSU_MSQ_OPE_OPC_ECI.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/config.xml	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/MedianImageNode.py	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/medianImageFilter/strench.jpg	1889
S_I.OSI.OVU.OSU.MSQ.MV.SQ.morphologyFilter	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/MorphologyFilterInterface.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/MorphologyFilterPanel.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/MorphologyFilterRun.java	1889

/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/MorphologyFilterView.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/config.xml	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/GrayFilterNode.py	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/morphologyFilter/strench.jpg	1889
S_I.OSI.OVU.OSU.MSQ.MV.SQ.sobelFilter	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/SobelFilterNodeInterface.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/ SobelFilterOSU_MSQ_AOI_AOD_AOU_AOQ_VES.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/ SobelFilterOSU_MSQ_AVQ_ASQ_OVQ_OSQ_VSQ.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/SobelFilterOSU_MSQ_OPE_OPC_ECI.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/config.xml	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/sobel.jpg	1889
/Trif_DNA-master/C_L_S/S_I/OSI/OVU/OSU/MSQ/MV/SQ/sobelFilter/SobelFilterNode.py	1889
S_I.OSI.PEI.PCI.PSI.tinShell	1889
/Trif_DNA-master/C_L_S/S_I/OSI/PEI/PCI/PSI/tinShell/ AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI.java	1889
/Trif_DNA-master/C_L_S/S_I/OSI/PEI/PCI/PSI/tinShell/AddTinShellPanel.java	1891
/Trif_DNA-master/C_L_S/S_I/OSI/PEI/PCI/PSI/tinShell/AddTinShellRun.java	1891
/Trif_DNA-master/C_L_S/S_I/OSI/PEI/PCI/PSI/tinShell/AddTinShellView.java	1895
/Trif_DNA-master/C_L_S/S_I/OSI/PEI/PCI/PSI/tinShell/TinMap.java	1896
/Trif_DNA-master/C_L_S/S_I/OSI/PEI/PCI/PSI/tinShell/config.xml	1899
/Trif_DNA-master/C_L_S/S_I/OSI/PEI/PCI/PSI/tinShell/editPaneReader.jpg	1899
/Trif_DNA-master/C_L_S/S_I/OSI/PEI/PCI/PSI/tinShell/EmbossFilterNode.py	1899
S_I.OSI.PSO.regex	1900
/Trif_DNA-master/C_L_S/S_I/OSI/PSO/regex/DoSplit.java	1900
S_I.OSI.SIQ.tinMap	1902
/Trif_DNA-master/C_L_S/S_I/OSI/SIQ/tinMap/TinMap.java	1902
S_I.SVQ.doc	1902
/Trif_DNA-master/C_L_S/S_I/SVQ/doc/InitonPDEModel.lyg	1902
S_logger	1902
/Trif_DNA-master/C_L_S/S_logger/Log.java	1902
1-11-/Trif_DNA-master/C_L_U-计算变换-格式变换-变量类变换-排序	1902
U_A.PEU.P.cache	1902
/Trif_DNA-master/C_L_U/U_A/PEU/P/cache/Cache_M.java	1902
/Trif_DNA-master/C_L_U/U_A/PEU/P/cache/Cache_MTest.java	1905
/Trif_DNA-master/C_L_U/U_A/PEU/P/cache/Cache.java	1905
/Trif_DNA-master/C_L_U/U_A/PEU/P/cache/CacheTest.java	1906
U_A.PEU.P.classification	1906
/Trif_DNA-master/C_L_U/U_A/PEU/P/classification/ObjectClassification.java	1906
/Trif_DNA-master/C_L_U/U_A/PEU/P/classification/ObjectClassificationTest.java	1909
U_A.PEU.P.combination	1909
/Trif_DNA-master/C_L_U/U_A/PEU/P/combination/ObjectCombination.java	1909
/Trif_DNA-master/C_L_U/U_A/PEU/P/combination/ObjectCombinationTest.java	1913
U_A.PEU.P.dna	1913
/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/DNAAuth.java	1913
/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/DNAAuthTest.java	1918
/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/StringUtil.java	1918
/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/StringUtilTest.java	1920
/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/Token.java	1920
/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/TokenCerts.java	1922
/Trif_DNA-master/C_L_U/U_A/PEU/P/dna/TokenUtil.java	1923
U_A.PEU.P.image	1925
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/ConvexHull.java	1925
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/ConvexHullTest.java	1930
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/Dilation.java	1930
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/DilationTest.java	1932
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/Erosion.java	1932
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/ErosionTest.java	1933
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/HitAndMiss.java	1933
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/HitAndMissTest.java	1935
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/HoughTransform.java	1935
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/HoughTransformTest.java	1936
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/ReadWritePng.java	1936
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/ReadWritePngTest.java	1940
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/RegionGet.java	1940
/Trif_DNA-master/C_L_U/U_A/PEU/P/image/RegionGetTest.java	1942
U_A.PEU.P.matrix	1942
/Trif_DNA-master/C_L_U/U_A/PEU/P/matrix/AnnSummingPath.java	1942

/Trif_DNA-master/C_L_U/U_A/PEU/P/matrix/AnnSummingPathTest.java	1942
/Trif_DNA-master/C_L_U/U_A/PEU/P/matrix/CnnMeasure.java	1942
/Trif_DNA-master/C_L_U/U_A/PEU/P/matrix/CnnMeasureTest.java	1943
U_A.PEU.P.md5	1943
/Trif_DNA-master/C_L_U/U_A/PEU/P/md5/Usr.java	1943
/Trif_DNA-master/C_L_U/U_A/PEU/P/md5/UsrToken.java	1945
U_A.PEU.P.movie	1945
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/AdataFrame.java	1945
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/AdataFrameTest.java	1946
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/Export.java	1946
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/Header.java	1946
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/LYGFileIO.java	1947
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/LYGFileIOTest.java	1958
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/LYGPlayer.java	1958
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/MdataFrame.java	1959
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/MdataFrameTest.java	1959
/Trif_DNA-master/C_L_U/U_A/PEU/P/movie/WAVtoLYG.java	1959
U_A.PEU.P.nlp	1959
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_ANN_HMM_L.java	1959
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_ANN_HMM_LTest.java	1960
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_ANN_HMM.java	1960
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_ANN_HMMTest.java	1961
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_DNN_L.java	1961
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_DNN_LTest.java	1965
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_DNN.java	1965
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/DETA_DNNTest.java	1968
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/StopSymbol_UTF8.java	1968
/Trif_DNA-master/C_L_U/U_A/PEU/P/nlp/TranditionChinese.java	1968
U_A.PEU.P.search	1968
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/BinarySearch.java	1968
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/BreadthRun.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/BreadthTreeSearch.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/Demo.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/DepthRun.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/DepthTreeSearch.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/LinerSearch.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/Node.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/PreorderRun.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/PreorderTreeSearch.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/search/RandomSearch.java	1969
U_A.PEU.P.sort	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/BBSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/BinarySort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/BTreeSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Compare.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Demo.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Heap_1D_Sort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Heap_2D_Sort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/InsertionSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Leaf.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/LinkSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/LYGSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/MergeSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/OrderEvenSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/OTreeSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Quick_1D_Sort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Quick_2D_Sort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Quick_3D_Sort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Quick_4D_Sort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/Quick_7D_Sort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/SelectionSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/ShellSort.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sort/TTreeSort.java	1969
U_A.PEU.P.sound	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/ButterworthPro.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/BytestoAIS.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/Convolution.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/Guassian2dPro.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/HoughPro.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/LaplacianPro.java	1969

/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/MaxMiniPro.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/MedianPro.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundAttribute.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundFrequency.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundLinesVector.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundMedVector.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundPlay.java	1969
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundRecord.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundRegressionVector.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundWaveVector.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/sound/SoundWaveVectorTest.java	1970
U_A.PEU.P.table	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/table/TableSorterZYNK.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/table/TableSorterZYNKTest.java	1970
U_A.PEU.P.time	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/time/Speed.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/time/TimeCheck.java	1970
U_A.PEU.P.wave	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Common.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/DFT.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/FFT.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Guassian_F.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Guassian.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Laplasian.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Median.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Shehold.java	1970
/Trif_DNA-master/C_L_U/U_A/PEU/P/wave/Tailor.java	1970
U_A.PEU.P.zip	1971
/Trif_DNA-master/C_L_U/U_A/PEU/P/zip/BootBackup.java	1971
/Trif_DNA-master/C_L_U/U_A/PEU/P/zip/GzipUtil.java	1971
/Trif_DNA-master/C_L_U/U_A/PEU/P/zip/UnZip.java	1971
U_A.PVU.P.image	1971
/Trif_DNA-master/C_L_U/U_A/PVU/P/image/Filter.java	1971
U_V.AVU.OEU.FLT	1971
/Trif_DNA-master/C_L_U/U_V/AVU/OEU/FLT/Flt.java	1971
/Trif_DNA-master/C_L_U/U_V/AVU/OEU/FLT/FltNew.java	1971
/Trif_DNA-master/C_L_U/U_V/AVU/OEU/FLT/FltNewTest.java	1971
U_V.ESU.array	1971
/Trif_DNA-master/C_L_U/U_V/ESU/array/Demo.java	1971
/Trif_DNA-master/C_L_U/U_V/ESU/array/Double_ESU.java	1971
/Trif_DNA-master/C_L_U/U_V/ESU/array/LYG9DWithDoubleQuickSort4D.java	1973
U_V.ESU.csv	1975
/Trif_DNA-master/C_L_U/U_V/ESU/csv/CSV_ESU.java	1975
U_V.ESU.date	1975
/Trif_DNA-master/C_L_U/U_V/ESU/date/Date_ESU.java	1975
U_V.ESU.encoding	1975
/Trif_DNA-master/C_L_U/U_V/ESU/encoding/Coding_ESU.java	1975
U_V.ESU.hash	1975
/Trif_DNA-master/C_L_U/U_V/ESU/hash/Hash_ESU.java	1975
U_V.ESU.http	1975
/Trif_DNA-master/C_L_U/U_V/ESU/http/HttpUnicode.java	1975
U_V.ESU.pdf_P	1975
/Trif_DNA-master/C_L_U/U_V/ESU/pdf_P/PDF_ESU.java	1975
U_V.ESU.query	1975
/Trif_DNA-master/C_L_U/U_V/ESU/query/Query_ESU.java	1975
U_V.ESU.sort	1975
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithDouble_ESU.java	1975
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithDoubleObject_ESU.java	1977
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithDoubleString_ESU.java	1978
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithFloatFloat_ESU.java	1980
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithFloatObject_ESU.java	1981
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithInt_ESU.java	1983
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithIntObject_ESU.java	1985
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithString_ESU_backup.java	1986
/Trif_DNA-master/C_L_U/U_V/ESU/sort/Quick9DLYGWithString_ESU.java	1986
U_V.ESU.stock	1989
/Trif_DNA-master/C_L_U/U_V/ESU/stock/StockCode.java	1989
U_V.ESU.string	1991
/Trif_DNA-master/C_L_U/U_V/ESU/string/QuickLuoyaoguang4D.java	1991
/Trif_DNA-master/C_L_U/U_V/ESU/string/StringSequency.java	1994

U_V.ESU.tree	1997
/Trif_DNA-master/C_L_U/U_V/ESU/tree/TreeLuoyaoguang4D.java	1997
U_V.ESU.tsp.....	1999
/Trif_DNA-master/C_L_U/U_V/ESU/tsp/TSP.java.....	1999
/Trif_DNA-master/C_L_U/U_V/ESU/tsp/TSP_Euler.java.....	2002
/Trif_DNA-master/C_L_U/U_V/ESU/tsp/YaoguangEulerTSP.java	2009
/Trif_DNA-master/C_L_U/U_V/ESU/tsp/YaoguangEulerTSP2D.java	2018
U_V.OEU.LYG4DQS4D	2025
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG10DWithChineseMixStringSort10D.java	2025
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG10DWithChineseMixStringSort11D.java	2025
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG10DWithChineseMixStringSort12D.java	2025
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG10DWithChineseMixStringSort12DTest.java	2041
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG4DWithChineseMixStringSort7D.java	2041
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG4DWithDoubleQuickSort4D.java.....	2041
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG4DWithDoubleQuickSort4DTest.java	2042
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG5DWithChineseMixStringSort8D.java.....	2042
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG5DWithDoubleQuickSort4D.java.....	2042
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG6DWithDoubleQuickSort4D.java.....	2042
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG6DWithDoubleQuickSort4DTest.java	2044
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DQS4DWithDoubleTopSort1D.java	2044
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithChineseMixStringSort9D.java	2044
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithDoubleQuickSort4D.java.....	2044
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithDoubleTopSort1D.java	2044
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithDoubleTopSort2D.java	2044
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithDoubleTopSort3D.java	2044
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithDoubleTopSort3DTest.java.....	2046
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithDoubleTopSort4D_U.java	2046
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithDoubleTopSort4D.java	2048
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithDoubleTopSort5D.java	2049
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/LYG9DWithIntTopSort4D.java.....	2051
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_5D_Sort.java.....	2053
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_6D_AOP_MVE_CSI_DUQ_Sort.java	2053
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_6D_luoyaoguang_Sort.java	2056
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_7D_luoyaoguang_Sort.java	2058
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_Luoyaoguang_1D.java.....	2060
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_Luoyaoguang_2D.java.....	2062
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_Luoyaoguang_3D.java.....	2063
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_Luoyaoguang_4D.java.....	2065
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_Luoyaoguang_4DWithoutComb.java.....	2067
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/Quick_Luoyaoguang_5D.java.....	2068
/Trif_DNA-master/C_L_U/U_V/OEU/LYG4DQS4D/说明文件.MD	2071
U_V.OEU.press	2071
/Trif_DNA-master/C_L_U/U_V/OEU/press/Node.java	2071
/Trif_DNA-master/C_L_U/U_V/OEU/press/Press.java	2071
U_V.PEU.S.UVIMIACI	2074
/Trif_DNA-master/C_L_U/U_V/PEU/S/UVIMIACI/VerbalSource.java	2074
U_V.SU.object.....	2074
/Trif_DNA-master/C_L_U/U_V/SU/object/Object_ESU.java	2074
1-12-/Trif_DNA-master/C_L_V-拼音笔画视觉-肽计算-animation动画-视频	2075
OSI.MSU.AVQ.ASQ.dnaSets	2075
/Trif_DNA-master/C_L_V/OSI/MSU/AVQ/ASQ/dnaSets/CMS.java.....	2075
/Trif_DNA-master/C_L_V/OSI/MSU/AVQ/ASQ/dnaSets/PDN.java	2075
OSI.OPE.OVU.PQE.flash	2076
/Trif_DNA-master/C_L_V/OSI/OPE/OVU/PQE/flash/Flash.java	2076
/Trif_DNA-master/C_L_V/OSI/OPE/OVU/PQE/flash/GUISample.java.....	2076
/Trif_DNA-master/C_L_V/OSI/OPE/OVU/PQE/flash/ThisCanvas.java	2078
V_A.SQ	2083
/Trif_DNA-master/C_L_V/V_A/SQ/ReadPinYinBihua.java	2083
V_A.VPC.cache	2084
/Trif_DNA-master/C_L_V/V_A/VPC/cache/DetaCache_M.java	2084
/Trif_DNA-master/C_L_V/V_A/VPC/cache/DetaCache_MTest.java.....	2086
/Trif_DNA-master/C_L_V/V_A/VPC/cache/DetaCache.java.....	2086
/Trif_DNA-master/C_L_V/V_A/VPC/cache/Test.java.....	2087
/Trif_DNA-master/C_L_V/V_A/VPC/cache/TimeCheck.java.....	2087
V_A.VPC.common.....	2087
/Trif_DNA-master/C_L_V/V_A/VPC/common/KeyStore.java	2087
/Trif_DNA-master/C_L_V/V_A/VPC/common/LYGPasswordDecode.java	2087
/Trif_DNA-master/C_L_V/V_A/VPC/common/NeedLogin.java	2087
/Trif_DNA-master/C_L_V/V_A/VPC/common/POSTREST.java	2087
/Trif_DNA-master/C_L_V/V_A/VPC/common/RequestLimit.java.....	2087

/Trif_DNA-master/C_L_V/V_A/VPC/common/WriteData.java	2087
V_A.VQ.SQ.demos	2087
/Trif_DNA-master/C_L_V/V_A/VQ/SQ/demos/DEMO_LYG_to_WAV.java	2087
/Trif_DNA-master/C_L_V/V_A/VQ/SQ/demos/DEMO_LYG_to_WAVTest.java	2087
/Trif_DNA-master/C_L_V/V_A/VQ/SQ/demos/DEMO_WAV_TO_LYG.java	2087
/Trif_DNA-master/C_L_V/V_A/VQ/SQ/demos/DEMO_WAV_TO_LYGTest.java	2087
/Trif_DNA-master/C_L_V/V_A/VQ/SQ/demos/DEMO.java	2087
V_A.VSQ.OPE.jogl	2087
/Trif_DNA-master/C_L_V/V_A/VSQ/OPE/jogl/JOGLDrawOBJ.java	2087
/Trif_DNA-master/C_L_V/V_A/VSQ/OPE/jogl/JOGLOBJFace.java	2087
/Trif_DNA-master/C_L_V/V_A/VSQ/OPE/jogl/JOGLObjLoader.java	2087
/Trif_DNA-master/C_L_V/V_A/VSQ/OPE/jogl/JOGLOBJResources.java	2087
/Trif_DNA-master/C_L_V/V_A/VSQ/OPE/jogl/JOGLOBJShape.java	2087
/Trif_DNA-master/C_L_V/V_A/VSQ/OPE/jogl/JOGLOBJVec4.java	2087
/Trif_DNA-master/C_L_V/V_A/VSQ/OPE/jogl/JOGLOBJVertex.java	2087
1-13-/Trif_DNA-master/C_R_A--肽计算-视频类,	2088
A_I.pde	2088
/Trif_DNA-master/C_R_A/A_I/pde/PDE_Decrement_FullDNAFormular_XCDX.java	2088
/Trif_DNA-master/C_R_A/A_I/pde/PDE_Decrement_FullDNAFormular_XCDXTest.java	2094
/Trif_DNA-master/C_R_A/A_I/pde/PDE_Increment_FullDNAFormular_XCDX.java	2094
/Trif_DNA-master/C_R_A/A_I/pde/PDE_Increment_FullDNAFormular_XCDXTest.java	2101
A_V.cap	2101
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X.java	2101
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_EyeScan.java	2107
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_EyeScanTest.java	2114
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_Ica.java	2114
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_IcaTest.java	2115
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_Pca.java	2115
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_PcaTest.java	2117
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_PcfButton.java	2117
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_PcfButtonTest.java	2121
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_Pde.java	2121
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation_PdeTest.java	2124
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Animation.java	2124
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_AnimationTest.java	2126
/Trif_DNA-master/C_R_A/A_V/cap/Monitor_X_Test.java	2126
ascii	2126
/Trif_DNA-master/C_R_A/ascii/GetASCMap.java	2126
/Trif_DNA-master/C_R_A/ascii/WriteASCMap.java	2128
dnn	2129
/Trif_DNA-master/C_R_A/dnn/GetDNNMap.java	2129
/Trif_DNA-master/C_R_A/dnn/WriteDNNMap.java	2131
fullDNATokenPDI	2133
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X.java	2133
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X_do_PDE_RNA_FullFormular.java	2139
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X_do_PDE_RNA_FullFormularTest.java	2145
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X_doDecrement.java	2145
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X_doDecrementTest.java	2149
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X_doIncrement.java	2149
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X_doIncrementTest.java	2153
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X_S.java	2153
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDI_X_Test.java	2153
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDITest_X.java	2153
/Trif_DNA-master/C_R_A/fullDNATokenPDI/PDITest_X_Test.java	2156
1-14-/Trif_DNA-master/C_R_C-分词控制器, 变量控制,	2157
C_A.OCI.ME.analysis.C	2157
/Trif_DNA-master/C_R_C/C_A/OCI/ME/analysis/C/A_X_Map.java	2157
/Trif_DNA-master/C_R_C/C_A/OCI/ME/analysis/C/ACTest.java	2157
/Trif_DNA-master/C_R_C/C_A/OCI/ME/analysis/C/ATest.java	2157
/Trif_DNA-master/C_R_C/C_A/OCI/ME/analysis/C/BinaryForest_ATest.java	2157
/Trif_DNA-master/C_R_C/C_A/OCI/ME/analysis/C/CogsBinaryForest_ATest.java	2157
C_A.OCI.ME.nlp.C	2157
/Trif_DNA-master/C_R_C/C_A/OCI/ME/nlp/C/Nlp_C_X.java	2157
/Trif_DNA-master/C_R_C/C_A/OCI/ME/nlp/C/Nlp_C_X_A_doPOSAndEMMCheck.java	2157
/Trif_DNA-master/C_R_C/C_A/OCI/ME/nlp/C/Nlp_C_X_A_doSlangPartAndPOSCheck.java	2158
/Trif_DNA-master/C_R_C/C_A/OCI/ME/nlp/C/Nlp_C_X_S_doSlangCheck.java	2158
C_A.OCI.ME.pos.C	2159
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/C/X.java	2159
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/C/X_E.java	2159
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/C/X_O_parserFirst.java	2160

C_A.OCI.ME.pos.CP	2160
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/CP/X_P_JieCi.java	2160
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/CP/X_P_LianCi.java	2160
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/CP/X_P_LiangCi.java	2161
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/CP/X_P_MingCi.java	2161
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/CP/X_P_ZhuCi.java	2162
/Trif_DNA-master/C_R_C/C_A/OCI/ME/pos/CP/X_P.java	2162
C_A.OSI.AOP.MS.VPC.C	2163
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/MS/VPC/C/Config_C.java	2163
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/MS/VPC/C/DBCategory_C.java	2163
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/MS/VPC/C/DC.java	2164
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/MS/VPC/C/IUC.java	2164
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/MS/VPC/C/QC.java	2165
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/MS/VPC/C/UC.java	2166
C_A.OSI.AOP.VPC.C	2167
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/VPC/C/Config_C.java	2167
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/VPC/C/D_C.java	2167
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/VPC/C/DBCategory_C.java	2167
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/VPC/C/IU_C.java	2167
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/VPC/C/Q_C.java	2167
/Trif_DNA-master/C_R_C/C_A/OSI/AOP/VPC/C/U_C.java	2168
1-15-/Trif_DNA-master/C_R_D	2169
constants	2169
/Trif_DNA-master/C_R_D/constants/DetaDBConstant.java	2169
1-16-/Trif_DNA-master/C_R_E	2170
E	2170
/Trif_DNA-master/C_R_E/E/Nlp_CE_X_A_doPOSAndEMMCheck.java	2170
/Trif_DNA-master/C_R_E/E/Nlp_CE_X_A_doSlangPartAndPOSCheck.java	2175
/Trif_DNA-master/C_R_E/E/Nlp_CE_X_S.java	2177
EAE	2182
/Trif_DNA-master/C_R_E/EAE/X_Map.java	2182
ECE	2187
/Trif_DNA-master/C_R_E/ECE/Pos_X.java	2187
/Trif_DNA-master/C_R_E/ECE/Pos_X_E.java	2188
/Trif_DNA-master/C_R_E/ECE/Pos_X_O.java	2191
/Trif_DNA-master/C_R_E/ECE/XCE_XAS_U.java	2193
ME	2193
/Trif_DNA-master/C_R_E/ME/XCE_XAS.java	2193
nodeView	2194
/Trif_DNA-master/C_R_E/nodeView/NodeShow_X_S.java	2194
P	2195
/Trif_DNA-master/C_R_E/P/Pos_X_P_JieCi.java	2195
/Trif_DNA-master/C_R_E/P/Pos_X_P_LianCi.java	2199
/Trif_DNA-master/C_R_E/P/Pos_X_P_LiangCi.java	2202
/Trif_DNA-master/C_R_E/P/Pos_X_P_MingCi.java	2205
/Trif_DNA-master/C_R_E/P/Pos_X_P_ZhuCi.java	2211
/Trif_DNA-master/C_R_E/P/Pos_X_P.java	2214
1-17-/Trif_DNA-master/C_R_I-增加-文档-词汇花类,	2217
bi_P	2217
/Trif_DNA-master/C_R_I/bi_P/CoDNAAuthorForWord_X_S.java	2217
YLJ_HRJ.ME.VPC.S.books	2219
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/books/BookIndex.java	2219
YLJ_HRJ.ME.VPC.S.books.xlsx	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/books/xlsx/BookXlsxIndex.java	2220
YLJ_HRJ.ME.VPC.S.configs	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/ConfigIndex.java	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/brewReplace.sh	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/DBconfig.lyg	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/kompose.sh	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/noUpdate.sh	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/opencv.sh	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/openGL.sh	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/preEnvironment.sh	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/configs/problem.sh	2220
YLJ_HRJ.ME.VPC.S.images	2220
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/images/ImageIndex.java	2220
YLJ_HRJ.ME.VPC.S.sc	2223
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/sc/WaveIndex.java	2223
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/sc/a.png	2223
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/sc/e.png	2223

/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/sc/i.png	2223
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/sc/o.png	2223
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/sc/u.png	2223
YLJ_HRJ.ME.VPC.S.yaoCaiTu.....	2223
/Trif_DNA-master/C_R_I/YLJ_HRJ/ME/VPC/S/yaoCaiTu/YaoCaiIndex.java.....	2223
1-18-/Trif_DNA-master/C_R_M-管理-数据库-网络类,	2223
M_V.ME.ConditionPLSQL	2223
/Trif_DNA-master/C_R_M/M_V/ME/ConditionPLSQL /X_ .java	2223
/Trif_DNA-master/C_R_M/M_V/ME/ConditionPLSQL /X_ Cache.java.....	2223
/Trif_DNA-master/C_R_M/M_V/ME/ConditionPLSQL /X_ Kernel.java.....	2227
/Trif_DNA-master/C_R_M/M_V/ME/ConditionPLSQL /X_ Map.java	2229
/Trif_DNA-master/C_R_M/M_V/ME/ConditionPLSQL /X_ Table.java.....	2232
M_V.ME.E.....	2238
/Trif_DNA-master/C_R_M/M_V/ME/E/PLSQLCommand_E_X_P_BaseName.java	2238
/Trif_DNA-master/C_R_M/M_V/ME/E/PLSQLCommand_E_X_P_Check.java.....	2238
/Trif_DNA-master/C_R_M/M_V/ME/E/PLSQLCommand_E_X_P_E_Kernel.java	2240
/Trif_DNA-master/C_R_M/M_V/ME/E/PLSQLCommand_E_X_P_E.java	2243
/Trif_DNA-master/C_R_M/M_V/ME/E/PLSQLCommand_E_X_P_Join.java	2244
/Trif_DNA-master/C_R_M/M_V/ME/E/PLSQLCommand_E_X_P_ListNeedStart.java.....	2245
/Trif_DNA-master/C_R_M/M_V/ME/E/PLSQLCommand_E_X_P_SetRoot.java	2245
/Trif_DNA-master/C_R_M/M_V/ME/E/PLSQLCommand_E_X_P_TableName.java	2246
M_V.ME.GetCulumnsPLSQL.....	2246
/Trif_DNA-master/C_R_M/M_V/ME/GetCulumnsPLSQL/X_getCulumnsMap.java.....	2246
/Trif_DNA-master/C_R_M/M_V/ME/GetCulumnsPLSQL/X_getCulumnsMapWithAs.java	2247
/Trif_DNA-master/C_R_M/M_V/ME/GetCulumnsPLSQL/X_P_GetCulumnsMap.java	2247
M_V.ME.RelationPLSQL	2248
/Trif_DNA-master/C_R_M/M_V/ME/RelationPLSQL /X_P_AndMap.java.....	2248
/Trif_DNA-master/C_R_M/M_V/ME/RelationPLSQL /X_P_OrMap.java	2251
M_V.ME.word.forest.utils	2253
/Trif_DNA-master/C_R_M/M_V/ME/word/forest/utils/Util_X_ContainsKey.java	2253
/Trif_DNA-master/C_R_M/M_V/ME/word/forest/utils/Util_X_NotContainsKey.java	2254
/Trif_DNA-master/C_R_M/M_V/ME/word/forest/utils/Util_X_prefixWordEqualZero.java.....	2254
M_V.ME.word.frequency.utils	2255
/Trif_DNA-master/C_R_M/M_V/ME/word/frequency/utils/Util_X_CompareCheck.java	2255
/Trif_DNA-master/C_R_M/M_V/ME/word/frequency/utils/Util_X_FindCheck.java.....	2255
M_V.MS.OP.SM.AOP.MEC.SIQ.cache	2256
/Trif_DNA-master/C_R_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaCache_M.java	2256
/Trif_DNA-master/C_R_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaDBBufferCache_M.java	2258
/Trif_DNA-master/C_R_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/Test.java.....	2261
M_V.MS.VPC.PP	2261
/Trif_DNA-master/C_R_M/M_V/MS/VPC/PP/Cache_P.java.....	2261
/Trif_DNA-master/C_R_M/M_V/MS/VPC/PP/Json_P.java.....	2261
/Trif_DNA-master/C_R_M/M_V/MS/VPC/PP/ThirdParty_P.java	2261
M_V.MS.VPC.V	2261
/Trif_DNA-master/C_R_M/M_V/MS/VPC/V/Forward_V.java	2261
/Trif_DNA-master/C_R_M/M_V/MS/VPC/V/RestMap_V.java	2263
/Trif_DNA-master/C_R_M/M_V/MS/VPC/V/RestMap.java.....	2265
M_V.MSD.OP.SM.AOP.MEC.SIQ.SM.OSD.E	2266
/Trif_DNA-master/C_R_M/M_V/MSD/OP/SM/AOP/MEC/SIQ/SM/OSD/E/ D_Rows_E_X_D_RowByAttributesOfCondition.java.....	2266
/Trif_DNA-master/C_R_M/M_V/MSD/OP/SM/AOP/MEC/SIQ/SM/OSD/E/ D_Rows_E_X_D_RowByTablePathAndIndex.java	2268
M_V.MSI.OP.SM.AOP.MEC.SIQ.SM.OSU.E.....	2269
/Trif_DNA-master/C_R_M/M_V/MSI/OP/SM/AOP/MEC/SIQ/SM/OSU/E/IU_Rows_E_X_IU_RowByAttributes.java	2269
/Trif_DNA-master/C_R_M/M_V/MSI/OP/SM/AOP/MEC/SIQ/SM/OSU/E/IU_Rows_E_X_IU_RowByBaseName.java	2269
/Trif_DNA-master/C_R_M/M_V/MSI/OP/SM/AOP/MEC/SIQ/SM/OSU/E/ IU_Rows_E_X_IU_RowByTablePathAndIndex.java	2272
M_V.MSQ.OP.SM.AOP.MEC.SIQ.SM.OSQ.E.joinRows.....	2273
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/joinRows/ Q_JoinRows_E_X_selectRowsByAttributesOfJoinAggregation.java.....	2273
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/joinRows/ Q_JoinRows_E_X_selectRowsByAttributesOfJoinCondition.java.....	2274
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/joinRows/ Q_JoinRows_E_X_selectRowsByAttributesOfJoinGetCulumns.java	2276
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/joinRows/ Q_JoinRows_E_X_selectRowsByAttributesOfJoinRelation.java	2277
M_V.MSQ.OP.SM.AOP.MEC.SIQ.SM.OSQ.E.nestRows.....	2278
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/nestRows/	

Q_NestRows_E_X_selectRowsByAttributesOfNestAggregation.java	2278
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/nestRows/	
Q_NestRows_E_X_selectRowsByAttributesOfNestCondition.java	2279
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/nestRows/	
Q_NestRows_E_X_selectRowsByAttributesOfNestGetCulumns.java	2281
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/nestRows/	
Q_NestRows_E_X_selectRowsByAttributesOfNestRelation.java	2281
M_V.MSQ.OP.SM.AOP.MEC.SIQ.SM.OSQ.E.rows	2282
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/rows/	
Q_Rows_E_X_selectRowsByAttribute.java	2282
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/rows/	
Q_Rows_E_X_selectRowsByAttributesOfAggregation.java	2285
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/rows/	
Q_Rows_E_X_selectRowsByAttributesOfCondition.java	2286
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/rows/	
Q_Rows_E_X_selectRowsByAttributesOfGetCulumns.java	2288
/Trif_DNA-master/C_R_M/M_V/MSQ/OP/SM/AOP/MEC/SIQ/SM/OSQ/E/rows/	
Q_Rows_E_X_selectRowsByTablePath.java	2289
M_V.MSU.AMS.VQS.SQV.SI.OSU.SMV.http	2292
/Trif_DNA-master/C_R_M/M_V/MSU/AMS/VQS/SQV/SI/OSU/SMV/http/RestCall_X_backEndRequest.java	2292
/Trif_DNA-master/C_R_M/M_V/MSU/AMS/VQS/SQV/SI/OSU/SMV/http/RestCall_X_cacheRequest.java	2294
/Trif_DNA-master/C_R_M/M_V/MSU/AMS/VQS/SQV/SI/OSU/SMV/http/RestCall_X_getJson.java	2296
/Trif_DNA-master/C_R_M/M_V/MSU/AMS/VQS/SQV/SI/OSU/SMV/http/RestCall_X_searchLinkWord.java	2298
M_V.MSU.OP.SM.AOP.MEC.SIQ.SM.update.EU.JoinRows.E	2299
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/JoinRows/E/	
X_U_RowsByAttributesOfJoinAggregation.java	2299
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/JoinRows/E/	
X_U_RowsByAttributesOfJoinCondition.java	2299
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/JoinRows/E/	
X_U_RowsByAttributesOfJoinGetCulumns.java	2301
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/JoinRows/E/	
X_U_RowsByAttributesOfJoinRelation.java	2302
M_V.MSU.OP.SM.AOP.MEC.SIQ.SM.update.EU.Rows.E	2303
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/Rows/E/	
X_U_RowByTablePathAndAttribute.java	2303
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/Rows/E/	
X_U_RowByTablePathAndIndex.java	2305
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/Rows/E/	
X_U_RowsByAttributesOfAggregation.java	2307
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/Rows/E/	
X_U_RowsByAttributesOfCondition.java	2308
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/Rows/E/	
X_U_RowsByAttributesOfGetCulumns.java	2310
/Trif_DNA-master/C_R_M/M_V/MSU/OP/SM/AOP/MEC/SIQ/SM/update/EU/Rows/E/	
X_U_RowsByRecordConditions.java	2310
M_V.MSU.sessionValidation	2312
/Trif_DNA-master/C_R_M/M_V/MSU/sessionValidation/X_sessionCheck.java	2312
/Trif_DNA-master/C_R_M/M_V/MSU/sessionValidation/X_sessionCheckWithOrderPDSKey.java	2312
/Trif_DNA-master/C_R_M/M_V/MSU/sessionValidation/X_sessionCheckWithPostPDSKey.java	2313
/Trif_DNA-master/C_R_M/M_V/MSU/sessionValidation/X_sessionInitByTokenCertsDNA.java	2314
/Trif_DNA-master/C_R_M/M_V/MSU/sessionValidation/X_sessionInitByTokenPDICertsDNA.java	2316
/Trif_DNA-master/C_R_M/M_V/MSU/sessionValidation/X_sessionTokenCertsInitWithHumanWordsByDNA_ETL.java ..	2317
/Trif_DNA-master/C_R_M/M_V/MSU/sessionValidation/X_sessionTokenCertsInitWithHumanWordsByDNA.java ..	2318
/Trif_DNA-master/C_R_M/M_V/MSU/sessionValidation/X_sessionTokenUpdateByDNA.java	2320
1-19-/Trif_DNA-master/C_R_O-操作, tinshell类,	2320
plshell	2320
/Trif_DNA-master/C_R_O/plshell/P_CO_XA_X.java	2320
/Trif_DNA-master/C_R_O/plshell/P_CO_XA_X_Cache.java	2320
/Trif_DNA-master/C_R_O/plshell/P_CO_XA_X_Kernel.java	2337
/Trif_DNA-master/C_R_O/plshell/P_CO_XA_X_Map.java	2338
1-20-/Trif_DNA-master/C_R_P-处理-轨迹类, 坐标类	2352
ASQ.attraction	2352
/Trif_DNA-master/C_R_P/ASQ/attraction/Attraction_X_getTwoPosition2DClusterAttraction.java	2352
/Trif_DNA-master/C_R_P/ASQ/attraction/Attraction_X_getTwoPosition3DClusterAttraction.java	2352
ASQ.basic	2352
/Trif_DNA-master/C_R_P/ASQ/basic/Distance_X_getDistance2D.java	2352
/Trif_DNA-master/C_R_P/ASQ/basic/Distance_X_getDistance3D.java	2352
/Trif_DNA-master/C_R_P/ASQ/basic/Distance_X_getPythagoreanDistance2D.java	2353
/Trif_DNA-master/C_R_P/ASQ/basic/Distance_X_getPythagoreanDistance3D.java	2353

ASQ.classification.AMV.MVS.VSQ.....	2353
/Trif_DNA-master/C_R_P/ASQ/classification/AMV/MVS/VSQ/Classification_X_addNewPositionWithHeart.java.....	2353
/Trif_DNA-master/C_R_P/ASQ/classification/AMV/MVS/VSQ/Classification_X_addNewPositionWithoutHeart.java.....	2354
/Trif_DNA-master/C_R_P/ASQ/classification/AMV/MVS/VSQ/	
HeartsSample_X_getShorterSamplePosition2DGroupsWithScale.java.....	2355
/Trif_DNA-master/C_R_P/ASQ/classification/AMV/MVS/VSQ/	
HeartsSample_X_getShortestSamplePosition2DGroup.java.....	2356
/Trif_DNA-master/C_R_P/ASQ/classification/AMV/MVS/VSQ/	
HeartsSample_X_getShortestSamplePosition3DGroup.java.....	2357
/Trif_DNA-master/C_R_P/ASQ/classification/AMV/MVS/VSQ/	
HeartsSample_X_getShortestSamplePosition3DGroupsWithScale.java.....	2358
ASQ.classification.fissile.....	2358
/Trif_DNA-master/C_R_P/ASQ/classification/fissile/Fissile_X_fissilePosition2D.java.....	2358
/Trif_DNA-master/C_R_P/ASQ/classification/fissile/Fissile_X_fissilePosition3D.java.....	2360
/Trif_DNA-master/C_R_P/ASQ/classification/fissile/FissileWithMatch_X_fissilePosition2DWithMatch.java.....	2361
/Trif_DNA-master/C_R_P/ASQ/classification/fissile/FissileWithMatch_X_fissilePosition3DWithMatch.java.....	2363
ASQ.cluster.....	2365
/Trif_DNA-master/C_R_P/ASQ/cluster/SideEnd_X_getSideEnd2D.java.....	2365
/Trif_DNA-master/C_R_P/ASQ/cluster/SideEnd_X_getSideEnd3D.java.....	2366
ASQ.correction.....	2367
/Trif_DNA-master/C_R_P/ASQ/correction/CorrectionCoosMap_X_CorrectionPosition2DMap.java.....	2367
/Trif_DNA-master/C_R_P/ASQ/correction/CorrectionCoosMap_X_CorrectionPosition3DMap.java.....	2367
/Trif_DNA-master/C_R_P/ASQ/correction/CorrectionCoosMap_X_getPosition2DRange.java.....	2368
/Trif_DNA-master/C_R_P/ASQ/correction/CorrectionCoosMap_X_getPosition3DRange.java.....	2368
ASQ.demension.AMV.MVS.VSQ.....	2369
/Trif_DNA-master/C_R_P/ASQ/demension/AMV/MVS/VSQ/FindHeart_X_getPosition2DGroupsHearts.java.....	2369
/Trif_DNA-master/C_R_P/ASQ/demension/AMV/MVS/VSQ/FindHeart_X_getPosition3DGroupsHearts.java.....	2370
/Trif_DNA-master/C_R_P/ASQ/demension/AMV/MVS/VSQ/FindMid_X_getPosition2DGroupsMids.java.....	2370
/Trif_DNA-master/C_R_P/ASQ/demension/AMV/MVS/VSQ/FindMid_X_getPosition3DGroupsMids.java.....	2371
ASQ.euclid.....	2371
/Trif_DNA-master/C_R_P/ASQ/euclid/Euclid_X_findCryptionPosition2D.java.....	2371
/Trif_DNA-master/C_R_P/ASQ/euclid/Euclid_X_findCryptionPosition3D.java.....	2372
/Trif_DNA-master/C_R_P/ASQ/euclid/Euclid_X_findHeartPosition2D.java.....	2372
/Trif_DNA-master/C_R_P/ASQ/euclid/Euclid_X_findHeartPosition3D.java.....	2373
/Trif_DNA-master/C_R_P/ASQ/euclid/Euclid_X_findMidPosition2D.java.....	2374
/Trif_DNA-master/C_R_P/ASQ/euclid/Euclid_X_findMidPosition3D.java.....	2375
ASQ.filter.....	2376
/Trif_DNA-master/C_R_P/ASQ/filter/Filter_X_getImagePix2DGroupFilter.java.....	2376
/Trif_DNA-master/C_R_P/ASQ/filter/Filter_X_getImagePix3DGroupFilter.java.....	2377
ASQ.find.AMV.MVS.VSQ.....	2378
/Trif_DNA-master/C_R_P/ASQ/find/AMV/MVS/VSQ/GroupPascalHearts_X_getPosition2DsGroupPascalDirection.java.....	2378
/Trif_DNA-master/C_R_P/ASQ/find/AMV/MVS/VSQ/GroupPascalHearts_X_getPosition2DsGroupPascalHearts.java.....	2379
/Trif_DNA-master/C_R_P/ASQ/find/AMV/MVS/VSQ/GroupPascalHearts_X_getPosition2DsGroupPascalMids.java.....	2379
/Trif_DNA-master/C_R_P/ASQ/find/AMV/MVS/VSQ/GroupPascalHearts_X_getPosition3DsGroupPascalDirection.java.....	2380
/Trif_DNA-master/C_R_P/ASQ/find/AMV/MVS/VSQ/GroupPascalHearts_X_getPosition3DsGroupPascalHearts.java.....	2381
/Trif_DNA-master/C_R_P/ASQ/find/AMV/MVS/VSQ/GroupPascalHearts_X_getPosition3DsGroupPascalMids.java.....	2381
ASQ.find.mean.....	2382
/Trif_DNA-master/C_R_P/ASQ/find/mean/Distance_X_findMeanDistanceFromPositions2D.java.....	2382
/Trif_DNA-master/C_R_P/ASQ/find/mean/Distance_X_findMeanDistanceFromPositions3D.java.....	2383
/Trif_DNA-master/C_R_P/ASQ/find/mean/Distance_X_findPascalMeanDistanceByEachPositions2D.java.....	2384
/Trif_DNA-master/C_R_P/ASQ/find/mean/Distance_X_findPascalMeanDistanceByEachPositions3D.java.....	2386
ASQ.forest.....	2387
/Trif_DNA-master/C_R_P/ASQ/forest/Forest_X_getTSPForestIsolationGroups2D.java.....	2387
/Trif_DNA-master/C_R_P/ASQ/forest/Forest_X_getTSPForestIsolationGroups3D.java.....	2388
ASQ.fusion.....	2390
/Trif_DNA-master/C_R_P/ASQ/fusion/Filter_X_filterFusion2DSetsWithCountScale.java.....	2390
/Trif_DNA-master/C_R_P/ASQ/fusion/Filter_X_filterFusion3DSetsWithCountScale.java.....	2390
/Trif_DNA-master/C_R_P/ASQ/fusion/Fusion_X_fusionPosition2DwithHeart.java.....	2391
/Trif_DNA-master/C_R_P/ASQ/fusion/Fusion_X_fusionPosition3DwithHeart.java.....	2393
/Trif_DNA-master/C_R_P/ASQ/fusion/FusionHeart_X_fusionPosition2DHeart.java.....	2395
/Trif_DNA-master/C_R_P/ASQ/fusion/FusionHeart_X_fusionPosition3DHeart.java.....	2395
ASQ.fuzz.....	2395
/Trif_DNA-master/C_R_P/ASQ/fuzz/Fuzz_X_getSimilarFuzzSet.java.....	2395
/Trif_DNA-master/C_R_P/ASQ/fuzz/Fuzz_X_getSimilarFuzzSetListWithScale.java.....	2396
ASQ.group.....	2397

/Trif_DNA-master/C_R_P/ASQ/group/Percentage_X_getClusterPercentByDeterPixGroup.java	2397
/Trif_DNA-master/C_R_P/ASQ/group/Percentage_X_getPixPercentByDeterPix.java	2398
/Trif_DNA-master/C_R_P/ASQ/group/Percentage_X_getPixPercentByDeterPixGroup.java	2398
ASQ.hint.AMV.MVS.VSQ	2399
/Trif_DNA-master/C_R_P/ASQ/hint/AMV/MVS/VSQ/HintDirection_X_getHintDirectionTrendFromPosition2Ds.java	2399
/Trif_DNA-master/C_R_P/ASQ/hint/AMV/MVS/VSQ/HintDirection_X_getHintDirectionTrendFromPosition3Ds.java	2400
ASQ.ica	2400
/Trif_DNA-master/C_R_P/ASQ/ica/CorrelationICA_X_frequencyDownSplit.java	2400
/Trif_DNA-master/C_R_P/ASQ/ica/CorrelationICA_X_frequencyDownSplitWithScale.java	2401
/Trif_DNA-master/C_R_P/ASQ/ica/CorrelationICA_X_frequencyUpSplit.java	2401
/Trif_DNA-master/C_R_P/ASQ/ica/CorrelationICA_X_frequencyUpSplitWithScale.java	2402
ASQ.image.pix	2402
/Trif_DNA-master/C_R_P/ASQ/image/pix/Extract_X_getHWaveFromImagePix.java	2402
/Trif_DNA-master/C_R_P/ASQ/image/pix/Extract_X_getVWaveFromImagePix.java	2403
ASQ.isolation	2403
/Trif_DNA-master/C_R_P/ASQ/isolation/Isolation_X_getCorrelation.java	2403
/Trif_DNA-master/C_R_P/ASQ/isolation/Isolation_X_getTSPIsolationGroups2D.java	2404
/Trif_DNA-master/C_R_P/ASQ/isolation/Isolation_X_getTSPIsolationGroups3D.java	2405
ASQ.martrix	2406
/Trif_DNA-master/C_R_P/ASQ/martrix/MartrixFilter_X_getIsolationFilterMartix2D.java	2406
/Trif_DNA-master/C_R_P/ASQ/martrix/MartrixFilter_X_getIsolationFilterMartix3D.java	2407
ASQ.nlp	2408
/Trif_DNA-master/C_R_P/ASQ/nlp/NLPTopicMatch_X_getNLPBestSentencesMatchScore.java	2408
/Trif_DNA-master/C_R_P/ASQ/nlp/NLPTopicMatch_X_getNLPBestSentencesMatchScoreRights.java	2409
/Trif_DNA-master/C_R_P/ASQ/nlp/NLPTopicMatch_X_NLPBestSentenceMatch.java	2410
/Trif_DNA-master/C_R_P/ASQ/nlp/NLPTopicMatch_X_NLPBestSentencesMatch.java	2412
ASQ.pca	2413
/Trif_DNA-master/C_R_P/ASQ/pca/AMV_MVS_VSQ_Filter_X_filterPosition2DsWithScaledDistance.java	2413
/Trif_DNA-master/C_R_P/ASQ/pca/AMV_MVS_VSQ_Filter_X_filterPosition3DsWithScaledDistance.java	2414
ASQ.probability.classification	2415
/Trif_DNA-master/C_R_P/ASQ/probability/classification/Classification_X_predictionMatrixResult.java	2415
/Trif_DNA-master/C_R_P/ASQ/probability/classification/Classification_X_predictionResult.java	2416
ASQ.trace.AMV.MVS.VSQ	2417
/Trif_DNA-master/C_R_P/ASQ/trace/AMV/MVS/VSQ/Hearts_X_trackTracePosition2DHeartsWithSingerGroup.java	2417
/Trif_DNA-master/C_R_P/ASQ/trace/AMV/MVS/VSQ/Hearts_X_trackTracePosition3DHeartsWithSingerGroup.java	2418
ASQ.trace.fissile.AMV.MVS.VSQ	2419
/Trif_DNA-master/C_R_P/ASQ/trace/fissile/AMV/MVS/VSQ/ByHearts_X_trackTracePosition2DByHearts.java	2419
/Trif_DNA-master/C_R_P/ASQ/trace/fissile/AMV/MVS/VSQ/ByHearts_X_trackTracePosition3DByHearts.java	2420
/Trif_DNA-master/C_R_P/ASQ/trace/fissile/AMV/MVS/VSQ/Hearts_X_trackTracePosition2DHeartsWithFissileGroups.java	2421
/Trif_DNA-master/C_R_P/ASQ/trace/fissile/AMV/MVS/VSQ/Hearts_X_trackTracePosition3DHeartsWithFissileGroups.java	2423
ASQ.YaoguangLuoEulerRing.tsp	2425
/Trif_DNA-master/C_R_P/ASQ/YaoguangLuoEulerRing/tsp/TSP_X_getYaoguangLuo2DEulerRingTSP.java	2425
/Trif_DNA-master/C_R_P/ASQ/YaoguangLuoEulerRing/tsp/TSP_X_getYaoguangLuo3DEulerRingTSP.java	2427
/Trif_DNA-master/C_R_P/ASQ/YaoguangLuoEulerRing/tsp/TSP2D_X_getYaoguangLuo2DEulerRingTSP2D.java	2430
/Trif_DNA-master/C_R_P/ASQ/YaoguangLuoEulerRing/tsp/TSP2D_X_getYaoguangLuo3DEulerRingTSP2D.java	2433
/Trif_DNA-master/C_R_P/ASQ/YaoguangLuoEulerRing/tsp/TSP3D_X_getYaoguangLuo2DEulerRingTSP3D.java	2436
/Trif_DNA-master/C_R_P/ASQ/YaoguangLuoEulerRing/tsp/TSP3D_X_getYaoguangLuo3DEulerRingTSP3D.java	2439
exception.file.O	2442
/Trif_DNA-master/C_R_P/exception/file/O/DetaBufferedReader.java	2442
/Trif_DNA-master/C_R_P/exception/file/O/DetaFile.java	2442
/Trif_DNA-master/C_R_P/exception/file/O/DetaFileInputStream.java	2443
/Trif_DNA-master/C_R_P/exception/file/O/DetaFileReader.java	2444
/Trif_DNA-master/C_R_P/exception/file/O/DetaFileWriter.java	2445
/Trif_DNA-master/C_R_P/exception/file/O/DetaInputStreamReader.java	2446
exception.net.O	2446
/Trif_DNA-master/C_R_P/exception/net/O/DetaURLEncoder.java	2446
exception.thread	2447
/Trif_DNA-master/C_R_P/exception/thread/DetaThread.java	2447
1-21-/Trif_DNA-master/C_R_S-分词-ETL-词汇花	2447
VSQ.etl	2447
/Trif_DNA-master/C_R_S/VSQ/etl/GUISample_X_S.java	2447
/Trif_DNA-master/C_R_S/VSQ/etl/LinkNode_X_S.java	2449
/Trif_DNA-master/C_R_S/VSQ/etl/NodeProject_X_S.java	2450
/Trif_DNA-master/C_R_S/VSQ/etl/NodeShow_X_S.java	2450

/Trif_DNA-master/C_R_S/VSQ/etl/ThisCanvas_X_S.java.....	2450
VSQ.ICAQDTVMQVM.....	2452
/Trif_DNA-master/C_R_S/VSQ/ICAQDTVMQVM/Monitor_X_S.java	2452
VSQ.neroCell	2457
/Trif_DNA-master/C_R_S/VSQ/neroCell/DrawArrowHead.java	2457
/Trif_DNA-master/C_R_S/VSQ/neroCell/DrawNeroCellMask31.java	2458
/Trif_DNA-master/C_R_S/VSQ/neroCell/DrawNeroCellMask32.java	2461
/Trif_DNA-master/C_R_S/VSQ/neroCell/DrawNeroCellMask33.java	2462
VSQ.parser	2464
/Trif_DNA-master/C_R_S/VSQ/parser/AE_X_S.java	2464
/Trif_DNA-master/C_R_S/VSQ/parser/EmotionInit_X_S.java	2467
/Trif_DNA-master/C_R_S/VSQ/parser/EnvironmentInit_X_S.java	2469
/Trif_DNA-master/C_R_S/VSQ/parser/FMHMMListOneTime_E_X_S.java	2472
VSQ.threeD.flowers	2473
/Trif_DNA-master/C_R_S/VSQ/threeD/flowers/CoAuthorForWord_X_S.java	2473
1-22-/Trif_DNA-master/C_R_U--卷积-变换-排序类	2478
P.image.....	2478
/Trif_DNA-master/C_R_U/P/image/CheckRange.java	2478
/Trif_DNA-master/C_R_U/P/image/CheckRangeTest.java	2479
/Trif_DNA-master/C_R_U/P/image/Closing.java	2479
/Trif_DNA-master/C_R_U/P/image/Color_P.java.....	2479
/Trif_DNA-master/C_R_U/P/image/Color_PTest.java.....	2480
/Trif_DNA-master/C_R_U/P/image/Emboss.java	2480
/Trif_DNA-master/C_R_U/P/image/GetMean.java	2481
/Trif_DNA-master/C_R_U/P/image/GetMeanTest.java	2481
/Trif_DNA-master/C_R_U/P/image/Group_O.java.....	2481
/Trif_DNA-master/C_R_U/P/image/Group_OTest.java	2482
/Trif_DNA-master/C_R_U/P/image/Guassian_X_P_1D.java	2482
/Trif_DNA-master/C_R_U/P/image/Guassian_X_P_2D.java	2483
/Trif_DNA-master/C_R_U/P/image/Guassian_X_P.java.....	2483
/Trif_DNA-master/C_R_U/P/image/Guassian.java	2483
/Trif_DNA-master/C_R_U/P/image/Laplacian.java	2483
/Trif_DNA-master/C_R_U/P/image/LaplacianTest.java	2483
/Trif_DNA-master/C_R_U/P/image/Mask.java	2483
/Trif_DNA-master/C_R_U/P/image/MaskTest.java	2483
/Trif_DNA-master/C_R_U/P/image/Median_X_P.java	2483
/Trif_DNA-master/C_R_U/P/image/Median.java.....	2485
/Trif_DNA-master/C_R_U/P/image/MedianTest.java	2485
/Trif_DNA-master/C_R_U/P/image/Opening.java	2485
/Trif_DNA-master/C_R_U/P/image/OpeningTest.java	2485
/Trif_DNA-master/C_R_U/P/image/PrintMoments.java	2485
/Trif_DNA-master/C_R_U/P/image/PrintMomentsTest.java	2486
/Trif_DNA-master/C_R_U/P/image/RedRaf.java.....	2486
/Trif_DNA-master/C_R_U/P/image/RedRafTest.java	2487
/Trif_DNA-master/C_R_U/P/image/Reflection.java	2487
/Trif_DNA-master/C_R_U/P/image/ReflectionTest.java.....	2488
/Trif_DNA-master/C_R_U/P/image/Sobel_X_dir.java.....	2488
/Trif_DNA-master/C_R_U/P/image/Sobel_X_dirTest.java.....	2489
/Trif_DNA-master/C_R_U/P/image/Sobel_X_mag.java.....	2489
/Trif_DNA-master/C_R_U/P/image/Sobel_X_magTest.java	2490
/Trif_DNA-master/C_R_U/P/image/Sobel_X_P_WithMask.java.....	2490
/Trif_DNA-master/C_R_U/P/image/Sobel_X_P.java.....	2490
/Trif_DNA-master/C_R_U/P/image/Sobel.java.....	2491
/Trif_DNA-master/C_R_U/P/image/Strech_X_P.java	2493
/Trif_DNA-master/C_R_U/P/image/Strech.java.....	2493
/Trif_DNA-master/C_R_U/P/image/Threshold_X_P_Section.java.....	2495
/Trif_DNA-master/C_R_U/P/image/Threshold_X_P_SectionTest.java	2495
/Trif_DNA-master/C_R_U/P/image/Threshold_X_P.java.....	2495
/Trif_DNA-master/C_R_U/P/image/Threshold_X_PTest.java	2496
P.PDEMask.....	2496
/Trif_DNA-master/C_R_U/P/PDEMask/Btest.java	2496
P.wave.copy	2496
/Trif_DNA-master/C_R_U/P/wave/copy/Copy_X_copy1d.java	2496
/Trif_DNA-master/C_R_U/P/wave/copy/Copy_X_copy1dTest.java	2496
/Trif_DNA-master/C_R_U/P/wave/copy/Copy_X_copy1dx2.java	2496
/Trif_DNA-master/C_R_U/P/wave/copy/Copy_X_copy1dx2Test.java	2497
/Trif_DNA-master/C_R_U/P/wave/copy/Copy_X_copy2d.java	2497
/Trif_DNA-master/C_R_U/P/wave/copy/Copy_X_copy2dTest.java	2497
P.wave.maxAndMin	2497
/Trif_DNA-master/C_R_U/P/wave/maxAndMin/MaxAndMin_X_max_i.java	2497

/Trif_DNA-master/C_R_U/P/wave/maxAndMin/MaxAndMin_X_max_iTest.java	2497
/Trif_DNA-master/C_R_U/P/wave/maxAndMin/MaxAndMin_X_max_v.java	2497
/Trif_DNA-master/C_R_U/P/wave/maxAndMin/MaxAndMin_X_max_vTest.java	2499
/Trif_DNA-master/C_R_U/P/wave/maxAndMin/MaxAndMin_X_min_i.java	2499
/Trif_DNA-master/C_R_U/P/wave/maxAndMin/MaxAndMin_X_min_iTest.java	2499
/Trif_DNA-master/C_R_U/P/wave/maxAndMin/MaxAndMin_X_min_v.java	2499
/Trif_DNA-master/C_R_U/P/wave/maxAndMin/MaxAndMin_X_min_vTest.java	2499
P.wave.peakStatistic	2500
/Trif_DNA-master/C_R_U/P/wave/peakStatistic/PeakStatistic_X_fengPaixX.java	2500
/Trif_DNA-master/C_R_U/P/wave/peakStatistic/PeakStatistic_X_fengPaixXTest.java	2501
/Trif_DNA-master/C_R_U/P/wave/peakStatistic/PeakStatistic_X_fengPaixY.java	2501
/Trif_DNA-master/C_R_U/P/wave/peakStatistic/PeakStatistic_X_fengPaixYTest.java	2502
/Trif_DNA-master/C_R_U/P/wave/peakStatistic/PeakStatistic_X_fengTong1.java	2502
/Trif_DNA-master/C_R_U/P/wave/peakStatistic/PeakStatistic_X_fengTong1Test.java	2502
P.wave.proportion	2502
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newX.java	2502
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newXTest.java	2502
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newXY.java	2502
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newXYBest.java	2503
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newXYBestTest.java	2506
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newXYTest.java	2506
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newYYYwithoutBound.java	2506
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newYYYwithoutBoundTest.java	2506
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newY.java	2506
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newYTest.java	2507
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newYwithoutBound.java	2507
/Trif_DNA-master/C_R_U/P/wave/proportion/Proportion_X_newYwithoutBoundTest.java	2507
P.wave.quantification	2507
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaDengChaAdd.java	2507
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaDengChaAddTest.java	2508
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaDengChaMines.java	2508
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaDengChaMinesTest.java	2508
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaEqualDelete.java	2508
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaEqualDeleteTest.java	2509
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaXiHua.java	2509
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaXiHuaHalfSide.java	2510
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaXiHuaHalfSideTest.java	2510
/Trif_DNA-master/C_R_U/P/wave/quantification/Quantification_X_liangHuaXiHuaTest.java	2510
U_V.ESU.array	2510
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToIterator.java	2510
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToIteratorTest.java	2511
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToJson.java	2511
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToJsonTest.java	2511
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToList.java	2511
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToListTest.java	2512
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToMap.java	2512
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToMapTest.java	2512
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToMatrix.java	2512
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToMatrix3D.java	2513
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToMatrix3DTest.java	2513
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToMatrixTest.java	2513
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToSet.java	2513
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToSetTest.java	2514
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToVector.java	2514
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToVectorTest.java	2514
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToXml.java	2514
/Trif_DNA-master/C_R_U/U_V/ESU/array/Array_ESU_X_arrayToXmlTest.java	2515
U_V.ESU.image	2515
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_bufferedImageToFile.java	2515
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_bufferedImageToFileTest.java	2515
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_bufferedImageToScaleImageFile.java	2515
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_bufferedImageToScaleImageFileTest.java	2516
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_imageToFile.java	2516
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_imageToFileTest.java	2516
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_pixRGBTableToFile.java	2516
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_pixRGBTableToFileTest.java	2517
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_pixTableToFile.java	2517
/Trif_DNA-master/C_R_U/U_V/ESU/image/Image_ESU_X_pixTableToFileTest.java	2518
U_V.ESU.imageToList	2518
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageFileToArrayMapWithBlack.java	2518

/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageFileToArrayMapWithBlackTest.java	2519
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageFileToArrayWithBlack.java	2519
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageFileToArrayWithBlackTest.java	2520
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageFileToArrayWithColor.java	2520
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageFileToArrayWithColorTest.java	2520
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageToArrayMapWithBlack.java	2520
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageToArrayMapWithBlackTest.java	2521
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageToArrayWithBlack.java	2521
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageToArrayWithBlackTest.java	2522
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageToArrayWithColor.java	2522
/Trif_DNA-master/C_R_U/U_V/ESU/imageToList/ImageToList_X_waveImageToArrayWithColorTest.java	2523
U_V.ESU.iterator	2523
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToJsonString.java	2523
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToJsonStringTest.java	2523
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToList.java	2523
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToListTest.java	2524
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToMap.java	2524
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToMapTest.java	2524
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToObjectArray.java	2524
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToObjectArrayTest.java	2525
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToXml.java	2525
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_iteratorToXmlTest.java	2525
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_stringIteratorToJsonString.java	2525
/Trif_DNA-master/C_R_U/U_V/ESU/iterator/Iterator_ESU_X_stringIteratorToJsonStringTest.java	2526
U_V.ESU.json	2526
/Trif_DNA-master/C_R_U/U_V/ESU/json/Json_ESU_X_jsonArrayToList.java	2526
/Trif_DNA-master/C_R_U/U_V/ESU/json/Json_ESU_X_jsonArrayToListTest.java	2526
/Trif_DNA-master/C_R_U/U_V/ESU/json/Json_ESU_X_jsonObjectToHashtable.java	2526
/Trif_DNA-master/C_R_U/U_V/ESU/json/Json_ESU_X_jsonObjectToHashtableTest.java	2527
/Trif_DNA-master/C_R_U/U_V/ESU/json/Json_ESU_X_jsonObjectToString.java	2527
/Trif_DNA-master/C_R_U/U_V/ESU/json/Json_ESU_X_jsonObjectToStringTest.java	2527
/Trif_DNA-master/C_R_U/U_V/ESU/json/Json_ESU_X_jsonObjectToXml.java	2527
/Trif_DNA-master/C_R_U/U_V/ESU/json/Json_ESU_X_jsonObjectToXmlTest.java	2528
U_V.ESU.list	2528
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToArray.java	2528
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToIterator.java	2528
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToIteratorTest.java	2528
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToJsonString.java	2528
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToJsonStringTest.java	2529
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToMap.java	2529
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToMapTest.java	2529
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToObjectArray.java	2529
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToObjectArrayTest.java	2529
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToObjectMatrix.java	2529
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToSet.java	2529
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToSetTest.java	2530
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToVector.java	2530
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToVectorTest.java	2530
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToXml.java	2530
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_listToXmlTest.java	2531
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_stringListToJsonString.java	2531
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_stringListToJsonStringTest.java	2531
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_stringlistToStringArray.java	2531
/Trif_DNA-master/C_R_U/U_V/ESU/list/List_ESU_X_stringlistToStringArrayTest.java	2532
U_V.ESU.map	2532
/Trif_DNA-master/C_R_U/U_V/ESU/map/Map_ESU_X_mapToHash.java	2532
/Trif_DNA-master/C_R_U/U_V/ESU/map/Map_ESU_X_mapToHashTest.java	2532
/Trif_DNA-master/C_R_U/U_V/ESU/map/Map_ESU_X_mapToList.java	2532
U_V.ESU.matrix	2533
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixInclineSwap.java	2533
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixInclineSwapTest.java	2533
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixInSwap.java	2533
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixInSwapTest.java	2534
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixPostSwap.java	2534
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixPostSwapTest.java	2534
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixReverseSwap.java	2534
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixReverseSwapTest.java	2535
/Trif_DNA-master/C_R_U/U_V/ESU/matrix/Matrix_ESU_X_matrixToArrayTest.java	2535
U_V.ESU.matrix3D	2535
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixInclineSwapXY.java	2535

/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixInclineSwapXYTest.java	2536
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixInclineSwapXZ.java	2536
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixInclineSwapXZTest.java	2536
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixInclineSwapYZ.java	2536
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixInclineSwapYZTest.java	2537
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixReverseSwap.java	2537
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixReverseSwapTest.java	2538
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixReverseSwapXY.java	2538
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixReverseSwapXYTest.java	2538
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixReverseSwapXZ.java	2538
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixReverseSwapXZTest.java	2539
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixReverseSwapYZ.java	2539
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixReverseSwapYZTest.java	2539
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixShiftSwapXYZ.java	2539
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixShiftSwapXYZTest.java	2540
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixToArray.java	2540
/Trif_DNA-master/C_R_U/U_V/ESU/matrix3D/Matrix3D_ESU_X_matrixToArrayTest.java	2540
U_V.ESU.office	2540
/Trif_DNA-master/C_R_U/U_V/ESU/office/DetaDBBufferToXls.java	2540
/Trif_DNA-master/C_R_U/U_V/ESU/office/DetaDBBufferToXlsSample.java	2544
/Trif_DNA-master/C_R_U/U_V/ESU/office/DetaDBBufferToXlsSampleTest.java	2547
/Trif_DNA-master/C_R_U/U_V/ESU/office/DetaDBBufferToXlsTest.java	2547
/Trif_DNA-master/C_R_U/U_V/ESU/office/DetaYLJBufferToXls.java	2547
/Trif_DNA-master/C_R_U/U_V/ESU/office/DetaYLJBufferToXlsTest.java	2550
/Trif_DNA-master/C_R_U/U_V/ESU/office/XlsToDetaDBBuffer.java	2550
/Trif_DNA-master/C_R_U/U_V/ESU/office/XlsToDetaDBBufferTest.java	2553
/Trif_DNA-master/C_R_U/U_V/ESU/office/XlsToYLJBufferPage.java	2553
/Trif_DNA-master/C_R_U/U_V/ESU/office/XlsToYLJBufferPageBackUp.java	2557
/Trif_DNA-master/C_R_U/U_V/ESU/office/XlsToYLJBufferPageBackUpTest.java	2557
/Trif_DNA-master/C_R_U/U_V/ESU/office/XlsToYLJBufferPageTest.java	2557
/Trif_DNA-master/C_R_U/U_V/ESU/office/XlsxToYLJBufferPage.java	2557
/Trif_DNA-master/C_R_U/U_V/ESU/office/XlsxToYLJBufferPageTest.java	2562
U_V.ESU.string	2562
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_charsetSwap.java	2562
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_charsetSwapTest.java	2562
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_stringArrayToString.java	2562
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_stringArrayToStringTest.java	2563
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_stringToArray.java	2563
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_stringToArrayTest.java	2563
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_stringToCharASCII.java	2563
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_stringToCharASCIITest.java	2563
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_stringToURLencode.java	2563
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_stringToURLencodeTest.java	2564
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_uRLencodeToURIdecode.java	2564
/Trif_DNA-master/C_R_U/U_V/ESU/string/String_ESU_X_uRLencodeToURIdecodeTest.java	2564
/Trif_DNA-master/C_R_U/U_V/ESU/string/StringReplace.java	2564
/Trif_DNA-master/C_R_U/U_V/ESU/string/StringReplaceTest.java	2566
U_V.ESU.stringBuilder	2566
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToCharArray.java	2566
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToCharArrayTest.java	2566
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToJson.java	2566
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToJsonTest.java	2567
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToMap.java	2567
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToMapTest.java	2567
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToSplitArray.java	2567
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToSplitArrayTest.java	2568
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToString.java	2568
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToStringTest.java	2568
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToXml.java	2568
/Trif_DNA-master/C_R_U/U_V/ESU/stringBuilder/S_ESU_X_stringBuilderToXmlTest.java	2568
U_V.ESU.tinos.vector	2568
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToArray.java	2568
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToArrayTest.java	2569
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToIterator.java	2569
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToIteratorTest.java	2569
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToJsonString.java	2569
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToJsonStringTest.java	2570
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToList.java	2570
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToListTest.java	2570
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToMap.java	2570

/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToMapTest.java.....	2571
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToSet.java.....	2571
/Trif_DNA-master/C_R_U/U_V/ESU/tinos/vector/Vector_ESU_X_vectorToSetTest.java.....	2571
U_V.ESU.toolkitImageToBufferImage.....	2571
/Trif_DNA-master/C_R_U/U_V/ESU/toolkitImageToBufferImage/U_X_toolkitImageIconToBufferImage.java.....	2571
/Trif_DNA-master/C_R_U/U_V/ESU/toolkitImageToBufferImage/U_X_toolkitImageIconToBufferImageTest.java.....	2572
/Trif_DNA-master/C_R_U/U_V/ESU/toolkitImageToBufferImage/U_X_toolkitImageToBufferImage.java.....	2572
/Trif_DNA-master/C_R_U/U_V/ESU/toolkitImageToBufferImage/	
U_X_toolkitImageToBufferImageAboutIconConfig.java.....	2572
/Trif_DNA-master/C_R_U/U_V/ESU/toolkitImageToBufferImage/	
U_X_toolkitImageToBufferImageAboutIconConfigTest.java.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/toolkitImageToBufferImage/U_X_toolkitImageToBufferImageTest.java.....	2573
U_V.ESU.tree.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/tree/Tree_ESU_X_stringTreeToJsonString.java.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/tree/Tree_ESU_X_treeToIterator.java.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/tree/Tree_ESU_X_treeToJsonString.java.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/tree/Tree_ESU_X_treeToMap.java.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/tree/Tree_ESU_X_treeToObjectArray.java.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/tree/Tree_ESU_X_treeToXml.java.....	2573
U_V.ESU.txt.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToListString.java.....	2573
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToListStringArray.java.....	2574
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToListStringArrayTest.java.....	2574
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToListStringTest.java.....	2574
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToListStringWithRange.java.....	2574
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToListStringWithRangeTest.java.....	2575
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToObjectMatrix.java.....	2575
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToObjectMatrixTest.java.....	2576
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToObjectMatrixWithRange.java.....	2576
/Trif_DNA-master/C_R_U/U_V/ESU/txt/TXT_ESU_X_txtFileToObjectMatrixWithRangeTest.java.....	2577
U_V.ESU.xml.....	2577
/Trif_DNA-master/C_R_U/U_V/ESU/xml/XML_ESU_X_xmlToHashtable.java.....	2577
/Trif_DNA-master/C_R_U/U_V/ESU/xml/XML_ESU_X_xmlToHashtableTest.java.....	2577
/Trif_DNA-master/C_R_U/U_V/ESU/xml/XML_ESU_X_xmlToList.java.....	2577
/Trif_DNA-master/C_R_U/U_V/ESU/xml/XML_ESU_X_xmlToListTest.java.....	2577
/Trif_DNA-master/C_R_U/U_V/ESU/xml/XML_ESU_X_xmlToSets.java.....	2577
/Trif_DNA-master/C_R_U/U_V/ESU/xml/XML_ESU_X_xmlToSetsTest.java.....	2577
/Trif_DNA-master/C_R_U/U_V/ESU/xml/XML_ESU_X_xmlToVector.java.....	2577
/Trif_DNA-master/C_R_U/U_V/ESU/xml/XML_ESU_X_xmlToVectorTest.java.....	2577
U_V.OEU.xqpx.wcms.sort.....	2577
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CA.java.....	2577
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CAC.java.....	2579
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CACTest.java.....	2579
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CAS_C.java.....	2579
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CAS_CTest.java.....	2580
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CAS.java.....	2580
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CATest.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CUA_C.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CUA.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_CUATest.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PA.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PAC.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PACTest.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PAS_C.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PAS_CTest.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PAS.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PASTest.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PATest.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PUA_C.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PUA_CTest.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PUA.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_PUATest.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_S.java.....	2584
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_SC.java.....	2586
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D13D_X_STest.java.....	2587
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D14D_X_CUA.java.....	2587
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D14D_X_CUATest.java.....	2592
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D14D_X_PUA.java.....	2592
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D14D_X_PUATest.java.....	2592
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D15D_X_CUA.java.....	2592

/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D15D_X_CUATest.java	2592
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D15D_X_PUA.java	2592
/Trif_DNA-master/C_R_U/U_V/OEU/xqpx/wcms/sort/LYG10D15D_X_PUATest.java	2592
1-23-/Trif_DNA-master/C_R_V--网络-文档-读写-肽计算	2592
Gzip	2592
** /Trif_DNA-master/C_R_V/Gzip/GzipUtil_X_compress.java	2592
** /Trif_DNA-master/C_R_V/Gzip/GzipUtil_X_uncompress.java	2592
OSI.OPE.gui	2592
/Trif_DNA-master/C_R_V/OSI/OPE/gui/Sample_X_I_Map.java	2592
/Trif_DNA-master/C_R_V/OSI/OPE/gui/Sample_X_I_MapAddPlugin.java	2604
/Trif_DNA-master/C_R_V/OSI/OPE/gui/Sample_X_I_MapTest.java	2607
/Trif_DNA-master/C_R_V/OSI/OPE/gui/Sample_X_Register.java	2607
/Trif_DNA-master/C_R_V/OSI/OPE/gui/Sample_X_RegisterTest.java	2621
OSI.OPE.OEI.PVI.SOI.SMQ.load	2621
/Trif_DNA-master/C_R_V/OSI/OPE/OEI/PVI/SOI/SMQ/load/File_X_getOrigianlTextByLock.java	2621
/Trif_DNA-master/C_R_V/OSI/OPE/OEI/PVI/SOI/SMQ/load/File_X_getOrigianlTextByLockTest.java	2621
/Trif_DNA-master/C_R_V/OSI/OPE/OEI/PVI/SOI/SMQ/load/File_X_linkNode.java	2621
/Trif_DNA-master/C_R_V/OSI/OPE/OEI/PVI/SOI/SMQ/load/File_X_linkNodeTest.java	2622
/Trif_DNA-master/C_R_V/OSI/OPE/OEI/PVI/SOI/SMQ/load/File_X_Load.java	2622
/Trif_DNA-master/C_R_V/OSI/OPE/OEI/PVI/SOI/SMQ/load/File_X_LoadTest.java	2625
OSI.OPE.OVU.PQE.flash	2625
/Trif_DNA-master/C_R_V/OSI/OPE/OVU/PQE/flash/GUISampleTest.java	2625
/Trif_DNA-master/C_R_V/OSI/OPE/OVU/PQE/flash/ThisCanvas_X_S_ArrowTarget.java	2625
/Trif_DNA-master/C_R_V/OSI/OPE/OVU/PQE/flash/ThisCanvas_X_S_ArrowTargetTest.java	2627
/Trif_DNA-master/C_R_V/OSI/OPE/OVU/PQE/flash/ThisCanvasTest.java	2627
OSI.PVD.SOI.SMQ.delete	2627
/Trif_DNA-master/C_R_V/OSI/PVD/SOI/SMQ/delete/D_File.java	2627
utils	2627
/Trif_DNA-master/C_R_V/utils/DetaUtil.java	2627
/Trif_DNA-master/C_R_V/utils/DetaUtilTest.java	2628
VPC.DetaDB	2629
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_java	2629
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_backEndRequest.java	2629
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_backEndRequestTest.java	2630
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_cacheRequest.java	2630
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_cacheRequestTest.java	2631
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_DBRequest.java	2631
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_DBRequestTest.java	2632
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_IV_CulumnNameType.java	2632
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_IV_CulumnNameTypeTest.java	2632
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_Test.java	2632
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_withoutCulumnNameType.java	2632
/Trif_DNA-master/C_R_V/VPC/DetaDB/DetaDBUtil_X_withoutCulumnNameTypeTest.java	2633
VPC.DetaFrontEnd	2633
/Trif_DNA-master/C_R_V/VPC/DetaFrontEnd/DetaFrontEndUtil_X_backEndRequest.java	2633
/Trif_DNA-master/C_R_V/VPC/DetaFrontEnd/DetaFrontEndUtil_X_backEndRequestTest.java	2633
/Trif_DNA-master/C_R_V/VPC/DetaFrontEnd/DetaFrontEndUtil_X_cacheRequest.java	2633
/Trif_DNA-master/C_R_V/VPC/DetaFrontEnd/DetaFrontEndUtil_X_cacheRequestTest.java	2634
/Trif_DNA-master/C_R_V/VPC/DetaFrontEnd/DetaFrontEndUtil_X_huaRuiJiRequest.java	2634
/Trif_DNA-master/C_R_V/VPC/DetaFrontEnd/DetaFrontEndUtil_X_huaRuiJiRequestTest.java	2636
VPC.PdeSwap	2636
/Trif_DNA-master/C_R_V/VPC/PdeSwap/X_PdcToPde.java	2636
/Trif_DNA-master/C_R_V/VPC/PdeSwap/X_PdcToPdeTest.java	2639
/Trif_DNA-master/C_R_V/VPC/PdeSwap/X_PdcToPds.java	2639
/Trif_DNA-master/C_R_V/VPC/PdeSwap/X_PdcToPdsTest.java	2639
/Trif_DNA-master/C_R_V/VPC/PdeSwap/X_PdeToPds.java	2639
/Trif_DNA-master/C_R_V/VPC/PdeSwap/X_PdeToPdsTest.java	2641
/Trif_DNA-master/C_R_V/VPC/PdeSwap/X_PdsToPde.java	2641
/Trif_DNA-master/C_R_V/VPC/PdeSwap/X_PdsToPdeTest.java	2642
VPC.PdeSwapFix	2642
/Trif_DNA-master/C_R_V/VPC/PdeSwapFix/X_pdcToPde.java	2642
/Trif_DNA-master/C_R_V/VPC/PdeSwapFix/X_pdcToPds.java	2645
/Trif_DNA-master/C_R_V/VPC/PdeSwapFix/X_pdeToPds.java	2645
/Trif_DNA-master/C_R_V/VPC/PdeSwapFix/X_pdsToPde.java	2647
/Trif_DNA-master/C_R_V/VPC/PdeSwapFix/X_pdwToPdc.java	2648
/Trif_DNA-master/C_R_V/VPC/PdeSwapFix/X_textToPdw.java	2649
VPC.transaction	2650
/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_Login.java	2650
/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_LoginDB_DNA.java	2651
/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_LoginDB_DNATest.java	2653

/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_LoginDB.java	2653
/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_LoginTest.java	2653
/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_Register.java	2653
/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_RegisterDB_DNA.java	2654
/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_RegisterDB_DNATest.java	2655
/Trif_DNA-master/C_R_V/VPC/transaction/DelegateDB_X_RegisterTest.java	2655
第二部分-元基花调用部分	2655
/Trif_DNA-master/Blooming	2655
2-1-OSI.OSU	2655
/Trif_DNA-master/Blooming/OSI/OSU/temp.txt	2655
2-2-OSI.OSU.addAction--指令句类注册函数	2655
/Trif_DNA-master/Blooming/OSI/OSU/addAction/AddActionExample.java	2655
/Trif_DNA-master/Blooming/OSI/OSU/addAction/AddActionInterfaceClass.java	2656
/Trif_DNA-master/Blooming/OSI/OSU/addAction/AddFindColumnsInMemoryClass.java	2659
/Trif_DNA-master/Blooming/OSI/OSU/addAction/AddParserMixedStringClass.java	2664
/Trif_DNA-master/Blooming/OSI/OSU/addAction/AddParserMixedStringToListClass.java	2667
/Trif_DNA-master/Blooming/OSI/OSU/addAction/LimitedRowAttributesOfColumnsInMemoryClass.java	2670
/Trif_DNA-master/Blooming/OSI/OSU/addAction/UpdateColorAttributesOfColumnsInMemoryClass.java	2678
/Trif_DNA-master/Blooming/OSI/OSU/addAction/笔记.txt	2684
2-3-OSI.OSU.crab--函数扩展的螃蟹模型	2684
/Trif_DNA-master/Blooming/OSI/OSU/crab/CrabInterface.java	2684
/Trif_DNA-master/Blooming/OSI/OSU/crab/CrabInterfaceClass.java	2686
/Trif_DNA-master/Blooming/OSI/OSU/crab/CrabNode_MCI_MCU_MSI.java	2689
/Trif_DNA-master/Blooming/OSI/OSU/crab/CrabNodePanel.java	2691
/Trif_DNA-master/Blooming/OSI/OSU/crab/CrabNodeRun.java	2692
/Trif_DNA-master/Blooming/OSI/OSU/crab/CrabNodeView.java	2693
/Trif_DNA-master/Blooming/OSI/OSU/crab/config.xml	2694
/Trif_DNA-master/Blooming/OSI/OSU/crab/CrabNode.py	2694
/Trif_DNA-master/Blooming/OSI/OSU/crab/editPaneReader.jpg	2694
第三部分-元基花功能测试部分	2694
3-1-test.java.interfaces.net.http.socket--网络类	2694
/Trif_DNA-master/src/test/java/interfaces/net/http/socket/BoothHVPCS.java	2694
/Trif_DNA-master/src/test/java/interfaces/net/http/socket/ImageUploadTest.java	2695
/Trif_DNA-master/src/test/java/interfaces/net/http/socket/InteractiveWebTest.java	2695
/Trif_DNA-master/src/test/java/interfaces/net/http/socket/RestCallTest.java	2695
/Trif_DNA-master/src/test/java/interfaces/net/http/socket/RestHVPCS.java	2697
/Trif_DNA-master/src/test/java/interfaces/net/http/socket/RestHvpcsServer.java	2699
3-2-test.java.interfaces.test	2700
/Trif_DNA-master/src/test/java/interfaces/test/CommonTestInition.java	2700
3-3-test.java.InterfaceTest.chineseParser--中文切词类	2705
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoAfterPOSTest.java	2705
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoEX.java	2712
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoEX1.java	2716
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoEX2.java	2716
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoEXTest.java	2716
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoPOSTest.java	2716
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/EffectDemoTest.java	2732
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/EffectForestIsolationDemoTest.java	2741
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/FMHMMListOneTime_ETest.java	2752
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap.java	2758
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap1.java	2760
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap2.java	2760
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap3.java	2760
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap4.java	2760
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap5.java	2760
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap6.java	2760
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap7.java	2761
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap8.java	2761
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalInputMap9.java	2761
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap.java	2761
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap1.java	2763
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap2.java	2763
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap3.java	2764
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap4.java	2764
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap5.java	2764
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap6.java	2764
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap7.java	2764

/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap8.java	2764
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/LoadVerbalOutputMap9.java	2764
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/ParserCharsFix.java	2764
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoEXTest_annotation.txt	2770
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoEXTest_output_1.txt	2770
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoEXTest_output_2.txt	2770
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/DemoEXTest_output.txt	2770
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/sample.txt	2770
/Trif_DNA-master/src/test/java/InterfaceTest/chineseParser/分词的工程思想BPM.txt	2770
3-4-test.java.InterfaceTest.computingPDE.linePage--肽腐蚀类, 综合时微分函数,	2770
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/linePage/PageQ_ETest.java	2770
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/linePage/时微分PDE酸性增敏观测.png	2772
3-5-test.java.InterfaceTest.computingPDE.matrixPage--肽计算类,	2772
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/RangeImagePDITest.java	2772
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/RangePDITest.java	2775
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/input_geog.jpg	2778
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/input_horri.jpg	2778
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/output_geog.png	2778
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/output_horri.jpg	2778
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/PDITestinput.jpg	2778
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/PDITestOutput.jpg	2778
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/matrixPage/老教材的骨折图非腐蚀肽组合观测.jpeg ...	2778
3-6-test.java.InterfaceTest.computingPDE.pdi	2778
/Trif_DNA-master/src/test/java/InterfaceTest/computingPDE/pdi/RangePDITest.java	2778
3-7-test.java.InterfaceTest.convolution--卷积类,	2781
/Trif_DNA-master/src/test/java/InterfaceTest/convolution/ClosingTest.java	2781
/Trif_DNA-master/src/test/java/InterfaceTest/convolution/EmbossTest.java	2782
/Trif_DNA-master/src/test/java/InterfaceTest/convolution/GuassianTest.java	2783
/Trif_DNA-master/src/test/java/InterfaceTest/convolution/SobelTest.java	2784
/Trif_DNA-master/src/test/java/InterfaceTest/convolution/StrechTest.java	2786
3-8-test.java.InterfaceTest.coordinatesTracement	2788
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoords1Test.java	2788
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoords2Test.java	2790
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoords3Test.java	2793
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoords4Test.java	2795
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoords5Test.java	2798
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoords6Test.java	2802
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoords7Test.java	2807
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoords8Test.java	2811
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoordsAllTest.java	2813
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/DemoCoordsTest.java	2815
/Trif_DNA-master/src/test/java/InterfaceTest/coordinatesTracement/ TSP2D_X_getYaoguangLuo2DEulerRingTSP2DTest.java	2816
3-9-test.java.InterfaceTest.DNASession	2818
/Trif_DNA-master/src/test/java/InterfaceTest/DNASession/FullDNATokenPDITest.java	2818
/Trif_DNA-master/src/test/java/InterfaceTest/DNASession/FullDNATokenPDITestTest.java	2824
3-10-test.java.InterfaceTest.initon.util	2832
/Trif_DNA-master/src/test/java/InterfaceTest/initon/util/FindAllPackageFiles.java	2832
/Trif_DNA-master/src/test/java/InterfaceTest/initon/util/FindAllProjectFiles.java	2834
3-11-test.java.InterfaceTest.sort.arraySort.chineseSort	2852
/Trif_DNA-master/src/test/java/InterfaceTest/sort/arraySort/chineseSort/LYG10D13D_X_CASTest.java	2852
3-12-test.java.InterfaceTest.sort.arraySort.LYG4DQS4D --排序类,	2853
/Trif_DNA-master/src/test/java/InterfaceTest/sort/arraySort/LYG4DQS4D/LYG9DWithDoubleTopSort4DTest.java	2853
/Trif_DNA-master/src/test/java/InterfaceTest/sort/arraySort/LYG4DQS4D/LYG9DWithDoubleTopSort5DTest.java	2854
/Trif_DNA-master/src/test/java/InterfaceTest/sort/arraySort/LYG4DQS4D/LYG9DWithFloatTopSort5D.java	2857
/Trif_DNA-master/src/test/java/InterfaceTest/sort/arraySort/LYG4DQS4D/LYG9DWithIntTopSort5D.java	2858
/Trif_DNA-master/src/test/java/InterfaceTest/sort/arraySort/LYG4DQS4D/LYG9DWithLongTopSort5D.java	2860
3-13-test.java.InterfaceTest.sort.arraySort.reflection.IntStringSort	2862
/Trif_DNA-master/src/test/java/InterfaceTest/sort/arraySort/reflection/IntStringSort/ Quick9DLYGWithString_ESUTest.java	2862
3-14-test.java.InterfaceTest.sort.objectSort.listSort	2868
/Trif_DNA-master/src/test/java/InterfaceTest/sort/objectSort/listSort/List_ESU_X_listToArrayTest.java	2868
3-15-test.java.InterfaceTest.sort.objectSort.mapSort	2870
/Trif_DNA-master/src/test/java/InterfaceTest/sort/objectSort/mapSort/Map_ESU_X_mapToListTest.java	2870
3-16-test.java.InterfaceTest.TextDNN	2872

/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/ANNTestTest.java	2872
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/DemoTSLTTest.java	2876
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/DNNTestTest.java	2878
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/EducationLevelTestTest.java	2881
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/InitBehaviorICAKernelTest.java	2883
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/LenovoInitTest.java	2885
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/LiterarinessLevelTestTest.java	2889
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/RestNLPPortImplTest.java	2891
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/RNN_IDETestTest.java	2893
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/SensingTestTest.java	2896
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/SuccessICATestTest.java	2899
/Trif_DNA-master/src/test/java/InterfaceTest/TextDNN/SampleDNN.png	2900
test.java.InterfaceTest.timeNorms.jniLYGAFDCDFFT	2900
/Trif_DNA-master/src/test/java/InterfaceTest/timeNorms/jniLYGAFDCDFFT/LYGAFDCDFFT_FTest.java	2900
/Trif_DNA-master/src/test/java/InterfaceTest/timeNorms/jniLYGAFDCDFFT/timeNor.png	2904
/Trif_DNA-master/src/test/java/InterfaceTest/timeNorms/jniLYGAFDCDFFT/timeNor1.png	2904
3-17-test.java.InterfaceTest.tinShell	2904
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/FastCartesianIdentifyTest.java	2904
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/FlowerTalkEncodingTest.java	2921
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/ShellJPanelSeparationTest.java	2926
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/ShellJPanelTest.java	2936
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/SimpleTinShellLineTest.java	2953
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/SortFlowerTalkTest.java	2955
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/StaticFunctionMapQ_VECS_ETest.java	2958
3-18-test.java.InterfaceTest.tinShell.bootBatchFilePLETL	2961
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/bootBatchFilePLETL/Sample_X_I_MapAddPluginTest.java	2961
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/bootBatchFilePLETL/tinshellExample.etl	2968
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/bootBatchFilePLETL/tinshellExample.etl.dna	2968
/Trif_DNA-master/src/test/java/InterfaceTest/tinShell/bootBatchFilePLETL/tinshellSample.jpeg	2968
3-19-test.java.InterfaceTest.ui.bootETL	2968
/Trif_DNA-master/src/test/java/InterfaceTest/ui/bootETL/BootETLTest.java	2968
/Trif_DNA-master/src/test/java/InterfaceTest/ui/bootETL/ETLSample.jpeg	2971
源码著作权溯源	2972

前言

这个源码工程对应古拉丁文元基编码的华瑞集最后一个版本，用来索引对照用途。之后全部函数名走元基索引。方便我之后快速比对还原。

源码包含3个部分

第一部分 23组模拟染色体索引，将之前的著作权文件对应的函数全部整合在一起然后按函数级别的功能分类排列。

--这些函数主要来自于 德塔分词，微分催化排序，极速象契排序，拼音笔画排序，java卷积分析，VPCS服务器，数据库，tinshell，三维词汇花罗盘，tinshell ETL，坐标轨迹欧拉计算，数据变换，元基花新陈代谢 等罗瑶光先生个人著作权作品。

第二部分是 函数的扩展方式，目前支持直接编码，继承编码 和 OSGI 插件编码。

第三部分是函数的测试方式，目前是民用功能类测试，包含了大量的第一部分函数集的组合使用案例。我会在7版本中包含更丰富的web测试和sonar测试接地气，

目录中加了**开头的函数 涉及我在网上，书籍，借鉴和摘录，我依旧会标记进行 refer，这些refer，我之前的40余著作权文件都有详细地描述。我保留这个标题头，但是在文章中我会全部删除，因为3000页多不在乎这2页。另外一些特别简单逻辑的充数类函数这里全部剔除，因为已经3000多页了。。

--个人作者:罗瑶光--

/*

* 个人著作权人, 第一作者: 罗瑶光, 1985年5月25日出生，性别男, 湖南省浏阳市人.

* --浏阳德塔软件开发有限公司-创始人，法人，总经理-- 永久非盈利--

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

* 15116110525- (唯一标识 2018年2月开始使用此手机卡至今从未拓机，欠费和关机，手机是实名身份证发票机，警惕我家附近类似GSM电子狗频率充斥欺诈)

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 当前居住地址: 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

* --最近思考家的地址从2009年的北岭美林花苑9号 改成石霜路阳光花园9号，又石霜路阳光花园10号，又大塘路阳光家园10号，再大塘路一段阳光家园第10栋到现在集里大塘冲路一段208阳光家园别墅小区第10栋，我在想，频繁换名，有问题，于是准确介绍下家地址。

* --湖南省浏阳市集里街道-大塘冲路一段208阳光家园别墅小区第10栋别墅--，位于大塘冲路一段208阳光家园小区内的别墅房区西南门入口内左侧，靠近美林花苑入口北门大门（美林花苑小区内见到阳光家园的最前面金属围栏后便是(10号-面朝建筑左户-罗瑶光家)和（11号-面朝建筑右户）2户联体别墅），美林花苑小区位于大塘冲路一段的十字路口，十字路线的以点轴心斜对面是--浏阳市 大塘实验(普特融合)小学 建筑--。

* --身份证上地址2011年后姑姑罗庆华一家入住。中国身份证件标注地址是户籍地址不是居民地址，罗瑶光先生2018年后一直居住在阳光家园的第三层。

*/

第一部分-元基索引花函数

1-1-/Trif_DNA-master/C_L_A-分析-肽计算-欧拉环路-六元催化接口

A_I.MPE.EDVQMSAXQCTI

/Trif_DNA-master/C_L_A/A_I/MPE/EDVQMSAXQCTI/AOPM_VECS_IDUQ.java

A_I.MPE.MAQPVDCOISIV.full

```
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDCOISIV/full/FullDNATokenPDI_do_PDE_RNA_FullFormular_FullBack.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDCOISIV/full/FullDNATokenPDI.java
package A_I.MPE.MAQPVDCOISIV.full;
```

```
import A_I.MPE.MAQPVDCOISIV.initon.Initon;
```

```
import A_I.MPE.MAQPVDCOISIV.initon.InitonLinkDNA;
```

```
import A_I.MPE.MAQPVDCOISIV.pde.PDE_Decrement_FullDNAFormular;
```

```
import A_I.MPE.MAQPVDCOISIV.pde.PDE_Decrement_FullFormular;
```

```
import A_I.MPE.MAQPVDCOISIV.pde.PDE_Increment_FullDNAFormular;
```

```
import A_I.MPE.MAQPVDCOISIV.pde.PDE_Increment_FullFormular;
```

```
import S_A.SVQ.stable.S_Common;
```

```
import S_A.pheromone.IMV_SQI;
```

```
/*
```

```
* 个人著作权人，作者 罗瑶光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.

//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

```
public class FullDNATokenPDI {
    public double[] key = new double[4];
    public String bys = "";
    public String pdw = "";
    public String pds = "";
    public String pde = "";
    public String time = "";
    public String session_key = "";
    public String text;
    public String cacheId;
    public String code = "";
    public String lock = "";
    public String pdedeKey = "";
    public String pdedsKey = "";
    public String pdeicKey = "";
    public String pdeisKey = "";
```

```

public void doKeyPress(String initons, FullDNATokenPDI pDE_RNA_FullFormular,
        boolean bYS) {
    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {
        if (initon[i] == null) {
            initon[i] = new Initon();
        }
        initon[i].I_Initon("" + initons.charAt(i));
        if (i + 1 < initons.length()) {
            if (initon[i + 1] == null) {
                initon[i + 1] = new Initon();
            }
            initon[i].next = initon[i + 1];
            initon[i + 1].prev = initon[i];
        }
    }
    if (null != initons && initons.length() > 0) {
        do_pDE_RNA_FullFormular(initon[0], pDE_RNA_FullFormular, bYS);
    }
}

public String[] encodeDNA(String humanWord) {
    String[] string = new String[5];
    // 0e 1s 2 humanword 3 seccode 4dna initons
    FullDNATokenPDI pDE_RNA_FullFormular = new FullDNATokenPDI();
    pDE_RNA_FullFormular.code = pDE_RNA_FullFormular.initonSect(humanWord);
    // 20230106-S_logger.Log.logger.info("" + pDE_RNA_FullFormular.code);
    pDE_RNA_FullFormular.enSecCode(pDE_RNA_FullFormular, false);
    pDE_RNA_FullFormular.pdedeKey = pDE_RNA_FullFormular.pdeieKey;
    pDE_RNA_FullFormular.pdedesKey = pDE_RNA_FullFormular.pdeisKey;
    pDE_RNA_FullFormular.code = pDE_RNA_FullFormular.pde;
    pDE_RNA_FullFormular.pde = "";
    pDE_RNA_FullFormular.deSecCode(pDE_RNA_FullFormular, true);
    string[0] = pDE_RNA_FullFormular.pdeieKey;
    string[1] = pDE_RNA_FullFormular.pdeisKey;
    string[2] = humanWord;
    string[3] = pDE_RNA_FullFormular.code;
    string[4] = pDE_RNA_FullFormular.pde;
    // 20230106-S_logger.Log.logger.info("" + pDE_RNA_FullFormular.pde);
    return string;
}

public void deSecCode(FullDNATokenPDI pDE_RNA_FullFormular, boolean b) {
    //
    String initons = pDE_RNA_FullFormular.code;
    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {
        if (initon[i] == null) {
            initon[i] = new Initon();

```

```

    }
    initon[i].I_Initon("" + initons.charAt(i));
    if (i + 1 < initons.length()) {
if (initon[i + 1] == null) {
    initon[i + 1] = new Initon();
}
initon[i].next = initon[i + 1];
initon[i + 1].prev = initon[i];
    }
}
if (null != initons && initons.length() > 0) {
    de_PDE_RNA_FullFormularForDB(initon[0], pDE_RNA_FullFormular, true);
    // do_PDE_RNA_FullFormular_FullBack(initon[0],
    // pDE_RNA_FullFormular, true);
}
}

// Initon InitonPDE1A= doIncrementA(InitonPDE1S, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1O= doIncrementO(InitonPDE1A, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1P= doIncrementP(InitonPDE1O, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE2= doIncrementM(InitonPDE1P, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDEM= doDecrementM(InitonPDE2,
// initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDEP= doDecrementP(InitonPDEM, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDEO= doDecrementO(InitonPDEP, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDEA= doDecrementA(InitonPDEO, initonLinkDNA, pDE_RNA_FullFormular);
//
// Initon InitonPDES= doDecrementS(InitonPDEA, initonLinkDNA, pDE_RNA_FullFormular, bYS);
// Initon InitonPDEC= doDecrementC(InitonPDES, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDEE= doDecrementE(InitonPDEC, initonLinkDNA, pDE_RNA_FullFormular, bYS);
// Initon InitonPDE1= doDecrementV(InitonPDEE, initonLinkDNA, pDE_RNA_FullFormular);
// while(InitonPDE1.hasNext()) {
//pDE_RNA_FullFormular.pds+= InitonPDE1.getStore();
//InitonPDE1= InitonPDE1.next;
//    }
//    pDE_RNA_FullFormular.pds+= InitonPDE1.getStore();
//    while(InitonPDE1.hasPrev()) {
//InitonPDE1= InitonPDE1.prev;
//    }

private void de_PDE_RNA_FullFormularForDB(Initon initon,
    FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
    Initon InitonPDE = initon;
    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
    Initon InitonPDE1V = doIncrementV(InitonPDE, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDE1E = doIncrementE(InitonPDE1V, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
    Initon InitonPDE1C = doIncrementC(InitonPDE1E, initonLinkDNA,

```

```

pDE_RNA_FullFormular);

    Initon InitonPDE1S = doIncrementS(InitonPDE1C, initonLinkDNA,
pDE_RNA_FullFormular, bYS);

    while (InitonPDE1S.hasNext()) {
        pDE_RNA_FullFormular.pde += InitonPDE1S.getStore();
        InitonPDE1S = InitonPDE1S.next;
    }

    pDE_RNA_FullFormular.pde += InitonPDE1S.getStore();
    while (InitonPDE1S.hasPrev()) {
        InitonPDE1S = InitonPDE1S.prev;
    }
}

public void enSecCode(FullDNATokenPDI pDE_RNA_FullFormular, boolean bys) {
    String initons = pDE_RNA_FullFormular.code;
    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {
        if (initon[i] == null) {
            initon[i] = new Initon();
        }
        initon[i].I_Initon("" + initons.charAt(i));
        if (i + 1 < initons.length()) {
            if (initon[i + 1] == null) {
                initon[i + 1] = new Initon();
            }
            initon[i].next = initon[i + 1];
            initon[i + 1].prev = initon[i];
        }
    }
    if (null != initons && initons.length() > 0) {
        do_PDE_RNA_FullFormularForDB(initon[0], pDE_RNA_FullFormular, bys);
    }
}

//      public static void main(String[] argv) {
//FullDNATokenPDI pDE_RNA_FullFormular= new FullDNATokenPDI();
//String a= "luoyaoguang";
//pDE_RNA_FullFormular.encodeDNA(a);
//
//@SuppressWarnings("unused")
//String initonKeys= "EIU/0.6/EDU/0.4/si/0.3/sq/0.7/EIU/0.5/EDU/0.5/si/0.632/sq/0.368";
//pDE_RNA_FullFormular.key[0]= 0.6;
//pDE_RNA_FullFormular.key[1]= 0.3;
//pDE_RNA_FullFormular.key[2]= 0.5;
//pDE_RNA_FullFormular.key[3]= 0.632;
//
//pDE_RNA_FullFormular.text= "控制吸收";
//pDE_RNA_FullFormular.pdw= pDE_RNA_FullFormular.IV_onSect(pDE_RNA_FullFormular.text);
////20230106-S_logger.Log.logger.info("" + "原文: " + pDE_RNA_FullFormular.text);

```

```
////pDE_RNA_FullFormular.pdw= "字典保密: MSIOCUOCIPCUPCI";

//String[] lock= new String[12];

//    lock[0]= "A"; lock[3]= "O"; lock[6]= "P"; lock[9]= "M";

//    lock[1]= "V"; lock[4]= "E"; lock[7]= "C"; lock[10]= "S";

//    lock[2]= "I"; lock[5]= "D"; lock[8]= "U"; lock[11]= "Q";

//    int i= (int)(Math.random()* 12)% 12;

//    pDE_RNA_FullFormular.lock+= lock[i];

//    i= (int)(Math.random()* 12)% 12;

//    pDE_RNA_FullFormular.lock+= lock[i];

//    i= (int)(Math.random()* 12)% 12;

//    pDE_RNA_FullFormular.lock+= lock[i];

//    i= (int)(Math.random()* 12)% 12;

//    pDE_RNA_FullFormular.lock+= lock[i];

//

//for(i= 0; i< pDE_RNA_FullFormular.pdw.length(); i++) {

//    pDE_RNA_FullFormular.code+= pDE_RNA_FullFormular.lock

//        + pDE_RNA_FullFormular.pdw.charAt(i);

//}

////20230106-S_logger.Log.logger.info("'" + "肽语: " + pDE_RNA_FullFormular.pdw);

////20230106-S_logger.Log.logger.info("'" + "肽锁: " + pDE_RNA_FullFormular.lock);

////20230106-S_logger.Log.logger.info("'" + "散列肽语: " + pDE_RNA_FullFormular.code);

//pDE_RNA_FullFormular.bys= "0.6/0.3/0.5/0.632";

////20230106-S_logger.Log.logger.info("'" + "静态密钥: " + pDE_RNA_FullFormular.bys);

//pDE_RNA_FullFormular.doKeyPress(pDE_RNA_FullFormular.code, pDE_RNA_FullFormular, false);

////20230106-S_logger.Log.logger.info("'" + "静态肽展降元概率钥匙E: " + pDE_RNA_FullFormular.pdedeKey);

////20230106-S_logger.Log.logger.info("'" + "静态肽展降元概率钥匙S: " + pDE_RNA_FullFormular.pdedeKey);

////20230106-S_logger.Log.logger.info("'" + "静态肽展降元: " + pDE_RNA_FullFormular.pds);

//

////20230106-S_logger.Log.logger.info("'" + "静态肽展增元概率钥匙E: " + pDE_RNA_FullFormular.pdeieKey);

////20230106-S_logger.Log.logger.info("'" + "静态肽展增元概率钥匙S: " + pDE_RNA_FullFormular.pdeisKey);

////20230106-S_logger.Log.logger.info("'" + "静态肽展增元: " + pDE_RNA_FullFormular.pde);

//

//pDE_RNA_FullFormular.time= "" + System.currentTimeMillis();

//pDE_RNA_FullFormular.cacheId= "ID" + Math.random()

//    + S_Common.STRING_SYMBOL_PER + Math.random();

////20230106-S_logger.Log.logger.info("'" + "时间: " + pDE_RNA_FullFormular.time);

////20230106-S_logger.Log.logger.info("'" + "账号随机缓存字符串: " + pDE_RNA_FullFormular.cacheId);

//pDE_RNA_FullFormular.session_key= pDE_RNA_FullFormular.pde;

////20230106-S_logger.Log.logger.info("'" + "Session: " + pDE_RNA_FullFormular.session_key);

////20230106-S_logger.Log.logger.info("'" + "=====");

////20230106-S_logger.Log.logger.info("'" + "开始前序验证: ");

////20230106-S_logger.Log.logger.info("'" + "开始Session解析: " + pDE_RNA_FullFormular.session_key);

////20230106-S_logger.Log.logger.info("'" + "开始概率钥匙解析: " + pDE_RNA_FullFormular.pdedeKey

//    + pDE_RNA_FullFormular.pdedeKey

//    + pDE_RNA_FullFormular.pdeieKey+ pDE_RNA_FullFormular.pdeisKey);

//

//FullDNATokenPDI pDE_RNA_FullFormular1= new FullDNATokenPDI();
```



```
//pDE_RNA_FullFormular1.pdedeKey= pDE_RNA_FullFormular.pdedeKey.toString();
//pDE_RNA_FullFormular1.pdedsKey= pDE_RNA_FullFormular.pdedsKey.toString();
//pDE_RNA_FullFormular1.pdeieKey= pDE_RNA_FullFormular.pdeieKey.toString();
//pDE_RNA_FullFormular1.pdeisKey= pDE_RNA_FullFormular.pdeisKey.toString();
//
//pDE_RNA_FullFormular.doKeyUnPress(pDE_RNA_FullFormular.code, pDE_RNA_FullFormular1, true);
////20230106-S_logger.Log.logger.info(" " + "");
////20230106-S_logger.Log.logger.info(" " + "得到原降元元基DNA序列: " + pDE_RNA_FullFormular.pds);
////20230106-S_logger.Log.logger.info(" " + "得到新降元元基DNA序列: " + pDE_RNA_FullFormular1.pds);
////20230106-S_logger.Log.logger.info(" " + "得到原元基DNA序列: " + pDE_RNA_FullFormular.pde);
////20230106-S_logger.Log.logger.info(" " + "得到新元基DNA序列: " + pDE_RNA_FullFormular1.pde);
////20230106-S_logger.Log.logger.info(" " + "验证正确? ");
////20230106-S_logger.Log.logger.info(" " + pDE_RNA_FullFormular.pde.equals(pDE_RNA_FullFormular1.pde)
// ? "正确": "失败");
/////20230106-S_logger.Log.logger.info(" " + "=====");
////20230106-S_logger.Log.logger.info(" " + "开始后序验证: ");
//FullDNATokenPDI pDE_RNA_FullFormular2= new FullDNATokenPDI();
//pDE_RNA_FullFormular2.pdeieKey= pDE_RNA_FullFormular.pdedeKey.toString();
//pDE_RNA_FullFormular2.pdeisKey= pDE_RNA_FullFormular.pdedsKey.toString();
//pDE_RNA_FullFormular2.pdedeKey= pDE_RNA_FullFormular.pdeieKey.toString();
//pDE_RNA_FullFormular2.pdedsKey= pDE_RNA_FullFormular.pdeisKey.toString();
////20230106-S_logger.Log.logger.info(" " + "准备计算元基DNA序列: " + pDE_RNA_FullFormular1.pde);
//pDE_RNA_FullFormular2.doSessionKeyUnPress(pDE_RNA_FullFormular1.pde
// , pDE_RNA_FullFormular2, true);
////20230106-S_logger.Log.logger.info(" " + "");
////20230106-S_logger.Log.logger.info(" " + "得到原续降元元基DNA序列: " + pDE_RNA_FullFormular1.pds);
////20230106-S_logger.Log.logger.info(" " + "得到后续降元元基DNA序列: " + pDE_RNA_FullFormular2.pds);
////20230106-S_logger.Log.logger.info(" " + "验证正确? ");
////20230106-S_logger.Log.logger.info(" " + pDE_RNA_FullFormular1.pds.equals(pDE_RNA_FullFormular2.pds)
// ? "正确": "失败");
//
////20230106-S_logger.Log.logger.info(" " + "=====");
////20230106-S_logger.Log.logger.info(" " + "开始整序验证: ");
//FullDNATokenPDI pDE_RNA_FullFormular3= new FullDNATokenPDI();
//
//
//pDE_RNA_FullFormular3.pdeieKey= pDE_RNA_FullFormular.pdeieKey.toString();
//pDE_RNA_FullFormular3.pdeisKey= pDE_RNA_FullFormular.pdeisKey.toString();
//pDE_RNA_FullFormular3.pdedeKey= pDE_RNA_FullFormular.pdeieKey.toString();
//pDE_RNA_FullFormular3.pdedsKey= pDE_RNA_FullFormular.pdeisKey.toString();
//
//
////20230106-S_logger.Log.logger.info(" " + "准备计算元基DNA序列: " + pDE_RNA_FullFormular1.pde);//pde加成
//pDE_RNA_FullFormular3.doFullSessionKeyUnPress(pDE_RNA_FullFormular1.pde
// , pDE_RNA_FullFormular3, true);
////20230106-S_logger.Log.logger.info(" " + "");
////20230106-S_logger.Log.logger.info(" " + "得到原续降元元基DNA序列: " + pDE_RNA_FullFormular1.pds);
```

```

////20230106-S_logger.Log.logger.info("'" + "得到后续降元元基DNA序列: " + pDE_RNA_FullFormular3.pds);
////20230106-S_logger.Log.logger.info("'" + "验证正确? ");
////20230106-S_logger.Log.logger.info("'" + pDE_RNA_FullFormular1.pds.equals(pDE_RNA_FullFormular3.pds)
//? "正确": "失败");
////20230106-S_logger.Log.logger.info("'" + "准备整序计算元基DNA序列: " + pDE_RNA_FullFormular1.pde);
////20230106-S_logger.Log.logger.info("'" + "准备整序计算元基DNA序列: " + pDE_RNA_FullFormular3.pde);
////20230106-S_logger.Log.logger.info("'" + pDE_RNA_FullFormular1.pde.equals(pDE_RNA_FullFormular3.pde)
//? "正确": "失败");
//
//    }
//
public void doFullSessionKeyUnPress(String initons,
    FullDNATokenPDI pDE_RNA_FullFormular3, boolean bYS) {
    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {
        if (initon[i] == null) {
            initon[i] = new Initon();
        }
        initon[i].I_Initon("'" + initons.charAt(i));
        if (i + 1 < initons.length()) {
            if (initon[i + 1] == null) {
                initon[i + 1] = new Initon();
            }
            initon[i].next = initon[i + 1];
            initon[i + 1].prev = initon[i];
        }
    }
    do_PDE_RNA_FullFormular_FullBack(initon[0], pDE_RNA_FullFormular3, bYS);
}
private void do_PDE_RNA_FullFormular_FullBack(Initon initon,
    FullDNATokenPDI pDE_RNA_FullFormular3, boolean bYS) {
    Initon InitonPDE = initon;
    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
    Initon InitonPDE1V = doIncrementV(InitonPDE, initonLinkDNA,
pDE_RNA_FullFormular3);
    Initon InitonPDE1E = doIncrementE(InitonPDE1V, initonLinkDNA,
pDE_RNA_FullFormular3, bYS);
    Initon InitonPDE1C = doIncrementC(InitonPDE1E, initonLinkDNA,
pDE_RNA_FullFormular3);
    Initon InitonPDE1S = doIncrementS(InitonPDE1C, initonLinkDNA,
pDE_RNA_FullFormular3, bYS);
    Initon InitonPDE1A = doIncrementA(InitonPDE1S, initonLinkDNA,
pDE_RNA_FullFormular3);
    Initon InitonPDE1O = doIncrementO(InitonPDE1A, initonLinkDNA,
pDE_RNA_FullFormular3);
    Initon InitonPDE1P = doIncrementP(InitonPDE1O, initonLinkDNA,
pDE_RNA_FullFormular3);
    Initon InitonPDE2 = doIncrementM(InitonPDE1P, initonLinkDNA,

```

```

pDE_RNA_FullFormular3);

    while (InitonPDE2.hasNext()) {
        pDE_RNA_FullFormular3.pde += InitonPDE2.getStore();
        InitonPDE2 = InitonPDE2.next;
    }

    pDE_RNA_FullFormular3.pde += InitonPDE2.getStore();
    while (InitonPDE2.hasPrev()) {
        InitonPDE2 = InitonPDE2.prev;
    }

    Initon InitonPDEM = doDecrementM(InitonPDE2, initonLinkDNA,
pDE_RNA_FullFormular3);

    Initon InitonPDEP = doDecrementP(InitonPDEM, initonLinkDNA,
pDE_RNA_FullFormular3);

    Initon InitonPDEO = doDecrementO(InitonPDEP, initonLinkDNA,
pDE_RNA_FullFormular3);

    Initon InitonPDEA = doDecrementA(InitonPDEO, initonLinkDNA,
pDE_RNA_FullFormular3);

    Initon InitonPDES = doDecrementS(InitonPDEA, initonLinkDNA,
pDE_RNA_FullFormular3, bYS);

    Initon InitonPDEC = doDecrementC(InitonPDES, initonLinkDNA,
pDE_RNA_FullFormular3);

    Initon InitonPDEE = doDecrementE(InitonPDEC, initonLinkDNA,
pDE_RNA_FullFormular3, bYS);

    Initon InitonPDE1 = doDecrementV(InitonPDEE, initonLinkDNA,
pDE_RNA_FullFormular3);

    while (InitonPDE1.hasNext()) {
        pDE_RNA_FullFormular3.pds += InitonPDE1.getStore();
        InitonPDE1 = InitonPDE1.next;
    }

    pDE_RNA_FullFormular3.pds += InitonPDE1.getStore();
    while (InitonPDE1.hasPrev()) {
        InitonPDE1 = InitonPDE1.prev;
    }
}

public void doSessionKeyUnPress(String initons,
    FullDNATokenPDI pDE_RNA_FullFormular2, boolean bYS) {

    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {
        if (initon[i] == null) {
            initon[i] = new Initon();
        }
        initon[i].I_Initon("'" + initons.charAt(i));
        if (i + 1 < initons.length()) {
            if (initon[i + 1] == null) {
                initon[i + 1] = new Initon();
            }
            initon[i].next = initon[i + 1];
        }
    }
}

```

```

        initon[i + 1].prev = initon[i];
    }
}

do_PDE_RNA_FullFormular_Back(initon[0], pDE_RNA_FullFormular2, bYS);
}

private void do_PDE_RNA_FullFormular_Back(Initon initon,
    FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
    Initon InitonPDE = initon;
    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
    Initon InitonPDEM = doDecrementM(InitonPDE, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEP = doDecrementP(InitonPDEM, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEO = doDecrementO(InitonPDEP, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEA = doDecrementA(InitonPDEO, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDES = doDecrementS(InitonPDEA, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
    Initon InitonPDEC = doDecrementC(InitonPDES, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEE = doDecrementE(InitonPDEC, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
    Initon InitonPDE1 = doDecrementV(InitonPDEE, initonLinkDNA,
pDE_RNA_FullFormular);
    while (InitonPDE1.hasNext()) {
        pDE_RNA_FullFormular.pds += InitonPDE1.getStore();
        InitonPDE1 = InitonPDE1.next;
    }
    pDE_RNA_FullFormular.pds += InitonPDE1.getStore();
    while (InitonPDE1.hasPrev()) {
        InitonPDE1 = InitonPDE1.prev;
    }
}

// Initon InitonPDE1S= doIncrementS(InitonPDE1, initonLinkDNA, pDE_RNA_FullFormular, bYS);
// Initon InitonPDE1C= doIncrementC(InitonPDE1S, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1E= doIncrementE(InitonPDE1C, initonLinkDNA, pDE_RNA_FullFormular, bYS);
// Initon InitonPDE1V= doIncrementV(InitonPDE1E, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1M= doIncrementM(InitonPDE1V, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1P= doIncrementP(InitonPDE1M, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1O= doIncrementO(InitonPDE1P, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE2= doIncrementA(InitonPDE1O, initonLinkDNA, pDE_RNA_FullFormular);
// while(InitonPDE2.hasNext()) {
//pDE_RNA_FullFormular.pde+= InitonPDE2.getStore();
//InitonPDE2= InitonPDE2.next;
// }
// pDE_RNA_FullFormular.pde+= InitonPDE2.getStore();

```

```

//      while(InitonPDE2.hasPrev()) {
//InitonPDE2= InitonPDE2.prev;
//      }

    public void doKeyUnPress(String initons,
        FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
        Initon[] initon = new Initon[initons.length()];
        for (int i = 0; i < initons.length(); i++) {
            if (initon[i] == null) {
                initon[i] = new Initon();
            }
            initon[i].I_Initon("" + initons.charAt(i));
            if (i + 1 < initons.length()) {
                if (initon[i + 1] == null) {
                    initon[i + 1] = new Initon();
                }
                initon[i].next = initon[i + 1];
                initon[i + 1].prev = initon[i];
            }
        }
        do_PDE_RNA_FullFormular(initon[0], pDE_RNA_FullFormular, bYS);
    }
// Initon InitonPDE1A= doIncrementA(InitonPDE1S,
// initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1O= doIncrementO(InitonPDE1A, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1P= doIncrementP(InitonPDE1O, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE2= doIncrementM(InitonPDE1P, initonLinkDNA, pDE_RNA_FullFormular);
    private void do_PDE_RNA_FullFormularForDB(Initon initon,
        FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
        Initon InitonPDE = initon;
        InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
        Initon InitonPDE1V = doDecrementV(InitonPDE, initonLinkDNA,
pDE_RNA_FullFormular);
        Initon InitonPDE1E = doDecrementE(InitonPDE1V, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
        Initon InitonPDE1C = doDecrementC(InitonPDE1E, initonLinkDNA,
pDE_RNA_FullFormular);
        Initon InitonPDE1S = doDecrementS(InitonPDE1C, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
        while (InitonPDE1S.hasNext()) {
            pDE_RNA_FullFormular.pde += InitonPDE1S.getStore();
            InitonPDE1S = InitonPDE1S.next;
        }
        pDE_RNA_FullFormular.pde += InitonPDE1S.getStore();
        while (InitonPDE1S.hasPrev()) {
            InitonPDE1S = InitonPDE1S.prev;
        }
    }
}

```

```

public void do_PDE_RNA_FullFormular(Initon initon,
    FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
    Initon InitonPDE = initon;
    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
    // a->b
    Initon InitonPDEA = doDecrementA(InitonPDE, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEO = doDecrementO(InitonPDEA, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEP = doDecrementP(InitonPDEO, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEM = doDecrementM(InitonPDEP, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEV = doDecrementV(InitonPDEM, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEE = doDecrementE(InitonPDEV, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
    Initon InitonPDEC = doDecrementC(InitonPDEE, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDE1 = doDecrementS(InitonPDEC, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
    while (InitonPDE1.hasNext()) {
        pDE_RNA_FullFormular.pds += InitonPDE1.getStore();
        InitonPDE1 = InitonPDE1.next;
    }
    pDE_RNA_FullFormular.pds += InitonPDE1.getStore();
    while (InitonPDE1.hasPrev()) {
        InitonPDE1 = InitonPDE1.prev;
    }
    // b->c
    Initon InitonPDE1V = doIncrementV(InitonPDE1, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDE1E = doIncrementE(InitonPDE1V, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
    Initon InitonPDE1C = doIncrementC(InitonPDE1E, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDE1S = doIncrementS(InitonPDE1C, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
    Initon InitonPDE1A = doIncrementA(InitonPDE1S, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDE1O = doIncrementO(InitonPDE1A, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDE1P = doIncrementP(InitonPDE1O, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDE2 = doIncrementM(InitonPDE1P, initonLinkDNA,
pDE_RNA_FullFormular);
    while (InitonPDE2.hasNext()) {

```

```

        pDE_RNA_FullFormular.pde += InitonPDE2.getStore();
        InitonPDE2 = InitonPDE2.next;
    }
    pDE_RNA_FullFormular.pde += InitonPDE2.getStore();
    while (InitonPDE2.hasPrev()) {
        InitonPDE2 = InitonPDE2.prev;
    }
}

/////INITONS SWAP

public Initon doIncrementA(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementA(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-System.out.print("A->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doIncrementO(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementO(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-System.out.print("O->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
}

```

```

    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doIncrementP(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementP(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("P-");
    // 20230106-System.out.print("P->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doIncrementM(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementM(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("M-");
    // 20230106-System.out.print("M->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }

```



```
}
    return InitonPDE;
}

public Initon doIncrementV(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementV(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("V-");
    // 20230106-System.out.print("V->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doIncrementC(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("C-");
    // 20230106-System.out.print("C->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}
```

```
public Initon doIncrementE(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullDNAFormular()
.PDE_IncrementE_DU(initonLinkDNA, pDE_RNA_FullFormular, bYS);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info(" " + " ");
    // 20230106-System.out.print("E->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doIncrementS(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullDNAFormular()
.PDE_IncrementS_IQ(initonLinkDNA, pDE_RNA_FullFormular);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info(" " + " ");
    // 20230106-System.out.print("S->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doDecrementA(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
```

```

        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementA(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-System.out.print("A->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doDecrementO(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementO(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-System.out.print("O->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doDecrementP(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);

```

```
        InitonPDE = new PDE_Decrement_FullFormular()

        .PDE_DecrementP(intonLinkDNA);

        while (InitonPDE.hasPrev()) {

            InitonPDE = InitonPDE.prev;

        }

        // 20230106-S_logger.Log.logger.info(" " + " ");

        // 20230106-System.out.print("P->");

        while (InitonPDE.hasNext()) {

            // 20230106-System.out.print(InitonPDE.getStore());

            InitonPDE = InitonPDE.next;

        }

        while (InitonPDE.hasPrev()) {

            InitonPDE = InitonPDE.prev;

        }

        return InitonPDE;

    }

    public Initon doDecrementM(Initon InitonPDE, InitonLinkDNA intonLinkDNA,

        FullDNATokenPDI pDE_RNA_FullFormular) {

        while (InitonPDE.hasPrev()) {

            InitonPDE = InitonPDE.prev;

        }

        intonLinkDNA.I_InitonLink(InitonPDE);

        InitonPDE = new PDE_Decrement_FullFormular()

        .PDE_DecrementM(intonLinkDNA);

        while (InitonPDE.hasPrev()) {

            InitonPDE = InitonPDE.prev;

        }

        // 20230106-S_logger.Log.logger.info(" " + " ");

        // 20230106-System.out.print("M->");

        while (InitonPDE.hasNext()) {

            // 20230106-System.out.print(InitonPDE.getStore());

            InitonPDE = InitonPDE.next;

        }

        while (InitonPDE.hasPrev()) {

            InitonPDE = InitonPDE.prev;

        }

        return InitonPDE;

    }

    public Initon doDecrementV(Initon InitonPDE, InitonLinkDNA intonLinkDNA,

        FullDNATokenPDI pDE_RNA_FullFormular) {

        while (InitonPDE.hasPrev()) {

            InitonPDE = InitonPDE.prev;

        }

        intonLinkDNA.I_InitonLink(InitonPDE);

        InitonPDE = new PDE_Decrement_FullFormular()

        .PDE_DecrementV(intonLinkDNA);

        while (InitonPDE.hasPrev()) {
```

```

        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-System.out.print("V->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doDecrementC(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-System.out.print("C->");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

public Initon doDecrementE(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_FullDNAFormular()
.PDE_DecrementE_DU(initonLinkDNA, pDE_RNA_FullFormular, bYS);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    // 20230106-S_logger.Log.logger.info("" + "");

```

```

        // 20230106-System.out.print("E->");
        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    public Initon doDecrementS(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        FullDNATokenPDI pDE_RNA_FullFormular, boolean bYS) {
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Decrement_FullDNAFormular()
        .PDE_DecrementS_IQ(initonLinkDNA, pDE_RNA_FullFormular, bYS);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        // 20230106-S_logger.Log.logger.info(""" + "");
        // 20230106-System.out.print("S->");
        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.next;
        }
        // 20230106-S_logger.Log.logger.info(""" + "");
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    @SuppressWarnings("unchecked")
    public String initonSect(String key) {
        String pdis = "";
        IMV_SQI map = new IMV_SQI();
        map.put("0", "A");
        map.put("1", "O");
        map.put("2", "P");
        map.put("3", "M");
        map.put("4", "V");
        map.put("5", "E");
        map.put("6", "C");
        map.put("7", "S");
        map.put("8", "I");
        map.put("9", "D");
    }

```

```
        map.put(".", "Q");

        int lineChars = 0; // 让元基码分段
        for (int i = 0; i < key.length(); i++) {
            lineChars++;
            if (lineChars == 60) { // 1行60个char
                pdis += "\r\n";
                lineChars = 0;
            }
            int c = key.charAt(i);
            String ints = S_Common.STRING_EMPTY + c;
            for (int j = 0; j < ints.length(); j++) {
                pdis += map.get("" + ints.charAt(j));
            }
            pdis += "U";
        }
        return pdis;
    }

    @SuppressWarnings("unchecked")
    public String initonDeSect(String key) {
        String pdis = "";
        IMV_SQI map = new IMV_SQI();
        map.put("A", "0");
        map.put("O", "1");
        map.put("P", "2");
        map.put("M", "3");
        map.put("V", "4");
        map.put("E", "5");
        map.put("C", "6");
        map.put("S", "7");
        map.put("I", "8");
        map.put("D", "9");
        map.put("Q", ".");
        String[] values = key.split("U");
        for (int i = 0; i < values.length; i++) {
            String c = values[i];
            String ints = "";
            for (int j = 0; j < c.length(); j++) {
                ints += map.get("" + c.charAt(j));
            }
            if (!ints.isEmpty() && !ints.contains("null")) { // 优化下null check
                int a = Integer.valueOf(ints);
                char b = (char) a;
                pdis += b;
            }
        }
        return pdis;
    }
}
```

```

}

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/full/FullTokenPDI.java
package A_I.MPE.MAQPVDISCOISIV.full;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;

import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

import A_I.MPE.MAQPVDISCOISIV.pde.PDE_Decrement_FullFormular;

import A_I.MPE.MAQPVDISCOISIV.pde.PDE_Increment_FullFormular;

import S_A.SVQ.stable.S_Common;

import S_A.pheromone.IMV_SQL;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.
//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

public class FullTokenPDI {

    public double[] key = new double[4];

    public String bys = "";

    public String pdw = "";

    public String pds = "";

    public String pde = "";

    public String time = "";

    public String session_key = "";

    public String text;

    public String cacheId;

    public String code = "";

    public String lock = "";

    public String pdedeKey = "";

    public String pdedsKey = "";

    public String pdeieKey = "";

    public String pdeisKey = "";

    public void doKeyPress(String initons, FullTokenPDI pDE_RNA_FullFormular,

        boolean bYS) {

        Initon[] initon = new Initon[initons.length()];

        for (int i = 0; i < initons.length(); i++) {

            if (initon[i] == null) {

                initon[i] = new Initon();

            }

            initon[i].I_Initon("" + initons.charAt(i));

            if (i + 1 < initons.length()) {

                if (initon[i + 1] == null) {

                    initon[i + 1] = new Initon();

```



```

    }

    initon[i].next = initon[i + 1];
    initon[i + 1].prev = initon[i];
    }
}

if (null != initons && initons.length() > 0) {
    do_PDE_RNA_FullFormular(initon[0], pDE_RNA_FullFormular, bYS);
}

}

public void run(String[] argv) {
    // long begin=System.currentTimeMillis();
    FullTokenPDI pDE_RNA_FullFormular = new FullTokenPDI();
    @SuppressWarnings("unused")
    String initonKeys = "EIU/0.6/EDU/0.4/si/0.3/sq/0.7/EIU/0.5/EDU/0.5/si/0.632/sq/0.368";
    pDE_RNA_FullFormular.key[0] = 0.6;
    pDE_RNA_FullFormular.key[1] = 0.3;
    pDE_RNA_FullFormular.key[2] = 0.5;
    pDE_RNA_FullFormular.key[3] = 0.632;
    pDE_RNA_FullFormular.text = "控制吸收";
    pDE_RNA_FullFormular.pdw = pDE_RNA_FullFormular
.initonSect(pDE_RNA_FullFormular.text);
    // 20230106-S_logger.Log.logger.info(" " + "原文: " +
    // pDE_RNA_FullFormular.text);
    // pDE_RNA_FullFormular.pdw= "字典保密: MSIOCUOCIPCUPCI";
    String[] lock = new String[12];
    lock[0] = "A";
    lock[3] = "O";
    lock[6] = "P";
    lock[9] = "M";
    lock[1] = "V";
    lock[4] = "E";
    lock[7] = "C";
    lock[10] = "S";
    lock[2] = "I";
    lock[5] = "D";
    lock[8] = "U";
    lock[11] = "Q";
    int i = (int) (Math.random() * 12) % 12;
    pDE_RNA_FullFormular.lock += lock[i];
    i = (int) (Math.random() * 12) % 12;
    pDE_RNA_FullFormular.lock += lock[i];
    i = (int) (Math.random() * 12) % 12;
    pDE_RNA_FullFormular.lock += lock[i];
    i = (int) (Math.random() * 12) % 12;
    pDE_RNA_FullFormular.lock += lock[i];
    for (i = 0; i < pDE_RNA_FullFormular.pdw.length(); i++) {
        pDE_RNA_FullFormular.code += pDE_RNA_FullFormular.lock

```

```
+ pDE_RNA_FullFormular.pdw.charAt(i);
}
// 20230106-S_logger.Log.logger.info("" + "肽语: " +
// pDE_RNA_FullFormular.pdw);
// 20230106-S_logger.Log.logger.info("" + "肽锁: " +
// pDE_RNA_FullFormular.lock);
// 20230106-S_logger.Log.logger.info("" + "散列肽语: 保密");
pDE_RNA_FullFormular.bys = "0.6/0.3/0.5/0.632";
// 20230106-S_logger.Log.logger.info("" + "静态密钥: " +
// pDE_RNA_FullFormular.bys);
pDE_RNA_FullFormular.doKeyPress(pDE_RNA_FullFormular.code,
pDE_RNA_FullFormular, false);
// 20230106-S_logger.Log.logger.info("" + "静态肽展降元概率钥匙E: " +
// pDE_RNA_FullFormular.pdedeKey);
// 20230106-S_logger.Log.logger.info("" + "静态肽展降元概率钥匙S: " +
// pDE_RNA_FullFormular.pdedsKey);
// 20230106-S_logger.Log.logger.info("" + "静态肽展降元: " +
// pDE_RNA_FullFormular.pds);
// 20230106-S_logger.Log.logger.info("" + "静态肽展增元概率钥匙E: " +
// pDE_RNA_FullFormular.pdeieKey);
// 20230106-S_logger.Log.logger.info("" + "静态肽展增元概率钥匙S: " +
// pDE_RNA_FullFormular.pdeisKey);
// 20230106-S_logger.Log.logger.info("" + "静态肽展增元: " +
// pDE_RNA_FullFormular.pde);
pDE_RNA_FullFormular.time = "" + System.currentTimeMillis();
pDE_RNA_FullFormular.cacheId = "ID" + Math.random()
+ S_Common.STRING_SYMBOL_PER + Math.random();
// 20230106-S_logger.Log.logger.info("" + "时间: " +
// pDE_RNA_FullFormular.time);
// 20230106-S_logger.Log.logger.info("" + "账号随机缓存字符串: " +
// pDE_RNA_FullFormular.cacheId);
pDE_RNA_FullFormular.session_key = pDE_RNA_FullFormular.pde;
// 20230106-S_logger.Log.logger.info("" + "Session: " +
// pDE_RNA_FullFormular.session_key);
// 20230106-S_logger.Log.logger.info("" +
"=====");
// 20230106-S_logger.Log.logger.info("" + "开始前序验证: ");
// 20230106-S_logger.Log.logger.info("" + "开始Session解析: " +
// pDE_RNA_FullFormular.session_key);
// 20230106-S_logger.Log.logger.info("" + "开始概率钥匙解析: " +
// pDE_RNA_FullFormular.pdedeKey
// + pDE_RNA_FullFormular.pdedsKey+
// pDE_RNA_FullFormular.pdeieKey
// + pDE_RNA_FullFormular.pdeisKey);
FullTokenPDI pDE_RNA_FullFormular1 = new FullTokenPDI();
pDE_RNA_FullFormular1.pdedeKey = pDE_RNA_FullFormular.pdedeKey
.toString();
pDE_RNA_FullFormular1.pdedsKey = pDE_RNA_FullFormular.pdedsKey
```

```

.toString();
        pDE_RNA_FullFormular1.pdeieKey = pDE_RNA_FullFormular.pdeieKey
.toString();
        pDE_RNA_FullFormular1.pdeisKey = pDE_RNA_FullFormular.pdeisKey
.toString();
        pDE_RNA_FullFormular.doKeyUnPress(pDE_RNA_FullFormular.code,
pDE_RNA_FullFormular1, true);
        // 20230106-S_logger.Log.logger.info(" " + "得到原降元元基DNA序列: " +
        // pDE_RNA_FullFormular.pds);
        // 20230106-S_logger.Log.logger.info(" " + "得到新降元元基DNA序列: " +
        // pDE_RNA_FullFormular1.pds);
        // 20230106-S_logger.Log.logger.info(" " + "得到原元基DNA序列: " +
        // pDE_RNA_FullFormular.pde);
        // 20230106-S_logger.Log.logger.info(" " + "得到新元基DNA序列: " +
        // pDE_RNA_FullFormular1.pde);
        // 20230106-S_logger.Log.logger.info(" " + "验证正确? ");
        // 20230106-S_logger.Log.logger.info(" " + pDE_RNA_FullFormular.pde.equals(pDE_RNA_FullFormular1.pde)
        // ? "正确": "失败");
        // 20230106-S_logger.Log.logger.info(" " +
        "=====");
        // 20230106-S_logger.Log.logger.info(" " + "开始后序验证: ");
        FullTokenPDI pDE_RNA_FullFormular2 = new FullTokenPDI();
        pDE_RNA_FullFormular2.pdeieKey = pDE_RNA_FullFormular.pdedeKey
.toString();
        pDE_RNA_FullFormular2.pdeisKey = pDE_RNA_FullFormular.pdedeKey
.toString();
        pDE_RNA_FullFormular2.pdedeKey = pDE_RNA_FullFormular.pdeieKey
.toString();
        pDE_RNA_FullFormular2.pdedeKey = pDE_RNA_FullFormular.pdeisKey
.toString();
        // 20230106-S_logger.Log.logger.info(" " + "准备计算元基DNA序列: " +
        // pDE_RNA_FullFormular1.pde);
        pDE_RNA_FullFormular2.doSessionKeyUnPress(pDE_RNA_FullFormular1.pde,
pDE_RNA_FullFormular2, true);
        // 20230106-S_logger.Log.logger.info(" " + "得到原续降元元基DNA序列: " +
        // pDE_RNA_FullFormular1.pds);
        // 20230106-S_logger.Log.logger.info(" " + "得到后续降元元基DNA序列: " +
        // pDE_RNA_FullFormular2.pds);
        // 20230106-S_logger.Log.logger.info(" " + "验证正确? ");
        // 20230106-S_logger.Log.logger.info(" " + pDE_RNA_FullFormular1.pds.equals(pDE_RNA_FullFormular2.pds)
        // ? "正确": "失败");
//long end=System.currentTimeMillis();
////20230106-S_logger.Log.logger.info(" " + end-begin);
}

private void doSessionKeyUnPress(String initons,
        FullTokenPDI pDE_RNA_FullFormular2, boolean bYS) {
    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {

```

```

        if (initon[i] == null) {
            initon[i] = new Initon();
        }
        initon[i].I_Initon("'" + initons.charAt(i));
        if (i + 1 < initons.length()) {
            if (initon[i + 1] == null) {
                initon[i + 1] = new Initon();
            }
            initon[i].next = initon[i + 1];
            initon[i + 1].prev = initon[i];
        }
        do_PDE_RNA_FullFormular_Back(initon[0], pDE_RNA_FullFormular2, bYS);
    }

    private void do_PDE_RNA_FullFormular_Back(Initon initon,
        FullTokenPDI pDE_RNA_FullFormular, boolean bYS) {
        Initon InitonPDE = initon;
        InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
        Initon InitonPDEM = doDecrementM(InitonPDE, initonLinkDNA,
pDE_RNA_FullFormular);
        Initon InitonPDEP = doDecrementP(InitonPDEM, initonLinkDNA,
pDE_RNA_FullFormular);
        Initon InitonPDEO = doDecrementO(InitonPDEP, initonLinkDNA,
pDE_RNA_FullFormular);
        Initon InitonPDEA = doDecrementA(InitonPDEO, initonLinkDNA,
pDE_RNA_FullFormular);
        Initon InitonPDES = doDecrementS(InitonPDEA, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
        Initon InitonPDEC = doDecrementC(InitonPDES, initonLinkDNA,
pDE_RNA_FullFormular);
        Initon InitonPDEE = doDecrementE(InitonPDEC, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
        Initon InitonPDE1 = doDecrementV(InitonPDEE, initonLinkDNA,
pDE_RNA_FullFormular);
        while (InitonPDE1.hasNext()) {
            pDE_RNA_FullFormular.pds += InitonPDE1.getStore();
            InitonPDE1 = InitonPDE1.next;
        }
        pDE_RNA_FullFormular.pds += InitonPDE1.getStore();
        while (InitonPDE1.hasPrev()) {
            InitonPDE1 = InitonPDE1.prev;
        }
    }

// Initon InitonPDE1S= doIncrementS(InitonPDE1, initonLinkDNA, pDE_RNA_FullFormular, bYS);
// Initon InitonPDE1C= doIncrementC(InitonPDE1S, initonLinkDNA, pDE_RNA_FullFormular);
// Initon InitonPDE1E= doIncrementE(InitonPDE1C, initonLinkDNA, pDE_RNA_FullFormular, bYS);
// Initon InitonPDE1V= doIncrementV(InitonPDE1E, initonLinkDNA, pDE_RNA_FullFormular);

```

```

//      Initon InitonPDE1M= doIncrementM(InitonPDE1V, initonLinkDNA, pDE_RNA_FullFormular);
//      Initon InitonPDE1P= doIncrementP(InitonPDE1M, initonLinkDNA, pDE_RNA_FullFormular);
//      Initon InitonPDE1O= doIncrementO(InitonPDE1P, initonLinkDNA, pDE_RNA_FullFormular);
//      Initon InitonPDE2= doIncrementA(InitonPDE1O, initonLinkDNA, pDE_RNA_FullFormular);
//      while(InitonPDE2.hasNext()) {
//pDE_RNA_FullFormular.pde+= InitonPDE2.getStore();
//InitonPDE2= InitonPDE2.next;
//      }
//      pDE_RNA_FullFormular.pde+= InitonPDE2.getStore();
//      while(InitonPDE2.hasPrev()) {
//InitonPDE2= InitonPDE2.prev;
//      }

private void doKeyUnPress(String initons, FullTokenPDI pDE_RNA_FullFormular,
                        boolean bYS) {
    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {
        if (initon[i] == null) {
            initon[i] = new Initon();
        }
        initon[i].I_Initon("" + initons.charAt(i));
        if (i + 1 < initons.length()) {
            if (initon[i + 1] == null) {
                initon[i + 1] = new Initon();
            }
            initon[i].next = initon[i + 1];
            initon[i + 1].prev = initon[i];
        }
    }
    do_PDE_RNA_FullFormular(initon[0], pDE_RNA_FullFormular, bYS);
}

//
public void do_PDE_RNA_FullFormular(Initon initon,
                        FullTokenPDI pDE_RNA_FullFormular, boolean bYS) {
    Initon InitonPDE = initon;
    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
    Initon InitonPDEA = doDecrementA(InitonPDE, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEO = doDecrementO(InitonPDEA, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEP = doDecrementP(InitonPDEO, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEM = doDecrementM(InitonPDEP, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEV = doDecrementV(InitonPDEM, initonLinkDNA,
pDE_RNA_FullFormular);
    Initon InitonPDEE = doDecrementE(InitonPDEV, initonLinkDNA,
pDE_RNA_FullFormular, bYS);
}

```

```

        Initon InitonPDEC = doDecrementC(InitonPDEE, initonLinkDNA,
pDE_RNA_FullFormular);

        Initon InitonPDE1 = doDecrementS(InitonPDEC, initonLinkDNA,
pDE_RNA_FullFormular, bYS);

        while (InitonPDE1.hasNext()) {
            pDE_RNA_FullFormular.pds += InitonPDE1.getStore();
            InitonPDE1 = InitonPDE1.next;
        }

        pDE_RNA_FullFormular.pds += InitonPDE1.getStore();
        while (InitonPDE1.hasPrev()) {
            InitonPDE1 = InitonPDE1.prev;
        }

        Initon InitonPDE1V = doIncrementV(InitonPDE1, initonLinkDNA,
pDE_RNA_FullFormular);

        Initon InitonPDE1E = doIncrementE(InitonPDE1V, initonLinkDNA,
pDE_RNA_FullFormular, bYS);

        Initon InitonPDE1C = doIncrementC(InitonPDE1E, initonLinkDNA,
pDE_RNA_FullFormular);

        Initon InitonPDE1S = doIncrementS(InitonPDE1C, initonLinkDNA,
pDE_RNA_FullFormular, bYS);

        Initon InitonPDE1A = doIncrementA(InitonPDE1S, initonLinkDNA,
pDE_RNA_FullFormular);

        Initon InitonPDE1O = doIncrementO(InitonPDE1A, initonLinkDNA,
pDE_RNA_FullFormular);

        Initon InitonPDE1P = doIncrementP(InitonPDE1O, initonLinkDNA,
pDE_RNA_FullFormular);

        Initon InitonPDE2 = doIncrementM(InitonPDE1P, initonLinkDNA,
pDE_RNA_FullFormular);

        while (InitonPDE2.hasNext()) {
            pDE_RNA_FullFormular.pde += InitonPDE2.getStore();
            InitonPDE2 = InitonPDE2.next;
        }

        pDE_RNA_FullFormular.pde += InitonPDE2.getStore();
        while (InitonPDE2.hasPrev()) {
            InitonPDE2 = InitonPDE2.prev;
        }
    }
}

```

/////INITONS SWAP

```

private Initon doIncrementA(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
FullTokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }

    initonLinkDNA.I_InitonLink(InitonPDE);

    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementA(initonLinkDNA);

    while (InitonPDE.hasPrev()) {

```

```

        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doIncrementO(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementO(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doIncrementP(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementP(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

```

```

}

private Initon doIncrementM(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementM(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doIncrementV(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementV(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doIncrementC(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementC(initonLinkDNA);

```



```

        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    private Initon doIncrementE(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        FullTokenPDI pDE_RNA_FullFormular, boolean bYS) {
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementE_DU(initonLinkDNA, pDE_RNA_FullFormular, bYS);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    private Initon doIncrementS(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        FullTokenPDI pDE_RNA_FullFormular, boolean bYS) {
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_FullFormular()
.PDE_IncrementS_IQ(initonLinkDNA, pDE_RNA_FullFormular);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
    }

```

```
        return InitonPDE;
    }

    private Initon doDecrementA(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        FullTokenPDI pDE_RNA_FullFormular) {
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementA(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    private Initon doDecrementO(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        FullTokenPDI pDE_RNA_FullFormular) {
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementO(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    private Initon doDecrementP(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        FullTokenPDI pDE_RNA_FullFormular) {
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Decrement_FullFormular()
```

```

.PDE_DecrementP(intonLinkDNA);

    while (IntonPDE.hasPrev()) {
        IntonPDE = IntonPDE.prev;
    }
    while (IntonPDE.hasNext()) {
        IntonPDE = IntonPDE.next;
    }
    while (IntonPDE.hasPrev()) {
        IntonPDE = IntonPDE.prev;
    }
    return IntonPDE;
}

private Inton doDecrementM(Inton IntonPDE, IntonLinkDNA intonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular) {
    while (IntonPDE.hasPrev()) {
        IntonPDE = IntonPDE.prev;
    }
    intonLinkDNA.I_IntonLink(IntonPDE);
    IntonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementM(intonLinkDNA);
    while (IntonPDE.hasPrev()) {
        IntonPDE = IntonPDE.prev;
    }
    while (IntonPDE.hasNext()) {
        IntonPDE = IntonPDE.next;
    }
    while (IntonPDE.hasPrev()) {
        IntonPDE = IntonPDE.prev;
    }
    return IntonPDE;
}

private Inton doDecrementV(Inton IntonPDE, IntonLinkDNA intonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular) {
    while (IntonPDE.hasPrev()) {
        IntonPDE = IntonPDE.prev;
    }
    intonLinkDNA.I_IntonLink(IntonPDE);
    IntonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementV(intonLinkDNA);
    while (IntonPDE.hasPrev()) {
        IntonPDE = IntonPDE.prev;
    }
    while (IntonPDE.hasNext()) {
        IntonPDE = IntonPDE.next;
    }
    while (IntonPDE.hasPrev()) {
        IntonPDE = IntonPDE.prev;
    }

```

```

    }
    return InitonPDE;
}

private Initon doDecrementC(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doDecrementE(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_FullFormular()
.PDE_DecrementE_DU(initonLinkDNA, pDE_RNA_FullFormular, bYS);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doDecrementS(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_FullFormular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);

```

```

        InitonPDE = new PDE_Decrement_FullFormular()
        .PDE_DecrementS_IQ(intonLinkDNA, pDE_RNA_FullFormular, bYS);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        // 20230106-S_logger.Log.logger.info("" + "");
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }
    @SuppressWarnings("unchecked")
    public String initonSect(String key) {
        String pdis = "";
        IMV_SQI map = new IMV_SQI();
        map.put("0", "A");
        map.put("1", "O");
        map.put("2", "P");
        map.put("3", "M");
        map.put("4", "V");
        map.put("5", "E");
        map.put("6", "C");
        map.put("7", "S");
        map.put("8", "I");
        map.put("9", "D");
        map.put(".", "U");
        for (int i = 0; i < key.length(); i++) {
            int c = key.charAt(i);
            String ints = S_Common.STRING_EMPTY + c;
            for (int j = 0; j < ints.length(); j++) {
                pdis += map.get("" + ints.charAt(j));
            }
            pdis += "U";
        }
        return pdis;
    }
}

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/full/FullTokenPDITest.java

A_I.MPE.MAQPVDISCOISIV.initon

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/inton/Initon.java
package A_I.MPE.MAQPVDISCOISIV.initon;

import S_A.SVQ.stable.S_Common;

/*

```

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class Initon {  
    private String store;  
    public Initon prev;  
    public Initon next;  
    public String getStore() {  
        return store;  
    }  
    public Initon addNextInitons(Initon initon) {  
        Initon thisIniton = this;  
        while (null != thisIniton.next) {  
            thisIniton = thisIniton.next;  
        }  
        thisIniton.next = initon;  
        return thisIniton;  
    }  
    public Initon addPrevInitons(Initon initon) {  
        Initon thisIniton = this;  
        while (null != thisIniton.prev) {  
            thisIniton = thisIniton.prev;  
        }  
        thisIniton.prev = initon;  
        return thisIniton;  
    }  
    public void I_Initon(String initon) {  
        this.store = initon;  
    }  
    public void I_A() {  
        this.store = "A";  
    }  
    public void I_O() {  
        this.store = "O";  
    }  
    public void I_P() {  
        this.store = "P";  
    }  
    public void I_M() {  
        this.store = "M";  
    }  
    public void I_V() {
```

```
        this.store = "V";
    }
    public void I_E() {
        this.store = "E";
    }
    public void I_C() {
        this.store = "C";
    }
    public void I_S() {
        this.store = "S";
    }
    public void I_I() {
        this.store = "I";
    }
    public void I_D() {
        this.store = "D";
    }
    public void I_U() {
        this.store = "U";
    }
    public void I_Q() {
        this.store = "Q";
    }
    public boolean hasNext() {
        if (null != this.next) {
            return true;
        }
        return false;
    }
    public Initon forwardNext() {
        return this.next;
    }
    public boolean hasPrev() {
        if (null != this.prev) {
            return true;
        }
        return false;
    }
    public Initon forwardPrev() {
        return this.prev;
    }
    // 用于计算复制
    public Initon copyRNA(Initon initonPDE) {
        while (initonPDE.hasPrev()) {
            initonPDE.forwardPrev();
        }
        Initon InitonRNA = new Initon();
```

```
        InitonRNA.store = initonPDE.store;
        while (initonPDE.next != null) {
            initonPDE = initonPDE.next;
            Initon initon = new Initon();
            initon.store = initonPDE.store;
            InitonRNA.next = initon;
            initon.prev = InitonRNA;
            InitonRNA = initon;
        }
        // 完成copy
        while (initonPDE.prev != null) {
            initonPDE = initonPDE.prev;
        }
        while (InitonRNA.prev != null) {
            InitonRNA = InitonRNA.prev;
        }
        return InitonRNA;
    }
    // 用于遗传复制
    public Initon swapRNA(Initon initonPDE) {
        while (initonPDE.hasPrev()) {
            initonPDE.forwardPrev();
        }
        Initon InitonRNA = new Initon();
        InitonRNA.store = initonPDE.store;
        while (initonPDE.next != null) {
            initonPDE = initonPDE.next;
            Initon initon = new Initon();
            if (initonPDE.store.equalsIgnoreCase("IQ")) {
                initon.store = "S";
            }
            if (initonPDE.store.equalsIgnoreCase("S")) {
                initon.store = "IQ";
            }
            // if(initonPDE.store.equalsIgnoreCase("I")) {
            //     initon.store= "U";
            // }//这里我会做补码 规范化. 罗瑶光.20201105 7:56
            if (initonPDE.store.equalsIgnoreCase("D")) {
                initon.store= "DD";
            }
            if (initonPDE.store.equalsIgnoreCase("E")) {
                initon.store = "DU";
            }
            if (initonPDE.store.equalsIgnoreCase("DU")) {
                initon.store = "E";
            }
            if (initonPDE.store.equalsIgnoreCase("C")) {
```

```

        initon.store = "ID";
    }
    if (initonPDE.store.equalsIgnoreCase("ID")) {
initon.store = "C";
    }
    if (initonPDE.store.equalsIgnoreCase("UQ")) {
initon.store = "V";
    }
    if (initonPDE.store.equalsIgnoreCase("V")) {
initon.store = "UQ";
    }
    initon.store = initonPDE.store;
    InitonRNA.next = initon;
    initon.prev = InitonRNA;
    InitonRNA = initon;
}
// 完成copy
while (initonPDE.prev != null) {
    initonPDE = initonPDE.prev;
}
while (InitonRNA.prev != null) {
    InitonRNA = InitonRNA.prev;
}
return InitonRNA;
}
// 用于反码复制
public Initon maskRNA(Initon initonPDE) {
    while (initonPDE.hasPrev()) {
        initonPDE.forwardPrev();
    }
    Initon InitonRNA = new Initon();
    InitonRNA.store = initonPDE.store;
    while (initonPDE.next != null) {
        initonPDE = initonPDE.next;
        Initon initon = new Initon();
        if (initonPDE.store.equalsIgnoreCase("I")) {
initon.store = "D";
        }
        if (initonPDE.store.equalsIgnoreCase("D")) {
initon.store = "I";
        }
        if (initonPDE.store.equalsIgnoreCase("U")) {
initon.store = "Q";
        }
        if (initonPDE.store.equalsIgnoreCase("Q")) {
initon.store = "U";
        }
    }
}

```

```
        initon.store = initonPDE.store;
        InitonRNA.next = initon;
        initon.prev = InitonRNA;
        InitonRNA = initon;
    }
    // 完成copy
    while (initonPDE.prev != null) {
        initonPDE = initonPDE.prev;
    }
    while (InitonRNA.prev != null) {
        InitonRNA = InitonRNA.prev;
    }
    return InitonRNA;
}
// 用于补码复制
public Initon compsRNA(Initon initonPDE) {
    while (initonPDE.hasPrev()) {
        initonPDE.forwardPrev();
    }
    Initon InitonRNA = new Initon();
    InitonRNA.store = initonPDE.store;
    while (initonPDE.next != null) {
        initonPDE = initonPDE.next;
        Initon initon = new Initon();
        if (initonPDE.store.equalsIgnoreCase("I")) {
            initon.store = "U";
        }
        if (initonPDE.store.equalsIgnoreCase("D")) {
            initon.store = "I";
        }
        if (initonPDE.store.equalsIgnoreCase("U")) {
            initon.store = "Q";
        }
        if (initonPDE.store.equalsIgnoreCase("Q")) {
            initon.store = "DD";
        }
        initon.store = initonPDE.store;
        InitonRNA.next = initon;
        initon.prev = InitonRNA;
        InitonRNA = initon;
    }
    // 完成copy
    while (initonPDE.prev != null) {
        initonPDE = initonPDE.prev;
    }
    while (InitonRNA.prev != null) {
        InitonRNA = InitonRNA.prev;
    }
}
```

```

    }
    return InitonRNA;
}
// 举例V的 SWAP initon.store= "UQ"; -> initon.store= "U"; 和
// NEXT initon.store= "Q";
// 用于RNA 肽展遗传和增元计算.
public Initon storeRNAExtension(Initon initonPDE) {
    while (initonPDE.hasPrev()) {
        initonPDE.forwardPrev();
    }
    Initon InitonRNA = new Initon();
    InitonRNA.store = initonPDE.store;
    while (initonPDE != null) {
        if (initonPDE.store.length() == 1) {
            Initon initon = new Initon();
            initon.store = initonPDE.store;
            initon.prev = initonPDE.prev;
            initon.next = initonPDE.next;
            InitonRNA = initon;
        }
        if (initonPDE.store.length() == 2) {
            // 初始
            Initon initonFirst = new Initon();
            Initon initonSecond = new Initon();
            // 赋值
            initonFirst.store = S_Common.STRING_EMPTY
                + initonPDE.store.charAt(0);
            initonSecond.store = S_Common.STRING_EMPTY
                + initonPDE.store.charAt(1);
            // 连接
            initonFirst.next = initonSecond;
            initonSecond.prev = initonFirst;
            // 已计算的前序融入
            if (initonPDE.prev != null) {
                initonFirst.prev = initonPDE.prev;
                initonFirst.prev.next = initonFirst;
            }
            // 未计算的后序关联
            if (initonPDE.next != null) {
                initonSecond.next = initonPDE.next;
                initonSecond.next.prev = initonSecond;
            }
            // 替进
            InitonRNA = initonSecond;
        }
        if (null == initonPDE.next) {
            while (initonPDE.prev != null) {

```

```

        initonPDE = initonPDE.prev;
    }
    while (InitonRNA.prev != null) {
        InitonRNA = InitonRNA.prev;
    }
    return InitonRNA;
}

        initonPDE = initonPDE.next;
    }
    // 完成copy
    while (initonPDE.prev != null) {
        initonPDE = initonPDE.prev;
    }
    while (InitonRNA.prev != null) {
        InitonRNA = InitonRNA.prev;
    }
    return InitonRNA;
}

public static void main(String[] argv) {
    Initon initonV = new Initon();
    initonV.store = "UQ";
    Initon initonV1 = new Initon();
    initonV1.store = "Q";
    Initon initonV2 = new Initon();
    initonV2.store = "IQ";
    initonV.next = initonV1;
    initonV1.prev = initonV;
    initonV1.next = initonV2;
    initonV2.prev = initonV1;
    initonV = initonV.storeRNAExtension(initonV);
    while (initonV.hasNext()) {
        // 20230106-S_logger.Log.logger.info("" + initonV.getStore());
        initonV = initonV.forwardNext();
    }
    // 20230106-S_logger.Log.logger.info("" + initonV.getStore());
}
}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVD COISIV/initon/InitonLinkDNA.java
package A_I.MPE.MAQPVD COISIV.initon;
import java.util.List;
/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class InitonLinkDNA {
    private Initon initonLink;
    private List<Initon> initonList;
    public void I_NextIniton(Initon initon) {
        initonLink = initonLink.addNextInitons(initon);
        initonList.add(initon);
    }
    public void I_PrevIniton(Initon initon) {

```

```

        initonLink = initonLink.addPrevInitons(inton);
        initonList.add(0, initon);
    }
    public Initon getInitonLink() {
        return initonLink;
    }
    public void I_InitonLink(Initon initons) {
        initonLink = initons;
    }
}
/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDCOISIV/inton/InitonLinkDNATest.java
/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDCOISIV/inton/InitonsPDI.java
package A_I.MPE.MAQPVDCOISIV.initon;

```

```
@SuppressWarnings("unused")
```

```
/*
```

```
* 个人著作权人，作者 罗瑶光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```

public class InitonsPDI {
    // M-E-C
    public static void main(String[] argv) {
    }
    public String EUM_QD_MQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_QI_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "V", "E", "C", "S" }, { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        // WHILE
        // AEP+=PM_ISO+CED[][];
        // ...
        return AEP;
    }
    public String EUM_QD_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_Q_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "V", "E", "C", "S" },
        { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }
}

```

```

    }

    public String EUM_QU_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "V", "E", "C", "S" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String EUP_QD_PQU(String AEP, int QSU) {
        String[][] CED = new String[][] { { "D", "I", "U", "Q" },
        { "H", "T", "X" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String OCS(int[][] VDE) {
        String SEP = "";
        for (int i = 0; i < VDE.length; i++) {
            for (int j = 0; j < VDE[0].length; j++) {
                VDE[0][j] += SEP.charAt(i) | '0';
            }
        }
        return null;
    }

    public String SDI_P_M_PDS(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) & '0';
            SQV += AEP.charAt(i) & '1';
        }
        for (int i = 0; i < AEP.length(); i++) {
            for (int j = 0; j < SQV.length(); j++) {
                AEP += AEP.charAt(i) | SQV.charAt(j);
            }
        }
        return AEP + SQV;
    }

    public String SDD_P_M_PDS(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) | '0';
            SQV += AEP.charAt(i) | '0';
        }
        return AEP + SQV;
    }
}

/*
/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVD COISIV/initon/InitonsPDITest.java
/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVD COISIV/initon/InitonsPDM.java
package A_I.MPE.MAQPVD COISIV.initon;

@SuppressWarnings("unused")

```

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class InitonsPDM {
    public static void main(String[] argv) {
    }
    public String EUM_QD_MQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_QI_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "V", "E", "C", "S" }, { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_QD_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_Q_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "V", "E", "C", "S" },
        { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_QU_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "V", "E", "C", "S" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUP_QD_PQU(String AEP, int QSU) {
        String[][] CED = new String[][] { { "D", "I", "U", "Q" },
        { "H", "T", "X" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String OCS(int[][] VDE) {
        return null;
    }
}
```

```

    }

    public String SDI_P_M_PDS(String SQV, int[][] SVE, int[][] mask) {

        String AEP = "";

        for (int i = 0; i < SQV.length(); i++) {

            AEP += SQV.charAt(i) & '0';

        }

        return AEP;

    }

    public String SDD_P_M_PDS(String SQV, int[][] SVE, int[][] mask) {

        String AEP = "";

        for (int i = 0; i < SQV.length(); i++) {

            AEP += SQV.charAt(i) | '0';

        }

        return AEP;

    }

    public String[][] SDD_P_M_PDI(String SQV[], int[][] SDE, String[][] SVE) {

        String[][] CEI = new String[][] { { "D", "I", "U", "Q" } };
        String[][] CED = new String[][] { { "V", "E", "C", "S" } };
        String[][] CEU = new String[][] { { "A", "O", "P", "M" } };
        String[][] CEQ = new String[][] { { "T", "X", "H" } };

        for (int i = 0; i < SDE.length; i++) {

            for (int j = 0; j < SDE[0].length; j++) {

                SQV[i][j] = new String(SVE[SDE[i][j]][SDE[i][j]]);

            }

        }

        // CEI&CED+SQV
        // CEU&CEQ+SQV
        // ...

        return SQV;

    }

    // shit

    public int[][] ISV_QSU_S(int[][] IMI, int UMQ, int UMV) {

        for (int i = 0; i < IMI.length; i++) {

            for (int j = 0; j < IMI[0].length; j++) {

                if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math

                    .abs(UMQ)) {

                    IMI[i][j] = IMI[i][j] - 1;

                } else {

                    // IMI[i][j]=IMI[i][j]-1;

                    // .. AEC.VES...

                }

            }

        }

        return IMI;

    }

    // vivi 1-3-1-3 1313-3131-cce,

    // idc-1133-3311-2213-1312-cs.idq.vec

```



```

public int[][] ISV_QSU_V(int[][] IMI, int UMQ, int UMV) {
    // int a= ;
    for (int i = 0; i < IMI.length; i++) {
        for (int j = 0; j < IMI[0].length; j++) {
            // if(IMI[i][j]& 00110011 > 0) {
            // }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
                // IDC...
            } else {
                // IMI[i][j]= IMI[i][j]- 1;
                // ..
            }
        }
    }
    return IMI;
}

// vivi 1-3-1-3 148913-3131-cce,
// idc-1133-3311-2213-1312-ces.idq.vec
public int[][] ISV_QSU_VS(int[][] IMI, int UMQ, int UMV) {
    for (int i = 0; i < IMI.length; i++) {
        Here: for (int j = 0; j < IMI[0].length; j++) {
            if (IMI[i][j] << 1 != (int) Math.pow(IMI[i][j], 2)
                / IMI[i][j] >> 1) {
                continue Here;
            }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
            } else {
                IMI[i][j] = IMI[i][j] - 1;
            }
        }
    }
    return IMI;
}

public int[][] ISV_QSU_VV(int[][] IMI, int UMQ, int UMV) {
    // int a= ;
    for (int i = 0; i < IMI.length; i++) {
        for (int j = 0; j < IMI[0].length; j++) {
            // if(IMI[i][j]& 00110011 > 0) {
            // }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
                // IDC...
            }
        }
    }
}

```

```

    } else {
        // IMI[i][j]= IMI[i][j]- 1;
        // ..
    }

    }

    }

    return IMI;

}

// vivi 1-3-1-3 148913-3131-cce,
// idc-1133-3311-2213-1312-ces.idq.vec
public int[][] ISV_QCU_VS(int[][] IMI, int UMV, int UMV) {
    for (int i = 0; i < IMI.length; i++) {
        Here: for (int j = 0; j < IMI[0].length; j++) {
            if (IMI[i][j] << 1 != (int) Math.pow(IMI[i][j], 2)
                / IMI[i][j] >> 1) {
                continue Here;
            }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
            } else {
                IMI[i][j] = IMI[i][j] - 1;
            }
        }
    }
    return IMI;
}

public int[][] ISV_QEU_V(int[][] IMI, int UMV, int UMV) {
    for (int i = 0; i < IMI.length; i++) {
        Here: for (int j = 0; j < IMI[0].length; j++) {
            if (IMI[i][j] << 1 == (int) Math.pow(IMI[i][j], 2)
                / IMI[i][j] >> 1) {
                continue Here;
            }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
            } else {
                IMI[i][j] = IMI[i][j] - 1;
            }
        }
    }
    return IMI;
}

// vivi 1-3-1-3 148913-3131-cce,
// idc-1133-3311-2213-1312-ces.idq.vec
public int[][] ISV_QVU_VS(int[][] IMI, int UMV, int UMV) {

```

```

        for (int i = 0; i < IMI.length; i++) {
            Here: for (int j = 0; j < IMI[0].length; j++) {
if (IMI[i][j] << 2 != (int) Math.pow(IMI[i][j], 2)
            / IMI[i][j] >> 2) {
                continue Here;
            }
if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
            .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
            } else {
                IMI[i][j] = IMI[i][j] - 1;
            }
        }
    }
    return IMI;
}

public int[][] ISV_MSU_V(int[][] IMI, int UMQ, int UMV) {
    // int a= ;
    for (int i = 0; i < IMI.length; i++) {
        for (int j = 0; j < IMI[0].length; j++) {
// if(IMI[i][j]& 00110011 > 0) {
// }
if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
            .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
                // IDC...
            } else {
                // IMI[i][j]= IMI[i][j]- 1;
                // ..
            }
        }
    }
    return IMI;
}

// vivi 1-3-1-3 148913-3131-cce,
// idc-1133-3311-2213-1312-ces.idq.vec
public int[][] ISV_PSU_VS(int[][] IMI, int UMQ, int UMV) {
    for (int i = 0; i < IMI.length; i++) {
        Here: for (int j = 0; j < IMI[0].length; j++) {
if (IMI[i][j] << 3 != (int) Math.pow(IMI[i][j], 2)
            / IMI[i][j] >> 3) {
                continue Here;
            }
if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
            .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
            } else {

```

```

        IMI[i][j] = IMI[i][j] - 1;
    }

    }

    }

    return IMI;
}

public int[][] ISV_OSU_V(int[][] IMI, int UMV, int UMV) {
    // int a= ;
    for (int i = 0; i < IMI.length; i++) {
        for (int j = 0; j < IMI[0].length; j++) {
            // if(IMI[i][j]& 00110011 > 0) {
            // }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMV)) {
                IMI[i][j] = IMI[i][j] + 1;
                // IDC...
            } else {
                // IMI[i][j]= IMI[i][j]- 1;
                // ..
            }
        }
    }

    }

    return IMI;
}

// vivi 1-3-1-3 148913-3131-cce,
// idc-1133-3311-2213-1312-cs.idq.vec

public int[][] ISV_ASU_VS(int[][] IMI, int UMV, int UMV) {
    for (int i = 0; i < IMI.length; i++) {
        Here: for (int j = 0; j < IMI[0].length; j++) {
            if (IMI[i][j] << 4 != (int) Math.pow(IMI[i][j], 2)
                / IMI[i][j] >> 4) {
                continue Here;
            }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMV)) {
                IMI[i][j] = IMI[i][j] + 1;
            } else {
                IMI[i][j] = IMI[i][j] - 1;
            }
        }
    }

    return IMI;
}

public int[][] ICV_QSU_V(int[][] IMI, int UMV, int UMV) {
    // int a= ;
    for (int i = 0; i < IMI.length; i++) {
        for (int j = 0; j < IMI[0].length; j++) {

```

```

// if(IMI[i][j]& 00110011 > 0) {
// }

if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
    .abs(UMQ)) {
    IMI[i][j] = IMI[i][j] + 1;
    // IDC...
} else {
    // IMI[i][j]= IMI[i][j]- 1;
    // ..
}

    }

}

return IMI;
}

// vivi 1-3-1-3 148913-3131-cce,
// idc-1133-3311-2213-1312-ces.idq.vec
public int[][] IEV_QSU_VS(int[][] IMI, int UMQ, int UMV) {
    for (int i = 0; i < IMI.length; i++) {
        Here: for (int j = 0; j < IMI[0].length; j++) {
            if (IMI[i][j] << 5 != (int) Math.pow(IMI[i][j], 2)
                / IMI[i][j] >> 5) {
                continue Here;
            }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
            } else {
                IMI[i][j] = IMI[i][j] - 1;
            }
        }
    }
    return IMI;
}

public int[][] IVV_QSU_V(int[][] IMI, int UMQ, int UMV) {
    // int a= ;
    for (int i = 0; i < IMI.length; i++) {
        for (int j = 0; j < IMI[0].length; j++) {
            // if(IMI[i][j]& 00110011 > 0) {
            // }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
                // IDC...
            } else {
                // IMI[i][j]= IMI[i][j]- 1;
                // ..
            }
        }
    }
}

```

```

    }

    }

    return IMI;

}

// VIVI + SHIT= VS
// vivi 1-3-1-3 148913-3131-cce,
// idc-1133-3311-2213-1312-ces.idq.vec

public int[][] A_QSV_QSU_VS(int[][] IMI, int UMQ, int UMV) {
    for (int i = 0; i < IMI.length; i++) {
        Here: for (int j = 0; j < IMI[0].length; j++) {
            if (IMI[i][j] << 1 != (int) Math.pow(IMI[i][j], 2)
                / IMI[i][j] >> 1) {
                continue Here;
            }
        }
        if (Math.abs(Math.random() * Math.pow(10, UMV)) > UMQ) {
            IMI[i][j] = IMI[i][j] + 1;
        } else {
            IMI[i][j] = IMI[i][j] - 1;
        }
    }
}

return IMI;

}

public int[][] USV_QSU_V(int[][] IMI, int UMQ, int UMV) {
    // int a= ;
    for (int i = 0; i < IMI.length; i++) {
        for (int j = 0; j < IMI[0].length; j++) {
            // if(IMI[i][j]& 00110011 > 0) {
            // }
            if (Math.abs(Math.random() * Math.pow(10, UMV)) > Math
                .abs(UMQ)) {
                IMI[i][j] = IMI[i][j] + 1;
                // IDC...
            } else {
                // IMI[i][j]= IMI[i][j]- 1;
                // ..
            }
        }
    }

    return IMI;

}

// vivi 1-3-1-3 148913-3131-cce,
// idc-1133-3311-2213-1312-ces.idq.vec

public int[][] DSV_QSU_VS(int[][] IMI, int UMQ, int UMV) {
    for (int i = 0; i < IMI.length; i++) {
        Here: for (int j = 0; j < IMI[0].length; j++) {
            if (IMI[i][j] << 1 != (int) Math.pow(IMI[i][j], 2)

```

```

        / IMI[i][j] >> 1) {
            continue Here;
        }
//          if(Math.abs(Math.random()* Math.pow(10, UMI))> Math.abs(UMQ)) {
//              IMI[i][j]= IMI[i][j]+ 1;
//          }else {
//              IMI[i][j]= IMI[i][j]- 1;
//          }
int UMI = -1;
if (Math.abs(Math.random() * Math.pow(10, UMI)) > (UMQ & UMI)) {
    IMI[i][j] = IMI[i][j] + 1;
}
    }
}
return IMI;
}
}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDcoISIV/initon/InitonsPDMTest.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDcoISIV/initon/InitonsPDS.java
package A_I.MPE.MAQPVDcoISIV.initon;

import ME.VPC.M.app.App;

import S_A.SVQ.stable.S_MapsInitons;

@SuppressWarnings("unused")
/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class InitonsPDS {
    public static void main(String[] argv) {
    }

    public static int DO_ACP_IDV_16_Array(char[] IDC, int QVM, App NE) {
        int OSI = 0;
        for (int i = 0; i < IDC.length - 1; i++) {
            String string = NE.app_S_S_MapsInitons.initonsMap16.get("" + IDC[i]);
            int temp = NE.app_S_S_MapsInitons.numberSet16.get(string);
            OSI += temp * Math.pow(QVM, IDC.length - i - 1);
        }
        int IC = IDC.length;
        String string = NE.app_S_S_MapsInitons.initonsMap16.get("" + IDC[IC - 1]);
        IC = NE.app_S_S_MapsInitons.numberSet16.get(string);
        return OSI + IC;
    }

    public String EUM_QD_MQD(String AEP, int QSV) {

```

```

        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String EUM_QI_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "V", "E", "C", "S" }, { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String EUM_QD_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String EUM_Q_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "V", "E", "C", "S" },
        { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String EUM_QU_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "V", "E", "C", "S" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String EUP_QD_PQU(String AEP, int QSU) {
        String[][] CED = new String[][] { { "D", "I", "U", "Q" },
        { "H", "T", "X" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String OCS(int[][] VDE) {
        return null;
    }

    public String SDI_P_M_PDS(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) & '0';
            SQV += AEP.charAt(i) | '0';
        }
        return AEP;
    }

    // s-e-i
    public String SDD_P_M_PDS(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";

```



```

        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) | '0';
            SQV += AEP.charAt(i) & '0';
        }
        return AEP;
    }

    public String SDI_P_M_PDS_I(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) & '0';
            SQV += AEP.charAt(i) | '1';
        }
        return AEP;
    }

    // s-e-i
    public String SDD_P_M_PDS_I(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) | '1';
            SQV += AEP.charAt(i) & '0';
        }
        return AEP;
    }

    public String SDI_P_M_PDS_D(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) & '1';
            // SQV+= AEP.charAt(i) | '0';
        }
        return AEP;
    }

    // s-e-i
    public String SDD_P_M_PDS_D(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) | '0';
            // SQV+= AEP.charAt(i) & '1';
        }
        return AEP;
    }

    public String SDI_P_M_PDS_ID(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) & '1';
            // SQV+= AEP.charAt(i) | '0';
        }
        return AEP;
    }

```

```

    }
    // s-e-i
    public String SDD_P_M_PDS_ID(String SQV, int[][] SVE, int[][] mask) {
        String AEP = "";
        for (int i = 0; i < SQV.length(); i++) {
            AEP += SQV.charAt(i) | '0';
            // SQV += AEP.charAt(i) & '1';
        }
        int S = 1;
        int E = 1;
        int I = 1;
        String SEI = "DO ACP.IDV";
        // ...
        return AEP;
    }
    // 51MV_51_D1_SVI 20 2* 41
    public static int DO_ACP_IDV(int[] IDC, int QVM) {
        int OSI = 0;
        for (int j = 0; j < IDC.length - 1; j++) {
            OSI += IDC[j] * Math.pow(QVM, IDC.length - j - 1);
        }
        int IC = IDC.length;
        return OSI + IDC[IC - 1];
    }
    // 稍后会把map并到stable文件去。
    public static int DO_ACP_IDV_17(char[] IDC, int QVM, App NE) {
        int OSI = 0;
        for (int j = 0; j < IDC.length - 1; j++) {
            int temp = NE.app_S_S_MapsInitons.numberSet
                .get(NE.app_S_S_MapsInitons.initonsMap.get("" + IDC[j]));
            OSI += temp * Math.pow(QVM, IDC.length - j - 1);
        }
        int IC = IDC.length;
        IC = NE.app_S_S_MapsInitons.numberSet
            .get(NE.app_S_S_MapsInitons.initonsMap.get("" + IDC[IC - 1]));
        return OSI + IC;
    }
    public int DO_ACP_IDV_17_S_(char[][] IDC, int QVM, App NE) {
        int OSI = 0;
        for (int i = 0; i < IDC.length; i++) {
            for (int j = 0; j < IDC[0].length - 1; j++) {
                int temp = NE.app_S_S_MapsInitons.numberSet
                    .get(NE.app_S_S_MapsInitons.initonsMap.get("" + IDC[i][j]));
                OSI += temp * Math.pow(QVM, IDC[0].length - j - 1);
            }
        }
        int IC = IDC[0].length;
    }

```

```

        IC = NE.app_S_S_MapsInitons.numberSet
        .get(NE.app_S_S_MapsInitons.initonsMap.get("" + IDC[0][IC - 1]));
        return OSI + IC;
    }

    public static int DO_ACP_IDV_16(char[] IDC, int QVM, App NE) {
        int OSI = 0;
        for (int j = 0; j < IDC.length - 1; j++) {
            int temp = NE.app_S_S_MapsInitons.numberSet
            .get(NE.app_S_S_MapsInitons.initonsMap.get("" + IDC[j]));
            OSI += temp * Math.pow(QVM, IDC.length - j - 1);
        }
        int IC = IDC.length;
        IC = NE.app_S_S_MapsInitons.numberSet
        .get(NE.app_S_S_MapsInitons.initonsMap.get("" + IDC[IC - 1]));
        return OSI + IC;
    }

    // later in new file.
}

//// IMV_SQI initonsMap = new IMV_SQI();
//// initonsMap.put("A", "7");
//// initonsMap.put("O", "A");
//// initonsMap.put("P", "2");
//// initonsMap.put("M", "8");
//// initonsMap.put("V", "D");
//// initonsMap.put("E", "3");
//// initonsMap.put("C", "1");
//// initonsMap.put("S", "9");
//// initonsMap.put("I", "E");
//// initonsMap.put("D", "0");
//// initonsMap.put("U", "F");
//// initonsMap.put("Q", "G");
//// initonsMap.put("T", "C");
//// initonsMap.put("X", "6");
//// initonsMap.put("+", "B");
//// initonsMap.put("-", "5");
//// initonsMap.put("H", "4");
////
//// Map<String, Integer> numberSet = new IMV_SQI();
//// numberSet.put("0", 0);
//// numberSet.put("1", 1);
//// numberSet.put("2", 2);
//// numberSet.put("3", 3);
//// numberSet.put("4", 4);
//// numberSet.put("5", 5);
//// numberSet.put("6", 6);
//// numberSet.put("7", 7);
//// numberSet.put("8", 8);

```

```
////    numberSet.put("9", 9);
////    numberSet.put("A", 10);
////    numberSet.put("B", 11);
////    numberSet.put("C", 12);
////    numberSet.put("D", 13);
////    numberSet.put("E", 14);
////    numberSet.put("F", 15);
////    numberSet.put("G", 16);
//    IMV_SQI initonsMap = new IMV_SQI();
//    initonsMap.put("D", "0");
//    initonsMap.put("C", "1");
//    initonsMap.put("P", "2");
//    initonsMap.put("E", "3");
//    initonsMap.put("T", "4");
//    initonsMap.put("H", "5");
//    initonsMap.put("O", "6");
//    initonsMap.put("S", "7");
//    initonsMap.put("M", "8");
//    initonsMap.put("A", "9");
//    initonsMap.put("X", "A");
//    initonsMap.put("F", "B");
//    initonsMap.put("V", "C");
//    initonsMap.put("I", "D");
//    initonsMap.put("U", "E");
//    initonsMap.put("Q", "F");
//
//    Map<String, Integer> numberSet = new IMV_SQI();
//    numberSet.put("0", 0);
//    numberSet.put("1", 1);
//    numberSet.put("2", 2);
//    numberSet.put("3", 3);
//    numberSet.put("4", 4);
//    numberSet.put("5", 5);
//    numberSet.put("6", 6);
//    numberSet.put("7", 7);
//    numberSet.put("8", 8);
//    numberSet.put("9", 9);
//    numberSet.put("A", 10);
//    numberSet.put("B", 11);
//    numberSet.put("C", 12);
//    numberSet.put("D", 13);
//    numberSet.put("E", 14);
//    numberSet.put("F", 15);
//155
//decadeToPDS.initonsMap.put("A", "7");
//    decadeToPDS.initonsMap.put("O", "A");
//    decadeToPDS.initonsMap.put("P", "2");
```

```
//      decadeToPDS.initonsMap.put("M", "8");
//      decadeToPDS.initonsMap.put("V", "D");
//      decadeToPDS.initonsMap.put("E", "3");
//      decadeToPDS.initonsMap.put("C", "1");
//      decadeToPDS.initonsMap.put("S", "9");
//      decadeToPDS.initonsMap.put("I", "E");
//      decadeToPDS.initonsMap.put("D", "0");
//      decadeToPDS.initonsMap.put("U", "F");
//      decadeToPDS.initonsMap.put("Q", "G");
//      decadeToPDS.initonsMap.put("T", "C");
//      decadeToPDS.initonsMap.put("X", "6");
//      decadeToPDS.initonsMap.put("+", "B");
//      decadeToPDS.initonsMap.put("-", "5");
//      decadeToPDS.initonsMap.put("H", "4");
//
//      decadeToPDS.numberSet.put("0", 0);
//      decadeToPDS.numberSet.put("1", 1);
//      decadeToPDS.numberSet.put("2", 2);
//      decadeToPDS.numberSet.put("3", 3);
//      decadeToPDS.numberSet.put("4", 4);
//      decadeToPDS.numberSet.put("5", 5);
//      decadeToPDS.numberSet.put("6", 6);
//      decadeToPDS.numberSet.put("7", 7);
//      decadeToPDS.numberSet.put("8", 8);
//      decadeToPDS.numberSet.put("9", 9);
//      decadeToPDS.numberSet.put("A", 10);
//      decadeToPDS.numberSet.put("B", 11);
//      decadeToPDS.numberSet.put("C", 12);
//      decadeToPDS.numberSet.put("D", 13);
//      decadeToPDS.numberSet.put("E", 14);
//      decadeToPDS.numberSet.put("F", 15);
//      decadeToPDS.numberSet.put("G", 16);

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/initon/InitonsPDSTest.java
/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/initon/InitonTest.java
```

A_I.MPE.MAQPVDISCOISIV.pdc

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pdc/PDC.lyg

A_I.MPE.MAQPVDISCOISIV.pde

```
/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Decrement_Formular.java
package A_I.MPE.MAQPVDISCOISIV.pde;
```

```
import A_I.MPE.MAQPVDISCOISIV.initon.Initon;

import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

import A_I.MPE.MAQPVDISCOISIV.pdi.TokenPDI;

/*
```

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.

//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

```
public class PDE_Decrement_Formular {
    // A= V + S LINK 数据结构对象 (简单测试)
    // 作用发现有A 的initon进行VPCS级展开. 按我在 印度 ANSI C代码风格进行编写方式.
    public Initon PDE_DecrementA(InitonLinkDNA initonLinkDNA) {
        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("A")) {
                Initon initonIncrementV = new Initon();
                initonIncrementV.I_V(); // 新增一个数据V
                Initon initonIncrementS = new Initon();
                initonIncrementS.I_S(); // 新增一个数据S
                initonIncrementV.next = initonIncrementS; // V初始
                initonIncrementV.prev = initonLink.prev;
                if (null != initonIncrementV.prev) {
                    initonIncrementV.prev.next = initonIncrementV;
                }
                initonIncrementS.prev = initonIncrementV; // S初始
                initonIncrementS.next = initonLink.next;
                if (null != initonIncrementS.next) {
                    initonIncrementS.next.prev = initonIncrementS;
                }
                initonLink = initonIncrementS; // 最后S代替
            }
            if (!initonLink.hasNext()) {
                return initonLink;
            }
            initonLink = initonLink.next; // while loop 替增.
        }
        return initonLink;
    }
    // O= E + S LINK 数据结构对象 (简单测试)
    public Initon PDE_DecrementO(InitonLinkDNA initonLinkDNA) {
        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("O")) {
                Initon initonIncrementE = new Initon();
                initonIncrementE.I_E(); // 新增一个数据E
                Initon initonIncrementS = new Initon();
                initonIncrementS.I_S(); // 新增一个数据S
```

```

    initonIncrementE.next = initonIncrementS; // E初始
    initonIncrementE.prev = initonLink.prev;
    if (null != initonIncrementE.prev) {
        initonIncrementE.prev.next = initonIncrementE;
    }
    initonIncrementS.prev = initonIncrementE; // S初始
    initonIncrementS.next = initonLink.next;
    if (null != initonIncrementS.next) {
        initonIncrementS.next.prev = initonIncrementS;
    }
    initonLink = initonIncrementS; // 最后S代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.next; // while loop 替增.
}
return initonLink;
}

// P= E + C LINK 数据结构对象 (简单测试)
public Initon PDE_DecrementP(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("P")) {
Initon initonIncrementE = new Initon();
initonIncrementE.I_E(); // 新增一个数据E
Initon initonIncrementC = new Initon();
initonIncrementC.I_C(); // 新增一个数据C
initonIncrementE.next = initonIncrementC; // E初始
initonIncrementE.prev = initonLink.prev;
if (null != initonIncrementE.prev) {
    initonIncrementE.prev.next = initonIncrementE;
}
initonIncrementC.prev = initonIncrementE; // C初始
initonIncrementC.next = initonLink.next;
if (null != initonIncrementC.next) {
    initonIncrementC.next.prev = initonIncrementC;
}
initonLink = initonIncrementC; // 最后C代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.next; // while loop 替增.
}
return initonLink;
}
}

```

```
// M= C + S

public Initon PDE_DecrementM(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("M")) {
            Initon initonIncrementC = new Initon();
            initonIncrementC.I_C(); // 新增一个数据C
            Initon initonIncrementS = new Initon();
            initonIncrementS.I_S(); // 新增一个数据S
            initonIncrementC.next = initonIncrementS; // C初始
            initonIncrementC.prev = initonLink.prev;
            if (null != initonIncrementC.prev) {
                initonIncrementC.prev.next = initonIncrementC;
            }
            initonIncrementS.prev = initonIncrementC; // s初始
            initonIncrementS.next = initonLink.next;
            if (null != initonIncrementS.next) {
                initonIncrementS.next.prev = initonIncrementS;
            }
            initonLink = initonIncrementS; // 最后S代替
        }
        if (!initonLink.hasNext()) {
            return initonLink;
        }
        initonLink = initonLink.next; // while loop 替增.
    }
    return initonLink;
}

// V= U + Q

public Initon PDE_DecrementV(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("V")) {
            Initon initonIncrementU = new Initon();
            initonIncrementU.I_U(); // 新增一个数据U
            Initon initonIncrementQ = new Initon();
            initonIncrementQ.I_Q(); // 新增一个数据Q
            initonIncrementU.next = initonIncrementQ; // U初始
            initonIncrementU.prev = initonLink.prev;
            if (null != initonIncrementU.prev) {
                initonIncrementU.prev.next = initonIncrementU;
            }
            initonIncrementQ.prev = initonIncrementU; // Q初始
            initonIncrementQ.next = initonLink.next;
            if (null != initonIncrementQ.next) {
                initonIncrementQ.next.prev = initonIncrementQ;
            }
        }
    }
}
```



```

        initonLink = initonIncrementQ; // 最后Q代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.next; // while loop 替增.
}
return initonLink;
}
// E= I + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
public Initon PDE_DecrementE_IU(InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular, boolean bYS) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    int index = 0;
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("E")) {
Initon initonIncrementI = new Initon();
if (bYS) {
    if (pDE_RNA_Formular.pdedeKey.charAt(index++) == '0') {
        initonIncrementI.I_I(); // 新增一个数据I
    } else {
        initonIncrementI.I_D(); // 新增一个数据D
    }
} else {
    if (Math.random() < pDE_RNA_Formular.key[2]) { // later
        // null。
        pDE_RNA_Formular.pdedeKey += "0";
        initonIncrementI.I_I(); // 新增一个数据I
    } else {
        pDE_RNA_Formular.pdedeKey += "1";
        initonIncrementI.I_D(); // 新增一个数据D
    }
}
}
Initon initonIncrementU = new Initon(); // u
initonIncrementU.I_U(); // 新增一个数据u
initonIncrementI.next = initonIncrementU; // I初始
initonIncrementI.prev = initonLink.prev;
if (null != initonIncrementI.prev) {
    initonIncrementI.prev.next = initonIncrementI;
}
initonIncrementU.prev = initonIncrementI; // U初始
initonIncrementU.next = initonLink.next;
if (null != initonIncrementU.next) {
    initonIncrementU.next.prev = initonIncrementU;
}
initonLink = initonIncrementU; // 最后U代替

```

```

    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.next;// while loop 替增.
}
return initonLink;
}
// C= I + D
public Initon PDE_DecrementC(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("C")) {
Initon initonIncrementI = new Initon();
initonIncrementI.I_I(); // 新增一个数据I
Initon initonIncrementD = new Initon();
initonIncrementD.I_D(); // 新增一个数据D
initonIncrementI.next = initonIncrementD; // I初始
initonIncrementI.prev = initonLink.prev;
if (null != initonIncrementI.prev) {
    initonIncrementI.prev.next = initonIncrementI;
}
initonIncrementD.prev = initonIncrementI; // D初始
initonIncrementD.next = initonLink.next;
if (null != initonIncrementD.next) {
    initonIncrementD.next.prev = initonIncrementD;
}
initonLink = initonIncrementD;// 最后D代替
        }
        if (!initonLink.hasNext()) {
return initonLink;
        }
        initonLink = initonLink.next;// while loop 替增.
    }
    return initonLink;
}
// S= I + Q
public Initon PDE_DecrementS(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("S")) {
Initon initonIncrementI = new Initon();
initonIncrementI.I_I(); // 新增一个数据I
Initon initonIncrementQ = new Initon();
initonIncrementQ.I_Q(); // 新增一个数据Q
initonIncrementI.next = initonIncrementQ; // I初始
initonIncrementI.prev = initonLink.prev;

```

```

    if (null != initonIncrementI.prev) {
        initonIncrementI.prev.next = initonIncrementI;
    }
    initonIncrementQ.prev = initonIncrementI; // Q初始
    initonIncrementQ.next = initonLink.next;
    if (null != initonIncrementQ.next) {
        initonIncrementQ.next.prev = initonIncrementQ;
    }
    initonLink = initonIncrementQ; // 最后Q代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.next; // while loop 替增.
    }
    return initonLink;
}
// E= D + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
public Initon PDE_DecrementE_DU(InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("E")) {
            Initon initonIncrementD = new Initon();
            if (Math.random() < pDE_RNA_Formular.key[2]) {
                initonIncrementD.I_I(); // 新增一个数据I
            } else {
                initonIncrementD.I_D(); // 新增一个数据D
            }
            Initon initonIncrementU = new Initon();
            initonIncrementU.I_U(); // 新增一个数据U
            initonIncrementD.next = initonIncrementU; // D初始
            initonIncrementD.prev = initonLink.prev;
            if (null != initonIncrementD.prev) {
                initonIncrementD.prev.next = initonIncrementD;
            }
            initonIncrementU.prev = initonIncrementD; // U初始
            initonIncrementU.next = initonLink.next;
            if (null != initonIncrementU.next) {
                initonIncrementU.next.prev = initonIncrementU;
            }
            initonLink = initonIncrementU; // 最后U代替
        }
        if (!initonLink.hasNext()) {
return initonLink;
        }
    }
}

```

```

        initonLink = initonLink.next;// while loop 替增.

    }

    return initonLink;

}

// S= I
// 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q
public Initon PDE_DecrementS_I(InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular, boolean bYS) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    int index = 0;
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("S")) {
            Initon initonIncrementI = new Initon();
            if (bYS) {
                try {
                    if (pDE_RNA_Formular.pdedKey.charAt(index++) == '0') {
                        initonIncrementI.I_I(); // 新增一个数据I
                    } else {
                        initonIncrementI.I_Q(); // 新增一个数据Q
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                }
            } else {
                if (Math.random() < pDE_RNA_Formular.key[1]) {
                    pDE_RNA_Formular.pdedKey += "0";
                    initonIncrementI.I_I();
                } else {
                    pDE_RNA_Formular.pdedKey += "1";
                    initonIncrementI.I_Q();
                }
            }
        }
        initonIncrementI.prev = initonLink.prev;
        if (null != initonIncrementI.prev) {
            initonIncrementI.prev.next = initonIncrementI;
        }
        initonIncrementI.next = initonLink.next;
        if (null != initonIncrementI.next) {
            initonIncrementI.next.prev = initonIncrementI;
        }
        initonLink = initonIncrementI;// 最后I代替
    }
    if (!initonLink.hasNext()) {
        //// 20230106-S_logger.Log.logger.info("" + pDE_RNA_Formular.pdedKey);
        return initonLink;
    }
    initonLink = initonLink.next;// while loop 替增.

```

```

    }

    /// 20230106-S_logger.Log.logger.info("" + pDE_RNA_Formular.pdedKey);

    return initonLink;

}

// S= Q

// 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q

public Initon PDE_DecrementS_Q(InitonLinkDNA initonLinkDNA,

    TokenPDI pDE_RNA_Formular) {

    Initon initonLink = initonLinkDNA.getInitonLink();

    while (null != initonLink) {

        if (initonLink.getStore().equalsIgnoreCase("S")) {

            Initon initonIncrementQ = new Initon();

            if (Math.random() < pDE_RNA_Formular.key[1]) {

                initonIncrementQ.I_I();

            } else {

                initonIncrementQ.I_Q();

            }

            initonIncrementQ.prev = initonLink.prev;

            if (null != initonIncrementQ.prev) {

                initonIncrementQ.prev.next = initonIncrementQ;

            }

            initonIncrementQ.next = initonLink.next;

            if (null != initonIncrementQ.next) {

                initonIncrementQ.next.prev = initonIncrementQ;

            }

            initonLink = initonIncrementQ; // 最后Q代替

        }

        if (!initonLink.hasNext()) {

            return initonLink;

        }

        initonLink = initonLink.next; // while loop 替增.

    }

    return initonLink;

}

}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Decrement_FormularTest.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Decrement_FullDNAFormular.java
package A_I.MPE.MAQPVDISCOISIV.pde;

import A_I.MPE.MAQPVDISCOISIV.full.FullDNATokenPDI;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;

import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

/*

* 个人著作权人 , 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

```

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.

//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

```
public class PDE_Decrement_FullDNAFormular {
    // A= V + S LINK 数据结构对象 (简单测试)
    // 作用发现有A 的initon进行VPCS级展开. 按我在 印度 ANSI C代码风格进行编写方式.
    public Initon PDE_DecrementA(InitonLinkDNA initonLinkDNA) {
        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("A")) {
                Initon initonIncrementV = new Initon();
                initonIncrementV.I_V(); // 新增一个数据V
                Initon initonIncrementS = new Initon();
                initonIncrementS.I_S(); // 新增一个数据S
                initonIncrementV.next = initonIncrementS; // V初始
                initonIncrementV.prev = initonLink.prev;
                if (null != initonIncrementV.prev) {
                    initonIncrementV.prev.next = initonIncrementV;
                }
                initonIncrementS.prev = initonIncrementV; // S初始
                initonIncrementS.next = initonLink.next;
                if (null != initonIncrementS.next) {
                    initonIncrementS.next.prev = initonIncrementS;
                }
                initonLink = initonIncrementS; // 最后S代替
            }
            if (!initonLink.hasNext()) {
                return initonLink;
            }
            initonLink = initonLink.next; // while loop 替增.
        }
        return initonLink;
    }
    // O= E + S LINK 数据结构对象 (简单测试)
    public Initon PDE_DecrementO(InitonLinkDNA initonLinkDNA) {
        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("O")) {
                Initon initonIncrementE = new Initon();
                initonIncrementE.I_E(); // 新增一个数据E
                Initon initonIncrementS = new Initon();
                initonIncrementS.I_S(); // 新增一个数据S
                initonIncrementE.next = initonIncrementS; // E初始
                initonIncrementE.prev = initonLink.prev;
```

```

    if (null != initonIncrementE.prev) {
        initonIncrementE.prev.next = initonIncrementE;
    }
    initonIncrementS.prev = initonIncrementE; // S初始
    initonIncrementS.next = initonLink.next;
    if (null != initonIncrementS.next) {
        initonIncrementS.next.prev = initonIncrementS;
    }
    initonLink = initonIncrementS; // 最后S代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.next; // while loop 替增.
}
return initonLink;
}

// P= E + C LINK 数据结构对象 (简单测试)
public Initon PDE_DecrementP(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("P")) {
            Initon initonIncrementE = new Initon();
            initonIncrementE.I_E(); // 新增一个数据E
            Initon initonIncrementC = new Initon();
            initonIncrementC.I_C(); // 新增一个数据C
            initonIncrementE.next = initonIncrementC; // E初始
            initonIncrementE.prev = initonLink.prev;
            if (null != initonIncrementE.prev) {
                initonIncrementE.prev.next = initonIncrementE;
            }
            initonIncrementC.prev = initonIncrementE; // C初始
            initonIncrementC.next = initonLink.next;
            if (null != initonIncrementC.next) {
                initonIncrementC.next.prev = initonIncrementC;
            }
            initonLink = initonIncrementC; // 最后C代替
        }
        if (!initonLink.hasNext()) {
return initonLink;
        }
        initonLink = initonLink.next; // while loop 替增.
    }
    return initonLink;
}

// M= C + S
public Initon PDE_DecrementM(InitonLinkDNA initonLinkDNA) {

```

```

        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("M")) {
                Initon initonIncrementC = new Initon();
                initonIncrementC.I_C(); // 新增一个数据C
                Initon initonIncrementS = new Initon();
                initonIncrementS.I_S(); // 新增一个数据S
                initonIncrementC.next = initonIncrementS; // C初始
                initonIncrementC.prev = initonLink.prev;
                if (null != initonIncrementC.prev) {
                    initonIncrementC.prev.next = initonIncrementC;
                }
                initonIncrementS.prev = initonIncrementC; // s初始
                initonIncrementS.next = initonLink.next;
                if (null != initonIncrementS.next) {
                    initonIncrementS.next.prev = initonIncrementS;
                }
                initonLink = initonIncrementS; // 最后S代替
            }
            if (!initonLink.hasNext()) {
                return initonLink;
            }
            initonLink = initonLink.next; // while loop 替增
        }
        return initonLink;
    }
    // V= U + Q
    public Initon PDE_DecrementV(InitonLinkDNA initonLinkDNA) {
        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("V")) {
                Initon initonIncrementU = new Initon();
                initonIncrementU.I_U(); // 新增一个数据U
                Initon initonIncrementQ = new Initon();
                initonIncrementQ.I_Q(); // 新增一个数据Q
                initonIncrementU.next = initonIncrementQ; // U初始
                initonIncrementU.prev = initonLink.prev;
                if (null != initonIncrementU.prev) {
                    initonIncrementU.prev.next = initonIncrementU;
                }
                initonIncrementQ.prev = initonIncrementU; // Q初始
                initonIncrementQ.next = initonLink.next;
                if (null != initonIncrementQ.next) {
                    initonIncrementQ.next.prev = initonIncrementQ;
                }
                initonLink = initonIncrementQ; // 最后Q代替
            }

```



```

        if (!initonLink.hasNext()) {

            return initonLink;

        }

        initonLink = initonLink.next;// while loop 替增.

    }

    return initonLink;

}

// E= D + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
// E= D + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
// 2021年2月10, 优化次黄嘌呤 对应 D 和 U 弱碱强碱. 罗瑶光
// 。这里受酸碱的浓度控制 正如 S= I和 S= Q 强酸弱酸。
// 之后研究 u是否可以略去。今改下E= D 和 E= U
public Initon PDE_DecrementE_DU(InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_Formular, boolean bYS) {

    Initon initonLink = initonLinkDNA.getInitonLink();

    int index = 0;

    while (null != initonLink) {

        if (initonLink.getStore().equalsIgnoreCase("E")) {

            Initon initonIncrementI = new Initon();

            if (bYS) {

                try {

                    if (pDE_RNA_Formular.pdedeKey.charAt(index) == '0') {

                        initonIncrementI.I_D(); // 新增一个数据D

                    }

                    if (pDE_RNA_Formular.pdedeKey.charAt(index) == '1') {

                        initonIncrementI.I_U(); // 新增一个数据U

                    }

                    // else {

                    // initonIncrementI.I_E(); //新增一个数据E

                    // }

                    index++;

                } catch (Exception e) {

                    e.printStackTrace();

                }

            } else {

                if (Math.random() < pDE_RNA_Formular.key[1]) {

                    pDE_RNA_Formular.pdedeKey += "0";

                    initonIncrementI.I_D();

                } else {

                    pDE_RNA_Formular.pdedeKey += "1";

                    initonIncrementI.I_U();

                }

            }

        }

        initonIncrementI.prev = initonLink.prev;

        if (null != initonIncrementI.prev) {

```

```

        initonIncrementI.prev.next = initonIncrementI;
    }
    initonIncrementI.next = initonLink.next;
    if (null != initonIncrementI.next) {
        initonIncrementI.next.prev = initonIncrementI;
    }
    initonLink = initonIncrementI; // 最后I代替
    }
    // else if(initonLink.getStore().equalsIgnoreCase("D")) {
    // pDE_RNA_Formular.pdedeKey+= "2";
    // }else if(initonLink.getStore().equalsIgnoreCase("U")) {
    // pDE_RNA_Formular.pdedeKey+= "2";
    // }
    if (!initonLink.hasNext()) {
    //// 20230106-S_logger.Log.logger.info(" " + pDE_RNA_Formular.pdedeKey);
    return initonLink;
    }
    initonLink = initonLink.next; // while loop 替增.
    }
    //// 20230106-S_logger.Log.logger.info(" " + pDE_RNA_Formular.pdedeKey);
    return initonLink;
}
// C= I + D
public Initon PDE_DecrementC(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("C")) {
            Initon initonIncrementI = new Initon();
            initonIncrementI.I_I(); // 新增一个数据I
            Initon initonIncrementD = new Initon();
            initonIncrementD.I_D(); // 新增一个数据D
            initonIncrementI.next = initonIncrementD; // I初始
            initonIncrementI.prev = initonLink.prev;
            if (null != initonIncrementI.prev) {
                initonIncrementI.prev.next = initonIncrementI;
            }
            initonIncrementD.prev = initonIncrementI; // D初始
            initonIncrementD.next = initonLink.next;
            if (null != initonIncrementD.next) {
                initonIncrementD.next.prev = initonIncrementD;
            }
            initonLink = initonIncrementD; // 最后D代替
        }
        if (!initonLink.hasNext()) {
            return initonLink;
        }
        initonLink = initonLink.next; // while loop 替增.
    }
}

```

```

    }
    return initonLink;
}
// S= I
// 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q
public Initon PDE_DecrementS_IQ(InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_Formular, boolean bYS) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    int index = 0;
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("S")) {
            Initon incrementI = new Initon();
            if (bYS) {
                try {
                    if (pDE_RNA_Formular.pdedKey.charAt(index) == '0') {
                        incrementI.I_I(); // 新增一个数据D
                    }
                    if (pDE_RNA_Formular.pdedKey.charAt(index) == '1') {
                        incrementI.I_Q(); // 新增一个数据U
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                }
            } else {
                incrementI.I_S(); // 新增一个数据E
            }
            index++;
        } else {
            if (Math.random() < pDE_RNA_Formular.key[1]) {
                pDE_RNA_Formular.pdedKey += "0";
                incrementI.I_I();
            } else {
                pDE_RNA_Formular.pdedKey += "1";
                incrementI.I_Q();
            }
        }
        incrementI.prev = initonLink.prev;
        if (null != incrementI.prev) {
            incrementI.prev.next = incrementI;
        }
        incrementI.next = initonLink.next;
        if (null != incrementI.next) {
            incrementI.next.prev = incrementI;
        }
        initonLink = incrementI; // 最后I代替
    }
    else if (initonLink.getStore().equalsIgnoreCase("I")) {

```

```
//          pDE_RNA_Formular.pdedKey+= "2";
//      }else if(initonLink.getStore().equalsIgnoreCase("Q")) {
//          pDE_RNA_Formular.pdedKey+= "2";
//      }

      if (!initonLink.hasNext()) {
// 20230106-S_logger.Log.logger.info("" + pDE_RNA_Formular.pdedKey);
return initonLink;
      }

      initonLink = initonLink.next;// while loop 替增.
  }
  //// 20230106-S_logger.Log.logger.info("" + pDE_RNA_Formular.pdedKey);
  return initonLink;
}
}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDCOISIV/pde/PDE_Decrement_FullDNAFormularTest.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDCOISIV/pde/PDE_Decrement_FullFormular.java
package A_I.MPE.MAQPVDCOISIV.pde;

import A_I.MPE.MAQPVDCOISIV.full.FullTokenPDI;

import A_I.MPE.MAQPVDCOISIV.initon.Initon;

import A_I.MPE.MAQPVDCOISIV.initon.InitonLinkDNA;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.
//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.
public class PDE_Decrement_FullFormular {
    // A= V + S LINK 数据结构对象 (简单测试)
    // 作用发现有A 的initon进行VPCS级展开. 按我在 印度 ANSI C代码风格进行编写方式.
    public Initon PDE_DecrementA(InitonLinkDNA initonLinkDNA) {
        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("A")) {
                Initon initonIncrementV = new Initon();
                initonIncrementV.I_V(); // 新增一个数据V
                Initon initonIncrementS = new Initon();
                initonIncrementS.I_S(); // 新增一个数据S
                initonIncrementV.next = initonIncrementS; // V初始
                initonIncrementV.prev = initonLink.prev;
                if (null != initonIncrementV.prev) {
                    initonIncrementV.prev.next = initonIncrementV;
                }
            }
            initonLink = initonLink.next;
        }
    }
}
```

```

    }
    initonIncrementS.prev = initonIncrementV; // S初始
    initonIncrementS.next = initonLink.next;
    if (null != initonIncrementS.next) {
        initonIncrementS.next.prev = initonIncrementS;
    }
    initonLink = initonIncrementS; // 最后S代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.next; // while loop 替增.
    }
    return initonLink;
}

// O= E + S LINK 数据结构对象 (简单测试)
public Initon PDE_DecrementO(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("O")) {
            Initon initonIncrementE = new Initon();
            initonIncrementE.I_E(); // 新增一个数据E
            Initon initonIncrementS = new Initon();
            initonIncrementS.I_S(); // 新增一个数据S
            initonIncrementE.next = initonIncrementS; // E初始
            initonIncrementE.prev = initonLink.prev;
            if (null != initonIncrementE.prev) {
                initonIncrementE.prev.next = initonIncrementE;
            }
            initonIncrementS.prev = initonIncrementE; // S初始
            initonIncrementS.next = initonLink.next;
            if (null != initonIncrementS.next) {
                initonIncrementS.next.prev = initonIncrementS;
            }
            initonLink = initonIncrementS; // 最后S代替
            }
            if (!initonLink.hasNext()) {
return initonLink;
            }
            initonLink = initonLink.next; // while loop 替增.
            }
            return initonLink;
        }
    }

// P= E + C LINK 数据结构对象 (简单测试)
public Initon PDE_DecrementP(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {

```

```

        if (initonLink.getStore().equalsIgnoreCase("P")) {

            Initon initonIncrementE = new Initon();
            initonIncrementE.I_E(); // 新增一个数据E
            Initon initonIncrementC = new Initon();
            initonIncrementC.I_C(); // 新增一个数据C
            initonIncrementE.next = initonIncrementC; // E初始
            initonIncrementE.prev = initonLink.prev;
            if (null != initonIncrementE.prev) {
                initonIncrementE.prev.next = initonIncrementE;
            }
            initonIncrementC.prev = initonIncrementE; // C初始
            initonIncrementC.next = initonLink.next;
            if (null != initonIncrementC.next) {
                initonIncrementC.next.prev = initonIncrementC;
            }
            initonLink = initonIncrementC; // 最后C代替
        }
        if (!initonLink.hasNext()) {
            return initonLink;
        }
        initonLink = initonLink.next; // while loop 替增.
    }
    return initonLink;
}

// M= C + S
public Initon PDE_DecrementM(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("M")) {
            Initon initonIncrementC = new Initon();
            initonIncrementC.I_C(); // 新增一个数据C
            Initon initonIncrementS = new Initon();
            initonIncrementS.I_S(); // 新增一个数据S
            initonIncrementC.next = initonIncrementS; // C初始
            initonIncrementC.prev = initonLink.prev;
            if (null != initonIncrementC.prev) {
                initonIncrementC.prev.next = initonIncrementC;
            }
            initonIncrementS.prev = initonIncrementC; // s初始
            initonIncrementS.next = initonLink.next;
            if (null != initonIncrementS.next) {
                initonIncrementS.next.prev = initonIncrementS;
            }
            initonLink = initonIncrementS; // 最后S代替
        }
        if (!initonLink.hasNext()) {
            return initonLink;
        }
    }
}

```

```

    }
    initonLink = initonLink.next;// while loop 替增.
}
return initonLink;
}
// V= U + Q
public Initon PDE_DecrementV(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("V")) {
            Initon initonIncrementU = new Initon();
            initonIncrementU.I_U(); // 新增一个数据U
            Initon initonIncrementQ = new Initon();
            initonIncrementQ.I_Q(); // 新增一个数据Q
            initonIncrementU.next = initonIncrementQ; // U初始
            initonIncrementU.prev = initonLink.prev;
            if (null != initonIncrementU.prev) {
                initonIncrementU.prev.next = initonIncrementU;
            }
            initonIncrementQ.prev = initonIncrementU; // Q初始
            initonIncrementQ.next = initonLink.next;
            if (null != initonIncrementQ.next) {
                initonIncrementQ.next.prev = initonIncrementQ;
            }
            initonLink = initonIncrementQ; // 最后Q代替
        }
        if (!initonLink.hasNext()) {
            return initonLink;
        }
        initonLink = initonLink.next;// while loop 替增.
    }
    return initonLink;
}
// E= D + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
// E= D + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
// 2021年2月10, 优化次黄嘌呤 对应 D 和 U 弱碱强碱. 罗瑶光
// 。这里受酸碱的浓度控制 正如 S= I和 S= Q 强酸弱酸。
// 之后研究 u是否可以略去。今改下E= D 和 E= U
public Initon PDE_DecrementE_DU(InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_Formular, boolean bYS) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    int index = 0;
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("E")) {
            Initon initonIncrementI = new Initon();

```

```

    if (bYS) {
        try {
            if (pDE_RNA_Formular.pdedeKey.charAt(index) == '0') {
                initonIncrementL.I_D(); // 新增一个数据D
            }
            if (pDE_RNA_Formular.pdedeKey.charAt(index) == '1') {
                initonIncrementL.I_U(); // 新增一个数据U
            }
            // else {
            // initonIncrementL.I_E(); //新增一个数据E
            // }
            index++;
        } catch (Exception e) {
            e.printStackTrace();
        }
    } else {
        if (Math.random() < pDE_RNA_Formular.key[1]) {
            pDE_RNA_Formular.pdedeKey += "0";
            initonIncrementL.I_D();
        } else {
            pDE_RNA_Formular.pdedeKey += "1";
            initonIncrementL.I_U();
        }
    }
    initonIncrementL.prev = initonLink.prev;
    if (null != initonIncrementL.prev) {
        initonIncrementL.prev.next = initonIncrementI;
    }
    initonIncrementL.next = initonLink.next;
    if (null != initonIncrementL.next) {
        initonIncrementL.next.prev = initonIncrementI;
    }
    initonLink = initonIncrementI; // 最后I代替
    }
    // else if (initonLink.getStore().equalsIgnoreCase("D")) {
    // pDE_RNA_Formular.pdedeKey += "2";
    // } else if (initonLink.getStore().equalsIgnoreCase("U")) {
    // pDE_RNA_Formular.pdedeKey += "2";
    // }
    if (!initonLink.hasNext()) {
    /// 20230106-S_logger.Log.logger.info(" " + pDE_RNA_Formular.pdedeKey);
    return initonLink;
    }
    initonLink = initonLink.next; // while loop 替增.
}
/// 20230106-S_logger.Log.logger.info(" " + pDE_RNA_Formular.pdedeKey);
return initonLink;

```



```

    }
    // C= I + D
    public Initon PDE_DecrementC(InitonLinkDNA initonLinkDNA) {
        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("C")) {
                Initon initonIncrementI = new Initon();
                initonIncrementI.I_I(); // 新增一个数据I
                Initon initonIncrementD = new Initon();
                initonIncrementD.I_D(); // 新增一个数据D
                initonIncrementI.next = initonIncrementD; // I初始
                initonIncrementI.prev = initonLink.prev;
                if (null != initonIncrementI.prev) {
                    initonIncrementI.prev.next = initonIncrementI;
                }
                initonIncrementD.prev = initonIncrementI; // D初始
                initonIncrementD.next = initonLink.next;
                if (null != initonIncrementD.next) {
                    initonIncrementD.next.prev = initonIncrementD;
                }
                initonLink = initonIncrementD; // 最后D代替
            }
            if (!initonLink.hasNext()) {
                return initonLink;
            }
            initonLink = initonLink.next; // while loop 替增.
        }
        return initonLink;
    }
    // S= I
    // 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q
    public Initon PDE_DecrementS_IQ(InitonLinkDNA initonLinkDNA,
        FullTokenPDI pDE_RNA_Formular, boolean bYS) {
        Initon initonLink = initonLinkDNA.getInitonLink();
        int index = 0;
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("S")) {
                Initon initonIncrementI = new Initon();
                if (bYS) {
                    try {
                        if (pDE_RNA_Formular.pdedKey.charAt(index) == '0') {
                            initonIncrementI.I_I(); // 新增一个数据D
                        }
                        if (pDE_RNA_Formular.pdedKey.charAt(index) == '1') {
                            initonIncrementI.I_Q(); // 新增一个数据U
                        }
                    }
                }
            }
        }
    }
    // else {

```

```

//          initonIncrementI.I_S(); //新增一个数据E
//      }

          index++;
      } catch (Exception e) {
          e.printStackTrace();
      }
  } else {
      if (Math.random() < pDE_RNA_Formular.key[1]) {
          pDE_RNA_Formular.pdedKey += "0";
          initonIncrementI.I_I();
      } else {
          pDE_RNA_Formular.pdedKey += "1";
          initonIncrementI.I_Q();
      }
  }

  initonIncrementI.prev = initonLink.prev;
  if (null != initonIncrementI.prev) {
      initonIncrementI.prev.next = initonIncrementI;
  }
  initonIncrementI.next = initonLink.next;
  if (null != initonIncrementI.next) {
      initonIncrementI.next.prev = initonIncrementI;
  }
  initonLink = initonIncrementI; // 最后I代替
  }

//      else if (initonLink.getStore().equalsIgnoreCase("I")) {
//          pDE_RNA_Formular.pdedKey += "2";
//      } else if (initonLink.getStore().equalsIgnoreCase("Q")) {
//          pDE_RNA_Formular.pdedKey += "2";
//      }

          if (!initonLink.hasNext()) {
// 20230106-S_logger.Log.logger.info(" " + pDE_RNA_Formular.pdedKey);
return initonLink;
          }

          initonLink = initonLink.next; // while loop 替增.
      }

      /// 20230106-S_logger.Log.logger.info(" " + pDE_RNA_Formular.pdedKey);
      return initonLink;
  }
}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Decrement_FullFormularTest.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Formular.java
package A_I.MPE.MAQPVDISCOISIV.pde;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;
import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.

//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

```
public class PDE_Formular {  
    public static void main(String[] argv) {  
        Initon initonA = new Initon();  
        initonA.I_M(); // 改成 O 测试下  
        Initon initonV = new Initon();  
        initonV.I_S();  
        Initon initonS = new Initon();  
        initonS.I_I();  
        Initon initonS1 = new Initon();  
        initonS1.I_O();  
        Initon initonS2 = new Initon();  
        initonS2.I_C();  
        Initon initonS3 = new Initon();  
        initonS3.I_U();  
        Initon initonS4 = new Initon();  
        initonS4.I_O();  
        Initon initonS5 = new Initon();  
        initonS5.I_C();  
        Initon initonS6 = new Initon();  
        initonS6.I_I();  
        Initon initonS7 = new Initon();  
        initonS7.I_P();  
        Initon initonS8 = new Initon();  
        initonS8.I_C();  
        Initon initonS9 = new Initon();  
        initonS9.I_U();  
        Initon initonS10 = new Initon();  
        initonS10.I_P();  
        Initon initonS11 = new Initon();  
        initonS11.I_C();  
        Initon initonS12 = new Initon();  
        initonS12.I_I();  
        initonA.next = initonV;  
        initonV.prev = initonA;  
        initonV.next = initonS;  
        initonS.prev = initonV;  
        initonS.next = initonS1;  
        initonS1.prev = initonS;  
        initonS.next = initonS1;
```

```
        initonS1.prev = initonS;  
        initonS1.next = initonS2;  
        initonS2.prev = initonS1;  
        initonS2.next = initonS3;  
        initonS3.prev = initonS2;  
        initonS3.next = initonS4;  
        initonS4.prev = initonS3;  
        initonS4.next = initonS5;  
        initonS5.prev = initonS4;  
        initonS5.next = initonS6;  
        initonS6.prev = initonS5;  
        initonS6.next = initonS7;  
        initonS7.prev = initonS6;  
        initonS7.next = initonS8;  
        initonS8.prev = initonS7;  
        initonS8.next = initonS9;  
        initonS9.prev = initonS8;  
        initonS9.next = initonS10;  
        initonS10.prev = initonS9;  
        initonS10.next = initonS11;  
        initonS11.prev = initonS10;  
        initonS11.next = initonS12;  
        initonS12.prev = initonS11;  
  
        PDE_Formular pDE_RNA_Formular = new PDE_Formular();  
        pDE_RNA_Formular.do_PDE_RNA_Formular(intonA);  
    }  
  
    public void do_PDE_RNA_Formular(Initon inton) {  
        // 初始  
        Initon InitonPDE = inton;  
        // 20230106-System.out.print("input:" +  
        // InitonPDE.getStore());  
        while (InitonPDE.hasNext()) {  
            InitonPDE = InitonPDE.forwardNext();  
            // 20230106-System.out.print(InitonPDE.getStore());  
        }  
        // 20230106-S_logger.Log.logger.info(" " + " ");  
        // 20230106-S_logger.Log.logger.info(" " + "肽展 降元");  
        while (InitonPDE.hasPrev()) {  
            InitonPDE = InitonPDE.forwardPrev();  
        }  
        InitonLinkDNA intonLinkDNA = new InitonLinkDNA();  
        intonLinkDNA.I_InitonLink(InitonPDE);  
        InitonPDE = new PDE_Decrement_Formular().PDE_DecrementA(intonLinkDNA);  
        while (InitonPDE.hasPrev()) {  
            InitonPDE = InitonPDE.forwardPrev();  
        }  
        // 20230106-S_logger.Log.logger.info(" " + " ");  
    }  
}
```

```
// 20230106-S_logger.Log.logger.info("" + "降元A= V + S");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementO(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + "降元O= E + S");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementP(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + "降元P= E + C");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementM(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + "降元M= C + S");
while (InitonPDE.hasNext()) {
```

```

// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementV(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + "降元V= U + Q");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + "1降元概率IU");
doE_IU(InitonPDE, initonLinkDNA);
initonLinkDNA.I_InitonLink(InitonPDE_COPY);
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + "2降元概率DU");
doE_DU(InitonPDE_COPY, initonLinkDNA);
}

private static void doE_IU(Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Decrement_Formular()
.PDE_DecrementE_IU(initonLinkDNA, null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "11降元E= I + U");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
}

```

```

    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "11降元C= I + D");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    // 20230106-S_logger.Log.logger.info("" + "");
    String s = "1111概率S_";
    String si = "1111概率S_I";
    String sq = "1111概率S_Q";
    // 20230106-S_logger.Log.logger.info("" + "1111概率S");
    doS(s, InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSI);
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "1112概率S_I");
    doS_I(si, InitonPDE_COPYSI, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSQ);
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "1113概率S_Q");
    doS_Q(sq, InitonPDE_COPYSQ, initonLinkDNA);
}

private static void doS_Q(String sq, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS_Q(initonLinkDNA,
null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + sq+ "降元S= Q");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
}

```

```

    }

    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    // 全部收
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + sq+ "肽展 增元");
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA = new InitonLinkDNA();
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + sq+ "V= U + Q");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + sq+ "概率Increment IU");
    doIncrementE_IU(sq, InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPY);
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + sq+ "概率Increment DU");
    doIncrementE_DU(sq, InitonPDE_COPY, initonLinkDNA);
}

private static void doS_I(String si, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS_I(initonLinkDNA,
    null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + si+ "11降元S= I");

```

```

        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        // 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        // 全部收
        // 20230106-S_logger.Log.logger.info("" + "");
        // 20230106-S_logger.Log.logger.info("" + si+ "11肽展 增元");
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA = new InitonLinkDNA();
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        // 20230106-S_logger.Log.logger.info("" + "");
        // 20230106-S_logger.Log.logger.info("" + si+ "11V= U + Q");
        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        // 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
        // 20230106-S_logger.Log.logger.info("" + "");
        // 20230106-S_logger.Log.logger.info("" + si+ "111概率Increment IU");
        doIncrementE_IU(si, InitonPDE, initonLinkDNA);
        initonLinkDNA.I_InitonLink(InitonPDE_COPY);
        // 20230106-S_logger.Log.logger.info("" + "");
        // 20230106-S_logger.Log.logger.info("" + si+ "112概率Increment DU");
        doIncrementE_DU(si, InitonPDE_COPY, initonLinkDNA);
    }

    private static void doS(String s, Initon InitonPDE,
        InitonLinkDNA initonLinkDNA) {
        InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
    }

```

```

// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + s+ "11降元S= I + Q");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
// 全部收
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + s+ "11肽展 增元");
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA = new InitonLinkDNA();
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + s+ "11V= U + Q");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + s+ "111概率Increment IU");
doIncrementE_IU(s, InitonPDE, initonLinkDNA);
initonLinkDNA.I_InitonLink(InitonPDE_COPY);
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + s+ "112概率Increment DU");
doIncrementE_DU(s, InitonPDE_COPY, initonLinkDNA);
}

private static void doE_DU(Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Decrement_Formular()
.PDE_DecrementE_DU(initonLinkDNA, null);
    while (InitonPDE.hasPrev()) {

```

```

        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "21降元E= D + U");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "21降元C= I + D");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    String s = "2222概率S_";
    String si = "2222概率S_I_";
    String sq = "2222概率S_Q_";
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "2111概率S");
    doS(s, InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSI);
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "2112概率S_I");
    doS_I(si, InitonPDE_COPYSI, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSQ);
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + "2113概率S_Q");
    doS_Q(sq, InitonPDE_COPYSQ, initonLinkDNA);
}

private static void doIncrementE_DU(String sq, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {

```

```

        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementE_DU(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        // 20230106-S_logger.Log.logger.info(" " + " ");
        // 20230106-S_logger.Log.logger.info(" " + sq+ "E= D + U");
        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        // 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        // 20230106-S_logger.Log.logger.info(" " + " ");
        // 20230106-S_logger.Log.logger.info(" " + sq+ "C= I + D");
        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        // 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        initonLinkDNA.I_InitonLink(InitonPDE);
        Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
        Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
        doIncrementS(sq + "EDU_IQ_", InitonPDE, initonLinkDNA);
        doIncrementS_I(sq + "EDU_I_", InitonPDE_COPYSI, initonLinkDNA);
        doIncrementS_Q(sq + "EDU_Q_", InitonPDE_COPYSQ, initonLinkDNA);
    }
    private static void doIncrementE_IU(String s, Initon InitonPDE,
        InitonLinkDNA initonLinkDNA) {
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementE_IU(initonLinkDNA, null, false);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
    }

```

```

// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + s+ "E= I + U");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + s+ "C= I + D");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Increment_Formular().PDE_IncrementS(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
doIncrementS(s + "EIU_IQ_", InitonPDE, initonLinkDNA);
doIncrementS_I(s + "EIU_I_", InitonPDE_COPYSI, initonLinkDNA);
doIncrementS_Q(s + "EIU_Q_", InitonPDE_COPYSQ, initonLinkDNA);
}

private static void doIncrementS_Q(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementS_Q(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + s+ "S= Q");

```

```

        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        // 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        doIncrementAOPM(s, InitonPDE, initonLinkDNA);
    }

    private static void doIncrementS_I(String s, Initon InitonPDE,
        InitonLinkDNA initonLinkDNA) {
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementS_I(initonLinkDNA,
null);

        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        // 20230106-S_logger.Log.logger.info("" + "");
        // 20230106-S_logger.Log.logger.info("" + s+ "S= I");
        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        // 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        doIncrementAOPM(s, InitonPDE, initonLinkDNA);
    }

    private static void doIncrementS(String s, Initon InitonPDE,
        InitonLinkDNA initonLinkDNA) {
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementS(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        // 20230106-S_logger.Log.logger.info("" + "");
        // 20230106-S_logger.Log.logger.info("" + s+ "S= I + Q");
        while (InitonPDE.hasNext()) {
            // 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        // 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {

```

```

        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    doIncrementAOPM(s, InitonPDE, initonLinkDNA);
}

private static void doIncrementAOPM(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementA(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info(" " + " ");
    // 20230106-S_logger.Log.logger.info(" " + s+ "A= V + S");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementO(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info(" " + " ");
    // 20230106-S_logger.Log.logger.info(" " + s+ "O= E + S");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementP(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info(" " + " ");
    // 20230106-S_logger.Log.logger.info(" " + s+ "P= E + C");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
}

```

```

// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Increment_Formular().PDE_IncrementM(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
// 20230106-S_logger.Log.logger.info("" + "");
// 20230106-S_logger.Log.logger.info("" + s+ "M= C + S");
while (InitonPDE.hasNext()) {
    // 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
}
}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_FormularTest.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_FormularWithKey.java
package A_I.MPE.MAQPVDISCOISIV.pde;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;

import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.
//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

public class PDE_FormularWithKey {
    public double[] key = new double[4];

    public static void main(String[] argv) {
        Initon initonA = new Initon();
        initonA.I_M(); // 改成 O 测试下
        Initon initonV = new Initon();

```



```
initonV.I_S();
Initon initonS = new Initon();
initonS.I_I();
Initon initonS1 = new Initon();
initonS1.I_O();
Initon initonS2 = new Initon();
initonS2.I_C();
Initon initonS3 = new Initon();
initonS3.I_U();
Initon initonS4 = new Initon();
initonS4.I_O();
Initon initonS5 = new Initon();
initonS5.I_C();
Initon initonS6 = new Initon();
initonS6.I_I();
Initon initonS7 = new Initon();
initonS7.I_P();
Initon initonS8 = new Initon();
initonS8.I_C();
Initon initonS9 = new Initon();
initonS9.I_U();
Initon initonS10 = new Initon();
initonS10.I_P();
Initon initonS11 = new Initon();
initonS11.I_C();
Initon initonS12 = new Initon();
initonS12.I_I();
initonA.next = initonV;
initonV.prev = initonA;
initonV.next = initonS;
initonS.prev = initonV;
initonS.next = initonS1;
initonS1.prev = initonS;
initonS.next = initonS1;
initonS1.prev = initonS;
initonS1.next = initonS2;
initonS2.prev = initonS1;
initonS2.next = initonS3;
initonS3.prev = initonS2;
initonS3.next = initonS4;
initonS4.prev = initonS3;
initonS4.next = initonS5;
initonS5.prev = initonS4;
initonS5.next = initonS6;
initonS6.prev = initonS5;
initonS6.next = initonS7;
initonS7.prev = initonS6;
```

```

initonS7.next = initonS8;
initonS8.prev = initonS7;
initonS8.next = initonS9;
initonS9.prev = initonS8;
initonS9.next = initonS10;
initonS10.prev = initonS9;
initonS10.next = initonS11;
initonS11.prev = initonS10;
initonS11.next = initonS12;
initonS12.prev = initonS11;
PDE_FormularWithKey pDE_RNA_Formular = new PDE_FormularWithKey();
@SuppressWarnings("unused")
String initonKeys = "EIU/0.6/EDU/0.4/si/0.3/sq/0.7/EIU/0.5/EDU/0.5/si/0.632/sq/0.368";
pDE_RNA_Formular.key[0] = 0.6;
pDE_RNA_Formular.key[1] = 0.3;
pDE_RNA_Formular.key[2] = 0.5;
pDE_RNA_Formular.key[3] = 0.632;
pDE_RNA_Formular.do_PDE_RNA_Formular(initonA, pDE_RNA_Formular);
}

public void do_PDE_RNA_Formular(Initon initon, PDE_FormularWithKey pDE_RNA_Formular) {
    //初始
    Initon InitonPDE = initon;
    //20230106-System.out.print("input:" + InitonPDE.getStore());
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.forwardNext();
        //20230106-System.out.print(InitonPDE.getStore());
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "肽展 降元");
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementA(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "降元A= V + S");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
}

```

```

}

initonLinkDNA.I_InitonLink(InitonPDE);

InitonPDE = new PDE_Decrement_Formular().PDE_DecrementO(initonLinkDNA);

while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "降元O= E + S");

while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}

//20230106-System.out.print(InitonPDE.getStore());

while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

initonLinkDNA.I_InitonLink(InitonPDE);

InitonPDE = new PDE_Decrement_Formular().PDE_DecrementP(initonLinkDNA);

while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "降元P= E + C");

while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}

//20230106-System.out.print(InitonPDE.getStore());

while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

initonLinkDNA.I_InitonLink(InitonPDE);

InitonPDE = new PDE_Decrement_Formular().PDE_DecrementM(initonLinkDNA);

while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "降元M= C + S");

while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}

//20230106-System.out.print(InitonPDE.getStore());

while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

initonLinkDNA.I_InitonLink(InitonPDE);

```

```

InitonPDE = new PDE_Decrement_Formular().PDE_DecrementV(intonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "降元V= U + Q");
while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
intonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
if (Math.random() < pDE_RNA_Formular.key[0]) {
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "1降元概率IU");
    doE_IU(InitonPDE, intonLinkDNA, pDE_RNA_Formular);
} else {
    intonLinkDNA.I_InitonLink(InitonPDE_COPY);
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "2降元概率DU");
    doE_DU(InitonPDE_COPY, intonLinkDNA, pDE_RNA_Formular);
}
}

private static void doE_IU(Initon InitonPDE, InitonLinkDNA intonLinkDNA
, PDE_FormularWithKey pDE_RNA_Formular) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementE_IU(intonLinkDNA, null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "11降元E= I + U");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    intonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }

```

```

}

//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "11降元C= I + D");
while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}

//20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

initonLinkDNA.I_InitonLink(InitonPDE);
initonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
//20230106-S_logger.Log.logger.info("" + "");
// String s= "1111概率S_";
String si = "1111概率S_I_";
String sq = "1111概率S_Q_";
////20230106-S_logger.Log.logger.info("" + "1111概率S");
//doS(s, InitonPDE, initonLinkDNA);
initonLinkDNA.I_InitonLink(InitonPDE_COPYSI);
if (Math.random() < pDE_RNA_Formular.key[1]) {
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "1112概率S_I");
    doS_I(si, InitonPDE_COPYSI, initonLinkDNA, pDE_RNA_Formular);
} else {
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSQ);
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "1113概率S_Q");
    doS_Q(sq, InitonPDE_COPYSQ, initonLinkDNA, pDE_RNA_Formular);
}
}

private static void doS_Q(String sq, Initon InitonPDE, InitonLinkDNA initonLinkDNA
, PDE_FormularWithKey pDE_RNA_Formular) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementsS_Q(initonLinkDNA, null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + sq+ "降元S= Q");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {

```

```

    InitonPDE = InitonPDE.forwardPrev();
}

initonLinkDNA.I_InitonLink(InitonPDE);
//全部收
//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + sq+ "肽展 增元");
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

initonLinkDNA = new InitonLinkDNA();
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + sq+ "V= U + Q");
while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}

//20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}

initonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
if (Math.random() < pDE_RNA_Formular.key[2]) {
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + sq+ "概率Increment IU");
    doIncrementE_IU(sq, InitonPDE, initonLinkDNA, pDE_RNA_Formular);
} else {
    initonLinkDNA.I_InitonLink(InitonPDE_COPY);
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + sq+ "概率Increment DU");
    doIncrementE_DU(sq, InitonPDE_COPY, initonLinkDNA, pDE_RNA_Formular);
}
}

private static void doS_I(String si, Initon InitonPDE, InitonLinkDNA initonLinkDNA
, PDE_FormularWithKey pDE_RNA_Formular) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS_I(initonLinkDNA, null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }

    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + si+ "11降元S= I");
    while (InitonPDE.hasNext()) {

```

```

//20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
//全部 收
//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + si+ "11肽展 增元");
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA = new InitonLinkDNA();
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + si+ "11V= U + Q");
while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
if (Math.random() < pDE_RNA_Formular.key[2]) {
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + si+ "111概率Increment IU");
    doIncrementE_IU(si, InitonPDE, initonLinkDNA, pDE_RNA_Formular);
} else {
    initonLinkDNA.I_InitonLink(InitonPDE_COPY);
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + si+ "112概率Increment DU");
    doIncrementE_DU(si, InitonPDE_COPY, initonLinkDNA, pDE_RNA_Formular);
}
}
//      private static void doS(String s, Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
//InitonPDE= new PDE_Decrement_Formular().PDE_DecrementS(initonLinkDNA);
//while(InitonPDE.hasPrev()) {
//      InitonPDE= InitonPDE.forwardPrev();

```

```
//}  
////20230106-S_logger.Log.logger.info(" + ");  
//  
////20230106-S_logger.Log.logger.info(" + s+ "11降元S= I + Q");  
//while(InitonPDE.hasNext()) {  
//    //20230106-System.out.print(InitonPDE.getStore());  
//    InitonPDE= InitonPDE.forwardNext();  
//}  
////20230106-System.out.print(InitonPDE.getStore());  
//while(InitonPDE.hasPrev()) {  
//    InitonPDE= InitonPDE.forwardPrev();  
//}  
//initonLinkDNA.I_InitonLink(InitonPDE);  
//  
////全部 收  
////20230106-S_logger.Log.logger.info(" + ");  
////20230106-S_logger.Log.logger.info(" + s+ "11肽展 增元");  
//while(InitonPDE.hasPrev()) {  
//    InitonPDE= InitonPDE.forwardPrev();  
//}  
//initonLinkDNA= new InitonLinkDNA();  
//initonLinkDNA.I_InitonLink(InitonPDE);  
//InitonPDE= new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);  
//while(InitonPDE.hasPrev()) {  
//    InitonPDE= InitonPDE.forwardPrev();  
//}  
////20230106-S_logger.Log.logger.info(" + ");  
////20230106-S_logger.Log.logger.info(" + s+ "11V= U + Q");  
//while(InitonPDE.hasNext()) {  
//    //20230106-System.out.print(InitonPDE.getStore());  
//    InitonPDE= InitonPDE.forwardNext();  
//}  
////20230106-System.out.print(InitonPDE.getStore());  
//while(InitonPDE.hasPrev()) {  
//    InitonPDE= InitonPDE.forwardPrev();  
//}  
//initonLinkDNA.I_InitonLink(InitonPDE);  
//Initon InitonPDE_COPY= InitonPDE.copyRNA(InitonPDE);  
////20230106-S_logger.Log.logger.info(" + ");  
////20230106-S_logger.Log.logger.info(" + s+ "111概率Increment IU");  
//doIncrementE_IU(s, InitonPDE, initonLinkDNA);  
//  
//initonLinkDNA.I_InitonLink(InitonPDE_COPY);  
////20230106-S_logger.Log.logger.info(" + ");  
////20230106-S_logger.Log.logger.info(" + s+ "112概率Increment DU");  
//doIncrementE_DU(s, InitonPDE_COPY, initonLinkDNA);  
//    }
```

```

private static void doE_DU(Initon InitonPDE, InitonLinkDNA initonLinkDNA
, PDE_FormularWithKey pDE_RNA_Formular) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementE_DU(initonLinkDNA, null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "21降元E= D + U");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + "21降元C= I + D");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    // String s= "2222概率S_";
    String si = "2222概率S_I_";
    String sq = "2222概率S_Q_";
    // //20230106-S_logger.Log.logger.info("" + "");
    // //20230106-S_logger.Log.logger.info("" + "2111概率S");
    // doS(s, InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSI);
    if (Math.random() < pDE_RNA_Formular.key[1]) {
        //20230106-S_logger.Log.logger.info("" + "");
        //20230106-S_logger.Log.logger.info("" + "2112概率S_I");
        doS_I(si, InitonPDE_COPYSI, initonLinkDNA, pDE_RNA_Formular);
    } else {
        initonLinkDNA.I_InitonLink(InitonPDE_COPYSQ);
    }
}

```

```

//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + "2113概率S_Q");
doS_Q(sq, InitonPDE_COPYSQ, initonLinkDNA, pDE_RNA_Formular);
}
}

private static void doIncrementE_DU(String sq, Initon InitonPDE
, InitonLinkDNA initonLinkDNA, PDE_FormularWithKey pDE_RNA_Formular) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementE_DU(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + sq+ "E= D + U");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + sq+ "C= I + D");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    //doIncrementS(sq + "EDU_IQ_", InitonPDE, initonLinkDNA);
    if (Math.random() < pDE_RNA_Formular.key[3]) {
        doIncrementS_I(sq + "EDU_I_", InitonPDE_COPYSI, initonLinkDNA);
    } else {
        doIncrementS_Q(sq + "EDU_Q_", InitonPDE_COPYSQ, initonLinkDNA);
    }
}

```

```

}

private static void doIncrementE_IU(String s, Initon InitonPDE
, InitonLinkDNA initonLinkDNA, PDE_FormularWithKey pDE_RNA_Formular) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementE_IU(intonLinkDNA, null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + s + "E= I + U");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + s + "C= I + D");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementS(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    //doIncrementS(s + "EIU_IQ_", InitonPDE, initonLinkDNA);
    if (Math.random() < pDE_RNA_Formular.key[3]) {
        doIncrementS_I(s + "EIU_I_", InitonPDE_COPYSI, initonLinkDNA);
    } else {
        doIncrementS_Q(s + "EIU_Q_", InitonPDE_COPYSQ, initonLinkDNA);
    }
}

```

```

}

private static void doIncrementS_Q(String s, Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementS_Q(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + s+ "S= Q");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    doIncrementAOPM(s, InitonPDE, initonLinkDNA);
}

private static void doIncrementS_I(String s, Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementS_I(initonLinkDNA, null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + s+ "S= I");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    doIncrementAOPM(s, InitonPDE, initonLinkDNA);
}

//      @SuppressWarnings("unused")
//      private static void doIncrementS(String s, Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
//          initonLinkDNA.I_InitonLink(InitonPDE);
//          InitonPDE= new PDE_Increment_Formular().PDE_IncrementS(initonLinkDNA);
//          while(InitonPDE.hasPrev()) {
//              InitonPDE= InitonPDE.forwardPrev();
//          }
//          ///20230106-S_logger.Log.logger.info("" + "");

```

```

////20230106-S_logger.Log.logger.info("" + s+ "S= I + Q");
//while(InitonPDE.hasNext()) {
//    //20230106-System.out.print(InitonPDE.getStore());
//    InitonPDE= InitonPDE.forwardNext();
//}
////20230106-System.out.print(InitonPDE.getStore());
//while(InitonPDE.hasPrev()) {
//    InitonPDE= InitonPDE.forwardPrev();
//}
//initonLinkDNA.I_InitonLink(InitonPDE);
//doIncrementAOPM(s, InitonPDE, initonLinkDNA);
//    }

private static void doIncrementAOPM(String s, Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementA(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + s+ "A= V + S");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementO(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //20230106-S_logger.Log.logger.info("" + "");
    //20230106-S_logger.Log.logger.info("" + s+ "O= E + S");
    while (InitonPDE.hasNext()) {
        //20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementP(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
}

```

```

//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + s+ "P= E + C");
while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Increment_Formular().PDE_IncrementM(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
//20230106-S_logger.Log.logger.info("" + "");
//20230106-S_logger.Log.logger.info("" + s+ "M= C + S");
while (InitonPDE.hasNext()) {
    //20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
}
}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_FormularWithKeyPress.java
package A_I.MPE.MAQPVDISCOISIV.pde;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;
import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;
//import A_I.OPE.ASQ.ASU.MPE.procedure.pde.Initon;
//import A_I.OPE.ASQ.ASU.MPE.procedure.pde.InitonLinkDNA;
//import A_I.OPE.ASQ.ASU.MPE.procedure.pde.PDE_Decrement_Formular;
//import A_I.OPE.ASQ.ASU.MPE.procedure.pde.PDE_FormularWithKeyPress;
//import A_I.OPE.ASQ.ASU.MPE.procedure.pde.PDE_Increment_Formular;
import S_A.SVQ.stable.S_Common;

/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.

```

//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

@SuppressWarnings("unused")

```
public class PDE_FormularWithKeyPress {  
    public double[] key = new double[4];  
    public String bys = "";  
    public String pdw = "";  
    public String pds = "";  
    public String pde = "";  
    public String time = "";  
    public String session_key = "";  
    private String text;  
    private String cacheId;  
    public void doKeyPress(String initons,  
        PDE_FormularWithKeyPress pDE_RNA_Formular) {  
        Initon[] initon = new Initon[initons.length()];  
        for (int i = 0; i < initons.length(); i++) {  
            if (initon[i] == null) {  
initon[i] = new Initon();  
                }  
                initon[i].I_Initon("" + initons.charAt(i));  
                if (i + 1 < initons.length()) {  
if (initon[i + 1] == null) {  
                    initon[i + 1] = new Initon();  
                }  
                initon[i].next = initon[i + 1];  
                initon[i + 1].prev = initon[i];  
            }  
        }  
        do_PDE_RNA_Formular(initon[0], pDE_RNA_Formular);  
    }  
    public static void main(String[] argv) {  
        PDE_FormularWithKeyPress pDE_RNA_Formular = new PDE_FormularWithKeyPress();  
        @SuppressWarnings("unused")  
        String initonKeys = "EIU/0.6/EDU/0.4/si/0.3/sq/0.7/EIU/0.5/EDU/0.5/si/0.632/sq/0.368";  
        pDE_RNA_Formular.key[0] = 0.6;  
        pDE_RNA_Formular.key[1] = 0.3;  
        pDE_RNA_Formular.key[2] = 0.5;  
        pDE_RNA_Formular.key[3] = 0.632;  
        pDE_RNA_Formular.text = "控制吸收";  
        // 20230106-S_logger.Log.logger.info("" + "原文: " +  
        // pDE_RNA_Formular.text);  
        pDE_RNA_Formular.pdw = "MSIOCUOCIPCUPCI";  
        // 20230106-S_logger.Log.logger.info("" + "肽语: " +  
        // pDE_RNA_Formular.pdw);  
        pDE_RNA_Formular.bys = "0.6/0.3/0.5/0.632";  
        // 20230106-S_logger.Log.logger.info("" + "静态密钥: " +  
        // pDE_RNA_Formular.bys);  
    }  
}
```

```

pDE_RNA_Formular.doKeyPress("MSIOCUOCIPCUPCI", pDE_RNA_Formular);

// 20230106-S_logger.Log.logger.info(" + "静态肽展降元: "+
// pDE_RNA_Formular.pds);

// 20230106-S_logger.Log.logger.info(" + "静态肽展增元: "+
// pDE_RNA_Formular.pde);

PDE_FormularWithKeyPress pDE_RNA_Formular1 = new PDE_FormularWithKeyPress();

pDE_RNA_Formular1.key[0] = 0.63;
pDE_RNA_Formular1.key[1] = 0.2;
pDE_RNA_Formular1.key[2] = 0.7;
pDE_RNA_Formular1.key[3] = 0.677;

pDE_RNA_Formular1.doKeyPress(pDE_RNA_Formular.pde, pDE_RNA_Formular1);

pDE_RNA_Formular1.bys = "0.63/0.2/0.7/0.677";

// 20230106-S_logger.Log.logger.info(" + "动态密钥: " +
// pDE_RNA_Formular1.bys);

// 20230106-S_logger.Log.logger.info(" + "动态肽展降元: "+
// pDE_RNA_Formular1.pds);

// 20230106-S_logger.Log.logger.info(" + "动态肽展增元: "+
// pDE_RNA_Formular1.pde);

pDE_RNA_Formular.time = "" + System.currentTimeMillis();

pDE_RNA_Formular.cacheId = "ID" + Math.random()

+ S_Common.STRING_SYMBOL_PER + Math.random();

// 20230106-S_logger.Log.logger.info(" + "时间: " +
// pDE_RNA_Formular.time);

// 20230106-S_logger.Log.logger.info(" + "账号随机缓存字符串: " +
// pDE_RNA_Formular.cacheId);

pDE_RNA_Formular.session_key = pDE_RNA_Formular.pds

+ pDE_RNA_Formular1.pds;

// 20230106-S_logger.Log.logger.info(" + "Session: " +
// pDE_RNA_Formular.session_key);

}

public void do_PDE_RNA_Formular(Initon initon,

    PDE_FormularWithKeyPress pDE_RNA_Formular) {

    // 初始
    Initon InitonPDE = initon;

    //// 20230106-System.out.print("input: " +
    //// InitonPDE.getStore());

    while (InitonPDE.hasNext()) {

        InitonPDE = InitonPDE.forwardNext();

        //// 20230106-System.out.print(InitonPDE.getStore());

    }

    //// 20230106-S_logger.Log.logger.info(" + "");

    //// 20230106-S_logger.Log.logger.info(" + "肽展 降元");

    while (InitonPDE.hasPrev()) {

        InitonPDE = InitonPDE.forwardPrev();

    }

    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();

    initonLinkDNA.I_InitonLink(InitonPDE);

```

```

InitonPDE = new PDE_Decrement_Formular().PDE_DecrementA(intonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
//////// 20230106-S_logger.Log.logger.info("" + "");
//////// 20230106-S_logger.Log.logger.info("" + "降元A= V + S");
while (InitonPDE.hasNext()) {
    //// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
intonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementO(intonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
//////// 20230106-S_logger.Log.logger.info("" + "");
//////// 20230106-S_logger.Log.logger.info("" + "降元O= E + S");
while (InitonPDE.hasNext()) {
    //// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
intonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementP(intonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
//////// 20230106-S_logger.Log.logger.info("" + "");
//////// 20230106-S_logger.Log.logger.info("" + "降元P= E + C");
while (InitonPDE.hasNext()) {
    //// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
intonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementM(intonLinkDNA);
while (InitonPDE.hasPrev()) {

```

```

        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info(" + ");
    // 20230106-S_logger.Log.logger.info(" + "降元M= C + S");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementV(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info(" + ");
    // 20230106-S_logger.Log.logger.info(" + "降元V= U + Q");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
    if (Math.random() < pDE_RNA_Formular.key[0]) {
        // 20230106-S_logger.Log.logger.info(" + ");
        // 20230106-S_logger.Log.logger.info(" + "1降元概率IU");
        doE_IU(InitonPDE, initonLinkDNA, pDE_RNA_Formular);
    } else {
        initonLinkDNA.I_InitonLink(InitonPDE_COPY);
        // 20230106-S_logger.Log.logger.info(" + ");
        // 20230106-S_logger.Log.logger.info(" + "2降元概率DU");
        doE_DU(InitonPDE_COPY, initonLinkDNA, pDE_RNA_Formular);
    }
}

private static void doE_IU(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    InitonPDE = new PDE_Decrement_Formular()
        .PDE_DecrementE_IU(intonLinkDNA, null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
}

```

```

//////// 20230106-S_logger.Log.logger.info("" + "");
//////// 20230106-S_logger.Log.logger.info("" + "11降元E= I + U");
while (InitonPDE.hasNext()) {
    /// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
//////// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
//////// 20230106-S_logger.Log.logger.info("" + "");
/// 20230106-S_logger.Log.logger.info("" + "11降元C= I + D");
while (InitonPDE.hasNext()) {
    /// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
/// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
initonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
/// 20230106-S_logger.Log.logger.info("" + "");
// String s= "1111概率S_";
// String si= "1111概率S_I_";
// String sq= "1111概率S_Q_";
//////// 20230106-S_logger.Log.logger.info("" + "1111概率S");
// doS(s, InitonPDE, initonLinkDNA);
initonLinkDNA.I_InitonLink(InitonPDE_COPYSI);
if (Math.random() < pDE_RNA_Formular.key[1]) {
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "1112概率S_I");
    doS_I("", InitonPDE_COPYSI, initonLinkDNA, pDE_RNA_Formular);
} else {
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSQ);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "1113概率S_Q");
    doS_Q("", InitonPDE_COPYSQ, initonLinkDNA, pDE_RNA_Formular);
}
}

```

```

private static void doS_Q(String sq, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS_Q(initonLinkDNA,
null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //// 20230106-S_logger.Log.logger.info("" + "");
    //// 20230106-S_logger.Log.logger.info("" + sq+ "降元S= Q");
    while (InitonPDE.hasNext()) {
        //// 20230106-System.out.print(InitonPDE.getStore());
        pDE_RNA_Formular.pds += InitonPDE.getStore();
        InitonPDE = InitonPDE.forwardNext();
    }
    //// 20230106-System.out.print(InitonPDE.getStore());
    pDE_RNA_Formular.pds += InitonPDE.getStore();
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    // 全部收
    //// 20230106-S_logger.Log.logger.info("" + "");
    //// 20230106-S_logger.Log.logger.info("" + sq+ "肽展 增元");
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA = new InitonLinkDNA();
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    //// 20230106-S_logger.Log.logger.info("" + "");
    //// 20230106-S_logger.Log.logger.info("" + sq+ "V= U + Q");
    while (InitonPDE.hasNext()) {
        //// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    //// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
    if (Math.random() < pDE_RNA_Formular.key[2]) {
        //// 20230106-S_logger.Log.logger.info("" + "");

```

```

    /// 20230106-S_logger.Log.logger.info("" + sq+ "概率Increment IU");
    doIncrementE_IU(sq, InitonPDE, initonLinkDNA, pDE_RNA_Formular);

} else {

    initonLinkDNA.I_InitonLink(InitonPDE_COPY);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + sq+ "概率Increment DU");
    doIncrementE_DU(sq, InitonPDE_COPY, initonLinkDNA,
pDE_RNA_Formular);
}

}

private static void doS_I(String si, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS_I(initonLinkDNA,
null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + si+ "11降元S= I");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        pDE_RNA_Formular.pds += InitonPDE.getStore();
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    pDE_RNA_Formular.pds += InitonPDE.getStore();
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    // 全部 收
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + si+ "11肽展 增元");
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA = new InitonLinkDNA();
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + si+ "11V= U + Q");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());

```

```

        InitonPDE = InitonPDE.forwardNext();
    }
    //// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
    if (Math.random() < pDE_RNA_Formular.key[2]) {
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + si+ "111概率Increment IU");
        doIncrementE_IU(si, InitonPDE, initonLinkDNA, pDE_RNA_Formular);
    } else {
        initonLinkDNA.I_InitonLink(InitonPDE_COPY);
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + si+ "112概率Increment DU");
        doIncrementE_DU(si, InitonPDE_COPY, initonLinkDNA,
            pDE_RNA_Formular);
    }
}

// private static void doS(String s, Initon InitonPDE,
// InitonLinkDNA initonLinkDNA) {
// InitonPDE= new
// PDE_Decrement_Formular().PDE_DecrementS(initonLinkDNA);
// while(InitonPDE.hasPrev()) {
// InitonPDE= InitonPDE.forwardPrev();
// }
// ////20230106-S_logger.Log.logger.info("" + "");
//
// ////20230106-S_logger.Log.logger.info("" + s+ "11降元S= I + Q");
// while(InitonPDE.hasNext()) {
// ////20230106-System.out.print(InitonPDE.getStore());
// InitonPDE= InitonPDE.forwardNext();
// }
// ////20230106-System.out.print(InitonPDE.getStore());
// while(InitonPDE.hasPrev()) {
// InitonPDE= InitonPDE.forwardPrev();
// }
// initonLinkDNA.I_InitonLink(InitonPDE);
//
// //全部 收
// ////20230106-S_logger.Log.logger.info("" + "");
// ////20230106-S_logger.Log.logger.info("" + s+ "11肽展 增元");
// while(InitonPDE.hasPrev()) {
// InitonPDE= InitonPDE.forwardPrev();
// }
// initonLinkDNA= new InitonLinkDNA();

```

```

// initonLinkDNA.I_InitonLink(InitonPDE);
// InitonPDE= new
// PDE_Increment_Formular().PDE_IncrementV(intonLinkDNA);
// while(InitonPDE.hasPrev()) {
// InitonPDE= InitonPDE.forwardPrev();
// }
// ///20230106-S_logger.Log.logger.info("" + "");
// ///20230106-S_logger.Log.logger.info("" + s+ "11V= U + Q");
// while(InitonPDE.hasNext()) {
// ///20230106-System.out.print(InitonPDE.getStore());
// InitonPDE= InitonPDE.forwardNext();
// }
// ///20230106-System.out.print(InitonPDE.getStore());
// while(InitonPDE.hasPrev()) {
// InitonPDE= InitonPDE.forwardPrev();
// }
// initonLinkDNA.I_InitonLink(InitonPDE);
// Initon InitonPDE_COPY= InitonPDE.copyRNA(InitonPDE);
// ///20230106-S_logger.Log.logger.info("" + "");
// ///20230106-S_logger.Log.logger.info("" + s+ "111概率Increment IU");
// doIncrementE_IU(s, InitonPDE, initonLinkDNA);
//
// initonLinkDNA.I_InitonLink(InitonPDE_COPY);
// ///20230106-S_logger.Log.logger.info("" + "");
// ///20230106-S_logger.Log.logger.info("" + s+ "112概率Increment DU");
// doIncrementE_DU(s, InitonPDE_COPY, initonLinkDNA);
// }
private static void doE_DU(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    InitonPDE = new PDE_Decrement_Formular()
.PDE_DecrementE_DU(intonLinkDNA, null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "21降元E= D + U");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(intonLinkDNA);
    while (InitonPDE.hasPrev()) {

```

```

        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "21降元C= I + D");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    // String s= "2222概率S_";
    // String si= "2222概率S_I_";
    // String sq= "2222概率S_Q_";
    // /// 20230106-S_logger.Log.logger.info("" + "");
    // /// 20230106-S_logger.Log.logger.info("" + "2111概率S");
    // doS(s, InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSI);
    if (Math.random() < pDE_RNA_Formular.key[1]) {
        /// 20230106-S_logger.Log.logger.info("" + "");
        /// 20230106-S_logger.Log.logger.info("" + "2112概率S_I");
        doS_I("", InitonPDE_COPYSI, initonLinkDNA, pDE_RNA_Formular);
    } else {
        initonLinkDNA.I_InitonLink(InitonPDE_COPYSQ);
        /// 20230106-S_logger.Log.logger.info("" + "");
        /// 20230106-S_logger.Log.logger.info("" + "2113概率S_Q");
        doS_Q("", InitonPDE_COPYSQ, initonLinkDNA, pDE_RNA_Formular);
    }
}

private static void doIncrementE_DU(String sq, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementE_DU(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + sq+ "E= D + U");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
}

```



```

    }

    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + sq+ "C= I + D");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    // doIncrementS(sq + "EDU_IQ_", InitonPDE, initonLinkDNA);
    if (Math.random() < pDE_RNA_Formular.key[3]) {
        doIncrementS_I(sq + "EDU_I_", InitonPDE_COPYSI, initonLinkDNA,
        pDE_RNA_Formular);
    } else {
        doIncrementS_Q(sq + "EDU_Q_", InitonPDE_COPYSQ, initonLinkDNA,
        pDE_RNA_Formular);
    }
}

private static void doIncrementE_IU(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementE_IU(initonLinkDNA, null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "E= I + U");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
}

```

```

    }

    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "C= I + D");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementS(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    // doIncrementS(s + "EIU_IQ_", InitonPDE, initonLinkDNA);
    if (Math.random() < pDE_RNA_Formular.key[3]) {
        doIncrementS_I(s + "EIU_I_", InitonPDE_COPYSI, initonLinkDNA,
pDE_RNA_Formular);
    } else {
        doIncrementS_Q(s + "EIU_Q_", InitonPDE_COPYSQ, initonLinkDNA,
pDE_RNA_Formular);
    }
}

private static void doIncrementS_Q(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementS_Q(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");

```

```

    /// 20230106-S_logger.Log.logger.info("" + s+ "S= Q");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    doIncrementAOPM(s, InitonPDE, initonLinkDNA, pDE_RNA_Formular);
}

private static void doIncrementS_I(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementS_I(initonLinkDNA,
null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "S= I");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    doIncrementAOPM(s, InitonPDE, initonLinkDNA, pDE_RNA_Formular);
}

// @SuppressWarnings("unused")
// private static void doIncrementS(String s, Initon
// InitonPDE
// , InitonLinkDNA initonLinkDNA) {
// initonLinkDNA.I_InitonLink(InitonPDE);
// InitonPDE= new
// PDE_Increment_Formular().PDE_IncrementS(initonLinkDNA);
// while(InitonPDE.hasPrev()) {
// InitonPDE= InitonPDE.forwardPrev();
// }
// /// 20230106-S_logger.Log.logger.info("" + "");
// /// 20230106-S_logger.Log.logger.info("" + s+ "S= I + Q");
// while(InitonPDE.hasNext()) {

```

```
// ///20230106-System.out.print(InitonPDE.getStore());
// InitonPDE= InitonPDE.forwardNext();
// }
// ///20230106-System.out.print(InitonPDE.getStore());
// while(InitonPDE.hasPrev()) {
// InitonPDE= InitonPDE.forwardPrev();
// }
// initonLinkDNA.I_InitonLink(InitonPDE);
// doIncrementAOPM(s, InitonPDE, initonLinkDNA);
// }
private static void doIncrementAOPM(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA,
    PDE_FormularWithKeyPress pDE_RNA_Formular) {
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementA(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "A= V + S");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementO(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "O= E + S");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementP(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
}
```

```

    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "P= E + C");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementM(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "肽展增元: ");
    /// 20230106-S_logger.Log.logger.info("" + s+ "M= C + S");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());

        pDE_RNA_Formular.pde += InitonPDE.getStore();

        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
}

}

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_FormularWithKeyTest.java
/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Increment_Formular.java
package A_I.MPE.MAQPVDISCOISIV.pde;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;

import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

import A_I.MPE.MAQPVDISCOISIV.pdi.TokenPDI;

import java.util.Iterator;

import java.util.List;

import java.util.concurrent.CopyOnWriteArrayList;

/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342

```

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.

//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

```
public class PDE_Increment_Formular {

    // A= V + S LINK 数据结构对象 (简单测试)

    // 这个函数我没有按sonar模式 修改, 因为我担心 sonar会潜意识 改变我的写作模式,

    // 于是 按照我很多年的ANSI C代码风格进行编写, 因为这种方式是我的基础. 我个人觉

    // 得 一个人的研发能力来自于他的母语水平. 所以我是C基础, 我应该继续跟进这个天赋. 正如下面这函数.

    public Initon PDE_IncrementA(InitonLinkDNA initonLinkDNA) {

        Initon initonLink = initonLinkDNA.getInitonLink();

        while (null != initonLink) {

            if (initonLink.getStore().equalsIgnoreCase("V")) {

                if (initonLink.hasNext()) {

                    Initon initonNext = initonLink.forwardNext();

                    if (initonNext.getStore().equalsIgnoreCase("S")) {

                        Initon initonIncrementA = new Initon();

                        initonIncrementA.I_A(); // 新增一个数据A

                        if (initonNext.hasNext()) {

                            initonIncrementA.next = initonNext.next; // A后序替换

                            initonIncrementA.next.prev = initonIncrementA; // A后序前序恒等

                        }

                        if (null != initonNext.prev.prev) {

                            initonIncrementA.prev = initonNext.prev.prev; // A前序替换

                            initonIncrementA.prev.next = initonIncrementA; // A前序后序恒等

                        }

                        initonLink = initonIncrementA; // 最后A代替

                    }

                }

            }

            if (!initonLink.hasNext()) {

                return initonLink;

            }

            initonLink = initonLink.forwardNext(); // while loop 替增.

        }

        return initonLink;

    }

    // A= V + S LIST jdk util对象, 下面函数是直接用JDK的虚拟机函数 编写的, 逻辑比较清晰,

    // 各有各的用处.
```

```

public List<Initon> PDE_IncrementA(List<Initon> Initons) {

    List<Initon> output = new CopyOnWriteArrayList<>();

    Iterator<Initon> iterator = Initons.iterator();

    HERE: while (iterator.hasNext()) {

        Initon initon = iterator.next();

        if (initon.getStore().equalsIgnoreCase("V")) {

if (iterator.hasNext()) {

            Initon initonNext = iterator.next();

            if (initonNext.getStore().equalsIgnoreCase("S")) {

                Initon initonIncrementA = new Initon();

                initonIncrementA.I_A();

                output.add(initonIncrementA);

                continue HERE;

            }

            Initon initonIncrementV = new Initon();

            initonIncrementV.I_V();

            Initon initonIncrementS = new Initon();

            initonIncrementS.I_S();

            output.add(initonIncrementV);

            output.add(initonIncrementS);

            continue HERE;

        }

        Initon initonIncrementV = new Initon();

        initonIncrementV.I_V();

        output.add(initonIncrementV);

        continue HERE;

    }

    output.add(initon);

}

return output;

}

// O= E + S LINK 数据结构对象 (简单测试)
public Initon PDE_IncrementO(InitonLinkDNA initonLinkDNA) {

    Initon initonLink = initonLinkDNA.getInitonLink();

    while (null != initonLink) {

        if (initonLink.getStore().equalsIgnoreCase("E")) {

if (initonLink.hasNext()) {

            Initon initonNext = initonLink.forwardNext();

            if (initonNext.getStore().equalsIgnoreCase("S")) {

                Initon initonIncrementO = new Initon();

                initonIncrementO.I_O(); // 新增一个数据O

                if (initonNext.hasNext()) {

                    initonIncrementO.next = initonNext.next; // O后序替换

                    initonIncrementO.next.prev = initonIncrementO; // O后序前序恒等

                }

                if (null != initonNext.prev.prev) {

```

```

        initonIncrementO.prev = initonNext.prev.prev; // O前序替换
        initonIncrementO.prev.next = initonIncrementO; // O前序后序恒等
    }
    initonLink = initonIncrementO; // 最后O代替
}
}

    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext(); // while loop 替增.
}
return initonLink;
}

// P= E + C LINK 数据结构对象 (简单测试)
public Initon PDE_IncrementP(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("E")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("C")) {
                Initon initonIncrementP = new Initon();
                initonIncrementP.I_P(); // 新增一个数据P
                if (initonNext.hasNext()) {
                    initonIncrementP.next = initonNext.next; // P后序替换
                    initonIncrementP.next.prev = initonIncrementP; // P后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementP.prev = initonNext.prev.prev; // P前序替换
                    initonIncrementP.prev.next = initonIncrementP; // P前序后序恒等
                }
                initonLink = initonIncrementP; // 最后P代替
            }
        }
    }

    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext(); // while loop 替增.
}
return initonLink;
}

// M= C + S
public Initon PDE_IncrementM(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {

```



```

        if (initonLink.getStore().equalsIgnoreCase("C")) {
            if (initonLink.hasNext()) {
                Initon initonNext = initonLink.forwardNext();
                if (initonNext.getStore().equalsIgnoreCase("S")) {
                    Initon initonIncrementM = new Initon();
                    initonIncrementM.I_M(); // 新增一个数据M
                    if (initonNext.hasNext()) {
                        initonIncrementM.next = initonNext.next; // M后序替换
                        initonIncrementM.next.prev = initonIncrementM; // M后序前序恒等
                    }
                    if (null != initonNext.prev.prev) {
                        initonIncrementM.prev = initonNext.prev.prev; // M前序替换
                        initonIncrementM.prev.next = initonIncrementM; // M前序后序恒等
                    }
                    initonLink = initonIncrementM; // 最后M代替
                }
            }
        }
        if (!initonLink.hasNext()) {
            return initonLink;
        }
        initonLink = initonLink.forwardNext(); // while loop 替增.
    }
    return initonLink;
}

// V= U + Q
public Initon PDE_IncrementV(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("U")) {
            if (initonLink.hasNext()) {
                Initon initonNext = initonLink.forwardNext();
                if (initonNext.getStore().equalsIgnoreCase("Q")) {
                    Initon initonIncrementV = new Initon();
                    initonIncrementV.I_V(); // 新增一个数据V
                    if (initonNext.hasNext()) {
                        initonIncrementV.next = initonNext.next; // V后序替换
                        initonIncrementV.next.prev = initonIncrementV; // V后序前序恒等
                    }
                    if (null != initonNext.prev.prev) {
                        initonIncrementV.prev = initonNext.prev.prev; // V前序替换
                        initonIncrementV.prev.next = initonIncrementV; // V前序后序恒等
                    }
                    initonLink = initonIncrementV; // 最后V代替
                }
            }
        }
    }
}

```

```

        if (!initonLink.hasNext()) {

            return initonLink;

        }

        initonLink = initonLink.forwardNext();// while loop 替增.

    }

    return initonLink;

}

// E= I + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
public Initon PDE_IncrementE_IU(InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular, boolean bYS) {

    Initon initonLink = initonLinkDNA.getInitonLink();

    // int index= 0;

    while (null != initonLink) {

        String initonString = initonLink.getStore();

        if (initonLink.getStore().equalsIgnoreCase("I")

            || initonLink.getStore().equalsIgnoreCase("D")) {

            if (initonLink.hasNext()) {

                Initon initonNext = initonLink.forwardNext();

                if (initonNext.getStore().equalsIgnoreCase("U")) {

                    Initon initonIncrementE = new Initon();

                    initonIncrementE.I_E(); // 新增一个数据E

                    if (initonString.equalsIgnoreCase("I")) {

                        pDE_RNA_Formular.pdeieKey += "0";

                    } else if (initonString.equalsIgnoreCase("D")) {

                        pDE_RNA_Formular.pdeieKey += "1";

                    }

                    if (initonNext.hasNext()) {

                        initonIncrementE.next = initonNext.next; // E后序替换

                        initonIncrementE.next.prev = initonIncrementE; // E后序前序恒等

                    }

                    if (null != initonNext.prev.prev) {

                        initonIncrementE.prev = initonNext.prev.prev; // E前序替换

                        initonIncrementE.prev.next = initonIncrementE; // E前序后序恒等

                    }

                    initonLink = initonIncrementE; // 最后E代替

                }

            }

        }

        if (!initonLink.hasNext()) {

            return initonLink;

        }

        initonLink = initonLink.forwardNext();// while loop 替增.

    }

    return initonLink;

}

// C= I + D

```

```

public Initon PDE_IncrementC(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("I")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("D")) {
                Initon initonIncrementC = new Initon();
                initonIncrementC.I_C(); // 新增一个数据C
                if (initonNext.hasNext()) {
                    initonIncrementC.next = initonNext.next; // C后序替换
                    initonIncrementC.next.prev = initonIncrementC; // C后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementC.prev = initonNext.prev.prev; // M前序替换
                    initonIncrementC.prev.next = initonIncrementC; // M前序后序恒等
                }
                initonLink = initonIncrementC; // 最后C代替
            }
        }
        if (!initonLink.hasNext()) {
return initonLink;
        }
        initonLink = initonLink.forwardNext(); // while loop 替增.
    }
    return initonLink;
}

// S= I + Q
public Initon PDE_IncrementS(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("I")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("Q")) {
                Initon initonIncrementS = new Initon();
                initonIncrementS.I_S(); // 新增一个数据S
                if (initonNext.hasNext()) {
                    initonIncrementS.next = initonNext.next; // S后序替换
                    initonIncrementS.next.prev = initonIncrementS; // S后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementS.prev = initonNext.prev.prev; // S前序替换
                    initonIncrementS.prev.next = initonIncrementS; // S前序后序恒等
                }
                initonLink = initonIncrementS; // 最后S代替
            }
        }
    }
}

```

```

    }
}

    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}
// E= D + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
public Initon PDE_IncrementE_DU(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("D")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("U")) {
                Initon initonIncrementE = new Initon();
                initonIncrementE.I_E(); // 新增一个数据E
                if (initonNext.hasNext()) {
                    initonIncrementE.next = initonNext.next; // E后序替换
                    initonIncrementE.next.prev = initonIncrementE; // E后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementE.prev = initonNext.prev.prev; // E前序替换
                    initonIncrementE.prev.next = initonIncrementE; // E前序后序恒等
                }
                initonLink = initonIncrementE; // 最后E代替
            }
        }
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}
// S= I
// 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q
public Initon PDE_IncrementS_I(InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        String initonString = initonLink.getStore();

```

```

        if (initonLink.getStore().equalsIgnoreCase("I")) {
            || initonLink.getStore().equalsIgnoreCase("Q")) {
        Initon initonIncrementS = new Initon();
        initonIncrementS.I_S(); // 新增一个数据S
        if (initonString.equalsIgnoreCase("I")) {
            pDE_RNA_Formular.pdeisKey += "0";
        } else if (initonString.equalsIgnoreCase("Q")) {
            pDE_RNA_Formular.pdeisKey += "1";
        }
        if (initonLink.hasNext()) {
            initonIncrementS.next = initonLink.next; // S后序替换
            initonIncrementS.next.prev = initonIncrementS; // S后序前序恒等
        }
        if (null != initonLink.prev) {
            initonIncrementS.prev = initonLink.prev; // S前序替换
            initonIncrementS.prev.next = initonIncrementS; // S前序后序恒等
        }
        initonLink = initonIncrementS; // 最后S代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext(); // while loop 替增.
}
return initonLink;
}
// S= Q
// 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q
public Initon PDE_IncrementS_Q(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("Q")) {
        Initon initonIncrementS = new Initon();
        initonIncrementS.I_S(); // 新增一个数据S
        if (initonLink.hasNext()) {
            initonIncrementS.next = initonLink.next; // S后序替换
            initonIncrementS.next.prev = initonIncrementS; // S后序前序恒等
        }
        if (null != initonLink.prev) {
            initonIncrementS.prev = initonLink.prev; // S前序替换
            initonIncrementS.prev.next = initonIncrementS; // S前序后序恒等
        }
        initonLink = initonIncrementS; // 最后S代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
}

```

```

        initonLink = initonLink.forwardNext();// while loop 替增.

    }

    return initonLink;

}

}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Increment_FormularTest.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Increment_FullDNAFormular.java
package A_I.MPE.MAQPVDISCOISIV.pde;

import A_I.MPE.MAQPVDISCOISIV.full.FullDNATokenPDI;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;

import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

import java.util.Iterator;

import java.util.List;

import java.util.concurrent.CopyOnWriteArrayList;

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.

//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

public class PDE_Increment_FullDNAFormular {

    // A= V + S LINK 数据结构对象 (简单测试)

    // 这个函数我没有按sonar模式 修改, 因为我担心 sonar会潜意识 改变我的写作模式,

    // 于是 按照我很多年的ANSI C代码风格进行编写, 因为这种方式是我的基础. 我个人觉

    // 得 一个人的研发能力来自于他的母语水平.所以我是C基础, 我应该继续跟进这个天赋.正如下面这函数.

    public Initon PDE_IncrementA(InitonLinkDNA initonLinkDNA) {

        Initon initonLink = initonLinkDNA.getInitonLink();

        while (null != initonLink) {

            if (initonLink.getStore().equalsIgnoreCase("V")) {

                if (initonLink.hasNext()) {

                    Initon initonNext = initonLink.forwardNext();

                    if (initonNext.getStore().equalsIgnoreCase("S")) {

                        Initon initonIncrementA = new Initon();

                        initonIncrementA.I_A0(); // 新增一个数据A

                        if (initonNext.hasNext()) {

                            initonIncrementA.next = initonNext.next; // A后序替换

                            initonIncrementA.next.prev = initonIncrementA;// A后序前序恒等

                        }

                        if (null != initonNext.prev.prev) {

                            initonIncrementA.prev = initonNext.prev.prev;// A前序替换

                            initonIncrementA.prev.next = initonIncrementA;// A前序后序恒等

```

```

    }
    initonLink = initonIncrementA;// 最后A代替
}
}
return initonLink;
}
    initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}

```

// A= V + S LIST jdk util对象, 下面函数是直接JDK的虚拟机函数 编写的, 逻辑比较清晰,
// 各有各的用处.

```

public List<Initon> PDE_IncrementA(List<Initon> Initons) {
    List<Initon> output = new CopyOnWriteArrayList<>();
    Iterator<Initon> iterator = Initons.iterator();
    HERE: while (iterator.hasNext()) {
        Initon initon = iterator.next();
        if (initon.getStore().equalsIgnoreCase("V")) {
if (iterator.hasNext()) {
            Initon initonNext = iterator.next();
            if (initonNext.getStore().equalsIgnoreCase("S")) {
                Initon initonIncrementA = new Initon();
                initonIncrementA.I_A();
                output.add(initonIncrementA);
                continue HERE;
            }
            Initon initonIncrementV = new Initon();
            initonIncrementV.I_V();
            Initon initonIncrementS = new Initon();
            initonIncrementS.I_S();
            output.add(initonIncrementV);
            output.add(initonIncrementS);
            continue HERE;
        }
        Initon initonIncrementV = new Initon();
        initonIncrementV.I_V();
        output.add(initonIncrementV);
        continue HERE;
    }
    output.add(initon);
}
return output;
}
// O= E + S LINK 数据结构对象 (简单测试)
public Initon PDE_IncrementO(InitonLinkDNA initonLinkDNA) {

```

```

        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("E")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("S")) {
                Initon initonIncrementO = new Initon();
                initonIncrementO.I_O(); // 新增一个数据O
                if (initonNext.hasNext()) {
                    initonIncrementO.next = initonNext.next; // O后序替换
                    initonIncrementO.next.prev = initonIncrementO; // O后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementO.prev = initonNext.prev.prev; // O前序替换
                    initonIncrementO.prev.next = initonIncrementO; // O前序后序恒等
                }
                initonLink = initonIncrementO; // 最后O代替
            }
        }
        if (!initonLink.hasNext()) {
return initonLink;
        }
        initonLink = initonLink.forwardNext(); // while loop 替增.
    }
    return initonLink;
}

// P= E + C LINK 数据结构对象 (简单测试)
public Initon PDE_IncrementP(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("E")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("C")) {
                Initon initonIncrementP = new Initon();
                initonIncrementP.I_P(); // 新增一个数据P
                if (initonNext.hasNext()) {
                    initonIncrementP.next = initonNext.next; // P后序替换
                    initonIncrementP.next.prev = initonIncrementP; // P后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementP.prev = initonNext.prev.prev; // P前序替换
                    initonIncrementP.prev.next = initonIncrementP; // P前序后序恒等
                }
                initonLink = initonIncrementP; // 最后P代替
            }
        }
    }
}

```



```

    }
        }
        if (!initonLink.hasNext()) {
return initonLink;
        }
        initonLink = initonLink.forwardNext();// while loop 替增.
    }
    return initonLink;
}

// M= C + S
public Initon PDE_IncrementM(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("C")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("S")) {
                Initon initonIncrementM = new Initon();
                initonIncrementM.I_M(); // 新增一个数据M
                if (initonNext.hasNext()) {
                    initonIncrementM.next = initonNext.next; // M后序替换
                    initonIncrementM.next.prev = initonIncrementM; // M后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementM.prev = initonNext.prev.prev; // M前序替换
                    initonIncrementM.prev.next = initonIncrementM; // M前序后序恒等
                }
                initonLink = initonIncrementM; // 最后M代替
            }
        }
    }

    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}

// V= U + Q
public Initon PDE_IncrementV(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("U")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("Q")) {
                Initon initonIncrementV = new Initon();

```

```

        initonIncrementV.I_V(); // 新增一个数据V
        if (initonNext.hasNext()) {
            initonIncrementV.next = initonNext.next; // V后序替换
            initonIncrementV.next.prev = initonIncrementV; // V后序前序恒等
        }
        if (null != initonNext.prev.prev) {
            initonIncrementV.prev = initonNext.prev.prev; // V前序替换
            initonIncrementV.prev.next = initonIncrementV; // V前序后序恒等
        }
        initonLink = initonIncrementV; // 最后V代替
    }
}

    }
    if (!initonLink.hasNext()) {
return initonLink;
    }

    initonLink = initonLink.forwardNext(); // while loop 替增.
}
return initonLink;
}

// E= D + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
public Initon PDE_IncrementE_DU(InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_Formular, boolean bYS) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        String initonString = initonLink.getStore();
        if (initonLink.getStore().equalsIgnoreCase("D"))
            || initonLink.getStore().equalsIgnoreCase("U")) {
            Initon initonIncrementE = new Initon();
            initonIncrementE.I_E(); // 新增一个数据S
            if (initonString.equalsIgnoreCase("D")) {
                pDE_RNA_Formular.pdeieKey += "0";
            } else if (initonString.equalsIgnoreCase("U")) {
                pDE_RNA_Formular.pdeieKey += "1";
            }
            if (initonLink.hasNext()) {
                initonIncrementE.next = initonLink.next; // S后序替换
                initonIncrementE.next.prev = initonIncrementE; // S后序前序恒等
            }
            if (null != initonLink.prev) {
                initonIncrementE.prev = initonLink.prev; // S前序替换
                initonIncrementE.prev.next = initonIncrementE; // S前序后序恒等
            }
            initonLink = initonIncrementE; // 最后S代替
        }
        if (!initonLink.hasNext()) {

```

```

        return initonLink;
    }
    initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}
// C= I + D
public Initon PDE_IncrementC(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("I")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("D")) {
                Initon initonIncrementC = new Initon();
                initonIncrementC.I_C(); // 新增一个数据C
                if (initonNext.hasNext()) {
                    initonIncrementC.next = initonNext.next; // C后序替换
                    initonIncrementC.next.prev = initonIncrementC; // C后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementC.prev = initonNext.prev.prev; // M前序替换
                    initonIncrementC.prev.next = initonIncrementC; // M前序后序恒等
                }
                initonLink = initonIncrementC; // 最后C代替
            }
        }
        if (!initonLink.hasNext()) {
return initonLink;
        }
        initonLink = initonLink.forwardNext();// while loop 替增.
    }
    return initonLink;
}
// S= I
// 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q
public Initon PDE_IncrementS_IQ(InitonLinkDNA initonLinkDNA,
    FullDNATokenPDI pDE_RNA_Formular) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    // int index= 0;
    while (null != initonLink) {
        String initonString = initonLink.getStore();
        if (initonLink.getStore().equalsIgnoreCase("I")
            || initonLink.getStore().equalsIgnoreCase("Q")) {
            Initon initonIncrementS = new Initon();
            initonIncrementS.I_S(); // 新增一个数据S

```

```

        if (initonString.equalsIgnoreCase("I")) {
            pDE_RNA_Formular.pdeisKey += "0";
        } else if (initonString.equalsIgnoreCase("Q")) {
            pDE_RNA_Formular.pdeisKey += "1";
        }
//            if(pDE_RNA_Formular.pdedKey.charAt(index)=='0') {
//                initonIncrementS.I_S0; //新增一个数据S
//            } else if(pDE_RNA_Formular.pdedKey.charAt(index)=='1') {
//                initonIncrementS.I_S0; //新增一个数据S
//            } else if(pDE_RNA_Formular.pdedKey.charAt(index)=='2') {
//                if(initonString.equalsIgnoreCase("I")) {
//                    initonIncrementS.I_I0;
//                    //pDE_RNA_Formular.pdeisKey+= "0";
//                } else if (initonString.equalsIgnoreCase("Q")) {
//                    initonIncrementS.I_Q0;
//                    //pDE_RNA_Formular.pdeisKey+= "1";
//                }
//            }
//            index++;
        if (initonLink.hasNext()) {
            initonIncrementS.next = initonLink.next; // S后序替换
            initonIncrementS.next.prev = initonIncrementS; // S后序前序恒等
        }
        if (null != initonLink.prev) {
            initonIncrementS.prev = initonLink.prev; // S前序替换
            initonIncrementS.prev.next = initonIncrementS; // S前序后序恒等
        }
        initonLink = initonIncrementS; // 最后S代替
    }

    if (!initonLink.hasNext()) {
return initonLink;
    }

    initonLink = initonLink.forwardNext(); // while loop 替增.
}

return initonLink;
}
}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Increment_FullDNAFormularTest.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Increment_FullFormular.java
package A_I.MPE.MAQPVDISCOISIV.pde;

import A_I.MPE.MAQPVDISCOISIV.full.FullTokenPDI;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;

import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

import java.util.Iterator;

import java.util.List;

import java.util.concurrent.CopyOnWriteArrayList;

```

```

/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.
//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

public class PDE_Increment_FullFormular {

    // A= V + S LINK 数据结构对象 (简单测试)
    // 这个函数我没有按sonar模式 修改, 因为我担心 sonar会潜意识 改变我的写作模式,
    // 于是 按照我很多年的ANSI C代码风格进行编写, 因为这种方式是我的基础. 我个人觉
    // 得 一个人的研发能力来自于他的母语水平.所以我是C基础, 我应该继续跟进这个天赋.正如下面这函数.
    public Initon PDE_IncrementA(InitonLinkDNA initonLinkDNA) {

        Initon initonLink = initonLinkDNA.getInitonLink();
        while (null != initonLink) {
            if (initonLink.getStore().equalsIgnoreCase("V")) {
                if (initonLink.hasNext()) {
                    Initon initonNext = initonLink.forwardNext();
                    if (initonNext.getStore().equalsIgnoreCase("S")) {
                        Initon initonIncrementA = new Initon();
                        initonIncrementA.I_A(); // 新增一个数据A
                        if (initonNext.hasNext()) {
                            initonIncrementA.next = initonNext.next; // A后序替换
                            initonIncrementA.next.prev = initonIncrementA; // A后序前序恒等
                        }
                        if (null != initonNext.prev.prev) {
                            initonIncrementA.prev = initonNext.prev.prev; // A前序替换
                            initonIncrementA.prev.next = initonIncrementA; // A前序后序恒等
                        }
                        initonLink = initonIncrementA; // 最后A代替
                    }
                }
            }
            if (!initonLink.hasNext()) {
                return initonLink;
            }
            initonLink = initonLink.forwardNext(); // while loop 替增.
        }
        return initonLink;
    }

    // A= V + S LIST jdk util对象, 下面函数是直接JDK的虚拟机函数 编写的, 逻辑比较清晰,
    // 各有各的用处.
    public List<Initon> PDE_IncrementA(List<Initon> Initons) {

```

```

List<Initon> output = new CopyOnWriteArrayList<>();
Iterator<Initon> iterator = Initons.iterator();
HERE: while (iterator.hasNext()) {
    Initon initon = iterator.next();
    if (initon.getStore().equalsIgnoreCase("V")) {
if (iterator.hasNext()) {
    Initon initonNext = iterator.next();
    if (initonNext.getStore().equalsIgnoreCase("S")) {
        Initon initonIncrementA = new Initon();
        initonIncrementA.I_A();
        output.add(initonIncrementA);
        continue HERE;
    }
    Initon initonIncrementV = new Initon();
    initonIncrementV.I_V();
    Initon initonIncrementS = new Initon();
    initonIncrementS.I_S();
    output.add(initonIncrementV);
    output.add(initonIncrementS);
    continue HERE;
}
    Initon initonIncrementV = new Initon();
    initonIncrementV.I_V();
    output.add(initonIncrementV);
    continue HERE;
    }
    output.add(initon);
}
return output;
}

// O= E + S LINK 数据结构对象 (简单测试)
public Initon PDE_IncrementO(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("E")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("S")) {
                Initon initonIncrementO = new Initon();
                initonIncrementO.I_O(); // 新增一个数据O
                if (initonNext.hasNext()) {
                    initonIncrementO.next = initonNext.next; // O后序替换
                    initonIncrementO.next.prev = initonIncrementO; // O后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementO.prev = initonNext.prev.prev; // O前序替换
                    initonIncrementO.prev.next = initonIncrementO; // O前序后序恒等

```

```

    }
    initonLink = initonIncrementO;// 最后O代替
}
}

}
if (!initonLink.hasNext()) {
return initonLink;
}
initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}

// P= E + C LINK 数据结构对象 (简单测试)
public Initon PDE_IncrementP(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("E")) {
if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("C")) {
                Initon initonIncrementP = new Initon();
                initonIncrementP.I_P(); // 新增一个数据P
                if (initonNext.hasNext()) {
                    initonIncrementP.next = initonNext.next; // P后序替换
                    initonIncrementP.next.prev = initonIncrementP; // P后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementP.prev = initonNext.prev.prev; // P前序替换
                    initonIncrementP.prev.next = initonIncrementP; // P前序后序恒等
                }
                initonLink = initonIncrementP; // 最后P代替
            }
        }
    }
    if (!initonLink.hasNext()) {
return initonLink;
}
initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}

// M= C + S
public Initon PDE_IncrementM(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("C")) {
if (initonLink.hasNext()) {

```

```

        Initon initonNext = initonLink.forwardNext();
        if (initonNext.getStore().equalsIgnoreCase("S")) {
            Initon initonIncrementM = new Initon();
            initonIncrementM.I_M0(); // 新增一个数据M
            if (initonNext.hasNext()) {
                initonIncrementM.next = initonNext.next; // M后序替换
                initonIncrementM.next.prev = initonIncrementM; // M后序前序恒等
            }
            if (null != initonNext.prev.prev) {
                initonIncrementM.prev = initonNext.prev.prev; // M前序替换
                initonIncrementM.prev.next = initonIncrementM; // M前序后序恒等
            }
            initonLink = initonIncrementM; // 最后M代替
        }
    }

    if (!initonLink.hasNext()) {
        return initonLink;
    }

    initonLink = initonLink.forwardNext(); // while loop 替增.
}

return initonLink;
}

// V= U + Q
public Initon PDE_IncrementV(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("U")) {
            if (initonLink.hasNext()) {
                Initon initonNext = initonLink.forwardNext();
                if (initonNext.getStore().equalsIgnoreCase("Q")) {
                    Initon initonIncrementV = new Initon();
                    initonIncrementV.I_V0(); // 新增一个数据V
                    if (initonNext.hasNext()) {
                        initonIncrementV.next = initonNext.next; // V后序替换
                        initonIncrementV.next.prev = initonIncrementV; // V后序前序恒等
                    }
                    if (null != initonNext.prev.prev) {
                        initonIncrementV.prev = initonNext.prev.prev; // V前序替换
                        initonIncrementV.prev.next = initonIncrementV; // V前序后序恒等
                    }
                    initonLink = initonIncrementV; // 最后V代替
                }
            }
        }
    }

    if (!initonLink.hasNext()) {
        return initonLink;
    }
}

```



```

    }
    initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}
// 201-
// Initon initonLink= initonLinkDNA.getInitonLink();
// //int index= 0;
// while(null!= initonLink) {
// String initonString= initonLink.getStore();
// if(initonLink.getStore().equalsIgnoreCase("I")
// || initonLink.getStore().equalsIgnoreCase("D")) {
// if(initonLink.hasNext()) {
// Initon initonNext= initonLink.forwardNext();
// if(initonNext.getStore().equalsIgnoreCase("U")) { //
// Initon initonIncrementE= new Initon();
// initonIncrementE.I_E(); //新增一个数据E
// if(initonString.equalsIgnoreCase("I")) {
// pDE_RNA_Formular.pdeieKey+= "0";
// }else if(initonString.equalsIgnoreCase("D")) {
// pDE_RNA_Formular.pdeieKey+= "1";
// }
// if(initonNext.hasNext()) {
// initonIncrementE.next= initonNext.next; //E后序替换
// initonIncrementE.next.prev= initonIncrementE; //E后序前序恒等
// }
// if(null!= initonNext.prev.prev) {
// initonIncrementE.prev= initonNext.prev.prev; //E前序替换
// initonIncrementE.prev.next= initonIncrementE; //E前序后序恒等
// }
// initonLink= initonIncrementE; //最后E代替
// }
// }
// }
// if(!initonLink.hasNext()) {
// return initonLink;
// }
// initonLink= initonLink.forwardNext();//while loop 替增.
// }
// return initonLink;
// E= D + U
// 肽展公式的概率问题, 先这样命名, 之后讨论 是IU还是DU
public Initon PDE_IncrementE_DU(InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_Formular, boolean bYS) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        String initonString = initonLink.getStore();

```

```

        if (initonLink.getStore().equalsIgnoreCase("D"))
            || initonLink.getStore().equalsIgnoreCase("U")) {
        Initon initonIncrementE = new Initon();
        initonIncrementE.I_E(); // 新增一个数据S
        if (initonString.equalsIgnoreCase("D")) {
            pDE_RNA_Formular.pdeieKey += "0";
        } else if (initonString.equalsIgnoreCase("U")) {
            pDE_RNA_Formular.pdeieKey += "1";
        }
        if (initonLink.hasNext()) {
            initonIncrementE.next = initonLink.next; // S后序替换
            initonIncrementE.next.prev = initonIncrementE; // S后序前序恒等
        }
        if (null != initonLink.prev) {
            initonIncrementE.prev = initonLink.prev; // S前序替换
            initonIncrementE.prev.next = initonIncrementE; // S前序后序恒等
        }
        initonLink = initonIncrementE; // 最后S代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext(); // while loop 替增.
}
return initonLink;
}
// C= I + D
public Initon PDE_IncrementC(InitonLinkDNA initonLinkDNA) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    while (null != initonLink) {
        if (initonLink.getStore().equalsIgnoreCase("I")) {
        if (initonLink.hasNext()) {
            Initon initonNext = initonLink.forwardNext();
            if (initonNext.getStore().equalsIgnoreCase("D")) {
                Initon initonIncrementC = new Initon();
                initonIncrementC.I_C(); // 新增一个数据C
                if (initonNext.hasNext()) {
                    initonIncrementC.next = initonNext.next; // C后序替换
                    initonIncrementC.next.prev = initonIncrementC; // C后序前序恒等
                }
                if (null != initonNext.prev.prev) {
                    initonIncrementC.prev = initonNext.prev.prev; // M前序替换
                    initonIncrementC.prev.next = initonIncrementC; // M前序后序恒等
                }
                initonLink = initonIncrementC; // 最后C代替
            }
        }
    }
}

```

```

    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}
// 315-
// if(pDE_RNA_Formular.pdedKey.charAt(index)== '0') {
// initonIncrementS.I_S(); //新增一个数据S
// }else if(pDE_RNA_Formular.pdedKey.charAt(index)== '1') {
// initonIncrementS.I_S(); //新增一个数据S
// }else if(pDE_RNA_Formular.pdedKey.charAt(index)== '2') {
// if(initonString.equalsIgnoreCase("I")) {
// initonIncrementS.I_I();
// //pDE_RNA_Formular.pdeisKey+= "0";
// }else if(initonString.equalsIgnoreCase("Q")) {
// initonIncrementS.I_Q();
// //pDE_RNA_Formular.pdeisKey+= "1";
// }
// }
// index++;
// S= I
// 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q
public Initon PDE_IncrementS_IQ(InitonLinkDNA initonLinkDNA,
    FullTokenPDI pDE_RNA_Formular) {
    Initon initonLink = initonLinkDNA.getInitonLink();
    // int index= 0;
    while (null != initonLink) {
        String initonString = initonLink.getStore();
        if (initonLink.getStore().equalsIgnoreCase("I"))
            || initonLink.getStore().equalsIgnoreCase("Q")) {
        Initon initonIncrementS = new Initon();
        initonIncrementS.I_S(); // 新增一个数据S
        if (initonString.equalsIgnoreCase("I")) {
            pDE_RNA_Formular.pdeisKey += "0";
        } else if (initonString.equalsIgnoreCase("Q")) {
            pDE_RNA_Formular.pdeisKey += "1";
        }
        if (initonLink.hasNext()) {
            initonIncrementS.next = initonLink.next; // S后序替换
            initonIncrementS.next.prev = initonIncrementS; // S后序前序恒等
        }
        if (null != initonLink.prev) {
            initonIncrementS.prev = initonLink.prev; // S前序替换
            initonIncrementS.prev.next = initonIncrementS; // S前序后序恒等
        }
    }
}

```

```

    }
    initonLink = initonIncrementS;// 最后S代替
    }
    if (!initonLink.hasNext()) {
return initonLink;
    }
    initonLink = initonLink.forwardNext();// while loop 替增.
}
return initonLink;
}
// S= Q
// 肽展公式的概率问题, 先这样命名, 之后讨论 是I, Q还是I + Q
// public Initon PDE_IncrementS_Q(InitonLinkDNA
// initonLinkDNA) {
// Initon initonLink= initonLinkDNA.getInitonLink();
// while(null!= initonLink) {
// if(initonLink.getStore().equalsIgnoreCase("Q")) {
// Initon initonIncrementS= new Initon();
// initonIncrementS.I_S(); //新增一个数据S
// if(initonLink.hasNext()) {
// initonIncrementS.next= initonLink.next; //S后序替换
// initonIncrementS.next.prev= initonIncrementS;//S后序前序恒等
// }
// if(null!= initonLink.prev) {
// initonIncrementS.prev= initonLink.prev; //S前序替换
// initonIncrementS.prev.next= initonIncrementS;//S前序后序恒等
// }
// initonLink= initonIncrementS;//最后S代替
//
// }
// if(!initonLink.hasNext()) {
// return initonLink;
// }
// initonLink= initonLink.forwardNext();//while loop 替增.
// }
// return initonLink;
// }
}

//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_Increment_FullFormularTest.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_RNA_Formular.java
package A_I.MPE.MAQPVDISCOISIV.pde;

import A_I.MPE.MAQPVDISCOISIV.initon.Initon;
import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;

/*
* 个人著作权人 , 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342

```

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.

//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

```
public class PDE_RNA_Formular {  
    public static void main(String[] argv) {  
        Initon initonA = new Initon();  
        initonA.I_M(); // 改成 O 测试下  
        Initon initonV = new Initon();  
        initonV.I_S();  
        Initon initonS = new Initon();  
        initonS.I_I();  
        Initon initonS1 = new Initon();  
        initonS1.I_O();  
        Initon initonS2 = new Initon();  
        initonS2.I_C();  
        Initon initonS3 = new Initon();  
        initonS3.I_U();  
        Initon initonS4 = new Initon();  
        initonS4.I_O();  
        Initon initonS5 = new Initon();  
        initonS5.I_C();  
        Initon initonS6 = new Initon();  
        initonS6.I_I();  
        Initon initonS7 = new Initon();  
        initonS7.I_P();  
        Initon initonS8 = new Initon();  
        initonS8.I_C();  
        Initon initonS9 = new Initon();  
        initonS9.I_U();  
        Initon initonS10 = new Initon();  
        initonS10.I_P();  
        Initon initonS11 = new Initon();  
        initonS11.I_C();  
        Initon initonS12 = new Initon();  
        initonS12.I_I();  
        initonA.next = initonV;  
        initonV.prev = initonA;  
        initonV.next = initonS;  
        initonS.prev = initonV;  
        initonS.next = initonS1;  
        initonS1.prev = initonS;  
        initonS.next = initonS1;  
        initonS1.prev = initonS;  
        initonS1.next = initonS2;
```

```

        initonS2.prev = initonS1;
        initonS2.next = initonS3;
        initonS3.prev = initonS2;
        initonS3.next = initonS4;
        initonS4.prev = initonS3;
        initonS4.next = initonS5;
        initonS5.prev = initonS4;
        initonS5.next = initonS6;
        initonS6.prev = initonS5;
        initonS6.next = initonS7;
        initonS7.prev = initonS6;
        initonS7.next = initonS8;
        initonS8.prev = initonS7;
        initonS8.next = initonS9;
        initonS9.prev = initonS8;
        initonS9.next = initonS10;
        initonS10.prev = initonS9;
        initonS10.next = initonS11;
        initonS11.prev = initonS10;
        initonS11.next = initonS12;
        initonS12.prev = initonS11;
        PDE_RNA_Formular pDE_RNA_Formular = new PDE_RNA_Formular();
        pDE_RNA_Formular.do_PDE_RNA_Formular(intonA);
    }

    public void do_PDE_RNA_Formular(Initon initon) {
        // 初始
        Initon InitonPDE = initon;
        // 20230106-System.out.print("input:" +
        // InitonPDE.getStore());
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.forwardNext();
            // 20230106-System.out.print(InitonPDE.getStore());
        }
        // 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + "肽展 降元");
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Decrement_Formular().PDE_DecrementA(intonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + "降元A= V + S");
        while (InitonPDE.hasNext()) {

```

```

    /// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
/// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementO(intonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
/// 20230106-S_logger.Log.logger.info("" + "");
/// 20230106-S_logger.Log.logger.info("" + "降元O= E + S");
while (InitonPDE.hasNext()) {
    /// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
/// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementP(intonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
/// 20230106-S_logger.Log.logger.info("" + "");
/// 20230106-S_logger.Log.logger.info("" + "降元P= E + C");
while (InitonPDE.hasNext()) {
    /// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
/// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Decrement_Formular().PDE_DecrementM(intonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
/// 20230106-S_logger.Log.logger.info("" + "");
/// 20230106-S_logger.Log.logger.info("" + "降元M= C + S");
while (InitonPDE.hasNext()) {
    /// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}

```

```

    }

    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementV(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "降元V= U + Q");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "1降元概率IU");
    doE_IU(InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPY);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "2降元概率DU");
    doE_DU(InitonPDE_COPY, initonLinkDNA);
}

private static void doE_IU(Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Decrement_Formular()
.PDE_DecrementE_IU(initonLinkDNA, null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "11降元E= I + U");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);

```

```

    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "11降元C= I + D");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    /// 20230106-S_logger.Log.logger.info("" + "");
    String s = "1111概率S_";
    String si = "1111概率S_I_";
    String sq = "1111概率S_Q_";
    /// 20230106-S_logger.Log.logger.info("" + "1111概率S");
    doS(s, InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSI);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "1112概率S_I");
    doS_I(si, InitonPDE_COPYSI, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSQ);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "1113概率S_Q");
    doS_Q(sq, InitonPDE_COPYSQ, initonLinkDNA);
}

private static void doS_Q(String sq, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS_Q(intonLinkDNA,
null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + sq+ "降元S= Q");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());

```

```

        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        // 全部 收
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + sq+ "肽展 增元");
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA = new InitonLinkDNA();
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + sq+ "V= U + Q");
        while (InitonPDE.hasNext()) {
            //// 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        //// 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + sq+ "概率Increment IU");
        doIncrementE_IU(sq, InitonPDE, initonLinkDNA);
        initonLinkDNA.I_InitonLink(InitonPDE_COPY);
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + sq+ "概率Increment DU");
        doIncrementE_DU(sq, InitonPDE_COPY, initonLinkDNA);
    }

    private static void doS_I(String si, Initon InitonPDE,
        InitonLinkDNA initonLinkDNA) {
        InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS_I(initonLinkDNA,
null, false);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + si+ "11降元S= I");
        while (InitonPDE.hasNext()) {
            //// 20230106-System.out.print(InitonPDE.getStore());

```

```

        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    // 全部收
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + si+ "11肽展 增元");
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA = new InitonLinkDNA();
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + si+ "11V= U + Q");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + si+ "111概率Increment IU");
    doIncrementE_IU(si, InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPY);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + si+ "112概率Increment DU");
    doIncrementE_DU(si, InitonPDE_COPY, initonLinkDNA);
}

private static void doS(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "11降元S= I + Q");

```

```

while (InitonPDE.hasNext()) {
    /// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
/// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
// 全部收
/// 20230106-S_logger.Log.logger.info("" + "");
/// 20230106-S_logger.Log.logger.info("" + s+ "11肽展 增元");
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA = new InitonLinkDNA();
initonLinkDNA.I_InitonLink(InitonPDE);
InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(initonLinkDNA);
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
/// 20230106-S_logger.Log.logger.info("" + "");
/// 20230106-S_logger.Log.logger.info("" + s+ "11V= U + Q");
while (InitonPDE.hasNext()) {
    /// 20230106-System.out.print(InitonPDE.getStore());
    InitonPDE = InitonPDE.forwardNext();
}
/// 20230106-System.out.print(InitonPDE.getStore());
while (InitonPDE.hasPrev()) {
    InitonPDE = InitonPDE.forwardPrev();
}
initonLinkDNA.I_InitonLink(InitonPDE);
Initon InitonPDE_COPY = InitonPDE.copyRNA(InitonPDE);
/// 20230106-S_logger.Log.logger.info("" + "");
/// 20230106-S_logger.Log.logger.info("" + s+ "111概率Increment IU");
doIncrementE_IU(s, InitonPDE, initonLinkDNA);
initonLinkDNA.I_InitonLink(InitonPDE_COPY);
/// 20230106-S_logger.Log.logger.info("" + "");
/// 20230106-S_logger.Log.logger.info("" + s+ "112概率Increment DU");
doIncrementE_DU(s, InitonPDE_COPY, initonLinkDNA);
}

private static void doE_DU(Initon InitonPDE, InitonLinkDNA initonLinkDNA) {
    InitonPDE = new PDE_Decrement_Formular()
.PDE_DecrementE_DU(initonLinkDNA, null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
}

```

```

    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "21降元E= D + U");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "21降元C= I + D");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    String s = "2222概率S_";
    String si = "2222概率S_I_";
    String sq = "2222概率S_Q_";
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "2111概率S");
    doS(s, InitonPDE, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSI);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "2112概率S_I");
    doS_I(si, InitonPDE_COPYSI, initonLinkDNA);
    initonLinkDNA.I_InitonLink(InitonPDE_COPYSQ);
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + "2113概率S_Q");
    doS_Q(sq, InitonPDE_COPYSQ, initonLinkDNA);
}

private static void doIncrementE_DU(String sq, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular()

```

```

.PDE_IncrementE_DU(intonLinkDNA);

    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + sq+ "E= D + U");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + sq+ "C= I + D");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    initonLinkDNA.I_InitonLink(InitonPDE);
    Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
    Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
    doIncrementS(sq + "EDU_IQ_", InitonPDE, initonLinkDNA);
    doIncrementS_I(sq + "EDU_I_", InitonPDE_COPYSI, initonLinkDNA);
    doIncrementS_Q(sq + "EDU_Q_", InitonPDE_COPYSQ, initonLinkDNA);
}

private static void doIncrementE_IU(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementE_IU(intonLinkDNA, null, false);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "E= I + U");

```

```

        while (InitonPDE.hasNext()) {
            /// 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        /// 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        /// 20230106-S_logger.Log.logger.info("" + "");
        /// 20230106-S_logger.Log.logger.info("" + s+ "C= I + D");
        while (InitonPDE.hasNext()) {
            /// 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        /// 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementS(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        Initon InitonPDE_COPYSI = InitonPDE.copyRNA(InitonPDE);
        Initon InitonPDE_COPYSQ = InitonPDE.copyRNA(InitonPDE);
        doIncrementS(s + "EIU_IQ_", InitonPDE, initonLinkDNA);
        doIncrementS_I(s + "EIU_I_", InitonPDE_COPYSI, initonLinkDNA);
        doIncrementS_Q(s + "EIU_Q_", InitonPDE_COPYSQ, initonLinkDNA);
    }

    private static void doIncrements_Q(String s, Initon InitonPDE,
        InitonLinkDNA initonLinkDNA) {
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementS_Q(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        /// 20230106-S_logger.Log.logger.info("" + "");
        /// 20230106-S_logger.Log.logger.info("" + s+ "S= Q");
        while (InitonPDE.hasNext()) {
            /// 20230106-System.out.print(InitonPDE.getStore());

```

```

        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    doIncrementAOPM(s, InitonPDE, initonLinkDNA);
}

private static void doIncrementS_I(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementS_I(initonLinkDNA,
null);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "S= I");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    doIncrementAOPM(s, InitonPDE, initonLinkDNA);
}

private static void doIncrementS(String s, Initon InitonPDE,
    InitonLinkDNA initonLinkDNA) {
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementS(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    /// 20230106-S_logger.Log.logger.info("" + s+ "S= I + Q");
    while (InitonPDE.hasNext()) {
        /// 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    /// 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
}

```



```
        initonLinkDNA.I_InitonLink(InitonPDE);
        doIncrementAOPM(s, InitonPDE, initonLinkDNA);
    }

    private static void doIncrementAOPM(String s, Initon InitonPDE,
        InitonLinkDNA initonLinkDNA) {
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementA(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + s+ "A= V + S");
        while (InitonPDE.hasNext()) {
            //// 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        //// 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementO(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + s+ "O= E + S");
        while (InitonPDE.hasNext()) {
            //// 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        //// 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementP(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.forwardPrev();
        }
        //// 20230106-S_logger.Log.logger.info("" + "");
        //// 20230106-S_logger.Log.logger.info("" + s+ "P= E + C");
        while (InitonPDE.hasNext()) {
            //// 20230106-System.out.print(InitonPDE.getStore());
            InitonPDE = InitonPDE.forwardNext();
        }
        //// 20230106-System.out.print(InitonPDE.getStore());
        while (InitonPDE.hasPrev()) {
```

```

        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementM(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    // 20230106-S_logger.Log.logger.info("" + "");
    // 20230106-S_logger.Log.logger.info("" + s+ "M= C + S");
    while (InitonPDE.hasNext()) {
        // 20230106-System.out.print(InitonPDE.getStore());
        InitonPDE = InitonPDE.forwardNext();
    }
    // 20230106-System.out.print(InitonPDE.getStore());
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.forwardPrev();
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
}
}

```

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pde/PDE_RNA_FormularTest.java

A_I.MPE.MAQPVDISCOISIV.pdi

```

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pdi/PowerPDI.java
package A_I.MPE.MAQPVDISCOISIV.pdi;
import S_A.SVQ.stable.S_Common;
@SuppressWarnings("unused")
/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class PowerPDI {
    // 1-r-1-r-1s-e-p-0-v
    public static void main(String[] argv) {
    }
    public String injectPDI(String pdi) {
        return pdi;
    }
    public String PDSEncode(String VSQ) {
        while (VSQ.length() > 0) {
            // VSQ.concat(VSQ.replace("", ""))> 0;
        }
        return VSQ;
    }
    public String PDS_OEC_IIU(String VSQ_IIE, int OCI_PPE) {
        //
        String $ _CID = "AOPMVECS";
        // VSQ_IIE= $_CID.charAt(VSQ_IIE.length());
        return VSQ_IIE;
    }
    public String PDS_P_USQ_ECM(int P_VSQ, int MSP) {
        String ISQ = "";
        while (P_VSQ > 0) {
            ISQ += P_VSQ / MSP;
            P_VSQ %= MSP;
        }
        ISQ += P_VSQ;
        return ISQ;
    }
    public String ESU_M_SVQ_PDS_OEQ(String SQA) {

```

```

        String[] PDS = new String[] { "AOPM", "VECS", "DIUQ", "HTX" };
        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }
    public String ESU_ECS_SVQ_PDS_OEQ(String SQA) {
        String[] PDS = new String[] { "AOPM", "VECS" };
        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }
    public String[][] ESU_P_SVQ_PDS_OEQ(String SQA) {
        String[][] PDS = new String[1][SQA.length()];
        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        for (int i = 0; i < SQA.length(); i++) {
            PDS[0][i] = S_Common.STRING_EMPTY + SQA.charAt(i);
        }
        return PDS;
    }
    public String EUM_QD_MQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_QI_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "V", "E", "C", "S" }, { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_QD_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_Q_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "V", "E", "C", "S" },
        { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUM_QU_PQD(String AEP, int QSV) {
        String[][] CED = new String[][] { { "V", "E", "C", "S" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String EUP_QD_PQU(String AEP, int QSU) {
        String[][] CED = new String[][] { { "D", "I", "U", "Q" },
        { "H", "T", "X" } };
        String PM_ISQ = "";
        return AEP;
    }
    public String OCS(int[][] VDE) {
        return null;
    }
}
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVD COISIV/pdi/PowerPDI Test.java
//Trif_DNA-master/C_L_A/A_I/MPE/MAQPVD COISIV/pdi/RangePDI.java
package A_I.MPE.MAQPVD COISIV.pdi;

```

```
import A_I.MPE.MAQPVD COISIV.initon.InitonsPDS;
```

```
import ME.VPC.M.app.App;
```

```
import S_A.SVQ.stable.S_MapsInitons;
```

```
@SuppressWarnings("unused")
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
```

```
* 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道

* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class RangePDI {

    public static void main(String[] argv) {

        //// 20230106-S_logger.Log.logger.info("" + new RangePDI().IOE(16,
        //// 20));

    }

    // 64/4 8/4 2

    public static String PDS_P_USQ_ECP_I(int P_VSQ, int MSP) {

        String ISQ = "";

        while (P_VSQ >= MSP) {

            ISQ += P_VSQ / MSP;

            P_VSQ %= MSP;

        }

        ISQ += P_VSQ;

        return ISQ;

    }

    public static double[] Double_IPE_AOPM_VECS_IDUQ_TXH_AC(double[] ps,

        App NE) {

        int VECS = NE.app_S.sourcePagePDE;

        for (int i = 0; i < ps.length; i++) {

            boolean isNeg = false;

            int temp = (int) ps[i];

            if (temp < 0) {

isNeg = true;

            }

            temp = Math.abs(temp);

            temp = temp > 255 ? 255 : temp;

            String IDUQ = RangePDI.PDS_P_USQ_ECP_I_16(temp, 16, NE);

            char[] OIQ = new char[IDUQ.length()];

            for (int k = 0; k < IDUQ.length(); k++) {

double tempRandom = Math.random() * 100;

            OIQ[k] = doACAK(IDUQ, k, tempRandom, VECS);

            }

            ps[i] = InitonsPDS.DO_ACP_IDV_16_Array(OIQ, 16, NE);

            if (isNeg) {

ps[i] = -ps[i];

            }

        }

    }

}
```

```

        return ps;
    }

    public static float[] Double_IPE_AOPM_VECS_IDUQ_TXH_AC(float[] ps, App NE) {
        int VECS = NE.app_S.sourcePagePDE;
        for (int i = 0; i < ps.length; i++) {
            boolean isNeg = false;
            int temp = (int) ps[i];
            if (temp < 0) {
isNeg = true;
            }
            temp = Math.abs(temp);
            temp = temp > 255 ? 255 : temp;
            String IDUQ = RangePDI.PDS_P_USQ_ECP_I_16(temp, 16, NE);
            char[] OIQ = new char[IDUQ.length()];
            for (int k = 0; k < IDUQ.length(); k++) {
double tempRandom = Math.random() * 100;
OIQ[k] = doACAK(IDUQ, k, tempRandom, VECS);
            }
            ps[i] = InitonsPDS.DO_ACP_IDV_16_Array(OIQ, 16, NE);
            if (isNeg) {
ps[i] = -ps[i];
            }
        }
        return ps;
    }

    // P_VSQ;
    public static String PDS_P_USQ_ECP_I_16(int P_VSQ, int MSP, App NE) {
        StringBuilder ISQ = new StringBuilder();
        while (P_VSQ >= MSP) {
            String string = NE.app_S_S_MapsInitons.initonsSet16.get("" + P_VSQ / MSP);
            ISQ.append(NE.app_S_S_MapsInitons.initonsCode16.get(string));
            P_VSQ %= MSP;
        }
        String string = NE.app_S_S_MapsInitons.initonsSet16.get("" + P_VSQ);
        ISQ.append(NE.app_S_S_MapsInitons.initonsCode16.get(string));
        return ISQ.toString();
    }

    // 新增 T = V + HE; --page204
    // T = V + HE, 酸= C + E= P, 碱= V + S= A
    // 新增 X = V + HC; --page204
    // X = V + HC, 酸= C + C= C, 碱= V + V= V
    // A= V + S, 酸= C + E= P, 碱= V + S= A
    // O= E + S, 酸= E + E= E, 碱= V + S= A
    // P= E + C, 酸= E + C= P, 碱= S + V= A
    // M= C + S, 酸= C + E= P, 碱= V + S= A
    // F= E+ C + S, 酸= H, 碱= V

    public static char doACAK(String IDUQ, int k, double tempRandom, int VECS) {

```

```

        if (IDUQ.charAt(k) == 'U') { // g
            if (tempRandom < VECS) {
                return 'Q';
            } else {
                return 'U';
            }
        } else if (IDUQ.charAt(k) == 'Q') { // s
            if (tempRandom < VECS) {
                return 'Q';
            } else {
                return 'U';
            }
        } else if (IDUQ.charAt(k) == 'I') { // s
            if (tempRandom < VECS) {
                return 'I';
            } else {
                return 'D';
            }
        } else if (IDUQ.charAt(k) == 'D') { // g
            if (tempRandom < VECS) {
                return 'I';
            } else {
                return 'D';
            }
        } else if (IDUQ.charAt(k) == 'V') { // U // DIUQ
            if (tempRandom < VECS) { // SEVC 相对应
                return 'C';
            } else {
                return 'V';
            }
        } else if (IDUQ.charAt(k) == 'E') { // I
            if (tempRandom < VECS) {
                return 'E';
            } else {
                return 'S';
            }
        } else if (IDUQ.charAt(k) == 'C') { // Q
            if (tempRandom < VECS) {
                return 'C';
            } else {
                return 'V';
            }
        } else if (IDUQ.charAt(k) == 'S') { // D
            if (tempRandom < VECS) {
                return 'E';
            } else {
                return 'S';
            }
        }

```

```

    }
    } else if (IDUQ.charAt(k) == 'A') {
        if (tempRandom < VECS) {
return 'P';
        } else {
return 'A';
        }
    } else if (IDUQ.charAt(k) == 'O') {
        if (tempRandom < VECS) {
return 'E';
        } else {
return 'A';
        }
    } else if (IDUQ.charAt(k) == 'P') {
        if (tempRandom < VECS) {
return 'P';
        } else {
return 'A';
        }
    } else if (IDUQ.charAt(k) == 'M') {
        if (tempRandom < VECS) {
return 'P';
        } else {
return 'A';
        }
    } else if (IDUQ.charAt(k) == 'F') {
        if (tempRandom < VECS) {
return 'H';
        } else {
return 'V';
        }
    } else {
        return IDUQ.charAt(k);
    }
}
// 240/4 600
public static int[][] IOE(int[][] ps, int VECS) {
    for (int i = 0; i < ps.length; i++) {
        for (int j = 0; j < ps[0].length; j++) {
String IDUQ = RangePDI.PDS_P_USQ_ECP_I(ps[i][j], 4);
int[] OIQ = new int[IDUQ.length()];
for (int k = 0; k < IDUQ.length(); k++) {
    if (IDUQ.charAt(k) == '2') {
        if (Math.random() * 100 > VECS) {
            OIQ[k] = 3;
        } else {
            OIQ[k] = 1;

```

```

    }
    } else {
        OIQ[k] = Integer.valueOf("'" + IDUQ.charAt(k));
    }
}
ps[i][j] = InitonsPDS.DO_ACP_IDV(OIQ, 4);
    }
}
return ps;
}

// ##### 元基数字= 元基符号= 肽展公式元基数字变换= (肽概率展开数字逻辑集合)
// ##### 1位 ##### E= I= I=(I)
// ##### F= U= I++ OR Q--=(I, Q)
// ##### G= Q= Q=(Q)
// ##### 1~2位
// ##### 0= D= DD=(D, DD)
// ##### 2位
// ##### 1= C= DI=(DI)
// ##### 3= E= IU, DU=(IU, DU)
// ##### D= V= UQ=(UQ)
// ##### 9= S= QI=(QI)
// ##### 2~4位
// ##### 4= H= (IU, DU) OR DI=(IU, DU, DI) OR (IUDI, DUDI)
// ##### 4位
// ##### 2= P= (IU, DU) + DI=(IUDI, DUDI)
// ##### A= O= (IU, DU) + QI=(IUQI, DUQI)
// ##### 7= A= UQQI=(UQQI)
// ##### 4~6位
// ##### 5= HC-= ((IU, DU) OR DI) + DI=(IUDI, DUDI, DIDI) OR
// (IUDIDI, DUDIDI)
// ##### B= HE+= ((IU, DU) OR DI) + (IU, DU)=(IUIU, IUDU,
// DUIU, DUDU, DIIU, DIDU)
// OR (IUDIU, IUDIDU, DUDIU, DUDIDU)
// ##### 6~8位
// ##### 8= M= ((IU, DU) OR DI) + DI + QI=(IUDI, DUDI,
// DIDI, QI) OR (IUDIDI, DUDIDI, QI)
// ##### 6= X= UQ + ((IU, DU) OR DI) + DI=(UQIUDI, UQDUDI,
// UQDIDI) OR (UQIUDIDI, UQDUDIDI)
// ##### C= T= UQ + ((IU, DU) OR DI) + (IU, DU)=(UQIUIU,
// UQIUDU, UQDUIU, UQDUDU
// , UQDIIU, UQDIDU) OR (UQIUDIIU, UQIUDIDU, UQDUDIIU,
// UQDUDIDU)

public static int[][] IPE_AOPM_VECS_IDUQ_TXH(int[][] ps, int VECS, App NE) {
    for (int i = 0; i < ps.length; i++) {
        for (int j = 0; j < ps[0].length; j++) {
            String IDUQ = RangePDI.PDS_P_USQ_ECP_I_17(ps[i][j], 17, NE);
            char[] OIQ = new char[IDUQ.length()];

```

```

for (int k = 0; k < IDUQ.length(); k++) {
    if (IDUQ.charAt(k) == 'U') { // g //DIUQ
        if (Math.random() * 100 > VECS) { // 按生化计算来。大于酸 小数, 小于碱
            // 大数
            OIQ[k] = 'Q';
        } else {
            OIQ[k] = IDUQ.charAt(k);
        }
    } else if (IDUQ.charAt(k) == 'Q') { // s
        if (Math.random() * 100 < VECS) {
            OIQ[k] = 'D';
        } else {
            OIQ[k] = IDUQ.charAt(k);
        }
    } else if (IDUQ.charAt(k) == 'I') { // s
        if (Math.random() * 100 < VECS) {
            OIQ[k] = 'U';
        } else {
            OIQ[k] = IDUQ.charAt(k);
        }
    } else if (IDUQ.charAt(k) == 'D') { // g
        if (Math.random() * 100 > VECS) {
            OIQ[k] = 'I';
        } else {
            OIQ[k] = IDUQ.charAt(k);
        }
    } else if (IDUQ.charAt(k) == 'V') { // U //DIUQ
        if (Math.random() * 100 > VECS) { // SEVC 相对应 //符号写翻了纠正
            // >20210820
            OIQ[k] = 'C';
        } else {
            OIQ[k] = 'V';
        }
    } else if (IDUQ.charAt(k) == 'E') { // I
        if (Math.random() * 100 < VECS) {
            OIQ[k] = 'V';
        } else {
            OIQ[k] = 'E';
        }
    } else if (IDUQ.charAt(k) == 'C') { // Q
        if (Math.random() * 100 < VECS) {
            OIQ[k] = 'S';
        } else {
            OIQ[k] = 'C';
        }
    } else if (IDUQ.charAt(k) == 'S') { // D
        if (Math.random() * 100 > VECS) { // 符号写翻了纠正 >20210820

```

```

        OIQ[k] = 'E';
    } else {
        OIQ[k] = 'S';
    }
} else {
    OIQ[k] = IDUQ.charAt(k);
}
}
ps[i][j] = InitonsPDS.DO_ACP_IDV_17(OIQ, 17, NE);
}
}
return ps;
}

```

// 上面, 统一 VECS 小数是碱 大数是酸。按标准肽展公式模拟下计算机视觉

```

public static int[][] IPE_AOPM_VECS_IDUQ_TXH_AC(int[][] ps, int VECS, App NE) {
    for (int i = 0; i < ps.length; i++) {
        for (int j = 0; j < ps[0].length; j++) {
            String IDUQ = RangePDI.PDS_P_USQ_ECP_I_16(ps[i][j], 16, NE);
            char[] OIQ = new char[IDUQ.length()];
            for (int k = 0; k < IDUQ.length(); k++) {
                if (IDUQ.charAt(k) == 'U') { // g
                    if (Math.random() * 100 < VECS) {
                        OIQ[k] = 'Q';
                    } else {
                        OIQ[k] = 'U';
                    }
                } else if (IDUQ.charAt(k) == 'Q') { // s
                    if (Math.random() * 100 < VECS) {
                        OIQ[k] = 'Q';
                    } else {
                        OIQ[k] = 'U';
                    }
                } else if (IDUQ.charAt(k) == 'T') { // s
                    if (Math.random() * 100 < VECS) {
                        OIQ[k] = 'T';
                    } else {
                        OIQ[k] = 'D';
                    }
                } else if (IDUQ.charAt(k) == 'D') { // g
                    if (Math.random() * 100 < VECS) {
                        OIQ[k] = 'T';
                    } else {
                        OIQ[k] = 'D';
                    }
                } else if (IDUQ.charAt(k) == 'V') { // U //DIUQ
                    if (Math.random() * 100 < VECS) { // SEVC 相对应
                        OIQ[k] = 'C';
                    }
                }
            }
        }
    }
}

```

```
        } else {  
            OIQ[k] = 'V';  
        }  
    } else if (IDUQ.charAt(k) == 'E') { // I  
        if (Math.random() * 100 < VECS) {  
            OIQ[k] = 'E';  
        } else {  
            OIQ[k] = 'S';  
        }  
    } else if (IDUQ.charAt(k) == 'C') { // Q  
        if (Math.random() * 100 < VECS) {  
            OIQ[k] = 'C';  
        } else {  
            OIQ[k] = 'V';  
        }  
    } else if (IDUQ.charAt(k) == 'S') { // D  
        if (Math.random() * 100 < VECS) {  
            OIQ[k] = 'E';  
        } else {  
            OIQ[k] = 'S';  
        }  
    } else if (IDUQ.charAt(k) == 'A') { // A= V + S, 酸= C + E= P,  
        // 碱= V + S= A  
        if (Math.random() * 100 < VECS) {  
            OIQ[k] = 'P';  
        } else {  
            OIQ[k] = 'A';  
        }  
    } else if (IDUQ.charAt(k) == 'O') { // O= E + S, 酸= E + E= E,  
        // 碱= V + S= A  
        if (Math.random() * 100 < VECS) {  
            OIQ[k] = 'E';  
        } else {  
            OIQ[k] = 'A';  
        }  
    } else if (IDUQ.charAt(k) == 'P') { // P= E + C, 酸= E + C= P,  
        // 碱= S + V= A  
        if (Math.random() * 100 < VECS) {  
            OIQ[k] = 'P';  
        } else {  
            OIQ[k] = 'A';  
        }  
    } else if (IDUQ.charAt(k) == 'M') { // M= C + S, 酸= C + E= P,  
        // 碱= V + S= A  
        if (Math.random() * 100 < VECS) {  
            OIQ[k] = 'P';  
        } else {
```

```

        OIQ[k] = 'A';
    }
} else if (IDUQ.charAt(k) == 'F') { // F= E+ C + S, 酸= H, 碱=
    // V
    if (Math.random() * 100 < VECS) {
        OIQ[k] = 'H';
    } else {
        OIQ[k] = 'V';
    }
} else {
    OIQ[k] = IDUQ.charAt(k);
}
}
ps[i][j] = InitonsPDS.DO_ACP_IDV_16(OIQ, 16, NE);
}
}
return ps;
}

public static int[][] IPE(int[][] ps, int VECS) {
    for (int i = 0; i < ps.length; i++) {
        for (int j = 0; j < ps[0].length; j++) {
String IDUQ = RangePDI.PDS_P_USQ_ECP_I(ps[i][j], 4);
int[] OIQ = new int[IDUQ.length()];
for (int k = 0; k < IDUQ.length(); k++) {
    if (IDUQ.charAt(k) == '2') { // g
        if (Math.random() * 100 > VECS) {
            OIQ[k] = 3;
        } else {
            OIQ[k] = Integer.valueOf("" + IDUQ.charAt(k));
        }
    } else if (IDUQ.charAt(k) == '3') { // s
        if (Math.random() * 100 < VECS) {
            OIQ[k] = 0;
        } else {
            OIQ[k] = Integer.valueOf("" + IDUQ.charAt(k));
        }
    } else if (IDUQ.charAt(k) == '1') { // s
        if (Math.random() * 100 < VECS) {
            OIQ[k] = 2;
        } else {
            OIQ[k] = Integer.valueOf("" + IDUQ.charAt(k));
        }
    } else if (IDUQ.charAt(k) == '0') { // g
        if (Math.random() * 100 > VECS) {
            OIQ[k] = 1;
        } else {
            OIQ[k] = Integer.valueOf("" + IDUQ.charAt(k));
        }
    }
}
}
}

```

```

    }
}

ps[i][j] = InitonsPDS.DO_ACP_IDV(OIQ, 4);
    }
}

return ps;
}

public static int[][] QPE(int[][] ps, int VECS) {
    for (int i = 0; i < ps.length; i++) {
        for (int j = 0; j < ps[0].length; j++) {
            String IDUQ = RangePDI.PDS_P_USQ_ECP_I(ps[i][j], 4);
            int[] OIQ = new int[IDUQ.length()];
            for (int k = 0; k < IDUQ.length(); k++) {
                if (IDUQ.charAt(k) == '0') { // g D I U Q
                    if (Math.random() * 100 > VECS) {
                        OIQ[k] = 1;
                    } else {
                        OIQ[k] = 0;
                    }
                } else if (IDUQ.charAt(k) == '1') { // s
                    if (Math.random() * 100 > VECS) {
                        OIQ[k] = 1;
                    } else {
                        OIQ[k] = 2;
                    }
                } else if (IDUQ.charAt(k) == '2') { // s
                    if (Math.random() * 100 > VECS) {
                        OIQ[k] = 3;
                    } else {
                        OIQ[k] = 0;
                    }
                } else if (IDUQ.charAt(k) == '3') { // g
                    if (Math.random() * 100 > VECS) {
                        OIQ[k] = 3;
                    } else {
                        OIQ[k] = 0;
                    }
                }
            }
        }
    }

    ps[i][j] = InitonsPDS.DO_ACP_IDV(OIQ, 4);
    }
}

return ps;
}

public String injectPDI(String pdi) {
    return pdi;
}

```

```

}

public String PDSEncode(String VSQ) {
    while (VSQ.length() > 0) {
        // VSQ.concat(VSQ.replace("", ""))> 0;
    }
    return VSQ;
}

public String PDS_OEC_IID(String VSQ_IIE, int OCI_PPE) {
    //
    String $_CID = "AOPMVECS";
    // VSQ_IIE= $_CID.charAt(VSQ_IIE.length());
    return VSQ_IIE;
}

public String PDS_P_USQ_ECP(int P_VSQ, int MSP) {
    String ISQ = "";
    while (P_VSQ > 0) {
        ISQ += P_VSQ / MSP;
        P_VSQ %= MSP;
    }
    ISQ += P_VSQ;
    return ISQ;
}

// 现在PDS函数太大了, 不想new, 之后会用DecadeToPDS函数
public static String PDS_P_USQ_ECP_I_17(int P_VSQ, int MSP, App NE) {
    String ISQ = "";
    while (P_VSQ >= MSP) {
        ISQ += NE.app_S_S_MapsInitons.initonsCode
        .get(NE.app_S_S_MapsInitons.initonsSet.get("" + P_VSQ / MSP)); // P_VSQ;
        P_VSQ %= MSP;
    }
    ISQ += NE.app_S_S_MapsInitons.initonsCode
    .get(NE.app_S_S_MapsInitons.initonsSet.get("" + P_VSQ)); // P_VSQ;
    return ISQ;
}

public String PDS_P_USQ_ECP_I_17_S_(int P_VSQ, int MSP, App NE) {
    String ISQ = "";
    while (P_VSQ >= MSP) {
        ISQ += NE.app_S_S_MapsInitons.initonsCode
        .get(NE.app_S_S_MapsInitons.initonsSet.get("" + P_VSQ / MSP)); // P_VSQ;
        P_VSQ %= MSP;
    }
    ISQ += NE.app_S_S_MapsInitons.initonsCode
    .get(NE.app_S_S_MapsInitons.initonsSet.get("" + P_VSQ)); // P_VSQ;
    return ISQ;
}

public String ESU_M_SVQ_PDS_OEU(String SQA) {
    String[] PDS = new String[] { "AOPM", "VECS", "DIUQ", "HTX" };

```

```

        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }

    public String ESU_ECS_SVQ_PDS_OEU(String SQA) {
        String[] PDS = new String[] { "AOPM", "VECS" };
        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }

    public String ESU_P_SVQ_PDS_OEU(String SQA) {
        String[] PDS = new String[] { "AOPM", "VECS", "DIUQ", "HTX" };
        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }

    public String ESU_P_SEQ_PDS_OEU(String SQA) {
        String[] PDS = new String[] { "AOPM", "VECS", "DIUQ", "HTX" };
        String ISQ_PSD = "";
        int i = 0;
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
            SQA += PDSEncode(PDS[i]);
        }
        return ISQ_PSD;
    }

    public String EUP_QD_PQI(String AEP, int QSV) {
        String[][] CED = new String[][] { { "A", "O", "P", "M" },
        { "V", "E", "C", "S" }, { "D", "I", "U", "Q" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String EUP_QD_PQU(String AEP, int QSU) {
        String[][] CED = new String[][] { { "D", "I", "U", "Q" },
        { "H", "T", "X" } };
        String PM_ISQ = "";
        return AEP;
    }

    public String ESU_M_SVQ_PDS_OEU_M(String SQA) {
        String[] PDS = new String[] { "AOPM" };
        String ISQ_PSD = "";
    }

```

```

        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }

    public String ESU_M_SVQ_PDS_OEU_P(String SQA) {
        String[] PDS = new String[] { "VECS" };
        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }

    public String ESU_M_SVQ_PDS_OEU_O(String SQA) {
        String[] PDS = new String[] { "DIUQ" };
        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }

    public String ESU_M_SVQ_PDS_OEU_A(String SQA) {
        String[] PDS = new String[] { "HTX" };
        String ISQ_PSD = "";
        while (injectPDI(ISQ_PSD).length() > 0) {
            ISQ_PSD += PDSEncode(SQA);
        }
        return ISQ_PSD;
    }

    public String ESU_M_SVQ_PDS_OEU_OA(String SQA) {
        // AEC.VSQ.IC.IE
        return SQA;
    }
}

//436
//    IMV_SQI initonsSet = new IMV_SQI();
//    initonsSet.put("0", "0");//later remove out
//    initonsSet.put("1", "1");
//    initonsSet.put("2", "2");
//    initonsSet.put("3", "3");
//    initonsSet.put("4", "4");
//    initonsSet.put("5", "5");
//    initonsSet.put("6", "6");
//    initonsSet.put("7", "7");
//    initonsSet.put("8", "8");
//    initonsSet.put("9", "9");
//    initonsSet.put("10", "A");

```



```
//  initonsSet.put("11", "B");
//  initonsSet.put("12", "C");
//  initonsSet.put("13", "D");
//  initonsSet.put("14", "E");
//  initonsSet.put("15", "F");
//
//  IMV_SQI initonsCode = new IMV_SQI();
//  initonsCode.put("0", "D");
//  initonsCode.put("1", "C");
//  initonsCode.put("2", "P");
//  initonsCode.put("3", "E");
//  initonsCode.put("4", "T");
//  initonsCode.put("5", "H");
//  initonsCode.put("6", "O");
//  initonsCode.put("7", "S");
//  initonsCode.put("8", "M");
//  initonsCode.put("9", "A");
//  initonsCode.put("A", "X");
//  initonsCode.put("B", "F");
//  initonsCode.put("C", "V");
//  initonsCode.put("D", "I");
//  initonsCode.put("E", "U");
//  initonsCode.put("F", "Q");
//
//  IMV_SQI initonsSet = new IMV_SQI();
//  initonsSet.put("0", "0");
//  initonsSet.put("1", "1");
//  initonsSet.put("2", "2");
//  initonsSet.put("3", "3");
//  initonsSet.put("4", "4");
//  initonsSet.put("5", "5");
//  initonsSet.put("6", "6");
//  initonsSet.put("7", "7");
//  initonsSet.put("8", "8");
//  initonsSet.put("9", "9");
//  initonsSet.put("10", "A");
//  initonsSet.put("11", "B");
//  initonsSet.put("12", "C");
//  initonsSet.put("13", "D");
//  initonsSet.put("14", "E");
//  initonsSet.put("15", "F");
//  initonsSet.put("16", "G");
//  IMV_SQI initonsCode = new IMV_SQI();
//  initonsCode.put("0", "D");
//  initonsCode.put("1", "C");
//  initonsCode.put("2", "P");
//  initonsCode.put("3", "E");
```

```
//      initonsCode.put("4", "H");
//      initonsCode.put("5", "-");
//      initonsCode.put("6", "X");
//      initonsCode.put("7", "A");
//      initonsCode.put("8", "M");
//      initonsCode.put("9", "S");
//      initonsCode.put("A", "O");
//      initonsCode.put("B", "+");
//      initonsCode.put("C", "T");
//      initonsCode.put("D", "V");
//      initonsCode.put("E", "I");
//      initonsCode.put("F", "U");
//      initonsCode.put("G", "Q");
//346
//      //阿拉伯数字变元基数字
//      decadeToPDS.initonsSet.put("0", "0");
//      decadeToPDS.initonsSet.put("1", "1");
//      decadeToPDS.initonsSet.put("2", "2");
//      decadeToPDS.initonsSet.put("3", "3");
//      decadeToPDS.initonsSet.put("4", "4");
//      decadeToPDS.initonsSet.put("5", "5");
//      decadeToPDS.initonsSet.put("6", "6");
//      decadeToPDS.initonsSet.put("7", "7");
//      decadeToPDS.initonsSet.put("8", "8");
//      decadeToPDS.initonsSet.put("9", "9");
//      decadeToPDS.initonsSet.put("10", "A");
//      decadeToPDS.initonsSet.put("11", "B");
//      decadeToPDS.initonsSet.put("12", "C");
//      decadeToPDS.initonsSet.put("13", "D");
//      decadeToPDS.initonsSet.put("14", "E");
//      decadeToPDS.initonsSet.put("15", "F");
//      decadeToPDS.initonsSet.put("16", "G");
//
//      //元基数字变阿拉伯数字
//      decadeToPDS.numberSet.put("0", 0);
//      decadeToPDS.numberSet.put("1", 1);
//      decadeToPDS.numberSet.put("2", 2);
//      decadeToPDS.numberSet.put("3", 3);
//      decadeToPDS.numberSet.put("4", 4);
//      decadeToPDS.numberSet.put("5", 5);
//      decadeToPDS.numberSet.put("6", 6);
//      decadeToPDS.numberSet.put("7", 7);
//      decadeToPDS.numberSet.put("8", 8);
//      decadeToPDS.numberSet.put("9", 9);
//      decadeToPDS.numberSet.put("A", 10);
//      decadeToPDS.numberSet.put("B", 11);
//      decadeToPDS.numberSet.put("C", 12);
```

```
//      decadeToPDS.numberSet.put("D", 13);
//      decadeToPDS.numberSet.put("E", 14);
//      decadeToPDS.numberSet.put("F", 15);
//      decadeToPDS.numberSet.put("G", 16);

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pdi/TokenPDI.java
package A_I.MPE.MAQPVDISCOISIV.pdi;

import A_I.MPE.MAQPVDISCOISIV.pde.PDE_Decrement_Formular;
import A_I.MPE.MAQPVDISCOISIV.pde.PDE_Increment_Formular;
import A_I.MPE.MAQPVDISCOISIV.initon.Initon;
import A_I.MPE.MAQPVDISCOISIV.initon.InitonLinkDNA;
import S_A.SVQ.stable.S_Common;
import S_A.pheromone.IMV_SQI;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguangleo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//注意: 该 文件对应的是罗瑶光先生 DNA 编码 与 计算的两本 国家软著作 思想的编码 实现.
//公安部 与 知识产权委员会 已经备案, 可阅读 相关 著作权 原文 进行逻辑辨别.

public class TokenPDI {

    public double[] key = new double[4];
    public String bys = "";
    public String pdw = "";
    public String pds = "";
    public String pde = "";
    public String time = "";
    public String session_key = "";
    public String text;
    public String cacheId;
    public String code = "";
    public String lock = "";
    public String pdedeKey = "";
    public String pdedsKey = "";
    public String pdeieKey = "";
    public String pdeisKey = "";
    public void doKeyPress(String initons, TokenPDI pDE_RNA_Formular,
        boolean bYS) {
        Initon[] initon = new Initon[initons.length()];
        for (int i = 0; i < initons.length(); i++) {
            if (initon[i] == null) {
                initon[i] = new Initon();
            }
            initon[i].I_Initon("" + initons.charAt(i));
            if (i + 1 < initons.length()) {
                if (initon[i + 1] == null) {
```

```

        initon[i + 1] = new Initon();
    }
    initon[i].next = initon[i + 1];
    initon[i + 1].prev = initon[i];
    }
    }
    if (null != initons && initons.length() > 0) {
        do_PDE_RNA_Formular(initon[0], pDE_RNA_Formular, bYS);
    }
}

public static void main(String[] argv) {
    TokenPDI pDE_RNA_Formular = new TokenPDI();
    @SuppressWarnings("unused")
    String initonKeys = "EIU/0.6/EDU/0.4/si/0.3/sq/0.7/EIU/0.5/EDU/0.5/si/0.632/sq/0.368";
    pDE_RNA_Formular.key[0] = 0.6;
    pDE_RNA_Formular.key[1] = 0.3;
    pDE_RNA_Formular.key[2] = 0.5;
    pDE_RNA_Formular.key[3] = 0.632;
    pDE_RNA_Formular.text = "控制吸收";
    pDE_RNA_Formular.pdw = pDE_RNA_Formular
.initonSect(pDE_RNA_Formular.text);
    // 20230106-S_logger.Log.logger.info(" " + "原文: " +
    // pDE_RNA_Formular.text);
    // pDE_RNA_Formular.pdw= "字典保密: MSIOCUOCIPCUPCI";
    String[] lock = new String[12];
    lock[0] = "A";
    lock[3] = "O";
    lock[6] = "P";
    lock[9] = "M";
    lock[1] = "V";
    lock[4] = "E";
    lock[7] = "C";
    lock[10] = "S";
    lock[2] = "I";
    lock[5] = "D";
    lock[8] = "U";
    lock[11] = "Q";
    int i = (int) (Math.random() * 12) % 12;
    pDE_RNA_Formular.lock += lock[i];
    i = (int) (Math.random() * 12) % 12;
    pDE_RNA_Formular.lock += lock[i];
    i = (int) (Math.random() * 12) % 12;
    pDE_RNA_Formular.lock += lock[i];
    i = (int) (Math.random() * 12) % 12;
    pDE_RNA_Formular.lock += lock[i];
    for (i = 0; i < pDE_RNA_Formular.pdw.length(); i++) {
        pDE_RNA_Formular.code += pDE_RNA_Formular.lock

```

```
+ pDE_RNA_Formular.pdw.charAt(i);
}

// 20230106-S_logger.Log.logger.info("" + "肽语:" + pDE_RNA_Formular.pdw);
// 20230106-S_logger.Log.logger.info("" + "肽锁:" +
// pDE_RNA_Formular.lock);
// 20230106-S_logger.Log.logger.info("" + "散列肽语:" +
// pDE_RNA_Formular.code);
pDE_RNA_Formular.bys = "0.6/0.3/0.5/0.632";
// 20230106-S_logger.Log.logger.info("" + "静态密钥: " +
// pDE_RNA_Formular.bys);
pDE_RNA_Formular.doKeyPress(pDE_RNA_Formular.code, pDE_RNA_Formular,
false);

// 20230106-S_logger.Log.logger.info("" + "静态肽展降元概率钥匙E: " +
// pDE_RNA_Formular.pdedeKey);
// 20230106-S_logger.Log.logger.info("" + "静态肽展降元概率钥匙S: " +
// pDE_RNA_Formular.pdedeKey);
// 20230106-S_logger.Log.logger.info("" + "静态肽展降元: " +
// pDE_RNA_Formular.pds);
// 20230106-S_logger.Log.logger.info("" + "静态肽展增元概率钥匙E: " +
// pDE_RNA_Formular.pdeieKey);
// 20230106-S_logger.Log.logger.info("" + "静态肽展增元概率钥匙S: " +
// pDE_RNA_Formular.pdeieKey);
// 20230106-S_logger.Log.logger.info("" + "静态肽展增元: " +
// pDE_RNA_Formular.pde);
pDE_RNA_Formular.time = "" + System.currentTimeMillis();
pDE_RNA_Formular.cacheId = "ID" + Math.random()
+ S_Common.STRING_SYMBOL_PER + Math.random();

// 20230106-S_logger.Log.logger.info("" + "时间: " +
// pDE_RNA_Formular.time);
// 20230106-S_logger.Log.logger.info("" + "账号随机缓存字符串: " +
// pDE_RNA_Formular.cacheId);
pDE_RNA_Formular.session_key = pDE_RNA_Formular.pde;
// 20230106-S_logger.Log.logger.info("" + "Session: " +
// pDE_RNA_Formular.session_key);
// 20230106-S_logger.Log.logger.info("" +
"=====");

// 20230106-S_logger.Log.logger.info("" + "开始前序验证: ");
// 20230106-S_logger.Log.logger.info("" + "开始Session解析: " +
// pDE_RNA_Formular.session_key);
// 20230106-S_logger.Log.logger.info("" + "开始概率钥匙解析: " +
// pDE_RNA_Formular.pdedeKey
// + pDE_RNA_Formular.pdedeKey+ pDE_RNA_Formular.pdeieKey
// + pDE_RNA_Formular.pdeieKey);
TokenPDI pDE_RNA_Formular1 = new TokenPDI();
pDE_RNA_Formular1.pdedeKey = pDE_RNA_Formular.pdedeKey.toString();
pDE_RNA_Formular1.pdedeKey = pDE_RNA_Formular.pdedeKey.toString();
pDE_RNA_Formular1.pdeieKey = pDE_RNA_Formular.pdeieKey.toString();
pDE_RNA_Formular1.pdeieKey = pDE_RNA_Formular.pdeieKey.toString();
```

```

// 20230106-S_logger.Log.logger.info("" + "观测: " +
// pDE_RNA_Formular.code);
pDE_RNA_Formular.doKeyUnPress(pDE_RNA_Formular.code, pDE_RNA_Formular1,
true);

// 20230106-S_logger.Log.logger.info("" + "得到原降元元基DNA序列: "+
// pDE_RNA_Formular.pds);
// 20230106-S_logger.Log.logger.info("" + "得到新降元元基DNA序列: "+
// pDE_RNA_Formular1.pds);
// 20230106-S_logger.Log.logger.info("" + "得到原元基DNA序列: "+
// pDE_RNA_Formular.pde);
// 20230106-S_logger.Log.logger.info("" + "得到新元基DNA序列: "+
// pDE_RNA_Formular1.pde);
// 20230106-S_logger.Log.logger.info("" + "验证正确? ");
// 20230106-S_logger.Log.logger.info("" + pDE_RNA_Formular.pde.equals(pDE_RNA_Formular1.pde)?
// "正确": "失败");
"=====// 20230106-S_logger.Log.logger.info("" +
// 20230106-S_logger.Log.logger.info("" + "开始后序验证: ");
TokenPDI pDE_RNA_Formular2 = new TokenPDI();
pDE_RNA_Formular2.pdeicKey = pDE_RNA_Formular.pdedeKey.toString();
pDE_RNA_Formular2.pdeisKey = pDE_RNA_Formular.pdedeKey.toString();
pDE_RNA_Formular2.pdedeKey = pDE_RNA_Formular.pdeicKey.toString();
pDE_RNA_Formular2.pdedeKey = pDE_RNA_Formular.pdeisKey.toString();
// 20230106-S_logger.Log.logger.info("" + "准备计算元基DNA序列: "+
// pDE_RNA_Formular1.pde);
pDE_RNA_Formular2.doSessionKeyUnPress(pDE_RNA_Formular1.pde,
pDE_RNA_Formular2, true);

// 20230106-S_logger.Log.logger.info("" + "得到原续降元元基DNA序列: "+
// pDE_RNA_Formular1.pds);
// 20230106-S_logger.Log.logger.info("" + "得到后续降元元基DNA序列: "+
// pDE_RNA_Formular2.pds);
// 20230106-S_logger.Log.logger.info("" + "验证正确? ");
// 20230106-S_logger.Log.logger.info("" + pDE_RNA_Formular1.pds.equals(pDE_RNA_Formular2.pds)?
// "正确": "失败");
}

public void doSessionKeyUnPress(String initons, TokenPDI pDE_RNA_Formular2,
boolean bYS) {
    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {
        if (initon[i] == null) {
            initon[i] = new Initon();
        }
        initon[i].I_Initon("" + initons.charAt(i));
        if (i + 1 < initons.length()) {
            if (initon[i + 1] == null) {
                initon[i + 1] = new Initon();
            }
            initon[i].next = initon[i + 1];
        }
    }
}

```

```

        initon[i + 1].prev = initon[i];
    }
}

do_PDE_RNA_Formular_Back(initon[0], pDE_RNA_Formular2, bYS);
}

private void do_PDE_RNA_Formular_Back(Initon initon,
    TokenPDI pDE_RNA_Formular, boolean bYS) {
    Initon InitonPDE = initon;
    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
    Initon InitonPDEM = doDecrementM(InitonPDE, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEP = doDecrementP(InitonPDEM, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEO = doDecrementO(InitonPDEP, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEA = doDecrementA(InitonPDEO, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDES = doDecrementS(InitonPDEA, initonLinkDNA,
pDE_RNA_Formular, bYS);
    Initon InitonPDEC = doDecrementC(InitonPDES, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEE = doDecrementE(InitonPDEC, initonLinkDNA,
pDE_RNA_Formular, bYS);
    Initon InitonPDE1 = doDecrementV(InitonPDEE, initonLinkDNA,
pDE_RNA_Formular);
    while (InitonPDE1.hasNext()) {
        pDE_RNA_Formular.pds += InitonPDE1.getStore();
        InitonPDE1 = InitonPDE1.next;
    }
    pDE_RNA_Formular.pds += InitonPDE1.getStore();
    while (InitonPDE1.hasPrev()) {
        InitonPDE1 = InitonPDE1.prev;
    }
}

// 175-
// Initon InitonPDE1S= doIncrementS(InitonPDE1,
// initonLinkDNA, pDE_RNA_Formular, bYS);
// Initon InitonPDE1C= doIncrementC(InitonPDE1S,
// initonLinkDNA, pDE_RNA_Formular);
// Initon InitonPDE1E= doIncrementE(InitonPDE1C,
// initonLinkDNA, pDE_RNA_Formular, bYS);
// Initon InitonPDE1V= doIncrementV(InitonPDE1E,
// initonLinkDNA, pDE_RNA_Formular);
// Initon InitonPDE1M= doIncrementM(InitonPDE1V,
// initonLinkDNA, pDE_RNA_Formular);
// Initon InitonPDE1P= doIncrementP(InitonPDE1M,
// initonLinkDNA, pDE_RNA_Formular);

```

```

// Initon InitonPDE1O= doIncrementO(InitonPDE1P,
// initonLinkDNA, pDE_RNA_Formular);
// Initon InitonPDE2= doIncrementA(InitonPDE1O,
// initonLinkDNA, pDE_RNA_Formular);
// while(InitonPDE2.hasNext()) {
// pDE_RNA_Formular.pde+= InitonPDE2.getStore();
// InitonPDE2= InitonPDE2.next;
// }
// pDE_RNA_Formular.pde+= InitonPDE2.getStore();
// while(InitonPDE2.hasPrev()) {
// InitonPDE2= InitonPDE2.prev;
// }

public void doKeyUnPress(String initons, TokenPDI pDE_RNA_Formular,
                        boolean bYS) {
    Initon[] initon = new Initon[initons.length()];
    for (int i = 0; i < initons.length(); i++) {
        if (initon[i] == null) {
            initon[i] = new Initon();
        }
        initon[i].I_Initon("" + initons.charAt(i));
        if (i + 1 < initons.length()) {
            if (initon[i + 1] == null) {
                initon[i + 1] = new Initon();
            }
            initon[i].next = initon[i + 1];
            initon[i + 1].prev = initon[i];
        }
    }
    do_pDE_RNA_Formular(initon[0], pDE_RNA_Formular, bYS);
}

public void do_pDE_RNA_Formular(Initon initon, TokenPDI pDE_RNA_Formular,
                                boolean bYS) {
    Initon InitonPDE = initon;
    InitonLinkDNA initonLinkDNA = new InitonLinkDNA();
    Initon InitonPDEA = doDecrementA(InitonPDE, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEO = doDecrementO(InitonPDEA, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEP = doDecrementP(InitonPDEO, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEM = doDecrementM(InitonPDEP, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEV = doDecrementV(InitonPDEM, initonLinkDNA,
pDE_RNA_Formular);
    Initon InitonPDEE = doDecrementE(InitonPDEV, initonLinkDNA,
pDE_RNA_Formular, bYS);
    Initon InitonPDEC = doDecrementC(InitonPDEE, initonLinkDNA,

```



```

pDE_RNA_Formular);

    Initon InitonPDE1 = doDecrementS(InitonPDEC, initonLinkDNA,
pDE_RNA_Formular, bYS);

    while (InitonPDE1.hasNext()) {

        pDE_RNA_Formular.pds += InitonPDE1.getStore();

        InitonPDE1 = InitonPDE1.next;

    }

    pDE_RNA_Formular.pds += InitonPDE1.getStore();

    while (InitonPDE1.hasPrev()) {

        InitonPDE1 = InitonPDE1.prev;

    }

    Initon InitonPDE1V = doIncrementV(InitonPDE1, initonLinkDNA,
pDE_RNA_Formular);

    Initon InitonPDE1E = doIncrementE(InitonPDE1V, initonLinkDNA,
pDE_RNA_Formular, bYS);

    Initon InitonPDE1C = doIncrementC(InitonPDE1E, initonLinkDNA,
pDE_RNA_Formular);

    Initon InitonPDE1S = doIncrementS(InitonPDE1C, initonLinkDNA,
pDE_RNA_Formular, bYS);

    Initon InitonPDE1A = doIncrementA(InitonPDE1S, initonLinkDNA,
pDE_RNA_Formular);

    Initon InitonPDE1O = doIncrementO(InitonPDE1A, initonLinkDNA,
pDE_RNA_Formular);

    Initon InitonPDE1P = doIncrementP(InitonPDE1O, initonLinkDNA,
pDE_RNA_Formular);

    Initon InitonPDE2 = doIncrementM(InitonPDE1P, initonLinkDNA,
pDE_RNA_Formular);

    while (InitonPDE2.hasNext()) {

        pDE_RNA_Formular.pde += InitonPDE2.getStore();

        InitonPDE2 = InitonPDE2.next;

    }

    pDE_RNA_Formular.pde += InitonPDE2.getStore();

    while (InitonPDE2.hasPrev()) {

        InitonPDE2 = InitonPDE2.prev;

    }

}

///// INITONS SWAP

private Initon doIncrementA(Initon InitonPDE, InitonLinkDNA initonLinkDNA,

    TokenPDI pDE_RNA_Formular) {

    while (InitonPDE.hasPrev()) {

        InitonPDE = InitonPDE.prev;

    }

    initonLinkDNA.I_InitonLink(InitonPDE);

    InitonPDE = new PDE_Increment_Formular().PDE_IncrementA(initonLinkDNA);

    while (InitonPDE.hasPrev()) {

        InitonPDE = InitonPDE.prev;

    }

}

```

```

        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    private Initon doIncrementO(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        TokenPDI pDE_RNA_Formular) {
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementO(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    private Initon doIncrementP(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        TokenPDI pDE_RNA_Formular) {
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Increment_Formular().PDE_IncrementP(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

    private Initon doIncrementM(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
        TokenPDI pDE_RNA_Formular) {
        while (InitonPDE.hasPrev()) {

```

```

        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementM(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doIncrementV(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementV(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doIncrementC(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementC(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {

```

```

        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doIncrementE(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular()
.PDE_IncrementE_IU(initonLinkDNA, pDE_RNA_Formular, bYS);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doIncrementS(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Increment_Formular().PDE_IncrementS_I(initonLinkDNA,
pDE_RNA_Formular);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doDecrementA(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }

```

```

        initonLinkDNA.I_InitonLink(InitonPDE);
        InitonPDE = new PDE_Decrement_Formular().PDE_DecrementA(initonLinkDNA);
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        while (InitonPDE.hasNext()) {
            InitonPDE = InitonPDE.next;
        }
        while (InitonPDE.hasPrev()) {
            InitonPDE = InitonPDE.prev;
        }
        return InitonPDE;
    }

private Initon doDecrementO(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementO(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doDecrementP(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementP(initonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
}

```

```

        return InitonPDE;
    }

private Initon doDecrementM(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementM(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doDecrementV(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementV(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doDecrementC(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementC(intonLinkDNA);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }

```

```

    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doDecrementE(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular()
.PDE_DecrementE_IU(initonLinkDNA, pDE_RNA_Formular, bYS);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

private Initon doDecrementS(Initon InitonPDE, InitonLinkDNA initonLinkDNA,
    TokenPDI pDE_RNA_Formular, boolean bYS) {
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    initonLinkDNA.I_InitonLink(InitonPDE);
    InitonPDE = new PDE_Decrement_Formular().PDE_DecrementS_I(initonLinkDNA,
pDE_RNA_Formular, bYS);
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    while (InitonPDE.hasNext()) {
        InitonPDE = InitonPDE.next;
    }
    /// 20230106-S_logger.Log.logger.info("" + "");
    while (InitonPDE.hasPrev()) {
        InitonPDE = InitonPDE.prev;
    }
    return InitonPDE;
}

```

```
}

@SuppressWarnings("unchecked")
public String initonSect(String key) {
    String pdis = "";
    IMV_SQI map = new IMV_SQI();
    map.put("0", "A");
    map.put("1", "O");
    map.put("2", "P");
    map.put("3", "M");
    map.put("4", "V");
    map.put("5", "E");
    map.put("6", "C");
    map.put("7", "S");
    map.put("8", "I");
    map.put("9", "D");
    map.put(".", "U");
    for (int i = 0; i < key.length(); i++) {
        int c = key.charAt(i);
        String ints = S_Common.STRING_EMPTY + c;
        for (int j = 0; j < ints.length(); j++) {
            pdis += map.get("" + ints.charAt(j));
        }
        pdis += "U";
    }
    return pdis;
}

@SuppressWarnings("unchecked")
public String initonSectByOthoDNA(String key) {
    String pdis = "";
    IMV_SQI map = new IMV_SQI();
    map.put("0", "D");
    map.put("1", "I");
    map.put("2", "U");
    map.put("3", "Q");
    map.put("4", "A");
    map.put("5", "O");
    map.put("6", "P");
    map.put("7", "M");
    map.put("8", "V");
    map.put("9", "C");
    map.put(".", "X");
    // ADD otho
    for (int i = 0; i < key.length(); i++) {
        int c = key.charAt(i);
        String ints = S_Common.STRING_EMPTY + c;
        for (int j = 0; j < ints.length(); j++) {
            pdis += map.get("" + ints.charAt(j));
        }
    }
}
```



```
}
    pdis += "X";
}
return pdis;
}
}

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pdi/TokenPDITest.java

A_I.MPE.MAQPVDISCOISIV.pdw

/Trif_DNA-master/C_L_A/A_I/MPE/MAQPVDISCOISIV/pdw/PDW.txt

A_I.MPE.MSU.euler

/Trif_DNA-master/C_L_A/A_I/MPE/MSU/euler/Q_OulerRing.java
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package A_I.MPE.MSU.euler;
import S_A.SVQ.stable.S_Common;
import S_A.pheromone.IMV_SQI;
import java.util.Map;

public class Q_OulerRing{
    //这段函数用于观测元基映射的欧拉回路模型
    //思想 罗瑶光
    //算法 欧拉
    //程序员 罗瑶光
    //QUIVT+OSMAX-HEPCD 9 结果输出 有很多,我先任意选一种,就这个了 参考第48行。

//下面是所有的输出结果,没时间耗在这上面。
//      HEPCD3
//      2
//      1
//      UIVT+OSMAX-11
//      10
//      9
//      8
//      Q8
//      7
//      6
//      5
//      4
//      3
//      2
//      1
//      0
```

```
//      AMSO+HEPCD8
//      7
//      6
//      UIVT9
//      8
//      7
//      Q7
//      6
//      5
//      -X6
//      5
//      4
//      3
//      2
//      1
//      0
//
// , , , , , , , , , , , , , , , , , , --» » » » »      OSMAX-HEPCD9
//      8
//      7
//      UIVT+11
//      10
//      9
//      8
//      Q8
//      7
//      6
//      5
//      4
//      3
//      2
//      1
//      0
//
//      PEHCD3
//      2
//      +OSMAX-8
//      7
//      6
//      5
//      QUIVT9
//      8
//      7
//      6
//      5
//      4
```

```
//      3
//      2
//      1
//      0
//
//      MAX-HEPCD7
//      6
//      5
//      UIVT+OSQ12
//      11
//      10
//      9
//      8
//      7
//      6
//      5
//      4
//      3
//      2
//      1
//      0
//
//      VIUEHCP5
//      D5
//      4
//      +OSMAX-10
//      9
//      8
//      7
//      Q7
//      6
//      5
//      T5
//      4
//      3
//      2
//      1
//      0
//
//      EHCP2
//      D2
//      1
//      +OSMAX-7
//      6
//      5
//      4
//      QUIVT8
```

```
//      7
//      6
//      5
//      4
//      3
//      2
//      1
//      0
//
//      CHEP2
//      UIVT+OSMAX-12
//      11
//      10
//      9
//      Q9
//      8
//      7
//      6
//      5
//      4
//      3
//      2
//      1
//      0
//      D0
//
//      SO+HEPCD6
//      5
//      4
//      UIVT7
//      6
//      5
//      Q5
//      4
//      3
//      -MAX6
//      5
//      4
//      3
//      2
//      1
//      0
//
//      IVTEHCP5
//      D5
//      4
//      +OSMAX-10
```

// 9
// 8
// 7
// QU8
// 7
// 6
// 5
// 4
// 3
// 2
// 1
// 0
//
// DCHEP3
// UIVT+OSMAX-13
// 12
// 11
// 10
// Q10
// 9
// 8
// 7
// 6
// 5
// 4
// 3
// 2
// 1
// 0
//
// UEHCP3
// D3
// 2
// +OSMAX-8
// 7
// 6
// 5
// Q5
// 4
// 3
// TVI5
// 4
// 3
// 2
// 1
// 0
//

// QSO+HEPCD7

// 6

// 5

// UIVT8

// 7

// 6

// 5

// 4

// -MAX7

// 6

// 5

// 4

// 3

// 2

// 1

// 0

//

// TVIUEHCP6

// D6

// 5

// +OSMAX-11

// 10

// 9

// 8

// Q8

// 7

// 6

// 5

// 4

// 3

// 2

// 1

// 0

//

// XAMSO+HEPCD9

// 8

// 7

// UIVT10

// 9

// 8

// Q8

// 7

// 6

// -6

// 5

// 4

// 3

```
//      2
//      1
//      0
//
//      +HEPCD4
//      3
//      2
//      UIVT5
//      4
//      3
//      QSO5
//      MAX-8
//      7
//      6
//      5
//      4
//      3
//      2
//      1
//      0
//
//      -HEPCD4
//      3
//      2
//      UIVT+OSMAX11
//      10
//      9
//      Q9
//      8
//      7
//      6
//      5
//      4
//      3
//      2
//      1
//      0
//
//      0

@SuppressWarnings("unchecked")
public static void main(String[] args) {
    //init AOPM VECS IDUQ TXH DD
    //初始环路
    Map<String, Boolean> initonsLink= new IMV_SQI();
    //环路探索
    Map<String, Boolean> didInitonsLink= new IMV_SQI();
```

```
initonsLink.put("DC", true);
initonsLink.put("CD", true);
initonsLink.put("IV", true);
initonsLink.put("VT", true);
initonsLink.put("IU", true);
initonsLink.put("UI", true);
initonsLink.put("UE", true);
initonsLink.put("EU", true);
initonsLink.put("UQ", true);
initonsLink.put("QU", true);
initonsLink.put("QS", true);
initonsLink.put("SQ", true);

initonsLink.put("VT", true);
initonsLink.put("TV", true);
initonsLink.put("ET", true);
initonsLink.put("TE", true);
initonsLink.put("EH", true);
initonsLink.put("HE", true);
initonsLink.put("EP", true);
initonsLink.put("PE", true);
initonsLink.put("H+", true); //HE + HC -
initonsLink.put("+H", true);
initonsLink.put("H-", true);
initonsLink.put("-H", true);
initonsLink.put("HC", true);
initonsLink.put("CH", true);
initonsLink.put("CP", true);
initonsLink.put("PC", true);
initonsLink.put("SM", true);
initonsLink.put("MS", true);
initonsLink.put("SO", true);
initonsLink.put("OS", true);
initonsLink.put("XA", true);
initonsLink.put("AX", true);
initonsLink.put("MA", true);
initonsLink.put("AM", true);
initonsLink.put("X-", true);
initonsLink.put("-X", true);
initonsLink.put("M-", true);
initonsLink.put("-M", true);
initonsLink.put("T+", true);
initonsLink.put("+T", true);
initonsLink.put("O+", true);
initonsLink.put("+O", true);
```

```
String[] initons= new String[]{"H", "A", "O", "P", "M", "V", "E", "C", "S"
```



```
, "I", "D", "U", "Q", "T", "X", "+", "-");  
    int[] initonsCount= new int[17];  
    //for loop  
    //开始计算 路径总数  
    //String didInitons= "";  
  
int count= 0;  
  
    for(int i= 0; i< initons.length; i++) {  
        ///20230106-S_logger.Log.logger.info("" + temp);  
        //20230106-System.out.print(initons[i]);  
        initonsCount[i]++;  
        recur(initons[i], initonsLink, didInitonsLink, initons, initonsCount  
            , count, i);  
        ///20230106-S_logger.Log.logger.info("" + count);  
        //下一个  
        count= 0;  
        //20230106-S_logger.Log.logger.info("" + "");  
        didInitonsLink.clear();  
        initonsCount= new int[17];  
    }  
    // print loop initons  
    //打印可能模式  
    //20230106-S_logger.Log.logger.info("" + count);  
}  
  
//递归 继续修改。等会加 隔开观测。  
public static void recur(String firstChar, Map<String, Boolean> initonsLink  
    , Map<String, Boolean> didInitonsLink, String[] initons  
    , int[] initonsCount, int count, int i) {  
    for(int j= 0; j< initons.length; j++) {  
        if(!firstChar.equals(initons[j])) {  
String temp= S_Common.STRING_EMPTY+ firstChar+ initons[j];  
        //有路径                                     //没有遍历 //遍历了两次  
if(initonsLink.containsKey(temp)&& !didInitonsLink.containsKey(temp)  
            && initonsCount[j]< 1) {  
            initonsCount[j]++;  
            didInitonsLink.put(""+ firstChar+ initons[j], true);  
            //20230106-System.out.print(initons[j]);  
            recur(initons[j], initonsLink, didInitonsLink, initons  
                , initonsCount, count+ 1, j);  
            //20230106-S_logger.Log.logger.info("" + count);  
        }  
    }  
}  
  
}
```

* 个人著作权人，作者 罗瑶光，浏阳

* yaoguangleuo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

package A_IMPE.MSU.euler;

import S_A.SVQ.stable.S_Common;

import S_A.pheromone.IMV_SQI;

import java.util.Map;

public class Q_OulerRing16{

 //这段函数用于观测元基映射的欧拉回路模型

 //思想 罗瑶光

 //算法 欧拉

 //程序员 罗瑶光

 //基于32行组 求接出 16进制元基为 DCPE THOS MAXF VIUQ

/*

* HOSMAXFVIUEPCD12

11

10

T10

9

Q9

8

7

6

5

4

3

2

1

0

//AMSOHEPCD7 //基于该组数据求解 罗瑶光 20211129

//6

//5

//UIVT8

//FX9

//8

//7

//6

//Q6

//5

//4

//3

//2

//1

//0

OHEPCD4

3

2

UIVT5

FMAX8

7

SQ8

7

6

5

4

3

2

1

0

PEHOSMAXFVIUQ11

10

9

T9

8

7

6

5

4

3

2

CD3

2

1

0

MAXFHOSQUEPCD11

10

9

TVI11

10

9

8

7

6

5

4

3

2

1

0

VIUEHOSMAXF9

8

7	
6	
Q6	
5	
4	
CP5	
D5	
4	
T4	
3	
2	
1	
0	
EHOSMAXFVIUQ10	
9	
8	
T8	
7	
6	
5	
4	
3	
2	
1	
CP2	
D2	
1	
0	
CHOSMAXFVIUEP11	
T11	
10	
Q10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	
D0	
SOHEPCD5	
4	
3	
UIVT6	

FMAX9

- 8
- 7
- 6
- 5
- 4
- Q4
- 3
- 2
- 1
- 0

IVTHOSMAXF8

- 7
- 6
- 5

QUEPCD10

- 9
- 8
- 7
- 6
- 5
- 4
- 3
- 2
- 1
- 0

DCHOSMAXFVIUEP12

T12

- 11
- Q11
- 10
- 9
- 8
- 7
- 6
- 5
- 4
- 3
- 2
- 1
- 0

UEHOSMAXFVI9

T9

- 8
- 7
- 6
- 5

4	
Q4	
3	
2	
CP3	
D3	
2	
1	
0	
QSOHEPCD6	
5	
4	
UIVT7	
FMAX10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	
THOSMAXFVIUEPCD13	
12	
11	
10	
Q10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	
XAMSOHEPCD8	
7	
6	
UIVT9	
F9	
8	
7	
Q7	

```
6
5
4
3
2
1
0
+
FHOSMAX5
4
3
QUEPCD8
7
6
TVI8
7
6
5
4
3
2
1
0
0
**/

@SuppressWarnings("unchecked")
public static void main(String[] args) {
    //init AOPM VECS IDUQ TXH DD
    //初始环路
    Map<String, Boolean> initonsLink= new IMV_SQI();
    //环路探索
    Map<String, Boolean> didInitonsLink= new IMV_SQI();
    initonsLink.put("DC", true);
    initonsLink.put("CD", true);
    initonsLink.put("IV", true);
    initonsLink.put("VI", true);
    initonsLink.put("IU", true);
    initonsLink.put("UI", true);
    initonsLink.put("UE", true);
    initonsLink.put("EU", true);
    initonsLink.put("UQ", true);
    initonsLink.put("QU", true);
    initonsLink.put("QS", true);
    initonsLink.put("SQ", true);

    initonsLink.put("VT", true);
```

```
        initonsLink.put("TV", true);
        initonsLink.put("ET", true);
        initonsLink.put("TE", true);
        initonsLink.put("EH", true);
        initonsLink.put("HE", true);
        initonsLink.put("EP", true);
        initonsLink.put("PE", true);
//initonsLink.put("H+", true); //HE + HC -
//initonsLink.put("+H", true);
//initonsLink.put("H-", true);
//initonsLink.put("-H", true);
        initonsLink.put("HC", true);
        initonsLink.put("CH", true);
        initonsLink.put("CP", true);
        initonsLink.put("PC", true);
        initonsLink.put("SM", true);
        initonsLink.put("MS", true);
        initonsLink.put("SO", true);
        initonsLink.put("OS", true);

        initonsLink.put("HF", true);
        initonsLink.put("FH", true);
        initonsLink.put("VF", true);
        initonsLink.put("FV", true);

        initonsLink.put("XA", true);
        initonsLink.put("AX", true);
        initonsLink.put("MA", true);
        initonsLink.put("AM", true);
//initonsLink.put("X-", true);
//initonsLink.put("-X", true);
//initonsLink.put("M-", true);
//initonsLink.put("-M", true);

        initonsLink.put("XF", true);
        initonsLink.put("FX", true);
        initonsLink.put("MF", true);
        initonsLink.put("FM", true);

//initonsLink.put("T+", true);
//initonsLink.put("+T", true);
//initonsLink.put("O+", true);
//initonsLink.put("+O", true);

        initonsLink.put("TH", true);
        initonsLink.put("HT", true);
        initonsLink.put("OH", true);
```

```

        initonsLink.put("HO", true);

        String[] initons= new String[] {"H", "A", "O", "P", "M", "V", "E", "C", "S"
, "I", "D", "U", "Q", "T", "X", "+", "F"};
        int[] initonsCount= new int[17];
        //for loop
        //开始计算 路径总数
        //String didInitons= "";

int count= 0;
        for(int i= 0; i< initons.length; i++) {
            ///20230106-S_logger.Log.logger.info("" + temp);
            //20230106-System.out.print(initons[i]);

            initonsCount[i]++;
            recur(initons[i], initonsLink, didInitonsLink, initons, initonsCount
, count, i);
            ///20230106-S_logger.Log.logger.info("" + count);
            //下一个
            count= 0;
            //20230106-S_logger.Log.logger.info("" + "");
            didInitonsLink.clear();
            initonsCount= new int[17];
        }
        // print loop initons
        //打印可能模式
        //20230106-S_logger.Log.logger.info("" + count);
    }

    //递归 继续修改。等会加 隔开观测。
    public static void recur(String firstChar, Map<String, Boolean> initonsLink
, Map<String, Boolean> didInitonsLink, String[] initons
, int[] initonsCount, int count, int i) {
        for(int j= 0; j< initons.length; j++) {
            if(!firstChar.equals(initons[j])) {
String temp= S_Common.STRING_EMPTY+ firstChar+ initons[j];
//有路径                                //没有遍历 //遍历了两次
            if(initonsLink.containsKey(temp)&& !didInitonsLink.containsKey(temp)
&& initonsCount[j]< 1) {
                initonsCount[j]++;
                didInitonsLink.put(""+ firstChar+ initons[j], true);
                //20230106-System.out.print(initons[j]);
                recur(initons[j], initonsLink, didInitonsLink, initons
, initonsCount, count+ 1, j);
                //20230106-S_logger.Log.logger.info("" + count);
            }
        }
    }
}
}
}
}

```

```
//Trif_DNA-master/C_L_A/A_I/MPE/MSU/ouler/Q_OulerRing16Test.java
//Trif_DNA-master/C_L_A/A_I/MPE/MSU/ouler/Q_OulerRingTest.java
```

A_I.MPE.MSU.pde

```
//Trif_DNA-master/C_L_A/A_I/MPE/MSU/pde/DecadeToPDS.java
/*
```

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
package A_I.MPE.MSU.pde;
```

```
import A_I.MPE.MSU.pds.PDE_PDS_DL;
```

```
import S_A.pheromone.IMV_SQI;
```

```
import java.util.Map;
```

```
//这个函数集用于将常数变换成十七进制元基数字,
```

```
//这个函数集用于将十七进制元基数字进行元基变换
```

```
//这个函数集用于将元基变换进行肽展概率丝化展开
```

```
//这个函数用于将肽展丝化的肽增十七进制进行元基变换
```

```
//这个函数用于将肽展丝化的肽增十七进制变换成元基数字
```

```
//这个函数用于十七进制元基数字进行十进制还原。
```

```
/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,

* 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道

* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class DecadeToPDS {
```

```
    // 思想：肽展公式 1.2.2, 元基数字逻辑； 十七进制元基组合数学； 概率论
```

```
    // 算法：进制计算, 肽展计算
```

```
    // 程序员： 罗瑶光,
```

```
    public IMV_SQI initonsMap = new IMV_SQI();
```

```
    public IMV_SQI initonsCode = new IMV_SQI();
```

```
    public IMV_SQI initonsSet = new IMV_SQI();
```

```
    @SuppressWarnings("unchecked")
```

```
    public Map<String, Integer> numberSet = new IMV_SQI();
```

```
    public static void main(String[] Args) {
```

```
        DecadeToPDS decadeToPDS = new DecadeToPDS();
```

```
        decadeToPDS.IV_(decadeToPDS);
```

```
        int decade = (int) (Math.random() * 1000 % 256); // 随便写一个数
```

```
        double pDE_KEY_rate = 0.25; // 随便模拟一个0-1之间的概率钥匙, 假设 0~0.5为酸
```

```
        // , 0.5~1 为碱;
```

```
        decadeToPDS.doPDS(decadeToPDS, decade, pDE_KEY_rate);
```

```
    }
```

```

private int doPDS(DecadeToPDS decadeToPDS, int decade,
    double pDE_KEY_rate) {
    String seventeen = decadeToPDS.decadeToSeventeen(decade, decadeToPDS);
    String initons = decadeToPDS.seventeenToIntons(seventeen, decadeToPDS);
    String pDS = decadeToPDS.initonsToPDS(initons, pDE_KEY_rate,
decadeToPDS);
    pDS = pDS.replace(".", "");
    String pDSInitons = decadeToPDS.PDSToInitons(pDS, pDE_KEY_rate,
decadeToPDS);
    String pDSSeventeen = decadeToPDS.initonsToSeventeen(pDSInitons,
decadeToPDS);
    int pDSDecade = decadeToPDS.seventeenToDecade(pDSSeventeen,
decadeToPDS);
    return pDSDecade;
}
@SuppressWarnings("unchecked")
public void IV_(DecadeToPDS decadeToPDS) {
    decadeToPDS.initonsMap.put("A", "7");
    decadeToPDS.initonsMap.put("O", "A");
    decadeToPDS.initonsMap.put("P", "2");
    decadeToPDS.initonsMap.put("M", "8");
    decadeToPDS.initonsMap.put("V", "D");
    decadeToPDS.initonsMap.put("E", "3");
    decadeToPDS.initonsMap.put("C", "1");
    decadeToPDS.initonsMap.put("S", "9");
    decadeToPDS.initonsMap.put("I", "E");
    decadeToPDS.initonsMap.put("D", "0");
    decadeToPDS.initonsMap.put("U", "F");
    decadeToPDS.initonsMap.put("Q", "G");
    decadeToPDS.initonsMap.put("T", "C");
    decadeToPDS.initonsMap.put("X", "6");
    decadeToPDS.initonsMap.put("+", "B");
    decadeToPDS.initonsMap.put("-", "5");
    decadeToPDS.initonsMap.put("H", "4");
    // 元基数字变元基符号
    decadeToPDS.initonsCode.put("0", "D");
    decadeToPDS.initonsCode.put("1", "C");
    decadeToPDS.initonsCode.put("2", "P");
    decadeToPDS.initonsCode.put("3", "E");
    decadeToPDS.initonsCode.put("4", "H");
    decadeToPDS.initonsCode.put("5", "-");
    decadeToPDS.initonsCode.put("6", "X");
    decadeToPDS.initonsCode.put("7", "A");
    decadeToPDS.initonsCode.put("8", "M");
    decadeToPDS.initonsCode.put("9", "S");
    decadeToPDS.initonsCode.put("A", "O");
    decadeToPDS.initonsCode.put("B", "+");

```

```

decadeToPDS.initonsCode.put("C", "T");
decadeToPDS.initonsCode.put("D", "V");
decadeToPDS.initonsCode.put("E", "I");
decadeToPDS.initonsCode.put("F", "U");
decadeToPDS.initonsCode.put("G", "Q");
// 阿拉伯数字变元基数字
decadeToPDS.initonsSet.put("0", "0");
decadeToPDS.initonsSet.put("1", "1");
decadeToPDS.initonsSet.put("2", "2");
decadeToPDS.initonsSet.put("3", "3");
decadeToPDS.initonsSet.put("4", "4");
decadeToPDS.initonsSet.put("5", "5");
decadeToPDS.initonsSet.put("6", "6");
decadeToPDS.initonsSet.put("7", "7");
decadeToPDS.initonsSet.put("8", "8");
decadeToPDS.initonsSet.put("9", "9");
decadeToPDS.initonsSet.put("10", "A");
decadeToPDS.initonsSet.put("11", "B");
decadeToPDS.initonsSet.put("12", "C");
decadeToPDS.initonsSet.put("13", "D");
decadeToPDS.initonsSet.put("14", "E");
decadeToPDS.initonsSet.put("15", "F");
decadeToPDS.initonsSet.put("16", "G");
// 元基数字变阿拉伯数字
decadeToPDS.numberSet.put("0", 0);
decadeToPDS.numberSet.put("1", 1);
decadeToPDS.numberSet.put("2", 2);
decadeToPDS.numberSet.put("3", 3);
decadeToPDS.numberSet.put("4", 4);
decadeToPDS.numberSet.put("5", 5);
decadeToPDS.numberSet.put("6", 6);
decadeToPDS.numberSet.put("7", 7);
decadeToPDS.numberSet.put("8", 8);
decadeToPDS.numberSet.put("9", 9);
decadeToPDS.numberSet.put("A", 10);
decadeToPDS.numberSet.put("B", 11);
decadeToPDS.numberSet.put("C", 12);
decadeToPDS.numberSet.put("D", 13);
decadeToPDS.numberSet.put("E", 14);
decadeToPDS.numberSet.put("F", 15);
decadeToPDS.numberSet.put("G", 16);
}
// ///准备集成第二卷的AOPM 级别 肽展公式 , 已经并入PDSToInitons 函数中
// private String PDSToPDE(String pds, double pDE_KEY_rate,
// DecadeToPDS
// decadeToPDS) {
//

```

```

//// pds= pds.replace("UQ", "V");
//// pds= pds.replace("DI", "C");
//// pds= pds.replace("IQ", "S");
//// pds= pds.replace("VS", "A");
//// pds= pds.replace("ES", "O");
//// pds= pds.replace("EC", "P");
//// pds= pds.replace("CS", "M");
//// pds= pds.replace("VE", "T");
//// pds= pds.replace("VC", "X");
////
// return pds;
// }
// 这个函数集用于将常数变换成十七进制元基数字,
public String decadeToSeventeen(int decade, DecadeToPDS decadeToPDS) {
    String seventeen = "";
    int decad = decade;
    while (0 < decad / 17) {
        int seventeenth = decad % 17;
        seventeen = decadeToPDS.initonsSet.get("" + seventeenth)
+ seventeen;
        decad = decad / 17;
    }
    seventeen = decadeToPDS.initonsSet.get("" + decad) + seventeen;
    //
    return seventeen;
}
// 这个函数集用于将十七进制元基数字进行元基变换
public String seventeenToIntons(String seventeen, DecadeToPDS decadeToPDS) {
    String initons = "";
    for (int i = 0; i < seventeen.length(); i++) {
        initons += decadeToPDS.initonsCode.get("" + seventeen.charAt(i));
    }
    //
    return initons;
}
// 这个函数集用于将元基变换进行肽展概率丝化展开
public String initonsToPDS(String initons, double pDE_KEY_rate,
    DecadeToPDS decadeToPDS) {
    String PDS = "";
    StringBuilder PDEKey = new StringBuilder("");
    for (int i = 0; i < initons.length(); i++) {
        PDS += new PDE_PDS_DL().initonPDSwithBYS("" + initons.charAt(i),
pDE_KEY_rate, PDEKey, true) + ".";
    }
    //// 20230106-S_logger.Log.logger.info("" + "生成钥匙: "+ PDEKey);
    //
    return PDS;
}

```

```

    }

    // 这个函数用于将肽展丝化的肽增十七进制进行元基变换
    public String PDSToInitons(String pDS, double pDE_KEY_rate,
        DecadeToPDS decadeToPDS) {
        String initons = "";
        // initons= new PDE_PDS_DL().initonPDIwithBYS(pDS, 0, new
        // StringBuilder()
        // , false);
        // initons= new PDE_PDS_DL().initonPDEwithBYS(pDS,
        // pDE_KEY_rate
        // , new StringBuilder(), true);
        initons = new PDE_PDS_DL().initonPDE_DCDLwithBYS(pDS, pDE_KEY_rate,
        new StringBuilder(), true);
        return initons;
    }

    // 这个函数用于将肽展丝化的肽增十七进制变换成元基数字
    public String initonsToSeventeen(String initons, DecadeToPDS decadeToPDS) {
        String seventeen = "";
        //
        for (int i = 0; i < initons.length(); i++) {
            seventeen += decadeToPDS.initonsMap.get("" + initons.charAt(i));
        }
        return seventeen;
    }

    // 这个函数用于十七进制元基数字进行十进制还原。
    public int seventeenToDecade(String seventeen, DecadeToPDS decadeToPDS) {
        int decade = 0;
        // A11 10*17*17 + 1*17 + 1
        for (int i = 0; i < seventeen.length(); i++) {
            int value = decadeToPDS.numberSet.get("" + seventeen.charAt(i));
            decade += value * Math.pow(17, seventeen.length() - 1 - i);
        }
        return decade;
    }

    // 这个函数用于十七进制元基数字进行十进制矩阵变换。
    public int[][] doPDSMatrix(DecadeToPDS decadeToPDS, int[][] rp,
        double facx) {
        for (int i = 0; i < rp.length; i++) {
            for (int j = 0; j < rp[0].length; j++) {
                rp[i][j] = decadeToPDS.doPDS(decadeToPDS, rp[i][j], facx);
            }
        }
        return rp;
    }
}

// 元基符号变元基数字
// 20230106-S_logger.Log.logger.info("" + "输入十进制数: " + decade);

```

```
//      String seventeen= decadeToPDS.decadeToSeventeen(decade, decadeToPDS);
//      ///20230106-S_logger.Log.logger.info("'" + "元基进制数为: " + seventeen);
//      String initons= decadeToPDS.seventeenToIntons(seventeen, decadeToPDS);
//      ///20230106-S_logger.Log.logger.info("'" + "变换为元基: "+initons);
//      //initons= "AOPMVE";
//      ///20230106-S_logger.Log.logger.info("'" + "输入元基: "+ initons);
//      ///20230106-S_logger.Log.logger.info("'" + "输入概率: "+ pDE_KEY_rate);
//      String pDS= decadeToPDS.initonsToPDS(initons, pDE_KEY_rate, decadeToPDS);
//      ///20230106-S_logger.Log.logger.info("'" + "输出肽丝:"+ pDS);
//      pDS= pDS.replace(".", "");
//      String pDSInitons= decadeToPDS.PDSToInitons(pDS, pDE_KEY_rate, decadeToPDS);
//      ///20230106-S_logger.Log.logger.info("'" + "肽丝增元:"+ pDSInitons);
//      //第二卷的肽展公式 可以用到了
//      //String pDEInitons= decadeToPDS.PDSToPDE(pDSInitons, pDE_KEY_rate, decadeToPDS);
//      ///20230106-S_logger.Log.logger.info("'" + "肽展增元:"+ pDEInitons);
//      String pDSSeventeen= decadeToPDS.initonsToSeventeen(pDSInitons, decadeToPDS);
//      ///20230106-S_logger.Log.logger.info("'" + "元基数字:"+ pDSSeventeen);
//      String pDSDecade= decadeToPDS.seventeenToDecade(pDSSeventeen, decadeToPDS);
//      ///20230106-S_logger.Log.logger.info("'" + "输出十进制数:"+ pDSDecade);
//      return Integer.valueOf(pDSDecade);
```

/*Trif_DNA-master/C_L_A/A_I/MPE/MSU/pde/DecadeToPDS16.java

* 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

package A_I.MPE.MSU.pde;

import A_I.MPE.MSU.pds.PDE_PDS_DL16;

import S_A.pheromone.IMV_SQL;

import java.util.Map;

```
//这个函数集用于将常数变换成十六进制元基数字,
//这个函数集用于将十六进制元基数字进行元基变换
//这个函数集用于将元基变换进行肽展概率丝化展开
//这个函数用于将肽展丝化的肽增十六进制进行元基变换
//这个函数用于将肽展丝化的肽增十六进制变换成元基数字
//这个函数用于十六进制元基数字进行十进制还原。
```

/*

* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

```

public class DecadeToPDS16 {

    // 思想：肽展公式 1.2.2, 元基数字逻辑； 十六进制元基组合数学； 概率论
    // 算法：进制计算, 肽展计算
    // 程序员： 罗瑶光,

    public IMV_SQI initonsMap = new IMV_SQI();
    public IMV_SQI initonsCode = new IMV_SQI();
    public IMV_SQI initonsSet = new IMV_SQI();
    @SuppressWarnings("unchecked")
    public Map<String, Integer> numberSet = new IMV_SQI();
    public static void main(String[] Args) {

        DecadeToPDS16 decadeToPDS = new DecadeToPDS16();
        decadeToPDS.IV_(decadeToPDS);
        int decade = (int) (Math.random() * 1000 % 256); // 随便写一个数
        double pDE_KEY_rate = 0.25; // 随便模拟一个0-1之间的概率钥匙, 假设 0~0.5为酸
        // , 0.5~1 为碱;
        decadeToPDS.doPDS(decadeToPDS, decade, pDE_KEY_rate);
        // 20230106-S_logger.Log.logger.info("" + 1);
    }

    // 元基符号变元基数字
    // 20230106-S_logger.Log.logger.info("" + "输入十进制数: " + decade);
    String seventeen = decadeToPDS.decadeToSeventeen(decade, decadeToPDS);
    // 20230106-S_logger.Log.logger.info("" + "元基进制数为: " + seventeen);
    String initons = decadeToPDS.seventeenToIntons(seventeen, decadeToPDS);
    // 20230106-S_logger.Log.logger.info("" + "变换为元基: " + initons);
    // initons = "AOPMVE";
    // 20230106-S_logger.Log.logger.info("" + "输入元基: " + initons);
    // 20230106-S_logger.Log.logger.info("" + "输入概率: " + pDE_KEY_rate);
    String pDS = decadeToPDS.initonsToPDS(initons, pDE_KEY_rate, decadeToPDS);
    // 20230106-S_logger.Log.logger.info("" + "输出肽丝:" + pDS);
    pDS = pDS.replace(".", "");
    String pDSInitons = decadeToPDS.PDSToInitons(pDS, pDE_KEY_rate, decadeToPDS);
    // 20230106-S_logger.Log.logger.info("" + "肽丝增元:" + pDSInitons);
    // 第二卷的肽展公式 可以用到了
    // String pDEInitons = decadeToPDS.PDSToPDE(pDSInitons, pDE_KEY_rate, decadeToPDS);
    // 20230106-S_logger.Log.logger.info("" + "肽展增元:" + pDEInitons);
    String pDSSeventeen = decadeToPDS.initonsToSeventeen(pDSInitons, decadeToPDS);
    // 20230106-S_logger.Log.logger.info("" + "元基数字:" + pDSSeventeen);
    String pDSDecade = decadeToPDS.seventeenToDecade(pDSSeventeen, decadeToPDS);
    // 20230106-S_logger.Log.logger.info("" + "输出十进制数:" + pDSDecade);
    return Integer.valueOf(pDSDecade);
    private int doPDS(DecadeToPDS16 decadeToPDS, int decade,
        double pDE_KEY_rate) {
        String sixteen = decadeToPDS.decadeToSixteen(decade, decadeToPDS);
        String initons = decadeToPDS.sixteenToIntons(sixteen, decadeToPDS);
        String pDS = decadeToPDS.initonsToPDS(initons, pDE_KEY_rate,
decadeToPDS);
        pDS = pDS.replace(".", "");
    }
}

```

```

String pDSInitons = decadeToPDS.PDSToInitons(pDS, pDE_KEY_rate,
decadeToPDS);

String pDSSixteen = decadeToPDS.initonsToSixteen(pDSInitons,
decadeToPDS);

int pDSDecade = decadeToPDS.sixteenToDecade(pDSSixteen, decadeToPDS);
return pDSDecade;
}

@SuppressWarnings("unchecked")
public void IV_(DecadeToPDS16 decadeToPDS) {
    decadeToPDS.initonsMap.put("A", "9");
    decadeToPDS.initonsMap.put("O", "6");
    decadeToPDS.initonsMap.put("P", "2");
    decadeToPDS.initonsMap.put("M", "8");
    decadeToPDS.initonsMap.put("V", "C");
    decadeToPDS.initonsMap.put("E", "3");
    decadeToPDS.initonsMap.put("C", "1");
    decadeToPDS.initonsMap.put("S", "7");
    decadeToPDS.initonsMap.put("I", "D");
    decadeToPDS.initonsMap.put("D", "0");
    decadeToPDS.initonsMap.put("U", "E");
    decadeToPDS.initonsMap.put("Q", "F");
    decadeToPDS.initonsMap.put("T", "4");
    decadeToPDS.initonsMap.put("X", "A");
    decadeToPDS.initonsMap.put("F", "B");
    decadeToPDS.initonsMap.put("H", "5");
    // 元基数字变元基符号
    decadeToPDS.initonsCode.put("0", "D");
    decadeToPDS.initonsCode.put("1", "C");
    decadeToPDS.initonsCode.put("2", "P");
    decadeToPDS.initonsCode.put("3", "E");
    decadeToPDS.initonsCode.put("4", "T");
    decadeToPDS.initonsCode.put("5", "H");
    decadeToPDS.initonsCode.put("6", "O");
    decadeToPDS.initonsCode.put("7", "S");
    decadeToPDS.initonsCode.put("8", "M");
    decadeToPDS.initonsCode.put("9", "A");
    decadeToPDS.initonsCode.put("A", "X");
    decadeToPDS.initonsCode.put("B", "F");
    decadeToPDS.initonsCode.put("C", "V");
    decadeToPDS.initonsCode.put("D", "I");
    decadeToPDS.initonsCode.put("E", "U");
    decadeToPDS.initonsCode.put("F", "Q");
    // 阿拉伯数字变元基数字
    decadeToPDS.initonsSet.put("0", "0");
    decadeToPDS.initonsSet.put("1", "1");
    decadeToPDS.initonsSet.put("2", "2");
    decadeToPDS.initonsSet.put("3", "3");

```

```

        decadeToPDS.initonsSet.put("4", "4");
        decadeToPDS.initonsSet.put("5", "5");
        decadeToPDS.initonsSet.put("6", "6");
        decadeToPDS.initonsSet.put("7", "7");
        decadeToPDS.initonsSet.put("8", "8");
        decadeToPDS.initonsSet.put("9", "9");
        decadeToPDS.initonsSet.put("10", "A");
        decadeToPDS.initonsSet.put("11", "B");
        decadeToPDS.initonsSet.put("12", "C");
        decadeToPDS.initonsSet.put("13", "D");
        decadeToPDS.initonsSet.put("14", "E");
        decadeToPDS.initonsSet.put("15", "F");
        // 元基数字变阿拉伯数字
        decadeToPDS.numberSet.put("0", 0);
        decadeToPDS.numberSet.put("1", 1);
        decadeToPDS.numberSet.put("2", 2);
        decadeToPDS.numberSet.put("3", 3);
        decadeToPDS.numberSet.put("4", 4);
        decadeToPDS.numberSet.put("5", 5);
        decadeToPDS.numberSet.put("6", 6);
        decadeToPDS.numberSet.put("7", 7);
        decadeToPDS.numberSet.put("8", 8);
        decadeToPDS.numberSet.put("9", 9);
        decadeToPDS.numberSet.put("A", 10);
        decadeToPDS.numberSet.put("B", 11);
        decadeToPDS.numberSet.put("C", 12);
        decadeToPDS.numberSet.put("D", 13);
        decadeToPDS.numberSet.put("E", 14);
        decadeToPDS.numberSet.put("F", 15);
    }

    ///准备集成第二卷的AOPM 级别 肽展公式, 已经并入PDSToInitons 函数中
    // private String PDSToPDE(String pds, double pDE_KEY_rate,
    // DecadeToPDS
    // decadeToPDS) {
    //
    //
    // /// pds= pds.replace("UQ", "V");
    // /// pds= pds.replace("DI", "C");
    // /// pds= pds.replace("IQ", "S");
    // /// pds= pds.replace("VS", "A");
    // /// pds= pds.replace("ES", "O");
    // /// pds= pds.replace("EC", "P");
    // /// pds= pds.replace("CS", "M");
    // /// pds= pds.replace("VE", "T");
    // /// pds= pds.replace("VC", "X");
    // ///
    // return pds;
    // }

```

// 这个函数集用于将常数变换成十六进制元基数字,

```
public String decadeToSixteen(int decade, DecadeToPDS16 decadeToPDS) {
    String sixteen = "";
    int decad = decade;
    while (0 < decad / 16) {
        int sixteenth = decad % 16;
        sixteen = decadeToPDS.initonsSet.get("" + sixteenth) + sixteen;
        decad = decad / 16;
    }
    sixteen = decadeToPDS.initonsSet.get("" + decad) + sixteen;
    //
    return sixteen;
}
```

// 这个函数集用于将十六进制元基数字进行元基变换

```
public String sixteenToIntons(String sixteen, DecadeToPDS16 decadeToPDS) {
    String initons = "";
    for (int i = 0; i < sixteen.length(); i++) {
        initons += decadeToPDS.initonsCode.get("" + sixteen.charAt(i));
    }
    //
    return initons;
}
```

// 这个函数集用于将元基变换进行肽展概率丝化展开

```
public String initonsToPDS(String initons, double pDE_KEY_rate,
    DecadeToPDS16 decadeToPDS) {
    String PDS = "";
    StringBuilder PDEKey = new StringBuilder("");
    for (int i = 0; i < initons.length(); i++) {
        PDS += new PDE_PDS_DL16().initonPDSwithBYS("" + initons.charAt(i),
        pDE_KEY_rate, PDEKey, true) + ".";
    }
    //// 20230106-S_logger.Log.logger.info("" + "生成钥匙: " + PDEKey);
    //
    return PDS;
}
```

// 这个函数用于将肽展丝化的肽增十六进制进行元基变换, 之后利用下 PDEKey 在肽erosion中的作用。

```
public String PDSToInitons(String pDS, double pDE_KEY_rate,
    DecadeToPDS16 decadeToPDS) {
    String initons = "";
    // initons= new PDE_PDS_DL().initonPDIwithBYS(pDS, 0, new
    // StringBuilder()
    // , false);
    // initons= new PDE_PDS_DL().initonPDEwithBYS(pDS,
    // pDE_KEY_rate
    // , new StringBuilder(), true);
    initons = new PDE_PDS_DL16().initonPDE_DCDLwithBYS(pDS, pDE_KEY_rate,
    new StringBuilder(), true);
}
```

```

        return initons;
    }

    // 这个函数用于将肽展丝化的肽增十六进制变换成元基数字
    public String initonsToSixteen(String initons, DecadeToPDS16 decadeToPDS) {
        String sixteen = "";
        //
        for (int i = 0; i < initons.length(); i++) {
            sixteen += decadeToPDS.initonsMap.get("" + initons.charAt(i));
        }
        return sixteen;
    }

    // 这个函数用于十六进制元基数字进行十进制还原。
    public int sixteenToDecade(String sixteen, DecadeToPDS16 decadeToPDS) {
        int decade = 0;
        // A11 10*16*16 + 1*16 + 1
        for (int i = 0; i < sixteen.length(); i++) {
            int value = decadeToPDS.numberSet.get("" + sixteen.charAt(i));
            decade += value * Math.pow(16, sixteen.length() - 1 - i);
        }
        return decade;
    }

    // 这个函数用于十六进制元基数字进行十进制矩阵变换。
    public int[][] doPDSMatrix(DecadeToPDS16 decadeToPDS, int[][] rp,
        double facx) {
        for (int i = 0; i < rp.length; i++) {
            for (int j = 0; j < rp[0].length; j++) {
                rp[i][j] = decadeToPDS.doPDS(decadeToPDS, rp[i][j], facx);
            }
        }
        return rp;
    }
}

//Trif_DNA-master/C_L_A/A_I/MPE/MSU/pde/DecadeToPDS16Test.java
//Trif_DNA-master/C_L_A/A_I/MPE/MSU/pde/DecadeToPDS1Test.java

    A_I.MPE.MSU.pds

//Trif_DNA-master/C_L_A/A_I/MPE/MSU/pds/PDE_PDS_DL.java
package A_I.MPE.MSU.pds;

//这个函数用于元基进行数字逻辑丝化变换
//思想：肽展公式, 十七进制元基数字, 元基数字逻辑
//作者：罗瑶光
//算法参考如下（肽展公式在离散数学中根据贝叶斯进行数字逻辑变换）
//##### 元基数字 = 元基符号= 肽展公式元基数字变换=（肽概率展开数字逻辑集合）
//##### 0      =          D =DD=(D, DD)
//##### E=      I =I =(I)
//##### F=      U =I++ OR Q-- =(I, Q)
//##### G=      Q =   Q =(Q)
//
//##### 1      =          C = DI =(DI)

```

```

##### 3=      E = IU, DU =(IU, DU)
##### 4=      H = (IU, DU) OR DI =(IU, DU, DI) OR (IUDI, DUDI)
##### D=      V =UQ =(UQ)
##### 9=      S =QI =(QI)
//
//
##### 2=      P =(IU, DU) + DI =(IUDI, DUDI)
##### 5=      HC=  ((IU, DU) OR DI) + DI =(IUDI, DUDI, DIDI) OR (IUDIDI
//, DUDIDI)
##### B=      HE=  ((IU, DU) OR DI) + (IU, DU) =(IUIU, IUDU, DUIU, DUDU
//, DIU, DIDU) OR (IUDIU, IUDIDU, DUDIU, DUDIDU)
##### A=      O =(IU, DU) + QI =(IUQI, DUQI)
##### 7=      A =UQQI =(UQQI)
//
//
##### 8=      M =(IU, DU) OR DI) + DI + QI =(IUDIQI, DUDIQI, DIDIQI)
//OR (IUDIDIQI, DUDIDIQI)
##### 6=      X = UQ + ((IU, DU) OR DI) + DI =(UQIUDI, UQDUDI, UQDIDI)
//OR (UQIUDIDI, UQDUDIDI)
##### C=      T =UQ + ((IU, DU) OR DI) + (IU, DU) =(UQIUIU, UQIUDU,
//UQDUIU, UQDUDU, UQDIU, UQDIDU) OR (UQIUDIU, UQIUDIDU, UQDUDIU, UQDUDIDU)
/*
* 个人著作权人,作者 罗瑶光,浏阳 yaoguanguo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class PDE_PDS_DL {
    public String initonPDSwithBYS(String initon, double bys, StringBuilder pDEKey
        , boolean isBys) {
        if(initon.equalsIgnoreCase("D")) {
            return "D";
        }
        if(initon.equalsIgnoreCase("I")) {
            return "I";
        }
        if(initon.equalsIgnoreCase("U")) {
            if(!isBys) {
                if(Math.random()< 0.5) {
                    pDEKey.append("0");
                    return "I";
                }else {
                    pDEKey.append("1");
                    return "Q";
                }
            }

```

```

        }else {
            if(Math.random()< bys) {
                pDEKey.append("0");
                return "I";
            }else {
                pDEKey.append("1");
                return "Q";
            }
        }
    }
    if(initon.equalsIgnoreCase("Q")) {
        return "Q";
    }
    if(initon.equalsIgnoreCase("C")) {
        return "DI";
    }
    if(initon.equalsIgnoreCase("E")) {
        if(!isBys) {
            if(Math.random()< 0.5) {
                pDEKey.append("0");
                return "IU";
            }else {
                pDEKey.append("1");
                return "DU";
            }
        }else {
            if(Math.random()< bys) {
                pDEKey.append("0");
                return "IU";
            }else {
                pDEKey.append("1");
                return "DU";
            }
        }
    }
    //#### 4=          H = (IU, DU) OR DI =(IU, DU, DI) OR (IUDI, DUDI)
    if(initon.equalsIgnoreCase("H")) {
        if(!isBys) {
            if(Math.random()< 0.5) {
                pDEKey.append("0");
                return "IUDI";
            }else {
                pDEKey.append("1");
                return "DUDI";
            }
        }else {
            if(Math.random()< bys) {

```

```

        pDEKey.append("0");
        return "IUDI";
    }else {
        pDEKey.append("1");
        return "DUDI";
    }
    }
    }

    //+- 符号见 FindOulerRing 函数 的 332行。
    //#### 5=          HC=  ((IU, DU) OR DI) + DI
    //=(IUDI, DUDI, DIDI) OR (IUDIDI, DUDIDI)
    if(initon.equalsIgnoreCase("-")) {
        if(!isBys) {
if(Math.random()< 0.5) {
            pDEKey.append("0");
            return "IUDIDI";
        }else {
            pDEKey.append("1");
            return "DUDIDI";
        }
        }else {
if(Math.random()< bys) {
            pDEKey.append("0");
            return "IUDIDI";
        }else {
            pDEKey.append("1");
            return "DUDIDI";
        }
        }
    }

    //+- 符号见 FindOulerRing 函数 的 332行。
    //#### B=          HE=  ((IU, DU) OR DI) + (IU, DU)
    //=(IUIU, IUDU, DUIU, DUDU, DIIU, DIDU) OR (IUDIIU, IUDIDU, DUDIIU
    //, DUDIDU)
    if(initon.equalsIgnoreCase("+")) {
        if(!isBys) {
if(Math.random()< 0.5) {
            pDEKey.append("0");
            if(Math.random()< 0.5) {
                pDEKey.append("0");
                return "IUDIIU";
            }else {
                pDEKey.append("1");
                return "IUDIDU";
            }
        }else {
            pDEKey.append("1");
        }
    }

```

```

        if(Math.random()< 0.5) {
            pDEKey.append("0");
            return "DUDIIU";
        }else {
            pDEKey.append("1");
            return "DUDIDU";
        }
    }

    }else {
if(Math.random()< bys) {
    pDEKey.append("0");
    if(Math.random()< bys) {
        pDEKey.append("0");
        return "IUDIIU";
    }else {
        pDEKey.append("1");
        return "IUDIDU";
    }
}
}else {
    pDEKey.append("1");
    if(Math.random()< bys) {
        pDEKey.append("0");
        return "DUDIIU";
    }else {
        pDEKey.append("1");
        return "DUDIDU";
    }
}
}

    }

}

if(initon.equalsIgnoreCase("V")) {
    return "UQ";
}

if(initon.equalsIgnoreCase("S")) {
    return "QI";
}

//#### 2=          P =(IU, DU) + DI =(IUDI, DUDI)
if(initon.equalsIgnoreCase("P")) {
    if(!isBys) {
if(Math.random()< 0.5) {
    pDEKey.append("0");
    return "IUDI";
}
}else {
    pDEKey.append("1");
    return "DUDI";
}

    }else {

```

```

    if(Math.random() < bys) {
        pDEKey.append("0");
        return "IUDI";
    } else {
        pDEKey.append("1");
        return "DUDI";
    }
}

}

}

//#### A=          O =(IU, DU) + QI =(IUQI, DUQI)
if(initon.equalsIgnoreCase("O")) {
    if(!isBys) {
if(Math.random() < 0.5) {
    pDEKey.append("0");
    return "IUQI";
} else {
    pDEKey.append("1");
    return "DUQI";
}

    } else {
if(Math.random() < bys) {
    pDEKey.append("0");
    return "IUQI";
} else {
    pDEKey.append("1");
    return "DUQI";
}

    }

}

}

if(initon.equalsIgnoreCase("A")) {
    return "UQQI";
}

//#### 8=          M =((IU, DU) OR DI) + DI + QI
//=(IUDIQI, DUDIQI, DIDIQI) OR (IUDIDIQI, DUDIDIQI)
if(initon.equalsIgnoreCase("M")) {
    if(!isBys) {
if(Math.random() < 0.5) {
    pDEKey.append("0");
    return "IUDIDIQI";
} else {
    pDEKey.append("1");
    return "DUDIDIQI";
}

    } else {
if(Math.random() < bys) {
    pDEKey.append("0");
    return "IUDIDIQI";

```

```

    }else {
        pDEKey.append("1");
        return "DUDIDIQI";
    }

    }

    }

    //##### 6=          X = UQ + ((IU, DU) OR DI) + DI
    //=(UQIUDI, UQDUDI, UQDIDI) OR (UQIUDIDI, UQDUDIDI)
    if(initon.equalsIgnoreCase("X")) {
        if(!isBys) {
if(Math.random()< 0.5) {
            pDEKey.append("0");
            return "UQIUDIDI";
}else {
            pDEKey.append("1");
            return "UQDUDIDI";
        }

        }else {
if(Math.random()< bys) {
            pDEKey.append("0");
            return "UQIUDIDI";
}else {
            pDEKey.append("1");
            return "UQDUDIDI";
        }

        }

        }

        //##### C=          T =UQ + ((IU, DU) OR DI) + (IU, DU)
        //=(UQIUIU, UQIUDU, UQDUIU, UQDUDU, UQDIIU, UQDIDU)
        //OR (UQIUDIIU, UQIUDIDU, UQDUDIIU, UQDUDIDU)
        if(initon.equalsIgnoreCase("T")) {
            if(!isBys) {
if(Math.random()< 0.5) {
                pDEKey.append("0");
                if(Math.random()< 0.5) {
                    pDEKey.append("0");
                    return "UQIUDIIU";
                }else {
                    pDEKey.append("1");
                    return "UQIUDIDU";
                }
            }else {
                pDEKey.append("1");
                if(Math.random()< 0.5) {
                    pDEKey.append("0");
                    return "UQDUDIIU";
                }else {

```

```

        pDEKey.append("1");
        return "UQDUDIDU";
    }
}

    }else {
if(Math.random()< bys) {
    pDEKey.append("0");
    if(Math.random()< bys) {
        pDEKey.append("0");
        return "UQIUDIIU";
    }else {
        pDEKey.append("1");
        return "UQIUDIDU";
    }
}
}else {
    pDEKey.append("1");
    if(Math.random()< bys) {
        pDEKey.append("0");
        return "UQDUDIIU";
    }else {
        pDEKey.append("1");
        return "UQDUDIDU";
    }
}
    }

    }

    }

    return null;
}

//IUQ D CEVS H POA -+ MXT
//反向排列后如下
//TXM +- AOP H SVEC D QUI

public String initonPDIwithBYS(String pds, double bys, StringBuilder pDEKey
    , boolean isBys) {
    pds= pds.replace("UQIUDIIU", "T");
    pds= pds.replace("UQIUDIDU", "T");
    pds= pds.replace("UQDUDIIU", "T");
    pds= pds.replace("UQDUDIDU", "T");
    pds= pds.replace("UQIUDIDI", "X");
    pds= pds.replace("UQDUDIDI", "X");
    pds= pds.replace("IUDIDIQI", "M");
    pds= pds.replace("DUDIDIQI", "M");
    pds= pds.replace("IUDIIU", "+");
    pds= pds.replace("IUDIDU", "+");
    pds= pds.replace("DUDIIU", "+");
    pds= pds.replace("DUDIDU", "+");
    pds= pds.replace("IUDIDI", "-");
    pds= pds.replace("DUDIDI", "-");

```

```

        pds= pds.replace("UQQI", "A");
        pds= pds.replace("IUQI", "O");
        pds= pds.replace("DUQI", "O");
        pds= pds.replace("IUDI", "P");
        pds= pds.replace("DUDI", "P");
        pds= pds.replace("IUDI", "H");
        pds= pds.replace("DUDI", "H");
        pds= pds.replace("QI", "S");
        pds= pds.replace("UQ", "V");
        pds= pds.replace("IU", "E");
        pds= pds.replace("DU", "E");
        pds= pds.replace("DI", "C");
        pds= pds.replace("D", "D");
        pds= pds.replace("Q", "Q");
        pds= pds.replace("U", "U");
        pds= pds.replace("I", "I");
        return pds;
    }

//用于肽展公式逐级变换
//QUI D SVEC H AOP +- TXM
public String initonPDEwithBYS(String pds, double pDE_KEY_rate, StringBuilder
        pDEKey, boolean isBys) {
    if(!isBys) {
        pds= pds.replace("Q", "Q");
        pds= pds.replace("U", "D");
        pds= pds.replace("I", "I");
        pds= pds.replace("D", "D");
        pds= pds.replace("QI", "S");
        pds= pds.replace("UQ", "V");
        pds= pds.replace("IU", "E");
        pds= pds.replace("DU", "E");
        pds= pds.replace("DI", "C");
        pds= pds.replace("IUDI", "H");
        pds= pds.replace("DUDI", "H");
        pds= pds.replace("UQQI", "A");
        pds= pds.replace("IUQI", "O");
        pds= pds.replace("DUQI", "O");
        pds= pds.replace("IUDI", "P");
        pds= pds.replace("DUDI", "P");
        pds= pds.replace("IUDIIU", "+");
        pds= pds.replace("IUDIDU", "+");
        pds= pds.replace("DUDIIU", "+");
        pds= pds.replace("DUDIDU", "+");
        pds= pds.replace("IUDIDI", "-");
        pds= pds.replace("DUDIDI", "-");
        pds= pds.replace("UQIUDIIU", "T");
        pds= pds.replace("UQIUDIDU", "T");
    }
}

```

```

        pds= pds.replace("UQDUDIIU", "T");
        pds= pds.replace("UQDUDIDU", "T");
        pds= pds.replace("UQIUDIDI", "X");
        pds= pds.replace("UQDUDIDI", "X");
        pds= pds.replace("IUDIDIQI", "M");
        pds= pds.replace("DUDIDIQI", "M");
        return pds;
    }

    pds= pds.replace("Q", "Q");
    pds= pds.replace("U", "U");
    pds= pds.replace("I", "I");
    pds= pds.replace("D", "D");
    pds= pds.replace("QI", "S");
    pds= pds.replace("UQ", "V");
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IU", "E");
    }else {
        pds= pds.replace("DU", "E");
    }
    pds= pds.replace("DI", "C");
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDI", "H");
    }else {
        pds= pds.replace("DUDI", "H");
    }
    pds= pds.replace("UQQI", "A");
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUQI", "O");
    }else {
        pds= pds.replace("DUQI", "O");
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDI", "P");
    }else {
        pds= pds.replace("DUDI", "P");
    }
    if(Math.random()< pDE_KEY_rate) {
        if(Math.random()<pDE_KEY_rate) {
pds= pds.replace("IUDIIU", "+");
        }else {
pds= pds.replace("IUDIDU", "+");
        }
    }else {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("DUDIIU", "+");
        }else {
pds= pds.replace("DUDIDU", "+");

```

```

    }

    }

    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDIDI", "-");
    }else {
        pds= pds.replace("DUDIDI", "-");
    }

    if(Math.random()< pDE_KEY_rate) {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("UQIUDIIU", "T");
        }else {
pds= pds.replace("UQIUDIDU", "T");
        }
    }else {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("UQDUDIIU", "T");
        }else {
pds= pds.replace("UQDUDIDU", "T");
        }
    }

    }

    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("UQIUDIDI", "X");
    }else {
        pds= pds.replace("UQDUDIDI", "X");
    }

    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUDIDIQI", "M");
    }else {
        pds= pds.replace("DUDIDIQI", "M");
    }

    return pds;
}

//融合肽展公式, 离散数学和数字逻辑 的元基变换
//    pds= pds.replace("UQ", "V");
//    pds= pds.replace("DI", "C");
//    pds= pds.replace("IQ", "S");
//    pds= pds.replace("VS", "A");
//    pds= pds.replace("ES", "O");
//    pds= pds.replace("EC", "P");
//    pds= pds.replace("CS", "M");
//    pds= pds.replace("VE", "T");
//    pds= pds.replace("VC", "X");

// 我的思维逻辑是先将PDS的数字逻辑和离散数学归纳识别, 然后走肽展识别, 最大缩短元基长度
public String initonPDE_DC DLwithBYS(String pds, double pDE_KEY_rate
    , StringBuilder pDEKey, boolean isBys) {
    if(!isBys) {
        pds= pds.replace("UQIUDIIU", "T");
    }
}

```

```

        pds= pds.replace("UQIUDIDU", "T");
        pds= pds.replace("UQDUDIIU", "T");
        pds= pds.replace("UQDUDIDU", "T");
        pds= pds.replace("UQIUDIDI", "X");
        pds= pds.replace("UQDUDIDI", "X");
        pds= pds.replace("IUDIDIQI", "M");
        pds= pds.replace("DUDIDIQI", "M");
        pds= pds.replace("IUDIIU", "+");
        pds= pds.replace("IUDIDU", "+");
        pds= pds.replace("DUDIIU", "+");
        pds= pds.replace("DUDIDU", "+");
        pds= pds.replace("IUDIDI", "-");
        pds= pds.replace("DUDIDI", "-");
        pds= pds.replace("IUDI", "H");
        pds= pds.replace("DUDI", "H");
        pds= pds.replace("UQQI", "A");
        pds= pds.replace("IUQI", "O");
        pds= pds.replace("DUQI", "O");
        pds= pds.replace("IUDI", "P");
        pds= pds.replace("DUDI", "P");
        pds= pds.replace("QI", "S");
        pds= pds.replace("UQ", "V");
        pds= pds.replace("IU", "E");
        pds= pds.replace("DU", "E");
        pds= pds.replace("DI", "C");
        pds= pds.replace("Q", "Q");
        pds= pds.replace("U", "D");
        pds= pds.replace("I", "I");
        pds= pds.replace("D", "D");
        //PDE
        pds= pds.replace("VS", "A");
        pds= pds.replace("ES", "O");
        pds= pds.replace("EC", "P");
        pds= pds.replace("HE", "+");
        pds= pds.replace("HC", "-");
        pds= pds.replace("VE", "T");
        pds= pds.replace("VC", "X");
        pds= pds.replace("CS", "M");
        return pds;
    }

    if(Math.random()< pDE_KEY_rate) {
        if(Math.random()< pDE_KEY_rate) {
            pds= pds.replace("UQIUDIIU", "T");
        } else {
            pds= pds.replace("UQIUDIDU", "T");
        }
    } else {

```

```

        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("UQDUDIIU", "T");
        }else {
pds= pds.replace("UQDUDIDU", "T");
        }
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("UQIUDIDI", "X");
    }else {
        pds= pds.replace("UQDUDIDI", "X");
    }
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUDIDIQI", "M");
    }else {
        pds= pds.replace("DUDIDIQI", "M");
    }
    if(Math.random()< pDE_KEY_rate) {
        if(Math.random()<pDE_KEY_rate) {
pds= pds.replace("IUDIIU", "+");
        }else {
pds= pds.replace("IUDIDU", "+");
        }
    }else {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("DUDIIU", "+");
        }else {
pds= pds.replace("DUDIDU", "+");
        }
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDIDI", "-");
    }else {
        pds= pds.replace("DUDIDI", "-");
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDI", "H");
    }else {
        pds= pds.replace("DUDI", "H");
    }
    pds= pds.replace("UQQI", "A");
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUQI", "O");
    }else {
        pds= pds.replace("DUQI", "O");
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDI", "P");
    }

```



```

    }else {
        pds= pds.replace("DUDI", "P");
    }
    pds= pds.replace("QI", "S");
    pds= pds.replace("UQ", "V");
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IU", "E");
    }else {
        pds= pds.replace("DU", "E");
    }
    pds= pds.replace("DI", "C");
    pds= pds.replace("D", "D");
    pds= pds.replace("Q", "Q");
    pds= pds.replace("U", "U");
    pds= pds.replace("I", "I");
    //PDE
    pds= pds.replace("VS", "A");
    pds= pds.replace("ES", "O");
    pds= pds.replace("EC", "P");
    pds= pds.replace("HE", "+");
    pds= pds.replace("HC", "-");
    pds= pds.replace("VE", "T");
    pds= pds.replace("VC", "X");
    pds= pds.replace("CS", "M");
    return pds;
}
}

```

//Trif_DNA-master/C_L_A/A_I/MPE/MSU/pds/PDE_PDS_DL16.java
package A_I.MPE.MSU.pds;

//这个函数用于元基进行数字逻辑丝化变换

//思想：肽展公式, 十六 十七进制元基数字, 元基数字逻辑

//作者：罗瑶光

//算法参考如下（肽展公式在离散数学中根据贝叶斯进行数字逻辑变换）

////#### 元基数字 = 元基符号= 肽展公式元基数字变换= (肽概率展开数字逻辑集合)

////#### 0 = D =DD=(D, DD)

////#### E= I =I =(I)

////#### F= U =I++ OR Q-- =(I, Q)

////#### G= Q = Q =(Q)

//

////#### 1 = C = DI =(DI)

////#### 3= E = IU, DU =(IU, DU)

////#### 4= H = (IU, DU) OR DI =(IU, DU, DI) OR (IUDI, DUDI)

////#### D= V =UQ =(UQ)

////#### 9= S =QI =(QI)

//

//

////#### 2= P =(IU, DU) + DI =(IUDI, DUDI)

////#### 5= HC= ((IU, DU) OR DI) + DI =(IUDI, DUDI, DIDI) OR (IUDIDI


```

        pDEKey.append("1");
        return "Q";
    }

    }

    }

    if(initon.equalsIgnoreCase("Q")) {
        return "Q";
    }

    if(initon.equalsIgnoreCase("C")) {
        return "DI";
    }

    if(initon.equalsIgnoreCase("E")) {
        if(!isBys) {
if(Math.random() < 0.5) {
            pDEKey.append("0");
            return "IU";
        }else {
            pDEKey.append("1");
            return "DU";
        }

        }else {
if(Math.random() < bys) {
            pDEKey.append("0");
            return "IU";
        }else {
            pDEKey.append("1");
            return "DU";
        }

        }

        }

        //#### 4=          H = (IU, DU) OR DI =(IU, DU, DI) OR (IUDI, DUDI)
        if(initon.equalsIgnoreCase("H")) {
            if(!isBys) {
if(Math.random() < 0.5) {
                pDEKey.append("0");
                return "IUDI";
            }else {
                pDEKey.append("1");
                return "DUDI";
            }

            }else {
if(Math.random() < bys) {
                pDEKey.append("0");
                return "IUDI";
            }else {
                pDEKey.append("1");
                return "DUDI";
            }
        }
    }

```

```

    }

    }

    }

    //+- 符号见 FindOulerRing 函数的 332行。
    //##### 5=          HC=  ((IU, DU) OR DI) + DI
    //=(IUDI, DUDI, DIDI) OR (IUDIDI, DUDIDI)
    if(initon.equalsIgnoreCase("-")) {
        if(!isBys) {
if(Math.random()< 0.5) {
    pDEKey.append("0");
    return "IUDIDI";
} else {
    pDEKey.append("1");
    return "DUDIDI";
}

        } else {
if(Math.random()< bys) {
    pDEKey.append("0");
    return "IUDIDI";
} else {
    pDEKey.append("1");
    return "DUDIDI";
}

        }

    }

    //+- 符号见 FindOulerRing 函数的 332行。
    //##### B=          HE=  ((IU, DU) OR DI) + (IU, DU)
    //=(IUIU, IUDU, DUIU, DUDU, DIIU, DIDU) OR (IUDIIU, IUDIDU, DUDIIU
    //, DUDIDU)
    if(initon.equalsIgnoreCase("+")) {
        if(!isBys) {
if(Math.random()< 0.5) {
    pDEKey.append("0");
    if(Math.random()< 0.5) {
        pDEKey.append("0");
        return "IUDIIU";
    } else {
        pDEKey.append("1");
        return "IUDIDU";
    }
} else {
    pDEKey.append("1");
    if(Math.random()< 0.5) {
        pDEKey.append("0");
        return "DUDIIU";
    } else {
        pDEKey.append("1");
    }
}

```

```

        return "DUDIDU";
    }
}

        }else {
if(Math.random()< bys) {
    pDEKey.append("0");
    if(Math.random()< bys) {
        pDEKey.append("0");
        return "IUDIIU";
    }else {
        pDEKey.append("1");
        return "IUDIDU";
    }
}
}else {
    pDEKey.append("1");
    if(Math.random()< bys) {
        pDEKey.append("0");
        return "DUDIIU";
    }else {
        pDEKey.append("1");
        return "DUDIDU";
    }
}
}

        }
    }

    if(inton.equalsIgnoreCase("V")) {
        return "UQ";
    }

    if(inton.equalsIgnoreCase("S")) {
        return "QI";
    }

    //#### 2=          P =(IU, DU) + DI =(IUDI, DUDI)
    if(inton.equalsIgnoreCase("P")) {
        if(!isBys) {
if(Math.random()< 0.5) {
    pDEKey.append("0");
    return "IUDI";
}
}else {
    pDEKey.append("1");
    return "DUDI";
}

        }else {
if(Math.random()< bys) {
    pDEKey.append("0");
    return "IUDI";
}
}else {
    pDEKey.append("1");

```

```

        return "DUDI";
    }

    }

    }

    //#### A=          O =(IU, DU) + QI =(IUQI, DUQI)
    if(initon.equalsIgnoreCase("O")) {
        if(!isBys) {
            if(Math.random() < 0.5) {
                pDEKey.append("0");
                return "IUQI";
            } else {
                pDEKey.append("1");
                return "DUQI";
            }
        } else {
            if(Math.random() < bys) {
                pDEKey.append("0");
                return "IUQI";
            } else {
                pDEKey.append("1");
                return "DUQI";
            }
        }
    }

    }

    }

    if(initon.equalsIgnoreCase("A")) {
        return "UQOI";
    }

    //根据结构式进行分析 全嘌呤肽展公式PDE PDS。罗瑶光20211201
    //F全嘌呤= E OR C OR S= E OR S OR C= (IU, DU) OR QI OR DI= (IUQIDI, DUQIDI)
    // 排列方式为 活泼组合E + 稳定组合S + 中性组合C
    if(initon.equalsIgnoreCase("F")) {
        if(!isBys) {
            if(Math.random() < 0.5) {
                pDEKey.append("0");
                return "IUQIDI";
            } else {
                pDEKey.append("1");
                return "DUQIDI";
            }
        } else {
            if(Math.random() < bys) {
                pDEKey.append("0");
                return "IUQIDI";
            } else {
                pDEKey.append("1");
                return "DUQIDI";
            }
        }
    }

```

```

    }

    }

    //#### 8=          M=((IU, DU) OR DI) + DI + QI
    //=(IUDIQL, DUDIQL, DIDIQL) OR (IUDIDIQL, DUDIDIQL)
    if(initon.equalsIgnoreCase("M")) {
        if(!isBys) {
            if(Math.random() < 0.5) {
                pDEKey.append("0");
                return "IUDIDIQL";
            } else {
                pDEKey.append("1");
                return "DUDIDIQL";
            }
        } else {
            if(Math.random() < bys) {
                pDEKey.append("0");
                return "IUDIDIQL";
            } else {
                pDEKey.append("1");
                return "DUDIDIQL";
            }
        }
    }

    }

    //#### 6=          X = UQ + ((IU, DU) OR DI) + DI
    //=(UQIUDI, UQDUDI, UQDIDI) OR (UQIUDIDI, UQDUDIDI)
    if(initon.equalsIgnoreCase("X")) {
        if(!isBys) {
            if(Math.random() < 0.5) {
                pDEKey.append("0");
                return "UQIUDIDI";
            } else {
                pDEKey.append("1");
                return "UQDUDIDI";
            }
        } else {
            if(Math.random() < bys) {
                pDEKey.append("0");
                return "UQIUDIDI";
            } else {
                pDEKey.append("1");
                return "UQDUDIDI";
            }
        }
    }

    }

    //#### C=          T=UQ + ((IU, DU) OR DI) + (IU, DU)
    //=(UQIUIU, UQIUUDU, UQDUUIU, UQDUDU, UQDIU, UQDIDU) OR (UQIUDUIU, UQIUDIDU, UQDUDUIU, UQDUDIDU)
    if(initon.equalsIgnoreCase("T")) {

```

```

        if(!isBys) {
            if(Math.random() < 0.5) {
                pDEKey.append("0");
                if(Math.random() < 0.5) {
                    pDEKey.append("0");
                    return "UQIUDIIU";
                } else {
                    pDEKey.append("1");
                    return "UQIUDIDU";
                }
            } else {
                pDEKey.append("1");
                if(Math.random() < 0.5) {
                    pDEKey.append("0");
                    return "UQDUDIIU";
                } else {
                    pDEKey.append("1");
                    return "UQDUDIDU";
                }
            }
        } else {
            if(Math.random() < bys) {
                pDEKey.append("0");
                if(Math.random() < bys) {
                    pDEKey.append("0");
                    return "UQIUDIIU";
                } else {
                    pDEKey.append("1");
                    return "UQIUDIDU";
                }
            } else {
                pDEKey.append("1");
                if(Math.random() < bys) {
                    pDEKey.append("0");
                    return "UQDUDIIU";
                } else {
                    pDEKey.append("1");
                    return "UQDUDIDU";
                }
            }
        }
    }

    return initon;//出现null 的脏字符串,
}

//IUQ D CEVS H POA -+ MXT
//反向排列后如下
//TXM +- AOP H SVEC D QUI

```



```

public String initonPDIwithBYS(String pds, double bys, StringBuilder pDEKey
    , boolean isBys) {
    pds= pds.replace("UQIUDIIU", "T");
    pds= pds.replace("UQIUDIDU", "T");
    pds= pds.replace("UQDUDIIU", "T");
    pds= pds.replace("UQDUDIDU", "T");
    pds= pds.replace("UQIUDIDI", "X");
    pds= pds.replace("UQDUDIDI", "X");
    pds= pds.replace("IUDIDIQI", "M");
    pds= pds.replace("DUDIDIQI", "M");
    pds= pds.replace("IUDIIU", "+");
    pds= pds.replace("IUDIDU", "+");
    pds= pds.replace("DUDIIU", "+");
    pds= pds.replace("DUDIDU", "+");
    pds= pds.replace("IUDIDI", "-");
    pds= pds.replace("IUQIDI", "F");//增加F PDE
    pds= pds.replace("DUQIDI", "F");
    pds= pds.replace("UQQI", "A");
    pds= pds.replace("IUQI", "O");
    pds= pds.replace("DUQI", "O");
    pds= pds.replace("IUDI", "P");
    pds= pds.replace("DUDI", "P");
    pds= pds.replace("IUDI", "H");
    pds= pds.replace("DUDI", "H");
    pds= pds.replace("QI", "S");
    pds= pds.replace("UQ", "V");
    pds= pds.replace("IU", "E");
    pds= pds.replace("DU", "E");
    pds= pds.replace("DI", "C");
    pds= pds.replace("D", "D");
    pds= pds.replace("Q", "Q");
    pds= pds.replace("U", "U");
    pds= pds.replace("I", "I");
    return pds;
}

//用于肽展公式逐级变换
//QUI D SVEC H AOP +- TXM
public String initonPDEwithBYS(String pds, double pDE_KEY_rate, StringBuilder
    pDEKey, boolean isBys) {
    if(!isBys) {
        pds= pds.replace("Q", "Q");
        pds= pds.replace("U", "D");
        pds= pds.replace("I", "I");
        pds= pds.replace("D", "D");
        pds= pds.replace("QI", "S");
        pds= pds.replace("UQ", "V");
        pds= pds.replace("IU", "E");
    }
}

```

```

        pds= pds.replace("DU", "E");
        pds= pds.replace("DI", "C");
        pds= pds.replace("IUDI", "H");
        pds= pds.replace("DUDI", "H");
        pds= pds.replace("UQQI", "A");
        pds= pds.replace("IUQI", "O");
        pds= pds.replace("DUQI", "O");
        pds= pds.replace("IUDI", "P");
        pds= pds.replace("DUDI", "P");
        pds= pds.replace("IUDIIU", "+");
        pds= pds.replace("IUDIDU", "+");
        pds= pds.replace("DUDIIU", "+");
        pds= pds.replace("DUDIDU", "+");
        pds= pds.replace("IUDIDI", "-");
        pds= pds.replace("DUDIDI", "-");
        pds= pds.replace("UQIUDIIU", "T");
        pds= pds.replace("UQIUDIDU", "T");
        pds= pds.replace("UQDUDIIU", "T");
        pds= pds.replace("UQDUDIDU", "T");
        pds= pds.replace("UQIUDIDI", "X");
        pds= pds.replace("UQDUDIDI", "X");
        pds= pds.replace("IUDIDIQI", "M");
        pds= pds.replace("DUDIDIQI", "M");
        return pds;
    }

    pds= pds.replace("Q", "Q");
    pds= pds.replace("U", "U");
    pds= pds.replace("I", "I");
    pds= pds.replace("D", "D");
    pds= pds.replace("QI", "S");
    pds= pds.replace("UQ", "V");
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IU", "E");
    }else {
        pds= pds.replace("DU", "E");
    }

    pds= pds.replace("DI", "C");
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDI", "H");
    }else {
        pds= pds.replace("DUDI", "H");
    }

    pds= pds.replace("UQQI", "A");
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUQI", "O");
    }else {
        pds= pds.replace("DUQI", "O");
    }

```

```

    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDI", "P");
    }else {
        pds= pds.replace("DUDI", "P");
    }
    if(Math.random()< pDE_KEY_rate) {
        if(Math.random()<pDE_KEY_rate) {
pds= pds.replace("IUDIIU", "+");
        }else {
pds= pds.replace("IUDIDU", "+");
        }
    }else {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("DUDIIU", "+");
        }else {
pds= pds.replace("DUDIDU", "+");
        }
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDIDI", "-");
    }else {
        pds= pds.replace("DUDIDI", "-");
    }
    if(Math.random()< pDE_KEY_rate) {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("UQIUDIIU", "T");
        }else {
pds= pds.replace("UQIUDIDU", "T");
        }
    }else {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("UQDUDIIU", "T");
        }else {
pds= pds.replace("UQDUDIDU", "T");
        }
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("UQIUDIDI", "X");
    }else {
        pds= pds.replace("UQDUDIDI", "X");
    }
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUDIDIQI", "M");
    }else {
        pds= pds.replace("DUDIDIQI", "M");
    }

```

```

        if(Math.random()<pDE_KEY_rate) {
            pds= pds.replace("IUQIDI", "F");//增加F PDE
        }else {
            pds= pds.replace("DUQIDI", "F");
        }

        return pds;
    }
}

//融合肽展公式, 离散数学和数字逻辑 的元基变换
//      pds= pds.replace("UQ", "V");
//      pds= pds.replace("DI", "C");
//      pds= pds.replace("IQ", "S");
//      pds= pds.replace("VS", "A");
//      pds= pds.replace("ES", "O");
//      pds= pds.replace("EC", "P");
//      pds= pds.replace("CS", "M");
//      pds= pds.replace("VE", "T");
//      pds= pds.replace("VC", "X");

// 我的思维逻辑是先将PDS的数字逻辑和离散数学归纳识别, 然后走肽展识别, 最大缩短元基长度
public String initonPDE_DCDLwithBYS(String pds, double pDE_KEY_rate
    , StringBuilder pDEKey, boolean isBys) {
    if(!isBys) {
        pds= pds.replace("UQIUDIIU", "T");//UQIUDIIU
        pds= pds.replace("UQIUDIDU", "T");
        pds= pds.replace("UQDUDIIU", "T");
        pds= pds.replace("UQDUDIDU", "T");
        pds= pds.replace("UQIUDIDI", "X");
        pds= pds.replace("UQDUDIDI", "X");
        pds= pds.replace("IUDIDIQI", "M");
        pds= pds.replace("DUDIDIQI", "M");
        pds= pds.replace("IUDIIU", "+");
        pds= pds.replace("IUDIDU", "+");
        pds= pds.replace("DUDIIU", "+");
        pds= pds.replace("DUDIDU", "+");
        pds= pds.replace("IUDIDI", "-");
        pds= pds.replace("DUDIDI", "-");
        pds= pds.replace("IUQIDI", "F");//增加F PDE
        pds= pds.replace("DUQIDI", "F");
        pds= pds.replace("IUDI", "H");
        pds= pds.replace("DUDI", "H");
        pds= pds.replace("UQQI", "A");
        pds= pds.replace("IUQI", "O");
        pds= pds.replace("DUQI", "O");
        pds= pds.replace("IUDI", "P");
        pds= pds.replace("DUDI", "P");
        pds= pds.replace("QI", "S");
        pds= pds.replace("UQ", "V");
    }
}

```

```

        pds= pds.replace("IU", "E");
        pds= pds.replace("DU", "E");
        pds= pds.replace("DI", "C");
        pds= pds.replace("Q", "Q");
        pds= pds.replace("U", "D");
        pds= pds.replace("I", "I");
        pds= pds.replace("D", "D");
        //PDE
        pds= pds.replace("VS", "A");
        pds= pds.replace("ES", "O");
        pds= pds.replace("EC", "P");
        pds= pds.replace("HE", "+");
        pds= pds.replace("HC", "-");
        pds= pds.replace("VE", "T");
        pds= pds.replace("VC", "X");
        pds= pds.replace("CS", "M");
        return pds;
    }
    if(Math.random()< pDE_KEY_rate) {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("UQIUDIIU", "T");
        }else {
pds= pds.replace("UQIUDIDU", "T");
        }
    }else {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("UQDUDIIU", "T");
        }else {
pds= pds.replace("UQDUDIDU", "T");
        }
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("UQIUDIDI", "X");
    }else {
        pds= pds.replace("UQDUDIDI", "X");
    }
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUDIDIQI", "M");
    }else {
        pds= pds.replace("DUDIDIQI", "M");
    }
    if(Math.random()< pDE_KEY_rate) {
        if(Math.random()<pDE_KEY_rate) {
pds= pds.replace("IUDIIU", "+");
        }else {
pds= pds.replace("IUDIDU", "+");
        }
    }

```

```

    }else {
        if(Math.random()< pDE_KEY_rate) {
pds= pds.replace("DUDIIU", "+");
        }else {
pds= pds.replace("DUDIDU", "+");
        }
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDIDI", "-");
    }else {
        pds= pds.replace("DUDIDI", "-");
    }

    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUQIDI", "F");//增加F PDE
    }else {
        pds= pds.replace("DUQIDI", "F");
    }

    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDI", "H");
    }else {
        pds= pds.replace("DUDI", "H");
    }
    pds= pds.replace("UQQI", "A");
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IUQI", "O");
    }else {
        pds= pds.replace("DUQI", "O");
    }
    if(Math.random()< pDE_KEY_rate) {
        pds= pds.replace("IUDI", "P");
    }else {
        pds= pds.replace("DUDI", "P");
    }
    pds= pds.replace("QI", "S");
    pds= pds.replace("UQ", "V");
    if(Math.random()<pDE_KEY_rate) {
        pds= pds.replace("IU", "E");
    }else {
        pds= pds.replace("DU", "E");
    }
    pds= pds.replace("DI", "C");
    pds= pds.replace("D", "D");
    pds= pds.replace("Q", "Q");
    pds= pds.replace("U", "U");
    pds= pds.replace("I", "I");

```

```

//PDE
pds= pds.replace("VS", "A");
pds= pds.replace("ES", "O");
pds= pds.replace("EC", "P");
pds= pds.replace("HE", "+");
pds= pds.replace("HC", "-");
pds= pds.replace("VE", "T");
pds= pds.replace("VC", "X");
pds= pds.replace("CS", "M");
return pds;
}
}

/Trif_DNA-master/C_L_A/A_I/MPE/MSU/pds/PDE_PDS_DL16Test.java
/Trif_DNA-master/C_L_A/A_I/MPE/MSU/pds/PDE_PDS_DLTest.java

A_I.MPE.P

/Trif_DNA-master/C_L_A/A_I/MPE/P/CopyLaw.java
package A_I.MPE.P;

import S_A.pheromone.IMV_SQI;

import java.util.Iterator;

import java.util.concurrent.ConcurrentHashMap;

import java.util.concurrent.ConcurrentLinkedDeque;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class CopyLaw {
    public DNA copy(DNA dna) {
        DNA newDna = new DNA();
        newDna.functionLineDeque = copy(dna.getMaskLineDeque());
        newDna.maskLineDeque = mask(newDna.functionLineDeque,
dna.getFunctionLineDeque(), dna.getMaskLineDeque());
        newDna.functionLineMap = copyMap(newDna.maskLineDeque);
        newDna.maskLineMap = copyMap(newDna.functionLineDeque);
        return newDna;
    }

    @SuppressWarnings("unchecked")
    public ConcurrentHashMap<Integer, PDN> copyMap(
        ConcurrentLinkedDeque<PDN> functionLineDeque) {
        ConcurrentHashMap<Integer, PDN> output = new IMV_SQI();
        Iterator<PDN> iterator = functionLineDeque.iterator();
        int count = 0;
        while (iterator.hasNext()) {
            PDN pdn = iterator.next();
            boolean[] booleanPdn = pdn.getPdn();

```

```

        boolean[] booleanPdnNew = new boolean[booleanPdn.length];
        for (int i = 0; i < booleanPdn.length; i++) {
            booleanPdnNew[i] = !booleanPdn[i];
        }
        PDN pdnNew = new PDN();
        pdnNew.I_Pdn(booleanPdnNew);
        output.put(count++, pdnNew);
    }
    return output;
}

public ConcurrentLinkedDeque<PDN> mask(
    ConcurrentLinkedDeque<PDN> originDeque,
    ConcurrentLinkedDeque<PDN> newDeque,
    ConcurrentLinkedDeque<PDN> maskOutput) {
    Iterator<PDN> iteratorOriginDeque = originDeque.iterator();
    Iterator<PDN> iteratorNewDeque = newDeque.iterator();
    while (iteratorOriginDeque.hasNext()) {
        PDN pdnOriginDeque = iteratorOriginDeque.next();
        PDN pdnNewDeque = iteratorNewDeque.next();
        boolean[] booleanPdnOriginDeque = pdnOriginDeque.getPdn();
        boolean[] booleanPdnNewDeque = pdnNewDeque.getPdn();
        for (int i = 0; i < booleanPdnOriginDeque.length; i++) {
            if (booleanPdnOriginDeque[i] != booleanPdnNewDeque[i]) {
                return null;
            }
        }
    }
    return maskOutput;
}

public ConcurrentLinkedDeque<PDN> copy(
    ConcurrentLinkedDeque<PDN> maskLineDeque) {
    ConcurrentLinkedDeque<PDN> output = new ConcurrentLinkedDeque<>();
    Iterator<PDN> iterator = maskLineDeque.iterator();
    while (iterator.hasNext()) {
        PDN pdn = iterator.next();
        boolean[] booleanPdn = pdn.getPdn();
        boolean[] booleanPdnNew = new boolean[booleanPdn.length];
        for (int i = 0; i < booleanPdn.length; i++) {
            booleanPdnNew[i] = !booleanPdn[i];
        }
        PDN pdnNew = new PDN();
        pdnNew.I_Pdn(booleanPdnNew);
        output.add(pdnNew);
    }
    return output;
}

public DNA mutationCopy(DNA dna) {

```



```

        DNA newDna = new DNA();

        newDna.maskLineDeque = copy(dna.getMaskLineDeque());

        newDna.functionLineDeque = mask(newDna.maskLineDeque,
dna.getFunctionLineDeque(), dna.getMaskLineDeque());

        newDna.functionLineMap = copyMap(newDna.maskLineDeque);
        newDna.maskLineMap = copyMap(newDna.functionLineDeque);

        return newDna;

    }

}

/Trif_DNA-master/C_L_A/A_I/MPE/P/DatingLaw.java
package A_I.MPE.P;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class DatingLaw{

    public RST dating(RST father, RST mother) {

        RST son= new RST();

        //random
        if(Math.random()>0.5) {

            son.I_FatherDeque(father.getFatherDeque());

        }else {

            son.I_FatherDeque(father.getMotherDeque());

        }

        if(Math.random()<0.5) {

            son.I_MotherDeque(mother.getMotherDeque());

        }else {

            son.I_MotherDeque(mother.getFatherDeque());

        }

        return son;

    }

}

/Trif_DNA-master/C_L_A/A_I/MPE/P/DNA.java
package A_I.MPE.P;

import java.util.concurrent.ConcurrentHashMap;

import java.util.concurrent.ConcurrentLinkedDeque;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

```

```
*/  
  
public class DNA{  
    public ConcurrentLinkedDeque<PDN> getFunctionLineDeque() {  
        return functionLineDeque;  
    }  
  
    public void I_FunctionLineDeque(ConcurrentLinkedDeque<PDN> functionLineDeque) {  
        this.functionLineDeque= functionLineDeque;  
    }  
  
    public ConcurrentLinkedDeque<PDN> getMaskLineDeque() {  
        return maskLineDeque;  
    }  
  
    public void I_MaskLineDeque(ConcurrentLinkedDeque<PDN> maskLineDeque) {  
        this.maskLineDeque= maskLineDeque;  
    }  
  
    public ConcurrentHashMap<Integer, PDN> getFunctionLineMap() {  
        return functionLineMap;  
    }  
  
    public void I_FunctionLineMap(ConcurrentHashMap<Integer, PDN> functionLineMap) {  
        this.functionLineMap= functionLineMap;  
    }  
  
    public ConcurrentHashMap<Integer, PDN> getMaskLineMap() {  
        return maskLineMap;  
    }  
  
    public void I_MaskLineMap(ConcurrentHashMap<Integer, PDN> maskLineMap) {  
        this.maskLineMap= maskLineMap;  
    }  
  
    public ConcurrentLinkedDeque<PDN> functionLineDeque;  
    public ConcurrentLinkedDeque<PDN> maskLineDeque;  
    public ConcurrentHashMap<Integer, PDN> functionLineMap;  
    public ConcurrentHashMap<Integer, PDN> maskLineMap;  
}
```

```
/Trif_DNA-master/C_L_A/A_I/MPE/P/Family.java  
package A_I.MPE.P;
```

```
import S_A.pheromone.IMV_SQI;
```

```
import java.util.concurrent.ConcurrentHashMap;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
```

```
* 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
```

* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道

* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Family {
    public ConcurrentHashMap<String, Life> getFamily() {
        return family;
    }
    public void I_Family(ConcurrentHashMap<String, Life> family) {
        this.family = family;
    }
    @SuppressWarnings("unchecked")
    ConcurrentHashMap<String, Life> family = new IMV_SQI();
}
```

/Trif_DNA-master/C_L_A/A_I/MPE/P/InheritanceLaw.java
package A_I.MPE.P;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,

* 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道

* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class InheritanceLaw{
    public RST inheritance(RST father, RST mother) {
        RST son= new RST();
        //random
        if(Math.random()> 0.997) {
            son=new MutationLaw().mutation(father, mother);
        }else {
            son=new DatingLaw().dating(father, mother);
        }
        return son;
    }
}
```

/Trif_DNA-master/C_L_A/A_I/MPE/P/Life.java
package A_I.MPE.P;
import java.util.concurrent.ConcurrentLinkedDeque;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,

* 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道

* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Life{
    public ConcurrentLinkedDeque<RST> getLife() {
        return lifeDeque;
    }
    public void I_Life(ConcurrentLinkedDeque<RST> lifeDeque) {
        this.lifeDeque= lifeDeque;
    }
    private ConcurrentLinkedDeque<RST> lifeDeque;
```

```

    public void born() {
        PDN helpFather= new PDN();
        PDN loveFather= new PDN();
        PDN workFather= new PDN();
        PDN studyFather= new PDN();
        PDN safeFather= new PDN();
        PDN creativeFather= new PDN();
        helpFather.IV_(true);
        loveFather.IV_(true);
        workFather.IV_(true);
        studyFather.IV_(true);
        safeFather.IV_(true);
        creativeFather.IV_(true);
        ConcurrentLinkedDeque<PDN> PdnsFather= new ConcurrentLinkedDeque<>();
        PdnsFather.add(loveFather);
        PdnsFather.add(safeFather);
        PdnsFather.add(studyFather);
        PdnsFather.add(creativeFather);
        PdnsFather.add(workFather);
        PdnsFather.add(helpFather);
        DNA dnaFather= new DNA();
        dnaFather.I_MaskLineDeque(PdnsFather);
        dnaFather.I_FunctionLineDeque(new CopyLaw().copy(PdnsFather));
        dnaFather.I_MaskLineMap(new CopyLaw().copyMap(dnaFather.getFunctionLineDeque()));
        dnaFather.I_FunctionLineMap(new CopyLaw().copyMap(dnaFather.getMaskLineDeque()));

        PDN helpMother= new PDN();
        PDN loveMother= new PDN();
        PDN workMother= new PDN();
        PDN studyMother= new PDN();
        PDN safeMother= new PDN();
        PDN creativeMother= new PDN();
        helpMother.IV_(false);
        loveMother.IV_(false);
        workMother.IV_(false);
        studyMother.IV_(false);
        safeMother.IV_(false);
        creativeMother.IV_(false);
        ConcurrentLinkedDeque<PDN> PdnsMother= new ConcurrentLinkedDeque<>();
        PdnsMother.add(loveMother);
        PdnsMother.add(safeMother);
        PdnsMother.add(studyMother);
        PdnsMother.add(creativeMother);
        PdnsMother.add(workMother);
        PdnsMother.add(helpMother);
        DNA dnaMother= new DNA();
        dnaMother.I_MaskLineDeque(PdnsMother);
        dnaMother.I_FunctionLineDeque(new CopyLaw().copy(PdnsMother));
        dnaMother.I_MaskLineMap(new CopyLaw().copyMap(dnaMother.getFunctionLineDeque()));
        dnaMother.I_FunctionLineMap(new CopyLaw().copyMap(dnaMother.getMaskLineDeque()));

        RST rst= new RST();
        rst.I_FatherDeque(dnaFather);
        rst.I_MotherDeque(dnaMother);
        ConcurrentLinkedDeque<RST> born= new ConcurrentLinkedDeque<>();
        born.add(rst);
        this.I_Life(born);
    }
    public void evolve() {
    }
    public void study(String token) {
    }

    public void IQ_Information() {
    }
}
//Trif_DNA-master/C_L_A/A_I/MPE/P/MutationLaw.java
package A_I.MPE.P;

/*

```

* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,

* 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道

* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

```

*/
public class MutationLaw{

    public RST mutation(RST father, RST mother) {

        RST son= new RST();

        //random

        if(Math.random()>0.5) {

            son.I_FatherDeque(father.getFatherDeque());

        }else {

            son.I_FatherDeque(mother.getFatherDeque());

        }

        if(Math.random()<0.5) {

            son.I_MotherDeque(mother.getMotherDeque());

        }else {

            son.I_MotherDeque(father.getMotherDeque());

        }

        return son;

    }

}

/Trif_DNA-master/C_L_A/A_I/MPE/P/PDN_CODE.java
package A_I.MPE.P;
/*
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public interface PDN_CODE{
    public final boolean positive= true;
    public final boolean negative= false;
}
/Trif_DNA-master/C_L_A/A_I/MPE/P/PDN.java
package A_I.MPE.P;
/*
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class PDN{
    public boolean[] getPdn() {
        return pdn;
    }
    public void I_Pdn(boolean[] pdn) {
        this.pdn= pdn;
    }
    private boolean[] pdn;
    public void IV_(boolean isMale) {
        pdn= new boolean[1985525];
        for(int i= 0; i< pdn.length; i++) {
            if(isMale) {
                pdn[i]= true;
            }else {
                pdn[i]= false;
            }
        }
    }
}
/Trif_DNA-master/C_L_A/A_I/MPE/P/Race.java
package A_I.MPE.P;

import S_A.pheromone.IMV_SQL;

import java.util.concurrent.ConcurrentHashMap;

```

```
/*
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguangluo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class Race {
    public ConcurrentHashMap<String, Family> getRace() {
        return race;
    }
    public void I_Race(ConcurrentHashMap<String, Family> race) {
        this.race = race;
    }
    @SuppressWarnings("unchecked")
    ConcurrentHashMap<String, Family> race = new IMV_SQI();
}

//Trif_DNA-master/C_L_A/A_I/MPE/P/RST.java
package A_I.MPE.P;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguangluo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class RST{
    public DNA getFatherDeque() {
        return fatherDeque;
    }

    public void I_FatherDeque(DNA fatherDeque) {
        this.fatherDeque= fatherDeque;
    }

    public DNA getMotherDeque() {
        return motherDeque;
    }

    public void I_MotherDeque(DNA motherDeque) {
        this.motherDeque= motherDeque;
    }

    private DNA fatherDeque;
    private DNA motherDeque;
```

```
}

/Trif_DNA-master/C_L_A/A_I/MPE/P/SeparaterLaw.java
package A_I.MPE.P;

import java.util.Iterator;

import java.util.concurrent.ConcurrentLinkedDeque;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class SeparaterLaw{

    public ConcurrentLinkedDeque<DNA> separater(ConcurrentLinkedDeque<DNA> original) {

        ConcurrentLinkedDeque<DNA> output= new ConcurrentLinkedDeque<>();

        //random

        if(Math.random()> 0.999997) {

            Iterator<DNA> iterator= original.iterator();

            while(iterator.hasNext()) {

                output.add(new CopyLaw().mutationCopy(iterator.next()));

            }

        }else {

            Iterator<DNA> iterator= original.iterator();

            while(iterator.hasNext()) {

                output.add(new CopyLaw().copy(iterator.next()));

            }

        }

        return output;

    }

}

/Trif_DNA-master/C_L_A/A_I/MPE/P/SwapLaw.java
package A_I.MPE.P;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class SwapLaw{
```

```

    public DNA swap(DNA original) {

        original.I_FunctionLineDeque(new CopyLaw().copy(original.getMaskLineDeque()));

        original.I_FunctionLineMap(new CopyLaw().copyMap(original.maskLineDeque));

        original.I_MaskLineMap(new CopyLaw().copyMap(original.functionLineDeque));

        return original;

    }

}

```

A_V.ASQ.PSU.test

```

/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoCogs.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoEX2.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoEXTTest.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoPOS.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoPOSforSpecial_UTF8.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/DemoPOSMedicine.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/EffectDemo.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/MakeImage.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/S_Count.java
/Trif_DNA-master/C_L_A/A_V/ASQ/PSU/test/TimeCheck.java

```

A_V.ATIPDAXQDTIV

```

/Trif_DNA-master/C_L_A/A_V/ATIPDAXQDTIV/TaskMakeCoWorker.java
package A_V.ATIPDAXQDTIV;

```

```
import ME.VPC.S.ne.App_S;
```

```
import E_A.ME.analysis.E.CogsBinaryForest_AE;
```

```
import I_A.IMO.EVU.bi_P.CoAuthorForWord;
```

```
import ME.VPC.M.app.App;
```

```
import S_A.pheromone.IMV_SQI;
```

```
import javax.swing.JScrollBar;
```

```
import java.awt.Color;
```

```
import java.awt.Container;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
```

```
* 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
```

```
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
*/
```

```
public class TaskMakeCoWorker extends Thread {
```

```
    public App_S u;
```

```
    public CogsBinaryForest_AE _A;
```

```
    public Container jpanelFirstu;
```

```
    public IMV_SQI pos;
```

```
    App NE;
```

```
    public TaskMakeCoWorker(App_S u, Container jpanelFirstu, CogsBinaryForest_AE _A
```

```
        , IMV_SQI pos, App app_NE) {
```

```
        NE = app_NE;
```

```
        this.u = u;
```

```
        this.jpanelFirstu = jpanelFirstu;
```



```

        this._A = _A;
        this.pos = pos;
    }

    @Override
    public void run() {
        u.coAuthorForWord = new CoAuthorForWord(NE);
        u.coAuthorForWord.setBackground(Color.white);
        JScrollPane jsp_bagua = new JScrollPane(u.coAuthorForWord);//3d外框
        jsp_bagua.setBounds(695, 270, 750 - 130, 440 - 50);
        jpanelFirstu.add(jsp_bagua);
        jpanelFirstu.validate();
    }
}

/Trif_DNA-master/C_L_A/A_V/ATIPDAXQDTIV/TaskMakeCoWorkerTest.java

```

A_V.C

/Trif_DNA-master/C_L_A/A_V/C/RatioMap.java

```

package A_V.C;
import ME.VPC.M.app.App;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_S_;
import S_A.pheromone.IMV_SQI_X_;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public interface RatioMap {
    double findTotalPositiveCount(IMV_SQI_X_ emotionSampleMap);
    double findTotalNegativeCount(IMV_SQI_X_ emotionSampleMap);
    double findTotalKeyCount(IMV_SQI_X_ emotionSampleMap);
    void IQ_Motivation(IMV_SQI_X_ emotionSampleMap, IMV_SQI motivation);
    void IQ_Trending(IMV_SQI_X_ emotionSampleMap, IMV_SQI trending);
    void IQ_Prediction(IMV_SQI_X_ emotionSampleMap, IMV_SQI prediction);
    void IQ_Distinction(IMV_SQI_X_ emotionSampleMap, IMV_SQI distinction);
    IMV_SQI_X_ getEnvironmentSampleMap(IMV_SQI_S_ wordFrequencyMap);
    IMV_SQI_X_ getEmotionSampleMap(IMV_SQI_S_ wordFrequencyMap
        , IMV_SQI positive, IMV_SQI negative, App NE);
    void IQ_EmotionRatio(IMV_SQI_X_ emotionSampleMap, double positiveCount
        , double negativeCount, double medCount);
    void IQ_MotivationRatio(IMV_SQI_X_ emotionSampleMap, double sumOfEmotion);
    void IQ_CorrelationRatio(IMV_SQI_X_ emotionSampleMap, double sumOfEmotion);
    void IQ_ContinusRatio(IMV_SQI_X_ emotionSampleMap, double emotionRatio);
    void IQ_TrendsRatio(IMV_SQI_X_ emotionSampleMap);
}

```

```

void IQ_PredictionRatio(IMV_SQI_X_ emotionSampleMap);

void IQ_GuessRatio(IMV_SQI_X_ emotionSampleMap);

void IQ_SensingRatio(IMV_SQI_X_ emotionSampleMap);

}

```

/Trif_DNA-master/C_L_A/A_V/C/RatioMapTest.java

A_V.cap

```

/Trif_DNA-master/C_L_A/A_V/cap/Cordination.java
package A_V.cap;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Cordination{
    public int h0;
    public int h1;
    public int w0;
    public int w1;
}
/Trif_DNA-master/C_L_A/A_V/cap/test.java
/Trif_DNA-master/C_L_A/A_V/cap/test1.java

```

A_V.E

```

/Trif_DNA-master/C_L_A/A_V/E/RatioMap_E.java
package A_V.E;

import A_V.C.RatioMap;

import ME.VPC.M.app.App;

import S_A.SVQ.stable.S_Pos;

import S_A.VSQ.parser.EmotionSample;

import S_A.pheromone.IMV_SQI;

import S_A.pheromone.IMV_SQI_S_;

import S_A.pheromone.IMV_SQI_X_;

import java.util.Iterator;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class RatioMap_E implements RatioMap {

    @Override

    public IMV_SQI_X_ getEmotionSampleMap(IMV_SQI_S_ wordFrequencyMap,

        IMV_SQI positive, IMV_SQI negative, App NE) {

        IMV_SQI_X_ output = new IMV_SQI_X_();

        for (int i = wordFrequencyMap.size()

            - S_Pos.INT_ONE; i >= S_Pos.INT_ZERO; i--) {

            if (wordFrequencyMap.getW(i).get_word()

                .length() > S_Pos.INT_ONE) {

                EmotionSample emotionSample;

                if (output.containsKey(wordFrequencyMap.getW(i)

```

```

        .get_word())) {
            emotionSample = output.get_S(wordFrequencyMap
                .getW(i).get_word());
        } else {
            emotionSample = new EmotionSample();
        }
        if (positive.containsKey(wordFrequencyMap.getW(i)
            .get_word())) {
            emotionSample.I_PositiveCount(wordFrequencyMap
                .getW(i).get_frequency());
        } else if (negative.containsKey(wordFrequencyMap.getW(
            i).get_word())) {
            emotionSample.I_NegativeCount(wordFrequencyMap
                .getW(i).get_frequency());
        } else {
            emotionSample.I_MedCount(wordFrequencyMap.getW(i)
                .get_frequency());
        }
        output.put(wordFrequencyMap.getW(i).get_word(),
            emotionSample);
    }
}

return output;
}

@Override
public void IQ_MotivationRatio(IMV_SQI_X_ emotionSampleMap,
    double sumOfEmotion) {
    Iterator<String> Iterator = emotionSampleMap.keySet()
        .iterator();
    while (Iterator.hasNext()) {
        String word = Iterator.next();
        EmotionSample emotionSample = emotionSampleMap.get_S(
word);
        emotionSample.I_MotivationRatio(emotionSample
            .getEmotionRatio() / sumOfEmotion);
        emotionSampleMap.put(word, emotionSample);
    }
}

@Override
public void IQ_CorrelationRatio(IMV_SQI_X_ emotionSampleMap,
    double sumOfEmotion) {
    Iterator<String> Iterator = emotionSampleMap.keySet()
        .iterator();
    while (Iterator.hasNext()) {
        String word = Iterator.next();
        EmotionSample emotionSample = emotionSampleMap.get_S(
word);

```

```

        emotionSample.I_CorrelationRatio((emotionSample
        .getPositiveCount() + emotionSample.getNegativeCount()
        + emotionSample.getMedCount()) / sumOfEmotion);
        emotionSampleMap.put(word, emotionSample);
    }
}

@Override
public void IQ_ContinusRatio(IMV_SQI_X_ emotionSampleMap,
    double emotionRatio) {
    Iterator<String> Iterator = emotionSampleMap.keySet()
        .iterator();
    while (Iterator.hasNext()) {
        String word = Iterator.next();
        EmotionSample emotionSample = emotionSampleMap.get_S(
word);
        emotionSample.I_ContinusRatio((emotionSample
        .getPositiveCount() + emotionSample.getNegativeCount()
        + emotionSample.getMedCount()) * emotionRatio);
        emotionSampleMap.put(word, emotionSample);
    }
}

@Override
public void IQ_TrendsRatio(IMV_SQI_X_ emotionSampleMap) {
    Iterator<String> Iterator = emotionSampleMap.keySet()
        .iterator();
    while (Iterator.hasNext()) {
        String word = Iterator.next();
        EmotionSample emotionSample = emotionSampleMap.get_S(
word);
        emotionSample.I_TrendsRatio(emotionSample
        .getEmotionRatio() * emotionSample.getContinusRatio()
        * emotionSample.getCorrelationRatio());
        emotionSampleMap.put(word, emotionSample);
    }
}

@Override
public void IQ_PredictionRatio(IMV_SQI_X_ emotionSampleMap) {
    Iterator<String> Iterator = emotionSampleMap.keySet()
        .iterator();
    while (Iterator.hasNext()) {
        String word = Iterator.next();
        EmotionSample emotionSample = emotionSampleMap.get_S(
word);
        emotionSample.I_PredictionRatio(emotionSample
        .getMotivationRatio() * emotionSample
        .getCorrelationRatio());
        emotionSampleMap.put(word, emotionSample);
    }
}

```

```
    }  
}  
  
@Override  
public void IQ_GuessRatio(IMV_SQI_X_ emotionSampleMap) {  
    Iterator<String> Iterator = emotionSampleMap.keySet()  
        .iterator();  
    while (Iterator.hasNext()) {  
        String word = Iterator.next();  
        EmotionSample emotionSample = emotionSampleMap.get_S(  
word);  
        emotionSample.I_GuessRatio(emotionSample  
.getPredictionRatio() * emotionSample  
.getTrendsRatio());  
        emotionSampleMap.put(word, emotionSample);  
    }  
}  
  
@Override  
public void IQ_SensingRatio(IMV_SQI_X_ emotionSampleMap) {  
    Iterator<String> Iterator = emotionSampleMap.keySet()  
        .iterator();  
    while (Iterator.hasNext()) {  
        String word = Iterator.next();  
        EmotionSample emotionSample = emotionSampleMap.get_S(  
word);  
        if (0 == emotionSample.getTrendsRatio()) {  
emotionSample.I_SensingRatio(0.0);  
        } else {  
emotionSample.I_SensingRatio(emotionSample  
.getPredictionRatio() / emotionSample  
.getTrendsRatio());  
        }  
        emotionSampleMap.put(word, emotionSample);  
    }  
}  
  
@Override  
public double findTotalPositiveCount(  
    IMV_SQI_X_ emotionSampleMap) {  
    double output = 1.0;  
    Iterator<String> Iterator = emotionSampleMap.keySet()  
        .iterator();  
    while (Iterator.hasNext()) {  
        String word = Iterator.next();  
        EmotionSample emotionSample = emotionSampleMap.get_S(  
word);  
        output += emotionSample.getPositiveCount();  
    }  
    return output;  
}
```

```

    }

    @Override
    public double findTotalNegativeCount(
        IMV_SQI_X_ emotionSampleMap) {
        double output = 1.0;
        Iterator<String> Iterator = emotionSampleMap.keySet()
            .iterator();
        while (Iterator.hasNext()) {
            String word = Iterator.next();
            EmotionSample emotionSample = emotionSampleMap.get_S(
word);

            output += 0.0;
            // if (null != emotionSample.getNegativeCount()) {
            output += emotionSample.getNegativeCount();
            // }
        }
        return output;
    }

    @Override
    public void IQ_EmotionRatio(IMV_SQI_X_ emotionSampleMap,
        double positiveCount, double negativeCount, double medCount) {
        Iterator<String> Iterator = emotionSampleMap.keySet()
            .iterator();
        while (Iterator.hasNext()) {
            String word = Iterator.next();
            EmotionSample emotionSample = emotionSampleMap.get_S(
word);

            double negRate = emotionSample.getNegativeCount()
/ negativeCount;
            double posRate = emotionSample.getPositiveCount()
/ positiveCount;
            double medRate = emotionSample.getMedCount() / medCount;
            emotionSample.I_EmotionRatio(negRate + posRate + medRate);
            emotionSampleMap.put(word, emotionSample);
        }
    }

    @Override
    public double findTotalKeyCount(IMV_SQI_X_ emotionSampleMap) {
        double output = 1.0; // LATER IN BOLD
        Iterator<String> Iterator = emotionSampleMap.keySet()
            .iterator();
        while (Iterator.hasNext()) {
            String word = Iterator.next();
            EmotionSample emotionSample = emotionSampleMap.get_S(
word);

            output += emotionSample.getNegativeCount() + emotionSample
.getPositiveCount() + emotionSample.getMedCount();

```

```
}
    return output;
}

@SuppressWarnings({ "unused", "unchecked" })
@Override
public void IQ_Motivation(IMV_SQI_X_ emotionSampleMap,
    IMV_SQI motivation) {
    Iterator<String> IteratorsMotivation = motivation.keySet()
        .iterator();
    Iterator<String> Iterator = emotionSampleMap.keySet()
        .iterator();
    while (Iterator.hasNext()) {
        String word = Iterator.next();
        EmotionSample emotionSample = emotionSampleMap.get_S(
word);
        if (motivation.containsKey(word)) {
emotionSample.I_Motivation(motivation.getString(
word));
        } else {
emotionSample.I_Motivation("语料库空");
        }
        emotionSampleMap.put(word, emotionSample);
    }
}

@Override
public void IQ_Trending(IMV_SQI_X_ emotionSampleMap,
    IMV_SQI trending) {
    Iterator<String> Iterator = emotionSampleMap.keySet()
        .iterator();
    while (Iterator.hasNext()) {
        String word = Iterator.next();
        EmotionSample emotionSample = emotionSampleMap.get_S(
word);
        if (trending.containsKey(word)) {
emotionSample.I_Trending(trending.getString(word));
        } else {
emotionSample.I_Trending("语料库空");
        }
        emotionSampleMap.put(word, emotionSample);
    }
    S_logger.Log.logger.info("'" + "trif");
}

@Override
public void IQ_Prediction(IMV_SQI_X_ emotionSampleMap,
    IMV_SQI prediction) {
    Iterator<String> Iterator = emotionSampleMap.keySet()
        .iterator();
```

```

        while (Iterator.hasNext()) {
            String word = Iterator.next();
            EmotionSample emotionSample = emotionSampleMap.get_S(
word);
            if (prediction.containsKey(emotionSample.getTrending())) {
emotionSample.I_Prediction(prediction.get(
emotionSample.getTrending()).toString());
            } else if (prediction.containsKey(emotionSample
.getMotivation())) {
emotionSample.I_Prediction(prediction.get(
emotionSample.getMotivation()).toString());
            } else {
emotionSample.I_Prediction("语料库空");
            }
            emotionSampleMap.put(word, emotionSample);
        }
    }
    @Override
    public void IQ_Distinction(IMV_SQI_X_ emotionSampleMap,
        IMV_SQI distinction) {
        Iterator<String> Iterator = emotionSampleMap.keySet()
            .iterator();
        while (Iterator.hasNext()) {
            String word = Iterator.next();
            EmotionSample emotionSample = emotionSampleMap.get_S(
word);
            if (distinction.containsKey(word)) {
emotionSample.I_Distinction(distinction.get(word)
.toString());
            }
            emotionSampleMap.put(word, emotionSample);
        }
    }
    /*
    * 开始思考单字的索引价值，以后换成元基索引，会出现更多的单字母。那么之后的distinct表就要
    * 进行map reduce，添加字词要谨慎。终于和 业界 的 map reduce 接轨了。同行要争气。
    *
    * --罗瑶光
    */
    @Override
    public IMV_SQI_X_ getEnvironmentSampleMap(
        IMV_SQI_S_ wordFrequencyMap) {
        IMV_SQI_X_ output = new IMV_SQI_X_();
        for (int i = wordFrequencyMap.size()
            - S_Pos.INT_ONE; i >= S_Pos.INT_ZERO; i--) {
            if (wordFrequencyMap.getW(i).get_word()
.length() > S_Pos.INT_ZERO) {

```



```

//if (wordFrequencyMap.getW(i).get_word().length() > S_Pos.INT_ONE) {

EmotionSample emotionSample = new EmotionSample();

if (!output.containsKey(wordFrequencyMap.getW(i)

    .get_word())) {

    output.put(wordFrequencyMap.getW(i).get_word(),

        emotionSample);

}

}

}

return output;

}

}

```

/Trif_DNA-master/C_L_A/A_V/E/RatioMap_ETest.java

A_V.IAVPDAXQDT

/Trif_DNA-master/C_L_A/A_V/IAVPDAXQDT/PaserException.java

A_V.ica

/Trif_DNA-master/C_L_A/A_V/ica/EducationRatio.java
package A_V.ica;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import ME.VPC.M.app.App;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI;

import S_A.pheromone.IMV_SQI_S_;

import java.util.Iterator;

import java.util.List;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,

* 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道

* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class EducationRatio {

public double[] getEducationKernel(String text,

CogsBinaryForest_AE _AE, App NE) {

CogsBinaryForest_AE _A = _AE;

IMV_SQI pos = _A.getPosCnToCn();

List<String> sets = _A.parserMixedString(text);

IMV_SQI_S_ map = _A.getWordFrequencyByReturnSortMap(sets, NE);

double[] output = new double[S_Pos.INT_SEVEN];

output[S_Pos.INT_ZERO] = (double) (sets.size());

Iterator<Integer> iterator = map.keySet().iterator();

Here: while (iterator.hasNext()) {

WordFrequency wordFrequency = map.getW(iterator.next());

if (!pos.containsKey(wordFrequency.get_word())) {

```

        continue Here;
    }
    if (pos.getString(wordFrequency.get_word()).contains(
S_Pos.NLP_ZI_MING)) {
output[S_Pos.INT_ONE] += S_Pos.INT_ONE;
continue Here;
    }
    if (pos.getString(wordFrequency.get_word()).contains(
S_Pos.NLP_ZI_DONG)) {
output[S_Pos.INT_TWO] += S_Pos.INT_ONE;
continue Here;
    }
    if (pos.getString(wordFrequency.get_word()).contains(
S_Pos.NLP_ZI_WEI)) {
output[S_Pos.INT_FOUR] += S_Pos.INT_ONE;
continue Here;
    }
    if (pos.getString(wordFrequency.get_word()).contains(
S_Pos.NLP_ZI_XING)) {
output[S_Pos.INT_THREE] += S_Pos.INT_ONE;
continue Here;
    }
    if (pos.getString(wordFrequency.get_word()).contains(
S_Pos.NLP_ZI_FU)) {
output[S_Pos.INT_FIVE] += S_Pos.INT_ONE;
continue Here;
    }
    if (pos.getString(wordFrequency.get_word()).contains(
S_Pos.NLP_ZI_JIE)) {
output[S_Pos.INT_SIX] += S_Pos.INT_ONE;
    }
    }
    return output;
}

public double[] getEducationRatioKernel(double[] input) {
    double[] output = new double[input.length];
    for (int i = S_Pos.INT_ZERO; i < input.length; i++) {
        output[i] = input[i] / input[S_Pos.INT_ZERO];
    }
    return output;
}
}

//Trif_DNA-master/C_L_A/A_V/ica/EducationRatioTest.java
//Trif_DNA-master/C_L_A/A_V/ica/InitBehaviorICAKernel.java
package A_V.ica;

import A_V.IVDTIXQCTIV.EmotionInit;
import A_V.IVDTIXQCTIV.EnvironmentInit;
import ME.VPC.M.app.App;

```

```
import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQL_X_;

import java.util.Iterator;

import java.util.LinkedList;

import java.util.List;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,

* 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道

* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class InitBehaviorICAKernel {

    private double[] kernel;

    public double[] getKernel() {

        return kernel;

    }

    public List<String> getForRestReturn() {

        return forRestReturn;

    }

    private List<String> forRestReturn;

    public double getTrustRate(String text, App NE) {

        EmotionInit emotionInitEnvironment = NE.app_S.emotionInit;

        emotionInitEnvironment.IV_(text, NE);

        double positiveCountEnvironment = emotionInitEnvironment

            .getPositiveCount();

        double totalCountEnvironment = emotionInitEnvironment

            .getTotalCount();

        positiveCountEnvironment += S_Pos.INT_ONE;

        return positiveCountEnvironment / totalCountEnvironment;

    }

    public double[] getBehaviorICAKernel(String text, App NE) {

        forRestReturn = new LinkedList<>();

        kernel = new double[S_Pos.INT_SEVEN];

        //EmotionInit emotionInit= emotionInit;

        NE.app_S.emotionInit.IV_(text, NE);

        double positiveCount = NE.app_S.emotionInit

            .getPositiveCount();

        double negativeCount = NE.app_S.emotionInit

            .getNegativeCount();

        double totalCount = NE.app_S.emotionInit.getTotalCount();

        forRestReturn.add("正面情感: " + positiveCount);

        forRestReturn.add("负面情感: " + negativeCount);

        if (positiveCount == S_Pos.INT_ZERO) {

            positiveCount = 1.0;

        }

    }

}
```

```

        if (negativeCount == S_Pos.INT_ZERO) {
            negativeCount = 1.0;
        }
        double adjRatio = Math.abs(positiveCount / negativeCount
            - negativeCount / positiveCount);
        forRestReturn.add("渲染比率: " + adjRatio);
        double psychologicRatio = (positiveCount + negativeCount)
            / totalCount;
        forRestReturn.add("情绪比率: " + psychologicRatio);
        double infectionRatio = adjRatio * psychologicRatio;
        forRestReturn.add("感染比率: " + infectionRatio);
        kernel[S_Pos.INT_ZERO] = adjRatio;
        kernel[S_Pos.INT_ONE] = psychologicRatio;
        kernel[S_Pos.INT_TWO] = infectionRatio;
        EnvironmentInit environmentInit = NE.app_S.environmentInit;//new EnvironmentInit();//later
        environmentInit.IV_FromEmotion(NE.app_S.emotionInit
            .getWordFrequencyMap(), NE);
        IMV_SQL_X_ environmentSampleMap = environmentInit
            .getEmotionSampleMap();
        forRestReturn.add("观测角度: ");
        String environmentText = "";
        Iterator<String> Iterator = environmentSampleMap.keySet()
            .iterator();
        while (Iterator.hasNext()) {
            String word = Iterator.next();
            NE.app_S.emotionSample = environmentSampleMap.get_S(word);//later..
            if (null != NE.app_S.emotionSample.getDistinction()) {
environmentText += NE.app_S.emotionSample
                .getDistinction() + S_Pos.SPACE_STRING;
            }
        }
        forRestReturn.add(environmentText);
        kernel[S_Pos.INT_THREE] = getTrustRate(environmentText, NE);
        forRestReturn.add(S_Pos.EMPTY_STRING
            + kernel[S_Pos.INT_THREE]);
        forRestReturn.add("信任比率: ");
        String motivationText = S_Pos.EMPTY_STRING;
        Iterator = environmentSampleMap.keySet().iterator();
        while (Iterator.hasNext()) {
            String word = Iterator.next();
            NE.app_S.emotionSample = environmentSampleMap.get_S(word);//later s-w
            if (null != NE.app_S.emotionSample.getMotivation()) {
if (NE.app_S.emotionSample.getMotivation().contains(
                "语料库")) {
                    continue;
                }
            }
            motivationText += NE.app_S.emotionSample

```

```

        .getMotivation() + S_Pos.SPACE_STRING;
    }
}

forRestReturn.add(motivationText);

kernel[S_Pos.INT_FOUR] = getTrustRate(motivationText, NE); //distinguide with old version later
forRestReturn.add(S_Pos.EMPTY_STRING
    + kernel[S_Pos.INT_FOUR]);

forRestReturn.add("执行比率: ");
String trendingText = S_Pos.EMPTY_STRING;
Iterator = environmentSampleMap.keySet().iterator();
while (Iterator.hasNext()) {
    String word = Iterator.next();
    NE.app_S.emotionSample = environmentSampleMap.get_S(word);
    if (null != NE.app_S.emotionSample.getTrending()) {
if (NE.app_S.emotionSample.getTrending().contains(
    "语料库")) {
        continue;
    }
    trendingText += NE.app_S.emotionSample.getTrending()
        + S_Pos.SPACE_STRING;
    }
}

forRestReturn.add(trendingText);

kernel[S_Pos.INT_FIVE] = getTrustRate(trendingText, NE);
forRestReturn.add(S_Pos.EMPTY_STRING
    + kernel[S_Pos.INT_FIVE]);

forRestReturn.add("成功比率: ");
String predictionText = S_Pos.EMPTY_STRING;
Iterator = environmentSampleMap.keySet().iterator();
while (Iterator.hasNext()) {
    String word = Iterator.next();
    NE.app_S.emotionSample = environmentSampleMap.get_S(word);
    if (null != NE.app_S.emotionSample.getPrediction()) {
if (NE.app_S.emotionSample.getPrediction().contains(
    "语料库")) {
        continue;
    }
    predictionText += NE.app_S.emotionSample
        .getPrediction() + S_Pos.SPACE_STRING;
    }
}

forRestReturn.add(predictionText);

kernel[S_Pos.INT_SIX] = getTrustRate(predictionText, NE);
forRestReturn.add(S_Pos.EMPTY_STRING + kernel[S_Pos.INT_SIX]);
return kernel;
}
}

```

```
// public double[] getBehaviorICAKernel(String text) {  
//     forRestReturn = new LinkedList<>();  
//     kernel = new double[S_Pos.INT_SEVEN];  
//     NE.app_S.emotionInit.IV_Exclude_A(text);  
//     double positiveCount = NE.app_S.emotionInit.getPositiveCount();  
//     double negativeCount = NE.app_S.emotionInit.getNegativeCount();  
//     double totalCount = NE.app_S.emotionInit.getTotalCount();  
//     forRestReturn.add("正面情感: " + positiveCount);  
//     forRestReturn.add("负面情感: " + negativeCount);  
//     if (positiveCount == S_Pos.INT_ZERO) {  
//         positiveCount = 1.0;  
//     }  
//     if (negativeCount == S_Pos.INT_ZERO) {  
//         negativeCount = 1.0;  
//     }  
//     double adjRatio = Math.abs(positiveCount / negativeCount  
//         - negativeCount / positiveCount);  
//     forRestReturn.add("渲染比率: " + adjRatio);  
//     double psychologicRatio = (positiveCount + negativeCount) / totalCount;  
//     forRestReturn.add("情绪比率: " + psychologicRatio);  
//     double infectionRatio = adjRatio * psychologicRatio;  
//     forRestReturn.add("感染比率: " + infectionRatio);  
//     kernel[S_Pos.INT_ZERO] = adjRatio;  
//     kernel[S_Pos.INT_ONE] = psychologicRatio;  
//     kernel[S_Pos.INT_TWO] = infectionRatio;  
//     NE.app_S.environmentInit.IV_FromEmotionExcludeEmotion(NE.app_S.emotionInit.getWordFrequencyMap());  
//     Map<String, EmotionSample> environmentSampleMap  
//         = NE.app_S.environmentInit.getEmotionSampleMap();  
//     forRestReturn.add("观测角度: ");  
//     String environmentText = S_Pos.EMPTY_STRING;  
//     Iterator<String> Iterator = environmentSampleMap.keySet().iterator();  
//     while (Iterator.hasNext()) {  
//         String word = Iterator.next();  
//         NE.app_S.emotionSample = environmentSampleMap.get(word);  
//         if (null != NE.app_S.emotionSample.getDistinction()) {  
//             environmentText += NE.app_S.emotionSample.getDistinction() + S_Pos.SPACE_STRING;  
//         }  
//     }  
//     forRestReturn.add(environmentText);  
//     kernel[S_Pos.INT_THREE] = getTrustRate(environmentText);  
//     forRestReturn.add(S_Pos.EMPTY_STRING + kernel[S_Pos.INT_THREE]);  
//     forRestReturn.add("信任比率: ");  
//     String motivationText = S_Pos.EMPTY_STRING;  
//     Iterator = environmentSampleMap.keySet().iterator();  
//     while (Iterator.hasNext()) {  
//         String word = Iterator.next();  
//         NE.app_S.emotionSample = environmentSampleMap.get(word);
```

```
//      if (null != NE.app_S.emotionSample.getMotivation()) {  
//          motivationText += NE.app_S.emotionSample.getMotivation()  
//              + S_Pos.SPACE_STRING;  
//      }  
//  }  
//  forRestReturn.add(motivationText);  
//  kernel[S_Pos.INT_FOUR] = getTrustRate(motivationText);  
//  forRestReturn.add(S_Pos.EMPTY_STRING + kernel[S_Pos.INT_FOUR]);  
//  forRestReturn.add("执行比率: ");  
//  String trendingText = S_Pos.EMPTY_STRING;  
//  Iterator = environmentSampleMap.keySet().iterator();  
//  while (Iterator.hasNext()) {  
//      String word = Iterator.next();  
//      NE.app_S.emotionSample = environmentSampleMap.get(word);  
//      if (null != NE.app_S.emotionSample.getTrending()) {  
//          trendingText += NE.app_S.emotionSample.getTrending()  
//              + S_Pos.SPACE_STRING;  
//      }  
//  }  
//  forRestReturn.add(trendingText);  
//  kernel[S_Pos.INT_FIVE] = getTrustRate(trendingText);  
//  forRestReturn.add(S_Pos.EMPTY_STRING + kernel[S_Pos.INT_FIVE]);  
//  forRestReturn.add("成功比率: ");  
//  String predictionText = S_Pos.EMPTY_STRING;  
//  Iterator = environmentSampleMap.keySet().iterator();  
//  while (Iterator.hasNext()) {  
//      String word = Iterator.next();  
//      NE.app_S.emotionSample = environmentSampleMap.get(word);  
//      if (null != NE.app_S.emotionSample.getPrediction()) {  
//          predictionText += NE.app_S.emotionSample.getPrediction()  
//              + S_Pos.SPACE_STRING;  
//      }  
//  }  
//  forRestReturn.add(predictionText);  
//  kernel[S_Pos.INT_SIX] = getTrustRate(predictionText);  
//  forRestReturn.add(S_Pos.EMPTY_STRING + kernel[S_Pos.INT_SIX]);  
//  return kernel;  
//  }  
//45  
//reduce later.  
//  public double getTrustRate(String text) {  
//      EmotionInit emotionInitEnvironment = NE.app_S.emotionInit;  
//      emotionInitEnvironment.IV_Exclude_A(text);  
//      //reduce  
//      double positiveCountEnvironment = emotionInitEnvironment.getPositiveCount();  
//      double totalCountEnvironment = emotionInitEnvironment.getTotalCount();  
//      positiveCountEnvironment += S_Pos.INT_ONE;
```

```
//    return positiveCountEnvironment / totalCountEnvironment;
// }
//32
//    public void I_Kernel(double[] kernel) {
//        this.kernel = kernel;
//    }
//    public void I_ForRestReturn(List<String> forRestReturn) {
//        this.forRestReturn = forRestReturn;
//    }
//Trif_DNA-master/C_L_A/A_V/ica/InitBehaviorICAKernelTest.java
    A_V.IVDTIXQCTIV
//Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/EmotionInit.java
package A_V.IVDTIXQCTIV;
import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_String;
import VSQ.parser.EmotionInit_X_S;
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
public class EmotionInit extends EmotionInit_X_S {
    public static void main(String[] argv) {
        EmotionInit emotionInit = new EmotionInit();
        emotionInit.IV_(S_String.text1, new App());
    }
    public void IV_(String text, App NE) {
        emotionMap = NE.app_S.emotionMap_E;
        _A = NE.app_S._A;
        positive = emotionMap.getPositiveMap();
        negative = emotionMap.getNegativeMap();
        sets = _A.parserMixedString(text);
        wordFrequencyMap = _A.getWordFrequencyByReturnSortMap(sets, NE);
        rationMap = NE.app_S.rationMap_E;
        emotionSampleMap = rationMap.getEmotionSampleMap(wordFrequencyMap
            , positive, negative, NE);
        positiveCount = rationMap.findTotalPositiveCount(emotionSampleMap);
        negativeCount = rationMap.findTotalNegativeCount(emotionSampleMap);
        totalCount = rationMap.findTotalKeyCount(emotionSampleMap);
    }
}
//
//public void IV_Exclude_A(String text, App NE) {
```



```
// emotionMap = NE.app_S.emotionMap_E;
// _A = NE.app_S._A;
// positive = emotionMap.getPositiveMap();
// negative = emotionMap.getNegativeMap();
// sets = _A.parserMixedString(text);
// wordFrequencyMap = _A.getWordFrequencyByReturnSortMap(sets, NE);
// rationMap = NE.app_S.rationMap_E;
// emotionSampleMap = rationMap.getEmotionSampleMap(wordFrequencyMap
// , positive, negative, NE);
// positiveCount = rationMap.findTotalPositiveCount(emotionSampleMap);
// negativeCount = rationMap.findTotalNegativeCount(emotionSampleMap);
// totalCount = rationMap.findTotalKeyCount(emotionSampleMap);
//}
//Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/EmotionInitTest.java
//Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/EnvironmentInit.java
package A_V.IVDTIXQCTIV;

import java.util.List;
import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_String;
import VSQ.parser.EnvironmentInit_X_S;
import S_A.pheromone.IMV_SQI_S_;
import S_A.pheromone.IMV_SQI_X_;
/*
* 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
* 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
* 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
*/
public class EnvironmentInit extends EnvironmentInit_X_S {
    @SuppressWarnings("unused")
    public static void main(String[] argv) {
        EnvironmentInit environmentInit = new EnvironmentInit();
        environmentInit.IV_(S_String.text1, new App());
        IMV_SQI_X_ environmentSampleMap = environmentInit
            .getEmotionSampleMap();
    }
    public void IV_(String text, App NE) {
        emotionMap = NE.app_S.emotionMap_E;
        _A = NE.app_S._A;
        motivation = emotionMap.getMotivationMap();
        trending = emotionMap.getTrendingMap();
        prediction = emotionMap.getPredictionMap();
        distinction = emotionMap.getDistinctionMap();
        //sets= _A.parserMixedString(text);//稍后统一更新下接口为parserMixedString
        sets = _A.parserMixedString(text);//
    }
}
```

```

        wordFrequencyMap = _A.getWordFrequencyByReturnSortMap(sets,
            NE);

        rationMap = NE.app_S.rationMap_E;//later。。局部对象需要重新单例 iv text，不能apps取。
        emotionSampleMap = rationMap.getEnvironmentSampleMap(
            wordFrequencyMap);
        rationMap.IQ_Motivation(emotionSampleMap, motivation);
        rationMap.IQ_Trending(emotionSampleMap, trending);
        rationMap.IQ_Prediction(emotionSampleMap, prediction);
        rationMap.IQ_Distinction(emotionSampleMap, distinction);
    }

    public void IV_SetsExclude_A(List<String> set, App NE) {
        emotionMap = NE.app_S.emotionMap_E;
        _A = NE.app_S._A;
        motivation = emotionMap.getMotivationMap();//later
        trending = emotionMap.getTrendingMap();
        prediction = emotionMap.getPredictionMap();
        distinction = emotionMap.getDistinctionMap();
        sets = set;
        //S_logger.Log.logger.info(" " + "400-8-0001-004-" + sets.size());
        wordFrequencyMap = _A.getWordFrequencyByReturnSortMap(sets,
            NE);
        rationMap = NE.app_S.rationMap_E;
        emotionSampleMap = rationMap.getEnvironmentSampleMap(
            wordFrequencyMap);
        rationMap.IQ_Motivation(emotionSampleMap, motivation);
        rationMap.IQ_Trending(emotionSampleMap, trending);
        rationMap.IQ_Prediction(emotionSampleMap, prediction);
        rationMap.IQ_Distinction(emotionSampleMap, distinction);
        //S_logger.Log.logger.info(" " + "400-8-0001-008-" + emotionSampleMap.size());
    }

    //if(emotionSampleMap.containsKey("街")) {
    //    S_logger.Log.logger.info(" " + "400-8-0001-007-04-" + emotionSampleMap.size());
    // }
    //S_logger.Log.logger.info(" " + "400-8-0001-006-" + emotionSampleMap.size());
    //S_logger.Log.logger.info(" " + "400-8-0001-007-" + distinction.size());
    //if(distinction.containsKey("街")) {
    //    S_logger.Log.logger.info(" " + "400-8-0001-007-01-" + distinction.getString("街"));
    //}
    //if(distinction.containsKey("餐饮")) {
    //    S_logger.Log.logger.info(" " + "400-8-0001-007-01-" + distinction.getString("餐饮"));
    //}
    /*
    * 说明我的函数还可以细化切割成更小的块。
    */

    public void IV_Exclude_A(String text, App NE) {
        emotionMap = NE.app_S.emotionMap_E;
        _A = NE.app_S._A;
    }

```

```
motivation = emotionMap.getMotivationMap(); //later
trending = emotionMap.getTrendingMap();
prediction = emotionMap.getPredictionMap();
distinction = emotionMap.getDistinctionMap();
sets = _A.parserString(text);
wordFrequencyMap = _A.getWordFrequencyByReturnSortMap(sets,
    NE);
rationMap = NE.app_S.ratioMap_E;
emotionSampleMap = rationMap.getEnvironmentSampleMap(
    wordFrequencyMap);
rationMap.IQ_Motivation(emotionSampleMap, motivation);
rationMap.IQ_Trending(emotionSampleMap, trending);
rationMap.IQ_Prediction(emotionSampleMap, prediction);
rationMap.IQ_Distinction(emotionSampleMap, distinction);
}

public void IV_FromEmotion(IMV_SQI_S_ getWordFrequencyMap,
    App NE) {
    emotionMap = NE.app_S.emotionMap_E;
    //parser sentence
    motivation = emotionMap.getMotivationMap();
    trending = emotionMap.getTrendingMap();
    prediction = emotionMap.getPredictionMap();
    distinction = emotionMap.getDistinctionMap();
    //map
    rationMap = NE.app_S.ratioMap_E;
    emotionSampleMap = rationMap.getEnvironmentSampleMap(
        getWordFrequencyMap);
    rationMap.IQ_Motivation(emotionSampleMap, motivation);
    rationMap.IQ_Trending(emotionSampleMap, trending);
    rationMap.IQ_Prediction(emotionSampleMap, prediction);
    rationMap.IQ_Distinction(emotionSampleMap, distinction);
}

}

//
//public void IV_FromEmotionExcludeEmotion(IMV_SQI_S_ getWordFrequencyMap, App NE) {
//    emotionMap = NE.app_S.emotionMap_E;
//    motivation = emotionMap.getMotivationMap();
//    trending = emotionMap.getTrendingMap();
//    prediction = emotionMap.getPredictionMap();
//    distinction = emotionMap.getDistinctionMap();
//    rationMap = NE.app_S.ratioMap_E;
//    emotionSampleMap = rationMap.getEnvironmentSampleMap(getWordFrequencyMap);
//    rationMap.IQ_Motivation(emotionSampleMap, motivation);
//    rationMap.IQ_Trending(emotionSampleMap, trending);
//    rationMap.IQ_Prediction(emotionSampleMap, prediction);
//    rationMap.IQ_Distinction(emotionSampleMap, distinction);
//}
```

```
// emotionMap.IV_MotivationMap();
// emotionMap.IV_TrendingMap();
// emotionMap.IV_PredictionMap();
// emotionMap.IV_DistinctionMap();
// emotionMap.IV_MotivationMap();
// emotionMap.IV_TrendingMap();
// emotionMap.IV_PredictionMap();
// emotionMap.IV_DistinctionMap();

//Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/EnvironmentInitTest.java
//Trif_DNA-master/C_L_A/A_V/IVDTIXQCTIV/LenovoInit.java
package A_V.IVDTIXQCTIV;

import java.util.List;

import A_V.E.RatioMap_E;

import C_A.OCLAMV.ECS.SVQ.MPC.fhmm.C.SensingMap;

import E_A.OELAMV.ECS.SVQ.MPC.SOQ.OEM.E.SensingMap_E;

import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;

import E_A.ME.analysis.E.BinaryForest_AE;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import ME.VPC.M.app.App;

import S_A.SVQ.stable.S_String;

import S_A.VSQ.parser.EmotionSample;

import S_A.pheromone.IMV_SQL;

import S_A.pheromone.IMV_SQL_X_;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利- 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道
 * 神仙坳社区 大塘冲路一段 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class LenovoInit {

    public SensingMap getSensingMap() {

        return sensingMap;

    }

    public void I_SensingMap(SensingMap sensingMap) {

        this.sensingMap = sensingMap;

    }

    public EnvironmentInit getEnvironmentInit() {

        return environmentInit;

    }

    // public void I_EnvironmentInit(EnvironmentInit
    // environmentInit) {
    //this.environmentInit= environmentInit;
    //      }

    private SensingMap sensingMap;

    private EnvironmentInit environmentInit;

    @SuppressWarnings("unused")
```

```

    public static void main(String[] argv,
        EnvironmentInit environmentInit, EmotionMap_E emotionMap_E,
        RatioMap_E ratioMap_E, CogsBinaryForest_AE _AE,
        SensingMap_E sensingMap_E, EmotionSample emotionSample) {
        LenovoInit lenovoInit = new LenovoInit();
        lenovoInit.IV_(S_String.text1, new App());
        IMV_SQI_X_ environmentSampleMap = lenovoInit
            .getEnvironmentInit().getEmotionSampleMap();
        IMV_SQI lenovo = lenovoInit.getSensingMap().getLenovoMap();
    }

    public void IV_(String text, App NE) {
        environmentInit.IV_(text, NE);
        //this.environmentInit = environmentInit;
        //this.environmentInit = NE.app_S.environmentInit;//later trif
        sensingMap = NE.app_S.sensingMap_E;
        sensingMap.IV_LenovoMap(((BinaryForest_AE) environmentInit
            .get_A()));
    }

    public void IV_Exclude_A(String text, App NE) {
        NE.app_S.environmentInit.IV_Exclude_A(text, NE);
        this.environmentInit = NE.app_S.environmentInit;
        sensingMap = NE.app_S.sensingMap_E;
        sensingMap.IV_LenovoMap(((BinaryForest_AE) environmentInit
            .get_A()));
    }

    public void IV_SetsExclude_A(List<String> set, App NE) {
        NE.app_S.environmentInit.IV_SetsExclude_A(set, NE);
        this.environmentInit = NE.app_S.environmentInit;
        sensingMap = NE.app_S.sensingMap_E;
        sensingMap.IV_LenovoMap(((BinaryForest_AE) environmentInit
            .get_A()));
    }
}

```

/Trif_DNA-master/C_L_A/A_V/IVDTIXOCTIV/LenovoInitTest.java
 1-2-/Trif_DNA-master/C_L_C-控制-分词-VPCS

C

/Trif_DNA-master/C_L_C/C/Verbal_C.java
 /Trif_DNA-master/C_L_C/C/VerbalMap_C.java

C_A.ME.analysis.C

/Trif_DNA-master/C_L_C/C_A/ME/analysis/C/A.java
 package C_A.ME.analysis.C;

import ME.VPC.M.app.App;

import java.util.List;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface A {

    // public List<String> parserMixedString(StringBuilder mixedString);

// public List<String> parserMixedString(String mixedString);

    public List<String> parserString(String inputString);

    public void I_FixWords(int charPosition, String inputString

        , StringBuilder[] fixWords);

    public String[] parserEnglishString(String englishString);

    public void IV_(App NE);

    public void IV_Mixed(App NE);

    public void IV_All(App NE);

    public void IV_condition(int indexType, App NE);

}

//void IV_0 ;

//void IV_Mixed() ;

//List<String> parserString(String input);

//void I_FixWords(int charPosition, String inputString, StringBuilder[] fixWords);

//String[] parserEnglishString(String englishString);

//List<String> parserMixedString(String mixedString);

//Trif_DNA-master/C_L_C/C_A/ME/analysis/C/AC.java
package C_A.ME.analysis.C;
import java.util.List;
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
public interface AC {
    public List<String> parserMixedString(StringBuilder mixedString);
    public List<String> parserMixedString(String mixedString);
}
//void IV_0 ;
//void IV_Mixed() ;
//List<String> parserString(String input);
//void I_FixWords(int charPosition, String inputString, StringBuilder[] fixWords);
//String[] parserEnglishString(String englishString);
//List<String> parserMixedString(String mixedString);
//Trif_DNA-master/C_L_C/C_A/ME/analysis/C/BinaryForest_A.java
package C_A.ME.analysis.C;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import S_A.pheromone.IMV_SQI;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

```

```
*/
```

```
public interface BinaryForest_A {
    void studyNewWord(String study, String token, String posStudy);
    IMV_SQI getStudyPos();
    IMV_SQI getCtK();
    IMV_SQI getCtG();
    IMV_SQI getCtS();
    IMV_SQI getPosEnToCn();
    IMV_SQI getPosEnToEn();
    IMV_SQI getPosCnToCn();
    IMV_SQI getEnToCn();
    IMV_SQI getCnToEn();
    IMV_SQI getFullEnToCn();
    IMV_SQI getFullCnToEn();
    SensingTest getSensingTest();
}
```

```
/Trif_DNA-master/C_L_C/C_A/ME/analysis/C/CogsBinaryForest_A.java
package C_A.ME.analysis.C;
```

```
import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;
```

```
import ME.VPC.M.app.App;
```

```
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
```

```
import S_A.pheromone.IMV_SQI_SS;
```

```
import S_A.pheromone.IMV_SQI_S_;
```

```
import java.util.List;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
*/
```

```
public interface CogsBinaryForest_A extends BinaryForest_A {
    IMV_SQI_SS parserMixStringByReturnFrequencyMap(String key, App NE);
    IMV_SQI_SS getWordFrequencyMap(List<String> sets, App NE);
    IMV_SQI_S_ getWordFrequencyByReturnSortMap(List<String> sets, App NE);
    EmotionMap getEmotionMap();
    IMV_SQI_S_ sortWordFrequencyMapToSortMap(IMV_SQI_SS map);
    List<WordFrequency> sortWordFrequencyMap(IMV_SQI_SS map);
}
```

```
C_A.ME.euclid.C
```

```
/Trif_DNA-master/C_L_C/C_A/ME/euclid/C/Euclid_C.java
package C_A.ME.euclid.C;
```

```
import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;
```

```
import S_A.SVQ.stable.S_Pos;
```

```
import java.util.Map;
```

```
/*
```

```

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface Euclid_C {

    @SuppressWarnings({S_Pos.RAW_TYPES})
    Map<Integer, Map> mCogsEuclid(Map<Long, FMHMMNode> concurrentHashMap);

}

/Trif_DNA-master/C_L_C/C_A/ME/euclid/C/EuclidCTest.java

    C_A.ME.liner.C

/Trif_DNA-master/C_L_C/C_A/ME/liner/C/Quick6DLuoYaoguangSort.java
package C_A.ME.liner.C;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.pheromone.IMV_SQI_SS;

import S_A.pheromone.IMV_SQI_S_;

import java.util.List;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface Quick6DLuoYaoguangSort {

    void quick6DLuoYaoGuangSortWordFrequency(List<WordFrequency> list, int leftPosition
        , int rightPosition);

    int partitionList(List<WordFrequency> list, int leftPosition, int rightPosition);

    List<WordFrequency> frequencyWordMapToList(IMV_SQI_SS map);

    void quick6DLuoYaoGuangSortWordFrequency(IMV_SQI_S_ map, int leftPosition
        , int rightPosition);

    int partition(IMV_SQI_S_ map, int leftPosition, int rightPosition);

    IMV_SQI_S_ frequencyWordMapToMap(IMV_SQI_SS map);

}

/Trif_DNA-master/C_L_C/C_A/ME/liner/C/Quick6DLuoYaoguangSortTest.java

    C_A.ME.nero.C

/Trif_DNA-master/C_L_C/C_A/ME/nero/C/NERO_C_OneTime.java
package C_A.ME.nero.C;

import ME.VPC.M.app.App;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

import java.util.Map;

/*

```



```

* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface NERO_C_OneTime {

    StringBuilder getBinaryForestRecurWords(StringBuilder inputStringWordNode,
        StringBuilder inputString, int charPosition, int inputLength,
        Map<Long, FMHMMNode> forestRoots, int forestDepth,
        int charPositionNext);

    StringBuilder getBinaryForestsRecurWords(StringBuilder inputStringWordNode,
        String inputString, int charPosition, int inputLength,
        Map<Long, FMHMMNode>[] forestsRoots, int forestDepth,
        int charPositionNext);

    StringBuilder getQuickForestRecurWord(StringBuilder inputStringWordNode,
        String inputString, int charPosition, int inputLength,
        int forestDepth, int charPositionNext, App NE);

}

//Trif_DNA-master/C_L_C/C_A/ME/nero/C/NERO_C_OneTimeTest.java
//Trif_DNA-master/C_L_C/C_A/ME/nero/C/NERO_C.java
package C_A.ME.nero.C;

import S_A.SVQ.stable.S_Pos;

import java.util.Map;

/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface NERO_C {

    @SuppressWarnings(S_Pos.RAW_TYPES)

    StringBuilder getBinaryForestRecurWord(StringBuilder inputStringWordNode,
        String inputString, int charPosition, int inputLength,
        Map<Integer, Map> forestRoots, int forestDepth,
        int charPositionNext);

}

//Trif_DNA-master/C_L_C/C_A/ME/nero/C/NERO_CTest.java

C_A.OCL.AMV.ECS.SVQ.MPC.fhmm.C

//Trif_DNA-master/C_L_C/C_A/OCI/AMV/ECS/SVQ/MPC/fhmm/C/SensingMap.java

package C_A.OCL.AMV.ECS.SVQ.MPC.fhmm.C;

import C_A.ME.analysis.C.BinaryForest_A;

import S_A.pheromone.IMV_SQL;

```

```

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface SensingMap{

    void IV_LenovoMap(BinaryForest_A _A) ;

    public IMV_SQI getLenovoMap();

    //public void I_LenovoMap(IMV_SQI lenovoMap);

}

C_A.OCL.AVC.SUQ.SVQ.MPC.fhmm.C

/Trif_DNA-master/C_L_C/C_A/OCI/AVC/SUQ/SVQ/MPC/fhmm/C/EmotionMap.java
package C_A.OCL.AVC.SUQ.SVQ.MPC.fhmm.C;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface EmotionMap{

    void IV_MotivationMap(App NE) ;

    void IV_PositiveMap(App NE) ;

    void IV_NegativeMap(App NE) ;

    void IV_TrendingMap(App NE) ;

    void IV_PredictionMap(App NE) ;

    void IV_DistinctionMap(App NE) ;

    public IMV_SQI getPositiveMap();

    public void I_PositiveMap(IMV_SQI positiveMap);

    public IMV_SQI getNegativeMap();

    public void I_NegativeMap(IMV_SQI negativeMap);

    public IMV_SQI getMotivationMap() ;

    public void I_MotivationMap(IMV_SQI motivationMap);

    public IMV_SQI getTrendingMap() ;

    public void I_TrendingMap(IMV_SQI trendingMap);

    public IMV_SQI getPredictionMap() ;

    public void I_PredictionMap(IMV_SQI predictionMap);

    public IMV_SQI getDistinctionMap() ;

    public void I_DistinctionMap(IMV_SQI distinctionMap);

```

```

}

/Trif_DNA-master/C_L_C/C_A/OCI/AVC/SUQ/SVQ/MPC/fhmm/C/EmotionMapTest.java

C_A.OCI.OPM.VEC.SVQ.MPC.fhmm.C

/Trif_DNA-master/C_L_C/C_A/OCI/OPM/VEC/SVQ/MPC/fhmm/C/BehaviorMap.java
package C_A.OCI.OPM.VEC.SVQ.MPC.fhmm.C;
import S_A.pheromone.IMV_SQI;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public interface BehaviorMap{
    IMV_SQI initBusinessMap();
    IMV_SQI initTradeMap();
    IMV_SQI initFacilityMap();
    IMV_SQI initAckuisitionMap();
    IMV_SQI initCoorporationMap();
    IMV_SQI initPromiseMap();
}
/Trif_DNA-master/C_L_C/C_A/OCI/OPM/VEC/SVQ/MPC/fhmm/C/BehaviorMapTest.java

C_A.OCI.SVQ.MPC.fhmm.C

/Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FHMMList_indexAll_X.java
package C_A.OCI.SVQ.MPC.fhmm.C;

//词汇翻译系统

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public interface FHMMList_indexAll_X {

    void loadLanguage(String cInputString, String cInputStringKorea,

        String cInputStringJapan, String cInputStringTrandition,

        String cInputStringEnglish, String cInputStringRs,

        String cInputStringAb, String cInputStringFn, String cInputStringGm,

        String cInputStringSp);

    void loagPOS(String cInputString, String cInputStringKorea,

        String cInputStringJapan, String cInputStringTrandition,

        String cInputStringEnglish, String cInputStringRs,

        String cInputStringAb, String cInputStringFn, String cInputStringGm,

        String cInputStringSp);

}

//public void indexAll();

/Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FHMMList_indexAll.java
package C_A.OCI.SVQ.MPC.fhmm.C;
//词汇翻译系统
import ME.VPC.M.app.App;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

```

```

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
public interface FHMMList_indexAll extends FHMMList {
    // void index();
    public void indexMixed(App NE);
    public void flushRedundant();
}
//public void indexAll();
//Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FHMMList.java
package C_A.OCI.SVQ.MPC.fhmm.C;

//词汇翻译系统

//import E_A.OEI.SVQ.MPC.fhmm.E.FastReadProjectFile;

import ME.VPC.M.app.App;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

//import S_A.SVQ.stable.S_Maps;

//import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI;

//import U_A.PEU.P.nlp.StopSymbol_UTF8;

//import exception.file.O.DetaBufferedReader;

//import exception.file.O.DetaInputStreamReader;

//

//import java.io.InputStream;

//import java.util.Iterator;

//import java.util.LinkedList;

import java.util.List;

import java.util.Map;

//import java.util.concurrent.CopyOnWriteArrayList;

/*

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public interface FHMMList {

    public List<String> englishStringToWordsList(String string);

    public Map<Long, FMHMMNode> getMap();

    public Map<Long, FMHMMNode>[] getMaps();

    // 稍后translator的index 去重 统一用index mixed

    public void index();

    public void indexFullEnToCn();

    public void indexFullCnToEn();

    public void indexFullCnToFn();

    public void indexFullCnToKo();

    public void indexFullCnToJp();

    public void indexFullCnToGm();

```

```
public void indexFullCnToSp();
public void indexFullCnToRs();
public void indexFullCnToAb();
public void indexFullCnToPy();
public void indexFullCnToTt();
public IMV_SQI getFullEnToCn();
public IMV_SQI getFullCnToEn();
public void indexPosEnToCn(App NE);
public void indexFn(App NE);
public void indexKo(App NE);
public void indexJp(App NE);
public void indexGm(App NE);
public void indexSp(App NE);
public void indexAb(App NE);
public void indexRs(App NE);
public void indexPy(App NE);
public void indexTt(App NE);
public void indexPosEnToEn(App NE);
public void indexEnToCn(App NE);
public void indexCnToEn(App NE);
public Map<Long, FMHMMNode> loopLoadForest(String cInputString);
public Map<Long, FMHMMNode> doNeroPostCognitive(FMHMMNode fFHMMNode,
        String cInputString, int i);
public Map<Long, FMHMMNode> doCheckAndRunNeroPostFix(FMHMMNode fFHMMNode,
        String cInputString, int i);
public IMV_SQI getPosCnToCn();
public IMV_SQI getEnToCn();
public IMV_SQI getCnToEn();
public IMV_SQI getPosEnToCn();
public IMV_SQI getPosEnToEn();
public IMV_SQI getFullCnToJp();
public IMV_SQI getFullCnToRs();
public IMV_SQI getFullCnToAb();
public IMV_SQI getFullCnToFn();
public IMV_SQI getFullCnToGm();
public IMV_SQI getFullCnToKo();
public IMV_SQI getFullCnToSp();
public IMV_SQI getFullCnToPy();
public IMV_SQI getFullCnToTt();
public void indexFullNegative();
public void indexFullPositive();
public IMV_SQI getFullNegative();
public IMV_SQI getFullPositive();
public Map<Long, IMV_SQI> getWordsForests();
public void studyNewPos(String string, String posStudy);
public IMV_SQI getStudyPos();
```

```
}
```

```
//Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FMHMMListTest.java
//Trif_DNA-master/C_L_C/C_A/OCI/SVQ/MPC/fhmm/C/FMHMMListTest.java
package C_A.OCI.SVQ.MPC.fhmm.C;
```

```
import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;
```

```
import S_A.pheromone.IMV_SQI;
```

```
import java.util.List;
```

```
import java.util.Map;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
*/
```

```
public interface FMHMMList extends FMHMMList {
```

```
    @SuppressWarnings("rawtypes")
```

```
    public Map<Integer, Map> getRoot();
```

```
    public void index();
```

```
    public Map<Long, FMHMMNode> loopLoadForest(String cInputString);
```

```
    public Map<Long, FMHMMNode> doNeroPostCognitive(FMHMMNode fFMHMMNode,
```

```
        String cInputString, int i);
```

```
    public Map<Long, FMHMMNode> doCheckAndRunNeroPostFix(FMHMMNode fFMHMMNode,
```

```
        String cInputString, int i);
```

```
    public IMV_SQI getWords();
```

```
    public Map<Long, FMHMMNode> getMap();
```

```
    public IMV_SQI getPosEnToEn();
```

```
    public IMV_SQI getEnToCn();
```

```
    public IMV_SQI getCnToEn();
```

```
    public void indexEnToCn();
```

```
    public void indexCnToEn();
```

```
    public IMV_SQI getPosEnToCn();
```

```
    public IMV_SQI getPosCnToCn();
```

```
    public void indexPosEnToCn();
```

```
    public void indexPosEnToEn();
```

```
    public void indexFullEnToCn();
```

```
    public void indexFullCnToEn();
```

```
    public IMV_SQI getFullEnToCn();
```

```
    public IMV_SQI getFullCnToEn();
```

```
    public List<String> englishStringToWordsList(String string);
```

```
    public void indexFullCnToJp();
```

```
    public void indexFullCnToRs();
```

```
    public void indexFullCnToAb();
```

```
    public void indexFullCnToFn();
```

```
    public void indexFullCnToGm();
```

```
    public void indexFullCnToKo();
```

```
    public void indexFullCnToSp();
```

```

    public void indexFullCnToPy();
    public IMV_SQI getFullCnToJp();
    public IMV_SQI getFullCnToRs();
    public IMV_SQI getFullCnToAb();
    public IMV_SQI getFullCnToFn();
    public IMV_SQI getFullCnToGm();
    public IMV_SQI getFullCnToKo();
    public IMV_SQI getFullCnToSp();
    public IMV_SQI getFullCnToPy();
    public Map<Long, FMHMMNode>[] getMaps();
    public Map<Long, IMV_SQI> getWordsForests();
    public void studyNewPos(String string, String posStudy);
    public IMV_SQI getStudyPos();
    public IMV_SQI getFullCnToTt();
    public void indexFullCnToTt();
}

C_A.OSI.AOP.MS.VPC.S.hall

/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/S/hall/DatabaseLog_H.java

```

```

package C_A.OSI.AOP.MS.VPC.S.hall;
import M_V.ME.SM.OP.SMA.AOP.MEC.SIQ.E.E_PLSQL_E;
import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaFileReader;
import S_A.SVQ.stable.S_Web;
import U_A.PEU.P.cache.Cache_M;
import U_A.PEU.P.zip.GzipUtil;
import U_A.PEU.P.zip.UnZip;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.UnsupportedEncodingException;
@SuppressWarnings("unused")
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳 yaoguanguo@outlook.com,
 * 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用) 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342 204925063,
 * 389418686, F2406501, 0626136 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class DatabaseLog_H {
    static String logCategoryPath;
    static String logCurrentFilePath;
    static String logCurrentFile;
    public static void I_BinLog_H() {
        // db write operation
        IV_LogCategoryPath();
    }
}

```

```
IV_CurrentFilePath();
// write error rollback
// binlog
}

private static void IV_CurrentFilePath() {
    long yearMonthDay = System.currentTimeMillis();
    long day = yearMonthDay / (1000 * 60 * 60 * 24);
    logCurrentFilePath = logCategoryPath + "/log/logger" + day + ".det";
}

private static void IV_LogCategoryPath() {
    if (null != Cache_M.getCacheInfo("DBPath")) {
        logCategoryPath = Cache_M.getCacheInfo("DBPath").getValue()
        .toString();
    } else {
        try {
            throw new Exception();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}

public static void writeLogFile(long when, String who, String plsql) {
    Q_CurrentFileRange();
    // zip text string and write; im thinking about a new method
    // to make string write small.
    String logString = "#@:" + when + "@:" + who + "@:" + plsql;
    FileWriter fw = null;
    try {
        fw = new FileWriter(logCurrentFilePath, true);
        fw.write("\n\r\n");
        fw.write(new String(
            GzipUtil.compress(logString.getBytes(S_Web.CHARSET_UTF8)),
            S_Web.CHARSET_UTF8));
        // fw.write(logString);
        fw.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static void Q_CurrentFileRange() {
    long yearMonthDay = System.currentTimeMillis();
    long day = yearMonthDay / (1000 * 60 * 60 * 24);
    String willMakeFile = logCategoryPath + "/log/logger" + day + ".det";
    String willMakeFilePath = logCategoryPath + "/log";
    File file = new File(willMakeFile);
    File fileWillMakeFilePath = new File(willMakeFilePath);
    if (!fileWillMakeFilePath.exists()) {
```



```

        fileWillMakeFilePath.mkdirs();

        // dont make new for this class, will get memory leakage
    }

    if (!file.exists()) {

        // dont make new for this class, will get memory leakage
        logCurrentFilePath = willMakeFile;
    }

    // if currentfiletime - currenttime > 1 day
    // if currentfilesize > 100mb will make discussion later,
    // now just make one file per day.
    // makenew file;
}

@SuppressWarnings("resource")
private static void coverageByTime(long time) {

    // 1删除已损坏的数据库 已完成
    File needClear = new File("C:/DetaDB");
    needClear.delete();

    // 2解压备份数据库 已完成
    UnZip.unZipWithPath("C:/DetaLog/zipCover/zip_1549583065203.zip",
"C:/DetaLog/zipCover/cover");

    // 3循环执行备份plsql命令,直到等于大于时间戳完成返回。
    DetaBufferedReader reader = null;
    reader = new DetaBufferedReader(
DetaFileReader.E("C:/DetaLog/log/logger.det");

    String tempString;
    while ((tempString = reader.readDetaLine()) != null) {

        // 解gzip压缩并执行数据库恢复
        try {
tempString = new String(
            GzipUtil.uncompress(
tempString.getBytes(S_Web.CHARSET_UTF8)),
            S_Web.CHARSET_UTF8);
        } catch (UnsupportedEncodingException e) {
throw new RuntimeException(e);
        }

        long currentTime = Long.valueOf(tempString.split("@:")[1]);
        if (currentTime < time) { // 逐条恢复到点。
E_PLSQL_E.E_PLSQL(tempString.split("@:")[3], true);
        }
    }

    reader.closeDeta();
}

}

/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/S/hall/DatabaseLog_HTest.java
C_A.OSI.AOP.MS.VPC.server

/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/BootVPCSBackEnd.java
package C_A.OSI.AOP.MS.VPC.server;

```

```

import ME.VPC.S.ne.App_S;

import A_V.IVDTIXQCTIV.EnvironmentInit;

import A_V.E.RatioMap_E;

import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;

import E_A.OELAVC.SUQ.SVQ.MPC.fhmm.E.EmotionMap_E;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import E_A.ME.liner.E.Quick6DLuoYaoguangSort3DMap_E;

import E_A.ME.nero.E.NERO_C_OneTime_E;

import E.Nlp_CE_X_S;

import P.Pos_X_P;

import E_A.OELSVQ.MPC.fhmm.E.FMHMMListOneTime_E;

import ME.VPC.M.app.App;

import exception.thread.DetaThread;

import P_V.PCS.thread.SocketThread;

import P_V.PCS.thread.SocketThreadPool;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import S_logger.Log;

import java.net.ServerSocket;

import java.util.Properties;

import java.util.Random;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
@SuppressWarnings("unused")
public class BootVPCBackEnd extends Thread {
    private ServerSocket server;
    private Properties properties;
    private CogsBinaryForest_AE _A;
    private EmotionMap emotionMap;// later remove
    private RatioMap_E ratioMap_E;
    private EnvironmentInit environmentInit;
    public App_S app;
    public BootVPCBackEnd(App_S app, App NE) {
        properties = new Properties();
        this.app = app;
        S_logger.Log.logger.info("" + "启动400编码调试开始-00000052");
        if (null == app._A) {
            this._A = new CogsBinaryForest_AE();
            this._A.IV_Mixed(NE);
        } else {
            this._A = app._A;

```

```

    }
    S_logger.Log.logger.info("启动400编码调试开始-00000052-01");
}
public BootVPCSBackEnd() {
}
public void IV_(App NE) { // 代入逻辑必定都是E实体。不确定的具体引用加Q
    try {
        S_logger.Log.logger.info("启动400编码调试开始-00000052-02");
        // port= Integer.parseInt(properties.getProperty("port"));
        int port = Integer.parseInt(app.backendTxt);
        S_logger.Log.logger.info("启动400编码调试开始-00000053-" + port);
        // port= Config.detaVPCSBackEndPort;
        server = new ServerSocket(port);
        if (null == this._A) {
this._A = new CogsBinaryForest_AE();
this._A.IV_Mixed(NE);
        }
        emotionMap = NE.app_S.emotionMap_E;
        ratioMap_E = NE.app_S.ratioMap_E; // later。。局部对象需要单例，不能apps取。
        environmentInit = NE.app_S.environmentInit; // later。。局部对象需要单例，不能apps取。
        // 20230106-S_logger.Log.logger.info("启动400编码调试开始-00000052-01-02");
        // 20230106-S_logger.Log.logger.info("启动400编码调试开始-00000052-01-03");
        // port);
        // MrTin.born.start();
        // 20230106-S_logger.Log.logger.info("启动400编码调试开始-00000052-01-04");
        // 第一代类人催化生命特征开启:" + port);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
@SuppressWarnings("unused")
public void bootBackEnd(SocketThreadPool socketThreadPool, App NE) {
    // SocketThreadPool socketThreadPool = new
    // SocketThreadPool();
    long before = System.currentTimeMillis();
    S_logger.Log.logger.info("启动400编码调试开始-00000052-01-02");
    IV_(NE);
    S_logger.Log.logger.info("启动400编码调试开始-00000052-01-03");
    I_RestService();
    S_logger.Log.logger.info("启动400编码调试开始-00000052-01-04");
    long now = System.currentTimeMillis();
    S_logger.Log.logger.info("DETA VPCS BackEnd--2.0");
    S_logger.Log.logger.info("Author: 罗瑶光");
    S_logger.Log.logger.info("浏阳德塔软件开发有限公司开源项目");
    S_logger.Log.logger.info("德塔VPCS后端服务器");
    S_logger.Log.logger.info("德塔VPCS后端服务器启动一切正常-总耗时:" + (now - before) + "毫秒");
    S_logger.Log.logger.info("启动400编码调试开始-00000052-01-04-01");
}

```

```

        while (true) {
            try {
                if (socketThreadPool.getThreadsCount() < 300) {
                    S_logger.Log.logger.info("" + "启动400编码调试开始-00000052-01-04-02");
                    SocketThread clientSocket = new SocketThread(
                        socketThreadPool, server.accept(),
                        System.currentTimeMillis() + ""
                            + new Random().nextLong(),
                        NE);
                    S_logger.Log.logger.info("" + "启动400编码调试开始-00000052-01-04-03");
                    socketThreadPool.I_E_Socket(clientSocket.getSid(),
                        clientSocket);
                    clientSocket.start();
                    S_logger.Log.logger.info("" + "启动400编码调试开始-00000052-01-04-04");
                    DetaThread.sleepDeta(30); // lag limited
                }
            } catch (Exception e) {
                break;
            }
        }
    }

    public static void main(String[] args, EnvironmentInit environmentInit,
        EmotionMap_E emotionMap_E, RatioMap_E ratioMap_QE,
        EnvironmentInit environmentInit_QE,
        SocketThreadPool socketThreadPool,
        NERO_C_OneTime_E nERO_C_OneTime_E, Nlp_CE_X_S nlp_CE_X_S,
        Pos_X_P pos_X_P, SensingTest sensingTest,
        FMHMMListOneTime_E fMHMMListOneTime_E,
        Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E,
        App NE) {
        new BootVPCSEnd().bootBackEnd(socketThreadPool, NE);
    }

    public void run(SocketThreadPool socketThreadPool, App NE) {
        S_logger.Log.logger.info("" + "启动400编码调试开始-00000052-01-01");
        bootBackEnd(socketThreadPool, NE);
    }

    private void I_RestService() {
    }
}

//55
// 因为首页已经init了,我之后会改成_A带入就是了. 罗瑶光20210420
//  EmotionMap_E emotionMap_E, RatioMap_E ratioMap_QE
//      , EnvironmentInit environmentInit_QE, NERO_C_OneTime_E nERO_C_OneTime_E
//      , Nlp_CE_X_S nlp_CE_X_S, Pos_X_P pos_X_P, SensingTest sensingTest
//      , FMHMMListOneTime_E fMHMMListOneTime_E
//      , Quick6DLuoYaoguangSort3DMap_E quick6DLuoYaoguangSort3DMap_E
//      emotionMap_E, nERO_C_OneTime_E, nlp_CE_X_S, pos_X_P

```

```
//          , sensingTest, fMHMMListOneTime_E
//          , quick6DLuoYaoguangSort3DMap_E
//78
//      emotionMap.IV_NegativeMap();
//      emotionMap.IV_PositiveMap();
//      emotionMap.IV_MotivationMap();
//      emotionMap.IV_TrendingMap();
//      emotionMap.IV_PredictionMap();
//      emotionMap.IV_DistinctionMap();

//Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/BootVPCSEndTest.java
//Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/RequestFilter_C.java
package C_A.OSI.AOP.MS.VPC.server;

import S_A.pheromone.IMV_SQL;

import java.net.InetAddress;

import java.net.Socket;

import java.net.UnknownHostException;

import java.util.Map;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class RequestFilter_C {

    static Map<String, Boolean> ipBlock;

    public static void main(String[] args) {

    }

    public static void requestIpFilter(Socket socket) {

    }

    public static void requestLinkFilter(Socket socket) {

    }

    public static void requestIpFilter(VPCSRequest vPCSRequest,
                                       VPCSResponse vPCSResponse) {

        // 过滤block
        if (ipBlock.containsKey(vPCSRequest.getRequestIp())) {

            vPCSResponse.setErrorCode(403);

        }

        // 同源csrf
        try {

            if (vPCSRequest.getRequestIp().equalsIgnoreCase(
                InetAddress.getLocalHost().getHostAddress())) {

                // vPCSResponse.setErrorCode(405);

            }

        }

    }

}
```

```

        } catch (UnknownHostException e) {
            throw new RuntimeException(e);
        }
    }
}

public static void requestLinkFilter(VPCSRequest vpcsRequest,
    VPCSResponse vPCSResponse) {
    // //限制ddos
    // String ipCount= DetaDBUtil.cacheRequest("get?key="
    // + vpcsRequest.getRequestIp() + "&email="
    // + "313699483@qq.com" + "&password=" + "Fengyue1985!");
    // if(null== ipCount){
    // DetaDBUtil.cacheRequest("put?key="
    // + vpcsRequest.getRequestIp() + "&value=" + "1" + "&time="
    // + (2000+System.currentTimeMillis())
    // + "&email=" + "313699483@qq.com" + "&password=" +
    // "Fengyue1985!");
    // }else if(ipCount.contains("unsucces")){
    // DetaDBUtil.cacheRequest("put?key="
    // + vpcsRequest.getRequestIp() + "&value=" + "1" + "&time="
    // + (2000+System.currentTimeMillis())
    // + "&email=" + "313699483@qq.com" + "&password=" +
    // "Fengyue1985!");
    // }else if(Integer.valueOf(ipCount) <= 30){
    // int digit= Integer.valueOf(ipCount) + 1;
    // DetaDBUtil.cacheRequest("put?key=" +
    // vpcsRequest.getRequestIp()
    // + "&value=" + digit + "&time="
    // + 0 + "&email=" + "313699483@qq.com"
    // + "&password=" + "Fengyue1985!");
    // }else{
    // vPCSResponse.returnErrorCode(400);
    // }
}

// 先注释掉 之后测试 20210405 罗瑶光
// DB在此处出错后能读DMA读写了. 罗瑶光 20210406 稍后进行block list函数设计 优先级降低
@SuppressWarnings("unchecked")
public static void IV_BlockList() {
    ipBlock = new IMV_SQI();
    // FileInputStream fileInputStream= new FileInputStream(
    // new File("src/main/resources/ipBlock.ips"));
    // InputStreamReader inputStreamReader= new
    // InputStreamReader(fileInputStream, "UTF-8");
    // DetaBufferedReader DetaBufferedReader= new
    // DetaBufferedReader(inputStreamReader);
    // String line= null;
    // while ((line= DetaBufferedReader.readDetaLine()) != null)
    // {

```

```

        // ipBlock.put(line, true);

        // }

        // DetaBufferedReader.close();

    }

}

//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestFilter_CTest.java
//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestFix_C.java
package C_A.OSI.AOP.MS.VPC.server;

import java.net.Socket;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class RequestFix_C {

    public static void main(String[] args){

    }

    public static void requestIpFix(Socket socket) {

    }

    public static void requestLinkFix(Socket socket) {

    }

    public static void requestIpFix(VPCSRequest vPCSRequest

        , VPCSResponse vPCSResponse) {

    }

    public static void requestLinkFix(VPCSRequest vPCSRequest

        , VPCSResponse vPCSResponse) {

    }

}

//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestFix_CTest.java
//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/RequestRecord_C.java
package C_A.OSI.AOP.MS.VPC.server;

import exception.file.O.DetaBufferedReader;

import exception.file.O.DetaInputStreamReader;

import S_A.SVQ.stable.S_Pos;

import S_A.SVQ.stable.S_Web;

import S_A.pheromone.IMV_SQI;

import java.io.IOException;

import java.io.UnsupportedEncodingException;

import java.net.URLDecoder;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*

* 个人著作权人, 作者 罗瑶光. 浏阳

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class RequestRecord_C {  
    public static void requestIpRecoder(VPCSRequest vPCSRequest,  
        VPCSResponse vPCSResponse) {  
        vPCSRequest.setRequestIp(  
vPCSResponse.getSocket().getInetAddress().getHostAddress());  
        vPCSRequest.setRequestName(  
vPCSResponse.getSocket().getInetAddress().getHostName());  
    }  
    @SuppressWarnings({ "unused", "resource", "unchecked" })  
    public static void requestLinkRecoder(VPCSRequest vPCSRequest,  
        VPCSResponse vPCSResponse) {  
        DataBufferedReader br = null;  
        try {  
            br = new DataBufferedReader(DataInputStreamReader.E(  
vPCSResponse.getSocket().getInputStream(),  
S_Web.CHARSET_UTF8));  
        } catch (IOException e) {  
            throw new RuntimeException(e);  
        }  
        String mess = br.readDataLine();  
        if (mess.contains("替") || mess.contains("改") || mess.contains("写")  
|| mess.contains("换") || mess.contains("删")  
|| mess.contains("增") || mess.contains("更")  
|| mess.contains("rop") || mess.contains("rea")  
|| mess.contains("elet") || mess.contains("ate")  
|| mess.contains("ert") || mess.contains("lace")  
|| mess.contains("转")) {  
            vPCSResponse.returnValue(S_Web.HTTP_400);  
            return;  
        }  
        if (null == mess) {  
            vPCSResponse.returnValue(S_Web.HTTP_400);  
            return;  
        }  
        if (mess.equalsIgnoreCase(S_Web.STRING_EMPTY)) {  
            vPCSResponse.returnValue(S_Web.HTTP_400);  
            return;  
        }  
        String[] type = mess.split(S_Web.STRING_SPACE);  
        if (type.length < S_Pos.INT_TWO) {
```



```

        vPCSResponse.setErrorCode(S_Web.HTTP_500);

        return;
    }

    String[] content = type[S_Pos.INT_ONE]
.split(S_Web.STRING_SLASH_QUESTION);
    if (content.length == S_Pos.INT_TWO) {
        vPCSRequest.setRequestIsRest(true);
        if (content[S_Pos.INT_ONE] == null) {
            vPCSResponse.setErrorCode(S_Web.HTTP_500);
            return;
        }
    }
    if (content[S_Pos.INT_ZERO].contains(S_Web.STRING_QUATE)) {
        // vPCSRequest.I_RequestIsRest(false);
    }
    if (vPCSRequest.getRequestIsRest()) {
        String[] column = content[S_Pos.INT_ONE]
.split(S_Web.STRING_JUNCTION);
        IMV_SQI data = new IMV_SQI();
        for (String cell : column) {
            String[] cells = cell.split(S_Web.MATH_EQUAL);
            try {
                data.put(cells[S_Pos.INT_ZERO], URLDecoder
                    .decode(cells[S_Pos.INT_ONE], S_Web.CHARSET_UTF_8));
            } catch (UnsupportedEncodingException e) {
                throw new RuntimeException(e);
            }
        }
        vPCSRequest.setRequestValue(data);
    }
    vPCSRequest.setRequestLink(content[S_Pos.INT_ZERO]);
}
}

```

/Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/RequestRecord_CTest.java
 /Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/Response_C.java
 package C_A.OSI.AOP.MS.VPC.server;

import java.io.IOException;

import java.io.PrintWriter;

import java.net.Socket;

import I_Q.hvpcs.boot.vpc.sleeper.SleeperHall;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class Response_C {
    public static void main(String[] args) {
    }
    @SuppressWarnings("unused")
    private static void error404(Socket socket, SleeperHall sleeper_H, Integer sId) {
        PrintWriter pw = null;
        try {
            pw = new PrintWriter(socket.getOutputStream(), true);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        pw.println("HTTP/1.1 404 OK\n\n");
        pw.flush();
        pw.close();
        try {
            socket.close();
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        sleeper_H.removeThreadById(sId);
        return;
    }
    @SuppressWarnings("unused")
    private static void error500(Socket socket, SleeperHall sleeper_H, Integer sId) {
        PrintWriter pw = null;
        try {
            pw = new PrintWriter(socket.getOutputStream(), true);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        pw.println("HTTP/1.1 500 OK\n\n");
        pw.flush();
        pw.close();
        try {
            socket.close();
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        sleeper_H.removeThreadById(sId);
        return;
    }
}

//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ResponseCTest.java
//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerForward_Standard.java
package C_A.OSI.AOP.MS.VPC.server;

import O_V.OSI.AOP.VPC.rest.VPCYangliaojing;

import S_A.SVQ.stable.S_Web;
```

```
import java.net.Socket;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

import I_Q.hvpcs.boot.vpc.vision.RestMapVision_XP_bytes;

import I_Q.hvpcs.boot.vpc.vision.RestMapVision_XP_rest;

import ME.VPC.M.app.App;

//import OSI.AOP.VPC.PP.port_E.RestLoginPort_E;

//import PEU.P.map.VtoV;

//合并 数据库和 前端的 vision文件

//罗瑶光 20210730

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class ServerForward_Standard {

    public static void main(String[] args) {

    }

    public static void IQ_ForwardType(Socket socket) {

    }

    public static void forwardToRestMap(Socket socket) {

    }

    public static void IQ_ForwardType(VPCSRequest vPCSRequest,

        VPCSResponse vPCSResponse) {

        if (vPCSRequest.getRequestIsRest()) {

            String filePath = VPCYangliaoJing

                .getFilePath(vPCSRequest.getRequestLink());

            if (filePath.contains(S_Web.FILE_TTF)

                || filePath.contains(S_Web.FILE_EOT)

                || filePath.contains(S_Web.FILE_SVG)

                || filePath.contains(S_Web.FILE_WOFF)

                || filePath.contains(S_Web.FILE_WOFF2)

                || filePath.contains(S_Web.FILE_OTF)) {

                String code = VPCYangliaoJing.getCode(filePath);

                vPCSRequest.setRequestFilePath(filePath);

                vPCSRequest.setRequestFileCode(code);

                vPCSRequest.setRequestForwardType(S_Web.STREAM_BUFFER);

                vPCSResponse.setResponseContentType(

                    S_Web.HEADER_CONTENT_TYPE_FONT_WOFF);

                return;

            }

            vPCSRequest.setRequestForwardType(S_Web.STREAM_REST);

        } else {
```

```
String filePath = VPCYangliaoJing
    .getFilePath(vPCSRRequest.getRequestLink());
    String code = VPCYangliaoJing.getCode(filePath);
    vPCSRRequest.setRequestFilePath(filePath);
    vPCSRRequest.setRequestFileCode(code);
    if (filePath.contains(S_Web.FILE_PNG)) {
vPCSRRequest.setRequestForwardType(S_Web.STREAM_BYTES);
vPCSRResponse
    .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_PNG);
    }
    if (filePath.contains(S_Web.FILE_JPEG)) {
vPCSRRequest.setRequestForwardType(S_Web.STREAM_BYTES);
vPCSRResponse
    .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_JPEG);
    }
    if (filePath.contains(S_Web.FILE_JPG)) {
vPCSRRequest.setRequestForwardType(S_Web.STREAM_BYTES);
vPCSRResponse
    .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_JPG);
    }
    if (filePath.contains(S_Web.FILE_GIF)) {
vPCSRRequest.setRequestForwardType(S_Web.STREAM_BYTES);
vPCSRResponse
    .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_GIF);
    }
    if (filePath.contains(S_Web.FILE_JS)
        && code.equalsIgnoreCase(S_Web.CHARSET_UTF_8)) {
vPCSRRequest.setRequestForwardType(S_Web.STREAM_BYTES_BUFFER);
vPCSRResponse
    .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_JS);
    }
    if (filePath.contains(S_Web.FILE_CSS)) {
vPCSRRequest.setRequestForwardType(S_Web.STREAM_BUFFER);
vPCSRResponse
    .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_CSS);
    }
    if (filePath.contains(S_Web.FILE_HTML)) {
vPCSRRequest.setRequestForwardType(S_Web.STREAM_BUFFER);
vPCSRResponse
    .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_HTML);
    }
    if (filePath.contains(S_Web.FILE_PDF)) {
vPCSRRequest
    .setRequestForwardType(S_Web.STREAM_BYTES_WITHOUT_ZIP);
vPCSRResponse
    .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_PDF);
    }
```

```

        if (filePath.contains(S_Web.FILE_ZIP)) {

vPCSRequest

                .setRequestForwardType(S_Web.STREAM_BYTES_WITHOUT_ZIP);

vPCSResponse

                .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_ZIP);
        }

        if (filePath.contains(S_Web.FILE_WAV)) {
vPCSRequest.setRequestForwardType(S_Web.STREAM_BUFFER);
vPCSResponse

                .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_WAV);
        }

        if (filePath.contains(S_Web.FILE_RAR)) {
vPCSRequest

                .setRequestForwardType(S_Web.STREAM_BYTES_WITHOUT_ZIP);

vPCSResponse

                .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_RAR);
        }

        if (filePath.contains(S_Web.FILE_XML)) {
vPCSRequest

                .setRequestForwardType(S_Web.STREAM_BYTES_WITHOUT_ZIP);

vPCSResponse

                .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_XML);
        }

        if (filePath.contains(S_Web.FILE_TXT)) {
vPCSRequest

                .setRequestForwardType(S_Web.STREAM_BYTES_WITHOUT_ZIP);

vPCSResponse

                .setResponseContentType(S_Web.HEADER_CONTENT_TYPE_TXT);
        }
    }
}

public static void forwardToRestMap(VPCSRequest vPCSRequest,
    VPCSResponse vPCSResponse, App NE) {
    if (vPCSRequest.getRequestForwardType() == null) {
        vPCSResponse.return404();
        return;
    }
    if (vPCSRequest.getRequestForwardType()
.equalsIgnoreCase(S_Web.STREAM_REST)) {
        // ServerRestMap_Standard.P_Rest(vPCSRequest, vPCSResponse);
        RestMapVision_XP_rest.processRest(vPCSRequest, vPCSResponse, NE);
        // 注意vpc是否有那个link转发不然会null。trif 2025
        return;
    }
    // later 统一byte with gzip
    if (vPCSRequest.getRequestForwardType()
.equalsIgnoreCase(S_Web.STREAM_BYTES)) {

```

```

        System.out.flush();
        S_logger.Log.logger.info("'" + "-bytes");
        RestMapVision_XP_bytes.processBytes(vPCSRequest, vPCSResponse);
        return;
        // ServerRestMap_Standard.P_Bytes(vPCSRequest,
        // vPCSResponse);
    }

    if (vPCSRequest.getRequestForwardType()
.equalsIgnoreCase(S_Web.STREAM_BUFFER)) {
        System.out.flush();
        S_logger.Log.logger.info("'" + "-bytes");
        RestMapVision_XP_bytes.processBytes(vPCSRequest, vPCSResponse);
        return;
        // ServerRestMap_Standard.P_Buffer(vPCSRequest,
        // vPCSResponse);
    }

    if (vPCSRequest.getRequestForwardType()
.equalsIgnoreCase(S_Web.STREAM_BYTES_BUFFER)) {
        System.out.flush();
        S_logger.Log.logger.info("'" + "-bytes");
        RestMapVision_XP_bytes.processBytes(vPCSRequest, vPCSResponse);
        return;
        // ServerRestMap_Standard.P_BufferBytes(vPCSRequest,
        // vPCSResponse);
    }

    if (vPCSRequest.getRequestForwardType()
.equalsIgnoreCase(S_Web.STREAM_BYTES_WITHOUT_ZIP)) {
        System.out.flush();
        S_logger.Log.logger.info("'" + "-bytes");
        RestMapVision_XP_bytes.processBytes(vPCSRequest, vPCSResponse);
        return;
        // ServerRestMap_Standard.P_BytesWithoutZip(vPCSRequest,
        // vPCSResponse);
    }
}

//
// public static void forwardToRestMap(App app, VPCSRequest vPCSRequest
// , VPCSResponse vPCSResponse) {
//if(null== vPCSRequest || null== vPCSRequest.getRequestForwardType()){
//    vPCSResponse.return404();
//    return;
//}
//if(vPCSRequest.getRequestForwardType().equalsIgnoreCase(S_Web.STREAM_REST)){
//    //与其设计个sleep继承,不如在这里设计个condition函数
//,然后根据port识别 来继承这个condition 就是了。 , 罗瑶光 20210730
//    if(vPCSRequest.getRequestPort().equals(S_HTTP.PORT_DATABASE)) {
//        RestMap_V.P_RestDB(app, vPCSRequest, vPCSResponse);

```

```
//      }else {

//          RestMap_V.P_Rest(app, vPCSRequest, vPCSResponse);

//      }

//}

//if(vPCSRequest.getRequestForwardType().equalsIgnoreCase(S_Web.STREAM_BYTES)){

//      RestMap_V.P_Bytes(vPCSRequest, vPCSResponse);

//}

//if(vPCSRequest.getRequestForwardType().equalsIgnoreCase(S_Web.STREAM_BUFFER)){

//      RestMap_V.P_Buffer(vPCSRequest, vPCSResponse);

//}

//if(vPCSRequest.getRequestForwardType().equalsIgnoreCase(S_Web.STREAM_BYTES_BUFFER)){

//      RestMap_V.P_BufferBytes(vPCSRequest, vPCSResponse);

//}

//if(vPCSRequest.getRequestForwardType().equalsIgnoreCase(S_Web.STREAM_BYTES_WITHOUT_ZIP)){

//      RestMap_V.P_BytesWithoutZip(vPCSRequest, vPCSResponse);

//}

//      }

//Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/ServerForward_StandardTest.java
//Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/ServerInit_C_VPCSEnd.java
package C_A.OSI.AOP.MS.VPC.server;

import ME.VPC.S.ne.App_S;

import M_V.MS.VPC.PP.Time_P;

import exception.thread.DetaThread;

import utils.DetaUtil;

import YIJ_HRJ.ME.VPC.S.configs.ConfigIndex;

import java.io.IOException;

import java.net.ServerSocket;

import java.util.Properties;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

import I_Q.hvpcs.boot.vpc.sleeper.Sleeper;

import I_Q.hvpcs.boot.vpc.sleeper.SleeperHall;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class ServerInit_C_VPCSEnd {

    private ServerSocket server;

    @SuppressWarnings("unused")

    private Properties properties;

    private App_S app;

    private int port;

    {
```

```

        properties = new Properties();

        // 20230106-S_logger.Log.logger.info(" + "----德塔VPCS前端服务器资源载入:成功!");
    }

    public void IV_() {
        try {
            port = Integer.valueOf(app.frontendTxt);
            S_logger.Log.logger.info(" + "启动400编码调试开始-00000051-" + port);
            // port= Config.detaVPCSFrontEndPort;
            server = new ServerSocket(port);
            // 20230106-S_logger.Log.logger.info(" + "----德塔VPCS前端服务器端口启动:" +
            // port);
            if (null != app.dbConfigPath && !app.dbConfigPath.isEmpty()) {
                DetaUtil.IV_DB(app.dbConfigPath); // 稍后整理下
            } else {
                DetaUtil.IV_DB(ConfigIndex.detaDBPath);
            }
            // 20230106-S_logger.Log.logger.info(" + "----德塔VPCS前端服务器DMA确认:成功!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private void haoHiYooFaker(SleeperHall sleeper_H) {
        sleeper_H.callSkivvy();
    }

    public void IV_Server(App_S app) {
        this.app = app;
        S_logger.Log.logger.info(" + ");
        S_logger.Log.logger.info(" + "----DETA VPCS--2.0");
        S_logger.Log.logger.info(" + "----Author: 罗瑶光");
        S_logger.Log.logger.info(" + "----浏阳德塔软件开发有限公司开源项目");
        Time_P time_P = new Time_P();
        time_P.begin();
        SleeperHall sleeper_H = new SleeperHall();
        IV_();
        time_P.end();
        S_logger.Log.logger.info(" +
        "----德塔VPCS前端服务器启动一切正常-总耗时:" + time_P.duration() + "毫秒");
        while (true) {
            if (sleeper_H.getThreadsCount() < 500) {
                VPCSRequest vPCSRequest = new VPCSRequest();
                VPCSResponse vPCSResponse = new VPCSResponse();
                Sleeper sleeper = new Sleeper();
                sleeper.goToBedroom(vPCSRequest, vPCSResponse);
                try {
                    sleeper.hugPillow(sleeper_H, server.accept(),
                    sleeper.hashCode());
                } catch (IOException e) {

```



```

        throw new RuntimeException(e);
    }
    sleeper.start();
    DetaThread.sleepDeta(30);
    } else {
        haoHiYooFaker(sleeper_H);
    }
}
}
}

//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerInit_C_VPCSFrontEndTest.java
//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerInit_C.java
package C_A.OSI.AOP.MS.VPC.server;

import C_A.OSI.AOP.MS.VPC.S.hall.DatabaseLog_H;

import I_Q.hvpcs.boot.sets.StableBlob;

import I_Q.hvpcs.boot.sets.StableSets;

//import I_Q.hvpcs.boot.sets.VPCSRequest;

//import I_Q.hvpcs.boot.sets.VPCSResponse;

import I_Q.hvpcs.boot.vpc.process.TimeProcess;

import I_Q.hvpcs.boot.vpc.sleeper.SleeperHall;

import I_Q.hvpcs.boot.vpc.sleeper.Sleeper_X;

import I_Q.hvpcs.boot.vpc.sleeper.TimeHall;

import ME.VPC.M.app.App;

import M_V.MS.OP.SM.AOP.MEC.SIQ.cache.DetaDBBufferCache_M;

import M_V.MS.VPC.PP.Time_P;

//import S_A.SVQ.stable.S_HTTP;

//import S_A.SVQ.stable.S_Web;

import utils.DetaUtil;

import YLJ_HRJ.ME.VPC.S.configs.ConfigIndex;

import javax.swing.JOptionPane;

import java.io.IOException;

import java.net.ServerSocket;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class ServerInit_C {
    private ServerSocket server;

    // private static Properties properties;

    private static int port;

    private static String dbConfigPath;

    public void IV_Service(JOptionPane jTextPane, String frontendTxt) {
        try {

```

```
// port=
// Integer.parseInt(properties.getProperty(S_Data.TCP_PORT));
// port = 3306;
port = Integer.valueOf(frontendTxt);
S_logger.Log.logger.info("'" + "启动400编码调试开始-00000054-" + port);
// port= Config.detaVPCSDBPort;
server = new ServerSocket(port);
// 20230106-S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器端口启动:" +
// port);
if (jTextPane != null) {
String text = jTextPane.getText();
text = "\r\n" + "----德塔VPCS数据库服务器端口启动:" + port + text;
text = "\r\n" + "...";
jTextPane.setText(text);
}
if (null != dbConfigPath && !dbConfigPath.isEmpty()) {
dbConfigPath.replace("\\", "/"); // 支持 linux reg 格式
DetaUtil.IV_DB(dbConfigPath);
} else {
dbConfigPath = ConfigIndex.detaDBPath;
DetaUtil.IV_DB(dbConfigPath);
}
S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器DMA确认:成功!");
if (jTextPane != null) {
String text = jTextPane.getText();
text = "\r\n" + "----德塔VPCS数据库服务器DMA确认:成功!" + text;
text = "\r\n" + "...";
jTextPane.setText(text);
}
RequestFilter_C.IV_BlockList();
S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器IP过滤服务启动:成功!");
if (jTextPane != null) {
String text = jTextPane.getText();
text = "\r\n" + "----德塔VPCS数据库服务器IP过滤服务启动:成功!" + text;
text = "\r\n" + "...";
jTextPane.setText(text);
}
DetaDBBufferCache_M.reflection();
// 20230106-S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器启动整库过程映射服务:成功!");
if (jTextPane != null) {
String text = jTextPane.getText();
text = "\r\n" + "----德塔VPCS数据库服务器启动整库过程映射服务:成功!" + text;
text = "\r\n" + "...";
jTextPane.setText(text);
}
DatabaseLog_H.I_BinLog_H();
// 20230106-S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器启动整库过程映射服务:成功!");
```

```

        if (jTextPane != null) {

String text = jTextPane.getText();

text = "\r\n" + "----德塔VPCS数据库服务器启动整库过程映射服务:成功!" + text;

text = "\r\n" + "...";

jTextPane.setText(text);

        }

        // BootBackup.bootBackupByUsingGzip(Cache_M

        // .getCacheInfo("LogPath").getValue().toString()+"/zipCover");

        // UnZip.unZipWithPath("C:/DetaLog/zipCover/zip_1549583065203.zip"

        // , "C:/DetaLog/zipCover/cover");

    } catch (Exception e) {

        e.printStackTrace();

    }

}

@SuppressWarnings("unused")

private void haoHiYooFaker(SleeperHall sleeper_H) {

    sleeper_H.callSkivvy();

}

@SuppressWarnings("unused")

public void IV_Server(App app) throws InterruptedException, IOException {

    dbConfigPath = app.app_S.dbConfigPath;

    // 20230106-S_logger.Log.logger.info(""" + "----DETA VPCS--2.0");

    // 20230106-S_logger.Log.logger.info(""" + "----Author: 罗瑶光");

    // 20230106-S_logger.Log.logger.info(""" + "----浏阳德塔软件开发有限公司开源项目");

    if (app.app_S.appConfig.SectionJPanel.jTextPane != null) {

        String text = app.app_S.appConfig.SectionJPanel.jTextPane.getText();

        text = "\r\n" + "载入德塔 PLSQL数据库 映射添加完毕!" + text;

        text = "\r\n" + "----Author: 罗瑶光" + text;

        text = "\r\n" + "----浏阳德塔软件开发有限公司开源项目" + text;

        text = "\r\n" + "..." + text;

        app.app_S.appConfig.SectionJPanel.jTextPane.setText(text);

    }

    Time_P time_P = new Time_P();

    time_P.begin();

    SleeperHall sleeper_H = new SleeperHall();

    IV_Service(app.app_S.appConfig.SectionJPanel.jTextPane,

app.app_S.databasePortTxt);

    time_P.end();

    if (app.app_S.appConfig.SectionJPanel.jTextPane != null) {

        String text = app.app_S.appConfig.SectionJPanel.jTextPane.getText();

        text = "\r\n" + "----德塔VPCS数据库服务器启动一切正常-总耗时:" + time_P.duration()

+ "毫秒" + text;

        text = "\r\n" + "..." + text;

        app.app_S.appConfig.SectionJPanel.jTextPane.setText(text);

    }

    // 20230106-S_logger.Log.logger.info(""" + "----德塔VPCS数据库服务器启动一切正常-总耗时:"

// + time_P.duration() + "毫秒");

```

```

app.app_S.ready = true;
App appReg = app;
System.out.flush();
S_logger.Log.logger.info("'" + "----DETA HVPCS--1.9070");
System.out.flush();
S_logger.Log.logger.info("'" + "----Author: 罗瑶光");
System.out.flush();
S_logger.Log.logger.info("'" + "----浏阳德塔软件开发有限公司开源项目");
TimeProcess timeProcess = new TimeProcess();
timeProcess.begin();
SleeperHall sleeperHall = new SleeperHall();
port = 8000;
System.out.flush();
S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器端口启动:" + port);
// DetaUtil.initDB();
System.out.flush();
S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器DMA确认:成功!");
S_logger.Log.logger.info("'" + "----初始大文件:成功!");
StableBlob.initBlobLimit();
S_logger.Log.logger.info("'" + "----初始固定数据:成功!");
StableSets.intListPartBytes();
S_logger.Log.logger.info("'" + "----初始http数据:成功!");
StableSets.initHttpContentTypeMap();
timeProcess.end();
System.out.flush();
S_logger.Log.logger.info("'" +
"----德塔VPCS前端服务器启动一切正常-总耗时:" + timeProcess.duration() + "毫秒");
// ServerSocket server = new ServerSocket(port);
long clearTime = System.currentTimeMillis();
TimeHall timeHall = new TimeHall();
timeHall.registerServer(server);
timeHall.registerSleeperHall(sleeperHall);
timeHall.start();
/*
 * S2189 later
 */
Here: while (true) { // later do 6 dead id filter
    if (sleeper_H.getThreadsCount() > 2) {
sleeper_H.callSkivvy();
Thread.sleep(3000);
continue Here;
    }
    if (0 >= timeHall.getMaxLimist()) {
System.out.flush();
S_logger.Log.logger.info("'" + sleeper_H.getThreadsCount());
timeHall.resetLimists();// 20250523
continue Here;

```

```

    }

    Sleeper_X sleeper = new Sleeper_X();
    if (!server.isClosed()) {
        sleeper.pillows.vPCSResponse.socket = server.accept();
    }
    if (!server.isClosed()
        && null != sleeper.pillows.vPCSResponse.socket) {
        sleeper_H.addExecSleeper(sleeper.hashCode(), sleeper);
        sleeper.start();
    }
}

// private App app;
// static {
////properties= new Properties();
////try {
////    properties.load(new FileInputStream
////        (new File("src/main/resources/property.proterties")));
////    //20230106-S_logger.Log.logger.info("" + "----德塔VPCS数据库服务器资源载入:成功!");
////}catch (IOException e) {
////    e.printStackTrace();
////}
// }
//138
//while (true) {
//if (sleeper_H.getThreadsCount() < S_Web.SLEEPERS_RANGE) {
// VPCSRequest vPCSRequest = new VPCSRequest();
// VPCSResponse vPCSResponse = new VPCSResponse();
// Sleeper_X sleeper = new Sleeper_X();
// sleeper.goToBedroom(S_HTTP.PORT_DATABASE, vPCSRequest, vPCSResponse);
// try {
//     sleeper.hugPillow(sleeper_H, server.accept(), sleeper.hashCode());
//     sleeper.sleeping();
// } catch (IOException e) {
//     e.printStackTrace();
// }
//} else {
// haoHiYooFaker(sleeper_H);
//}
//}

//Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/ServerInitCTest.java
//Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/ServerInitStandard.java
package C_A.OSI.AOP.MS.VPC.server;

import M_V.MS.VPC.PP.Time_P;

import S_A.SVQ.stable.S_Web;

import java.io.IOException;

import java.net.ServerSocket;

```

```
import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

import I_Q.hvpcs.boot.vpc.sleeper.SleeperHall;

//VPCS标准函数,准备之后所有服务器 走继承这个文件。

//罗瑶光 20200811

/*

* 个人著作权人,作者 罗瑶光,浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class ServerInit_Standard {

    private ServerSocket server;

    private int port;

    public void IV_Service(String frontendTxt, String serverName) {

        try {

            //port= Integer.parseInt(properties.getProperty(S_Data.TCP_PORT));

            port = Integer.valueOf(frontendTxt);

            //port= Config.detaVPCSDBPort;

            server = new ServerSocket(port);

            //20230106-S_logger.Log.logger.info(" " + "----德塔VPCS"+ serverName+ "端口启动:" + port);

            //20230106-S_logger.Log.logger.info(" " + "----德塔VPCS"+ serverName+ "DMA确认:成功!");

            RequestFilter_C.IV_BlockList();

            //20230106-S_logger.Log.logger.info(" " + "----德塔VPCS"+ serverName+ "IP过滤服务启动:成功!");

        } catch (Exception e) {

            e.printStackTrace();

        }

    }

    private void haoHiYooFaker(SleeperHall sleeper_H) {

        sleeper_H.callSkivvy();

    }

    public void IV_Server(String frontendTxt, String serverName) {

        //20230106-S_logger.Log.logger.info(" " + "----DETA VPCS--3.0");

        //20230106-S_logger.Log.logger.info(" " + "----Author: 罗瑶光");

        //20230106-S_logger.Log.logger.info(" " + "----浏阳德塔软件开发有限公司开源项目");

        Time_P time_P = new Time_P();

        time_P.begin();

        SleeperHall sleeper_H = new SleeperHall();

        IV_Service(frontendTxt, serverName);

        time_P.end();

        //20230106-S_logger.Log.logger.info(" " + "----德塔VPCS"+ serverName+ "启动一切正常-总耗时:"

        //+ time_P.duration()+ "毫秒");

        while (true) {

            if (sleeper_H.getThreadsCount() < S_Web.SLEEPERS_RANGE) {
```

```

VPCSRequest vPCSRequest = new VPCSRequest();
VPCSResponse vPCSResponse = new VPCSResponse();
ServerSleeper_Standard sleeper
    = new ServerSleeper_Standard(frontendTxt, vPCSRequest, vPCSResponse);
try {
    sleeper.hugPillow(sleeper_H, server.accept()
        , sleeper.hashCode());
    sleeper.start();
} catch (IOException e) {
    e.printStackTrace();
}
} else {
    haoHiYooFaker(sleeper_H);
}
}
}
}

//Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/ServerInit_StandardTest.java
//Trif_DNA-master/C-L-C/C-A/OSI/AOP/MS/VPC/server/ServerRestMap_Standard.java
package C_A.OSI.AOP.MS.VPC.server;

import exception.thread.DetaThread;
import M_V.MS.OP.SM.AOP.MEC.SIQ.cache.DetaCache_M;
import O_V.OSI.AOP.VPC.rest.VPC;
import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaFileInputStream;
import exception.file.O.DetaInputStreamReader;
import S_A.SVQ.stable.S_Pos;
import S_A.SVQ.stable.S_Web;
import U_A.PEU.P.zip.GzipUtil;
import java.io.BufferedWriter;
import java.io.ByteArrayOutputStream;
import java.io.DataOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.io.UnsupportedEncodingException;
import java.net.Socket;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import I_Q.hvpcs.boot.sets.VPCSRequest;
import I_Q.hvpcs.boot.sets.VPCSResponse;
import ME.VPC.M.app.App;

//这里出现了VPC的标识, 让-去小广告-看清楚, 我有很多方法来继承, 我就不用-去小广告-的那种
//. VPCS 的STABLE就可以解决

```

//是不是我写的. (详细描述下, -去小广告-当年php 多线程卡到爆, -去小广告-总被我念的头皮发麻

//最后用了java, 我overload了接口)

//去app 函数标识, 准备继承 用standard来区分。(当时是php 之前员工已经写好了

//, 我只是java翻译而已, 所以没用我的思维写,

//下面代码我的习惯是如下, 我喜欢类c的小片段继承。从不用overload)

// 罗瑶光

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class ServerRestMap_Standard {
    public static void main(String[] args) {
    }
    public static void IQ_Response(Socket socket) {
    }
    public static void returnResponse(Socket socket) {
    }
    public static void IQ_Response(VPCSRequest vPCSRequest,
                                   VPCSResponse vPCSResponse) {
    }
    public static void returnResponse(VPCSRequest vPCSRequest,
                                   VPCSResponse vPCSResponse) {
        vPCSResponse.getSleeperHall()
        .removeThreadById(vPCSResponse.getHashCode());
    }
    public static void P_Rest(VPCSRequest vPCSRequest,
                              VPCSResponse vPCSResponse, App NE) {
        // VPC属于子继承, 如果不用 override 来分配, 也有很多方法, 如osgi
        // , 当然, 我现在用最快map标识, 更爽.
        // indexVPCMapPillows.get(vPCSRequest.gettag())...;
        // 现在仅仅deta的网站处理服务器有web页, 养疗经app还没有涉及
        // , 因此 pillow tag 区分VPC 函数的rest map 设计优先级稍后
        try {
            String output = VPC.forward(vPCSRequest.getRequestLink(),
            vPCSRequest.getRequestValue(), NE);
            output = output.length() > 0 ? output : " ";
            PrintWriter printWriter = null;
            printWriter = new PrintWriter(
            new BufferedWriter(new OutputStreamWriter(
                vPCSResponse.getSocket().getOutputStream(),
                S_Web.CHARSET_UTF_8)),
            true);
```



```

        printWriter.println(S_Web.HEADER_HTTP_200_OK_DOUBLE_ENTER);

        output = output.charAt(S_Pos.INT_ZERO) == ""

? output.substring(S_Pos.INT_ONE, output.length())

: output;

        output = output.charAt(output.length() - S_Pos.INT_ONE) == ""

? output.substring(S_Pos.INT_ZERO,

        output.length() - S_Pos.INT_ONE)

: output;

        printWriter.println(output.replace("\\\\", "\\"));
        // 20230106-S_logger.Log.logger.info("" + "db:"+4);
        printWriter.flush();
        printWriter.close();
        vPCSResponse.getSleeperHall()

.removeThreadById(vPCSResponse.getSocket().hashCode());
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

public static void P_View(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
}

public static void P_Bytes(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
    List<byte[]> list;
    DataOutputStream dataOutputStream = null;
    try {
        dataOutputStream = new DataOutputStream(
vPCSResponse.getSocket().getOutputStream());
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    if (null != DetaCache_M

.getCacheOfBytesList(vPCSRequest.getRequestFilePath())) {
        list = DetaCache_M

.getCacheOfBytesList(vPCSRequest.getRequestFilePath());
    } else {
        FileInputStream fileInputStream = DetaFileInputStream

.E(new File(vPCSRequest.getRequestFilePath()));
        ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();
        byte[] byteArray = new byte[S_Web.BUFFER_RANGE_MAX];
        int lengthOfFile = S_Pos.INT_ZERO;
        list = new ArrayList<>();
        // 这段while函数思想来自 这篇文章:
        // https://blog.csdn.net/top_code/article/details/41042413
        // 非常轻松处理len的长度溢出。谢谢。
        while ((lengthOfFile = DetaFileInputStream.I(fileInputStream,
byteArray, S_Pos.INT_ZERO,

```

```

        S_Web.BUFFER_RANGE_MAX)) != S_Pos.INT_ERROR) {
    byteArrayOutputStream.write(byteArray, S_Pos.INT_ZERO,
        lengthOfFile);
    }
    DetaFileInputStream.D(fileInputStream);
    byte[] sniper = GzipUtil
        .compress(byteArrayOutputStream.toByteArray());
    try {
list.add(0, vPCSResponse.getResponseContentType()
        .getBytes(S_Web.CHARSET_UTF8));
list.add(0,
        (S_Web.HEADER_CONTENT_LENGTH + sniper.length
        + S_Web.STRING_SPACE_ENTER)
        .getBytes(S_Web.CHARSET_UTF8));
list.add(0, S_Web.HEADER_CACHE_CONTROL
        .getBytes(S_Web.CHARSET_UTF8));
list.add(0, S_Web.HEADER_CONTENT_ENCODING_GZIP
        .getBytes(S_Web.CHARSET_UTF8));
list.add(0, S_Web.HEADER_ACCEPT_RANGES_BYTES
        .getBytes(S_Web.CHARSET_UTF8));
list.add(0, S_Web.HEADER_HOST.getBytes(S_Web.CHARSET_UTF8));
list.add(0,
        S_Web.HEADER_HTTP_200_OK.getBytes(S_Web.CHARSET_UTF8));
    } catch (UnsupportedEncodingException e) {
throw new RuntimeException(e);
    }
    if (null != sniper && sniper.length > S_Pos.INT_ZERO) {
list.add(sniper);
    }
    DetaCache_M.putCacheOfBytesList(vPCSRequest.getRequestFilePath(),
list);
    }
    try {
        Iterator<byte[]> iterator = list.iterator();
        while (iterator.hasNext()) {
dataOutputStream.write(iterator.next());
        }
        dataOutputStream.flush();
        dataOutputStream.close();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
@SuppressWarnings("resource")
public static void P_Buffer(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
    String builderToString;

```

```

        if (null != DetaCache_M
.getCacheOfString(vPCSRequest.getRequestFilePath())) {
            builderToString = DetaCache_M
.getCacheOfString(vPCSRequest.getRequestFilePath());
        } else {
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.append(S_Web.HEADER_HTTP_200_OK);
            stringBuilder.append(S_Web.HEADER_HOST);
            stringBuilder.append(S_Web.HEADER_CACHE_CONTROL);
            stringBuilder.append(vPCSResponse.getResponseContentType());
            FileInputStream fileInputStream = DetaFileInputStream
.E(new File(vPCSRequest.getRequestFilePath()));
            InputStreamReader inputStreamReader = DetaInputStreamReader
.E(fileInputStream, vPCSRequest.getRequestFileCode());
            DetabufferedReader DetabufferedReader = new DetabufferedReader(
inputStreamReader);
            String line = null;
            while ((line = DetabufferedReader.readDetaLine()) != null) {
stringBuilder.append(line);
            }
            DetabufferedReader.closeDeta();
            builderToString = stringBuilder.toString();
            DetaCache_M.putCacheOfString(vPCSRequest.getRequestFilePath(),
builderToString);
        }
        BufferedWriter bufferedWriter = null;
        try {
            bufferedWriter = new BufferedWriter(new OutputStreamWriter(
vPCSResponse.getSocket().getOutputStream(),
vPCSRequest.getRequestFileCode()));
            bufferedWriter.write(builderToString);
            bufferedWriter.flush();
            bufferedWriter.close();
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    public static void P_BufferBytes(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
        StringBuilder stringBuilder = new StringBuilder();
        stringBuilder.append(S_Web.HEADER_HTTP_200_OK);
        stringBuilder.append(S_Web.HEADER_HOST);
        stringBuilder.append(S_Web.HEADER_CACHE_CONTROL);
        stringBuilder.append(S_Web.HEADER_CONTENT_ENCODING_GZIP);
        stringBuilder.append(vPCSResponse.getResponseContentType());
        String builderToString = stringBuilder.toString();
        String contentBuilderToString;

```

```

        try {
            if (null != DetaCache_M
                .getCacheOfString(vPCSRequest.getRequestFilePath())) {
                contentBuilderToString = DetaCache_M
                    .getCacheOfString(vPCSRequest.getRequestFilePath());
            } else {
                StringBuilder contentBuilder = new StringBuilder();
                FileInputStream fileInputStream = DetaFileInputStream
                    .E(new File(vPCSRequest.getRequestFilePath()));
                int lengthOfFile = S_Pos.INT_ZERO;
                byte[] byteArray = new byte[S_Web.BUFFER_RANGE_MAX];
                while ((lengthOfFile = DetaFileInputStream.I(fileInputStream,
                    byteArray)) != S_Pos.INT_ERROR) {
                    contentBuilder.append(new String(byteArray, S_Pos.INT_ZERO,
                        lengthOfFile, S_Web.CHARSET_UTF_8));
                }
                fileInputStream.close();
                contentBuilderToString = contentBuilder.toString();
                DetaCache_M.putCacheOfString(vPCSRequest.getRequestFilePath(),
                    contentBuilderToString);
            }
            DataOutputStream dataOutputStream = new DataOutputStream(
                vPCSResponse.getSocket().getOutputStream());
            dataOutputStream
                .write(builderToString.getBytes(S_Web.CHARSET_UTF8));
            dataOutputStream.write(GzipUtil.compress(
                contentBuilderToString.getBytes(S_Web.CHARSET_UTF8)));
            dataOutputStream.flush();
            dataOutputStream.close();
        } catch (UnsupportedEncodingException e) {
            throw new RuntimeException(e);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    public static void P_BytesWithoutZip(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
        List<byte[]> list;
        DataOutputStream dataOutputStream = null;
        try {
            dataOutputStream = new DataOutputStream(
                vPCSResponse.getSocket().getOutputStream());
            if (DetaCache_M.getCacheOfBytesList(
                vPCSRequest.getRequestFilePath()) != null) {
                list = DetaCache_M
                    .getCacheOfBytesList(vPCSRequest.getRequestFilePath());
            } else {

```

```
FileInputStream fileInputStream = new FileInputStream(  
    new File(vPCSRequest.getRequestFilePath()));  
  
ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();  
byte[] byteArray = new byte[1024];  
int len = 0;  
list = new ArrayList<>();  
// 下面三行while函数思想来自  
// 这篇文章: https://blog.csdn.net/top\_code/article/details/41042413  
// 非常轻松处理len的长度溢出。谢谢。  
while ((len = fileInputStream.read(byteArray, 0, 1024)) != -1) {  
    byteArrayOutputStream.write(byteArray, 0, len);  
}  
fileInputStream.close();  
byte[] sniper = byteArrayOutputStream.toByteArray();  
list.add(0, vPCSResponse.getResponseContentType()  
    .getBytes(S_Web.CHARSET_UTF8));  
list.add(0,  
    (S_Web.HEADER_CONTENT_LENGTH + sniper.length + " \n")  
    .getBytes(S_Web.CHARSET_UTF8));  
list.add(0, (S_Web.HEADER_CACHE_CONTROL)  
    .getBytes(S_Web.CHARSET_UTF8));  
list.add(0, S_Web.HEADER_ACCEPT_RANGES_BYTES  
    .getBytes(S_Web.CHARSET_UTF8));  
list.add(0, S_Web.HEADER_HOST.getBytes(S_Web.CHARSET_UTF8));  
list.add(0,  
    S_Web.HEADER_HTTP_200_OK.getBytes(S_Web.CHARSET_UTF8));  
list.add(sniper);  
DetaCache_M.putCacheOfBytesList(  
    vPCSRequest.getRequestFilePath(), list);  
}  
Iterator<byte[]> iterator = list.iterator();  
while (iterator.hasNext()) {  
byte[] bytes = null;  
int i = 0;  
try {  
    bytes = iterator.next();  
    if (bytes.length > 10000) {  
        int last = bytes.length % 10000;  
        for (i = 0; i < bytes.length - 10000; i += 10000) {  
            byte[] serparBytes = new byte[10000];  
            for (int j = 0; j < 10000; j++) {  
serparBytes[j] = bytes[i + j];  
            }  
            DetaThread.sleepDeta(250);  
            dataOutputStream.write(serparBytes);  
            dataOutputStream.flush();  
        }  
    }  
}
```

```

        byte[] serparBytes = new byte[last];

        // i=10000;

        for (int j = 0; j < last; j++) {

            serparBytes[j] = bytes[i + j];

        }

        dataOutputStream.write(serparBytes);

    } else {

        dataOutputStream.write(bytes);

    }

} catch (Exception e) {

    // 20230106-System.out.print(i);

    // 20230106-System.out.print(bytes.length);

    // 20230106-System.out.print(vPCSRequest.getRequestLink());

    // 20230106-System.out.print(vPCSRequest.getRequestFilePath());

    e.printStackTrace();

}

}

dataOutputStream.flush();

dataOutputStream.close();

} catch (IOException e) {

    throw new RuntimeException(e);

}

}

}

//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerRestMap_StandardTest.java
//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerSleepor_Standard.java
package C_A.OSI.AOP.MS.VPC.server;

import java.net.Socket;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

import I_Q.hvpcs.boot.vpc.sleeper.SleeperHall;

import I_Q.hvpcs.boot.vpc.sleeper.Sleeper_X;

import ME.VPC.M.app.App;

//这是标准的sleep函数,我稍后会设计 frontend sleeper, backend sleeper

//, cache sleeper, database sleeper 继承它, 避免循环嵌套。

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class ServerSleepor_Standard extends Thread implements Runnable {

    public App NE;

    public VPCSRequest vPCSRequest;

```

```
public VPCSResponse vPCSResponse;

public ServerSleeper_Standard() {
}

public ServerSleeper_Standard(String port, VPCSRequest vPCSRequest,
    VPCSResponse vPCSResponse) {
    this.vPCSRequest = vPCSRequest;
    vPCSRequest.I_RequestPort(port);
    this.vPCSResponse = vPCSResponse;
    vPCSResponse.setHashCode(this.hashCode());
}

// 合并
public ServerSleeper_Standard(VPCSRequest vPCSRequest,
    VPCSResponse vPCSResponse) {
    this.vPCSRequest = vPCSRequest;
    this.vPCSResponse = vPCSResponse;
    vPCSResponse.setHashCode(this.hashCode());
}

// public void run() {
//RequestRecord_C.requestIpRecoder(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//    return;
//}
//RequestRecord_C.requestLinkRecoder(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//    return;
//}
//RequestFilter_C.requestIpFilter(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//    return;
//}
//RequestFilter_C.requestLinkFilter(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//    return;
//}
//RequestFix_C.requestIpFix(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//    return;
//}
//RequestFix_C.requestLinkFix(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//    return;
//}
//ServerForward_Standard.IQ_ForwardType(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//    return;
//}
//ServerForward_Standard.forwardToRestMap(vPCSRequest, vPCSResponse);
```

```

//if (vPCSResponse.getSocket().isClosed()) {
//
//    return;
//}
//ServerRestMap_Standard.IQ_Response(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//
//    return;
//}
//ServerRestMap_Standard.returnResponse(vPCSRequest, vPCSResponse);
//if (vPCSResponse.getSocket().isClosed()) {
//
//    return;
//}
//
//    }
//
//    public void hugPillow(SleeperHall sleeper_H, Socket accept, int hashCode) {
//
//        Sleeper_X sleeper_X = (Sleeper_X) this;
//        sleeper_H.addExecSleeper(hashCode, sleeper_X);
//        vPCSResponse.setSocket(accept);
//        vPCSResponse.setSleeperHall(sleeper_H);
//
//    }
//
//}
//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerSleeper_StandardTest.java
//Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerVPC_Standard.java
package C_A.OSI.AOP.MS.VPC.server;

import C_A.OSI.AOP.MS.VPC.C.DBCategory_C;

import C_A.OSI.AOP.VPC.C.Config_C;

import C_A.OSI.AOP.VPC.C.D_C;

import C_A.OSI.AOP.VPC.C.IU_C;

import C_A.OSI.AOP.VPC.C.Q_C;

import C_A.OSI.AOP.VPC.C.U_C;

import DSU.V.VtoV;

import E_A.OSI.AOP.VPC.PP.port_E.RestAskPort_E;

import E_A.OSI.AOP.VPC.PP.port_E.RestLoginPort_E;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

import java.io.File;

//作者 罗瑶光

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class ServerVPC_Standard {

    @SuppressWarnings("unchecked")

    public static String forward(String string, IMV_SQI data, App NE) {

//if(!DNAAuth.DNAAuthStatusCheckEmailAndPassword(app, string, data)) {

```



```
//      return "";
//}

// controller
if (string.contains("/select")) {
    return Q_C.exec(string, data);
}
if (string.contains("/setDB")) {
    return Config_C.exec(string, data);
}
if (string.contains("/insert")) {
    return IU_C.exec(string, data);
}
if (string.contains("/delete")) {
    return D_C.exec(string, data);
}
if (string.contains("/update")) {
    return U_C.exec(string, data);
}
if (string.contains("DBCategory")) {
    return DBCategory_C.exec(string, data);
}
// restMap // 准备改成1次进入由原来的 前端 后端数据库, 变成前端 数据库。 模式为前端加cache
// , 后端加数据库。 罗瑶光
if (string.equalsIgnoreCase("/login")) {
    return VtoV.ObjectToJsonString(RestLoginPort_E.login(
data.getString("uEmail"), data.getString("uPassword")));
}
if (string.equalsIgnoreCase("/find")) {
    return VtoV.ObjectToJsonString(
RestLoginPort_E.find(data.getString("uEmail")));
}
if (string.equalsIgnoreCase("/logout")) {
    return VtoV.ObjectToJsonString(RestLoginPort_E.logout(
data.getString("uEmail"), data.getString("uToken")));
}

//if(string.equalsIgnoreCase("/register")){
//      return VtoV.ObjectToJsonString(RestLoginPort_E.register(data.getString("uEmail")
//      , data.getString("uEmailEnsure"), data.getString("uName")
//      , data.getString("uPassword"), data.getString("uPassWDEnsure")
//      , data.getString("uAddress"), data.getString("uPhone")
//      , data.getString("uWeChat"), data.getString("uQq")
//      , data.getString("uAge"), data.getString("uSex")));
//}

if (string.equalsIgnoreCase("/register")) {
    return VtoV.ObjectToJsonString(RestLoginPort_E.register(
data.getString("uEmail"), data.getString("uName"),
data.getString("uPassword")));
}
```

```
}  
if (string.equalsIgnoreCase("/change")) {  
    return VtoV.ObjectToJsonString(RestLoginPort_E.change(  
data.getString("uEmail"), data.getString("uChange"),  
data.getString("uChangeEnsure"), data.getString("uToken"),  
data.getString("uPassword")));  
}  
// if(string.equalsIgnoreCase("/checkStatus")){  
// return  
// VtoV.ObjectToJsonString(RestLoginPortImpl.checkStatus(data.getString("token")));  
// }  
if (string.equalsIgnoreCase("/ask")) {  
    return VtoV.ObjectToJsonString(RestAskPort_E.ask(  
data.getString("ip"), data.getString("token"),  
data.getString("message"), data.getString("pointIp")));  
}  
if (string.equalsIgnoreCase("/feedBack")) {  
    return VtoV.ObjectToJsonString(RestAskPort_E.feedBack(  
data.getString("ip"), data.getString("token"),  
data.getString("pointIp")));  
}  
if (string.equalsIgnoreCase("/getAskers")) {  
    return VtoV.ObjectToJsonString(  
RestAskPort_E.getAskers(data.getString("token")));  
}  
if (string.equalsIgnoreCase("/dataWS")) {  
    return RestAskPort_E.dataWS(data.getString("message"), NE);  
}  
if (string.equalsIgnoreCase("/dataCX")) {  
    return RestAskPort_E.dataCX(data.getString("message"));  
}  
if (string.equalsIgnoreCase("/dataCY")) {  
    return RestAskPort_E.dataCY(data.getString("message"));  
}  
if (string.equalsIgnoreCase("/dataXL")) {  
    return RestAskPort_E.dataXL(data.getString("message"));  
}  
if (string.equalsIgnoreCase("/dataRN")) {  
    return RestAskPort_E.dataRN(data.getString("message"));  
}  
if (string.equalsIgnoreCase("/dataCG")) {  
    return RestAskPort_E.dataCG(data.getString("message"));  
}  
if (string.equalsIgnoreCase("/dataCJ")) {  
    return RestAskPort_E.dataCJ(data.getString("message"));  
}  
if (string.equalsIgnoreCase("/dataCL")) {
```

```
        return RestAskPort_E.dataCL(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataXX")) {
        return RestAskPort_E.dataXX(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataHF")) {
        return RestAskPort_E.dataHF(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataCP")) {
        return RestAskPort_E.dataCP(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataZF")) {
        return RestAskPort_E.dataZF(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataZY")) {
        return RestAskPort_E.dataZY(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataXY")) {
        return RestAskPort_E.dataXY(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataYT")) {
        return RestAskPort_E.dataYT(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataZT")) {
        return RestAskPort_E.dataZT(data.getString("message"));
    }
    if (string.equalsIgnoreCase("/dataXT")) {
        return RestAskPort_E.dataXT(data.getString("message"));
    }
    return "";
}

public static String getCode(String filePath) {
    if (filePath.contains(".html") || filePath.contains(".js")) {
        return "UTF-8";
    }
    return "UTF8";
}

public static String getFilePath(String string) {
    String root = new File("src/main/resources/static").getAbsolutePath()
.replace("\\", "/");
    if (null == string || string.equalsIgnoreCase(""))
|| string.equalsIgnoreCase("/") {
        string = "/index.html";
    }
    return root + string;
}
}
```

```
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/ServerVPC_StandardTest.java
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/VPCSRequest.java
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/VPCSRequestTest.java
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/VPCSResponse.java
/Trif_DNA-master/C_L_C/C_A/OSI/AOP/MS/VPC/server/VPCSResponseTest.java
1-3-/Trif_DNA-master/C_L_D
```

DSU.constants

/Trif_DNA-master/C_L_D/DSU/constants/DetaDBConstant.java

```
package DSU.constants;
```

```
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
```

```
public interface DetaDBConstant {

    // common
    final static int DIGIT_ONE = 1;
    final static int DIGIT_60000 = 60000;
    // Token expired.
    final static String ERRORCODE_ER1001 = "ER1001";
    // rest tire
    final static String REST_JSON_CONFIG = "application/json;charset=utf-8";
    final static String REST_LOGIN_LOGIN = "/login";
    final static String REST_LOGIN_LOGOUT = "/logout";
    final static String REST_LOGIN_REGISTER = "/register";
    final static String REST_LOGIN_FIND = "/find";
    final static String REST_LOGIN_CHANGE = "/change";
    final static String DB_LOGIN_U_NAME = "uName";
    final static String DB_LOGIN_U_CHANGE = "uChange";
    final static String DB_LOGIN_U_CHANGE_ENSURE = "uChangeEnsure";
    final static String DB_LOGIN_U_AGE = "uAge";
    final static String DB_LOGIN_U_EMAIL = "uEmail";
    final static String DB_LOGIN_U_EMAIL_ENSURE = "uEmailEnsure";
    final static String DB_LOGIN_U_ID = "uId";
    final static String DB_LOGIN_U_SEX = "uSex";
    final static String DB_LOGIN_U_PHONE = "uPhone";
    final static String DB_LOGIN_U_ADDRESS = "uAddress";
    final static String DB_LOGIN_U_WECHAT = "uWeChat";
    final static String DB_LOGIN_U_PSWD = "uPassword";
    final static String DB_LOGIN_U_PSWD_ENSURE = "uPasswordEnsure";
    final static String DB_LOGIN_U_PSTOKEN = "uToken";
    final static String DB_LOGIN_U_CLASS = "uClass";
    final static String DB_LOGIN_U_QQ = "uQq";
    // service tire
```

```

// scheduler tire
// dao tire
// database tire
// condition tire
}

DSU.exceptions

/Trif_DNA-master/C_L_D/DSU/exceptions/DetaDBException.java
package DSU.exceptions;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
@SuppressWarnings("serial")
public class DetaDBException extends RuntimeException {

    private String errorId;
    private String errorMessage;
    public String getErrorId() {
        return errorId;
    }
    public void I_ErrorId(String errorId) {
        this.errorId= errorId;
    }
    public String getErrorMessage() {
        return errorMessage;
    }
    public void I_ErrorMessage(String errorMessage) {
        this.errorMessage= errorMessage;
    }
    public DetaDBException() {
        super();
    }
    public DetaDBException(String errorId, String errorMessage) {
        super(errorMessage);
        this.errorId= errorId;
        this.errorMessage= errorMessage;
    }
}

/Trif_DNA-master/C_L_D/DSU/exceptions/DetaDBExceptionTest.java
/Trif_DNA-master/C_L_D/DSU/exceptions/ErrorCodeEnum.java
package DSU.exceptions;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳

```

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```
public enum ErrorCodeEnum { //later..
```

```
    ER1001("error."),  
    ER1002("invalid key.");  
    private String msg;  
    private ErrorCodeEnum(String msg) {  
        this.msg= msg;  
    }  
  
    public String getMsg() {  
        return msg;  
    }  
    public void I_Msg(String msg) {  
        this.msg= msg;  
    }  
}
```

```
/Trif_DNA-master/C_L_D/DSU/exceptions/RequestLimitException.java
```

DSU.V

```
**/Trif_DNA-master/C_L_D/DSU/V/VtoV.java  
package DSU.V;
```

```
import S_A.pheromone.IMV_SQI;  
import com.google.gson.Gson;  
import com.google.gson.reflect.TypeToken;  
import org.json.JSONObject;  
import java.lang.reflect.Type;  
/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```
public class VtoV {  
    public static IMV_SQI JsonObjectToMap(JSONObject response) {  
        Gson gson = new Gson();  
        Type type = new TypeToken<IMV_SQI>() {  
        }.getType();  
        IMV_SQI responseMap = gson.fromJson(response.toString(), type);  
        return responseMap;  
    }  
}
```

```

    }

    public static String ObjectToJsonString(Object response) {

        Gson gson = new Gson();

        return gson.toJson(response);

    }

}

```

//VPCS VtoV整合在这里是为了避嫌.

//逐步剔除著作权范围代码, 大家可以看到, 这些注释掉的函数已经说明了我的整个工程 就

//没有任何XML的应用. 具体见Deta 的 data swap包.

//除了两个 JsonObjectToMap 和 ObjectToJsonString(用到 谷歌 的Guson 官方开源

//DEMO 原函数) 就没有用到过-去小广告- 时候的VToV(保留着词汇是因为VtoV是我罗瑶光命名的。)函数,

//2019~2020 data swap 著作权申请文档很清晰了.

//罗瑶光

```
//      public JSONObject XmlToJsonObject(String response ){
```

-----已经删除

```
//}
```

```
//      public String MapToJsonString(IMV_SQI response ){
```

-----已经删除

```
//      }
```

```
//      public String ListToJsonString(List<Object> response ){
```

-----已经删除

```
//
```

```
//      public IMV_SQI XmlToMap(String response){
```

-----已经删除

```
//      }
```

```
//      public String MapToXml(IMV_SQI response){
```

-----已经删除

```
//      }
```

```
//      public List<Object> JsonArrayToList(JSONArray job) {
```

-----已经删除

```
//}
```

```
//return output;
```

```
//      }
```

```
//      public Object StringListToJsonString(List<String> sets) {
```

-----已经删除

```
//      }
```

/Trif_DNA-master/C_L_D/DSU/V/VtoVTest.java

exceptions

/Trif_DNA-master/C_L_D/exceptions/DetaDBException.java

/Trif_DNA-master/C_L_D/exceptions/ErrorCodeEnum.java

/Trif_DNA-master/C_L_D/exceptions/RequestLimitException.java

package exceptions;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

@SuppressWarnings("serial")

```
public class RequestLimitException extends Exception {
```

```
    public RequestLimitException() {
```

```
        super("HTTP Request Limit...");
```

```
    }
```

```
    public RequestLimitException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

1-4-/Trif_DNA-master/C_L_E-执行-分词-数据库-ETL-界面UI-卷积-网络传输

E_A.ME.analysis.E

```
/Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE_M.java
```

```
package E_A.ME.analysis.E;
```

```
import C_A.ME.analysis.C.A;
```

```
import C_A.ME.nero.C.NERO_C_OneTime;
```

```
import C_A.OCLSVQ.MPC.fhmm.C.FHMMList_indexAll;
```

```
import E.Nlp_CE_X_S;
```

```
import P.Pos_X_P;
```

```
import ME.VPC.M.app.App;
```

```
import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;
```

```
import S_A.SVQ.stable.S_Pos;
```

```
import java.util.Iterator;
```

```
import java.util.LinkedList;
```

```
import java.util.List;
```

```
import java.util.Map;
```

```
/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class AE_M extends AS implements A {
```

```
    public FHMMList_indexAll fhMMList;// gjs list later..
```

```
    public Nlp_CE_X_S nlp_C;// later fix.. //later getter setter
```

```
    public NERO_C_OneTime nero_C;
```

```
    public Pos_X_P pos_C;
```

```
    public Map<Long, FMHMMNode> forestRoots;
```

```
    public Map<Long, FMHMMNode>[] forestsRoots;
```

```
    public void I_FixWords(int charPosition, StringBuilder inputString,
```



```

        StringBuilder[] fixWords) {
    fixWords[S_Pos.INT_ONE].delete(S_Pos.INT_ZERO,
fixWords[S_Pos.INT_ONE].length());
    if (charPosition + S_Pos.INT_EIGHT < inputString.length()) {
        fixWords[S_Pos.INT_ONE].append(
inputString.substring(charPosition + S_Pos.INT_THREE,
charPosition + S_Pos.INT_EIGHT));
        return;
    }
    fixWords[S_Pos.INT_ONE].append(inputString.substring(
charPosition + S_Pos.INT_THREE, inputString.length()));
}
// 82行 通过 S AE的 一次新陈代谢分离
@SuppressWarnings("unused")
private void doFind() {
    if (S_Pos.INT_ONE == find) {
        find = S_Pos.INT_ZERO;
        Iterator<String> it = fHMMList.englishStringToWordsList(
fixWords[S_Pos.INT_ZERO].toString()).iterator();
        StringBuilder number = new StringBuilder();
        while (it.hasNext()) {
String temp = it.next();
if (S_Pos.NUMBERS.contains(temp)) {
    number.append(temp);
} else {
    if (number.length() > 0) {
        outputList.add(number.toString());
        number.delete(0, number.length());
    }
    outputList.add(temp);
}
        }
        if (number.length() > 0) {
outputList.add(number.toString());
number.delete(0, number.length());
        }
        fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
fixWords[S_Pos.INT_ZERO].length());
    }
}

public List<String> parserString(String inputString) {
    mixedString = new StringBuilder(inputString);
    List<String> outputList = new LinkedList<>();
    int inputLength = mixedString.length();
    int forestDepth = S_Pos.INT_ZERO;
    int countLength;
    StringBuilder[] fixWords = new StringBuilder[S_Pos.INT_TWO];

```

```

fixWords[S_Pos.INT_ZERO] = new StringBuilder();
fixWords[S_Pos.INT_ONE] = new StringBuilder();
StringBuilder stringBuilder = new StringBuilder();
int find = S_Pos.INT_ZERO;
S_Pos.INT_ZERO Here: for (int charPosition = S_Pos.INT_ZERO; charPosition < inputLength; charPosition += (countLength !=
? countLength
: S_Pos.INT_ONE)) {
    if (S_Pos.INT_ONE_TWO_EIGHT > mixedString.charAt(charPosition)) {
if (fixWords[S_Pos.INT_ZERO].length() > S_Pos.INT_ZERO) {
    if (fixWords[S_Pos.INT_ZERO].charAt(fixWords[S_Pos.INT_ZERO]
        .length()
        - S_Pos.INT_ONE) < S_Pos.INT_ONE_TWO_EIGHT) {
        fixWords[S_Pos.INT_ZERO]
.append(mixedString.charAt(charPosition));
        countLength = S_Pos.INT_ONE;
        find = S_Pos.INT_ONE;
        continue Here;
    }
    fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
        fixWords[S_Pos.INT_ZERO].length());
}
find = S_Pos.INT_ONE;
fixWords[S_Pos.INT_ZERO]
    .append(mixedString.charAt(charPosition));
countLength = S_Pos.INT_ONE;
continue Here;
    }
    if (find == S_Pos.INT_ONE) {
find = S_Pos.INT_ZERO;
outputList.add(fixWords[S_Pos.INT_ZERO].toString());
    }
    stringBuilder.delete(S_Pos.INT_ZERO, stringBuilder.length());
    stringBuilder = nero_C.getBinaryForestRecurWords(
stringBuilder.append(mixedString.charAt(charPosition)),
mixedString, charPosition, inputLength, forestRoots,
forestDepth, charPosition + S_Pos.INT_ONE);
    String countWordNode = stringBuilder.toString();
    int compare = countLength = countWordNode.length();
    if (compare == S_Pos.INT_ONE) {
outputList.add(countWordNode);
fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
    fixWords[S_Pos.INT_ZERO].length());
fixWords[S_Pos.INT_ZERO].append(countWordNode);
continue Here;
    }
    if (compare == S_Pos.INT_TWO) {
countLength = nlp_C.forTwoChar(countLength, outputList,

```

```

        stringBuilder, fixWords, charPosition, mixedString);

    continue Here;

    }

    if (compare == S_Pos.INT_THREE) {
I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C.ofThree(countLength, outputList,
        stringBuilder, fixWords, charPosition, mixedString);

    continue Here;

    }

    if (compare == S_Pos.INT_FOUR) {
I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C._E(countLength, outputList, stringBuilder,
        fixWords, charPosition, mixedString);
    }

    }

    return outputList;
}

public String[] parserEnglishString(String englishString) {
    String[] words = englishString
.replaceAll(S_Pos.NLP_SPASE_REP, S_Pos.SPACE_STRING)
.split(S_Pos.SPACE_STRING);

    if (S_Pos.INT_ZERO == words.length) {
        return new String[] { S_Pos.SPACE_STRING };
    }

    return words;
}

public void IV_(App NE) {
    IV_condition(1, NE);
}

public void IV_Mixed(App NE) {
    IV_condition(2, NE);
}

public void IV_All(App NE) {
    IV_condition(3, NE);
}

@SuppressWarnings("unchecked")
public void IV_condition(int indexType, App NE) {
    if (null != NE) {
        this.fHMMLList = NE.app_S.fHMMLListOneTime_E;
        pos_C = NE.app_S.pos_X_P;
        nlp_C = NE.app_S.nlp_CE_X_S;
        nero_C = NE.app_S.nERO_C_OneTime_E;
        emotionMap = NE.app_S.emotionMap_E;
        sensingTest = NE.app_S.sensingTest;
        quick6DLuoYaoguangSort = NE.app_S.quick6DLuoYaoguangSort3DMap_E;
    } else {
    }
}

```

```

        if (1 == indexType) {
            fHMMList.index();
        }
        if (2 == indexType) {
            fHMMList.indexMixed(NE);
        }
        if (3 == indexType) {
            // fHMMList.indexAll();
        }

        fHMMList.indexPosEnToCn(NE);
        fHMMList.indexPosEnToEn(NE);
        fHMMList.indexEnToCn(NE);
        fHMMList.indexCnToEn(NE);
        fHMMList.indexFullEnToCn();
        fHMMList.indexFullCnToEn();
        fHMMList.indexFullCnToPy();
        fHMMList.indexFullCnToKo();
        fHMMList.indexFullCnToJp();
        fHMMList.indexFullCnToTt();
        fHMMList.indexFullCnToRs();
        fHMMList.indexFullCnToAb();
        fHMMList.indexFullCnToFn();
        fHMMList.indexFullCnToGm();
        fHMMList.indexFullCnToSp();
        forestRoots = fHMMList.getMap();
        forestsRoots = fHMMList.getMaps();
        wordsForest = fHMMList.getPosCnToCn();
        wordsForests = fHMMList.getWordsForests();
        //
        this.fHMMList.flushRedundant();
    }
}

//emotionMap.IV_MotivationMap();
//emotionMap.IV_NegativeMap();
//emotionMap.IV_PositiveMap();
//emotionMap.IV_TrendingMap();
//emotionMap.IV_PredictionMap();

//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE_MTest.java
//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE_X.java
package E_A.ME.analysis.E;

import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;
import C_A.ME.analysis.C.A;
import C_A.ME.liner.C.Quick6DLuoYaoguangSort;
import C_A.ME.nero.C.NERO_C_OneTime;
import C_A.OCLSVQ.MPC.fhmm.C.FHMMList_indexAll;
import E.Nlp_CE_X_S;
import P.Pos_X_P;
import ME.VPC.M.app.App;

```

```

import P_V.PEQ.AMV.ECS.test.SensingTest;

import Q_V.QSD.VQS.MSO.Q_parser;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI;

import java.util.Iterator;

import java.util.LinkedList;

import java.util.List;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class AE_X extends AS implements A {
    public FHMMList_indexAll fHMMList;
    public Nlp_CE_X_S nlp_C;
    public NERO_C_OneTime nero_C;
    public Map<Long, IMV_SQI> wordsForests;
    public Quick6DLuoYaoguangSort quick6DLuoYaoguangSort;
    public Map<Long, FMHMMNode>[] forestsRoots;
    public Map<Long, FMHMMNode> forestRoots;
    public EmotionMap emotionMap;
    public SensingTest sensingTest;
    public Pos_X_P pos_C;//later
    public Q_parser _Q_parser;
    public List<String> parserMixedString(String inputString) {
        mixedString = new StringBuilder(inputString + S_Pos.SPACE_STRING_DISTINCTION);
        return parserMixedString(mixedString);
    }
    public List<String> parserMixedString(StringBuilder inputString) {
        mixedString = new StringBuilder(inputString + S_Pos.SPACE_STRING_DISTINCTION);
        inputLength = mixedString.length();
        outputList = new LinkedList<>();
        forestDepth = S_Pos.INT_ZERO;
        fixWords = new StringBuilder[S_Pos.INT_TWO];
        fixWords[S_Pos.INT_ZERO] = new StringBuilder();
        fixWords[S_Pos.INT_ONE] = new StringBuilder();
        stringBuilder = new StringBuilder();
        find = S_Pos.INT_ZERO;
        Here:
        for (charPosition = S_Pos.INT_ZERO; charPosition < inputLength;

```

```

charPosition += (countLength == S_Pos.INT_ZERO ? S_Pos.INT_ONE
    : countLength)) {
if (mixedString.charAt(charPosition) < S_Pos.INT_TEN_SOUTHANDS
    && charPosition < inputLength - S_Pos.INT_ONE) {
if (find == S_Pos.INT_ZERO) {
    fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO
        , fixWords[S_Pos.INT_ZERO].length());
}
fixWords[S_Pos.INT_ZERO].append(mixedString.charAt(charPosition));
countLength = S_Pos.INT_ONE;
find = S_Pos.INT_ONE;
continue Here;
}
if (S_Pos.INT_ONE == find) {
    find = S_Pos.INT_ZERO;
    Iterator<String> it = fHMMList.englishStringToWordsList(
        fixWords[S_Pos.INT_ZERO].toString()).iterator();
    StringBuilder number = new StringBuilder();
    while (it.hasNext()) {
        String temp = it.next();
        if (S_Pos.NUMBERS.contains(temp)) {
            number.append(temp);
        } else {
            if (number.length() > 0) {
                outputList.add(number.toString());
                number.delete(0, number.length());
            }
            outputList.add(temp);
        }
    }
    if (number.length() > 0) {
        outputList.add(number.toString());
        number.delete(0, number.length());
    }
    fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO
        , fixWords[S_Pos.INT_ZERO].length());
}
stringBuilder.delete(S_Pos.INT_ZERO, stringBuilder.length());
stringBuilder = nero_C.getBinaryForestRecurWords(stringBuilder
    .append(mixedString.charAt(charPosition)), mixedString
    , charPosition, inputLength, forestRoots, forestDepth
    , charPosition + S_Pos.INT_ONE);
String countWordNode = stringBuilder.toString();
int compare = countLength = countWordNode.length();
if (S_Pos.INT_ONE == compare) {
    outputList.add(countWordNode);
    fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO

```

```

        , fixWords[S_Pos.INT_ZERO].length());
    fixWords[S_Pos.INT_ZERO].append(countWordNode);
    continue Here;
}
if (S_Pos.INT_TWO == compare) {
    countLength = nlp_C.forTwoChar(countLength, outputList
        , stringBuilder, fixWords, charPosition, mixedString);
    continue Here;
}
if (S_Pos.INT_THREE == compare) {
    I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C.ofThree(countLength, outputList
        , stringBuilder, fixWords, charPosition, mixedString);
    continue Here;
}
if (S_Pos.INT_FOUR == compare) {
    I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C._E(countLength, outputList, stringBuilder
        , fixWords, charPosition, mixedString);
}
}
return outputList;
}

public List<String> parserString(String inputString) {
    mixedString = new StringBuilder(inputString);
    List<String> outputList = new LinkedList<>();
    int inputLength = mixedString.length();
    int forestDepth = S_Pos.INT_ZERO;
    int countLength;
    StringBuilder[] fixWords = new StringBuilder[S_Pos.INT_TWO];
    fixWords[S_Pos.INT_ZERO] = new StringBuilder();
    fixWords[S_Pos.INT_ONE] = new StringBuilder();
    StringBuilder stringBuilder = new StringBuilder();
    int find = S_Pos.INT_ZERO;
    Here:
    for (int charPosition = S_Pos.INT_ZERO; charPosition < inputLength
        ; charPosition += (countLength != S_Pos.INT_ZERO
            ? countLength : S_Pos.INT_ONE)) {
        if (S_Pos.INT_ONE_TWO_EIGHT > mixedString.charAt(charPosition)) {
            if (fixWords[S_Pos.INT_ZERO].length() > S_Pos.INT_ZERO) {
                if (fixWords[S_Pos.INT_ZERO].charAt(fixWords[S_Pos.INT_ZERO]
                    .length() - S_Pos.INT_ONE) < S_Pos.INT_ONE_TWO_EIGHT) {
                    fixWords[S_Pos.INT_ZERO].append(mixedString.charAt(charPosition));
                    countLength = S_Pos.INT_ONE;
                    find = S_Pos.INT_ONE;
                    continue Here;
                }
            }

```

```

        fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO
            , fixWords[S_Pos.INT_ZERO].length());
    }
    find = S_Pos.INT_ONE;
    fixWords[S_Pos.INT_ZERO].append(mixedString.charAt(charPosition));
    countLength = S_Pos.INT_ONE;
    continue Here;
}
if (find == S_Pos.INT_ONE) {
    find = S_Pos.INT_ZERO;
    outputList.add(fixWords[S_Pos.INT_ZERO].toString());
}
stringBuilder.delete(S_Pos.INT_ZERO, stringBuilder.length());
stringBuilder = nero_C.getBinaryForestRecurWords(stringBuilder
    .append(mixedString.charAt(charPosition)), mixedString
    , charPosition, inputLength, forestRoots, forestDepth
    , charPosition + S_Pos.INT_ONE);
String countWordNode = stringBuilder.toString();
int compare = countLength = countWordNode.length();
if (compare == S_Pos.INT_ONE) {
    outputList.add(countWordNode);
    fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO
        , fixWords[S_Pos.INT_ZERO].length());
    fixWords[S_Pos.INT_ZERO].append(countWordNode);
    continue Here;
}
if (compare == S_Pos.INT_TWO) {
    countLength = nlp_C.forTwoChar(countLength, outputList, stringBuilder
        , fixWords, charPosition, mixedString);
    continue Here;
}
if (compare == S_Pos.INT_THREE) {
    I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C.ofThree(countLength, outputList
        , stringBuilder, fixWords, charPosition, mixedString);
    continue Here;
}
if (compare == S_Pos.INT_FOUR) {
    I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C._E(countLength, outputList, stringBuilder
        , fixWords, charPosition, mixedString);
}
}
return outputList;
}
public void I_FixWords(int charPosition, StringBuilder inputString
    , StringBuilder[] fixWords) {

```

```

fixWords[S_Pos.INT_ONE].delete(S_Pos.INT_ZERO, fixWords[S_Pos.INT_ONE].length());

if (charPosition + S_Pos.INT_EIGHT < inputString.length()) {
    fixWords[S_Pos.INT_ONE].append(inputString.substring(charPosition
        + S_Pos.INT_THREE, charPosition + S_Pos.INT_EIGHT));
    return;
}

fixWords[S_Pos.INT_ONE].append(inputString.substring(charPosition
    + S_Pos.INT_THREE, inputString.length()));
}

public String[] parserEnglishString(String englishString) {
    String[] words = englishString.replaceAll(S_Pos.NLP_SPASE_REP
        , S_Pos.SPACE_STRING).split(S_Pos.SPACE_STRING);
    if (S_Pos.INT_ZERO == words.length) {
        return new String[] {S_Pos.SPACE_STRING};
    }
    return words;
}

public void IV_(App NE) {
    IV_condition(1, NE);
}

public void IV_Mixed(App NE) {
    IV_condition(2, NE);
}

public void IV_All(App NE) {
    IV_condition(3, NE);
}

@SuppressWarnings("unchecked")
    public void IV_condition(int indexType, App NE) {
        this.fHMMLList = NE.app_S.fMHMMLListOneTime_E;
        if (1 == indexType) {
            fHMMLList.index();
        }
        if (2 == indexType) {
            fHMMLList.indexMixed(NE);
        }
        if (3 == indexType) {
            //fHMMLList.indexAll();
        }

        fHMMLList.indexPosEnToCn(NE);
        fHMMLList.indexPosEnToEn(NE);
        fHMMLList.indexEnToCn(NE);
        fHMMLList.indexCnToEn(NE);
        fHMMLList.indexFullEnToCn();
        fHMMLList.indexFullCnToEn();
        fHMMLList.indexFullCnToPy();
        fHMMLList.indexFullCnToKo();
        fHMMLList.indexFullCnToJp();
    }

```

```

fHMMList.indexFullCnToTt();
fHMMList.indexFullCnToRs();
fHMMList.indexFullCnToAb();
nero_C = NE.app_S.nERO_C_OneTime_E;
nlp_C = NE.app_S.nlp_CE_X_S;
pos_C = NE.app_S.pos_X_P;
quick6DLuoYaoguangSort = NE.app_S.quick6DLuoYaoguangSort3DMap_E;
forestRoots = fHMMList.getMap();
forestsRoots = fHMMList.getMaps();
wordsForest = fHMMList.getPosCnToCn();
wordsForests = fHMMList.getWordsForests();
emotionMap = NE.app_S.emotionMap_E;
sensingTest = sensingTest;//trif
//
this.fHMMList.flushRedundant();
}
}
//emotionMap.IV_MotivationMap();
//emotionMap.IV_NegativeMap();
//emotionMap.IV_PositiveMap();
//emotionMap.IV_TrendingMap();
//emotionMap.IV_PredictionMap();
//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE_XTest.java
//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AE.java
package E_A.ME.analysis.E;

import C_A.ME.analysis.C.AC;

import S_A.SVQ.stable.S_Pos;

import java.util.Iterator;

import java.util.LinkedList;

import java.util.List;

/*
* 个人著作权人,作者 罗瑶光,浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
* */
//最近2年没人挑战我,我不能老闲着,那就继续自己挑战自己的模式,定个目标优化下。trif-
//countLength比较 是可以在子迭代层进行分解掉的,分词速度可以提升5%,既然提了,于是刚刚我就干脆分解掉了。
public class AE extends AE_M implements AC {
    public List<String> parserMixedString(String inputString) {
        mixedString = new StringBuilder(inputString);
        return parserMixedString(mixedString);
    }
}
/*
// for (charPosition = S_Pos.INT_ZERO; charPosition <

```

```

// inputLength
// ; charPosition += (countLength != S_Pos.INT_ZERO
// ? countLength: S_Pos.INT_ONE)) { //trif-
// countLength比较 是可以在子迭代层进行分解掉的，分词速度可以提升5%
///
*/

public List<String> parserMixedString(StringBuilder inputString) {
    mixedString = new StringBuilder(inputString);
    mixedString.append(S_Pos.SPACE_STRING_DISTINCTION);
    inputLength = mixedString.length();
    outputList = new LinkedList<>();
    forestDepth = S_Pos.INT_ZERO;
    fixWords = new StringBuilder[S_Pos.INT_TWO];
    fixWords[S_Pos.INT_ZERO] = new StringBuilder();
    fixWords[S_Pos.INT_ONE] = new StringBuilder();
    stringBuilder = new StringBuilder();
    find = S_Pos.INT_ZERO;
    // trif-countLength
    Here: for (charPosition = S_Pos.INT_ZERO; charPosition
        < inputLength; charPosition += countLength) {
        if (charPosition < inputLength - S_Pos.INT_ONE
&& mixedString.charAt(
            charPosition) < S_Pos.INT_TEN_SOUTHANDS) {
if (find == S_Pos.INT_ZERO) {
            fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
                fixWords[S_Pos.INT_ZERO].length());
        }
        fixWords[S_Pos.INT_ZERO].append(mixedString.charAt(
            charPosition));
        countLength = S_Pos.INT_ONE;
        find = S_Pos.INT_ONE;
        continue Here;
    }
    if (S_Pos.INT_ONE == find) { // 奇怪了怎么2个重复
        find = S_Pos.INT_ZERO;
        Iterator<String> it = fHMMList
            .englishStringToWordsList(fixWords[S_Pos.INT_ZERO]
                .toString()).iterator();
        StringBuilder number = new StringBuilder();
        Has: while (it.hasNext()) { //trif 紧凑缩进
            String temp = it.next();
            if (S_Pos.NUMBERS.contains(temp)) {
                number.append(temp);
                continue Has;
            }
            if (number.length() > 0) {
                outputList.add(number.toString());
            }
        }
    }
}

```

```

        number.delete(0, number.length());
    }
    outputList.add(temp);
}
if (number.length() > 0) {
    outputList.add(number.toString());
    number.delete(0, number.length());
}
fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
    fixWords[S_Pos.INT_ZERO].length());
}
stringBuilder.delete(S_Pos.INT_ZERO, stringBuilder
.length());

    stringBuilder.append(mixedString.charAt(charPosition));
    stringBuilder = nero_C.getBinaryForestRecurWords(
stringBuilder, mixedString, charPosition, inputLength,
forestRoots, forestDepth, charPosition
    + S_Pos.INT_ONE);
    String countWordNode = stringBuilder.toString();
    int compare = countLength = countWordNode.length();
    if (S_Pos.INT_ONE == compare) {
outputList.add(countWordNode);
fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
    fixWords[S_Pos.INT_ZERO].length());
fixWords[S_Pos.INT_ZERO].append(countWordNode);
continue Here;
    }
    if (S_Pos.INT_TWO == compare) { // trif-countLength
countLength = nlp_C.forTwoChar(countLength,
    outputList, stringBuilder, fixWords, charPosition,
    mixedString);
continue Here;
    }
    if (S_Pos.INT_THREE == compare) { // trif-countLength
I_FixWords(charPosition, mixedString, fixWords);
countLength = nlp_C.ofThree(countLength, outputList,
    stringBuilder, fixWords, charPosition,
    mixedString);
continue Here;
    }
    // if (S_Pos.INT_FOUR == compare) { // trif-countLength
    // 我引擎最长只有4
    I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C._E(countLength, outputList,
stringBuilder, fixWords, charPosition, mixedString);
    // }
}

```

```

        return outputList;
    }
}

//24
//if(null==mixedString){
//    mixedString = inputString;
// }else{
//    mixedString.delete(0, mixedString.length());
//    mixedString.append(inputString);
// }
//    mixedString.append(S_Pos.SPACE_STRING_DISTINCTION);
//    inputLength = mixedString.length();
//    if(null==outputList){
//        outputList = new LinkedList<>();
//    }else{
//        outputList.clear();
//    }
//    forestDepth = S_Pos.INT_ZERO;
//    if(null== fixWords){
//        fixWords = new StringBuilder[S_Pos.INT_TWO];
//        fixWords[S_Pos.INT_ZERO] = new StringBuilder();
//        fixWords[S_Pos.INT_ONE] = new StringBuilder();
//    }else{
//        fixWords[S_Pos.INT_ZERO].delete(0, fixWords[S_Pos.INT_ZERO].length());
//        fixWords[S_Pos.INT_ONE].delete(0, fixWords[S_Pos.INT_ONE].length());
//    }
//    if(null== stringBuilder){
//        stringBuilder = new StringBuilder();
//    }else{
//        stringBuilder.delete(0, stringBuilder.length());
//    }

//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AETest.java
//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/AS.java
package E_A.ME.analysis.E;

import C_A.ME.liner.C.Quick6DLuoYaoguangSort;

import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;

import ME.XCE_XAS;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import Q_V.QSD.VQS.MSO.Q_parser;

import S_A.pheromone.IMV_SQI;

import java.util.List;

import java.util.Map;

//import S_A.SVQ.stable.S_Pos;

/*

```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class AS extends XCE_XAS {
    public Quick6DLuoYaoguangSort quick6DLuoYaoguangSort;
    public Map<Long, IMV_SQI> wordsForests;
    public int find;
    public int inputLength;
    public int forestDepth;
    public int countLength;
    public int charPosition;
    public int charPositionNext;
    public StringBuilder mixedString;
    public StringBuilder stringBuilder;
    public StringBuilder[] fixWords;
    public List<String> outputList;
    public Q_parser _Q_parser;
    public EmotionMap emotionMap;
    public SensingTest sensingTest;
    //public S_Pos _S_Pos;
}

//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/BinaryForest_AE.java
package E_A.ME.analysis.E;

import C_A.ME.analysis.C.BinaryForest_A;
import P_V.PEQ.AMV.ECS.test.SensingTest;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;
import S_A.SVQ.stable.S_Pos;
import S_A.pheromone.IMV_SQI;
import java.util.Map;

/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class BinaryForest_AE extends AE implements BinaryForest_A {
    @SuppressWarnings("unchecked")
    public void studyNewWord(String study, String token, String posStudy) {
        // learn new word
        FMHMMNode fFHMMNode = forestRoots
.get(Long.valueOf(study.charAt(S_Pos.INT_ZERO)));
        Map<String, Integer> map;
        if (null == fFHMMNode) {
            fFHMMNode = new FMHMMNode();
```

```
        map = new IMV_SQI();
    } else {
        map = fHMMNode.getNext();
    }
    map.put(token, S_Pos.INT_ONE);
    fHMMNode.L_Next(map);
    forestRoots.put(Long.valueOf(study.charAt(S_Pos.INT_ZERO)), fHMMNode);
    // learn new pos
    fHMMList.studyNewPos(study + token, posStudy);
}

@Override
public IMV_SQI getStudyPos() {
    return fHMMList.getStudyPos();
}

@Override
public IMV_SQI getCtK() {
    return fHMMList.getFullCnToKo();
}

@Override
public IMV_SQI getCtS() {
    return fHMMList.getFullCnToSp();
}

@Override
public IMV_SQI getCtG() {
    return fHMMList.getFullCnToGm();
}

@Override
public SensingTest getSensingTest() {
    return this.sensingTest;
}

public IMV_SQI getPosEnToCn() {
    return fHMMList.getPosEnToCn();
}

public IMV_SQI getPosEnToEn() {
    return fHMMList.getPosEnToEn();
}

@Override
public IMV_SQI getPosCnToCn() {
    return fHMMList.getPosCnToCn();
}

public IMV_SQI getFullEnToCn() {
    return fHMMList.getFullEnToCn();
}

public IMV_SQI getFullCnToEn() {
    return fHMMList.getFullCnToEn();
}

public IMV_SQI getEnToCn() {
```

```

        return fHMMList.getEnToCn();
    }

    public IMV_SQI getCnToEn() {
        return fHMMList.getCnToEn();
    }
}

//      @Override
//      public IMV_SQI getPinYin() {
//return fHMMList.getFullCnToPy();
//      }
//
//      @Override
//      public IMV_SQI getCtT() {
//return fHMMList.getFullCnToTt();
//      }
//      @Override
//      public IMV_SQI getCtF() {
//return fHMMList.getFullCnToFn();
//      }
//
//      @Override
//      public IMV_SQI getCtJ() {
//return fHMMList.getFullCnToJp();
//      }
//
//      @Override
//      public IMV_SQI getCtR() {
//return fHMMList.getFullCnToRs();
//      }
//
//      @Override
//      public IMV_SQI getCtA() {
//return fHMMList.getFullCnToAb();
//      }
//
//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/BinaryForest_AETest.java
//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/CogsBinaryForest_AE.java
package E_A.ME.analysis.E;

import C_A.OCLAVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;

import C_A.ME.analysis.C.CogsBinaryForest_A;

import ME.VPC.M.app.App;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.SVQ.stable.S_Maps;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI_SS;

import S_A.pheromone.IMV_SQI_S_;

import java.util.Iterator;

import java.util.List;

```

```

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CogsBinaryForest_AE extends BinaryForest_AE implements
    CogsBinaryForest_A {
    /*
     * positionStackPoint改为char的position 位置。
     */
    public IMV_SQI_SS getWordFrequencyMap(List<String> sets, App NE) {
        IMV_SQI_SS map = new IMV_SQI_SS();
        Iterator<String> iterator = sets.iterator();
        int positionStackPoint = 1;
        Here: while (iterator.hasNext()) {
            String setOfi = iterator.next();
            if (map.containsKey(setOfi)) {
                WordFrequency wordFrequency = map.getW(setOfi);
                wordFrequency.I_frequency(wordFrequency
                    .get_frequency() + S_Pos.INT_ONE);
                wordFrequency.positions.add(positionStackPoint);
                map.put(setOfi, wordFrequency);
                continue Here;
            }
            WordFrequency wordFrequency = new WordFrequency(1.0,
                setOfi);
            wordFrequency.positions.add(positionStackPoint);
            map.put(setOfi, wordFrequency);
            positionStackPoint += setOfi.length();
        }
        return map;
    }
    // 计算函数以后移出 DNA元基组 到RNA.
    public List<WordFrequency> sortWordFrequencyMap(IMV_SQI_SS map) {
        List<WordFrequency> list = quick6DLuoYaoguangSort
            .frequencyWordMapToList(map);
        quick6DLuoYaoguangSort.quick6DLuoYaoGuangSortWordFrequency(
            list, S_Pos.INT_ZERO, list.size() - S_Pos.INT_ONE);
        return list;
    }
    /*
    //      for(String string : sets) {

```

```

//      System.out.print("-" + string);
//}
//
//      for(String string : _IMV_SQI_SS.keySet()) {
//      System.out.print("+" + string);
//} //循环逻辑中不出现打印类。
//if (_IMV_SQI_SS.containsKey("街")) {
//      S_logger.Log.logger.info("'" + "400-8-0001-007-03-01-" + _IMV_SQI_SS.size());
//}
//
//
*/

public IMV_SQI_S_ getWordFrequencyByReturnSortMap(
    List<String> sets, App NE) {
    IMV_SQI_SS _IMV_SQI_SS = getWordFrequencyMap(sets, NE);
    return sortWordFrequencyMapToSortMap(_IMV_SQI_SS);
}

public IMV_SQI_S_ sortWordFrequencyMapToSortMap(IMV_SQI_SS map) {
    //S_logger.Log.logger.info("'" + "400-8-0001-007-03-05-" + map.size());
    IMV_SQI_S_ listMap = quick6DLuoYaoguangSort
        .frequencyWordMapToMap(map);
    //S_logger.Log.logger.info("'" + "400-8-0001-007-03-06-" + listMap.size());
    quick6DLuoYaoguangSort.quick6DLuoYaoGuangSortWordFrequency(
        listMap, S_Pos.INT_ZERO, listMap.size() - S_Pos.INT_ONE);
    return listMap;
}

public IMV_SQI_SS parserMixStringByReturnFrequencyMap(
    String inputString, App NE) {
    mixedString = new StringBuilder(inputString
        + S_Pos.SPACE_STRING);
    IMV_SQI_SS outputList = new IMV_SQI_SS();
    Map<Long, FMHMMNode> forestRoots = fHMMList.getMap();// .getRoot();
    int inputLength = mixedString.length();
    int forestDepth = S_Pos.INT_ZERO;
    int countLength;
    StringBuilder[] fixWords = new StringBuilder[S_Pos.INT_TWO];
    fixWords[S_Pos.INT_ZERO] = new StringBuilder();
    fixWords[S_Pos.INT_ONE] = new StringBuilder();
    StringBuilder stringBuilder = new StringBuilder();
    int find = S_Pos.INT_ZERO;
    Here: for (int charPosition = S_Pos.INT_ZERO; charPosition
        < inputLength; charPosition += (countLength == S_Pos.INT_ZERO
            ? S_Pos.INT_ONE
            : countLength)) { // luan ma
        NE._I_U.parserCharPosition = charPosition;
        if (mixedString.charAt(
            charPosition) < S_Pos.INT_ONE_TWO_EIGHT
            && charPosition < mixedString.length()

```

```

        - S_Pos.INT_ONE) {
    if (find == S_Pos.INT_ZERO) {
        fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
            fixWords[S_Pos.INT_ZERO].length());
    }
    fixWords[S_Pos.INT_ZERO].append(mixedString.charAt(
        charPosition));
    countLength = S_Pos.INT_ONE;
    find = S_Pos.INT_ONE;
    continue Here;
    }
    if (find == S_Pos.INT_ONE) {
    find = S_Pos.INT_ZERO;
    Iterator<String> it = fHMMList
        .englishStringToWordsList(fixWords[S_Pos.INT_ZERO]
            .toString()).iterator();
    while (it.hasNext()) {
        String temp = it.next();
        if (outputList.containsKey(temp)) {
            WordFrequency wordFrequency = outputList.getW(
                temp);
            wordFrequency.I_frequency(wordFrequency
                .get_frequency() + S_Pos.INT_ONE);
            wordFrequency.positions.add(
                NE._I_U.parserCharPosition);
            outputList.put(temp, wordFrequency);
        } else {
            WordFrequency wordFrequency = new WordFrequency(
                1.0, temp);
            wordFrequency.positions.add(
                NE._I_U.parserCharPosition);
            outputList.put(temp, wordFrequency);
        }
    }
    fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
        fixWords[S_Pos.INT_ZERO].length());
    }
    stringBuilder.delete(S_Pos.INT_ZERO, stringBuilder
        .length());
    stringBuilder = nero_C.getBinaryForestRecurWords(
    stringBuilder.append(mixedString.charAt(
        charPosition)), mixedString, charPosition,
    inputLength, forestRoots, forestDepth, charPosition
        + S_Pos.INT_ONE);
    String countWordNode = stringBuilder.toString();
    int compare = countLength = countWordNode.length();
    if (compare == S_Pos.INT_TWO) {

```

```

        countLength = nlp_C.forTwoCharForMap(countLength,
            outputList, stringBuilder, fixWords, NE);
    continue Here;
    }
    if (compare == S_Pos.INT_THREE) {
    I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C.ofThreeForMap(countLength,
        outputList, stringBuilder, fixWords, NE);
    continue Here;
    }
    if (compare == S_Pos.INT_ONE) {
    if (outputList.containsKey(countWordNode)) {
        WordFrequency wordFrequency = outputList.getW(
            countWordNode);
        wordFrequency.I_frequency(wordFrequency
            .get_frequency() + S_Pos.INT_ONE);
        wordFrequency.positions.add(
            NE._I_U.parserCharPosition);
        outputList.put(countWordNode, wordFrequency);
    } else {
        WordFrequency wordFrequency = new WordFrequency(
            1.0, fixWords[S_Pos.INT_ZERO].toString());
        wordFrequency.positions.add(
            NE._I_U.parserCharPosition);
        // xian dan hou duo..later..
        outputList.put(countWordNode, wordFrequency);
    }
    fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO,
        fixWords[S_Pos.INT_ZERO].length());
    fixWords[S_Pos.INT_ZERO].append(countWordNode);
    continue Here;
    }
    if (compare == S_Pos.INT_FOUR) {
    I_FixWords(charPosition, mixedString, fixWords);
    countLength = nlp_C.forMap(countLength, outputList,
        stringBuilder, fixWords, NE);
    }
    }
    return outputList;
}

public EmotionMap getEmotionMap() {
    return this.emotionMap;
}

@SuppressWarnings("unchecked")
public void initPCAWordPOS(IMV_SQI_SS mapSearchWithoutSort,
    App NE) {
    Iterator<String> iterator = mapSearchWithoutSort.keySet()

```

```

        .iterator();
        while (iterator.hasNext()) {
            String string = iterator.next();
            WordFrequency wordFrequency = mapSearchWithoutSort.getW(
string);

            StringBuilder stringBuilder = new StringBuilder();
            if (S_Maps.mingCi.containsKey(string)) {
stringBuilder.append("名词");
                } else if (S_Maps.dongCi.containsKey(string)) {
stringBuilder.append("动词");
                } else if (S_Maps.xingRongCi.containsKey(string)) {
stringBuilder.append("形容词");
                } else if (S_Maps.weiCi.containsKey(string)) {
stringBuilder.append("谓词");
                } else if (S_Maps.fuCi.containsKey(string)) {
stringBuilder.append("副词");
                } else if (S_Maps.jieCi.containsKey(string)) {
stringBuilder.append("介词");
                } else if (S_Maps.baDongCi.containsKey(string)) {
stringBuilder.append("把动词");
                } else if (S_Maps.liangCi.containsKey(string)) {
stringBuilder.append("量词");
                } else if (S_Maps.dingMingCi.containsKey(string)) {
stringBuilder.append("定名词");// later dingming 地名统一纠正
                } else if (S_Maps.daiCi.containsKey(string)) {
stringBuilder.append("代词");
                } else if (S_Maps.lianCi.containsKey(string)) {
stringBuilder.append("连词");
                } else if (S_Maps.qingTaiCi.containsKey(string)) {
stringBuilder.append("情态词");
                } else if (S_Maps.shiTaiCi.containsKey(string)) {
stringBuilder.append("时态词");
                } else if (S_Maps.xianDingCi.containsKey(string)) {
stringBuilder.append("限定词");
                } else if (S_Maps.zhuCi.containsKey(string)) {
stringBuilder.append("助词");
                } else if (S_Maps.shengLueCi.containsKey(string)) {
stringBuilder.append("省略词");
                } else {
stringBuilder.append("未知");
// 关联studyMap later -trif
if (null != NE.app_S.workVerbalMap.command_V) {
            NE.app_S.workVerbalMap.command_V.unknown_map.put(
                string, true);
        }
    }

    wordFrequency.L_pos(stringBuilder.toString());

```

```

        mapSearchWithoutSort.put(string, wordFrequency);
    }
}

//108 WordFrequency wordFrequency = new WordFrequency();
//    wordFrequency.I_frequency(1.0);
//    wordFrequency.I_Word(setOfi);
//180 WordFrequency wordFrequency = new WordFrequency();
//    wordFrequency.I_frequency(1.0);
//    wordFrequency.I_Word(temp);
//212 WordFrequency wordFrequency = new WordFrequency();
//    wordFrequency.I_frequency(1.0);
//    wordFrequency.I_Word(fixWords[S_Pos.INT_ZERO].toString());
// public IMV_SQL_SS parserStringByReturnFrequencyMap(StringBuilder inputString, App NE) {
//    int countLength;
//    int find = S_Pos.INT_ZERO;
//    int forestDepth = S_Pos.INT_ZERO;
//    int inputLength = inputString.length();
//    StringBuilder[] fixWords = new StringBuilder[S_Pos.INT_TWO];
//    fixWords[S_Pos.INT_ZERO] = new StringBuilder();
//    fixWords[S_Pos.INT_ONE] = new StringBuilder();
//    StringBuilder stringBuilder = new StringBuilder();
//    //Map<String, String> wordsForest = fHMMList.getPosCnToCn();
//    IMV_SQL_SS outputList = new IMV_SQL_SS();
//    Map<Long, FMHMMNode> forestRoots = fHMMList.getMap();
//    for (int charPosition = S_Pos.INT_ZERO; charPosition < inputLength; charPosition
//        += (countLength == S_Pos.INT_ZERO ? S_Pos.INT_ONE : countLength)) {
//        if (inputString.charAt(charPosition) < S_Pos.INT_ONE_TWO_EIGHT) {
//            if (fixWords[S_Pos.INT_ZERO].length() > S_Pos.INT_ZERO) {
//                if (fixWords[S_Pos.INT_ZERO].charAt(fixWords[S_Pos.INT_ZERO].length()
//                    - S_Pos.INT_ONE) < S_Pos.INT_ONE_TWO_EIGHT) {
//                    fixWords[S_Pos.INT_ZERO].append(inputString.charAt(charPosition));
//                    countLength = S_Pos.INT_ONE;
//                    find = S_Pos.INT_ONE;
//                    continue;
//                }
//                fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO
//                    , fixWords[S_Pos.INT_ZERO].length());
//            }
//            find = S_Pos.INT_ONE;
//            fixWords[S_Pos.INT_ZERO].append(inputString.charAt(charPosition));
//            countLength = S_Pos.INT_ONE;
//            continue;
//        }
//        if (find == S_Pos.INT_ONE) {
//            find = S_Pos.INT_ZERO;
//            Util_X_FindCheck._E(outputList, fixWords, NE);

```

```

//      }
//      stringBuilder.delete(S_Pos.INT_ZERO, stringBuilder.length());
//      stringBuilder = nero_C.getBinaryForestRecurWords(stringBuilder.append(
//          inputString.charAt(charPosition)), inputString, charPosition, inputLength
//          , forestRoots, forestDepth, charPosition + S_Pos.INT_ONE);
//      String countWordNode = stringBuilder.toString();
//      int compare = countLength = countWordNode.length();
//      if (compare == S_Pos.INT_ONE) {
//          Util_X_CompareCheck._E(outputList, fixWords, countWordNode, NE);
//          fixWords[S_Pos.INT_ZERO].delete(S_Pos.INT_ZERO
//              , fixWords[S_Pos.INT_ZERO].length());
//          fixWords[S_Pos.INT_ZERO].append(countWordNode);
//          continue;
//      }
//      if (compare == S_Pos.INT_TWO) {
//          countLength = nlp_C.forTwoCharForMap(countLength, outputList
//              , stringBuilder, fixWords, NE);
//          continue;
//      }
//      if (compare == S_Pos.INT_THREE) {
//          I_FixWords(charPosition, inputString, fixWords);
//          countLength = nlp_C.ofThreeForMap(countLength, outputList
//              , stringBuilder, fixWords, NE);
//          continue;
//      }
//      if (compare == S_Pos.INT_FOUR) {
//          I_FixWords(charPosition, inputString, fixWords);
//          countLength = nlp_C.forMap(countLength, outputList, stringBuilder
//              , fixWords, NE);
//      }
//  }
//  return outputList;
//  }
//62
//public IMV_SQL_S_ sortWordFrequencyMapToUnsortMap(IMV_SQL_SS map) {
//  IMV_SQL_S_ listMap = quick6DLuoYaoguangSort.frequencyWordMapToMap(map);
//  return listMap;
//}
//163
//public List<WordFrequency> getWordFrequency(List<String> sets, App NE) {
//  return sortWordFrequencyMap(getWordFrequencyMap(sets, NE));
//}
//Trif_DNA-master/C_L_E/E_A/ME/analysis/E/CogsBinaryForest_AETest.java
//      E_A.ME.euclid.E
//Trif_DNA-master/C_L_E/E_A/ME/euclid/E/Euclid_CE.java
package E_A.ME.euclid.E;
import C_A.ME.euclid.C.Euclid_C;

```

```
import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI;

import java.util.Iterator;

import java.util.Map;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class Euclid_CE implements Euclid_C {

    @SuppressWarnings({S_Pos.RAW_TYPES, S_Pos.UNCHECKED})

    public Map<Integer, Map> mCogsEuclid(Map<Long, FMHMMNode> HashMap) {

        Map<Integer, Map> HashMapRoot = new IMV_SQI();

        Iterator<Long> iter = HashMap.keySet().iterator();

        while (iter.hasNext()) {

            Long keyValue = iter.next();

            Integer charOfKeyValueToInteger = Integer.valueOf(S_Pos.EMPTY_STRING + keyValue);

            int range = (charOfKeyValueToInteger >> S_Pos.INT_SIX);

            int rangeHigh = range >> S_Pos.INT_FOUR;

            if (!HashMapRoot.containsKey(rangeHigh)) {

                Map<Long, FMHMMNode> innerHashMap = new IMV_SQI();

                innerHashMap.put(keyValue, HashMap.get(keyValue));

                Map<Integer, Map> root = new IMV_SQI();

                root.put(range, innerHashMap);

                HashMapRoot.put(rangeHigh, root);

                continue;

            }

            Map<Integer, Map> root = HashMapRoot.get(rangeHigh);

            if (!root.containsKey(range)) {

                Map<Long, FMHMMNode> innerHashMap = new IMV_SQI();

                innerHashMap.put(keyValue, HashMap.get(keyValue));

                root.put(range, innerHashMap);

                HashMapRoot.put(rangeHigh, root);

                continue;

            }

            Map<Long, FMHMMNode> innerHashMap = root.get(range);

            innerHashMap.put(keyValue, HashMap.get(keyValue));

            root.put(range, innerHashMap);

            HashMapRoot.put(rangeHigh, root);

        }

        return HashMapRoot;

    }

}
```



```

}

/Trif_DNA-master/C_L_E/E_A/ME/euclid/E/Euclid_CETest.java

    E_A.ME.liner.E

/Trif_DNA-master/C_L_E/E_A/ME/liner/E/Quick6DLuoYaoguangSort_E.java
package E_A.ME.liner.E;

import C_A.ME.liner.C.Quick6DLuoYaoguangSort;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQL_SS;

import S_A.pheromone.IMV_SQL_S_;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

/*

** 快排6小高峰修正算法 作者 罗瑶光

** 快排10小高峰修正算法 作者 罗瑶光 20200921

*/

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class Quick6DLuoYaoguangSort_E implements Quick6DLuoYaoguangSort {

    public void quick6DLuoYaoGuangSortWordFrequency(List<WordFrequency> list

        , int leftPosition, int rightPosition) {

        int c = rightPosition - leftPosition + S_Pos.INT_ONE;

        if (c > S_Pos.INT_FOUR) {

            int pos = partitionList(list, leftPosition, rightPosition);

            if (leftPosition < pos - S_Pos.INT_ONE) {

                quick6DLuoYaoGuangSortWordFrequency(list, leftPosition, pos - S_Pos.INT_ONE);

            }

            if (pos + S_Pos.INT_ONE < leftPosition) {

                quick6DLuoYaoGuangSortWordFrequency(list, ++pos, rightPosition);

            }

            return;

        }

        int i = leftPosition;

        for (int j = i + S_Pos.INT_ONE; j < leftPosition + c; j = i++) {

            while (j > leftPosition) {

                if (list.get(j).get_frequency() < list.get(--j).get_frequency()) {

                    WordFrequency wordFrequency = list.get(j + S_Pos.INT_ONE);

                    list.set(j + S_Pos.INT_ONE, list.get(j));

```

```

        list.set(j, wordFrequency);
    }
}
}

public int partitionList(List<WordFrequency> list, int leftPosition, int rightPosition) {
    int leftPositionNew = leftPosition;
    WordFrequency wordFrequencyX = list.get(leftPosition);
    WordFrequency wordFrequencyY = list.get(rightPosition);
    //小高峰修正边缘均衡开始
    if (wordFrequencyX.get_frequency() < wordFrequencyY.get_frequency()) {
        wordFrequencyY = wordFrequencyX;
    }
    //小高峰修正边缘均衡结束
    while (leftPositionNew++ < rightPosition) {
        while (!(list.get(leftPositionNew++).get_frequency() > wordFrequencyY.get_frequency())
            || (leftPositionNew > rightPosition)) {
        }
        while (list.get(rightPosition--).get_frequency() > wordFrequencyY.get_frequency()) {
        }
        if (--leftPositionNew < ++rightPosition) {
            WordFrequency wordFrequency = list.get(rightPosition);
            list.set(rightPosition, list.get(leftPositionNew));
            list.set(leftPositionNew, wordFrequency);
        }
    }
    list.set(leftPosition, list.get(rightPosition));
    list.set(rightPosition, wordFrequencyY);
    return rightPosition;
}

@SuppressWarnings(S_Pos.RAW_TYPES)
public List<WordFrequency> frequencyWordMapToList(IMV_SQL_SS map) {
    List<WordFrequency> list = new ArrayList<>();
    Iterator iterator = map.keySet().iterator(); //later..
    while (iterator.hasNext()) {
        list.add(map.getW(iterator.next().toString())); //later run exe to check
    }
    return list;
}

@Override
public void quick6DLuoYaoGuangSortWordFrequency(IMV_SQL_S_map, int leftPosition
    , int rightPosition) {
}

@Override
public int partition(IMV_SQL_S_map, int leftPosition, int rightPosition) {
    return S_Pos.INT_ZERO;
}

```

```

@Override

public IMV_SQI_S_ frequencyWordMapToMap(IMV_SQI_SS map) {

    return null;

}

}

/Trif_DNA-master/C_L_E/E_A/ME/liner/E/Quick6DLuoYaoguangSort_ETest.java
/Trif_DNA-master/C_L_E/E_A/ME/liner/E/Quick6DLuoYaoguangSort3DMap_E.java
package E_A.ME.liner.E;

import C_A.ME.liner.C.Quick6DLuoYaoguangSort;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI_SS;

import S_A.pheromone.IMV_SQI_S_;

import java.util.Iterator;

import java.util.List;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class Quick6DLuoYaoguangSort3DMap_E implements Quick6DLuoYaoguangSort {

    @Override

    public void quick6DLuoYaoGuangSortWordFrequency(IMV_SQI_S_ map

        , int leftPosition, int rightPosition) {

        if (leftPosition < rightPosition) {

            int c = rightPosition - leftPosition + S_Pos.INT_ONE;

            if (c < S_Pos.INT_FOUR) {

                int j;

                for (int i = S_Pos.INT_ONE + leftPosition; i < leftPosition + c; i++) {

                    j = i;

                    while (j >= S_Pos.INT_ONE + leftPosition) {

                        if (map.getW(j).get_frequency()

                            < map.getW(j - S_Pos.INT_ONE).get_frequency()) {

                            WordFrequency wordFrequency = map.getW(j);

                            map.put(j, map.get(j - S_Pos.INT_ONE));

                            map.put(j - S_Pos.INT_ONE, wordFrequency);

                        }

                        j--;

                    }

                }

            }

            return;

        }

        int pos = partition(map, leftPosition, rightPosition);

```

```

        quick6DLuoYaoGuangSortWordFrequency(map, leftPosition, pos - S_Pos.INT_ONE);
        quick6DLuoYaoGuangSortWordFrequency(map, ++pos, rightPosition);
    }
}

@Override
public int partition(IMV_SQI_S_map, int leftPosition, int rightPosition) {
    int leftPositionNew = leftPosition;
    WordFrequency wordFrequencyX = map.getW(leftPosition);
    WordFrequency wordFrequencyY = map.getW(rightPosition);
    wordFrequencyY = wordFrequencyX.get_frequency() <= wordFrequencyY.get_frequency()
        ? wordFrequencyX : wordFrequencyY;
    while (leftPositionNew < rightPosition) {
        while (!(map.getW(leftPositionNew++).get_frequency() > wordFrequencyY.get_frequency()
            || leftPositionNew > rightPosition)) {
        }
        while (map.getW(rightPosition--).get_frequency() > wordFrequencyY.get_frequency()) {
        }
        if (--leftPositionNew < ++rightPosition) {
            WordFrequency wordFrequency = map.getW(rightPosition);
            map.put(rightPosition, map.get(leftPositionNew));
            map.put(leftPositionNew, wordFrequency);
        }
    }
    map.put(leftPosition, map.get(rightPosition));
    map.put(rightPosition, wordFrequencyY);
    return rightPosition;
}

@SuppressWarnings(S_Pos.RAW_TYPES)
public IMV_SQI_S_frequencyWordMapToMap(IMV_SQI_SS map) {
    IMV_SQI_S_listMap = new IMV_SQI_S_0();
    Iterator iterator = map.keySet().iterator();
    int c = S_Pos.INT_ZERO;
    while (iterator.hasNext()) {
        listMap.put(c++, map.get(iterator.next()));
    }
    return listMap;
}

public void quick6DLuoYaoGuangSortWordFrequency(List<WordFrequency> list
    , int leftPosition, int rightPosition) {
}

public int partitionList(List<WordFrequency> list, int leftPosition, int rightPosition) {
    return S_Pos.INT_ZERO;
}

public List<WordFrequency> frequencyWordMapToList(IMV_SQI_SS map) {
    return null;
}
}

```

/Trif_DNA-master/C_L_E/E_A/ME/liner/E/Quick6DLuoYaoguangSort3DMap_ETest.java

E_A.ME.nero.E

/Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_C_OneTime_E.java
package E_A.ME.nero.E;

import C_A.ME.nero.C.NERO_C_OneTime;

import ME.VPC.M.app.App;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

import S_A.SVQ.stable.S_Pos;

import java.util.Map;

//著作权人+作者= 罗瑶光

/*

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class NERO_C_OneTime_E implements NERO_C_OneTime {

public StringBuilder getBinaryForestRecurWords(StringBuilder outputWordNode

, String inputString, int charPosition, int inputLength

, Map<Long, FMHMMNode> forestRoots, int forestDepth, int charPositionNext) {

if (S_Pos.INT_THREE == forestDepth) {

return outputWordNode;

}

FMHMMNode fFHMMNode = forestRoots.get(Long.valueOf(inputString.charAt(charPosition)));

if (null == fFHMMNode) {

return outputWordNode;

}

Map<String, Integer> outputList = fFHMMNode.getNext();

if (null == outputList || charPositionNext >= inputLength) {

return outputWordNode;

}

char positionOfi = inputString.charAt(charPositionNext);

if (outputList.containsKey(String.valueOf(positionOfi))) {

outputWordNode = getBinaryForestRecurWords(outputWordNode.append(positionOfi)

, inputString, charPositionNext, inputLength, forestRoots, ++forestDepth

, ++charPositionNext);

}

return outputWordNode;

}

//prepare for the big map collection in the future.

public StringBuilder getBinaryForestsRecurWords(StringBuilder outputWordNode

, String inputString, int charPosition, int inputLength

, Map<Long, FMHMMNode>[] forestsRoots, int forestDepth, int charPositionNext) {

if (S_Pos.INT_THREE == forestDepth) {

```

        return outputWordNode;
    }

    FMHMMNode fFHMMNode = getFMHMMNode(forestsRoots, inputString, charPosition);
    if (null == fFHMMNode) {
        return outputWordNode;
    }

    Map<String, Integer> outputList = fFHMMNode.getNext();
    if (null == outputList || charPositionNext >= inputLength) {
        return outputWordNode;
    }

    char positionOfi = inputString.charAt(charPositionNext);
    if (outputList.containsKey(String.valueOf(positionOfi))) {
        outputWordNode = getBinaryForestsRecurWords(outputWordNode.append(positionOfi)
            , inputString, charPositionNext, inputLength, forestsRoots
            , ++forestDepth, ++charPositionNext);
    }

    return outputWordNode;
}

private FMHMMNode getFMHMMNode(Map<Long, FMHMMNode>[] forestsRoots, String inputString
    , int charPosition) {
    for (Map<Long, FMHMMNode> forestsRoot : forestsRoots) {
        if (forestsRoot.containsKey(Long.valueOf(inputString.charAt(charPosition)))) {
            return forestsRoot.get(Long.valueOf(inputString.charAt(charPosition)));
        }
    }

    return null;
}

public StringBuilder getQuickForestRecurWord(StringBuilder outputWordNode
    , String inputString, int charPosition, int inputLength
    , int forestDepth, int charPositionNext, App NE) {
    if (S_Pos.INT_THREE == forestDepth || charPositionNext >= inputLength) {
        return outputWordNode;
    }

    char positionOfi = inputString.charAt(charPositionNext);
    if (NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn
        .containsKey(String.valueOf(outputWordNode.toString() + positionOfi))) {
        outputWordNode = getQuickForestRecurWord(outputWordNode.append(positionOfi)
            , inputString, charPositionNext, inputLength, ++forestDepth
            , ++charPositionNext, NE);
    }

    return outputWordNode;
}
}

/Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_C_OneTime_ES.java
package E_A.ME.nero.E;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

import java.util.Map;

//著作权人+作者= 罗瑶光

```

```

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

public class NERO_C_OneTime_ES {
    public FMHMMNode fFMHMMNode;
    public Map<String, Integer> outputList;
    public char positionOfi;
}

//Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_C_OneTime_ETest.java
//Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_CE.java
package E_A.ME.nero.E;

import C_A.ME.nero.C.NERO_C;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;

import S_A.SVQ.stable.S_Pos;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

public class NERO_CE implements NERO_C {
    @SuppressWarnings({S_Pos.RAW_TYPES, S_Pos.UNCHECKED})
    public StringBuilder getBinaryForestRecurWord(StringBuilder outputWordNode
        , String inputString, int charPosition, int inputLength
        , Map<Integer, Map> forestRoots, int forestDepth, int charPositionNext) {
        if (S_Pos.INT_THREE == forestDepth) {
            return outputWordNode;
        }
        char charAtPosition = inputString.charAt(charPosition);
        int rangeHigh = charAtPosition >> S_Pos.INT_TEN;
        Map<Integer, Map> trees = forestRoots.get(rangeHigh);
        if (null == trees) {
            return outputWordNode;
        }
        int range = charAtPosition >> S_Pos.INT_SIX;
        if (!trees.containsKey(range)) {
            return outputWordNode;
        }
    }
}

```

```

    Map<Long, FMHMMNode> maps = trees.get(range);
    FMHMMNode fFHMMNode = maps.get(Long.valueOf(charAtPosition));
    if (null == fFHMMNode) {
        return outputWordNode;
    }
    Map<String, Integer> outputList = fFHMMNode.getNext();
    if (null == outputList || charPositionNext >= inputLength) {
        return outputWordNode;
    }
    char positionOfi = inputString.charAt(charPositionNext);
    if (outputList.containsKey(String.valueOf(positionOfi))) {
        outputWordNode = getBinaryForestRecurWord(outputWordNode.append(positionOfi)
            , inputString, charPositionNext, inputLength, forestRoots
            , forestDepth + S_Pos.INT_ONE, ++charPositionNext);
    }
    return outputWordNode;
}
}
/Trif_DNA-master/C_L_E/E_A/ME/nero/E/NERO_CETest.java

    E_A.OELAMV.ECS.SVQ.MPC.SOQ.OEM.E

/Trif_DNA-master/C_L_E/E_A/OEL/AMV/ECS/SVQ/MPC/SOQ/OEM/E/SensingMap_E.java
package E_A.OELAMV.ECS.SVQ.MPC.SOQ.OEM.E;

import C_A.OCLAMV.ECS.SVQ.MPC.fhmm.C.SensingMap;

import C_A.ME.analysis.C.BinaryForest_A;

import S_A.pheromone.IMV_SQI;

import java.util.Iterator;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class SensingMap_E implements SensingMap {
    private IMV_SQI lenovoMap;

    @Override
    public IMV_SQI getLenovoMap() {
        return this.lenovoMap;
    }

    @SuppressWarnings("unchecked")
    @Override
    public void IV_LenovoMap(BinaryForest_A _A) {
        lenovoMap = new IMV_SQI();
        IMV_SQI CnToEnMap = _A.getFullCnToEn();
        IMV_SQI EnToCnMap = _A.getEnToCn();
    }

```



```

        Iterator<String> iterator = CnToEnMap.keySet().iterator();
        while (iterator.hasNext()) {
            String word = iterator.next();
            if (!CnToEnMap.containsKey(word)) {
                continue;
            }
            if (!EnToCnMap.containsKey(CnToEnMap.get(word))) {
                lenovoMap.put(word, word);
                continue;
            }
            lenovoMap.put(word, EnToCnMap.get(CnToEnMap.get(word)));
        }
    }
}

/Trif_DNA-master/C_L_E/E_A/OEI/AMV/ECS/SVQ/MPC/SOQ/OEM/E/SensingMap_ETest.java

E_A.OEI.AVC.SUQ.SVQ.MPC.fhmm.E

/Trif_DNA-master/C_L_E/E_A/OEI/AVC/SUQ/SVQ/MPC/fhmm/E/EmotionMap_E.java
package E_A.OEI.AVC.SUQ.SVQ.MPC.fhmm.E;

import C_A.OEI.AVC.SUQ.SVQ.MPC.fhmm.C.EmotionMap;
import ME.VPC.M.app.App;
import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaFile;
import exception.file.O.DetaFileInputStream;
import exception.file.O.DetaInputStreamReader;
import S_A.SVQ.stable.S_Pos;
import S_A.VSQ.parser.EmotionMap_ES;
import S_A.pheromone.IMV_SQI;
import java.io.InputStream;
import java.io.InputStreamReader;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
@SuppressWarnings({"unchecked", "resource"})
public class EmotionMap_E extends EmotionMap_ES implements EmotionMap {
    //NE.resourceTail, 因为eclipse打包比ideaij简洁明确, 对于程序员编码工程不是sdk研发者。
    //导致工程转ideaij打包缺少一把资源文件上层路径问题, 于是
    //加个变量来校正
    public void IV_PositiveMap(App NE) {
        positiveMap = new IMV_SQI();
        DetaFile file = new DetaFile(S_Pos.WORDS_SOURCE_LINK_POS_POSITIVE);
        InputStream in = DetaFileInputStream.E(file.getDetaCanonicalPath())

```

```

        .replace(file.getPath(), NE.resourceTail + "lygs/"
            + file.getPath()));

DetaBufferedReader cReader = new DetaBufferedReader(
    DetaInputStreamReader.E(in, S_Pos.UTF8_STRING));
String cInputString;
while ((cInputString = cReader.readDetaLine()) != null) {
    if (!positiveMap.containsKey(cInputString)) {
        positiveMap.put(cInputString, true);
    }
}
cReader.closeDeta();
}

public void IV_MotivationMap(App NE) {
    motivationMap = new IMV_SQI();
    DetaFile file = new DetaFile(S_Pos.WORDS_SOURCE_LINK_MOTIVATION);
    InputStream in = DetaFileInputStream.E(file.getDetaCanonicalPath()
        .replace(file.getDetaPath(), NE.resourceTail + "lygs/"
            + file.getDetaPath()));
    DetaBufferedReader cReader = new DetaBufferedReader(
        DetaInputStreamReader.E(in, S_Pos.UTF8_STRING));
    String cInputString;
    while ((cInputString = cReader.readDetaLine()) != null) {
        String[] value = cInputString.split(S_Pos.NLP_SYMBOL_SLASH);
        if (!motivationMap.containsKey(value[S_Pos.INT_ZERO])) {
            motivationMap.put(value[S_Pos.INT_ZERO], value[S_Pos.INT_ONE]);
        }
    }
    cReader.closeDeta();
}

    public void IV_TrendingMap(App NE) {
        trendingMap = new IMV_SQI();
        DetaFile file = new DetaFile(S_Pos.WORDS_SOURCE_LINK_TRENDING);
        InputStream in = DetaFileInputStream.E(file.getDetaCanonicalPath()
            .replace(file.getPath(), NE.resourceTail + "lygs/"
                + file.getPath()));
        DetaBufferedReader cReader = new DetaBufferedReader(
            DetaInputStreamReader.E(in, S_Pos.UTF8_STRING));
        String cInputString;
        while ((cInputString = cReader.readDetaLine()) != null) {
            String[] value = cInputString.split(S_Pos.NLP_SYMBOL_SLASH);
            if (!trendingMap.containsKey(value[S_Pos.INT_ZERO])) {
                trendingMap.put(value[S_Pos.INT_ZERO], value[S_Pos.INT_ONE]);
            }
        }
        cReader.closeDeta();
    }

    public void IV_NegativeMap(App NE) {

```

```

negativeMap = new IMV_SQI();
//URL url= new URL(S_Pos.WORDS_SOURCE_LINK_POS_NEGATIVE);
DetaFile file = new DetaFile(S_Pos.WORDS_SOURCE_LINK_POS_NEGATIVE);
InputStream inputStream = DetaFileInputStream.E(file.getDetaCanonicalPath()
    .replace(file.getPath(), NE.resourceTail + "lygs/"
        + file.getPath()));
InputStreamReader inputStreamReader = DetaInputStreamReader.E(inputStream
    , S_Pos.UTF8_STRING);
DetaBufferedReader cReader = new DetaBufferedReader(inputStreamReader);
String cInputString;
while ((cInputString = cReader.readDetaLine()) != null) {
    if (!negativeMap.containsKey(cInputString)) {
        negativeMap.put(cInputString, true);
    }
}
cReader.closeDeta();
}

public void IV_PredictionMap(App NE) {
    predictionMap = new IMV_SQI();
    DetaFile file = new DetaFile(S_Pos.WORDS_SOURCE_LINK_PREDICTION);
    InputStream in = DetaFileInputStream.E(file.getDetaCanonicalPath()
        .replace(file.getPath(), NE.resourceTail + "lygs/"
            + file.getPath()));
    DetaBufferedReader cReader
        = new DetaBufferedReader(DetaInputStreamReader.E(in, S_Pos.UTF8_STRING));
    String cInputString;
    while ((cInputString = cReader.readDetaLine()) != null) {
        String[] value = cInputString.split(S_Pos.NLP_SYMBOL_SLASH);
        if (!predictionMap.containsKey(value[S_Pos.INT_ZERO])) {
            predictionMap.put(value[S_Pos.INT_ZERO], value[S_Pos.INT_ONE]);
        }
    }
    cReader.closeDeta();
}

public void IV_DistinctionMap(App NE) {
    distinctionMap = new IMV_SQI();
    DetaFile file = new DetaFile(S_Pos.WORDS_SOURCE_LINK_DISTINCTION);
    InputStream in = DetaFileInputStream.E(file.getDetaCanonicalPath()
        .replace(file.getPath(), NE.resourceTail + "lygs/"
            + file.getPath()));
    DetaBufferedReader cReader = new DetaBufferedReader(
        DetaInputStreamReader.E(in, S_Pos.UTF8_STRING));
    String cInputString;
    while ((cInputString = cReader.readDetaLine()) != null) {
        String[] value = cInputString.split(S_Pos.NLP_SYMBOL_SLASH);
        if (!distinctionMap.containsKey(value[S_Pos.INT_ZERO])) {
            distinctionMap.put(value[S_Pos.INT_ZERO], value[S_Pos.INT_ONE]);
        }
    }
}

```

```
}
}
cReader.closeDeta();
}
}
/Trif_DNA-master/C_L_E/E_A/OEI/AVC/SUQ/SVQ/MPC/fhmm/E/EmotionMap_ETest.java

E_A.OEI.OPM.VEC.SVQ.MPC.fhmm.E

/Trif_DNA-master/C_L_E/E_A/OEI/OPM/VEC/SVQ/MPC/fhmm/E/BehaviorMap_E.java
package E_A.OEI.OPM.VEC.SVQ.MPC.fhmm.E;

import C_A.OEI.OPM.VEC.SVQ.MPC.fhmm.C.BehaviorMap;

import S_A.pheromone.IMV_SQI;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class BehaviorMap_E implements BehaviorMap{

    @Override
    public IMV_SQI initBusinessMap() {

        return null;

    }

    @Override
    public IMV_SQI initTradeMap() {

        return null;

    }

    @Override
    public IMV_SQI initFacilityMap() {

        return null;

    }

    @Override
    public IMV_SQI initAckuisationMap() {

        return null;

    }

    @Override
    public IMV_SQI initCoorporationMap() {

        return null;

    }

    @Override
    public IMV_SQI initPromiseMap() {

        return null;

    }

}

/Trif_DNA-master/C_L_E/E_A/OEI/OPM/VEC/SVQ/MPC/fhmm/E/BehaviorMap_ETest.java
```

E_A.OEI.OSQ.SVQ.DU.CM.E

```
/Trif_DNA-master/C_L_E/E_A/OEI/OSQ/SVQ/DU/CM/E/Verbal_CE.java
package E_A.OEI.OSQ.SVQ.DU.CM.E;
```

```
import C.Verbal_C;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光. 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
public class Verbal_CE implements Verbal_C{
```

```
    //get verbal
```

```
    //近义词 synonyms
```

```
    //反义词 antonyms
```

```
    //词性the part of speech
```

```
    //词根root
```

```
    //词频word frequency
```

```
    //词态word fix
```

```
    //词感sense of word
```

```
    //词音pronunciation
```

```
    //翻译translation
```

```
}
/Trif_DNA-master/C_L_E/E_A/OEI/OSQ/SVQ/DU/CM/E/VerbalMap_CE.java
E_A.OEI.SVQ.MPC.fhmm.E
```

```
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FastReadProjectFile.java
package E_A.OEI.SVQ.MPC.fhmm.E;
```

```
import exception.file.O.DetaBufferedReader;
```

```
import exception.file.O.DetaInputStreamReader;
```

```
import YLJ_HRJ.ME.VPC.S.books.BookIndex;
```

```
import javax.swing.ImageIcon;
```

```
import java.io.File;
```

```
import java.io.FileInputStream;
```

```
import java.io.IOException;
```

```
import java.io.InputStream;
```

```
import java.net.URL;
```

```
/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class FastReadProjectFile {

    public static URL getDetaURLReader(String string, String afterProjectMiddlePath) {

        try {

            File file = new File(string);

            String stringPath = file.getCanonicalPath().replace(file.getPath()

                , afterProjectMiddlePath + file.getPath());

            return new URL(stringPath);

        } catch (Exception e) {

            e.printStackTrace();

        }

        return null;

    }

    public static ImageIcon getDetaImageIconReader(String string

        , String afterProjectMiddlePath) {

        try {

            File file = new File(string);

            String stringPath = file.getCanonicalPath().replace(file.getPath()

                , afterProjectMiddlePath + file.getPath());

            return new ImageIcon(stringPath);

        } catch (Exception e) {

            e.printStackTrace();

        }

        return null;

    }

    public static DetabufferedReader getDetaBufferedReader(String string

        , String afterProjectMiddlePath, String charset) {

        File file = new File(string);

        InputStream inputStream = null;

        try {

            inputStream = new FileInputStream(

                file.getCanonicalPath().replace(

                    file.getPath(), afterProjectMiddlePath + file.getPath()));

        } catch (IOException e) {

            throw new RuntimeException(e);

        }

        DetabufferedReader cReader = null;

        cReader = new DetabufferedReader(DetaInputStreamReader.E(inputStream

            , charset));

    }

}
```

```

        return cReader;
    }

    public static DetabufferedReader getDetabufferedReader(File file
        , String afterProjectMiddlePath, String charset) {
        InputStream inputStream = null;
        try {
            inputStream = new FileInputStream(file.getCanonicalPath().replace(
                file.getPath(), afterProjectMiddlePath + file.getPath()));
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        DetabufferedReader cReader = null;
        cReader = new DetabufferedReader(DetaInputStreamReader.E(inputStream, charset));
        return cReader;
    }

    public static DetabufferedReader getDetaBookDetabufferedReader(String booktxt
        , String charset) {
        InputStream in = new BookIndex().BookIndex(booktxt);
        DetabufferedReader cReader = null;
        cReader = new DetabufferedReader(DetaInputStreamReader.E(in, charset));
        return cReader;
    }

    public static void close() {
        //shaohou sikao static is zenme close
    }
}

//Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FastReadProjectFileTest.java
//Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMList_E.java
package E_A.OEI.SVQ.MPC.fhmm.E;

import C_A.OCL.SVQ.MPC.fhmm.C.FMHMMList;
import E_A.ME.euclid.E.Euclid_CE;
import ME.VPC.M.app.App;
import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaInputStreamReader;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;
import S_A.SVQ.stable.S_Pos;
import S_A.pheromone.IMV_SQI;
import java.io.InputStream;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136

```

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
@SuppressWarnings({ "unchecked", "resource", "unused" })

public class FMHMMList_E implements FMHMMList {

    private Map<Long, FMHMMNode> linkedHashMap;

    private IMV_SQI words;

    @SuppressWarnings(S_Pos.RAW_TYPES)
    private Map<Integer, Map> linkedHashMapRoot;

    @SuppressWarnings(S_Pos.RAW_TYPES)
    public Map<Integer, Map> getRoot() {

        return this.linkedHashMapRoot;

    }

    public void index() {

        words = new IMV_SQI();

        linkedHashMap = new IMV_SQI();

        linkedHashMapRoot = new IMV_SQI();

        InputStream inputStream = getClass()

.getResourceAsStream(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_CN);

        DataBufferedReader cReader = new DataBufferedReader(

DataInputStreamReader.E(inputStream, S_Pos.UTF8_STRING));

        String cInputString;

        while ((cInputString = cReader.readDataLine()) != null) {

            if (!cInputString.replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING)

.equals(S_Pos.EMPTY_STRING)

&& cInputString.split(

                S_Pos.NLP_SYMBOL_SLASH).length > S_Pos.INT_ONE)) {

continue;

            }

            words.put(cInputString.split(S_Pos.NLP_SYMBOL_SLASH)[S_Pos.INT_ZERO],

new StringBuilder(cInputString

                .split(S_Pos.NLP_SYMBOL_SLASH)[S_Pos.INT_ONE]));

            linkedHashMap = loopLoadForest(cInputString);

        }

        cReader.closeData();

        linkedHashMapRoot = new Euclid_CE().mCogsEuclid(linkedHashMap);

    }

    public Map<Long, FMHMMNode> loopLoadForest(String cInputString) {

        Here: for (int i = S_Pos.INT_ZERO; i < cInputString.length(); i++) {

            if (linkedHashMap

                .containsKey(Long.valueOf(cInputString.charAt(i)))) {

FMHMMNode fHHMMNode = linkedHashMap

                .get(Long.valueOf(cInputString.charAt(i)));

linkedHashMap = doNeroPostCognitive(fHHMMNode, cInputString, i);

continue Here;

            } else {

FMHMMNode fHHMMNode = new FMHMMNode();

fHHMMNode.I_Vb(S_Pos.EMPTY_STRING + cInputString.charAt(i));
```



```

    if (i + S_Pos.INT_ONE < cInputString.length()) {
        Map<String, Integer> next = new IMV_SQI();
        next.put(
            S_Pos.EMPTY_STRING
            + cInputString.charAt(i + S_Pos.INT_ONE),
            S_Pos.INT_ONE);
        fHHMMNode.I_Next(next);
    }
    linkedHashMap.put(Long.valueOf(cInputString.charAt(i)),
        fHHMMNode);
    }
    }
    return linkedHashMap;
}

public Map<Long, FMHMMNode> doNeroPostCognitive(FMHMMNode fHHMMNode,
    String cInputString, int i) {
    if (fHHMMNode.getNext() != null) {
        if (i + S_Pos.INT_ONE < cInputString.length()) {
linkedHashMap = doCheckAndRunNeroPostFix(fHHMMNode,
            cInputString, i);
        }
    } else {
        ConcurrentHashMap<String, Integer> concurrentHashMap = new IMV_SQI();
        if (i + S_Pos.INT_ONE < cInputString.length()) {
concurrentHashMap.put(
            S_Pos.EMPTY_STRING
            + cInputString.charAt(i + S_Pos.INT_ONE),
            S_Pos.INT_ONE);
        }
        fHHMMNode.I_Next(concurrentHashMap);
        linkedHashMap.put(Long.valueOf(cInputString.charAt(i)), fHHMMNode);
    }
    return linkedHashMap;
}

public Map<Long, FMHMMNode> doCheckAndRunNeroPostFix(FMHMMNode fHHMMNode,
    String cInputString, int i) {
    if (!fHHMMNode.getNext().containsKey(
S_Pos.EMPTY_STRING + cInputString.charAt(i + S_Pos.INT_ONE))) {
        Map<String, Integer> map = fHHMMNode.getNext();
        map.put(S_Pos.EMPTY_STRING + cInputString.charAt(i + S_Pos.INT_ONE),
            S_Pos.INT_ONE);
        fHHMMNode.I_Next(map);
        linkedHashMap.put(Long.valueOf(cInputString.charAt(i)), fHHMMNode);
    }
    return linkedHashMap;
}

public IMV_SQI getWords() {

```

```

        return this.words;
    }

    public Map<Long, FMHMMNode> getMap() {
        return this.linkedHashMap;
    }

    public IMV_SQI getPosEnToEn() {
        return null;
    }

    public IMV_SQI getEnToCn() {
        return null;
    }

    public IMV_SQI getCnToEn() {
        return null;
    }

    public void indexEnToCn() {
    }

    public void indexCnToEn() {
    }

    public IMV_SQI getPosEnToCn() {
        return null;
    }

    public IMV_SQI getPosCnToCn() {
        return null;
    }

    public void indexPosEnToCn() {
    }

    public void indexPosEnToEn() {
    }

    public void indexFullEnToCn() {
    }

    public void indexFullCnToEn() {
    }

    public IMV_SQI getFullEnToCn() {
        return null;
    }

    public IMV_SQI getFullCnToEn() {
        return null;
    }
}

```

//later trif eclipse 转 idea出现的问题

```

@Override
public void indexPosEnToCn(App NE) {
}

@Override
public void indexFn(App NE) {
}

@Override
public void indexKo(App NE) {
}

```

```
}  
  
@Override  
public void indexJp(App NE) {  
}  
  
@Override  
public void indexGm(App NE) {  
}  
  
@Override  
public void indexSp(App NE) {  
}  
  
@Override  
public void indexAb(App NE) {  
}  
  
@Override  
public void indexRs(App NE) {  
}  
  
@Override  
public void indexPy(App NE) {  
}  
  
@Override  
public void indexTt(App NE) {  
}  
  
@Override  
public void indexPosEnToEn(App NE) {  
}  
  
@Override  
public void indexEnToCn(App NE) {  
}  
  
@Override  
public void indexCnToEn(App NE) {  
}  
  
@Override  
public List<String> englishStringToWordsList(String string) {  
    return null;  
}  
  
@Override  
public void indexFullCnToJp() {  
}  
  
@Override  
public void indexFullCnToRs() {  
}  
  
@Override  
public void indexFullCnToAb() {  
}  
  
@Override  
public void indexFullCnToFn() {  
}
```

```
@Override
public void indexFullCnToGm() {
}

@Override
public void indexFullCnToKo() {
}

@Override
public void indexFullCnToSp() {
}

@Override
public void indexFullCnToPy() {
}

@Override
public IMV_SQI getFullCnToJp() {
    return null;
}

@Override
public IMV_SQI getFullCnToRs() {
    return null;
}

@Override
public IMV_SQI getFullCnToAb() {
    return null;
}

@Override
public IMV_SQI getFullCnToFn() {
    return null;
}

@Override
public IMV_SQI getFullCnToGm() {
    return null;
}

@Override
public IMV_SQI getFullCnToKo() {
    return null;
}

@Override
public IMV_SQI getFullCnToSp() {
    return null;
}

@Override
public IMV_SQI getFullCnToPy() {
    return null;
}

@Override
public Map<Long, FMHMMNode>[] getMaps() {
    return null;
}
```

```

    }

    @Override
    public Map<Long, IMV_SQI> getWordsForests() {
        return null;
    }

    @Override
    public void studyNewPos(String string, String posStudy) {
    }

    @Override
    public IMV_SQI getStudyPos() {
        return null;
    }

    @Override
    public IMV_SQI getFullCnToTt() {
        return null;
    }

    @Override
    public void indexFullNegative() {
    }

    @Override
    public void indexFullPositive() {
    }

    @Override
    public IMV_SQI getFullNegative() {
        return null;
    }

    @Override
    public IMV_SQI getFullPositive() {
        return null;
    }

    @Override
    public void indexFullCnToTt() {
    }
}

/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMList_ETest.java
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMListOneTime_E_maps.java
package E_A.OEI.SVQ.MPC.fhmm.E;

import C_A.OCL.SVQ.MPC.fhmm.C.FHMMList;
import ME.VPC.M.app.App;
import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaInputStreamReader;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.FMHMMNode;
import S_A.SVQ.stable.S_Maps;
import S_A.SVQ.stable.S_Pos;
import VSQ.parser.FMHMMListOneTime_E_X_S;
import S_A.pheromone.IMV_SQI;
import U_A.PEU.P.nlp.StopSymbol_UTF8;
import java.io.InputStream;

```

```

import java.util.Iterator;

import java.util.LinkedList;

import java.util.List;

import java.util.Map;

import java.util.concurrent.CopyOnWriteArrayList;

//I will build a collection class for managing this maps. at the next version.

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

@SuppressWarnings({ "unchecked", "resource", "unused" })

public class FMHMMListOneTime_E_maps extends FMHMMListOneTime_E_X_S

    implements FHMMList {

    // 之后去掉一切古老的继承关系接口规范 如extends implements, 等。
    public List<String> englishStringToWordsList(String string) {

        List<String> list = new LinkedList<>();

        string = string.replaceAll(S_Pos.NLP_SPASE_REP, S_Pos.SPACE_STRING);

        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < string.length(); i++) {

            if (StopSymbol_UTF8.symbol

                .contains(S_Pos.EMPTY_STRING + string.charAt(i))) {

list.add(sb.toString().toLowerCase());

sb.delete(S_Pos.INT_ZERO, sb.length());

list.add(String.valueOf(string.charAt(i)));

            } else {

sb.append(string.charAt(i));

            }

        }

        if (S_Pos.INT_ZERO < sb.length()) {

            list.add(sb.toString().toLowerCase());

        }

        return list;

    }

    public Map<Long, FMHMMNode> getMap() {

        return this.linkedHashMap;

    }

    public Map<Long, FMHMMNode>[] getMaps() {

        int segment = this.linkedHashMap.size();

        int perRatio = segment / S_Pos.INT_SIX;

        Map<Long, FMHMMNode>[] maps = new IMV_SQI[S_Pos.INT_SIX];

        Iterator<Long> iterator = this.linkedHashMap.keySet().iterator();

        maps[S_Pos.INT_ZERO] = new IMV_SQI();

```

```

        int index = S_Pos.INT_ZERO;
        int count = S_Pos.INT_ONE;
        while (iterator.hasNext()) {
            if (S_Pos.INT_ZERO == count++ % perRatio) {
if (index < S_Pos.INT_FIVE) {
                index++;
                maps[index] = new IMV_SQI();
            }
                }
            Long key = iterator.next();
            maps[index].put(key, this.linkedHashMap.get(key));
        }
        return maps;
    }
// 稍后translator的index 去重 统一用index mixed
    public void index() {
        posCnToCn = new IMV_SQI();
        linkedHashMap = new IMV_SQI();
        listCn = new CopyOnWriteArrayList<>();
        InputStream inputStream = getClass()
.getResourceAsStream(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_CN);
        DetabufferedReader cReader = new DetabufferedReader(
DetabufferedReader.E(inputStream, S_Pos.UTF8_STRING));
        String cInputString;
        Here: while ((cInputString = cReader.readDetabLine()) != null) {
            listCn.add(cInputString);
            if (!cInputString.replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING)
.equals(S_Pos.EMPTY_STRING)
&& cInputString.split(
                S_Pos.NLP_SYMBO_SLASH).length > S_Pos.INT_ONE)) {
continue Here;
            }
            if (!S_Maps.fuCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
                .contains(S_Pos.NLP_CI_FU)) {
S_Maps.fuCi.put(
                cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
            }
            if (!S_Maps.dongCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
                .contains(S_Pos.NLP_CI_DONG)) {
S_Maps.dongCi.put(

```

```
        cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.liangCi.containsKey(
    cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
    && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_LIANG)) {
S_Maps.liangCi.put(
    cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.lianCi.containsKey(
    cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
    && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_LIAN)) {
S_Maps.lianCi.put(
    cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.baDongCi.containsKey(
    cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
    && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_BA_DONG)) {
S_Maps.baDongCi.put(
    cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.xianDingCi.containsKey(
    cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
    && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_XIAN_DING)) {
S_Maps.xianDingCi.put(
    cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.mingCi.containsKey(
    cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
```



```
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_MING)) {
S_Maps.mingCi.put(
        cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.daiCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_DAI)) {
S_Maps.daiCi.put(
        cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.jieCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_JIE)) {
S_Maps.jieCi.put(
        cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.xingRongCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_XING_RONG)) {
S_Maps.xingRongCi.put(
        cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.zhuCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_ZHU)) {
S_Maps.zhuCi.put(
        cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
```

```
    }

    if (!S_Maps.weiCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
            .contains(S_Pos.NLP_CI_WEI)) {
S_Maps.weiCi.put(
    cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }

    if (!S_Maps.shengLueCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
            .contains(S_Pos.NLP_CI_SHENG_LUE)) {
S_Maps.shengLueCi.put(
    cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }

    if (!S_Maps.qingTaiCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
            .contains(S_Pos.NLP_CI_QING_TAI)) {
S_Maps.qingTaiCi.put(
    cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }

    if (!S_Maps.xingWeiCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
            .contains(S_Pos.NLP_CI_XING_WEI)) {
S_Maps.xingWeiCi.put(
    cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }

    if (!S_Maps.shiTaiCi.containsKey(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
        && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
            .contains(S_Pos.NLP_CI_SHI_TAI)) {
S_Maps.shiTaiCi.put(
    cInputString
```

```

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.dingMingCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_DING_MING)) {
S_Maps.dingMingCi.put(
    cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    posCnToCn.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    linkedHashMap = loopLoadForest(cInputString);
}
cReader.closeDeta();
}
public void indexFullEnToCn() {
    Iterator<String> listcc = listCn.iterator();
    Iterator<String> listee = listEn.iterator();
    fullEnToCn = new IMV_SQI();
    while (listcc.hasNext() && listee.hasNext()) {
        fullEnToCn.put(
listee.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
            .toLowerCase(),
listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]);
    }
}
public void indexFullCnToEn() {
    Iterator<String> listcc = listCn.iterator();
    Iterator<String> listee = listEn.iterator();
    fullCnToEn = new IMV_SQI();
    while (listcc.hasNext() && listee.hasNext()) {
        fullCnToEn.put(
listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
listee.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
            .toLowerCase());
    }
}
public void indexFullCnToFn() {
    Iterator<String> listcc = listCn.iterator();
    Iterator<String> listfn = listFn.iterator();

```

```
        fullCnToFn = new IMV_SQI();
        while (listcc.hasNext() && listfn.hasNext()) {
            fullCnToFn.put(
                listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                listfn.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
                    .toLowerCase());
        }
    }

    public void indexFullCnToKo() {
        Iterator<String> listcc = listCn.iterator();
        Iterator<String> listko = listKo.iterator();
        fullCnToKo = new IMV_SQI();
        while (listcc.hasNext() && listko.hasNext()) {
            fullCnToKo.put(
                listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                listko.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
                    .toLowerCase());
        }
    }

    public void indexFullCnToJp() {
        Iterator<String> listcc = listCn.iterator();
        Iterator<String> listjp = listJp.iterator();
        fullCnToJp = new IMV_SQI();
        while (listcc.hasNext() && listjp.hasNext()) {
            fullCnToJp.put(
                listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                listjp.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
                    .toLowerCase());
        }
    }

    public void indexFullCnToGm() {
        Iterator<String> listcc = listCn.iterator();
        Iterator<String> listgm = listGm.iterator();
        fullCnToGm = new IMV_SQI();
        while (listcc.hasNext() && listgm.hasNext()) {
            fullCnToGm.put(
                listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                listgm.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
                    .toLowerCase());
        }
    }

    public void indexFullCnToSp() {
        Iterator<String> listcc = listCn.iterator();
        Iterator<String> listsp = listSp.iterator();
        fullCnToSp = new IMV_SQI();
        while (listcc.hasNext() && listsp.hasNext()) {
            fullCnToSp.put(
```

```
listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
listsp.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase());
    }
}

public void indexFullCnToRs() {
    Iterator<String> listcc = listCn.iterator();
    Iterator<String> listrs = listRs.iterator();
    fullCnToRs = new IMV_SQI();
    while (listcc.hasNext() && listrs.hasNext()) {
        fullCnToRs.put(
listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
listrs.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase());
    }
}

public void indexFullCnToAb() {
    Iterator<String> listcc = listCn.iterator();
    Iterator<String> listab = listAb.iterator();
    fullCnToAb = new IMV_SQI();
    while (listcc.hasNext() && listab.hasNext()) {
        fullCnToAb.put(
listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
listab.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase());
    }
}

public void indexFullCnToPy() {
    Iterator<String> listcc = listCn.iterator();
    Iterator<String> listpy = listPy.iterator();
    fullCnToPy = new IMV_SQI();
    while (listcc.hasNext() && listpy.hasNext()) {
        fullCnToPy.put(
listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
listpy.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase());
    }
}

public void indexFullCnToTt() {
    Iterator<String> listcc = listCn.iterator();
    Iterator<String> listtt = listTt.iterator();
    fullCnToTt = new IMV_SQI();
    while (listcc.hasNext() && listtt.hasNext()) {
        fullCnToTt.put(
listcc.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
listtt.next().split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase());
    }
}
```

```

    }

}

public IMV_SQI getFullEnToCn() {
    return this.fullEnToCn;
}

public IMV_SQI getFullCnToEn() {
    return this.fullCnToEn;
}

public void indexPosEnToCn(App NE) {
    posEnToCn = new IMV_SQI();
    // InputStream in=
    // getClass().getResourceAsStream(S_Pos.WORDS_SOURCE_LINK_POS_EN_TO_CN);
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_EN_TO_CN, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    Here: while (null != (cInputString = cReader.readDetaLine())) {
        if (!cInputString.replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING)
.equals(S_Pos.EMPTY_STRING)
&& cInputString.split(
                S_Pos.NLP_SYMBO_SLASH).length > S_Pos.INT_ONE)) {
continue Here;
        }
        posEnToCn.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
                .toLowerCase(),
new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    cReader.closeDeta();
}

public void indexFn(App NE) {
    listFn = new CopyOnWriteArrayList<>();
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_FN, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    while (null != (cInputString = cReader.readDetaLine())) {
        listFn.add(cInputString);
    }
    cReader.closeDeta();
}

public void indexKo(App NE) {
    listKo = new CopyOnWriteArrayList<>();
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_KO, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);

```

```
String cInputString;
while (null != (cInputString = cReader.readDetaLine())) {
    listKo.add(cInputString);
}
cReader.closeDeta();
}

public void indexJp(App NE) {
    listJp = new CopyOnWriteArrayList<>();
    DetabufferedReader cReader = FastReadProjectFile.getDetabufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_JP, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    while (null != (cInputString = cReader.readDetaLine())) {
        listJp.add(cInputString);
    }
    cReader.closeDeta();
}

public void indexGm(App NE) {
    listGm = new CopyOnWriteArrayList<>();
    DetabufferedReader cReader = FastReadProjectFile.getDetabufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_GM, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    while (null != (cInputString = cReader.readDetaLine())) {
        listGm.add(cInputString);
    }
    cReader.closeDeta();
}

public void indexSp(App NE) {
    listSp = new CopyOnWriteArrayList<>();
    DetabufferedReader cReader = FastReadProjectFile.getDetabufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_SP, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    while (null != (cInputString = cReader.readDetaLine())) {
        listSp.add(cInputString);
    }
    cReader.closeDeta();
}

public void indexAb(App NE) {
    listAb = new CopyOnWriteArrayList<>();
    DetabufferedReader cReader = FastReadProjectFile.getDetabufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_AB, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    while (null != (cInputString = cReader.readDetaLine())) {
        listAb.add(cInputString);
    }
}
```

```

    }
    cReader.closeDeta();
}

public void indexRs(App NE) {
    listRs = new CopyOnWriteArrayList<>();
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_RS, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    while (null != (cInputString = cReader.readDetaLine())) {
        listRs.add(cInputString);
    }
    cReader.closeDeta();
}

public void indexPy(App NE) {
    listPy = new CopyOnWriteArrayList<>();
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_PY, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    while (null != (cInputString = cReader.readDetaLine())) {
        listPy.add(cInputString);
    }
    cReader.closeDeta();
}

public void indexTt(App NE) {
    listTt = new CopyOnWriteArrayList<>();
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_TT, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    while (null != (cInputString = cReader.readDetaLine())) {
        listTt.add(cInputString);
    }
    cReader.closeDeta();
}

public void indexPosEnToEn(App NE) {
    posEnToEn = new IMV_SQI();
    listEn = new CopyOnWriteArrayList<>();
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_EN_TO_EN, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    Here: while (null != (cInputString = cReader.readDetaLine())) {
        listEn.add(cInputString);
        if (!cInputString.replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING)
.equals(S_Pos.EMPTY_STRING)

```



```

        && cInputString.split(
            S_Pos.NLP_SYMBO_SLASH).length > S_Pos.INT_ONE)) {
continue Here;
    }
    posEnToEn.put(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
            .toLowerCase(),
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
            .toLowerCase());
    }
    cReader.closeDeta();
}

public void indexEnToCn(App NE) {
    enToCn = new IMV_SQI();
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_EN_TO_CN, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    Here: while (null != (cInputString = cReader.readDetaLine())) {
        if (!(!cInputString.replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING)
.equals(S_Pos.EMPTY_STRING)
        && cInputString.split(
            S_Pos.NLP_SYMBO_SLASH).length > S_Pos.INT_ONE)) {
continue Here;
        }
        enToCn.put(
            cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
                .toLowerCase(),
            cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]);
        }
        cReader.closeDeta();
    }
}

public void indexCnToEn(App NE) {
    cnToEn = new IMV_SQI();
    DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_CN_TO_EN, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
    String cInputString;
    Here: while (null != (cInputString = cReader.readDetaLine())) {
        if (!(!cInputString.replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING)
.equals(S_Pos.EMPTY_STRING)
        && cInputString.split(
            S_Pos.NLP_SYMBO_SLASH).length > S_Pos.INT_ONE)) {
continue Here;
        }
        cnToEn.put(
            cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

```

```

        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
            .toLowerCase());
    }
    cReader.closeData();
}

public Map<Long, FMHMMNode> loopLoadForest(String cInputString) {
    Here: for (int i = S_Pos.INT_ZERO; i < cInputString.length(); i++) {
        if (linkedHashMap
            .containsKey(Long.valueOf(cInputString.charAt(i)))) {
            FMHMMNode fHHMMNode = linkedHashMap
                .get(Long.valueOf(cInputString.charAt(i)));
            linkedHashMap = doNeroPostCognitive(fHHMMNode, cInputString, i);
            continue Here;
        }
        FMHMMNode fHHMMNode = new FMHMMNode();
        fHHMMNode.I_Vb(S_Pos.EMPTY_STRING + cInputString.charAt(i));
        if (i + S_Pos.INT_ONE < cInputString.length()) {
            Map<String, Integer> next = new IMV_SQI();
            next.put(
                S_Pos.EMPTY_STRING
                + cInputString.charAt(i + S_Pos.INT_ONE),
                S_Pos.INT_ONE);
            fHHMMNode.I_Next(next);
        }
        linkedHashMap.put(Long.valueOf(cInputString.charAt(i)), fHHMMNode);
    }
    return linkedHashMap;
}

public Map<Long, FMHMMNode> doNeroPostCognitive(FMHMMNode fHHMMNode,
    String cInputString, int i) {
    if (null != fHHMMNode.getNext()) {
        if (i + S_Pos.INT_ONE < cInputString.length()) {
            linkedHashMap = doCheckAndRunNeroPostFix(fHHMMNode,
                cInputString, i);
        }
        return linkedHashMap;
    }
    Map<String, Integer> HashMap = new IMV_SQI();
    if (i + S_Pos.INT_ONE < cInputString.length()) {
        HashMap.put(
            S_Pos.EMPTY_STRING + cInputString.charAt(i + S_Pos.INT_ONE),
            S_Pos.INT_ONE);
    }
    fHHMMNode.I_Next(HashMap);
    linkedHashMap.put(Long.valueOf(cInputString.charAt(i)), fHHMMNode);
    return linkedHashMap;
}
}

```

```

    public Map<Long, FMHMMNode> doCheckAndRunNeroPostFix(FMHMMNode fFHMMNode,
        String cInputString, int i) {
        if (!fFHMMNode.getNext().containsKey(
            S_Pos.EMPTY_STRING + cInputString.charAt(i + S_Pos.INT_ONE))) {
            Map<String, Integer> map = fFHMMNode.getNext();
            map.put(S_Pos.EMPTY_STRING + cInputString.charAt(i + S_Pos.INT_ONE),
                S_Pos.INT_ONE);
            fFHMMNode.L_Next(map);
            linkedHashMap.put(Long.valueOf(cInputString.charAt(i)), fFHMMNode);
        }
        return linkedHashMap;
    }

    public IMV_SQI getPosCnToCn() {
        return this.posCnToCn;
    }

    public IMV_SQI getEnToCn() {
        return enToCn;
    }

    public IMV_SQI getCnToEn() {
        return cnToEn;
    }

    public IMV_SQI getPosEnToCn() {
        return this.posEnToCn;
    }

    public IMV_SQI getPosEnToEn() {
        return this.posEnToEn;
    }

    public IMV_SQI getFullCnToJp() {
        return this.fullCnToJp;
    }

    public IMV_SQI getFullCnToRs() {
        return this.fullCnToRs;
    }

    public IMV_SQI getFullCnToAb() {
        return this.fullCnToAb;
    }

    public IMV_SQI getFullCnToFn() {
        return this.fullCnToFn;
    }

    public IMV_SQI getFullCnToGm() {
        return this.fullCnToGm;
    }

    public IMV_SQI getFullCnToKo() {
        return this.fullCnToKo;
    }

    public IMV_SQI getFullCnToSp() {
        return this.fullCnToSp;
    }

```

```
}

public IMV_SQI getFullCnToPy() {

    this.fullCnToPy.put("鼋", "yuan");// 这些我稍后也会移动到stable文件中, 免得乱码。
    this.fullCnToPy.put("鹑", "chun");
    this.fullCnToPy.put("鸬鹚", "qu yu");
    this.fullCnToPy.put("鲚鱼", "ji yu");
    this.fullCnToPy.put("鲎", "hou");
    this.fullCnToPy.put("醍醐", "ti hu");
    this.fullCnToPy.put("豹", "bao");
    this.fullCnToPy.put("蠮", "zhe");
    this.fullCnToPy.put("蛭", "qi cao");
    this.fullCnToPy.put("藁", "huai");
    this.fullCnToPy.put("菰", "jin");
    this.fullCnToPy.put("槩", "bo");
    this.fullCnToPy.put("濮", "pu");
    this.fullCnToPy.put("稻", "lv");
    this.fullCnToPy.put("穰", "kuang");
    this.fullCnToPy.put("籩", "jun");
    this.fullCnToPy.put("芰", "ji");
    this.fullCnToPy.put("蒺藜", "ci gu");
    this.fullCnToPy.put("荏", "ren");
    this.fullCnToPy.put("茈", "rao");
    this.fullCnToPy.put("蓁", "lu");
    this.fullCnToPy.put("菰", "song");
    this.fullCnToPy.put("藪", "ji");
    this.fullCnToPy.put("藜藿", "fan cai");

    return this.fullCnToPy;

}

public IMV_SQI getFullCnToTt() {

    return this.fullCnToTt;

}

public void indexFullNegative() {

    fullNegative = new IMV_SQI();

    InputStream in = getClass()

.getResourceAsStream(S_Pos.WORDS_SOURCE_LINK_POS_NEGATIVE);

    DetabufferedReader cReader = new DetabufferedReader(

DetabufferedReader.E(in, S_Pos.UTF8_STRING));

    String cInputString;

    while (null != (cInputString = cReader.readDetabufferLine())) {

        if (!fullNegative.containsKey(cInputString)) {

fullNegative.put(cInputString, S_Pos.EMPTY_STRING);

        }

    }

    cReader.closeDetabuffer();

}

public void indexFullPositive() {

    fullPositive = new IMV_SQI();
```

```

        InputStream in = getClass()
            .getResourceAsStream(S_Pos.WORDS_SOURCE_LINK_POS_POSITIVE);

        DataBufferedReader cReader = new DataBufferedReader(
            DataInputStreamReader.E(in, S_Pos.UTF8_STRING));

        String cInputString;
        while (null != (cInputString = cReader.readDataLine())) {
            if (!fullPositive.containsKey(cInputString)) {
                fullPositive.put(cInputString, S_Pos.EMPTY_STRING);
            }
        }

        cReader.closeData();
    }

    public IMV_SQI getFullNegative() {
        return this.fullNegative;
    }

    public IMV_SQI getFullPositive() {
        return this.fullPositive;
    }

    @Override
    public Map<Long, IMV_SQI> getWordsForests() {
        Map<Long, IMV_SQI> output = new IMV_SQI();
        Iterator<String> WordTree = posCnToCn.keySet().iterator();
        while (WordTree.hasNext()) {
            String treeName = WordTree.next();
            if (0 < treeName.length()) {
                IMV_SQI treeLeafs;
                if (output.containsKey(
                    Long.valueOf(treeName.charAt(S_Pos.INT_ZERO)))) {
                    treeLeafs = output
                        .get(Long.valueOf(treeName.charAt(S_Pos.INT_ZERO)));
                    treeLeafs.put(treeName, posCnToCn.get(treeName));
                    output.put(Long.valueOf(treeName.charAt(S_Pos.INT_ZERO)),
                        treeLeafs);
                } else {
                    treeLeafs = new IMV_SQI();
                }
                treeLeafs.put(treeName, posCnToCn.get(treeName));
                output.put(Long.valueOf(treeName.charAt(S_Pos.INT_ZERO)),
                    treeLeafs);
            }
        }

        return output;
    }

    public void studyNewPos(String string, String posStudy) {
        posCnToCn.put(string, posStudy);
    }

    @Override

```

```

        public IMV_SQI getStudyPos() {

            return this.studyPos;

        }

    }

//604 // if((string.charAt(i)> S_Data.INT_SIXTY_FOUR && string.charAt(i)<= S_Data.INT_NINTY)
////    ||(string.charAt(i)>= S_Data.INT_NINTY_SEVEN && string.charAt(i)<= S_Data.INT_ONE_TWO_TWO)
////    ||symbol.contains(S_Data.EMPTY_STRING+ string.charAt(i)) {
////sb.append(string.charAt(i));
////}else {
////list.add(sb.toString().toLowerCase());
////sb.delete(S_Data.INT_ZERO, sb.length());
////list.add(String.valueOf(string.charAt(i)));
////}

//Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMListOneTime_E_mapsTest.java
//Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMListOneTime_E.java
package E_A.OEI.SVQ.MPC.fhmm.E;

import C_A.OCLSVQ.MPC.fhmm.C.FHMMList_indexAll;

import ME.VPC.M.app.App;

import S_A.SVQ.stable.S_Maps;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI;

import exception.file.O.DetaBufferedReader;

import java.util.concurrent.CopyOnWriteArrayList;

/*

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

@SuppressWarnings({ "unchecked", "resource", "unused" })

public class FMHMMListOneTime_E extends FMHMMListOneTime_EX

    implements FHMMList_indexAll {

    public void indexMixed(App NE) {

        // init

        studyPos = new IMV_SQI();

        posCnToCn = new IMV_SQI();

        linkedHashMap = new IMV_SQI();

        listCn = new CopyOnWriteArrayList<>();

        listAb = new CopyOnWriteArrayList<>();

        listKo = new CopyOnWriteArrayList<>();

        listJp = new CopyOnWriteArrayList<>();

        listRs = new CopyOnWriteArrayList<>();

        listTt = new CopyOnWriteArrayList<>();

        listEn = new CopyOnWriteArrayList<>();

        listPy = new CopyOnWriteArrayList<>();

```

```
listSp = new CopyOnWriteArrayList<>();
listFn = new CopyOnWriteArrayList<>();
listGm = new CopyOnWriteArrayList<>();
DetaBufferedReader cReader = FastReadProjectFile.getDetaBufferedReader(
S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_CN, NE.resourceTail + "lygs/",
S_Pos.UTF8_STRING);
DetaBufferedReader cReaderKorea = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_KO,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderJapan = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_JP,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderTrandition = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_TT,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderEnglish = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_EN_TO_EN,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderPinYin = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_PY,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderRs = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_RS,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderAb = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_AB,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderSp = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_SP,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderFn = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_FN,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
DetaBufferedReader cReaderGm = FastReadProjectFile
.getDetaBufferedReader(S_Pos.WORDS_SOURCE_LINK_POS_CN_TO_GM,
NE.resourceTail + "lygs/", S_Pos.UTF8_STRING);
String cInputString;
String cInputStringKorea;
String cInputStringJapan;
String cInputStringTrandition;
String cInputStringEnglish;
String cInputStringPinYin;
String cInputStringRs;
String cInputStringAb;
String cInputStringSp;
String cInputStringGm;
String cInputStringFn;
```

```

// read
Here: while ((cInputString = cReader.readDetaLine()) != null) {
    cInputStringKorea = cReaderKorea.readDetaLine();
    cInputStringJapan = cReaderJapan.readDetaLine();
    cInputStringTrandition = cReaderTrandition.readDetaLine();
    cInputStringEnglish = cReaderEnglish.readDetaLine();
    cInputStringPinYin = cReaderPinYin.readDetaLine();
    cInputStringRs = cReaderRs.readDetaLine();
    cInputStringAb = cReaderAb.readDetaLine();
    cInputStringSp = cReaderSp.readDetaLine();
    cInputStringGm = cReaderGm.readDetaLine();
    cInputStringFn = cReaderFn.readDetaLine();
    listCn.add(cInputString);
    if (null != cInputStringKorea) {
listKo.add(cInputStringKorea);
        }
        if (null != cInputStringJapan) {
listJp.add(cInputStringJapan);
        }
        if (null != cInputStringTrandition) {
listTt.add(cInputStringTrandition);
        }
        if (null != cInputStringEnglish) {
listEn.add(cInputStringEnglish);
        }
        if (null != cInputStringPinYin) {
listPy.add(cInputStringPinYin);
        }
        if (null != cInputStringRs) {
listRs.add(cInputStringRs);
        }
        if (null != cInputStringAb) {
listAb.add(cInputStringAb);
        }
        if (null != cInputStringSp) {
listSp.add(cInputStringSp);
        }
        if (null != cInputStringFn) {
listFn.add(cInputStringFn);
        }
        if (null != cInputStringGm) {
listGm.add(cInputStringGm);
        }
        //
        if (!(!cInputString.replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING)
.equals(S_Pos.EMPTY_STRING)
&& cInputString.split(

```

```

        S_Pos.NLP_SYMBO_SLASH).length > S_Pos.INT_ONE)) {

continue Here;

        }

        if (S_Pos.INT_ONE == cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
S_Maps.CiOne.put(

        cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

        new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        }

        if (S_Pos.INT_TWO == cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
S_Maps.CiTwo.put(

        cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

        new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        }

        if (S_Pos.INT_THREE == cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
S_Maps.CiThree.put(

        cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

        new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        }

        if (S_Pos.INT_FOUR == cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
S_Maps.CiFour.put(

        cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

        new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        }

        // pos

        loagPOS(cInputString, cInputStringKorea, cInputStringJapan,

        cInputStringTrandition, cInputStringEnglish, cInputStringRs,

        cInputStringAb, cInputStringFn, cInputStringGm,

        cInputStringSp);

        // langu

        loadLanguage(cInputString, cInputStringKorea, cInputStringJapan,

        cInputStringTrandition, cInputStringEnglish, cInputStringRs,

        cInputStringAb, cInputStringFn, cInputStringGm,

        cInputStringSp);

    }

    // end

```

```
cReader.closeDeta();
cReaderKorea.closeDeta();
cReaderJapan.closeDeta();
cReaderTrandition.closeDeta();
cReaderEnglish.closeDeta();
cReaderPinYin.closeDeta();
cReaderRs.closeDeta();
cReaderAb.closeDeta();
cReaderFn.closeDeta();
cReaderSp.closeDeta();
cReaderGm.closeDeta();
// inputStream.closeDeta();
cReader.closeDeta();
// inputStreamKorea.closeDeta();
cReaderKorea.closeDeta();
// inputStreamJapan.closeDeta();
cReaderJapan.closeDeta();
// inputStreamTrandition.closeDeta();
cReaderTrandition.closeDeta();
// inputStreamEnglish.closeDeta();
cReaderEnglish.closeDeta();
// inputStreamPinYin.closeDeta();
cReaderPinYin.closeDeta();
// inputStreamRs.closeDeta();
cReaderRs.closeDeta();
// inputStreamAb.closeDeta();
cReaderAb.closeDeta();
cReaderSp.closeDeta();
cReaderGm.closeDeta();
cReaderFn.closeDeta();
// trim
S_Maps.baDongCi.remove("");
S_Maps.daiCi.remove("");
S_Maps.dingMingCi.remove("");
S_Maps.dongCi.remove("");
S_Maps.fuCi.remove("");
S_Maps.jieCi.remove("");
S_Maps.lianCi.remove("");
S_Maps.liangCi.remove("");
S_Maps.mingCi.remove("");
S_Maps.qingTaiCi.remove("");
S_Maps.shengLueCi.remove("");
S_Maps.shiTaiCi.remove("");
S_Maps.weiCi.remove("");
S_Maps.xianDingCi.remove("");
S_Maps.xingRongCi.remove("");
S_Maps.xingWeiCi.remove("");
```

```
S_Maps.zhuCi.remove("");
// 20230106-S_logger.Log.logger.info("" + S_Maps.CiOne.size());
// 20230106-S_logger.Log.logger.info("" + S_Maps.CiTwo.size());
// 20230106-S_logger.Log.logger.info("" + S_Maps.CiThree.size());
// 20230106-S_logger.Log.logger.info("" + S_Maps.CiFour.size());
// System.gc();
// DetaThread.sleepDeta(200);
}
@Override
public void flushRedundant() {
    listEn.clear();
    listCn.clear();
    listFn.clear();
    listKo.clear();
    listJp.clear();
    listSp.clear();
    listGm.clear();
    listRs.clear();
    listAb.clear();
    listPy.clear();
    listTt.clear();
    //
    listEn = null;
    listCn = null;
    listFn = null;
    listKo = null;
    listJp = null;
    listSp = null;
    listGm = null;
    listRs = null;
    listAb = null;
    listPy = null;
    listTt = null;
    System.gc();
}
}
/*
* public void indexAll() { studyPos = new IMV_SQI();
* posCnToCn = new IMV_SQI(); linkedHashMap = new IMV_SQI();
* listCn = new CopyOnWriteArrayList<>(); listAb = new
* CopyOnWriteArrayList<>(); listKo = new
* CopyOnWriteArrayList<>(); listJp = new
* CopyOnWriteArrayList<>(); listRs = new
* CopyOnWriteArrayList<>(); listTt = new
* CopyOnWriteArrayList<>(); listEn = new
* CopyOnWriteArrayList<>(); listPy = new
* CopyOnWriteArrayList<>(); listFn = new
```

```
* CopyOnWriteArrayList<>(); listSp = new
* CopyOnWriteArrayList<>(); listGm = new
* CopyOnWriteArrayList<>(); InputStream inputStream =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_CN); DataBufferedReader
* cReader = new
* DataBufferedReader(DataInputStreamReader.E(inputStream,
* S_Pos.UTF8_STRING)); InputStream inputStreamKorea =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_KO); DataBufferedReader
* cReaderKorea = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamKorea, S_Pos.UTF8_STRING)); InputStream
* inputStreamJapan = getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_JP); DataBufferedReader
* cReaderJapan = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamJapan, S_Pos.UTF8_STRING)); InputStream
* inputStreamTrandition =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_TT); DataBufferedReader
* cReaderTrandition = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamTrandition, S_Pos.UTF8_STRING)); InputStream
* inputStreamEnglish =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_EN_TO_EN); DataBufferedReader
* cReaderEnglish = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamEnglish, S_Pos.UTF8_STRING)); InputStream
* inputStreamPinYin = getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_PY); DataBufferedReader
* cReaderPinYin = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamPinYin, S_Pos.UTF8_STRING));
*
* InputStream inputStreamRs =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_RS); DataBufferedReader
* cReaderRs = new
* DataBufferedReader(DataInputStreamReader.E(inputStreamRs,
* S_Pos.UTF8_STRING));
*
* InputStream inputStreamAb =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_AB); DataBufferedReader
* cReaderAb = new
```

```
* DetabufferedReader(DetaInputStreamReader.E(inputStreamAb,
* S_Pos.UTF8_STRING));
*
* InputStream inputStreamFn =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_FN); DetabufferedReader
* cReaderFn = new
* DetabufferedReader(DetaInputStreamReader.E(inputStreamFn,
* S_Pos.UTF8_STRING));
*
* InputStream inputStreamGm =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_GM); DetabufferedReader
* cReaderGm = new
* DetabufferedReader(DetaInputStreamReader.E(inputStreamGm,
* S_Pos.UTF8_STRING));
*
* InputStream inputStreamSp =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_SP); DetabufferedReader
* cReaderSp = new
* DetabufferedReader(DetaInputStreamReader.E(inputStreamSp,
* S_Pos.UTF8_STRING));
*
* String cInputString; String cInputStringKorea; String
* cInputStringJapan; String cInputStringTrandition; String
* cInputStringEnglish; String cInputStringPinYin; String
* cInputStringRs; String cInputStringAb; String
* cInputStringFn; String cInputStringGm; String
* cInputStringSp;
*
* Here: while ((cInputString = cReader.readDetaLine()) !=
* null) { cInputStringKorea = cReaderKorea.readDetaLine();
* cInputStringJapan = cReaderJapan.readDetaLine();
* cInputStringTrandition =
* cReaderTrandition.readDetaLine(); cInputStringEnglish =
* cReaderEnglish.readDetaLine(); cInputStringPinYin =
* cReaderPinYin.readDetaLine(); cInputStringRs =
* cReaderRs.readDetaLine(); cInputStringAb =
* cReaderAb.readDetaLine();
*
* cInputStringFn = cReaderFn.readDetaLine(); cInputStringSp
* = cReaderGm.readDetaLine(); cInputStringGm =
* cReaderSp.readDetaLine(); listCn.add(cInputString); if
* (null != cInputStringKorea) {
* listKo.add(cInputStringKorea); } if (null !=
* cInputStringJapan) { listJp.add(cInputStringJapan); } if
```

```

* (null != cInputStringTrandition) {
* listTt.add(cInputStringTrandition); if (S_Pos.INT_ONE ==
* cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
* S_Maps.CiOne.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
*
* } if (S_Pos.INT_TWO == cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
* S_Maps.CiTwo.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE])); } if
* (S_Pos.INT_THREE == cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
* S_Maps.CiThree.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE])); } if
* (S_Pos.INT_FOUR == cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
* S_Maps.CiFour.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE])); } } if
* (null != cInputStringEnglish) {
* listEn.add(cInputStringEnglish); } if (null !=
* cInputStringPinYin) { listPy.add(cInputStringPinYin); }
* if (null != cInputStringRs) { listRs.add(cInputStringRs);
* } if (null != cInputStringAb) {
* listAb.add(cInputStringAb); } if (null != cInputStringFn)
* { listFn.add(cInputStringFn); } if (null !=
* cInputStringSp) { listSp.add(cInputStringSp); } if (null
* != cInputStringGm) { listGm.add(cInputStringGm); } if
* (!(cInputString.replace(S_Pos.SPACE_STRING,
* S_Pos.EMPTY_STRING) .equals(S_Pos.EMPTY_STRING) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH) .length >
* S_Pos.INT_ONE)) { continue Here; } if (S_Pos.INT_ONE ==
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
* .length()) {
* S_Maps.CiOne.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH
* )[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (S_Pos.INT_TWO ==
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]

```

```

* .length()) {
* S_Maps.CiTwo.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH
* )[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (S_Pos.INT_THREE ==
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
* .length()) { S_Maps.CiThree.put(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (S_Pos.INT_FOUR ==
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
* .length()) { S_Maps.CiFour.put(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if
* (!S_Maps.fuCi.containsKey(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_FU)) {
* S_Maps.fuCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)
* [S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.fuCi.put(cInputStringKorea.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.fuCi.put(cInputStringJapan.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.fuCi.put(cInputStringTrandition.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.fuCi.put(cInputStringEnglish.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.dongCi.containsKey(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_DONG)) {
* S_Maps.dongCi.put(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {

```

```
* S_Maps.dongCi.put(cInputStringKorea.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {  
* S_Maps.dongCi.put(cInputStringJapan.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {  
* S_Maps.dongCi.put(cInputStringTrandition.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {  
* S_Maps.dongCi.put(cInputStringEnglish.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } } if  
* (!S_Maps.liangCi.containsKey(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
* .contains(S_Pos.NLP_CI_LIANG)) {  
* S_Maps.liangCi.put(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {  
* S_Maps.liangCi.put(cInputStringKorea.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {  
* S_Maps.liangCi.put(cInputStringJapan.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {  
* S_Maps.liangCi.put(cInputStringTrandition.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {  
* S_Maps.liangCi.put(cInputStringEnglish.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } } if  
* (!S_Maps.lianCi.containsKey(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
* .contains(S_Pos.NLP_CI_LIAN)) {  
* S_Maps.lianCi.put(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
```



```
* S_Pos.INT_ONE)); if (null != cInputStringKorea) {
* S_Maps.lianCi.put(cInputStringKorea.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.lianCi.put(cInputStringJapan.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.lianCi.put(cInputStringTrandition.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.lianCi.put(cInputStringEnglish.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.baDongCi.containsKey(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_BA_DONG)) {
* S_Maps.baDongCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.baDongCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.baDongCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.baDongCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.baDongCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.xianDingCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_XIAN_DING)) {
* S_Maps.xianDingCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
```

```
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.xianDingCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.xianDingCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.xianDingCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.xianDingCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.mingCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_MING)) {
* S_Maps.mingCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.mingCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.mingCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.mingCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.mingCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.daiCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_DAI)) {
* S_Maps.daiCi.put(cInputString
```

```
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.daiCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.daiCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.daiCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.daiCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.jieCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE].
* contains(S_Pos.NLP_CI_JIE)) {
* S_Maps.jieCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH
* )[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.jieCi.put(cInputStringKorea.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.jieCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.jieCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.jieCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.xingRongCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_XING_RONG)) {
```

```
* S_Maps.xingRongCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.xingRongCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.xingRongCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.xingRongCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.xingRongCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.zhuCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_ZHU)) {
* S_Maps.zhuCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.zhuCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.zhuCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.zhuCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.zhuCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.weiCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE].
```

```
* contains(S_Pos.NLP_CI_WEI)) {  
  
* S_Maps.weiCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH  
*)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {  
* S_Maps.weiCi.put(cInputStringKorea  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {  
* S_Maps.weiCi.put(cInputStringJapan  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {  
* S_Maps.weiCi.put(cInputStringTrandition  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {  
* S_Maps.weiCi.put(cInputStringEnglish  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } } if  
* (!S_Maps.shengLueCi.containsKey(cInputString  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
* .contains(S_Pos.NLP_CI_SHENG_LUE)) {  
* S_Maps.shengLueCi.put(cInputString  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {  
* S_Maps.shengLueCi.put(cInputStringKorea  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {  
* S_Maps.shengLueCi.put(cInputStringJapan  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {  
* S_Maps.shengLueCi.put(cInputStringTrandition  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {  
* S_Maps.shengLueCi.put(cInputStringEnglish  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } } if  
* (!S_Maps.qingTaiCi.containsKey(cInputString  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
```

```
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]

* .contains(S_Pos.NLP_CI_QING_TAI)) {

* S_Maps.qingTaiCi.put(cInputString

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); if (null != cInputStringKorea) {

* S_Maps.qingTaiCi.put(cInputStringKorea

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {

* S_Maps.qingTaiCi.put(cInputStringJapan

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {

* S_Maps.qingTaiCi.put(cInputStringTrandition

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {

* S_Maps.qingTaiCi.put(cInputStringEnglish

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); } } if

* (!S_Maps.xingWeiCi.containsKey(cInputString

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&

* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]

* .contains(S_Pos.NLP_CI_XING_WEI)) {

* S_Maps.xingWeiCi.put(cInputString

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); if (null != cInputStringKorea) {

* S_Maps.xingWeiCi.put(cInputStringKorea

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {

* S_Maps.xingWeiCi.put(cInputStringJapan

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {

* S_Maps.xingWeiCi.put(cInputStringTrandition

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {

* S_Maps.xingWeiCi.put(cInputStringEnglish

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[

* S_Pos.INT_ONE])); } } if

* (!S_Maps.shiTaiCi.containsKey(cInputString
```

```
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_SHI_TAI)) {
* S_Maps.shiTaiCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.shiTaiCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.shiTaiCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.shiTaiCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.shiTaiCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.dingMingCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_DING_MING)) {
* S_Maps.dingMingCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.dingMingCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.dingMingCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.dingMingCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.dingMingCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } }
```

```

* studyPos.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE]));
* posCnToCn.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); linkedHashMap =
* loopLoadForest(cInputString); if (null !=
* cInputStringKorea) { posCnToCn.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); linkedHashMap =
* loopLoadForest(cInputStringKorea); } if (null !=
* cInputStringJapan) { if
* (!posCnToCn.containsKey(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])) {
* posCnToCn.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] ,
* cInputStringJapan.split(S_Pos.NLP_SYMBO_SLASH).length >
* S_Pos.INT_ONE ? cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE] : "未知");
* linkedHashMap = loopLoadForest(cInputStringJapan); } } if
* (null != cInputStringTrandition) { if
* (!posCnToCn.containsKey(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])) {
* posCnToCn.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); linkedHashMap =
* loopLoadForest(cInputStringTrandition); } } if (null !=
* cInputStringEnglish) { posCnToCn.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase
* (), new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringRs) {
* posCnToCn.put(cInputStringRs
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase
* (), new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringAb) {
* posCnToCn.put(cInputStringAb
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase
* (), new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); }
*

```



```
* if (null != cInputStringFn) {
* posCnToCn.put(cInputStringFn
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase
* () , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringGm) {
* posCnToCn.put(cInputStringGm
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase
* () , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringSp) {
* posCnToCn.put(cInputStringSp
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase
* () , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } cReader.closeDeta();
* cReaderKorea.closeDeta(); cReaderJapan.closeDeta();
* cReaderTrandition.closeDeta();
* cReaderEnglish.closeDeta(); cReaderPinYin.closeDeta();
* cReaderRs.closeDeta(); cReaderAb.closeDeta();
* cReaderFn.closeDeta(); cReaderGm.closeDeta();
* cReaderSp.closeDeta(); try { inputStream.close();
* cReader.closeDeta(); inputStreamKorea.close();
* cReaderKorea.closeDeta(); inputStreamJapan.close();
* cReaderJapan.closeDeta(); inputStreamTrandition.close();
* cReaderTrandition.closeDeta();
* inputStreamEnglish.close(); cReaderEnglish.closeDeta();
* inputStreamPinYin.close(); cReaderPinYin.closeDeta();
* inputStreamRs.close(); cReaderRs.closeDeta();
* inputStreamAb.close(); cReaderAb.closeDeta();
* inputStreamFn.close(); cReaderFn.closeDeta();
* inputStreamSp.close(); cReaderSp.closeDeta();
* inputStreamFn.close(); cReaderFn.closeDeta();
*
* } catch (IOException e) { throw new RuntimeException(e);
* } //trim S_Maps.baDongCi.remove("");
* S_Maps.daiCi.remove(""); S_Maps.dingMingCi.remove("");
* S_Maps.dongCi.remove(""); S_Maps.fuCi.remove("");
* S_Maps.jieCi.remove(""); S_Maps.lianCi.remove("");
* S_Maps.liangCi.remove(""); S_Maps.mingCi.remove("");
* S_Maps.qingTaiCi.remove("");
* S_Maps.shengLueCi.remove(""); S_Maps.shiTaiCi.remove("");
* S_Maps.weiCi.remove(""); S_Maps.xianDingCi.remove("");
* S_Maps.xingRongCi.remove("");
* S_Maps.xingWeiCi.remove(""); S_Maps.zhuCi.remove("");
* //20230106-S_logger.Log.logger.info("" + S_Maps.CiOne.size());
* //20230106-S_logger.Log.logger.info("" + S_Maps.CiTwo.size());
```

```
* //20230106-S_logger.Log.logger.info("'" + S_Maps.CiThree.size());
* //20230106-S_logger.Log.logger.info("'" + S_Maps.CiFour.size()); }
*/
//
// public void loadLanguage(String cInputString, String
// cInputStringKorea
//     , String cInputStringJapan, String cInputStringTrandition
//     , String cInputStringEnglish, String cInputStringRs, String cInputStringAb
//     , String cInputStringFn, String cInputStringGm, String cInputStringSp) {
//     studyPos.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//         , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//     posCnToCn.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//         , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//     linkedHashMap = loopLoadForest(cInputString);
//     if (null != cInputStringKorea) {
//         posCnToCn.put(cInputStringKorea
//             .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//             , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//         linkedHashMap = loopLoadForest(cInputStringKorea);
//     }
//     if (null != cInputStringJapan) {
//         if (!posCnToCn.containsKey(cInputStringJapan
//             .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])) {
//             posCnToCn.put(cInputStringJapan
//                 .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//                 , cInputStringJapan.split(S_Pos.NLP_SYMBO_SLASH).length
//                 > S_Pos.INT_ONE ? cInputStringJapan.split(
//                     S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE] : "未知");
//             linkedHashMap = loopLoadForest(cInputStringJapan);
//         }
//     }
//     if (null != cInputStringTrandition) {
//         if (!posCnToCn.containsKey(cInputStringTrandition
//             .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])) {
//             posCnToCn.put(cInputStringTrandition
//                 .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//                 , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//             linkedHashMap = loopLoadForest(cInputStringTrandition);
//         }
//     }
//     if (null != cInputStringEnglish) {
//         posCnToCn.put(cInputStringEnglish
//             .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase()
//             , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//     }
//     if (null != cInputStringRs) {
//         posCnToCn.put(cInputStringRs
```

```
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase()
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringAb) {
//      posCnToCn.put(cInputStringAb
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase()
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringFn) { //注意拉丁语 系列碎片去重map混淆
//      posCnToCn.put(cInputStringFn
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase()
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringGm) {
//      posCnToCn.put(cInputStringGm
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase()
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringSp) {
//      posCnToCn.put(cInputStringSp
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase()
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  }
//
//  public void loagPOS(String cInputString, String cInputStringKorea
//      , String cInputStringJapan, String cInputStringTrandition
//      , String cInputStringEnglish, String cInputStringRs, String cInputStringAb
//      , String cInputStringFn, String cInputStringGm, String cInputStringSp) {
//      if (!S_Maps.fuCi.containsKey(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//          && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//          .contains(S_Pos.NLP_CI_FU)) {
//          S_Maps.fuCi.put(cInputString
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//          if (null != cInputStringKorea) {
//              S_Maps.fuCi.put(cInputStringKorea
//                  .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//                  , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//          }
//          if (null != cInputStringJapan) {
//              S_Maps.fuCi.put(cInputStringJapan
//                  .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//                  , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//          }
//      }
```

```
//      if (null != cInputStringTrandition) {
//          S_Maps.fuCi.put(cInputStringTrandition.
//              split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringEnglish) {
//          S_Maps.fuCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.dongCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_DONG)) {
//      S_Maps.dongCi.put(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.dongCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringJapan) {
//          S_Maps.dongCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringTrandition) {
//          S_Maps.dongCi.put(cInputStringTrandition
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringEnglish) {
//          S_Maps.dongCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.liangCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE].contains(S_Pos.NLP_CI_LIANG)) {
//      S_Maps.liangCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.liangCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
```

```
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringJapan) {
//      S_Maps.liangCi.put(cInputStringJapan
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringTrandition) {
//      S_Maps.liangCi.put(cInputStringTrandition
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringEnglish) {
//      S_Maps.liangCi.put(cInputStringEnglish
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  }
//  if (!S_Maps.lianCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_LIAN)) {
//      S_Maps.lianCi.put(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  if (null != cInputStringKorea) {
//      S_Maps.lianCi.put(cInputStringKorea
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringJapan) {
//      S_Maps.lianCi.put(cInputStringJapan
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringTrandition) {
//      S_Maps.lianCi.put(cInputStringTrandition
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringEnglish) {
//      S_Maps.lianCi.put(cInputStringEnglish
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  }
//  if (!S_Maps.baDongCi.containsKey(cInputString
```

```
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_BA_DONG)) {
//      S_Maps.baDongCi.put(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.baDongCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringJapan) {
//          S_Maps.baDongCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringTrandition) {
//          S_Maps.baDongCi.put(cInputStringTrandition
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringEnglish) {
//          S_Maps.baDongCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.xianDingCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_XIAN_DING)) {
//      S_Maps.xianDingCi.put(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.xianDingCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringJapan) {
//          S_Maps.xianDingCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringTrandition) {
//          S_Maps.xianDingCi.put(cInputStringTrandition
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
```

```
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringEnglish) {
//      S_Maps.xianDingCi.put(cInputStringEnglish
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  }
//  if (!S_Maps.mingCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_MING)) {
//      S_Maps.mingCi.put(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.mingCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringJapan) {
//          S_Maps.mingCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringTrandition) {
//          S_Maps.mingCi.put(cInputStringTrandition.
//              split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringEnglish) {
//          S_Maps.mingCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.daiCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_DAI)) {
//      S_Maps.daiCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.daiCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
```

```
//      if (null != cInputStringJapan) {
//          S_Maps.daiCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringTrandition) {
//          S_Maps.daiCi.put(cInputStringTrandition.
//              split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringEnglish) {
//          S_Maps.daiCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.jieCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_JIE)) {
//      S_Maps.jieCi.put(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.jieCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringJapan) {
//          S_Maps.jieCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringTrandition) {
//          S_Maps.jieCi.put(cInputStringTrandition
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringEnglish) {
//          S_Maps.jieCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.xingRongCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
```



```
//      .contains(S_Pos.NLP_CI_XING_RONG)) {  
//      S_Maps.xingRongCi.put(cInputString  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      if (null != cInputStringKorea) {  
//      S_Maps.xingRongCi.put(cInputStringKorea  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      }  
//      if (null != cInputStringJapan) {  
//      S_Maps.xingRongCi.put(cInputStringJapan  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      }  
//      if (null != cInputStringTrandition) {  
//      S_Maps.xingRongCi.put(cInputStringTrandition  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      }  
//      if (null != cInputStringEnglish) {  
//      S_Maps.xingRongCi.put(cInputStringEnglish  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      }  
//      }  
//      if (!S_Maps.zhuCi.containsKey(cInputString  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])  
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
//      .contains(S_Pos.NLP_CI_ZHU)) {  
//      S_Maps.zhuCi.put(cInputString  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      if (null != cInputStringKorea) {  
//      S_Maps.zhuCi.put(cInputStringKorea  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      }  
//      if (null != cInputStringJapan) {  
//      S_Maps.zhuCi.put(cInputStringJapan  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      }  
//      if (null != cInputStringTrandition) {  
//      S_Maps.zhuCi.put(cInputStringTrandition  
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]  
//      , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));  
//      }  
//      }
```

```
//      if (null != cInputStringEnglish) {
//          S_Maps.zhuCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.weiCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE].contains(S_Pos.NLP_CI_WEI)) {
//      S_Maps.weiCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.weiCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringJapan) {
//          S_Maps.weiCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringTrandition) {
//          S_Maps.weiCi.put(cInputStringTrandition
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringEnglish) {
//          S_Maps.weiCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.shengLueCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_SHENG_LUE)) {
//      S_Maps.shengLueCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      if (null != cInputStringKorea) {
//          S_Maps.shengLueCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringJapan) {
//          S_Maps.shengLueCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
```

```
//    }
//    if (null != cInputStringTrandition) {
//        S_Maps.shengLueCi.put(cInputStringTrandition
//            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//            , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//    }
//    if (null != cInputStringEnglish) {
//        S_Maps.shengLueCi.put(cInputStringEnglish
//            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//            , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//    }
// }
// if (!S_Maps.qingTaiCi.containsKey(cInputString
//     .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//     && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//     .contains(S_Pos.NLP_CI_QING_TAI)) {
//     S_Maps.qingTaiCi.put(cInputString
//         .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//         , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
// }
// if (null != cInputStringKorea) {
//     S_Maps.qingTaiCi.put(cInputStringKorea
//         .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//         , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
// }
// if (null != cInputStringJapan) {
//     S_Maps.qingTaiCi.put(cInputStringJapan
//         .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//         , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
// }
// if (null != cInputStringTrandition) {
//     S_Maps.qingTaiCi.put(cInputStringTrandition
//         .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//         , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
// }
// if (null != cInputStringEnglish) {
//     S_Maps.qingTaiCi.put(cInputStringEnglish
//         .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//         , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
// }
// }
// if (!S_Maps.xingWeiCi.containsKey(cInputString
//     .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//     && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//     .contains(S_Pos.NLP_CI_XING_WEI)) {
//     S_Maps.xingWeiCi.put(cInputString
//         .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//         , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
// }
```

```
//      if (null != cInputStringKorea) {
//          S_Maps.xingWeiCi.put(cInputStringKorea
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringJapan) {
//          S_Maps.xingWeiCi.put(cInputStringJapan
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringTrandition) {
//          S_Maps.xingWeiCi.put(cInputStringTrandition
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//      if (null != cInputStringEnglish) {
//          S_Maps.xingWeiCi.put(cInputStringEnglish
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//      }
//  }
//  if (!S_Maps.shiTaiCi.containsKey(cInputString
//      .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//      && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//      .contains(S_Pos.NLP_CI_SHI_TAI)) {
//      S_Maps.shiTaiCi.put(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringKorea) {
//      S_Maps.shiTaiCi.put(cInputStringKorea
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringJapan) {
//      S_Maps.shiTaiCi.put(cInputStringJapan
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringTrandition) {
//      S_Maps.shiTaiCi.put(cInputStringTrandition
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
//  if (null != cInputStringEnglish) {
//      S_Maps.shiTaiCi.put(cInputStringEnglish
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//          , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//  }
```

```
//      }
//      }
//      if (!S_Maps.dingMingCi.containsKey(cInputString
//          .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
//          && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
//          .contains(S_Pos.NLP_CL_DING_MING)) {
//          S_Maps.dingMingCi.put(cInputString
//              .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//              , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//          if (null != cInputStringKorea) {
//              S_Maps.dingMingCi.put(cInputStringKorea
//                  .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//                  , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//          }
//          if (null != cInputStringJapan) {
//              S_Maps.dingMingCi.put(cInputStringJapan
//                  .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//                  , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//          }
//          if (null != cInputStringTrandition) {
//              S_Maps.dingMingCi.put(cInputStringTrandition
//                  .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//                  , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//          }
//          if (null != cInputStringEnglish) {
//              S_Maps.dingMingCi.put(cInputStringEnglish
//                  .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
//                  , new StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
//          }
//      }
//  }
//  }
//  private void loadLength() {
//  }
//  }
```

```
/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/FMHMMListOneTime_EX.java
package E_A.OEI.SVQ.MPC.fhmm.E;
```

```
import C_A.OCLSVQ.MPC.fhmm.C.FHMMList_indexAll_X;
```

```
import S_A.SVQ.stable.S_Maps;
```

```
import S_A.SVQ.stable.S_Pos;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光. 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

 **/

```

@SuppressWarnings({ "unchecked", "resource", "unused" })

public class FMHMMListOneTime_EX extends FMHMMListOneTime_E_maps
    implements FMHMMList_indexAll_X {

    public void loadLanguage(String cInputString, String cInputStringKorea,
        String cInputStringJapan, String cInputStringTrandition,
        String cInputStringEnglish, String cInputStringRs,
        String cInputStringAb, String cInputStringFn, String cInputStringGm,
        String cInputStringSp) {
        studyPos.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        posCnToCn.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        linkedHashMap = loopLoadForest(cInputString);
        if (null != cInputStringKorea) {
            posCnToCn.put(
cInputStringKorea
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
            linkedHashMap = loopLoadForest(cInputStringKorea);
        }
        if (null != cInputStringJapan) {
            if (!posCnToCn.containsKey(cInputStringJapan
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])) {
posCnToCn.put(
                cInputStringJapan
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                cInputStringJapan.split(
S_Pos.NLP_SYMBO_SLASH).length > S_Pos.INT_ONE
                    ? cInputStringJapan.split(
S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
                    : "未知");
linkedHashMap = loopLoadForest(cInputStringJapan);
            }
        }
        if (null != cInputStringTrandition) {
            if (!posCnToCn.containsKey(cInputStringTrandition
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])) {
posCnToCn.put(
                cInputStringTrandition
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
linkedHashMap = loopLoadForest(cInputStringTrandition);
            }
        }
    }
}

```

```

    }
}

if (null != cInputStringEnglish) {
    posCnToCn.put(
cInputStringEnglish
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase(),
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}

if (null != cInputStringRs) {
    posCnToCn.put(
cInputStringRs.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase(),
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}

if (null != cInputStringAb) {
    posCnToCn.put(
cInputStringAb.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase(),
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}

if (null != cInputStringFn) { // 注意拉丁语 系列碎片去重map混淆
    posCnToCn.put(
cInputStringFn.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase(),
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}

if (null != cInputStringGm) {
    posCnToCn.put(
cInputStringGm.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase(),
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}

if (null != cInputStringSp) {
    posCnToCn.put(
cInputStringSp.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
        .toLowerCase(),
new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}
}

public void loagPOS(String cInputString, String cInputStringKorea,

```

```

        String cInputStringJapan, String cInputStringTrandition,
        String cInputStringEnglish, String cInputStringRs,
        String cInputStringAb, String cInputStringFn, String cInputStringGm,
        String cInputStringSp) {
    if (!S_Maps.fuCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_FU)) {
        S_Maps.fuCi.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        if (null != cInputStringKorea) {
S_Maps.fuCi.put(
            cInputStringKorea
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringJapan) {
S_Maps.fuCi.put(
            cInputStringJapan
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringTrandition) {
S_Maps.fuCi.put(
            cInputStringTrandition
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringEnglish) {
S_Maps.fuCi.put(
            cInputStringEnglish
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
    }
    if (!S_Maps.dongCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_DONG)) {
        S_Maps.dongCi.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

```



```

        new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        if (null != cInputStringKorea) {
S_Maps.dongCi.put(
            cInputStringKorea
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringJapan) {
S_Maps.dongCi.put(
            cInputStringJapan
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringTrandition) {
S_Maps.dongCi.put(
            cInputStringTrandition
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringEnglish) {
S_Maps.dongCi.put(
            cInputStringEnglish
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
    }
    if (!S_Maps.liangCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_LIANG)) {
        S_Maps.liangCi.put(
            cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        if (null != cInputStringKorea) {
S_Maps.liangCi.put(
            cInputStringKorea
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringJapan) {

```

```
S_Maps.liangCi.put(
    cInputStringJapan
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}
if (null != cInputStringTrandition) {
S_Maps.liangCi.put(
    cInputStringTrandition
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}
if (null != cInputStringEnglish) {
S_Maps.liangCi.put(
    cInputStringEnglish
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}
}
if (!S_Maps.lianCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
    .contains(S_Pos.NLP_CI_LIAN)) {
    S_Maps.lianCi.put(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    if (null != cInputStringKorea) {
S_Maps.lianCi.put(
    cInputStringKorea
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}
if (null != cInputStringJapan) {
S_Maps.lianCi.put(
    cInputStringJapan
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
}
if (null != cInputStringTrandition) {
S_Maps.lianCi.put(
    cInputStringTrandition
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
```

```

        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringEnglish) {
S_Maps.lianCi.put(
        cInputStringEnglish
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (!S_Maps.baDongCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_BA_DONG)) {
        S_Maps.baDongCi.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        if (null != cInputStringKorea) {
S_Maps.baDongCi.put(
            cInputStringKorea
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringJapan) {
S_Maps.baDongCi.put(
            cInputStringJapan
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringTrandition) {
S_Maps.baDongCi.put(
            cInputStringTrandition
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringEnglish) {
S_Maps.baDongCi.put(
            cInputStringEnglish
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
    }
}

```

```

    }

    if (!S_Maps.xianDingCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
.contains(S_Pos.NLP_CI_XIAN_DING)) {
        S_Maps.xianDingCi.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        if (null != cInputStringKorea) {
S_Maps.xianDingCi.put(
cInputStringKorea
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringJapan) {
S_Maps.xianDingCi.put(
cInputStringJapan
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringTrandition) {
S_Maps.xianDingCi.put(
cInputStringTrandition
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringEnglish) {
S_Maps.xianDingCi.put(
cInputStringEnglish
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
    }

    if (!S_Maps.mingCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
.contains(S_Pos.NLP_CI_MING)) {
        S_Maps.mingCi.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        if (null != cInputStringKorea) {

```

```

    S_Maps.mingCi.put(
        cInputStringKorea
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringJapan) {
S_Maps.mingCi.put(
        cInputStringJapan
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringTrandition) {
S_Maps.mingCi.put(
        cInputStringTrandition
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringEnglish) {
S_Maps.mingCi.put(
        cInputStringEnglish
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
}
if (!S_Maps.daiCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
.contains(S_Pos.NLP_CI_DAI)) {
    S_Maps.daiCi.put(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    if (null != cInputStringKorea) {
S_Maps.daiCi.put(
        cInputStringKorea
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringJapan) {
S_Maps.daiCi.put(
        cInputStringJapan
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

```

```

        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringTrandition) {
S_Maps.daiCi.put(
        cInputStringTrandition
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringEnglish) {
S_Maps.daiCi.put(
        cInputStringEnglish
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
}
if (!S_Maps.jieCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
.contains(S_Pos.NLP_CI_JIE)) {
    S_Maps.jieCi.put(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    if (null != cInputStringKorea) {
S_Maps.jieCi.put(
        cInputStringKorea
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringJapan) {
S_Maps.jieCi.put(
        cInputStringJapan
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringTrandition) {
S_Maps.jieCi.put(
        cInputStringTrandition
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
}

```

```
        if (null != cInputStringEnglish) {
            S_Maps.jieCi.put(
                cInputStringEnglish
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
    }
    if (!S_Maps.xingRongCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
        .contains(S_Pos.NLP_CI_XING_RONG)) {
        S_Maps.xingRongCi.put(
            cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
            new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        if (null != cInputStringKorea) {
            S_Maps.xingRongCi.put(
                cInputStringKorea
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringJapan) {
            S_Maps.xingRongCi.put(
                cInputStringJapan
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringTrandition) {
            S_Maps.xingRongCi.put(
                cInputStringTrandition
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringEnglish) {
            S_Maps.xingRongCi.put(
                cInputStringEnglish
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
    }
    if (!S_Maps.zhuCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
```

```

    && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]

        .contains(S_Pos.NLP_CI_ZHU)) {

        S_Maps.zhuCi.put(

cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        if (null != cInputStringKorea) {

S_Maps.zhuCi.put(

        cInputStringKorea

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

        new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        }

        if (null != cInputStringJapan) {

S_Maps.zhuCi.put(

        cInputStringJapan

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

        new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        }

        if (null != cInputStringTrandition) {

S_Maps.zhuCi.put(

        cInputStringTrandition

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

        new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        }

        if (null != cInputStringEnglish) {

S_Maps.zhuCi.put(

        cInputStringEnglish

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

        new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        }

        }

        if (!S_Maps.weiCi.containsKey(

cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])

&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]

        .contains(S_Pos.NLP_CI_WEI)) {

        S_Maps.weiCi.put(

cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

new StringBuilder(cInputString

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));

        if (null != cInputStringKorea) {

S_Maps.weiCi.put(

        cInputStringKorea

        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],

```



```

        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringJapan) {
S_Maps.weiCi.put(
        cInputStringJapan
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringTrandition) {
S_Maps.weiCi.put(
        cInputStringTrandition
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringEnglish) {
S_Maps.weiCi.put(
        cInputStringEnglish
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
}
if (!S_Maps.shengLueCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
.contains(S_Pos.NLP_CI_SHENG_LUE)) {
    S_Maps.shengLueCi.put(
        cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
            .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    if (null != cInputStringKorea) {
S_Maps.shengLueCi.put(
        cInputStringKorea
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringJapan) {
S_Maps.shengLueCi.put(
        cInputStringJapan
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
        new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
}

```

```
        if (null != cInputStringTrandition) {
            S_Maps.shengLueCi.put(
                cInputStringTrandition
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringEnglish) {
            S_Maps.shengLueCi.put(
                cInputStringEnglish
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (!S_Maps.qingTaiCi.containsKey(
            cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
            && cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
                .contains(S_Pos.NLP_CI_QING_TAI)) {
            S_Maps.qingTaiCi.put(
                cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
            if (null != cInputStringKorea) {
                S_Maps.qingTaiCi.put(
                    cInputStringKorea
                    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                    new StringBuilder(cInputString
                    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
            }
            if (null != cInputStringJapan) {
                S_Maps.qingTaiCi.put(
                    cInputStringJapan
                    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                    new StringBuilder(cInputString
                    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
            }
            if (null != cInputStringTrandition) {
                S_Maps.qingTaiCi.put(
                    cInputStringTrandition
                    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                    new StringBuilder(cInputString
                    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
            }
            if (null != cInputStringEnglish) {
                S_Maps.qingTaiCi.put(
                    cInputStringEnglish
```

```
.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
}

if (!S_Maps.xingWeiCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
    .contains(S_Pos.NLP_CI_XING_WEI)) {
    S_Maps.xingWeiCi.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    if (null != cInputStringKorea) {
S_Maps.xingWeiCi.put(
    cInputStringKorea
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringJapan) {
S_Maps.xingWeiCi.put(
    cInputStringJapan
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringTrandition) {
S_Maps.xingWeiCi.put(
    cInputStringTrandition
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringEnglish) {
S_Maps.xingWeiCi.put(
    cInputStringEnglish
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
        .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
}

if (!S_Maps.shiTaiCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
    .contains(S_Pos.NLP_CI_SHI_TAI)) {
    S_Maps.shiTaiCi.put(
```

```
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    if (null != cInputStringKorea) {
S_Maps.shiTaiCi.put(
    cInputStringKorea
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringJapan) {
S_Maps.shiTaiCi.put(
    cInputStringJapan
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringTrandition) {
S_Maps.shiTaiCi.put(
    cInputStringTrandition
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    if (null != cInputStringEnglish) {
S_Maps.shiTaiCi.put(
    cInputStringEnglish
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    }
    if (!S_Maps.dingMingCi.containsKey(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])
&& cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
    .contains(S_Pos.NLP_CI_DING_MING)) {
    S_Maps.dingMingCi.put(
cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    if (null != cInputStringKorea) {
S_Maps.dingMingCi.put(
    cInputStringKorea
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
    new StringBuilder(cInputString
    .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
    }
    }
```

```

        if (null != cInputStringJapan) {
            S_Maps.dingMingCi.put(
                cInputStringJapan
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringTrandition) {
            S_Maps.dingMingCi.put(
                cInputStringTrandition
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
        if (null != cInputStringEnglish) {
            S_Maps.dingMingCi.put(
                cInputStringEnglish
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO],
                new StringBuilder(cInputString
                .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
        }
    }
}

/*
 * public void indexAll() { studyPos = new IMV_SQI();
 * posCnToCn = new IMV_SQI(); linkedHashMap = new IMV_SQI();
 * listCn = new CopyOnWriteArrayList<>(); listAb = new
 * CopyOnWriteArrayList<>(); listKo = new
 * CopyOnWriteArrayList<>(); listJp = new
 * CopyOnWriteArrayList<>(); listRs = new
 * CopyOnWriteArrayList<>(); listTt = new
 * CopyOnWriteArrayList<>(); listEn = new
 * CopyOnWriteArrayList<>(); listPy = new
 * CopyOnWriteArrayList<>(); listFn = new
 * CopyOnWriteArrayList<>(); listSp = new
 * CopyOnWriteArrayList<>(); listGm = new
 * CopyOnWriteArrayList<>(); InputStream inputStream =
 * getClass().getResourceAsStream(S_Pos.
 * WORDS_SOURCE_LINK_POS_CN_TO_CN); DetaBufferedReader
 * cReader = new
 * DetaBufferedReader(DetaInputStreamReader.E(inputStream,
 * S_Pos.UTF8_STRING)); InputStream inputStreamKorea =
 * getClass().getResourceAsStream(S_Pos.
 * WORDS_SOURCE_LINK_POS_CN_TO_KO); DetaBufferedReader
 * cReaderKorea = new
 * DetaBufferedReader(DetaInputStreamReader.E(

```

```
* inputStreamKorea, S_Pos.UTF8_STRING)); InputStream
* inputStreamJapan = getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_JP); DataBufferedReader
* cReaderJapan = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamJapan, S_Pos.UTF8_STRING)); InputStream
* inputStreamTrandition =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_TT); DataBufferedReader
* cReaderTrandition = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamTrandition, S_Pos.UTF8_STRING)); InputStream
* inputStreamEnglish =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_EN_TO_EN); DataBufferedReader
* cReaderEnglish = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamEnglish, S_Pos.UTF8_STRING)); InputStream
* inputStreamPinYin = getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_PY); DataBufferedReader
* cReaderPinYin = new
* DataBufferedReader(DataInputStreamReader.E(
* inputStreamPinYin, S_Pos.UTF8_STRING));
*
* InputStream inputStreamRs =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_RS); DataBufferedReader
* cReaderRs = new
* DataBufferedReader(DataInputStreamReader.E(inputStreamRs,
* S_Pos.UTF8_STRING));
*
* InputStream inputStreamAb =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_AB); DataBufferedReader
* cReaderAb = new
* DataBufferedReader(DataInputStreamReader.E(inputStreamAb,
* S_Pos.UTF8_STRING));
*
* InputStream inputStreamFn =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_FN); DataBufferedReader
* cReaderFn = new
* DataBufferedReader(DataInputStreamReader.E(inputStreamFn,
* S_Pos.UTF8_STRING));
*
* InputStream inputStreamGm =
* getClass().getResourceAsStream(S_Pos.
```

```

* WORDS_SOURCE_LINK_POS_CN_TO_GM); DataBufferedReader
* cReaderGm = new
* DataBufferedReader(DataInputStreamReader.E(inputStreamGm,
* S_Pos.UTF8_STRING));
*
* InputStream inputStreamSp =
* getClass().getResourceAsStream(S_Pos.
* WORDS_SOURCE_LINK_POS_CN_TO_SP); DataBufferedReader
* cReaderSp = new
* DataBufferedReader(DataInputStreamReader.E(inputStreamSp,
* S_Pos.UTF8_STRING));
*
* String cInputString; String cInputStringKorea; String
* cInputStringJapan; String cInputStringTrandition; String
* cInputStringEnglish; String cInputStringPinYin; String
* cInputStringRs; String cInputStringAb; String
* cInputStringFn; String cInputStringGm; String
* cInputStringSp;
*
* Here: while ((cInputString = cReader.readDetaLine()) !=
* null) { cInputStringKorea = cReaderKorea.readDetaLine();
* cInputStringJapan = cReaderJapan.readDetaLine();
* cInputStringTrandition =
* cReaderTrandition.readDetaLine(); cInputStringEnglish =
* cReaderEnglish.readDetaLine(); cInputStringPinYin =
* cReaderPinYin.readDetaLine(); cInputStringRs =
* cReaderRs.readDetaLine(); cInputStringAb =
* cReaderAb.readDetaLine();
*
* cInputStringFn = cReaderFn.readDetaLine(); cInputStringSp
* = cReaderGm.readDetaLine(); cInputStringGm =
* cReaderSp.readDetaLine(); listCn.add(cInputString); if
* (null != cInputStringKorea) {
* listKo.add(cInputStringKorea); } if (null !=
* cInputStringJapan) { listJp.add(cInputStringJapan); } if
* (null != cInputStringTrandition) {
* listTt.add(cInputStringTrandition); if (S_Pos.INT_ONE ==
* cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
* S_Maps.CiOne.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]));
*
* } if (S_Pos.INT_TWO == cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
* S_Maps.CiTwo.put(cInputStringTrandition

```

```

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE])); } if
* (S_Pos.INT_THREE == cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
* S_Maps.CiThree.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE])); } if
* (S_Pos.INT_FOUR == cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].length()) {
* S_Maps.CiFour.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE])); } if
* (null != cInputStringEnglish) {
* listEn.add(cInputStringEnglish); } if (null !=
* cInputStringPinYin) { listPy.add(cInputStringPinYin); }
* if (null != cInputStringRs) { listRs.add(cInputStringRs);
* } if (null != cInputStringAb) {
* listAb.add(cInputStringAb); } if (null != cInputStringFn)
* { listFn.add(cInputStringFn); } if (null !=
* cInputStringSp) { listSp.add(cInputStringSp); } if (null
* != cInputStringGm) { listGm.add(cInputStringGm); } if
* (!cInputString.replace(S_Pos.SPACE_STRING,
* S_Pos.EMPTY_STRING) .equals(S_Pos.EMPTY_STRING) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH) .length >
* S_Pos.INT_ONE)) { continue Here; } if (S_Pos.INT_ONE ==
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
* .length()) {
* S_Maps.CiOne.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH
* )[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (S_Pos.INT_TWO ==
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
* .length()) {
* S_Maps.CiTwo.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH
* )[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (S_Pos.INT_THREE ==
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
* .length()) { S_Maps.CiThree.put(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (S_Pos.INT_FOUR ==
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]
* .length()) { S_Maps.CiFour.put(cInputString.split(S_Pos.

```



```
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if
* (!S_Maps.fuCi.containsKey(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_FU)) {
* S_Maps.fuCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)
* [S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.fuCi.put(cInputStringKorea.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.fuCi.put(cInputStringJapan.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.fuCi.put(cInputStringTrandition.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.fuCi.put(cInputStringEnglish.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.dongCi.containsKey(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_DONG)) {
* S_Maps.dongCi.put(cInputString.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.dongCi.put(cInputStringKorea.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.dongCi.put(cInputStringJapan.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.dongCi.put(cInputStringTrandition.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
```

```
* S_Maps.dongCi.put(cInputStringEnglish.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } } if  
* (!S_Maps.liangCi.containsKey(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
* .contains(S_Pos.NLP_CI_LIANG)) {  
* S_Maps.liangCi.put(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {  
* S_Maps.liangCi.put(cInputStringKorea.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {  
* S_Maps.liangCi.put(cInputStringJapan.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {  
* S_Maps.liangCi.put(cInputStringTrandition.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {  
* S_Maps.liangCi.put(cInputStringEnglish.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } } if  
* (!S_Maps.lianCi.containsKey(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
* .contains(S_Pos.NLP_CI_LIAN)) {  
* S_Maps.lianCi.put(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {  
* S_Maps.lianCi.put(cInputStringKorea.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {  
* S_Maps.lianCi.put(cInputStringJapan.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {  
* S_Maps.lianCi.put(cInputStringTrandition.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
```

```
* S_Pos.INT_ONE)); } if (null != cInputStringEnglish) {  
* S_Maps.lianCi.put(cInputStringEnglish.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } } if  
* (!S_Maps.baDongCi.containsKey(cInputString.split(S_Pos.  
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
* .contains(S_Pos.NLP_CI_BA_DONG)) {  
* S_Maps.baDongCi.put(cInputString  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {  
* S_Maps.baDongCi.put(cInputStringKorea  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {  
* S_Maps.baDongCi.put(cInputStringJapan  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {  
* S_Maps.baDongCi.put(cInputStringTrandition  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {  
* S_Maps.baDongCi.put(cInputStringEnglish  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } } if  
* (!S_Maps.xianDingCi.containsKey(cInputString  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
* .contains(S_Pos.NLP_CI_XIAN_DING)) {  
* S_Maps.xianDingCi.put(cInputString  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {  
* S_Maps.xianDingCi.put(cInputStringKorea  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {  
* S_Maps.xianDingCi.put(cInputStringJapan  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {  
* S_Maps.xianDingCi.put(cInputStringTrandition  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
```

```
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.xianDingCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.mingCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_MING)) {
* S_Maps.mingCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.mingCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.mingCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.mingCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.mingCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.daiCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_DAI)) {
* S_Maps.daiCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.daiCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.daiCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.daiCi.put(cInputStringTrandition
```

```
*.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.daiCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.jieCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE].
* contains(S_Pos.NLP_CI_JIE)) {
* S_Maps.jieCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH
* )[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.jieCi.put(cInputStringKorea.split(S_Pos.
* NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.jieCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.jieCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.jieCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.xingRongCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_XING_RONG)) {
* S_Maps.xingRongCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.xingRongCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.xingRongCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
```

```
* S_Maps.xingRongCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.xingRongCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.zhuCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_ZHU)) {
* S_Maps.zhuCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.zhuCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.zhuCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.zhuCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.zhuCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.weiCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE].
* contains(S_Pos.NLP_CI_WEI)) {
* S_Maps.weiCi.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH
* )[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.weiCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.weiCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
```

```
* S_Pos.INT_ONE)); } if (null != cInputStringTrandition) {  
  
* S_Maps.weiCi.put(cInputStringTrandition  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); } if (null != cInputStringEnglish) {  
  
* S_Maps.weiCi.put(cInputStringEnglish  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); } } if  
  
* (!S_Maps.shengLueCi.containsKey(cInputString  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
  
* .contains(S_Pos.NLP_CI_SHENG_LUE)) {  
  
* S_Maps.shengLueCi.put(cInputString  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); if (null != cInputStringKorea) {  
  
* S_Maps.shengLueCi.put(cInputStringKorea  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); } if (null != cInputStringJapan) {  
  
* S_Maps.shengLueCi.put(cInputStringJapan  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); } if (null != cInputStringTrandition) {  
  
* S_Maps.shengLueCi.put(cInputStringTrandition  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); } if (null != cInputStringEnglish) {  
  
* S_Maps.shengLueCi.put(cInputStringEnglish  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); } } if  
  
* (!S_Maps.qingTaiCi.containsKey(cInputString  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&  
  
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]  
  
* .contains(S_Pos.NLP_CI_QING_TAI)) {  
  
* S_Maps.qingTaiCi.put(cInputString  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); if (null != cInputStringKorea) {  
  
* S_Maps.qingTaiCi.put(cInputStringKorea  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE)); } if (null != cInputStringJapan) {  
  
* S_Maps.qingTaiCi.put(cInputStringJapan  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
```

```
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.qingTaiCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.qingTaiCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.xingWeiCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_XING_WEI)) {
* S_Maps.xingWeiCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.xingWeiCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.xingWeiCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.xingWeiCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.xingWeiCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.shiTaiCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_SHI_TAI)) {
* S_Maps.shiTaiCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.shiTaiCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.shiTaiCi.put(cInputStringJapan
```



```
*.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.shiTaiCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.shiTaiCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } if
* (!S_Maps.dingMingCi.containsKey(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO]) &&
* cInputString.split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE]
* .contains(S_Pos.NLP_CI_DING_MING)) {
* S_Maps.dingMingCi.put(cInputString
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); if (null != cInputStringKorea) {
* S_Maps.dingMingCi.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringJapan) {
* S_Maps.dingMingCi.put(cInputStringJapan
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringTrandition) {
* S_Maps.dingMingCi.put(cInputStringTrandition
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } if (null != cInputStringEnglish) {
* S_Maps.dingMingCi.put(cInputStringEnglish
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } }
* studyPos.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE]));
* posCnToCn.put(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); linkedHashMap =
* loopLoadForest(cInputString); if (null !=
* cInputStringKorea) { posCnToCn.put(cInputStringKorea
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
```

```
* S_Pos.INT_ONE)); linkedHashMap =  
  
* loopLoadForest(cInputStringKorea); } if (null !=  
  
* cInputStringJapan) { if  
  
* (!posCnToCn.containsKey(cInputStringJapan  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])) {  
  
* posCnToCn.put(cInputStringJapan  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] ,  
  
* cInputStringJapan.split(S_Pos.NLP_SYMBO_SLASH).length >  
  
* S_Pos.INT_ONE ? cInputStringJapan  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ONE] : "未知");  
* linkedHashMap = loopLoadForest(cInputStringJapan); } } if  
  
* (null != cInputStringTrandition) { if  
  
* (!posCnToCn.containsKey(cInputStringTrandition  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO])) {  
  
* posCnToCn.put(cInputStringTrandition  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO] , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE])); linkedHashMap =  
  
* loopLoadForest(cInputStringTrandition); } } if (null !=  
  
* cInputStringEnglish) { posCnToCn.put(cInputStringEnglish  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase  
  
* () , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE])); } if (null != cInputStringRs) {  
  
* posCnToCn.put(cInputStringRs  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase  
  
* () , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE])); } if (null != cInputStringAb) {  
  
* posCnToCn.put(cInputStringAb  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase  
  
* () , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE])); }  
  
*  
  
* if (null != cInputStringFn) {  
  
* posCnToCn.put(cInputStringFn  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase  
  
* () , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE])); } if (null != cInputStringGm) {  
  
* posCnToCn.put(cInputStringGm  
  
* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase  
  
* () , new  
  
* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[  
  
* S_Pos.INT_ONE])); } if (null != cInputStringSp) {  
  
* posCnToCn.put(cInputStringSp
```

```

* .split(S_Pos.NLP_SYMBO_SLASH)[S_Pos.INT_ZERO].toLowerCase

* () , new

* StringBuilder(cInputString.split(S_Pos.NLP_SYMBO_SLASH)[
* S_Pos.INT_ONE])); } } cReader.closeDeta();

* cReaderKorea.closeDeta(); cReaderJapan.closeDeta();

* cReaderTrandition.closeDeta();

* cReaderEnglish.closeDeta(); cReaderPinYin.closeDeta();

* cReaderRs.closeDeta(); cReaderAb.closeDeta();

* cReaderFn.closeDeta(); cReaderGm.closeDeta();

* cReaderSp.closeDeta(); try { inputStream.close();

* cReader.closeDeta(); inputStreamKorea.close();

* cReaderKorea.closeDeta(); inputStreamJapan.close();

* cReaderJapan.closeDeta(); inputStreamTrandition.close();

* cReaderTrandition.closeDeta();

* inputStreamEnglish.close(); cReaderEnglish.closeDeta();

* inputStreamPinYin.close(); cReaderPinYin.closeDeta();

* inputStreamRs.close(); cReaderRs.closeDeta();

* inputStreamAb.close(); cReaderAb.closeDeta();

* inputStreamFn.close(); cReaderFn.closeDeta();

* inputStreamSp.close(); cReaderSp.closeDeta();

* inputStreamFn.close(); cReaderFn.closeDeta();

*

* } catch (IOException e) { throw new RuntimeException(e);

* } //trim S_Maps.baDongCi.remove("");

* S_Maps.daiCi.remove(""); S_Maps.dingMingCi.remove("");

* S_Maps.dongCi.remove(""); S_Maps.fuCi.remove("");

* S_Maps.jieCi.remove(""); S_Maps.lianCi.remove("");

* S_Maps.liangCi.remove(""); S_Maps.mingCi.remove("");

* S_Maps.qingTaiCi.remove("");

* S_Maps.shengLueCi.remove(""); S_Maps.shiTaiCi.remove("");

* S_Maps.weiCi.remove(""); S_Maps.xianDingCi.remove("");

* S_Maps.xingRongCi.remove("");

* S_Maps.xingWeiCi.remove(""); S_Maps.zhuCi.remove("");

* //20230106-S_logger.Log.logger.info("" + S_Maps.CiOne.size());

* //20230106-S_logger.Log.logger.info("" + S_Maps.CiTwo.size());

* //20230106-S_logger.Log.logger.info("" + S_Maps.CiThree.size());

* //20230106-S_logger.Log.logger.info("" + S_Maps.CiFour.size()); }

*/

//Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/PillowsSet.java
package E_A.OEI.SVQ.MPC.fhmm.E;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI;

import U_A.PEU.P.time.TimeCheck;

import java.util.Hashtable;

import java.util.LinkedHashMap;

import java.util.Map;

import java.util.TreeMap;

/*

```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

@SuppressWarnings({ "unchecked", "resource", "unused" })

```
public class PillowsSet {

    public long index;

    public boolean entry = false;

    public Object object;

    public PillowsSet smallOrder;

    public PillowsSet largeOrder;

    public PillowsSet preSmallOrder;

    public PillowsSet preLargeOrder;

    public void arrangePillow(long index, Object object, int pillows, int depth,
                               int currentDepth) {
        if (null == this.object) {
            this.index = index;
            this.object = object;
            return;
        }
        if (index < this.index) {
            if (null == smallOrder) {
                smallOrder = new PillowsSet();
                smallOrder.preLargeOrder = this;
            }
            smallOrder.arrangePillow(index, object, pillows, depth,
                                      currentDepth + S_Pos.INT_ONE);
        }
        if (index > this.index) {
            if (null == largeOrder) {
                largeOrder = new PillowsSet();
                largeOrder.preSmallOrder = this;
            }
            largeOrder.arrangePillow(index, object, pillows, depth,
                                      currentDepth + S_Pos.INT_ONE);
        }
    }

    public Object getPillow(long index) {
        if (index == this.index) {
            return object;
        }
        if (index < this.index) {
            return smallOrder.getPillow(index);
        }
    }
}
```

```
}
    if (index > this.index) {
        return largeOrder.getPillow(index);
    }
    return null;
}

public void superBalance() {
}

public void show() {
    entry = true;
    if (smallOrder != null && !smallOrder.entry) {
        smallOrder.show();
    }
    if (largeOrder != null && !largeOrder.entry) {
        largeOrder.show();
    }
}

public static void main(String[] argv) {
    int pillows = S_Pos.INT_ZERO;
    int depth = pillows >> S_Pos.INT_ONE;
    int currentDepth = S_Pos.INT_ZERO;
    PillowsSet pillowsMap = new PillowsSet();
    for (int i = S_Pos.INT_ZERO; i < 5000; i++) {
        pillowsMap.arrangePillow(i, i, pillows++, depth, currentDepth);
    }
    pillowsMap.fixEntry();
    pillowsMap.show();
    // TimeCheck timecheck= new TimeCheck();
    // timecheck.begin();
    // for(int i=0;i<100000;i++) {
    // // pillowsMap.getPillow(100);
    // }
    // timecheck.end();
    // timecheck.duration();
    // 20230106-S_logger.Log.logger.info("" + pillowsMap.getPillow(100));
    Map<Long, Object> map = new IMV_SQI();
    for (long i = 0; i < 5000000; i++) {
        map.put(i, i);
    }
    long v = 1000;
    TimeCheck timecheck = new TimeCheck();
    timecheck.begin();
    for (int i = 0; i < 5000000; i++) {
        map.get(v);
    }
    timecheck.end();
    timecheck.duration();
}
```

```
// 20230106-S_logger.Log.logger.info("" + map.get(v));  
  
// c  
  
map = new IMV_SQI();  
for (long i = 0; i < 5000000; i++) {  
    map.put(i, i);  
}  
  
v = 1000;  
  
timecheck = new TimeCheck();  
timecheck.begin();  
for (int i = 0; i < 5000000; i++) {  
    map.get(v);  
}  
  
timecheck.end();  
timecheck.duration();  
  
// 20230106-S_logger.Log.logger.info("" + map.get(v));  
  
// map compare  
  
map = new LinkedHashMap<>();  
for (long i = 0; i < 5000000; i++) {  
    map.put(i, i);  
}  
  
v = 1000;  
  
timecheck.begin();  
for (int i = 0; i < 5000000; i++) {  
    map.get(v);  
}  
  
timecheck.end();  
timecheck.duration();  
  
// 20230106-S_logger.Log.logger.info("" + map.get(v));  
  
//  
  
Hashtable<Long, Object> table = new Hashtable<>();  
for (long i = 0; i < 5000000; i++) {  
    table.put(i, i);  
}  
  
v = 1000;  
  
timecheck.begin();  
for (int i = 0; i < 5000000; i++) {  
    table.get(v);  
}  
  
timecheck.end();  
timecheck.duration();  
  
// 20230106-S_logger.Log.logger.info("" + table.get(v));  
  
//  
  
TreeMap<Long, Object> tree = new TreeMap<>();  
for (long i = 0; i < 5000000; i++) {  
    tree.put(i, i);  
}  
  
v = 1000;
```

```

        timecheck.begin();
        for (int i = 0; i < 5000000; i++) {
            tree.get(v);
        }
        timecheck.end();
        timecheck.duration();
        // 20230106-S_logger.Log.logger.info("" + tree.get(v));
    }
    private void fixEntry() {
        int small = 0;
        int large = 0;
        if (largeOrder == null && preLargeOrder != null) {
            largeOrder = preLargeOrder;
            if (largeOrder.preLargeOrder != null) {
                preLargeOrder = largeOrder.preLargeOrder;
                preLargeOrder.smallOrder = this;
            } else if (largeOrder.preSmallOrder != null) {
                preSmallOrder = largeOrder.preSmallOrder;
                preSmallOrder.largeOrder = this;
            }
            largeOrder.smallOrder = null;
            large = 1;
        }
        if (smallOrder == null && preSmallOrder != null) {
            smallOrder = preSmallOrder;
            if (smallOrder.preSmallOrder != null) {
                preSmallOrder = smallOrder.preSmallOrder;
                preSmallOrder.largeOrder = this;
            } else if (smallOrder.preLargeOrder != null) {
                preLargeOrder = smallOrder.preLargeOrder;
                preLargeOrder.smallOrder = this;
            }
            smallOrder.largeOrder = null;
            small = 1;
        }
        if (smallOrder != null && small == 0) {
            smallOrder.fixEntry();
        }
        if (largeOrder != null && large == 0) {
            largeOrder.fixEntry();
        }
    }
}

/Trif_DNA-master/C_L_E/E_A/OEI/SVQ/MPC/fhmm/E/PillowsSetTest.java

E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.company.E

/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/company/E/LoginService_E.java
/Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/company/E/LoginService_ETest.java

```

E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.factory.E

```
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/factory/E/LoginDAO_E.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/factory/E/LoginDAO_ETest.java
```

E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.port.E

```
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/Rest_C_Port_E.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/Rest_C_Port_ETest.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_DE.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_DETest.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_TUE.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_TUETest.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_QE.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_QETest.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_UE.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDB_UETest.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDBConfig_E.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDBConfig_ETest.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDBPLSQL_E.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestDBPLSQL_ETest.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestLoginPort_E.java  
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/port/E/RestLoginPort_ETest.java
```

E_A.OP.SM.AOP.MEC.SIQ.VPC.PP.project.E

```
//Trif_DNA-master/C_L_E/E_A/OP/SM/AOP/MEC/SIQ/VPC/PP/project/E/LoginProject_E.java
```

E_A.org.tinos.deta.hooks.main

```
//Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/ExceptInformation.java  
package E_A.org.tinos.deta.hooks.main;
```

import java.util.List;

import java.util.concurrent.CopyOnWriteArrayList;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//用于异常的并发循环登记。

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class ExceptInformation{

public long exceptionCount= 0;//问题发生的次数

public String exceptionFile= new String();//问题发生在哪个文件

public String exceptionSession= new String();//问题的登记编号记录

public List<Long> exceptionTimes= new CopyOnWriteArrayList<>();//问题的发生时间

public String throwableString= new String();

private Exception exception;

public void putException(Exception inputException) {

this.exception= inputException;

}


```

    public Exception getException() {
        return this.exception;
    }
    //...
}

//Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/ExceptInformationTest.java
//Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Hook_C.java
package E_A.org.tinos.deta.hooks.main;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
//时间: 20220623
//描述: 这个函数主要用于Exception的传入进行登记. 通过序列号区分, 时间记录,
//文件记录和异常描述的4个维度进行异常登记。
//输出结果如下, 注意下 间隔符号为‘: ’
//归纳序列号: 时间: 文件名: 异常描述
//稍后处理异常循环准备用map, 避免内存递增。
public interface Hook_C {
    //public List<String> exceptionList= new CopyOnWriteArrayList<>();
    //处理异常循环准备用map, 避免内存递增。
    //public Map<String, ExceptInformation> exceptionMap= new IMV_SQI();
    //这个函数用于异常描述
    public String getCurrentEnvironment(String session);
    public String getCurrentFrequentEnvironment(String session, String timeMillis);
    public String getCurrentFrequentEnvironment(String session, Long timeMillis);
    //用于登记异常
    public void hookException(Exception exception);
    //用于显示异常
    public void showException(Exception exception);
    //这里出现一个问题, 便是session的随机循环异常无法有效的命名,
    //于是准备将session提取出来到接口外定义, 这样map价值就出来了。于是内容为捕捉的times记录。
    public void hookFrequentException(String uniqueSessionKey, Long timeMillis
        , Exception exception);
    //这个函数用于异常描述
    public void hookThrowable(Throwable throwable);
    public void hookThrowable(String throwableString);
}

//Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Hook_CTest.java
//Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Hook_E.java
package E_A.org.tinos.deta.hooks.main;

import S_A.pheromone.IMV_SQI;

import java.util.List;

import java.util.Map;

```

```
import java.util.concurrent.CopyOnWriteArrayList;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
//时间: 20220623
//描述: 这个函数主要用于Exception的传入进行登记, 通过序列号区分, 时间记录,
//文件记录和异常描述的4个维度进行异常登记。
//输出结果如下, 注意下 间隔符号为‘: ’
//归纳序列号: 时间: 文件名: 异常描述
//稍后处理异常循环准备用map, 避免内存递增。
public class Hook_E extends Throwable implements Hook_C {
    public List<String> exceptionList = new CopyOnWriteArrayList<>();
    //处理异常循环准备用map, 避免内存递增。
    public Map<String, ExceptInformation> exceptionMap = new IMV_SQI();
    //这个函数用于异常描述
    public String getCurrentEnvironment(String session) {
        String CurrentEnvironment = new String(session + "://序列号
        CurrentEnvironment += System.currentTimeMillis() + "://时间
        return CurrentEnvironment;
    }
    public String getCurrentFrequentEnvironment(String session, String timeMillis) {
        String CurrentEnvironment = new String(session + "://序列号
        CurrentEnvironment += timeMillis + "://时间
        return CurrentEnvironment;
    }
    public String getCurrentFrequentEnvironment(String session, Long timeMillis) {
        String CurrentEnvironment = new String(session + "://序列号
        CurrentEnvironment += timeMillis.toString() + "://时间
        return CurrentEnvironment;
    }
    //用于登记异常
    public void hookException(Exception exception) {
        String session = "" + Math.random();
        exceptionList.add(getCurrentEnvironment(session) + exception.toString());
        StackTraceElement[] stackTraceElements = exception.getStackTrace();
        for (StackTraceElement stackTraceElement : stackTraceElements) {
            exceptionList.add(getCurrentEnvironment(session) + stackTraceElement.toString());
        }
    }
    //用于显示异常
    @SuppressWarnings("unused")
```

```
public void showException(Exception exception) {
    //20230106-S_logger.Log.logger.info("" + exception.toString());
    StackTraceElement[] stackTraceElements = exception.getStackTrace();
    for (StackTraceElement stackTraceElement : stackTraceElements) {
        //20230106-S_logger.Log.logger.info("" + stackTraceElement.toString());
    }
}

//这里出现一个问题,便是session的随机循环异常无法有效的命名,
//于是准备将session提取出来到接口外定义,这样map价值就出来了。于是内容为捕捉的times记录。
public void hookFrequentException(String uniqueSessionKey, Long timeMillis
    , Exception exception) {
    //如果出现同session 不同内容异常,不讨论,在函数上层进行区分。
    if (exceptionMap.containsKey(uniqueSessionKey)) {
        //map计数递增
        ExceptInformation exceptInformation = exceptionMap.get(uniqueSessionKey);
        exceptInformation.exceptionCount += 1;
        exceptInformation.exceptionTimes.add(timeMillis);
        exceptionMap.put(uniqueSessionKey, exceptInformation);
        return;
    }
    //map登记录入
    ExceptInformation exceptInformation = new ExceptInformation();
    exceptInformation.exceptionCount = 1;
    exceptInformation.exceptionSession = uniqueSessionKey;
    exceptInformation.exceptionTimes.add(timeMillis);
    exceptInformation.putException(exception);
    exceptionMap.put(uniqueSessionKey, exceptInformation);
    //list 同步登记
    exceptionList.add(getCurrentFrequentEnvironment(uniqueSessionKey, timeMillis)
        + exception.toString());
    StackTraceElement[] stackTraceElements = exception.getStackTrace();
    for (StackTraceElement stackTraceElement : stackTraceElements) {
        exceptionList.add(getCurrentFrequentEnvironment(uniqueSessionKey, timeMillis)
            + stackTraceElement.toString());
    }
}

//这个函数用于异常描述
public void hookThrowable(Throwable throwable) {
    String session = "" + Math.random();
    exceptionList.add(getCurrentEnvironment(session) + throwable.toString());
    ///20230106-S_logger.Log.logger.info("" + new Hook().getCurrentEnvironment() + throwable.toString());
    StackTraceElement[] stackTraceElements = throwable.getStackTrace();
    for (StackTraceElement stackTraceElement : stackTraceElements) {
        exceptionList.add(getCurrentEnvironment(session) + stackTraceElement.toString());
    }
}

public void hookThrowable(String throwableString) {
```

```
String session = "" + Math.random();

exceptionList.add(getCurrentEnvironment(session) + throwableString);

}

public void hookSleeper(Object object) {

}

}

/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Hook_ETest.java
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/Test.java
/Trif_DNA-master/C_L_E/E_A/org/tinos/deta/hooks/main/TestTest.java

E_A.OSI.AOP.PCS.PP.company_E

/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/company_E/LoginServiceImpl.java
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/company_E/LoginServiceImplTest.java

E_A.OSI.AOP.PCS.PP.factory_E

/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/factory_E/LoginDAOImpl.java
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/factory_E/LoginDAOImplTest.java

E_A.OSI.AOP.PCS.PP.port_E

/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/port_E/RestLoginPortImpl.java
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/port_E/RestLoginPortImplTest.java
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/PCS/PP/port_E/RestNLPPortImpl.java

E_A.OSI.AOP.VPC.PP.company_E

/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/company_E/LoginService_E.java

E_A.OSI.AOP.VPC.PP.factory_E

/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/factory_E/LoginDAO_E.java

E_A.OSI.AOP.VPC.PP.port_E

/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/port_E/RestAskPort_E.java
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/port_E/RestAskPort_ETest.java
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/port_E/RestLoginPort_E.java
/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/port_E/RestLoginPort_ETest.java

E_A.OSI.AOP.VPC.PP.project_E

/Trif_DNA-master/C_L_E/E_A/OSI/AOP/VPC/PP/project_E/LoginProject_E.java

OPE.MSQ.OVU.PQE.nodeInfo

/Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeInfo/NodeInfo.java
package OPE.MSQ.OVU.PQE.nodeInfo;

import E_A.OEL.SVQ.MPC.fhmm.E.FastReadProjectFile;

import S_A.SVQ.stable.S_Annotation;

import javax.swing.ImageIcon;

import javax.swing.JComboBox;

import javax.swing.JScrollPane;

import java.awt.Image;

@SuppressWarnings({S_Annotation.TAG_UNCHECKED, S_Annotation.TAG_RAW_TYPES})

/*

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

//图片是百度上搜国旗下的。
```

```

public class NodeInfo extends JScrollPane {

    String[] countryStrings= {"china", "ca", "denmark", "fr", "genmany"
        , "india", "norway", "uk", "us"};

    private ImageIcon[] images= {
        FastReadProjectFile.getDetaImageIconReader("china.gif", "DetaResources/gifFlag/"),
        FastReadProjectFile.getDetaImageIconReader("us.gif", "DetaResources/gifFlag/"),
        FastReadProjectFile.getDetaImageIconReader("denmark.gif", "DetaResources/gifFlag/"),
        FastReadProjectFile.getDetaImageIconReader("fr.gif", "DetaResources/gifFlag/"),
        FastReadProjectFile.getDetaImageIconReader("germany.gif", "DetaResources/gifFlag/"),
        FastReadProjectFile.getDetaImageIconReader("india.gif", "DetaResources/gifFlag/"),
        FastReadProjectFile.getDetaImageIconReader("norway.gif", "DetaResources/gifFlag/"),
        FastReadProjectFile.getDetaImageIconReader("uk.gif", "DetaResources/gifFlag/"),
        FastReadProjectFile.getDetaImageIconReader("ca.gif", "DetaResources/gifFlag/")
    };

    public NodeInfo() {
        Integer[] intArray= new Integer[countryStrings.length];
        JComboBox countryList= new JComboBox(intArray);
        countryList.setMaximumRowCount(5);
        for (int i= 0; i< countryStrings.length; i++) {
            intArray[i]= new Integer(i);
            if (images[i]!= null) {
                images[i].setImage(images[i].getImage().getScaledInstance
                    (50, 50, Image.SCALE_DEFAULT));
                images[i].setDescription(countryStrings[i]);
            }
        }
        countryList.removeAllItems();
        for(int i=0; i<images.length; i++) {
            countryList.addItem(images[i]);
        }
        this.setViewportView(countryList);
        this.validate();
    }
}

```

/Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeInfo/NodeInfoTest.java

OPE.MSQ.OVU.PQE.nodeProject

/Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeProject/NodeProject.java
 package OPE.MSQ.OVU.PQE.nodeProject;

```

import E_A.OE1.SVQ.MPC.fhmm.E.FastReadProjectFile;
import VSQ.etl.NodeProject_X_S;
import javax.swing.ImageIcon;
import javax.swing.JLabel;
import java.awt.event.ComponentEvent;
import java.awt.event.ComponentListener;
/*

```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

*(lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class NodeProject extends NodeProject_X_S implements ComponentListener {
    public NodeProject() {
        images = FastReadProjectFile.getDetaImageIconReader("imageread.jpg"
            , "DetaResources/imgETL/");
        jlabel = new JLabel();
        jlabel.setIcon(images);
        this.add(jlabel);
        this.validate();
        this.addComponentListener(this);
    }
    @Override
    public void componentResized(ComponentEvent e) {
        img = images.getImage().getScaledInstance(this.width <= 0 ? 100 : this.width
            , this.height <= 0 ? 100 : this.height, java.awt.Image.SCALE_SMOOTH);
        jlabel.setIcon(new ImageIcon(img));
        jlabel.validate();
        this.validate();
    }
    @Override
    public void componentMoved(ComponentEvent e) {
    }
    @Override
    public void componentShown(ComponentEvent e) {
    }
    @Override
    public void componentHidden(ComponentEvent e) {
    }
}

//Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeProject/NodeProjectTest.java
//Trif_DNA-master/C_L_E/OPE/MSQ/OVU/PQE/nodeProject/etl.png

    OPE.MVU.OVU.OVU.PQE.nodeEdit.C

//Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/CheckRange.java
package OPE.MVU.OVU.OVU.PQE.nodeEdit.C;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;

import java.awt.event.MouseEvent;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
```

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public interface CheckRange{  
    public void doCheckRange(LinkNode first, LinkNode node, MouseEvent mouseEvent);  
}
```

```
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/ChooseCheck.java  
package OPE.MVU.OVU.OVU.PQE.nodeEdit.C;
```

```
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
```

```
import java.awt.event.MouseEvent;
```

```
/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public interface ChooseCheck{  
    public Object[] chooseCheckNode(LinkNode first, MouseEvent mouseEvent);  
}
```

```
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/DrawArrow.java  
package OPE.MVU.OVU.OVU.PQE.nodeEdit.C;
```

```
import java.awt.Graphics2D;
```

```
/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public interface DrawArrow {  
    public void doDrawArrow(Graphics2D graphics2D  
        , int x, int y, int connectX, int connectY);  
    public void drawCurve(Graphics2D graphics2D  
        , int x, int y, int connectX, int connectY, double scale);  
}
```

```
/Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/DynamicLine_U.java  
package OPE.MVU.OVU.OVU.PQE.nodeEdit.C;
```

```
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
```

```
/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public interface DynamicLine_U{  
    public void exec(LinkNode first, LinkNode node);  
}
```

```
//Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/Sort.java  
package OPE.MVU.OVU.OVU.PQE.nodeEdit.C;
```

```
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
```

```
/*
```

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public interface Sort{  
    public LinkNode sort(LinkNode first);  
}
```

```
//Trif_DNA-master/C_L_E/OPE/MVU/OVU/OVU/PQE/nodeEdit/C/U_RelatedLine.java  
package OPE.MVU.OVU.OVU.PQE.nodeEdit.C;
```

```
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
```

```
/*
```

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public interface U_RelatedLine{  
    public void doUpdateRelatedLine(LinkNode first, String currentNodeName  
        , int currentNodeID, String currentNodePrimaryKey);  
}
```

OPE.OEQ.MCQ.OVU.PQE.osgi

```
//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGI.java  
package OPE.OEQ.MCQ.OVU.PQE.osgi;
```

```
/*
```

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

 **/

```

public class LinkOSGI {

    public NodeOSGI addNode(NodeOSGI currentNode

        , OSU_MCI_MCU_MSI currentFace) {

        if (null == currentNode) {

            currentNode = new NodeOSGI();

            currentNode.I_Name(currentFace);

            currentNode.next = null;

            currentNode.pre = null;

            return currentNode;

        }

        while (null != currentNode.next) {

            currentNode = currentNode.next;

        }

        NodeOSGI node = new NodeOSGI();

        node.I_Name(currentFace);

        node.pre = currentNode;

        currentNode.next = node;

        while (null != currentNode.pre) {

            currentNode = currentNode.pre;

        }

        return currentNode;

    }

}

//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGITest.java
//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/NodeOSGI.java
package OPE.OEQ.MCQ.OVU.PQE.osgi;

import S_A.VSQ.etl.NodeOSGI_S;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class NodeOSGI extends NodeOSGI_S {

    @Override

    public Object clone() {

        NodeOSGI obj = null;

        try {

            obj = (NodeOSGI) super.clone();

        } catch (CloneNotSupportedException e) {

            e.printStackTrace();

        }

        return obj;

    }

}

```

```

public NodeOSGI() {
    next = null;
    pre = null;
    currentFace = null;
    thisIcon = null;
    _SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS = null;
}

public void I_Name(OSU_MCI_MCU_MSI thisface) {
    next = null;
    pre = null;
    currentFace = thisface;
    thisIcon = currentFace.thisIcon;
    _SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS
        = new String(currentFace._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS);
}
}

//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/NodeOSGITest.java
//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/OSU_AOU_AOQ_VES.java
package OPE.OEQ.MCQ.OVU.PQE.osgi;

import ME.VPC.M.app.App;

import S_A.VSQ.etl.OSU_AVQ_ASQ_AOI_AOD_AOU_AOQ_VES_S;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class OSU_AOU_AOQ_VES
extends OSU_AVQ_ASQ_AOI_AOD_AOU_AOQ_VES_S implements Cloneable{
    public OSU_AOU_AOQ_VES(){
    }

    public void config() {
    }

    public OSU_AOU_AOQ_VES AOP_MVE_CSI_DUQ(App NE) {
        return objectPanel;
    }
}

//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/OSU_MCI_MCU_MSI.java
package OPE.OEQ.MCQ.OVU.PQE.osgi;

import ME.VPC.M.app.App;

import S_A.VSQ.etl.OSU_MCI_MCU_MSI_S;

import OSI.OPE.OVU.PQE.flash.ThisCanvas;

import javax.swing.JTextPane;

import java.util.List;

//作者 罗瑶光 等会我函数全部加 作者罗瑶光
/*

```

* 个人著作权人，作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class OSU_MCI_MCU_MSI extends OSU_MCI_MCU_MSI_S implements Cloneable {
    public boolean showed = false;
    public OSU_MCI_MCU_MSI AOP_MVE_CSI_DUQ(App NE) {
        S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-02-00-01");
        return _SQ_OSU_MSQ_OSU_MCI_MCU_MSI;
        //tinshell因为走一键执行, 没有new到子类, 而子类的button又是新的panel执行,
        //于是产生了非遗传隔层, 准备解决。
    }
    public OSU_MCI_MCU_MSI() {
        S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-02-00-01-01");
        thisIcon = null;
        thisImage = null;
        _SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS = null;
        S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-02-00-01-02");
        _SQ_AOU_AOQ_VES = new OSU_AOU_AOQ_VES();
        S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-02-00-01-03");
        _SQ_OPE_OPC_ECI = new OSU_OPE_OPC_ECI();
        S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-02-00-01-04");
        _SQ_OVQ_OSQ_VSQ = new OSU_OVQ_OSQ_VSQ();
        S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-02-00-01-05");
    }
    public void MEI_MSU(JTextPane rightBotJTextPane, ThisCanvas canvas) {
    }
    public void MEI_MSU(JTextPane rightBotJTextPane) {
    }
    //图片识别
    public void AOP_VEC_SQ(JTextPane rightBotJTextPane, String input, App NE) {
    }
    public void memoryRecovery(JTextPane rightBotJTextPane) {
    }
    public void OPE_E(JTextPane rightBotJTextPane, App NE) { //later..
    }
    public void AVQ_ASQ_OVQ_OSQ_VSQ(JTextPane rightBotJTextPane, App NE) {
    }
    public void register(Object[][] tableData_old, JTextPane text, App NE) {
        this.u = NE.app_S;
        this._A = NE.app_S._A;
        this.pos = NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn;
        this.text = text;
```

```

this.tableData_old = tableData_old;

}

public void IMP_PSU(App NE) {

}

public void AOP_VEC_SQ_PDE(JTextPane rightBotJTextPane, List<String> pathRankList) {

}

}

/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/OSU_OPE_OPC_ECI.java
package OPE.OEQ.MCQ.OVU.PQE.osgi;

import S_A.VSQ.etl.OSU_AVQ_ASQ_OPE_OPC_ECI_S;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class OSU_OPE_OPC_ECI extends OSU_AVQ_ASQ_OPE_OPC_ECI_S

    implements Cloneable{

    public OSU_OPE_OPC_ECI(){

    }

    @Override

    public OSU_OPE_OPC_ECI clone() {

        return addr;

    }

}

/Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/OSU_OVQ_OSQ_VSQ.java
package OPE.OEQ.MCQ.OVU.PQE.osgi;

import ME.VPC.M.app.App;

import S_A.VSQ.etl.OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ_S;

//import java.util.HashMap;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class OSU_OVQ_OSQ_VSQ

extends OSU_AVQ_ASQ_AVQ_ASQ_OVQ_OSQ_VSQ_S implements Cloneable{

    public OSU_OVQ_OSQ_VSQ() {

    }

    public void view(App NE) {

```

```

    }

    public OSU_OVQ_OSQ_VSQ clone() {

        return objectView;

    }

}

//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGI.pde
//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGI.pds
//Trif_DNA-master/C_L_E/OPE/OEQ/MCQ/OVU/PQE/osgi/LinkOSGI.tvn

    OPE.OVQ.MSQ.OVU.PQE.platForm

//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornJSplitPane.java
package OPE.OVQ.MSQ.OVU.PQE.platForm;

import javax.swing.JSplitPane;

import javax.swing.plaf.basic.BasicSplitPaneDivider;

import javax.swing.plaf.basic.BasicSplitPaneUI;

import java.awt.Rectangle;

/**
 * 作者对jdk 开源插件进行2次开发。不在个人著作申请范围内。
 *
 * @author LYG refer JDK authors from SUN Tech, 。
 */
* 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class UnicornJSplitPane extends JSplitPane {

    private int dividerDragSize = 3;

    private int dividerDragOffset = 3;

    public UnicornJSplitPane() {

        setDividerSize(0);

        setContinuousLayout(true);

    }

    @SuppressWarnings("deprecation")
    @Override
    public void layout() {

        super.layout();

        BasicSplitPaneDivider divider = ((BasicSplitPaneUI) getUI()).getDivider();

        Rectangle bounds = divider.getBounds();

        if (orientation == HORIZONTAL_SPLIT) {

            bounds.x -= dividerDragOffset;

            bounds.width = dividerDragSize;

        } else {

            bounds.y -= dividerDragOffset;

            bounds.height = dividerDragSize;

        }

    }

}

```

```

    }

    divider.setBounds(bounds);

}

}

//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornJSplitPaneTest.java
//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornScrollBarUI.java
package OPE.OVQ.MSQ.OVU.PQE.platForm;

import javax.swing.JButton;

import javax.swing.JComponent;

import javax.swing.JScrollBar;

import javax.swing.plaf.ComponentUI;

import javax.swing.plaf.basic.BasicScrollBarUI;

import java.awt.Color;

import java.awt.Dimension;

import java.awt.Graphics;

import java.awt.Rectangle;

/**
 * 作者对jdk 开源插件进行2次开发。不在个人著作申请范围内。
 *
 * @author LuoYaoGuang refer JDK authors from SUN Tech, 。
 */
* 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public final class UnicornScrollBarUI extends BasicScrollBarUI {

    public Dimension getPreferredSize(JComponent c) {

        return (scrollbar.getOrientation() == JScrollBar.VERTICAL)

            ? new Dimension(15, 40) : new Dimension(40, 15);

    }

    public static ComponentUI createUI(JComponent c) {

        return new UnicornScrollBarUI();

    }

    public void paintThumb(Graphics g, JComponent c, Rectangle thumbBounds) {

        if (thumbBounds.isEmpty() || !scrollbar.isEnabled())

            return;

        int w = thumbBounds.width;

        int h = thumbBounds.height;

        g.translate(thumbBounds.x, thumbBounds.y);

        g.setColor(Color.ORANGE); //thumbHighlightColor

        g.drawLine(0, 0, 0, h - 2); // Left side.

        g.drawLine(0, 0, w - 2, 0); // Top.

        g.setColor(Color.BLUE); //thumbDarkShadowColor

        g.drawLine(w - 1, 0, w - 1, h - 1); // Right side.

```

```
g.drawLine(0, h - 1, w - 1, h - 1); // Bottom.
g.setColor(Color.green); // thumbLightShadowColor
g.drawLine(w - 2, 1, w - 2, h - 2); // Right side.
g.drawLine(1, h - 2, w - 2, h - 2); // Bottom.
// Fill in the middle.
//g.setColor(Color.gray); // thumbColor
g.setColor(Color.pink); // thumbColor
g.fillRect(3, 3, w - 6, h - 6);
//.fillRect(6, 6, w - 12, h - 12);
// Done.
g.translate(-thumbBounds.x, -thumbBounds.y);
}

public JButton createDecreaseButton(int orientation) {
    return new ArrowButton(orientation);
}

public JButton createIncreaseButton(int orientation) {
    return new ArrowButton(orientation);
}

private static class ArrowButton extends JButton {

    private final int direction;
    private final int h;
    private final int w;
    private final Color thumb;
    private final Color shadow;
    private final Color darkShadow;
    private final Color highlight;
    public ArrowButton(int direction) {
        this.direction = direction;
        h = w = 15;
        //thumb= Color.pink;
        thumb = Color.darkGray;
        shadow = Color.pink;
        darkShadow = Color.orange;
        highlight = Color.blue;
        setRequestFocusEnabled(false);
    }

    public void paint(Graphics g) {
        final Color origColor = g.getColor();
        final boolean isPressed = getModel().isPressed();
        if (isPressed) {
            g.setColor(shadow);
            g.drawRect(0, 0, w - 1, h - 1);
        } else {
            g.setColor(highlight);
            g.drawLine(0, 0, 0, h - 1);
            g.drawLine(1, 0, w - 2, 0);
        }
    }
}
```

```

        g.setColor(shadow);
        g.drawLine(1, h - 2, w - 2, h - 2);
        g.drawLine(w - 2, 1, w - 2, h - 3);
        g.setColor(darkShadow);
        g.drawLine(0, h - 1, w - 1, h - 1);
        g.drawLine(w - 1, 0, w - 1, h - 1);
    }
    g.setColor(thumb);
    g.fillRect(1, 1, w - 3, h - 3);
    if (isPressed)
        g.translate(1, 1);
    paintTriangle(g);
    if (isPressed)
        g.translate(-1, -1);
    g.setColor(origColor);
}

public Dimension getPreferredSize() {
    return new Dimension(w, h);
}

public Dimension getMinimumSize() {
    return new Dimension(w, h);
}

public Dimension getMaximumSize() {
    return new Dimension(w, h);
}

//    public boolean isFocusTraversable() {
//        return false;
//    }

private void paintTriangle(Graphics g) {
    final int size = 4;
    int x = (w - size) / 2;
    int y = (h - size) / 2;
    if (direction == NORTH)
        --y;
    else if (direction == WEST)
        --x;
    g.translate(x, y);
    g.setColor(isEnabled() ? darkShadow : shadow);
    final int mid = (size / 2) - 1;
    switch (direction) {
        case NORTH:
            for (int i = 0; i < size; i++)
                g.drawLine(mid - i, i, mid + i, i);
            break;
        case SOUTH:
            for (int i = size, j = 0; i-- > 0;) {
                g.drawLine(mid - i, j, mid + i, j);
            }

```



```

        j++;
    }
    break;
case WEST:
    for (int i = 0; i < size; i++)
        g.drawLine(i, mid - i, i, mid + i);
    break;
case EAST:
    for (int i = size, j = 0; i-- > 0;) {
        g.drawLine(j, mid - i, j, mid + i);
        j++;
    }
    break;
}
g.translate(-x, -y);
}
}
}

//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornScrollBarUITest.java
//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornSplitPaneUI.java
package OPE.OVQ.MSQ.OVU.PQE.platForm;

import javax.swing.JButton;
import javax.swing.JComponent;
import javax.swing.JSplitPane;
import javax.swing.UIManager;
import javax.swing.border.Border;
import javax.swing.plaf.ComponentUI;
import javax.swing.plaf.basic.BasicSplitPaneDivider;
import javax.swing.plaf.basic.BasicSplitPaneUI;
import java.awt.Color;
import java.awt.Component;
import java.awt.Container;
import java.awt.Cursor;
import java.awt.Dimension;
import java.awt.Graphics;
import java.awt.Insets;
import java.awt.LayoutManager;

/**
 * 作者对jdk 开源插件进行2次开发。不在个人著作申请范围内。
 *
 * @author LuoYaoGuang refer JDK authors from SUN Tech, 。
 */
/**
 * 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

```

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class UnicornSplitPaneUI extends BasicSplitPaneUI {
    public UnicornSplitPaneUI() {
        super();
    }
    public static ComponentUI createUI(JComponent c) {
        return new UnicornSplitPaneUI();
    }
    @Override
    public void paint(Graphics g, JComponent c) {
        super.paint(g, c);
        g.setColor(new Color(255, 0, 255));
        int width = c.getWidth();
        int height = c.getHeight();
        g.drawRect(3, 3, width - 6, height - 6);
    }
    /**
     * Creates the default divider.
     */
    @Override
    public BasicSplitPaneDivider createDefaultDivider() {
        return new MyBasicSplitPaneDivider(this);
    }
    private class MyBasicSplitPaneDivider extends BasicSplitPaneDivider {
        /**
         *
         */

        private int oneTouchSize, oneTouchOffset;
        boolean centerOneTouchButtons;
        //center空白区域
        private int x1, y1;
        public MyBasicSplitPaneDivider(UnicornSplitPaneUI ui) {
            super(ui);
            oneTouchSize = ONE_TOUCH_SIZE;
            oneTouchOffset = ONE_TOUCH_OFFSET;
            centerOneTouchButtons = true;
            setLayout(new DividerLayout());
            setBasicSplitPaneUI(ui);
            orientation = splitPane.getOrientation();
            setCursor((orientation == JSplitPane.HORIZONTAL_SPLIT)
                ? Cursor.getPredefinedCursor(Cursor.E_RESIZE_CURSOR)
                : Cursor.getPredefinedCursor(Cursor.S_RESIZE_CURSOR));
            setBackground(UIManager.getColor("SplitPane.background"));
        }
        /**
```

```

* Creates and return an instance of JButton that can be used to
* collapse the right component in the split pane.
*/
@Override
public JButton createRightOneTouchButton() {
    JButton b = new JButton() {

        @SuppressWarnings("unused")
        public void L_Border(Border border) {
        }

        @Override
        public void paint(Graphics g) {
            if (splitPane != null) {
                int[] xs = new int[3];
                int[] ys = new int[3];
                int blockSize;

                // Fill the background first ...
                g.setColor(new Color(255, 0, 255));
                g.fillRect(0, 0, this.getWidth(),
                    this.getHeight());
                // ... then draw the arrow.
                if (orientation == JSplitPane.VERTICAL_SPLIT) {
                    blockSize = Math.min(getHeight(), oneTouchSize);
                    xs[0] = blockSize;
                    xs[1] = blockSize << 1;
                    xs[2] = 0;
                    ys[0] = blockSize;
                    ys[1] = ys[2] = 0;
                } else {
                    blockSize = Math.min(getWidth(), oneTouchSize);
                    xs[0] = xs[2] = 0;
                    xs[1] = blockSize;
                    ys[0] = 0;
                    ys[1] = blockSize;
                    ys[2] = blockSize << 1;
                }
                g.setColor(new Color(255, 0, 255));
                g.fillPolygon(xs, ys, 3);
            }
        }
    };

    b.setMinimumSize(new Dimension(oneTouchSize, oneTouchSize));
    b.setCursor(Cursor.getPredefinedCursor(Cursor.DEFAULT_CURSOR));
    b.setFocusPainted(false);
    b.setBorderPainted(false);
    b.setRequestFocusEnabled(false);
    return b;
}

```

```

    }

    /**
     * Creates and return an instance of JButton that can be used to
     * collapse the left component in the split pane.
     */
    public JButton createLeftOneTouchButton() {
        JButton b = new JButton() {

            @SuppressWarnings("unused")
            public void I_Border(Border b) {
            }

            public void paint(Graphics g) {
                if (splitPane != null) {
                    int[] xs = new int[3];
                    int[] ys = new int[3];
                    int blockSize;

                    // Fill the background first ...
                    g.setColor(new Color(255, 0, 255));
                    g.fillRect(0, 0, this.getWidth(),
                        this.getHeight());
                    // ... then draw the arrow.
                    g.setColor(new Color(255, 0, 255));
                    if (orientation == JSplitPane.VERTICAL_SPLIT) {
                        blockSize = Math.min(getHeight(), oneTouchSize);
                        xs[0] = blockSize;
                        xs[1] = 0;
                        xs[2] = blockSize << 1;
                        ys[0] = 0;
                        ys[1] = ys[2] = blockSize;
                        g.drawPolygon(xs, ys, 3); // Little trick to make the
                        // arrows of equal size
                    } else {
                        blockSize = Math.min(getWidth(), oneTouchSize);
                        xs[0] = xs[2] = blockSize;
                        xs[1] = 0;
                        ys[0] = 0;
                        ys[1] = blockSize;
                        ys[2] = blockSize << 1;
                    }
                    g.fillPolygon(xs, ys, 3);
                }
            }

            // Don't want the button to participate in focus traversable.
            public boolean isFocusTraversable() {
                return false;
            }
        };
    }

```

```
b.setMinimumSize(new Dimension(oneTouchSize, oneTouchSize));
b.setCursor(Cursor.getPredefinedCursor(Cursor.DEFAULT_CURSOR));
b.setFocusPainted(false);
b.setBorderPainted(false);
b.setRequestFocusEnabled(false);
return b;
}

@Override
public void paint(Graphics g) {
    super.paint(g);
    Dimension size = getSize();
    g.setColor(Color.white);
    g.fillRect(1, 1, size.width - 1, size.height - 1);
    g.setColor(Color.white);
    if (orientation == JSplitPane.VERTICAL_SPLIT) {
        g.fillRect(x1, 1, 30, size.height - 1);
    } else {
        g.fillRect(1, y1, size.width - 1, 30);
    }
    if (leftButton != null) {
        leftButton.repaint();
    }
    if (rightButton != null) {
        rightButton.repaint();
    }
}

/**
 * Used to layout a <code>BasicSplitPaneDivider</code>.
 * Layout for the divider
 * involves appropriately moving the left/right buttons around.
 * <p>
 */
public class DividerLayout implements LayoutManager {
    public void layoutContainer(Container c) {
        if (leftButton != null && rightButton != null) {
            if (splitPane.isOneTouchExpandable()) {
                Insets insets = getInsets();
                if (orientation == JSplitPane.VERTICAL_SPLIT) {
                    int extraX = (insets != null) ? insets.left : 0;
                    int blockSize = getHeight();
                    if (insets != null) {
                        blockSize -= (insets.top + insets.bottom);
                        blockSize = Math.max(blockSize, 0);
                    }
                    blockSize = Math.min(blockSize, oneTouchSize);
                    int y = (c.getSize().height - blockSize) / 2;
                    if (!centerOneTouchButtons) {
```

```

        y = (insets != null) ? insets.top : 0;
        extraX = 0;
    }

    int width = (int) MyBasicSplitPaneDivider.this.getSize().getWidth();
    x1 = width / 2 - oneTouchSize;
    leftButton.setBounds(extraX - oneTouchOffset + width / 2, y,
        blockSize * 2, blockSize);
    rightButton.setBounds(extraX - oneTouchOffset +
        oneTouchSize * 2 + width / 2, y,
        blockSize * 2, blockSize);
} else {
    int extraY = (insets != null) ? insets.top : 0;
    int blockSize = getWidth();
    if (insets != null) {
        blockSize -= (insets.left + insets.right);
        blockSize = Math.max(blockSize, 0);
    }
    blockSize = Math.min(blockSize, oneTouchSize);
    int x = (c.getSize().width - blockSize) / 2;
    if (!centerOneTouchButtons) {
        x = (insets != null) ? insets.left : 0;
        extraY = 0;
    }
    int height = (int) MyBasicSplitPaneDivider.this.getSize().getHeight();
    y1 = height / 2 - oneTouchSize;
    leftButton.setBounds(x, extraY - oneTouchOffset + height / 2,
        blockSize, blockSize * 2);
    rightButton.setBounds(x, extraY - oneTouchOffset +
        oneTouchSize * 2 + height / 2, blockSize,
        blockSize * 2);
}
} else {
    leftButton.setBounds(-5, -5, 1, 1);
    rightButton.setBounds(-5, -5, 1, 1);
}
}
}

public Dimension minimumLayoutSize(Container c) {
    // NOTE: This isn't really used, refer to
    // BasicSplitPaneDivider.getPreferredSize for the reason.
    // I leave it in hopes of having this used at some point.
    if (splitPane == null) {
        return new Dimension(0, 0);
    }
    Dimension buttonMinSize = null;
    if (splitPane.isOneTouchExpandable() && leftButton != null) {
        buttonMinSize = leftButton.getMinimumSize();
    }
}

```

```

    }

    Insets insets = getInsets();
    int width = getDividerSize();
    int height = width;
    if (orientation == JSplitPane.VERTICAL_SPLIT) {
        if (buttonMinSize != null) {
            int size = buttonMinSize.height;
            if (insets != null) {
                size += insets.top + insets.bottom;
            }
            height = Math.max(height, size);
        }
        width = 1;
    } else {
        if (buttonMinSize != null) {
            int size = buttonMinSize.width;
            if (insets != null) {
                size += insets.left + insets.right;
            }
            width = Math.max(width, size);
        }
        height = 1;
    }
    return new Dimension(width, height);
}

public Dimension preferredLayoutSize(Container c) {
    return minimumLayoutSize(c);
}

public void removeLayoutComponent(Component c) {
}

public void addLayoutComponent(String string, Component c) {
}

} // End of class BasicSplitPaneDivider.DividerLayout
}

}

//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornSplitPaneUITest.java
//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornTreeCellRenderrer.java
package OPE.OVQ.MSQ.OVU.PQE.platForm;

import javax.swing.JLabel;
import javax.swing.JTree;
import javax.swing.tree.DefaultMutableTreeNode;
import javax.swing.tree.DefaultTreeCellRenderer;
import java.awt.Component;

/**
 * 作者对jdk 开源插件进行2次开发。不在个人著作申请范围内。
 * @author LYG refer JDK authors from SUN Tech, 。
 */
* 作者 罗瑶光, 浏阳

```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
public class UnicornTreeCellRenderer extends DefaultTreeCellRenderer {
    public Component getTreeCellRendererComponent(JTree tree, Object value
        , boolean selected, boolean expanded, boolean leaf, int row, boolean hasFocus) {
        DefaultMutableTreeNode node= (DefaultMutableTreeNode)value;
        Object obj= node.getUserObject();
        if(obj instanceof JLabel) {
            JLabel label= (JLabel)obj;
            DefaultTreeCellRenderer tempCellRenderer= new DefaultTreeCellRenderer();
            tempCellRenderer.setLeafIcon(label.getIcon());
            return tempCellRenderer.getTreeCellRendererComponent(tree, label.getText()
                , selected, expanded, true, row, hasFocus);
        }
        return super.getTreeCellRendererComponent(tree, value, selected
            , expanded, leaf, row, hasFocus);
    }
}
```

```
//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornTreeCellRendererTest.java
//Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornTreeUI.java
/*
```

```
* %W% %E%
```

```
*
```

```
* Copyright (c) 2006, Oracle and/or its affiliates. All rights reserved.
```

```
* ORACLE PROPRIETARY/CONFIDENTIAL. Use is subject to license terms.
```

```
*/
```

```
package OPE.OVQ.MSQ.OVU.PQE.platForm;
```

```
import javax.swing.JComponent;
```

```
import javax.swing.UIManager;
```

```
import javax.swing.plaf.ComponentUI;
```

```
import javax.swing.plaf.basic.BasicTreeUI;
```

```
import javax.swing.tree.TreePath;
```

```
import java.awt.Color;
```

```
import java.awt.Graphics;
```

```
import java.awt.Insets;
```

```
import java.awt.Rectangle;
```

```
import java.beans.PropertyChangeEvent;
```

```
import java.beans.PropertyChangeListener;
```

```
/**
```

```
* 作者对jdk 开源插件进行2次开发。不在个人著作申请范围内。
```

```
*
```

```
* @author LYG refer JDK authors from SUN Tech, 。
```

```
*/
```

```

* 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
/**
* The metal look and feel implementation of <code>TreeUI</code>.
* <p>
* <code>MetalTreeUI</code> allows for configuring how to
* visually render the spacing and delineation between nodes. The following
* hints are supported:
*
* <table summary="Descriptions of supported hints: Angled, Horizontal, and None">
*   <tr>
*     <th><p align="left">Angled</p></th>
*     <td>A line is drawn connecting the child to the parent. For handling
*       of the root node refer to
*       {@link javax.swing.JTree#setRootVisible} and
*       {@link javax.swing.JTree#setShowsRootHandles}.
*     </td>
*   </tr>
*   <tr>
*     <th><p align="left">Horizontal</p></th>
*     <td>A horizontal line is drawn dividing the children of the root node.</td>
*   </tr>
*   <tr>
*     <th><p align="left">None</p></th>
*     <td>Do not draw any visual indication between nodes.</td>
*   </tr>
* </table>
*
* <p>
* As it is typically impractical to obtain the <code>TreeUI</code> from
* the <code>JTree</code> and cast to an instance of <code>MetalTreeUI</code>
* you enable this property via the client property
* <code>JTree.lineStyle</code>. For example, to switch to
* <code>Horizontal</code> style you would do:
* <code>tree.putClientProperty("JTree.lineStyle", "Horizontal");</code>
* <p>
* The default is <code>Angled</code>.
*/
public class UnicornTreeUI extends BasicTreeUI {
    private static Color lineColor;
    private static final String LINE_STYLE = "JTree.lineStyle";

```

```
private static final String LEG_LINE_STYLE_STRING = "Angled";
private static final String HORIZ_STYLE_STRING = "Horizontal";
private static final String NO_STYLE_STRING = "None";
private static final int LEG_LINE_STYLE = 2;
private static final int HORIZ_LINE_STYLE = 1;
private static final int NO_LINE_STYLE = 0;
private int lineStyle = LEG_LINE_STYLE;
private PropertyChangeListener lineStyleListener = new LineListener();
// Boilerplate
public static ComponentUI createUI(JComponent x) {
    return new UnicornTreeUI();
}
public UnicornTreeUI() {
    super();
}
public int getHorizontalLegBuffer() {
    return 3;
}
public void installUI(JComponent c) {
    super.installUI(c);
    lineColor = UIManager.getColor("Tree.line");
    Object lineStyleFlag = c.getClientProperty(LINE_STYLE);
    decodeLineStyle(lineStyleFlag);
    c.addPropertyChangeListener(lineStyleListener);
}
public void uninstallUI(JComponent c) {
    c.removePropertyChangeListener(lineStyleListener);
    super.uninstallUI(c);
}
/**
 * this function converts between the string passed into the client property
 * and the internal representation (currently and int)
 */
public void decodeLineStyle(Object lineStyleFlag) {
    if (lineStyleFlag == null || lineStyleFlag.equals(LEG_LINE_STYLE_STRING)) {
        lineStyle = LEG_LINE_STYLE; // default case
    } else {
        if (lineStyleFlag.equals(NO_STYLE_STRING)) {
            lineStyle = NO_LINE_STYLE;
        } else if (lineStyleFlag.equals(HORIZ_STYLE_STRING)) {
            lineStyle = HORIZ_LINE_STYLE;
        }
    }
}
public boolean isLocationInExpandControl(int row, int rowLevel, int mouseX, int mouseY) {
    if (tree != null && !isLeaf(row)) {
        int boxWidth;
```

```

        if (getExpandedIcon() != null)
            boxWidth = getExpandedIcon().getIconWidth() + 6;
        else
            boxWidth = 8;
        Insets i = tree.getInsets();
        int boxLeftX = (i != null) ? i.left : 0;
        boxLeftX += (((rowLevel + depthOffset - 1) * totalChildIndent)
            + getLeftChildIndent()) - boxWidth / 2;
        int boxRightX = boxLeftX + boxWidth;
        return mouseX >= boxLeftX && mouseX <= boxRightX;
    }
    return false;
}

public void paint(Graphics g, JComponent c) {
    super.paint(g, c);
    // Paint the lines
    if (lineStyle == HORIZ_LINE_STYLE && !largeModel) {
        paintHorizontalSeparators(g, c);
    }
}

public void paintHorizontalSeparators(Graphics g, JComponent c) {
    g.setColor(lineColor);
    Rectangle clipBounds = g.getClipBounds();
    int beginRow = getRowForPath(tree, getClosestPathForLocation(tree, 0, clipBounds.y));
    int endRow = getRowForPath(tree, getClosestPathForLocation(tree, 0
        , clipBounds.y + clipBounds.height - 1));
    if (beginRow <= -1 || endRow <= -1) {
        return;
    }
    for (int i = beginRow; i <= endRow; ++i) {
        TreePath path = getPathForRow(tree, i);
        if (path != null && path.getPathCount() == 2) {
            Rectangle rowBounds = getPathBounds(tree, getPathForRow(tree, i));
            // Draw a line at the top
            if (rowBounds != null) {
                g.setColor(Color.black);
                g.drawLine(clipBounds.x, rowBounds.y, clipBounds.x
                    + clipBounds.width, rowBounds.y);
                g.setColor(Color.white);
                g.drawLine(clipBounds.x, rowBounds.y + 1, clipBounds.x
                    + clipBounds.width, rowBounds.y + 1);
                g.setColor(Color.orange);
                g.drawLine(clipBounds.x, rowBounds.y + 2, clipBounds.x
                    + clipBounds.width, rowBounds.y + 2);
            }
        }
    }
}

```

```

    }

    public void paintVerticalPartOfLeg(Graphics g, Rectangle clipBounds
        , Insets insets, TreePath path) {
        if (lineStyle == LEG_LINE_STYLE) {
            super.paintVerticalPartOfLeg(g, clipBounds, insets, path);
        }
    }

    public void paintHorizontalPartOfLeg(Graphics g, Rectangle clipBounds,
        Insets insets, Rectangle bounds,
        TreePath path, int row,
        boolean isExpanded,
        boolean hasBeenExpanded, boolean
            isLeaf) {
        if (lineStyle == LEG_LINE_STYLE) {
            super.paintHorizontalPartOfLeg(g, clipBounds, insets, bounds,
                path, row, isExpanded,
                hasBeenExpanded, isLeaf);
        }
    }
}

/**
 * This class listens for changes in line style
 */

class LineListener implements PropertyChangeListener {
    public void propertyChange(PropertyChangeEvent e) {
        String name = e.getPropertyName();
        if (name.equals(LINE_STYLE)) {
            decodeLineStyle(e.getNewValue());
        }
    }
} // end class PaletteListener
}

/Trif_DNA-master/C_L_E/OPE/OVQ/MSQ/OVU/PQE/platForm/UnicornTreeUITest.java

    OPE.OVU.MVQ.OVU.PQE.nodeView

/Trif_DNA-master/C_L_E/OPE/OVU/MVQ/OVU/PQE/nodeView/CacuString.java
package OPE.OVU.MVQ.OVU.PQE.nodeView;

/**
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CacuString {
    //因未加速计算, 所以算子名不进行码农的修养化
    public String cauString(String string) {

```

```

StringBuilder stringBuilder = new StringBuilder();

if (string.equals("Node")) {
    return null;
}

char[] a = string.toCharArray();
for (int i = 0; i < string.length(); i++) {
    if (a[i] == 't' && a[i + 1] == 'e' && a[i + 2] == 'x' && a[i + 3] == 't') {
        for (int j = i + 5; a[j] != ';'; j++) {
            stringBuilder.append(a[j]);
        }
        return stringBuilder.toString();
    }
}

return stringBuilder.toString();
}
}

//Trif_DNA-master/C_L_E/OPE/OVU/MVQ/OVU/PQE/nodeView/CacuStringTest.java
//Trif_DNA-master/C_L_E/OPE/OVU/MVQ/OVU/PQE/nodeView/NodeShow.java
package OPE.OVU.MVQ.OVU.PQE.nodeView;

import OPE.OEQ.MCQ.OVU.PQE.osgi.LinkOSGI;

import OPE.OVQ.MSQ.OVU.PQE.platForm.UnicornTreeCellRenderer;

import OPE.SI.MCI.OEI.OVU.PQE.extOSGI.OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register;

import ME.VPC.M.app.App;

import OSI.OPE.OVU.PQE.flash.GUISample;

import nodeView.NodeShow_X_S;

import javax.swing.JLabel;

import javax.swing.JTextPane;

import javax.swing.JTree;

import javax.swing.tree.DefaultMutableTreeNode;

import javax.swing.tree.DefaultTreeModel;

import javax.swing.tree.TreeSelectionModel;

import java.awt.Font;

import java.awt.PopupMenu;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
@SuppressWarnings("serial")

public class NodeShow extends NodeShow_X_S {
    public NodeShow(GUISample gUISample, Object[][] tableData_old,
        JTextPane text, PopupMenu popupMenu, App NE) {
        this.text = text;
        this.tableData_old = tableData_old;
    }
}

```

```

        this.popupMenu = popupMenu;
        this.gUISample = gUISample;
        this.u = NE.app_S;
        this._A = NE.app_S._A;
        this.pos = NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn;
        link = new LinkOSGI();
        S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01");
        first = new OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register(
this.tableData_old, this.text, NE).Rigester(first, link, NE);
        S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-02");
        U_Tree();
        S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-03");
    }
    public void U_Tree() {
        DefaultMutableTreeNode root = new DefaultMutableTreeNode("Node");
        treeModel = new DefaultTreeModel(root);
        if (null == tree) {
            tree = new JTree(treeModel);
        } else {
            tree.clearSelection();
            tree.removeAll();
        }
        tree.setModel(treeModel);
        // tree= new JTree(treeModel);
        tree.setExpandsSelectedPaths(true);
        tree.getSelectionModel()
.setSelectionMode(TreeSelectionMode.SINGLE_TREE_SELECTION);
        tree.putClientProperty("JTree.lineStyle", "Horizontal");
        tree.setEditable(false);
        UnicornTreeCellRenderer myCellRenderer = new UnicornTreeCellRenderer();
        myCellRenderer.setFont(new Font("Serif", Font.ITALIC, 12));
        tree.setCellRenderer(myCellRenderer);
        DefaultMutableTreeNode BI = new DefaultMutableTreeNode("BI");
        DefaultMutableTreeNode SOUND = new DefaultMutableTreeNode("SOUND");
        DefaultMutableTreeNode IMAGE = new DefaultMutableTreeNode("IMAGE");
        DefaultMutableTreeNode MOVIE = new DefaultMutableTreeNode("MOVIE");
        root.add(BI);
        root.add(SOUND);
        root.add(IMAGE);
        root.add(MOVIE);
        while (null != first) {
            if (null == first.currentFace.AMV_MVS_VSQ) {
JLabel label;
label = new JLabel();
label.setIcon(first.thisIcon);
label.setText(first._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS);
DefaultMutableTreeNode node = new DefaultMutableTreeNode(label);

```

```

        root.add(node);

        } else if (first.currentFace.AMV_MVS_VSQ.equals("BI")) {

JLabel label;
label = new JLabel();
label.setIcon(first.thisIcon);
label.setText(first._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS);
DefaultMutableTreeNode node = new DefaultMutableTreeNode(label);
BI.add(node);

        } else if (first.currentFace.AMV_MVS_VSQ.equals("SOUND")) {

JLabel label;
label = new JLabel();
label.setIcon(first.thisIcon);
label.setText(first._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS);
DefaultMutableTreeNode node = new DefaultMutableTreeNode(label);
SOUND.add(node);

        } else if (first.currentFace.AMV_MVS_VSQ.equals("IMAGE")) {

JLabel label;
label = new JLabel();
label.setIcon(first.thisIcon);
label.setText(first._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS);
DefaultMutableTreeNode node = new DefaultMutableTreeNode(label);
IMAGE.add(node);

        } else if (first.currentFace.AMV_MVS_VSQ.equals("MOVIE")) {

JLabel label;
label = new JLabel();
label.setIcon(first.thisIcon);
label.setText(first._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS);
DefaultMutableTreeNode node = new DefaultMutableTreeNode(label);
MOVIE.add(node);

        } else {

JLabel label;
label = new JLabel();
label.setIcon(first.thisIcon);
label.setText(first._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS);
DefaultMutableTreeNode node = new DefaultMutableTreeNode(label);
root.add(node);

        }

        if (null == first.next) {

break;

        }

        first = first.next;

    }

    // tree.addMouseListener(GUI_Sample);
    this.setViewportView(tree);

}

}

```

```

/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/CheckRangeVPS.java
package OPE.OVU.MVU.OVU.PQE.nodeEdit;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.CheckRange;

import java.awt.event.MouseEvent;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CheckRangeVPS implements CheckRange {

    public void doCheckRange(LinkNode first, LinkNode node, MouseEvent arg0) {

        LinkNode linkNode= first;

        int x, y;

        x= arg0.getX();

        y= arg0.getY();

        while(null!= linkNode){

            if((x> linkNode.x- 20)&& (x< linkNode.x+ 100)&& (y> linkNode.y- 100)&& (y< linkNode.y+ 16)

            && (!node.primaryKey.equalsIgnoreCase(linkNode.primaryKey)))){

linkNode.beconnect= true;

linkNode.tBeconnect= true;

linkNode.tBeconnectX= node.x;

linkNode.tBeconnectY= node.y;

linkNode.tBeconnectID= node.ID;

linkNode.tBeconnectPrimaryKey= node.primaryKey;

linkNode.tBeconnectName= new String(node.name);

linkNode.tNode= node;

return;

            }

            if((x> linkNode.x- 20)&& (x< linkNode.x+ 50)&& (y> linkNode.y+ 16)&& (y< linkNode.y+ 32)

            && (!node.primaryKey.equalsIgnoreCase(linkNode.primaryKey)))){

linkNode.beconnect= true;

linkNode.mBeconnect= true;

linkNode.mBeconnectX= node.x;

linkNode.mBeconnectY= node.y;

linkNode.mBeconnectID= node.ID;

linkNode.mBeconnectPrimaryKey= node.primaryKey;

linkNode.mBeconnectName= new String(node.name);

linkNode.mNode= node;

return;

            }

            if((x> linkNode.x- 20)&& (x< linkNode.x+ 50)&& (y> linkNode.y+ 32)&& (y< linkNode.y+ 100)

            && (!node.primaryKey.equalsIgnoreCase(linkNode.primaryKey)))){

```



```

        if (null == node.next) {
            break;
        }
        node = node.next;
    }
    Object[] object = new Object[3];
    object[0] = "";
    object[1] = 0;
    object[2] = "";
    return object;
}
}

//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/ChooseQ_VPSTest.java
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawArrowVPS.java
package OPE.OVU.MVU.OVU.PQE.nodeEdit;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.DrawArrow;

import java.awt.BasicStroke;

import java.awt.Graphics2D;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class DrawArrowVPS implements DrawArrow {

    public void doDrawArrow(Graphics2D graphics2D, int x, int y, int connectX, int connectY) {
        x += 10;
        connectX -= 10;
        graphics2D.setStroke(new BasicStroke(2, BasicStroke.CAP_SQUARE, BasicStroke.JOIN_ROUND));
        drawCurve(graphics2D, x, y, connectX, connectY, 6.0);
        DrawSinLineVPS.drawHead(connectX - 8, connectY - 3, graphics2D);
    }

    //下面这个化三角箭头函数是JAVA2D awt demo
    //罗瑶光先生进行了2次修改.

    public void drawCurve(Graphics2D graphics2D, int x, int y
        , int connectX, int connectY, double scale) {
        double distanceX = (double) Math.abs(x - connectX);
        double distanceY = (double) Math.abs(y - connectY);
        double signOfPointX = (x - connectX < 0) ? 1.0 : -1;
        double signOfPointY = (y - connectY < 0) ? 1.0 : -1;
        double averageOfDistanceY = (distanceX == 0) ? 0 : distanceY / distanceX;
        double signOfPointYWithaverageOfDistanceY = averageOfDistanceY * signOfPointY;
        double oldRegisterY = 0.0;
        boolean firstTime = true;
        double signOfPointYWithaverageOfDistanceYKernel = signOfPointYWithaverageOfDistanceY * 8;

```

```

double averageOfDistanceYKernel = averageOfDistanceY * 8 / 6;

double signOfPointYWithaverageOfDistanceYCount = signOfPointYWithaverageOfDistanceYKernel;

double averageOfDistanceYCount = averageOfDistanceYKernel;

if (1 == signOfPointX) {
    double registerY = (double) y;
    int endX = x;
    for (int i = x; i < connectX - 16; i += 8) {
        registerY = y + signOfPointYWithaverageOfDistanceYCount + scale
            * Math.sin(averageOfDistanceYCount);
        graphics2D.drawLine(i, true == firstTime ? (int) registerY
            : (int) oldRegisterY, i + 8, (int) registerY);
        oldRegisterY = registerY;
        endX = i + 8;
        firstTime = false;
        signOfPointYWithaverageOfDistanceYCount += signOfPointYWithaverageOfDistanceYKernel;
        averageOfDistanceYCount += averageOfDistanceYKernel;
    }
    graphics2D.drawLine(endX, (int) registerY, connectX - 8, connectY);
    return;
}

if (-1 == signOfPointX) {
    for (int i = x; i > connectX + 2; i -= 8) {
        double registerY = y + signOfPointYWithaverageOfDistanceYCount + scale
            * Math.sin(averageOfDistanceYCount);
        graphics2D.drawLine(i, true == firstTime ? (int) registerY
            : (int) oldRegisterY, i - 8, (int) registerY);
        oldRegisterY = registerY;
        firstTime = false;
        signOfPointYWithaverageOfDistanceYCount += signOfPointYWithaverageOfDistanceYKernel;
        averageOfDistanceYCount += averageOfDistanceYKernel;
    }
}
}

}

//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawArrowVPSTest.java
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawFlashSide.java
package OPE.OVU.MVU.OVU.PQE.nodeEdit;

import java.awt.Color;

import java.awt.Graphics2D;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层

```

 **/

```

public class DrawFlashSide {

    public static void drawFlashSide(Graphics2D graphics2D, int x, int y, int flash) {

        if (0 >= flash) {

            graphics2D.setColor(Color.blue);

            DrawSinLineVPS.drawCosLine(x, y, graphics2D);

            graphics2D.setColor(Color.pink);

            DrawSinLineVPS.drawSinLine(x, y, graphics2D);

        }

        if (1 == flash) {

            graphics2D.setColor(Color.ORANGE);

            DrawSinLineVPS.drawCosLine(x, y, graphics2D);

            graphics2D.setColor(Color.blue);

            DrawSinLineVPS.drawSinLine(x, y, graphics2D);

        }

        if (2 <= flash) {

            graphics2D.setColor(Color.ORANGE);

            DrawSinLineVPS.drawCosLine(x, y, graphics2D);

            graphics2D.setColor(Color.RED);

            DrawSinLineVPS.drawSinLine(x, y, graphics2D);

        }

        drawConnect(graphics2D, x, y);

    }

    //for cell postfix

    private static void drawConnect(Graphics2D graphics2D, int x, int y) {

        graphics2D.drawOval(x + 10, y - 8, 4, 4);

        graphics2D.drawOval(x - 8, y + 22, 4, 4);

        graphics2D.drawOval(x + 2, y + 52, 4, 4);

        graphics2D.drawOval(x + 62, y + 26, 4, 4);

    }

    public static void D_FlashSide(Graphics2D graphics2D, int x, int y) {

        graphics2D.setColor(new Color(255, 255, 255));

        DrawSinLineVPS.drawCosLine(x, y, graphics2D);

        graphics2D.setColor(new Color(255, 255, 255));

        DrawSinLineVPS.drawSinLine(x, y, graphics2D);

        drawConnect(graphics2D, x, y);

    }

}

//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawFlashSideTest.java
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawSinLineVPS.java
package OPE.OVU.MVU.OVU.PQE.nodeEdit;

import VSQ.neroCell.DrawArrowHead;

import VSQ.neroCell.DrawNeroCellMask31;

import VSQ.neroCell.DrawNeroCellMask32;

import VSQ.neroCell.DrawNeroCellMask33;

import java.awt.Graphics2D;

//作者罗瑶光

/*

```

* */

```

public class DrawSinLineVPS {
    public static void fastDraw() {
        DrawNeroCellMask31.coordSwap(DrawNeroCellMask31.neroShape);
        DrawNeroCellMask32.coordSwap(DrawNeroCellMask32.neroShape);
        DrawNeroCellMask33.coordSwap(DrawNeroCellMask33.neroShape);
        DrawArrowHead.coordSwap(DrawArrowHead.neroShape);
    }
    //快速计算改写
    public static void drawCosLine(int x0, int y0, Graphics2D graphics2D) {
        //DrawSinLineVPS.fastDraw();
        for (int y = 0; y < DrawNeroCellMask31.coords[0].length; y++) {
            if (-1 == DrawNeroCellMask31.coords[0][y]) {
                return;
            }
            graphics2D.drawLine(DrawNeroCellMask31.coords[0][y] + x0,
                DrawNeroCellMask31.coords[1][y] + y0,
                DrawNeroCellMask31.coords[0][y] + x0,
                DrawNeroCellMask31.coords[1][y] + y0);
        }
    }
    public static void drawSinLine(int x0, int y0, Graphics2D graphics2D) {
        for (int y = 0; y < DrawNeroCellMask32.coords[0].length; y++) {
            if (-1 == DrawNeroCellMask32.coords[0][y]) {
                return;
            }
            graphics2D.drawLine(DrawNeroCellMask32.coords[0][y] + x0,
                DrawNeroCellMask32.coords[1][y] + y0,
                DrawNeroCellMask32.coords[0][y] + x0,
                DrawNeroCellMask32.coords[1][y] + y0);
        }
    }
    public static void drawHead(int x0, int y0, Graphics2D graphics2D) {
        for (int y = 0; y < DrawArrowHead.coords[0].length; y++) {
            if (-1 == DrawArrowHead.coords[0][y]) {
                return;
            }
            graphics2D.drawLine(DrawArrowHead.coords[0][y] + x0,
                DrawArrowHead.coords[1][y] + y0,
                DrawArrowHead.coords[0][y] + x0,
                DrawArrowHead.coords[1][y] + y0);
        }
    }
}

```

```

        DrawArrowHead.coords[1][y] + y0);
    }
}

//public static void drawCosLine(int x0, int y0, Graphics2D graphics2D) {
//    for(int y= 0; y< DrawNeroCellMask31.neroShape.length; y++) {
//for(int x= 0; x< DrawNeroCellMask31.neroShape[0].length; x++) {
//    if(1== DrawNeroCellMask31.neroShape[y][x]) {
//        graphics2D.drawLine(x + x0, y + y0, x + x0, y + y0);
//    }
//}
//    }
//}
//
//public static void drawSinLine(int x0, int y0, Graphics2D graphics2D) {
//    for(int y= 0; y< DrawNeroCellMask32.neroShape.length; y++) {
//for(int x= 0; x< DrawNeroCellMask32.neroShape[0].length; x++) {
//    if(1== DrawNeroCellMask32.neroShape[y][x]) {
//        graphics2D.drawLine(x + x0, y + y0, x + x0, y + y0);
//    }
//}
//    }
//}
//
//public static void drawHead(int x0, int y0, Graphics2D graphics2D) {
//    for(int y= 0; y< DrawArrowHead.neroShape.length; y++) {
//for(int x= 0; x< DrawArrowHead.neroShape[0].length; x++) {
//    if(1== DrawArrowHead.neroShape[y][x]) {
//        graphics2D.drawLine(x + x0, y + y0, x + x0, y + y0);
//    }
//}
//    }
//}

//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DrawSinLineVPSTest.java
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DynamicLineU_VPS.java
package OPE.OVU.MVU.OVU.PQE.nodeEdit;

import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.DynamicLine_U;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class DynamicLineU_VPS implements DynamicLine_U {

```

```

public void exec(LinkNode first, LinkNode node) {
    LinkNode linkNode = first;
    while (null != linkNode) {
        linkNode = linkNode.next;
    }
    linkNode = null;
}
}
/*
*
* public class DynamicLineU_VPS implements DynamicLine_U {
    public void exec(LinkNode first, LinkNode node) {
        LinkNode linkNode = first;
        while (null != linkNode) {
            if (linkNode.primaryKey.equalsIgnoreCase(node.tBeconnectPrimaryKey)) {
//                node.tBeconnectX= linkNode.x;
//                node.tBeconnectY= linkNode.y;
            }
            if (linkNode.tBeconnectPrimaryKey.equalsIgnoreCase(node.primaryKey)) {
//                linkNode.tBeconnectX= node.x;
//                linkNode.tBeconnectY= node.y;
            }
            if (linkNode.primaryKey.equalsIgnoreCase(node.mBeconnectPrimaryKey)) {
//                node.mBeconnectX= linkNode.x;
//                node.mBeconnectY= linkNode.y;
            }
            if (linkNode.mBeconnectPrimaryKey.equalsIgnoreCase(node.primaryKey)) {
//                linkNode.mBeconnectX= node.x;
//                linkNode.mBeconnectY= node.y;
            }
            if (linkNode.primaryKey.equalsIgnoreCase(node.dBeconnectPrimaryKey)) {
//                node.dBeconnectX= linkNode.x;
//                node.dBeconnectY= linkNode.y;
            }
            if (linkNode.dBeconnectPrimaryKey.equalsIgnoreCase(node.primaryKey)) {
//                linkNode.dBeconnectX= node.x;
//                linkNode.dBeconnectY= node.y;
            }
            linkNode = linkNode.next;
        }
        linkNode = null;
    }
}
*
**/
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/DynamicLineU_VPSTest.java
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/LinkList.java
package OPE.OVU.MVU.OVU.PQE.nodeEdit;

```

```
import OPE.OEQ.MCQ.OVU.PQE.osgi.NodeOSGI;
```

```
import ME.VPC.M.app.App;
```

```
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
```

```
public class LinkList {
    int index = 0;
    String key;
    public LinkNode first;
    public int sum_of_nude = 0;
    public LinkList() {
    }
    public boolean search(LinkNode linkNode, String key) {
        if (null == linkNode) {
            return false;
        }
        if (linkNode.name.equals(key)) {
            return true;
        }
        while (null != linkNode.next) {
            linkNode = linkNode.next;
            if (linkNode.name.equals(key)) {
                while (null != linkNode.pre) {
                    linkNode = linkNode.pre;
                }
                return true;
            }
        }
        return false;
    }
    public LinkNode addNodeOnlyWithFace(LinkNode linkNode, NodeOSGI nOSGI, App NE) {
        NodeOSGI currentOSGI = nOSGI;
        while (null != currentOSGI && null != currentOSGI.pre) {
            currentOSGI = currentOSGI.pre;
        }
        if (null != linkNode) {
            while (null != currentOSGI) {
                if (currentOSGI._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS.equals(linkNode.name)) {
                    linkNode.thisFace = currentOSGI.currentFace.AOP_MVE_CSI_DUQ(NE);
                    sum_of_nude++;
                    index++;
                }
            }
        }
    }
}
```



```

        return linkNode;
    }
    currentOSGI = currentOSGI.next;
}
}
index++;
sum_of_nude++;
return linkNode;
}

public LinkNode addNode(LinkNode linkNode, String treeNodeName
, int x, int y, NodeOSGI nOSGI, App NE) {
    NodeOSGI currentOSGI = nOSGI;
    while (null != currentOSGI && null != currentOSGI.pre) {
        currentOSGI = currentOSGI.pre;
    }
    if (null == linkNode) {
        while (null != currentOSGI) {
            if (currentOSGI._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS.equals(treeNodeName)) {
                linkNode = new LinkNode();
                linkNode.I_Name(treeNodeName, x, y, ++index);
                linkNode.thisFace = currentOSGI.currentFace.AOP_MVE_CSI_DUQ(NE);
                linkNode.next = null;
                linkNode.pre = null;
                sum_of_nude++;
                return linkNode;
            }
            currentOSGI = currentOSGI.next;
        }
    }
    while (null != linkNode && null != linkNode.next) {
        linkNode = linkNode.next;
    }
    while (null != currentOSGI) {
        if (currentOSGI._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS.equals(treeNodeName)) {
            //linkNode=new linkNode();
            LinkNode node = new LinkNode();
            node.I_Name(treeNodeName, x, y, ++index);
            node.thisFace = currentOSGI.currentFace.AOP_MVE_CSI_DUQ(NE);
            node.pre = linkNode;
            linkNode.next = node;
            sum_of_nude++;
            return linkNode;
        }
        currentOSGI = currentOSGI.next;
    }
    while (null != linkNode && null != linkNode.pre) {
        linkNode = linkNode.pre;
    }
}

```

```

    }
    sum_of_nude++;
    return linkNode;
}

public LinkNode deletNode(LinkNode linkNode, String name, int ID, String primaryKey) {
    if (null != linkNode) {
        if (linkNode.name.equals(name) && linkNode.ID == ID
            && linkNode.primaryKey.equalsIgnoreCase(primaryKey)) {
            if (null != linkNode.next) {
                linkNode = linkNode.next;
                linkNode.pre = null;
                return linkNode;
            }
            if (null == linkNode.next) {
                linkNode = null;
                return linkNode;
            }
        }
        while (null != linkNode.next) {
            linkNode = linkNode.next;
            if (linkNode.name.equals(name) && linkNode.ID == ID
                && linkNode.primaryKey.equalsIgnoreCase(primaryKey)) {
                if (null != linkNode.next) {
                    @SuppressWarnings("unused")
                    LinkNode node = linkNode;
                    linkNode = linkNode.next;
                    linkNode.pre = linkNode.pre.pre;
                    linkNode.pre.next = linkNode;
                    node = null;
                    linkNode = Sort.sort(linkNode);
                    return linkNode;
                }
                if (null == linkNode.next) {
                    linkNode = linkNode.pre;
                    linkNode.next = null;
                    linkNode = Sort.sort(linkNode);
                    return linkNode;
                }
            }
        }
    }
    return linkNode;
}
}

//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/LinkListTest.java
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/LinkNode.java
package OPE.OVU.MVU.OVU.PQE.nodeEdit;

import VSQ.etl.LinkNode_X_S;

```

```

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class LinkNode extends LinkNode_X_S implements Cloneable {
    public void I_Name(String SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS, int x1, int y1, int id1) {
        beconnect = false;
        rightChoose = false;
        leftChoose = false;
        tBeconnect = false;
        mBeconnect = false;
        dBeconnect = false;
        x = x1;
        y = y1;
        name = new String(SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS);
        ID = id1;
        tBeconnectPrimaryKey = "";
        mBeconnectPrimaryKey = "";
        dBeconnectPrimaryKey = "";
        tBeconnectID = 0;
        mBeconnectID = 0;
        dBeconnectID = 0;
        primaryKey = "" + Math.random();
    }
    public void I_xy(int x1, int y1) {
        x = x1;
        y = y1;
    }
    public void I_choose(Boolean choose) {
        leftChoose = choose;
    }
    public LinkNode clone() {
        return this;
    }
}

//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/LinkNodeTest.java
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/Sort.java
package OPE.OVU.MVU.OVU.PQE.nodeEdit;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Sort {  
    public static LinkNode sort(LinkNode first) {  
        while (null != first && null != first.pre) {  
            first = first.pre;  
        }  
        return first;  
    }  
}  
  
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/SortTest.java  
//Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/U_RelatedLineVPS.java  
package OPE.OVU.MVU.OVU.PQE.nodeEdit;
```

```
import OPE.MVU.OVU.OVU.PQE.nodeEdit.C.U_RelatedLine;
```

```
/*
```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class U_RelatedLineVPS implements U_RelatedLine {  
    public void doUpdateRelatedLine(LinkNode first, String currentNodeName  
        , int currentNodeID, String currentNodePrimaryKey) {  
        first = Sort.sort(first);  
        while (null != first) {  
            if (first.tBeconnect && first.tBeconnectName.equals(currentNodeName)  
                && first.tBeconnectID == currentNodeID  
                && first.tBeconnectPrimaryKey.equalsIgnoreCase(currentNodePrimaryKey)) {  
                first.tBeconnect = false;  
            }  
            if (first.mBeconnect && first.mBeconnectName.equals(currentNodeName)  
                && first.mBeconnectID == currentNodeID  
                && first.mBeconnectPrimaryKey.equalsIgnoreCase(currentNodePrimaryKey)) {  
                first.mBeconnect = false;  
            }  
            if (first.dBeconnect && first.dBeconnectName.equals(currentNodeName)  
                && first.dBeconnectID == currentNodeID  
                && first.dBeconnectPrimaryKey.equalsIgnoreCase(currentNodePrimaryKey)) {  
                first.dBeconnect = false;  
            }  
            if (null == first.next) {  
                break;  
            }  
        }  
    }  
}
```

```

        first = first.next;
    }
}
}

/Trif_DNA-master/C_L_E/OPE/OVU/MVU/OVU/PQE/nodeEdit/U_RelatedLineVPSTest.java

    OPE.SLMCI.OEI.OVU.PQE.extOSGI

/Trif_DNA-master/C_L_E/OPE/SL/MCI/OEI/OVU/PQE/extOSGI/OSGI_chansfer.java
package OPE.SLMCI.OEI.OVU.PQE.extOSGI;

import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.Sort;

//作者 罗瑶光

public class OSGI_chansfer {

    //增加tin shell output map, 罗瑶光20211008

    //链接是手操作的, 在一键链接的时候输出不适用, 准备修改成执行后操作, tableout记录

    //准备把执行shellpanel后的 output tinmap 拿出来

    //return; // 涉及多个节点测试

    public OSGI_chansfer(LinkNode node, LinkNode first) {

        first = Sort.sort(first);

        LinkNode linkNode = new LinkNode();

        linkNode = first;

        //节点只有上中下3个input, 于是优化成 max=3;

        int max = 0;

        while (null != linkNode) {

            if (node.tBeconnect

                //&&node.tBeconnectID== linkNode.ID

                && node.tBeconnectName.equals(linkNode.name)

                && (node.tBeconnectPrimarykey.equalsIgnoreCase(linkNode.primaryKey))) {

                node.thisFace._SQ__OPE_OPC_ECI.toptablein

                    = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.tableout;

                node.thisFace._SQ__OPE_OPC_ECI.topgin

                    = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.gout;

                node.thisFace._SQ__OPE_OPC_ECI.topOutput

                    = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.outputOut;

                max++;

            }

            if (node.mBeconnect

                //&&node.mBeconnectID== linkNode.ID

                && node.mBeconnectName.equals(linkNode.name)

                && (node.mBeconnectPrimarykey.equalsIgnoreCase(linkNode.primaryKey))) {

                node.thisFace._SQ__OPE_OPC_ECI.midtablein

                    = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.tableout;

                node.thisFace._SQ__OPE_OPC_ECI.midgin

                    = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.gout;

                node.thisFace._SQ__OPE_OPC_ECI.midOutput

                    = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.outputOut;

                //return;

                max++;
            }
        }
    }
}

```

```

    }
    if (node.dBeconnect
        //&&node.dBeconnectID== linkNode.ID
        && node.dBeconnectName.equals(linkNode.name)
        && (node.dBeconnectPrimaryKey.equalsIgnoreCase(linkNode.primaryKey))) {
        node.thisFace._SQ__OPE_OPC_ECI.downtablein
            = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.tableout;
        node.thisFace._SQ__OPE_OPC_ECI.downgin
            = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.gout;
        node.thisFace._SQ__OPE_OPC_ECI.downOutput
            = linkNode.thisFace._SQ__OVQ_OSQ_VSQ.outputOut;
        //return;
        max++;
    }
    if (null == linkNode.next || 3 == max) { //以后节点类型多了就重新设计。20211011 罗瑶光
        break;
    }
    linkNode = linkNode.next;
}
}
}
}

/*
Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/PQE/extOSGI/OSGI_findNode.java

* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package OPE.SI.MCI.OEI.OVU.PQE.extOSGI;
import OPE.OVU.MVU.OVU.PQE.nodeEdit.LinkNode;

//作者 罗瑶光
public class OSGI_findNode implements Cloneable{
    private LinkNode findNode;
    public OSGI_findNode() {
    }
    public LinkNode getFindNode() {
        return findNode;
    }
    //增加tin shell output map, 罗瑶光20211008
    public LinkNode findNode(String primaryKeyID, LinkNode first)
        {
            LinkNode linkNode= first.clone();
            while(null!= linkNode) {
                if(null!= linkNode.primaryKey) {
                    if(linkNode.primaryKey.equalsIgnoreCase(primaryKeyID)) {

```

```

        findNode= linkNode.clone();

        return findNode;

    }

    }

    linkNode= linkNode.next;

}

return null;

}

}

//Trif_DNA-master/C_L_E/OPE/SL/MCI/OEI/OVU/PQE/extOSGI/OSGI_findNodeTest.java
//Trif_DNA-master/C_L_E/OPE/SL/MCI/OEI/OVU/PQE/extOSGI/OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register.java
package OPE.SL.MCI.OEI.OVU.PQE.extOSGI;

import C_A.ME.analysis.C.A;

import ME.VPC.M.app.App;

import ME.VPC.S.ne.App_S;

import OPE.OEQ.MCQ.OVU.PQE.osgi.LinkOSGI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.NodeOSGI;

import OPE.OEQ.MCQ.OVU.PQE.osgi.OSU_MCI_MCU_MSI;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.fft.FFTFilterNodeInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.fft2DFilter.Ft2DFilterInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.freqCount.FreqCountNodeInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.gaussianWav2DFilter.GaussianWav2DFilterNodeInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.houghWavFilter.HoughWavFilterNodeInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.laplacianFilter.LaplacianFilterNodeInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.logFFT.LogFFTInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.logFFTcount.LogFFTcountInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.lygFilter.LygFilterNodeInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.lygSlaveFilter.LygSlaveFilterInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.maxMiniFilter.MaxMiniFilterNodeInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.medianFilter.MedianFilterNodeInterface;

import OSI.OEU.OSU.MSQ.OSU.AVU.OSQ.wavRead.WavReadNodeInterface;

import OSI.OSU.MSQ.ASU.OSU.PSU.MSU.AVQ.ASQ.OPE.xlsReaderNode.XlsReaderNodeInterface;

import OSI.OSU.PSI.OSU.MSQ.VQ.SQ.lygWrite.LYGWriteNodeInterface;

import OSI.OSU.PSU.OSU.MSQ.VQ.SQ.aviToLyg.AVItoLYGNodeInterface;

import OSI.OSU.PSU.OSU.MSQ.VQ.SQ.movieTransfer.MovieTransferNodeInterface;

import
OSI.OSU.SI.OVI.OSI.AVL.AEL.ACL.ASI.OVI.OEI.OCI.OSI.PVI.PEI.PCI.PSI.addPGSearchPage.AddPGSearchPageNodeASQ_OCQ_OSI_PCI_PCU_
MCI_MCU_MSI;

import OSI.OVU.OSU.MSQ.OSU.AVU.OSQ.butterworthFilter.ButterworthFilterNodeInterface;

import O_V.OSI.OSU.AVU.OSQ.wavePlay.WavePlayNodeInterface;

import S_A.pheromone.IMV_SQI;

import S_I.OSI.OSU.VSQ.OSU.MSQ.VQ.SQ.lygPlayer.LYGPlayerNodeInterface;

import S_I.OSI.OSU.VSQ.OSU.MSQ.VQ.SQ.lygRead.LYGReadNodeInterface;

import S_I.OSI.OVI.OSU.MSQ.MV.SQ.imageRead.ImageReadNodeInterface;

import S_I.OSI.OVQ.OSU.MSQ.MV.SQ.findColorB.FindColorBInterface;

import S_I.OSI.OVQ.OSU.MSQ.MV.SQ.findColorG.FindColorGInterface;

import S_I.OSI.OVQ.OSU.MSQ.MV.SQ.findColorR.FindColorRInterface;

import S_I.OSI.OVQ.OSU.MSQ.MV.SQ.show3D.Show3DInterface;

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.embossFilter.EmbossFilterInterface;

```

```

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.grayFilter.GrayFilterNodeInterface;

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.guassainFilter.GuassainFilterInterface;

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.houghTransform.HoughTransformInterface;

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.imageStrech.ImageStrechNodeInterface;

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.laplacianFilter.LaplacianFilterInterface;

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.medianImageFilter.MedianImageNodeInterface;

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.morphologyFilter.MorphologyFilterInterface;

import S_I.OSI.OVU.OSU.MSQ.MV.SQ.sobelFilter.SobelFilterNodeInterface;

import S_I.OSI.PEL.PCI.PSI.tinShell.AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI;

import javax.swing.JOptionPane;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register {

    JOptionPane text;

    Object[][] tableData_old;

    public App_S u;

    public A _A;

    public IMV_SQI pos;

    public OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_register(

        Object[][] tableData_old, JOptionPane text, App NE) {

        this.text = text;

        this.tableData_old = tableData_old;

        this.u = NE.app_S;

        this._A = this.u._A;

        this.pos = this.u.fMHMMListOneTime_E_X_S.posCnToCn;

        S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-01");

    }

    @SuppressWarnings({ "unchecked", "unused" })

    public NodeOSGI Rigester(NodeOSGI first, LinkOSGI link, App NE) {

        S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-02");

        // 注册

        // XlsReaderNodeInterface xlsReaderNodeInterface= new

        // XlsReaderNodeInterface();

        S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-02-000");

        // OSU_MCI_MCU_MSI XlsReadernode = (OSU_MCI_MCU_MSI)

        // xlsReaderNodeInterface;

        S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-02-01-01-01");

        // first = link.addNode(first, XlsReadernode);

        S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-02-01");

        // OSU_MCI_MCU_MSI imageReadNode = new

```



```
// ImageReadNodeInterface();
// first = link.addNode(first, imageReadNode);
// OSU_MCI_MCU_MSI imageStrechNode = new
// ImageStrechNodeInterface();
// first = link.addNode(first, imageStrechNode);
// OSU_MCI_MCU_MSI MedianImageNode = new
// MedianImageNodeInterface();
// first = link.addNode(first, MedianImageNode);
// OSU_MCI_MCU_MSI GrayFilterNode = new
// GrayFilterNodeInterface();
// first = link.addNode(first, GrayFilterNode);
// OSU_MCI_MCU_MSI GuassianFilterNode = new
// GuassianFilterInterface();
// first = link.addNode(first, GuassianFilterNode);
S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-03");
// OSU_MCI_MCU_MSI FindColorRNode = new
// FindColorRInterface();
// first = link.addNode(first, FindColorRNode);
// OSU_MCI_MCU_MSI FindColorGNode = new
// FindColorGInterface();
// first = link.addNode(first, FindColorGNode);
// OSU_MCI_MCU_MSI FindColorBNode = new
// FindColorBInterface();
// first = link.addNode(first, FindColorBNode);
// OSU_MCI_MCU_MSI SobelFilterNode= new
// SobelFilterNodeInterface();
// first = link.addNode(first, SobelFilterNode);
// OSU_MCI_MCU_MSI EmbossFilterNode
// = new EmbossFilterInterface();
// first = link.addNode(first, EmbossFilterNode);
S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-04");
// OSU_MCI_MCU_MSI LaplacianFilterNode
// = new LaplacianFilterInterface();
// first = link.addNode(first, LaplacianFilterNode);

// OSU_MCI_MCU_MSI HoughTransformNode = new HoughTransformInterface();
// first = link.addNode(first, HoughTransformNode);
//
// OSU_MCI_MCU_MSI WavReadNode = new WavReadNodeInterface();
// first = link.addNode(first, WavReadNode);
//
// OSU_MCI_MCU_MSI MedianFilterNode = new MedianFilterNodeInterface();
// first = link.addNode(first, MedianFilterNode);
//
// OSU_MCI_MCU_MSI ButterworthFilterNode = new ButterworthFilterNodeInterface();
// first = link.addNode(first, ButterworthFilterNode);
//
// OSU_MCI_MCU_MSI LaplacianWaveFilterNode = new LaplacianFilterNodeInterface();
```

```
// first = link.addNode(first, LaplacianWaveFilterNode);
// S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-05");
// OSU_MCI_MCU_MSI HoughWavFilterNode = new HoughWavFilterNodeInterface();
// first = link.addNode(first, HoughWavFilterNode);
//
// OSU_MCI_MCU_MSI GuassianWav2DFilterNode = new GuassianWav2DFilterNodeInterface();
// first = link.addNode(first, GuassianWav2DFilterNode);
//
// OSU_MCI_MCU_MSI MaxMiniFilterNode = new MaxMiniFilterNodeInterface();
// first = link.addNode(first, MaxMiniFilterNode);
//
// OSU_MCI_MCU_MSI wavePlayNode = new WavePlayNodeInterface();
// first = link.addNode(first, wavePlayNode);
//
// OSU_MCI_MCU_MSI Show3DNode = new Show3DInterface();
// first = link.addNode(first, Show3DNode);
//
// OSU_MCI_MCU_MSI MorphologyFilter = new MorphologyFilterInterface();
// first = link.addNode(first, MorphologyFilter);
// S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-06");
// OSU_MCI_MCU_MSI LYGReadNode = new LYGReadNodeInterface();
// first = link.addNode(first, LYGReadNode);
//
// OSU_MCI_MCU_MSI LYGWriteNode = new LYGWriteNodeInterface();
// first = link.addNode(first, LYGWriteNode);
//
// OSU_MCI_MCU_MSI MovieTransferNode = new MovieTransferNodeInterface();
// first = link.addNode(first, MovieTransferNode);
//
// OSU_MCI_MCU_MSI AVItoImagesNode = new AVItoLYGNodeInterface();
// first = link.addNode(first, AVItoImagesNode);
//
// OSU_MCI_MCU_MSI LYGPlayerNode = new LYGPlayerNodeInterface();
// first = link.addNode(first, LYGPlayerNode);
//
// OSU_MCI_MCU_MSI FFTFilterNode = new FFTFilterNodeInterface();
// first = link.addNode(first, FFTFilterNode);
//
// OSU_MCI_MCU_MSI freqCountNode = new FreqCountNodeInterface();
// first = link.addNode(first, freqCountNode);
// S_logger.Log.logger.info("" + "启动400编码调试开始-00000055-001-02-04-01-01-07");
// OSU_MCI_MCU_MSI lygFilterNode = new LygFilterNodeInterface();
// first = link.addNode(first, lygFilterNode);
//
// OSU_MCI_MCU_MSI lygFilterComp = new Ft2DFilterInterface();
// first = link.addNode(first, lygFilterComp);
//
```

```
// OSU_MCI_MCU_MSI lygSlave = new LygSlaveFilterInterface();
// first = link.addNode(first, lygSlave);
//
// OSU_MCI_MCU_MSI logFFT = new LogFFTInterface();
// first = link.addNode(first, logFFT);
// S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-08");
// OSU_MCI_MCU_MSI logFFTcount = new LogFFTcountInterface();
// first = link.addNode(first, logFFTcount);
//OSU_MCI_MCU_MSI dNA3DShow
// = new dNA3DShowNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI();
//first= link.addNode(first, dNA3DShow);
// 医学图片页添加
OSU_MCI_MCU_MSI searchPG = new AddPGSearchPageNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI();
if (null == searchPG) {
    //later do fatal changes
    S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-04-err");
}
searchPG.pageName = "图片搜索";
// first= link.addNode(first, searchPG);
S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-04");
OSU_MCI_MCU_MSI _OSU_MCI_MCU_MSI = searchPG;
S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-05");
u.gUISample.nodeReflection
.put(_OSU_MCI_MCU_MSI._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS, "");
_OSU_MCI_MCU_MSI.register(u.gUISample.tableData_old, u.gUISample.text,
NE);
_OSU_MCI_MCU_MSI.IMP_PSU(NE);
//u.gUISample.nodeView.first
// =
// u.gUISample.nodeView.link.addNode(u.gUISample.nodeView.first
// , OSU_MCI_MCU_MSI);
first = link.addNode(first, _OSU_MCI_MCU_MSI);
u.searchList.add(searchPG);
S_logger.Log.logger.info("'" + "启动400编码调试开始-00000055-001-02-04-01-01-06");
OSU_MCI_MCU_MSI tinShell = new AddTinShellNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI();
tinShell.pageName = "Tin语言";
// first= link.addNode(first, searchPG);
_OSU_MCI_MCU_MSI = (OSU_MCI_MCU_MSI) tinShell;
u.gUISample.nodeReflection
.put(_OSU_MCI_MCU_MSI._SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS, "");
_OSU_MCI_MCU_MSI.register(u.gUISample.tableData_old, u.gUISample.text,
NE);
_OSU_MCI_MCU_MSI.IMP_PSU(NE);
//u.gUISample.nodeView.first
// = u.gUISample.nodeView.link.addNode(u.gUISample.nodeView.first
// , OSU_MCI_MCU_MSI);
first = link.addNode(first, _OSU_MCI_MCU_MSI);
```

```
        u.searchList.add(tinShell);

        return first;

    }

}

//ddPGSearchPageNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI
//////////

//OSU_MCI_MCU_MSI editPanelReader
//= new EditPanelReaderNodeInterface(this.text);
//first= link.addNode(first, editPanelReader);
//
//OSU_MCI_MCU_MSI editPanelReaderH
//= new EditPanelReaderHNodeInterface(this.text);
//first= link.addNode(first, editPanelReaderH);
//
//OSU_MCI_MCU_MSI addChuFangAttributeH
//= new AddChuFangAttributeHNodeInterface(this.tableData_old
//, this.text);
//first= link.addNode(first, addChuFangAttributeH);
//
//OSU_MCI_MCU_MSI filterChuFangJinJiAttributeH=
//new filterChuFangJinJiAttributeHNodeInterface(this.tableData_old, this.text);
//first= link.addNode(first, filterChuFangJinJiAttributeH);
//
//OSU_MCI_MCU_MSI chuFangWuXingShowHInterface=
//new ChuFangWuXingShowHNodeInterface(this.tableData_old, this.text);
//first= link.addNode(first, chuFangWuXingShowHInterface);
//////扫描jar、 、添加jar
//OSU_MCI_MCU_MSI filterChuFangXingWeiKeywordsAttributeH=
//new filterChuFangXingWeiKeywordsAttributeHNodeInterface(this.tableData_old, this.text);
//first= link.addNode(first, filterChuFangXingWeiKeywordsAttributeH);
//
//OSU_MCI_MCU_MSI filterChuFangJinJiKeywordsAttributeH=
//new filterChuFangJinJiKeywordsAttributeHNodeInterface(this.tableData_old, this.text);
//first= link.addNode(first, filterChuFangJinJiKeywordsAttributeH);
//
//OSU_MCI_MCU_MSI updateToEditPane=
//new updateToEditPaneNodeInterface(this.tableData_old, this.text);
//first= link.addNode(first, updateToEditPane);
//
//OSU_MCI_MCU_MSI readNodeInterface=
//new ReadNodeInterface(this.tableData_old, this.text);
//first= link.addNode(first, readNodeInterface);
//OSU_MCI_MCU_MSI DNN3DInterface=
//new ChuFangDNN3DShowNodeInterface(this.tableData_old, this.text, this.u, this._A, this.pos);
//first= link.addNode(first, DNN3DInterface);
//try {
//////1扫描
```

```
////1.1设计一个文件夹
////扫描文件夹下面资源包录入
//String jarCategoryLink= "";
//FileDialog filedialog= new FileDialog(new Frame(), S_Data.ATTENSION_LOAD_HISTORY
//, FileDialog.LOAD);
//filedialog.setFilenameFilter(new TXTFilter(S_Data.FILE_FORMAT_ETL));
//filedialog.setVisible(true);
//jarCategoryLink= filedialog.getDirectory();
////////20230106-S_logger.Log.logger.info("" + jarCategoryLink);
//if(null== jarCategoryLink || jarCategoryLink.isEmpty() || jarCategoryLink.contains
//(S_Data.FILE_FORMAT_JAR)) {
////20230106-S_logger.Log.logger.info("" + S_Data.ATTENSION_RECHOICE);
//return first;
//}
//File file= new File(jarCategoryLink);
//if(file.isFile()) {
////20230106-S_logger.Log.logger.info("" + S_Data.ATTENSION_FILE_CHOICE);
//return first;
//}
//File[] files= file.listFiles();
//for(int i= 0; i< files.length; i++) {
//@SuppressWarnings({ "deprecation", "resource" })
//URLClassLoader loader= new URLClassLoader(new URL[] { files[i].toURL() });
//String filename= files[i].getName().replace(S_Data.FILE_FORMAT_JAR, S_Data.STRING_EMPTY);
//String[] columns= filename.split("\\.");
////如下注释2行代码 refer https://www.cnblogs.com/chinaxin/p/3678442.html 这小子以后有前途。哈哈
////Class<?> myclass= loader.loadClass("hand.java.loadjar.TestClass");
////Gene new object
////Object myobject= myclass.newInstance();
//Class<?> myclass= null;
//try {
//myclass= loader.loadClass(filename+ "." + columns[columns.length- 1]
//+ S_Data.NODE_NODE_INTERFACE);
//} catch (ClassNotFoundException e) {
//
//e.printStackTrace();
//}
//Object myobject= null;
//try {
//myobject= myclass.newInstance();
//} catch (InstantiationException | IllegalAccessException e) {
//
//e.printStackTrace();
//}
//OSU_MCI_MCU_MSI OSU_MCI_MCU_MSI
//= (OSU_MCI_MCU_MSI) myobject;
//first= link.addNode(first, OSU_MCI_MCU_MSI);
```

```

//}

//}catch(Exception e) {

//

//}

//208 //add extp/////

/////声诊断, 该接口已经2年停止开源研发。

///OSU_MCI_MCU_MSI soundCheck

//    // = new AddZNSZPageNodeASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI();

///soundCheck.pageName= "智能声诊";

/////first= link.addNode(first, searchPG);

///OSU_MCI_MCU_MSI OSU_MCI_MCU_MSI

///= (OSU_MCI_MCU_MSI) soundCheck;

///u.gUISample.nodeReflection.put(

//    //OSU_MCI_MCU_MSI_SQ_OSU_MSQ_OSU_AVQ_ASQ_SQ_VPC_PCS, null);

///OSU_MCI_MCU_MSI.register(

//    //u.gUISample.tableData_old, u.gUISample.text

///

//    , u.gUISample.u, u.gUISample._A, u.gUISample.pos);

///try {

///    OSU_MCI_MCU_MSI.IP;

///} catch (IOException e1) {

///    e1.printStackTrace();

///}

/////u.gUISample.nodeView.first

//    // = u.gUISample.nodeView.link.addNode(u.gUISample.nodeView.first

/////

//    , OSU_MCI_MCU_MSI);

///first= link.addNode(first, OSU_MCI_MCU_MSI);

///u.searchList.add(soundCheck);

//86

//OSU_MCI_MCU_MSI arffTransferNode

// = new arffTransferNodeInterface();

//first= link.addNode(first, arffTransferNode);

//OSU_MCI_MCU_MSI WekaPilot2DNode

// = new WekaPilot2DNodeInterface();

//first= link.addNode(first, WekaPilot2DNode);

//Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/POE/extOSGI/OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_registerTest.java

//Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/POE/extOSGI/OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_OSI_PCI.pde

//Trif_DNA-master/C_L_E/OPE/SI/MCI/OEI/OVU/POE/extOSGI/OSI_OSU_ASQ_OCQ_OSI_PCI_PCU_MCI_MCU_MSI_OSI_PCI.tvm

resources

/Trif_DNA-master/C_L_E/resources/ipBlock.ips

1-5-/Trif_DNA-master/C_L_I-增加-三维词汇花-卷积-VPCS-网络-编辑

DVIAQVIMIV

/Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering_17.java

package DVIAQVIMIV;

import java.io.File;

import java.io.FileWriter;

import java.io.IOException;

import ME.VPC.M.app.App;

import test.java.interfaces.test.CommonTestInitition;

```

```
//import PEU.P.image.ReadWritePng;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class DoDeciphering_17 {
    public String outputFilePath;
    public void setOutputFilePath(String outputPath) {
        outputFilePath = outputPath;
    }
    public void imageDeciphering(String indexFileCategoryPath, App NE) {
        //1 输入文件夹字符串
        File file = new File(indexFileCategoryPath);
        doInnerFiles(file, NE);
        //2 遍历文件夹下所有图片
        //3 每个图片进行训练取值
        //4 训练值进行记录
    }
    public void doInnerFiles(File file, App NE) {
        try {
            if (file.isDirectory()) {
                File[] files = file.listFiles();
                for (File currentFile : files) {
                    doInnerFiles(currentFile, NE);
                }
            } else {
                if (file.isFile()) {
                    if (null != file.getPath()) {
                        String filePath = file.getPath();
                        if (filePath.contains(".jpg") || filePath
                            .contains(".jpeg") || filePath.contains(
                                ".png")) {
                            //if(filePath.contains(".png")) {
                            //3 每个图片进行训练取值
                            int[][] getSkinBy = new GetSkinBy()
                                .getSkinBy(filePath);
                            int[][] doPDEofSkin = new DoPDEofSkin()
                                .doPDEofSkin_AOPM_VECS_IDUQ_TXH(
                                    getSkinBy, 95, NE);
                            //new ReadWritePng().writePNG("C:/Users/yaogu
                            ///Desktop/project/开源门户/deciphering/F_DB.jpg", doPDEofSkin);
                            double[] getStatisticCount = new GetStatisticCount()
```

```

        .getStatisticCount(doPDEofSkin, 10, 2,
            8);

        double[] getStatisticRatio = new GetStatisticRatio()
            .getStatisticRatio(getStatisticCount);

        //4 训练值进行记录
        File fileOut = new File(outputFilePath);
        //我不建议用数据库, 建议用txt 行操作即可.
        //以后训练数据比较完善准确了, 再用德塔数据库.
        FileWriter fileWriter = null;
        fileWriter = new FileWriter(fileOut,
            true);

        fileWriter.write("f>" + file.getName());
        for (int i = 0; i < getStatisticRatio.length; i++) {
            fileWriter.write(">d>"
                + getStatisticRatio[i] * 10000);
        }
        fileWriter.write("\r\n");
        fileWriter.close();
    }
}

} catch (IOException e) {
    throw new RuntimeException(e);
}
}

public static void main(String[] args) {
    CommonTestInition commonTestInition = new CommonTestInition();
    commonTestInition.initEnvironment("去弹窗组件流测试");
    DoDeciphering_17 doDeciphering = new DoDeciphering_17();
    doDeciphering.setOutputFilePath(
        "C:/Users/yaogu/Desktop/project"
        + "/开源门户/deciphering/F_DB6_17.txt");
    doDeciphering.imageDeciphering("D:/bu20210606"
        + "/搜索图片/搜索图片/pgSearch", commonTestInition.NE);
    commonTestInition.endEnvironment();
}
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering_17Test.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering_AC_17.java
package DVIAQVIMIV;

import java.io.File;

import java.io.FileWriter;

import java.io.IOException;

import ME.VPC.M.app.App;

import test.java.interfaces.test.CommonTestInition;

//import PEU.P.image.ReadWritePng;

/*

```


* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class DoDeciphering_AC_17 {
    public String outputFilePath;
    public void setOutputFilePath(String outputPath) {
        outputFilePath = outputPath;
    }
    public void imageDeciphering(String indexFileCategoryPath,
        App NE) {
        //1 输入文件夹字符串
        File file = new File(indexFileCategoryPath);
        doInerFiles(file, NE);
        //2 遍历文件夹下所有图片
        //3 每个图片进行训练取值
        //4 训练值进行记录
    }
    public void doInerFiles(File file, App NE) {
        try {
            if (file.isDirectory()) {
                File[] files = file.listFiles();
                for (File currentFile : files) {
                    doInerFiles(currentFile, NE);
                }
            } else {
                if (file.isFile()) {
                    if (null != file.getPath()) {
                        String filePath = file.getPath();
                        if (filePath.contains(".jpg") || filePath
                            .contains(".jpeg") || filePath.contains(
                                ".png")) {
                            //if(filePath.contains("阿洪病")) {
                            //3 每个图片进行训练取值
                            int[][] getSkinBy = new GetSkinBy()
                                .getSkinBy(filePath);
                            int[][] doPDEofSkin = new DoPDEofSkin()
                                .doPDEofSkin_AOPM_VECS_IDUQ_TXH_AC(
                                    getSkinBy, 95, NE);
                            // new ReadWritePng().writePNG("C:/Users/yaogu/Desktop
                                ///project/开源门户/deciphering/F_DB_20210820.jpg", doPDEofSkin);
                            double[] getStatisticCount = new GetStatisticCount()
                                .getStatisticCount(doPDEofSkin, 10, 5,
```

```

8);

        double[] getStatisticRatio = new GetStatisticRatio()
        .getStatisticRatio(getStatisticCount);

        //4 训练值进行记录
        File fileOut = new File(outputFilePath);
        //我不建议用数据库, 建议用txt 行操作即可.
        //以后训练数据比较完善准确了, 再用德塔数据库.
        FileWriter fileWriter = null;
        fileWriter = new FileWriter(fileOut,

true);

        fileWriter.write("f>" + file.getName());
        for (int i = 0; i < getStatisticRatio.length; i++) {
fileWriter.write(">d>"
        + getStatisticRatio[i] * 10000);
        }
        fileWriter.write("\r\n");
        fileWriter.close();
    } //}
}

    }
} catch (IOException e) {
    throw new RuntimeException(e);
}
}

public static void main(String[] args) {
    CommonTestInition commonTestInition = new CommonTestInition();
    commonTestInition.initEnvironment("去弹窗组件流测试");
    DoDeciphering_AC_17 doDeciphering = new DoDeciphering_AC_17();
    doDeciphering.setOutputFilePath("C:/Users/yaogu"
        + "/Desktop/project/开源门户/deciphering/F_DB6_suan_17.txt");
    doDeciphering.imageDeciphering("D:/bu20210606"
        + "/搜索图片/搜索图片/pgSearch", commonTestInition.NE);
    commonTestInition.endEnvironment();
}
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering_AC_17Test.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDeciphering.java
package DVIAQVIMIV;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
//import PEU.P.image.ReadWritePng;
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class DoDeciphering {
    public String outputFilePath;
    public void setOutputFilePath(String outputPath) {
        outputFilePath = outputPath;
    }
    public void imageDeciphering(String indexFileCategoryPath) {
        //1 输入文件夹字符串
        File file = new File(indexFileCategoryPath);
        doInerFiles(file);
        //2 遍历文件夹下所有图片
        //3 每个图片进行训练取值
        //4 训练值进行记录
    }
    public void doInerFiles(File file) {
        try {
            if (file.isDirectory()) {
                File[] files = file.listFiles();
                for (File currentFile : files) {
                    doInerFiles(currentFile);
                }
            } else {
                if (file.isFile()) {
                    if (null != file.getPath()) {
                        String filePath = file.getPath();
                        if (filePath.contains(".jpg") || filePath.contains(".jpeg")
                            || filePath.contains(".png")) {
                            //if(filePath.contains(".png")) {
                            //3 每个图片进行训练取值
                            int[][] getSkinBy = new GetSkinBy().getSkinBy(filePath);
                            int[][] doPDEofSkin = new DoPDEofSkin().doPDEofSkin_IDUQ(getSkinBy, 95);
                            //new ReadWritePng().writePNG("C:/Users/yaogu/Desktop
                            ///project/开源门户/deciphering/F_DB.jpg", doPDEofSkin);
                            double[] getStatisticCount
                                = new GetStatisticCount().getStatisticCount(doPDEofSkin, 10, 8, 8);
                            double[] getStatisticRatio
                                = new GetStatisticRatio().getStatisticRatio(getStatisticCount);
                            //4 训练值进行记录
                            File fileOut = new File(outputFilePath);
                            //我不建议用数据库, 建议用txt 行操作即可.
                            //以后训练数据比较完善准确了, 再用德塔数据库.
                            FileWriter fileWriter = null;
                            fileWriter = new FileWriter(fileOut, true);
                            fileWriter.write("f>" + file.getName());
                        }
                    }
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

        for (int i = 0; i < getStatisticRatio.length; i++) {
            fileWriter.write(">d>" + getStatisticRatio[i] * 10000);
        }
        fileWriter.write("\r\n");
        fileWriter.close();
    }
}

} catch (IOException e) {
    throw new RuntimeException(e);
}
}

public static void main(String[] args) {
    DoDeciphering doDeciphering = new DoDeciphering();
    doDeciphering.setOutputFilePath("C:/Users/yaogu/Desktop"
        + "/project/开源门户/deciphering/F_DB6.txt");
    doDeciphering.imageDeciphering("D:/bu20210606/搜索图片"
        + "/搜索图片/pgSearch");
}
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDecipheringTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDesiphline_17.java
package DVIAQVIMIV;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
//import PEU.P.image.ReadWritePng;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class DoDesiphline_17 {
    public String outputFilePath;

    public void setOutputFilePath(String outputPath) {
        outputFilePath = outputPath;
    }

    public void imageDesiphline(String indexFileCategoryPath) {
        //1 输入文件夹字符串
        File file = new File(indexFileCategoryPath);
        doInerFiles(file);
        //2 遍历文件夹下所有图片
        //3 每个图片进行训练取值
    }
}

```

```
//4 训练值进行记录
}

public void doInerFiles(File file) {
    try {
        if (file.isDirectory()) {
            File[] files = file.listFiles();
            for (File currentFile : files) {
                doInerFiles(currentFile);
            }
        } else {
            if (file.isFile() && null != file.getPath()) {
                String filePath = file.getPath();
                if (filePath.contains(".jpg") || filePath.contains(".jpeg")
                    || filePath.contains(".png")) {
                    //if(filePath.contains(".png")) {
                    //3 每个图片进行训练取值
                    int[][] getSkinBy = new GetSkinBy().getSkinBy(filePath);
                    int[][] doPDEofSkin = new DoPDEofSkin().doPDEofSkin_IDUQ(getSkinBy, 5);
                    //new      ReadWritePng().writePNG("C:/Users/yaogu
                    ///Desktop/project/开源门户/deciphering/F_DB.jpg", doPDEofSkin);
                    double[] getStatisticCount = new GetStatisticCount()
                        .getStatisticCount(doPDEofSkin, 10, 5, 8);
                    double[] getStatisticRatio = new GetStatisticRatio()
                        .getStatisticRatio(getStatisticCount);
                    //4 训练值进行记录
                    File fileOut = new File(outputFilePath);
                    //我不建议用数据库, 建议用txt 行操作即可.
                    //以后训练数据比较完善准确了, 再用德塔数据库.
                    FileWriter fileWriter = null;
                    fileWriter = new FileWriter(fileOut, true);
                    fileWriter.write("f>" + file.getName());
                    for (int i = 0; i < getStatisticRatio.length; i++) {
                        fileWriter.write(">d>" + getStatisticRatio[i] * 10000);
                    }
                    fileWriter.write("\r\n");
                    fileWriter.close();
                }
            }
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

public static void main(String[] args) {
    DoDesiphline_17 doDeciphering = new DoDesiphline_17();
    doDeciphering.setOutputFilePath("C:/Users/yaogu"
        + "/Desktop/project/开源门户/deciphering/F_DB6_17_.txt");
}
```

```

doDeciphering.imageDesiphline("D:/bu20210606"
    + "/搜索图片/搜索图片/pgSearch");
}
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDesiphline_17Test.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDesiphline.java
package DVIAQVIMIV;

import java.io.File;

import java.io.FileWriter;

import java.io.IOException;

//import PEU.P.image.ReadWritePng;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class DoDesiphline {

    public String outputFilePath;

    public void setOutputFilePath(String outputPath) {
        outputFilePath = outputPath;
    }

    public void imageDesiphline(String indexFileCategoryPath) {
        //1 输入文件夹字符串
        File file = new File(indexFileCategoryPath);
        doInerFiles(file);
        //2 遍历文件夹下所有图片
        //3 每个图片进行训练取值
        //4 训练值进行记录
    }

    public void doInerFiles(File file) {
        try {
            if (file.isDirectory()) {
                File[] files = file.listFiles();
                for (File currentFile : files) {
                    doInerFiles(currentFile);
                }
            } else {
                if (file.isFile()) {
                    if (null != file.getPath()) {
                        String filePath = file.getPath();
                        if (filePath.contains(".jpg") || filePath.contains(".jpeg")
                            || filePath.contains(".png")) {
                            //if(filePath.contains(".png")) {
                            //3 每个图片进行训练取值

```

```

int[][] getSkinBy = new GetSkinBy().getSkinBy(filePath);
int[][] doPDEofSkin = new DoPDEofSkin().doPDEofSkin_IDUQ(getSkinBy, 5);
//new    ReadWritePng().writePNG("C:/Users/yaogu
///Desktop/project/开源门户/deciphering/F_DB.jpg", doPDEofSkin);
double[] getStatisticCount = new GetStatisticCount()
    .getStatisticCount(doPDEofSkin, 10, 5, 8);
double[] getStatisticRatio = new GetStatisticRatio()
    .getStatisticRatio(getStatisticCount);
//4 训练值进行记录
File fileOut = new File(outputFilePath);
//我不建议用数据库, 建议用txt 行操作即可.
//以后训练数据比较完善准确了, 再用德塔数据库.
FileWriter fileWriter = null;
fileWriter = new FileWriter(fileOut, true);
fileWriter.write("f>" + file.getName());
for (int i = 0; i < getStatisticRatio.length; i++) {
    fileWriter.write(">d>" + getStatisticRatio[i] * 10000);
}
fileWriter.write("\r\n");
fileWriter.close();
}
}
}
} catch (IOException e) {
    throw new RuntimeException(e);
}
}

public static void main(String[] args) {
    DoDesiphline doDeciphering = new DoDesiphline();
    doDeciphering.setOutputFilePath("C:/Users/yaogu"
        + "/Desktop/project/开源门户/deciphering/F_DB6.txt");
    doDeciphering.imageDesiphline("D:/bu20210606"
        + "/搜索图片/搜索图片/pgSearch");
}
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoDesiphlineTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoPDEofSkin.java
package DVIAQVIMIV;

import A_I.MPE.MAQPVDCOISIV.pdi.RangePDI;
import ME.VPC.M.app.App;

//准备用肽展公式来设计个 皮肤病检测程序.
//罗瑶光 20210710
//软件思想 肽展erosion变换. 计算机视觉.
//硬件工具, imageIO, javaCV. 像素头
//因为肽展公式和思想GPL2.0开源, 本程序源码同样GPL2.0开源.
/*
* 个人著作权人, 作者 罗瑶光, 浏阳

```

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class DoPDEofSkin {
    public void doPDEofSkin() {
    }
    public void doPDSofSkin() {
    }
    public void doPDNofSkin() {
    }
    public void doTVMofSkin() {
    }
    public void doDNAofSkin() {
    }
    public void doRNAofSkin() {
    }
    public void doErosionofSkin() {
    }
    public int[][] doPDEofSkin_IDUQ(int[][] rgb, int ratio) {
        int[][] output = new int[3][];
        //int ratio= 50;
        int[] rp = RangePDI.IPE(rgb[0], ratio);
        int[] gp = RangePDI.IPE(rgb[1], ratio);
        int[] bp = RangePDI.IPE(rgb[2], ratio);
        output[0] = rp;
        output[1] = gp;
        output[2] = bp;
        return output;
    }
    public int[][] doPDEofSkin_AOPM_VECS_IDUQ_TXH(int[][] rgb, int ratio, App NE) {
        int[][] output = new int[3][];
        //int ratio= 50;
        int[] rp = RangePDI.IPE_AOPM_VECS_IDUQ_TXH(rgb[0], ratio, NE);
        int[] gp = RangePDI.IPE_AOPM_VECS_IDUQ_TXH(rgb[1], ratio, NE);
        int[] bp = RangePDI.IPE_AOPM_VECS_IDUQ_TXH(rgb[2], ratio, NE);
        output[0] = rp;
        output[1] = gp;
        output[2] = bp;
        return output;
    }
    public int[][] doPDEofSkin_AOPM_VECS_IDUQ_TXH_AC(int[][] rgb, int ratio, App NE) {
        int[][] output = new int[3][];
        //int ratio= 50;
```



```

    int[][] rp = RangePDI.IPE_AOPM_VECS_IDUQ_TXH_AC(rgb[0], ratio, NE);
    int[][] gp = RangePDI.IPE_AOPM_VECS_IDUQ_TXH_AC(rgb[1], ratio, NE);
    int[][] bp = RangePDI.IPE_AOPM_VECS_IDUQ_TXH_AC(rgb[2], ratio, NE);

    output[0] = rp;
    output[1] = gp;
    output[2] = bp;

    return output;
}
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/DoPDEofSkinTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetColorRatioScore.java
package DVIAQVIMIV;

import java.util.Iterator;

import java.util.List;

//准备用肽展公式来设计个 皮肤病检测程序.
//罗瑶光 20210710
//软件思想 肽展erosion变换. 计算机视觉.
//硬件工具, imageIO, javaCV.
//因为肽展公式和思想GPL2.0开源, 本程序源码同样GPL2.0开源.
/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class GetColorRatioScore {
    //RGBY
    public void getRatioScore() {
    }
    public void getRGRatioScore() {
    }
    public void getRBRatioScore() {
    }
    public void getGBRatioScore() {
    }
    public void getGYRatioScore() {
    }
    public void getBYRatioScore() {
    }
    public void getRYRatioScore() {
    }

    static void scoreDeiphring(double[] score, String[] nameScore, double[] getStatisticRatio
        , List<double[]> fileCells, List<String> fileNames) {
        int lineCount = 0;
        Iterator<double[]> iteratorCells = fileCells.iterator();

```

```

Iterator<String> iteratorNames = fileNames.iterator();
while (iteratorNames.hasNext()) {
    String name = iteratorNames.next();
    nameScore[lineCount++] = name.toString();
}
lineCount = 0;
while (iteratorCells.hasNext()) {
    double[] Doubles = iteratorCells.next();
    score[lineCount] = new Double(0);
    for (int i = 0; i < getStatisticRatio.length; i++) {
        try {
            if (0 == Doubles[i] || 0 == getStatisticRatio[i]) {
                score[lineCount] += 10000;
            } else {
                double ratio = (getStatisticRatio[i] * 10000) / Doubles[i];
                if (ratio > 2 || ratio < 0.5) {
                    score[lineCount] += 10000;
                } else {
                    score[lineCount] += ratio;
                }
            }
        } catch (Exception e) {
            //20230106-S_logger.Log.logger.info(" " + Doubles[i]);
            score[lineCount] += 0;
        }
    }
    score[lineCount] = Math.abs(score[lineCount] / 45 - 1); //nor 均值进行了优化 20210714
    lineCount++;
}
}

//Deiphring 分解为 Deciphring + Desiphine
static void scoreDeciphring(double[] score, String[] nameScore, double[] getStatisticRatio
, List<double[]> fileCells, List<String> fileNames, int pcaScale
, int upcaScale, int icaScale, int ecaScale) {
    int lineCount = 0;
    Iterator<double[]> iteratorCells = fileCells.iterator();
    Iterator<String> iteratorNames = fileNames.iterator();
    while (iteratorNames.hasNext()) {
        String name = iteratorNames.next();
        nameScore[lineCount++] = name.toString();
    }
    lineCount = 0;
    while (iteratorCells.hasNext()) {
        double[] Doubles = iteratorCells.next();
        score[lineCount] = new Double(0);
        int PCA = 0;
        int UPCA = 0;

```

```

int ICA = 0;
int ECA = 0;
for (int i = 0; i < getStatisticRatio.length; i++) {
    try {
        if (0 == Doubles[i] || 0 == getStatisticRatio[i]) {
            score[lineCount] += 10000;
            ICA++;
            UPCA++;
        } else {
            double ratio = (getStatisticRatio[i] * 10000) / Doubles[i];
            if (ratio > 2 || ratio < 0.5) {
                UPCA++;
                score[lineCount] += 10000;
            } else if (ratio < 1.5 || ratio > 0.75) {
                score[lineCount] += ratio;
                PCA++;
                if (i > 2) {
                    ICA++;
                }
                if (ratio < 1.1 || ratio > 0.9) {
                    ECA++;
                }
            }
        }
    } catch (Exception e) {
        //20230106-S_logger.Log.logger.info(" " + Doubles[i]);
        score[lineCount] += 0;
    }
}
if (PCA > pcaScale && UPCA < upcaScale && ICA > icaScale && ECA > ecaScale) {
    score[lineCount] = Math.abs(score[lineCount] / 45 - 1); //nor 均值进行了优化 20210714
} else {
    score[lineCount] = 99999.0; //nor 均值进行了优化 20210714
}
lineCount++;
}
}

// static void scoreDesiphine(double[] score, String[] nameScore, double[] getStatisticRatio
// , List<double[] > fileCells, List<String> fileNames) {
//int lineCount= 0;
//int ECA= 0;
//Iterator<double[] > iteratorCells= fileCells.iterator();
//Iterator<String> iteratorNames= fileNames.iterator();
//while(iteratorNames.hasNext()) {
//    String name= iteratorNames.next();
//    nameScore[lineCount++]= name.toString();
//}

```

```

//lineCount= 0;

//while(iteratorCells.hasNext()) {
//    double[] Doubles= iteratorCells.next();
//    score[lineCount]= new Double(0);
//    for(int i= 0; i< getStatisticRatio.length; i++) {
//        try {
//            if(0== Doubles[i] || 0== getStatisticRatio[i]) {
//                score[lineCount]+= 10000;
//            }else {
//                double ratio= (getStatisticRatio[i]* 10000)/ Doubles[i];
//                if(ratio>2 || ratio<0.5) {
//                    score[lineCount]+= 10000;
//                }else {
//                    score[lineCount]+= ratio;
//                }
//            }
//        } catch(Exception e) {
//            //20230106-S_logger.Log.logger.info("" + Doubles[i]);
//            score[lineCount]+= 0;
//        }
//    }
//    score[lineCount]= Math.abs(score[lineCount]/ 45 - 1);//nor 均值进行了优化 20210714
//    lineCount++;
//}
//    }
//}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetColorRatioScoreTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetICASStatisticRatio.java
package DVIAQVIMIV;

//准备用肽展公式来设计个 皮肤病检测程序.
//罗瑶光 20210710
//软件思想 肽展erosion变换. 计算机视觉.
//硬件工具, imageIO, javaCV.
//因为肽展公式和思想GPL2.0开源, 本程序源码同样GPL2.0开源.
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class GetICASStatisticRatio {
    int corr = 15;
    int scale = 7;
    public GetICASStatisticRatio(int scale, int corr) {

```

```
this.scale = scale;

this.corr = corr;
}

public double getPinkStatisticCount(int[][] rgb) {
    int ri = 255;
    int gi = 0;
    int bi = 255;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getWhiteStatisticCount(int[][] rgb) {
    int ri = 255;
    int gi = 255;
    int bi = 255;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getPupilStatisticCount(int[][] rgb) {
    int ri = 255;
    int gi = 0;
    int bi = 255;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getYellowStatisticCount(int[][] rgb) {
    int ri = 255;
    int gi = 255;
    int bi = 0;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getRedStatisticCount(int[][] rgb) {
    int ri = 255;
    int gi = 0;
    int bi = 0;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getGreenStatisticCount(int[][] rgb) {
    int ri = 0;
    int gi = 255;
    int bi = 0;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getBlueStatisticCount(int[][] rgb) {
    int ri = 0;
    int gi = 0;
    int bi = 255;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getMidPinkStatisticCount(int[][] rgb) {
    int ri = 128;
```

```
int gi = 0;
int bi = 128;
return getCountOfKernel(rgb, ri, gi, bi);
}

public double getMidWhiteStatisticCount(int[][] rgb) {
    int ri = 128;
    int gi = 128;
    int bi = 128;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getMidPupilStatisticCount(int[][] rgb) {
    int ri = 128;
    int gi = 0;
    int bi = 128;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getMidYellowStatisticCount(int[][] rgb) {
    int ri = 128;
    int gi = 128;
    int bi = 0;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getMidRedStatisticCount(int[][] rgb) {
    int ri = 128;
    int gi = 0;
    int bi = 0;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getMidGreenStatisticCount(int[][] rgb) {
    int ri = 0;
    int gi = 128;
    int bi = 0;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getMidBlueStatisticCount(int[][] rgb) {
    int ri = 0;
    int gi = 0;
    int bi = 128;
    return getCountOfKernel(rgb, ri, gi, bi);
}

public double getCountOfKernel(int[][] rgb, int ri, int gi, int bi) {
    double count = 0.0;
    for (int i = 0; i < rgb[0].length; i++) {
        for (int j = 0; j < rgb[0][0].length; j++) {
            if (rgb[0][i][j] * scale > ri - corr && rgb[0][i][j] * scale < ri + corr) {
                if (rgb[1][i][j] * scale > gi - corr && rgb[1][i][j] * scale < gi + corr) {
                    if (rgb[2][i][j] * scale > bi - corr && rgb[2][i][j] * scale < bi + corr) {
```

```

        count++;
    }
}
}
}
}
return count;
}
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetJCAStatisticRatioTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetRatioScoreSort.java
package DVIAQVIMIV;

//准备用肽展公式来设计个 皮肤病检测程序.
//罗瑶光 20210710
//软件思想 肽展erosion变换. 计算机视觉.
//硬件工具, imageIO, javaCV.
//因为肽展公式和思想GPL2.0开源, 本程序源码同样GPL2.0开源.
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class GetRatioScoreSort {
    public void getRatioScoreSort() {
    }
    public void getDoubleRatioScoreSort() {
    }
    public void getLongRatioScoreSort() {
    }
    public void getErrorRateScoreSort() {
    }
    public void getErrorRateExpand() {
    }
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetShapeRatioScore.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetSkinBy.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetSkinByTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetStatisticCount.java
package DVIAQVIMIV;

import P_V.PCI.ASQ.statistic.ShareholdCount;

//准备用肽展公式来设计个 皮肤病检测程序.
//罗瑶光 20210710
//软件思想 肽展erosion变换. 计算机视觉.
//硬件工具, imageIO, javaCV.
//因为肽展公式和思想GPL2.0开源, 本程序源码同样GPL2.0开源.
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class GetStatisticCount{
    public void getStatisticCount() {
    }
    public void getIncreasementStatisticCount() {
    }
    public void getPCAStatisticCount() {
    }
    public void getICAStatisticCount() {
//
    }
    public void getCorrelationStatisticCount() {
    }
    public void getBayansStatisticCount() {
    }
    public double[] getStatisticCount(int[][] rgbIPE, int sharehold, int scale, int corr) {
        double[] output= new double[17];
        //int sharehold= 127;
        double countR= ShareholdCount.getShareholdCount(rgbIPE[0], sharehold);
        double countB= ShareholdCount.getShareholdCount(rgbIPE[1], sharehold);
        double countG= ShareholdCount.getShareholdCount(rgbIPE[2], sharehold);

        double countYellow= new GetICAStatisticRatio(scale, corr).getYellowStatisticCount(rgbIPE);
        double countBlue= new GetICAStatisticRatio(scale, corr).getBlueStatisticCount(rgbIPE);
        double countRed= new GetICAStatisticRatio(scale, corr).getRedStatisticCount(rgbIPE);
        double countGreen= new GetICAStatisticRatio(scale, corr).getGreenStatisticCount(rgbIPE);
        double countPupil= new GetICAStatisticRatio(scale, corr).getPupilStatisticCount(rgbIPE);
        double countWhite= new GetICAStatisticRatio(scale, corr).getWhiteStatisticCount(rgbIPE);
        double countPink= new GetICAStatisticRatio(scale, corr).getPinkStatisticCount(rgbIPE);
        double countMidYellow= new GetICAStatisticRatio(scale, corr).getYellowStatisticCount(rgbIPE);
        double countMidBlue= new GetICAStatisticRatio(scale, corr).getBlueStatisticCount(rgbIPE);
        double countMidRed= new GetICAStatisticRatio(scale, corr).getRedStatisticCount(rgbIPE);
        double countMidGreen= new GetICAStatisticRatio(scale, corr).getGreenStatisticCount(rgbIPE);
        double countMidPupil= new GetICAStatisticRatio(scale, corr).getPupilStatisticCount(rgbIPE);
        double countMidWhite= new GetICAStatisticRatio(scale, corr).getWhiteStatisticCount(rgbIPE);
        double countMidPink= new GetICAStatisticRatio(scale, corr).getPinkStatisticCount(rgbIPE);
        output[0]= countR;
        output[1]= countB;
        output[2]= countG;
        output[3]= countYellow;
        output[4]= countBlue;
```



```

        output[5]= countRed;
        output[6]= countGreen;
        output[7]= countPupil;
        output[8]= countWhite;
        output[9]= countPink;
        output[10]= countMidYellow;
        output[11]= countMidBlue;
        output[12]= countMidRed;
        output[13]= countMidGreen;
        output[14]= countMidPupil;
        output[15]= countMidWhite;
        output[16]= countMidPink;
        return output;
    }
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetStatisticCountTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/GetStatisticRatio.java
package DVIAQVIMIV;

//准备用肽展公式来设计个 皮肤病检测程序.
//罗瑶光 20210710
//软件思想 肽展erosion变换. 计算机视觉.
//硬件工具, imageIO, javaCV.
//因为肽展公式和思想GPL2.0开源, 本程序源码同样GPL2.0开源.
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class GetStatisticRatio {
    public void getStatisticRatio() {
    }
    public void getInceasementStatisticRatio() {
    }
    public void getPCAStatisticRatio() {
    }
    public void getICAStatisticRatio() {
    }
    //
    }
    public void getCorrelationStatisticRatio() {
    }
    public void getBayansStatisticRatio() {
    }
    public double[] getStatisticRatio(double[] rgbIPECount) {

```

```
double countR = rgbIPECount[0];
double countB = rgbIPECount[1];
double countG = rgbIPECount[2];
double countYellow = rgbIPECount[3];
double countBlue = rgbIPECount[4];
double countRed = rgbIPECount[5];
double countGreen = rgbIPECount[6];
double countPupil = rgbIPECount[7];
double countWhite = rgbIPECount[8];
double countPink = rgbIPECount[9];
double countMidYellow = rgbIPECount[10];
double countMidBlue = rgbIPECount[11];
double countMidRed = rgbIPECount[12];
double countMidGreen = rgbIPECount[13];
double countMidPupil = rgbIPECount[14];
double countMidWhite = rgbIPECount[15];
double countMidPink = rgbIPECount[16];
double[] output = new double[45];
double ratioRB = 0 == countB ? 0 : countR / countB;
double ratioRG = 0 == countG ? 0 : countR / countG;
double ratioGB = 0 == countB ? 0 : countG / countB;
double ratioYellowR = 0 == countR ? 0 : countYellow / countR;
double ratioYellowG = 0 == countG ? 0 : countYellow / countG;
double ratioYellowB = 0 == countB ? 0 : countYellow / countB;
double countBlueR = 0 == countR ? 0 : countBlue / countR;
double countBlueG = 0 == countG ? 0 : countBlue / countG;
double countBlueB = 0 == countB ? 0 : countBlue / countB;
double countRedR = 0 == countR ? 0 : countRed / countR;
double countRedG = 0 == countG ? 0 : countRed / countG;
double countRedB = 0 == countB ? 0 : countRed / countB;
double countGreenR = 0 == countR ? 0 : countGreen / countR;
double countGreenG = 0 == countG ? 0 : countGreen / countG;
double countGreenB = 0 == countB ? 0 : countGreen / countB;
double countPupilR = 0 == countR ? 0 : countPupil / countR;
double countPupilG = 0 == countG ? 0 : countPupil / countG;
double countPupilB = 0 == countB ? 0 : countPupil / countB;
double countWhiteR = 0 == countR ? 0 : countWhite / countR;
double countWhiteG = 0 == countG ? 0 : countWhite / countG;
double countWhiteB = 0 == countB ? 0 : countWhite / countB;
double countPinkR = 0 == countR ? 0 : countPink / countR;
double countPinkG = 0 == countG ? 0 : countPink / countG;
double countPinkB = 0 == countB ? 0 : countPink / countB;
double ratioMidYellowR = 0 == countR ? 0 : countMidYellow / countR;
double ratioMidYellowG = 0 == countG ? 0 : countMidYellow / countG;
double ratioMidYellowB = 0 == countB ? 0 : countMidYellow / countB;
double countMidBlueR = 0 == countR ? 0 : countMidBlue / countR;
double countMidBlueG = 0 == countG ? 0 : countMidBlue / countG;
```

```
double countMidBlueB = 0 == countB ? 0 : countMidBlue / countB;
double countMidRedR = 0 == countR ? 0 : countMidRed / countR;
double countMidRedG = 0 == countG ? 0 : countMidRed / countG;
double countMidRedB = 0 == countB ? 0 : countMidRed / countB;
double countMidGreenR = 0 == countR ? 0 : countMidGreen / countR;
double countMidGreenG = 0 == countG ? 0 : countMidGreen / countG;
double countMidGreenB = 0 == countB ? 0 : countMidGreen / countB;
double countMidPupilR = 0 == countR ? 0 : countMidPupil / countR;
double countMidPupilG = 0 == countG ? 0 : countMidPupil / countG;
double countMidPupilB = 0 == countB ? 0 : countMidPupil / countB;
double countMidWhiteR = 0 == countR ? 0 : countMidWhite / countR;
double countMidWhiteG = 0 == countG ? 0 : countMidWhite / countG;
double countMidWhiteB = 0 == countB ? 0 : countMidWhite / countB;
double countMidPinkR = 0 == countR ? 0 : countMidPink / countR;
double countMidPinkG = 0 == countG ? 0 : countMidPink / countG;
double countMidPinkB = 0 == countB ? 0 : countMidPink / countB;
output[0] = ratioRB;
output[1] = ratioRG;
output[2] = ratioGB;
output[3] = ratioYellowR;
output[4] = ratioYellowG;
output[5] = ratioYellowB;
output[6] = countBlueR;
output[7] = countBlueG;
output[8] = countBlueB;
output[9] = countRedR;
output[10] = countRedG;
output[11] = countRedB;
output[12] = countGreenR;
output[13] = countGreenG;
output[14] = countGreenB;
output[15] = countPupilR;
output[16] = countPupilG;
output[17] = countPupilB;
output[18] = countWhiteR;
output[19] = countWhiteG;
output[20] = countWhiteB;
output[21] = countPinkR;
output[22] = countPinkG;
output[23] = countPinkB;
output[24] = ratioMidYellowR;
output[25] = ratioMidYellowG;
output[26] = ratioMidYellowB;
output[27] = countMidBlueR;
output[28] = countMidBlueG;
output[29] = countMidBlueB;
output[30] = countMidRedR;
```

```

        output[31] = countMidRedG;

        output[32] = countMidRedB;

        output[33] = countMidGreenR;

        output[34] = countMidGreenG;

        output[35] = countMidGreenB;

        output[36] = countMidPupilR;

        output[37] = countMidPupilG;

        output[38] = countMidPupilB;

        output[39] = countMidWhiteR;

        output[40] = countMidWhiteG;

        output[41] = countMidWhiteB;

        output[42] = countMidPinkR;

        output[43] = countMidPinkG;

        output[44] = countMidPinkB;

        return output;
    }
}

//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinDetect.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinDetectTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetect.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectDis.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectDisTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectString.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectStringTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTrip_E.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTrip_ETest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTrip.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDetectTripTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTrip.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2D_E.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2D_ETest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2D.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFix2DTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripFixTest.java
//Trif_DNA-master/C_L_I/DVIAQVIMIV/SkinPathDoubleDetectTripTest.java

```

I_A.IMO.EVU.bi_P

```

//Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/BigWord.java
//Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/BigWordTest.java
//Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/Classify.java
//Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/ClassifyTest.java
//Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthor.java
package I_A.IMO.EVU.bi_P;

```

```

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

import S_A.pheromone.IMV_SQI_SS;

import V_A.VSQ.OPE.jogl.JOGLOBJShape;

import com.jogamp.opengl.GL;

import com.jogamp.opengl.GL2;

import com.jogamp.opengl.GLAutoDrawable;

import com.jogamp.opengl.GLCapabilities;

import com.jogamp.opengl.GLEventListener;

import com.jogamp.opengl.GLProfile;

import com.jogamp.opengl.awt.GLCanvas;

import com.jogamp.opengl.glu.GLU;

import com.jogamp.opengl.util.FPSAnimator;

import com.jogamp.opengl.util.gl2.GLUT;

import javax.swing.Box;

import javax.swing.BoxLayout;

```

```
import javax.swing.JButton;

import javax.swing.JFrame;

import javax.swing.JPanel;

import javax.swing.JSlider;

import javax.swing.event.ChangeEvent;

import javax.swing.event.ChangeListener;

import java.awt.Component;

import java.awt.Dimension;

import java.awt.HeadlessException;

import java.awt.Toolkit;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.nio.ByteBuffer;

import java.util.Iterator;

import java.util.Map;

import java.util.concurrent.ConcurrentHashMap;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

 * *///, 基于gleem J3D官方demo 来写的。

public class CoAuthor extends JPanel implements GLEventListener {

    public IMV_SQI root;

    public Map<String, Integer> findLeaf;

    public IMV_SQI dic_li;

    public CogsBinaryForest_AE _A;

    public int c = 0;

    public int[] array;

    public FPSAnimator animator = null;

    ChangeListener listener;

    Box sliderBox = new Box(BoxLayout.Y_AXIS);

    Box buttonBox = new Box(BoxLayout.X_AXIS);

    JSlider sliderx;

    JSlider slidery;

    JSlider sliderz;

    JSlider slidert;

    JButton top;

    JButton down;

    JButton left;

    JButton right;

    GL2 gl;

    GLU glu;
```

```
GLUT glut;

float whiteLight[] = {0.2f, 0.2f, 0.2f, 1.0f};
float sourceLight[] = {0.8f, 0.8f, 0.8f, 1.0f};
float lightPos[] = {0.0f, 0.0f, 0.0f, 1.0f};
float moveshape = 0.0f;
float fEarthRot = 0.0f;
float xrot = 0.0f;
float yrot = 0.0f;
float zrot = 0.0f;
float trot = 0.0f;
float lrot = -2.0f;
float brot = 0.0f;
ByteBuffer buffer;
int[][] g;
JOGLOBSShape shape = null;
public double t = 1.0d;
public String rootWord;
// timeCheck ch=new timeCheck();
public CoAuthor(IMV_SQI dic_li, App NE) throws HeadlessException {
    this._A = NE.app_S._A;
    this.dic_li = dic_li;
    this.setLayout(null);
    GLProfile glp = GLProfile.getDefault();
    GLCapabilities glcaps = new GLCapabilities(glp);
    GLCanvas canvas = new GLCanvas(glcaps);
    canvas.addGLEventListener(this);
    canvas.setBounds(0, 0, 610, 370);
    this.add(canvas);
    animator = new FPSAnimator(canvas, 32, true);
    centerWindow(this);
    animator.start();
    sliderx = new JSlider(0, 360);
    sliderx.setSnapToTicks(true);
    sliderx.setPaintTicks(true);
    sliderx.setMajorTickSpacing(20);
    sliderx.setMinorTickSpacing(5);
    sliderx.addChangeListener(
        new ChangeListener() {
            public void stateChanged(ChangeEvent event) {
                JSlider source = (JSlider) event.getSource();
                xrot = source.getValue();
            }
        });
    slidery = new JSlider(0, 360);
    slidery.setSnapToTicks(true);
    slidery.setPaintTicks(true);
    slidery.setMajorTickSpacing(20);
```

```
slidery.setMinorTickSpacing(5);

slidery.addChangeListener(
    new ChangeListener() {
        public void stateChanged(ChangeEvent event) {
            JSlider source = (JSlider) event.getSource();
            yrot = source.getValue();
        }
    });

sliderz = new JSlider(0, 360);
sliderz.setSnapToTicks(true);
sliderz.setPaintTicks(true);
sliderz.setMajorTickSpacing(20);
sliderz.setMinorTickSpacing(5);
sliderz.addChangeListener(
    new ChangeListener() {
        public void stateChanged(ChangeEvent event) {
            JSlider source = (JSlider) event.getSource();
            zrot = source.getValue();
        }
    });

slidert = new JSlider(1, 50);
slidert.setSnapToTicks(true);
slidert.setPaintTicks(true);
slidert.setMajorTickSpacing(20);
slidert.setMinorTickSpacing(1);
slidert.addChangeListener(
    new ChangeListener() {
        public void stateChanged(ChangeEvent event) {
            JSlider source = (JSlider) event.getSource();
            trot = source.getValue();
        }
    });

top = new JButton("top");
top.addActionListener(
    new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            brot += 0.5;
        }
    });

down = new JButton("down");
down.addActionListener(
    new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            brot -= 0.5;
        }
    });

left = new JButton("left");
```

```
left.addActionListener(  
    new ActionListener() {  
        public void actionPerformed(ActionEvent e) {  
            lrot -= 0.5;  
        }  
    });  
right = new JButton("right");  
right.addActionListener(  
    new ActionListener() {  
        public void actionPerformed(ActionEvent e) {  
            lrot += 0.5;  
        }  
    });  
buttonBox.add(top);  
buttonBox.add(down);  
buttonBox.add(left);  
buttonBox.add(right);  
sliderBox.add(sliderx);  
sliderBox.add(slidery);  
sliderBox.add(sliderz);  
sliderBox.add(slidert);  
sliderBox.add(buttonBox);  
sliderBox.setBounds(0, 370, 610, 700);  
add(sliderBox);  
}  
  
private void centerWindow(Component frame) {  
    Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();  
    Dimension frameSize = frame.getSize();  
    if (frameSize.width > screenSize.width)  
        frameSize.width = screenSize.width;  
    if (frameSize.height > screenSize.height)  
        frameSize.height = screenSize.height;  
    frame.setLocation((screenSize.width - frameSize.width) >> 1,  
        (screenSize.height - frameSize.height) >> 1);  
}  
  
@SuppressWarnings("static-access")  
public void init(GLAutoDrawable drawable) {  
    gl = drawable.getGL().getGL2();  
    glu = new GLU();  
    glut = new GLUT();  
    gl.glEnable(GL.GL_TEXTURE_2D);  
    gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MIN_FILTER, gl.GL_LINEAR);  
    gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MAG_FILTER, gl.GL_NEAREST);  
    gl.glEnable(GL.GL_DEPTH_TEST);  
    gl.glDepthFunc(GL.GL_LEQUAL);  
    gl.glShadeModel(GL2.GL_SMOOTH);  
    gl.glClearColor(0f, 0f, 0f, 0f);           // The Type Of Depth Testing To Do
```

```

gl.glHint(gl.GL_PERSPECTIVE_CORRECTION_HINT, gl.GL_NICEST);

//20230106-S_logger.Log.logger.info("" + "error");

array = arrayInit();
}

private static int[] arrayInit() {
    int[] array = new int[100];
    java.util.Random r = new java.util.Random();
    for (int i = 100, j = 0; i > 0; i--, j++) {
        array[j] = r.nextInt(100);
    }
    return array;
}

public void display(GLAutoDrawable drawable) {
    GL2 gl = drawable.getGL().getGL2();
    gl.glClear(GL.GL_COLOR_BUFFER_BIT | GL.GL_DEPTH_BUFFER_BIT);
    gl.glLoadIdentity();
    gl.glTranslatef(lrot, brot, -6.0f); // translate left and into the screen
    gl.glRotatef(xrot, 1.0f, 0.0f, 0.0f); // Rotate On The X Axis
    gl.glRotatef(yrot, 0.0f, 1.0f, 0.0f); // Rotate On The Y Axis
    gl.glRotatef(zrot, 0.0f, 0.0f, 1.0f);
    gl.glPointSize(1);
    gl.glOrtho(0, //left
        trot, //right
        0, //bottom
        trot, //top
        1, -1);
    double section[] = new double[12];
    section[0] = 5.0;
    section[1] = 6.0;
    section[2] = 3.0;
    section[3] = 3.0;
    section[4] = 3.0;
    section[5] = 3.0;
    section[6] = 3.0;
    section[7] = 3.0;
    section[8] = 3.0;
    section[9] = 3.0;
    section[10] = 3.0;
    section[11] = 3.0;
    int a = 25;
    double root[] = new double[3];
    double pation = section[0] * section[1];
    int m = array[0];
    int l = array[0];
    for (int i = 0; i < array.length; i++) {
        if (array[i] > m) {
            m = array[i];

```

```

    }
    if (array[i] < 1) {
        l = array[i];
    }
}
double bina = m / pation;
draw(section, 0, root, a, 0, 0, bina);
gl.glEnd();
gl.glFlush();
}

@SuppressWarnings("static-access")
private void draw(double[] s, int se, double[] root, int a, int j, int k, double bina) {
    if (se < s.length) {
        for (int i = 1; i <= s[se]; i++) {
            gl.glBegin(gl.GL_LINES);
            gl.glColor3f(0, 0, 255);
            gl.glVertex3d(root[0], root[1], root[2]);
            gl.glVertex3d(root[0] + Math.cos(2 * Math.PI * i / s[se]) * a
                , root[1] + Math.sin(2 * Math.PI * i / s[se]) * a, root[2] + 1);
            gl.glEnd();
            gl.glColor3f(0, 0, 255);
            gl.glRasterPos3d(root[0] + Math.cos(2 * Math.PI * i / s[se]) * a
                , root[1] + Math.sin(2 * Math.PI * i / s[se]) * a, root[2] + 1);
            glut.glutBitmapString(GLUT.BITMAP_HELVETICA_12, (i) + "yaoguanguo");
            double root1[] = new double[3];
            root1[0] = root[0] + Math.cos(2 * Math.PI * i / s[se]) * a;
            root1[1] = root[1] + Math.sin(2 * Math.PI * i / s[se]) * a;
            root1[2] = root[2] + 1;
            j = i;
            k = se;
            draw(s, se + 1, root1, a / 2, j, k, bina);
        }
    } else {
        for (int i = 0; i < array.length; i++) {
            if (array[i] <= bina * j * s[k - 1] && array[i] > bina * (j - 1) * s[k - 1]) {
                gl.glBegin(gl.GL_LINES);
                gl.glColor3f(0, 255, 0);
                gl.glVertex3d(root[0], root[1], root[2]);
                gl.glVertex3d(root[0] + Math.cos(2 * Math.PI * i / 180) * a, root[1]
                    + Math.sin(2 * Math.PI * i / 180) * a, root[2] + 1);
                gl.glEnd();
                gl.glColor3f(255, 0, 0);
                gl.glRasterPos3d(root[0] + Math.cos(2 * Math.PI * i / 180) * a, root[1]
                    + Math.sin(2 * Math.PI * i / 180) * a, root[2] + 1);
                glut.glutBitmapString(GLUT.BITMAP_HELVETICA_12, (i) + "yaoguanguo");
            }
        }
    }
}

```

```

    }
}

public void reshape(GLAutoDrawable drawable, int x, int y, int width, int height) {
    float fAspect;
    if (height == 0) {
        height = 1;
    }

    gl.glViewport(0, 0, width, height);
    fAspect = (float) width / height;
    gl.glMatrixMode(GL2.GL_PROJECTION);
    gl.glLoadIdentity();
    glu.gluPerspective(53.0f, fAspect, 1.0, 400.0);
    gl.glMatrixMode(GL2.GL_MODELVIEW);
    gl.glLoadIdentity();
}

public void dispose(GLAutoDrawable arg0) {
}

public static void main(String[] args) {
    App NE = new App();
    CoAuthor app = new CoAuthor(null, NE);
    app.setSize(850, 700);
    app.setVisible(true);
    JFrame f = new JFrame();
    f.add(app);
    f.setTitle("ButtonDemo");
    f.setLocationRelativeTo(null);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setSize(650, 700);
    f.setVisible(true);
}

public void bootWordmap(String string, App NE) {
    rootWord = string;
    root = new IMV_SQI();
    findLeaf = new ConcurrentHashMap<String, Integer>();
    IMV_SQI leaf = new IMV_SQI();
    kerner(rootWord, leaf, findLeaf, NE);
    root.put(rootWord, leaf);
}

private void kerner(String temp, IMV_SQI root, Map<String, Integer> findLeaf, App NE) {
    if (dic_li.containsKey(temp)) {
        IMV_SQI leaf = new IMV_SQI();
        IMV_SQI_SS map = _A.parserMixStringByReturnFrequencyMap(dic_li.get(rootWord).toString(), NE);
        Iterator<String> it = map.keySet().iterator();
        while (it.hasNext()) {
            String tempRec = it.next();
            if (!findLeaf.containsKey(tempRec)) {
                findLeaf.put(tempRec, 1);
            }
        }
    }
}

```

```

        kerner(tempRec, leaf, findLeaf, NE);
    } else {
        findLeaf.put(tempRec, findLeaf.get(tempRec) + 1);
    }
}
root.put(temp, leaf);
}
}
}

/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWord_XCDX_M.java
package I_A.IMO.EVU.bi_P;

import ME.VPC.M.app.App;
import S_A.SVQ.stable.S_Maps;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_SS;
import java.awt.Font;
import java.awt.Graphics2D;
import java.util.Iterator;
import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 * *///, 基于gleem J3D官方demo 来写的。
public class CoAuthorForWord_XCDX_M extends CoAuthorForWord_XCDX_S {
    //dnn bingming
    public IMV_SQI kernerDnnBingMingForest(String[] dnn, IMV_SQI root
        , Map<String, Integer> findLeaf, int depth) {
        IMV_SQI leaf = new IMV_SQI();
        if (++depth > 6) {
            return leaf;
        }
        //s[se]= root3.size()> (int)iro? (int)iro: root3.size();
        int max = dnn.length > (int) irot ? (int) irot : dnn.length;
        for (int i = 1; i < max; i++) {
            String tempRec = dnn[i].split(":")[0];
            if (tempRec.length() > 1) {
                if (this.pos.containsKey(tempRec)) {
                    int find = 0;
                    if (S_Maps.mingCi.containsKey(tempRec)) {
                        find++;
                    }
                    if (S_Maps.dongCi.containsKey(tempRec)) {
                        find++;
                    }
                }
            }
        }
    }
}

```

```

    }
    if (S_Maps.xingRongCi.containsKey(tempRec)) {
        find++;
    }
    if (S_Maps.weiCi.containsKey(tempRec)) {
        find++;
    }
    if (find > 0) {
        if (!findLeaf.containsKey(tempRec)) {
            findLeaf.put(tempRec, 1);
            frequencyLeaf.put(tempRec, -1);
            Iterator<String> iterator = u.xyscPage.dic_shi_ying.keySet().iterator();
            IMV_SQI gongxiao = new IMV_SQI();
            Here:
            while (iterator.hasNext()) {
                String yaoming = iterator.next();
                IMV_SQI gongxiaoyaoming = new IMV_SQI();
                String li = u.xyscPage.dic_shi_ying.get(yaoming).toString();
                li += u.xyscPage.dic_yao_li.get(yaoming).toString();
                if (li.contains(tempRec)) {
                    String hai = u.xyscPage.dic_bu_liang_fan_ying
                        .get(yaoming).toString();
                    hai += u.xyscPage.dic_zhu_yi.get(yaoming).toString();
                    String xw = li;
                    if (!frequencyLeaf.containsKey(yaoming)) {
                        frequencyLeaf.put(yaoming, 1);
                        if (u.risk_filter_box.isSelected()) {
                            String temp = u.name_filter.getText();
                            for (int j = 0; j < temp.length(); j++) {
                                if (hai.contains("'" + temp.charAt(j))) {
                                    continue Here;
                                }
                            }
                        }
                    }
                }
                //li
                if (u.feel_filter_box.isSelected()) {
                    String temp = u.name_filter.getText();
                    for (int j = 0; j < temp.length(); j++) {
                        if (li.contains("'" + temp.charAt(j))) {
                            continue Here;
                        }
                    }
                }
                //xw
                if (!u.nameFeelFilter.getText().isEmpty()) {
                    String temp = u.nameFeelFilter.getText();
                    for (int j = 0; j < temp.length(); j++) {

```

```

        if (xw.contains("'" + temp.charAt(j))) {
            continue Here;
        }
    }
}

//sm
if (u.shuming_filter_box.isSelected()) {
    String temp = u.name_filter.getText();
    for (int j = 0; j < temp.length(); j++) {
        if (yaoming.contains("'" + temp.charAt(j))) {
            continue Here;
        }
    }
}

yaoming = yaoming.split("适应症")[0];
yaoming = yaoming.split(" ")[0];
yaoming = yaoming.split("\\(")[0];
gongxiao.put(yaoming, gongxiaoyaoming);
} else {
    frequencyLeaf.put(yaoming, frequencyLeaf.get(yaoming) + 1);
    //hai
}
//    }
}
}

leaf.put(tempRec, gongxiao);
} else {
    frequencyLeaf.put(tempRec, frequencyLeaf.get(tempRec) + 1);
}
}
}
}

return leaf;
}

//dnn
public IMV_SQI kernerDnnForest(String[] dnn, IMV_SQI root
, Map<String, Integer> findLeaf, int depth) {
    IMV_SQI leaf = new IMV_SQI();
    if (++depth > 6) {
        return leaf;
    }

    int max = dnn.length > (int) irot ? (int) irot : dnn.length;
    for (int i = 1; i < max; i++) {
        String tempRec = dnn[i].split(":")[0];
        if (tempRec.length() > 1) {
            if (this.pos.containsKey(tempRec)) {

```

```
int find = 0;
if (S_Maps.mingCi.containsKey(tempRec)) {
    find++;
}
if (S_Maps.dongCi.containsKey(tempRec)) {
    find++;
}
if (S_Maps.xingRongCi.containsKey(tempRec)) {
    find++;
}
if (S_Maps.weiCi.containsKey(tempRec)) {
    find++;
}
if (find > 0) {
    if (!findLeaf.containsKey(tempRec)) {
        findLeaf.put(tempRec, 1);
        frequencyLeaf.put(tempRec, -1);
        Iterator<String> iterator = u.dic_li.keySet().iterator();
        IMV_SQI gongxiao = new IMV_SQI();
        Here:
        while (iterator.hasNext()) {
            String yaoming = iterator.next();
            IMV_SQI gongxiaoyaoming = new IMV_SQI();
            if (u.dic_li.get(yaoming).toString().contains(tempRec)) {
                String hai = u.dic_hai.getString(yaoming);
                String li = u.dic_li.getString(yaoming);
                String xw = u.dic_xw.getString(yaoming);
                String jm = u.dic_jm.getTrimString(yaoming);//later..
                if (!frequencyLeaf.containsKey(yaoming)) {
                    frequencyLeaf.put(yaoming, 1);
                    //hai
                    if (u.risk_filter_box.isSelected()) {
                        String temp = u.name_filter.getText();
                        for (int j = 0; j < temp.length(); j++) {
                            if (hai.contains("" + temp.charAt(j))) {
                                continue Here;
                            }
                        }
                    }
                    //li
                    if (u.feel_filter_box.isSelected()) {
                        String temp = u.name_filter.getText();
                        for (int j = 0; j < temp.length(); j++) {
                            if (li.contains("" + temp.charAt(j))) {
                                continue Here;
                            }
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
    //xw
    if (!u.nameFeelFilter.getText().isEmpty()) {
        String temp = u.nameFeelFilter.getText();
        for (int j = 0; j < temp.length(); j++) {
            if (xw.contains("'" + temp.charAt(j))) {
                continue Here;
            }
        }
    }
    //jm
    if (null != u.name_filter_not_have.getText()) {
        if (!u.name_filter_not_have.getText().replace(" "
            , "").isEmpty()) {
            String temp = u.name_filter_not_have.getText();
            int findJM = 0;
            for (int j = 0; j < temp.length(); j++) {
                if (jm.contains("'" + temp.charAt(j))) {
                    findJM = 1;
                }
            }
            if (findJM == 0) {
                continue Here;
            }
        }
    }
    //sm
    if (u.shuming_filter_box.isSelected()) {
        String temp = u.name_filter.getText();
        for (int j = 0; j < temp.length(); j++) {
            if (yaoming.contains("'" + temp.charAt(j))) {
                continue Here;
            }
        }
        gongxiao.put(yaoming, gongxiaoyaoming);
    } else {
        frequencyLeaf.put(yaoming, frequencyLeaf.get(yaoming) + 1);
    }
}
}
leaf.put(tempRec, gongxiao);
} else {
    frequencyLeaf.put(tempRec, frequencyLeaf.get(tempRec) + 1);
}
}
}

```



```

    }
}
return leaf;
}
//neike JINJI
public IMV_SQI kernerZynkJForest(String temp, IMV_SQI root
, Map<String, Integer> findLeaf, int depth, App NE) {
    IMV_SQI leaf = new IMV_SQI();
    if (++depth > 6) {
        return leaf;
    }
    int ml = 0;
    IMV_SQI_SS map = _A.parserMixStringByReturnFrequencyMap(temp, NE);
    Iterator<String> it = map.keySet().iterator();
    int max = 0;
    while (it.hasNext()) {
        String tempRec = it.next();
        if (!findLeaf.containsKey(tempRec)) {
            findLeaf.put(tempRec, 1);
            frequencyLeaf.put(tempRec, -1);
            if (u.dic_map.containsKey(tempRec)) {
                IMV_SQI subLeaf = new IMV_SQI();
                if (u.dic_hai.containsKey(tempRec)) {
                    if (!u.dic_hai.get(tempRec).toString().contains("笔记")) {
                        IMV_SQI leafLiSub = new IMV_SQI();
                        subLeaf.put(u.dic_hai.get(tempRec).toString(), leafLiSub);
                    }
                }
                if (max++ < 25) {
                    leaf.put(tempRec, subLeaf);
                }
                //return leaf;
            } else if (ml < 8) {
                IMV_SQI subLeaf = kernerForest(tempRec, leaf, findLeaf, depth, NE);
                if (subLeaf.size() > 0) {
                    leaf.put(tempRec, subLeaf);
                    ml++;
                }
            }
            } else {
                frequencyLeaf.put(tempRec, frequencyLeaf.get(tempRec) + 1);
            }
        }
    }
    return leaf;
}
//li
public IMV_SQI kernerForest(String temp, IMV_SQI root

```

```

, Map<String, Integer> findLeaf, int depth, App NE) {
    IMV_SQI leaf = new IMV_SQI();
    if (++depth > 6) {
        return leaf;
    }
    //int ml=0;
    if (u.dic_li.containsKey(temp) || u.dic_yl.containsKey(temp) || u.dic_zf.containsKey(temp)) {
        IMV_SQI_SS map = _A.parserMixStringByReturnFrequencyMap(
            (u.dic_li.get(temp) == null ? "" : u.dic_li.get(temp)) +
            (u.dic_yl.get(temp) == null ? "" : "") +
            (u.dic_zf.get(temp) == null ? "" : ""))
        , NE);
        Iterator<String> it = map.keySet().iterator();
        while (it.hasNext()) { //gongxiao
            String tempRec = it.next();
            if (tempRec.length() > 1) {
                if (this.pos.containsKey(tempRec)) {
                    int find = 0;
                    if (S_Maps.mingCi.containsKey(tempRec)) {
                        find++;
                    }
                    if (S_Maps.dongCi.containsKey(tempRec)) {
                        find++;
                    }
                    if (S_Maps.xingRongCi.containsKey(tempRec)) {
                        find++;
                    }
                    if (S_Maps.weiCi.containsKey(tempRec)) {
                        find++;
                    }
                    if (find > 0) {
                        if (!findLeaf.containsKey(tempRec)) {
                            findLeaf.put(tempRec, 1);
                            frequencyLeaf.put(tempRec, -1);
                            Iterator<String> iterator = u.dic_li.keySet().iterator();
                            IMV_SQI gongxiao = new IMV_SQI();
                            while (iterator.hasNext()) {
                                String yaoming = iterator.next();
                                IMV_SQI gongxiaoyaoming = new IMV_SQI();
                                if (u.dic_li.get(yaoming).toString().contains(tempRec)) {
                                    String hai = u.dic_hai.get(yaoming).toString();
                                    if (!hai.contains("慎") && !hai.contains("禁") && !hai.contains("恶")
                                        && !hai.contains("毒") && !hai.contains("畏")
                                        && !hai.contains("孕") && !hai.contains("忌") && !hai.contains("胎")) {
                                        if (!frequencyLeaf.containsKey(yaoming)) {
                                            frequencyLeaf.put(yaoming, 1);
                                            String xingWeiRatio = u.dic_xw.get(yaoming).toString();

```

```
String xingWei = u.dic_xw.get(temp).toString();
String maiLuoRatio = u.dic_jm.get(yaoming).toString();
String maiLuo = u.dic_jm.get(temp).toString();
boolean _V_O = ((maiLuoRatio.contains("大肠") && maiLuo.contains("大肠"))
    || (maiLuoRatio.contains("小肠") && maiLuo.contains("小肠"))
    || (maiLuoRatio.contains("肝") && maiLuo.contains("肝"))
    || (maiLuoRatio.contains("脾") && maiLuo.contains("脾"))
    || (maiLuoRatio.contains("膀胱") && maiLuo.contains("膀胱"))
    || (maiLuoRatio.contains("胆") && maiLuo.contains("胆"))
    || (maiLuoRatio.contains("肺") && maiLuo.contains("肺"))
    || (maiLuoRatio.contains("肾") && maiLuo.contains("肾"))
    || (maiLuoRatio.contains("十二") && maiLuo.contains("十二"))
    || (maiLuoRatio.contains("任") && maiLuo.contains("任"))
    || (maiLuoRatio.contains("督") && maiLuo.contains("督"))
    || (maiLuoRatio.contains("三焦") && maiLuo.contains("三焦"))
    || (maiLuoRatio.contains("胃") && maiLuo.contains("胃"))
);
boolean _V_A = (!maiLuoRatio.contains("大肠") && !maiLuo.contains("大肠")
    && !maiLuoRatio.contains("小肠") && !maiLuo.contains("小肠")
    && !maiLuoRatio.contains("肝") && !maiLuo.contains("肝")
    && !maiLuoRatio.contains("脾") && !maiLuo.contains("脾")
    && !maiLuoRatio.contains("膀胱") && !maiLuo.contains("膀胱")
    && !maiLuoRatio.contains("胆") && !maiLuo.contains("胆")
    && !maiLuoRatio.contains("肺") && !maiLuo.contains("肺")
    && !maiLuoRatio.contains("肾") && !maiLuo.contains("肾")
    && !maiLuoRatio.contains("十二") && !maiLuo.contains("十二")
    && !maiLuoRatio.contains("任") && !maiLuo.contains("任")
    && !maiLuoRatio.contains("督") && !maiLuo.contains("督")
    && !maiLuoRatio.contains("三焦") && !maiLuo.contains("三焦")
    && !maiLuoRatio.contains("胃") && !maiLuo.contains("胃")
    && !maiLuoRatio.contains("肠") && !maiLuo.contains("肠")));
if (((xingWeiRatio.contains("辛") && xingWei.contains("辛"))
    || (xingWeiRatio.contains("温") && xingWei.contains("温"))
    || (xingWeiRatio.contains("燥") && xingWei.contains("燥"))
    || (xingWeiRatio.contains("热") && xingWei.contains("热"))
    || (xingWeiRatio.contains("苦") && xingWei.contains("苦"))
    || (xingWeiRatio.contains("寒") && xingWei.contains("寒"))
    || (xingWeiRatio.contains("凉") && xingWei.contains("凉"))
)) {
    if (maiLuoRatio.contains("心包")) {
        if ((maiLuo.contains("心包"))) {
            gongxiao.put(yaoming, gongxiaoyaoming);
        }
    }
}
if (!maiLuoRatio.contains("心包") && maiLuoRatio.contains("心")) {
    if ((!maiLuo.contains("心包") && maiLuo.contains("心"))) {
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
}
```

```

    }
    }
    if (_V_O) {
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
    if (_V_A) {
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
}

if (((!xingWeiRatio.contains("辛") && !xingWei.contains("辛")
    && !xingWeiRatio.contains("温") && !xingWei.contains("温")
    && !xingWeiRatio.contains("燥") && !xingWei.contains("燥")
    && !xingWeiRatio.contains("热") && !xingWei.contains("热")
    && !xingWeiRatio.contains("苦") && !xingWei.contains("苦")
    && !xingWeiRatio.contains("寒") && !xingWei.contains("寒")
    && !xingWeiRatio.contains("凉") && !xingWei.contains("凉")
))) {
    if (maiLuoRatio.contains("心包")) {
        if ((maiLuo.contains("心包"))) {
            gongxiao.put(yaoming, gongxiaoyaoming);
        }
    }
    if (!maiLuoRatio.contains("心包") && maiLuoRatio.contains("心")) {
        if ((!maiLuo.contains("心包") && maiLuo.contains("心"))) {
            gongxiao.put(yaoming, gongxiaoyaoming);
        }
    }
    if (_V_O) {
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
    if (_V_A) {
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
}
} else {
    frequencyLeaf.put(yaoming, frequencyLeaf.get(yaoming) + 1);
}
}
}
}
} else {
    frequencyLeaf.put(tempRec, frequencyLeaf.get(tempRec) + 1);
}
}
}
}

```

```

    }
}
return leaf;
}
//neike
public IMV_SQI kernerZynkForest(String temp, IMV_SQI root
, Map<String, Integer> findLeaf, int depth, App NE) {
    IMV_SQI leaf = new IMV_SQI();
    if (++depth > 6) {
        return leaf;
    }
    int ml = 0;
    int max = 0;
    IMV_SQI_SS map = _A.parserMixStringByReturnFrequencyMap(temp, NE);
    Iterator<String> it = map.keySet().iterator();
    while (it.hasNext()) {
        String tempRec = it.next();
        if (!findLeaf.containsKey(tempRec)) {
            findLeaf.put(tempRec, 1);
            frequencyLeaf.put(tempRec, -1);
            if (u.dic_map.containsKey(tempRec)) {
                IMV_SQI subLeaf = new IMV_SQI();
                if (u.dic_li.containsKey(tempRec)) {
                    IMV_SQI_SS liMap
                        = this._A.parserMixStringByReturnFrequencyMap(u.dic_li.get(tempRec).toString(), NE);
                    Iterator<String> itli = liMap.keySet().iterator();
                    while (itli.hasNext()) {
                        String word = itli.next();
                        if (this.pos.containsKey(word)) {
                            if (word.length() > 1) {
                                if (!frequencyLeaf.containsKey(word)) {
                                    frequencyLeaf.put(word, 1);
                                } else {
                                    frequencyLeaf.put(word, frequencyLeaf.get(word) + 1);
                                }
                            }
                            IMV_SQI leafLiSub = new IMV_SQI();
                            if (pos.getString(word).contains("动")) {
                                subLeaf.put(word, leafLiSub);
                            }
                            if (pos.getString(word).contains("形")) {
                                subLeaf.put(word, leafLiSub);
                            }
                            if (pos.getString(word).contains("谓")) {
                                subLeaf.put(word, leafLiSub);
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
    }
    if (max++ < 25) {
        leaf.put(tempRec, subLeaf);
    }
    //return leaf;
} else if (ml < 8) {
    IMV_SQI subLeaf = kernerForest(tempRec, leaf, findLeaf, depth, NE);
    if (subLeaf.size() > 0) {
        leaf.put(tempRec, subLeaf);
        ml++;
    }
}
} else {
    frequencyLeaf.put(tempRec, frequencyLeaf.get(tempRec) + 1);
}
}
return leaf;
}
//gongxiao
public IMV_SQI kernerClassification(String temp, IMV_SQI root
, Map<String, Integer> findLeaf, int depth, App NE) {
    IMV_SQI leaf = new IMV_SQI();
    if (++depth > 6) {
        return leaf;
    }
    frequencyLeaf.put(temp, -1);
    if (u.dic_li.containsKey(temp)) {
        String className = temp + "功效";
        findLeaf.put(className, 1);
        frequencyLeaf.put(className, -1);
        //add sub sets
        IMV_SQI_SS liMap
            = this._A.parserMixStringByReturnFrequencyMap(u.dic_li.get(temp).toString(), NE);
        Iterator<String> it = liMap.keySet().iterator();
        IMV_SQI leafLi = new IMV_SQI();
        int ml = 0;
        while (it.hasNext()) {
            String word = liMap.getW(it.next()).get_word();
            if (this.pos.containsKey(word)) {
                if (word.length() > 1) {
                    IMV_SQI leafLiSub = new IMV_SQI();
                    leafLi.put(word, leafLiSub);
                    frequencyLeaf.put(word, -1);
                } else if (ml < 10) {
                    IMV_SQI leafLiSub = new IMV_SQI();
                    if (leafLiSub.size() > 0) {

```

```

        leaf.put(word, leafLiSub);
        ml++;
    }
    ml++;
}
}
}
leaf.put(className, leafLi);
}
if (u.dic_hai.containsKey(temp)) {
    String className = temp + "禁忌";
    findLeaf.put(className, 1);
    frequencyLeaf.put(className, -1);
    IMV_SQI leafLi = new IMV_SQI();
    String string = u.dic_hai.get(temp).toString().length() > 40
        ? u.dic_hai.get(temp).toString().substring(0, 40) : u.dic_hai.get(temp).toString();
    leafLi.put(string, leafLi);
    frequencyLeaf.put(string, -1);
    leaf.put(className, leafLi);
}
if (u.dic_xw.containsKey(temp)) {
    String className = temp + "性味";
    findLeaf.put(className, 1);
    frequencyLeaf.put(className, -1);
    //add sub sets
    IMV_SQI_SS liMap
        = this._A.parserMixStringByReturnFrequencyMap(u.dic_xw.get(temp).toString(), NE);
    Iterator<String> it = liMap.keySet().iterator();
    IMV_SQI leafLi = new IMV_SQI();
    int ml = 0;
    while (it.hasNext()) {
        String word = liMap.getW(it.next()).get_word();
        if (this.pos.containsKey(word)) {
            if (word.length() > 1) {
                IMV_SQI leafLiSub = new IMV_SQI();
                leafLi.put(word, leafLiSub);
                frequencyLeaf.put(word, -1);
            } else if (ml < 10) {
                IMV_SQI leafLiSub = new IMV_SQI();
                leafLi.put(word, leafLiSub);
                frequencyLeaf.put(word, -1);
                ml++;
            }
        }
    }
}
leaf.put(className, leafLi);
}

```

```

    if (u.dic_yl.containsKey(temp)) {
        String className = temp + "用量备注";
        findLeaf.put(className, 1);
        frequencyLeaf.put(className, -1);
        //add sub sets
        IMV_SQI leafLi = new IMV_SQI();
        leafLi.put(u.dic_yl.get(temp).toString(), leafLi);
        frequencyLeaf.put(u.dic_yl.get(temp).toString(), -1);
        leaf.put(className, leafLi);
    }
    return leaf;
}

//hai
public IMV_SQI kernerSea(String temp, IMV_SQI root
    , Map<String, Integer> findLeaf, int depth, App NE) {
    IMV_SQI leaf = new IMV_SQI();
    if (++depth > 6) {
        return leaf;
    }
    //int ml=0;
    if (u.dic_yw.containsKey(temp)) {
        if (u.dic_yw.get(temp).toString().isEmpty()) {
            return leaf;
        }
    } else {
        return leaf;
    }
    if (u.dic_li.containsKey(temp) || u.dic_yl.containsKey(temp)
        || u.dic_zf.containsKey(temp)) {
        IMV_SQI_SS map = _A.parserMixStringByReturnFrequencyMap(
            (u.dic_li.get(temp) == null ? "" : u.dic_li.get(temp)) +
            (u.dic_yl.get(temp) == null ? "" : "") +
            (u.dic_zf.get(temp) == null ? "" : "")
            , NE
        );
        Iterator<String> it = map.keySet().iterator();
        while (it.hasNext()) { //gongxiao
            String tempRec = it.next();
            if (tempRec.length() > 1) {
                if (this.pos.containsKey(tempRec)) {
                    int find = 0;
                    if (S_Maps.mingCi.containsKey(tempRec)) {
                        find++;
                    }
                    if (S_Maps.dongCi.containsKey(tempRec)) {
                        find++;
                    }
                }
            }
        }
    }
}

```



```

if (S_Maps.xingRongCi.containsKey(tempRec)) {
    find++;
}
if (S_Maps.weiCi.containsKey(tempRec)) {
    find++;
}
if (find > 0) {
    if (!findLeaf.containsKey(tempRec)) {
        findLeaf.put(tempRec, 1);
        frequencyLeaf.put(tempRec, -1);
        Iterator<String> iterator = u.dic_li.keySet().iterator();
        IMV_SQI gongxiao
            = new IMV_SQI();
        //int max= 0;
        //bianli gongxiao
        Here:
        while (iterator.hasNext()) {
            String yaoming = iterator.next();
            IMV_SQI gongxiaoyaoming = new IMV_SQI();
            if (u.dic_li.get(yaoming).toString().contains(tempRec)) {
                String hai = u.dic_hai.get(yaoming).toString();
                if (hai.contains("慎") || hai.contains("禁") || hai.contains("恶")
                    || hai.contains("毒") || hai.contains("畏")
                    || hai.contains("孕") || hai.contains("忌") || hai.contains("胎")) {
                    if (!yaoming.contains("穴") && !frequencyLeaf.containsKey(yaoming)) {
                        frequencyLeaf.put(yaoming, 1);
                        String xingWeiRatio = u.dic_xw.get(yaoming).toString();
                        String xingWei = u.dic_xw.get(temp).toString();
                        String maiLuoRatio = u.dic_jm.get(yaoming).toString();
                        String maiLuo = u.dic_jm.get(temp).toString();

//
                        if (u.risk_filter_box.isSelected()) {
                            String tempRisk = u.name_filter.getText();
                            for (int j = 0; j < tempRisk.length(); j++) {
                                if (hai.contains("'" + tempRisk.charAt(j))) {
                                    continue Here;
                                }
                            }
                        }
                        if (((xingWeiRatio.contains("辛") && xingWei.contains("辛"))
                            || (xingWeiRatio.contains("温") && xingWei.contains("温"))
                            || (xingWeiRatio.contains("燥") && xingWei.contains("燥"))
                            || (xingWeiRatio.contains("热") && xingWei.contains("热"))
                            || (xingWeiRatio.contains("苦") && xingWei.contains("苦"))
                            || (xingWeiRatio.contains("寒") && xingWei.contains("寒"))
                            || (xingWeiRatio.contains("凉") && xingWei.contains("凉"))
                        )) {

```

```
if (maiLuoRatio.contains("心包")) {
    if ((maiLuo.contains("心包"))) {
        gongxiaoyaoming.put(hai, new IMV_SQI());
        findLeaf.put(yaoming, 1);
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
}
if (!maiLuoRatio.contains("心包") && maiLuoRatio.contains("心")) {
    if ((!maiLuo.contains("心包") && maiLuo.contains("心"))) {
        gongxiaoyaoming.put(hai, new IMV_SQI());
        findLeaf.put(yaoming, 1);
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
}
if ((maiLuoRatio.contains("大肠") && maiLuo.contains("大肠"))
    || (maiLuoRatio.contains("小肠") && maiLuo.contains("小肠"))
    || (maiLuoRatio.contains("肝") && maiLuo.contains("肝"))
    || (maiLuoRatio.contains("脾") && maiLuo.contains("脾"))
    || (maiLuoRatio.contains("膀胱") && maiLuo.contains("膀胱"))
    || (maiLuoRatio.contains("胆") && maiLuo.contains("胆"))
    || (maiLuoRatio.contains("肺") && maiLuo.contains("肺"))
    || (maiLuoRatio.contains("肾") && maiLuo.contains("肾"))
    || (maiLuoRatio.contains("十二") && maiLuo.contains("十二"))
    || (maiLuoRatio.contains("任") && maiLuo.contains("任"))
    || (maiLuoRatio.contains("督") && maiLuo.contains("督"))
    || (maiLuoRatio.contains("三焦") && maiLuo.contains("三焦"))
    || (maiLuoRatio.contains("胃") && maiLuo.contains("胃"))) {
    gongxiaoyaoming.put(hai, new IMV_SQI());
    findLeaf.put(yaoming, 1);
    gongxiao.put(yaoming, gongxiaoyaoming);
}
if ((!maiLuoRatio.contains("大肠") && !maiLuo.contains("大肠")
    && !maiLuoRatio.contains("小肠") && !maiLuo.contains("小肠")
    && !maiLuoRatio.contains("肝") && !maiLuo.contains("肝")
    && !maiLuoRatio.contains("脾") && !maiLuo.contains("脾")
    && !maiLuoRatio.contains("膀胱") && !maiLuo.contains("膀胱")
    && !maiLuoRatio.contains("胆") && !maiLuo.contains("胆")
    && !maiLuoRatio.contains("肺") && !maiLuo.contains("肺")
    && !maiLuoRatio.contains("肾") && !maiLuo.contains("肾")
    && !maiLuoRatio.contains("十二") && !maiLuo.contains("十二")
    && !maiLuoRatio.contains("任") && !maiLuo.contains("任")
    && !maiLuoRatio.contains("督") && !maiLuo.contains("督")
    && !maiLuoRatio.contains("三焦") && !maiLuo.contains("三焦")
    && !maiLuoRatio.contains("胃") && !maiLuo.contains("胃"))) {
    gongxiaoyaoming.put(hai, new IMV_SQI());
```

```

        findLeaf.put(yaoming, 1);
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
}

if (((!xingWeiRatio.contains("辛") && !xingWei.contains("辛"))
    && !xingWeiRatio.contains("温") && !xingWei.contains("温"))
    && !xingWeiRatio.contains("燥") && !xingWei.contains("燥"))
    && !xingWeiRatio.contains("热") && !xingWei.contains("热"))
    && !xingWeiRatio.contains("苦") && !xingWei.contains("苦"))
    && !xingWeiRatio.contains("寒") && !xingWei.contains("寒"))
    && !xingWeiRatio.contains("凉") && !xingWei.contains("凉"))
)) {
    if (maiLuoRatio.contains("心包")) {
        if ((maiLuo.contains("心包"))) {
            gongxiaoyaoming.put(hai, new IMV_SQI());
            findLeaf.put(yaoming, 1);
            gongxiao.put(yaoming, gongxiaoyaoming);
        }
    }

    if (!maiLuoRatio.contains("心包") && maiLuoRatio.contains("心")) {
        if ((!maiLuo.contains("心包") && maiLuo.contains("心"))) {
            gongxiaoyaoming.put(hai, new IMV_SQI());
            findLeaf.put(yaoming, 1);
            gongxiao.put(yaoming, gongxiaoyaoming);
        }
    }

    if ((maiLuoRatio.contains("大肠") && maiLuo.contains("大肠"))
        || (maiLuoRatio.contains("小肠") && maiLuo.contains("小肠"))
        || (maiLuoRatio.contains("肝") && maiLuo.contains("肝"))
        || (maiLuoRatio.contains("脾") && maiLuo.contains("脾"))
        || (maiLuoRatio.contains("膀胱") && maiLuo.contains("膀胱"))
        || (maiLuoRatio.contains("胆") && maiLuo.contains("胆"))
        || (maiLuoRatio.contains("肺") && maiLuo.contains("肺"))
        || (maiLuoRatio.contains("肾") && maiLuo.contains("肾"))
        || (maiLuoRatio.contains("十二") && maiLuo.contains("十二"))
        || (maiLuoRatio.contains("任") && maiLuo.contains("任"))
        || (maiLuoRatio.contains("督") && maiLuo.contains("督"))
        || (maiLuoRatio.contains("三焦") && maiLuo.contains("三焦"))
        || (maiLuoRatio.contains("胃") && maiLuo.contains("胃")))
    ) {
        gongxiaoyaoming.put(hai, new IMV_SQI());
        findLeaf.put(yaoming, 1);
        gongxiao.put(yaoming, gongxiaoyaoming);
    }

    if ((!maiLuoRatio.contains("大肠") && !maiLuo.contains("大肠") && (!maiLuoRatio.contains("小肠") && !
        && !maiLuoRatio.contains("肝") && !maiLuo.contains("肝"))
        && !maiLuoRatio.contains("脾") && !maiLuo.contains("脾"))
    ) {
        gongxiaoyaoming.put(hai, new IMV_SQI());
        findLeaf.put(yaoming, 1);
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
}

maiLuo.contains("小肠"))

```

```

        && !maiLuoRatio.contains("膀胱") && !maiLuo.contains("膀胱")
        && !maiLuoRatio.contains("胆") && !maiLuo.contains("胆")
        && !maiLuoRatio.contains("肺") && !maiLuo.contains("肺")
        && !maiLuoRatio.contains("肾") && !maiLuo.contains("肾")
        && !maiLuoRatio.contains("十二") && !maiLuo.contains("十二")
        && !maiLuoRatio.contains("任") && !maiLuo.contains("任")
        && !maiLuoRatio.contains("督") && !maiLuo.contains("督")
        && !maiLuoRatio.contains("三焦") && !maiLuo.contains("三焦")
        && !maiLuoRatio.contains("胃") && !maiLuo.contains("胃"))
    ) {
        gongxiaoyaoming.put(hai, new IMV_SQI());
        findLeaf.put(yaoming, 1);
        gongxiao.put(yaoming, gongxiaoyaoming);
    }
}
}
}
}
}
leaf.put(tempRec, gongxiao);
} else {
    frequencyLeaf.put(tempRec, frequencyLeaf.get(tempRec) + 1);
}
}
}
}
}
return leaf;
}

public void weightPrint(String prePrint, Graphics2D g, int AMV_MVS_VSQX
    , int AMV_MVS_VSQY, Font font) {
    g.setFont(font);
    g.drawString(prePrint, AMV_MVS_VSQX, AMV_MVS_VSQY);
}

public void heightPrint(String prePrint, Graphics2D g, int AMV_MVS_VSQX
    , int AMV_MVS_VSQY, Font font) {
    g.setFont(font);
    g.drawString(prePrint, AMV_MVS_VSQX, AMV_MVS_VSQY);
}
}

//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/CoAuthorForWord_XCDX_MTest.java
//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/CoAuthorForWord_XCDX_S.java
package I_A.IMO.EVU.bi_P;

import ME.VPC.S.ne.App_S;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import S_A.pheromone.IMV_SQI;

import V_A.VSQ.OPE.jogl.JOGLOBJShape;
```

```
import com.jogamp.opengl.GL2;

import com.jogamp.opengl.awt.GLCanvas;

import com.jogamp.opengl.glu.GLU;

import com.jogamp.opengl.util.FPSAnimator;

import com.jogamp.opengl.util.GLBuffers;

import javax.swing.Box;

import javax.swing.BoxLayout;

import javax.swing.JButton;

import javax.swing.JFrame;

import javax.swing.JPanel;

import javax.swing.JSlider;

import javax.swing.event.ChangeListener;

import java.nio.ByteBuffer;

import java.util.Map;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

 * */, 基于gleem J3D官方demo 来写的。

public class CoAuthorForWord_XCDX_S extends JPanel {

    public String rootWord;

    public ByteBuffer checkImageBuf = GLBuffers.newDirectByteBuffer(20 * 100 * 3);

    public boolean hook = false;

    public Boolean clicked = false;

    public int c = 0;

    public int mousex = 0;

    public int mousey = 0;

    public int zynk = 0;

    public int zybc = 0;

    public int needChange = 0;

    public int[][] g;

    public int section[];

    //public float whiteLight[] = {0.2f, 0.2f, 0.2f, 1.0f};

    // public float sourceLight[] = {0.8f, 0.8f, 0.8f, 1.0f};

    //public float lightPos[] = {0.0f, 0.0f, 0.0f, 1.0f};

    public float prot = 1.0f;

    public float lrot = -28.0f;

    public float brot = -1.5f;

    public float irot = 18.0f;

    public float srot = 0.7f;

    public float xrotr = 0.0f;

    public float yrotr = 0.0f;
```

```
public float zrotr = 0.0f;
public float trotr = 0.0f;
public float protr = 0.0f;
public float lrotr = 0.0f;
public float brotr = 0.0f;
public float irotr = 0.0f;
public float srotr = 0.0f;
public float moveshape = 0.0f;
public float fEarthRot = 0.0f;
public float xrot = 1.0f;
public float yrot = 1.0f;
public float zrot = 1.0f;
public float trot = 1.0f;
public double t = 1.0d;
public String prevSearchString = "";
//ui action
public JButton top;
public JButton down;
public JButton left;
public JButton right;
public JButton bhl;
public JButton cfc;
public JButton whh;
public JButton rst;
public JButton save;
public JButton load;
public JSlider sliderx;
public JSlider slidery;
public JSlider sliderz;
public JSlider slidert;
public Box sliderBoxx1 = new Box(BoxLayout.X_AXIS);
public Box sliderBoxx2 = new Box(BoxLayout.X_AXIS);
public Box sliderBox = new Box(BoxLayout.Y_AXIS);
public Box buttonBox = new Box(BoxLayout.X_AXIS);
public Box buttonBox1 = new Box(BoxLayout.X_AXIS);
public ChangeListener listener;
//util
public Map<String, Integer> didLeaf;
public IMV_SQI dic_li;
public IMV_SQI dic_yl;
public IMV_SQI rootLeaf;
public IMV_SQI pos;
public Map<String, Integer> findLeaf;
public Map<String, Integer> frequencyLeaf;
//native
public JOGLOBJShape shape = null;
public GLCanvas canvas;
```

```

public ByteBuffer buffer;

public GL2 gl;

public GLU glu;

//public GLUT glut;

public FPSAnimator animator = null;

//3d continuing object

public App_S u;

public CogsBinaryForest_AE _A;

public JFrame jframe;
}

/Trif_DNA-master/C_L_I/I_A/IMO/EVU/bi_P/CoAuthorForWord_XCDX.java
package I_A.IMO.EVU.bi_P;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

import java.awt.event.KeyEvent;

import java.awt.event.KeyListener;

import java.awt.event.MouseEvent;

import java.awt.event.MouseListener;

import java.awt.event.MouseMotionListener;

import java.awt.event.MouseWheelEvent;

import java.awt.event.MouseWheelListener;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

* *///, 基于gleem J3D官方demo 来写的。

public class CoAuthorForWord_XCDX extends CoAuthorForWord_XCDX_M implements

    MouseMotionListener, MouseListener, MouseWheelListener, KeyListener {

    //hai

    public String bootFlowerSea(String string, boolean isButton, App NE) {

        zynk = 0;

        zybc = 1;

        if (rootWord == null) {

            return "";

        }

        if (rootWord.equalsIgnoreCase(string) && isButton == false) {

            return "";

        }

        rootWord = string;

        rootLeaf.clear();

        findLeaf.clear();

        frequencyLeaf.clear();

        findLeaf.put(rootWord, 1);

        frequencyLeaf.put(rootWord, 1);

```

```
IMV_SQI leaf = new IMV_SQI();

int depth = 0;

IMV_SQI subLeaf = kernerSea(rootWord, leaf, findLeaf, depth, NE);

leaf.put(rootWord, subLeaf);

rootLeaf = leaf;

String output = subLeaf.keySet().toString();

return output;

}

public void bootClassificationMap(String string, boolean isButton, App NE) {

    zynk = 0;
    zybc = 1;
    if (rootWord == null) {
        return;
    }
    if (rootWord.equalsIgnoreCase(string) && isButton == false) {
        return;
    }
    rootWord = string;
    rootLeaf.clear();
    findLeaf.clear();
    frequencyLeaf.clear();
    findLeaf.put(rootWord, 1);
    frequencyLeaf.put(rootWord, 1);
    IMV_SQI leaf = new IMV_SQI();
    int depth = 0;
    IMV_SQI subLeaf = kernerClassification(rootWord, leaf, findLeaf, depth, NE);
    leaf.put(rootWord, subLeaf);
    rootLeaf = leaf;
}

public String bootFlowerForest(String string, boolean isButton, App NE) {

    zynk = 0;
    zybc = 1;
    if (rootWord == null) {
        return "";
    }
    if (rootWord.equalsIgnoreCase(string) && isButton == false) {
        return "";
    }
    rootWord = string;
    rootLeaf.clear();
    findLeaf.clear();
    frequencyLeaf.clear();
    findLeaf.put(rootWord, 1);
    frequencyLeaf.put(rootWord, 1);
    IMV_SQI leaf = new IMV_SQI();
    int depth = 0;
    IMV_SQI subLeaf = kernerForest(rootWord, leaf, findLeaf, depth, NE);
```



```
leaf.put(rootWord, subLeaf);
rootLeaf = leaf;
String output = subLeaf.keySet().toString();
return output;
}

public void bootDetaDnnFlowerForest(String mingCheng, String[] dnn
, boolean isButton) {
    zynk = 1;
    zybc = 1;
    if (rootWord == null) {
        return;
    }
    if (dnn.length < 2 && isButton == false) {
        return;
    }
    rootWord = mingCheng;
    rootLeaf.clear();
    findLeaf.clear();
    frequencyLeaf.clear();
    findLeaf.put(mingCheng, 1);
    frequencyLeaf.put(mingCheng, 1);
    IMV_SQI leaf = new IMV_SQI();
    int depth = 0;
    IMV_SQI subLeaf = kernerDnnForest(dnn, leaf, findLeaf, depth);
    leaf.put(mingCheng, subLeaf);
    rootLeaf = leaf;
}

public void bootDetaBingMingDnnFlowerForest(String mingCheng, String[] dnn
, boolean isButton) {
    zynk = 1;
    zybc = 0;
    u.leafrot = 2;
    if (rootWord == null) {
        return;
    }
    if (dnn.length < 2 && isButton == false) {
        return;
    }
    rootWord = mingCheng;
    rootLeaf.clear();
    findLeaf.clear();
    frequencyLeaf.clear();
    findLeaf.put(mingCheng, 1);
    frequencyLeaf.put(mingCheng, 1);
    IMV_SQI leaf = new IMV_SQI();
    int depth = 0;
    IMV_SQI subLeaf = kernerDnnBingMingForest(dnn, leaf, findLeaf, depth);
```

```
leaf.put(mingCheng, subLeaf);
rootLeaf = leaf;
}

public String bootZynkFlowerForest(String fangji, String zucheng
, boolean isButton, App NE) {
    zynk = 1;
    zybc = 1;
    if (rootWord == null) {
        return "";
    }
    if (rootWord.equalsIgnoreCase(fangji) && isButton == false) {
        return "";
    }
    rootWord = fangji;
    rootLeaf.clear();
    findLeaf.clear();
    frequencyLeaf.clear();
    findLeaf.put(fangji, 1);
    frequencyLeaf.put(fangji, 1);
    IMV_SQI leaf = new IMV_SQI();
    int depth = 0;
    IMV_SQI subLeaf = kernerZynkForest(zucheng, leaf, findLeaf, depth, NE);
    leaf.put(fangji, subLeaf);
    String output = subLeaf.keySet().toString();
    rootLeaf = leaf;
    return output;
}

public void bootZynkJFlowerForest(String fangji, String zucheng, boolean isButton, App NE) {
    zynk = 1;
    zybc = 1;
    if (rootWord == null) {
        return;
    }
    if (rootWord.equalsIgnoreCase(fangji) && isButton == false) {
        return;
    }
    rootWord = fangji;
    rootLeaf.clear();
    findLeaf.clear();
    frequencyLeaf.clear();
    findLeaf.put(fangji, 1);
    frequencyLeaf.put(fangji, 1);
    IMV_SQI leaf = new IMV_SQI();
    int depth = 0;
    IMV_SQI subLeaf = kernerZynkJForest(zucheng, leaf, findLeaf, depth, NE);
    leaf.put(fangji, subLeaf);
    rootLeaf = leaf;
```

```
}

@Override
public void mouseDragged(MouseEvent arg0) {
    if (arg0.getX() != mousex) {
        yrot += (arg0.getX() - mousex) / 5;
    }
    if (arg0.getY() != mousey) {
        xrot += (arg0.getY() - mousey) / 5;
    }
    mousex = arg0.getX();
    mousey = arg0.getY();
}

@Override
public void mouseMoved(MouseEvent arg0) {
}

@Override
public void keyPressed(KeyEvent arg0) {
    if (arg0.getKeyCode() == 38) {
        brot += 0.5;
    }
    if (arg0.getKeyCode() == 37) {
        lrot -= 0.5;
    }
    if (arg0.getKeyCode() == 40) {
        brot -= 0.5;
    }
    if (arg0.getKeyCode() == 39) {
        lrot += 0.5;
    }
}

@Override
public void keyReleased(KeyEvent arg0) {
}

@Override
public void mouseWheelMoved(MouseWheelEvent arg0) {
    trot += arg0.getWheelRotation();
}

@Override
public void mouseClicked(MouseEvent arg0) {
}

@Override
public void mouseEntered(MouseEvent arg0) {
    clicked = false;
}

public void mouseExited(MouseEvent arg0) {
    clicked = false;
}
```

```
@Override

public void mousePressed(MouseEvent arg0) {

    mousex = arg0.getX();

    mousey = arg0.getY();

}

@Override

public void mouseReleased(MouseEvent arg0) {

}

@Override

public void keyTyped(KeyEvent e) {

}

}

//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/CoAuthorForWord_XCDXTest.java
//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/CoAuthorForWord.java
package I_A.IMO.EVU.bi_P;

import ME.VPC.M.app.App;

import M_V.MVQ.button.DetaButton;

import M_V.MVQ.slider.DetaSlider;

import exception.thread.DetaThread;

import Q_V.OSI.AOP.MEC.SIQ.search.ZhongYaoSearch;

import S_A.SVQ.stable.S_Maps;

import S_A.pheromone.IMV_SQI;

import com.jogamp.opengl.GL;

import com.jogamp.opengl.GL2;

import com.jogamp.opengl.GLAutoDrawable;

import com.jogamp.opengl.GLCapabilities;

import com.jogamp.opengl.GLEventListener;

import com.jogamp.opengl.GLProfile;

import com.jogamp.opengl.awt.GLCanvas;

import com.jogamp.opengl.glu.GLU;

import com.jogamp.opengl.util.FPSAnimator;

import com.jogamp.opengl.util.GLBuffers;

import javax.swing.Box;

import javax.swing.BoxLayout;

import javax.swing.JButton;

import javax.swing.JFrame;

import javax.swing.JSlider;

import javax.swing.event.ChangeEvent;

import javax.swing.event.ChangeListener;

import java.awt.Color;

import java.awt.Component;

import java.awt.Dimension;

import java.awt.Font;

import java.awt.Graphics2D;

import java.awt.HeadlessException;

import java.awt.Toolkit;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;
```

```
import java.awt.image.BufferedImage;

import java.util.Iterator;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
/**, 基于gleem J3D官方demo 来写的。
public class CoAuthorForWord extends CoAuthorForWord_XCDX implements GLEventListener {
    public CoAuthorForWord(App NE) throws HeadlessException {
//System.load("/Users/joe/chilkatJava/libchilkat.jnilib");
//System.loadLibrary();

        DetaThread.sleepDeta(100);
        rootWord = "";
        rootLeaf = new IMV_SQI();
        findLeaf = new IMV_SQI();
        frequencyLeaf = new IMV_SQI();
        didLeaf = new IMV_SQI();
        this._A = NE.app_S._A;
        this.pos = NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn;
        this.u = NE.app_S;
        this.setLayout(null);
        DetaThread.sleepDeta(100);
        GLProfile glp = GLProfile.getDefault();
        GLCapabilities glcap = new GLCapabilities(glp);
        canvas = new GLCanvas(glcap);
        canvas.addGLEventListener(this);
        canvas.addMouseListener(this);
        DetaThread.sleepDeta(100);
        canvas.addMouseMotionListener(this);
        canvas.addKeyListener(this);
        canvas.addMouseWheelListener(this);
        canvas.setBounds(0, 0, 618, 350 - 50 - 30 - 20); //3d canvas
        DetaThread.sleepDeta(100);
        this.add(canvas);
        animator = new FPSAnimator(canvas, 24, true);
        centerWindow(this);
        animator.start();
        DetaThread.sleepDeta(100);
        sliderx = new DetaSlider(0, 360);
        sliderx.setSnapToTicks(true);
        sliderx.setPaintTicks(true);
        sliderx.setMajorTickSpacing(20);
```

```
sliderx.setMinorTickSpacing(5);

sliderx.addChangeListener(
    event -> {
        JSlider source = (JSlider) event.getSource();
        xrot = source.getValue();
    });
DetaThread.sleepDeta(100);
slidery = new DetaSlider(0, 360);
slidery.setSnapToTicks(true);
slidery.setPaintTicks(true);
slidery.setMajorTickSpacing(20);
slidery.setMinorTickSpacing(5);
slidery.addChangeListener(
    new ChangeListener() {
        public void stateChanged(ChangeEvent event) {
            JSlider source = (JSlider) event.getSource();
            yrot = source.getValue();
        }
    });
DetaThread.sleepDeta(100);
sliderz = new DetaSlider(0, 360);
sliderz.setSnapToTicks(true);
sliderz.setPaintTicks(true);
sliderz.setMajorTickSpacing(20);
sliderz.setMinorTickSpacing(5);
sliderz.addChangeListener(
    new ChangeListener() {
        public void stateChanged(ChangeEvent event) {
            JSlider source = (JSlider) event.getSource();
            zrot = source.getValue();
        }
    });
DetaThread.sleepDeta(100);
slidert = new DetaSlider(1, 21);
slidert.setSnapToTicks(true);
slidert.setPaintTicks(true);
slidert.setMajorTickSpacing(20);
slidert.setMinorTickSpacing(1);
slidert.addChangeListener(
    new ChangeListener() {
        public void stateChanged(ChangeEvent event) {
            JSlider source = (JSlider) event.getSource();
            irot = source.getValue();
        }
    });
top = new DetaButton("上", 100, 50, Color.pink);
top.addActionListener(
```

```
new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        brot += 0.5;
    }
});
down = new DetaButton("下", 100, 50, Color.orange);
down.addActionListener(
    new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            brot -= 0.5;
        }
    });
left = new DetaButton("左", 100, 50, Color.pink);
left.addActionListener(
    new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            lrot -= 0.5;
        }
    });
right = new DetaButton("右", 100, 50, Color.green);
right.addActionListener(
    e -> lrot += 0.5);
bhl = new DetaButton("功效森林", 100, 50, Color.pink);
bhl.addActionListener(
    e -> {
        String searchYaos = u.coAuthorForWord.bootFlowerForest(rootWord, true, NE);
        ZhongYaoSearch.zhongYaoSearch(u, searchYaos, "", searchYaos, NE);
    });
cfc = new DetaButton("属性植株", 100, 50, Color.orange);
cfc.addActionListener(
    e -> {
        bootClassificationMap(rootWord, true, NE);
        ZhongYaoSearch.zhongYaoSearch(u, u.key, "", u.key, NE);
    });
whh = new DetaButton("禁忌森林", 200, 50, Color.magenta);
whh.addActionListener(
    e -> {
        String searchYaos = u.coAuthorForWord.bootFlowerSea(rootWord, true, NE);
        ZhongYaoSearch.zhongYaoSearch(u, searchYaos, "", searchYaos, NE);
    });
rst = new DetaButton("重置", 200, 50, Color.pink);
rst.addActionListener(
    e -> {
        hook = !hook;
        if (null == jframe) {
            u.coAuthorForWord.canvas.setSize(900, 750);
            jframe = new JFrame("药物观测");
```

```
jframe.setLayout(null);
jframe.setSize(900, 750);
jframe.setVisible(false);
jframe.setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);
}
//拿出来
if (hook) {
    animator.stop();
    DetaThread.sleepDeta(200);
    u.coAuthorForWord.canvas.setSize(900, 750);
    DetaThread.sleepDeta(200);
    jframe.add(u.coAuthorForWord.canvas);
    DetaThread.sleepDeta(200);
    jframe.setVisible(true);
    u.coAuthorForWord.canvas.validate();
    DetaThread.sleepDeta(200);
    u.coAuthorForWord.validate();
    DetaThread.sleepDeta(200);
    jframe.validate();
    DetaThread.sleepDeta(200);
    animator.start();
    DetaThread.sleepDeta(1200);
    //semaphone问题被我hook解决了.
} else {
    animator.stop();
    DetaThread.sleepDeta(200);
    u.coAuthorForWord.canvas.setSize(618, 350 - 50 - 30 - 20);
    DetaThread.sleepDeta(200);
    u.coAuthorForWord.add(u.coAuthorForWord.canvas);
    DetaThread.sleepDeta(200);
    jframe.setVisible(false);
    DetaThread.sleepDeta(200);
    u.coAuthorForWord.canvas.validate();
    DetaThread.sleepDeta(200);
    u.coAuthorForWord.validate();
    DetaThread.sleepDeta(200);
    u.coAuthorForWord.xrot = 1.0f;
    u.coAuthorForWord.yrot = 1.0f;
    u.coAuthorForWord.zrot = 1.0f;
    u.coAuthorForWord.trot = 1.0f;
    u.coAuthorForWord.prot = 1.0f;
    u.coAuthorForWord.lrot = -28.0f;
    u.coAuthorForWord.brot = -1.5f;
    u.coAuthorForWord.irot = 18.0f;
    u.coAuthorForWord.srot = 0.7f;
    DetaThread.sleepDeta(200);
    animator.start();
```



```
DetaThread.sleepDeta(1200);
    //semaphone问题被我hook解决了.
}
});
save = new DetaButton("保存", 200, 50, Color.green);
//  rst.setBounds(0, 0, 100, 20);
save.addActionListener(e -> {
    xrotr = xrot;
    yrotr = yrot;
    zrotr = zrot;
    trotr = trot;
    protr = prot;
    lrotr = lrot;
    brotr = brot;
    irotr = irot;
    srotr = srot;
});
load = new DetaButton("读取", 200, 50, Color.magenta);
//  rst.setBounds(0, 0, 100, 20);
load.addActionListener(
    e -> {
        xrot = xrotr;
        yrot = yrotr;
        zrot = zrotr;
        trot = trotr;
        prot = protr;
        lrot = lrotr;
        brot = brotr;
        irot = irotr;
        srot = srotr;
    });
buttonBox.add(top);
buttonBox.add(down);
buttonBox.add(left);
DetaThread.sleepDeta(100);
buttonBox.add(right);
buttonBox.add(whh);
buttonBox.add(bhl);
buttonBox.add(cfc);
buttonBox1.add(rst);
buttonBox1.add(save);
buttonBox1.add(load);
Box buttonBoxX = new Box(BoxLayout.X_AXIS);
JButton xzb = new DetaButton("X坐标", 200, 50, Color.pink);
buttonBoxX.add(xzb);
buttonBoxX.add(sliderx);
sliderBoxx1.add(buttonBoxX);
```

```

Box buttonBoxY = new Box(BoxLayout.X_AXIS);
JButton yzb = new JButton("Y坐标", 200, 50, Color.pink);
buttonBoxY.add(yzb);
buttonBoxY.add(slidery);
sliderBoxx1.add(buttonBoxY);
Box buttonBoxZ = new Box(BoxLayout.X_AXIS);
JButton zzb = new JButton("Z坐标", 200, 50, Color.pink);
buttonBoxZ.add(zzb);
buttonBoxZ.add(sliderz);
sliderBoxx2.add(buttonBoxZ);
Box buttonBoxT = new Box(BoxLayout.X_AXIS);
JButton tzb = new JButton("T坐标", 200, 50, Color.pink);
buttonBoxT.add(tzb);
buttonBoxT.add(slidert);
sliderBoxx2.add(buttonBoxT);
sliderBox.add(sliderBoxx1);
sliderBox.add(sliderBoxx2);
sliderBox.add(buttonBox);
sliderBox.add(buttonBox1);
sliderBox.setBounds(0, 350 - 50 - 30 - 20, 627 - 10, 300 - 50); //3d
add(sliderBox);
}

private void centerWindow(Component frame) {
    Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
    Dimension frameSize = frame.getSize();
    if (frameSize.width > screenSize.width)
        frameSize.width = screenSize.width;
    if (frameSize.height > screenSize.height)
        frameSize.height = screenSize.height;
    frame.setLocation((screenSize.width - frameSize.width) >> 1,
        (screenSize.height - frameSize.height) >> 1);
}

@SuppressWarnings("static-access")
public void init(GLAutoDrawable drawable) {
    gl = drawable.getGL().getGL2();
    glu = new GLU();
    //glut = new GLUT();
    DetaThread.sleepDeta(100);
    //gl.glEnable(GL.GL_TEXTURE_2D);
    //gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MIN_FILTER, gl.GL_LINEAR);
    //gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MAG_FILTER, gl.GL_NEAREST);
    DetaThread.sleepDeta(100);
    gl.glEnable(GL.GL_DEPTH_TEST);
    gl.glDepthFunc(GL.GL_LEQUAL);
    gl.glShadeModel(GL2.GL_SMOOTH);
    DetaThread.sleepDeta(100);
    gl.glClearColor(0f, 0f, 0f, 0f);

```

```

gl.glHint(gl.GL_PERSPECTIVE_CORRECTION_HINT, gl.GL_NICEST);

gl.glMatrixMode(GL2.GL_PROJECTION);

gl.glLoadIdentity();

section = new int[21];

section[0] = 32;
section[1] = 32;
section[2] = 32;
section[3] = 32;
section[4] = 10;
section[5] = 10;
section[6] = 10;
section[7] = 10;
section[8] = 10;
section[9] = 10;
section[10] = 10;
section[11] = 10;

}

public void display(GLAutoDrawable drawable) {
    GL2 gl = drawable.getGL().getGL2();
    //gl.glMatrixMode(GL2.GL_PROJECTION);
    gl.glLoadIdentity();
    gl.glTranslatef(lrot, brot, -36.0f);
    gl.glRotatef(xrot, 1, 0.0f, 0.0f);
    gl.glRotatef(yrot, 0.0f, 1, 0.0f);
    gl.glRotatef(zrot, 0.0f, 0.0f, 1);
    gl.glPointSize(prot);
    gl.glOrtho(0, trot, 0, trot, 1, -1);
    float a = 10;
    double root[] = new double[3];
    for (int i = 0; i < root.length; i++) {
        root[i] = 0.0;
    }
    didLeaf.clear();
    gl.glClear(GL.GL_COLOR_BUFFER_BIT | GL.GL_DEPTH_BUFFER_BIT);
    gl.glBegin(GL.GL_TRIANGLES);
    drawWords(gl, section, 0, root, a, 0, 0, this.rootLeaf, didLeaf);
    gl.glEnd();
    gl.glFlush();
}

@SuppressWarnings({"unchecked", "static-access"})
private void drawWords(GL2 gl, int[] s, int se, double[] root, float a, int j
    , int k, IMV_SQI root3, Map<String, Integer> didLeaf) {
    DetaThread.sleepDeta(3);
    if (root3 == null || root3.size() == 0) {
        return;
    }
    int i = 0;

```

```
s[se] = root3.size() > (int) irot ? (int) irot : root3.size();
Iterator<String> iteratorLeaf = root3.keySet().iterator();
Here:
while (iteratorLeaf.hasNext()) {
    if (k >= 78) {
        return;
    }
    double a1 = 1.0;
    String leafName = iteratorLeaf.next();
    if (leafName.length() == 0) {
        continue Here;
    }
    int type = 0;
    IMV_SQI leaf = (IMV_SQI) root3.get(leafName);
    if (leaf.size() > 0) {
        a1 = 1.0;
    }
    k++;
    if (k > 25) {
        return;
    }
    if (se < 2) {
        if (leafName.equalsIgnoreCase("功能")
            || leafName.equalsIgnoreCase("引起")
            || leafName.equalsIgnoreCase("缺乏")
            || leafName.equalsIgnoreCase("作用")
            || leafName.equalsIgnoreCase("组成")
            || leafName.equalsIgnoreCase("输出")
            || leafName.equalsIgnoreCase("各种")
            || leafName.equalsIgnoreCase("引起")
            || leafName.equalsIgnoreCase("意义")
            || leafName.equalsIgnoreCase("传导")
            || leafName.equalsIgnoreCase("手术")
            || leafName.equalsIgnoreCase("性性")
            || leafName.equalsIgnoreCase("药物")
            || leafName.equalsIgnoreCase("过量")
            || leafName.equalsIgnoreCase("时可")
            || leafName.equalsIgnoreCase("治疗")
            || leafName.equalsIgnoreCase("预防")
            || leafName.equalsIgnoreCase("注射")
            || leafName.equalsIgnoreCase("应用")
            || leafName.equalsIgnoreCase("适用")
            || leafName.equalsIgnoreCase("适用于")
            || leafName.equalsIgnoreCase("适应")) {
            continue Here;
        }
    }
}
```

```

int freq = -1;
if (frequencyLeaf.containsKey(leafName)) {
    freq = frequencyLeaf.get(leafName);
}
if ((se < 2) || (se == 3) || (se == 2 && zybc == 1 && (freq == -1))
    || (se == 2 && (freq >= u.leafrot))) {
    gl.glBegin(gl.GL_LINES);
    if (j % 7 == 0) {
        gl.glColor3f(0, 0, 255);
    }
    if (j % 7 == 1) {
        gl.glColor3f(0, 255, 0);
    }
    if (j % 7 == 2) {
        gl.glColor3f(255, 0, 0);
    }
    if (j % 7 == 3) {
        gl.glColor3f(255, 0, 255);
    }
    if (j % 7 == 4) {
        gl.glColor3f(255, 255, 0);
    }
    if (j % 7 == 5) {
        gl.glColor3f(255, 255, 255);
    }
    if (j % 7 == 6) {
        gl.glColor3f(0, 255, 255);
    }
    gl.glVertex3d(root[0], root[1], root[2]);
    gl.glVertex3d(root[0] + Math.cos((2 * Math.PI / s[se]) * i) * a
        , root[1] + Math.sin((2 * Math.PI / s[se]) * i) * a
        , root[2] + (srot * a1));
    gl.glEnd();
    gl.glColor3f(0, 0, 255);
    gl.glRasterPos3d(root[0] + Math.cos((2 * Math.PI / s[se]) * i) * a
        , root[1] + Math.sin((2 * Math.PI / s[se]) * i) * a
        , root[2] + (srot * a1));
    String prePrint = leafName;
    prePrint = prePrint.substring(0, prePrint.length() > 20
        ? 20 : prePrint.length());
    Font font = new Font(prePrint, 20, 20);
    try {
        checkImageBuf = GLBuffers.newDirectByteBuffer(20
            * prePrint.length() * 22 * 3);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

BufferedImage image = new BufferedImage(prePrint.length() * 32
    , 20, BufferedImage.TYPE_INT_RGB);
Graphics2D ga = image.createGraphics();
ga.setColor(Color.white);
if (pos.containsKey(leafName)) {
    if (S_Maps.mingCi.containsKey(leafName)) {
        ga.setColor(new Color(255, 125, 255));
        type = 1;
    }
    if (S_Maps.dongCi.containsKey(leafName)) {
        ga.setColor(new Color(125, 255, 125));
    }
    if (S_Maps.xingRongCi.containsKey(leafName)) {
        ga.setColor(new Color(255, 255, 125));
    }
    if (S_Maps.weiCi.containsKey(leafName)) {
        ga.setColor(new Color(125, 125, 255));
    }
}
int AMV_MVS_VSQU = 1;
int AMV_MVS_VSQY = 17;
this.weightPrint(prePrint, ga, AMV_MVS_VSQU, AMV_MVS_VSQY, font);
this.heightPrint(prePrint, ga, AMV_MVS_VSQU, AMV_MVS_VSQY, font);
checkImageBuf.clear();
int h = image.getHeight();
for (int i1 = h - 1; i1 >= 0; i1--) {
    for (int j1 = 0; j1 < prePrint.length() * 20; j1++) {
        checkImageBuf.put((byte) (image.getRGB(j1, i1) & (byte) 0xFF));
        checkImageBuf.put((byte) (image.getRGB(j1, i1) >> 8 & (byte) 0xFF));
        checkImageBuf.put((byte) (image.getRGB(j1, i1) >> 16 & (byte) 0xFF));
    }
}
checkImageBuf.rewind();
try {
    gl.glDrawPixels(prePrint.length() * 20, 20, GL2.GL_RGB
        , GL.GL_UNSIGNED_BYTE, checkImageBuf);
} catch (Exception e) {
    e.printStackTrace();
    S_logger.Log.logger.info("'" + "2");
}
double root1[] = new double[3];
for (int regi = 0; regi < root1.length; regi++) {
    root1[regi] = 0.0;
}
root1[0] = root[0] + Math.cos((2 * Math.PI / s[se]) * i) * a;
root1[1] = root[1] + Math.sin((2 * Math.PI / s[se]) * i) * a;
root1[2] = root[2] + (srot * a1);

```

```

        if (!didLeaf.containsKey(leafName) && findLeaf.containsKey(leafName)) {
            didLeaf.put(leafName, 1);
            if (type == 1) {
                if (zynk == 0) {
                    drawWords(gl, s, se + 1, root1
                        , a * 0.5f, j + i, k, leaf, didLeaf);
                } else {
                    drawWords(gl, s, se + 1, root1
                        , a * 0.36f, j + i, k, leaf, didLeaf);
                }
            } else {
                drawWords(gl, s, se + 1, root1
                    , a * 0.5f, j + i, k, leaf, didLeaf);
            }
        }
        i++;
    }
}

public void reshape(GLAutoDrawable drawable, int x, int y, int width, int height) {
    float fAspect;
    if (height == 0) {
        height = 1;
    }
    gl.glViewport(0, 0, width, height);
    fAspect = (float) width / height;
    gl.glMatrixMode(GL2.GL_PROJECTION);
    gl.glLoadIdentity();
    glu.gluPerspective(53.0f, fAspect, 1.0, 400.0);
    gl.glMatrixMode(GL2.GL_MODELVIEW);
    gl.glLoadIdentity();
}

public void dispose(GLAutoDrawable arg0) {
}

public static void main(String[] args) throws HeadlessException {
    CoAuthorForWord app = new CoAuthorForWord(new App()); //later
    app.setSize(850, 700);
    app.setVisible(true);
    JFrame f = new JFrame();
    f.add(app);
    f.setTitle("ButtonDemo");
    f.setLocationRelativeTo(null);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setSize(650, 700);
    f.setVisible(true);
}
}

```

```
//372 //    int section[] = new int[21];

////    section[0] = 32;
////    section[1] = 32;
////    section[2] = 32;
////    section[3] = 32;
////    section[4] = 10;
////    section[5] = 10;
////    section[6] = 10;
////    section[7] = 10;
////    section[8] = 10;
////    section[9] = 10;
////    section[10] = 10;
////    section[11] = 10;

//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/CoAuthorForWordTest.java
//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/CoAuthorTest.java
//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/CoDNAAuthorForWord.java
package L_A.IMO.EVU.bi_P;

import ME.VPC.M.app.App;

import M_V.MVQ.button.DetaButton;

import M_V.MVQ.slider.DetaSlider;

import Q_A.life.Application;

import S_A.pheromone.IMV_SQI;

import bi_P.CoDNAAuthorForWord_X_S;

import com.jogamp.opengl.GL;

import com.jogamp.opengl.GL2;

import com.jogamp.opengl.GLAutoDrawable;

import com.jogamp.opengl.GLCapabilities;

import com.jogamp.opengl.GLEventListener;

import com.jogamp.opengl.GLProfile;

import com.jogamp.opengl.awt.GLCanvas;

import com.jogamp.opengl.glu.GLU;

import com.jogamp.opengl.util.FPSAnimator;

import com.jogamp.opengl.util.GLBuffers;

import com.jogamp.opengl.util.gl2.GLUT;

import exception.thread.DetaThread;

import javax.swing.Box;

import javax.swing.BoxLayout;

import javax.swing.JButton;

import javax.swing.JFrame;

import javax.swing.JSlider;

import java.awt.Color;

import java.awt.Component;

import java.awt.Dimension;

import java.awt.Font;

import java.awt.Graphics2D;

import java.awt.HeadlessException;

import java.awt.Toolkit;

import java.awt.event.KeyEvent;

import java.awt.event.KeyListener;
```



```
import java.awt.event.MouseEvent;

import java.awt.event.MouseListener;

import java.awt.event.MouseMotionListener;

import java.awt.event.MouseWheelEvent;

import java.awt.event.MouseWheelListener;

import java.awt.image.BufferedImage;

import java.util.Iterator;

import java.util.Map;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

 * 208号 阳光家园别墅小区 第十栋别墅 第三层

 * ///, 基于gleem J3D官方demo 来写的。

public class CoDNAAuthorForWord extends CoDNAAuthorForWord_X_S implements MouseMotionListener

, MouseListener, MouseWheelListener, KeyListener, GLEventListener {

public CoDNAAuthorForWord(App NE) throws HeadlessException {

    DetaThread.sleepDeta(100);

    rootWord = "";

    rootLeaf = new IMV_SQI();

    findLeaf = new IMV_SQI();

    frequencyLeaf = new IMV_SQI();

    didLeaf = new IMV_SQI();

    this._A = NE.app_S._A;

    this.pos = NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn;

    this.u = NE.app_S;

    this.setLayout(null);

    DetaThread.sleepDeta(100);

    GLProfile glp = GLProfile.getDefault();

    GLCapabilities glcaps = new GLCapabilities(glp);

    GLCanvas canvas = new GLCanvas(glcaps);

    canvas.addGLEventListener(this);

    canvas.addMouseListener(this);

    DetaThread.sleepDeta(100);

    canvas.addMouseMotionListener(this);

    canvas.addKeyListener(this);

    canvas.addMouseWheelListener(this);

    canvas.setBounds(0, 0, 1700, 700);

    DetaThread.sleepDeta(100);

    this.add(canvas);

    animator = new FPSAnimator(canvas, 24, true);

    centerWindow(this);

    animator.start();
```

```
DetaThread.sleepDeta(100);
sliderx = new DetaSlider(0, 360);
sliderx.setSnapToTicks(true);
sliderx.setPaintTicks(true);
sliderx.setMajorTickSpacing(20);
sliderx.setMinorTickSpacing(5);
sliderx.addChangeListener(
    event -> {
        JSlider source = (JSlider) event.getSource();
        xrot = source.getValue();
    });
DetaThread.sleepDeta(100);
slidery = new DetaSlider(0, 360);
slidery.setSnapToTicks(true);
slidery.setPaintTicks(true);
slidery.setMajorTickSpacing(20);
slidery.setMinorTickSpacing(5);
slidery.addChangeListener(
    event -> {
        JSlider source = (JSlider) event.getSource();
        yrot = source.getValue();
    });
DetaThread.sleepDeta(100);
sliderz = new DetaSlider(0, 360);
sliderz.setSnapToTicks(true);
sliderz.setPaintTicks(true);
sliderz.setMajorTickSpacing(20);
sliderz.setMinorTickSpacing(5);
sliderz.addChangeListener(
    event -> {
        JSlider source = (JSlider) event.getSource();
        zrot = source.getValue();
    });
DetaThread.sleepDeta(100);
slidert = new DetaSlider(1, 150);
slidert.setSnapToTicks(true);
slidert.setPaintTicks(true);
slidert.setMajorTickSpacing(20);
slidert.setMinorTickSpacing(1);
slidert.addChangeListener(
    event -> {
        JSlider source = (JSlider) event.getSource();
        irot = source.getValue();
    });
top = new DetaButton("上", 100, 50, Color.pink);
top.addActionListener(
    e -> brot += 0.5);
```

```
down = new DetaButton("下", 100, 50, Color.orange);
down.addActionListener(
    e -> brot -= 0.5);
left = new DetaButton("左", 100, 50, Color.pink);
left.addActionListener(
    e -> lrot -= 0.5);
right = new DetaButton("右", 100, 50, Color.green);
right.addActionListener(
    e -> lrot += 0.5);
bhl = new DetaButton("染色体", 100, 50, Color.pink);
bhl.addActionListener(
    e -> bootFlowerForest(rootWord, true));
cfc = new DetaButton("元基", 100, 50, Color.orange);
cfc.addActionListener(
    e -> bootClassificationMap(rootWord, true));
whh = new DetaButton("DNA", 200, 50, Color.magenta);
whh.addActionListener(
    e -> bootFlowerSea(rootWord, true));
rst = new DetaButton("重置", 200, 50, Color.pink);
// rst.setBounds(0, 0, 100, 20);
rst.addActionListener(
    e -> {
        xrot = 18.0f;
        yrot = 2.0f;
        zrot = 1.0f;
        trot = 6.0f;
        prot = 1.0f;
        lrot = -3.5f;
        brot = 1.0f;
        irot = 78.0f;
        srot = 0.7f;
    });
save = new DetaButton("保存", 200, 50, Color.green);
// rst.setBounds(0, 0, 100, 20);
save.addActionListener(
    e -> {
        xrotr = xrot;
        yrotr = yrot;
        zrotr = zrot;
        trotr = trot;
        protr = prot;
        lrotr = lrot;
        brotr = brot;
        irotr = irot;
        srotr = srot;
    });
load = new DetaButton("读取", 200, 50, Color.magenta);
```

```
// rst.setBounds(0, 0, 100, 20);

load.addActionListener(
    e -> {
        xrot = xrotr;
        yrot = yrotr;
        zrot = zrotr;
        trot = trotr;
        prot = protr;
        lrot = lrotr;
        brot = brotr;
        irot = irotr;
        srot = srotr;
    });

buttonBox.add(top);
buttonBox.add(down);
buttonBox.add(left);
DetaThread.sleepDeta(100);
buttonBox.add(right);
buttonBox.add(whh);
buttonBox.add(bhl);
buttonBox.add(cfc);
buttonBox.add(rst);
buttonBox.add(save);
buttonBox.add(load);

Box buttonBoxX = new Box(BoxLayout.X_AXIS);
JButton xzb = new DetaButton("X坐标", 200, 50, Color.pink);
buttonBoxX.add(xzb);
buttonBoxX.add(sliderx);
sliderBoxx1.add(buttonBoxX);

Box buttonBoxY = new Box(BoxLayout.X_AXIS);
JButton yzb = new DetaButton("Y坐标", 200, 50, Color.pink);
buttonBoxY.add(yzb);
buttonBoxY.add(slidery);
sliderBoxx1.add(buttonBoxY);

Box buttonBoxZ = new Box(BoxLayout.X_AXIS);
JButton zzb = new DetaButton("Z坐标", 200, 50, Color.pink);
buttonBoxZ.add(zzb);
buttonBoxZ.add(sliderz);
sliderBoxx2.add(buttonBoxZ);

Box buttonBoxT = new Box(BoxLayout.X_AXIS);
JButton tzb = new DetaButton("T坐标", 200, 50, Color.pink);
buttonBoxT.add(tzb);
buttonBoxT.add(slidert);
sliderBoxx2.add(buttonBoxT);

sliderBox.add(sliderBoxx1);
sliderBox.add(sliderBoxx2);
sliderBox.add(buttonBox);
```

```
sliderBox.setBounds(0, 750, 627, 300);

add(sliderBox);
}

public void bootFlowerSea(String rootWord2, boolean b) {
}

public void bootClassificationMap(String rootWord2, boolean b) {
}

public void bootFlowerForest(String rootWord2, boolean b) {
}

private void centerWindow(Component frame) {
    Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
    Dimension frameSize = frame.getSize();
    if (frameSize.width > screenSize.width)
        frameSize.width = screenSize.width;
    if (frameSize.height > screenSize.height)
        frameSize.height = screenSize.height;
    frame.setLocation((screenSize.width - frameSize.width) >> 1,
        (screenSize.height - frameSize.height) >> 1);
}

@SuppressWarnings("static-access")
public void init(GLAutoDrawable drawable) {
    gl = drawable.getGL().getGL2();
    glu = new GLU();
    glut = new GLUT();
    DetaThread.sleepDeta(100);
    gl.glEnable(GL.GL_TEXTURE_2D);
    gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MIN_FILTER, gl.GL_LINEAR);
    gl.glTexParameteri(gl.GL_TEXTURE_2D, gl.GL_TEXTURE_MAG_FILTER, gl.GL_NEAREST);
    DetaThread.sleepDeta(100);
    gl.glEnable(GL.GL_DEPTH_TEST);
    gl.glDepthFunc(GL.GL_LEQUAL);
    gl.glShadeModel(GL2.GL_SMOOTH);
    DetaThread.sleepDeta(100);
    gl.glClearColor(0f, 0f, 0f, 0f);
    gl.glHint(gl.GL_PERSPECTIVE_CORRECTION_HINT, gl.GL_NICEST);
}

public void display(GLAutoDrawable drawable) {
    GL2 gl = drawable.getGL().getGL2();
    gl.glClear(GL.GL_COLOR_BUFFER_BIT | GL.GL_DEPTH_BUFFER_BIT);
    gl.glLoadIdentity();
    gl.glTranslatef(1rot, brot, -6.0f);
    gl.glRotatef(xrot, 1, 0.0f, 0.0f);
    gl.glRotatef(yrot, 0.0f, 1, 0.0f);
    gl.glRotatef(zrot, 0.0f, 0.0f, 1);
    gl.glPointSize(prot);
    gl.glOrtho(0, trot, 0, trot, 1, -1);
    int section[] = new int[21];
```

```

    section[0] = 32;
    section[1] = 32;
    section[2] = 32;
    section[3] = 32;
    section[4] = 10;
    section[5] = 10;
    section[6] = 10;
    section[7] = 10;
    section[8] = 10;
    section[9] = 10;
    section[10] = 10;
    section[11] = 10;
    float a = 10;
    double root[] = new double[3];
    didLeaf.clear();
    drawWords(section, 0, root, a, 0, 0, this.rootLeaf, didLeaf);
    gl.glEnd();
    gl.glFlush();
}

@SuppressWarnings({"unchecked", "static-access"})
private void drawWords(int[] s, int se, double[] root, float a, int j, int k
    , IMV_SQI root3, Map<String, Integer> didLeaf) {
    DetaThread.sleepDeta(3);
    if (root3 == null || root3.size() == 0) {
        return;
    }
    int i = 0;
    s[se] = root3.size();
    Iterator<String> iteratorLeaf = root3.keySet().iterator();
    Here:
    while (iteratorLeaf.hasNext()) {
        if (k >= irot) {
            return;
        }
        double a1 = 1.0;
        String leafName = iteratorLeaf.next();
        if (leafName.length() == 0) {
            continue Here;
        }
        IMV_SQI leaf = (IMV_SQI) root3.get(leafName);
        if (leaf.size() > 0) {
            a1 = 1.0;
        }
        k++;
        gl.glBegin(gl.GL_LINES);
        if (j % 7 == 0) {
            gl.glColor3f(0, 0, 255);

```

```
}
if (j % 7 == 1) {
    gl.glColor3f(0, 255, 0);
}
if (j % 7 == 2) {
    gl.glColor3f(255, 0, 0);
}
if (j % 7 == 3) {
    gl.glColor3f(255, 0, 255);
}
if (j % 7 == 4) {
    gl.glColor3f(255, 255, 0);
}
if (j % 7 == 5) {
    gl.glColor3f(255, 255, 255);
}
if (j % 7 == 6) {
    gl.glColor3f(0, 255, 255);
}
gl.glVertex3d(root[0], root[1], root[2]);
if (se < 2) {
    gl.glEnd();
    gl.glColor3f(0, 255, 255);
    gl.glRasterPos3d(root[0] + i * a
        , root[1] + i * a
        , root[2] + (srot * a1));
} else {
    gl.glVertex3d(root[0] + Math.cos(2 * Math.PI * i / s[se]) * a
        , root[1] + Math.sin(2 * Math.PI * i / s[se]) * a
        , root[2] + (srot * a1));
    gl.glEnd();
    gl.glColor3f(0, 0, 255);
    gl.glRasterPos3d(root[0] + Math.cos(2 * Math.PI * i / s[se]) * a
        , root[1] + Math.sin(2 * Math.PI * i / s[se]) * a
        , root[2] + (srot * a1));
}
String prePrint = leafName;
Font font = new Font(prePrint, 20, 20);
checkImageBuf = GLBuffers.newDirectByteBuffer(20 * prePrint.length() * 22 * 3);
BufferedImage image = new BufferedImage(prePrint.length() * 32
    , 20, BufferedImage.TYPE_INT_RGB);
Graphics2D ga = image.createGraphics();
ga.setColor(Color.white);
int AMV_MVS_VSQX = 1;
int AMV_MVS_VSQY = 17;
this.weightPrint(prePrint, ga, AMV_MVS_VSQX, AMV_MVS_VSQY, font);
this.heightPrint(prePrint, ga, AMV_MVS_VSQX, AMV_MVS_VSQY, font);
```

```

    checkImageBuf.clear();
    int h = image.getHeight();
    for (int i1 = h - 1; i1 >= 0; i1--) {
        for (int j1 = 0; j1 < prePrint.length() * 20; j1++) {
            checkImageBuf.put((byte) (image.getRGB(j1, i1) & (byte) 0xFF));
            checkImageBuf.put((byte) (image.getRGB(j1, i1) >> 8 & (byte) 0xFF));
            checkImageBuf.put((byte) (image.getRGB(j1, i1) >> 16 & (byte) 0xFF));
        }
    }
    checkImageBuf.rewind();
    gl.glDrawPixels(prePrint.length() * 20, 20, GL2.GL_RGB
        , GL.GL_UNSIGNED_BYTE, checkImageBuf);
    double root1[] = new double[3];
    if (se < 2) {
        root1[0] = root[0] + i * a;
        root1[1] = root[1] + i * a;
        root1[2] = root[2] + (srot * a1);
    } else {
        root1[0] = root[0] + Math.cos(2 * Math.PI * i / s[se]) * a;
        root1[1] = root[1] + Math.sin(2 * Math.PI * i / s[se]) * a;
        root1[2] = root[2] + (srot * a1);
    }
    if (!leaf.isEmpty() && se < 1) {
        drawWords(s, se + 1, root1, a * 1f, j + i, k, leaf, didLeaf);
    } else if (!leaf.isEmpty() && se < 2) {
        drawWords(s, se + 1, root1, a * 0.7f, j + i, k, leaf, didLeaf);
    } else if (!leaf.isEmpty() && se >= 2) {
        drawWords(s, se + 1, root1, a * 1.2f, j + i, k, leaf, didLeaf);
    }
    i++;
}
}

public void reshape(GLAutoDrawable drawable, int x, int y, int width, int height) {
    float fAspect;
    if (height == 0) {
        height = 1;
    }
    gl.glViewport(0, 0, width, height);
    fAspect = (float) width / height;
    gl.glMatrixMode(GL2.GL_PROJECTION);
    gl.glLoadIdentity();
    glu.gluPerspective(53.0f, fAspect, 1.0, 400.0);
    gl.glMatrixMode(GL2.GL_MODELVIEW);
    gl.glLoadIdentity();
}

public void dispose(GLAutoDrawable arg0) {
}

```



```
public static void main(String[] args) throws HeadlessException {  
    App NE = new App();  
    Application application = new Application();  
    application.IV_();  
    CoDNAAuthorForWord app = new CoDNAAuthorForWord(NE);  
    app.setSize(1700, 950);  
    app.setVisible(true);  
    JFrame f = new JFrame();  
    f.add(app);  
    f.setTitle("ButtonDemo");  
    f.setLocationRelativeTo(null);  
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    f.setSize(1700, 950);  
    f.setVisible(true);  
    app.bootDNAFlowerForest("DNA三维观测", application, true);  
}  
  
public void bootDNAFlowerForest(String string  
    , Application application, boolean isButton) {  
    cms = application.cmss;  
    zynk = 1;  
    zybc = 1;  
    if (rootWord == null) {  
        return;  
    }  
    if (rootWord.equalsIgnoreCase(string) && isButton == false) {  
        return;  
    }  
    rootWord = string;  
    rootLeaf.clear();  
    findLeaf.clear();  
    frequencyLeaf.clear();  
    findLeaf.put(string, 1);  
    frequencyLeaf.put(string, 1);  
    IMV_SQI leaf = new IMV_SQI();  
    int depth = 0;  
    IMV_SQI subLeaf = kernerDNAForest("DNA", leaf, findLeaf, depth);  
    leaf.put(string, subLeaf);  
    rootLeaf = leaf;  
}  
  
private IMV_SQI kernerDNAForest(String string, IMV_SQI root  
    , Map<String, Integer> findLeaf, int depth) {  
    IMV_SQI leaf = new IMV_SQI();  
    //chos  
    Iterator<String> iteratorChos = cms.javaCMSsroot.keySet().iterator();//24  
    while (iteratorChos.hasNext()) {  
        String chos = iteratorChos.next();  
        IMV_SQI leafChosLeaf = new IMV_SQI();
```

```

Iterator<String> iteratorChosLeaf
    = cms.javaCMSsroot.get(chos).keySet().iterator(); //24*3i
while (iteratorChosLeaf.hasNext()) {
    String chosLeafs = iteratorChosLeaf.next();
    IMV_SQI leafChos = new IMV_SQI(); //24
    Iterator<String> iteratorChosLeafPDNs = cms.javaCMSsroot.get(chos)
        .get(chosLeafs).keySet().iterator();
    while (iteratorChosLeafPDNs.hasNext()) {
        String chosLeaf = iteratorChosLeafPDNs.next();
        IMV_SQI leafChosLeafs = new IMV_SQI();
        Iterator<String> iteratorChosLeafPDN = cms.javaCMSsroot.get(chos)
            .get(chosLeafs).get(chosLeaf).javaPDNs.keySet().iterator();
        while (iteratorChosLeafPDN.hasNext()) {
            String chosLeafPDN = iteratorChosLeafPDN.next();
            IMV_SQI leafChosLeafMap = new IMV_SQI();
            Iterator<String> iteratorChosLeafPDNSet = cms.javaCMSsroot
                .get(chos).get(chosLeafs).get(chosLeaf).javaPDNs.get(chosLeafPDN).iterator();
            while (iteratorChosLeafPDNSet.hasNext()) {
                String chosLeafPDNSet = iteratorChosLeafPDNSet.next();
                leafChosLeafMap.put(chosLeafPDNSet, new IMV_SQI()); //java name
            }
            leafChosLeafs.put(chosLeafPDN, leafChosLeafMap); //initons name
        }
        leafChos.put(chosLeaf, leafChosLeafs); //chos name
    }
    leafChosLeaf.put(chosLeafs, leafChos); //initons name
}
leaf.put(chos, leafChosLeaf); // rootlink
}
//findLeaf
//frequencyLeaf
return leaf;
}

public void weightPrint(String prePrint, Graphics2D g, int AMV_MVS_VSQX
    , int AMV_MVS_VSQY, Font font) {
    g.setFont(font);
    g.drawString(prePrint, AMV_MVS_VSQX, AMV_MVS_VSQY);
}

public void heightPrint(String prePrint, Graphics2D g, int AMV_MVS_VSQX
    , int AMV_MVS_VSQY, Font font) {
    g.setFont(font);
    g.drawString(prePrint, AMV_MVS_VSQX, AMV_MVS_VSQY);
}

public void mouseDragged(java.awt.event.MouseEvent arg0) {
    if (arg0.getX() != mousex) {
        yrot += (arg0.getX() - mousex) / 5;
    }
}

```

```

        if (arg0.getY() != mousey) {
            xrot += (arg0.getY() - mousey) / 5;
        }

        mousex = arg0.getX();
        mousey = arg0.getY();
    }

    public void mouseMoved(java.awt.event.MouseEvent arg0) {
    }

    public void keyPressed(KeyEvent arg0) {
        if (arg0.getKeyCode() == 38) {
            brot += 0.5;
        }
        if (arg0.getKeyCode() == 37) {
            lrot -= 0.5;
        }
        if (arg0.getKeyCode() == 40) {
            brot -= 0.5;
        }
        if (arg0.getKeyCode() == 39) {
            lrot += 0.5;
        }
    }

    public void keyReleased(KeyEvent arg0) {
    }

    public void keyTyped(KeyEvent arg0) {
    }

    public void mouseWheelMoved(MouseWheelEvent arg0) {
        trot += arg0.getWheelRotation();
    }

    public void mouseClicked(MouseEvent arg0) {
    }

    public void mouseEntered(MouseEvent arg0) {
        clicked = false;
    }

    public void mouseExited(MouseEvent arg0) {
        clicked = false;
    }

    public void mousePressed(MouseEvent arg0) {
        mousex = arg0.getX();
        mousey = arg0.getY();
    }

    public void mouseReleased(MouseEvent arg0) {
    }
}

```

```

//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/CoDNAAuthorForWordTest.java
//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/Histogram.java
//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/HistogramTest.java
//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/LineMed.java
//Trif_DNA-master/C_L_I/L_A/IMO/EVU/bi_P/LineMedTest.java

```

I_A.IMS.EUQ.recoder

```
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/Ipv.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/IpvTest.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/Recoder.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/RecoderTest.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/VoicePanel.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/recoder/VoicePanelTest.java
```

I_A.IMS.EUQ.SP

```
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/BootLine.java
package I_A.IMS.EUQ.SP;
```

```
import ME.VPC.M.app.App;
```

```
/*
```

```
* 个人著作权人，作者 罗瑶光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
public class BootLine {
```

```
    public SoundWave soundWave;
```

```
    public void boot(App NE) {
```

```
        soundWave = new SoundWave();
```

```
        new ReadingLine(NE).start();
```

```
        ;
```

```
        //new ComputingLine(soundWave).start();
```

```
        new ShowingLine(NE).start();
```

```
        ;
```

```
    }
```

```
}
```

```
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/BootLineTest.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ComputingLine.java
package I_A.IMS.EUQ.SP;
```

```
import ME.VPC.M.app.App;
```

```
import S_A.linePage.Line2025;
```

```
import exception.thread.DetaThread;
```

```
import javax.sound.sampled.AudioFormat;
```

```
import javax.sound.sampled.AudioInputStream;
```

```
import java.io.ByteArrayInputStream;
```

```
import java.io.ByteArrayOutputStream;
```

```
import java.io.IOException;
```

```
import java.util.Iterator;
```

```
import java.util.List;
```

```
/*
```

```
* 个人著作权人，作者 罗瑶光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

* 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class ComputingLine extends Thread implements Runnable {
    App NE;
    public ComputingLine(App app_NE) {
        NE = app_NE;
    }
    @SuppressWarnings({"unchecked"})
    public void run() {
        if (null != NE.app_S.line2025) {
            //reg
            Line2025 line2025 = NE.app_S.line2025;
            if(null==line2025){
                return;
            }
            if(null==line2025.format){
                return;
            }
            AudioFormat format = line2025.format;
            List<byte[]> temps = line2025.temps;
            SoundWaveVectorNew swn = line2025.swn;
            //ini
            byte[] audioBytes;
            ByteArrayOutputStream out;
            Iterator<byte[]> iterator;
            ByteArrayInputStream bais;
            AudioInputStream ais;
            AudioFormat af;
            while (NE.app_S.reg == 1) {
                try {
                    DetaThread.sleepDeta(1);
                    int size = temps.size();
                    if (size > 0) {
                        out = new ByteArrayOutputStream();
                        iterator = temps.iterator();
                        while (iterator.hasNext() && 1 == NE.app_S.reg) {
                            out.write(iterator.next());
                        }
                        temps.clear();
                        audioBytes = out.toByteArray();
                        bais = new ByteArrayInputStream(audioBytes);
                        long longS = audioBytes.length / format.getFrameSize();
                        ais = new AudioInputStream(bais, format, longS);
                        af = ais.getFormat();
                        if (null != audioBytes) {
                            swn.audioBytes = audioBytes;
                            swn.doAudioBytesTask(af, NE);
                        }
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

```

    }
    }//显示最大值
    } catch (IOException e) {
        e.printStackTrace();
    }
    }//显示波振幅
}
return;
}
}
}
//Trif_DNA-master/C-L-I/I_A/IMS/EUQ/SP/ComputingLineTest.java
//Trif_DNA-master/C-L-I/I_A/IMS/EUQ/SP/LYGFileIO.java
package I_A.IMS.EUQ.SP;

import ME.VPC.C.skivvy.MakeContainerZNSZ;

import ME.VPC.M.app.App;

import exception.file.O.DetaBufferedReader;

import exception.file.O.DetaInputStreamReader;

import U_A.PEU.P.movie.AdataFrame;

import U_A.PEU.P.movie.Header;

import U_A.PEU.P.movie.MdataFrame;

import U_A.PEU.P.sound.BytestoAIS;

import U_A.PEU.P.wave.DFT;

import javax.sound.sampled.AudioFormat;

import javax.sound.sampled.AudioFormat.Encoding;

import java.awt.image.BufferedImage;

import java.io.BufferedWriter;

import java.io.FileInputStream;

import java.io.FileOutputStream;

import java.io.InputStream;

import java.io.OutputStreamWriter;

import java.io.RandomAccessFile;

/*
 * 个人著作权人,作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class LYGFileIO {
    public Header header;

    public AdataFrame adataFrame;

    public MdataFrame mdataframe;

    public double duration;

    public String inputPath;

    public String outputPath;

    public int[] fsize;

```

```

public double[] master;

public double[] slave;

public double[] [] fftmatrix;

public int[] fftLogCount;

public BufferedImage image;

public MakeContainerZNSZ makeContainerZNSZ;

public LYGFileIO() {
}

public LYGFileIO initWithFFT(double fft[], App NE) {
    LYGFileIO IYGFileIO = new LYGFileIO();
    IYGFileIO.makeContainerZNSZ = NE._I_U.makeContainerZNSZ;
    IYGFileIO.header = new Header();
    IYGFileIO.header.MFrameRate = 0;
    IYGFileIO.header.MHFrame = 0;
    IYGFileIO.header.MWFrame = 0;
    IYGFileIO.header.MFrameLeangth = 0;
    IYGFileIO.header.SEn = new Encoding("PCM_SIGNED");
    IYGFileIO.header.SFrameLeangth = 0;
    IYGFileIO.header.SSampleRate = 19200;
    IYGFileIO.header.SSampleSizeInBits = 16;
    IYGFileIO.header.SChannels = 2;
    IYGFileIO.header.SFrameSize = 4;
    IYGFileIO.header.SFrameRate = 19200;
    IYGFileIO.header.SBigEndian = true;
    IYGFileIO.adataFrame = new AdataFrame();
    //IYGFileIO.adataFrame.audioArray= new double[(int)IYGFileIO.header.SFrameRate];
    DFT dFT = new DFT();
    dFT.IV_(1024);
    fft[10] = 10000.0;
    double[] array = dFT.ft(fft);
    IYGFileIO.adataFrame.audioArray = array;
    IYGFileIO.adataFrame.fftarray = fft;
    AudioFormat af = new AudioFormat(IYGFileIO.header.SEn,
        IYGFileIO.header.SSampleRate,
        IYGFileIO.header.SSampleSizeInBits,
        IYGFileIO.header.SChannels,
        IYGFileIO.header.SFrameSize,
        IYGFileIO.header.SFrameRate,
        IYGFileIO.header.SBigEndian);
    IYGFileIO.IV_ByAudioBytes(af);
    //double times;
    //long milliseconds= (long)((header.SFrameLeangth * 1000)/ header.SFrameRate);
    //IYGFileIO.header.times= milliseconds/ 1000.0;

    return IYGFileIO;
}

public void lygRead(String filepath) {
    try {

```

```
//read head by av
InputStream is = new FileInputStream(filepath);
String line;
DataBufferedReader reader = new DataBufferedReader(DataInputStreamReader.E(is));
while ((line = reader.readDataLine()) != null) {
    //read head
    if (line.equals("<HEAD>")) {
        //20230106-S_logger.Log.logger.info("" + "has HEAD");
        header = new Header();
    }
    if (line.equals("<MFR>")) {
        line = reader.readDataLine();
        header.MFrameRate = Integer.parseInt(line);
    }
    if (line.equals("<MHR>")) {
        line = reader.readDataLine();
        header.MHFrame = Integer.parseInt(line);
    }
    if (line.equals("<MWR>")) {
        line = reader.readDataLine();
        header.MWFrame = Integer.parseInt(line);
    }
    if (line.equals("<MFL>")) {
        line = reader.readDataLine();
        header.MFrameLeangth = Integer.parseInt(line);
    }
    //
    if (line.equals("<AEN>")) {
        line = reader.readDataLine();
        header.SEn = new Encoding(line);
    }
    if (line.equals("<AFL>")) {
        line = reader.readDataLine();
        header.SFrameLeangth = Long.parseLong(line);
    }
    if (line.equals("<ASR>")) {
        line = reader.readDataLine();
        header.SSampleRate = Float.parseFloat(line);
    }
    if (line.equals("<ASS>")) {
        line = reader.readDataLine();
        header.SSampleSizeInBits = Integer.parseInt(line);
    }
    if (line.equals("<ACH>")) {
        line = reader.readDataLine();
        header.SChannels = Integer.parseInt(line);
    }
}
```



```
if (line.equals("<AFS>")) {
    line = reader.readDetaLine();
    header.SFrameSize = Integer.parseInt(line);
}
if (line.equals("<AFR>")) {
    line = reader.readDetaLine();
    header.SFrameRate = Float.parseFloat(line);
}
if (line.equals("<ABE>")) {
    line = reader.readDetaLine();
    header.SBigEndian = Boolean.parseBoolean(line);
}
//read frame
if (line.equals("<FRAME>")) {
    //20230106-S_logger.Log.logger.info(" " + "has FRAME");
}
if (line.equals("<AF>")) {
    adataFrame = new AdataFrame();
    adataFrame.audioArray = new double[(int) header.SFrameRate];
    int i = 0;
    while ((!(line = reader.readDetaLine()).equals("</AF>")))
        && i < (int) header.SFrameRate {
        adataFrame.audioArray[i++] = Double.parseDouble(line);
    }
    double times;
    long milliseconds = (long) ((header.SFrameLeangth * 1000) / header.SFrameRate);
    times = milliseconds / 1000.0;
    //
    int cur = (int)times;
    if (cur < times) {
        times = cur + 1.0;
    }
    for (int i1 = 0; i1 < times; i1++) {
        adataFrame.next = new AdataFrame();
        adataFrame.next.prev = adataFrame;
        adataFrame = adataFrame.next;
        adataFrame.audioArray = new double[(int) header.SFrameRate];
        int i2 = 0;
        line = reader.readDetaLine();
        while (!(line.equals("</AF>")))
            && i2 < (int) header.SFrameRate
            && !line.equals("<MF>")
        ) {
            //20230106-S_logger.Log.logger.info(" " + i2);
            adataFrame.audioArray[i2++] = Double.parseDouble(line);
            line = reader.readDetaLine();
        }
    }
```

```

    }
}

if (line.equals("<MF>")) {
    int frameID = 0;
    mdataframe = new MdataFrame();
    mdataframe.RGBimage = new int[header.MHFrame][header.MWFrame];
    for (int i = 0; i < header.MHFrame; i++) {
        for (int j = 0; j < header.MWFrame; j++) {
            if (!(line = reader.readDetaLine()).equals("</MF>")) {
                mdataframe.RGBimage[i][j] = Integer.parseInt(line);
                mdataframe.frameID = frameID++;
            }
        }
    }
}

while (!(line = reader.readDetaLine()).equals("</MF>")) {
    mdataframe.next = new MdataFrame();
    mdataframe.next.prev = mdataframe;
    mdataframe = mdataframe.next;
    mdataframe.RGBimage = new int[header.MHFrame][header.MWFrame];
    for (int i = 0; i < header.MHFrame; i++) {
        for (int j = 0; j < header.MWFrame; j++) {
            if (!(line = reader.readDetaLine()).equals("</MF>")) {
                mdataframe.RGBimage[i][j] = Integer.parseInt(line);
                mdataframe.frameID = frameID++;
            }
        }
    }
}

}

}

is.close();
reader.close();
} catch (Exception e) {
    e.printStackTrace();
}

}

public void reset() {
    header = null;
    adataFrame = null;
    mdataframe = null;
}

public void lygWrite(String string) {
    try {
        BufferedWriter wr = new BufferedWriter(new OutputStreamWriter(new FileOutputStream(string), "UTF8"));
        wr.write("<HEAD>" + "\n");
        wr.write("<MFR>\n" + header.MFrameRate + "\n</MFR>\n");
        //20230106-S_logger.Log.logger.info("'" + "<MFR>\n"+header.MFrameRate+"\n</MFR>\n");
    }
}

```

```

wr.write("<MHR>\n" + header.MHFrame + "\n</MHR>\n");
//20230106-S_logger.Log.logger.info("'" + "<MHR>\n"+header.MHFrame+"\n</MHR>\n");
wr.write("<MWR>\n" + header.MWFrame + "\n</MWR>\n");
//20230106-S_logger.Log.logger.info("'" + "<MWR>\n"+header.MWFrame+"\n</MWR>\n");
wr.write("<MFL>\n" + header.MFrameLeangth + "\n</MFL>\n");
//20230106-S_logger.Log.logger.info("'" + "<MFL>\n"+header.MFrameLeangth+"\n</MFL>\n");
//en /1sample rate /2samplesize /3 channels /4framesize /5 framrate/bigendianture
wr.write("<AEN>\n" + header.SEn.toString() + "\n</AEN>\n");
//20230106-S_logger.Log.logger.info("'" + "<AEN>\n"+header.SEn.toString()+"\n</AEN>\n");
wr.write("<ASR>\n" + header.SSampleRate + "\n</ASR>\n");
//20230106-S_logger.Log.logger.info("'" + "<ASR>\n"+header.SSampleRate+"\n</ASR>\n");
wr.write("<ASS>\n" + header.SSampleSizeInBits + "\n</ASS>\n");
//20230106-S_logger.Log.logger.info("'" + "<ASS>\n"+header.SSampleSizeInBits+"\n</ASS>\n");
wr.write("<ACH>\n" + header.SChannels + "\n</ACH>\n");
//20230106-S_logger.Log.logger.info("'" + "<ACH>\n"+header.SChannels+"\n</ACH>\n");
wr.write("<AFS>\n" + header.SFrameSize + "\n</AFS>\n");
//20230106-S_logger.Log.logger.info("'" + "<AFS>\n"+header.SFrameSize+"\n</AFS>\n");
wr.write("<AFR>\n" + header.SFrameRate + "\n</AFR>\n");
//20230106-S_logger.Log.logger.info("'" + "<AFR>\n"+header.SFrameRate+"\n</AFR>\n");
wr.write("<ABE>\n" + header.SBigEndian + "\n</ABE>\n");
//20230106-S_logger.Log.logger.info("'" + "<ABE>\n"+header.SBigEndian+"\n</ABE>\n");
wr.write("<AFL>\n" + header.SFrameLeangth + "\n</AFL>\n");
//20230106-S_logger.Log.logger.info("'" + "<AFL>\n"+header.SFrameLeangth+"\n</AFL>\n");
wr.write("</HEAD>" + "\n");
wr.write("<FRAME>" + "\n");
wr.write("<AF>" + "\n");
if (adataFrame != null) {
    for (int i = 0; i < adataFrame.audioArray.length; i++) {
        wr.write(adataFrame.audioArray[i] + "\n");
    }
    while (adataFrame.next != null) {
        adataFrame = adataFrame.next;
        for (int i = 0; i < adataFrame.audioArray.length; i++) {
            wr.write(adataFrame.audioArray[i] + "\n");
        }
    }
}
wr.write("</AF>" + "\n");
wr.write("<MF>" + "\n");
long fr = 0;
if (mdataframe != null) {
    for (int i = 0; i < header.MHFrame; i++) {
        for (int j = 0; j < header.MWFrame; j++) {
            wr.write(mdataframe.image.getRGB(j, i) + "\n");
        }
    }
}
fr++;

```

```

        wr.write(fr + "\n");
        while (mdataframe.next != null) {
            mdataframe = mdataframe.next;
            for (int i = 0; i < header.MHFrame; i++) {
                for (int j = 0; j < header.MWFrame; j++) {
                    wr.write(mdataframe.image.getRGB(j, i) + "\n");
                }
            }
            fr++;
            wr.write(fr + "\n");
        }
    }
    wr.write("</MF>" + "\n");
    wr.write("</FRAME>" + "\n");
    wr.flush();
    wr.close();
} catch (Exception e) {
    e.printStackTrace();
}
}

public void toHead() {
    while (adataFrame != null && adataFrame.prev != null) {
        adataFrame = adataFrame.prev;
    }
    while (mdataframe != null && mdataframe.prev != null) {
        mdataframe = mdataframe.prev;
    }
}

public void IV_() {
    toHead();
    AudioFormat af = new AudioFormat(header.SEn,
        header.SSampleRate,
        header.SSampleSizeInBits,
        header.SChannels,
        header.SFrameSize,
        header.SFrameRate,
        header.SBigEndian);
    //en /1sample rate /2samplesize /3 channels /4framesize /5 framrate /bigendianture
    int times = 0;
    if (adataFrame != null) {
        times++;
        adataFrame.audioIS = new BytestoAIS().getAIS(adataFrame.audioArray, af, adataFrame.audioIS);
        while (adataFrame.next != null) {
            times++;
            adataFrame = adataFrame.next;
            adataFrame.audioIS = new BytestoAIS().getAIS(adataFrame.audioArray, af, adataFrame.audioIS);
        }
    }
}

```

```
}  
  
this.duration = (double)times;  
toHead();  
  
}  
  
public void IQ_header(RandomAccessFile raf) {  
    try {  
        header = new Header();  
        raf.seek(0);  
        long headpos = 0;  
        byte[] iv = new byte[4];  
        raf.read(iv);  
        int f0 = ((int) iv[0] & 0x000000ff);  
        int f1 = ((int) iv[1] << 8 & 0x0000ff00);  
        int f2 = ((int) iv[2] << 16 & 0x00ff0000);  
        int f3 = ((int) iv[3] << 24 & 0xff000000);  
        f0 = f3 | f2 | f1 | f0;  
        header.MWFrame = f0;  
        //20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.MWFrame+"\n</MWR>\n");  
        headpos += 4;  
        iv = new byte[4];  
        raf.read(iv);  
        f0 = ((int) iv[0] & 0x000000ff);  
        f1 = ((int) iv[1] << 8 & 0x0000ff00);  
        f2 = ((int) iv[2] << 16 & 0x00ff0000);  
        f3 = ((int) iv[3] << 24 & 0xff000000);  
        f0 = f3 | f2 | f1 | f0;  
        header.MHFrame = f0;  
        //20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.MHFrame+"\n</MWR>\n");  
        headpos += 4;  
        //3  
        iv = new byte[4];  
        raf.read(iv);  
        f0 = ((int) iv[0] & 0x000000ff);  
        f1 = ((int) iv[1] << 8 & 0x0000ff00);  
        f2 = ((int) iv[2] << 16 & 0x00ff0000);  
        f3 = ((int) iv[3] << 24 & 0xff000000);  
        f0 = f3 | f2 | f1 | f0;  
        header.MFrameLeangth = f0;  
        //20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.MFrameLeangth+"\n</MWR>\n");  
        headpos += 4;  
        //4  
        iv = new byte[4];  
        raf.read(iv);  
        f0 = (int) iv[0] & 0x000000ff;  
        f1 = (int) iv[1] << 8 & 0x0000ff00;  
        f2 = (int) iv[2] << 16 & 0x00ff0000;  
        f3 = (int) iv[3] << 24 & 0xff000000;
```

```
f0 = f3 | f2 | f1 | f0;
header.MFrameRate = f0;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.MFrameLeangth+"\n</MWR>\n");
headpos += 4;
//5
byte[] size = new byte[1];
raf.read(size);
headpos += 1;
if (size[0] != 0) {
    byte array[] = new byte[size[0] * 2];
    raf.read(array);
    char[] ch = new char[size[0]];
    for (int reg = 0; reg < size[0]; reg++) {
        int c0 = ((int) array[reg * 2] << 8 & 0xff00);
        int c1 = (int) array[reg * 2 + 1] & 0x00ff;
        ch[reg] = (char) (c1 | c0);
    }
    String se = new String(ch);
    header.SEn = new Encoding(se);
    //20230106-S_logger.Log.logger.info("" + "good");
    headpos += array.length;
}
//6
iv = new byte[4];
raf.read(iv);
f0 = ((int) iv[0] & 0x000000ff);
f1 = ((int) iv[1] << 8 & 0x0000ff00);
f2 = ((int) iv[2] << 16 & 0x00ff0000);
f3 = ((int) iv[3] << 24 & 0xff000000);
f0 = f3 | f2 | f1 | f0;
float fl = Float.intBitsToFloat(f0); //.floatToIntBits(IO.header.SSampleRate);
header.SSampleRate = fl;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SSampleRate+"\n</MWR>\n");
headpos += 4;
//7
iv = new byte[4];
raf.read(iv);
f0 = (int) iv[0] & 0x000000ff;
f1 = (int) iv[1] << 8 & 0x0000ff00;
f2 = (int) iv[2] << 16 & 0x00ff0000;
f3 = (int) iv[3] << 24 & 0xff000000;
f0 = f3 | f2 | f1 | f0;
header.SSampleSizeInBits = f0;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SSampleSizeInBits+"\n</MWR>\n");
headpos += 4;
//8
iv = new byte[4];
```

```
raf.read(iv);
f0 = (int) iv[0] & 0x000000ff;
f1 = (int) iv[1] << 8 & 0x0000ff00;
f2 = (int) iv[2] << 16 & 0x00ff0000;
f3 = (int) iv[3] << 24 & 0xff000000;
f0 = f3 | f2 | f1 | f0;
header.SChannels = f0;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SChannels+"\n</MWR>\n");
headpos += 4;
//9
iv = new byte[4];
raf.read(iv);
f0 = (int) iv[0] & 0x000000ff;
f1 = (int) iv[1] << 8 & 0x0000ff00;
f2 = (int) iv[2] << 16 & 0x00ff0000;
f3 = (int) iv[3] << 24 & 0xff000000;
f0 = f3 | f2 | f1 | f0;
header.SFrameSize = f0;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SFrameSize+"\n</MWR>\n");
headpos += 4;
//10
iv = new byte[4];
raf.read(iv);
f0 = ((int) iv[0] & 0x000000ff);
f1 = ((int) iv[1] << 8 & 0x0000ff00);
f2 = ((int) iv[2] << 16 & 0x00ff0000);
f3 = ((int) iv[3] << 24 & 0xff000000);
f0 = f3 | f2 | f1 | f0;
fl = Float.intBitsToFloat(f0);//.floatToIntBits(IO.header.SSampleRate);
header.SFrameRate = fl;
//20230106-S_logger.Log.logger.info("" + "<MWR>\n"+header.SFrameRate+"\n</MWR>\n");
headpos += 4;
//11
byte[] bein = new byte[1];
raf.read(bein);
if (bein[0] == 0) {
    header.SBigEndian = false;
} else {
    header.SBigEndian = true;
}
headpos += 1;
//12
byte[] audiopos = new byte[8];
raf.read(audiopos);
long l0 = ((long) audiopos[0] & 0x00000000000000ff);
long l1 = ((long) audiopos[1] & 0x00000000000000ff) << 8;
long l2 = ((long) audiopos[2] & 0x00000000000000ff) << 16;
```

```

    long l3 = ((long) audiopos[3] & 0x00000000000000ff) << 24;
    long l4 = ((long) audiopos[4] & 0x00000000000000ff) << 32;
    long l5 = ((long) audiopos[5] & 0x00000000000000ff) << 40;
    long l6 = ((long) audiopos[6] & 0x00000000000000ff) << 48;
    long l7 = ((long) audiopos[7] & 0x00000000000000ff) << 56;
    l0 = l7 | l6 | l5 | l4 | l3 | l2 | l1 | l0;
    header.SFrameLeangth = l0;

    headpos += 8;

    //13

    audiopos = new byte[16];
    raf.read(audiopos);

    headpos += 16;

    //14

    audiopos = new byte[8];
    raf.read(audiopos);

    l0 = ((long) audiopos[0] & 0x00000000000000ff);
    l1 = ((long) audiopos[1] & 0x00000000000000ff) << 8;
    l2 = ((long) audiopos[2] & 0x00000000000000ff) << 16;
    l3 = ((long) audiopos[3] & 0x00000000000000ff) << 24;
    l4 = ((long) audiopos[4] & 0x00000000000000ff) << 32;
    l5 = ((long) audiopos[5] & 0x00000000000000ff) << 40;
    l6 = ((long) audiopos[6] & 0x00000000000000ff) << 48;
    l7 = ((long) audiopos[7] & 0x00000000000000ff) << 56;
    l0 = l7 | l6 | l5 | l4 | l3 | l2 | l1 | l0;
    header.SWavPos = l0;

    headpos += 8;

    header.MovPos = headpos;

    duration = (double)(header.MFrameLeangth / header.MFrameRate);

    //20230106-S_logger.Log.logger.info("" + "end header get");
} catch (Exception e) {
    e.printStackTrace();
}
}

public void IV_ByAudioBytes(AudioFormat af) {
    //en /1sample rate /2samplesize /3 channels /4framesize /5 framrate /bigendianture
    adataFrame.audioIS = new BytestoAIS().getAIS(adataFrame.fftarray, af, adataFrame.audioIS);

    // TODO Auto - generated method stub
}
}

//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/LYGFileIOTest.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/ReadingLine.java
package I_A.IMS.EUQ.SP;

import ME.VPC.M.app.App;

import exception.thread.DetaThread;

import java.io.ByteArrayOutputStream;

import java.io.IOException;

import java.util.List;

import javax.sound.sampled.AudioFormat;
```



```
import javax.sound.sampled.AudioSystem;

import javax.sound.sampled.DataLine;

import javax.sound.sampled.TargetDataLine;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class ReadingLine extends Thread implements Runnable {
    public SoundWave soundWave;
    public TargetDataLine line;
    public AudioFormat.Encoding encoding;
    ByteArrayOutputStream out;
    byte[] data;//19200bytes++s
    int channels;
    int sampleSize;
    int numBytesRead;
    int bufferLengthInBytes;
    List<byte[]> temps;
    int maxPipeSize;
    boolean bigEndian;
    int frameSizeInBytes;
    int bufferLengthInFrames;
    App NE;
    String signedString = "signed";
    public ReadingLine(App app_NE) {
        NE = app_NE;
        line = null;
        channels = 1;
        //1 2 4 8 16 32 64 128
        int temp = 1 << (NE.app_S.sourcePageFR - 1);
        NE.app_S.rateWave = 32f * temp;//1024 * 32== 4 times 1024=
        sampleSize = 16;
        bigEndian = true;
        encoding = AudioFormat.Encoding.PCM_SIGNED;
    }
    public void run() {
        if (null == NE.app_S.line2025) {
            return;
        }
        this.soundWave = NE.app_S.line2025;
        while (NE.app_S.reg == 1) {
            //录音
```

```
try {
    temps = soundWave.temps;
    maxPipeSize = soundWave.maxPipeSize;
    soundWave.swn = new SoundWaveVectorNew();
    soundWave.format = new AudioFormat(encoding, NE.app_S.rateWave, sampleSize,
        channels, (sampleSize / 8) * channels, NE.app_S.rateWave, bigEndian);
    DetaThread.sleepDeta(1);
    DataLine.Info info = new DataLine.Info(TargetDataLine.class, soundWave.format);
    if (!AudioSystem.isLineSupported(info)) {
        S_logger.Log.logger.info("" + "-----Line-Unsupported-----");
        return;
    }
    line = (TargetDataLine) AudioSystem.getLine(info);
    line.open(soundWave.format, line.getBufferSize());
    frameSizeInBytes = soundWave.format.getFrameSize();
    bufferLengthInFrames = line.getBufferSize() / 8;
    bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    out = new ByteArrayOutputStream();
    DetaThread.sleepDeta(1);
    data = new byte[bufferLengthInBytes]; //19200bytes++s
    //audioData = new double[nlengthInSamples];
    numBytesRead = 0;
    line.start();
    while (NE.app_S.reg == 1 && temps.size() < maxPipeSize) {
        if (-1 == (numBytesRead = line.read(data, 0, bufferLengthInBytes))) {
            break;
        }
        out.reset();
        out.write(data, 0, numBytesRead);
        byte[] audioBytes = out.toByteArray();
        if (temps.size() < maxPipeSize) {
            temps.add(0, audioBytes);
        }
        DetaThread.sleepDeta(1);
    }
    // stop and close the output stream
    //later 去中断 演示中去fft计算展示来加速, 仅仅 slider 操作展示 fft和时函数高级计算
    line.stop();
    line.close();
    line = null;
    out.flush();
    out.close();
    //显示最大值
} catch (IOException e) {
    e.printStackTrace();
} catch (Exception ex) {
    ex.printStackTrace();
}
```

```

    } //显示波振幅

    return;
}

}

}

//later temps to linklistQ 20250107-0049

//一秒钟1小段， later

//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ReadingLineTest.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/Recorder.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ShowingLine.java
package I_A.IMS.EUQ.SP;

import ME.VPC.M.app.App;

import java.util.concurrent.CopyOnWriteArrayList;

/*
* 个人著作权人， 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class ShowingLine extends Thread implements Runnable {
    public SoundWave soundWave;

    App NE;

    public ShowingLine(App app_NE) {
        NE = app_NE;

        if (null != NE.app_S.line2025) {
            this.soundWave = NE.app_S.line2025;
        }
    }

    public CopyOnWriteArrayList<byte[]> temps;

    public int maxPipeSize = 1024;

    public void run() {
        while (NE.app_S.reg == 1) {
            //显示波振幅

            return;
        }
    }
}

//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/ShowingLineTest.java
//Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundPlay.java
package I_A.IMS.EUQ.SP;

import exception.thread.DetaThread;

import O_V.OSI.OSU.AVU.OSQ.wavePlay.Jp;

import U_A.PEU.P.movie.AdataFrame;

import U_A.PEU.P.movie.LYGFileIO;

import U_A.PEU.P.sound.BytestoAIS;

import U_A.PEU.P.wave.DFT;

import javax.sound.sampled.AudioFormat;

```

```
import javax.sound.sampled.AudioInputStream;

import javax.sound.sampled.AudioSystem;

import javax.sound.sampled.DataLine;

import javax.sound.sampled.LineUnavailableException;

import javax.sound.sampled.SourceDataLine;

import java.io.IOException;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

 */

public class SoundPlay implements Runnable {

    private LYGFileIO cur;

    private AdataFrame af;

    private Jp jp;

    @SuppressWarnings("unused")

    private Object bootSound;

    public SoundPlay() {

    }

    public void Play(AudioInputStream ais1) {

        if (ais1 == null) {

            return;

        }

        AudioFormat format = ais1.getFormat();

        AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, ais1);

        if (playbackInputStream == null) {

            return;

        }

        SourceDataLine line1;

        DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);

        try {

            line1 = (SourceDataLine) AudioSystem.getLine(info1);

            int frameSizeInBytes = ais1.getFormat().getFrameSize();

            int bufferLengthInFrames = line1.getBufferSize() / (1 * 2 * 2 * frameSizeInBytes);

            int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;

            //20230106-S_logger.Log.logger.info("" + "bufferLengthInBytes : " + bufferLengthInBytes + " *");

            byte[] data = new byte[bufferLengthInBytes];

            line1.open(format, line1.getBufferSize());

            int numBytesRead = 0;

            line1.start();

            while (true) {

                try {

                    if ((numBytesRead = ais1.read(data)) == -1) {
```

```
        break;
    }
    int numBytesRemaining = numBytesRead;
    while (numBytesRemaining > 0) {
        numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
        DetaThread.sleepDeta(1);
    }
} catch (Exception e) {
    break;
}
}
line1.stop();
line1.close();
line1 = null;
} catch (LineUnavailableException ex) {
    return;
}
}

public void Play(AdataFrame af) {
    if (af.audioIS == null) {
        return;
    }
    AudioFormat format = af.audioIS.getFormat();
    AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);
    if (playbackInputStream == null) {
        return;
    }
    SourceDataLine line1;
    DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
    try {
        line1 = (SourceDataLine) AudioSystem.getLine(info1);
        int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
        int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
        int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
        int bufSize = bufferLengthInBytes;
        line1.open(format, bufSize);
    } catch (LineUnavailableException ex) {
        return;
    }
    int frameSizeInBytes = format.getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    byte[] data = new byte[bufferLengthInBytes];
    int numBytesRead = 0;
    line1.start();
    try {
        af.audioIS.reset();
```

```
} catch (IOException e) {
    throw new RuntimeException(e);
}
while (true) {
    try { // try local ..
        if ((numBytesRead = af.audioIS.read(data)) == -1) {
            break;
        }
        int numBytesRemaining = numBytesRead;
        while (numBytesRemaining > 0) {
            numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
        }
    } catch (Exception e) {
        break;
    }
}
while (af.next != null) {
    af = af.next;
    try {
        af.audioIS.reset();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
    while (true) {
        try {
            if ((numBytesRead = af.audioIS.read(data)) == -1) {
                break;
            }
            int numBytesRemaining = numBytesRead;
            while (numBytesRemaining > 0) {
                numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
            }
        } catch (Exception e) {
            break;
        }
    }
}
line1.stop();
line1.close();
line1 = null;
}

public void Play(byte[] secbytesarray, AudioFormat af, SourceDataLine line1, int sp, byte[] data) throws LineUnavailableException {
    for (int i = 0; i < sp; i++) {
        for (int j = 0; j < data.length; j++) {
            data[j] = secbytesarray[j + i * data.length];
        }
        line1.write(data, 0, data.length);
    }
}
```

```
}  
}  
@SuppressWarnings("unused")  
public void Play1(AudioInputStream ais1, AudioFormat af) {  
    SourceDataLine line1;  
    DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, af);  
    try {  
        line1 = (SourceDataLine) AudioSystem.getLine(info1);  
        int bufSize = line1.getBufferSize() / 8;  
        line1.open(af, bufSize);  
    } catch (LineUnavailableException ex) {  
        return;  
    }  
    int bufSize = line1.getBufferSize() / 8;  
    byte[] data = new byte[bufSize];  
    int numBytesRead = 0;  
    line1.start();  
    byte all[] = new byte[(int) (ais1.getFrameLength() * 2)];  
    try { //later  
        ais1.read(all);  
    } catch (IOException e) {  
        throw new RuntimeException(e);  
    }  
    int sp = all.length / data.length;  
    int pos = 0;  
    for (int i = 0; i < sp; i++) {  
        for (int j = 0; j < data.length; j++) {  
            data[j] = all[j + i * data.length];  
        }  
        int numBytesRemaining = data.length; //343  
        while (numBytesRemaining > 0) {  
            numBytesRemaining -= line1.write(data, 0, numBytesRemaining);  
        }  
    }  
    line1.stop();  
    line1.close();  
    line1 = null;  
}  
public void PlayArray(AdataFrame af) {  
    if (af.audioIS == null) {  
        return;  
    }  
    AudioInputStream ais = new BytestoAIS().getAIS(af.audioArray, af.audioIS);  
    AudioFormat format = af.audioIS.getFormat();  
    AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);  
    if (playbackInputStream == null) {  
        return;  
    }  
}
```

```
}

SourceDataLine line1;

DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);

try {
    line1 = (SourceDataLine) AudioSystem.getLine(info1);
    int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    int bufSize = bufferLengthInBytes;
    line1.open(format, bufSize);
} catch (LineUnavailableException ex) {
    return;
}

int frameSizeInBytes = format.getFrameSize();
int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
byte[] data = new byte[bufferLengthInBytes];
int numBytesRead = 0;
line1.start();
while (true) {
    try {
        if ((numBytesRead = ais.read(data)) == -1) {
            break;
        }
        int numBytesRemaining = numBytesRead;
        while (numBytesRemaining > 0) {
            numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
        }
    } catch (Exception e) {
        break;
    }
}

while (af.next != null) {
    af = af.next;
    ais = new ByteToAIS().getAIS(af.audioArray, af.audioIS);
    while (true) {
        try {
            if ((numBytesRead = ais.read(data)) == -1) {
                break;
            }
            int numBytesRemaining = numBytesRead;
            while (numBytesRemaining > 0) {
                numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
            }
        } catch (Exception e) {
            break;
        }
    }
}
```



```
    }
}

line1.stop();
line1.close();
line1 = null;
}

public void PlayMasterArray(AdataFrame af) {
    if (af.audioIS == null) {
        return;
    }
    double master[] = new double[af.master.length];
    int psita = 0;
    for (int i = 2; i < master.length - 2; i++) {
        psita = 0;
        for (int p = -2; p < 3; p++) {
            psita += (af.master[i + p]);
        }
        master[i] = psita / 5.0;
    }
    AudioInputStream ais = new BytestoAIS().getAIS(master, af.audioIS);
    AudioFormat format = af.audioIS.getFormat();
    AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);
    if (playbackInputStream == null) {
        return;
    }
    SourceDataLine line1;
    DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
    try {
        line1 = (SourceDataLine) AudioSystem.getLine(info1);
        int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
        int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
        int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
        int bufSize = bufferLengthInBytes;
        line1.open(format, bufSize);
    } catch (LineUnavailableException ex) {
        return;
    }
    int frameSizeInBytes = format.getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    byte[] data = new byte[bufferLengthInBytes];
    int numBytesRead = 0;
    line1.start();
    while (true) {
        try {
            if ((numBytesRead = ais.read(data)) == -1) {
                break;
            }
        }
    }
}
```

```

    }
    int numBytesRemaining = numBytesRead;
    while (numBytesRemaining > 0) {
        numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
    }
} catch (Exception e) {
    break;
}
}
while (af.next != null) {
    af = af.next;
    psita = 0;
    for (int i = 2; i < master.length - 2; i++) {
        //find high
        psita = 0;
        for (int p = -2; p < 3; p++) {
            psita += (af.master[i + p]);
        }
        master[i] = psita / 5.0;
    }
    ais = new BytestoAIS().getAIS(master, af.audioIS);
    while (true) {
        try {
            if ((numBytesRead = ais.read(data)) == -1) {
                break;
            }
            int numBytesRemaining = numBytesRead;
            while (numBytesRemaining > 0) {
                numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
            }
        } catch (Exception e) {
            break;
        }
    }
}
line1.stop();
line1.close();
line1 = null;
}
public void PlaySlaveArray(AdataFrame af) {
    if (af.audioIS == null) {
        return;
    }
    double slave[] = new double[af.slave.length];
    int psita = 0;
    for (int i = 2; i < slave.length - 2; i++) {
        //find high

```

```
    psita = 0;
    for (int p = -2; p < 3; p++) {
        psita += (af.slave[i + p]);
    }
    slave[i] = psita / 5.0;
}

AudioInputStream ais = new BytestoAIS().getAIS(slave, af.audioIS);
AudioFormat format = af.audioIS.getFormat();
AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);
if (playbackInputStream == null) {
    return;
}

SourceDataLine line1;
DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
try {
    line1 = (SourceDataLine) AudioSystem.getLine(info1);
    int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    int bufSize = bufferLengthInBytes;
    line1.open(format, bufSize);
} catch (LineUnavailableException ex) {
    return;
}

int frameSizeInBytes = format.getFrameSize();
int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
byte[] data = new byte[bufferLengthInBytes];
int numBytesRead = 0;
line1.start();
while (true) {
    try {
        if ((numBytesRead = ais.read(data)) == -1) {
            break;
        }
        int numBytesRemaining = numBytesRead;
        while (numBytesRemaining > 0) {
            numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
        }
    } catch (Exception e) {
        break;
    }
}

while (af.next != null) {
    af = af.next;
    psita = 0;
    for (int i = 2; i < slave.length - 2; i++) {
```

```

    //find high
    psita = 0;
    for (int p = -2; p < 3; p++) {
        psita += (af.slave[i + p]);
    }
    slave[i] = psita / 5.0;
}

ais = new BytestoAIS().getAIS(slave, af.audioIS);
while (true) {
    try {
        if ((numBytesRead = ais.read(data)) == -1) {
            break;
        }
        int numBytesRemaining = numBytesRead;
        while (numBytesRemaining > 0) {
            numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
        }
    } catch (Exception e) {
        break;
    }
}

line1.stop();
line1.close();
line1 = null;
}

public void PlayFftArray(AdataFrame af, LYGFileIO cur, Jp jp, Object bootSound) throws InterruptedException {
    if (af.audioIS == null) {
        return;
    }
    this.cur = cur;
    this.af = af;
    this.jp = jp;
    this.bootSound = bootSound;
}

@Override
public void run() {
    while (jp.reg == 5) {
        DetaThread.sleepDeta(10);
        DFT dFT = new DFT();
        dFT.IV_(1024);
        double[] fft = new double[1024]; // 内自应体 «-» 核糖体不一致性，分区域应用。
        for (int i = 0; i < fft.length; i++) {
            fft[i] = 0.0;
        }
        fft[2] = Double.valueOf(cur.makeContainerZNSZ.source2 << 5);
        fft[4] = Double.valueOf(cur.makeContainerZNSZ.source3 << 5);
    }
}

```

```

fft[8] = Double.valueOf(cur.makeContainerZNSZ.source4 << 5);
fft[16] = Double.valueOf(cur.makeContainerZNSZ.source5 << 5);
fft[32] = Double.valueOf(cur.makeContainerZNSZ.source6 << 5);
fft[32] = Double.valueOf(cur.makeContainerZNSZ.source7 << 5);//无效先
fft[64] = Double.valueOf(cur.makeContainerZNSZ.source8 << 5);
fft[128] = Double.valueOf(cur.makeContainerZNSZ.source16 << 5);
fft[200] = Double.valueOf(cur.makeContainerZNSZ.source32 << 5);
fft[256] = Double.valueOf(cur.makeContainerZNSZ.source64 << 5);
fft[300] = Double.valueOf(cur.makeContainerZNSZ.source128 << 5);
fft[400] = Double.valueOf(cur.makeContainerZNSZ.source256 << 5);
fft[512] = Double.valueOf(cur.makeContainerZNSZ.source512 << 5);
double[] array = dFT.ft(fft);
//for(int i=0;i<5;i++) {
AudioInputStream ais = new ByteToAIS().getAIS(array, af.audioIS);
AudioFormat format = af.audioIS.getFormat();
AudioInputStream playbackInputStream
    = AudioSystem.getAudioInputStream(format, af.audioIS);
if (playbackInputStream == null) {
    return;
}
SourceDataLine line1;
DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
try {
    line1 = (SourceDataLine) AudioSystem.getLine(info1);
    int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    int bufSize = bufferLengthInBytes;
    line1.open(format, bufSize);
} catch (LineUnavailableException ex) {
    return;
}
int frameSizeInBytes = format.getFrameSize();
int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
byte[] data = new byte[bufferLengthInBytes];
byte[] initData = new byte[bufferLengthInBytes];
boolean firstTime = true;
int numBytesRead = 0;
line1.start();
while (jp.reg == 5) {
    try {
        if (firstTime) {
            if ((numBytesRead = ais.read(data)) == -1) {
                break;
            }
        }
        initData = data.clone();
    }
}

```

```

        firstTime = !firstTime;
    } else {
        data = initData.clone();
    }

    int numBytesRemaining = numBytesRead;
    while (numBytesRemaining > 0) {
        DetaThread.sleepDeta(2);
        numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
    }
} catch (Exception e) {
    break;
}
}
line1.stop();
line1.close();
}
// }
}
}

/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundPlaying.java
package I_A.IMS.EUQ.SP;

import O_V.OSI.OSU.AVU.OSQ.wavePlay.Jp;

import exception.thread.DetaThread;

import U_A.PEU.P.movie.AdataFrame;

import U_A.PEU.P.movie.LYGFileIO;

import U_A.PEU.P.sound.BytestoAIS;

import U_A.PEU.P.wave.DFT;

import javax.sound.sampled.AudioFormat;

import javax.sound.sampled.AudioInputStream;

import javax.sound.sampled.AudioSystem;

import javax.sound.sampled.DataLine;

import javax.sound.sampled.LineUnavailableException;

import javax.sound.sampled.SourceDataLine;

import java.io.IOException;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 * *///, 基于 java sound demo JDK 官方 demo 写的

public class SoundPlaying implements Runnable {
    private LYGFileIO cur;
    private AdataFrame af;
    private Jp jp;

    @SuppressWarnings("unused")

```

```
private Object bootSound;

public SoundPlaying() {
}

public void Play(AudioInputStream ais1) {
    if (ais1 == null) {
        return;
    }
    AudioFormat format = ais1.getFormat();
    AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, ais1);
    if (playbackInputStream == null) {
        return;
    }
    SourceDataLine line1;
    DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
    try {
        line1 = (SourceDataLine) AudioSystem.getLine(info1);
        int frameSizeInBytes = ais1.getFormat().getFrameSize();
        int bufferLengthInFrames = line1.getBufferSize() / (1 * 2 * 2 * frameSizeInBytes);
        int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
        //20230106-S_logger.Log.logger.info("" + "bufferLengthInBytes : "+ bufferLengthInBytes + " *");
        byte[] data = new byte[bufferLengthInBytes];
        line1.open(format, line1.getBufferSize());
        int numBytesRead = 0;
        line1.start();
        while (true) {
            try {
                if ((numBytesRead = ais1.read(data)) == -1) {
                    break;
                }
                int numBytesRemaining = numBytesRead;
                while (numBytesRemaining > 0) {
                    numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
                    DataThread.sleep(Deta(1));
                }
            } catch (Exception e) {
                break;
            }
        }
        line1.stop();
        line1.close();
        line1 = null;
    } catch (LineUnavailableException ex) {
        return;
    }
}

public void Play(AdataFrame af) {
    if (af.audioIS == null) {
```

```
        return;
    }

    AudioFormat format = af.audioIS.getFormat();
    AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);
    if (playbackInputStream == null) {
        return;
    }

    SourceDataLine line1;
    DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
    try {
        line1 = (SourceDataLine) AudioSystem.getLine(info1);
        int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
        int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
        int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
        int bufSize = bufferLengthInBytes;
        line1.open(format, bufSize);
    } catch (LineUnavailableException ex) {
        return;
    }

    int frameSizeInBytes = format.getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    byte[] data = new byte[bufferLengthInBytes];
    int numBytesRead = 0;
    line1.start();
    try {
        af.audioIS.reset();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }

    while (true) {
        try {
            if ((numBytesRead = af.audioIS.read(data)) == -1) {
                break;
            }

            int numBytesRemaining = numBytesRead;
            while (numBytesRemaining > 0) {
                numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
            }
        } catch (Exception e) {
            break;
        }
    }

    while (af.next != null) {
        af = af.next;
        try {
            af.audioIS.reset();
        }
```



```
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    while (true) {
        try {
            if ((numBytesRead = af.audioIS.read(data)) == -1) {
                break;
            }
            int numBytesRemaining = numBytesRead;
            while (numBytesRemaining > 0) {
                numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
            }
        } catch (Exception e) {
            break;
        }
    }
}
line1.stop();
line1.close();
line1 = null;
}

public void Play(byte[] secbytesarray, AudioFormat af, SourceDataLine line1, int sp, byte[] data) throws LineUnavailableException {
    for (int i = 0; i < sp; i++) {
        for (int j = 0; j < data.length; j++) {
            data[j] = secbytesarray[j + i * data.length];
        }
        line1.write(data, 0, data.length);
    }
}

@SuppressWarnings("unused")
public void Play1(AudioInputStream ais1, AudioFormat af) {
    SourceDataLine line1;
    DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, af);
    try {
        line1 = (SourceDataLine) AudioSystem.getLine(info1);
        int bufSize = line1.getBufferSize() / 8;
        line1.open(af, bufSize);
    } catch (LineUnavailableException ex) {
        return;
    }
    int bufSize = line1.getBufferSize() / 8;
    byte[] data = new byte[bufSize];
    int numBytesRead = 0;
    line1.start();
    byte all[] = new byte[(int) (ais1.getFrameLength() * 2)];
    try {
        ais1.read(all);
```

```
} catch (IOException e) {
    throw new RuntimeException(e);
}

int sp = all.length / data.length;
int pos = 0;
for (int i = 0; i < sp; i++) {
    for (int j = 0; j < data.length; j++) {
        data[j] = all[j + i * data.length];
    }
    int numBytesRemaining = data.length; // 343
    while (numBytesRemaining > 0) {
        numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
    }
}
line1.stop();
line1.close();
line1 = null;
}

public void PlayArray(AdataFrame af) {
    if (af.audioIS == null) {
        return;
    }
    AudioInputStream ais = new BytestoAIS().getAIS(af.audioArray, af.audioIS);
    AudioFormat format = af.audioIS.getFormat();
    AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);
    if (playbackInputStream == null) {
        return;
    }
    SourceDataLine line1;
    DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
    try {
        line1 = (SourceDataLine) AudioSystem.getLine(info1);
        int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
        int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
        int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
        int bufSize = bufferLengthInBytes;
        line1.open(format, bufSize);
    } catch (LineUnavailableException ex) {
        return;
    }
    int frameSizeInBytes = format.getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    byte[] data = new byte[bufferLengthInBytes];
    int numBytesRead = 0;
    line1.start();
    while (true) {
```

```
try {
    if ((numBytesRead = ais.read(data)) == -1) {
        break;
    }
    int numBytesRemaining = numBytesRead;
    while (numBytesRemaining > 0) {
        numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
    }
} catch (Exception e) {
    break;
}
}

while (af.next != null) {
    af = af.next;
    ais = new BytestoAIS().getAIS(af.audioArray, af.audioIS);
    while (true) {
        try {
            if ((numBytesRead = ais.read(data)) == -1) {
                break;
            }
            int numBytesRemaining = numBytesRead;
            while (numBytesRemaining > 0) {
                numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
            }
        } catch (Exception e) {
            break;
        }
    }
}

line1.stop();
line1.close();
line1 = null;
}

public void PlayMasterArray(AdataFrame af) {
    if (af.audioIS == null) {
        return;
    }
    double master[] = new double[af.master.length];
    int psita = 0;
    for (int i = 2; i < master.length - 2; i++) {
        psita = 0;
        for (int p = -2; p < 3; p++) {
            psita += (af.master[i + p]);
        }
        master[i] = psita / 5.0;
    }
    AudioInputStream ais = new BytestoAIS().getAIS(master, af.audioIS);
```

```

AudioFormat format = af.audioIS.getFormat();

AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);

if (playbackInputStream == null) {
    return;
}

SourceDataLine line1;

DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);

try {
    line1 = (SourceDataLine) AudioSystem.getLine(info1);
    int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    int bufSize = bufferLengthInBytes;
    line1.open(format, bufSize);
} catch (LineUnavailableException ex) {
    return;
}

int frameSizeInBytes = format.getFrameSize();
int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
byte[] data = new byte[bufferLengthInBytes];
int numBytesRead = 0;
line1.start();
while (true) {
    try {
        if ((numBytesRead = ais.read(data)) == -1) {
            break;
        }
        int numBytesRemaining = numBytesRead;
        while (numBytesRemaining > 0) {
            numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
        }
    } catch (Exception e) {
        break;
    }
}

while (af.next != null) {
    af = af.next;
    psita = 0;
    for (int i = 2; i < master.length - 2; i++) {
        //find high
        psita = 0;
        for (int p = -2; p < 3; p++) {
            psita += (af.master[i + p]);
        }
        master[i] = psita / 5.0;
    }
}

```

```
ais = new BytestoAIS().getAIS(master, af.audioIS);
while (true) {
    try {
        if ((numBytesRead = ais.read(data)) == -1) {
            break;
        }
        int numBytesRemaining = numBytesRead;
        while (numBytesRemaining > 0) {
            numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
        }
    } catch (Exception e) {
        break;
    }
}
line1.stop();
line1.close();
line1 = null;
}

public void PlaySlaveArray(AdataFrame af) {
    if (af.audioIS == null) {
        return;
    }
    double slave[] = new double[af.slave.length];
    int psita = 0;
    for (int i = 2; i < slave.length - 2; i++) {
        //find high
        psita = 0;
        for (int p = -2; p < 3; p++) {
            psita += (af.slave[i + p]);
        }
        slave[i] = psita / 5.0;
    }
    AudioInputStream ais = new BytestoAIS().getAIS(slave, af.audioIS);
    AudioFormat format = af.audioIS.getFormat();
    AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);
    if (playbackInputStream == null) {
        return;
    }
    SourceDataLine line1;
    DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
    try {
        line1 = (SourceDataLine) AudioSystem.getLine(info1);
        int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
        int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
        int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
        int bufSize = bufferLengthInBytes;
```

```
line1.open(format, bufSize);
} catch (LineUnavailableException ex) {
    return;
}
int frameSizeInBytes = format.getFrameSize();
int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
byte[] data = new byte[bufferLengthInBytes];
int numBytesRead = 0;
line1.start();
while (true) {
    try {
        if ((numBytesRead = ais.read(data)) == -1) {
            break;
        }
        int numBytesRemaining = numBytesRead;
        while (numBytesRemaining > 0) {
            numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
        }
    } catch (Exception e) {
        break;
    }
}
while (af.next != null) {
    af = af.next;
    psita = 0;
    for (int i = 2; i < slave.length - 2; i++) {
        //find high
        psita = 0;
        for (int p = -2; p < 3; p++) {
            psita += (af.slave[i + p]);
        }
        slave[i] = psita / 5.0;
    }
    ais = new BytestoAIS().getAIS(slave, af.audioIS);
    while (true) {
        try {
            if ((numBytesRead = ais.read(data)) == -1) {
                break;
            }
            int numBytesRemaining = numBytesRead;
            while (numBytesRemaining > 0) {
                numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
            }
        } catch (Exception e) {
            break;
        }
    }
}
```

```

    }
}

line1.stop();
line1.close();
line1 = null;
}

public void PlayFftArray(AdataFrame af, LYGFileIO cur, Jp jp, Object bootSound)
    throws InterruptedException {
    if (af.audioIS == null) {
        return;
    }
    this.cur = cur;
    this.af = af;
    this.jp = jp;
    this.bootSound = bootSound;
}

@Override
public void run() {
    while (jp.reg == 5) {
        DetaThread.sleepDeta(10);
        DFT dFT = new DFT();
        dFT.IV_(1024);
        double[] fft = new double[1024];
        fft[2] = (double) (cur.makeContainerZNSZ.source2 << 5);
        fft[3] = (double) (cur.makeContainerZNSZ.source3 << 5);
        fft[4] = (double) (cur.makeContainerZNSZ.source4 << 5);
        fft[5] = (double) (cur.makeContainerZNSZ.source5 << 5);
        fft[6] = (double) (cur.makeContainerZNSZ.source6 << 5);
        fft[7] = (double) (cur.makeContainerZNSZ.source7 << 5);
        fft[8] = (double) (cur.makeContainerZNSZ.source8 << 5);
        fft[16] = (double) (cur.makeContainerZNSZ.source16 << 5);
        fft[32] = (double) (cur.makeContainerZNSZ.source32 << 5);
        fft[64] = (double) (cur.makeContainerZNSZ.source64 << 5);
        fft[128] = (double) (cur.makeContainerZNSZ.source128 << 5);
        fft[256] = (double) (cur.makeContainerZNSZ.source256 << 5);
        fft[512] = (double) (cur.makeContainerZNSZ.source512 << 5);
        double[] array = dFT.ft(fft);
        //for(int i=0;i<5;i++) {
        AudioInputStream ais = new BytestoAIS().getAIS(array, af.audioIS);
        AudioFormat format = af.audioIS.getFormat();
        AudioInputStream playbackInputStream = AudioSystem.getAudioInputStream(format, af.audioIS);
        if (playbackInputStream == null) {
            return;
        }
        SourceDataLine line1;
        DataLine.Info info1 = new DataLine.Info(SourceDataLine.class, format);
        try {

```

```

        line1 = (SourceDataLine) AudioSystem.getLine(info1);
        int frameSizeInBytes = af.audioIS.getFormat().getFrameSize();
        int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
        int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
        int bufSize = bufferLengthInBytes;
        line1.open(format, bufSize);
    } catch (LineUnavailableException ex) {
        return;
    }
    int frameSizeInBytes = format.getFrameSize();
    int bufferLengthInFrames = line1.getBufferSize() / (8 * frameSizeInBytes);
    int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
    byte[] data = new byte[bufferLengthInBytes];
    byte[] initData = new byte[bufferLengthInBytes];
    boolean firstTime = true;
    int numBytesRead = 0;
    line1.start();
    while (jp.reg == 5) {
        DetaThread.sleepDeta(10);
        try {
            if (firstTime) {
                if ((numBytesRead = ais.read(data)) == -1) {
                    break;
                }
                initData = data.clone();
                firstTime = !firstTime;
            } else {
                data = initData.clone();
            }
            int numBytesRemaining = numBytesRead;
            while (numBytesRemaining > 0) {
                DetaThread.sleepDeta(1);
                numBytesRemaining -= line1.write(data, 0, numBytesRemaining);
            }
        } catch (Exception e) {
            break;
        }
    }
    line1.stop();
    line1.close();
}
// }
}
}

```

```

//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundPlayingTest.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundPlayTest.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWave.java
package L_A.IMS.EUQ.SP;

```

```
import ME.VPC.M.app.App;
```



```
import M_V.MVQ.button.DetaButton;

import S_A.pheromone.IMV_SQI;

import U_A.PEU.P.movie.LYGFileIO;

import U_A.PEU.P.sound.SoundRecord;

import jniFFT.LYGFFT;

import javax.sound.sampled.AudioFormat;

import javax.swing.JButton;

import javax.swing.JPanel;

import java.awt.geom.Line2D.Double;

import java.awt.image.BufferedImage;

import java.util.List;

import java.util.Vector;

import java.util.concurrent.CopyOnWriteArrayList;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)

 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

 * 208号 阳光家园别墅小区 第十栋别墅 第三层

 **///, 基于 java sound demo JDK 官方 demo 写的

public class SoundWave extends JPanel {

    public O_V.OSI.OSU.AVU.OSQ.wavePlay.Jp waveJpanel;

    public App NE;

    public boolean isRecoding = false;

    public int maxPipeSize = 512;

    public int type = 0;

    public int frameSizeInBytes;

    public int bufferLengthInFrames;

    public int bufferLengthInBytes;

    //稍后过滤private

    public LYGFileIO cur;

    public Thread t1;

    public JButton j;

    public JButton j4;

    public DetaButton detaButton;

    public LYGFFT fFT;

    public Thread time;

    public SoundRecord rec;

    public BufferedImage imageForOutput = new BufferedImage(1400

        , 700, BufferedImage.TYPE_INT_RGB);

    public AudioFormat format;

    public SoundWaveVector sw;

    public SoundWaveVectorNew swn;

    public Vector<Double> lines;
```

```

public Vector<java.lang.Float> forOutput;

public IMV_SQI cache_sj = new IMV_SQI();

public IMV_SQI cache_bz = new IMV_SQI();

public List<byte[]> temps = new CopyOnWriteArrayList<>();
}

//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWaveJPanel.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWaveJPanelNew.java
package L_A.IMS.EUQ.SP;

import ME.VPC.M.app.App;

import M_V.MVQ.button.DetaButton;

import M_V.MVQ.filenameFilter.TXTFilter;

import U_A.PEU.P.movie.LYGFileIO;

import U_A.PEU.P.sound.SoundRecord;

import jniFFT.LYGFFT;

import javax.imageio.ImageIO;

import javax.swing.JButton;

import javax.swing.JTextPane;

import java.awt.Color;

import java.awt.Container;

import java.awt.FileDialog;

import java.awt.Frame;

import java.awt.Graphics2D;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.awt.image.BufferedImage;

import java.io.File;

import java.io.IOException;

import java.util.Iterator;

import java.util.Vector;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 **///, 基于 java sound demo JDK 官方 demo 写的
public class SoundWaveJPanelNew extends SoundWave implements Runnable {
    ///
    public SoundWaveJPanelNew(Container jpanelFourth, LYGFFT fft, JTextPane text
        , BufferedImage imageForOutput, App app_NE) {
        NE = app_NE;
        this.imageForOutput = imageForOutput;
        this.fft = fft;
        this.setBackground(new Color(100, 100, 100));
        forOutput = new Vector<java.lang.Float>();

```

```
rec = new SoundRecord(null);

double[] fft = new double[1024];

for (int regi = 0; regi < fft.length; regi++) {
    fft[regi] = 0.0;
}

fft[4] = 500.0;

double[] rooms = new double[1024];

for (int regi = 0; regi < fft.length; regi++) {
    rooms[regi] = 0.0;
}

LYGFileIO IYGFileIO = new LYGFileIO().initWithFFT(rooms, NE);

waveJpanel = new O_V.OSI.OSU.AVU.OSQ.wavePlayJp(IYGFileIO);

JButton j = new DetaButton("录音", 100 - 20, 50, Color.pink);

j.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 1;
        type = 0;
        t1 = new Thread(SoundWaveJPanelNew.this);
        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        jpanelFourth.validate();
    }
});

j.setSize(100, 30);

j.setBounds(500, 20, 100, 30);

detaButton = new DetaButton("关闭", 100 - 20, 50, Color.black);

detaButton.setForeground(Color.black);

detaButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 0;
        if (null != waveJpanel) {
            waveJpanel.reg = 0;
        }
        if (null != jpanelFourth) {
            jpanelFourth.validate();
        }
    }
});

detaButton.setSize(100, 30);

JButton j2 = new DetaButton("生成数列", 100 - 20, 50, Color.green);

j2.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 2;
        if (forOutput == null) {
            return;
        }
        if (forOutput.size() <= 0) {
```

```
        return;
    }
    if (text.getText().length() > 5000) {
        return;
    }
    StringBuilder page = new StringBuilder();
    Iterator<java.lang.Float> iterator = forOutput.iterator();
    int i = 0;
    while (iterator.hasNext()) {
        double setOfi = iterator.next();
        page.append(" " + (i++) + ":" + setOfi);
    }
    if (!text.getText().isEmpty()) {
        text.setText(text.getText() + "\r\n\r\n" + page.toString());
    } else {
        text.setText(page.toString());
    }
    text.validate();
}
});
j2.setSize(100, 30);
JButton j21 = new JButton("读取数列", 100 - 20, 50, Color.green);
j21.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 2;
        if (forOutput == null) {
            return;
        }
        if (forOutput.size() <= 0) {
            return;
        }
        if (text.getText().length() > 5000) {
            return;
        }
        StringBuilder page = new StringBuilder();
        Iterator<java.lang.Float> iterator = forOutput.iterator();
        int i = 0;
        while (iterator.hasNext()) {
            double setOfi = iterator.next();
            page.append(" " + (i++) + ":" + setOfi);
        }
        if (!text.getText().isEmpty()) {
            text.setText(text.getText() + "\r\n\r\n" + page.toString());
        } else {
            text.setText(page.toString());
        }
        text.validate();
    }
});
```

```
}  
});  
j21.setSize(100, 30);  
JButton j3 = new JButton("存为图片", 100 - 20, 50, Color.magenta);  
j3.addActionListener(new ActionListener() {  
    @SuppressWarnings("unused")  
    public void actionPerformed(ActionEvent e) {  
        NE.app_S.reg = 0;  
        //获取图片  
        //弹出保存框  
        if (null == imageForOutput) {  
            return;  
        }  
        //get path  
        FileDialog filedialog = new FileDialog(new Frame(), "导入到这里", FileDialog.LOAD);  
        filedialog.setFilenameFilter(new TXTFilter("png"));  
        filedialog.setVisible(true);  
        String filepath = filedialog.getDirectory() + filedialog.getFile();  
        //20230106-S_logger.Log.logger.info("" + filepath);  
        try {  
            File outputBin = new File(filepath + ".png");  
            if (null == imageForOutput) {  
                return;  
            }  
            ImageIO.write(imageForOutput, "png", outputBin);  
        } catch (IOException writePngError) {  
            //20230106-S_logger.Log.logger.info("" + filepath);  
        }  
        //输出图片  
    }  
});  
j3.setSize(100, 30);  
JButton j4 = new JButton("高敏噪音", 100 - 20, 50, Color.black);  
j4.setForeground(Color.black);  
j4.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        if (null != waveJpanel) {  
            waveJpanel.validate();  
            waveJpanel.updateUI();  
            validate();  
            updateUI();  
            NE.app_S.jTabbedPane.validate();  
            NE.app_S.jTabbedPane.updateUI();  
        }  
        NE.app_S.reg = 1;  
        type = 1;  
        t1 = new Thread(SoundWaveJPanelNew.this);
```

```
t1.setPriority(Thread.MIN_PRIORITY);
t1.start();
jpanelFourth.validate();
}
});
j4.setSize(100, 30);
JButton j5 = new JButton("读取图片", 100 - 20, 50, Color.black);
j5.setForeground(Color.black);
j5.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 1;
        type = 1;
        t1 = new Thread(SoundWaveJPanelNew.this);
        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        jpanelFourth.validate();
    }
});
j5.setSize(100, 30);
JButton j51 = new JButton("调制解调", 100 - 20, 50, Color.black);
j51.setForeground(Color.black);
j51.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 1;
        type = 1;
        //new fft[];
        //double[] fft= new double[1024];
        //fft[4]= 500;
        //LYGFileIO IYGFileIO= new LYGFileIO().initWithFFT(new double[1024]
        //, makeContainerZNSZ);
        //waveJpanel= new org.LYG.node.sound.wavePlay.jp(LYGFileIO);
        waveJpanel.reg = 5;
        t1 = new Thread(waveJpanel);
        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        ;
        jpanelFourth.validate();
    }
});
j51.setSize(100, 30);
JButton j6 = new JButton("山林", 100 - 10, 50, Color.green);
j6.setForeground(Color.black);
j6.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 1;
        type = 1;
        t1 = new Thread(SoundWaveJPanelNew.this);
```

```
t1.setPriority(Thread.MIN_PRIORITY);
t1.start();
jpanelFourth.validate();
}
});
j6.setSize(100, 30);
DetaButton j7 = new DetaButton("海边", 100 - 20, 50, Color.blue);
j7.setForeground(Color.black);
j7.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 1;
        type = 1;
        t1 = new Thread(SoundWaveJPanelNew.this);
        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        jpanelFourth.validate();
    }
});
j7.setSize(100, 30);
JButton j8 = new DetaButton("美梦", 100 - 20, 50, Color.magenta);
j8.setForeground(Color.black);
j8.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 1;
        type = 1;
        t1 = new Thread(SoundWaveJPanelNew.this);
        t1.setPriority(Thread.MIN_PRIORITY);
        t1.start();
        jpanelFourth.validate();
    }
});
j8.setSize(100, 30);
this.add(j);
this.add(detaButton);
this.add(j2);
this.add(j21);
this.add(j3);
this.add(j5);
this.add(j4);
this.add(j51);
this.add(j6);
this.add(j7);
this.add(j8);
NE.app_S.jp3n = new SoundWavePanelVectorNew(imageForOutput.getGraphics());
NE.app_S.jp3n.setBounds(100, 0, 400, 400);
NE.app_S.jp3n.setVisible(true);
this.add(NE.app_S.jp3n);
```

```

}

public void run() {
    if (1 == NE.app_S.reg) {
        Graphics2D g = (Graphics2D) NE.app_S.jp3n.getGraphics();
        g.setBackground(new Color(110, 110, 110));
        g.setColor(new Color(110, 110, 110));
        g.clearRect(0, 40, 1400, 200);
        new ReadingLine(NE).start();
        ;
        // new ComputingLine(this).start();//2025
        ;
        new ShowingLine(NE).start();
        ;
    }
}

}

//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWaveJPanelNewTest.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWaveJPanelTest.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWaveJPanelTime.java
package I_A.IMS.EUQ.SP;

import ME.VPC.M.app.App;
import M_V.MVQ.button.DetaButton;
import M_V.MVQ.filenameFilter.TXTFilter;
import U_A.PEU.P.movie.LYGFileIO;
import U_A.PEU.P.sound.SoundRecord;
import jniFFT.LYGFFT;
import javax.imageio.ImageIO;
import javax.swing.JButton;
import javax.swing.JTextPane;
import java.awt.Color;
import java.awt.Container;
import java.awt.FileDialog;
import java.awt.Frame;
import java.awt.Graphics2D;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;
import java.util.Iterator;
import java.util.Vector;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

```


*/**, 基于 java sound demo JDK 官方 demo 写的

```
public class SoundWaveJPanelTime extends SoundWave implements Runnable {  
    //  
    public SoundWaveJPanelTime(Container jpanelFourth, LYGFFT fft, JTextPane text  
        , BufferedImage imageForOutput, App app_NE) {  
        NE = app_NE;  
        this.imageForOutput = imageForOutput;  
        this.fft = fft;  
        this.setBackground(new Color(100, 100, 100));  
        forOutput = new Vector<>();  
        rec = new SoundRecord(null);  
        double[] fft = new double[1024];  
        fft[4] = 500.0;  
        LYGFileIO IYGFileIO = new LYGFileIO().initWithFFT(new double[1024], NE);  
        waveJpanel = new O_V.OSI.OSU.AVU.OSQ.wavePlay.Jp(IYGFileIO);  
        JButton j = new DetaButton("录音", 100 - 20, 50, Color.pink);  
        j.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                NE.app_S.reg = 1;  
                type = 0;  
                t1 = new Thread(SoundWaveJPanelTime.this);  
                t1.setPriority(Thread.MIN_PRIORITY);  
                t1.start();  
                jpanelFourth.validate();  
            }  
        });  
        j.setSize(100, 30);  
        j.setBounds(500, 20, 100, 30);  
        detaButton = new DetaButton("关闭", 100 - 20, 50, Color.black);  
        detaButton.setForeground(Color.black);  
        detaButton.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                NE.app_S.reg = 0;  
                if (null != waveJpanel) {  
                    waveJpanel.reg = 0;  
                }  
                if (null != jpanelFourth) {  
                    jpanelFourth.validate();  
                }  
            }  
        });  
        detaButton.setSize(100, 30);  
        JButton j2 = new DetaButton("生成数列", 100 - 20, 50, Color.green);  
        j2.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                NE.app_S.reg = 2;  
                if (forOutput == null) {
```

```
        return;
    }
    if (forOutput.size() <= 0) {
        return;
    }
    if (text.getText().length() > 5000) {
        return;
    }
    StringBuilder page = new StringBuilder();
    Iterator<java.lang.Float> iterator = forOutput.iterator();
    int i = 0;
    while (iterator.hasNext()) {
        double setOfi = iterator.next();
        page.append(" " + (i++) + ":" + setOfi);
    }
    if (!text.getText().isEmpty()) {
        text.setText(text.getText() + "\r\n\r\n" + page.toString());
    } else {
        text.setText(page.toString());
    }
    text.validate();
}
});
j2.setSize(100, 30);
JButton j21 = new JButton("读取数列", 100 - 20, 50, Color.green);
j21.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 2;
        if (forOutput == null) {
            return;
        }
        if (forOutput.size() <= 0) {
            return;
        }
        if (text.getText().length() > 5000) {
            return;
        }
        StringBuilder page = new StringBuilder();
        Iterator<java.lang.Float> iterator = forOutput.iterator();
        int i = 0;
        while (iterator.hasNext()) {
            double setOfi = iterator.next();
            page.append(" " + (i++) + ":" + setOfi);
        }
        if (!text.getText().isEmpty()) {
            text.setText(text.getText() + "\r\n\r\n" + page.toString());
        } else {
```

```
        text.setText(page.toString());
    }
    text.validate();
}
});
j21.setSize(100, 30);
JButton j3 = new JButton("存为图片", 100 - 20, 50, Color.magenta);
j3.addActionListener(new ActionListener() {
    @SuppressWarnings("unused")
    public void actionPerformed(ActionEvent e) {
        NE.app_S.reg = 0;
        //获取图片
        //弹出保存框
        if (null == imageForOutput) {
            return;
        }
        //get path
        FileDialog filedialog = new FileDialog(new Frame(), "导入到这里", FileDialog.LOAD);
        filedialog.setFilenameFilter(new TXTFilter("png"));
        filedialog.setVisible(true);
        String filepath = filedialog.getDirectory() + filedialog.getFile();
        //20230106-S_logger.Log.logger.info("" + filepath);
        try {
            File outputBin = new File(filepath + ".png");
            if (null == imageForOutput) {
                return;
            }
            ImageIO.write(imageForOutput, "png", outputBin);
        } catch (IOException writePngError) {
            //20230106-S_logger.Log.logger.info("" + filepath);
        }
        //输出图片
    }
});
j3.setSize(100, 30);
JButton j4 = new JButton("高敏噪音", 100 - 20, 50, Color.black);
j4.setForeground(Color.black);
j4.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (null != waveJpanel) {
            waveJpanel.validate();
            waveJpanel.updateUI();
            validate();
            updateUI();
            NE.app_S.jTabbedPane.validate();
            NE.app_S.jTabbedPane.updateUI();
        }
    }
});
```

```
NE.app_S.reg = 1;

type = 1;

t1 = new Thread(SoundWaveJPanelTime.this);

t1.setPriority(Thread.MIN_PRIORITY);

t1.start();

jpanelFourth.validate();

}

});

j4.setSize(100, 30);

JButton j5 = new JButton("读取图片", 100 - 20, 50, Color.black);

j5.setForeground(Color.black);

j5.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        NE.app_S.reg = 1;

        type = 1;

        t1 = new Thread(SoundWaveJPanelTime.this);

        t1.setPriority(Thread.MIN_PRIORITY);

        t1.start();

        jpanelFourth.validate();

    }

});

j5.setSize(100, 30);

JButton j51 = new JButton("调制解调", 100 - 20, 50, Color.black);

j51.setForeground(Color.black);

j51.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        NE.app_S.reg = 1;

        type = 1;

        waveJpanel.reg = 5;

        t1 = new Thread(waveJpanel);

        t1.setPriority(Thread.MIN_PRIORITY);

        t1.start();

        ;

        jpanelFourth.validate();

    }

});

j51.setSize(100, 30);

JButton j6 = new JButton("山林", 100 - 10, 50, Color.green);

j6.setForeground(Color.black);

j6.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        NE.app_S.reg = 1;

        type = 1;

        t1 = new Thread(SoundWaveJPanelTime.this);

        t1.setPriority(Thread.MIN_PRIORITY);

        t1.start();

        jpanelFourth.validate();

    }

});
```

```
    }
    });
    j6.setSize(100, 30);
    DetaButton j7 = new DetaButton("海边", 100 - 20, 50, Color.blue);
    j7.setForeground(Color.black);
    j7.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            NE.app_S.reg = 1;
            type = 1;
            t1 = new Thread(SoundWaveJPanelTime.this);
            t1.setPriority(Thread.MIN_PRIORITY);
            t1.start();
            jpanelFourth.validate();
        }
    });
    j7.setSize(100, 30);
    JButton j8 = new DetaButton("美梦", 100 - 20, 50, Color.magenta);
    j8.setForeground(Color.black);
    j8.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            NE.app_S.reg = 1;
            type = 1;
            t1 = new Thread(SoundWaveJPanelTime.this);
            t1.setPriority(Thread.MIN_PRIORITY);
            t1.start();
            jpanelFourth.validate();
        }
    });
    j8.setSize(100, 30);
    this.add(j);
    this.add(detaButton);
    this.add(j2);
    this.add(j21);
    this.add(j3);
    this.add(j5);
    this.add(j4);
    this.add(j51);
    this.add(j6);
    this.add(j7);
    this.add(j8);
    NE.app_S.jp3n = new SoundWavePanelVectorNew(imageForOutput.getGraphics());
    NE.app_S.jp3n.setBounds(100, 0, 400, 400);
    NE.app_S.jp3n.setVisible(true);
    this.add(NE.app_S.jp3n);
}

public void run() {
    if (1 == NE.app_S.reg) {
```

```

Graphics2D g = (Graphics2D) NE.app_S.jp3n.getGraphics();

g.setBackground(new Color(110, 110, 110));

g.setColor(new Color(110, 110, 110));

g.clearRect(0, 40, 1400, 200);

new ReadingLine(NE).start();

new ShowingLine(NE).start();

    }

}

}

//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWavePanelTimeTest.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWavePanelVector.java
package L_A.IMS.EUQ.SP;

//import java.awt.Color;

//import java.awt.Font;

import S_A.linePage.PageP_M;

import java.awt.Graphics;

import java.awt.Graphics2D;

import java.awt.Panel;

import java.awt.geom.Line2D.Double;

import java.util.Vector;

//import ME.VPC.S.WindowsUI;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

* */, 基于 java sound demo JDK 官方 demo 写的

public class SoundWavePanelVector extends Panel {

    public int regLine = 0;

    public float[] temp1024 = new float[1024];

    public PageP_M pageP_M;

    Graphics g3;

    //NE.app_S.pageP_M;

    public SoundWavePanelVector(Graphics graphics) {

        this.g3 = graphics;

    }

    public Vector<Double> lines;

    public Vector<Double> forOutput = new Vector<Double>();

    public String v1;

    public Graphics2D g2;

    //

}

//public boolean isCloseSA = true;

// public Vector<Double> mlines1;

// public Vector<Double> mlines2;

```

```
// public Vector<Double> mlines3;
// public Vector<Double> mlines4;
// public Vector<Double> mlines5;
// public Vector<Double> mlines6;
// public Vector<Double> mlines7;
// public Vector<Double> mlines8;
// public Vector<Double> mlines9;
// public Vector<Double> mlines10;
// public Vector<Double> mlines11;
// public Vector<Double> mlines12;

//Trif_DNA-master/C-L-I/I_A/IMS/EUQ/SP/SoundWavePanelVectorEraser.java
//Trif_DNA-master/C-L-I/I_A/IMS/EUQ/SP/SoundWavePanelVectorNew.java
package I_A.IMS.EUQ.SP;

import ME.VPC.M.app.App;

import java.awt.Color;

import java.awt.Font;

import java.awt.Graphics;

import java.awt.Graphics2D;

import java.awt.geom.Line2D;

import java.awt.geom.Line2D.Double;

import java.util.Iterator;

import java.util.Vector;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 * *///, 基于 java sound demo JDK 官方 demo 写的
public class SoundWavePanelVectorNew extends SoundWavePanelVector {

    public void paint2(float[] ftkD, Color color, int rangedX
        , int rangedY, int midY) {
        if (null == g2 || null == ftkD) {
            return;
        }
        g2.setColor(color);
        for (int i = 0; i < ftkD.length - 1; i++) {
            g2.drawLine(i + rangedX, rangedY - midY - (int) ftkD[i]
                , i + 1 + rangedX, rangedY - midY - (int) ftkD[i + 1]);
        }
    }

    public void paint_string(Color color, int rangedX
        , int rangedY, String domainName) {
        if (null == g2) {
            return;
        }
    }
}
```

```
g2.setColor(color);
g2.drawString(domainName, 0 + rangedX, rangedY + 30);
}

public SoundWavePanelVectorNew(Graphics graphics) {
    super(graphics);
    // TODO Auto-generated constructor stub
}

public void paint2_string(Color color, int rangedX, int rangedY, String domainName) {
    if (null == g2) {
        return;
    }
    g2.setColor(color);
    g2.drawString(domainName, 0 + rangedX - 128, rangedY + 30 + 40);
}

public void paint2(float[] ftkDA, float[] ftkDBI, Color color
, int rangedX, int rangedY, int midY) {
    if (null == g2 || null == ftkDA) {
        return;
    }
    g2.setColor(color);
    for (int i = 0; i < ftkDA.length - 1; i++) {
        g2.drawLine((int) ftkDA[i] * 3 + rangedX
, rangedY - midY - (int) ftkDBI[i]
, (int) ftkDA[i + 1] * 3 + rangedX
, rangedY - midY - (int) ftkDBI[i + 1]);
    }
}

public void paint9(Vector<Double> mlines) {
    if (null == g2) {
        return;
    }
    if (mlines != null) {
        g2.setColor(Color.pink);
        if (mlines.size() > 2) {
            Iterator<Double> iterator = mlines.iterator();
            while (iterator.hasNext()) {
                Line2D v = iterator.next();//later int remove
                g2.drawLine((int) v.getX1() + 530, 570
, (int) v.getX2() + 530, 570 - (int) v.getY2());
            }
        }
    }
    g2.drawString("虚空分阶", 530, 600);
}

public void paint11(Vector<Double> mlines) { //later all in 1
    if (null == g2) {
        return;
    }
}
```



```

    }

    if (mlines != null) {
        g2.setColor(Color.white);
        if (mlines.size() > 2) {
            Iterator<Double> iterator = mlines.iterator();
            while (iterator.hasNext()) {
                Line2D v = iterator.next(); // later int remove
                g2.drawLine((int) v.getX1() + 790, 420
                    , (int) v.getX2() + 790, 420 - (int) v.getY2());
            }
        }
    }

    g2.drawString("薛定谔虚度漂移", 780 + 10, 350 + 120 - 20);
    g3.drawString("输出", 780 + 10, 350 + 120 - 20);
    g2.drawString("噪音报警类别:", 780 + 10, 350 + 220);
    g3.drawString("噪音报警类别:", 780 + 10, 350 + 220);
}

public void paintvalue(String v1) {
    g2.setColor(Color.black);
    g2.setFont(new Font("宋体", Font.BOLD + Font.ITALIC, 20));
    if (v1 != null)
        g2.drawString("你说的是" + v1, 10, 780);
}

//录制波是尾数复制，所以会remove，读取波是pos提取，会杂乱。
public void paint(Graphics g, float[] lines
    , Vector<java.lang.Float> forOutput, App NE) {
    //获取每一段数据的8等分，确定当前数据的8等分
    //确定等分比for loop，每更新一个等分前擦去当前的等分
    if (null == g2) {
        return;
    }
    g2 = (Graphics2D) g;
    g2.setColor(new Color(95, 95, 95));
    if (regLine + lines.length > 1024) {
        regLine = 0;
        forOutput.clear();
        if (NE.app_S.full1024) {
            NE.app_S.full1024Vector = temp1024.clone();
        }
    }
    int scaleX = regLine;
    int j = 1;
    for (int i = regLine; i < regLine + lines.length - 1; i++) {
        int x = (int) temp1024[i];
        int y = (int) temp1024[i + 1];
        g2.drawLine(j + 10 + scaleX, x, j++ + 11 + scaleX, y);
    }
}

```

```
g2.drawString(NE.app_S.rateWave + "Hz 时序波", 10, 170);
g2.setColor(Color.pink);
if (lines != null) {
    int x = 0;
    int y = 0;
    scaleX = regLine;//later trif
    j = 1;
    for (int i = 0; i < lines.length - 1; i++) {
        x = (int) lines[i] + 120;
        y = (int) lines[i + 1] + 120;
        forOutput.add(lines[i]);
        x = x < 42 ? 42 : x;
        x = x > 185 ? 185 : x;
        y = y < 42 ? 42 : y;
        y = y > 185 ? 185 : y;
        g2.drawLine(j + 10 + scaleX, x, j++ + 11 + scaleX, y);
        if (regLine + i > 1024) {
            return;
        }
        temp1024[regLine + i] = x;
    }
    temp1024[regLine + lines.length - 1] = y;
    regLine += lines.length;
}
g2.drawString(NE.app_S.rateWave + "Hz 时序波", 10, 170);
}

public void paintClean() {
    if (null != g2) {
        g2.setColor(new Color(95, 95, 95));
        g2.fillRect(0, 40, 1100, 160 * 3);
    }
}

public void paintSA(Color color) {
    if (null == g2) {
        return;
    }
    //十字标
    g2.setColor(color);
    g2.drawLine(10, 120, 1034, 120);
    g2.drawLine(522, 40, 522, 200);
    //刻度标
    int scale = 16;
    for (int xs = 0; xs < 1024; xs += scale) {
        g2.drawLine(xs + 10, 115, xs + 10, 125);
        if (xs % 128 == 0) {
            g2.drawLine(xs + 10, 110, xs + 10, 130);
            g2.drawString("" + xs, xs + 10, 115);//数字标
```

```

    }
}

for (int ys = 40; ys < 200; ys += scale) {
    g2.drawLine(512 + 5, ys, 512 + 15, ys);
}
}

public void paintSAm(int x, int y, Color color) {
    if (null == g2) {
        return;
    }
    g2.setColor(color);
    g2.drawLine(x, 41, x, 199); // 鼠十字
    g2.drawLine(0, y, 1024, y);
    g2.drawString("(" + x + ", " + -(y - 120) + ")", x, y + 30); // 鼠tag
}
}

//186
// public void paintValid(Graphics g, Vector<Double> lines
//     , Vector<Double> forOutput, Color color) {
//     g2 = (Graphics2D) g; // later be null check;
//     if (lines != null) {
//         forOutput.clear();
//         g2.setColor(color);
//         for (int i = 1, j = 1; i < lines.size() - 1; i++, j++) {
//             Line2D v = lines.get(i);
//             Line2D v1 = lines.get(i + 1);
//             forOutput.add(lines.get(i + 1));
//             int x = (int) v.getY1() + 120;
//             int y = (int) v1.getY2() + 120;
//             if (x < 42) {
//                 x = 42;
//             }
//             if (x > 185) {
//                 x = 185;
//             }
//             if (y < 42) {
//                 y = 42;
//             }
//             if (y > 185) {
//                 y = 185;
//             }
//             g2.drawLine(j + 10, x, j + 11, y);
//         }
//         // temp1024 = (Vector<Double>) lines.clone();
//     }
//     g2.drawString(NE.app_S.rateWave + "Hz 时序波", 10, 170);
// }

```

```
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWavePanelVectorNewTest.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWaveVector.java
//Trif_DNA-master/C_L_I/L_A/IMS/EUQ/SP/SoundWaveVectorNew.java
package I_A.IMS.EUQ.SP;
```

```
import ME.VPC.M.app.App;
```

```
import javax.sound.sampled.AudioFormat;
```

```
import javax.sound.sampled.AudioInputStream;
```

```
import java.util.Vector;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光, 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
* ///, 基于 java sound demo JDK 官方 demo 写的
```

```
public class SoundWaveVectorNew extends SoundWaveVector {
```

```
    @SuppressWarnings({"rawtypes", "unused"})
```

```
    public Vector getVectorLines1(AudioInputStream ais, App NE) {
```

```
        lines.removeAllElements();// clear the old vector
```

```
        AudioFormat af = ais.getFormat();
```

```
        audioBytes = null;
```

```
        if (audioBytes == null) {
```

```
            try {
```

```
                int temp = (int) (ais.getFrameLength() * af.getFrameSize());
```

```
                audioBytes = new byte[temp >> 1];
```

```
                ais.read(audioBytes);//loop later
```

```
                int len = 1;
```

```
                int c = 0;
```

```
                while (len > 0 && 1 == NE.app_S.reg && null != audioBytes) {
```

```
                    if (null == ais) {
```

```
                        break;
```

```
                    }
```

```
                    len = ais.read(audioBytes);//dangqian 4800 zhen = 1024fs
```

```
                    if (len > 0) {
```

```
                        doAudioBytesTask(af, NE);
```

```
                    }
```

```
                }
```

```
            } catch (Exception ex) {
```

```
                ex.printStackTrace();
```

```
                return lines;
```

```
            }
```

```
        }
```

```
        return lines;
```

```
    }
```

```
    public void doAudioBytesTask(AudioFormat af, App NE) {
```

```
        if (audioBytes.length < 1) {
```

```
        return;
    }

    pshock = 0.0;
    nshock = 0.0;
    if (af.getSampleSizeInBits() == 16) {
        int nlengthInSamples = audioBytes.length >> 1;
        if (nlengthInSamples != audioData.length) {
            if (null == audioData) {
                audioData = new float[nlengthInSamples];
            } else {
                if (audioData.length != nlengthInSamples) {
                    audioData = new float[nlengthInSamples];
                }
            }
        }
        if (af.isBigEndian()) {
            for (int i = 0; i < nlengthInSamples; i++) {
                int MSB = audioBytes[i << 1];
                int LSB = audioBytes[i * 2 + 1]; //later heap
                audioData[i] = (MSB << 8 | (255 & LSB));
            }
        } else {
            for (int i = 0; i < nlengthInSamples; i++) {
                int LSB = audioBytes[i << 1];
                int MSB = audioBytes[i * 2 + 1];
                audioData[i] = (MSB << 8 | (255 & LSB));
            }
        }
    } else if (af.getSampleSizeInBits() == 8) {
        if (audioBytes.length != audioData.length) {
            if (null == audioData) {
                audioData = new float[audioBytes.length];
            } else {
                if (audioData.length != audioBytes.length) {
                    audioData = new float[audioBytes.length];
                }
            }
        }
    }
    if (af.getEncoding().toString().startsWith("PCM_SIGN")) {
        for (int i = 0; i < audioBytes.length; i++) {
            audioData[i] = (audioBytes[i]);
        }
    } else {
        for (int i = 0; i < audioBytes.length; i++) {
            audioData[i] = (audioBytes[i]) - 128;
        }
    }
}
```

```

    }

    float newdataO[] = new float[N];
    int count = 0;//do loop
    int ratio = 0;
    //大于1024 loop
    //小于1024 叠满
    if (audioData.length < N) {
        if (countL < N) {
            for (int i = 0; i < audioData.length && (countL < N); i++) {
                audioDataL[countL++] = audioData[i];
            }
        }
        if (countL >= N) {
            countL = 1;
            float newdata[] = audioDataL.clone();
            audioDataL = new float[N]; //later acceleration
            //先做当前软件已显示的1024记录
            NE.app_S.pageQ.addDataToTail(newdata, NE);
            tempDoDraw(newdata, NE);
        }
    }

    if (audioData.length >= N) {
        while (count < audioData.length && 1 == NE.app_S.reg) {
            int rc = ratio * 1024;
            ratio++;
            if (N + rc > audioData.length) {
                return;//去range
            }
            for (int i = 0; i < N && audioData != null; i++) {
                newdataO[i] = audioData[i + rc]; //infinet later
                count++;
            }
            float newdata[] = newdataO;
            //先做当前软件已显示的1024记录
            NE.app_S.pageQ.addDataToTail(newdata, NE);
            tempDoDraw(newdata, NE);
        }
    }

    private void tempDoDraw(float[] newdata, App NE) {
        if (!NE.app_S.pageQ.registered) {
            NE.app_S.pageQ.regiter(lines, N, newdata, NE);
            NE.app_S.pageQ.newdata = newdata;
            v = NE.app_S.pageQ.doClt4(NE);
            return;
        }
        if (NE.app_S.line2025.isRecoding) {

```

```

        return;
    }

    //attension
    NE.app_S.pageQ.newdata = newdata;

    //ff
    v = NE.app_S.pageQ.doCl4(NE);
}
}

/Trif_DNA-master/C_L_I/I_A/IMS/EUQ/SP/SoundWaveVectorNewTest.java

    I_Q.hvpcs

    /Trif_DNA-master/C_L_I/I_Q/hvpcs/说明.txt

    I_Q.hvpcs.boot.controller

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/ConfigController.java
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/DBCategoryController.java
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/DeleteController.java
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/InsertController.java
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/SelectController.java
/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/controller/UpdateController.java

    I_Q.hvpcs.boot.rest

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/rest/LocalRestCall.java
package I_Q.hvpcs.boot.rest;

import java.io.IOException;
import java.io.InputStream;
import java.net.URLDecoder;
import java.util.Map;
import I_Q.hvpcs.boot.sets.VPCSRequest;
import I_Q.hvpcs.boot.sets.VPCSResponse;
import I_Q.hvpcs.html5.upload.FileUploaderSimple;
import I_Q.hvpcs.html5.upload.FileUploaderSimpleDNA;
import I_Q.hvpcs.html5.upload.FileUploaderSimpleSocket;
import I_Q.hvpcs.html5.upload.FileUploaderSimpleSocketPDE;
import I_Q.hvpcs.html5.upload.FileUploaderSimpleSocketSound;
import I_Q.hvpcs.html5.upload.FileUploaderSimpleSocketSoundImage;
import S_logger.Log;

//import org.html5.upload.FileUploaderSimpleSocketSoundImageMpeg;
/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class LocalRestCall {

    public static void doDataSB(String[] stringInputs
        , VPCSResponse vPCSResponse, VPCSRequest vPCSRequest
        , String stringInput, String mess, InputStream inputStream
        , byte[] bytes) throws IOException, InterruptedException {
        for (String contentLength : stringInputs) {

```

```

        if (contentLength.contains("Content-Length")) {
            String[] strings = contentLength.split(":");
            if (Long.valueOf(strings[1].replace(" ", "")) > 9999999) {
                vPCSResponse.returnCode(200);
                return;
            }
        }
    } //checkstatus
    System.out.flush();
    S_logger.Log.logger.info("" + "mess-" + mess);
    if (stringInput.contains(".png") || stringInput.contains(".jpeg")
        || stringInput.contains("JFIF") || stringInput.contains(".jpg")) {
        System.out.flush();
        S_logger.Log.logger.info("" + 2);
        //Thread.sleep(15);
        FileUploaderSimpleSocket.processForm(inputStream, bytes
            , ".jpg", vPCSRequest, vPCSResponse, stringInput);
    } else {
        vPCSResponse.returnCode(200);
        inputStream.close();
        return;
    }
    inputStream.close();
}

public static void doDataYL(String[] stringInputs
    , VPCSResponse vPCSResponse, VPCSRequest vPCSRequest
    , String stringInput, String mess, InputStream inputStream
    , byte[] bytes, int length, Map<String, String> data)
    throws IOException, InterruptedException {
    byte[] bytesNext = new byte[1600];
    if (length == 1600) {
        length = inputStream.read(bytesNext);
        String stringInputNext = new String(bytesNext);
        String[] stringInputsFull = (stringInput + stringInputNext).split("\r\n");
        for (String string : stringInputsFull) {
            if (string.contains("email=") && string.contains("token=")) {
                String[] stringsEmail = string.split("email=");
                String[] stringsEmailStop = stringsEmail[1].split(";");
                data.put("email", URLDecoder.decode(stringsEmailStop[0], "UTF-8"));
                String[] stringsToken = string.split("token=");
                data.put("token", URLDecoder.decode(stringsToken[1], "UTF-8"));
            }
        }
    }
} else {
    String[] stringInputsFull = (stringInput).split("\r\n");
    for (String string : stringInputsFull) {
        if (string.contains("email=") && string.contains("token=")) {

```



```

        String[] stringsEmail = string.split("email=");
        String[] stringsEmailStop = stringsEmail[1].split(";");
        data.put("email", URLDecoder.decode(stringsEmailStop[0], "UTF-8"));
        String[] stringsToken = string.split("token=");
        data.put("token", URLDecoder.decode(stringsToken[1], "UTF-8"));
    }
}
}

public static void doDataZT(String[] stringInputs
    , VPCSResponse vPCSResponse, VPCSRequest vPCSRequest
    , String stringInput, String mess, InputStream inputStream
    , byte[] bytes) throws IOException, InterruptedException {
    // 处理 //dz
    for (String contentLength : stringInputs) {
        if (contentLength.contains("Content-Length")) {
            String[] strings = contentLength.split(":");
            if (Long.valueOf(strings[1].replace(" ", "")) > 9999999) {
                vPCSResponse.returnCode(200);
                return;
            }
        }
    }
    System.out.flush();
    S_logger.Log.logger.info("" + "mess1-" + mess);
    if (stringInput.contains(".wav")) {
        System.out.flush();
        S_logger.Log.logger.info("" + 2);
        //Thread.sleep(15);
        FileUploaderSimpleSocketSoundImage.processForm(inputStream
            , bytes, ".wav", vPCSRequest, vPCSResponse, stringInput);
        return;
    } else if (stringInput.contains(".mp3")) {
        System.out.flush();
        S_logger.Log.logger.info("" + 2);
        //Thread.sleep(15); //later
        //FileUploaderSimpleSocketSoundImageMpeg.processForm(inputStream
        //, bytes, ".mp3", vPCSRequest, vPCSResponse, stringInput);
        return;
    } else {
        vPCSResponse.returnCode(200);
        return;
    }
}

public static void doDataPDE(String[] stringInputs
    , VPCSResponse vPCSResponse, VPCSRequest vPCSRequest
    , String stringInput, String mess, InputStream inputStream

```

```

, byte[] bytes) throws IOException, InterruptedException {
// 处理
// 处理
for (String contentLength : stringInputs) {
    if (contentLength.contains("Content-Length")) {
        String[] strings = contentLength.split(":");
        if (Long.valueOf(strings[1].replace(" ", "")) > 9999999) {
            vPCSResponse.returnValue(200);
            return;
        }
    }
}
//checkstatus
System.out.flush();
S_logger.Log.logger.info("" + "mess-2" + mess);
if (stringInput.contains(".png") || stringInput.contains(".jpeg")
    || stringInput.contains("JFIF") || stringInput.contains(".jpg")) {
    System.out.flush();
    S_logger.Log.logger.info("" + 2);
    //Thread.sleep(15);
    FileUploaderSimpleSocketPDE.processForm(inputStream, bytes
        , ".jpg", vPCSRequest, vPCSResponse, stringInput);
    inputStream.close();
    return;
} else {
    vPCSResponse.returnValue(200);
    inputStream.close();
    return;
}
}

public static void doDataTC(String[] stringInputs
    , VPCSResponse vPCSResponse, VPCSRequest vPCSRequest
    , String stringInput, String mess, InputStream inputStream
    , byte[] bytes) throws IOException, InterruptedException {
// 处理
// 处理
for (String contentLength : stringInputs) {
    if (contentLength.contains("Content-Length")) {
        String[] strings = contentLength.split(":");
        if (Long.valueOf(strings[1].replace(" ", "")) > 9999999) {
            vPCSResponse.returnValue(200);
            return;
        }
    }
}
}
System.out.flush();
S_logger.Log.logger.info("" + "mess-3" + mess);

```

```

        if (stringInput.contains(".png") || stringInput.contains(".jpeg")
            || stringInput.contains("JFIF") || stringInput.contains(".jpg")) {
            System.out.flush();
            S_logger.Log.logger.info("" + 2);
            //Thread.sleep(15);
            FileUploaderSimple.processForm(inputStream, bytes
                , ".jpg", vPCSRequest, vPCSResponse);
        } else {
            vPCSResponse.returnCode(200);
            return;
        }
    }

    public static void doDataSC(String[] stringInputs
        , VPCSResponse vPCSResponse, VPCSRequest vPCSRequest
        , String stringInput, String mess, InputStream inputStream
        , byte[] bytes) throws IOException, InterruptedException {
        // 测试 //
        for (String contentLength : stringInputs) {
            if (contentLength.contains("Content-Length")) {
                String[] strings = contentLength.split(":");
                if (Long.valueOf(strings[1].replace(" ", "")) > 9999999) {
                    vPCSResponse.returnCode(200);
                    return;
                }
            }
        }
        System.out.flush();
        S_logger.Log.logger.info("" + "mess-4" + mess);
        if (stringInput.contains(".png") || stringInput.contains(".jpeg")
            || stringInput.contains("JFIF") || stringInput.contains(".jpg")) {
            System.out.flush();
            S_logger.Log.logger.info("" + 2);
            //Thread.sleep(15);
            FileUploaderSimpleDNA.processForm(inputStream, bytes
                , ".jpg", vPCSRequest, vPCSResponse);
            return;
        } else {
            vPCSResponse.returnCode(200);
            return;
        }
    }

    public static void doDataSY(String[] stringInputs
        , VPCSResponse vPCSResponse, VPCSRequest vPCSRequest
        , String stringInput, String mess, InputStream inputStream
        , byte[] bytes) throws IOException, InterruptedException {
        // 测试 //
        for (String contentLength : stringInputs) {

```

```

        if (contentLength.contains("Content-Length")) {
            String[] strings = contentLength.split(":");
            if (Long.valueOf(strings[1].replace(" ", "")) > 9999999) {
                vPCSResponse.returnCode(200);
                return;
            }
        }
    }
    System.out.flush();
    S_logger.Log.logger.info("" + "mess-5" + mess);
    if (stringInput.contains(".wav")) {
        System.out.flush();
        S_logger.Log.logger.info("" + 2);
        //Thread.sleep(15);
        FileUploaderSimpleSocketSound.processForm(inputStream, bytes
            , ".wav", vPCSRequest, vPCSResponse, stringInput);
        return;
    } else {
        vPCSResponse.returnCode(200);
        return;
    }
}

}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/rest/VPC.java
package I_Q.hvpcs.boot.rest;

import java.io.File;
import java.io.IOException;
import java.util.Map;
import I_Q.hvpcs.lyg.common.maps.VtoV;
import I_Q.hvpcs.lyg.vpc.process.portImpl.RestAskPortImpl;
import I_Q.hvpcs.lyg.vpc.process.portImpl.RestLoginPortImpl;
import ME.VPC.M.app.App;
import I_Q.hvpcs.boot.controller.SelectController;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class VPC {
    //这个文件以后直接继承和扩展即可。
    public static String forward(String string, Map<String, String> data, App NE)
        throws Exception {
        // controller

```

```
        if (string.contains("/select")) {
            return SelectController.exec(string, data);
        }
        if (string.contains("/setDB")) {
            // return ConfigController.exec(string, data);
        }
        if (string.contains("/insert")) {
            // return InsertController.exec(string, data);
        }
        if (string.contains("/delete")) {
            // return DeleteController.exec(string, data);
        }
        if (string.contains("/update")) {
            // return UpdateController.exec(string, data);
        }
        if (string.contains("DBCategory")) {
            // return DBCategoryController.exec(string, data);
        }
        // restMap
        if (string.equalsIgnoreCase("/login")) {
            return VtoV.ObjectToJsonString(RestLoginPortImpl
                .login(data.get("uEmail"), data.get("uPassword")));
        }
        if (string.equalsIgnoreCase("/find")) {
            return VtoV.ObjectToJsonString(
                RestLoginPortImpl.find(data.get("uEmail")));
        }
        if (string.equalsIgnoreCase("/logout")) {
            return VtoV.ObjectToJsonString(RestLoginPortImpl
                .logout(data.get("uEmail"), data.get("uToken")));
        }
        if (string.equalsIgnoreCase("/register")) {
            return VtoV.ObjectToJsonString(RestLoginPortImpl.register(
                data.get("uEmail"), data.get("uEmailEnsure"),
                data.get("uName"), data.get("uPassword"),
                data.get("uPassWDEnsure"), data.get("uAddress"),
                data.get("uPhone"), data.get("uWeChat"), data.get("uQq"),
                data.get("uAge"), data.get("uSex")));
        }
        if (string.equalsIgnoreCase("/change")) {
            return VtoV.ObjectToJsonString(
                RestLoginPortImpl.change(data.get("uEmail"),
                    data.get("uChange"), data.get("uChangeEnsure"),
                    data.get("uToken"), data.get("uPassword")));
        }
        // if(string.equalsIgnoreCase("/checkStatus")){
        // return
```

```
// VtoV.ObjectToJsonString(RestLoginPortImpl.checkStatus(data.get("token")));  
// }  
if (string.equalsIgnoreCase("/ask")) {  
    return VtoV.ObjectToJsonString(  
RestAskPortImpl.ask(data.get("ip"), data.get("token"),  
        data.get("message"), data.get("pointIp")));  
}  
if (string.equalsIgnoreCase("/feedBack")) {  
    return VtoV.ObjectToJsonString(RestAskPortImpl.feedBack(  
data.get("ip"), data.get("token"), data.get("pointIp")));  
}  
if (string.equalsIgnoreCase("/getAskers")) {  
    return VtoV.ObjectToJsonString(  
RestAskPortImpl.getAskers(data.get("token")));  
}  
if (string.equalsIgnoreCase("/dataWS")) {  
    return RestAskPortImpl.dataWS(data.get("input"), NE);  
}  
if (string.equalsIgnoreCase("/dataCX")) {  
    return RestAskPortImpl.dataCX(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataCY")) {  
    return RestAskPortImpl.dataCY(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataXL")) {  
    return RestAskPortImpl.dataXL(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataRN")) {  
    return RestAskPortImpl.dataRN(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataCG")) {  
    return RestAskPortImpl.dataCG(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataCJ")) {  
    return RestAskPortImpl.dataCJ(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataCL")) {  
    return RestAskPortImpl.dataCL(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataXX")) {  
    return RestAskPortImpl.dataXX(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataHF")) {  
    return RestAskPortImpl.dataHF(data.get("message"));  
}  
if (string.equalsIgnoreCase("/dataCP")) {  
    return RestAskPortImpl.dataCP(data.get("message"));  
}
```

```

    }
    if (string.equalsIgnoreCase("/dataZF")) {
        return RestAskPortImpl.dataZF(data.get("message"));
    }
    if (string.equalsIgnoreCase("/dataYL")) {
        System.out.flush();
        S_logger.Log.logger.info("" + System.currentTimeMillis());
        System.out.flush();
        S_logger.Log.logger.info("" + "2huaruiji->" + data.get("email"));
        System.out.flush();
        S_logger.Log.logger.info("" + "2huaruiji->" + data.get("token"));
        if (null == data.get("token") || null == data.get("email")) {
            return "";
        }
        return RestAskPortImpl.dataYL(data.get("message"),
            data.get("email"), data.get("token"));
    }
    if (string.equalsIgnoreCase("/dataSC")) {
        System.out.flush();
        S_logger.Log.logger.info("" + System.currentTimeMillis());
    }
    return "";
}

public static String getCode(String filePath) throws IOException {
    if (filePath.contains(".html") || filePath.contains(".js")) {
        return "UTF-8";
    }
    return "UTF8";
}

public static String getFilePath(String string) {
    String root = new File("src/main/resources/static").getAbsolutePath()
        .replace("\\", "/");
    if (null == string || string.equalsIgnoreCase(""))
        || string.equalsIgnoreCase("/") {
        string = "/index.html";
    }
    return root + string;
}
}

```

I_Q.hvpcs.boot.server

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/server/BootVPCS.java
 package I_Q.hvpcs.boot.server;

import java.io.IOException;

import I_Q.hvpcs.boot.vpc.controller.ServerInitController;

import I_Q.hvpcs.html5.upload.FileUploaderSimpleDNA;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/

```
public class BootVPCS {
    public static void main(String[] args) throws IOException
    , InterruptedException {
        FileUploaderSimpleDNA.newADNAMap(null);
        ServerInitController serverInitController= new ServerInitController();
        serverInitController.initServer(null);
    }
}
```

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/server/BootVPCSTest.java
 //Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/server/BootVPCSTest2.java
 //Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/server/dsd.java

I_Q.hvpcs.boot.sets

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/BlobLimit.java
 package I_Q.hvpcs.boot.sets;

import java.util.concurrent.ConcurrentHashMap;

```
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 **/
```

```
public class BlobLimit{
    public int exceptions = 0;
    public boolean gc= false;
    public ConcurrentHashMap<String, Long> blobLimitMap;
    public void initBlobLimit() {
        blobLimitMap = new ConcurrentHashMap<String, Long>();
    }
    public boolean addBlobLimit(String string, long time) {
        if(blobLimitMap.containsKey(string)) {
            long timeOld = blobLimitMap.get(string);
            if(time - timeOld > 1000 * 60 * 3) { //fix later
                blobLimitMap.put(string, time);
            }
            return true;
        }
        return false;
    }
    blobLimitMap.put(string, time);
}
```



```
        return true;
    }

    public boolean clearBlobLimit(String string) {
        if(blobLimitMap.containsKey(string)) {
            blobLimitMap.remove(string);
            return true;
        }
        return false;
    }
}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/Request_S.java
package I_Q.hvpcs.boot.sets;

import java.util.Map;
import S_A.pheromone.IMV_SQI;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
@SuppressWarnings("unused")
public class Request_S {
    private String requestIp;
    private String requestName;
    private String requestLink;
    protected boolean requestIsRest;
    private String requestFilePath;
    private String requestFileCode;
    private String requestForwardType;
    protected String requestPort = "";
    private String requestIpFix;
    private String requestNameFix;
    private String requestLinkFix;
    // private Map<String, String> requestValue;
    private IMV_SQI requestValue;
    public boolean waitingForShake = true;
    public String getRequestIp() {
        return requestIp;
    }
    public void setRequestIp(String requestIp) {
        this.requestIp = requestIp;
    }
    public String getRequestName() {
        return requestName;
    }
}
```

```
public void setRequestName(String requestName) {
    this.requestName = requestName;
}
public String getRequestLink() {
    return requestLink;
}
public void setRequestLink(String requestLink) {
    this.requestLink = requestLink;
}
public String getRequestIpFix() {
    return requestIpFix;
}
public void setRequestIpFix(String requestIpFix) {
    this.requestIpFix = requestIpFix;
}
public String getRequestNameFix() {
    return requestNameFix;
}
public void setRequestNameFix(String requestNameFix) {
    this.requestNameFix = requestNameFix;
}
public String getRequestLinkFix() {
    return requestLinkFix;
}
public void setRequestLinkFix(String requestLinkFix) {
    this.requestLinkFix = requestLinkFix;
}
public IMV_SQI getRequestValue() {
    return requestValue;
}
public void setRequestValue(IMV_SQI requestValue) {
    this.requestValue = requestValue;
}
public String getRequestForwardType() {
    return requestForwardType;
}
public void setRequestForwardType(String requestForwardType) {
    this.requestForwardType = requestForwardType;
}
public String getRequestFilePath() {
    return requestFilePath;
}
public void setRequestFilePath(String requestFilePath) {
    this.requestFilePath = requestFilePath;
}
public String getRequestFileCode() {
    return requestFileCode;
}
```

```
}

    public void setRequestFileCode(String requestFileCode) {

        this.requestFileCode = requestFileCode;

    }

}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/Response_S.java
package I_Q.hvpcs.boot.sets;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.ByteArrayOutputStream;
import java.io.DataOutputStream;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.io.PrintWriter;
import java.net.Socket;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Response_S {

    public InputStream inputStream;

    public FileInputStream fileInputStream;

    public InputStreamReader inputStreamReader;

    public BufferedReader bufferedReader;

    protected OutputStream outputStream;

    public PrintWriter printWriter;

    public DataOutputStream dataOutputStream;

    public ByteArrayOutputStream byteArrayOutputStream;

    public BufferedWriter bufferedWriter;

    public Socket socket;

    public Socket getSocket() {

        return socket;

    }

    public void setSocket(Socket socket) {

        this.socket = socket;

    }

    public void closeInputStream() throws IOException {

        if (null != inputStream) {

            inputStream.close();

        }

    }

}
```

```
}

public void closeOutputStream() throws IOException {
    if (null != outputStream) {
        outputStream.close();
    }
}

public void closePrintWriter() throws IOException {
    if (null != printWriter) {
        printWriter.close();
    }
}

public void closeAllBeforeGC() {
    try {
        if (null != fileInputStream) {
fileInputStream.close();
        }
        fileInputStream = null;
        if (null != inputStreamReader) {
inputStreamReader.close();
        }
        inputStreamReader = null;
        if (null != bufferedReader) {
bufferedReader.close();
        }
        bufferedReader = null;
        if (null != inputStream) {
inputStream.close();
        }
        inputStream = null;
        if (null != dataOutputStream) {
dataOutputStream.close();
        }
        dataOutputStream = null;
        if (null != byteArrayOutputStream) {
byteArrayOutputStream.close();
        }
        byteArrayOutputStream = null;
        if (null != bufferedWriter) {
if (!socket.isClosed()) {
            bufferedWriter.close();
        }
        }
        bufferedWriter = null;
        if (null != printWriter) {
if (!socket.isClosed()) {
            printWriter.close();
        }
    }
}
```

```
    }
    printWriter = null;
    if (null != outputStream) {
outputStream.close();
    }
    outputStream = null;
    if (null != socket) {
socket.close();
while (!this.socket.isClosed()) {
    Thread.sleep(1000);
}

    }
    socket = null;
} catch (Exception e) {
    System.out.flush();
    S_logger.Log.logger.info("" + "-closeAllBeforeGC err");
    // StableBlob.addException();
    e.printStackTrace();
}
}

public void feedAndCloseAllBeforeGC() {
    try {
        if (null != this.socket) {
if (null != fileInputStream) {
            fileInputStream.close();
        }
if (null != inputStreamReader) {
            inputStreamReader.close();
        }
if (null != bufferedReader) {
            bufferedReader.close();
        }
if (null != inputStream) {
            inputStream.close();
        }
if (null != dataOutputStream) {
            dataOutputStream.close();
        }
if (null != byteArrayOutputStream) {
            byteArrayOutputStream.close();
        }
if (null != bufferedWriter) {
            if (!socket.isClosed()) {
                bufferedWriter.close();
            }
        }
    }
    if (null != printWriter) {
```

```
        if (!socket.isClosed()) {  
            printWriter.close();  
        }  
    }  
    if (null != outputStream) {  
        outputStream.close();  
    }  
    if (null != socket) {  
        socket.close();  
    }  
    while (!this.socket.isClosed()) {  
        Thread.sleep(1000);  
    }  
    }  
    } catch (Exception e) {  
        System.out.flush();  
        S_logger.Log.logger.info("" + "-closeAllBeforeGC err");  
        // StableBlob.addException();  
        // e.printStackTrace();  
    }  
}  
  
public void closeSocket() throws IOException {  
    if (null != socket) {  
        socket.close();  
    }  
}  
  
public void registInputStream(InputStream inputStream) {  
    // TODO Auto-generated method stub  
    this.inputStream = inputStream;  
}  
  
public void doPrintWriter(Integer errorCode) throws IOException {  
    if (null == outputStream) {  
        outputStream = this.socket.getOutputStream();  
        printWriter = new PrintWriter(outputStream, true);  
        printWriter.flush();  
        printWriter.println("HTTP/1.1 " + errorCode + " OK\n\n");  
        printWriter.close();  
    }  
}  
}
```

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/Sleeper_S.java
package I_Q.hvpcs.boot.sets;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

```
* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class Sleeper_S{

    public VPCSRRequest vPCSRRequest;

    public VPCSRResponse vPCSRResponse;

}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/Stable.java
package I_Q.hvpcs.boot.sets;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class Stable {

    public static int totalDownload= 0;

}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/StableBlob.java
package I_Q.hvpcs.boot.sets;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public interface StableBlob {

    public BlobLimit blobLimit = new BlobLimit();

    public static boolean goodState() {

        if(!blobLimit.gc) {

            return true;

        }

        return false;

    }

    public static void initBlobLimit() {

        blobLimit.initBlobLimit();

    }

    public static boolean addBlobLimit(String string, long time) {

        return blobLimit.addBlobLimit(string, time);

    }

}
```

```

    }

    public static boolean clearBlobLimit(String string) {
        if(null != string) {
            return blobLimit.clearBlobLimit(string);
        }
        return false;
    }

    public static void addException() {
        if(blobLimit.exceptions++ > 8) {
            System.out.flush();S_logger.Log.logger.info("" + "-more exceptions");
        }
    }

    public static void checkException() {
        if(blobLimit.exceptions > 15) {
            blobLimit.exceptions = 0;
            System.gc();
            System.out.flush();S_logger.Log.logger.info("" + "-gc exceptions");
        }
    }
}

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/StableSets.java
package I_Q.hvpcs.boot.sets;

import java.io.UnsupportedEncodingException;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public interface StableSets {

    public List<byte[]> stableListPartBytes= new ArrayList<>();
    public Map<String, String> contentTypes= new HashMap<>();
    public static List<byte[]> getListPartBytes() {
        List<byte[]> output= new ArrayList<>();
        output.addAll(stableListPartBytes);
        return output;
    }

    public static void intListPartBytes() {

```



```

        try {
            if(stableListPartBytes.isEmpty()) {
                stableListPartBytes.add("HTTP/1.1 200 OK \n".getBytes("UTF8"));
                stableListPartBytes.add("Host: deta software \n".getBytes("UTF8"));
                stableListPartBytes.add("Accept-Ranges: bytes \n".getBytes("UTF8"));
                stableListPartBytes.add("Content-Encoding: Gzip \n".getBytes("UTF8"));
                stableListPartBytes.add("Cache-control: max-age=315360000 \n".getBytes("UTF8"));
            }
        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        }
    }

    public static void initHttpContentTypeMap() {
        if(contentTypes.isEmpty()) {
            contentTypes.put("png", "content-type: image/png \n\n");
            contentTypes.put("ico", "content-type: image/ico \n\n");
            contentTypes.put("jpeg", "content-type: image/jpeg \n\n");
            contentTypes.put("jpg", "content-type: image/jpg \n\n");
            contentTypes.put("gif", "content-type: image/gif \n\n");
            contentTypes.put("wav", "content-type: audio/wav \n\n");
            contentTypes.put("js", "content-type: text/javascript; charset:UTF-8 \n\n");
            contentTypes.put("map", "content-type: text/plain; charset:UTF-8 \n\n");
            contentTypes.put("css", "content-type: text/css \n\n");
            contentTypes.put("html", "content-type: text/html \n\n");
            contentTypes.put("txt", "content-type: text/plain \n\n");
            contentTypes.put("xml", "content-type: application/xml \n\n");
            contentTypes.put("pdf", "content-type: application/pdf \n\n");
            contentTypes.put("zip", "content-type: application/zip \n\n");
            contentTypes.put("rar", "content-type: application/rar \n\n");
            contentTypes.put("jar", "content-type: application/jar \n\n");
            contentTypes.put("pptx", "content-type: application/pptx \n\n");
        }
    }
}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/VPCSRRequest.java
package I_Q.hvpcs.boot.sets;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class VPCSRRequest extends I_Q.hvpcs.boot.sets.Request_S {
    public String getRequestPort() {

```

```
        return requestPort;
    }

    public void I_RequestPort(String requestPort) {
        this.requestPort = requestPort;
    }

    public boolean getRequestIsRest() {
        return requestIsRest;
    }

    public void setRequestIsRest(boolean requestIsRest) {
        this.requestIsRest = requestIsRest;
    }

    public void closeAllBeforeGC() {
        // StableBlob.clearBlobLimit(getRequestFilePath());
    }

    public void setWaitingForShake(boolean isWaitingForShake) {
        waitingForShake = isWaitingForShake;
    }

    public boolean getWaitingForShake() {
        // TODO Auto-generated method stub
        return waitingForShake;
    }
}

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/sets/VPCSResponse.java
package I_Q.hvpcs.boot.sets;

import java.io.IOException;
import java.io.PrintWriter;
import I_Q.hvpcs.boot.vpc.sleeper.SleeperHall;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class VPCSResponse extends Response_S {
    public boolean huashengketest = false;
    public int errorCode;
    public Integer hashCode;
    private String ResponseContentType;
    public SleeperHall sleeperHall;
    public SleeperHall getSleeperHall() {
        return sleeperHall;
    }

    public void setSleeperHall(SleeperHall sleeperHall) {
        this.sleeperHall = sleeperHall;
    }
}
```

```
public Integer getHashCode() {
    return hashCode;
}

public void setHashCode(Integer hashCode) {
    this.hashCode = hashCode;
}

public int getErrorCode() {
    return errorCode;
}

public void setErrorCode(int errorCode) {
    this.errorCode = errorCode;
}

public String getResponseContentType() {
    return ResponseContentType;
}

public void setResponseContentType(String responseContentType) {
    ResponseContentType = responseContentType;
}

public void returnNull() throws IOException {
    if (null != inputStream) {
        inputStream.close();
    }
    if (null != this.socket) {
        socket.close();
    }
    this.sleeperHall.removeThreadById(this.hashCode);
    System.out.flush();
    S_logger.Log.logger.info("" + this.getHashCode() + "-fatar");
    this.sleeperHall.callSkivvy();
}

public void returnCode(Integer errorCode) {
    try {
        if (errorCode == 204) {
            System.out.flush();
            S_logger.Log.logger.info("" + this.getHashCode() + "-bicha");
        }
        if (null == this.socket) {
            closeAllBeforeGC();
            this.sleeperHall.removeThreadById(this.hashCode);
            return;
        }
        if (!this.socket.isClosed()) {
            doPrintWriter(errorCode);
            if (null != inputStream) {
                inputStream.close();
            }
            this.socket.close();
        }
    }
}
```

```
    }
    if (null != inputStream) {
inputStream.close();
    }
    closeAllBeforeGC();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public void clearBuffer() {
    this.sleeperHall.removeThreadById(this.hashCode);
    System.out.flush();
    S_logger.Log.logger.info("" + this.hashCode + " -clear");
}

// old interface
public void return404() {
    try {
        if (socket.isClosed()) {
this.sleeperHall.removeThreadById(this.hashCode);
return;
        }
        PrintWriter pw = new PrintWriter(this.socket.getOutputStream(),
true);
        pw.println("HTTP/1.1 404 OK\n\n");
        pw.flush();
        pw.close();
        socket.close();
        this.sleeperHall.removeThreadById(this.hashCode);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public void returnErrorCode(Integer errorCode) {
    try {
        if (socket.isClosed()) {
this.sleeperHall.removeThreadById(this.hashCode);
return;
        }
        PrintWriter pw = new PrintWriter(this.socket.getOutputStream(),
true);
        pw.println("HTTP/1.1 " + errorCode + " OK\n\n");
        pw.flush();
        pw.close();
        socket.close();
        this.sleeperHall.removeThreadById(this.hashCode);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```

    }

}

}

I_Q.hvpcs.boot.vpc.controller

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/RequestFilterController.java
package I_Q.hvpcs.boot.vpc.controller;

import java.net.Socket;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class RequestFilterController {

    public static void main(String[] args){

        //

    }

    public static void requestIpFilter(Socket socket) {

        //

    }

    public static void requestLinkFilter(Socket socket) {

        //

    }

    public static void requestIpFilter(VPCSRequest vPCSRequest, VPCSResponse vPCSResponse) {

        //

    }

    public static void requestLinkFilter(VPCSRequest vpcsRequest, VPCSResponse vPCSResponse) {

        //

    }

}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/RequestFixController.java
package I_Q.hvpcs.boot.vpc.controller;

import java.net.Socket;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

```

 **/

```

public class RequestFixController {
    public static void main(String[] args){
    }
    public static void requestIpFix(Socket socket) {

    }
    public static void requestLinkFix(Socket socket) {

    }
    public static void requestIpFix(VPCSRequest vPCSRequest, VPCSResponse vPCSResponse) {

    }
    public static void requestLinkFix(VPCSRequest vPCSRequest, VPCSResponse vPCSResponse) {

    }
}

```

```

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/RequestRecordController.java
package I_Q.hvpcs.boot.vpc.controller;

```

```

import java.io.IOException;
import java.io.UnsupportedEncodingException;
import java.net.URLDecoder;
import I_Q.hvpcs.boot.sets.VPCSRequest;
import I_Q.hvpcs.boot.sets.VPCSResponse;
import S_A.pheromone.IMV_SQI;

```

```

/*

```

```

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

```

```

public class RequestRecordController {
    public static void requestIpRecoder(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
        vPCSRequest.setRequestIp(
vPCSResponse.socket.getInetAddress().getHostAddress());
        vPCSRequest.setRequestName(
vPCSResponse.socket.getInetAddress().getHostName());
    }
    @SuppressWarnings("unchecked")
    public static void requestLinkRecoder(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
        int length = 0;
        byte[] rbytes = new byte[1600];
        try {
            vPCSResponse.inputStream = vPCSResponse.socket.getInputStream();

```

```
length = vPCSResponse.inputStream.read(rbytes);
    } catch (IOException e) {
        e.printStackTrace();
    }
    if (length < 1) {
        System.out.flush();
        S_logger.Log.logger.info("" + vPCSResponse.getHashCode() + "-check");
        vPCSResponse.setErrorCode(204);
        return;
    }
    byte[] bytes = new byte[length];
    for (int i = 0; i < length; i++) {
        bytes[i] = rbytes[i];
    }
    String stringInput = new String(bytes);
    rbytes = null;
    String temp = stringInput.substring(0, length > 60 ? 60 : length);
    System.out.flush();
    S_logger.Log.logger.info("" + vPCSResponse.getHashCode() + "->" + temp);
    String[] stringInputs = stringInput.split("\r\n");
    String mess = stringInputs[0];
    if (null == mess) {
        System.out.flush();
        S_logger.Log.logger.info("" + vPCSResponse.getHashCode() + "-check mess");
        vPCSResponse.setErrorCode(204);
        return;
    }
    if (mess.length() > 3000) {
        System.out.flush();
        S_logger.Log.logger.info("" + vPCSResponse.getHashCode() + "-check length");
        vPCSResponse.setErrorCode(204);
        return;
    }
    if (mess.equalsIgnoreCase("") || !mess.contains("HTTP")) {
        System.out.flush();
        S_logger.Log.logger.info("" + vPCSResponse.getHashCode() + "-check contains");
        vPCSResponse.setErrorCode(204);
        return;
    }
    String temp_mess = mess.toLowerCase();
    if (temp_mess.contains(".php") || temp_mess.contains(".jsp")
|| temp_mess.contains(".wav") || temp_mess.contains(".jar")
|| temp_mess.contains(".rar") || temp_mess.contains(".zip")
|| temp_mess.contains("phtun") || temp_mess.contains("who")
|| temp_mess.contains("xht") || temp_mess.contains("del")
|| temp_mess.contains("dir") || temp_mess.contains("net")
|| temp_mess.contains("def") || temp_mess.contains("0x"))
```

```
|| temp_mess.contains("/") || temp_mess.contains("ash")
|| temp_mess.contains("std") || temp_mess.contains("pid")
|| temp_mess.contains("upd") || temp_mess.contains("loa")
|| temp_mess.contains("file") || temp_mess.contains("#if")
|| temp_mess.contains("#el")) {
    System.out.flush();
    S_logger.Log.logger.info("'" + vPCSResponse.getHashCode() + "-check tails");
    vPCSResponse.setErrorCode(204);
    return;
}
String[] type = mess.split(" ");
if (type.length < 3) {
    vPCSResponse.setErrorCode(204);
    return;
}
if (type[1].contains("::")) {
    vPCSResponse.setErrorCode(204);
    return;
}
String[] content = type[1].split("\\?");
vPCSRequest.setRequestIsRest(false);
if (content.length == 2) {
    if (content[0].contains(".woff") || content[0].contains(".ttf")) {
vPCSResponse.setErrorCode(204);
return;// zhengque baohan + //cuowubao baohan
    }
    if (content[1] == null || content[1].equals(""))
        || !content[1].contains("=")) {
vPCSResponse.setErrorCode(204);
return;// cuowubao baohan
    }
    vPCSRequest.setRequestIsRest(true);
    if (content[0].equalsIgnoreCase("/") || content[0].contains(".ash")
        || content[0].contains(".txt") || content[0].contains(".js")
        || content[0].contains(".css")
        || content[0].contains(".html")) {
vPCSRequest.setRequestIsRest(false);
    }
}
vPCSRequest.setRequestLink(content[0]);
if (!vPCSRequest.getRequestIsRest()) {
    if (content[0].equals("/")) {
vPCSRequest.setRequestLink(content[0] + "index.html");
    }
    System.out.flush();
    S_logger.Log.logger.info("'" + vPCSResponse.getHashCode() + "-check"
        + vPCSRequest.getRequestLink());
```



```

        return;
    }

    String[] column = content[1].split("&");
    IMV_SQI data = new IMV_SQI();
    for (String cell : column) {
        String[] cells = cell.split("=");
        if (cells.length > 1) {
            String string = null;
            try {
                string = URLDecoder.decode(cells[1], "UTF-8");
            } catch (UnsupportedEncodingException e) {
                System.out.flush();
                S_logger.Log.logger.info("'" +
                    vPCSResponse.getHashCode() + "-check decoding");
                vPCSResponse.setErrorCode(204);
                return;
            }
            if (null == string) {
                System.out.flush();
                S_logger.Log.logger.info("'" +
                    vPCSResponse.getHashCode() + "-check rest");
                vPCSResponse.setErrorCode(204);
                return;
            }
            if (string.length() > 800) { // fix later
                string = string.substring(0, 800);
            }
            data.put(cells[0], string);
        }
    }

    // load
    vPCSRequest.setRequestValue(data);
}

}

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/RequestRecordControllerBUp.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/ResponseController.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/controller/ServerInitController.java
package I_Q.hvpcs.boot.vpc.controller;

import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.net.ServerSocket;
import java.util.Properties;
import I_Q.hvpcs.boot.sets.StableBlob;
import I_Q.hvpcs.boot.sets.StableSets;
import I_Q.hvpcs.boot.vpc.process.TimeProcess;
import I_Q.hvpcs.boot.vpc.sleeper.SleeperHall;
import I_Q.hvpcs.boot.vpc.sleeper.Sleeper_X;
import I_Q.hvpcs.boot.vpc.sleeper.TimeHall;

```

```
import ME.VPC.M.app.App;

import I_Q.hvpcs.lyg.common.utils.DetaUtil;

/*
 * 个人著作权人，作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com
 * , lyg.tin@gmail.com- lygtin@sina.com
 * 15116110525, ,
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段208号
 * 阳光家园别墅小区 第十栋别墅 第三层
 */

@SuppressWarnings("unused")
public class ServerInitController {
    private Properties properties;
    private int port;
    private App appReg;
    {
        properties = new Properties();
        System.out.flush();
        S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器资源载入:成功！");
    }
    public void init() throws IOException {
        port = 8000;
        System.out.flush();
        S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器端口启动:" + port);
        DetaUtil.initDB();
        System.out.flush();
        S_logger.Log.logger.info("'" + "----德塔VPCS数据库服务器DMA确认:成功！");
        S_logger.Log.logger.info("'" + "----初始大文件:成功！");
        StableBlob.initBlobLimit();
        S_logger.Log.logger.info("'" + "----初始固定数据:成功！");
        StableSets.intListPartBytes();
        S_logger.Log.logger.info("'" + "----初始http数据:成功！");
        StableSets.initHttpContentTypeMap();
    }
    public void initServer(App app) throws IOException, InterruptedException {
        appReg = app;
        System.out.flush();
        S_logger.Log.logger.info("'" + "----DETA HVPCS--1.9070");
        System.out.flush();
        S_logger.Log.logger.info("'" + "----Author: 罗瑶光");
        System.out.flush();
        S_logger.Log.logger.info("'" + "----浏阳德塔软件开发有限公司开源项目");
        TimeProcess timeProcess = new TimeProcess();
        timeProcess.begin();
        SleeperHall sleeperHall = new SleeperHall();
    }
}
```

```

        init();
        timeProcess.end();
        System.out.flush();
        S_logger.Log.logger.info("" +
"----德塔VPCS前端服务器启动一切正常-总耗时:" + timeProcess.duration() + "毫秒");
        ServerSocket server = new ServerSocket(port);
        long clearTime = System.currentTimeMillis();
        TimeHall timeHall = new TimeHall();
        timeHall.registerServer(server);
        timeHall.registerSleeperHall(sleeperHall);
        timeHall.start();
        runServer(server, sleeperHall, clearTime, timeHall);
    }

    public void runServer(ServerSocket server, SleeperHall sleeperHall,
        long clearTime, TimeHall timeHall)
        throws InterruptedException, IOException {
        Here: while (true) { // later do 6 dead id filter
            if (sleeperHall.getThreadsCount() > 2) {
sleeperHall.callSkivvy();
Thread.sleep(3000);
continue Here;
            }
            if (0 >= timeHall.getMaxLimist()) {
System.out.flush();
S_logger.Log.logger.info("" + sleeperHall.getThreadsCount());
timeHall.resetLimists();// 20250523
continue Here;
            }
            Sleeper_X sleeper = new Sleeper_X();
            sleeper.NE = appReg;
            if (!server.isClosed()) {
sleeper.pillows.vPCSResponse.socket = server.accept();
            }
            if (!server.isClosed()
                && null != sleeper.pillows.vPCSResponse.socket) {
sleeperHall.addExecSleeper(sleeper.hashCode(), sleeper);
sleeper.start();
            }
        }
    }
}

```

I_Q.hvpcs.boot.vpc.process

```

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/process/CacheProcess.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/process/JsonProcess.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/process/ThirdPartyProcess.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/process/TimeProcess.java

```

I_Q.hvpcs.boot.vpc.sleeper

```

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/Sleeper_O.java
package I_Q.hvpcs.boot.vpc.sleeper;

```

```

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Sleeper_O extends Sleeper implements Runnable {
    public boolean needReturn(String string) { // 7api sonar lint 4Z
        if (null == pillows.vPCSResponse.socket) { // fix later..
            pillows.vPCSResponse.closeAllBeforeGC();
            setWaitingForShake(false, pillows.vPCSResponse.getHashCode(),
                "-check " + string + " null err");
            return true;
        }
        if (204 == pillows.vPCSResponse.errorCode) { // fix later..
            pillows.vPCSResponse.returnValue(pillows.vPCSResponse.errorCode);
            setWaitingForShake(false, pillows.vPCSResponse.getHashCode(),
                "-check " + string + " 404 err");
            return true;
        }
        return false;
    }
}

//DetaSrc.org.deta.boot.vpc.controller.RequestFilterController
// .requestIpFilter(vPCSRequest, vPCSResponse);
// DetaSrc.org.deta.boot.vpc.controller.RequestFilterController
// .requestLinkFilter(vPCSRequest, vPCSResponse);
// DetaSrc.org.deta.boot.vpc.controller.RequestFixController
// .requestIpFix(vPCSRequest, vPCSResponse);
// DetaSrc.org.deta.boot.vpc.controller.RequestFixController
// .requestLinkFix(vPCSRequest, vPCSResponse);
//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/Sleeper_X.java
package I_Q.hvpcs.boot.vpc.sleeper;

import C_A.OSI.AOP.MS.VPC.server.ServerForward_Standard;

import I_Q.hvpcs.boot.sets.Sleeper_S;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

```

```
*/  
  
public class Sleeper_X extends Sleeper_O implements Runnable {  
    public Sleeper_X() {  
        Sleeper_S hugPillows = new Sleeper_S();  
        hugPillows.vPCSRequest = new VPCSRequest();  
        hugPillows.vPCSResponse = new VPCSResponse();  
        hugPillows.vPCSResponse.setHashCode(this.hashCode());  
        pillows = hugPillows;  
    }  
    public void run() {  
        // later trif  
        I_Q.hvpcs.boot.vpc.controller.RequestRecordController  
.requestIpRecoder(pillows.vPCSRequest, pillows.vPCSResponse);  
        if (null == pillows.vPCSResponse.socket) { // fix later..  
            setWaitingForShake(false, pillows.vPCSResponse.getHashCode(),  
"-check requestIpRecoder null err");  
            return;  
        }  
        I_Q.hvpcs.boot.vpc.controller.RequestRecordController  
.requestLinkRecoder(pillows.vPCSRequest, pillows.vPCSResponse);  
        if (needReturn("requestLinkRecoder")) {  
            return;  
        }  
        //DetaSrc.org.deta.boot.vpc.vision.ForwardVision  
        //        .getForwardType(pillows.vPCSRequest, pillows.vPCSResponse);  
        //if (needReturn("getForwardType")) {  
        //    return;  
        //}  
        //DetaSrc.org.deta.boot.vpc.vision.ForwardVision  
        //        .forwardToRestMap(pillows.vPCSRequest, pillows.vPCSResponse);  
        //if (needReturn("forwardToRestMap")) {  
        //    return;  
        //}  
        ServerForward_Standard.IQ_ForwardType(pillows.vPCSRequest, pillows.vPCSResponse);  
        if (needReturn("getForwardType")) {  
            return;  
        }  
        ServerForward_Standard.forwardToRestMap(pillows.vPCSRequest, pillows.vPCSResponse, NE);  
        if (needReturn("forwardToRestMap")) {  
            return;  
        }  
        //ServerForward_Standard.IQ_ForwardType(vPCSRequest, vPCSResponse);  
        //if (vPCSResponse.getSocket().isClosed()) {  
        //    return;  
        //}  
        //ServerForward_Standard.forwardToRestMap(vPCSRequest, vPCSResponse);  
        //if (vPCSResponse.getSocket().isClosed()) {
```

```
//      return;
//}

        I_Q.hvpcs.boot.vpc.vision.RestMapView.returnResponse(this);
        pillows.vPCSResponse.closeAllBeforeGC();
        System.out.flush();
        S_logger.Log.logger.info("" + pillows.vPCSResponse.getHashCode()
+ "-check returnResponse waiting for skivvy");
    }
}

//DetaSrc.org.deta.boot.vpc.controller.RequestFilterController
//.requestIpFilter(vPCSRequest, vPCSResponse);
// DetaSrc.org.deta.boot.vpc.controller.RequestFilterController
//.requestLinkFilter(vPCSRequest, vPCSResponse);
// DetaSrc.org.deta.boot.vpc.controller.RequestFixController
//.requestIpFix(vPCSRequest, vPCSResponse);
// DetaSrc.org.deta.boot.vpc.controller.RequestFixController
//.requestLinkFix(vPCSRequest, vPCSResponse);
//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/Sleeper.java
package I_Q.hvpcs.boot.vpc.sleeper;

import java.io.IOException;

import C_A.OSI.AOP.MS.VPC.server.ServerSleeper_Standard;

import I_Q.hvpcs.boot.sets.Sleeper_S;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/

public class Sleeper extends ServerSleeper_Standard {

    public Sleeper_S pillows;

    public void setWaitingForShake(boolean isWaitingForShake, Integer integer,
        String string) {

        pillows.vPCSRequest.setWaitingForShake(isWaitingForShake);
        System.out.flush();
        S_logger.Log.logger.info("" + integer + "- set WaitingForShake(false)");
        System.out.flush();
        S_logger.Log.logger.info("" + pillows.vPCSResponse.getHashCode() + string);
    }

    public void closeInputStream() throws IOException {

        if (null != this.pillows.vPCSResponse) {

            this.pillows.vPCSResponse.closeInputStream();

        }

    }
}
```

```
public void closeOutputStream() throws IOException {
    if (null != this.pillows.vPCSResponse) {
        this.pillows.vPCSResponse.closeOutputStream();
    }
}

public void closeAllBeforeGC() throws IOException, InterruptedException {
    if (null != this.pillows.vPCSRequest) {
        this.pillows.vPCSRequest.closeAllBeforeGC();
    }
    if (null != this.pillows.vPCSResponse) {
        pillows.vPCSResponse.closeAllBeforeGC();
    }
}

public void feedAndCloseAllBeforeGC()
    throws IOException, InterruptedException {
    if (null != this.pillows.vPCSResponse) {
        pillows.vPCSResponse.returnValue(204);
    }
}

public void closePrintWriter() throws IOException {
    if (null != this.pillows.vPCSResponse) {
        this.pillows.vPCSResponse.closePrintWriter();
    }
}

public void closeSocket() throws IOException {
    if (null != this.pillows.vPCSResponse) {
        this.pillows.vPCSResponse.closeSocket();
    }
}

public VPCSRequest getVPCSRequest() {
    return this.pillows.vPCSRequest;
}

public Sleeper() {
    pillows = new Sleeper_S();
    pillows.vPCSRequest = new VPCSRequest();
    pillows.vPCSResponse = new VPCSResponse();
    pillows.vPCSResponse.setHashCode(this.hashCode());
}

public boolean getWaitingForShake() {
    return pillows.vPCSRequest.getWaitingForShake();
}

public void goToBedroom(I_Q.hvpcs.boot.sets.VPCSRequest vPCSRequest,
    I_Q.hvpcs.boot.sets.VPCSResponse vPCSResponse) {
    this.vPCSRequest = vPCSRequest;
    this.vPCSResponse = vPCSResponse;
    vPCSResponse.setHashCode(this.hashCode());
}
```

```
public void goToBedroom(String port,
    I_Q.hvpcs.boot.sets.VPCSRequest vPCSRequest,
    I_Q.hvpcs.boot.sets.VPCSResponse vPCSResponse) {
    // this.app = app;
    vPCSRequest.I_RequestPort(port);
    goToBedroom(vPCSRequest, vPCSResponse);
}

public void sleeping() {
    this.start();
}

}

// public void run(){
// DetaSrc.org.deta.boot.vpc.controller.RequestRecordController
// .requestIpRecoder(vPCSRequest, vPCSResponse);
// if(null == vPCSResponse.socket) { // fix later..
//     setWaitingForShake(false);
//     System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//     + "- set WaitingForShake(false)");
//     System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//     + "-check requestIpRecoder null err");
//     return;
// }
// DetaSrc.org.deta.boot.vpc.controller.RequestRecordController
// .requestLinkRecoder(vPCSRequest, vPCSResponse);
// if(null == vPCSResponse.socket) { // fix later..
//     setWaitingForShake(false);
//     System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//     + "- set WaitingForShake(false)");
//     System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//     + "-check requestLinkRecoder null err");
//     return;
// }
// //DetaSrc.org.deta.boot.vpc.controller.RequestFilterController
// // .requestIpFilter(vPCSRequest, vPCSResponse);
// //
// //DetaSrc.org.deta.boot.vpc.controller.RequestFilterController
// // .requestLinkFilter(vPCSRequest, vPCSResponse);
// //
// //DetaSrc.org.deta.boot.vpc.controller.RequestFixController
// // .requestIpFix(vPCSRequest, vPCSResponse);
// //
// //DetaSrc.org.deta.boot.vpc.controller.RequestFixController
// // .requestLinkFix(vPCSRequest, vPCSResponse);
// //
// if(204 == vPCSResponse.errorCode) { // fix later..
//     vPCSResponse.returnValue(vPCSResponse.errorCode);
//     setWaitingForShake(false);
```



```
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"- set WaitingForShake(false)");
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"-check requestLinkRecoder 204 err");
//      return;
// }

// DetaSrc.org.deta.boot.vpc.vision.ForwardVision
// .getForwardType(vPCSRequest, vPCSResponse);
// if(null == vPCSResponse.socket) { //fix later..
//      vPCSResponse.closeAllBeforeGC();
//      setWaitingForShake(false);
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"- set WaitingForShake(false)");
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"-check getForwardType null err");
//      return;
// }
// if(204 == vPCSResponse.errorCode) { //fix later..
//      vPCSResponse.returnValue(vPCSResponse.errorCode);
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"- set WaitingForShake(false)");
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"-check getForwardType 204 err");
//      setWaitingForShake(false);
//      return;
// }

// DetaSrc.org.deta.boot.vpc.vision.ForwardVision
// .forwardToRestMap(vPCSRequest, vPCSResponse);
// if(null == vPCSResponse.socket) { //fix later..
//      vPCSResponse.closeAllBeforeGC();
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"- set WaitingForShake(false)");
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"-check forwardToRestMap null err");
//      setWaitingForShake(false);
//      return;
// }
// if(204 == vPCSResponse.errorCode) { //fix later..
//      vPCSResponse.returnValue(vPCSResponse.errorCode);
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"- set WaitingForShake(false)");
//      System.out.flush();S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
//      +"-check forwardToRestMap 404 err");
//      setWaitingForShake(false);
//      return;
// }

// DetaSrc.org.deta.boot.vpc.vision.RestMapView
```

```
// .returnResponse(this);

// vPCSRResponse.closeAllBeforeGC();

// System.out.flush();S_logger.Log.logger.info("" + vPCSRResponse.getHashCode()

// +"-check returnResponse waiting for skivvy");

//}

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/SleeperHall.java
package I_Q.hvpcs.boot.vpc.sleeper;

import java.io.IOException;

import java.util.Date;

import java.util.Iterator;

import java.util.concurrent.ConcurrentHashMap;

import I_Q.hvpcs.boot.rest.VPC;

/*

* 个人著作权人,作者 罗瑶光,浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class SleeperHall {

    private ConcurrentHashMap<Integer, Sleeper_X> sleepersMap;

    public SleeperHall() {

        sleepersMap = new ConcurrentHashMap<>();

    }

    public int getThreadsCount() {

        if (null == sleepersMap) {

            sleepersMap = new ConcurrentHashMap<>();

            return 0;

        }

        return sleepersMap.size();

    }

    public void addExecSleeper(Integer sid,

        Sleeper_X serverSleeper_Standard) {

        sleepersMap.put(sid, serverSleeper_Standard);

    }

    public void removeThreadById(Integer sid) {

        try {

            if (sleepersMap.containsKey(sid)) {

                Sleeper_X sleeper = (Sleeper_X) sleepersMap.get(sid);//later trif

                sleeper.closeAllBeforeGC();

                sleepersMap.remove(sid);

                sleeper = null;

                System.out.flush();

                S_logger.Log.logger.info("" + sid + " -free");

            }

        } catch (IOException e) {
```

```
        e.printStackTrace();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

public void callSkivvy() {
    Iterator<Integer> iterater = sleepersMap.keySet().iterator();
    while (iterater.hasNext()) {
        Integer integer = iterater.next();
        Sleeper_X sleeper = (Sleeper_X) sleepersMap.get(integer); //later trif
        if (null != sleeper) {
            if (!sleeper.getWaitingForShake()) {
                sleepersMap.remove(integer);
                sleeper = null;
                System.out.flush();
                S_logger.Log.logger.info("'" + integer + new Date(
                    System.currentTimeMillis()).toString()
                    + "-h free");
            }
        }
    }
}

public int getZipDownloadCount() {
    int output = 0;
    Iterator<Integer> iterator = sleepersMap.keySet().iterator();
    while (iterator.hasNext()) {
        if (VPC.getPath(sleepersMap.get(iterator.next()))
            .getVPCSRRequest().getRequestLink()).contains(
            ".zip")) {
            output++;
        }
    }
    return output;
}

public void clearWaitingState() {
    //TODO Auto-generated method stub
    //Iterator<Integer> iterater= sleepersMap.keySet().iterator();
    //while(iterater.hasNext()) {
    //    Integer integer= iterater.next();
    //    Sleeper sleeper= sleepersMap.get(integer);
    //    if(!sleeper.waitingForShake) {
    //        sleeper.closeAllBeforeGC();
    //        Thread.sleep(10);
    //    }
    //}
}
```

```
//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/sleeper/TimeHall.java
package I_Q.hvpcs.boot.vpc.sleeper;

import java.io.IOException;

import java.net.ServerSocket;

import I_Q.hvpcs.boot.sets.StableBlob;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class TimeHall extends Thread implements Runnable {

    public boolean waitingForShake = false;

    private ServerSocket serverSocket;

    private SleeperHall sleeperHall;

    private int maxLimist = 0;

    public void registerServer(ServerSocket server) throws IOException {

        serverSocket = server;

    }

    public void registerSleeperHall(SleeperHall hall) {

        if (null == sleeperHall) {

            this.sleeperHall = hall;

        }

    }

    public TimeHall() {

    }

    public void resetLimists() {

        maxLimist = 70;

    }

    public int getMaxLimist() {

        maxLimist -= 1;

        System.out.flush();

        S_logger.Log.logger.info("" + maxLimist);

        return maxLimist;

    }

    public void run() {

        while (true) {

            try {

                maxLimist = 70;

                Thread.sleep(1000 * 60 * 15);

                if (null != serverSocket) {

                    if (null != this.sleeperHall) {

                        sleeperHall.callSkivvy();

                        StableBlob.checkException();

                    }

                }

            }

        }

    }

}
```

```

        vPCSRequest.setRequestForwardType("bytes");
        vPCSResponse.setResponseContentType(
            "content-type: application/font-woff \n\n");
        return;
    }
    vPCSRequest.setRequestForwardType("rest");
    return;

    }
    String filePath = VPC.getFilePath(vPCSRequest.getRequestLink());
    String fileString = URLDecoder.decode(filePath, "UTF8");
    File file = new File(fileString);
    System.out.flush();
    S_logger.Log.logger.info("'" + "-fi-" + fileString);
    if (!file.exists() || !file.isFile()) {
fileString = URLDecoder.decode(filePath, "UTF8");
File fileUTF8 = new File(fileString);
System.out.flush();
S_logger.Log.logger.info("'" + "-fi-" + fileString);
if (!fileUTF8.exists() || !fileUTF8.isFile()) {
    System.out.flush();
    S_logger.Log.logger.info("'" + vPCSResponse.getHashCode()
        + "-getForwardType unfile err");
    vPCSResponse.setErrorCode(404);
    return; // fix later
}

    }
    String code = VPC.getCode(filePath);
    vPCSRequest.setRequestFilePath(fileString);
    vPCSRequest.setRequestFileCode(code);
    // init---
    vPCSRequest.setRequestForwardType("bytes");
    vPCSResponse.setResponseContentType("content-type: text/html \n\n");
    // ---
    String[] stings = filePath.split("\\.");
    if (1 < stings.length) {
stings[1] = stings[1] == null ? "" : stings[1];
if (StableSets.contentTypes.containsKey(stings[1])) {
    if (stings[1].contains(".js") || stings[1].contains(".map")) {
        if (code.equalsIgnoreCase("UTF-8")) {
            vPCSResponse.setResponseContentType(
                StableSets.contentTypes.get(stings[1]));
            return;
        }
    }
}
    vPCSResponse.setResponseContentType(
        StableSets.contentTypes.get(stings[1]));
    return;
}

```

```
}

    }

    } catch (Exception e) {

        System.out.flush();

        S_logger.Log.logger.info(""" + e.getMessage());

        System.out.flush();

        S_logger.Log.logger.info(""" +

vPCSResponse.getHashCode() + "-getForwardType 204 err");

        vPCSResponse.setErrorCode(204);

    }

}

public static void forwardToRestMap(VPCSRequest vPCSRequest,

    VPCSResponse vPCSResponse, App NE) {

    try {

        if (vPCSRequest.getRequestForwardType() == null) {

System.out.flush();

S_logger.Log.logger.info(""" + "-getRequestForwardType 204");

vPCSResponse.setErrorCode(204);

return;

        }

        if (vPCSRequest.getRequestForwardType().equalsIgnoreCase("rest")) {

System.out.flush();

S_logger.Log.logger.info(""" + "-rest");

RestMapVision_XP_rest.processRest(vPCSRequest, vPCSResponse, NE);

return;

        }

        if (vPCSRequest.getRequestForwardType().equalsIgnoreCase("bytes")) {

System.out.flush();

S_logger.Log.logger.info(""" + "-bytes");

RestMapVision_XP_bytes.processBytes(vPCSRequest, vPCSResponse);

return;

        }

    } catch (Exception e) {

        System.out.flush();

        S_logger.Log.logger.info(""" + e.getMessage());

        System.out.flush();

        S_logger.Log.logger.info(""" +

vPCSResponse.getHashCode() + "-forwardToRestMap 204 err");

        vPCSResponse.setErrorCode(204);

    }

}

}

// if(filePath.contains(".png")){

// vPCSResponse.setResponseContentType("content-type:

// image/png \n\n");

// return;

// }
```

```
// if(filePath.contains(".ico")){  
  
// vPCSRResponse.setResponseContentType("content-type:  
  
// image/ico \n\n");  
  
// return;  
  
// }  
  
// if(filePath.contains(".jpeg")){  
  
// vPCSRResponse.setResponseContentType("content-type:  
  
// image/jpeg \n\n");  
  
// return;  
  
// }  
  
// if(filePath.contains(".jpg")){  
  
// vPCSRResponse.setResponseContentType("content-type:  
  
// image/jpg \n\n");  
  
// return;  
  
// }  
  
// if(filePath.contains(".gif")){  
  
// vPCSRResponse.setResponseContentType("content-type:  
  
// image/gif \n\n");  
  
// return;  
  
// }  
  
// if(filePath.contains(".wav")){  
  
// vPCSRResponse.setResponseContentType("content-type:  
  
// audio/wav \n\n");  
  
// return;  
  
// }  
  
// if(filePath.contains(".js") &&  
// code.equalsIgnoreCase("UTF-8")){  
  
// vPCSRResponse.setResponseContentType(  
// "content-type: text/javascript; charset:UTF-8 \n\n");  
// return;  
// }  
  
// if(filePath.contains(".map") &&  
// code.equalsIgnoreCase("UTF-8")){//  
// vPCSRResponse.setResponseContentType(  
// "content-type: text/plain; charset:UTF-8 \n\n");  
// return;  
// }  
  
// if(filePath.contains(".css")){  
  
// vPCSRResponse.setResponseContentType("content-type:  
// text/css \n\n");  
  
// return;  
  
// }  
  
// if(filePath.contains(".html")){  
  
// vPCSRResponse.setResponseContentType("content-type:  
// text/html \n\n");  
  
// return;  
  
// }
```



```
// if(filePath.contains(".txt")){
// vPCSRResponse.setResponseContentType("content-type:
// text/plain \n\n");
// return;
// }
// if(filePath.contains(".xml")){
// vPCSRResponse.setResponseContentType("content-type:
// application/xml \n\n");
// return;
// }
// if(filePath.contains(".pdf")){
// vPCSRResponse.setResponseContentType("content-type:
// application/pdf \n\n");
// return;
// }
// if(filePath.contains(".zip")){
// vPCSRResponse.setResponseContentType("content-type:
// application/zip \n\n");
// return;
// }
// if(filePath.contains(".rar")){
// vPCSRResponse.setResponseContentType("content-type:
// application/rar \n\n");
// return;
// }
// if(filePath.contains(".jar")){
// vPCSRResponse.setResponseContentType("content-type:
// application/jar \n\n");
// return;
// }
// if(filePath.contains(".pptx")){
// vPCSRResponse.setResponseContentType("content-type:
// application/pptx \n\n");
// return;
// }
// if(!StableBlob.addBlobLimit(filePath,
// System.currentTimeMillis())) { //fix later
// System.out.flush();S_logger.Log.logger.info("" + vPCSRResponse.getHashCode()+"-dreaming
// asyc");
// vPCSRResponse.returnValue(204);
// return;
// }
//
// if(vPCSRRequest.getRequestForwardType().equalsIgnoreCase("buffer")){
//     System.out.flush();
//     S_logger.Log.logger.info("" + "-buffer");
//     RestMapVision_XP_buffer.processBuffer(vPCSRRequest, vPCSRResponse);
```

```
//      return;
// }
// if(vPCSRequest.getRequestForwardType().equalsIgnoreCase("bytesBuffer")){
//      System.out.flush();
//      S_logger.Log.logger.info("" + "-bytesBuffer");
//      RestMapVision_XP_bufferBytes.processBufferBytes(vPCSRequest, vPCSResponse);
//      return;
// }
// if(vPCSRequest.getRequestForwardType().equalsIgnoreCase("bytesWithoutZip")){
//      System.out.flush();
//      S_logger.Log.logger.info("" + "-bytesWithoutZip");
//      RestMapVision_XP_bytesZip.processBytesWithoutZip(vPCSRequest, vPCSResponse);
//      return;
// }

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_buffer.java
package I_Q.hvpcs.boot.vpc.vision;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileInputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.io.OutputStreamWriter;
import I_Q.hvpcs.cache.DetaCacheManager;
import I_Q.hvpcs.boot.sets.StableBlob;
import I_Q.hvpcs.boot.sets.VPCSRequest;
import I_Q.hvpcs.boot.sets.VPCSResponse;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class RestMapVision_XP_buffer extends RestMapVision {

    public static void processBuffer(VPCSRequest vPCSRequest,
                                     VPCSResponse vPCSResponse) {

        try {

            if (null == vPCSRequest.getRequestFilePath()
                || null == vPCSRequest.getRequestLink()) {

                return;

            }

            String builderToString;

            if (DetaCacheManager.getCacheOfString(
                vPCSRequest.getRequestFilePath()) != null) {

                builderToString = DetaCacheManager
```

```
.getCacheOfString(vPCSRequest.getRequestFilePath());
    } else {
        StringBuilder stringBuilder = new StringBuilder();
        stringBuilder.append("HTTP/1.1 200 OK").append("\n");
        stringBuilder.append("Host:deta software \n");
        stringBuilder.append("Cache-control: max-age=315360000 \n");
        stringBuilder.append(vPCSResponse.getResponseContentType());
        File file = new File(vPCSRequest.getRequestFilePath());
        vPCSResponse.fileInputStream = new FileInputStream(file);
        vPCSResponse.inputStreamReader = new InputStreamReader(
            vPCSResponse.fileInputStream,
            vPCSRequest.getRequestFileCode());
        vPCSResponse.bufferedReader = new BufferedReader(
            vPCSResponse.inputStreamReader);
        String line = null;
        while ((line = vPCSResponse.bufferedReader
            .readLine()) != null) {
            stringBuilder.append(line);
        }
        builderToString = stringBuilder.toString();
        DetaCacheManager.putCacheOfString(
            vPCSRequest.getRequestFilePath(), builderToString);
        vPCSResponse.bufferedReader.close();
    }
    if (null != vPCSResponse.socket) {
        if (!vPCSResponse.socket.isClosed()) {
            OutputStream outputStream = vPCSResponse.socket
                .getOutputStream();
            OutputStreamWriter outputStreamWriter = new OutputStreamWriter(
                outputStream, vPCSRequest.getRequestFileCode());
            vPCSResponse.bufferedWriter = new BufferedWriter(
                outputStreamWriter);
            int j = builderToString.length();
            vPCSResponse.bufferedWriter.flush();
            vPCSResponse.bufferedWriter.write(builderToString, 0, j);
            vPCSResponse.bufferedWriter.close();
        }
    }
    System.out.flush();
    S_logger.Log.logger.info(""" + vPCSResponse.getHashCode() + "-f");
} catch (Exception e) {
    System.out.flush();
    S_logger.Log.logger.info(""" + e.getMessage());
    System.out.flush();
    S_logger.Log.logger.info(""" + vPCSResponse.getHashCode()
        + "-processBuffer bufferedWriter err");
    System.out.flush();
}
```

```

        S_logger.Log.logger.info("'" + vPCSRequest.getRequestLink());

        System.out.flush();

        S_logger.Log.logger.info("'" + vPCSRequest.getRequestFilePath());

        StableBlob.addException();

        return;
    }
}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_bufferBytes.java
package I_Q.hvpcs.boot.vpc.vision;

import java.io.DataOutputStream;

import java.io.File;

import java.io.FileInputStream;

import I_Q.hvpcs.cache.DetaCacheManager;

import I_Q.hvpcs.lyg.common.utils.GzipUtil;

import I_Q.hvpcs.boot.sets.StableBlob;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*
 * 个人著作权人,作者 罗瑶光,浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */public class RestMapVision_XP_bufferBytes extends RestMapVision {
    public static void processBufferBytes(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
        try {
            StringBuilder stringBuilder = new StringBuilder();
            stringBuilder.append("HTTP/1.1 200 OK").append("\n");
            stringBuilder.append("Host:deta software \n");
            stringBuilder.append("Cache-control: max-age=315360000 \n");
            stringBuilder.append("Content-Encoding:gzip \n");
            stringBuilder.append(vPCSResponse.getResponseContentType());
            String builderToString = stringBuilder.toString();
            String contentBuilderToString;
            if (DetaCacheManager.getCacheOfString(
                vPCSRequest.getRequestFilePath()) != null) {
                contentBuilderToString = DetaCacheManager
                    .getCacheOfString(vPCSRequest.getRequestFilePath());
            } else {
                StringBuilder contentBuilder = new StringBuilder();
                File file = new File(vPCSRequest.getRequestFilePath());
                vPCSResponse.fileInputStream = new FileInputStream(file);
                int len = 0;
                byte[] byteArray = new byte[1024];

```

```
while ((len = vPCSResponse.fileInputStream
        .read(byteArray)) != -1) {
    contentBuilder
        .append(new String(byteArray, 0, len, "UTF-8"));
}
vPCSResponse.fileInputStream.close();
Thread.sleep(75);
contentBuilderToString = contentBuilder.toString();
DetaCacheManager.putCacheOfString(
    vPCSRequest.getRequestFilePath(),
    contentBuilderToString);
}
vPCSResponse.dataOutputStream = new DataOutputStream(
vPCSResponse.socket.getOutputStream());
vPCSResponse.dataOutputStream.flush();
vPCSResponse.dataOutputStream
.write(builderToString.getBytes("UTF-8"));
Thread.sleep(75);
byte[] bytes = null;
int i = 0;
long bytes_length = 0;
bytes = GzipUtil.compress(contentBuilderToString.getBytes("UTF-8"));
bytes_length = bytes == null ? 0 : bytes.length;
vPCSResponse.dataOutputStream.flush();
if (bytes_length > 10000) {
int last = bytes.length % 10000;
for (i = 0; i < bytes.length - 10000; i += 10000) {
    byte[] serparBytes = new byte[10000];
    for (int j = 0; j < 10000; j++) {
        serparBytes[j] = bytes[i + j];
    }
    vPCSResponse.dataOutputStream.write(serparBytes);
    vPCSResponse.dataOutputStream.flush();
}
byte[] serparBytes = new byte[last];
for (int j = 0; j < last; j++) {
    serparBytes[j] = bytes[i + j];
}
vPCSResponse.dataOutputStream.write(serparBytes);
vPCSResponse.dataOutputStream.flush();
} else if (bytes_length > 0) {
vPCSResponse.dataOutputStream.write(bytes);
vPCSResponse.dataOutputStream.flush();
}
vPCSResponse.dataOutputStream.close();
System.out.flush();
S_logger.Log.logger.info("'" + vPCSResponse.getHashCode() + "-f");
```

```

        } catch (Exception e) {
            System.out.flush();
            S_logger.Log.logger.info("" + e.getMessage());
            System.out.flush();
            S_logger.Log.logger.info("" + vPCSResponse.getHashCode()
+ "-processBufferBytes dataOutputStream err");
            System.out.flush();
            S_logger.Log.logger.info("" + vPCSRequest.getRequestLink());
            System.out.flush();
            S_logger.Log.logger.info("" + vPCSRequest.getRequestFilePath());
            StableBlob.addException();
            return;
        }
    }
}

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_bytes.java
package I_Q.hvpcs.boot.vpc.vision;

import java.io.ByteArrayOutputStream;
import java.io.DataOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.util.Iterator;
import java.util.List;
import I_Q.hvpcs.cache.DetaCacheManager;
import I_Q.hvpcs.lyg.common.utils.GzipUtil;
import I_Q.hvpcs.boot.sets.StableBlob;
import I_Q.hvpcs.boot.sets.StableSets;
import I_Q.hvpcs.boot.sets.VPCSRequest;
import I_Q.hvpcs.boot.sets.VPCSResponse;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */public class RestMapVision_XP_bytes extends RestMapVision {
    // 51-55这段while函数思想来自 这篇文章:
    // https://blog.csdn.net/top_code/article/details/41042413
    // 非常轻松处理len的长度溢出。谢谢。
    public static void processBytes(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse) {
        try {
            List<byte[]> list;
            vPCSResponse.dataOutputStream = new DataOutputStream(
                vPCSResponse.socket.getOutputStream());
            if (DetaCacheManager.getCacheOfBytesList(

```

```
vPCSRequest.getRequestFilePath()) != null) {  
    list = DetaCacheManager  
        .getCacheOfBytesList(vPCSRequest.getRequestFilePath());  
    } else {  
        // String fileString=  
        // URLDecoder.decode(vPCSRequest.getRequestFilePath(),  
        // "UTF8");  
        String fileString = vPCSRequest.getRequestFilePath();  
        File file = new File(fileString);  
        vPCSResponse.fileInputStream = new FileInputStream(file);  
        vPCSResponse byteArrayOutputStream = new ByteArrayOutputStream();  
        byte[] byteArray = new byte[1024];  
        int len = 0; // list = new ArrayList<>();  
        list = StableSets.getListPartBytes();  
        vPCSResponse byteArrayOutputStream.flush();  
        while ((len = vPCSResponse.fileInputStream.read(byteArray, 0,  
            1024)) != -1) {  
            vPCSResponse byteArrayOutputStream.write(byteArray, 0, len);  
            vPCSResponse byteArrayOutputStream.flush();  
        }  
        vPCSResponse.fileInputStream.close();  
        String temp = new String(vPCSResponse byteArrayOutputStream.toByteArray());  
        S_logger.Log.logger.info("-^-" + temp);  
        byte[] sniper = GzipUtil.compress(  
            vPCSResponse byteArrayOutputStream.toByteArray());  
        for(int i=0;i<sniper.length;i++) {  
            System.out.print(sniper[i]+"_");  
        }  
        long sniper_length = sniper == null ? 0 : sniper.length;  
        list.add(("Content-Length: " + sniper_length + " \n")  
            .getBytes("UTF8"));  
        list.add(  
            vPCSResponse.getResponseContentType().getBytes("UTF8"));  
        list.add(sniper);  
        DetaCacheManager.putCacheOfBytesList(  
            vPCSRequest.getRequestFilePath(), list);  
    }  
    Iterator<byte[]> iterator = list.iterator();  
    vPCSResponse.dataOutputStream.flush();  
    while (iterator.hasNext()) {  
        byte[] bytes = null;  
        int i = 0;  
        long bytes_length = 0;  
        bytes = iterator.next();  
        bytes_length = bytes == null ? 0 : bytes.length;  
        if (bytes_length > 10000) {  
            int last = bytes.length % 10000;
```

```

        for (i = 0; i < bytes.length - 10000; i += 10000) {
            byte[] serparBytes = new byte[10000];
            for (int j = 0; j < 10000; j++) {
                serparBytes[j] = bytes[i + j];
            }
            vPCSResponse.dataOutputStream.write(serparBytes);
            vPCSResponse.dataOutputStream.flush();
        }
        byte[] serparBytes = new byte[last];
        for (int j = 0; j < last; j++) {
            serparBytes[j] = bytes[i + j];
        }
        vPCSResponse.dataOutputStream.write(serparBytes);
        vPCSResponse.dataOutputStream.flush();
    } else if (bytes_length > 0) {
        vPCSResponse.dataOutputStream.write(bytes);
        vPCSResponse.dataOutputStream.flush();
    }
}

    }
    vPCSResponse.dataOutputStream.close();
    System.out.flush();
    S_logger.Log.logger.info(" " + vPCSResponse.getHashCode() + " -f");
} catch (Exception e) {
    System.out.flush();
    S_logger.Log.logger.info(" " + e.getMessage());
    System.out.flush();
    System.out
.println(vPCSResponse.getHashCode() + "-processBytes err");
    System.out.flush();
    S_logger.Log.logger.info(" " + vPCSRequest.getRequestLink());
    System.out.flush();
    S_logger.Log.logger.info(" " + vPCSRequest.getRequestFilePath());
    StableBlob.addException();
    return;
}
}
}

//list.add(0, vPCSResponse.getResponseContentType().getBytes("UTF8"));
// list.add(0, ("Content-Length: " + sniper_length + " \n").getBytes("UTF8"));
// list.add(0, ("Cache-control: max-age=315360000 \n").getBytes("UTF8"));
// list.add(0, ("Content-Encoding:Gzip \n").getBytes("UTF8"));
// list.add(0, "Accept-Ranges: bytes \n".getBytes("UTF8"));
// list.add(0, "Host:deta software \n".getBytes("UTF8"));
// list.add(0, "HTTP/1.1 200 OK \n".getBytes("UTF8"));

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_bytesZip.java
package I_Q.hvpcs.boot.vpc.vision;

import java.io.ByteArrayOutputStream;
import java.io.DataOutputStream;

```



```
import java.io.File;

import java.io.FileInputStream;

import java.net.URLDecoder;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

import I_Q.hvpcs.cache.DetaCacheManager;

import I_Q.hvpcs.boot.sets.StableBlob;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

 */public class RestMapView_XP_bytesZip extends RestMapView {

    // 327//dataOutputStream.write(GzipUtil.compress(contentBuilderToString.getBytes("UTF8")));

    public static void processBytesWithoutZip(VPCSRequest vPCSRequest,

        VPCSResponse vPCSResponse) {

        try {

            List<byte[]> list;

            vPCSResponse.dataOutputStream = new DataOutputStream(

                vPCSResponse.socket.getOutputStream());

            if (DetaCacheManager.getCacheOfBytesList(

                vPCSRequest.getRequestFilePath()) != null) {

                list = DetaCacheManager

                    .getCacheOfBytesList(vPCSRequest.getRequestFilePath());

            } else {

                vPCSResponse.fileInputStream = null;

                String string = vPCSRequest.getRequestFilePath();

                if (string.contains("/dna_pde_rna/")) {

                    string = URLDecoder.decode(string, "UTF8");

                    File file = new File(string);

                    vPCSResponse.fileInputStream = new FileInputStream(file);

                } else {

                    File file = new File(vPCSRequest.getRequestFilePath());

                    vPCSResponse.fileInputStream = new FileInputStream(file);

                }

                vPCSResponse byteArrayOutputStream = new ByteArrayOutputStream();

                byte[] byteArray = new byte[1024];

                int len = 0;

                list = new ArrayList<>();

                vPCSResponse.byteArrayOutputStream.flush();

                while ((len = vPCSResponse.fileInputStream.read(byteArray, 0,
```

```
1024)) != -1) {  
    vPCSResponse byteArrayOutputStream.write(byteArray, 0, len);  
    vPCSResponse byteArrayOutputStream.flush();  
}  
vPCSResponse.fileInputStream.close();  
byte[] sniper = vPCSResponse byteArrayOutputStream  
    .toArray();  
list.add(0,  
    vPCSResponse.getResponseContentType().getBytes("UTF8"));  
list.add(0, ("Content-Length: " + sniper.length + " \n")  
    .getBytes("UTF8"));  
list.add(0, ("Cache-control: max-age=315360000 \n")  
    .getBytes("UTF8"));  
list.add(0, "Accept-Ranges: bytes \n".getBytes("UTF8"));  
list.add(0, "Host:deta software \n".getBytes("UTF8"));  
list.add(0, "HTTP/1.1 200 OK \n".getBytes("UTF8"));  
list.add(sniper);  
DetaCacheManager.putCacheOfBytesList(  
    vPCSRequest.getRequestFilePath(), list);  
}  
Iterator<byte[]> iterator = list.iterator();  
vPCSResponse.dataOutputStream.flush();  
while (iterator.hasNext()) {  
byte[] bytes = null;  
int i = 0;  
long bytes_length = 0;  
bytes = iterator.next();  
bytes_length = bytes == null ? 0 : bytes.length;  
if (bytes_length > 10000) {  
    int last = bytes.length % 10000;  
    for (i = 0; i < bytes.length - 10000; i += 10000) {  
        byte[] serparBytes = new byte[10000];  
        for (int j = 0; j < 10000; j++) {  
            serparBytes[j] = bytes[i + j];  
        }  
        vPCSResponse.dataOutputStream.write(serparBytes);  
        vPCSResponse.dataOutputStream.flush();  
    }  
    byte[] serparBytes = new byte[last];  
    for (int j = 0; j < last; j++) {  
        serparBytes[j] = bytes[i + j];  
    }  
    vPCSResponse.dataOutputStream.write(serparBytes);  
    vPCSResponse.dataOutputStream.flush();  
} else if (bytes_length > 0) {  
    vPCSResponse.dataOutputStream.write(bytes);  
    vPCSResponse.dataOutputStream.flush();  
}
```

```

    }

    }

    vPCSResponse.dataOutputStream.close();

    if (null != vPCSResponse.byteArrayOutputStream) {
vPCSResponse.byteArrayOutputStream.close();

    }

    System.out.flush();

    S_logger.Log.logger.info("'" + vPCSResponse.getHashCode() + "-f");
} catch (Exception e) {

    System.out.flush();

    S_logger.Log.logger.info("'" + e.getMessage());

    System.out.flush();

    S_logger.Log.logger.info("'" + vPCSResponse.getHashCode()
+ "-processBytesWithoutZip fileInputStream err");

    System.out.flush();

    S_logger.Log.logger.info("'" + vPCSRequest.getRequestLink());

    System.out.flush();

    S_logger.Log.logger.info("'" + vPCSRequest.getRequestFilePath());

    StableBlob.addException();

    return;

}

}

}

/Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapVision_XP_rest.java
package I_Q.hvpcs.boot.vpc.vision;

import java.io.BufferedWriter;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import I_Q.hvpcs.boot.rest.VPC;
import I_Q.hvpcs.boot.sets.StableBlob;
import I_Q.hvpcs.boot.sets.VPCSRequest;
import I_Q.hvpcs.boot.sets.VPCSResponse;
import ME.VPC.M.app.App;

/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
*/public class RestMapVision_XP_rest extends RestMapVision {

    @SuppressWarnings("unchecked")

    public static void processRest(VPCSRequest vPCSRequest,
        VPCSResponse vPCSResponse, App NE) {

        String output;

        try {

            output = VPC.forward(vPCSRequest.getRequestLink(),

```

```

vPCSRequest.getRequestValue(), NE);

    vPCSResponse.printWriter = new PrintWriter(
new BufferedWriter(new OutputStreamWriter(
    vPCSResponse.socket.getOutputStream(), "UTF-8")),
true);
} catch (Exception e) {
    System.out.flush();
    S_logger.Log.logger.info(" " + e.getMessage());
    System.out.flush();
    S_logger.Log.logger.info(" " + vPCSResponse.getHashCode() + "-processRest err");
    System.out.flush();
    S_logger.Log.logger.info(" " + vPCSRequest.getRequestLink());
    System.out.flush();
    S_logger.Log.logger.info(" " + vPCSRequest.getRequestFilePath());
    StableBlob.addException();
    return;
}
if (output.length() > 0) {
    System.out.flush();
    S_logger.Log.logger.info(" " + "success YL");
    vPCSResponse.printWriter.flush();
    vPCSResponse.printWriter.println("HTTP/1.1 200 OK\n\n");
    output = output.charAt(0) == ""
? output.substring(1, output.length())
: output;
    output = output.charAt(output.length() - 1) == ""
? output.substring(0, output.length() - 1)
: output;
    vPCSResponse.printWriter.flush();
    vPCSResponse.printWriter.println(output.replace("\\\\", "\"));
    vPCSResponse.printWriter.close();
    System.out.flush();
    S_logger.Log.logger.info(" " + vPCSResponse.getHashCode() + "-f");
} else {
    vPCSResponse.printWriter.flush();
    vPCSResponse.printWriter.println("HTTP/1.1 204 OK\n\n");
    vPCSResponse.printWriter.close();
    System.out.flush();
    S_logger.Log.logger.info(" " +
vPCSResponse.getHashCode() + "-processRest length err");
    System.out.flush();
    System.out.print(vPCSRequest.getRequestLink());
    System.out.flush();
    System.out.print(vPCSRequest.getRequestFilePath());
    return;
}
}

```

```
}

//Trif_DNA-master/C_L_I/I_Q/hvpcs/boot/vpc/vision/RestMapView.java
package I_Q.hvpcs.boot.vpc.vision;

//import java.io.BufferedReader;

//import java.io.BufferedWriter;

//import java.io.ByteArrayOutputStream;

//import java.io.DataOutputStream;

//import java.io.File;

//import java.io.FileInputStream;

//import java.io.InputStreamReader;

//import java.io.OutputStream;

//import java.io.OutputStreamWriter;

//import java.io.PrintWriter;

//import java.net.URLDecoder;

//import java.util.ArrayList;

//import java.util.Iterator;

//import java.util.List;

//import org.lyg.cache.DetaCacheManager;

//import org.lyg.common.utils.GzipUtil;

//

//import DetaSrc.org.deta.boot.rest.VPC;

//import DetaSrc.org.deta.boot.sets.StableBlob;

import java.net.Socket;

import I_Q.hvpcs.boot.sets.VPCSRequest;

import I_Q.hvpcs.boot.sets.VPCSResponse;

import I_Q.hvpcs.boot.vpc.sleeper.Sleeper;

/*

* 个人著作权人,作者 罗瑶光,浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/public class RestMapView {

    public static void main(String[] args) {

    }

    public static void getResponse(Socket socket) {

    }

    public static void returnResponse(Sleeper sleeper) {

        sleeper.setWaitingForShake(false,

sleeper.pillows.vPCSResponse.getHashCode(),

sleeper.pillows.vPCSResponse.getHashCode() + "-e");

    }

    public static void getResponse(VPCSRequest vPCSRequest,

VPCSResponse vPCSResponse) {

        System.out.flush();

        S_logger.Log.logger.info("'" + vPCSResponse.getHashCode() + "-pe");
```

```
vPCSResponse.closeAllBeforeGC();

    }

}

//      public static void processRest(VPCSRequest vPCSRequest
//      , VPCSResponse vPCSResponse) {
// String output;
// try {
//      output = VPC.forward(vPCSRequest.getRequestLink()
//      , vPCSRequest.getRequestValue());
//      vPCSResponse.printWriter = new PrintWriter(new BufferedWriter(
//      new OutputStreamWriter(vPCSResponse.socket
//      .getOutputStream(), "UTF-8")), true);
// }catch(Exception e) {
//      System.out.flush();S_logger.Log.logger.info(" " + e.getMessage());
//      System.out.flush();S_logger.Log.logger.info(" " + vPCSResponse.getHashCode()
//      + "-processRest err");
//      System.out.flush();S_logger.Log.logger.info(" " + vPCSRequest.getRequestLink());
//      System.out.flush();S_logger.Log.logger.info(" " + vPCSRequest.getRequestFilePath());
//      StableBlob.addException();
//      return;
// }
// if(output.length()> 0) {
//      System.out.flush();S_logger.Log.logger.info(" " + "success YL");
//      vPCSResponse.printWriter.flush();
//      vPCSResponse.printWriter.println("HTTP/1.1 200 OK\n\n");
//      output=output.charAt(0)==' '?output.substring(1, output.length()):output;
//      output=output.charAt(output.length()-1)==' '?output.substring(0
//      , output.length()-1):output;
//      vPCSResponse.printWriter.flush();
//      vPCSResponse.printWriter.println(output.replace("\\\\", "\\"));
//      vPCSResponse.printWriter.close();
//      System.out.flush();S_logger.Log.logger.info(" " + vPCSResponse.getHashCode() + "-f");
// }else {
//      vPCSResponse.printWriter.flush();
//      vPCSResponse.printWriter.println("HTTP/1.1 204 OK\n\n");
//      vPCSResponse.printWriter.close();
//      System.out.flush();S_logger.Log.logger.info(" " + vPCSResponse.getHashCode()
//      + "-processRest length err");
//      System.out.flush();System.out.print(vPCSRequest.getRequestLink());
//      System.out.flush();System.out.print(vPCSRequest.getRequestFilePath());
//      return;
// }
// }

//      }

//
//      public static void processView(VPCSRequest vPCSRequest, VPCSResponse vPCSResponse) {
//
//
//      }
```

```
//  
//      //119-while_?https://blog.csdn.net/top_code/article/details/41042413  
//      //dzNlenijLl  
//      public static void processBytes(VPCSRequest vPCSRequest  
//      , VPCSResponse vPCSResponse) {  
// try {  
//      List<byte[]> list;  
//      vPCSResponse.dataOutputStream = new DataOutputStream(vPCSResponse.socket.getOutputStream());  
//      if(DetaCacheManager.getCacheOfBytesList(vPCSRequest.getRequestFilePath()) != null){  
//          list = DetaCacheManager.getCacheOfBytesList(vPCSRequest.getRequestFilePath());  
//      }else{  
//          String fileString= URLDecoder.decode(vPCSRequest.getRequestFilePath(), "UTF8");  
//          File file= new File(fileString);  
//          vPCSResponse.fileInputStream = new FileInputStream(file);  
//          vPCSResponse byteArrayOutputStream = new ByteArrayOutputStream();  
//          byte[] byteArray = new byte[1024];  
//          int len = 0;  
//          list = new ArrayList<>();  
//          vPCSResponse.byteArrayOutputStream.flush();  
//          while((len= vPCSResponse.fileInputStream.read(byteArray, 0, 1024))!=-1){  
//              vPCSResponse.byteArrayOutputStream.write(byteArray, 0, len);  
//              vPCSResponse.byteArrayOutputStream.flush();  
//          }  
//          vPCSResponse.fileInputStream.close();  
//          byte[] sniper = GzipUtil.compress(vPCSResponse.byteArrayOutputStream.toByteArray());  
//          long sniper_length= sniper==null? 0: sniper.length;  
//          list.add(0, vPCSResponse.getResponseContentType().getBytes("UTF8"));  
//          list.add(0, ("Content-Length: " + sniper_length + " \n").getBytes("UTF8"));  
//          list.add(0, ("Cache-control: max-age=315360000 \n").getBytes("UTF8"));  
//          list.add(0, ("Content-Encoding:Gzip \n").getBytes("UTF8"));  
//          list.add(0, "Accept-Ranges: bytes \n".getBytes("UTF8"));  
//          list.add(0, "Host:deta software \n".getBytes("UTF8"));  
//          list.add(0, "HTTP/1.1 200 OK \n".getBytes("UTF8"));  
//          list.add(sniper);  
//          DetaCacheManager.putCacheOfBytesList(vPCSRequest.getRequestFilePath(), list);  
//      }  
//      Iterator<byte[]> iterator = list.iterator();  
//      vPCSResponse.dataOutputStream.flush();  
//      while(iterator.hasNext()){  
//          byte[] bytes= null;  
//          int i= 0;  
//          long bytes_length= 0;  
//          bytes= iterator.next();  
//          bytes_length= bytes==null? 0: bytes.length;  
//          if(bytes_length>10000) {  
//              int last= bytes.length% 10000;  
//              for(i= 0; i< bytes.length- 10000; i+= 10000) {
```

```
//      byte[] serparBytes = new byte[10000];
//      for(int j= 0; j< 10000; j++) {
//          serparBytes[j]= bytes[i+ j];
//      }
//      vPCSResponse.dataOutputStream.write(serparBytes);
//      vPCSResponse.dataOutputStream.flush();
//          }
//          byte[] serparBytes = new byte[last];
//          for(int j=0; j<last;j++) {
//      serparBytes[j]= bytes[i+ j];
//          }
//          vPCSResponse.dataOutputStream.write(serparBytes);
//          vPCSResponse.dataOutputStream.flush();
//      }else if(bytes_length> 0){
//          vPCSResponse.dataOutputStream.write(bytes);
//          vPCSResponse.dataOutputStream.flush();
//      }
//  }
//  vPCSResponse.dataOutputStream.close();
//  System.out.flush();S_logger.Log.logger.info(" " + vPCSResponse.getHashCode() + " -f");
// }catch(Exception e) {
//      System.out.flush();S_logger.Log.logger.info(" " + e.getMessage());
//      System.out.flush();S_logger.Log.logger.info(" " + vPCSResponse.getHashCode()+"-processBytes err");
//      System.out.flush();S_logger.Log.logger.info(" " + vPCSRequest.getRequestLink());
//      System.out.flush();S_logger.Log.logger.info(" " + vPCSRequest.getRequestFilePath());
//      StableBlob.addException();
//      return;
// }
// }
//
// public static void processBuffer(VPCSRequest vPCSRequest
// , VPCSResponse vPCSResponse) {
// try {
//      if(null==vPCSRequest.getRequestFilePath()
//          | null==vPCSRequest.getRequestLink()) {
//          return;
//      }
//
//      String builderToString;
//      if(DetaCacheManager.getCacheOfString(vPCSRequest.getRequestFilePath()) != null){
//          builderToString = DetaCacheManager.getCacheOfString(vPCSRequest.getRequestFilePath());
//      }else{
//          StringBuilder stringBuilder = new StringBuilder();
//          stringBuilder.append("HTTP/1.1 200 OK").append("\n");
//          stringBuilder.append("Host:deta software \n");
//          stringBuilder.append("Cache-control: max-age=315360000 \n");
//          stringBuilder.append(vPCSResponse.getResponseContentType());
```



```
//      File file= new File(vPCSRequest.getRequestFilePath());
//      vPCSResponse.fileInputStream = new FileInputStream(file);
//      vPCSResponse.inputStreamReader
//      = new InputStreamReader(vPCSResponse.fileInputStream
//      , vPCSRequest.getRequestFileCode());
//      vPCSResponse.bufferedReader = new BufferedReader(vPCSResponse.inputStreamReader);
//      String line = null;
//      while ((line = vPCSResponse.bufferedReader.readLine()) != null) {
//          stringBuilder.append(line);
//      }
//      builderToString = stringBuilder.toString();
//      DetaCacheManager.putCacheOfString(vPCSRequest.getRequestFilePath()
//      , builderToString);
//      vPCSResponse.bufferedReader.close();
//  }
//  if(null!= vPCSResponse.socket) {
//      if(!vPCSResponse.socket.isClosed()) {
//          OutputStream outputStream= vPCSResponse.socket.getOutputStream();
//          OutputStreamWriter outputStreamWriter=new OutputStreamWriter(
//          outputStream, vPCSRequest.getRequestFileCode());
//          vPCSResponse.bufferedWriter = new BufferedWriter(outputStreamWriter);
//          int j=builderToString.length();
//          vPCSResponse.bufferedWriter.flush();
//          vPCSResponse.bufferedWriter.write(builderToString, 0, j);
//          vPCSResponse.bufferedWriter.close();
//      }
//  }
//  System.out.flush();S_logger.Log.logger.info(" " + vPCSResponse.getHashCode() + "-f");
// }catch(Exception e) {
//     System.out.flush();S_logger.Log.logger.info(" " + e.getMessage());
//     System.out.flush();S_logger.Log.logger.info(" " + vPCSResponse.getHashCode()
//     + "-processBuffer bufferedWriter err");
//     System.out.flush();S_logger.Log.logger.info(" " + vPCSRequest.getRequestLink());
//     System.out.flush();S_logger.Log.logger.info(" " + vPCSRequest.getRequestFilePath());
//     StableBlob.addException();
//     return;
// }
//
// public static void processBufferBytes(VPCSRequest vPCSRequest
// , VPCSResponse vPCSResponse) {
// try {
//     StringBuilder stringBuilder = new StringBuilder();
//     stringBuilder.append("HTTP/1.1 200 OK").append("\n");
//     stringBuilder.append("Host:deta software \n");
//     stringBuilder.append("Cache-control: max-age=315360000 \n");
//     stringBuilder.append("Content-Encoding:gzip \n");
```

```
//      stringBuilder.append(vPCSResponse.getResponseContentType());
//      String builderToString = stringBuilder.toString();
//      String contentBuilderToString;
//      if(DetaCacheManager.getCacheOfString(vPCSRequest.getRequestFilePath()) != null){
//          contentBuilderToString = DetaCacheManager.getCacheOfString(vPCSRequest.getRequestFilePath());
//      }else{
//          StringBuilder contentBuilder = new StringBuilder();
//          File file = new File(vPCSRequest.getRequestFilePath());
//          vPCSResponse.fileInputStream = new FileInputStream(file);
//          int len = 0;
//          byte[] byteArray = new byte[1024];
//          while ((len = vPCSResponse.fileInputStream.read(byteArray))!=-1){
//              contentBuilder.append(new String(byteArray, 0, len, "UTF-8"));
//          }
//          vPCSResponse.fileInputStream.close();
//          Thread.sleep(75);
//          contentBuilderToString = contentBuilder.toString();
//          DetaCacheManager.putCacheOfString(vPCSRequest.getRequestFilePath()
//      , contentBuilderToString);
//      }
//      vPCSResponse.dataOutputStream
//      = new DataOutputStream(vPCSResponse.socket.getOutputStream());
//      vPCSResponse.dataOutputStream.flush();
//      vPCSResponse.dataOutputStream.write(builderToString.getBytes("UTF8"));
//      Thread.sleep(75);
//      byte[] bytes= null;
//      int i= 0;
//      long bytes_length= 0;
//      bytes= GzipUtil.compress(contentBuilderToString.getBytes("UTF8"));
//      bytes_length= bytes==null? 0: bytes.length;
//      vPCSResponse.dataOutputStream.flush();
//      if(bytes_length>10000) {
//          int last= bytes.length%10000;
//          for(i= 0; i< bytes.length- 10000; i+= 10000) {
//              byte[] serparBytes = new byte[10000];
//              for(int j= 0; j< 10000; j++) {
//                  serparBytes[j]= bytes[i+ j];
//              }
//              vPCSResponse.dataOutputStream.write(serparBytes);
//              vPCSResponse.dataOutputStream.flush();
//          }
//          byte[] serparBytes = new byte[last];
//          for(int j=0; j<last;j++) {
//              serparBytes[j]= bytes[i+ j];
//          }
//          vPCSResponse.dataOutputStream.write(serparBytes);
//          vPCSResponse.dataOutputStream.flush();
```

```
//      }else if(bytes_length> 0){
//          vPCSResponse.dataOutputStream.write(bytes);
//          vPCSResponse.dataOutputStream.flush();
//      }
//      vPCSResponse.dataOutputStream.close();
//      System.out.flush();S_logger.Log.logger.info("'" + vPCSResponse.getHashCode() + "-f");
// }catch(Exception e) {
//      System.out.flush();S_logger.Log.logger.info("'" + e.getMessage());
//      System.out.flush();S_logger.Log.logger.info("'" + vPCSResponse.getHashCode()
//          + "-processBufferBytes dataOutputStream err");
//      System.out.flush();S_logger.Log.logger.info("'" + vPCSRequest.getRequestLink());
//      System.out.flush();S_logger.Log.logger.info("'" + vPCSRequest.getRequestFilePath());
//      StableBlob.addException();
//      return;
// }
//      }
//      //327//dataOutputStream.write(GzipUtil.compress(contentBuilderToString.getBytes("UTF8")));
//      public static void processBytesWithoutZip(VPCSRequest vPCSRequest
//          , VPCSResponse vPCSResponse) {
//      try {
//          List<byte[]> list;
//          vPCSResponse.dataOutputStream = new DataOutputStream(vPCSResponse.socket.getOutputStream());
//          if(DetaCacheManager.getCacheOfBytesList(vPCSRequest.getRequestFilePath()) != null){
//              list = DetaCacheManager.getCacheOfBytesList(vPCSRequest.getRequestFilePath());
//          }else{
//              vPCSResponse.fileInputStream = null;
//              String string= vPCSRequest.getRequestFilePath();
//              if(string.contains("/dna_pde_rna/")) {
//                  string= URLDecoder.decode(string, "UTF8");
//                  File file = new File(string);
//                  vPCSResponse.fileInputStream = new FileInputStream(file);
//              }else {
//                  File file = new File(vPCSRequest.getRequestFilePath());
//                  vPCSResponse.fileInputStream = new FileInputStream(file);
//              }
//              vPCSResponse byteArrayOutputStream = new ByteArrayOutputStream();
//              byte[] byteArray = new byte[1024];
//              int len = 0;
//              list = new ArrayList<>();
//              vPCSResponse.byteArrayOutputStream.flush();
//              while((len = vPCSResponse.fileInputStream.read(byteArray, 0, 1024))!= -1){
//                  vPCSResponse.byteArrayOutputStream.write(byteArray, 0, len);
//                  vPCSResponse.byteArrayOutputStream.flush();
//              }
//              vPCSResponse.fileInputStream.close();
//              byte[] sniper =vPCSResponse.byteArrayOutputStream.toByteArray();
//              list.add(0, vPCSResponse.getResponseContentType().getBytes("UTF8"));
```

```
// list.add(0, ("Content-Length: " + sniper.length + "\n").getBytes("UTF8"));
// list.add(0, ("Cache-control: max-age=315360000 \n").getBytes("UTF8"));
// list.add(0, "Accept-Ranges: bytes \n".getBytes("UTF8"));
// list.add(0, "Host:deta software \n".getBytes("UTF8"));
// list.add(0, "HTTP/1.1 200 OK \n".getBytes("UTF8"));
// list.add(sniper);
// DetaCacheManager.putCacheOfBytesList(vPCSRequest.getRequestFilePath(), list);
// }
// Iterator<byte[]> iterator = list.iterator();
// vPCSResponse.dataOutputStream.flush();
// while(iterator.hasNext()){
//     byte[] bytes = null;
//     int i= 0;
//     long bytes_length= 0;
//     bytes= iterator.next();
//     bytes_length= bytes==null? 0: bytes.length;
//     if(bytes_length>10000) {
//         int last= bytes.length%10000;
//         for(i= 0; i< bytes.length- 10000; i+= 10000) {
//             byte[] serparBytes = new byte[10000];
//             for(int j= 0; j< 10000; j++) {
//                 serparBytes[j]= bytes[i+ j];
//             }
//             vPCSResponse.dataOutputStream.write(serparBytes);
//             vPCSResponse.dataOutputStream.flush();
//         }
//         byte[] serparBytes = new byte[last];
//         for(int j=0; j<last;j++) {
//             serparBytes[j]= bytes[i+ j];
//         }
//         vPCSResponse.dataOutputStream.write(serparBytes);
//         vPCSResponse.dataOutputStream.flush();
//     }else if(bytes_length> 0){
//         vPCSResponse.dataOutputStream.write(bytes);
//         vPCSResponse.dataOutputStream.flush();
//     }
// }
// vPCSResponse.dataOutputStream.close();
// if(null!= vPCSResponse.byteArrayOutputStream) {
//     vPCSResponse.byteArrayOutputStream.close();
// }
// System.out.flush();S_logger.Log.logger.info("'" + vPCSResponse.getHashCode() + "-f");
// }catch(Exception e) {
//     System.out.flush();S_logger.Log.logger.info("'" + e.getMessage());
//     System.out.flush();S_logger.Log.logger.info("'" + vPCSResponse.getHashCode()
//         + "-processBytesWithoutZip fileInputStream err");
//     System.out.flush();S_logger.Log.logger.info("'" + vPCSRequest.getRequestLink());
```

```
//      System.out.flush();S_logger.Log.logger.info("" + vPCSRRequest.getRequestFilePath());
//      StableBlob.addException();
//      return;
// }
// }
// }

I_Q.hvpcs.cache

/*Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/Cache.java
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
package I_Q.hvpcs.cache;
public class Cache {
    private String key;
    private Object value;
    private long timeOut;
    private boolean expired;
    public Cache() {
        super();
    }
    public Cache(String key, Object value, long timeOut, boolean expired) {
        this.key = key;
        this.value = value;
        this.timeOut = timeOut;
        this.expired = expired;
    }
    public String getKey() {
        return key;
    }
    public long getTimeOut() {
        return timeOut;
    }
    public Object getValue() {
        return value;
    }
    public void setKey(String string) {
        key = string;
    }
    public void setTimeOut(long l) {
        timeOut = l;
    }
}
```

```

    public void setValue(Object object) {
        value = object;
    }

    public boolean isExpired() {
        return expired;
    }

    public void setExpired(boolean b) {
        expired = b;
    }
}

**JDK-- /Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/CacheManager.java
/*

* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

//refer java jdk 组件, 当年王浩雷要我搜网上buffer cache, 我copy了 jdk原生的函数这个 改的
package I_Q.hvpcs.cache;

import java.util.*;

public class CacheManager {
    @SuppressWarnings("rawtypes")
    private static HashMap cacheMap = new HashMap();
    private CacheManager() {
        super();
    }

    public static boolean getSimpleFlag(String key) {
        try {
            return (Boolean) cacheMap.get(key);
        } catch (NullPointerException e) {
            return false;
        }
    }

    public static long getServerStartdt(String key) {
        try {
            return (Long) cacheMap.get(key);
        } catch (Exception ex) {
            return 0;
        }
    }

    @SuppressWarnings("unchecked")
    public synchronized static boolean setSimpleFlag(String key, boolean flag) {
        if (flag && getSimpleFlag(key)) {
            return false;
        } else {

```

```
        cacheMap.put(key, flag);
        return true;
    }
}

@SuppressWarnings("unchecked")
public synchronized static boolean setSimpleFlag(String key,
        long serverbegrundt) {
    if (cacheMap.get(key) == null) {
        cacheMap.put(key, serverbegrundt);
        return true;
    } else {
        return false;
    }
}

// private synchronized static Cache getCache(String key) {
private synchronized static Cache getCache(String key) {
    return (Cache) cacheMap.get(key);
}

// private synchronized static boolean hasCache(String key)
// {
private synchronized static boolean hasCache(String key) {
    return cacheMap.containsKey(key);
}

public synchronized static void clearAll() {
    cacheMap.clear();
}

@SuppressWarnings({ "rawtypes", "unchecked" })
public synchronized static void clearAll(String type) {
    Iterator i = cacheMap.entrySet().iterator();
    String key;
    ArrayList arr = new ArrayList();
    try {
        while (i.hasNext()) {
            java.util.Map.Entry entry = (java.util.Map.Entry) i.next();
            key = (String) entry.getKey();
            if (key.startsWith(type)) {
                arr.add(key);
            }
        }
        for (int k = 0; k < arr.size(); k++) {
            clearOnly((String) arr.get(k));
        }
    } catch (Exception ex) {
        ex.printStackTrace();
    }
}

public synchronized static void clearOnly(String key) {
```

```
        cacheMap.remove(key);
    }
    // public synchronized static void putCache(String key,
    // Cache obj) {
    @SuppressWarnings("unchecked")
    public synchronized static void putCache(String key, Cache obj) {
        cacheMap.put(key, obj);
    }
    public synchronized static Cache getCacheInfo(String key) {
        if (hasCache(key)) {
            Cache cache = getCache(key);
            if (cacheExpired(cache)) {
                cache.setExpired(true);
            }
            return cache;
        } else
            return null;
    }
    @SuppressWarnings("unchecked")
    public static void putCacheInfo(String key, Cache obj, long dt,
        boolean expired) {
        Cache cache = new Cache();
        cache.setKey(key);
        cache.setTimeout(dt + System.currentTimeMillis());
        cache.setValue(obj);
        cache.setExpired(expired);
        cacheMap.put(key, cache);
    }
    @SuppressWarnings("unchecked")
    public static void putCacheInfo(String key, Cache obj, long dt) {
        Cache cache = new Cache();
        cache.setKey(key);
        cache.setTimeout(dt + System.currentTimeMillis());
        cache.setValue(obj);
        cache.setExpired(false);
        cacheMap.put(key, cache);
    }
    public static boolean cacheExpired(Cache cache) {
        if (null == cache) {
            return false;
        }
        long nowDt = System.currentTimeMillis();
        long cacheDt = cache.getTimeout();
        if (cacheDt <= 0 || cacheDt > nowDt) {
            return false;
        } else {
            return true;
        }
    }
}
```



```
    }  
}  
  
public synchronized static int getCacheSize() {  
    return cacheMap.size();  
}  
  
@SuppressWarnings("rawtypes")  
public static int getCacheSize(String type) {  
    int k = 0;  
    Iterator i = cacheMap.entrySet().iterator();  
    String key;  
    try {  
        while (i.hasNext()) {  
            java.util.Map.Entry entry = (java.util.Map.Entry) i.next();  
            key = (String) entry.getKey();  
            if (key.indexOf(type) != -1) {  
                k++;  
            }  
        }  
    } catch (Exception ex) {  
        ex.printStackTrace();  
    }  
    return k;  
}  
  
@SuppressWarnings({ "rawtypes", "unchecked", "finally" })  
public static ArrayList getCacheAllkey() {  
    ArrayList a = new ArrayList();  
    try {  
        Iterator i = cacheMap.entrySet().iterator();  
        while (i.hasNext()) {  
            java.util.Map.Entry entry = (java.util.Map.Entry) i.next();  
            a.add((String) entry.getKey());  
        }  
    } catch (Exception ex) {  
    } finally {  
        return a;  
    }  
}  
  
@SuppressWarnings({ "rawtypes", "unchecked", "finally" })  
public static ArrayList getCacheListkey(String type) {  
    ArrayList a = new ArrayList();  
    String key;  
    try {  
        Iterator i = cacheMap.entrySet().iterator();  
        while (i.hasNext()) {  
            java.util.Map.Entry entry = (java.util.Map.Entry) i.next();  
            key = (String) entry.getKey();  
            if (key.indexOf(type) != -1) {
```

```
a.add(key);

    }

        }

    } catch (Exception ex) {

    } finally {

        return a;

    }

}

}

}

/*
Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/DetaCache.java
*/

* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

package I_Q.hvpcs.cache;

public class DetaCache {

    public String getValue() {

        return value;

    }

    public void setValue(String value) {

        this.value = value;

    }

    public Object getObject() {

        return object;

    }

    public void setObject(Object object) {

        this.object = object;

    }

    public long getTimeout() {

        return timeout;

    }

    public void setTimeout(long timeout) {

        this.timeout = timeout;

    }

    private String value;

    private Object object;

    private long timeout;

    public DetaCache() {

        super();

    }

    public DetaCache(String value, Object object, long timeout) {

        this.value = value;

        this.object = object;
```

```
        this.timeOut = timeOut;
    }
}

/*Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/DetaCacheManager.java
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
package I_Q.hvpcs.cache;
import java.util.Iterator;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
public class DetaCacheManager {
    private static ConcurrentHashMap<String, DetaCache> cacheMap = new ConcurrentHashMap<>();
    private static ConcurrentHashMap<String, List<byte[]>> bytesMap = new ConcurrentHashMap<>();
    private static ConcurrentHashMap<String, String> stringMap = new ConcurrentHashMap<>();
    private DetaCacheManager() {
        super();
    }
    public static String putCache(String key, String value, long timeOut) {
        DetaCache c = new DetaCache();
        c.setValue(value);
        c.setTimeout(timeOut);
        cacheMap.put(key, c);
        return "success";
    }
    public static String getCache(String key) {
        DetaCache c = cacheMap.get(key);
        if (null == c) {
            return "unsuccess nofind cache";
        }
        long now = System.currentTimeMillis();
        if (c.getTimeOut() < now) {
            cacheMap.remove(key);
            return "unsuccess timeout";
        }
        return c.getValue();
    }
    @SuppressWarnings("rawtypes")
    public static Iterator getCacheIterator() {
        return cacheMap.entrySet().iterator();
    }
    public static void putCacheOfBytesList(String filePath, List<byte[]> list) {
```

```

        bytesMap.put(filePath, list);
    }

    public static List<byte[]> getCacheOfBytesList(String filePath) {
        if (bytesMap.containsKey(filePath)) {
            return bytesMap.get(filePath);
        }
        return null;
    }

    public static String getCacheOfString(String filePath) {
        if (stringMap.containsKey(filePath)) {
            return stringMap.get(filePath);
        }
        return null;
    }

    public static void putCacheOfString(String filePath, String stringBuilder) {
        stringMap.put(filePath, stringBuilder);
    }
}

//Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/test.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/cache/TimeCheck.java

I_Q.hvpcs.DetaSrc

I_Q.hvpcs.html5.upload

//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/code.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/DoubleListBytes.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploader.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimple.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleBackUp.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleDNA.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocket.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocketPDE.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocketSound.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocketSoundImage.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderSimpleSocketSoundImageMpeg.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/FileUploaderUnwrite.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/html5/upload/SoundWaveVectorMp3.java

I_Q.hvpcs.lyg.common

//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/KeyStore.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/LYGPasswordDecode.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/NeedLogin.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/POSTREST.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/RequestLimit.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/WriteData.java

I_Q.hvpcs.lyg.common.constants

//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/constants/DetaDBConstant.java

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
package I_Q.hvpcs.lyg.common.constants;

public interface DetaDBConstant {

    // common

```

```

final static int DIGIT_ONE = 1;

final static int DIGIT_60000 = 60000;

// Token expired.

final static String ERRORCODE_ER1001 = "ER1001";

// rest tire

final static String REST_JSON_CONFIG = "application/json;charset=utf-8";

final static String REST_LOGIN_LOGIN = "/login";

final static String REST_LOGIN_LOGOUT = "/logout";

final static String REST_LOGIN_REGISTER = "/register";

final static String REST_LOGIN_FIND = "/find";

final static String REST_LOGIN_CHANGE = "/change";

final static String DB_LOGIN_U_NAME = "uName";

final static String DB_LOGIN_U_CHANGE = "uChange";

final static String DB_LOGIN_U_CHANGE_ENSURE = "uChangeEnsure";

final static String DB_LOGIN_U_AGE = "uAge";

final static String DB_LOGIN_U_EMAIL = "uEmail";

final static String DB_LOGIN_U_EMAIL_ENSURE = "uEmailEnsure";

final static String DB_LOGIN_U_ID = "uId";

final static String DB_LOGIN_U_SEX = "uSex";

final static String DB_LOGIN_U_PHONE = "uPhone";

final static String DB_LOGIN_U_ADDRESS = "uAddress";

final static String DB_LOGIN_U_WECHAT = "uWeChat";

final static String DB_LOGIN_U_PSWD = "uPassword";

final static String DB_LOGIN_U_PSWD_ENSURE = "uPasswordEnsure";

final static String DB_LOGIN_U_PSTOKEN = "uToken";

final static String DB_LOGIN_U_CLASS = "uClass";

final static String DB_LOGIN_U_QQ = "uQq";

// service tire

// scheduler tire

// dao tire

// database tire

// condition tire
}

```

I_Q.hvpcs.lyg.common.exceptions

```

//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/exceptions/DetaDBException.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/exceptions/ErrorCodeEnum.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/exceptions/RequestLimitException.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/exceptions/RestExceptionHandler.java

```

I_Q.hvpcs.lyg.common.maps

```

//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/maps/VtoV.java

```

I_Q.hvpcs.lyg.common.utils

```

//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/utils/DateUtil.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/utils/DetaDBUtil.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/utils/DetaUtil.java
//Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/common/utils/GzipUtil.java

```

```
L_Q.hvpcs.lyg.vpc.process.companyImpl
```

```
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/companyImpl/LoginServiceImpl.java
```

```
L_Q.hvpcs.lyg.vpc.process.factoryImpl
```

```
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/factoryImpl/LoginDAOImpl.java
```

```
L_Q.hvpcs.lyg.vpc.process.portImpl
```

```
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/portImpl/RestAskPortImpl.java
```

```
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/portImpl/RestLoginPortImpl.java
```

```
L_Q.hvpcs.lyg.vpc.process.projectImpl
```

```
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/process/projectImpl/LoginProjectImpl.java
```

```
L_Q.hvpcs.lyg.vpc.transaction
```

```
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/transaction/TransactionDelegate.java
```

```
L_Q.hvpcs.lyg.vpc.view
```

```
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/Branch.java  
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/Talk.java  
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/Token.java  
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/Usr.java  
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/UsrFull.java  
/Trif_DNA-master/C_L_I/I_Q/hvpcs/lyg/vpc/view/UsrToken.java
```

```
L_Q.hvpcs.pixes.process
```

```
/Trif_DNA-master/C_L_I/I_Q/hvpcs/pixes/process/ColorProcess.java  
/Trif_DNA-master/C_L_I/I_Q/hvpcs/pixes/process/DistributeProcess.java  
/Trif_DNA-master/C_L_I/I_Q/hvpcs/pixes/process/RangeProcess.java  
package DetaSrc.org.deta.pixes.process;
```

```
import java.util.Map;
```

```
///2.1 像素的区间定义  
///2.2 像素的主要颜色区间  
///2.3 像素的特殊颜色区间  
///2.4 像素的特征颜色的区间  
///2.5 像素的区间提取
```

```
在设计这个api之前，我的德塔数据预测包已经有了像素的坐标存储函数  
AMV_MVS_VSQ_2D，2维坐标，对应图片中像素的坐标。
```

```
那就开始了  
作者 罗瑶光
```

```
* 个人著作权人，作者 罗瑶光，浏阳  
* yaoguangluo@outlook.com, 31369483@qq.com, 2080315360@qq.com, lyg.tin@gmail.com- lygtin@sina.com  
* 15116110525  
* 430181198505250014, G24402609, EB0581342  
* 204925063, 389418686, F2406501, 0626136  
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段208号 阳光家园别墅小区 第十栋别墅 第三层  
*/public interface RangeProcess{
```

```
///1 像素的区间定义  
///首先，我需要 一个图片，然后从图片中读取标准的三原色，和它的总数和区间定义规范。  
public int setRGBAndRangeThenReturnAmountOfThisRanges(int[] rgb, int rangeMin, int rangeMax);  
public int 设置一个RGB和区间定义然后提取出区间的像素数(int[] rgb, int rangeMin, int rangeMax);  
public int 设置一个RGB和区间定义然后提取出区间的PCA像素数(int[] rgb, int rangeMin, int rangeMax);  
public int 设置一个RGB和区间定义然后提取出区间的ICA像素数(int[] rgb, int rangeMin, int rangeMax);  
public int 设置一个RGB和区间定义然后提取出区间的PCA分类像素(int[] rgb, int rangeMin, int rangeMax);  
public int 设置一个RGB和区间定义然后提取出区间的ICA分类像素(int[] rgb, int rangeMin, int rangeMax);  
public Map<String, int[]> 设置一个RGB和区间定义然后提取出区间的ICA像素集(int[] rgb, int rangeMin, int rangeMax);  
public Map<String, int[]> 设置一个RGB和区间定义然后提取出区间的PCA像素集(int[] rgb, int rangeMin, int rangeMax);  
public Map<String, int[]> 设置一个RGB和区间定义然后提取出区间的ICA分类像素集(int[] rgb, int rangeMin, int rangeMax);  
public Map<String, int[]> 设置一个RGB和区间定义然后提取出区间的PCA分类像素集(int[] rgb, int rangeMin, int rangeMax);  
public Map<String, int[]> 设置一个RGB和区间进行指定rgb计算定义然后提取出区间的PCA分类像素集(int[] rgb, int rangeMin, int rangeMax, int r, int g, int b);  
public Map<String, int[]> 设置一个RGB和区间进行指定rgb计算定义然后提取出区间的ICA分类像素集(int[] rgb, int rangeMin, int rangeMax, int r, int g, int b);  
public Map<String, int[]> 设置一个RGB和区间进行指定rgb计算定义然后提取出区间的section分类像素集(int[] rgb, int rangeMin, int rangeMax, int r, int g, int b, int section);  
///2.2 像素的主要颜色区间  
///关于像素的主要颜色 我定位在七彩色，红橙黄绿青靛紫 区间定义规范。  
///2.3 像素的特殊颜色区间  
///关于像素的主要颜色 我定位在21颜色阶，浅红深，浅橙深，浅黄深，浅绿深，浅青深，浅靛深，浅紫深，区间定义规范。  
///2.4 像素的特征颜色的区间  
///关于像素的特征颜色 我定位在输入某一个rgb，某一个颜色阶段，某一个区间，输出需求的颜色统计。区间定义规范。  
///2.5 像素的区间提取  
///输入一个区间，得到这个区间的像素总数。和区间的涉及，或者涉及区间的内的像素统计分类。  
///2.6 区间的增量设计  
///如设计一个增量x，那么0，x，2x，3x，4x.....nx, 255]然后增量区间划分。  
package L_Q;  
Trif_DNA-master/C_L_I/I_Q/hvpcs/pixes/process/RatioProcess.java
```

1-6-/Trif_DNA-master/C_L_M-管理-报表-打印-网络-UI

M_V.AVQ.PVU.vedio

```
/Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/PlayerControl.java
package M_V.AVQ.PVU.vedio;

import M_V.MVQ.filenameFilter.TXTFilter;

import javax.imageio.ImageIO;

import javax.swing.Box;

import javax.swing.BoxLayout;

import javax.swing.JButton;

import javax.swing.JFrame;

import javax.swing.JSlider;

import javax.swing.event.ChangeEvent;

import javax.swing.event.ChangeListener;

import java.awt.Color;

import java.awt.FileDialog;

import java.awt.Frame;

import java.awt.ScrollPane;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.awt.event.MouseEvent;

import java.awt.event.MouseListener;

import java.awt.image.BufferedImage;

import java.io.File;

import java.io.FileNotFoundException;

import java.io.FileOutputStream;

import java.io.IOException;

import java.util.Iterator;

import java.util.Map;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

 */

public class PlayerControl {

    int[] control;

    public JSlider sliderx;

    Map<Integer, BufferedImage> flipImage;

    public PlayerControl(int[] control, Map<Integer, BufferedImage> flipImage) {

        super();

        this.control = control;

        this.flipImage = flipImage;

    }

}
```

```
}

public void run() {
    Box mainBox = new Box(BoxLayout.Y_AXIS);
    //弹出控件
    Box slideBox = new Box(BoxLayout.X_AXIS);
    sliderx = new JSlider(0, control[5]);
    sliderx.setSnapToTicks(true);
    sliderx.setPaintTicks(true);
    //sliderx.setMajorTickSpacing(1);
    sliderx.setMinorTickSpacing(1);
    sliderx.addChangeListener(new ChangeListener() {
        public void stateChanged(ChangeEvent event) {
            JSlider source = (JSlider) event.getSource();
            control[6] = source.getValue();
        }
    });
    sliderx.addMouseListener(new MouseListener() {
        @Override
        public void mouseClicked(MouseEvent arg0) {
        }
        @Override
        public void mouseEntered(MouseEvent arg0) {
        }
        @Override
        public void mouseExited(MouseEvent arg0) {
        }
        @Override
        public void mousePressed(MouseEvent arg0) {
        }
        @Override
        public void mouseReleased(MouseEvent arg0) {
            JSlider source = (JSlider) arg0.getSource();
            if (1 == control[1]) {
                control[6] = source.getValue() + 1;
            } else if (1 == control[2]) {
                control[6] = source.getValue() - 1;
            }
            control[7] = 1;
        }
    });
    slideBox.add(sliderx);
    //BOX
    Box processBox = new Box(BoxLayout.X_AXIS);
    //buttonADD= new JButton("添加到编辑页");
    JButton buttonPlayer = new JButton("播放");
    buttonPlayer.setBounds(0, 0, 135, 30);
    buttonPlayer.addActionListener(new ActionListener() {
```



```
public void actionPerformed(ActionEvent e) {
    control[0] = 1;
    if (control[1] == 0 && control[2] == 0) {
        control[2] = 1;
    }
    if (control[3] == 0 && control[4] == 0) {
        control[3] = 1;
    }
}
});
JButton buttonPause = new JButton("暂停");
buttonPause.setBounds(35, 0, 135, 30);
buttonPause.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        control[0] = 0;
    }
});
JButton buttonRollback = new JButton("正放");
buttonRollback.setBounds(70, 0, 135, 30);
buttonRollback.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        control[0] = 1;
        control[1] = 0;
        control[3] = 1;
        control[4] = 0;
        control[2] = 1;
    }
});
JButton buttonReDo = new JButton("倒放");
buttonReDo.setBounds(105, 0, 135, 30);
buttonReDo.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        control[0] = 1;
        control[3] = 1;
        control[4] = 0;
        control[1] = 1;
        control[2] = 0;
    }
});
JButton buttonFast = new JButton("快放");
buttonFast.setBounds(140, 0, 135, 30);
buttonFast.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        control[4] = 0;
        if (control[3] == 0) {
            control[3] = 1;
        } else if (control[3] == 1) {
```

```
        control[3] = 2;
    } else if (control[3] == 2) {
        control[3] = 3;
    } else if (control[3] == 3) {
        control[3] = 1;
    }
}

});

JButton buttonSlow = new JButton("慢放");
buttonSlow.setBounds(175, 0, 135, 30);
buttonSlow.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        control[3] = 0;
        if (control[4] == 0) {
            control[4] = 1;
        } else if (control[4] == 1) {
            control[4] = 2;
        } else if (control[4] == 2) {
            control[4] = 3;
        } else if (control[4] == 3) {
            control[4] = 1;
        }
    }
});

JButton buttonSave = new JButton("保存当前帧");
buttonSave.setBounds(210, 0, 135, 30);
buttonSave.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (0 == control[0]) {
            //获取图片
            //弹出保存框
            if (null == flipImage) {
                return;
            }
            //get path
            FileDialog filedialog = new FileDialog(new Frame(), "导入到这里", FileDialog.LOAD);
            filedialog.setFilenameFilter(new TXTFilter("png"));
            filedialog.setVisible(true);
            String filepath = filedialog.getDirectory() + filedialog.getFile();
            //20230106-S_logger.Log.logger.info("" + filepath);
            try {
                File outputBin = new File(filepath + ".png");
                if (null == flipImage) {
                    return;
                }
                ImageIO.write(flipImage.get(control[6]), "png", outputBin);
            } catch (IOException writePngError) {
```

```
//20230106-S_logger.Log.logger.info("" + filepath);
}
//输出图片
}
}
});
JButton buttonNew = new JButton("保存为一个新的视频");
buttonNew.setBounds(245, 0, 135, 30);
buttonNew.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (0 == control[0]) {
            if (flipImage.size() <= 0) {
                return;
            }
            FileDialog filedialog = new FileDialog(new Frame(), "导入到这里", FileDialog.LOAD);
            filedialog.setFilenameFilter(new TXTFilter(".lyg"));
            filedialog.setVisible(true);
            String filepath = filedialog.getDirectory() + filedialog.getFile();
            //20230106-S_logger.Log.logger.info("" + filepath);
            File outputBin = new File(filepath + ".lyg");
            //    Iterator<int[][]> iterator= flipImage.keySet().iterator();
            Iterator<Integer> iterator = flipImage.keySet().iterator();
            try {
                FileOutputStream wr = new FileOutputStream(outputBin);
                //写头, 宽, 长
                if (iterator.hasNext()) {
                    BufferedImage bufferedImage = flipImage.get(iterator.next());
                    //int[][] bufferedImage= v;
                    int width = bufferedImage.getWidth();
                    int height = bufferedImage.getHeight();
                    int flips = flipImage.size();
                    byte[] flipBytes = new byte[height * width * 4];
                    byte[] widthBytes = new byte[4];
                    widthBytes[0] = (byte) (width & 0xff);
                    widthBytes[1] = (byte) (width >> 8 & 0xff);
                    widthBytes[2] = (byte) (width >> 16 & 0xff);
                    widthBytes[3] = (byte) (width >> 24 & 0xff);
                    wr.write(widthBytes);
                    byte[] heightBytes = new byte[4];
                    heightBytes[0] = (byte) (height & 0xff);
                    heightBytes[1] = (byte) (height >> 8 & 0xff);
                    heightBytes[2] = (byte) (height >> 16 & 0xff);
                    heightBytes[3] = (byte) (height >> 24 & 0xff);
                    wr.write(heightBytes);
                    byte[] flipsBytes = new byte[4];
                    flipsBytes[0] = (byte) (flips & 0xff);
                    flipsBytes[1] = (byte) (flips >> 8 & 0xff);
```

```
flipsBytes[2] = (byte) (flips >> 16 & 0xff);
flipsBytes[3] = (byte) (flips >> 24 & 0xff);
wr.write(flipsBytes);
//写头帧
for (int i = 0; i < height; i++) {
    for (int j = 0; j < width; j++) {
        int pix = bufferedImage.getRGB(j, i);
        flipBytes[i * (width * 4) + j * 4 + 0] = (byte) (pix & 0xff);
        flipBytes[i * (width * 4) + j * 4 + 1] = (byte) (pix >> 8 & 0xff);
        flipBytes[i * (width * 4) + j * 4 + 2] = (byte) (pix >> 16 & 0xff);
        flipBytes[i * (width * 4) + j * 4 + 3] = (byte) (pix >> 24 & 0xff);
    }
}
wr.write(flipBytes);
wr.flush();
//写身体
while (iterator.hasNext()) {
    //写身体每一帧
    bufferedImage = flipImage.get(iterator.next());
    //int[][] flipImage= iterator.next();
    for (int i = 0; i < height; i++) {
        for (int j = 0; j < width; j++) {
            int pix = bufferedImage.getRGB(j, i);
            //byte[] flip=new byte[4];
            flipBytes[i * (width * 4) + j * 4 + 0] = (byte) (pix & 0xff);
            flipBytes[i * (width * 4) + j * 4 + 1] = (byte) (pix >> 8 & 0xff);
            flipBytes[i * (width * 4) + j * 4 + 2] = (byte) (pix >> 16 & 0xff);
            flipBytes[i * (width * 4) + j * 4 + 3] = (byte) (pix >> 24 & 0xff);
        }
    }
    wr.write(flipBytes);
    wr.flush();
}
}
wr.close();
} catch (FileNotFoundException e1) {
    e1.printStackTrace();
} catch (IOException e1) {
    e1.printStackTrace();
}
}
}
});
processBox.add(buttonPlayer);
processBox.add(buttonPause);
processBox.add(buttonRollback);
processBox.add(buttonReDo);
```

```

        processBox.add(buttonFast);
        processBox.add(buttonSlow);
        processBox.add(buttonSave);
        processBox.add(buttonNew);
        JFrame jframeControl = new JFrame();
        jframeControl.setLocation(200, 675);
        jframeControl.setSize(640, 150);
        jframeControl.setResizable(true);
        ScrollPane scrollPaneControl = new ScrollPane();
        mainBox.add(slideBox);
        mainBox.add(processBox);
        scrollPaneControl.add(mainBox);
        scrollPaneControl.validate();
        jframeControl.add(scrollPaneControl);
        jframeControl.setBackground(Color.pink);
        jframeControl.setVisible(true);
        jframeControl.validate();
    }
}

//Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/PlayerControlTest.java
//Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/PlayerShow.java
package M_V.AVQ.PVU.vedio;

import javax.swing.JFrame;
import java.awt.Color;
import java.awt.event.MouseEvent;
import java.awt.event.MouseMotionListener;
import java.awt.image.BufferedImage;
import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class PlayerShow extends Thread implements Runnable, MouseMotionListener {
    int widthInt;
    int heightInt;
    int[] crontrol;
    PlayerControl playerControl;
    Map<Integer, BufferedImage> flipImage;
    ThreadPanel panel;

    public PlayerShow(PlayerControl playerControl, int widthInt, int heightInt
        , Map<Integer, BufferedImage> flipImage, int[] crontrol) {
        this.widthInt = widthInt;
        this.heightInt = heightInt;

```

```

        this.flipImage = flipImage;

        this.crontol = crontol;

        this.playerControl = playerControl;
    }

    public void run() {
        JFrame jframe = new JFrame();
        jframe.setLayout(null);
        jframe.setLocation(200, 200);
        jframe.setSize(widthInt, heightInt);
        jframe.setResizable(true);
        panel = new ThreadPanel(playerControl, widthInt, heightInt, flipImage, crontol);
        panel.setLayout(null);
        jframe.add(panel);
        //panel.addMouseListener(this);
        panel.addMouseMotionListener(this);
        //jframe.addMouseListener(this);
        jframe.setBackground(Color.BLUE);
        jframe.setVisible(true);
        jframe.validate();
        panel.run();
    }

    @Override
    public void mouseDragged(MouseEvent arg0) {
        int x = arg0.getX();
        int y = arg0.getY();
        panel.paint(x, y);
    }

    @Override
    public void mouseMoved(MouseEvent arg0) {
    }
}

```

```

//Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/PlayerShowTest.java
//Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/ThreadPanel.java
package M_V.AVQ.PVU.vedio;

```

```

import exception.thread.DetaThread;

import java.awt.Graphics;

import java.awt.Panel;

import java.awt.image.BufferedImage;

import java.util.Map;

```

```

/*

```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class ThreadPanel extends Panel implements Runnable {

    int widthInt;
    int heightInt;
    int[] control;
    PlayerControl playerControl;
    Map<Integer, BufferedImage> flipImage;
    BufferedImage flip;

    public ThreadPanel(PlayerControl playerControl, int widthInt, int heightInt
        , Map<Integer, BufferedImage> flipImage, int[] control) {
        this.setBounds(0, 0, widthInt, heightInt);
        this.setVisible(true);
        this.widthInt = widthInt;
        this.heightInt = heightInt;
        this.flipImage = flipImage;
        this.control = control;
        this.playerControl = playerControl;
    }

    public void run() {
        //init;
        control[6] = 0;
        int j = flipImage.size();
        Graphics g = this.getGraphics();
        while (true) {
            if (control[6] < 0 || control[6] > j) {
                control[6] = 0;
            }
            DetaThread.sleepDeta(50);
            if (1 == control[0]) { //确定播放
                if (1 == control[7]) {
                    g.drawImage(flipImage.get(control[6]), 0, 0, this);
                    control[0] = 0;
                    control[7] = 0;
                }
                if (1 == control[1]) { //确定倒放
                    control[6]--;
                    if (control[6] < 0) { //停止条件
                        control[0] = 0;
                        control[6] = 0;
                    }
                }
                if (0 < control[3]) { //确定加速放
                    DetaThread.sleepDeta(300 / control[3]);
                    if (flipImage.containsKey(control[6])) {
                        if (null != playerControl.sliderx) {
                            playerControl.sliderx.setValue(control[6]);
                        }
                    }
                }
            }
        }
    }
}
```

```

        g.drawImage(flipImage.get(control[6]), 0, 0, this);
    }
} else if (0 < control[4]) { //确定减速速放
    DetaThread.sleepDeta(300 + 200 * control[4]);
    if (flipImage.containsKey(control[6])) {
        if (null != playerControl.sliderx) {
            playerControl.sliderx.setValue(control[6]);
        }
        g.drawImage(flipImage.get(control[6]), 0, 0, this);
    }
}
} else if (1 == control[2]) { //确定正放
    control[6]++;
    if (control[6] >= j) { //停止条件
        control[0] = 0;
        control[6] = 0;
    }
    if (0 < control[3]) { //确定加速放
        DetaThread.sleepDeta(300 / control[3]);
        if (flipImage.containsKey(control[6])) {
            if (null != playerControl.sliderx) {
                playerControl.sliderx.setValue(control[6]);
            }
            g.drawImage(flipImage.get(control[6]), 0, 0, this);
        }
    } else if (0 < control[4]) { //确定减速速放
        DetaThread.sleepDeta(300 + 200 * control[4]);
        if (flipImage.containsKey(control[6])) {
            if (null != playerControl.sliderx) {
                playerControl.sliderx.setValue(control[6]);
            }
            g.drawImage(flipImage.get(control[6]), 0, 0, this);
        }
    }
}
}
}
}
}

public void paint(int x, int y) {
    Graphics g = this.getGraphics();
    g.drawLine(x, y, x + 1, y + 1);
    g.drawLine(x, y + 1, x + 1, y);
    Graphics gf = flipImage.get(control[6]).getGraphics();
    gf.drawLine(x, y, x + 1, y + 1);
    gf.drawLine(x, y + 1, x + 1, y);
}
}
}

```

/Trif_DNA-master/C_L_M/M_V/AVQ/PVU/vedio/ThreadPanelTest.java

M_V.ME.APM.VSQ.editPane

/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane_X_.java
package M_V.ME.APM.VSQ.editPane;

import A_V.C.RatioMap;
import A_V.E.RatioMap_E;
import E_A.OEIAMV.ECS.SVQ.MPC.SOQ.OEM.E.SensingMap_E;
import E_A.ME.analysis.E.AE;
import ME.VPC.M.app.App;
import ME.VPC.S.ne.WindowsUI;
import M_V.MVQ.button.DetaButton;
import M_V.MVQ.filenameFilter.TXTFilter;
import O_V.OSI.AOP.neo.tts.ReadChinese;
import P.image.Color_P;
import P_V.PEQ.AMV.ECS.test.SensingTest;
import S_A.SVQ.stable.S_Maps;
import S_A.VSQ.parser.EmotionSample;
import S_A.pheromone.IMV_SQI;
import U_V.ESU.string.StringReplace;
import exception.thread.DetaThread;
import javax.swing.JScrollPane;
import javax.swing.JTextPane;
import java.awt.Color;
import java.awt.Container;
import java.awt.FileDialog;
import java.awt.Frame;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
public class EditPane_X_ extends Container implements KeyListener {
 private IMV_SQI ctj;
 private IMV_SQI ctk;

```
private IMV_SQI ctt;
private IMV_SQI ctr;
private IMV_SQI cko;
private IMV_SQI cfn;
private IMV_SQI csp;
private IMV_SQI cgm;
//
public App NE;
private IMV_SQI pinyin;
private EmotionSample emotionSample;
private RatioMap ratioMap;
private RatioMap_E ratioMap_E;
private SensingTest sensingTest;
private SensingMap_E sensingMap_E;
public String key;
public String dataValue;
public JTextPane data;
public JTextPane text;
public IMV_SQI pose;
public IMV_SQI etc;
public IMV_SQI cte;
public IMV_SQI pos;
public AE _A;
public Detabutton buttonHighLight;
public EditPane_X_(JTextPane text, App NE) {
    this.text = text;
    this.key = "";
    this.pose = NE.app_S.fMHMMListOneTime_E_X_S.posEnToEn;
    this.etc = NE.app_S.fMHMMListOneTime_E_X_S.fullEnToCn;
    this.cte = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToEn;
    this.pos = NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn;
    this._A = NE.app_S._A;
    this.ctj = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToJp;
    this.ctl = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToAb;
    this.ctt = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToTt;
    this.ctr = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToRs;
    this.cko = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToKo;
    this.cfn = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToFn;
    this.csp = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToSp;
    this.cgm = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToGm;
    this.ratioMap_E = NE.app_S.ratioMap_E;
    this.pinyin = NE.app_S.fMHMMListOneTime_E_X_S.fullCnToPy;
    this.emotionSample = NE.app_S.emotionSample;
    this.ratioMap = NE.app_S.ratioMap;
    this.sensingTest = NE.app_S.sensingTest;
    this.sensingMap_E = NE.app_S.sensingMap_E;
    this.setLayout(null);
```

```

this.setBounds(0, 0, WindowsUI.makeContainerWidth
    , WindowsUI.makeContainerHeight);
JScrollPane jsp_data = new JScrollPane(this.data(NE));
jsp_data.setBounds(605, 80, WindowsUI.basicWidth - 750, 550);
JScrollPane jsp_text = new JScrollPane(this.text);
jsp_text.setBounds(5, 80, WindowsUI.basicWidth - 850, 550);
I_OutputButton(NE);
I_TranslateButton(NE);
I_ExplainButton(NE);
I_ClearButton(NE);
I_TxtButton(NE);
I_PrintButton(NE);
I_ReadButton(NE);
I_PinYinButton(NE);
I_ArabicButton(NE);
I_TranditionButton(NE);
I_JapanButton(NE);
I_RussianButton(NE);
I_KoreanButton(NE);
I_FnButton(NE);
I_SpButton(NE);
I_GmButton(NE);
this.add(jsp_data);
this.add(jsp_text);
}

private void I_KoreanButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("韩语");
    button.setBounds(845, 33, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(
                text.getText().trim()).iterator();
            if (text.getSelectedText() != null
                && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(
                    text.getSelectedText().trim()).iterator();
            }
            text.setSelectionStart(0);
            text.setSelectionEnd(0);
            String split = "【";
            String splite = "】 ";
            Here:

```

```

while (iterator.hasNext()) {
    String setOfi = iterator.next();
    setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
    if (pos.get(setOfi) == null) {
        page.append("<span style=\"background:#F1F1F1\">" +
            "<font color=\"black\" \"\"
            + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
            + setOfi + (cko.containsKey(setOfi)
            ? split + cko.get(setOfi) + splite : "");
        continue Here;
    }
    if (!setOfi.equals("")) {
        if (StringReplace.pos(key, pos, cko, setOfi, page, NE)) {
            continue Here;
        }
        page.append("<span style=\"background:white\">" +
            "<font color=\"black\" \"\"
            + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
            + setOfi + (cko.containsKey(setOfi)
            ? split + cko.get(setOfi) + splite : "")
            + "</font></span>");
    }
}
data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
});
this.add(button);
}

private void I_FnButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("法语");
    button.setBounds(950, 33, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(
                text.getText().trim()).iterator();
            if (text.getSelectedText() != null
                && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(
                    text.getSelectedText().trim()).iterator();
            }
        }
    });
}

```

```

    }
    text.setSelectionStart(0);
    text.setSelectionEnd(0);
    String split = "【";
    String splite = "】 ";
    Here:
    while (iterator.hasNext()) {
        String setOfi = iterator.next();
        setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
        if (pos.get(setOfi) == null) {
            page.append("<span style=\"background:#F1F1F1\">" +
                "<font color=\"black\" \"\"
                + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
                + setOfi + (cfn.containsKey(setOfi)
                ? split + cfn.get(setOfi) + splite : ""));
            continue Here;
        }
        if (!setOfi.equals("")) {
            if (StringReplace.pos(key, pos, cfn, setOfi, page, NE)) {
                continue Here;
            }
            page.append("<span style=\"background:white\">" +
                "<font color=\"black\" \"\"
                + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
                + setOfi + (cfn.containsKey(setOfi)
                ? split + cfn.get(setOfi) + splite : "")
                + "</font></span>");
        }
    }
    data.setText(page.toString());
    data.setSelectionStart(0);
    data.setSelectionEnd(0);
    data.validate();
}
});
this.add(button);
}

private void I_SpButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("西语");
    button.setBounds(1055, 33, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();

```

```

Iterator<String> iterator = _A.parserMixedString(
    text.getText().trim()).iterator();
if (text.getSelectedText() != null
    && !text.getSelectedText().trim().isEmpty()) {
    iterator = _A.parserMixedString(
        text.getSelectedText().trim()).iterator();
}
text.setSelectionStart(0);
text.setSelectionEnd(0);
String split = "【";
String splite = "】 ";
Here:
while (iterator.hasNext()) {
    String setOfi = iterator.next();
    setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
    if (pos.get(setOfi) == null) {
        page.append("<span style=\"background:#F1F1F1\">" +
            "<font color=\"black\" \"\"
            + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
            + setOfi + (csp.containsKey(setOfi)
                ? split + csp.get(setOfi) + splite : ""));
        continue Here;
    }
    if (!setOfi.equals("")) {
        if (StringReplace.pos(key, pos, csp, setOfi, page, NE)) {
            continue Here;
        }
        page.append("<span style=\"background:white\">" +
            "<font color=\"black\" \"\"
            + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
            + setOfi + (csp.containsKey(setOfi)
                ? split + csp.get(setOfi) + splite : "")
            + "</font></span>");
    }
}
data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
});
this.add(button);
}
private void I_GmButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("德语");
    button.setBounds(1160, 33, 100, 20);
    button.addActionListener(new ActionListener() {

```

```
@Override
```

```
public void actionPerformed(ActionEvent arg0) {
    if (text.getText().equalsIgnoreCase("")) {
        return;
    }
    StringBuilder page = new StringBuilder();
    Iterator<String> iterator = _A.parserMixedString(
        text.getText().trim()).iterator();
    if (text.getSelectedText() != null
        && !text.getSelectedText().trim().isEmpty()) {
        iterator = _A.parserMixedString(
            text.getSelectedText().trim()).iterator();
    }
    text.setSelectionStart(0);
    text.setSelectionEnd(0);
    String split = " 【";
    String splite = "】 ";
    Here:
    while (iterator.hasNext()) {
        String setOfi = iterator.next();
        setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
        if (pos.get(setOfi) == null) {
            page.append("<span style=\"background:#F1F1F1\">" +
                "<font color=\"black\" \"\"
                + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
                + setOfi + (cgm.containsKey(setOfi)
                    ? split + cgm.get(setOfi) + splite : ""));
            continue Here;
        }
        if (!setOfi.equals("")) {
            if (StringReplace.pos(key, pos, cgm, setOfi, page, NE)) {
                continue Here;
            }
            page.append("<span style=\"background:white\">" +
                "<font color=\"black\" \"\"
                + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
                + setOfi + (cgm.containsKey(setOfi)
                    ? split + cgm.get(setOfi) + splite : "")
                + "</font></span>");
        }
    }
    data.setText(page.toString());
    data.setSelectionStart(0);
    data.setSelectionEnd(0);
    data.validate();
}
});
```

```

this.add(button);
}

private void I_ArabicButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("阿语");
    button.setBounds(950, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(
                text.getText().trim()).iterator();
            if (text.getSelectedText() != null
                && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(
                    text.getSelectedText().trim()).iterator();
            }
            text.setSelectionStart(0);
            text.setSelectionEnd(0);
            String split = " 【";
            String splite = "】 ";
            Here:
            while (iterator.hasNext()) {
                String setOfi = iterator.next();
                setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
                if (pos.get(setOfi) == null) {
                    page.append("<span style=\"background:#F1F1F1\">" +
                        "<font color=\"black\" \"\" +
                        + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
                        + setOfi + (ctk.containsKey(setOfi)
                            ? split + ctk.get(setOfi) + splite : ""));
                    continue Here;
                }
                if (!setOfi.equals("")) {
                    if (StringReplace.pos(key, pos, ctk, setOfi, page, NE)) {
                        continue Here;
                    }
                }
                page.append("<span style=\"background:white\">" +
                    "<font color=\"black\" \"\" +
                    + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
                    + setOfi + (ctk.containsKey(setOfi)
                        ? split + ctk.get(setOfi) + splite : ""
                        + "</font></span>");
            }
        }
    }
}

```



```

        data.setText(page.toString());
        data.setSelectionStart(0);
        data.setSelectionEnd(0);
        data.validate();
    }
});
this.add(button);
}

private void I_RussianButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("俄语");
    button.setBounds(1160, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(
                text.getText().trim()).iterator();
            if (text.getSelectedText() != null
                && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(
                    text.getSelectedText().trim()).iterator();
            }
            text.setSelectionStart(0);
            text.setSelectionEnd(0);
            String split = " 【";
            String splite = "】 ";
            Here:
            while (iterator.hasNext()) {
                String setOfi = iterator.next();
                setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
                if (pos.get(setOfi) == null) {
                    page.append("<span style=\"background:#F1F1F1\">" +
                        "<font color=\"black\" \"\"
                        + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
                        + setOfi + (ctr.containsKey(setOfi)
                            ? split + ctr.get(setOfi) + splite : ""));
                    continue Here;
                }
                if (!setOfi.equals("")) {
                    if (StringReplace.pos(key, pos, ctr, setOfi, page, NE)) {
                        continue Here;
                    }
                }
                page.append("<span style=\"background:white\">" +
                    "<font color=\"black\" \"\"

```

```

        + " size=\"" + NE.app_S.size_font_rot + "\">"
        + setOfi + (ctr.containsKey(setOfi)
        ? split + ctr.get(setOfi) + splite : "")
        + "</font></span>");
    }
}
data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
});
this.add(button);
}
private void I_TrandonButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("繁体");
    button.setBounds(845, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(
                text.getText().trim()).iterator();
            if (text.getSelectedText() != null
                && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(
                    text.getSelectedText().trim()).iterator();
            }
            text.setSelectionStart(0);
            text.setSelectionEnd(0);
            String split = "【";
            String splite = "】 ";
            Here:
            while (iterator.hasNext()) {
                String setOfi = iterator.next();
                setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
                if (pos.get(setOfi) == null) {
                    page.append("<span style=\""background:#F1F1F1\">" +
                        "<font color=\""black\">"
                        + " size=\"" + NE.app_S.size_font_rot + "\">"
                        + setOfi
                        + (ctt.containsKey(setOfi) ? split
                        + ctt.get(setOfi) + splite : ""));
                    continue Here;
                }
            }
        }
    });
}

```

```

    }
    if (!setOfi.equals("")) {
        if (StringReplace.pos(key, pos, ctt, setOfi, page, NE)) {
            continue Here;
        }
        page.append("<span style=\"background:white\">" +
            "<font color=\"black\" "
            + " size=\"" + NE.app_S.size_font_rot + "\">"
            + setOfi + (ctt.containsKey(setOfi)
            ? split + ctt.get(setOfi) + splite : "")
            + "</font></span>");
    }
}
data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
});
this.add(button);
}

private void I_JapanButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("日语");
    button.setBounds(1055, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(
                text.getText().trim()).iterator();
            if (text.getSelectedText() != null
                && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(
                    text.getSelectedText().trim()).iterator();
            }
            text.setSelectionStart(0);
            text.setSelectionEnd(0);
            String split = "【";
            String splite = "】";
            Here:
            while (iterator.hasNext()) {
                String setOfi = iterator.next();
                setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
                if (pos.get(setOfi) == null) {

```

```

        page.append("<span style=\"background:#F1F1F1\">" +
            "<font color=\"black\" "
            + " size=\" " + NE.app_S.size_font_rot + "\">"
            + setOfi + (ctj.containsKey(setOfi)
            ? split + ctj.get(setOfi) + splite : "");
        continue Here;
    }
    if (!setOfi.equals("")) {
        if (StringReplace.pos(key, pos, ctj, setOfi, page, NE)) {
            continue Here;
        }
        page.append("<span style=\"background:white\">" +
            "<font color=\"black\" "
            + "size=\"5\">"
            + setOfi
            + (ctj.containsKey(setOfi) ? split
            + ctj.get(setOfi) + splite : "") + "</font></span>");
    }
}
data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
});
this.add(button);
}

private void I_PinYinButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("拼音");
    button.setBounds(740, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(
                text.getText().trim()).iterator();
            if (text.getSelectedText() != null
                && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(
                    text.getSelectedText().trim()).iterator();
            }
            text.setSelectionStart(0);
            text.setSelectionEnd(0);
            Here:

```

```

while (iterator.hasNext()) {
    String setOfi = iterator.next();
    setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
    if (pos.get(setOfi) == null) {
        page.append("<span style=\"background:#F1F1F1\">" +
            "<font color=\"black\" "
            + " size=\"" + NE.app_S.size_font_rot + "\">"
            + setOfi + (pinyin.containsKey(setOfi)
                ? pinyin.get(setOfi) : "");
            continue Here;
        }
    if (!setOfi.equals("")) {
        if (StringReplace.pos(key, pos, pinyin, setOfi, page, NE)) {
            continue Here;
        }
        page.append("<span style=\"background:white\">" +
            "<font color=\"black\" "
            + " size=\"" + NE.app_S.size_font_rot + "\">"
            + setOfi + (pinyin.containsKey(setOfi)
                ? pinyin.get(setOfi) : "") + "</font></span>");
    }
}
data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
});
this.add(button);
}

private void I_ReadButton(App NE) {
    M_V.MVQ.button.DetaButton button = new M_V.MVQ.button.DetaButton("阅读关");
    button.setBounds(635, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        ReadChinese readChinese = new ReadChinese(button, NE.app_S);
        @SuppressWarnings("deprecation")
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (button.getText().equalsIgnoreCase("阅读开")) {
                readChinese.waitString = true;
                readChinese.waitWord = true;
                return;
            }
            if (readChinese.getState().toString()
                .equalsIgnoreCase("TERMINATED")) {
                readChinese.I_NullSap();
                readChinese = new ReadChinese(button, NE.app_S);
            }
        }
    });
}

```

```

    }
    if (text == null) {
        return;
    }
    if (readChinese.finish == 0) {
        if (!readChinese.isAlive()) {
            button.setLabel("阅读开");
            button.updateUI();
            DetaThread.sleepDeta(200);
            long stopBy = 25;
            char[] words = text.getText().toCharArray();
            List<String> list = new ArrayList<>();
            int i = 0;
            while (i < words.length) {
                int j = 0;
                StringBuilder stringBuilder = new StringBuilder();
                while (j < stopBy && i + j < words.length) {
                    stringBuilder.append(words[i + (j++)]);
                }
                list.add(stringBuilder.toString());
                i += j;
            }
            readChinese.I_PreReadList(list, NE);
            if (readChinese.finish == 2) {
                readChinese.start();
            }
        }
    }
}

});
this.add(button);
}

private void I_PrintButton(App NE) {
    DetaButton button = new DetaButton("打印");
    button.setBounds(530, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            String temp = text.getText();
            String[] lines = temp.split("\r\n");
            List<String> trueLines = new ArrayList<>();
            for (int i = 0; i < lines.length; i++) {
                int line_count = lines[i].length() / 30;
                String[] lineOfStrings = new String[line_count + 1];

```

```
        for (int j = 0; j <= line_count; j++) {
            lineOfStrings[j] = lines[i].substring(j * 30 > lines[i].length()
                ? lines[i].length() : j * 30, (j + 1) * 30
                > lines[i].length() ? lines[i].length() : (j + 1) * 30);
            trueLines.add(lineOfStrings[j]);
        }
    }

    int page = trueLines.size() / 46;
    for (int i = 0; i <= page; i++) {
        new PrintTest4(trueLines.subList((i * 46 > trueLines.size()
            ? trueLines.size() : (i * 46, (i + 1) * 46
            > trueLines.size() ? trueLines.size() : (i + 1) * 46));
    }
}

});
this.add(button);
}

private void I_TxtButton(App NE) {
    DetaButton button = new DetaButton("生成TXT");
    button.setBounds(425, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            //get path
            FileDialog filedialog = new FileDialog(new Frame()
                , "导入到这里", FileDialog.LOAD);
            filedialog.setFilenameFilter(new TXTFilter("txt"));
            filedialog.setVisible(true);
            String filepath = filedialog.getDirectory() + filedialog.getFile();
            //20230106-S_logger.Log.logger.info("" + filepath);
            //write file
            FileWriter fw = null;
            try {
                fw = new FileWriter(filepath, true);
                fw.write(text.getText());
                fw.close();
            } catch (IOException e) {
                try {
                    fw.close();
                } catch (IOException e1) {
                    fw = null;
                    e1.printStackTrace();
                }
            }
            e.printStackTrace();
        }
    });
}
```



```

        + " size=\"" + NE.app_S.size_font_rot + "\">"
        + setOfi + "</font></span>");
    continue Here;
}
if (S_Maps.dongCi.containsKey(setOfi)) {
    page.append("<span style=\"background:"
        + Color_P.P(245, 255, 245)
        + "\"><font color=\"black\" \"
        + " size=\"" + NE.app_S.size_font_rot + "\">"
        + setOfi + "</font></span>");
    continue Here;
}
if (S_Maps.xingRongCi.containsKey(setOfi)) {
    page.append("<span style=\"background:"
        + Color_P.P(255, 255, 245)
        + "\"><font color=\"black\" \"
        + " size=\"" + NE.app_S.size_font_rot + "\">"
        + setOfi + "</font></span>");
    continue Here;
}
if (S_Maps.fuCi.containsKey(setOfi)) {
    page.append("<span style=\"background:#F1FFFF\">" +
        "<font color=\"black\" \"
        + " size=\"" + NE.app_S.size_font_rot + "\">"
        + setOfi + "</font></span>");
    continue Here;
}
page.append("<span style=\"background:white\">" +
    "<font color=\"black\" \"
    + " size=\"" + NE.app_S.size_font_rot + "\">"
    + setOfi + "</font></span>");
}
}
data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
});
this.add(button);
}
private void I_ExplainButton(App NE) {
    DetaButton button = new DetaButton("解释");
    button.setBounds(5 + 105 + 105, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {

```

```

if (text.getText().equalsIgnoreCase("")) {
    return;
}
StringBuilder page = new StringBuilder();
Iterator<String> iterator = _A.parserMixedString(
    text.getText().trim()).iterator();
if (text.getSelectedText() != null
    && !text.getSelectedText().trim().isEmpty()) {
    iterator = _A.parserMixedString(
        text.getSelectedText().trim()).iterator();
}
text.setSelectionStart(0);
text.setSelectionEnd(0);
Here:
while (iterator.hasNext()) {
    String setOfi = iterator.next();
    setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
    if (pos.get(setOfi) == null) {
        page.append("<span style=\"background:#F1F1F1\">" +
            "<font color=\"black\"\"
            + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
            + setOfi + (cte.containsKey(setOfi)
            ? (etc.containsKey(cte.get(setOfi))
            ? ":" + etc.get(cte.get(setOfi))
            + ")" : "")) : "")) + "</font></span>");
        continue Here;
    }
    if (!setOfi.equals("")) {
        if (key.containsKey(setOfi) && (S_Maps.mingCi.containsKey(setOfi)
            || S_Maps.dongCi.containsKey(setOfi)
            || S_Maps.xingRongCi.containsKey(setOfi))) {
            page.append("<span style=\"background:red\">" +
                "<font color=\"white\">"
                + setOfi + (cte.containsKey(setOfi)
                ? (etc.containsKey(cte.get(setOfi))
                ? ":" + etc.get(cte.get(setOfi))
                + ")" : "")) : "")) + "</font></span>");
            continue Here;
        }
        if (S_Maps.mingCi.containsKey(setOfi)) {
            page.append("<span style=\"background:"
                + Color_P.P(255, 245, 255)
                + "\"><font color=\"black\"\"
                + \" size=\"\" + NE.app_S.size_font_rot + \"\">"
                + setOfi + (cte.containsKey(setOfi)
                ? (etc.containsKey(cte.get(setOfi))
                ? ":" + etc.get(cte.get(setOfi))

```

```

        + ")" : "" : "" : "" + "</font></span>");
        continue Here;
    }
    if (S_Maps.dongCi.containsKey(setOfi)) {
        page.append("<span style=\"background:"
            + Color_P.P(245, 255, 245)
            + "\"><font color=\"black\"
            + " size=\"\" + NE.app_S.size_font_rot + "\">"
            + setOfi + (cte.containsKey(setOfi)
            ? (etc.containsKey(cte.get(setOfi))
            ? "(" + etc.get(cte.get(setOfi))
            + ")" : "" : "" : "" + "</font></span>");
        continue Here;
    }
    if (S_Maps.xingRongCi.containsKey(setOfi)) {
        page.append("<span style=\"background:"
            + Color_P.P(255, 255, 245)
            + "\"><font color=\"black\"
            + " size=\"\" + NE.app_S.size_font_rot + "\">"
            + setOfi + (cte.containsKey(setOfi)
            ? (etc.containsKey(cte.get(setOfi))
            ? "(" + etc.get(cte.get(setOfi))
            + ")" : "" : "" : "" + "</font></span>");
        continue Here;
    }
    if (S_Maps.fuCi.containsKey(setOfi)) {
        page.append("<span style=\"background:#F1FFFF\">" +
            "<font color=\"black\"
            + " size=\"\" + NE.app_S.size_font_rot + "\">"
            + setOfi + (cte.containsKey(setOfi)
            ? (etc.containsKey(cte.get(setOfi))
            ? "(" + etc.get(cte.get(setOfi))
            + ")" : "" : "" : "" + "</font></span>");
        continue Here;
    }
    page.append("<span style=\"background:white\">" +
        "<font color=\"black\"
        + " size=\"\" + NE.app_S.size_font_rot + "\">"
        + setOfi + (cte.containsKey(setOfi)
        ? (etc.containsKey(cte.get(setOfi))
        ? "(" + etc.get(cte.get(setOfi))
        + ")" : "" : "" : "" + "</font></span>");
    }
}

data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);

```

```

        data.validate();
    }
});
this.add(button);
}

private void I_TranslateButton(App NE) {
    DetaButton button = new DetaButton("英译");
    button.setBounds(5 + 105, 3, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(
                text.getText().trim()).iterator();
            if (text.getSelectedText() != null
                && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(
                    text.getSelectedText().trim()).iterator();
            }
            text.setSelectionStart(0);
            text.setSelectionEnd(0);
            Here:
            while (iterator.hasNext()) {
                String setOfi = iterator.next();
                setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
                if (pos.get(setOfi) == null) {
                    page.append("<span style=\"background:#F1F1F1\">" +
                        "<font color=\"black\" \"\" "
                        + " size=\"\" + NE.app_S.size_font_rot + "\">"
                        + setOfi + (cte.containsKey(setOfi)
                            ? ":(\" + cte.get(setOfi) + \") : \"") + "</font></span>");
                    continue Here;
                }
                if (!setOfi.equals("")) {
                    if (StringReplace.pos(key, pos, cte, setOfi, page, NE)) {
                        continue Here;
                    }
                    page.append("<span style=\"background:white\">" +
                        "<font color=\"black\" \"\" "
                        + " size=\"\" + NE.app_S.size_font_rot + "\">"
                        + setOfi + (cte.containsKey(setOfi)
                            ? ":(\" + cte.get(setOfi) + \") : \"") + "</font></span>");
                }
            }
        }
    }
}

```

```

        data.setText(page.toString());
        data.setSelectionStart(0);
        data.setSelectionEnd(0);
        data.validate();
    }
});
this.add(button);
}

private void I_OutputButton(App NE) {
    buttonHighLight = new DetaButton("高亮");
    buttonHighLight.setBounds(5, 3, 100, 20);
    buttonHighLight.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            StringBuilder page = new StringBuilder();
            Iterator<String> iterator = _A.parserMixedString(text.getText().trim()).iterator();
            if (text.getSelectedText() != null && !text.getSelectedText().trim().isEmpty()) {
                iterator = _A.parserMixedString(text.getSelectedText().trim()).iterator();
            }
            text.setSelectionStart(0);
            text.setSelectionEnd(0);
            Here:
            while (iterator.hasNext()) {
                String setOfi = iterator.next();
                setOfi = StringReplace.StringHTMLReplaceARCP(setOfi);
                if (pos.get(setOfi) == null) {
                    page.append("<span style=\"background:#818181\">" + "<font color=\"black\"\"
                        + " size=\"" + NE.app_S.size_font_rot + "\">"
                        + setOfi + "</font></span>");
                    continue Here;
                }
                if (!setOfi.equals("")) {
                    if (key.contains(setOfi) && (S_Maps.mingCi.containsKey(setOfi)
                        || S_Maps.dongCi.containsKey(setOfi)
                        || S_Maps.xingRongCi.containsKey(setOfi))) {
                        page.append("<span style=\"background:red\">"
                            + "<font color=\"white\">" + setOfi + "</font></span>");
                        continue Here;
                    }
                    if (S_Maps.mingCi.containsKey(setOfi)) {
                        page.append("<span style=\"background:"
                            + Color_P.P(245, 215
                                - NE.app_S.raterot << 3, 245)
                            + "\"><font color=\"black\"\" + " size=\""

```

```

        + NE.app_S.size_font_rot + "\">" + setOfi
        + "</font></span>");
    continue Here;
}
if (S_Maps.dongCi.containsKey(setOfi)) {
    page.append("<span style=\"background:"
        + Color_P.P(245
            - NE.app_S.raterot << 3, 255
        , 245 - 40 - NE.app_S.raterot << 3)
        + "\"><font color=\"black\" \" + \" size=\" \"
        + NE.app_S.size_font_rot + "\">"
        + setOfi + "</font></span>");
    continue Here;
}
if (S_Maps.xingRongCi.containsKey(setOfi)) {
    page.append("<span style=\"background:"
        + Color_P.P(245, 255
        , 245 - 40 - NE.app_S.raterot << 3)
        + "\"><font color=\"black\" \" + \" size=\" \"
        + NE.app_S.size_font_rot + "\">" + setOfi
        + "</font></span>");
    continue Here;
}
if (S_Maps.fuCi.containsKey(setOfi)) {
    page.append("<span style=\"background:#81FFFF\">"
        + "<font color=\"black\" \" + \" size=\" \"
        + NE.app_S.size_font_rot + "\">" + setOfi
        + "</font></span>");
    continue Here;
}
page.append("<span style=\"background:white\">"
    + "<font color=\"black\" \" + \" size=\" \"
    + NE.app_S.size_font_rot + "\">" + setOfi
    + "</font></span>");
}
}
data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
});
this.add(buttonHighLight);
}
public int getPagesCount(String text) {
    int page = 0;
    int AMV_MVS_VSQ, count = 0;

```

```

String str = text;
while (str.length() > 0) {
    AMV_MVS_VSQ = str.indexOf("\n");
    count += 1;
    if (AMV_MVS_VSQ != 1) {
        str = str.substring(AMV_MVS_VSQ + 1);
    } else {
        str = "";
    }
}
if (count > 0) {
    page = count / 54 + 1;
}
return page;
}

private JTextPane data(App NE) {
    data = new JTextPane();
    data.setContentType("text/html");
    data.setBackground(new Color(210, 210, 210));
    return data;
}

@Override
public void keyPressed(KeyEvent arg0) {
    // TODO Auto-generated method stub
}

@Override
public void keyReleased(KeyEvent arg0) {
    // TODO Auto-generated method stub
}

@Override
public void keyTyped(KeyEvent arg0) {
    // TODO Auto-generated method stub
}
}

//339
//if(key.contains(setOfi)&&(pos.get(setOfi).contains("名")
//||pos.get(setOfi).contains("动")||pos.get(setOfi).contains("形"))) {
//page.append("<span style=\"background:red\"><font color=\"white\">" + setOfi
//      + (ctj.containsKey(setOfi)?split+ ctj.get(setOfi)+ splite:"")+ "</font></span>");
//continue Here;
//}
//if(pos.get(setOfi).contains("名")) {
//page.append("<span style=\"background:"+U_A.PEU.P.image.Color_P.P(255, 245, 255)
//      + "\"><font color=\"black\" size=\"5\">" + setOfi
//      + (ctj.containsKey(setOfi)?split+ ctj.get(setOfi)+ splite:"")+ "</font></span>");
//continue Here;
//}

```

```
//if(pos.get(setOfi).contains("动")) {
//page.append("<span style=\"background:"+U_A.PEU.P.image.Color_P.P(245, 255, 245)
//      + "\"><font color=\"black\" size=\"5\">\" + setOfi
//      + (ctj.containsKey(setOfi)?split+ ctj.get(setOfi)+ splite:\"")+ "</font></span>");
//continue Here;
//}
//if(pos.get(setOfi).contains("形")) {
//page.append("<span style=\"background:"+U_A.PEU.P.image.Color_P.P(255, 255, 245)
//      + "\"><font color=\"black\" size=\"5\">\" + setOfi
//      + (ctj.containsKey(setOfi)?split+ ctj.get(setOfi)+ splite:\"")+ "</font></span>");
//continue Here;
//}
//if(pos.get(setOfi).contains("副")) {
//page.append("<span style=\"background:#F1FFFF\"><font color=\"black\" size=\"5\">\"
//      + setOfi+ (ctj.containsKey(setOfi)?split+ ctj.get(setOfi)+ splite:\"")
//      + "</font></span>");
//continue Here;
//}
//389
//      //get path
//      FileDialog filedialog= new FileDialog(new Frame(), "选择要阅读的医学书籍", FileDialog.LOAD);
//      filedialog.setFilenameFilter(new TXTFilter("txt"));
//      filedialog.setVisible(true);
//      String filepath= filedialog.getDirectory()+ filedialog.getFile();
//      //20230106-S_logger.Log.logger.info("" + filepath);
//      if(filepath.endsWith(".doc")|| filepath.endsWith(".docx")
//      || filepath.endsWith(".txt")|| filepath.endsWith(".pdf")) {
//          try {
//              Runtime.getRuntime().exec("rundll32 url.dll, FileProtocolHandler "+filepath);
//          } catch (IOException e) {
//              //      e.printStackTrace();
//          }
//      }
//96
//      this.ratioMap = this.ratioMap_E;
//      this.emotionSample = NE.app_S.emotionSample;
//      this.sensingTest = NE.app_S.sensingTest;
//      this.sensingMap_E = NE.app_S.sensingMap_E;
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane_X_OS.java
/*
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
```



```

package M_V.ME.APM.VSQ.editPane;

import DSU.V.VtoV;

import E_A.OSI.AOP.PCS.PP.port_E.RestNLPPortImpl;

import ME.VPC.M.app.App;

import ME.VPC.V.appButton.AppButtonUtil_X_DNN;

import M_V.MSU.AMS.VQS.SQV.SI.OSU.SMV.http.RestCall_X_backEndRequest;

import P.image.Color_P;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;

import S_A.SVQ.stable.S_Maps;

import S_A.SVQ.stable.S_Pos;

import S_A.pheromone.IMV_SQI_SS;

import U_V.ESU.string.String_ESU_X_charsetSwap;

import U_V.ESU.string.String_ESU_X_stringToURIencode;

import U_V.ESU.string.String_ESU_X_uRIencodeToURIdecode;

import javax.swing.JTextPane;

import java.util.Iterator;

import java.util.List;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class EditPane_X_OS extends EditPane_X_ {

    public EditPane_X_OS(JTextPane text, App app_NE) {
        super(text, app_NE);
        NE = app_NE;
        // TODO Auto-generated constructor stub
    }

    public String sectionDNN(List<String> setsForGet, Iterator<String> iterator
        , String setOfi) {
        setsForGet = _A.parserMixedString(text.getText().trim());
        iterator = setsForGet.iterator();
        if (text.getSelectedText() != null
            && !text.getSelectedText().trim().isEmpty()) {
            iterator = setsForGet.iterator();
        }
        text.setSelectionStart(0);
        text.setSelectionEnd(0);
        int times = 0;
        while (times++ < 1000 && iterator.hasNext()) {
            setOfi += iterator.next();

```

```

    }

    String response = "";

    String keyCache = setOfi.length() > 30 ? setOfi.substring(0, 30)
        : setOfi.substring(0, setOfi.length() - 1);

    if (!NE.app_S.CacheString.containsKey(keyCache)) {
        String string = String_ESU_X_charsetSwap._E(setOfi
            , "UTF8", "UTF8");

        String encode = String_ESU_X_stringToURLEncoder._E(string, "UTF8");
        response = RestCall_X_backEndRequest._E(encode);
        String[] strings = response.split("\\");
        response = strings.length > 3 ? strings[3] : "";
        response = String_ESU_X_uRIencodeToURIdecode._E(response, "UTF8");
        if (NE.app_S.CacheStringPiple.size() < 301) { //buman
            NE.app_S.CacheString.put(keyCache, response);
            NE.app_S.CacheStringPiple.add(0, keyCache);
        } else { //man
            NE.app_S.CacheString.put(keyCache, response);
            NE.app_S.CacheStringPiple.add(0, keyCache);
            NE.app_S.CacheString.remove(
                NE.app_S.CacheStringPiple.get(300));
            NE.app_S.CacheStringPiple.remove(300);
        }
    } else {
        response = NE.app_S.CacheString.getString(keyCache);
    }

    return response;
}

public String sectionLocalDNN(List<String> setsForGet, Iterator<String> iterator
    , String setOfi) {
    setsForGet = _A.parserMixedString(text.getText().trim());
    iterator = setsForGet.iterator();

    if (text.getSelectedText() != null && !text.getSelectedText().trim().isEmpty()) {
        iterator = setsForGet.iterator();
    }

    text.setSelectionStart(0);
    text.setSelectionEnd(0);

    int times = 0;
    while (times++ < 1000 && iterator.hasNext()) {
        setOfi += iterator.next();
    }

    String response = "";

    String keyCache = setOfi.length() > 30 ? setOfi.substring(0, 30)
        : setOfi.substring(0, setOfi.length() - 1);

    if (NE.app_S.appConfig.SectionJPanel.jlabel_peizhi_di2515.isSelected()) {
        if (NE.app_S.CacheString.containsKey(keyCache)) {
            response = NE.app_S.CacheString.getString(keyCache);
        } else {

```

```

String string = String_ESU_X_charsetSwap._E(setOfi, "UTF8", "UTF8");
String encode = String_ESU_X_stringToURIencode._E(string, "UTF8");
//response= RestCall_X_backEndRequest._E(encode);
response = VtoV.ObjectToJsonString(RestNLPPortImpl.dataCG(encode, NE));
String[] strings = response.split("\\");
response = strings.length > 3 ? strings[3] : "";
response = String_ESU_X_uRIencodeToURIdecode._E(response, "UTF8");
if (NE.app_S.CacheStringPiple.size() < 301) { //human
    NE.app_S.CacheString.put(keyCache, response);
    NE.app_S.CacheStringPiple.add(0, keyCache);
} else { //man
    NE.app_S.CacheString.put(keyCache, response);
    NE.app_S.CacheStringPiple.add(0, keyCache);
    NE.app_S.CacheString.remove(NE.app_S.CacheStringPiple.get(300));
    NE.app_S.CacheStringPiple.remove(300);
}
}
} else { //shaohousuojin
String string = String_ESU_X_charsetSwap._E(setOfi
, "UTF8", "UTF8");
//String encode= String_ESU_X_stringToURIencode._E(string, "UTF8");
//response= RestCall_X_backEndRequest._E(encode);
response = VtoV.ObjectToJsonString(RestNLPPortImpl.dataCG(string, NE));
String[] strings = response.split("\\");
response = strings.length > 3 ? strings[3] : "";
response = String_ESU_X_uRIencodeToURIdecode._E(response, "UTF8");
if (NE.app_S.CacheStringPiple.size() < 301) { //human
    NE.app_S.CacheString.put(keyCache, response);
    NE.app_S.CacheStringPiple.add(0, keyCache);
} else { //man
    NE.app_S.CacheString.put(keyCache, response);
    NE.app_S.CacheStringPiple.add(0, keyCache);
    NE.app_S.CacheString.remove(
        NE.app_S.CacheStringPiple.get(300));
    NE.app_S.CacheStringPiple.remove(300);
}
}
}
return response;
}

public void showDataTagDNN(Iterator<String> iterator, List<String> setsForGet
, String setOfi, StringBuilder page, String response
, Map<String, Double> dNNrank) {
IMV_SQL_SS map = new IMV_SQL_SS();
iterator = setsForGet.iterator();
NE.app_S.listCharPosition = 0;
Here:
while (iterator.hasNext()) {

```

```

NE.app_S.listCharPosition++;
setOfi = iterator.next();
if (pos.get(setOfi) == null) {
    page.append("<span style=\"background:#F1F1F1\">"
        + "<font color=\"black\">"
        + " size=\"" + NE.app_S.size_font_rot + "\">"
        + setOfi + "</font></span>");
    continue Here;
}
if ((S_Maps.mingCi.containsKey(setOfi) || S_Maps.dongCi.containsKey(setOfi)
    || S_Maps.xingRongCi.containsKey(setOfi))) {
    if (map.containsKey(setOfi)) {
        WordFrequency wordFrequency = map.getW(setOfi);
        wordFrequency.I_frequency(wordFrequency.get_frequency() + S_Pos.INT_ONE);
        wordFrequency.positions.add(NE.app_S.listCharPosition);
        map.put(setOfi, wordFrequency);
    } else {
        WordFrequency wordFrequency = new WordFrequency(1.0, setOfi);
        wordFrequency.positions.add(NE.app_S.listCharPosition);
        map.put(setOfi, wordFrequency);
    }
}
if (!setOfi.equals("")) {
    if (response.contains(setOfi) && setOfi.length() > 1) {
        if (AppButtonUtil_X_DNN.doResponseContainsSetofl(page
            , setOfi, dNNrank, NE)) {
            continue Here;
        }
    }
    ;
}
if (key.contains(setOfi) && (S_Maps.mingCi.containsKey(setOfi)
    || S_Maps.dongCi.containsKey(setOfi)
    || S_Maps.xingRongCi.containsKey(setOfi))) {
    page.append("<span style=\"background:red\">"
        + "<font color=\"white\">"
        + setOfi + "</font></span>");
    continue Here;
}
if (S_Maps.mingCi.containsKey(setOfi)) {
    page.append("<span style=\"background:"
        + Color_P.P(255, 245, 255)
        + "\"><font color=\"black\">"
        + " size=\"" + NE.app_S.size_font_rot + "\">"
        + setOfi + "</font></span>");
    continue Here;
}
if (S_Maps.dongCi.containsKey(setOfi)) {

```

```

        page.append("<span style=\"background:"
            + Color_P.P(245, 255, 245)
            + "\"><font color=\"black\" \"
            + \" size=\"\" + NE.app_S.size_font_rot + "\">"
            + setOfi + "</font></span>");
        continue Here;
    }

    if (S_Maps.xingRongCi.containsKey(setOfi)) {
        page.append("<span style=\"background:"
            + Color_P.P(255, 255, 245)
            + "\"><font color=\"black\" \"
            + \" size=\"\" + NE.app_S.size_font_rot + "\">"
            + setOfi + "</font></span>");
        continue Here;
    }

    if (S_Maps.fuCi.containsKey(setOfi)) {
        page.append("<span style=\"background:#F1FFFF\">"
            + "<font color=\"black\" \"
            + \" size=\"\" + NE.app_S.size_font_rot + "\">"
            + setOfi + "</font></span>");
        continue Here;
    }

    page.append("<span style=\"background:white\">"
        + "<font color=\"black\" \"
        + \" size=\"\" + NE.app_S.size_font_rot + "\">"
        + setOfi
        + "</font></span>");
    }
}

data.setText(page.toString());
data.setSelectionStart(0);
data.setSelectionEnd(0);
data.validate();
}
}

//Map<String, WordFrequency> map= new IMV_SQI();
//iterator= setsForGet.iterator();
//Here:
//    while(iterator.hasNext()) {
//setOfi= iterator.next();
//if(pos.get(setOfi)== null) {
//    page.append("<span style=\"background:#F1F1F1\">"
//        + "<font color=\"black\" size=\"5\">"
//        + setOfi+ "</font></span>");
//    continue Here;
//}
//if((pos.get(setOfi).contains("名")) || pos.get(setOfi).contains("劲"))

```

```

//          || pos.get(setOfi).contains("形")) {
//      if (map.containsKey(setOfi)) {
//          WordFrequency wordFrequency= map.get(setOfi);
//          wordFrequency.I_frequency(wordFrequency.get_frequency()
//      + S_Pos.INT_ONE);
//          map.put(setOfi, wordFrequency);
//      } else {
//          WordFrequency wordFrequency= new WordFrequency();
//          wordFrequency.I_frequency(S_Pos.INT_ONE);
//          wordFrequency.I_Word(setOfi);
//          map.put(setOfi, wordFrequency);
//      }
//  }
//}
//if (!setOfi.equals("")) {
//    if(response.contains(setOfi)&& setOfi.length()> 1) {
//        if(AppButtonUtil_X_DNN.doResponseContainsSetofl(NE.app_S
//    , pos, page, setOfi, dNNrank)) {
//            continue Here;
//        };
//    }
//    if(key.contains(setOfi)&&(pos.get(setOfi).contains("名")
//        || pos.get(setOfi).contains("动")
//        || pos.get(setOfi).contains("形"))) {
//        page.append("<span style=\"background:red\">"
//    + "<font color=\"white\">"
//    + setOfi+ "</font></span>");
//        continue Here;
//    }
//    if(pos.get(setOfi).contains("名")) {
//        page.append("<span style=\"background:"
//    + U_A.PEU.P.image.Color_P.P(255, 245, 255)
//    + "\"><font color=\"black\" size=\"5\">"
//    + setOfi+ "</font></span>");
//        continue Here;
//    }
//    if(pos.get(setOfi).contains("动")) {
//        page.append("<span style=\"background:"
//    + U_A.PEU.P.image.Color_P.P(245, 255, 245)
//    + "\"><font color=\"black\" size=\"5\">"
//    + setOfi+ "</font></span>");
//        continue Here;
//    }
//    if(pos.get(setOfi).contains("形")) {
//        page.append("<span style=\"background:"
//    + U_A.PEU.P.image.Color_P.P(255, 255, 245)
//    + "\"><font color=\"black\" size=\"5\">"
//    + setOfi+ "</font></span>");

```

```
//
        continue Here;
//
    }
//
    if(pos.get(setOfi).contains("副")) {
//
        page.append("<span style=\"background:#F1FFFF\">"
//
        + "<font color=\"black\" size=\"5\">"
//
        + setOfi+ "</font></span>");
//
        continue Here;
//
    }
//
    page.append("<span style=\"background:white\">"
//
        + "<font color=\"black\" size=\"5\">"+ setOfi
//
        + "</font></span>");
//}
//
    }
//data.setText(page.toString());
//data.setSelectionStart(0);
//data.setSelectionEnd(0);
//data.validate();
//156 //
        WordFrequency wordFrequency = new WordFrequency();
////
        wordFrequency.I_frequency(1.0);
////
        wordFrequency.I_Word(setOfi);
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane_X_OSTest.java
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane_X_Test.java
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/EditPane.java
package M_V.ME.APM.VSQ.editPane;

import E_A.OSI.AOP.PCS.PP.port_E.RestNLPPortImpl;

import ME.VPC.M.app.App;

import ME.VPC.V.appButton.AppButtonUtil_X_DNN;

import M_V.MVQ.button.DetaButton;

import S_A.pheromone.IMV_SQI;

import javax.swing.JSlider;

import javax.swing.JTextPane;

import javax.swing.event.ChangeEvent;

import javax.swing.event.ChangeListener;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.io.IOException;

import java.net.URLDecoder;

import java.util.Iterator;

import java.util.List;

import java.util.Map;

/*
 * 个人著作权人,作者 罗瑶光.浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
```

```
public class EditPane extends EditPane_X_OS {

    public EditPane(JTextPane text, App app_NE) {
        super(text, app_NE); //shaohou duplication
        NE = app_NE;
        I_DNNButton();
        I_DNNButtonSort();
        I_DNNButtonWest();
        I_DNNButtonChinese();
        I_DNNSlider();
        I_Literature();
        I_MindReader();
    }

    //dataCY
    private void I_MindReader() {
        DetaButton button = new DetaButton("词义分析");
        button.setBounds(635, 33, 100, 20);
        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent arg0) {
                if (text.getText().equalsIgnoreCase("")) {
                    return;
                }
                try {
                    IMV_SQI output = RestNLPPortImpl
                        .dataCY(text.getText().trim(), NE);
                    String cy = output.get("cy").toString().trim();
                    cy = URLDecoder.decode(cy, "UTF-8");
                    cy = cy.replace("\r\n", "<br/> ");
                    cy = "<html>" + cy + "</html>";
                    data.setText(cy);
                    data.setSelectionStart(0);
                    data.setSelectionEnd(0);
                    data.validate();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        });
        this.add(button);
    }

    //dataZF
    private void I_Literature() {
        DetaButton button = new DetaButton("作文辅导");
        button.setBounds(740, 33, 100, 20);
        button.addActionListener(new ActionListener() {
            @Override
```



```
public void actionPerformed(ActionEvent arg0) {
    if (text.getText().equalsIgnoreCase("")) {
        return;
    }
    try {
        @SuppressWarnings("unchecked")
        IMV_SQI output = (IMV_SQI) RestNLPPortImpl
            .dataZF(text.getText().trim(), NE);
        String zf = output.get("zf").toString().trim();
        zf = URLDecoder.decode(zf, "UTF-8");
        zf = zf.replace("\r\n", "<br/> ");
        zf = "<html>" + zf + "</html>";
        data.setText(zf);
        data.setSelectionStart(0);
        data.setSelectionEnd(0);
        data.validate();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

});
this.add(button);
}

private void I_DNNButtonSort() {
    DetaButton button = new DetaButton("DNN重心");
    button.setBounds(5 + 105 * 1, 3 + 30, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            List<String> setsForGet = _A.parserMixedString(text.getText().trim());
            Iterator<String> iterator = setsForGet.iterator();
            String setOfi = new String();
            String response = null;
            response = sectionLocalDNN(setsForGet, iterator, setOfi);
            //稍后新button 罗瑶光20230905
            //dnn rank
            JTextPane statistic = new JTextPane();
            Map<String, Double> dNNrank = new IMV_SQI();
            AppButtonUtil_X_DNN.doRankDNN(statistic, response, pos, dNNrank, NE);
            //String temp= statistic.getText();
            data.setText(statistic.getText());
            data.setSelectionStart(0);
            data.setSelectionEnd(0);
            data.validate();
        }
    });
}
```

```
}
});
this.add(button);
}

private void I_DNNButtonWest() {
    DetaButton button = new DetaButton("DNN西药");
    button.setBounds(5 + 105 * 2, 3 + 30, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            //
            StringBuilder page = new StringBuilder().append("");
            List<String> setsForGet = _A.parserMixedString(text.getText().trim());
            Iterator<String> iterator = setsForGet.iterator();
            String setOfi = new String();
            //String response= sectionDNN(setsForGet, iterator, setOfi);
            String response = null;
            response = sectionLocalDNN(setsForGet, iterator, setOfi);
            //dnn森林//稍后新button 罗瑶光20230905
            try {
                String[] dnn = response.replace("\r\n", "<br/>")
                    .split("<br/>");
                NE.app_S.coAuthorForWord.bootDetaBingMingDnnFlowerForest(
                    "西药", dnn, true);
            } catch (Exception e1) {
                NE.validate();
            }
            //稍后新button 罗瑶光20230905
            //                //dnn rank
            JTextPane statistic = new JTextPane();
            Map<String, Double> dNNrank = new IMV_SQI();
            AppButtonUtil_X_DNN.doRankDNN(statistic, response, pos, dNNrank, NE);
            //DNN
            showDataTagDNN(iterator, setsForGet, setOfi, page, response, dNNrank);
        }
    });
    this.add(button);
}

private void I_DNNButtonChinese() {
    DetaButton button = new DetaButton("DNN中药");
    button.setBounds(5 + 105 * 3, 3 + 30, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
```

```
        if (text.getText().equalsIgnoreCase("")) {
            return;
        }
        //
        StringBuilder page = new StringBuilder().append("");
        List<String> setsForGet = _A.parserMixedString(text.getText().trim());
        Iterator<String> iterator = setsForGet.iterator();
        String setOfi = new String();
        String response = null;
        response = sectionLocalDNN(setsForGet, iterator, setOfi);
        //dnn森林//稍后新button 罗瑶光20230905
        String[] dnn = response.replace("\r\n", "<br/>").split("<br/>");
        NE.app_S.coAuthorForWord.bootDetaDnnFlowerForest("中药", dnn, true);
        //稍后新button 罗瑶光20230905
        //                //dnn rank
        JTextPane statistic = new JTextPane();
        Map<String, Double> dNNrank = new IMV_SQI();
        AppButtonUtil_X_DNN.doRankDNN(statistic, response, pos, dNNrank, NE);
        //DNN
        showDataTagDNN(iterator, setsForGet, setOfi, page, response, dNNrank);
    }
});
this.add(button);
}
//替换下准备
private void I_DNNButton() {
    DetaButton button = new DetaButton("DNN");
    button.setBounds(5, 3 + 30, 100, 20);
    button.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent arg0) {
            if (text.getText().equalsIgnoreCase("")) {
                return;
            }
            //
            StringBuilder page = new StringBuilder().append("");
            List<String> setsForGet = _A.parserMixedString(text.getText().trim());
            Iterator<String> iterator = setsForGet.iterator();
            String setOfi = new String();
            String response = null;
            response = sectionLocalDNN(setsForGet, iterator, setOfi);
            //稍后新button 罗瑶光20230905
            //                //dnn rank
            JTextPane statistic = new JTextPane();
            Map<String, Double> dNNrank = new IMV_SQI();
            AppButtonUtil_X_DNN.doRankDNN(statistic, response, pos, dNNrank, NE);
            //DNN
```

```

        showDataTagDNN(iterator, setsForGet, setOfi, page, response, dNNrank);
    }
});
this.add(button);
}

private void I_DNNSlider() {
    JSlider sliderDNN = new JSlider(0, 25);
    sliderDNN.setBounds(5 + 105 + 105 + 105 + 105, 3 + 30, 205, 20);
    sliderDNN.setSnapToTicks(true);
    sliderDNN.setPaintTicks(true);
    sliderDNN.setMajorTickSpacing(5);
    sliderDNN.setMinorTickSpacing(1);
    sliderDNN.addChangeListener(
        new ChangeListener() {
            public void stateChanged(ChangeEvent event) {
                JSlider source = (JSlider) event.getSource();
                NE.app_S.jsliderDNNRatio = (double) source.getValue();
                if (text.getText().equalsIgnoreCase("")) {
                    return;
                }
                //
                StringBuilder page = new StringBuilder().append("");
                List<String> setsForGet = _A.parserMixedString(text.getText().trim());
                Iterator<String> iterator = setsForGet.iterator();
                String setOfi = new String();
                String response = null;
                response = sectionLocalDNN(setsForGet, iterator, setOfi);
                //稍后新button 罗瑶光20230905
                //dnn rank
                JTextPane statistic = new JTextPane();
                Map<String, Double> dNNrank = new IMV_SQI();
                AppButtonUtil_X_DNN.doRankDNN(statistic, response, pos, dNNrank, NE);
                //DNN
                showDataTagDNN(iterator, setsForGet, setOfi, page, response, dNNrank);
            }
        });
    this.add(sliderDNN);
}

//Map<String, WordFrequency> map= new IMV_SQI();
//iterator= setsForGet.iterator();
//Here:
//    while(iterator.hasNext()) {
//setOfi= iterator.next();
//if(pos.get(setOfi)== null) {
//    page.append("<span style=\"background:#F1F1F1\">"
//        + "<font color=\"black\" size=\"5\">"

```

```
//          + setOfi+ "</font></span>");
//      continue Here;
//}
//if((pos.get(setOfi).contains("名") || pos.get(setOfi).contains("动")
//      || pos.get(setOfi).contains("形"))) {
//      if (map.containsKey(setOfi)) {
//          WordFrequency wordFrequency= map.get(setOfi);
//          wordFrequency.I_frequency(wordFrequency.get_frequency()
//      + S_Pos.INT_ONE);
//          map.put(setOfi, wordFrequency);
//      } else {
//          WordFrequency wordFrequency= new WordFrequency();
//          wordFrequency.I_frequency(S_Pos.INT_ONE);
//          wordFrequency.I_Word(setOfi);
//          map.put(setOfi, wordFrequency);
//      }
//}
//if (!setOfi.equals("")) {
//      if(response.contains(setOfi)&& setOfi.length()> 1) {
//          if(AppButtonUtil_X_DNN.doResponseContainsSetofl(NE.app_S
//      , pos, page, setOfi, dNNrank)) {
//              continue Here;
//          };
//      }
//      if(key.contains(setOfi)&&(pos.get(setOfi).contains("名")
//          || pos.get(setOfi).contains("动")
//          || pos.get(setOfi).contains("形"))) {
//          page.append("<span style=\"background:red\">"
//      + "<font color=\"white\">"
//      + setOfi+ "</font></span>");
//          continue Here;
//      }
//      if(pos.get(setOfi).contains("名")) {
//          page.append("<span style=\"background:"
//      + U_A.PEU.P.image.Color_P.P(255, 245, 255)
//      + "\"><font color=\"black\" size=\"5\">"
//      + setOfi+ "</font></span>");
//          continue Here;
//      }
//      if(pos.get(setOfi).contains("动")) {
//          page.append("<span style=\"background:"
//      + U_A.PEU.P.image.Color_P.P(245, 255, 245)
//      + "\"><font color=\"black\" size=\"5\">"
//      + setOfi+ "</font></span>");
//          continue Here;
//      }
//      if(pos.get(setOfi).contains("形")) {
```

```
//          page.append("<span style=\"background:\""
//      + U_A.PEU.P.image.Color_P.P(255, 255, 245)
//      + "\"><font color=\"black\" size=\"5\">"
//      + setOfi+ "</font></span>");
//          continue Here;
//      }
//      if(pos.get(setOfi).contains("副")) {
//          page.append("<span style=\"background:#F1FFFF\">"
//      + "<font color=\"black\" size=\"5\">"
//      + setOfi+ "</font></span>");
//          continue Here;
//      }
//      page.append("<span style=\"background:white\">"
//      + "<font color=\"black\" size=\"5\">"+ setOfi
//      + "</font></span>");
//}
//      }
//data.setText(page.toString());
//data.setSelectionStart(0);
//data.setSelectionEnd(0);
//data.validate();
** /Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/PrintTest4.java
package M_V.ME.APM.VSQ.editPane;

import java.awt.BasicStroke;

import java.awt.Color;

import java.awt.Component;

import java.awt.Font;

import java.awt.Graphics;

import java.awt.Graphics2D;

import java.awt.Image;

import java.awt.Toolkit;

import java.awt.print.Book;

import java.awt.print.PageFormat;

import java.awt.print.Paper;

import java.awt.print.Printable;

import java.awt.print.PrinterException;

import java.awt.print.PrinterJob;

import java.util.List;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class PrintTest4 implements Printable {
```

[illegible]

```

g2.setColor(Color.black);
//打印起点坐标
int x = (int) pageFormat.getImageableX();
int y = (int) pageFormat.getImageableY();
switch (pageIndex) {
    case 0:
        Font font = new Font("新宋体", Font.PLAIN, 16);
        g2.setFont(font);//设置字体
        float[] dash1 = {2.0f};
        //设置打印线的属性。
        //1.线宽 2、3、不知道, 4、空白的宽度, 5、虚线的宽度, 6、偏移量
        g2.setStroke(new BasicStroke(0.5f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER
            , 2.0f, dash1, 0.0f));
        //g2.setStroke(bs_3);//设置线宽
        float heigth = font.getSize2D();//字体高度
        //20230106-S_logger.Log.logger.info("" + ">>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>x="+x);
        Image src = Toolkit.getDefaultToolkit().getImage("D:/背景图片/yesorno.jpg");
        g2.drawImage(src, x, y, c);
        int img_Height = src.getHeight(c);
        g2.drawImage(src, x, (int) (y + 1 * heigth + img_Height + 11), c);
        //String[] lines= str.split("\r\n\r\n");
        int countOut = 0;
        for (int i = 0; i < prePrint.length; i++) {
            String lineOfStrings = prePrint[i].toString()
                .replace(" ", "").replaceAll(";", "+", ";");
            g2.drawString(lineOfStrings
                , x + 30,
                y + ((countOut++) + 1) * heigth + img_Height);
        }
        return PAGE_EXISTS;
    default:
        return NO_SUCH_PAGE;
}
}

public static void main(String[] args) {
    //new PrintTest4("怎么办? ");
}
}

/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/editPane/PrintTest4Test.java

M_V.ME.APM.VSQ.ESI.SVQ.search

/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/ESI/SVQ/search/CaiJPanel.java

package M_V.ME.APM.VSQ.ESI.SVQ.search;

import javax.swing.JLabel;

import javax.swing.JPanel;

import javax.swing.JTextField;

import java.awt.Color;
```



```
/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguanguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class CaiJPanel extends JPanel {
    /**
     *
     */
    public CaiJPanel(JTextField name) {
        this.setLayout(null);
        this.setBounds(0, 0, 780, 610);
        this.setBackground(Color.BLACK);
    }

    ///copy tab
    JLabel chineseVerbal = new JLabel("chineseVerbal: 之后这里横向扩展一个应用面 @^_^@ ");
    chineseVerbal.setForeground(Color.orange);
    chineseVerbal.setBounds(10, 5, 400, 50);
    this.add(chineseVerbal);
    JLabel chinesePinYin = new JLabel("chinesePinYin: 之后这里横向扩展一个应用面 @^_^@ ");
    chinesePinYin.setForeground(Color.WHITE);
    chinesePinYin.setBounds(10, 5 + 25, 400, 50);
    this.add(chinesePinYin);
    JLabel chineseExplain = new JLabel("chineseExplain: 之后这里横向扩展一个应用面 @^_^@ ");
    chineseExplain.setForeground(Color.WHITE);
    chineseExplain.setBounds(10, 5 + 50, 400, 50);
    this.add(chineseExplain);
    JLabel verbalExplain = new JLabel("verbalExplain: 之后这里横向扩展一个应用面 @^_^@ ");
    verbalExplain.setForeground(Color.WHITE);
    verbalExplain.setBounds(10, 5 + 75, 400, 50);
    this.add(verbalExplain);
    //jiuzheng later
    JLabel verbalSynorine = new JLabel("verbalSynorine: 之后这里横向扩展一个应用面 @^_^@ ");
    verbalSynorine.setForeground(Color.WHITE);
    verbalSynorine.setBounds(10, 5 + 100, 400, 50);
    this.add(verbalSynorine);
    JLabel verbalAsynorine = new JLabel("verbalAsynorine: 之后这里横向扩展一个应用面 @^_^@ ");
    verbalAsynorine.setForeground(Color.WHITE);
    verbalAsynorine.setBounds(10, 5 + 125, 400, 50);
    this.add(verbalAsynorine);
}
}
```

M_V.ME.APM.VSQ.xlsx

/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlsx/CoAuthorForWord_A.java

```
package M_V.ME.APM.VSQ.xlx;

import java.awt.event.KeyEvent;
import java.awt.event.MouseEvent;
import java.awt.event.MouseWheelEvent;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CoAuthorForWord_A extends CoAuthorForWord_X_Slider {

    public void mouseDragged(java.awt.event.MouseEvent arg0) {

        int registX = arg0.getX();

        if (registX > mousex) {

            zrot += (arg0.getX() + 10) / 70;

        } else {

            zrot -= (arg0.getX() + 10) / 70;

        }

        mousex = registX;

    }

    public void mouseMoved(java.awt.event.MouseEvent arg0) {

    }

    public void keyPressed(KeyEvent arg0) {

        if (arg0.getKeyCode() == 38) {

            brot += 0.5;

        }

        if (arg0.getKeyCode() == 37) {

            lrot -= 0.5;

        }

        if (arg0.getKeyCode() == 40) {

            brot -= 0.5;

        }

        if (arg0.getKeyCode() == 39) {

            lrot += 0.5;

        }

    }

    public void keyReleased(KeyEvent arg0) {

    }

    public void keyTyped(KeyEvent arg0) {

    }

    public void mouseWheelMoved(MouseWheelEvent arg0) {

        trot += arg0.getWheelRotation() * 2;

    }

    public void mouseClicked(MouseEvent arg0) {
```

```

}

public void mouseEntered(MouseEvent arg0) {
    clicked = false;
}

public void mouseExited(MouseEvent arg0) {
    clicked = false;
}

public void mousePressed(MouseEvent arg0) {
    mousex = arg0.getX();
    mousey = arg0.getY();
}

public void mouseReleased(MouseEvent arg0) {
}
}

//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlsx/CoAuthorForWord_ATest.java
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlsx/CoAuthorForWord_Q.java
package M_V.ME.APM.VSQ.xlsx;

import ME.VPC.M.app.App;

import S_A.pheromone.IMV_SQI;

import S_A.pheromone.IMV_SQI_SS;

import com.jogamp.opengl.GL;

import com.jogamp.opengl.GL2;

import com.jogamp.opengl.GLAutoDrawable;

import java.util.Iterator;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CoAuthorForWord_Q extends CoAuthorForWord_V {

    public void drawEngine(GLAutoDrawable drawable) {
        gl = drawable.getGL().getGL2();
        gl.glClear(GL.GL_COLOR_BUFFER_BIT | GL.GL_DEPTH_BUFFER_BIT);
        gl.glLoadIdentity();
        gl.glTranslatef(1rot, brot, -36.0f);
        gl.glRotatef(xrot, 1.0f, 0.0f, 0.0f);
        gl.glRotatef(yrot, 0.0f, 1.0f, 0.0f);
        gl.glRotatef(zrot, 0.0f, 0.0f, 1.0f); //稍后~
        gl.glPointSize(prot);
        gl.glOrtho(0, trot, 0, trot, 1, -1);
        gl.glClear(GL.GL_COLOR_BUFFER_BIT | GL.GL_DEPTH_BUFFER_BIT);
        gl.glBegin(GL.GL_TRIANGLES);

```

```
deleteALatest(gl);
drawANew(gl);
gl.glEnd();
gl.glFlush();
}

public void drawANew(GL2 gl) {
    float i = 1.0f;
    Iterator<String> iterators = rootClassMap.keySet().iterator();
    while (iterators.hasNext()) {
        String string = iterators.next();
        CommonRotClass commonRotClass = rootClassMap.getCommonRotClass(string);
        if (commonRotClass.buttonClicked) {
            rootLeaf = commonRotClass.rootLeaf;
            float rotRatio = commonRotClass.rot;
            float rotRatioZ = commonRotClass.rotZ;
            displayWords(gl, i, 1, 1, true
                , rotRatio, rotRatioZ, ((i * 2) % 6f); //稍后整合五行
            i += 0.5f;
        }
    }
}

//先长度，再弧度，最后错开，
public void deleteALatest(GL2 gl) {
    float i = 1.0f;
    boolean firstTime = true;
    Iterator<String> iterators = rootClassMap.keySet().iterator();
    while (iterators.hasNext()) {
        String string = iterators.next();
        CommonRotClass commonRotClass = rootClassMap.getCommonRotClass(string);
        if (commonRotClass.buttonClicked) {
            if (firstTime || (commonRotClass.rot != commonRotClass.rotl)) {
                firstTime = false;
                rootLeaf = commonRotClass.rootLeaf;
                float rotRatio = commonRotClass.rotl;
                float rotRatioZ = commonRotClass.rotZl;
                displayWords(gl, i, 1, 1, false
                    , rotRatio, rotRatioZ, (i * 2) % 6f);
                rotRatio = commonRotClass.rot;
                rotRatioZ = commonRotClass.rotZ;
                displayWords(gl, i, 1, 1, true
                    , rotRatio, rotRatioZ, (i * 2) % 6f); //稍后整合五行
                //更新
                commonRotClass.rotl = commonRotClass.rot;
                commonRotClass.rotZl = commonRotClass.rotZ;
            }
            i += 0.5f;
        }
    }
}
```

```
        rootClassMap.put(string, commonRotClass);
    }
}

//稍后移出去~

public void displayWords(GL2 gl, float length, float arc, float flop, boolean back
    , float rotRatio, float rotRatioZ, float colorType) {
    int section[] = new int[21];
    section[0] = 32;
    section[1] = 32;
    section[2] = 32;
    section[3] = 32;
    section[4] = 10;
    section[5] = 10;
    section[6] = 10;
    section[7] = 10;
    section[8] = 10;
    section[9] = 10;
    section[10] = 10;
    section[11] = 10;
    float a = 10;
    double root[] = new double[3];
    for(int regi=0;regi<root.length;regi++){
        root[regi]=0.0;
    }
    didLeaf.clear();
    if (back) {
        drawWords(gl, section, 0, root, a, 0, 0, this.rootLeaf, didLeaf
            , length, rotRatio, rotRatioZ, (int) colorType, true);
    } else {
        drawWords(gl, section, 0, root, a, 0, 0, this.rootLeaf, didLeaf
            , length, rotRatio, rotRatioZ, (int) colorType, false);
    }
}

//稍后去重

public void display(GLAutoDrawable drawable) {
    drawEngine(drawable);
}

public void reshape(GLAutoDrawable drawable, int x, int y, int width, int height) {
    float fAspect;
    if (height == 0) {
        height = 1;
    }
    gl.glViewport(0, 0, width, height);
    fAspect = (float) width / height;
    gl.glMatrixMode(GL2.GL_PROJECTION);
    gl.glLoadIdentity();
    glu.gluPerspective(53.0f, fAspect, 1.0, 400.0);
```

```

gl.glMatrixMode(GL2.GL_MODELVIEW);

gl.glLoadIdentity();

}

public void dispose(GLAutoDrawable arg0) {

}

@SuppressWarnings("unused")

private IMV_SQI kernerForest(String temp, IMV_SQI root
    , Map<String, Integer> findLeaf, int depth, App NE) {
    IMV_SQI leaf = new IMV_SQI();
    if (++depth > 6) {
        return leaf;
    }
    int ml = 0;
    if (u.app_S.dic_li.containsKey(temp) || u.app_S.dic_yl.containsKey(temp)
        || u.app_S.dic_zf.containsKey(temp)) {
        IMV_SQI_SS map = _A.parserMixStringByReturnFrequencyMap(
            (u.app_S.dic_li.get(temp) == null
                ? "" : u.app_S.dic_li.get(temp))
            + (u.app_S.dic_yl.get(temp) == null
                ? "" : u.app_S.dic_yl.get(temp).toString())
            + (u.app_S.dic_zf.get(temp) == null
                ? "" : u.app_S.dic_zf.get(temp).toString())
            , NE);
        Iterator<String> it = map.keySet().iterator();
        while (it.hasNext()) {
            String tempRec = it.next();
            if (!findLeaf.containsKey(tempRec)) {
                findLeaf.put(tempRec, 1);
                frequencyLeaf.put(tempRec, 1);
                if (u.app_S.dic_map.containsKey(tempRec)) {
                    IMV_SQI subLeaf
                        = kernerForest(tempRec, leaf, findLeaf, depth, NE);
                    leaf.put(tempRec, subLeaf);
                } else if (ml < 8) {
                    IMV_SQI subLeaf
                        = kernerForest(tempRec, leaf, findLeaf, depth, NE);
                    if (subLeaf.size() > 0) {
                        leaf.put(tempRec, subLeaf);
                        ml++;
                    }
                }
            } else {
                frequencyLeaf.put(tempRec, frequencyLeaf.get(tempRec) + 1);
            }
        }
    }
    return leaf;
}

```

```

    }
}

//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_QTest.java
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_V.java
package M_V.ME.APM.VSQ.xlx;

import S_A.SVQ.stable.S_Colors;

import S_A.pheromone.IMV_SQL;

import com.jogamp.opengl.GL;

import com.jogamp.opengl.GL2;

import com.jogamp.opengl.GLAutoDrawable;

import com.jogamp.opengl.util.GLBuffers;

import javax.swing.ImageIcon;

import java.awt.Color;

import java.awt.Font;

import java.awt.Graphics2D;

import java.awt.image.BufferedImage;

import java.nio.ByteBuffer;

import java.util.Iterator;

import java.util.LinkedHashMap;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CoAuthorForWord_V extends CoAuthorForWord_A {

    @SuppressWarnings({"unchecked", "static-access"})

    public void drawWords(GL2 gl, int[] s, int se, double[] root

        , float a, int j, int k, LinkedHashMap<String, Object> root3

        , Map<String, Integer> didLeaf, float length, float rotRatio

        , float rotRatioZ, int colorType, boolean animation) {

        if (root3 == null || root3.size() == 0) {

            return;

        }

        int i = 0;

        a = a * length;

        s[se] = root3.size();

        Iterator<String> iteratorLeaf = root3.keySet().iterator();

        //init

        double mcz = Math.cos(2 * Math.PI * (rotRatioZ / 100));

        double msz = Math.sin(2 * Math.PI * (rotRatioZ / 100));

        Here:

        while (iteratorLeaf.hasNext()) {

            if (k >= irot) {

```

```

    return;
}

k++;

String leafName = iteratorLeaf.next();

if (leafName.length() == 0) {
    continue Here;
}

LinkedHashMap<String, Object> leaf = (LinkedHashMap<String, Object>) root3.get(leafName);

//角坐标叠加
//当xy 固定z 旋转后 将 xz对应y旋转，会倾斜
//稍后优化 区间余弦叠加

double mca = Math.cos(2 * Math.PI * ((i + rotRatio / 100)) / (s[se])) * α;
double msa = Math.sin(2 * Math.PI * ((i + rotRatio / 100)) / (s[se])) * α;
gl.glBegin(gl.GL_LINES);
if (animation) {
    gl.glColor3f(S_Colors.colors[colorType][0]
        , S_Colors.colors[colorType][1]
        , S_Colors.colors[colorType][2]);
} else {
    gl.glColor3f(0, 0, 0);
}
gl.glVertex3d(root[0], root[1], root[2]);
gl.glVertex3d(root[0] + mca * mcz, root[1] + msa
    , root[2] + mca * msz);
gl.glEnd();
gl.glRasterPos3d(root[0] + mca * mcz, root[1] + msa
    , root[2] + mca * msz);
String prePrint = leafName;
Font font = new Font(prePrint, 20, 20);
checkImageBuf = GLBuffers.newDirectByteBuffer(prePrint.length() * 1320);
BufferedImage image = new BufferedImage(prePrint.length() << 5
    , 20, BufferedImage.TYPE_INT_RGB);
Graphics2D ga = image.createGraphics();
//紫色： megenda
ImageIcon bagua_dui = null;
if (animation) {
    ga.setColor(Color.white);
    bagua_dui = getColorTag(leafName, ga);
    DrawImage(bagua_dui);
} else {
    ga.setColor(Color.black);
    bagua_dui = getColorTagBack(leafName, ga);
    DrawImageBack(bagua_dui);
}
gl.glRasterPos3d(root[0] + mca * mcz, root[1] + msa
    , root[2] + mca * msz + 9);
int AMV_MVS_VSQX = 1;

```



```

int AMV_MVS_VSQY = 17;

this.weightPrint(prePrint, ga, AMV_MVS_VSQX, AMV_MVS_VSQY, font);
this.heightPrint(prePrint, ga, AMV_MVS_VSQX, AMV_MVS_VSQY, font);
checkImageBuf.clear();

int h = image.getHeight();
if (animation) {
    for (int i1 = h - 1; i1 >= 0; i1--) {
        for (int j1 = 0; j1 < prePrint.length() * 20; j1++) {
            checkImageBuf.put((byte) (image.getRGB(j1, i1)
                & (byte) 0xFF));
            checkImageBuf.put((byte) (image.getRGB(j1, i1)
                >> 8 & (byte) 0xFF));
            checkImageBuf.put((byte) (image.getRGB(j1, i1)
                >> 16 & (byte) 0xFF));
        }
    }
} else {
    for (int i1 = h - 1; i1 >= 0; i1--) {
        for (int j1 = 0; j1 < prePrint.length() * 20; j1++) {
            checkImageBuf.put((byte) (0 & (byte) 0xFF));
            checkImageBuf.put((byte) (0 >> 8 & (byte) 0xFF));
            checkImageBuf.put((byte) (0 >> 16 & (byte) 0xFF));
        }
    }
}

checkImageBuf.rewind();
gl.glDrawPixels(prePrint.length() * 20, 20, GL2.GL_RGB
    , GL.GL_UNSIGNED_BYTE, checkImageBuf);

double root1[] = new double[3];
root1[0] = root[0] + mca * mcz;
root1[1] = root[1] + msa;
root1[2] = root[2] + mca * msz;
//j= i;
if (!didLeaf.containsKey(leafName) && findLeaf.containsKey(leafName)) {
    didLeaf.put(leafName, 1);
    drawWords(gl, s, se + 1, root1, a * 0.66f, j + i, k
        , leaf, didLeaf, length, rotRatio, rotRatioZ, colorType
        , animation);
}
i++;
}
}

@SuppressWarnings({"unchecked", "static-access"})
public void drawWordsBlack(int[] s, int se, double[] root, float a, int j
    , int k, IMV_SQI root3, Map<String, Integer> didLeaf
    , float length, float rotRatio, float rotRatioZ) {
    if (root3 == null || root3.size() == 0) {

```

```

    return;
}

int i = 0;
s[se] = root3.size();
a = a * length;
Iterator<String> iteratorLeaf = root3.keySet().iterator();
//init
double mcz = Math.cos(2 * Math.PI * (rotRatioZ / 100));
double msz = Math.sin(2 * Math.PI * (rotRatioZ / 100));
Here:
while (iteratorLeaf.hasNext()) {
    if (k >= irot) {
        return;
    }
    k++;
    String leafName = iteratorLeaf.next();
    if (leafName.length() == 0) {
        continue Here;
    }
    IMV_SQI leaf = (IMV_SQI) root3.get(leafName);
    if (leaf.size() > 0) {
    }
    double mca = Math.cos(2 * Math.PI * ((i + rotRatio / 100)) / (s[se])) * a;
    double msa = Math.sin(2 * Math.PI * ((i + rotRatio / 100)) / (s[se])) * a;
    gl.glBegin(gl.GL_LINES);
    gl.glColor3f(0, 0, 0);
    gl.glVertex3d(root[0], root[1], root[2]);
    gl.glVertex3d(root[0] + mca * mcz, root[1] + msa
        , root[2] + mca * msz);
    gl.glEnd();
    gl.glRasterPos3d(root[0] + mca * mcz, root[1] + msa
        , root[2] + mca * msz);
    String prePrint = leafName;
    Font font = new Font(prePrint, 20, 20);
    checkImageBuf = GLBuffers.newDirectByteBuffer(prePrint.length() * 1320);
    BufferedImage image = new BufferedImage(prePrint.length() << 5
        , 20, BufferedImage.TYPE_INT_RGB);
    Graphics2D ga = image.createGraphics();
    //ga.setColor(Color.black);
    //紫色: megenda
    ImageIcon bagua_dui = getColorTagBack(leafName, ga);
    if (null != bagua_dui) {
        DrawImageBack(bagua_dui);
    }
    gl.glRasterPos3d(root[0] + mca * mcz, root[1] + msa
        , root[2] + mca * msz + 9);
    int AMV_MVS_VSQX = 1;

```

```

int AMV_MVS_VSQY = 17;

this.weightPrint(prePrint, ga, AMV_MVS_VSQX, AMV_MVS_VSQY, font);
this.heightPrint(prePrint, ga, AMV_MVS_VSQX, AMV_MVS_VSQY, font);
checkImageBuf.clear();

int h = image.getHeight();
for (int i1 = h - 1; i1 >= 0; i1--) {
    for (int j1 = 0; j1 < prePrint.length() * 20; j1++) {
        checkImageBuf.put((byte) (0 & (byte) 0xFF));
        checkImageBuf.put((byte) (0 >> 8 & (byte) 0xFF));
        checkImageBuf.put((byte) (0 >> 16 & (byte) 0xFF));
    }
}

checkImageBuf.rewind();
gl.glDrawPixels(prePrint.length() * 20, 20, GL2.GL_RGB
    , GL.GL_UNSIGNED_BYTE, checkImageBuf);

double root1[] = new double[3];
root1[0] = root[0] + mca * mcz;
root1[1] = root[1] + msa;
root1[2] = root[2] + mca * msz;
//j= i;
if (!didLeaf.containsKey(leafName) && findLeaf.containsKey(leafName)) {
    didLeaf.put(leafName, 1);
    drawWordsBlack(s, se + 1, root1, a * 0.66f, j + i
        , k, leaf, didLeaf, length, rotRatio, rotRatioZ);
}
i++;
}
}

private ImageIcon getColorTag(String leafName, Graphics2D ga) {
    ImageIcon bagua = null;
    if (leafName.contains("巽")) {
        bagua = bagua_xun;
        ga.setColor(new Color(226, 5, 9));
    } else if (leafName.contains("离")) {
        bagua = bagua_li;
        ga.setColor(new Color(240, 45, 216)); //zi
    } else if (leafName.contains("坤")) {
        bagua = bagua_kun;
        ga.setColor(new Color(0, 185, 185));
    } else if (leafName.contains("兑")) {
        bagua = bagua_dui;
        ga.setColor(new Color(30, 164, 19));
    } else if (leafName.contains("乾")) {
        bagua = bagua_qian;
        ga.setColor(Color.BLUE);
    } else if (leafName.contains("坎")) {
        bagua = bagua_kan;
    }
}

```

```

        ga.setColor(new Color(91, 15, 86));
    } else if (leafName.contains("艮")) {
        bagua = bagua_gen;
        ga.setColor(new Color(0, 255, 255));
    } else if (leafName.contains("震")) {
        bagua = bagua_zhen;
        ga.setColor(new Color(245, 249, 102));
    } else if (leafName.contains("木")) {
        ga.setColor(new Color(226, 5, 9));
    } else if (leafName.contains("火")) {
        ga.setColor(new Color(240, 45, 216));
    } else if (leafName.contains("土")) {
        ga.setColor(new Color(0, 255, 255));
    } else if (leafName.contains("金")) {
        ga.setColor(new Color(30, 164, 19));
    } else if (leafName.contains("水")) {
        ga.setColor(new Color(91, 15, 86));
    }
    return bagua;
}

private ImageIcon getColorTagBack(String leafName, Graphics2D ga) {
    ImageIcon bagua = null;
    if (leafName.contains("巽")) {
        bagua = bagua_xun;
    } else if (leafName.contains("离")) {
        bagua = bagua_li;
    } else if (leafName.contains("坤")) {
        bagua = bagua_kun;
    } else if (leafName.contains("兑")) {
        bagua = bagua_dui;
    } else if (leafName.contains("乾")) {
        bagua = bagua_qian;
    } else if (leafName.contains("坎")) {
        bagua = bagua_kan;
    } else if (leafName.contains("艮")) {
        bagua = bagua_gen;
    } else if (leafName.contains("震")) {
        bagua = bagua_zhen;
    }
    ga.setColor(new Color(0, 0, 0));
    return bagua_dui;
}

private void DrawImage(ImageIcon bagua_dui) {
    if (null == bagua_dui) {
        return;
    }
    BufferedImage bufferedImage = new BufferedImage(bagua_dui.getIconWidth(),

```

```

        bagua_dui.getIconHeight(), BufferedImage.TYPE_INT_RGB);

Graphics2D ga = bufferedImage.createGraphics();
ga.drawImage(bagua_dui.getImage(), 0, 0, bagua_dui.getImageObserver());
ByteBuffer graphImageBuf = GLBuffers.newDirectByteBuffer(
    bufferedImage.getHeight() * (bufferedImage.getWidth() + 10) * 3);
graphImageBuf.clear();
int h = bufferedImage.getHeight();
for (int i1 = h - 1; i1 >= 0; i1--) {
    for (int j1 = 0; j1 < bufferedImage.getWidth(); j1++) {
        if (bufferedImage.getRGB(j1, i1) != 0) {
            graphImageBuf.put((byte) (bufferedImage.getRGB(j1, i1)
                >> 0 & (byte) 0xFF));
            graphImageBuf.put((byte) (bufferedImage.getRGB(j1, i1)
                >> 8 & (byte) 0xFF));
            graphImageBuf.put((byte) (bufferedImage.getRGB(j1, i1)
                >> 16 & (byte) 0xFF));
        }
    }
}
graphImageBuf.rewind();
gl.glDrawPixels(bufferedImage.getWidth(), bufferedImage.getHeight()
    , GL2.GL_RGB, GL.GL_UNSIGNED_BYTE, graphImageBuf);
}

private void DrawImageBack(ImageIcon bagua_dui) {
    if (null == bagua_dui) {
        return;
    }
    BufferedImage bufferedImage = new BufferedImage(bagua_dui.getIconWidth(),
        bagua_dui.getIconHeight(), BufferedImage.TYPE_INT_RGB);
    Graphics2D ga = bufferedImage.createGraphics();
    ga.drawImage(bagua_dui.getImage(), 0, 0, bagua_dui.getImageObserver());
    ByteBuffer graphImageBuf = GLBuffers.newDirectByteBuffer(
        bufferedImage.getHeight() * (bufferedImage.getWidth() + 10) * 3);
    graphImageBuf.clear();
    int h = bufferedImage.getHeight();
    for (int i1 = h - 1; i1 >= 0; i1--) {
        for (int j1 = 0; j1 < bufferedImage.getWidth(); j1++) {
            if (bufferedImage.getRGB(j1, i1) != 0) {
                graphImageBuf.put((byte) (0 >> 0 & (byte) 0xFF));
                graphImageBuf.put((byte) (0 >> 8 & (byte) 0xFF));
                graphImageBuf.put((byte) (0 >> 16 & (byte) 0xFF));
            }
        }
    }
}
graphImageBuf.rewind();
gl.glDrawPixels(bufferedImage.getWidth(), bufferedImage.getHeight()
    , GL2.GL_RGB, GL.GL_UNSIGNED_BYTE, graphImageBuf);

```

```

}

public void reshape(GLAutoDrawable drawable, int x, int y, int width, int height) {
    float fAspect;
    if (height == 0) {
        height = 1;
    }
    gl.glViewport(0, 0, width, height);
    fAspect = (float) width / height;
    gl.glMatrixMode(GL2.GL_PROJECTION);
    gl.glLoadIdentity();
    glu.gluPerspective(53.0f, fAspect, 1.0, 400.0);
    gl.glMatrixMode(GL2.GL_MODELVIEW);
    gl.glLoadIdentity();
}

public void dispose(GLAutoDrawable arg0) {
}

public void weightPrint(String prePrint, Graphics2D g, int AMV_MVS_VSQX
    , int AMV_MVS_VSQY, Font font) {
    g.setFont(font);
    g.drawString(prePrint, AMV_MVS_VSQX, AMV_MVS_VSQY);
}

public void heightPrint(String prePrint, Graphics2D g, int AMV_MVS_VSQX
    , int AMV_MVS_VSQY, Font font) {
    g.setFont(font);
    g.drawString(prePrint, AMV_MVS_VSQX, AMV_MVS_VSQY);
}
}

//288
// } else if (leafName.contains("木")) {
//     ga.setColor(new Color(0, 0, 0));
// } else if (leafName.contains("火")) {
//     ga.setColor(new Color(0, 0, 0));
// } else if (leafName.contains("土")) {
//     ga.setColor(new Color(0, 0, 0));
// } else if (leafName.contains("金")) {
//     ga.setColor(new Color(0, 0, 0));
// } else if (leafName.contains("水")) {
//     ga.setColor(new Color(0, 0, 0));
// }

//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_V1.java
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_VTest.java
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_X_maps.java
package M_V.ME.APM.VSQ.xlx;

import VSQ.threeD.flowers.CoAuthorForWord_X_S;

import java.util.LinkedHashMap;

/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

```

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class CoAuthorForWord_X_maps extends CoAuthorForWord_X_S {
```

```
    public void bootCommonMap(String string, boolean isButton
```

```
        , CommonRotClass commonRotClass) {
```

```
        if (rootWord == null) {
```

```
            return;
```

```
        }
```

```
        if (rootWord.equalsIgnoreCase(string) && isButton == false) {
```

```
            return;
```

```
        }
```

```
        if (null == commonRotClass.inputs) {
```

```
            return;
```

```
        }
```

```
        rootWord = string;
```

```
        //rootLeaf.clear();
```

```
        findLeaf.clear();
```

```
        frequencyLeaf.clear();
```

```
        findLeaf.put(rootWord, 1);
```

```
        frequencyLeaf.put(rootWord, 1);
```

```
        LinkedHashMap<String, Object> leaf = new LinkedHashMap<>();
```

```
        LinkedHashMap<String, Object> subLeaf = new LinkedHashMap<>();
```

```
        for (int i = 0; i < commonRotClass.inputs.length; i++) {
```

```
            subLeaf.put(commonRotClass.inputs[i].toString()
```

```
                , new LinkedHashMap<String, Object>());
```

```
        }
```

```
        leaf.put(rootWord, subLeaf);
```

```
        commonRotClass.rootLeaf = subLeaf;
```

```
    }
```

```
}
```

```
//subLeaf.put("浆-花色-", new IMV_SQI());
```

```
//subLeaf.put("灰-曲直-", new IMV_SQI());
```

```
//subLeaf.put("林-呼怒-", new IMV_SQI());
```

```
//subLeaf.put("山-歌意-", new IMV_SQI());
```

```
//subLeaf.put("岸-禾蓄-", new IMV_SQI());
```

```
//subLeaf.put("北坎-河-恐呻", new IMV_SQI());
```

```
//subLeaf.put("溪-润健-", new IMV_SQI());
```

```
//subLeaf.put("沼-悦魄-", new IMV_SQI());
```

```
//subLeaf.put("石-商忧-", new IMV_SQI());
```

```
//subLeaf.put("岩-思顺-", new IMV_SQI());
```

```
//subLeaf.put("沙-宫赤-", new IMV_SQI());
```

```
//if(null== commonRotClass.inputs) {
```

```
//    commonRotClass.inputs= new String[11];
```

```
//    commonRotClass.inputs[0]= new String("浆-花色-");
```

```
//      commonRotClass.inputs[1]= new String("灰-曲直-");
//      commonRotClass.inputs[2]= new String("林-呼怒-");
//      commonRotClass.inputs[3]= new String("山-歌意-");
//      commonRotClass.inputs[4]= new String("岸-禾蓄-");
//      commonRotClass.inputs[5]= new String("北坎-河-恐呻");
//      commonRotClass.inputs[6]= new String("溪-润健-");
//      commonRotClass.inputs[7]= new String("沼-悦魄-");
//      commonRotClass.inputs[8]= new String("石-商忧-");
//      commonRotClass.inputs[9]= new String("岩-思顺-");
//      commonRotClass.inputs[10]= new String("沙-宫赤-");
//}

//subLeaf.put("马-午丞-灵境-甲午刑-休加制", new IMV_SQI());
//subLeaf.put("蛇-巳云-新象-辰戌刑", new IMV_SQI());
//subLeaf.put("龙-辰云-新象-辰戌刑", new IMV_SQI());
//subLeaf.put("兔-卯力-凶伤-开惊加制", new IMV_SQI());
//subLeaf.put("虎-寅月-山亭-甲申刑-殇杜加制", new IMV_SQI());
//subLeaf.put("牛-丑月-山亭-甲申刑-殇杜加制", new IMV_SQI());
//subLeaf.put("鼠-子目-屯才-生死加制", new IMV_SQI());
//subLeaf.put("猪-亥口-阴池-丙丁奇门", new IMV_SQI());
//subLeaf.put("狗-戌口-阴池-丙丁奇门", new IMV_SQI());
//subLeaf.put("鸡-酉早-水阁-丙丁奇门", new IMV_SQI());
//subLeaf.put("猴-申因-枯木-甲戌刑", new IMV_SQI());
//subLeaf.put("羊-未因-枯木-甲戌刑", new IMV_SQI());
//if(null== commonRotClass.inputs) {
//      commonRotClass.inputs= new String[12];
//      commonRotClass.inputs[0]= new String("马-午丞-灵境-甲午刑-休加制");
//      commonRotClass.inputs[1]= new String("蛇-巳云-新象-辰戌刑");
//      commonRotClass.inputs[2]= new String("龙-辰云-新象-辰戌刑");
//      commonRotClass.inputs[3]= new String("兔-卯力-凶伤-开惊加制");
//      commonRotClass.inputs[4]= new String("虎-寅月-山亭-甲申刑-殇杜加制");
//      commonRotClass.inputs[5]= new String("牛-丑月-山亭-甲申刑-殇杜加制");
//      commonRotClass.inputs[6]= new String("鼠-子目-屯才-生死加制");
//      commonRotClass.inputs[7]= new String("猪-亥口-阴池-丙丁奇门");
//      commonRotClass.inputs[8]= new String("狗-戌口-阴池-丙丁奇门");
//      commonRotClass.inputs[9]= new String("鸡-酉早-水阁-丙丁奇门");
//      commonRotClass.inputs[10]= new String("猴-申因-枯木-甲戌刑");
//      commonRotClass.inputs[11]= new String("羊-未因-枯木-甲戌刑");
//}

//subLeaf.put("景-层乱枯紫稠急-疏同", new IMV_SQI());
//subLeaf.put("杜-散灿鲜靛急瀑", new IMV_SQI());
//subLeaf.put("殇-衣嫩蓝青微湍", new IMV_SQI());
//subLeaf.put("生-软柔黄珊细缓", new IMV_SQI());
//subLeaf.put("修-滑透墨辉绵平-枢密", new IMV_SQI());
//subLeaf.put("开-坚鲜浓红呼清", new IMV_SQI());
//subLeaf.put("精-纯淡红橙当浊", new IMV_SQI());
//subLeaf.put("陨-混浓棕黄刺污", new IMV_SQI());
//
```



```
//if(null== commonRotClass.inputs) {  
  
//    commonRotClass.inputs= new String[8];  
//    commonRotClass.inputs[0]= new String("景-层乱枯紫稠急-疏同");  
//    commonRotClass.inputs[1]= new String("杜-散灿鲜靛急瀑");  
//    commonRotClass.inputs[2]= new String("殇-衣嫩蓝青微湍");  
//    commonRotClass.inputs[3]= new String("生-软柔黄珊细缓");  
//    commonRotClass.inputs[4]= new String("修-滑透墨辉绵平-枢密");  
//    commonRotClass.inputs[5]= new String("开-坚鲜浓红呼清");  
//    commonRotClass.inputs[6]= new String("精-纯淡红橙当浊");  
//    commonRotClass.inputs[7]= new String("陨-混浓棕黄刺污");  
//}  
  
//subLeaf.put("午-天广-酉", new IMV_SQI());  
//subLeaf.put("丙-太威-辛", new IMV_SQI());  
//subLeaf.put("巳-天屏-酉", new IMV_SQI());  
//subLeaf.put("巽-太乙-Kui", new IMV_SQI());  
//subLeaf.put("辰-天罡-Li", new IMV_SQI());  
//subLeaf.put("乙-天官-甲", new IMV_SQI());  
//subLeaf.put("卯-天苑-丁", new IMV_SQI());  
//subLeaf.put("甲-天苑-丙", new IMV_SQI());  
//  
//subLeaf.put("寅-天培-未", new IMV_SQI());  
//subLeaf.put("艮-天市-丁", new IMV_SQI());  
//subLeaf.put("丑-天厨-辰", new IMV_SQI());  
//subLeaf.put("Kui-阴光-巳", new IMV_SQI());  
//subLeaf.put("子-天垒-巳", new IMV_SQI());  
//subLeaf.put("壬-天辅-申", new IMV_SQI());  
//subLeaf.put("亥-天皇-乙", new IMV_SQI());  
//subLeaf.put("乾-天庭-卯", new IMV_SQI());  
//  
//subLeaf.put("戌-天魁-丑", new IMV_SQI());  
//subLeaf.put("辛-天乙-丑", new IMV_SQI());  
//subLeaf.put("酉-少微-寅", new IMV_SQI());  
//subLeaf.put("庚-天汉-午", new IMV_SQI());  
//subLeaf.put("申-天关-Kui", new IMV_SQI());  
//subLeaf.put("坤-天镜-乙", new IMV_SQI());  
//subLeaf.put("未-天常-Kui", new IMV_SQI());  
//subLeaf.put("丁-南极-寅", new IMV_SQI());  
//  
//if(null== commonRotClass.inputs) {  
  
//    commonRotClass.inputs= new String[24];  
//    commonRotClass.inputs[0]= new String("午-天广-酉");  
//    commonRotClass.inputs[1]= new String("丙-太威-辛");  
//    commonRotClass.inputs[2]= new String("巳-天屏-酉");  
//    commonRotClass.inputs[3]= new String("巽-太乙-Kui");  
//    commonRotClass.inputs[4]= new String("辰-天罡-Li");  
//    commonRotClass.inputs[5]= new String("乙-天官-甲");  
//    commonRotClass.inputs[6]= new String("卯-天苑-丁");
```

```
//      commonRotClass.inputs[7]= new String("甲-天苑-丙");
//
//      commonRotClass.inputs[8]= new String("寅-天培-未");
//      commonRotClass.inputs[9]= new String("艮-天市-丁");
//      commonRotClass.inputs[10]= new String("丑-天厨-辰");
//      commonRotClass.inputs[11]= new String("Kui-阴光-巳");
//      commonRotClass.inputs[12]= new String("子-天垒-巳");
//      commonRotClass.inputs[13]= new String("壬-天辅-申");
//      commonRotClass.inputs[14]= new String("亥-天皇-乙");
//      commonRotClass.inputs[15]= new String("乾-天庭-卯");
//
//
//      commonRotClass.inputs[16]= new String("戌-天魁-丑");
//      commonRotClass.inputs[17]= new String("辛-天乙-丑");
//      commonRotClass.inputs[18]= new String("酉-少微-寅");
//      commonRotClass.inputs[19]= new String("庚-天汉-午");
//      commonRotClass.inputs[20]= new String("申-天关-Kui");
//      commonRotClass.inputs[21]= new String("坤-天镜-乙");
//      commonRotClass.inputs[22]= new String("未-天常-Kui");
//      commonRotClass.inputs[23]= new String("丁-南极-寅");
//}
//if(null== commonRotClass.inputs) {
//      commonRotClass.inputs= new String[8];
//      commonRotClass.inputs[0]= new String("心 小肠 离 腕踝上 经");
//      commonRotClass.inputs[1]= new String("胆 肝 巽 掌趾前 荣");
//      commonRotClass.inputs[2]= new String("胆 肝 震 掌趾前 荣");
//      commonRotClass.inputs[3]= new String("血 胃 艮 肘膝中 合");
//      commonRotClass.inputs[4]= new String("肾 膀胱 坎 掌趾后 输");
//      commonRotClass.inputs[5]= new String("大肠 乾 手足端 井");
//      commonRotClass.inputs[6]= new String("肺 兑 手足端 井");
//      commonRotClass.inputs[7]= new String("血 脾 坤 肘膝中 合");
//}
//if(null== commonRotClass.inputs) {
//commonRotClass.inputs= new String[8];
//commonRotClass.inputs[0]= new String("上焦-血分-营分-心-小肠");
//commonRotClass.inputs[1]= new String("气分-胆-股");
//commonRotClass.inputs[2]= new String("下焦-气分-血分-肝-足");
//commonRotClass.inputs[3]= new String("中焦-气分-胃-手");
//commonRotClass.inputs[4]= new String("下焦-血分-卫分-耳-肾-膀胱");
//commonRotClass.inputs[5]= new String("气分-大肠-首-健");
//commonRotClass.inputs[6]= new String("上焦-卫分-肺-口");
//commonRotClass.inputs[7]= new String("中焦-血分-脾-腹");
//}
//640
////subLeaf.put("03-白羊-土-3m21_4m19", new IMV_SQI());
//subLeaf.put("04-金牛-金-4m20_5m20", new IMV_SQI());
//subLeaf.put("05-双子-金-5m21_6m21", new IMV_SQI());
```

```
//subLeaf.put("06-巨蟹-土-6m22_7m22", new IMV_SQI());
//subLeaf.put("07-狮子-火-7m23_8m22", new IMV_SQI());
//subLeaf.put("08-处女-火-8m23_9m22", new IMV_SQI());
//subLeaf.put("09-天秤-土-9m23_10m23", new IMV_SQI());
//subLeaf.put("10-天蝎-木-10m24_11m22", new IMV_SQI());
//subLeaf.put("11-射手-木-11m23_12m21", new IMV_SQI());
//subLeaf.put("12-魔蝎-土-12m22_1m19", new IMV_SQI());
//subLeaf.put("01-水瓶-水-1m20_2m18", new IMV_SQI());
//subLeaf.put("02-双鱼-水-2m19_3m20", new IMV_SQI());
//
//if(null== commonRotClass.inputs) {
//    commonRotClass.inputs= new String[12];
//    commonRotClass.inputs[0]= new String("03-白羊-土-3m21_4m19");
//    commonRotClass.inputs[1]= new String("04-金牛-金-4m20_5m20");
//    commonRotClass.inputs[2]= new String("05-双子-金-5m21_6m21");
//    commonRotClass.inputs[3]= new String("06-巨蟹-土-6m22_7m22");
//    commonRotClass.inputs[4]= new String("07-狮子-火-7m23_8m22");
//    commonRotClass.inputs[5]= new String("08-处女-火-8m23_9m22");
//    commonRotClass.inputs[6]= new String("09-天秤-土-9m23_10m23");
//    commonRotClass.inputs[7]= new String("10-天蝎-木-10m24_11m22");
//    commonRotClass.inputs[8]= new String("11-射手-木-11m23_12m21");
//    commonRotClass.inputs[9]= new String("12-魔蝎-土-12m22_1m19");
//    commonRotClass.inputs[10]= new String("01-水瓶-水-1m20_2m18");
//    commonRotClass.inputs[11]= new String("02-双鱼-水-2m19_3m20");
//}
//737
//if(null== inputs) {
//inputs= new String[8];
//inputs[0]="心 小肠 离 腕踝上 经";
//inputs[1]="胆 肝 巽 掌趾前 荣";
//inputs[2]="胆 肝 震 掌趾前 荣";
//inputs[3]="血 胃 艮 肘膝中 合";
//inputs[4]="肾 膀胱 坎 掌趾后 输";
//inputs[5]="大肠 乾 手足端 井";
//inputs[6]="肺 兑 手足端 井";
//inputs[7]="血 脾 坤 肘膝中 合";
//}
//
//691
//subLeaf.put("上焦-血分-营分-心-小肠", new IMV_SQI());
//subLeaf.put("气分-胆-股", new IMV_SQI());
//subLeaf.put("下焦-气分-血分-肝-足", new IMV_SQI());
//subLeaf.put("中焦-气分-胃-手", new IMV_SQI());
//subLeaf.put("下焦-血分-卫分-耳-肾-膀胱", new IMV_SQI());
//subLeaf.put("气分-大肠-首-健", new IMV_SQI());
//subLeaf.put("上焦-卫分-肺-口", new IMV_SQI());
//subLeaf.put("中焦-血分-脾-腹", new IMV_SQI());
```

```
//
//640
//subLeaf.put("03-白羊-土-3m21_4m19", new IMV_SQI());
//subLeaf.put("04-金牛-金-4m20_5m20", new IMV_SQI());
//subLeaf.put("05-双子-金-5m21_6m21", new IMV_SQI());
//subLeaf.put("06-巨蟹-土-6m22_7m22", new IMV_SQI());
//subLeaf.put("07-狮子-火-7m23_8m22", new IMV_SQI());
//subLeaf.put("08-处女-火-8m23_9m22", new IMV_SQI());
//subLeaf.put("09-天秤-土-9m23_10m23", new IMV_SQI());
//subLeaf.put("10-天蝎-木-10m24_11m22", new IMV_SQI());
//subLeaf.put("11-射手-木-11m23_12m21", new IMV_SQI());
//subLeaf.put("12-魔蝎-土-12m22_1m19", new IMV_SQI());
//subLeaf.put("01-水瓶-水-1m20_2m18", new IMV_SQI());
//subLeaf.put("02-双鱼-水-2m19_3m20", new IMV_SQI());
//
//if(null== commonRotClass.inputs) {
//    commonRotClass.inputs= new String[12];
//    commonRotClass.inputs[0]= new String("03-白羊-土-3m21_4m19");
//    commonRotClass.inputs[1]= new String("04-金牛-金-4m20_5m20");
//    commonRotClass.inputs[2]= new String("05-双子-金-5m21_6m21");
//    commonRotClass.inputs[3]= new String("06-巨蟹-土-6m22_7m22");
//    commonRotClass.inputs[4]= new String("07-狮子-火-7m23_8m22");
//    commonRotClass.inputs[5]= new String("08-处女-火-8m23_9m22");
//    commonRotClass.inputs[6]= new String("09-天秤-土-9m23_10m23");
//    commonRotClass.inputs[7]= new String("10-天蝎-木-10m24_11m22");
//    commonRotClass.inputs[8]= new String("11-射手-木-11m23_12m21");
//    commonRotClass.inputs[9]= new String("12-魔蝎-土-12m22_1m19");
//    commonRotClass.inputs[10]= new String("01-水瓶-水-1m20_2m18");
//    commonRotClass.inputs[11]= new String("02-双鱼-水-2m19_3m20");
//}
//subLeaf.put("夏至", new IMV_SQI());
//subLeaf.put("芒种", new IMV_SQI());
//subLeaf.put("小满", new IMV_SQI());
//subLeaf.put("立夏", new IMV_SQI());
//subLeaf.put("谷雨", new IMV_SQI());
//
//subLeaf.put("清明", new IMV_SQI());
//subLeaf.put("春分", new IMV_SQI());
//subLeaf.put("惊蛰", new IMV_SQI());
//subLeaf.put("雨水", new IMV_SQI());
//subLeaf.put("立春", new IMV_SQI());
//
//subLeaf.put("大寒", new IMV_SQI());
//subLeaf.put("小寒", new IMV_SQI());
//subLeaf.put("冬至", new IMV_SQI());
//subLeaf.put("大雪", new IMV_SQI());
//subLeaf.put("小雪", new IMV_SQI());
```

```
//
//subLeaf.put("立冬", new IMV_SQI());
//subLeaf.put("霜降", new IMV_SQI());
//subLeaf.put("寒露", new IMV_SQI());
//subLeaf.put("秋分", new IMV_SQI());
//subLeaf.put("白露", new IMV_SQI());
//
//subLeaf.put("处暑", new IMV_SQI());
//subLeaf.put("立秋", new IMV_SQI());
//subLeaf.put("大暑", new IMV_SQI());
//subLeaf.put("小暑", new IMV_SQI());
//if(null== commonRotClass.inputs) {
//    commonRotClass.inputs= new String[24];
//    commonRotClass.inputs[0]= new String("夏至");
//    commonRotClass.inputs[1]= new String("芒种");
//    commonRotClass.inputs[2]= new String("小满");
//    commonRotClass.inputs[3]= new String("立夏");
//    commonRotClass.inputs[4]= new String("谷雨");
//
//    commonRotClass.inputs[5]= new String("清明");
//    commonRotClass.inputs[6]= new String("春分");
//    commonRotClass.inputs[7]= new String("惊蛰");
//    commonRotClass.inputs[8]= new String("雨水");
//    commonRotClass.inputs[9]= new String("立春");
//
//    commonRotClass.inputs[10]= new String("大寒");
//    commonRotClass.inputs[11]= new String("小寒");
//    commonRotClass.inputs[12]= new String("冬至");
//    commonRotClass.inputs[13]= new String("大雪");
//    commonRotClass.inputs[14]= new String("小雪");
//
//    commonRotClass.inputs[15]= new String("立冬");
//    commonRotClass.inputs[16]= new String("霜降");
//    commonRotClass.inputs[17]= new String("寒露");
//    commonRotClass.inputs[18]= new String("秋分");
//    commonRotClass.inputs[19]= new String("白露");
//
//    commonRotClass.inputs[20]= new String("处暑");
//    commonRotClass.inputs[21]= new String("立秋");
//    commonRotClass.inputs[22]= new String("大暑");
//    commonRotClass.inputs[23]= new String("小暑");
//}
//subLeaf.put("心 小肠 离 腕踝上 经", new IMV_SQI());
//subLeaf.put("胆 肝 巽 掌趾前 荣", new IMV_SQI());
//subLeaf.put("胆 肝 震 掌趾前 荣", new IMV_SQI());
//subLeaf.put("血 胃 艮 肘膝中 合", new IMV_SQI());
//subLeaf.put("肾 膀胱 坎 掌趾后 输", new IMV_SQI());
```

```
//subLeaf.put("大肠 乾 手足端 井", new IMV_SQI());
//subLeaf.put("肺 兑 手足端 井", new IMV_SQI());
//subLeaf.put("血 脾 坤 肘膝中 合", new IMV_SQI());
//if(null== commonRotClass.inputs) {
//    commonRotClass.inputs= new String[8];
//    commonRotClass.inputs[0]= new String("心 小肠 离 腕踝上 经");
//    commonRotClass.inputs[1]= new String("胆 肝 巽 掌趾前 荣");
//    commonRotClass.inputs[2]= new String("胆 肝 震 掌趾前 荣");
//    commonRotClass.inputs[3]= new String("血 胃 艮 肘膝中 合");
//    commonRotClass.inputs[4]= new String("肾 膀胱 坎 掌趾后 输");
//    commonRotClass.inputs[5]= new String("大肠 乾 手足端 井");
//    commonRotClass.inputs[6]= new String("肺 兑 手足端 井");
//    commonRotClass.inputs[7]= new String("血 脾 坤 肘膝中 合");
//}
//subLeaf.put("XJ 离 手少阳 手少阴 手厥阴 手太阳 ↑ Ⅲ", new IMV_SQI());
//subLeaf.put("WS YQ 巽 足少阳 ↘ V", new IMV_SQI());
//subLeaf.put("YQ MG 震 足厥阴 ← IV", new IMV_SQI());
//subLeaf.put("MG 艮 手少阳 足阳明 ↗ Ⅶ", new IMV_SQI());
//subLeaf.put("JA 坎 足少阴 足太阴 手少阳 ↓ VI", new IMV_SQI());
//subLeaf.put("XH FG 乾 手阳明 ↘ I", new IMV_SQI());
//subLeaf.put("LZ SX 兑 手太阴 → II", new IMV_SQI());
//subLeaf.put("SX JZ 坤 足太阴 ↗ Ⅷ", new IMV_SQI());
//subLeaf.put("XJ禁XH 离 胆", new IMV_SQI());
//subLeaf.put("WS禁LZ 巽 胆 肝", new IMV_SQI());
//subLeaf.put("YQ MG禁LZ 震 消化 胃", new IMV_SQI());
//subLeaf.put("MG禁JA 艮 唾 胰", new IMV_SQI());
//subLeaf.put("JA禁JZ 坎 发", new IMV_SQI());
//subLeaf.put("XH FG禁XJ 乾 眠 泪", new IMV_SQI());
//subLeaf.put("LZ SX禁JA 兑 寔", new IMV_SQI());
//subLeaf.put("SX JZ禁JA 坤 鼻 汗", new IMV_SQI());
//
//if(null== commonRotClass.inputs) {
//    commonRotClass.inputs= new String[8];
//    commonRotClass.inputs[0]= new String("XJ禁XH 离 胆");
//    commonRotClass.inputs[1]= new String("WS禁LZ 巽 胆 肝");
//    commonRotClass.inputs[2]= new String("YQ MG禁LZ 震 消化 胃");
//    commonRotClass.inputs[3]= new String("MG禁JA 艮 唾 胰");
//    commonRotClass.inputs[4]= new String("JA禁JZ 坎 发");
//    commonRotClass.inputs[5]= new String("XH FG禁XJ 乾 眠 泪");
//    commonRotClass.inputs[6]= new String("LZ SX禁JA 兑 寔");
//    commonRotClass.inputs[7]= new String("SX JZ禁JA 坤 鼻 汗");
//}
//subLeaf.put("DH 景 酰 离 ↑ Ⅲ", new IMV_SQI());
//subLeaf.put("QA 杜 酸 巽 ↘ V", new IMV_SQI());
//subLeaf.put("PI 殇 酮 震 ← IV", new IMV_SQI());
//subLeaf.put("ST 生 酯 艮 ↗ Ⅶ", new IMV_SQI());
//subLeaf.put("UO 休 酚 坎 ↓ VI", new IMV_SQI());
```

```
//subLeaf.put("MV 开 醚 乾↘ I ", new IMV_SQI());
//subLeaf.put("CX 惊 醇 兑→ II ", new IMV_SQI());
//subLeaf.put("DD E 死 酶 坤↗ VIII ", new IMV_SQI());
//subLeaf.put("H PV DD 酰 火 ↑ ", new IMV_SQI());
//subLeaf.put("H OEU 醇 酮 木↘ ", new IMV_SQI());
//subLeaf.put("H CDI 酯 酸 土↗ ", new IMV_SQI());
//subLeaf.put("H MXS 醚 水↙ ", new IMV_SQI());
//subLeaf.put("H AQT 酚 酶 金↘ ", new IMV_SQI());
//if(null== commonRotClass.inputs) {
//    commonRotClass.inputs= new String[5];
//    commonRotClass.inputs[0]= new String("H PV DD 酰 火 ↑ ");
//    commonRotClass.inputs[1]= new String("H OEU 醇 酮 木↘ ");
//    commonRotClass.inputs[2]= new String("H CDI 酯 酸 土↗ ");
//    commonRotClass.inputs[3]= new String("H MXS 醚 水↙ ");
//    commonRotClass.inputs[4]= new String("H AQT 酚 酶 金↘ ");
//}
//subLeaf.put("PV 景 酰 离 ↑ III ", new IMV_SQI());
//subLeaf.put("UH 杜 酮 巽↘ V ", new IMV_SQI());
//subLeaf.put("OE 殇 醇 震← IV ", new IMV_SQI());
//subLeaf.put("IX 生 酯 艮↙ VII ", new IMV_SQI());
//subLeaf.put("MS 休 醚 坎 ↓ VI ", new IMV_SQI());
//subLeaf.put("Q 开 酶 乾↘ I ", new IMV_SQI());
//subLeaf.put("ACT 惊 酚 兑→ II ", new IMV_SQI());
//subLeaf.put("D 死 酸 坤↗ VIII ", new IMV_SQI());
//subLeaf.put("酰 离 DH 清润核酸 ↑ III ", new IMV_SQI());
//subLeaf.put("酸 巽 QA erosion调节↘ V ", new IMV_SQI());
//subLeaf.put("酮 震 IP 激素扩散← IV ", new IMV_SQI());
//subLeaf.put("酯 艮 TS 体能聚能↙ VII ", new IMV_SQI());
//subLeaf.put("酚 坎 OU 受体免疫 ↓ VI ", new IMV_SQI());
//subLeaf.put("醚 乾 VM 受体诱导↘ I ", new IMV_SQI());
//subLeaf.put("醇 兑 CX 扩散免疫→ II ", new IMV_SQI());
//subLeaf.put("酶 坤 DD E erosion诱导↗ VIII ", new IMV_SQI());
//public void bootFlowerForest(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//    return;
//}
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {
//    return;
//}
////这里要全部重写，整个插件 canvas
//rootWord= string;
////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
//frequencyLeaf.put(rootWord, 1);
//IMV_SQI leaf= new IMV_SQI();
```

```
//IMV_SQI subLeaf= new LinkedHashMap<>();

//for(int i= 0; i< commonRotClass.inputs.length; i++) {
//
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
//}

//leaf.put(rootWord, subLeaf);
//commonRotClass.rootLeaf= subLeaf;
//}

//public void bootFlowerSea(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//
//    return;
//}

//if(rootWord.equalsIgnoreCase(string) && isButton== false) {
//
//    return;
//}

//rootWord= string;
////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
//frequencyLeaf.put(rootWord, 1);
//IMV_SQI leaf= new IMV_SQI();
//IMV_SQI subLeaf= new LinkedHashMap<>();
//for(int i= 0; i< commonRotClass.inputs.length; i++) {
//
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
//}

//leaf.put(rootWord, subLeaf);
//commonRotClass.rootLeaf= subLeaf;
//}

//public void bootClassificationMap(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//
//    return;
//}

//if(rootWord.equalsIgnoreCase(string) && isButton== false) {
//
//    return;
//}

//rootWord= string;
////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
//frequencyLeaf.put(rootWord, 1);
//IMV_SQI leaf= new IMV_SQI();
//IMV_SQI subLeaf= new LinkedHashMap<>();
//for(int i= 0; i< commonRotClass.inputs.length; i++) {
//
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
//}

//leaf.put(rootWord, subLeaf);
```



```
//commonRotClass.rootLeaf= subLeaf;

//}

//public void bootBaYouMap(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//    return;
//}

//if(rootWord.equalsIgnoreCase(string) && isButton== false) {
//    return;
//}

//rootWord= string;
////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
//frequencyLeaf.put(rootWord, 1);
//IMV_SQI leaf= new IMV_SQI();
//IMV_SQI subLeaf= new LinkedHashMap<>();
//for(int i= 0; i< commonRotClass.inputs.length; i++) {
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
//}
//leaf.put(rootWord, subLeaf);
//commonRotClass.rootLeaf= subLeaf;
//}

//public void bootJingLuoMap(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//    return;
//}

//if(rootWord.equalsIgnoreCase(string) && isButton== false) {
//    return;
//}

//rootWord= string;
////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
//frequencyLeaf.put(rootWord, 1);
//IMV_SQI leaf= new IMV_SQI();
//IMV_SQI subLeaf= new LinkedHashMap<>();
//for(int i= 0; i< commonRotClass.inputs.length; i++) {
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
//}
//leaf.put(rootWord, subLeaf);
//commonRotClass.rootLeaf= subLeaf;
//}

//public void bootZiWuMap(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//    return;
```

```
//}  
  
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {  
//    return;  
//}  
  
//rootWord= string;  
////rootLeaf.clear();  
//findLeaf.clear();  
//frequencyLeaf.clear();  
//findLeaf.put(rootWord, 1);  
//frequencyLeaf.put(rootWord, 1);  
//IMV_SQI leaf= new IMV_SQI();  
//IMV_SQI subLeaf= new LinkedHashMap<>();  
//for(int i= 0; i< commonRotClass.inputs.length; i++) {  
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());  
//}  
//leaf.put(rootWord, subLeaf);  
//commonRotClass.rootLeaf= subLeaf;  
//}  
  
//public void bootLiuZhuMap(String string, boolean isButton, CommonRotClass commonRotClass) {  
//if(rootWord== null) {  
//    return;  
//}  
  
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {  
//    return;  
//}  
  
//rootWord= string;  
////rootLeaf.clear();  
//findLeaf.clear();  
//frequencyLeaf.clear();  
//findLeaf.put(rootWord, 1);  
//frequencyLeaf.put(rootWord, 1);  
//IMV_SQI leaf= new IMV_SQI();  
//IMV_SQI subLeaf= new LinkedHashMap<>();  
//for(int i= 0; i< commonRotClass.inputs.length; i++) {  
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());  
//}  
//leaf.put(rootWord, subLeaf);  
//commonRotClass.rootLeaf= subLeaf;  
//}  
  
//public void bootJieQiMap(String string, boolean isButton, CommonRotClass commonRotClass) {  
//if(rootWord== null) {  
//    return;  
//}  
  
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {  
//    return;  
//}  
  
//rootWord= string;
```

```
////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
//frequencyLeaf.put(rootWord, 1);
//IMV_SQI leaf= new IMV_SQI();
//IMV_SQI subLeaf= new LinkedHashMap<>();
//for(int i= 0; i< commonRotClass.inputs.length; i++) {
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
//}
//leaf.put(rootWord, subLeaf);
//commonRotClass.rootLeaf= subLeaf;
//}
//public void bootBaGangMap(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//    return;
//}
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {
//    return;
//}
//rootWord= string;
//////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
//frequencyLeaf.put(rootWord, 1);
//IMV_SQI leaf= new IMV_SQI();
//IMV_SQI subLeaf= new LinkedHashMap<>();
//
//for(int i= 0; i< commonRotClass.inputs.length; i++) {
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
//}
//leaf.put(rootWord, subLeaf);
//commonRotClass.rootLeaf= subLeaf;
//}
//public void bootKanYuMap(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//    return;
//}
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {
//    return;
//}
//rootWord= string;
//////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
```

```
//frequencyLeaf.put(rootWord, 1);

//IMV_SQI leaf= new IMV_SQI();

//IMV_SQI subLeaf= new LinkedHashMap<>();

//for(int i= 0; i< commonRotClass.inputs.length; i++) {

//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());

//}

//leaf.put(rootWord, subLeaf);

//commonRotClass.rootLeaf= subLeaf;

//}

//public void bootLiuRenMap(String string, boolean isButton, CommonRotClass commonRotClass) {

//if(rootWord== null) {

//    return;

//}

//if(rootWord.equalsIgnoreCase(string) && isButton== false) {

//    return;

//}

//rootWord= string;

////rootLeaf.clear();

//findLeaf.clear();

//frequencyLeaf.clear();

//findLeaf.put(rootWord, 1);

//frequencyLeaf.put(rootWord, 1);

//IMV_SQI leaf= new IMV_SQI();

//IMV_SQI subLeaf= new LinkedHashMap<>();

//for(int i= 0; i< commonRotClass.inputs.length; i++) {

//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());

//}

//leaf.put(rootWord, subLeaf);

//commonRotClass.rootLeaf= subLeaf;

//}

//public void bootYiShuMap(String string, boolean isButton, CommonRotClass commonRotClass) {

//if(rootWord== null) {

//    return;

//}

//if(rootWord.equalsIgnoreCase(string) && isButton== false) {

//    return;

//}

//rootWord= string;

////rootLeaf.clear();

//findLeaf.clear();

//frequencyLeaf.clear();

//findLeaf.put(rootWord, 1);

//frequencyLeaf.put(rootWord, 1);

//IMV_SQI leaf= new IMV_SQI();

//IMV_SQI subLeaf= new LinkedHashMap<>();

//for(int i= 0; i< commonRotClass.inputs.length; i++) {

//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
```

```
//}  
  
//  
//leaf.put(rootWord, subLeaf);  
//commonRotClass.rootLeaf= subLeaf;  
//}  
//  
  
////经仪数据来自刘基 袁天罡 市面通用八卦罗盘。  
//public void bootTianXingMap(String string, boolean isButton, CommonRotClass commonRotClass) {  
//if(rootWord== null) {  
//    return;  
//}  
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {  
//    return;  
//}  
//rootWord= string;  
////rootLeaf.clear();  
//findLeaf.clear();  
//frequencyLeaf.clear();  
//findLeaf.put(rootWord, 1);  
//frequencyLeaf.put(rootWord, 1);  
//IMV_SQI leaf= new IMV_SQI();  
//IMV_SQI subLeaf= new LinkedHashMap<>();  
//for(int i= 0; i< commonRotClass.inputs.length; i++) {  
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());  
//}  
//leaf.put(rootWord, subLeaf);  
//commonRotClass.rootLeaf= subLeaf;  
//}  
  
//public void bootJingYiMap(String string, boolean isButton, CommonRotClass commonRotClass) {  
//if(rootWord== null) {  
//    return;  
//}  
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {  
//    return;  
//}  
//rootWord= string;  
////rootLeaf.clear();  
//findLeaf.clear();  
//frequencyLeaf.clear();  
//findLeaf.put(rootWord, 1);  
//frequencyLeaf.put(rootWord, 1);  
//IMV_SQI leaf= new IMV_SQI();  
//IMV_SQI subLeaf= new LinkedHashMap<>();  
//  
//subLeaf.put("", new IMV_SQI());  
//  
//leaf.put(rootWord, subLeaf);
```

```
///// rootLeafList.add(subLeaf);

//rootLeafJingYi= subLeaf;

////commonRotClass.rootLeaf= subLeaf;

//}

//public void bootHeLuoMap(String string, boolean isButton, CommonRotClass commonRotClass) {

//if(rootWord== null) {

//    return;

//}

//if(rootWord.equalsIgnoreCase(string) && isButton== false) {

//    return;

//}

//rootWord= string;

////rootLeaf.clear();

//findLeaf.clear();

//frequencyLeaf.clear();

//findLeaf.put(rootWord, 1);

//frequencyLeaf.put(rootWord, 1);

//IMV_SQI leaf= new IMV_SQI();

//IMV_SQI subLeaf= new LinkedHashMap<>();

//

//for(int i= 0; i< commonRotClass.inputs.length; i++) {

//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());

//}

//leaf.put(rootWord, subLeaf);

//commonRotClass.rootLeaf= subLeaf;

//}

//public void bootSanJiaoMap(String string, boolean isButton, CommonRotClass commonRotClass) {

//if(rootWord== null) {

//    return;

//}

//if(rootWord.equalsIgnoreCase(string) && isButton== false) {

//    return;

//}

//rootWord= string;

////rootLeaf.clear();

//findLeaf.clear();

//frequencyLeaf.clear();

//findLeaf.put(rootWord, 1);

//frequencyLeaf.put(rootWord, 1);

//IMV_SQI leaf= new IMV_SQI();

//IMV_SQI subLeaf= new LinkedHashMap<>();

//for(int i= 0; i< commonRotClass.inputs.length; i++) {

//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());

//}

//leaf.put(rootWord, subLeaf);

//commonRotClass.rootLeaf= subLeaf;

//}
```

```

//zishiying chuankou

//tinshell add mingling

//jiemian add method

//

//public void bootRandomMap(String string, boolean isButton, CommonRotClass commonRotClass) {
//if(rootWord== null) {
//    return;
//}
//if(rootWord.equalsIgnoreCase(string) && isButton== false) {
//    return;
//}
//rootWord= string;
////rootLeaf.clear();
//findLeaf.clear();
//frequencyLeaf.clear();
//findLeaf.put(rootWord, 1);
//frequencyLeaf.put(rootWord, 1);
//IMV_SQI leaf= new IMV_SQI();
//IMV_SQI subLeaf= new LinkedHashMap<>();
//for(int i= 0; i< commonRotClass.inputs.length; i++) {
//    subLeaf.put(commonRotClass.inputs[i].toString(), new IMV_SQI());
//}
//leaf.put(rootWord, subLeaf);
//commonRotClass.rootLeaf= subLeaf;
//}

//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_X_mapsTest.java
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWord_X_Slider.java
package M_V.ME.APM.VSQ.xlx;

import M_V.MVQ.button.DetaButton;

import M_V.MVQ.slider.DetaSlider;

import exception.thread.DetaThread;

import javax.swing.Box;

import javax.swing.BoxLayout;

import javax.swing.JButton;

import javax.swing.JSlider;

import java.awt.Color;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.util.List;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

 */

```

```
public class CoAuthorForWord_X_Slider extends CoAuthorForWord_X_maps {
```

```
    public DetaSlider I_DetaSlider(String string, boolean isRotation
        , boolean isObservation) {
        DetaSlider detaSlider = new DetaSlider(0, 1500);
        detaSlider.setSnapToTicks(true);
        detaSlider.setPaintTicks(true);
        detaSlider.setMajorTickSpacing(20);
        detaSlider.setMinorTickSpacing(1);
        detaSlider.addChangeListener(
            event -> {
                JSlider source = (JSlider) event.getSource();
                CommonRotClass commonRotClass = new CommonRotClass();
                if (rootClassMap.containsKey(string)) {
                    commonRotClass = rootClassMap.getCommonRotClass(string);
                }
                if (isObservation) {
                    commonRotClass.rotZ = source.getValue() << 2;
                }
                if (isRotation) {
                    commonRotClass.rot = source.getValue() << 2;
                }
                rootClassMap.put(string, commonRotClass);
            });
        return detaSlider;
    }
}
```

```
public Box creatSlidersBox() {
    sliderFangDa = new DetaSlider(0, 1500);
    sliderFangDa.setSnapToTicks(true);
    sliderFangDa.setPaintTicks(true);
    sliderFangDa.setMajorTickSpacing(20);
    sliderFangDa.setMinorTickSpacing(1);
    sliderFangDa.addChangeListener(
        event -> {
            JSlider source = (JSlider) event.getSource();
            rotFangDa = source.getValue() << 2;
            trot = rotFangDa * 0.03F;
        });
    buttonFangDa = new DetaButton("放大", 100, 50, Color.RED);
    buttonFangDa.addActionListener(
        e -> {
            //to do..
        });
    Box sliderBoxFangDa = new Box(BoxLayout.X_AXIS);
    sliderBoxFangDa.add(buttonFangDa);
    sliderBoxFangDa.add(sliderFangDa);
    sliderSuoXiao = new DetaSlider(0, 1500);
    sliderSuoXiao.setSnapToTicks(true);
}
```



```
sliderSuoXiao.setPaintTicks(true);
sliderSuoXiao.setMajorTickSpacing(20);
sliderSuoXiao.setMinorTickSpacing(1);
sliderSuoXiao.addChangeListener(event -> {
    JSlider source = (JSlider) event.getSource();
    rotSuoXiao = source.getValue() < 2;
});
buttonSuoXiao = new DetaButton("缩小", 100, 50, Color.RED);
buttonSuoXiao.addActionListener(e -> {
    //to do..
});
Box sliderBoxSuoXiao = new Box(BoxLayout.X_AXIS);
sliderBoxSuoXiao.add(buttonSuoXiao);
sliderBoxSuoXiao.add(sliderSuoXiao);
sliderShunLiu = new DetaSlider(0, 1500);
sliderShunLiu.setSnapToTicks(true);
sliderShunLiu.setPaintTicks(true);
sliderShunLiu.setMajorTickSpacing(20);
sliderShunLiu.setMinorTickSpacing(1);
sliderShunLiu.addChangeListener(event -> {
    JSlider source = (JSlider) event.getSource();
    rotShunLiu = source.getValue() < 2;
});
buttonShunLiu = new DetaButton("顺流", 100, 50, Color.RED);
buttonShunLiu.addActionListener(e -> {
    //to do..
});
Box sliderBoxShunLiu = new Box(BoxLayout.X_AXIS);
sliderBoxShunLiu.add(buttonShunLiu);
sliderBoxShunLiu.add(sliderShunLiu);
sliderNiLiu = new DetaSlider(0, 1500);
sliderNiLiu.setSnapToTicks(true);
sliderNiLiu.setPaintTicks(true);
sliderNiLiu.setMajorTickSpacing(20);
sliderNiLiu.setMinorTickSpacing(1);
sliderNiLiu.addChangeListener(event -> {
    JSlider source = (JSlider) event.getSource();
    rotNiLiu = source.getValue() < 2;
});
buttonNiLiu = new DetaButton("逆流", 100, 50, Color.RED);
buttonNiLiu.addActionListener(e -> {
    //to do..
});
Box sliderBoxNiLiu = new Box(BoxLayout.X_AXIS);
sliderBoxNiLiu.add(buttonNiLiu);
sliderBoxNiLiu.add(sliderNiLiu);
//sliderBoxRight
```

```
//稍后优化 commonInputs
sliderSanJiao = I_DetaSlider("三焦", true, false);
//稍后缩进
sliderSanJiaoZ = I_DetaSlider("三焦", false, true);
//稍后缩进
buttonSanJiao = new DetaButton("三焦关", 100, 50, Color.green);
buttonSanJiao.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("三焦")) {
        commonRotClass = rootClassMap.getCommonRotClass("三焦");
    }
    if (commonRotClass.buttonClicked) {
        buttonSanJiao.setBackground(Color.blue);
        buttonSanJiao.setText("三焦关");
    } else {
        buttonSanJiao.setText("三焦开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("上焦-血分-营分-心-小肠");
            commonRotClass.inputs[1] = new String("气分-胆-股");
            commonRotClass.inputs[2] = new String("下焦-气分-血分-肝-足");
            commonRotClass.inputs[3] = new String("中焦-气分-胃-手");
            commonRotClass.inputs[4] = new String("下焦-血分-卫分-耳-肾-膀胱");
            commonRotClass.inputs[5] = new String("气分-大肠-首-健");
            commonRotClass.inputs[6] = new String("上焦-卫分-肺-口");
            commonRotClass.inputs[7] = new String("中焦-血分-脾-腹");
        }
        bootCommonMap("三焦", true, commonRotClass);
        buttonSanJiao.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("三焦", commonRotClass);
});
Box sliderBoxSanJiao = new Box(BoxLayout.X_AXIS);
sliderBoxSanJiao.add(buttonSanJiao);
sliderBoxSanJiao.add(sliderSanJiao);
sliderBoxSanJiao.add(sliderSanJiaoZ);
//
sliderBaGang = I_DetaSlider("八纲", true, false);
sliderBaGangZ = I_DetaSlider("八纲", false, true);
buttonBaGang = new DetaButton("八纲关", 100, 50, Color.BLUE);
buttonBaGang.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("八纲")) {
        commonRotClass = rootClassMap.getCommonRotClass("八纲");
```

```

}

if (commonRotClass.buttonClicked) {
    buttonBaGang.setBackground(Color.blue);
    buttonBaGang.setText("八纲关");
} else {
    buttonBaGang.setText("八纲开");
    if (null == commonRotClass.inputs) {
        commonRotClass.inputs = new String[8];
        commonRotClass.inputs[0] = new String("离 甜苦干热-火红紫粉");
        commonRotClass.inputs[1] = new String("巽 酸甜湿热-木蓝");
        commonRotClass.inputs[2] = new String("震 酸湿-木绿温");
        commonRotClass.inputs[3] = new String("艮 平温凉-土米黄");
        commonRotClass.inputs[4] = new String("坎 苦咸涩-水涩寒冷冻黑 紫红");
        commonRotClass.inputs[5] = new String("乾 辣涩干凉-金赤");
        commonRotClass.inputs[6] = new String("兑 辣涩干凉热燥-金白辛");
        commonRotClass.inputs[7] = new String("坤 腻涩干凉-土棕");
    }
    bootCommonMap("八纲", true, commonRotClass);
    buttonBaGang.setBackground(Color.gray);
}

commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("八纲", commonRotClass);
});

Box sliderBoxBaGang = new Box(BoxLayout.X_AXIS);
sliderBoxBaGang.add(buttonBaGang);
sliderBoxBaGang.add(sliderBaGang);
sliderBoxBaGang.add(sliderBaGangZ);
//sliderBoxRight.add(new )
sliderBaYou = I_DetaSlider("八酉", true, false);
sliderBaYouZ = I_DetaSlider("八酉", false, true);
buttonBaYou = new DetaButton("八酉关", 100, 50, Color.pink);
buttonBaYou.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("八酉")) {
        commonRotClass = rootClassMap.getCommonRotClass("八酉");
    }
    if (commonRotClass.buttonClicked) {
        buttonBaYou.setBackground(Color.blue);
        buttonBaYou.setText("八酉关");
    } else {
        buttonBaYou.setText("八酉开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("酰 离 DH 清润核酸↑ Ⅲ");
            commonRotClass.inputs[1] = new String("酸 巽 QA 腐蚀调节↘ V");
            commonRotClass.inputs[2] = new String("酮 震 IP 激素扩散← IV");

```

```

        commonRotClass.inputs[3] = new String("酯 艮 TS 体能聚能↗ VII");
        commonRotClass.inputs[4] = new String("酚 坎 OU 受体免疫↓ VI");
        commonRotClass.inputs[5] = new String("醚 乾 VM 受体诱导↘ I");
        commonRotClass.inputs[6] = new String("醇 兑 CX 扩散免疫→ II");
        commonRotClass.inputs[7] = new String("酶 坤 DD E 腐蚀诱导↗ VIII");
    }

    bootCommonMap("八酉", true, commonRotClass);
    buttonBaYou.setBackground(Color.gray);
}

commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("八酉", commonRotClass);
});

Box sliderBoxBaYou = new Box(BoxLayout.X_AXIS);
sliderBoxBaYou.add(buttonBaYou);
sliderBoxBaYou.add(sliderBaYou);
sliderBoxBaYou.add(sliderBaYouZ);
//
sliderYuanJi = I_DetaSlider("阴阳", true, false);
sliderYuanJiZ = I_DetaSlider("阴阳", false, true);
buttonYuanJi = new DetaButton("元基关", 100, 50, Color.green);
buttonYuanJi.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("阴阳")) {
        commonRotClass = rootClassMap.getCommonRotClass("阴阳");
    }
    if (commonRotClass.buttonClicked) {
        buttonYuanJi.setBackground(Color.blue);
        buttonYuanJi.setText("元基关");
    } else {
        buttonYuanJi.setText("元基开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("PV 景 酰 离 ↑ III");
            commonRotClass.inputs[1] = new String("UH 杜 酮 巽 ↘ V");
            commonRotClass.inputs[2] = new String("OE 殇 醇 震 ← IV");
            commonRotClass.inputs[3] = new String("IX 生 酯 艮 ↗ VII");
            commonRotClass.inputs[4] = new String("MS 休 醚 坎 ↓ VI");
            commonRotClass.inputs[5] = new String("Q 开 酶 乾 ↘ I");
            commonRotClass.inputs[6] = new String("ACT 惊 酚 兑 → II");
            commonRotClass.inputs[7] = new String("D 殒 酸 坤 ↗ VIII");
        }
        bootCommonMap("阴阳", true, commonRotClass);
        buttonYuanJi.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("阴阳", commonRotClass);
});

```

```

});

Box sliderBoxYuanJi = new Box(BoxLayout.X_AXIS);
sliderBoxYuanJi.add(buttonYuanJi);
sliderBoxYuanJi.add(sliderYuanJi);
sliderBoxYuanJi.add(sliderYuanJiZ);

//
sliderShuangYuan = I_DetaSlider("双元", true, false);
sliderShuangYuanZ = I_DetaSlider("双元", false, true);
buttonShuangYuan = new DetaButton("双元关", 100, 50, Color.green);
buttonShuangYuan.addActionListener(e -> {

    //to do..

    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("双元")) {
        commonRotClass = rootClassMap.getCommonRotClass("双元");
    }
    if (commonRotClass.buttonClicked) {
        buttonShuangYuan.setBackground(Color.blue);
        buttonShuangYuan.setText("双元关");
    } else {
        buttonShuangYuan.setText("双元开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("DH 景酰离↑Ⅲ");
            commonRotClass.inputs[1] = new String("QA 杜酸巽↘Ⅴ");
            commonRotClass.inputs[2] = new String("PI 殇酮震←Ⅳ");
            commonRotClass.inputs[3] = new String("ST 生酯艮↙Ⅶ");
            commonRotClass.inputs[4] = new String("UO 休酚坎↓Ⅵ");
            commonRotClass.inputs[5] = new String("MV 开醚乾↘Ⅰ");
            commonRotClass.inputs[6] = new String("CX 惊醇兑→Ⅱ");
            commonRotClass.inputs[7] = new String("DD E 殒酶坤↗Ⅷ");
        }
        bootCommonMap("双元", true, commonRotClass);
        buttonShuangYuan.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("双元", commonRotClass);
});

Box sliderBoxShuangYuan = new Box(BoxLayout.X_AXIS);
sliderBoxShuangYuan.add(buttonShuangYuan);
sliderBoxShuangYuan.add(sliderShuangYuan);
sliderBoxShuangYuan.add(sliderShuangYuanZ);
sliderJingLuo = I_DetaSlider("经络", true, false);
sliderJingLuoZ = I_DetaSlider("经络", false, true);
buttonJingLuo = new DetaButton("经络关", 100, 50, Color.blue);
buttonJingLuo.addActionListener(e -> {

    //to do..

    CommonRotClass commonRotClass = new CommonRotClass();

```

```

    if (rootClassMap.containsKey("经络")) {
        commonRotClass = rootClassMap.getCommonRotClass("经络");
    }
    if (commonRotClass.buttonClicked) {
        buttonJingLuo.setBackground(Color.blue);
        buttonJingLuo.setText("经络关");
    } else {
        buttonJingLuo.setText("经络开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("XJ 离 手少阳 手少阴 手厥阴 手太阳 ↑ Ⅲ");
            commonRotClass.inputs[1] = new String("WS YQ 巽 足少阳 \ V");
            commonRotClass.inputs[2] = new String("YQ MG 震 足厥阴 ← IV");
            commonRotClass.inputs[3] = new String("MG 艮 手少阳 足阳明 ↙ Ⅶ");
            commonRotClass.inputs[4] = new String("JA 坎 足少阴 足太阴 手少阳 ↓ VI");
            commonRotClass.inputs[5] = new String("XH FG 乾 手阳明 \ I");
            commonRotClass.inputs[6] = new String("LZ SX 兑 手太阴 → II");
            commonRotClass.inputs[7] = new String("SX JZ 坤 足太阴 ↗ Ⅷ");
        }
        bootCommonMap("经络", true, commonRotClass);
        buttonJingLuo.setBackground(Color.gray);
    }

    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("经络", commonRotClass);
});

Box sliderBoxJingLuo = new Box(BoxLayout.X_AXIS);
sliderBoxJingLuo.add(buttonJingLuo);
sliderBoxJingLuo.add(sliderJingLuo);
sliderBoxJingLuo.add(sliderJingLuoZ);

//
//sliderBoxRight

sliderHeLuo = I_DetaSlider("河洛", true, false);
sliderHeLuoZ = I_DetaSlider("河洛", false, true);
buttonHeLuo = new DetaButton("河洛关", 100, 50, Color.green);
buttonHeLuo.addActionListener(e -> {
    //to do..

    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("河洛")) {
        commonRotClass = rootClassMap.getCommonRotClass("河洛");
    }
    if (commonRotClass.buttonClicked) {
        buttonHeLuo.setBackground(Color.blue);
        buttonHeLuo.setText("河洛关");
    } else {
        buttonHeLuo.setText("河洛开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[12];

```

```

        commonRotClass.inputs[0] = new String("03-白羊-土-3m21_4m19");
        commonRotClass.inputs[1] = new String("04-金牛-金-4m20_5m20");
        commonRotClass.inputs[2] = new String("05-双子-金-5m21_6m21");
        commonRotClass.inputs[3] = new String("06-巨蟹-土-6m22_7m22");
        commonRotClass.inputs[4] = new String("07-狮子-火-7m23_8m22");
        commonRotClass.inputs[5] = new String("08-处女-火-8m23_9m22");
        commonRotClass.inputs[6] = new String("09-天秤-土-9m23_10m23");
        commonRotClass.inputs[7] = new String("10-天蝎-木-10m24_11m22");
        commonRotClass.inputs[8] = new String("11-射手-木-11m23_12m21");
        commonRotClass.inputs[9] = new String("12-魔蝎-土-12m22_1m19");
        commonRotClass.inputs[10] = new String("01-水瓶-水-1m20_2m18");
        commonRotClass.inputs[11] = new String("02-双鱼-水-2m19_3m20");
    }

    bootCommonMap("河洛", true, commonRotClass);
    buttonHeLuo.setBackground(Color.gray);
}

commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("河洛", commonRotClass);
});

Box sliderBoxHeLuo = new Box(BoxLayout.X_AXIS);
sliderBoxHeLuo.add(buttonHeLuo);
sliderBoxHeLuo.add(sliderHeLuo);
sliderBoxHeLuo.add(sliderHeLuoZ);

sliderLiuRen = I_DetaSlider("六壬", true, false);
sliderLiuRenZ = I_DetaSlider("六壬", false, true);
buttonLiuRen = new DetaButton("六壬关", 100, 50, Color.blue);
buttonLiuRen.addActionListener(e -> {
    //to do..

    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("六壬")) {
        commonRotClass = rootClassMap.getCommonRotClass("六壬");
    }

    if (commonRotClass.buttonClicked) {
        buttonLiuRen.setBackground(Color.red);
        buttonLiuRen.setText("六壬关");
    } else {
        buttonLiuRen.setText("六壬开");
    }
    if (null == commonRotClass.inputs) {
        commonRotClass.inputs = new String[12];
        commonRotClass.inputs[0] = new String("马-午丞-灵境-甲午刑-休加制");
        commonRotClass.inputs[1] = new String("蛇-巳云-新象-辰戌刑");
        commonRotClass.inputs[2] = new String("龙-辰云-新象-辰戌刑");
        commonRotClass.inputs[3] = new String("兔-卯力-凶伤-开惊加制");
        commonRotClass.inputs[4] = new String("虎-寅月-山亭-甲申刑-殇杜加制");
        commonRotClass.inputs[5] = new String("牛-丑月-山亭-甲申刑-殇杜加制");
        commonRotClass.inputs[6] = new String("鼠-子目-屯才-生死加制");
        commonRotClass.inputs[7] = new String("猪-亥口-阴池-丙丁奇门");
    }
});

```

```

        commonRotClass.inputs[8] = new String("狗-戌口-阴池-丙丁奇门");
        commonRotClass.inputs[9] = new String("鸡-酉早-水阁-丙丁奇门");
        commonRotClass.inputs[10] = new String("猴-申因-枯木-甲戌刑");
        commonRotClass.inputs[11] = new String("羊-未因-枯木-甲戌刑");
    }
    bootCommonMap("六壬", true, commonRotClass);
    buttonLiuRen.setBackground(Color.gray);
}

commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("六壬", commonRotClass);
});

Box sliderBoxLiuRen = new Box(BoxLayout.X_AXIS);
sliderBoxLiuRen.add(buttonLiuRen);
sliderBoxLiuRen.add(sliderLiuRen);
sliderBoxLiuRen.add(sliderLiuRenZ);
sliderLiuZhu = I_DetaSlider("流注", true, false);
sliderLiuZhuZ = I_DetaSlider("流注", false, true);
buttonLiuZhu = new DetaButton("流注关", 100, 50, Color.pink);
buttonLiuZhu.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("流注")) {
        commonRotClass = rootClassMap.getCommonRotClass("流注");
    }
    if (commonRotClass.buttonClicked) {
        buttonLiuZhu.setBackground(Color.blue);
        buttonLiuZhu.setText("流注关");
    } else {
        buttonLiuZhu.setText("流注开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("心 小肠 离 腕踝上 经");
            commonRotClass.inputs[1] = new String("胆 肝 巽 掌趾前 荣");
            commonRotClass.inputs[2] = new String("胆 肝 震 掌趾前 荣");
            commonRotClass.inputs[3] = new String("血 胃 艮 肘膝中 合");
            commonRotClass.inputs[4] = new String("肾 膀胱 坎 掌趾后 输");
            commonRotClass.inputs[5] = new String("大肠 乾 手足端 井");
            commonRotClass.inputs[6] = new String("肺 兑 手足端 井");
            commonRotClass.inputs[7] = new String("血 脾 坤 肘膝中 合");
        }
        bootCommonMap("流注", true, commonRotClass);
        buttonLiuZhu.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("流注", commonRotClass);
});

Box sliderBoxLiuZhu = new Box(BoxLayout.X_AXIS);

```



```
sliderBoxLiuZhu.add(buttonLiuZhu);
sliderBoxLiuZhu.add(sliderLiuZhu);
sliderBoxLiuZhu.add(sliderLiuZhuZ);
sliderTianXing = I_DetaSlider("天星", true, false);
sliderTianXingZ = I_DetaSlider("天星", false, true);
buttonTianXing = new DetaButton("天星关", 100, 50, Color.green);
buttonTianXing.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("天星")) {
        commonRotClass = rootClassMap.getCommonRotClass("天星");
    }
    if (commonRotClass.buttonClicked) {
        buttonTianXing.setBackground(Color.blue);
        buttonTianXing.setText("天星关");
    } else {
        buttonTianXing.setText("天星开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[24];
            commonRotClass.inputs[0] = new String("午-天广-酉");
            commonRotClass.inputs[1] = new String("丙-太威-辛");
            commonRotClass.inputs[2] = new String("巳-天屏-酉");
            commonRotClass.inputs[3] = new String("巽-太乙-Kui");
            commonRotClass.inputs[4] = new String("辰-天罡-Li");
            commonRotClass.inputs[5] = new String("乙-天官-甲");
            commonRotClass.inputs[6] = new String("卯-天苑-丁");
            commonRotClass.inputs[7] = new String("甲-天苑-丙");
            commonRotClass.inputs[8] = new String("寅-天培-未");
            commonRotClass.inputs[9] = new String("艮-天市-丁");
            commonRotClass.inputs[10] = new String("丑-天厨-辰");
            commonRotClass.inputs[11] = new String("Kui-阴光-巳");
            commonRotClass.inputs[12] = new String("子-天垒-巳");
            commonRotClass.inputs[13] = new String("壬-天辅-申");
            commonRotClass.inputs[14] = new String("亥-天皇-乙");
            commonRotClass.inputs[15] = new String("乾-天庭-卯");
            commonRotClass.inputs[16] = new String("戌-天魁-丑");
            commonRotClass.inputs[17] = new String("辛-天乙-丑");
            commonRotClass.inputs[18] = new String("酉-少微-寅");
            commonRotClass.inputs[19] = new String("庚-天汉-午");
            commonRotClass.inputs[20] = new String("申-天关-Kui");
            commonRotClass.inputs[21] = new String("坤-天镜-乙");
            commonRotClass.inputs[22] = new String("未-天常-Kui");
            commonRotClass.inputs[23] = new String("丁-南极-寅");
        }
        bootCommonMap("天星", true, commonRotClass);
        buttonTianXing.setBackground(Color.gray);
    }
}
```

```
commonRotClass.buttonClicked = !commonRotClass.buttonClicked;

rootClassMap.put("天星", commonRotClass);

});

Box sliderBoxTianXing = new Box(BoxLayout.X_AXIS);
sliderBoxTianXing.add(buttonTianXing);
sliderBoxTianXing.add(sliderTianXing);
sliderBoxTianXing.add(sliderTianXingZ);

//
sliderYiShu = I_DetaSlider("易数", true, false);
sliderYiShuZ = I_DetaSlider("易数", false, true);
buttonYiShu = new DetaButton("易数关", 100, 50, Color.blue);
buttonYiShu.addActionListener(e -> {

    //to do..

    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("易数")) {
        commonRotClass = rootClassMap.getCommonRotClass("易数");
    }
    if (commonRotClass.buttonClicked) {
        buttonYiShu.setBackground(Color.blue);
        buttonYiShu.setText("易数关");
    } else {
        buttonYiShu.setText("易数开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("景-层乱枯紫稠急-疏同");
            commonRotClass.inputs[1] = new String("杜-散灿鲜靛急瀑");
            commonRotClass.inputs[2] = new String("殇-衣嫩蓝青微湍");
            commonRotClass.inputs[3] = new String("生-软柔黄珊细缓");
            commonRotClass.inputs[4] = new String("修-滑透墨辉绵平-枢密");
            commonRotClass.inputs[5] = new String("开-坚鲜浓红呼清");
            commonRotClass.inputs[6] = new String("精-纯淡红橙当浊");
            commonRotClass.inputs[7] = new String("陨-混浓棕黄刺污");
        }
        bootCommonMap("易数", true, commonRotClass);
        buttonYiShu.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("易数", commonRotClass);

});

Box sliderBoxYiShu = new Box(BoxLayout.X_AXIS);
sliderBoxYiShu.add(buttonYiShu);
sliderBoxYiShu.add(sliderYiShu);
sliderBoxYiShu.add(sliderYiShuZ);

//
sliderZiWu = I_DetaSlider("子午", true, false);
sliderZiWuZ = I_DetaSlider("子午", false, true);
buttonZiWu = new DetaButton("子午关", 100, 50, Color.pink);
```

```
buttonZiWu.addActionListener(e -> {  
    //to do..  
    CommonRotClass commonRotClass = new CommonRotClass();  
    if (rootClassMap.containsKey("子午")) {  
        commonRotClass = rootClassMap.getCommonRotClass("子午");  
    }  
    if (commonRotClass.buttonClicked) {  
        buttonZiWu.setBackground(Color.blue);  
        buttonZiWu.setText("子午关");  
    } else {  
        buttonZiWu.setText("子午开");  
        if (null == commonRotClass.inputs) {  
            commonRotClass.inputs = new String[8];  
            commonRotClass.inputs[0] = new String("XJ禁XH 离 胆");  
            commonRotClass.inputs[1] = new String("WS禁LZ 巽 胆 肝");  
            commonRotClass.inputs[2] = new String("YQ MG禁LZ 震 消化 胃");  
            commonRotClass.inputs[3] = new String("MG禁JA 艮 唾 胰");  
            commonRotClass.inputs[4] = new String("JA禁JZ 坎 发");  
            commonRotClass.inputs[5] = new String("XH FG禁XJ 乾 眠 泪");  
            commonRotClass.inputs[6] = new String("LZ SX禁JA 兑 寔");  
            commonRotClass.inputs[7] = new String("SX JZ禁JA 坤 鼻 汗");  
        }  
        bootCommonMap("子午", true, commonRotClass);  
        buttonZiWu.setBackground(Color.gray);  
    }  
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;  
    rootClassMap.put("子午", commonRotClass);  
});  
Box sliderBoxZiWu = new Box(BoxLayout.X_AXIS);  
sliderBoxZiWu.add(buttonZiWu);  
sliderBoxZiWu.add(sliderZiWu);  
sliderBoxZiWu.add(sliderZiWuZ);  
//  
sliderKanYu = I_DetaSlider("堪舆", true, false);  
sliderKanYuZ = I_DetaSlider("堪舆", false, true);  
buttonKanYu = new DetaButton("堪舆关", 100, 50, Color.green);  
buttonKanYu.addActionListener(e -> {  
    //to do..  
    CommonRotClass commonRotClass = new CommonRotClass();  
    if (rootClassMap.containsKey("堪舆")) {  
        commonRotClass = rootClassMap.getCommonRotClass("堪舆");  
    }  
    if (commonRotClass.buttonClicked) {  
        buttonKanYu.setBackground(Color.blue);  
        buttonKanYu.setText("堪舆关");  
    } else {  
        buttonKanYu.setText("堪舆开");
```

```

    if (null == commonRotClass.inputs) {
        commonRotClass.inputs = new String[11];
        commonRotClass.inputs[0] = new String("浆-花色-");
        commonRotClass.inputs[1] = new String("灰-曲直-");
        commonRotClass.inputs[2] = new String("林-呼怒-");
        commonRotClass.inputs[3] = new String("山-歌意-");
        commonRotClass.inputs[4] = new String("岸-禾蓄-");
        commonRotClass.inputs[5] = new String("北坎-河-恐呻");
        commonRotClass.inputs[6] = new String("溪-润健-");
        commonRotClass.inputs[7] = new String("沼-悦魄-");
        commonRotClass.inputs[8] = new String("石-商忧-");
        commonRotClass.inputs[9] = new String("岩-思顺-");
        commonRotClass.inputs[10] = new String("沙-官赤-");
    }
    bootCommonMap("堪輿", true, commonRotClass);
    buttonKanYu.setBackground(Color.gray);
}
commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("堪輿", commonRotClass);
});
Box sliderBoxKanYu = new Box(BoxLayout.X_AXIS);
sliderBoxKanYu.add(buttonKanYu);
sliderBoxKanYu.add(sliderKanYu);
sliderBoxKanYu.add(sliderKanYuZ);
//
sliderWuXing = I_DetaSlider("五行", true, false);
sliderWuXingZ = I_DetaSlider("五行", false, true);
buttonWuXing = new DetaButton("五行关", 100, 50, Color.BLUE);
//boolean buttonWuXingClicked= true;
buttonWuXing.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("五行")) {
        commonRotClass = rootClassMap.getCommonRotClass("五行");
    }
    if (commonRotClass.buttonClicked) {
        buttonWuXing.setBackground(Color.blue);
        buttonWuXing.setText("五行关");
    } else {
        buttonWuXing.setText("五行开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[5];
            commonRotClass.inputs[0] = new String("H PV DD 酰 火 ↑");
            commonRotClass.inputs[1] = new String("H OEU 醇酮 木 ↘");
            commonRotClass.inputs[2] = new String("H CDI 酯酸 土 ↗");
            commonRotClass.inputs[3] = new String("H MXS 醚 水 ↙");
            commonRotClass.inputs[4] = new String("H AQT 酚酶 金 ↘");

```

```
}
    bootCommonMap("五行", true, commonRotClass);
    buttonWuXing.setBackground(Color.gray);
}
commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("五行", commonRotClass);
});
Box sliderBoxWuXing = new Box(BoxLayout.X_AXIS);
sliderBoxWuXing.add(buttonWuXing);
sliderBoxWuXing.add(sliderWuXing);
sliderBoxWuXing.add(sliderWuXingZ);
//
sliderJieQi = I_DetaSlider("节气", true, false);
sliderJieQiZ = I_DetaSlider("节气", false, true);
buttonJieQi = new DetaButton("节气关", 100, 50, Color.PINK);
buttonJieQi.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("节气")) {
        commonRotClass = rootClassMap.getCommonRotClass("节气");
    }
    if (commonRotClass.buttonClicked) {
        buttonJieQi.setBackground(Color.blue);
        buttonJieQi.setText("节气关");
    } else {
        buttonJieQi.setText("节气开");
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[24];
            commonRotClass.inputs[0] = new String("夏至");
            commonRotClass.inputs[1] = new String("芒种");
            commonRotClass.inputs[2] = new String("小满");
            commonRotClass.inputs[3] = new String("立夏");
            commonRotClass.inputs[4] = new String("谷雨");
            commonRotClass.inputs[5] = new String("清明");
            commonRotClass.inputs[6] = new String("春分");
            commonRotClass.inputs[7] = new String("惊蛰");
            commonRotClass.inputs[8] = new String("雨水");
            commonRotClass.inputs[9] = new String("立春");
            commonRotClass.inputs[10] = new String("大寒");
            commonRotClass.inputs[11] = new String("小寒");
            commonRotClass.inputs[12] = new String("冬至");
            commonRotClass.inputs[13] = new String("大雪");
            commonRotClass.inputs[14] = new String("小雪");
            commonRotClass.inputs[15] = new String("立冬");
            commonRotClass.inputs[16] = new String("霜降");
            commonRotClass.inputs[17] = new String("寒露");
            commonRotClass.inputs[18] = new String("秋分");
```

```
        commonRotClass.inputs[19] = new String("白露");
        commonRotClass.inputs[20] = new String("处暑");
        commonRotClass.inputs[21] = new String("立秋");
        commonRotClass.inputs[22] = new String("大暑");
        commonRotClass.inputs[23] = new String("小暑");
    }

    bootCommonMap("节气", true, commonRotClass);
    buttonJieQi.setBackground(Color.gray);
}

commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("节气", commonRotClass);
});

Box sliderBoxJieQi = new Box(BoxLayout.X_AXIS);
sliderBoxJieQi.add(buttonJieQi);
sliderBoxJieQi.add(sliderJieQi);
sliderBoxJieQi.add(sliderJieQiZ);
//
sliderQuDu = I_DetaSlider("导入一", true, false);
sliderQuDuZ = I_DetaSlider("导入一", false, true);
buttonQuDu = new DetaButton("导入关", 100, 50, Color.BLUE);
buttonQuDu.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //to do..
        CommonRotClass commonRotClass = new CommonRotClass();
        if (rootClassMap.containsKey("导入一")) {
            commonRotClass = rootClassMap.getCommonRotClass("导入一");
        }
        if (commonRotClass.buttonClicked) {
            buttonQuDu.setBackground(Color.blue);
            buttonQuDu.setText("导入关");
        } else {
            buttonQuDu.setText("导入开");
            //read
            List<String> list = GetCompassList.doGetCompassList();
            //get
            commonRotClass.inputs = new String[list.size()];
            for (int i = 0; i < list.size(); i++) {
                commonRotClass.inputs[i] = list.get(i).toString();
            }
            bootCommonMap("导入一", true, commonRotClass);
            buttonQuDu.setBackground(Color.gray);
        }
        commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
        rootClassMap.put("导入一", commonRotClass);
    }
});

Box sliderBoxQuDu = new Box(BoxLayout.X_AXIS);
```

```
sliderBoxQuDu.add(buttonQuDu);
sliderBoxQuDu.add(sliderQuDu);
sliderBoxQuDu.add(sliderQuDuZ);
//
sliderXianWei = I_DetaSlider("导入二", true, false);
sliderXianWeiZ = I_DetaSlider("导入二", false, true);
buttonXianWei = new DetaButton("导入二", 100, 50, Color.PINK);
buttonXianWei.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //to do..
        CommonRotClass commonRotClass = new CommonRotClass();
        if (rootClassMap.containsKey("导入二")) {
            commonRotClass = rootClassMap.getCommonRotClass("导入二");
        }
        if (commonRotClass.buttonClicked) {
            buttonXianWei.setBackground(Color.blue);
        } else {
            //read
            List<String> list = GetCompassList.doGetCompassList();
            //get
            commonRotClass.inputs = new String[list.size()];
            for (int i = 0; i < list.size(); i++) {
                commonRotClass.inputs[i] = list.get(i).toString();
            }
            bootCommonMap("导入二", true, commonRotClass);
            buttonXianWei.setBackground(Color.gray);
        }
        commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
        rootClassMap.put("导入二", commonRotClass);
    }
});
Box sliderBoxXianWei = new Box(BoxLayout.X_AXIS);
sliderBoxXianWei.add(buttonXianWei);
sliderBoxXianWei.add(sliderXianWei);
sliderBoxXianWei.add(sliderXianWeiZ);
sliderYinWei = I_DetaSlider("导入三", true, false);
sliderYinWeiZ = I_DetaSlider("导入三", false, true);
buttonYinWei = new DetaButton("导入三", 100, 50, Color.pink);
buttonYinWei.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("导入三")) {
        commonRotClass = rootClassMap.getCommonRotClass("导入三");
    }
    if (commonRotClass.buttonClicked) {
        buttonYinWei.setBackground(Color.blue);
    } else {
```

```
//read
List<String> list = GetCompassList.doGetCompassList();
//get
commonRotClass.inputs = new String[list.size()];
for (int i = 0; i < list.size(); i++) {
    commonRotClass.inputs[i] = list.get(i).toString();
}
bootCommonMap("导入三", true, commonRotClass);
buttonYinWei.setBackground(Color.gray);
}
commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("导入三", commonRotClass);
});
Box sliderBoxYinWei = new Box(BoxLayout.X_AXIS);
sliderBoxYinWei.add(buttonYinWei);
sliderBoxYinWei.add(sliderYinWei);
sliderBoxYinWei.add(sliderYinWeiZ);
//
sliderXueWei = I_DetaSlider("导入四", true, false);
sliderXueWeiZ = I_DetaSlider("导入四", false, true);
buttonXueWei = new DetaButton("导入四", 100, 50, Color.pink);
buttonXueWei.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("导入四")) {
        commonRotClass = rootClassMap.getCommonRotClass("导入四");
    }
    if (commonRotClass.buttonClicked) {
        buttonXueWei.setBackground(Color.blue);
    } else {
        //read
        List<String> list = GetCompassList.doGetCompassList();
        //get
        commonRotClass.inputs = new String[list.size()];
        for (int i = 0; i < list.size(); i++) {
            commonRotClass.inputs[i] = list.get(i).toString();
        }
        bootCommonMap("导入四", true, commonRotClass);
        buttonXueWei.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("导入四", commonRotClass);
});
Box sliderBoxXueWei = new Box(BoxLayout.X_AXIS);
sliderBoxXueWei.add(buttonXueWei);
sliderBoxXueWei.add(sliderXueWei);
sliderBoxXueWei.add(sliderXueWeiZ);
```



```
//
slider_five = I_DetaSlider("_five", true, false);
slider_fiveZ = I_DetaSlider("_five", false, true);
button_five = new DetaButton("_five", 100, 50, Color.pink);
button_five.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("_five")) {
        commonRotClass = rootClassMap.getCommonRotClass("_five");
    }
    if (commonRotClass.buttonClicked) {
        button_five.setBackground(Color.blue);
    } else {
        //read
        List<String> list = GetCompassList.doGetCompassList();
        //get
        commonRotClass.inputs = new String[list.size()];
        for (int i = 0; i < list.size(); i++) {
            commonRotClass.inputs[i] = list.get(i).toString();
        }
        bootCommonMap("_five", true, commonRotClass);
        button_five.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("_five", commonRotClass);
});
Box sliderBox_five = new Box(BoxLayout.X_AXIS);
sliderBox_five.add(button_five);
sliderBox_five.add(slider_five);
sliderBox_five.add(slider_fiveZ);
slider_six = I_DetaSlider("_six", true, false);
slider_sixZ = I_DetaSlider("_six", false, true);
button_six = new DetaButton("_six", 100, 50, Color.pink);
button_six.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("_six")) {
        commonRotClass = rootClassMap.getCommonRotClass("_six");
    }
    if (commonRotClass.buttonClicked) {
        button_six.setBackground(Color.blue);
    } else {
        //read
        List<String> list = GetCompassList.doGetCompassList();
        //get
        commonRotClass.inputs = new String[list.size()];
        for (int i = 0; i < list.size(); i++) {
```

```
        commonRotClass.inputs[i] = list.get(i).toString();
    }
    bootCommonMap("_six", true, commonRotClass);
    button_six.setBackground(Color.gray);
}
commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("_six", commonRotClass);
});
Box sliderBox_six = new Box(BoxLayout.X_AXIS);
sliderBox_six.add(button_six);
sliderBox_six.add(slider_six);
sliderBox_six.add(slider_sixZ);
//稍后缩进
slider_seven = I_DetaSlider("_seven", true, false);
slider_sevenZ = I_DetaSlider("_seven", false, true);
button_seven = new DetaButton("_seven", 100, 50, Color.pink);
button_seven.addActionListener(e -> {
    //to do..
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("_seven")) {
        commonRotClass = rootClassMap.getCommonRotClass("_seven");
    }
    if (commonRotClass.buttonClicked) {
        button_seven.setBackground(Color.blue);
    } else {
        //read
        List<String> list = GetCompassList.doGetCompassList();
        //get
        commonRotClass.inputs = new String[list.size()];
        for (int i = 0; i < list.size(); i++) {
            commonRotClass.inputs[i] = list.get(i).toString();
        }
        bootCommonMap("_seven", true, commonRotClass);
        button_seven.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("_seven", commonRotClass);
});
Box sliderBox_seven = new Box(BoxLayout.X_AXIS);
sliderBox_seven.add(button_seven);
sliderBox_seven.add(slider_seven);
sliderBox_seven.add(slider_sevenZ);
//
sliderJingYi = I_DetaSlider("导入八", true, false);
sliderJingYiZ = I_DetaSlider("导入八", false, true);
buttonJingYi = new DetaButton("导入八", 100, 50, Color.green);
buttonJingYi.addActionListener(e -> {
```

```
//to do..
CommonRotClass commonRotClass = new CommonRotClass();
if (rootClassMap.containsKey("导入八")) {
    commonRotClass = rootClassMap.getCommonRotClass("导入八");
}
if (commonRotClass.buttonClicked) {
    buttonJingYi.setBackground(Color.blue);
} else {
    //read
    List<String> list = GetCompassList.doGetCompassList();
    //get
    commonRotClass.inputs = new String[list.size()];
    for (int i = 0; i < list.size(); i++) {
        commonRotClass.inputs[i] = list.get(i).toString();
    }
    bootCommonMap("导入八", true, commonRotClass);
    buttonJingYi.setBackground(Color.gray);
}
commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("导入八", commonRotClass);
});
Box sliderBoxJingYi = new Box(BoxLayout.X_AXIS);
sliderBoxJingYi.add(buttonJingYi);
sliderBoxJingYi.add(sliderJingYi);
sliderBoxJingYi.add(sliderJingYiZ);
//
sliderBoxRight.add(sliderBoxFangDa);
sliderBoxRight.add(sliderBoxSuoXiao);
sliderBoxRight.add(sliderBoxShunLiu);
sliderBoxRight.add(sliderBoxNiLiu);
sliderBoxRight.add(sliderBoxSanJiao);
sliderBoxRight.add(sliderBoxBaGang);
sliderBoxRight.add(sliderBoxBaYou);
sliderBoxRight.add(sliderBoxYuanJi);
sliderBoxRight.add(sliderBoxShuangYuan);
sliderBoxRight.add(sliderBoxJingLuo);
sliderBoxRight.add(sliderBoxHeLuo);
sliderBoxRight.add(sliderBoxLiuRen);
sliderBoxRight.add(sliderBoxLiuZhu);
sliderBoxRight.add(sliderBoxTianXing);
sliderBoxRight.add(sliderBoxYiShu);
sliderBoxRight.add(sliderBoxZiWu);
sliderBoxRight.add(sliderBoxKanYu);
sliderBoxRight.add(sliderBoxWuXing);
sliderBoxRight.add(sliderBoxJieQi);
sliderBoxRight.add(sliderBoxQuDu);
sliderBoxRight.setBounds(720, 0, 300, 780);
```

```
        return sliderBoxRight;
    }

    public JSlider createSlider() {
        return null;
        //稍后优化。。~
    }

    public Box creatOperationsBox() {
        sliderx = new DetaSlider(0, 1500);
        sliderx.setSnapToTicks(true);
        sliderx.setPaintTicks(true);
        sliderx.setMajorTickSpacing(20);
        sliderx.setMinorTickSpacing(1);
        sliderx.addChangeListener(event -> {
            JSlider source = (JSlider) event.getSource();
            xrot = source.getValue() << 2;
        });
        DetaThread.sleepDeta(100);
        slidery = new DetaSlider(0, 1500);
        slidery.setSnapToTicks(true);
        slidery.setPaintTicks(true);
        slidery.setMajorTickSpacing(20);
        slidery.setMinorTickSpacing(1);
        slidery.addChangeListener(event -> {
            JSlider source = (JSlider) event.getSource();
            yrot = source.getValue() << 2;
        });
        DetaThread.sleepDeta(100);
        sliderz = new DetaSlider(0, 1500);
        sliderz.setSnapToTicks(true);
        sliderz.setPaintTicks(true);
        sliderz.setMajorTickSpacing(20);
        sliderz.setMinorTickSpacing(1);
        sliderz.addChangeListener(event -> {
            JSlider source = (JSlider) event.getSource();
            zrot = source.getValue() << 2;
        });
        DetaThread.sleepDeta(100);
        slidert = new DetaSlider(1, 150);
        slidert.setSnapToTicks(true);
        slidert.setPaintTicks(true);
        slidert.setMajorTickSpacing(20);
        slidert.setMinorTickSpacing(1);
        slidert.addChangeListener(event -> {
            JSlider source = (JSlider) event.getSource();
            irot = source.getValue() << 2;
        });
        top = new DetaButton("top", 100, 50, Color.pink);
```

```

top.addActionListener(e -> brot += 0.5);

down = new DetaButton("down", 100, 50, Color.orange);
down.addActionListener(e -> brot -= 0.5);

left = new DetaButton("left", 100, 50, Color.pink);
left.addActionListener(e -> lrot -= 0.5);

right = new DetaButton("right", 100, 50, Color.green);
right.addActionListener(e -> lrot += 0.5);

bhl = new DetaButton("元基五行", 100, 50, Color.pink);
bhl.addActionListener(e -> {

    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("五行")) {
        commonRotClass = rootClassMap.getCommonRotClass("五行");
    }
    if (commonRotClass.buttonClicked) {
        buttonSanJiao.setBackground(Color.blue);
    } else {
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[5];
            commonRotClass.inputs[0] = new String("H PV DD 酰 火 ↑");
            commonRotClass.inputs[1] = new String("H OEU 醇酮 木 ↖");
            commonRotClass.inputs[2] = new String("H CDI 酯酸 土 ↗");
            commonRotClass.inputs[3] = new String("H MXS 醚 水 ↘");
            commonRotClass.inputs[4] = new String("H AQT 酚酶 金 ↙");
        }
        bootCommonMap("五行", true, commonRotClass);
        buttonSanJiao.setBackground(Color.gray);
    }

    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("五行", commonRotClass);
});

cfc = new DetaButton("双元", 100, 50, Color.orange);
cfc.addActionListener(e -> {

    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("双元")) {
        commonRotClass = rootClassMap.getCommonRotClass("双元");
    }
    if (commonRotClass.buttonClicked) {
        buttonSanJiao.setBackground(Color.blue);
    } else {
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("DH 景酰 离 ↑ III");
            commonRotClass.inputs[1] = new String("QA 杜酸 巽 ↖ V");
            commonRotClass.inputs[2] = new String("PI 殇酮 震 ← IV");
            commonRotClass.inputs[3] = new String("ST 生酯 艮 ↘ VII");
            commonRotClass.inputs[4] = new String("UO 休酚 坎 ↓ VI");
            commonRotClass.inputs[5] = new String("MV 开醚 乾 ↙ I");

```

```

        commonRotClass.inputs[6] = new String("CX 惊 醇 兑→ Ⅱ");
        commonRotClass.inputs[7] = new String("DD E 殒 酶 坤↗ Ⅷ");
    }
    bootCommonMap("双元", true, commonRotClass);
    buttonSanJiao.setBackground(Color.gray);
}
commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
rootClassMap.put("双元", commonRotClass);
});

whh = new DetaButton("元基八卦", 200, 50, Color.magenta);
whh.addActionListener(e -> {
    CommonRotClass commonRotClass = new CommonRotClass();
    if (rootClassMap.containsKey("阴阳")) {
        commonRotClass = rootClassMap.getCommonRotClass("阴阳");
    }
    if (commonRotClass.buttonClicked) {
        buttonSanJiao.setBackground(Color.blue);
    } else {
        if (null == commonRotClass.inputs) {
            commonRotClass.inputs = new String[8];
            commonRotClass.inputs[0] = new String("PV 景 酰 离↑ Ⅲ");
            commonRotClass.inputs[1] = new String("UH 杜 酮 巽↘ V");
            commonRotClass.inputs[2] = new String("OE 殇 醇 震← IV");
            commonRotClass.inputs[3] = new String("IX 生 酯 艮↙ Ⅶ");
            commonRotClass.inputs[4] = new String("MS 休 醚 坎↓ VI");
            commonRotClass.inputs[5] = new String("Q 开 酶 乾↘ I");
            commonRotClass.inputs[6] = new String("ACT 惊 酚 兑→ Ⅱ");
            commonRotClass.inputs[7] = new String("D 殒 酸 坤↗ Ⅷ");
        }
        bootCommonMap("阴阳", true, commonRotClass);
        buttonSanJiao.setBackground(Color.gray);
    }
    commonRotClass.buttonClicked = !commonRotClass.buttonClicked;
    rootClassMap.put("阴阳", commonRotClass);
});

rst = new DetaButton("重置", 200, 50, Color.pink);
rst.addActionListener(e -> {
    xrot = -0.0f;
    yrot = -0.0f;
    zrot = 360.0f;
    trot = 0.0f;
    prot = 1.0f;
    lrot = -1.5f;
    brot = -0.5f;
    irot = 78.0f;
    srot = 0.7f;
});

```

```
save = new DetaButton("保存", 200, 50, Color.green);  
// rst.setBounds(0, 0, 100, 20);  
save.addActionListener(e -> {  
    xrotr = xrot;  
    yrotr = yrot;  
    zrotr = zrot;  
    trotr = trot;  
    protr = prot;  
    lrotr = lrot;  
    brotr = brot;  
    irotr = irot;  
    srotr = srot;  
});  
load = new DetaButton("读取", 200, 50, Color.magenta);  
// rst.setBounds(0, 0, 100, 20);  
load.addActionListener(e -> {  
    xrot = xrotr;  
    yrot = yrotr;  
    zrot = zrotr;  
    trot = trotr;  
    prot = protr;  
    lrot = lrotr;  
    brot = brotr;  
    irot = irotr;  
    srot = srotr;  
});  
buttonBox.add(top);  
buttonBox.add(down);  
buttonBox.add(left);  
DetaThread.sleepDeta(100);  
buttonBox.add(right);  
buttonBox.add(whh);  
buttonBox.add(bhl);  
buttonBox.add(cfc);  
buttonBox.add(rst);  
buttonBox.add(save);  
//buttonBox.add(load);  
Box buttonBoxX = new Box(BoxLayout.X_AXIS);  
JButton xzb = new DetaButton("X坐标", 200, 50, Color.pink);  
buttonBoxX.add(xzb);  
buttonBoxX.add(sliderx);  
sliderBoxx1.add(buttonBoxX);  
Box buttonBoxY = new Box(BoxLayout.X_AXIS);  
JButton yzb = new DetaButton("Y坐标", 200, 50, Color.pink);  
buttonBoxY.add(yzb);  
buttonBoxY.add(slidery);  
sliderBoxx1.add(buttonBoxY);
```

```

Box buttonBoxZ = new Box(BoxLayout.X_AXIS);
JButton zzb = new JButton("Z坐标", 200, 50, Color.pink);
buttonBoxZ.add(zzb);
buttonBoxZ.add(sliderz);
sliderBoxx2.add(buttonBoxZ);
Box buttonBoxT = new Box(BoxLayout.X_AXIS);
JButton tzb = new JButton("T坐标", 200, 50, Color.pink);
buttonBoxT.add(tzb);
buttonBoxT.add(slidert);
sliderBoxx2.add(buttonBoxT);
sliderBox.add(sliderBoxx1);
sliderBox.add(sliderBoxx2);
sliderBox.add(buttonBox);
sliderBox.setBounds(0, 660, 718, 300);
return sliderBox;
}
}

//Trif_DNA-master/C-L-M/M-V/ME/APM/VSQ/xlx/CoAuthorForWord_X_SliderTest.java
//Trif_DNA-master/C-L-M/M-V/ME/APM/VSQ/xlx/CoAuthorForWord.java
package M_V.ME.APM.VSQ.xlx;

import ME.VPC.M.app.App;
import exception.thread.DetaThread;
import S_A.pheromone.IMV_SQL;
import com.jogamp.opengl.GL;
import com.jogamp.opengl.GL2;
import com.jogamp.opengl.GLAutoDrawable;
import com.jogamp.opengl.GLCapabilities;
import com.jogamp.opengl.GLEventListener;
import com.jogamp.opengl.GLProfile;
import com.jogamp.opengl.awt.GLCanvas;
import com.jogamp.opengl.glu.GLU;
import com.jogamp.opengl.util.FPSAnimator;
import javax.swing.JFrame;
import java.awt.Component;
import java.awt.Dimension;
import java.awt.HeadlessException;
import java.awt.Toolkit;
import java.awt.event.KeyListener;
import java.awt.event.MouseListener;
import java.awt.event.MouseMotionListener;
import java.awt.event.MouseWheelListener;
import java.util.LinkedHashMap;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342

```


* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class CoAuthorForWord extends CoAuthorForWord_Q implements MouseMotionListener

, MouseListener, MouseWheelListener, KeyListener, GLEventListener {

public CoAuthorForWord(App NE) throws HeadlessException {

DetaThread.sleepDeta(100);

rootWord = "";

rootLeaf = new LinkedHashMap<String, Object>();

findLeaf = new IMV_SQI();

frequencyLeaf = new IMV_SQI();

didLeaf = new IMV_SQI();

this._A = NE.app_S._A;

this.pos = NE.app_S.fMHMMListOneTime_E_X_S.posCnToCn;

this.u = NE;

this.setLayout(null);

DetaThread.sleepDeta(100);

GLProfile glp = GLProfile.getDefault();

GLCapabilities glcaps = new GLCapabilities(glp);

GLCanvas canvas = new GLCanvas(glcaps);

canvas.addGLEventListener(this);

canvas.addMouseListener(this);

DetaThread.sleepDeta(100);

canvas.addMouseMotionListener(this);

canvas.addKeyListener(this);

canvas.addMouseWheelListener(this);

canvas.setBounds(0, 0, 718, 660);

DetaThread.sleepDeta(100);

this.add(canvas);

animator = new FPSAnimator(canvas, 24, true);

centerWindow(this);

animator.start();

DetaThread.sleepDeta(100);

add(creatOperationsBox());

add(creatSlidersBox());

}

private void centerWindow(Component frame) {

Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();

Dimension frameSize = frame.getSize();

if (frameSize.width > screenSize.width)

frameSize.width = screenSize.width;

if (frameSize.height > screenSize.height)

frameSize.height = screenSize.height;

frame.setLocation((screenSize.width - frameSize.width) >> 1,

(screenSize.height - frameSize.height) >> 1);

```

}

@SuppressWarnings("static-access")
public void init(GLAutoDrawable drawable) {
    gl = drawable.getGL().getGL2();
    glu = new GLU();
    //glut= new GLUT();
    DetaThread.sleepDeta(100);
    DetaThread.sleepDeta(100);
    gl.glEnable(GL.GL_DEPTH_TEST);
    gl.glDepthFunc(GL.GL_LEQUAL);
    gl.glShadeModel(GL2.GL_SMOOTH);
    DetaThread.sleepDeta(100);
    gl.glClearColor(0f, 0f, 0f, 0f);
    gl.glHint(gl.GL_PERSPECTIVE_CORRECTION_HINT, gl.GL_NICEST);
}

public static void main(String[] args) throws HeadlessException {
    App NE= new App();
    CoAuthorForWord app = new CoAuthorForWord(NE);
    app.setSize(850, 500);
    app.setVisible(true);
    JFrame f = new JFrame();
    f.add(app);
    f.setTitle("ButtonDemo");
    f.setLocationRelativeTo(null);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setSize(650, 480);
    f.setVisible(true);
}
}

//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CoAuthorForWordTest.java
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/CommonRotClass.java
package M_V.ME.APM.VSQ.xlx;
import java.util.LinkedHashMap;
public class CommonRotClass {
    public boolean buttonClicked;
    public float rot;
    public float rotL;
    public float rotZ;
    public float rotZl;
    public LinkedHashMap<String, Object> rootLeaf;
    public String[] inputs;
}
//Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xlx/GetCompassList.java
package M_V.ME.APM.VSQ.xlx;

import exception.file.O.DetaBufferedReader;

import exception.file.O.DetaInputStreamReader;

import java.awt.FileDialog;

import java.awt.Frame;

import java.io.File;

import java.io.FileInputStream;

import java.io.IOException;

import java.util.ArrayList;

import java.util.List;

/*
* 个人著作权人, 作者 罗瑶光, 浏阳

```

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class GetCompassList {
    public static List<String> doGetCompassList() {
        List<String> list;
        try {
            list = new ArrayList<>();
            FileDialog filedialog = new FileDialog(new Frame()
                , "选择罗盘文件", FileDialog.LOAD);
            filedialog.setVisible(true);
            String lyglpFilePath = filedialog.getDirectory() + filedialog.getFile();
            File file;//配置文件在同一个文件架夹下便于管理。
            file = new File(lyglpFilePath);
            if (file.exists()) { // 上面的string稍后写在配置面板里面。
                DetaBufferedReader bufferedWriter;
                bufferedWriter = new DetaBufferedReader(DetaInputStreamReader.E
                    (new FileInputStream(file), "UTF8"));
                String line;
                while (true) {
                    if (!(line = bufferedWriter.readDetaLine()) != null)) break;
                    if (!line.isEmpty()) {
                        list.add(line);
                    }
                }
                bufferedWriter.close();
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        return list;
    }
}
```

/Trif_DNA-master/C_L_M/M_V/ME/APM/VSQ/xx/GetCompassListTest.java

M_V.ME.OVQ.OSQ.VSQ.obj

/Trif_DNA-master/C_L_M/M_V/ME/OVQ/OSQ/VSQ/obj/WordInfo.java

M_V.ME.SM.OP.SM.AOP.MEC.SIQ.E

/Trif_DNA-master/C_L_M/M_V/ME/SM/OP/SM/AOP/MEC/SIQ/E/E_PLSQL_E.java
package M_V.ME.SM.OP.SM.AOP.MEC.SIQ.E;

import M_V.ME.E.PLSQLCommand_E_X_P_BaseName;

import M_V.ME.E.PLSQLCommand_E_X_P_Check;

import M_V.ME.E.PLSQLCommand_E_X_P_E;

import M_V.ME.E.PLSQLCommand_E_X_P_Join;

```

import M_V.ME.E.PLSQLCommand_E_X_P_ListNeedStart;

import M_V.ME.E.PLSQLCommand_E_X_P_SetRoot;

import M_V.ME.E.PLSQLCommand_E_X_P_TableName;

import O_V.OSI.AOP.MEC.SIQ.plorm.PLORM_C;

import S_A.SVQ.stable.S_ShellETL;

import S_A.pheromone.IMV_SQI;

import S_I.OSI.PEI.PCI.PSI.tinShell.TinMap;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class E_PLSQL_E {

    public static IMV_SQI E_PLSQL(String plsqli, boolean mod) {

        //working for here
        TinMap output= new TinMap();

        //1make container
        output.put("start", "0");

        output.put("countJoins", "0");

        //2make line
        String[] commands= plsqli.replace(" ", "").replace("\n",
            "").split(";");

        String[] acknowledge= null;

        for(String command:commands) {

            acknowledge= command.split(":");

            if(acknowledge[0].equals("setRoot")) {

                PLSQLCommand_E_X_P_SetRoot._E(acknowledge);

            }

            if(acknowledge[0].equals("baseName")) {

                PLSQLCommand_E_X_P_BaseName._E(acknowledge, output);

            }

            if(acknowledge[0].equals("tableName")) {

                PLSQLCommand_E_X_P_TableName._E(acknowledge, output);

            }

            if(acknowledge[0].equals(S_ShellETL.SHELL_ETL_CULUMNNAME)) {

                PLSQLCommand_E_X_P_ListNeedStart._E(acknowledge, output);

            }

            if(acknowledge[0].equals("修改列名")) { //later

                PLSQLCommand_E_X_P_ListNeedStart._E(acknowledge, output);

            }

            if(acknowledge[0].equals("columnValue")) {

                PLSQLCommand_E_X_P_ListNeedStart._E(acknowledge, output);

            }

        }

    }

}

```

```

        if(acknowledge[0].equals("join")) {
            PLSQLCommand_E_X_P_Join._E(acknowledge, output);
        }
        if(acknowledge[0].equals(S_ShellETL.SHELL_ETL_CONDITION)) {
            PLSQLCommand_E_X_P_ListNeedStart._E(acknowledge, output);
        }
        if(acknowledge[0].equals("relation")) {
            PLSQLCommand_E_X_P_ListNeedStart._E(acknowledge, output);
        }
        if(acknowledge[0].equals("aggregation")) {
            PLSQLCommand_E_X_P_ListNeedStart._E(acknowledge, output);
        }
        if(acknowledge[0].equals("getCulums")) {
            PLSQLCommand_E_X_P_ListNeedStart._E(acknowledge, output);
        }
        output.put("newCommand", acknowledge[0]);
        PLSQLCommand_E_X_P_E._E(acknowledge, output, mod);
        output.put("lastCommand", output.get("newCommand"));
    }
    if(null!= acknowledge) {
        if(output.get("start").toString().equals("1")) {
            PLSQLCommand_E_X_P_E._E(acknowledge, output, mod);
        }
    }
    //20230106-S_logger.Log.logger.info("" + "1");
    PLSQLCommand_E_X_P_Check._E(output.get("newCommand").toString(), output, mod);
    return output;
}

public static IMV_SQI E_PLORM(PLORM_C orm, boolean b) {
    return E_PLSQL(orm.getPLSQL(), true);
}
}

```

//Trif_DNA-master/C_L_M/M_V/ME/SM/OP/SM/AOP/MEC/SIQ/E/E_PLSQL_ETest.java
//Trif_DNA-master/C_L_M/M_V/ME/SM/OP/SM/AOP/MEC/SIQ/E/P_AggregationPLSQL.java
package M_V.ME.SM.OP.SM.AOP.MEC.SIQ.E;

import S_A.pheromone.IMV_SQI;

import java.math.BigDecimal;

import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;

@SuppressWarnings({"unused"})

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)

** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class P_AggregationPLSQL {
    public static void P_AggregationLimitMap(String[] sets
        , List<IMV_SQI> output) {
        List<IMV_SQI> outputTemp = new ArrayList<>();
        Iterator<IMV_SQI> iterator = output.iterator();
        int count = 0;
        while (iterator.hasNext()) {
            int rowid = count++;
            IMV_SQI row = iterator.next();
            IMV_SQI rowMap = new IMV_SQI();
            if (sets[1].equalsIgnoreCase("~")) {
                if (rowid >= new BigDecimal(sets[0]).doubleValue() && rowid
                    <= new BigDecimal(sets[2]).doubleValue()) {
                    outputTemp.add(row);
                }
            }
        }
        output.clear();
        output.addAll(outputTemp);
    }
}

/Trif_DNA-master/C_L_M/M_V/ME/SM/OP/SM/AOP/MEC/SIQ/E/P_AggregationPLSQLTest.java
```

M_V.MPI.xls.commonXLS

/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXls.java
package M_V.MPI.xls.commonXLS;

```
import ME.VPC.E.sleeper.PageButtons;
import ME.VPC.E.sleeper.PageUtil;
import ME.VPC.M.app.App;
import ME.VPC.S.buttons.Page_X_S_sButtonsreading;
import ME.VPC.S.ne.WindowsUI;
import ME.VPC.V.appButton.AppButtonUtil_X_DNN_Classify;
import ME.VPC.V.appButton.AppButtonUtil_X_DNN_Page;
import M_V.MVQ.button.DetaButton;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.JLabel;
import javax.swing.JScrollPane;
import javax.swing.JSlider;
import javax.swing.JTextPane;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;
import java.awt.Color;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
```

```
import java.awt.event.MouseListener;

import java.util.Iterator;
import java.util.List;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class CommonXls extends CommonXlsPage_X_S_sListeners implements MouseListener {

    //
    public CommonXls(JTextPane text, String fileType, String pageName, App app_NE) {
        NE = app_NE;
        this.emotionSample = NE.app_S.emotionSample;
        this.ratioMap = NE.app_S.ratioMap_E;
        this.ratioMap_E = NE.app_S.ratioMap_E;
        this.sensingTest = NE.app_S.sensingTest;
        this.pageName = pageName;
        this.xlsFilePath = NE._I_U.tempString;
        app_S = NE.app_S;
        initPage(text, NE);
        this.setLayout(null);
        this.setBounds(0, 0, WindowsUI.makeContainerWidth
            , WindowsUI.makeContainerHeight);
        JScrollPane jsp_name = new JScrollPane(this.name());
        jsp_name.setBounds(100, 15, 680, 50);
        JScrollPane jsp_data = new JScrollPane(this.data(NE));
        JScrollPane jsp_statistic = new JScrollPane(this.statistic());
        JScrollPane jsp = new JScrollPane(this.jTable(fileType, NE));
        JLabel jlabel = new JLabel("主要词汇 \\\ 操作选项");
        PageUtil.initPageTail(jsp_statistic, jsp_data, jsp, jlabel);
        this.add(jsp);
        this.add(jlabel);
        this.add(jsp_data);
        this.add(jsp_statistic);
    }

    public void PageButtons() {
        PageButtons._I(this, NE);
    }

    public JTextPane data(App NE) {
        PageButtons();
        DetaButton buttonADD = new DetaButton("添加到编辑页");
        buttonADD.setBounds(868, 0, 115, 30);
        buttonADD.addActionListener(new ActionListener() {
```

```
public void actionPerformed(ActionEvent e) {
    if (sets == null) {
        return;
    }
    if (text.getText().length() > 5000) {
        return;
    }
    StringBuilder page = new StringBuilder();
    List<String> setsForGet
        = sets.subList(currentPage * 2000, (currentPage + 1) * 2000 < sets.size()
            ? (currentPage + 1) * 2000 : sets.size());
    Iterator<String> iterator = setsForGet.iterator();
    while (iterator.hasNext()) {
        String setOfi = iterator.next();
        page.append(setOfi);
    }
    if (!text.getText().isEmpty()) {
        text.setText(text.getText() + "\r\n\r\n" + page.toString());
    } else {
        text.setText(page.toString());
    }
    text.validate();
}

});

DetaButton buttonKSLJ = new DetaButton("中药DNN");
buttonKSLJ.setBounds(990, 0, 115, 30);
buttonKSLJ.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (null == sets) {
            return;
        }
        if (text.getText().length() > 5000) {
            return;
        }
        StringBuilder page = new StringBuilder().append("");
        AppButtonUtil_X_DNN_Page.pageDNN(page
            , sets, currentPage, app_S, table, row, key
            , buttonSum, buttonCrt, data, statistic, "中药", NE);
    }
});

DetaButton buttonKSLJB = new DetaButton("西药DNN");
buttonKSLJB.setBounds(990 + 115 + 7, 0, 115, 30);
buttonKSLJB.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (null == sets) {
            return;
        }
    }
});
```



```

        if (text.getText().length() > 5000) {
            return;
        }
        StringBuilder page = new StringBuilder().append("");
        AppButtonUtil_X_DNN_Page.pageDNN(page
            , sets, currentPage, app_S, table, row, key
            , buttonSum, buttonCrt, data, statistic, "西药", NE);
    }
});
//dnn游标
JSlider sliderDNN = new JSlider(0, 25);
sliderDNN.setSnapToTicks(true);
sliderDNN.setPaintTicks(true);
sliderDNN.setMajorTickSpacing(5);
sliderDNN.setMinorTickSpacing(1);
sliderDNN.addChangeListener(
    new ChangeListener() {
        public void stateChanged(ChangeEvent event) {
            JSlider source = (JSlider) event.getSource();
            app_S.jsliderDNNRatio = (double) source.getValue();
            if (null == sets) {
                return;
            }
            if (text.getText().length() > 5000) {
                return;
            }
            StringBuilder page = new StringBuilder().append("");
            AppButtonUtil_X_DNN_Classify.classifyDNN(page
                , sets, currentPage, table, row, key, buttonSum
                , buttonCrt, data, statistic, "西药", NE);
        }
    });
buttonCTV = Page_X_S_sButtonsreading.data_X_reading(this, app_S, NE);
Box buttonBox = new Box(BoxLayout.X_AXIS);
buttonBox.add(buttonPrev);
buttonBox.add(buttonNext);
buttonBox.add(buttonSum);
buttonBox.add(buttonCrt);
buttonBox.add(buttonCTE);
buttonBox.add(buttonFRS);
buttonBox.add(buttonETC);
buttonBox.add(buttonCTV);
buttonBox.add(buttonADD);
buttonBox.add(buttonKSLJ);
buttonBox.add(buttonKSLJB);
buttonBox.add(sliderDNN);
buttonBox.setBounds(5 + 800 - 650, 290 + 100 - 80 + 200 - 260 + 120

```

```

    , 950 + 220, 20);

    this.add(buttonBox);

    return data;

}

public JTextPane statistic() {
    statistic = new JTextPane();

    statistic.setBackground(new Color(210, 210, 210));

    statistic.setBounds(850, 150, WindowsUI.basicWidth - 840, 800);

    return statistic;
}
}

//67
//    data = new JTextPane();
//    data.setBounds(850, 150, WindowsUI.basicWidth - 600, 800 - 70);
//    data.setBackground(new Color(210, 210, 210));
//
//    buttonSum = new DetaButton("共有 " + (sets == null ? 0 : (1 + sets.size() / 2001)) + " 页");
//    buttonSum.setBounds(0, 0, 100, 30);
//    buttonCrt = new DetaButton("当前页面: " + (currentPage + 1));
//    buttonCrt.setBounds(120, 0, 150, 30);
//    buttonPrev = Page_X_S_sButtonsrollback.data_X_rollback(this
//        , 2000);
//    buttonNext = Page_X_S_sButtonsrollafter.data_X_rollafter(this
//        , 2000);
//    buttonCTE = Page_X_S_sButtonsenglish.data_X_english(this
//        , 630, 0, 100, 30, 2000);
//    buttonFRS = Page_X_S_sButtonschinese.data_X_chinese(this
//        , 520, 0, 100, 30, 2000);
//    buttonETC = Page_X_S_sButtonssymnorine.data_X_symnorine(this
//        , 740, 0, 100, 30, 2000);

//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXLSASC.java
//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXLSASCTest.java
//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXLSDNN.java
package M_V.MPI.xls.commonXLS;

import ME.VPC.M.app.App;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import S_A.pheromone.IMV_SQI;

import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CommonXLSDNN {

    public static Map<String, IMV_SQI> createXLSDNN(Object[][] tableData_old

```

```

    , int[] identifyColumns, Map<String, IMV_SQI> subDNNmap, App NE){
//init deta parser
SensingTest sensingTest = NE.app_S._A.getSensingTest();
//    DNNTest dNNTest = new DNNTest();
//    ANNTTest aNNTest = new ANNTTest();
//    RNN_IDETest rNN_IDETest = new RNN_IDETest();
//loop
try {
    if (null != subDNNmap) {
        if(null==tableData_old){
            return null;
        }
        for (int i = 0; i < tableData_old.length; i++) {
            S_logger.Log.logger.info("" + i);
            double perRatio = 0.0;
            String needDNNString = "";
            String stringIdentify = "";
            for (int j = 0; j < identifyColumns.length; j++) {
                stringIdentify += (null != tableData_old[i][j]
                    ? tableData_old[i][j] : "").toString();
            }//Map<String, Double> dnnSet= NE.app_S.DNNmap.get(stringIdentify);
            IMV_SQI dnnSet = new IMV_SQI();
            for (int j = 0; j < tableData_old[0].length; j++) {
                needDNNString += (null != tableData_old[i][j]
                    ? tableData_old[i][j] : "").toString();
            }//do dnn row
            String[][] ann;
            ann = NE.app_S.aNNTest.getANNMatrix(needDNNString, NE);
            if (null != ann) {
                String[][] dnn = NE.app_S.dNNTest.getDNNMatrix(ann
                    , needDNNString, NE);
                for (String[] dnnTupe : dnn) {
                    double ratio = Double.valueOf(dnnTupe[1]);
                    //ratio = ratio.isInfinite() ? 1 : ratio;
                    dnnSet.put("" + (null == dnnTupe[0] ? "" : dnnTupe[0]), ratio);
                    perRatio += ratio;
                }
                dnnSet.put("总分", perRatio);
                subDNNmap.put(stringIdentify, dnnSet);
            } else {
                dnnSet.put("总分", 1.0);
                subDNNmap.put(stringIdentify, dnnSet);
            }
        }
    }
} catch (Exception e) {

```

```

        S_logger.Log.logger.info("'" + "读表错误");
        e.printStackTrace();
    }
    return null;
}
}

//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXLSDNNTTest.java
//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXlsPage_X_S_s.java
//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXlsPage_X_S_sListeners.java
package M_V.MPI.xls.commonXLS;

import dnn.GetDNNMap;
import dnn.WriteDNNMap;
import ME.VPC.M.app.App;
import ME.VPC.M.X.AppSearch_X_DataSearchWithDNN;
import S_A.pheromone.IMV_SQL;
import U_V.ESU.office.XlsToYlJBufferPage;
import U_V.ESU.office.XlsxToYlJBufferPage;
import U_V.ESU.sort.Quick9DLYGWithString_ESU;
import javax.swing.JTextField;
import javax.swing.table.DefaultTableModel;
import java.awt.event.MouseEvent;
import java.awt.event.MouseListener;
import java.io.File;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CommonXlsPage_X_S_sListeners extends CommonXlsPage_X_S_s
    implements MouseListener {
    //
    public JTextField name() {
        name = new JTextField();
        name.setBounds(180, 50, 380, 80);
        return name;
    }
    @SuppressWarnings({"serial"})
    public javax.swing.JTable jTable(String fileType, App app_NE) {
        NE = app_NE;
        if (fileType.equalsIgnoreCase("xls")) {

```

```

        tableData_old = new XlsToYlJBufferPage().readNonObjectXls(this
            , 6, NE.app_S.appConfig.SectionJPanel.xlsWithSpec.isSelected());
    } else if (fileType.equalsIgnoreCase("xlsx")) {
        tableData_old = new XlsxToYlJBufferPage().readNonObjectXlsx(this
            , 6, NE.app_S.appConfig.SectionJPanel.xlsWithSpec.isSelected());
    }
    //default
    if (null == identifyColumns) {
        identifyColumns = new int[1];
        identifyColumns[0] = 2;
    }
    //如果有dnn表，读取即可。
    String dnnPath = xlsFilePath.replace(".xls", ".lygDnn");
    dnnPath = dnnPath.replace(".xls", ".lygDnn");
    File file;//配置文件在同一个文件架夹下便于管理。
    file = new File(dnnPath);
    Map<String, IMV_SQI> subDNNmap = new IMV_SQI();
    if (file.exists()) {
        GetDNNMap.doGetDNNMap(app_S, NE.app_S.DNNmap, dnnPath, NE);
    } else { //如果没有，就生成小表。
        subDNNmap = CommonXLSDNN.createXLSDNN(tableData_old, identifyColumns
            , subDNNmap, NE);
        //小表写入dnn，
        WriteDNNMap.doWriteDNNMap(dnnPath, NE, subDNNmap);
    }
    if (!subDNNmap.isEmpty()) { //小表并入dnn大表。
        Iterator<String> iterator = subDNNmap.keySet().iterator();
        while (iterator.hasNext()) {
            String string = iterator.next();
            IMV_SQI dnnSet = subDNNmap.get(string);
            NE.app_S.DNNmap.put(string, dnnSet);
        }
    }
    //CommonXLSDNN.createXLSDNN(tableData_old, identifyColumns);
    table = new javax.swing.JTable();
    newTableModel = new DefaultTableModel(tableData_old, columnTitle) {
        @Override
        public boolean isCellEditable(int row, int column) {
            return false;
        }
    };
    tableSections(NE);
    table.addMouseListener(this);
    return table;
}

@Override
public void mouseClicked(MouseEvent arg0) {
    simpleClicked(NE);

```

```
}

@Override
public void mouseEntered(MouseEvent arg0) {
}

@Override
public void mouseExited(MouseEvent arg0) {
}

@Override
public void mousePressed(MouseEvent arg0) {
}

@Override
public void mouseReleased(MouseEvent arg0) {
}

public void doSearch1() {
}

public void doSearch() {
    List<Object> tableData_oldListObjects = new ArrayList<>();
    String identifyParser = "_-_" ;
    if (null == tableData_old) {
        return;
    }
    Here:
    for (int i = 0; i < tableData_old.length; i++) {
        StringBuilder lineStringBuilder = new StringBuilder();
        for (int k = 0; k < columnTitle.length; k++) {
            if (0 != k) {
                lineStringBuilder.append(identifyParser);
            }
            if (null != tableData_old[i][k]) {
                lineStringBuilder.append(tableData_old[i][k]); // .toString();
            } else {
                lineStringBuilder.append(" ");
            }
        }
        //later do a function call
        int itemCount = NE.app_S.newFilterLine.getItemCount();
        String[] strings = new String[itemCount];
        for (int j = 0; j < strings.length; j++) {
            strings[j] = new String(NE.app_S.newFilterLine.getItemAt(j).toString());
        }
        boolean needDelete = false;
        boolean needAdd = true;
        for (int j = 0; j < strings.length; j++) {
            String[] filters = strings[j].split("-");
            if (filters[0].equalsIgnoreCase(pageName)) {
                for (int k = 0; k < columnTitle.length; k++) {
                    if (columnTitle[k].toString().equalsIgnoreCase(filters[1])) {
```

```

        if (null == tableData_old[i][k]) {
            tableData_old[i][k] = "";
            tableData_oldListObjects.add(lineStringBuilder);
            continue Here;
        }
        if (filters[2].contains("剔除")) {
            if (tableData_old[i][k].toString().contains(filters[3])) {
                needDelete = true;
            }
        } else if (filters[2].contains("保留")) {
            if (tableData_old[i][k].toString().contains(filters[3])) {
                needAdd = true;
            } else {
                needAdd = false;
            }
        } else {
            tableData_oldListObjects.add(lineStringBuilder);
            continue Here;
        }
    }
}

if (!needDelete & needAdd) {
    tableData_oldListObjects.add(lineStringBuilder);
}

continue Here;
}

if (null == key || key.isEmpty()) {
    newTableModel.getDataVector().clear();
    Iterator<Object> iterator = tableData_oldListObjects.iterator();
    int i = 0;
    while (iterator.hasNext()) {
        Object[] objects = iterator.next().toString().split(identifyParser);
        newTableModel.insertRow(i++, objects);
    }
    newTableModel.fireTableDataChanged();
    return;
}

String[] score = new String[tableData_oldListObjects.size()];
String[] scoreOutput = new String[tableData_oldListObjects.size()];
int[] score_code = new int[tableData_oldListObjects.size()]; //稍后精简 入参
int count = new AppSearch_X_DataSearchWithDNN().detaSimpleSearch(score_code, score
    , newTableModel, key, true, identifyColumns, fileNamePrefix, scoreOutput
    , tableData_oldListObjects, identifyParser, NE);
if (-1 == count) {
    return;
}

```

```

    }
    new Quick9DLYGWithString_ESU().sort(score_code, scoreOutput);
    Object[][] tableData = new Object[tableData_oldListObjects.size()][tableData_old[0].length];
    int new_count = 0;
    newTableModel.getDataVector().clear();
    if (null == key || key.equals("")) {
        for (int i = 0; i < tableData_old.length; i++) {
            newTableModel.insertRow(i, tableData_old[i]);
        }
        newTableModel.fireTableDataChanged();
        return;
    }
    //checkfilter
    Here:
    for (int i = tableData_oldListObjects.size() - 1; i > -1; i--) {
        if (score_code[i] < 1) {
            continue Here;
        }
        String[] columns = scoreOutput[i].split(identifyParser);
        tableData[new_count] = new Object[columns.length];
        for (int j = 0; j < columns.length; j++) {
            columns[j] = columns[j] == null ? "" : columns[j];
            tableData[new_count][j] = columns[j].toString();
        }
        tableData[new_count][1] = score_code[i];
        newTableModel.insertRow(new_count, tableData[new_count]);
        new_count += 1;
        continue Here;
    }
    newTableModel.fireTableDataChanged();
}

}

////
//int itemCount= NE.app_S.newFilterLine.getItemCount();
//String[] strings= new String[itemCount];
//for(int j= 0; j< strings.length; j++) {
//    strings[j]= new String(NE.app_S.newFilterLine.getItemAt(j).toString());
//}
//for(int j= 0; j< strings.length; j++) {
//    String[] filters= strings[j].split("-");
//    if(filters[0].equalsIgnoreCase(pageName)) { //find filters[1]index
//int column;
//for(int k= 0; k< columnTitle.length; k++) {
//    if(columnTitle[k].toString().equalsIgnoreCase(filters[1])) {
//        column= k;
//        if(null== tableData[new_count][column]) {
//            tableData[new_count][column]= "";

```

```
//
//          newTableModel.addRow(new_count, tableData[new_count]);
//
//          new_count+=1;
//
//          continue Here;
//
//      }
//
//      if(filters[2].contains("剔除")) {
//
//          if(tableData[new_count][column].toString().contains(filters[3])) {
//
//              continue Here;
//
//          }
//
//      }else if(filters[2].contains("保留")) {
//
//          if(tableData[new_count][column].toString().contains(filters[3])) {
//
//              newTableModel.addRow(new_count, tableData[new_count]);
//
//              new_count+=1;
//
//              continue Here;
//
//          }else {
//
//              continue Here;
//
//          }
//
//      }else {
//
//          newTableModel.addRow(new_count, tableData[new_count]);
//
//          new_count+=1;
//
//          continue Here;
//
//      }
//
//  }
//}
//
//  }
//}
//146
//
//      else { //我就知道有问题。
//
//          needDelete = false;
//
//      }
//
//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXlsPage_X_S_sListenersTest.java
//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/CommonXlsTest.java
//Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/MakeContainerXLS.java
package M_V.MPI.xls.commonXLS;

import A_V.E.RatioMap_E;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import ME.VPC.C.skivvy.TableRender;

import ME.VPC.M.app.App;

import ME.VPC.S.ne.App_S;

import M_V.MVQ.tabbedPane.DetabbedPane;

import O_V.OSA.shell.XA_ShellTables;

import P_V.PEQ.AMV.ECS.test.ANNTTest;

import P_V.PEQ.AMV.ECS.test.DNNTTest;

import P_V.PEQ.AMV.ECS.test.RNN_IDETest;

import P_V.PEQ.AMV.ECS.test.SensingTest;

import S_A.VSQ.parser.EmotionSample;

import S_A.pheromone.IMV_SQI;

import exception.thread.DetaThread;

import javax.swing.ImageIcon;

import java.awt.Color;
```

```
import java.awt.Component;

import java.util.List;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

//非osgi调用方式
public class MakeContainerXLS extends Thread implements Runnable {
    public CommonXls jpanelSecond;
    public App_S u;
    public DetabbedPane jTabbedpane;
    public List<String> tabNames;
    public CogsBinaryForest_AE _A;
    public IMV_SQI pos;
    public IMV_SQI pose;
    public IMV_SQI etc;
    public IMV_SQI cte;
    public String fileType;
    public DNNTTest dNNTTest;
    public ANNTTest aNNTTest;
    public RNN_IDETest rNN_IDETest;
    public EmotionSample emotionSample;
    public RatioMap_E ratioMap_E;
    public SensingTest sensingTest;
    public String xlsFilePath;
    App NE;
    public MakeContainerXLS(CogsBinaryForest_AE _A, App_S u2
        , DetabbedPane jTabbedpane, List<String> tabNames2
        , IMV_SQI pos, IMV_SQI pose, IMV_SQI etc, IMV_SQI cte, String fileType
        , EmotionSample emotionSample, RatioMap_E ratioMap_E
        , SensingTest sensingTest, App app_NE) {
        super();
        NE = app_NE;
        //this.jpanelSecond= jpanelSecond;
        this.u = u2;
        this.tabNames = tabNames2;
        this.jTabbedpane = jTabbedpane;
        this._A = _A;
        this.pos = pos;
        this.pose = pose;
        this.etc = etc;
```

```

        this.cte = cte;
        this.fileType = fileType.toString();
        this.emotionSample = emotionSample;
        this.ratioMap_E = ratioMap_E;
        this.sensingTest = sensingTest;
    }

    public MakeContainerXLS(CommonXls component, CogsBinaryForest_AE _A, App_S u2
        , DetabbedPane jTabbedpane, List<String> tabNames2
        , IMV_SQI pos, IMV_SQI pose
        , IMV_SQI etc, IMV_SQI cte, String fileType
        , EmotionSample emotionSample, RatioMap_E ratioMap_E, SensingTest sensingTest
        , App app_NE) {
        super();
        NE = app_NE;
        //this.jpanelSecond= jpanelSecond;
        this.u = u2;
        this.tabNames = tabNames2;
        this.jTabbedpane = jTabbedpane;
        this._A = _A;
        this.pos = pos;
        this.pose = pose;
        this.etc = etc;
        this.cte = cte;
        this.fileType = fileType.toString();
        this.dNNTTest = dNNTTest;
        this.aNNTTest = aNNTTest;
        this.rNN_IDETest = rNN_IDETest;
        this.emotionSample = emotionSample;
        this.ratioMap_E = ratioMap_E;
        this.sensingTest = sensingTest;
        jpanelSecond = component;
    }

    @SuppressWarnings("unchecked")
    public void run() {
        try {
            //jTabbedpane.validate();
            DetaThread.sleepDeta(1000 * 3);
            long temp = System.currentTimeMillis();
            String tempName = "临时";
            String pageName;
            if (null == jpanelSecond) {
                pageName = tempName + "XLS" + temp;
                jpanelSecond = new CommonXls(u.text, fileType, pageName, NE);
                jpanelSecond.pageName = pageName;
                jpanelSecond.setName(jpanelSecond.pageName);
                u.appConfig.SectionJPanel.newPages.addItem(jpanelSecond.pageName);
                u.tempPages.put(jpanelSecond.pageName, jpanelSecond);
            }
        }
    }

```

```

        u.tableNameMap.put(jpanelSecond.pageName, true);
        u.appConfig.SectionJPanel.pagehookJCheckBoxMap.put(jpanelSecond.pageName, true);
    } else {
        pageName = jpanelSecond.getName();
    }
    if (null == jpanelSecond.pageName || null == tabNames) {
        return;
    }
    tabNames.add(jpanelSecond.pageName);
    //
    for (int i = 0; i < jTabbedPane.getComponentCount(); i++) {
        Component component = jTabbedPane.getComponentAt(i);
        String pageNameString = component.getName();
        if (pageNameString.contains(tempName)) {
            jTabbedPane.addTab(jpanelSecond.pageName, new ImageIcon()
                , jpanelSecond, jpanelSecond.pageName);
        }
    }
    XA_ShellTables.addNewSearchShellTable(pageName.replace(tempName, ""))
        , jpanelSecond.tableData_old, jpanelSecond.columnTitle, NE);
    Color[] colors = TableRender.getTableCellRender();
    jTabbedPane.setBackgroundAt(jTabbedPane.getComponentCount() - 1
        , colors[jTabbedPane.getComponentCount() % 3]);
    jTabbedPane.validate();
} catch (Exception e) {
    e.printStackTrace();
    jTabbedPane.validate();
}
}

public void AOP_VEC_SQ(String input) {
    jpanelSecond.key = input.toString();
    jpanelSecond.doSearch();
}
}

/Trif_DNA-master/C_L_M/M_V/MPI/xls/commonXLS/MakeContainerXLSTest.java

M_V.MPI.xls.unixUtil

/Trif_DNA-master/C_L_M/M_V/MPI/xls/unixUtil/FileEncoding.java

M_V.MS.OP.SM.AOP.MEC.SIQ.cache

/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaCache.java
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaCacheTest.java
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaDBBufferCache.java
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/DetaDBBufferCacheTest.java
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/TimeCheck.java
/Trif_DNA-master/C_L_M/M_V/MS/OP/SM/AOP/MEC/SIQ/cache/TimeCheckTest.java

M_V.MS.VPC.PP

/Trif_DNA-master/C_L_M/M_V/MS/VPC/PP/Time_P.java

M_V.MS.VPC.SH

/Trif_DNA-master/C_L_M/M_V/MS/VPC/SH/Sleeper_H.java
/Trif_DNA-master/C_L_M/M_V/MS/VPC/SH/Sleeper_HTest.java
/Trif_DNA-master/C_L_M/M_V/MS/VPC/SH/Sleeper.java

```

/Trif_DNA-master/C_L_M/M_V/MS/VPC/SH/SleeperTest.java

M_V.MSI.OP.SM.AOP.MEC.SIQ.SM.OSI.E

/Trif_DNA-master/C_L_M/M_V/MSI/OP/SM/AOP/MEC/SIQ/SM/OSI/E/I_Tables_E.java
package M_V.MSI.OP.SM.AOP.MEC.SIQ.SM.OSI.E;

import S_A.SVQ.stable.S_ShelleTL;

import S_A.pheromone.IMV_SQI;

import U_A.PEU.P.cache.Cache_M;

import VPC.DetaDB.DetaDBUtil_X_withoutCulumnNameType;

import java.io.File;

import java.io.FileWriter;

import java.util.Iterator;

import java.util.List;

@SuppressWarnings("unchecked")

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

public class I_Tables_E {

public static void I_Table(IMV_SQI object, boolean mod) {

if (!object.containsKey("baseName") || !object.containsKey("tableName")) {

return;

}

//get base

String DBPath = Cache_M.getCacheInfo("DBPath").getValue().toString()

+ "/" + object.get("baseName").toString();

File DBPathFile = new File(DBPath);

if (!DBPathFile.isDirectory()) {

return;

}

//make table dir

String tablePath = DBPath + "/" + object.get("tableName").toString();

File tablePathFile = new File(tablePath);

if (tablePathFile.exists()) {

return;

}

//make spec dir

tablePathFile.mkdirs();

String tableSpecPath = tablePath + "/spec";

String tableRowsPath = tablePath + "/rows";

File tableSpecPathFile = new File(tableSpecPath);

File tableRowsPathFile = new File(tableRowsPath);

tableSpecPathFile.mkdir();

```

tableRowsPathFile.mkdir();

//make data

List<String[]> columnNames

    = (List<String[]>) object.get(S_ShellETL.SHELL_ETL_CULUMNNAME);

Iterator<String[]> iterator = columnNames.iterator();

try {

    while (iterator.hasNext()) {

        String[] columnDefinition = iterator.next();

        //create name

        String columnNamePath = tableSpecPath + "/" + columnDefinition[2];

        File columnNameFile = new File(columnNamePath);

        if (DetaDBUtil_X_withoutColumnNameType._E(columnDefinition[3])) {

            throw new Exception();

        }

        columnNameFile.mkdir();

        //create file

        File file = new File(columnNamePath + "/value.lyg");

        if (file.exists()) {

            throw new Exception();

        }

        if (mod) {

            FileWriter fw = null;

            fw = new FileWriter(file, true);

            fw.write(columnDefinition[3]);

            fw.close();

        }

    }

} catch (Exception e) {

    throw new RuntimeException(e);

}

}

}

/Trif_DNA-master/C_L_M/M_V/MSI/OP/SM/AOP/MEC/SIQ/SM/OSI/E/I_Tables_ETest.java

```

M_V.MSU.OCI.ME.SMS.translator.C

```

/Trif_DNA-master/C_L_M/M_V/MSU/OCI/ME/SMS/translator/C/Translator.java
package M_V.MSU.OCI.ME.SMS.translator.C;

import E_A.ME.analysis.E.AE;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.Verbal;

import java.util.List;

/*

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

```

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public interface Translator{

    void IV_(CogsBinaryForest_AE _A);

    String EnglishStringToChineseString(AE _A, String EnglishString);

    String ChineseStringToEnglishString(AE _A, String ChineseString);

    String MixedStringToChineseString(AE _A, String key);

    String ChineseStringToEnglishStringWithPOS(AE _A, String v);

    List<Verbal> index(AE _A, String mixedString);

    List<Verbal> fixPos(List<Verbal> verbals);

    String getChineseSentenceFromVerbalList(List<Verbal> verbals);

    String getEnglishSentenceFromVerbalFixList(List<Verbal> verbalsFix);

}
```

/Trif_DNA-master/C_L_M/M_V/MSU/OCI/ME/SMS/translator/C/TranslatorTest.java

M_V.MSU.OEI.ME.SMS.SEU.OSD.OSI.E

/Trif_DNA-master/C_L_M/M_V/MSU/OEI/ME/SMS/SEU/OSD/OSI/E/Translator_E.java
package M_V.MSU.OEI.ME.SMS.SEU.OSD.OSI.E;

import C_A.OCLSVQ.MPC.fhmm.C.FHMMLList;

import E_A.ME.analysis.E.AE;

import E_A.ME.analysis.E.CogsBinaryForest_AE;

import E_A.OEI.SVQ.MPC.fhmm.E.FHMMLListOneTime_E_maps;

import M_V.MSU.OCI.ME.SMS.translator.C.Translator;

import S_A.AVQ.OVQ.OSQ.VSQ.obj.Verbal;

import S_A.SVQ.stable.S_Pos;

import VSQ.parser.FHMMLListOneTime_E_X_S;

import java.util.List;

import java.util.concurrent.CopyOnWriteArrayList;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

```
public class Translator_E implements Translator{

    public FHMMLListOneTime_E_X_S fHMMLListOneTime_E_X_S;

    public void IV_(CogsBinaryForest_AE _A) {

        FHMMLList fHMMLList= _A.fHMMLList;

        FHMMLListOneTime_E_maps fHMMLListOneTime_E_maps= (FHMMLListOneTime_E_maps) fHMMLList;

        fHMMLListOneTime_E_X_S= fHMMLListOneTime_E_maps;

    }

    public String EnglishStringToChineseString(AE _A, String EnglishString) {

        String[] nodes= _A.parserEnglishString(EnglishString);

        StringBuilder sb= new StringBuilder();

        for(String temp:nodes) {
```

```

        char[] caseTemp= temp.toCharArray();
        caseTemp[S_Pos.INT_ONE]= String.valueOf(caseTemp[S_Pos.INT_ONE])
        .toUpperCase().charAt(S_Pos.INT_ONE);
        if(fmHMMListOneTime_E_X_S.enToCn.containsKey(temp)) {
sb.append(fmHMMListOneTime_E_X_S.enToCn.getString(temp).split(S_Pos.NLP_DOT)[S_Pos.INT_ONE]);
        } else if(fmHMMListOneTime_E_X_S.fullEnToCn.containsKey(String.valueOf(caseTemp))) {
sb.append(fmHMMListOneTime_E_X_S.fullEnToCn.get(String.valueOf(caseTemp)));
        } else {
sb.append(temp);
        }
        sb.append(S_Pos.SPACE_STRING);
    }
    return sb.toString();
}

public String ChineseStringToEnglishString(AE _A, String ChineseString) {
    List<String> nodes= _A.parserMixedString(ChineseString);
    StringBuilder sb= new StringBuilder();
    for (String temp : nodes) {
        if (fmHMMListOneTime_E_X_S.cnToEn.containsKey(temp)) {
sb.append(fmHMMListOneTime_E_X_S.cnToEn.get(temp));
        } else if (fmHMMListOneTime_E_X_S.fullCnToEn.containsKey(temp)) {
sb.append(fmHMMListOneTime_E_X_S.fullCnToEn.get(temp));
        } else {
sb.append(temp);
        }
        sb.append(S_Pos.SPACE_STRING);
    }
    return sb.toString();
}

public String MixedStringToChineseString(AE _A, String mixedString) {
    List<String> nodes= _A.parserMixedString(mixedString.toLowerCase());
    StringBuilder sb= new StringBuilder();
    for (String temp : nodes) {
        if (fmHMMListOneTime_E_X_S.posCnToCn.containsKey(temp)) {
sb.append(temp);
        } else {
String[] strings = _A.parserEnglishString(temp);
for (String string : strings) {
    if (string == null || string.length() < S_Pos.INT_ONE) {
        string = S_Pos.EMPTY_STRING;
    }
    if (fmHMMListOneTime_E_X_S.enToCn.containsKey(string)) {
        sb.append(fmHMMListOneTime_E_X_S.enToCn.getString(string)
            .split(S_Pos.NLP_DOT)[S_Pos.INT_ZERO]);
    } else if (fmHMMListOneTime_E_X_S.fullEnToCn.containsKey(string)) {
        sb.append(fmHMMListOneTime_E_X_S.fullEnToCn.get(string));
    }
}
}
}

```



```

        } else {
            sb.append(temp);
        }
    }

    }

    }

    return sb.toString();
}

public String ChineseStringToEnglishStringWithPOS(AE _A, String ChineseString) {
    List<String> nodes= _A.parserMixedString(ChineseString);
    StringBuilder sb= new StringBuilder();
    for (String temp : nodes) {
        if (fMHMMListOneTime_E_X_S.cnToEn.containsKey(temp)) {
sb.append(fMHMMListOneTime_E_X_S.cnToEn.get(temp));
        } else if (fMHMMListOneTime_E_X_S.fullCnToEn.containsKey(temp)) {
sb.append(fMHMMListOneTime_E_X_S.fullCnToEn.get(temp));
        } else {
sb.append(temp);
        }
        if (fMHMMListOneTime_E_X_S.posCnToCn.containsKey(temp)) {
sb.append(S_Pos.NLP_SYMBO_SLASH);
sb.append(fMHMMListOneTime_E_X_S.posCnToCn.get(temp));
        }
        sb.append(S_Pos.SPACE_STRING);
    }
    return sb.toString();
}

public List<Verbal> index(AE _A, String mixedString) {
    List<Verbal> verbals= new CopyOnWriteArrayList<>();
    List<String> nodes= _A.parserMixedString(mixedString.toLowerCase());
    for (String word : nodes) {
        Verbal verbal = new Verbal();
        if (fMHMMListOneTime_E_X_S.posCnToCn.containsKey(word)) {
verbal.I_Chinese(word);
verbal.I_PartOfSpeech(fMHMMListOneTime_E_X_S.posCnToCn.getString(word));
if (fMHMMListOneTime_E_X_S.cnToEn.containsKey(word)) {
            verbal.I_English(fMHMMListOneTime_E_X_S.cnToEn.getString(word));
            if (fMHMMListOneTime_E_X_S.enToCn.containsKey(fMHMMListOneTime_E_X_S.cnToEn.get(word))) {
                verbal.I_Explain(fMHMMListOneTime_E_X_S.enToCn
                    .getString(fMHMMListOneTime_E_X_S.cnToEn.getString(word)));
            }
        } else if (fMHMMListOneTime_E_X_S.fullCnToEn.containsKey(word)) {
            verbal.I_English(fMHMMListOneTime_E_X_S.fullCnToEn.getString(word));
            verbal.I_Explain(fMHMMListOneTime_E_X_S.fullCnToEn.getString(word));
        }
        } else if (fMHMMListOneTime_E_X_S.posEnToEn.containsKey(word)) {
verbal.I_English(word);

```

```

        if (fMHMMListOneTime_E_X_S.fullEnToCn.containsKey(word)) {
            verbal.I_Chinese(fMHMMListOneTime_E_X_S.fullEnToCn.getString(word));
        }
        if (fMHMMListOneTime_E_X_S.posCnToCn.containsKey(fMHMMListOneTime_E_X_S.fullEnToCn.get(word))) {
            verbal.I_PartOfSpeech(fMHMMListOneTime_E_X_S.posCnToCn
                .getString(fMHMMListOneTime_E_X_S.fullEnToCn.getString(word)));
        }
        if (fMHMMListOneTime_E_X_S.enToCn.containsKey(word)) {
            verbal.I_Explain(fMHMMListOneTime_E_X_S.enToCn.getString(word));
        }
    } else {
        verbal.I_English(word);
        verbal.I_Chinese(fMHMMListOneTime_E_X_S.fullEnToCn.getString(word));
        verbal.I_PartOfSpeech(S_Pos.NLP_NULL);
        verbal.I_Explain(S_Pos.NLP_NULL);
    }
    verbals.add(verbal);
}
return verbals;
}

public List<Verbal> fixPos(List<Verbal> verbals) {
    for(int i= 0; i < verbals.size(); i++) {
        if(verbals.get(i).getPartOfSpeech() != null) {
            if(verbals.get(i).getPartOfSpeech().contains(S_Pos.NLP_ZI_DONG)){
                if(!verbals.get(i).getPartOfSpeech().contains(S_Pos.NLP_ZI_MING)){
                    if(i - S_Pos.INT_ONE > S_Pos.INT_ZERO && verbals.get(i
                        - S_Pos.INT_ONE)
                            .getEnglish().contains(S_Pos.NLP_ENGLISH_OF)) {
                        Verbal temp= verbals.get(i);
                        temp.I_PartOfSpeech(S_Pos.NLP_CI_DONG_MING);
                        String english= temp.getEnglish().replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING);
                        if(english.charAt(english.length() - S_Pos.INT_ONE)== S_Pos.NLP_CHAR_E) {
                            english= S_Pos.NLP_ENGLISH_THE + S_Pos.SPACE_STRING
                                + english.substring(S_Pos.INT_ZERO, english.length() - S_Pos.INT_ONE)
                                + S_Pos.NLP_ENGLISH_ING;
                        }else {
                            english= S_Pos.NLP_ENGLISH_THE + S_Pos.SPACE_STRING + english + S_Pos.NLP_ENGLISH_ING;
                        }
                        temp.I_English(english);
                    }else if(verbals.get(i + S_Pos.INT_ONE).getPartOfSpeech().contains(S_Pos.NLP_ZI_MING)){
                        if(i - S_Pos.INT_ONE >= S_Pos.INT_ZERO
                            && !verbals.get(i - S_Pos.INT_ONE).getPartOfSpeech()
                                .contains(S_Pos.NLP_ZI_MING)){
                            if(!verbals.get(i - S_Pos.INT_ONE).getPartOfSpeech().contains(S_Pos.NLP_ZI_DAI)){
                                if(verbals.get(i - S_Pos.INT_ONE).getPartOfSpeech().contains(S_Pos.NLP_ZI_JIE)){
                                    if(verbals.get(i - S_Pos.INT_ONE).getChinese().contains(S_Pos.NLP_ZI_ZAI)){
                                        if(verbals.get(i + S_Pos.INT_ONE).getChinese().contains(S_Pos.NLP_ZI_ZHONG)){

```

```

        Verbal temp= verbals.get(i + S_Pos.INT_ONE);
        temp.I_English(S_Pos.NLP_ENGLISH_STATUS);
    }
}

Verbal temp= verbals.get(i);
temp.I_PartOfSpeech(S_Pos.NLP_CI_DONG_MING);
String english= temp.getEnglish().replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING);
if(english.charAt(english.length()-S_Pos.INT_ONE)== S_Pos.NLP_CHAR_E) {
    english= S_Pos.NLP_ENGLISH_THE + S_Pos.SPACE_STRING
+ english.substring(S_Pos.INT_ZERO, english.length() - S_Pos.INT_ONE)
+ S_Pos.NLP_ENGLISH_ING;
}else {
    english= S_Pos.NLP_ENGLISH_THE + S_Pos.SPACE_STRING
+ english + S_Pos.NLP_ENGLISH_ING;
}
temp.I_English(english);
}

}

if(verbals.get(i + S_Pos.INT_TWO).getPartOfSpeech().contains(S_Pos.NLP_ZI_DONG)){
if(!verbals.get(i + S_Pos.INT_TWO).getPartOfSpeech().contains(S_Pos.NLP_ZI_MING)){
if(verbals.get(i + S_Pos.INT_THREE).getPartOfSpeech().contains(S_Pos.NLP_ZI_MING)){
    Verbal temp= verbals.get(i + S_Pos.INT_TWO);
    temp.I_PartOfSpeech(S_Pos.NLP_CI_DONG_MING);
    String english= temp.getEnglish().replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING);
    if(english.charAt(english.length() - S_Pos.INT_ONE)== S_Pos.NLP_CHAR_E) {
        english= S_Pos.NLP_ENGLISH_OF + S_Pos.SPACE_STRING +
S_Pos.NLP_ENGLISH_THE + S_Pos.SPACE_STRING
+ english.substring(S_Pos.INT_ZERO, english.length()-S_Pos.INT_ONE)
+ S_Pos.NLP_ENGLISH_ING;
    }else {
        english= S_Pos.NLP_ENGLISH_OF + S_Pos.SPACE_STRING
+ S_Pos.NLP_ENGLISH_THE + S_Pos.SPACE_STRING
+ english + S_Pos.NLP_ENGLISH_ING;
    }
    temp.I_English(english);
}
}

}

}else if(i - S_Pos.INT_ONE >= S_Pos.INT_ZERO && verbals.get(i - S_Pos.INT_ONE)
.getPartOfSpeech().contains(S_Pos.NLP_ZI_JIE)){
    Verbal temp= verbals.get(i);
    temp.I_PartOfSpeech(S_Pos.NLP_CI_DONG_MING);
    String english= temp.getEnglish().replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING);
    if(english.charAt(english.length() - S_Pos.INT_ONE)== S_Pos.NLP_CHAR_E) {
english= S_Pos.NLP_ENGLISH_THE + S_Pos.SPACE_STRING
+ english.substring(S_Pos.INT_ZERO

```

```

        , english.length() - S_Pos.INT_ONE) + S_Pos.NLP_ENGLISH_ING;
    } else {
        english= S_Pos.NLP_ENGLISH_THE + S_Pos.SPACE_STRING
            + english + S_Pos.NLP_ENGLISH_ING;
    }
    temp.I_English(english);
} else if(i - S_Pos.INT_ONE >= S_Pos.INT_ZERO
&& verbals.get(i - S_Pos.INT_ONE).getPartOfSpeech()
.contains(S_Pos.NLP_ZI_FU)){
    Verbal temp= verbals.get(i);
    temp.I_PartOfSpeech(S_Pos.NLP_CI_DONG_MING);
    String english= temp.getEnglish().replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING);
    if(english.charAt(english.length() - S_Pos.INT_ONE)== S_Pos.NLP_CHAR_E) {
        english= english.substring(S_Pos.INT_ZERO, english.length() - S_Pos.INT_ONE)
            + S_Pos.NLP_ENGLISH_ING;
    } else {
        english += S_Pos.NLP_ENGLISH_ING;
    }
    temp.I_English(english);
}
}
} else if(verbals.get(i).getPartOfSpeech().contains(S_Pos.NLP_ZI_MING)){
    if(i - S_Pos.INT_ONE >= S_Pos.INT_ZERO && verbals.get(i - S_Pos.INT_ONE)
        .getPartOfSpeech().contains(S_Pos.NLP_FU_SHU)){
        Verbal temp= verbals.get(i);
        String english= temp.getEnglish().replace(S_Pos.SPACE_STRING, S_Pos.EMPTY_STRING);
        if(english.charAt(english.length()-1)== S_Pos.NLP_CHAR_H
        || english.charAt(english.length()-1)== S_Pos.NLP_CHAR_S) {
            english += S_Pos.NLP_ENGLISH_ES;
        } else {
            english += S_Pos.NLP_ENGLISH_S;
        }
        temp.I_English(english);
    }
} else if(verbals.get(i).getPartOfSpeech().contains(S_Pos.NLP_CI_FU)){
    if(i - S_Pos.INT_ONE >= S_Pos.INT_ZERO && (verbals.get(i-S_Pos.INT_ONE)
        .getPartOfSpeech().contains(S_Pos.NLP_ZI_DAI)
        || verbals.get(i - S_Pos.INT_ONE).getPartOfSpeech().contains(S_Pos.NLP_ZI_MING))){
        if(i-S_Pos.INT_ONE >= S_Pos.INT_ZERO&&verbals.get(i - S_Pos.INT_ONE).getPartOfSpeech()
        .contains(S_Pos.NLP_ZI_again)
        || (verbals.get(i-S_Pos.INT_ONE).getPartOfSpeech().contains(S_Pos.NLP_ZI_single)
            && verbals.get(i-S_Pos.INT_ONE).getPartOfSpeech().contains(S_Pos.NLP_ZI_one))){
            if(verbals.get(i).getEnglish().contains(S_Pos.NLP_HAVE_HAS)) {
                Verbal temp= verbals.get(i);
                String english= temp.getEnglish().replace(S_Pos.NLP_HAVE_HAS, S_Pos.NLP_HAVE);
                temp.I_English(english);
            }

```

```

        }else {
            if(verbals.get(i).getEnglish().contains(S_Pos.NLP_HAVE_HAS)) {
                Verbal temp= verbals.get(i);
                String english= temp.getEnglish().replace(S_Pos.NLP_HAVE_HAS, S_Pos.NLP_HAS);
                temp.I_English(english);
            }
        }
    }
    if(i-S_Pos.INT_TWO>= S_Pos.INT_ZERO && (verbals.get(i-S_Pos.INT_TWO).getPartOfSpeech()
        .contains(S_Pos.NLP_ZI_DAI) || verbals.get(i-S_Pos.INT_TWO).getPartOfSpeech()
        .contains(S_Pos.NLP_ZI_MING))) {
        if(verbals.get(i-S_Pos.INT_TWO).getPartOfSpeech().contains(S_Pos.NLP_ZI_again)
            || (verbals.get(i-S_Pos.INT_TWO).getPartOfSpeech().contains(S_Pos.NLP_ZI_single)
                &&verbals.get(i-S_Pos.INT_TWO).getPartOfSpeech().contains(S_Pos.NLP_ZI_one))) {
            if(verbals.get(i).getEnglish().contains(S_Pos.NLP_HAVE_HAS)) {
                Verbal temp= verbals.get(i);
                String english= temp.getEnglish().replace(S_Pos.NLP_HAVE_HAS, S_Pos.NLP_HAVE);
                temp.I_English(english);
            }
        }else {
            if(verbals.get(i).getEnglish().contains(S_Pos.NLP_HAVE_HAS)) {
                Verbal temp= verbals.get(i);
                String english= temp.getEnglish().replace(S_Pos.NLP_HAVE_HAS, S_Pos.NLP_HAS);
                temp.I_English(english);
            }
        }
    }
}

}

}

return verbals;
}

public String getChineseSentenseFromVerbalList(List<Verbal> verbals) {
    StringBuilder sb= new StringBuilder();
    for (Verbal verbal : verbals) {
        sb.append(verbal.getChinese() == null
? S_Pos.EMPTY_STRING : verbal.getChinese());
    }
    return sb.toString()
.replaceAll(S_Pos.NLP_SPASE_REP, S_Pos.SPACE_STRING);
}

public String getEnglishSentenseFromVerbalFixList(List<Verbal> verbalsFix) {
    StringBuilder sb= new StringBuilder();
    for (Verbal fix : verbalsFix) {
        sb.append(fix.getEnglish() == null
? S_Pos.EMPTY_STRING : fix.getEnglish().toLowerCase());
        sb.append(S_Pos.SPACE_STRING);
    }
}

```

```

    }

    return sb.toString()

.replaceAll(S_Pos.NLP_SPASE_REP, S_Pos.SPACE_STRING);

}

}

/Trif_DNA-master/C_L_M/M_V/MSU/OEI/ME/SMS/SEU/OSD/OSI/E/Translator_ETest.java

M_V.MSV.OSQ.sets

/Trif_DNA-master/C_L_M/M_V/MSV/OSQ/sets/DetaDicimal.java
/Trif_DNA-master/C_L_M/M_V/MSV/OSQ/sets/DetaDouble.java
/Trif_DNA-master/C_L_M/M_V/MSV/OSQ/sets/DetaFloat.java
/Trif_DNA-master/C_L_M/M_V/MSV/OSQ/sets/DetaLong.java

M_V.MSV.OSU.array

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/array/ArrayValidation.java
package M_V.MSV.OSU.array;

/*
* 个人著作权人, 作者 罗瑶光. 浏阳
* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/

public class ArrayValidation{

    //以后 validation大体包括 check和fix, 2个部分。
    public static boolean arrayIntCheck(int[] array
        , int min, int max) {
        for (int j : array) {
            if (j > max || j < min) {
                return false;
            }
        }
        return true;
    }

    public static int[] arrayIntFix(int[] array
        , int min, int max) {
        for(int i= 0; i< array.length; i++) {
            if(array[i]> max) {
                array[i]= max;
            }
            if(array[i]< min) {
                array[i]= min;
            }
        }
        return array;
    }

    public static boolean arrayLongCheck(long[] array
        , long min, long max) {
        for (long l : array) {

```

```
        if (l > max || l < min) {

            return false;

        }

    }

    return true;

}

public static long[] arrayLongFix(long[] array
    , long min, long max) {
    for(int i= 0; i< array.length; i++) {
        if(array[i]> max) {
            array[i]= max;
        }
        if(array[i]< min) {
            array[i]= min;
        }
    }
    return array;
}

public static boolean arrayDoubleCheck(double[] array
    , double min, double max) {
    for (double aDouble : array) {
        if (aDouble > max || aDouble < min) {
            return false;
        }
    }
    return true;
}

public static double[] arrayDoubleFix(double[] array
    , double min, double max) {
    for(int i= 0; i< array.length; i++) {
        if(array[i]> max) {
            array[i]= max;
        }
        if(array[i]< min) {
            array[i]= min;
        }
    }
    return array;
}

public static boolean arrayFloatCheck(float[] array
    , float min, float max) {
    for (float v : array) {
        if (v > max || v < min) {
            return false;
        }
    }
    return true;
}
```

```

    }

    public static float[] arrayFloatFix(float[] array
        , float min, float max) {
        for(int i= 0; i< array.length; i++) {
            if(array[i]> max) {
                array[i]= max;
            }
            if(array[i]< min) {
                array[i]= min;
            }
        }
        return array;
    }
}

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/array/ArrayValidationTest.java

```

M_V.MSV.OSU.date

```

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/date/DateValidation.java
package M_V.MSV.OSU.date;

import S_A.ESU.code.stable.S_;
import java.io.IOException;
import java.net.MalformedURLException;
import java.net.URL;
import java.net.URLConnection;
import java.util.Date;

//refer www.bjtime.com 第 12, 13, 14行 共三行
//注意如果bjtime 网站关闭了这个功能, 该函数将无效, 请再互联网搜索其他提供时间校验的网址。
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class DateValidation {
    public static boolean currentTimeCheck(long offsetUTC8
        , long miliSecondsDistinction) {
        try {
            URL url = null;
            url = new URL(S_.STRING_BJTIME);
            URLConnection uRLConnection = url.openConnection();
            uRLConnection.connect();
            long bjtime = uRLConnection.getDate();
            long system = currentTimeToUTC(offsetUTC8);
            if (miliSecondsDistinction < Math.abs(bjtime - system)) {
                return false;
            }
        }
    }
}

```



```

    }
    } catch (MalformedURLException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
    return true;
}

public static long currentTimeSystemTimeToUTC(long offsetUTC8) {
    return new Date().getTime() + offsetUTC8;
}
}

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/date/DateValidationTest.java

```

M_V.MSV.OSU.list

```

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/list/ListValidation.java
package M_V.MSV.OSU.list;

import S_A.ESU.code.stable.S_;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class ListValidation {

    public static boolean ListSetsCheck(List<Object> list, String setsType) {
        Iterator<Object> iterator= list.iterator();
        while(iterator.hasNext()) {
            Object object= iterator.next();
            if(setsType.equalsIgnoreCase(S_.STRING_DOUBLE)) {
                if(!(object instanceof Double)) {
                    return false;
                }
            }
            if(setsType.equalsIgnoreCase(S_.STRING_INT)) {
                if(!(object instanceof Integer)) {
                    return false;
                }
            }
            if(setsType.equalsIgnoreCase(S_.STRING_FLOAT)) {
                if(!(object instanceof Float)) {
                    return false;
                }
            }
        }
    }
}

```

```

    }

    }

    if(setsType.equalsIgnoreCase(S_.STRING_STRING)) {
if(!(object instanceof String)) {
    return false;
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_SHORT)) {
if(!(object instanceof Short)) {
    return false;
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_BOOLEAN)) {
if(!(object instanceof Boolean)) {
    return false;
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_LONG)) {
if(!(object instanceof Long)) {
    return false;
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_BYTE)) {
if(!(object instanceof Byte)) {
    return false;
}

    }

    }

    return true;
}

public static List<Object> ListSetsFix(List<Object> list
    , String setsType) {
    List<Object> output= new ArrayList<>();
    Iterator<Object> iterator= list.iterator();
    while(iterator.hasNext()) {
        Object object= iterator.next();
        if(setsType.equalsIgnoreCase(S_.STRING_DOUBLE)) {
if(!(object instanceof Double)) {
            output.add(((double)0.00);
        }else {
            output.add(object);
        }

        }

        if(setsType.equalsIgnoreCase(S_.STRING_INT)) {
if(!(object instanceof Integer)) {
            output.add(((int)0);
        }else {

```

```
        output.add(object);
    }

    }

    if(setsType.equalsIgnoreCase(S_.STRING_FLOAT)) {
if(!(object instanceof Float)) {
    output.add((float)0.0);
}else {
    output.add(object);
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_STRING)) {
if(!(object instanceof String)) {
    output.add(S_.STRING_EMPTY);
}else {
    output.add(object);
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_SHORT)) {
if(!(object instanceof Short)) {
    output.add((short)0);
}else {
    output.add(object);
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_BOOLEAN)) {
if(!(object instanceof Boolean)) {
    output.add(false);
}else {
    output.add(object);
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_LONG)) {
if(!(object instanceof Long)) {
    output.add((long)(0));
}else {
    output.add(object);
}

    }

    if(setsType.equalsIgnoreCase(S_.STRING_BYTE)) {
if(!(object instanceof Byte)) {
    output.add((byte)0);
}else {
    output.add(object);
}

    }

    }

    return output;
}
```

```
}
}

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/list/ListValidationTest.java

    M_V.MSV.OSU.matrix

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/matrix/MatrixValidation.java
package M_V.MSV.OSU.matrix;

import S_A.ESU.code.stable.S_;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class MatrixValidation{

    public static boolean matrixCheck(Object[][] objects

        , String attribute){

        if(attribute.equalsIgnoreCase(S_.STRING_BOOLEAN)) {

            for(int i= 0; i< objects.length; i++) {

                for(int j= 0; j< objects[0].length; j++) {

                    if(!(objects[i][j] instanceof Boolean)) {

                        return false;

                    }

                }

            }

            if(attribute.equalsIgnoreCase(S_.STRING_INT)) {

                for(int i= 0; i<objects.length; i++) {

                    for(int j= 0; j<objects[0].length; j++) {

                        if(!(objects[i][j] instanceof Integer)) {

                            return false;

                        }

                    }

                }

                if(attribute.equalsIgnoreCase(S_.STRING_LONG)) {

                    for(int i= 0; i< objects.length; i++) {

                        for(int j= 0; j< objects[0].length; j++) {

                            if(!(objects[i][j] instanceof Long)) {

                                return false;

                            }

                        }

                    }

                    if(attribute.equalsIgnoreCase(S_.STRING_DOUBLE)) {
```

```
        for(int i= 0; i< objects.length; i++) {
            for(int j= 0; j< objects[0].length; j++) {
                if(!(objects[i][j] instanceof Double)) {
                    return false;
                }
            }
        }

        if(attribute.equalsIgnoreCase(S_.STRING_FLOAT)) {
            for(int i= 0; i< objects.length; i++) {
                for(int j= 0; j< objects[0].length; j++) {
                    if(!(objects[i][j] instanceof Float)) {
                        return false;
                    }
                }
            }
        }

        if(attribute.equalsIgnoreCase(S_.STRING_STRING)) {
            for(int i= 0; i< objects.length; i++) {
                for(int j= 0; j< objects[0].length; j++) {
                    if(!(objects[i][j] instanceof String)) {
                        return false;
                    }
                }
            }
        }

        if(attribute.equalsIgnoreCase(S_.STRING_SHORT)) {
            for(int i= 0; i< objects.length; i++) {
                for(int j= 0; j< objects[0].length; j++) {
                    if(!(objects[i][j] instanceof Short)) {
                        return false;
                    }
                }
            }
        }

        if(attribute.equalsIgnoreCase(S_.STRING_BYTE)) {
            for(int i= 0; i< objects.length; i++) {
                for(int j= 0; j< objects[0].length; j++) {
                    if(!(objects[i][j] instanceof Byte)) {
                        return false;
                    }
                }
            }
        }

        return true;
    }

    public static Object[][] matrixFix(Object[][] objects
```

```

        , String attribute){
            if(attribute.equalsIgnoreCase(S_.STRING_BOOLEAN)) {
                for(int i= 0; i< objects.length; i++) {
                    for(int j= 0; j< objects[0].length; j++) {
                        if(!(objects[i][j] instanceof Boolean)) {
                            objects[i][j]= false;
                        }
                    }
                }
            }
            if(attribute.equalsIgnoreCase(S_.STRING_INT)) {
                for(int i= 0; i< objects.length; i++) {
                    for(int j= 0; j< objects[0].length; j++) {
                        if(!(objects[i][j] instanceof Integer)) {
                            objects[i][j]= (int)0;
                        }
                    }
                }
            }
            if(attribute.equalsIgnoreCase(S_.STRING_LONG)) {
                for(int i= 0; i< objects.length; i++) {
                    for(int j= 0; j< objects[0].length; j++) {
                        if(!(objects[i][j] instanceof Long)) {
                            objects[i][j]= (long)0;
                        }
                    }
                }
            }
            if(attribute.equalsIgnoreCase(S_.STRING_DOUBLE)) {
                for(int i= 0; i< objects.length; i++) {
                    for(int j= 0; j< objects[0].length; j++) {
                        if(!(objects[i][j] instanceof Double)) {
                            objects[i][j]= (double)0.0;
                        }
                    }
                }
            }
            if(attribute.equalsIgnoreCase(S_.STRING_FLOAT)) {
                for(int i= 0; i< objects.length; i++) {
                    for(int j= 0; j< objects[0].length; j++) {
                        if(!(objects[i][j] instanceof Float)) {
                            objects[i][j]= (float)0.0;
                        }
                    }
                }
            }
            if(attribute.equalsIgnoreCase(S_.STRING_STRING)) {

```

```

        for(int i= 0; i< objects.length; i++) {
            for(int j= 0; j< objects[0].length; j++) {
                if(!(objects[i][j] instanceof String)) {
                    objects[i][j]= "";
                }
            }
        }

        if(attribute.equalsIgnoreCase(S_.STRING_SHORT)) {
            for(int i= 0; i< objects.length; i++) {
                for(int j= 0; j< objects[0].length; j++) {
                    if(!(objects[i][j] instanceof Short)) {
                        objects[i][j]= (short)0;
                    }
                }
            }
        }

        if(attribute.equalsIgnoreCase(S_.STRING_BYTE)) {
            for(int i= 0; i< objects.length; i++) {
                for(int j= 0; j< objects[0].length; j++) {
                    if(!(objects[i][j] instanceof Byte)) {
                        objects[i][j]= (byte)0;
                    }
                }
            }
        }

        return objects;
    }
}

```

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/matrix/MatrixValidationTest.java

M_V.MSV.OSU.string

/Trif_DNA-master/C_L_M/M_V/MSV/OSU/string/NullObject.java
 package M_V.MSV.OSU.string;

```

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class NullObject{
    public static String checkNULL(Object object) {
        return null== object? "": object.toString();
    }
}

```

M_V.MVQ.button

```
/Trif_DNA-master/C_L_M/M_V/MVQ/button/CfxButton.java
package M_V.MVQ.button;
```

```
import javax.swing.BorderFactory;
```

```
import javax.swing.JButton;
```

```
import javax.swing.Timer;
```

```
import java.awt.Color;
```

```
import java.awt.Dimension;
```

```
import java.awt.Graphics;
```

```
import java.awt.event.ActionEvent;
```

```
import java.awt.event.ActionListener;
```

```
import java.awt.event.MouseEvent;
```

```
import java.awt.event.MouseMotionListener;
```

```
import java.util.Random;
```

```
/*
```

```
 * 个人著作权人, 作者 罗瑶光, 浏阳
```

```
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
 * (lyg.tin@gmail.com 2018年回国后因国内G网屏蔽不再使用)
```

```
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
 * 430181198505250014, G24402609, EB0581342
```

```
 * 204925063, 389418686, F2406501, 0626136
```

```
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
 */
```

```
public class CfxButton extends JButton{
```

```
    /**
```

```
     * Author Yaoguang Luo
```

```
    */
```

```
    private int x, y=0;
```

```
    public CfxButton(String string, final int x, final int y, java.awt.Color c) {
```

```
        super(string);
```

```
        this.x= x;
```

```
        this.y= y;
```

```
        this.setContentAreaFilled(false);
```

```
        this.setBorder(BorderFactory.createRaisedBevelBorder());
```

```
        this.setPreferredSize(new Dimension(this.x+ 1, this.y+ 1));
```

```
        this.setOpaque(true);
```

```
        this.setBackground(c);
```

```
        this.addMouseMotionListener(new MouseMotionListener() {
```

```
            public void mouseDragged(MouseEvent e) {
```

```
            }
```

```
            public void mouseMoved(MouseEvent e) {
```

```
        Graphics g= getGraphics();
```

```
        I_color(g);
```

```
        //g.setColor(new Color(255, 0, 255, 128));
```

```
        int xPoints[]={8, 8+15, 8};
```



```
int yPoints[]={8, 8, 8+15};
g.fillPolygon(xPoints, yPoints, 3);
int xPoints1[]={x-16-5, x-8, x-8};
int yPoints1[]={y-8, y-16-5, y-8};
g.fillPolygon(xPoints1, yPoints1, 3);
g.fillRect(8, 8, x-16 , 5);
g.fillRect(8, 8, 5 , y-16);
g.fillRect(8, y-12, x-16 , 5);
g.fillRect(x-12, 8, 5 , y-16);
    }
});
Graphics g= getGraphics();
this.callback(g);
}
public void I_color(Graphics g) {

    int x=1+new Random().nextInt(9);
    if(1==x){
        g.setColor(Color.BLACK);
    }
    if(2==x){
        g.setColor(Color.blue);
    }
    if(3==x){
        g.setColor(Color.cyan);
    }
    if(4==x){
        g.setColor(Color.DARK_GRAY);
    }
    if(5==x){
        g.setColor(Color.green);
    }
    if(6==x){
        g.setColor(Color.orange);
    }
    if(7==x){
        g.setColor(Color.pink);
    }
    if(8==x){
        g.setColor(Color.pink);
    }
    if(9==x){
        g.setColor(Color.yellow);
    }
}
public void callback(Graphics g){
    Timer blinkTimer= new Timer(800, new ActionListener() {
```

```
public void actionPerformed(ActionEvent e) {

    Graphics g= getGraphics();
    if(null==g){
        return;
    }
    I_color(g);
    g.drawLine(0, 0, 0, y);
    g.drawLine(0, 0, x, 0);
    g.drawLine(x, y, x, 0);
    g.drawLine(x, y, 0, y);
    I_color(g);
    g.drawLine(2, 2, 2, y-2);
    g.drawLine(2, 2, x-2, 2);
    g.drawLine(x-2, y-2, x-2, 2);
    g.drawLine(x-2, y-2, 2, y-2);
    I_color(g);
    g.drawLine(4, 4, 4, y-4);
    g.drawLine(4, 4, x-4, 4);
    g.drawLine(x-4, y-4, x-4, 4);
    g.drawLine(x-4, y-4, 4, y-4);
    I_color(g);
    g.drawLine(6, 6, 6, y-6);
    g.drawLine(6, 6, x-6, 6);
    g.drawLine(x-6, y-6, x-6, 6);
    g.drawLine(x-6, y-6, 6, y-6);
    I_color(g);
    g.drawLine(8, 8, 8, y-8);
    g.drawLine(8, 8, x-8, 8);
    g.drawLine(x-8, y-8, x-8, 8);
    g.drawLine(x-8, y-8, 8, y-8);
    }

    private void I_color(Graphics g) {

    int x=1+new Random().nextInt(9);
    if(1==x){
        g.setColor(Color.BLACK);
    }
    if(2==x){
        g.setColor(Color.blue);
    }
    if(3==x){
        g.setColor(Color.cyan);
    }
    if(4==x){
        g.setColor(Color.DARK_GRAY);
    }
    if(5==x){
        g.setColor(Color.green);
    }
    }
```

```

    }

    if(6==x){

        g.setColor(Color.orange);

    }

    if(7==x){

        g.setColor(Color.pink);

    }

    if(8==x){

        g.setColor(Color.pink);

    }

    if(9==x){

        g.setColor(Color.yellow);

    }

    }

    });

    blinkTimer.start();

}

}

//Trif_DNA-master/C_L_M/M_V/MVQ/button/CfxButtonTest.java
//Trif_DNA-master/C_L_M/M_V/MVQ/button/DetaButton.java
package M_V.MVQ.button;

import javax.swing.JButton;

import java.awt.Color;

import java.awt.Graphics;

import java.awt.event.MouseEvent;

import java.awt.event.MouseListener;

import java.awt.event.MouseMotionListener;

import java.awt.event.MouseWheelEvent;

import java.awt.event.MouseWheelListener;

import java.awt.image.BufferedImage;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

 * 430181198505250014, G24402609, EB0581342

 * 204925063, 389418686, F2406501, 0626136

 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

 */

public class DetaButton extends JButton implements MouseListener, MouseMotionListener, MouseWheelListener{

    public BufferedImage bufferedImage;

    /**

     * Author Yaoguang Luo

     */

    public DetaButton(String string, final int x, final int y, java.awt.Color c) {

        super(string);

        this.setBackground(c);

    }

}

```

```
this.addMouseMotionListener(new MouseMotionListener() {  
    int motionI= 1;  
    public void mouseDragged(MouseEvent e) {  
    }  
    public void mouseMoved(MouseEvent e) {  
Color reg= Color.black;  
if(1== motionI++) {  
    reg= Color.black;  
}else if(2== motionI) {  
    reg= Color.blue;  
}else if(3== motionI) {  
    motionI= 1;  
    reg= Color.green;  
}  
Graphics g= getGraphics();  
g.setColor(reg);  
int xPoints[]= {1, 1+ 10, 1};  
int yPoints[]= {1, 1, 1+ 10};  
g.fillPolygon(xPoints, yPoints, 3);  
    }  
});  
}  
public DetaButton(String string) {  
    super(string);  
    int i= 1+ (int)(Math.abs(Math.random()*100))% 4;  
    Color regBackground= null;  
    if(1== i) {  
        regBackground= Color.orange;  
    }else if(2== i) {  
        regBackground= Color.magenta;  
    }else if(3== i) {  
        regBackground= Color.green;  
    }else if(4== i) {  
        regBackground= Color.pink;  
    }  
    this.setBackground(regBackground);  
    this.addMouseMotionListener(new MouseMotionListener() {  
        int motionI= 1;  
        public void mouseDragged(MouseEvent e) {  
        }  
        public void mouseMoved(MouseEvent e) {  
Color reg= Color.black;  
if(1== motionI++) {  
    reg= Color.black;  
}else if(2== motionI) {  
    reg= Color.blue;  
}else if(3== motionI) {
```

```
        motionI= 1;
        reg= Color.green;
    }
    Graphics g= getGraphics();
    g.setColor(reg);
    int xPoints[]= {1, 1+ 10, 1};
    int yPoints[]= {1, 1, 1+ 10};
    g.fillPolygon(xPoints, yPoints, 3);
        }
    });

    this.addMouseListener(this);
    this.addMouseWheelListener(this);
    this.addMouseMotionListener(this);
}
@Override
public void mouseWheelMoved(MouseWheelEvent e) {

}
@Override
public void mouseDragged(MouseEvent e) {

}
@Override
public void mouseMoved(MouseEvent e) {

        ///20230106-S_logger.Log.logger.info("" + e.getButton());
    }

@Override
public void mouseClicked(MouseEvent e) {
        ///20230106-S_logger.Log.logger.info("" + e.getButton());
    }
@Override
public void mousePressed(MouseEvent e) {

}
@Override
public void mouseReleased(MouseEvent e) {

}
@Override
public void mouseEntered(MouseEvent e) {
```

```

    }

    @Override

    public void mouseExited(MouseEvent e) {

    }

}

//Trif_DNA-master/C_L_M/M_V/MVQ/button/DetaButtonTest.java
//Trif_DNA-master/C_L_M/M_V/MVQ/button/HrjButton.java
package M_V.MVQ.button;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.Timer;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Graphics;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseEvent;
import java.awt.event.MouseMotionListener;
import java.util.Random;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class HrjButton extends JButton{

    /**
     * Author Yaoguang Luo
     */
    private int x, y=0;

    public HrjButton(String string, final int x, final int y, java.awt.Color c) {

        super(string);

        this.x= x;

        this.y= y;

        this.setContentAreaFilled(false);

        this.setBorder(BorderFactory.createRaisedBevelBorder());

        this.setPreferredSize(new Dimension(this.x+ 1, this.y+ 1));

        this.setOpaque(true);

        this.setBackground(c);

        this.addMouseMotionListener(new MouseMotionListener() {

            public void mouseDragged(MouseEvent e) {

            }

        })
    }

```

```
        public void mouseMoved(MouseEvent e) {

            Graphics g= getGraphics();
            I_color(g);
            //g.setColor(new Color(255, 0, 255, 128));
            int xPoints[]={8, 8+15, 8};
            int yPoints[]={8, 8, 8+15};
            g.fillPolygon(xPoints, yPoints, 3);
            int xPoints1[]={x-16-5, x-8, x-8};
            int yPoints1[]={y-8, y-16-5, y-8};
            g.fillPolygon(xPoints1, yPoints1, 3);
            g.fillRect(8, 8, x-16 , 5);
            g.fillRect(8, 8, 5 , y-16);
            g.fillRect(8, y-12, x-16 , 5);
            g.fillRect(x-12, 8, 5 , y-16);

        }

    });

}

public void I_color(Graphics g) {

    int x=1+new Random().nextInt(9);
    if(1==x){
        g.setColor(Color.BLACK);
    }
    if(2==x){
        g.setColor(Color.blue);
    }
    if(3==x){
        g.setColor(Color.cyan);
    }
    if(4==x){
        g.setColor(Color.DARK_GRAY);
    }
    if(5==x){
        g.setColor(Color.green);
    }
    if(6==x){
        g.setColor(Color.orange);
    }
    if(7==x){
        g.setColor(Color.pink);
    }
    if(8==x){
        g.setColor(Color.pink);
    }
    if(9==x){
        g.setColor(Color.yellow);
    }
}
```

```
}

public void callback(Graphics g){
    Timer blinkTimer= new Timer(800, new ActionListener() {
        public void actionPerformed(ActionEvent e) {

Graphics g= getGraphics();
if(null==g){
    return;
}
I_color(g);
g.drawLine(0, 0, 0, y);
g.drawLine(0, 0, x, 0);
g.drawLine(x, y, x, 0);
g.drawLine(x, y, 0, y);
I_color(g);
g.drawLine(2, 2, 2, y-2);
g.drawLine(2, 2, x-2, 2);
g.drawLine(x-2, y-2, x-2, 2);
g.drawLine(x-2, y-2, 2, y-2);
I_color(g);
g.drawLine(4, 4, 4, y-4);
g.drawLine(4, 4, x-4, 4);
g.drawLine(x-4, y-4, x-4, 4);
g.drawLine(x-4, y-4, 4, y-4);
I_color(g);
g.drawLine(6, 6, 6, y-6);
g.drawLine(6, 6, x-6, 6);
g.drawLine(x-6, y-6, x-6, 6);
g.drawLine(x-6, y-6, 6, y-6);
I_color(g);
g.drawLine(8, 8, 8, y-8);
g.drawLine(8, 8, x-8, 8);
g.drawLine(x-8, y-8, x-8, 8);
g.drawLine(x-8, y-8, 8, y-8);
        }

        private void I_color(Graphics g) {
int x=1+new Random().nextInt(9);
if(1==x){
    g.setColor(Color.BLACK);
}
if(2==x){
    g.setColor(Color.blue);
}
if(3==x){
    g.setColor(Color.cyan);
}
if(4==x){
    g.setColor(Color.DARK_GRAY);
}
```



```

    }

    if(5==x){

        g.setColor(Color.green);

    }

    if(6==x){

        g.setColor(Color.orange);

    }

    if(7==x){

        g.setColor(Color.pink);

    }

    if(8==x){

        g.setColor(Color.pink);

    }

    if(9==x){

        g.setColor(Color.yellow);

    }

        }

    });

    blinkTimer.start();

}

}

/Trif_DNA-master/C_L_M/M_V/MVQ/button/HrjButtonTest.java
/Trif_DNA-master/C_L_M/M_V/MVQ/button/YLJShowButton.java
package M_V.MVQ.button;

import javax.swing.ImageIcon;
import javax.swing.JButton;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Image;
import java.awt.event.MouseEvent;
import java.awt.event.MouseListener;
import java.awt.event.MouseMotionListener;
import java.awt.event.MouseWheelEvent;
import java.awt.event.MouseWheelListener;
import java.awt.image.BufferedImage;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class YLJShowButton extends JButton implements MouseListener
, MouseMotionListener, MouseWheelListener{

    /**
     * Author Yaoguang Luo

```

```
*/

public Image img;
public ImageIcon icon;
public int pointX= 0;
public int pointY= 0;
public int pointZ= 0;
public int coX= 0;
public int coY= 0;
public int press= 0;

public BufferedImage bufferedImage;
public void fix() {
    coX= 0;
    coY= 0;
    pointZ= 0;
}

public YLJShowButton(String string, final int x, final int y, java.awt.Color c) {
    super(string);
    this.setBackground(c);
    this.addMouseMotionListener(new MouseMotionListener() {
        int motionI= 1;
        public void mouseDragged(MouseEvent e) {
        }
        public void mouseMoved(MouseEvent e) {
Color reg= Color.black;
if(1== motionI++) {
    reg= Color.black;
}else if(2== motionI) {
    reg= Color.blue;
}else if(3== motionI) {
    motionI= 1;
    reg= Color.green;
}
Graphics g= getGraphics();
g.setColor(reg);
int xPoints[]= {1, 1+ 10, 1};
int yPoints[]= {1, 1, 1+ 10};
g.fillPolygon(xPoints, yPoints, 3);
        }
    });
}

public YLJShowButton(String string) {
    super(string);
    int i= 1+ (int)(Math.abs(Math.random()*100))% 4;
    Color regBackground= null;
    if(1== i) {
```

```

        regBackground= Color.orange;
    }else if(2== i) {
        regBackground= Color.magenta;
    }else if(3== i) {
        regBackground= Color.green;
    }else if(4== i) {
        regBackground= Color.pink;
    }
    this.setBackground(regBackground);
    this.addMouseMotionListener(new MouseMotionListener() {
        int motionI= 1;
        public void mouseDragged(MouseEvent e) {
        }
        public void mouseMoved(MouseEvent e) {
Color reg= Color.black;
if(1== motionI++) {
    reg= Color.black;
}else if(2== motionI) {
    reg= Color.blue;
}else if(3== motionI) {
    motionI= 1;
    reg= Color.green;
}
Graphics g= getGraphics();
g.setColor(reg);
int xPoints[]= {1, 1+ 10, 1};
int yPoints[]= {1, 1, 1+ 10};
g.fillPolygon(xPoints, yPoints, 3);
        }
    });
    this.addMouseListener(this);
    this.addMouseWheelListener(this);
    this.addMouseMotionListener(this);
}
@Override
public void mouseWheelMoved(MouseWheelEvent e) {

    pointZ+= e.getWheelRotation()* 100;
    pointZ= pointZ> 1200? 1200: pointZ;
    pointZ= pointZ< 50? 50: pointZ;
    Image imag= this.bufferedImage.getScaledInstance(605+ pointZ
, 365+ pointZ, java.awt.Image.SCALE_SMOOTH);
    ImageIcon bufferIcon= new ImageIcon();
    bufferIcon.setImage(imag);
    this.removeAll();
    this.setIcon(bufferIcon);
    this.setBounds(coX, coY, 605+ pointZ, 365+ pointZ);

```

```
        this.validate();
    }

    @Override
    public void mouseDragged(MouseEvent e) {

    }

    @Override
    public void mouseMoved(MouseEvent e) {

        ///20230106-S_logger.Log.logger.info("" + e.getButton());
    }

    @Override
    public void mouseClicked(MouseEvent e) {

        ///20230106-S_logger.Log.logger.info("" + e.getButton());
    }

    @Override
    public void mousePressed(MouseEvent e) {
        press= 1;
        pointX= e.getX();
        pointY= e.getY();
    }

    @Override
    public void mouseReleased(MouseEvent e) {
        press= 0;
        int mX= e.getX()- pointX;
        int mY= e.getY()- pointY;
        coX= coX+ mX;
        coY= coY+ mY;
        coX= coX> this.getWidth()? -this.getWidth(): coX;
        coY= coY> this.getHeight()? -this.getHeight(): coY;
        Image imag= this.bufferedImage.getScaledInstance(605+ pointZ
, 365+ pointZ, java.awt.Image.SCALE_SMOOTH);
        ImageIcon bufferIcon= new ImageIcon();
        bufferIcon.setImage(imag);
        this.removeAll();
        this.setIcon(bufferIcon);
        this.setBounds(coX, coY, 605+ pointZ, 365+ pointZ);
        this.validate();
    }

    @Override
    public void mouseEntered(MouseEvent e) {

    }

    @Override
    public void mouseExited(MouseEvent e) {
```

```

    }
}

/Trif_DNA-master/C_L_M/M_V/MVQ/button/YLJShowButtonTest.java

    M_V.MVQ.filenameFilter

/Trif_DNA-master/C_L_M/M_V/MVQ/filenameFilter/TXTFilter.java

    M_V.MVQ.label

/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaIndexLabel.java
package M_V.MVQ.label;

import ME.VPC.H.hospital.DetaLabelS_s;

import javax.swing.Box;

import javax.swing.JFrame;

import javax.swing.JLabel;

import java.awt.Color;

import java.awt.event.MouseEvent;

import java.awt.event.MouseListener;

import java.util.Iterator;

import java.util.List;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class DetaIndexLabel extends JLabel implements MouseListener{

    public String tagName= "";

    public Box box;

    //先碎片函数调试, 之后面板函数集成

    public JFrame frameTag= new JFrame("提示栏");

    public boolean ShowTag= false;

    public boolean inited= false;

    public int openx=0;

    public int openy=0;

    public int closex=0;

    public int closey=0;

    public boolean isIndex= false;

    public List<DetaLabel> jFrameList;

    public String indexString;

    //罗瑶光

    //为了避免花屏, 影响操作, 逻辑设计为点击打开, 鼠标移动触发关闭.

    public DetaIndexLabel(String name, List<DetaLabel> jFrameList) {

        super(name);

```

```
        this.setForeground(Color.lightGray);
        this.setBackground(Color.black);
        this.addMouseListener(this);
        this.jFrameList= jFrameList;
    }
    public void setTag(String string, boolean True) {

    }
    public void setIndex(String string) {
        if(null==DetaLabelS_s.stableImages){
            return;//mock trif
        }
        DetaLabelS_s.stableImages.setIndexString(string);
        this.indexString= string;
    }
    @Override
    public void mouseClicked(MouseEvent e) {
        if(!DetaLabelS_s.stableImages.isIndexed) {
            return;
        }
        if(this.indexString.equalsIgnoreCase("A")) {
            //do..
            Iterator<DetaLabel> iterator= jFrameList.iterator();
            while(iterator.hasNext()) {
                DetaLabel detaLabel= iterator.next();
                if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("A")) {
                    detaLabel.setForeground(Color.pink);
                }else {
                    detaLabel.setForeground(Color.white);
                }
                detaLabel.validate();
            }
            return;
        }
        if(this.indexString.equalsIgnoreCase("B")) {
            //do..
            Iterator<DetaLabel> iterator= jFrameList.iterator();
            while(iterator.hasNext()) {
                DetaLabel detaLabel= iterator.next();
                if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("B")) {
                    detaLabel.setForeground(Color.pink);
                }else {
                    detaLabel.setForeground(Color.white);
                }
                detaLabel.validate();
            }
            return;
        }
    }
```

```
}  
    if(this.indexString.equalsIgnoreCase("C")) {  
        //do..  
        Iterator<DetaLabel> iterator= jFrameList.iterator();  
        while(iterator.hasNext()) {  
DetaLabel detaLabel= iterator.next();  
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("C")) {  
    detaLabel.setForeground(Color.pink);  
}else {  
    detaLabel.setForeground(Color.white);  
}  
detaLabel.validate();  
    }  
    return;  
}  
    if(this.indexString.equalsIgnoreCase("D")) {  
        //do..  
        Iterator<DetaLabel> iterator= jFrameList.iterator();  
        while(iterator.hasNext()) {  
DetaLabel detaLabel= iterator.next();  
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("D")) {  
    detaLabel.setForeground(Color.pink);  
}else {  
    detaLabel.setForeground(Color.white);  
}  
detaLabel.validate();  
    }  
    return;  
}  
    if(this.indexString.equalsIgnoreCase("E")) {  
        //do..  
        Iterator<DetaLabel> iterator= jFrameList.iterator();  
        while(iterator.hasNext()) {  
DetaLabel detaLabel= iterator.next();  
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("E")) {  
    detaLabel.setForeground(Color.pink);  
}else {  
    detaLabel.setForeground(Color.white);  
}  
detaLabel.validate();  
    }  
    return;  
}  
    if(this.indexString.equalsIgnoreCase("F")) {  
        //do..  
        Iterator<DetaLabel> iterator= jFrameList.iterator();  
        while(iterator.hasNext()) {
```

```
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("F")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("G")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("G")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("H")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("H")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("I")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("I")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
```



```
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("J")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("J")) {
    detaLabel.setForeground(Color.pink);
} else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("K")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("K")) {
    detaLabel.setForeground(Color.pink);
} else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("L")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("L")) {
    detaLabel.setForeground(Color.pink);
} else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
```

```
        if(this.indexString.equalsIgnoreCase("M")) {
            //do..
            Iterator<DetaLabel> iterator= jFrameList.iterator();
            while(iterator.hasNext()) {
                DetaLabel detaLabel= iterator.next();
                if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("M")) {
                    detaLabel.setForeground(Color.pink);
                }else {
                    detaLabel.setForeground(Color.white);
                }
                detaLabel.validate();
            }
            return;
        }
        if(this.indexString.equalsIgnoreCase("N")) {
            //do..
            Iterator<DetaLabel> iterator= jFrameList.iterator();
            while(iterator.hasNext()) {
                DetaLabel detaLabel= iterator.next();
                if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("N")) {
                    detaLabel.setForeground(Color.pink);
                }else {
                    detaLabel.setForeground(Color.white);
                }
                detaLabel.validate();
            }
            return;
        }
        if(this.indexString.equalsIgnoreCase("O")) {
            //do..
            Iterator<DetaLabel> iterator= jFrameList.iterator();
            while(iterator.hasNext()) {
                DetaLabel detaLabel= iterator.next();
                if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("O")) {
                    detaLabel.setForeground(Color.pink);
                }else {
                    detaLabel.setForeground(Color.white);
                }
                detaLabel.validate();
            }
            return;
        }
        if(this.indexString.equalsIgnoreCase("P")) {
            //do..
            Iterator<DetaLabel> iterator= jFrameList.iterator();
            while(iterator.hasNext()) {
                DetaLabel detaLabel= iterator.next();
```

```
        if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("P")) {
            detaLabel.setForeground(Color.pink);
        }else {
            detaLabel.setForeground(Color.white);
        }
        detaLabel.validate();
    }
    return;
}

if(this.indexString.equalsIgnoreCase("Q")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("Q")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}

if(this.indexString.equalsIgnoreCase("R")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("R")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}

if(this.indexString.equalsIgnoreCase("S")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("S")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
}
```

```
detaLabel.validate();
    }
    return;
}

if(this.indexString.equalsIgnoreCase("T")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("T")) {
    detaLabel.setForeground(Color.pink);
} else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}

if(this.indexString.equalsIgnoreCase("U")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("U")) {
    detaLabel.setForeground(Color.pink);
} else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}

if(this.indexString.equalsIgnoreCase("V")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("V")) {
    detaLabel.setForeground(Color.pink);
} else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}

if(this.indexString.equalsIgnoreCase("W")) {
```

```
//do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("W")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("X")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("X")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("Y")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("Y")) {
    detaLabel.setForeground(Color.pink);
}else {
    detaLabel.setForeground(Color.white);
}
detaLabel.validate();
    }
    return;
}
if(this.indexString.equalsIgnoreCase("Z")) {
    //do..
    Iterator<DetaLabel> iterator= jFrameList.iterator();
    while(iterator.hasNext()) {
DetaLabel detaLabel= iterator.next();
if(null!= detaLabel&& null!= detaLabel.indexString&& detaLabel.indexString.contains("Z")) {
```

```

        detaLabel.setForeground(Color.pink);
    } else {
        detaLabel.setForeground(Color.white);
    }
    detaLabel.validate();
    }
    return;
}

}

@Override
public void mousePressed(MouseEvent e) {

}

@Override
public void mouseReleased(MouseEvent e) {
}

@Override
public void mouseEntered(MouseEvent e) {

}

@Override
public void mouseExited(MouseEvent e) {

}

}

}

/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaLabel.java
package M_V.MVQ.label;

import ME.VPC.H.hospital.DetaLabelS_s;
import javax.swing.Box;
import javax.swing.JFrame;
import javax.swing.JLabel;
import java.awt.Color;
import java.awt.Point;
import java.awt.event.MouseEvent;
import java.awt.event.MouseListener;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class DetaLabel extends JLabel {

    public String tagName = "";
    public JLabel jLabelTop;

```

```
public JLabel jLabelMed;

public JLabel jLabelBot;

public Box box;

//先碎片函数调试,之后面板函数集成

public JFrame frameTag = new JFrame("提示栏");

;

public boolean ShowTag = false;

public boolean inited = false;

public int openx = 0;

public int openy = 0;

public int closex = 0;

public int closey = 0;

public boolean isIndex = false;

public String indexString;

//罗瑶光

//为了避免花屏,影响操作,逻辑设计为点击打开,鼠标移动触发关闭.

public DetaLabel(String name) {

    super(name);

    this.setForeground(Color.lightGray);

    this.setBackground(Color.black);

    this.addMouseListener(new MouseListener() {

        @SuppressWarnings("deprecation")

        @Override

        public void mouseClicked(MouseEvent arg0) {

            if (!ShowTag) {

                return;

            }

            if (!inited) {

                //jlabel最简分行

                jLabelTop = new JLabel(tagName.substring(0, tagName.length() > 70 ? 70 : tagName.length()));

                jLabelTop.setBounds(0, 0, 900, 30);

                frameTag = new JFrame("提示栏");

                frameTag.setLayout(null);

                frameTag.setSize(900, 60);

                if (tagName.length() > 70) {

                    jLabelMed = new JLabel(tagName.substring(70, tagName.length() > 140 ? 140 : tagName.length()));

                    jLabelMed.setBounds(0, 30, 900, 30);

                    frameTag.setSize(900, 120);

                }

                if (tagName.length() > 140) {

                    jLabelBot = new JLabel(tagName.substring(140, tagName.length()));

                    jLabelBot.setBounds(0, 60, 900, 30);

                }

                //摘自第37行 https://blog.csdn.net/code\_better/article/details/53505962

                Point point = java.awt.MouseInfo.getPointerInfo().getLocation();

                //

                frameTag.setLocation(point.x, point.y);

            }

        }

    });

}
```

```

        frameTag.setResizable(false);
        frameTag.add(jLabelTop);
        if (tagName.length() > 70) {
            frameTag.add(jLabelMed);
        }
        if (tagName.length() > 140) {
            frameTag.add(jLabelBot);
        }
        frameTag.setVisible(true);
        frameTag.show();
        DetaLabelS_s.stableImages.setTarget(true);
        frameTag.setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE);
        openx = point.x;
        openy = point.y;
        inited = true;
        return;
    } else {
        if (!frameTag.isShowing()) {
            Point point = java.awt.MouseInfo.getPointerInfo().getLocation();
            frameTag.setSize(900, 150);
            frameTag.setLocation(point.x, point.y);
            frameTag.setVisible(true);
            frameTag.show();
            DetaLabelS_s.stableImages.setTarget(true);
            openx = point.x;
            openy = point.y;
            return;
        }
    }
}

@Override
public void mouseEntered(MouseEvent arg0) {
}

@Override
public void mouseExited(MouseEvent arg0) {
}

@Override
public void mousePressed(MouseEvent arg0) {
}

@Override
public void mouseReleased(MouseEvent arg0) {
}
});

//find and set tag
if (DetaLabelConjunction.detaLabelConjunction == null) {
    return;
}

```



```

DetaLabelConjunction.detaLabelConjunction.findAndSetTag(this, name);
}

public void setTag(String string, boolean True) {
    ShowTag = True;
    tagName = null == string ? "" : string.toString();
    if (null != jLabelTop) {
        jLabelTop.setText(tagName.substring(0, tagName.length() > 70 ? 70 : tagName.length()));
    } else {
        jLabelTop = new JLabel(tagName.substring(0, tagName.length() > 70 ? 70 : tagName.length()));
    }
    if (tagName.length() > 70) {
        if (null != jLabelBot) {
            jLabelMed.setText(tagName.substring(70, tagName.length() > 140 ? 140 : tagName.length()));
        } else {
            jLabelMed = new JLabel(tagName.substring(70, tagName.length() > 140 ? 140 : tagName.length()));
        }
    }
    if (tagName.length() > 140) {
        if (null != jLabelBot) {
            jLabelBot.setText(tagName.substring(140, tagName.length()));
        } else {
            jLabelBot = new JLabel(tagName.substring(140, tagName.length()));
        }
    }
}

//Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaLabelConjunction.java
package M_V.MVQ.label;

import exception.file.O.DetaBufferedReader;
import exception.file.O.DetaInputStreamReader;
import S_A.pheromone.IMV_SQI;
//import org.reportMap.ReportIndex;
import java.io.IOException;
import java.io.InputStream;
import java.util.Iterator;
import java.util.Map;

//设计 罗瑶光
/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class DetaLabelConjunction {
    public static DetaLabelConjunction detaLabelConjunction;

```

```
public Map<String, Boolean> checkSets = new IMV_SQI();

public void IV_() {
//    try {
//        InputStream inputStream = new ReportIndex().getClass()
//            .getResourceAsStream("report.txt");
//        DetabufferedReader cReader = new DetabufferedReader(
//            DetaInputStreamReader.E(inputStream));
//        String ctempString = null;
//        while (true) {
//            if (!(ctempString = cReader.readDetaLine()) != null)) break;
//            if (ctempString.replace(" ", "").equals(""))
//            if (!detaLabelConjunction.checkSets.containsKey(ctempString)) {
//                detaLabelConjunction.checkSets.put(ctempString, true);
//            }
//        }
//    } catch (IOException e) {
//        throw new RuntimeException(e);
//    }
}

public void findAndSetTag(DetaLabel detaLabel, String name) {
    Iterator<String> iterator = detaLabelConjunction.checkSets.keySet().iterator();
    while (iterator.hasNext()) {
        String string = iterator.next();
        String subString = name;
        subString = name.substring(0, 1);
        if (name.length() > 2) {
            subString = name.substring(0, 2);
        }
        if (name.length() > 5) {
            subString = name.substring(0, 5);
        }
        if (name.length() > 7) {
            subString = name.substring(0, 7);
        }
        if (name.length() > 8) {
            subString = name.substring(0, 8);
        }
        if (name.length() > 10) { //之后太长截的取中文的部分
            subString = name.substring(0, 7);
        }
        if (string.substring(5, 6 + subString.length())
            .toUpperCase().contains(subString.toUpperCase())) {
            detaLabel.setTag(string, true);
        }
    }
}
```

```

    }
}

/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaLabelConjunctionTest.java
/Trif_DNA-master/C_L_M/M_V/MVQ/label/DetaLabelTest.java

    M_V.MVQ.slider

    /Trif_DNA-master/C_L_M/M_V/MVQ/slider/DetaSlider.java

    M_V.MVQ.tabbedpane

/Trif_DNA-master/C_L_M/M_V/MVQ/tabbedpane/DetabbedPane.java
package M_V.MVQ.tabbedpane;

import ME.VPC.M.app.App;

import exception.thread.DetaThread;

import javax.swing.BorderFactory;

import javax.swing.JTabbedPane;

import javax.swing.event.ChangeEvent;

import javax.swing.event.ChangeListener;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
@SuppressWarnings("unused")
public class DetabbedPane extends JTabbedPane implements ChangeListener {

    ////

    private int x, y = 0;

    public String lastName = "";

    App NE;

    public DetabbedPane(final int x, final int y, java.awt.Color c, App app_NE) {

        super();

        this.setBorder(BorderFactory.createRaisedBevelBorder());

        this.setOpaque(true);

        this.setBackground(c);

        NE = app_NE;

    }

    public DetabbedPane() {

        super();

    }

    @Override

    public void stateChanged(ChangeEvent arg0) {

        if (null != arg0) {

            return;

        }

        if (null == NE) {

```

```
        return;
    }
    if (null == NE.app_S) {
        return;
    }
    if (null != NE.app_S.line2025) {
        if (null != NE.app_S.line2025.detaButton) {
            if (NE.app_S.jTabbedPane.lastName.contains("智能声学")) {
                NE.app_S.line2025.detaButton.doClick();
            }
        }
    }
    DetaThread.sleepDeta(1000);
    if (null != NE.app_S.monitor) {
        if (null != NE.app_S.monitor.monitor_X_S) {
            if (null != NE.app_S.monitor.monitor_X_S.bt00) {
                if (NE.app_S.jTabbedPane.lastName.contains("智能视觉")) {
                    NE.app_S.monitor.monitor_X_S.bt00.doClick();
                    NE.app_S.monitor.monitor_X_S.isStop = true;
                }
            }
        }
    }
    DetaThread.sleepDeta(500);
    DetaThread.sleepDeta(500);
    if (null != NE.app_S.jTabbedPane) {
        NE.app_S.jTabbedPane.getSelectedComponent().validate();
        DetaThread.sleepDeta(500);
    }
    //
    NE.app_S.jTabbedPane.lastName = NE.app_S.jTabbedPane
        .getSelectedComponent().getName();
}
}
//46
//    if (null != NE.app_S.soundWaveJPanelNew) {
//        if (NE.app_S.reg != 0) {
//            NE.app_S.reg = 0;
//            if (null != NE.app_S.soundWaveJPanelNew.detaButton) {
//                NE.app_S.soundWaveJPanelNew.detaButton.doClick();
//            }
//            NE.app_S.jTabbedPane.updateUI();
//        }
//    }
//    if (null != NE.app_S.monitor) {
//        if (null != NE.app_S.monitor.monitor_X_S) {
//            if (null != NE.app_S.monitor.monitor_X_S.bt00) {
```

```
//      if (!NE.app_S.monitor.monitor_X_S.isStop) {
//          NE.app_S.monitor.monitor_X_S.isStop = true;
//          //DetaThread.sleepDeta(200);
//          NE.app_S.jTabbedPane.updateUI();
//      }
//  }
//  }
//  }

/Trif_DNA-master/C_L_M/M_V/MVQ/tabbedPane/DeTabbedPaneTest.java

    M_V.MVQ.tableRender

/Trif_DNA-master/C_L_M/M_V/MVQ/tableRender/ColorTableRender.java
package M_V.MVQ.tableRender;

import ME.VPC.M.app.App;

import javax.swing.JTable;

import javax.swing.table.DefaultTableCellRenderer;

import java.awt.Color;

import java.awt.Component;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
 * 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class ColorTableRender extends DefaultTableCellRenderer {

    App NE;

    public ColorTableRender(App app_NE) {
        NE = app_NE;
    }

    public Component getTableCellRendererComponent(JTable table, Object value
        , boolean isSelected, boolean hasFocus, int row,
            int column) {
        if (isSelected && hasFocus && row == table.getSelectedRow()
            && column == table.getSelectedColumn()) {
            //2.设置当前Cell的颜色
            Component c = super.getTableCellRendererComponent(table, value, isSelected
                , hasFocus, row, column);
            c.setBackground(Color.CYAN);//设置背景色
            c.setForeground(Color.gray);//设置前景色
            return c;
        } else {
            //3.设置单数行, 偶数行的颜色
            if (row % 3 == 0) { //偶数行时的颜色 罗瑶光
```

```

        setBackground(new Color(253
            , 255 - NE.app_S.color_bg_rot
            , 254));
    } else if (row % 3 == 1) { //设置单数行的颜色
        setBackground(new Color(255 - NE.app_S.color_bg_rot
            , 254
            , 255 - NE.app_S.color_bg_rot));
    } else if (row % 3 == 2) { //设置单数行的颜色
        setBackground(new Color(255
            , 251
            , 255 - NE.app_S.color_bg_rot));
    }
    return super.getTableCellRendererComponent(table, value,
        isSelected, hasFocus, row, column);
}
}
}
}
/Trif_DNA-master/C_L_M/M_V/MVQ/tableRender/ColorTableRenderTest.java

```

M_V.MVQ.textarea

```

/Trif_DNA-master/C_L_M/M_V/MVQ/textarea/CfxTextArea.java
package M_V.MVQ.textarea;

import javax.swing.BorderFactory;
import javax.swing.JTextArea;
import javax.swing.Timer;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Graphics;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseEvent;
import java.awt.event.MouseMotionListener;
import java.util.Random;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class CfxTextArea extends JTextArea {

    private int x, y=0;

    public CfxTextArea(String string, final int x, final int y, java.awt.Color c) {
        super(string);
        this.x=x;
    }
}

```

```

        this.y=y;
        this.setBorder(BorderFactory.createRaisedBevelBorder());
        this.setPreferredSize(new Dimension(this.x+1, this.y+1));
        this.setOpaque(true);
        this.setBackground(c);
        this.addMouseListener(new MouseMotionListener() {
            public void mouseDragged(MouseEvent e) {
            }
            public void mouseMoved(MouseEvent e) {
//
                Graphics g= getGraphics();
//
                g.setColor(new Color(255, 0, 255, 128));
//
                int xPoints[]={8, 8+15, 8};
//
                int yPoints[]={8, 8, 8+15};
//
                g.fillPolygon(xPoints, yPoints, 3);
//
                int xPoints1[]={x-16-5, x-8, x-8};
//
                int yPoints1[]={y-8, y-16-5, y-8};
//
                g.fillPolygon(xPoints1, yPoints1, 3);
//
                g.fillRect(8, 8, x-16 , 5);
//
                g.fillRect(8, 8, 5 , y-16);
//
                g.fillRect(8, y-12, x-16 , 5);
//
                g.fillRect(x-12, 8, 5 , y-16);
            }
        });
        //Graphics g= getGraphics();
        //this.callback(g);
    }
    public void callback(Graphics g){
        Timer blinkTimer= new Timer(200, new ActionListener() {
            public void actionPerformed(ActionEvent e) {
Graphics g= getGraphics();
if(null==g){
            return;
        }
        I_color(g);
        g.drawLine(0, 0, 0, y);
        g.drawLine(0, 0, x, 0);
        g.drawLine(x, y, x, 0);
        g.drawLine(x, y, 0, y);
        I_color(g);
        g.drawLine(2, 2, 2, y-2);
        g.drawLine(2, 2, x-2, 2);
        g.drawLine(x-2, y-2, x-2, 2);
        g.drawLine(x-2, y-2, 2, y-2);
        I_color(g);
        g.drawLine(4, 4, 4, y-4);
        g.drawLine(4, 4, x-4, 4);
        g.drawLine(x-4, y-4, x-4, 4);

```

```
g.drawLine(x-4, y-4, 4, y-4);
I_color(g);
g.drawLine(6, 6, 6, y-6);
g.drawLine(6, 6, x-6, 6);
g.drawLine(x-6, y-6, x-6, 6);
g.drawLine(x-6, y-6, 6, y-6);
I_color(g);
g.drawLine(8, 8, 8, y-8);
g.drawLine(8, 8, x-8, 8);
g.drawLine(x-8, y-8, x-8, 8);
g.drawLine(x-8, y-8, 8, y-8);
    }

    private void I_color(Graphics g) {
int x=1+new Random().nextInt(9);
if(1==x){
    g.setColor(Color.BLACK);
}
if(2==x){
    g.setColor(Color.blue);
}
if(3==x){
    g.setColor(Color.cyan);
}
if(4==x){
    g.setColor(Color.DARK_GRAY);
}
if(5==x){
    g.setColor(Color.green);
}
if(6==x){
    g.setColor(Color.orange);
}
if(7==x){
    g.setColor(Color.pink);
}
if(8==x){
    g.setColor(Color.pink);
}
if(9==x){
    g.setColor(Color.yellow);
}
    }

    });
    blinkTimer.start();
}
}
```


M_V.MVQ.textfield

```
/Trif_DNA-master/C_L_M/M_V/MVQ/textfield/CfxTextField.java
package M_V.MVQ.textfield;

import javax.swing.BorderFactory;
import javax.swing.JTextField;
import javax.swing.Timer;
import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.RenderingHints;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseEvent;
import java.awt.event.MouseMotionListener;
import java.util.Random;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class CfxTextField extends JTextField {

    private int x, y= 0;

    @Override
    public void paintComponent(Graphics g){
        super.paintComponent(g);
        Graphics2D g2=(Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setStroke(new BasicStroke(2.0f));
        g2.setColor(Color.blue);
        g2.drawArc( 1, 1, 15, 15, 170, -90);
        g2.drawArc( 670, 1, 10, 10, 90, -90);
        g2.drawLine( 5, 1, 670, 1);
        g2.drawLine( 1, 5, 1, 42);
        g2.drawLine(670, 5, 670, 42);
        g2.drawLine( 1, 42, 670, 42);
    }

    public CfxTextField(String string, final int x, final int y, java.awt.Color c) {
        super(string);
        this.x= x;
        this.y= y;
        this.setBorder(BorderFactory.createRaisedBevelBorder());
    }
}
```

```

        this.setPreferredSize(new Dimension(this.x+1, this.y+1));

        this.setOpaque(true);

        this.setBackground(c);

        this.addMouseMotionListener(new MouseMotionListener() {

            public void mouseDragged(MouseEvent e) {

            }

            public void mouseMoved(MouseEvent e) {

//                Graphics g= getGraphics();
//                g.setColor(new Color(255, 0, 255, 128));
//                int xPoints[]={8, 8+15, 8};
//                int yPoints[]={8, 8, 8+15};
//                g.fillPolygon(xPoints, yPoints, 3);
//                int xPoints1[]={x-16-5, x-8, x-8};
//                int yPoints1[]={y-8, y-16-5, y-8};
//                g.fillPolygon(xPoints1, yPoints1, 3);
//                g.fillRect(8, 8, x-16 , 5);
//                g.fillRect(8, 8, 5 , y-16);
//                g.fillRect(8, y-12, x-16 , 5);
//                g.fillRect(x-12, 8, 5 , y-16);
            }

        });

        //Graphics g= getGraphics();

        //this.callback(g);

    }

    public void callback(Graphics g){

        Timer blinkTimer= new Timer(200, new ActionListener() {

            public void actionPerformed(ActionEvent e) {

Graphics g= getGraphics();
if(null==g){

            return;

        }

        I_color(g);

        g.drawLine(0, 0, 0, y);

        g.drawLine(0, 0, x, 0);

        g.drawLine(x, y, x, 0);

        g.drawLine(x, y, 0, y);

        I_color(g);

        g.drawLine(2, 2, 2, y-2);

        g.drawLine(2, 2, x-2, 2);

        g.drawLine(x-2, y-2, x-2, 2);

        g.drawLine(x-2, y-2, 2, y-2);

        I_color(g);

        g.drawLine(4, 4, 4, y-4);

        g.drawLine(4, 4, x-4, 4);

        g.drawLine(x-4, y-4, x-4, 4);

        g.drawLine(x-4, y-4, 4, y-4);

        I_color(g);

```

```
g.drawLine(6, 6, 6, y-6);
g.drawLine(6, 6, x-6, 6);
g.drawLine(x-6, y-6, x-6, 6);
g.drawLine(x-6, y-6, 6, y-6);
L_color(g);
g.drawLine(8, 8, 8, y-8);
g.drawLine(8, 8, x-8, 8);
g.drawLine(x-8, y-8, x-8, 8);
g.drawLine(x-8, y-8, 8, y-8);
    }
    private void L_color(Graphics g) {
int x=1+new Random().nextInt(9);
if(1==x){
    g.setColor(Color.BLACK);
}
if(2==x){
    g.setColor(Color.blue);
}
if(3==x){
    g.setColor(Color.cyan);
}
if(4==x){
    g.setColor(Color.DARK_GRAY);
}
if(5==x){
    g.setColor(Color.green);
}
if(6==x){
    g.setColor(Color.orange);
}
if(7==x){
    g.setColor(Color.pink);
}
if(8==x){
    g.setColor(Color.pink);
}
if(9==x){
    g.setColor(Color.yellow);
}
    }
    });
    blinkTimer.start();
}
}
```

M_V.MVQ.textpane

/Trif_DNA-master/C_L_M/M_V/MVQ/textpane/CfxTextPane.java

```
package M_V.MVQ.textpane;
import javax.swing.BorderFactory;
import javax.swing.JTextPane;
import java.awt.Dimension;
import java.awt.event.MouseEvent;
import java.awt.event.MouseMotionListener;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class CfxTextPane extends JTextPane {
    private int x, y=0;
    public CfxTextPane(String string, final int x, final int y, java.awt.Color c) {
        super();
        this.x=x;
        this.y=y;
        this.setBorder(BorderFactory.createRaisedBevelBorder());
        this.setPreferredSize(new Dimension(this.x+1, this.y+1));
        this.setOpaque(true);
        this.setBackground(c);
        this.addMouseMotionListener(new MouseMotionListener() {
            public void mouseDragged(MouseEvent e) {
            }
            public void mouseMoved(MouseEvent e) {
            }
        });
    }
}
```

1-7-/Trif_DNA-master/C_L_O-操作-tinshell-数据库-VPCS-文档

O_V.OM.config

/Trif_DNA-master/C_L_O/O_V/OM/config/Config.java

O_V.OP.SM.AOP.MEC.SIQ.SM.reflection

```
/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Base.java
package O_V.OP.SM.AOP.MEC.SIQ.SM.reflection;
import java.util.concurrent.ConcurrentHashMap;
/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
 ** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段

* 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class Base{

    public ConcurrentHashMap<String, Table> getTables() {

        return tables;

    }

    public void I_Tables(ConcurrentHashMap<String, Table> tables) {

        this.tables= tables;

    }

    public Table getTable(String tableName) {

        if(tables.containsKey(tableName)) {

            return tables.get(tableName);

        }

        return null;

    }

    public void putTable(String tableName, Table table) {

        this.tables.put(tableName, table);

    }

    private ConcurrentHashMap<String, Table> tables;

}

//Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/BaseTest.java
//Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Cell.java
package O_V.OP.SM.AOP.MEC.SIQ.SM.reflection;
```

/*

* 个人著作权人, 作者 罗瑶光. 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class Cell{

    public Object getCellValue() {

        return cellValue;

    }

    public String getCellStringValue() {

        return cellValue.toString();

    }

    public void I_CellValue(Object cellValue) {

        this.cellValue= cellValue;

    }

    private Object cellValue;

}
```

```
//Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/DB.java
package O_V.OP.SM.AOP.MEC.SIQ.SM.reflection;
```

```
import java.util.concurrent.ConcurrentHashMap;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光. 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年后因G网屏蔽不再使用)
```

```
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
```

```
* 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
public class DB{
```

```
    private ConcurrentHashMap<String, Base> bases;
```

```
    public ConcurrentHashMap<String, Base> getBases() {
```

```
        return bases;
```

```
    }
```

```
    public void I_Bases(ConcurrentHashMap<String, Base> bases) {
```

```
        this.bases= bases;
```

```
    }
```

```
    public Base getBase(String baseName) {
```

```
        if(bases.containsKey(baseName)) {
```

```
            return bases.get(baseName);
```

```
        }
```

```
        return null;
```

```
    }
```

```
    public void putBase(String baseName, Base base) {
```

```
        this.bases.put(baseName, base);
```

```
    }
```

```
}
```

```
//Trif_DNA-master/C-L-O/O-V/OP/SM/AOP/MEC/SIQ/SM/reflection/DBTest.java  
//Trif_DNA-master/C-L-O/O-V/OP/SM/AOP/MEC/SIQ/SM/reflection/Row.java  
package O_V.OP.SM.AOP.MEC.SIQ.SM.reflection;
```

```
import java.util.concurrent.ConcurrentHashMap;
```

```
/*
```

```
* 个人著作权人, 作者 罗瑶光. 浏阳
```

```
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
```

```
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
```

```
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

```
* 430181198505250014, G24402609, EB0581342
```

```
* 204925063, 389418686, F2406501, 0626136
```

```
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
```

```
**/
```

```
//我在想要不要数据库，要了麻烦。内存增量巨恶心。
```

```
public class Row{
```

```
    private ConcurrentHashMap<String, Cell> cells;
```

```
    public boolean containsCell(String cellName) {
```

```

        if(cells.containsKey(cellName)) {

            return true;

        }

        return false;

    }

    public ConcurrentHashMap<String, Cell> getCells() {

        return cells;

    }

    public void I_Cells(ConcurrentHashMap<String, Cell> cells) {

        this.cells= cells;

    }

    public Cell getCell(String cellName) {

        if(cells.containsKey(cellName)) {

            return cells.get(cellName);

        }

        return null;

    }

    public void putCell(String cellName, Cell cell) {

        this.cells.put(cellName, cell);

    }

    public void D_Cell(String cellName) {

        if(cells.containsKey(cellName)) {

            cells.remove(cellName);

        }

    }

}

//Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/RowTest.java
//Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Spec.java
package O_V.OP.SM.AOP.MEC.SIQ.SM.reflection;

import S_A.pheromone.IMV_SQI;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Spec{

    private IMV_SQI culumTypes;

    public IMV_SQI getCulumTypes() {

        return culumTypes;

    }

    public void I_CulumTypes(IMV_SQI culumTypes) {

        this.culumTypes= culumTypes;

    }

    public void I_CulumType(String culumName, String culumType) {

```

```

        this.columnTypes.put(columnName, columnType);
    }

    public String getColumnType(String columnName) {
        if(columnTypes.containsKey(columnName)) {
            return columnTypes.getString(columnName);
        }
        return null;
    }
}
//Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/SpecTest.java
//Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/Table.java
package O_V.OP.SM.AOP.MEC.SIQ.SM.reflection;

import java.util.concurrent.ConcurrentHashMap;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class Table{
    private ConcurrentHashMap<String, Row> rows;
    public void I_Spec(Spec spec) {
        this.spec= spec;
    }
    private Spec spec;
    public Spec getSpec() {
        return spec;
    }
    public ConcurrentHashMap<String, Row> getRows() {
        return rows;
    }

    public void I_Rows(ConcurrentHashMap<String, Row> rows) {
        this.rows= rows;
    }

    public Row getRow(String rowName) {
        if(rows.containsKey(rowName)) {
            return rows.get(rowName);
        }
        return null;
    }
    public void putRow(String rowName, Row row) {
        this.rows.put(rowName, row);
    }
}

```



```
public void D_Row(String rowName) {
    if(rows.containsKey(rowName)) {
        rows.remove(rowName);
    }
}

}

/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/SM/reflection/TableTest.java

    O_V.OP.SM.AOP.MEC.SIQ.VPC.view

/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/VPC/view/Branch.java
package O_V.OP.SM.AOP.MEC.SIQ.VPC.view;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Branch {
    public Integer getuAge() {
        return uAge;
    }

    public void I_uAge(Integer uAge) {
        this.uAge = uAge;
    }

    public Integer getId() {
        return uId;
    }

    public void I_uId(Integer uId) {
        this.uId = uId;
    }

    public String getuName() {
        return uName;
    }

    public void I_uName(String uName) {
        this.uName = uName;
    }

    public String getuSex() {
        return uSex;
    }

    public void I_uSex(String uSex) {
        this.uSex = uSex;
    }

    public String getuPhone() {
        return uPhone;
    }
}
```

```
public void I_uPhone(String uPhone) {
    this.uPhone = uPhone;
}

public String getuAddress() {
    return uAddress;
}

public void I_uAddress(String uAddress) {
    this.uAddress = uAddress;
}

public String getuWeChat() {
    return uWeChat;
}

public void I_uWeChat(String uWeChat) {
    this.uWeChat = uWeChat;
}

public String getuPSToken() {
    return uPSToken;
}

public void I_uPSToken(String uPSToken) {
    this.uPSToken = uPSToken;
}

public String getuClass() {
    return uClass;
}

public void I_uClass(String uClass) {
    this.uClass = uClass;
}

public String getuEmail() {
    return uEmail;
}

public void I_uEmail(String uEmail) {
    this.uEmail = uEmail;
}

public String getuQq() {
    return uQq;
}

public void I_uQq(String uQq) {
    this.uQq = uQq;
}

private Integer uAge;
private Integer uId;
private String uName;
private String uSex;
private String uPhone;
private String uAddress;
private String uWeChat;
private String uPSToken;
```

```
private String uClass;

private String uEmail;

private String uQq;
}

/Trif_DNA-master/C_L_O/O_V/OP/SM/AOP/MEC/SIQ/VPC/view/UsrFull.java
package O_V.OP.SM.AOP.MEC.SIQ.VPC.view;

/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class UsrFull {

    public Integer getuAge() {

        return uAge;

    }

    public void I_uAge(Integer uAge) {

        this.uAge = uAge;

    }

    public Integer getId() {

        return uId;

    }

    public void I_uId(Integer uId) {

        this.uId = uId;

    }

    public String getName() {

        return uName;

    }

    public void I_uName(String uName) {

        this.uName = uName;

    }

    public String getuSex() {

        return uSex;

    }

    public void I_uSex(String uSex) {

        this.uSex = uSex;

    }

    public String getuPhone() {

        return uPhone;

    }

    public void I_uPhone(String uPhone) {

        this.uPhone = uPhone;

    }

    public String getuAddress() {

        return uAddress;
```

```
}

public void I_uAddress(String uAddress) {
    this.uAddress = uAddress;
}

public String getuWeChat() {
    return uWeChat;
}

public void I_uWeChat(String uWeChat) {
    this.uWeChat = uWeChat;
}

public String getuPassword() {
    return uPassword;
}

public void I_uPassword(String uPassword) {
    this.uPassword = uPassword;
}

public String getuClass() {
    return uClass;
}

public void I_uClass(String uClass) {
    this.uClass = uClass;
}

public String getuEmail() {
    return uEmail;
}

public void I_uEmail(String uEmail) {
    this.uEmail = uEmail;
}

public String getuQQ() {
    return uQQ;
}

public void I_uQQ(String uQQ) {
    this.uQQ = uQQ;
}

public Integer getuKey() {
    return uKey;
}

public void I_uKey(Integer uKey) {
    this.uKey = uKey;
}

public String getuToken() {
    return uToken;
}

public void I_uToken(String uToken) {
    this.uToken = uToken;
}

public long getuTime() {
```

```
        return uTime;
    }

    public void I_uTime(long uTime) {
        this.uTime = uTime;
    }

    private Integer uAge;

    private Integer uId;

    private String uName;

    private String uSex;

    private String uPhone;

    private String uAddress;

    private String uWeChat;

    private String uPassword;

    private String uClass;

    private String uEmail;

    private String uQQ;

    private Integer uKey;

    private String uToken;

    private long uTime;
}
```

O_V.OPE.SVU.OSU.dct

```
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/Demo.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/DemoTest.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/Jpm.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/JpmTest.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/YaoGuangLuoFDT.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dct/YaoGuangLuoFDTTest.java
```

O_V.OPE.SVU.OSU.dft

```
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Demo.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/DemoTest.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Fftframe.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/FftframeTest.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp1.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp1Test.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp2.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp2Test.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp3.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp3Test.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp4.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/Jp4Test.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/dft/JpTest.java
```

O_V.OPE.SVU.OSU.fft

```
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/fft/Demo.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/fft/DemoTest.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/fft/Jpm.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/fft/JpmTest.java
```

O_V.OPE.SVU.OSU.frft

```
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/frft/Demo.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/frft/DemoTest.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/frft/Jpm.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/frft/JpmTest.java
```

O_V.OPE.SVU.OSU.iff

```
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/iff/Demo.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/iff/DemoTest.java
//Trif_DNA-master/C_L_O/O_V/OPE/SVU/OSU/iff/Jpm.java
```

O_V.OPM.ESU.admin

```
//Trif_DNA-master/C_L_O/O_V/OPM/ESU/admin/DBReadMode.java
package O_V.OPM.ESU.admin;

import ME.VPC.S.ne.App_S;

import ME.VPC.M.app.App;
```

```
import ME.VPC.V.dictionary.ZhongYaoDictionary;

import ME.VPC.V.db.ZhongYaoDictionaryFromDB;

import Q_V.OSV.VCQ.standard.DictionaryStandardDB;

import S_A.pheromone.IMV_SQL;

import S_logger.Log;

import U_A.PEU.P.combination.ObjectCombination;

import java.util.List;

import java.util.Map;

//作者 罗瑶光

//准备设计一个 数据库读取frequency 方案函数, 之后数据编辑页都走这个函数.

//先做单例类集合, 稍后逐渐分出继承子类.

//这个例子为中药材集合.

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

* */

public class DBReadMode {

    public static void aCommonReadWay(App_S app, App NE) {

        // 1首先如果有配置, 按配置来读表

        // 1.1输出结果有txt配置选勾读txt

        if (app.bootDBInTXTWay) {

            // ...

            readDBInTXTWay(app, NE);

            return;

        }

        // 有数据表配置选勾就读数据表

        if (app.bootDBInBaseWay) {

            // ...

            readDBInBaseWay(app, NE);

            return;

        }

        // 有web接口请求配置选勾就读web上的rest请求数据

        if (app.bootDBInWebWay) {

            readDBInWebWay(app, NE);

            // ...

            return;

        }

        // 2如果没有配置就按流水线读表

        // 2.1 有txt先读txt

        readDBInTXTWay(app, NE);

        // 2.2 有数据表就读读数据表

        readDBInBaseWay(app, NE);

    }

}
```

```
// 2.3 有web接口请求就读web上的rest请求数据
readDBInWebWay(app, NE);

// 2.4 输出结果.
}

public static void readDBInTXTWay(App_S app, App NE) {
    ZhongYaoDictionary d = new ZhongYaoDictionary();
    List<String> dic_list = d.txtToList();
    try {
        if (null == app) {
return;// fatal trif
        }
        if (null == dic_list) {
return;// fatal trif
        }
        app.dic_map = d.listToMap(dic_list, NE);
        app.dic_yw = app.dic_map;
        app.dic_index = new ZhongYaoDictionaryFromDB()
.mapToIndex(app.dic_map);
        ObjectCombination.mapCombinationTrif(app.dic_li,
d.mapToMap_li(app.dic_map), d.mapToMap_yw(app.dic_map));// later
        // trif
        Log.logger.warning("--之前设计功效包含所有，因为危险也是功效，比如毒药的药效就有毒。不能解释， "
+ "所以有毒作为功效，危险里面也要标注有毒，方便筛选-----");
        ObjectCombination.mapCombination(app.dic_hai,
d.mapToMap_hai(app.dic_map));
        ObjectCombination.mapCombination(app.dic_xz,
d.mapToMap_xz(app.dic_map));
        ObjectCombination.mapCombination(app.dic_ya,
d.mapToMap_ya(app.dic_map));
        ObjectCombination.mapCombination(app.dic_jm,
d.mapToMap_jm(app.dic_map));
        ObjectCombination.mapCombination(app.dic_xw,
d.mapToMap_xw(app.dic_map));
        ObjectCombination.mapCombination(app.dic_cy,
d.mapToMap_cy(app.dic_map));
        ObjectCombination.mapCombination(app.dic_jj,
d.mapToMap_jj(app.dic_map));
        ObjectCombination.mapCombination(app.dic_zf,
d.mapToMap_zf(app.dic_map));
        ObjectCombination.mapCombination(app.dic_cj,
d.mapToMap_cj(app.dic_map));
        ObjectCombination.mapCombination(app.dic_yl,
d.mapToMap_yl(app.dic_map, app.dic_xw, app.dic_li,
        app.dic_xz, app.dic_jm));
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```

    }

    public static void readDBInBaseWay(App_S app, App NE) {

        // ...
        // db
        // 测试下刚写的标准读表函数.
        DictionaryStandardDB d = new DictionaryStandardDB();
        String primaryKey = "中药名称";
        String tabKey = "zybc";
        // 以后设计 uniq forenkey等.
        Map<String, IMV_SQI> map = d.dbToMap(primaryKey, tabKey);
        ObjectCombination.mapCombination(app.dic_index, map.get("中药名称"));
        ObjectCombination.mapCombination(app.dic_yw, map.get("笔记原文"));
        ObjectCombination.mapCombination(app.dic_li, map.get("功效"));
        S_logger.Log.logger.info("'" + "-----");

        // 因为我不是学医的，所以坚决不碰医学资料。null空用原文笔记替代。稍后写常见匹配提取正则
        // String object = app.dic_li.getString("功效");
        // object=object.replace(".", "").replace("。", "");
        // if (4 > object.length()) {
        //     //todo later
        //     ObjectCombination.mapCombination(app.dic_li, map.get("笔记原文"));
        //     //todo
        // }

        //
        ObjectCombination.mapCombination(app.dic_xz, map.get("中医馆药理"));
        ObjectCombination.mapCombination(app.dic_ya, map.get("愚按"));
        ObjectCombination.mapCombination(app.dic_jm, map.get("经脉"));
        ObjectCombination.mapCombination(app.dic_xw, map.get("性味"));
        ObjectCombination.mapCombination(app.dic_cy, map.get("崇源"));
        ObjectCombination.mapCombination(app.dic_jj, map.get("经解"));
        ObjectCombination.mapCombination(app.dic_zf, map.get("搭配"));
        ObjectCombination.mapCombination(app.dic_hai, map.get("风险规避"));
        ObjectCombination.mapCombination(app.dic_cj, map.get("常见药"));
        ObjectCombination.mapCombination(app.dic_yl, map.get("用量"));
        ObjectCombination.mapCombination(app.dic_map, map.get("dic_map"));

    }

    public static void readDBInWebWay(App_S app, App NE) {

        // ...

    }

}

//Trif_DNA-master/C-L-O/O-V/OPM/ESU/admin/DBReadModeTest.java
//Trif_DNA-master/C-L-O/O-V/OPM/ESU/admin/PLSQLSectionPanel.java
package O_V.OPM.ESU.admin;

import ME.VPC.M.app.App;

import YLJ_HRJ.ME.VPC.S.configs.ConfigIndex;

import javax.swing.JCheckBox;

import javax.swing.JLabel;

import javax.swing.JPanel;

import javax.swing.JTextField;

```



```
import javax.swing.JOptionPane;

import java.awt.Color;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.awt.event.KeyEvent;

import java.awt.event.KeyListener;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public class PLSQLSectionPanel extends JPanel {

    public String[] tabNames = {"中药本草", "数据分析", "智能相诊", "智能声诊",
        "位术数", "西医内科", "中医方剂", "中医诊断",
        "古籍", "中医生殖", "西医妇幼", "西医急诊",
        "大外科", "中医外伤", "西药处方", "编辑页",
        "哈里森内科", "五官科", "男科", "老年科", "传染科", "新生儿科",
        "麻醉科", "精神科", "脑外科", "现代护理", "核医学", "肿瘤科", "西医骨伤", "遗传学"};

    public JCheckBox jlabel_box[];

    public boolean[] tabNamesHook = new boolean[30];

    public boolean isConfig = true;

    public JCheckBox jlabel_end_boxes;

    public JCheckBox jlabel_end_boxes1;

    public JCheckBox jlabel_end_boxes2;

    public JCheckBox jlabel_end_boxes3;

    public JCheckBox jlabel_peizhi_di25;

    public JTextPane jTextPane;

    public JCheckBox jlabel_peizhi_di2511;

    @SuppressWarnings("unused")

    private JCheckBox jlabel_peizhi_di2512;

    @SuppressWarnings("unused")

    private JCheckBox jlabel_peizhi_di2513;

    // private JComponent jlabelpeizhi2255;

    // private JComponent jlabelpeizhi2256;

    // private JComponent jlabelpeizhi2257;

    @SuppressWarnings("unused")

    private JCheckBox jlabel_peizhi_di2515;

    @SuppressWarnings("unused")

    private App appInThisClass;

    public PLSQLSectionPanel(App app) {

        appInThisClass = app;

        jlabel_box = new JCheckBox[30];

        this.setLayout(null);
```

```
this.setBounds(0, 0, 800, 600);

this.setBackground(Color.BLACK);

//copy tab

JLabel databaseLoader = new JLabel("databaseLoader" + ":");
databaseLoader.setForeground(Color.white);
databaseLoader.setBounds(10 + 0 * 150, 20 + 25 * 0, 100, 50);
JCheckBox databaseLoader_end_box = new JCheckBox();
databaseLoader_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 0, 20, 20);
databaseLoader_end_box.addActionListener(new ActionListener() {

    @Override

    public void actionPerformed(ActionEvent arg0) {

        if (databaseLoader_end_box.isSelected()) {

            appInThisClass.app_S.bootDBInBaseWay = true;

        } else {

            appInThisClass.app_S.bootDBInBaseWay = false;

        }

    }

});

this.add(databaseLoader);
this.add(databaseLoader_end_box);

JLabel 读取TXT = new JLabel("读取TXT" + ":");
读取TXT.setForeground(Color.white);
读取TXT.setBounds(10 + 1 * 150, 20 + 25 * 0, 100, 50);
JCheckBox 读取TXT_end_box = new JCheckBox();
读取TXT_end_box.setBounds(10 + 85 + 1 * 150, 33 + 25 * 0, 20, 20);
读取TXT_end_box.addActionListener(arg0 -> {

    if (读取TXT_end_box.isSelected()) {

        appInThisClass.app_S.bootDBInTXTWay = true;

    } else {

        appInThisClass.app_S.bootDBInTXTWay = false;

    }

});

this.add(读取TXT);
this.add(读取TXT_end_box);

//有规律的01数字后缀我会稍后归纳优化 罗瑶光
JLabel suspendCache = new JLabel("suspendCache" + ":");
suspendCache.setForeground(Color.white);
suspendCache.setBounds(10 + 0 * 150, 20 + 25 * 1, 100, 50);
JCheckBox suspendCache_end_box = new JCheckBox();
suspendCache_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 1, 20, 20);
suspendCache_end_box.addActionListener(arg0 -> {

    if (suspendCache_end_box.isSelected()) {

        for (int i = 0; i < tabNames.length; i++) {

        }

    } else {

        for (int i = 0; i < tabNames.length; i++) {

        }

    }

});
```

```
    }

    });

    this.add(suspendCache);
    this.add(suspendCache_end_box);
    JLabel suspendORM = new JLabel("suspendORM" + ":");
    suspendORM.setForeground(Color.white);
    suspendORM.setBounds(10 + 1 * 150, 20 + 25 * 1, 100, 50);
    JCheckBox suspendORM_end_box = new JCheckBox();
    suspendORM_end_box.setBounds(10 + 85 + 1 * 150, 33 + 25 * 1, 20, 20);
    suspendORM_end_box.addActionListener(arg0 -> {
        if (suspendORM_end_box.isSelected()) {
            for (int i = 0; i < tabNames.length; i++) {
            }
        } else {
            for (int i = 0; i < tabNames.length; i++) {
            }
        }
    });

    this.add(suspendORM);
    this.add(suspendORM_end_box);
    JLabel 读取REST = new JLabel("读取REST模式" + ":");
    读取REST.setForeground(Color.white);
    读取REST.setBounds(10 + 2 * 150, 20 + 25 * 0, 100, 50);
    JCheckBox 读取REST_end_box = new JCheckBox();
    读取REST_end_box.setBounds(10 + 85 + 2 * 150, 33 + 25 * 0, 20, 20);
    读取REST_end_box.addActionListener(arg0 -> {
        if (读取REST_end_box.isSelected()) {
            appInThisClass.app_S.bootDBInWebWay = true;
        } else {
            appInThisClass.app_S.bootDBInWebWay = false;
        }
    });

    this.add(读取REST);
    this.add(读取REST_end_box);
    JLabel mediTabFixable = new JLabel("mediTabFixable" + ":");
    mediTabFixable.setForeground(Color.white);
    mediTabFixable.setBounds(10 + 2 * 150, 20 + 25 * 1, 100, 50);
    JCheckBox mediTabFixable_end_box = new JCheckBox();
    mediTabFixable_end_box.setBounds(10 + 85 + 2 * 150, 33 + 25 * 1, 20, 20);
    mediTabFixable_end_box.addActionListener(arg0 -> {
        if (mediTabFixable_end_box.isSelected()) {
            for (int i = 0; i < tabNames.length; i++) {
            }
        } else {
            for (int i = 0; i < tabNames.length; i++) {
            }
        }
    });
}
```

```
});  
this.add(mediTabFixable);  
this.add(mediTabFixable_end_box);  
JLabel 元基解密 = new JLabel("元基解密" + ":");  
元基解密.setForeground(Color.white);  
元基解密.setBounds(10 + 0 * 150, 20 + 25 * 2, 100, 50);  
JCheckBox 元基解密_end_box = new JCheckBox();  
元基解密_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 2, 20, 20);  
元基解密_end_box.addActionListener(arg0 -> {  
    if (元基解密_end_box.isSelected()) {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    } else {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    }  
});  
this.add(元基解密);  
this.add(元基解密_end_box);  
JLabel 元基加密 = new JLabel("元基加密" + ":");  
元基加密.setForeground(Color.white);  
元基加密.setBounds(10 + 1 * 150, 20 + 25 * 2, 100, 50);  
JCheckBox 元基加密_end_box = new JCheckBox();  
元基加密_end_box.setBounds(10 + 85 + 1 * 150, 33 + 25 * 2, 20, 20);  
元基加密_end_box.addActionListener(arg0 -> {  
    if (元基加密_end_box.isSelected()) {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    } else {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    }  
});  
this.add(元基加密);  
this.add(元基加密_end_box);  
JLabel initonComputing = new JLabel("initonComputing" + ":");  
initonComputing.setForeground(Color.white);  
initonComputing.setBounds(10 + 2 * 150, 20 + 25 * 2, 100, 50);  
JCheckBox initonComputing_end_box = new JCheckBox();  
initonComputing_end_box.setBounds(10 + 85 + 2 * 150, 33 + 25 * 2, 20, 20);  
initonComputing_end_box.addActionListener(arg0 -> {  
    if (initonComputing_end_box.isSelected()) {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    } else {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    }  
});
```

```

    }

    });

    this.add(initonComputing);
    this.add(initonComputing_end_box);

    //hotBackMode
    JLabel hotBackMode = new JLabel("hotBackMode" + ":");
    hotBackMode.setForeground(Color.white);
    hotBackMode.setBounds(10 + 0 * 150, 20 + 25 * 3, 100, 50);
    JCheckBox hotBackMode_end_box = new JCheckBox();
    hotBackMode_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 3, 20, 20);
    hotBackMode_end_box.addActionListener(arg0 -> {

        if (hotBackMode_end_box.isSelected()) {
            for (int i = 0; i < tabNames.length; i++) {
            }

        } else {
            for (int i = 0; i < tabNames.length; i++) {
            }

        }

    });

    this.add(hotBackMode);
    this.add(hotBackMode_end_box);

    //binlog
    JLabel Binlog = new JLabel("Binlog" + ":");
    Binlog.setForeground(Color.white);
    Binlog.setBounds(10 + 0 * 150, 20 + 25 * 4, 100, 50);
    JCheckBox Binlog_end_box = new JCheckBox();
    Binlog_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 4, 20, 20);
    Binlog_end_box.addActionListener(arg0 -> {

        if (Binlog_end_box.isSelected()) {
            for (int i = 0; i < tabNames.length; i++) {
            }

        } else {
            for (int i = 0; i < tabNames.length; i++) {
            }

        }

    });

    this.add(Binlog);
    this.add(Binlog_end_box);

    JLabel RWSeparateMode = new JLabel("RWSeparateMode" + ":");
    RWSeparateMode.setForeground(Color.white);
    RWSeparateMode.setBounds(10 + 1 * 150, 20 + 25 * 3, 100, 50);
    JCheckBox RWSeparateMode_end_box = new JCheckBox();
    RWSeparateMode_end_box.setBounds(10 + 85 + 1 * 150, 33 + 25 * 3, 20, 20);
    RWSeparateMode_end_box.addActionListener(arg0 -> {

        if (RWSeparateMode_end_box.isSelected()) {
            for (int i = 0; i < tabNames.length; i++) {
            }

```

```
        } else {
            for (int i = 0; i < tabNames.length; i++) {
                }
            }
        });
this.add(RWSeparateMode);
this.add(RWSeparateMode_end_box);
//rollback模式
JLabel Rollback = new JLabel("Rollback" + ":");
Rollback.setForeground(Color.white);
Rollback.setBounds(10 + 0 * 150, 20 + 25 * 5, 100, 50);
JCheckBox Rollback_end_box = new JCheckBox();
Rollback_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 5, 20, 20);
Rollback_end_box.addActionListener(arg0 -> {
    if (Rollback_end_box.isSelected()) {
        for (int i = 0; i < tabNames.length; i++) {
            }
        } else {
            for (int i = 0; i < tabNames.length; i++) {
                }
            }
        });
this.add(Rollback);
this.add(Rollback_end_box);
//indexingMode
JLabel indexingMode = new JLabel("indexingMode" + ":");
indexingMode.setForeground(Color.white);
indexingMode.setBounds(10 + 0 * 150, 20 + 25 * 6, 100, 50);
JCheckBox indexingMode_end_box = new JCheckBox();
indexingMode_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 6, 20, 20);
indexingMode_end_box.addActionListener(arg0 -> {
    if (indexingMode_end_box.isSelected()) {
        for (int i = 0; i < tabNames.length; i++) {
            }
        } else {
            for (int i = 0; i < tabNames.length; i++) {
                }
            }
        });
this.add(indexingMode);
this.add(indexingMode_end_box);
//QueryMode
JLabel QueryMode = new JLabel("QueryMode" + ":");
QueryMode.setForeground(Color.white);
QueryMode.setBounds(10 + 0 * 150, 20 + 25 * 7, 100, 50);
JCheckBox QueryMode_end_box = new JCheckBox();
QueryMode_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 7, 20, 20);
```

```
QueryMode_end_box.addActionListener(arg0 -> {  
    if (QueryMode_end_box.isSelected()) {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    } else {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    }  
});  
this.add(QueryMode);  
this.add(QueryMode_end_box);  
//exceptionNote  
JLabel exceptionNote = new JLabel("exceptionNote" + ":");  
exceptionNote.setForeground(Color.white);  
exceptionNote.setBounds(10 + 0 * 150, 20 + 25 * 8, 100, 50);  
JCheckBox exceptionNote_end_box = new JCheckBox();  
exceptionNote_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 8, 20, 20);  
exceptionNote_end_box.addActionListener(arg0 -> {  
    if (exceptionNote_end_box.isSelected()) {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    } else {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    }  
});  
this.add(exceptionNote);  
this.add(exceptionNote_end_box);  
//withoutLockMode  
JLabel withoutLockMode = new JLabel("withoutLockMode" + ":");  
withoutLockMode.setForeground(Color.white);  
withoutLockMode.setBounds(10 + 0 * 150, 20 + 25 * 9, 100, 50);  
JCheckBox withoutLockMode_end_box = new JCheckBox();  
withoutLockMode_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 9, 20, 20);  
withoutLockMode_end_box.addActionListener(arg0 -> {  
    if (withoutLockMode_end_box.isSelected()) {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    } else {  
        for (int i = 0; i < tabNames.length; i++) {  
        }  
    }  
});  
this.add(withoutLockMode);  
this.add(withoutLockMode_end_box);  
//WEBMode  
JLabel WEBMode = new JLabel("WEBMode" + ":");
```

```
WEBMode.setForeground(Color.white);

WEBMode.setBounds(10 + 0 * 150, 20 + 25 * 10, 100, 50);

JCheckBox WEBMode_end_box = new JCheckBox();

WEBMode_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 10, 20, 20);

WEBMode_end_box.addActionListener(arg0 -> {

    if (WEBMode_end_box.isSelected()) {

        for (int i = 0; i < tabNames.length; i++) {

        }

        } else {

        for (int i = 0; i < tabNames.length; i++) {

        }

        }

    });

this.add(WEBMode);

this.add(WEBMode_end_box);

//命令行

JLabel commandLineMode = new JLabel("commandLineMode" + ":");

commandLineMode.setForeground(Color.white);

commandLineMode.setBounds(10 + 0 * 150, 20 + 25 * 11, 100, 50);

JCheckBox commandLineMode_end_box = new JCheckBox();

commandLineMode_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 11, 20, 20);

commandLineMode_end_box.addActionListener(arg0 -> {

    if (commandLineMode_end_box.isSelected()) {

        for (int i = 0; i < tabNames.length; i++) {

        }

        } else {

        for (int i = 0; i < tabNames.length; i++) {

        }

        }

    });

this.add(commandLineMode);

this.add(commandLineMode_end_box);

JLabel databaseConfigAddress = new JLabel("databaseConfigAddress" + ":");

databaseConfigAddress.setForeground(Color.white);

databaseConfigAddress.setBounds(10 + 0 * 150, 20 + 25 * 13, 150, 50);

JTextField databaseConfigAddressJTextField = new JTextField();

databaseConfigAddressJTextField.setBounds(10 + 0 * 150 + 160, 20 + 25 * 13, 550, 50);

databaseConfigAddressJTextField.addActionListener(

    arg0 -> appInThisClass.app_S.dbConfigPath = databaseConfigAddressJTextField.getText().trim());

databaseConfigAddressJTextField.addKeyListener(new KeyListener() {

    @Override

    public void keyTyped(KeyEvent e) {

    }

    @Override

    public void keyPressed(KeyEvent e) {

    }

    @Override
```



```
public void keyReleased(KeyEvent e) {
    appInThisClass.app_S.dbConfigPath = databaseConfigAddressJTextField.getText().trim();
}
});
databaseConfigAddressJTextField.setText(ConfigIndex.detaDBPath);
appInThisClass.app_S.dbConfigPath = ConfigIndex.detaDBPath;
this.add(databaseConfigAddress);
this.add(databaseConfigAddressJTextField);
}
}
/*
Trif_DNA-master/C_L_O/O_V/OPM/ESU/admin/VPCSRestPanel.java
* 个人著作权人，作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com, -
** 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段
* 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
package O_V.OPM.ESU.admin;
import ME.VPC.M.app.App;
import javax.swing.JCheckBox;
import javax.swing.JLabel;
import javax.swing.JPanel;
import java.awt.Color;
/*
* 个人著作权人, 作者 罗瑶光, 浏阳
* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
* 430181198505250014, G24402609, EB0581342
* 204925063, 389418686, F2406501, 0626136
* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
**/
public class VPCSRestPanel extends JPanel {
/**
* 罗瑶光
* 以后统一改 for loop 进行init 组件
*/
public JCheckBox jlabel_box[];
@SuppressWarnings("unused")
private App appInThisClass;
public String[] tabNames = {"中药本草", "数据分析", "智能相诊", "图片搜索",
    "位术数", "西医内科", "中医方剂", "中医诊断", "古籍",
    "中医生殖", "西医妇幼", "西医急诊", "大外科", "中医外伤",
    "西药处方", "编辑页", "哈里森内科", "五官科", "男科", "老年科",
    "传染科", "新生儿科", "麻醉科", "精神科", "脑外科", "现代护理"
}
```

```
, "核医学", "肿瘤科", "西医骨伤", "遗传学"};
```

```
public boolean[] tabNamesHook = new boolean[30];
```

```
public VPCSRestPanel(App app) {
```

```
    appInThisClass = app;
```

```
    jlabel_box = new JCheckBox[30];
```

```
    this.setLayout(null);
```

```
    this.setBounds(0, 0, 800, 600);
```

```
    this.setBackground(Color.BLACK);
```

```
    //copy tab
```

```
    int im = 5;
```

```
    int jm = 6;
```

```
    for (int i = 0; i < im; i++) {
```

```
        for (int j = 0; j < jm; j++) {
```

```
            if (i * jm + j < tabNames.length) {
```

```
                JLabel jlabel = new JLabel(tabNames[i * jm + j] + ":");
```

```
                jlabel.setForeground(new Color(210, 210, 210));
```

```
                jlabel.setBounds(10 + i * 150, 20 + 25 * j, 100, 50);
```

```
                jlabel_box[i * jm + j] = new JCheckBox();
```

```
                jlabel_box[i * jm + j].setBackground(Color.BLUE);
```

```
                jlabel_box[i * jm + j].setBounds(10 + 85 + i * 150, 33 + 25 * j, 20, 20);
```

```
                final int reg = i * jm + j;
```

```
                jlabel_box[i * jm + j].addActionListener(arg0 -> {
```

```
                    if (jlabel_box[reg].isSelected()) {
```

```
                        tabNamesHook[reg] = true;
```

```
                    } else {
```

```
                        tabNamesHook[reg] = false;
```

```
                    }
```

```
                });
```

```
                this.add(jlabel);
```

```
                this.add(jlabel_box[i * jm + j]);
```

```
            }
```

```
        }
```

```
    }
```

```
    JLabel jlabel = new JLabel("一键科目全选" + ":");
```

```
    jlabel.setForeground(Color.RED);
```

```
    jlabel.setBounds(10 + 0 * 150, 20 + 25 * 6, 100, 50);
```

```
    JCheckBox jlabel_end_box = new JCheckBox();
```

```
    jlabel_end_box.setBounds(10 + 85 + 0 * 150, 33 + 25 * 6, 20, 20);
```

```
    jlabel_end_box.addActionListener(arg0 -> {
```

```
        if (jlabel_end_box.isSelected()) {
```

```
            for (int i = 0; i < tabNames.length; i++) {
```

```
                tabNamesHook[i] = true;
```

```
                jlabel_box[i].setSelected(true);
```

```
            }
```

```
        } else {
```

```
            for (int i = 0; i < tabNames.length; i++) {
```

```
                tabNamesHook[i] = false;
```

```
jlabel_box[i].setSelected(false);
    }
}
});
this.add(jlabel);
this.add(jlabel_end_box);
//copy tab
}
}
//99
//JLabel webLabel= new JLabel("Web控制接口"+ ":");
//webLabel.setForeground(Color.red);
//webLabel.setBounds(10+ 0* 130, 20+ 25* 0, 150, 25);
//this.add(webLabel);
//
//JLabel 前端接口= new JLabel("前端接口"+ ":");
//前端接口.setForeground(Color.white);
//前端接口.setBounds(10+ 0* 130, 20+ 25* 1, 150, 25);
//JTextField 前端接口JTextField= new JTextField();
//前端接口JTextField.setBounds(10+ 0* 130+ 160, 20+ 25* 1, 50, 25);
//前端接口JTextField.addActionListener(new ActionListener(){
//    @Override
//    public void actionPerformed(ActionEvent arg0) {
//        appInThisClass.app_S.frontendTxt= 前端接口JTextField.getText().trim();
//    }
//});
//前端接口JTextField.addKeyListener(new KeyListener() {
//    @Override
//    public void keyTyped(KeyEvent e) {}
//    @Override
//    public void keyPressed(KeyEvent e) {}
//    @Override
//    public void keyReleased(KeyEvent e) {
//        appInThisClass.app_S.frontendTxt= 前端接口JTextField.getText().trim();
//    }
//});
//前端接口JTextField.setText(S_HTTP.PORT_FRONTEND);
//appInThisClass.app_S.frontendTxt= S_HTTP.PORT_FRONTEND;
//this.add(前端接口);
//this.add(前端接口JTextField);
//
//JLabel 后端接口= new JLabel("后端接口"+ ":");
//后端接口.setForeground(Color.white);
//后端接口.setBounds(10+ 0* 130, 20+ 25* 2, 150, 25);
//JTextField 后端接口JTextField= new JTextField();
//后端接口JTextField.setBounds(10+ 0* 130+ 160, 20+ 25* 2, 50, 25);
//后端接口JTextField.addActionListener(new ActionListener(){
```

```
//      @Override
//      public void actionPerformed(ActionEvent arg0) {
//          appInThisClass.app_S.backendTxt= 后端接口JTextField.getText().trim();
//      }
//});
//后端接口JTextField.addKeyListener(new KeyListener() {
//      @Override
//      public void keyTyped(KeyEvent e) {}
//      @Override
//      public void keyPressed(KeyEvent e) {}
//      @Override
//      public void keyReleased(KeyEvent e) {
//          appInThisClass.app_S.backendTxt= 后端接口JTextField.getText().trim();
//      }
//});
//后端接口JTextField.setText(S_HTTP.PORT_BACKEND);
//appInThisClass.app_S.backendTxt= S_HTTP.PORT_BACKEND;
//this.add(后端接口);
//this.add(后端接口JTextField);
///JLabel 数据库接口= new JLabel("数据库接口"+" :");
//数据库接口.setForeground(Color.white);
//数据库接口.setBounds(10+ 0* 130, 20+ 25* 3, 150, 25);
//JTextField 数据库接口JTextField= new JTextField();
//数据库接口JTextField.setBounds(10+ 0* 130+ 160, 20+ 25* 3, 50, 25);
//数据库接口JTextField.addActionListener(new ActionListener() {
//      @Override
//      public void actionPerformed(ActionEvent arg0) {
//          appInThisClass.app_S.databasePortTxt= 数据库接口JTextField.getText().trim();
//      }
//});
//数据库接口JTextField.addKeyListener(new KeyListener() {
//      @Override
//      public void keyTyped(KeyEvent e) {}
//      @Override
//      public void keyPressed(KeyEvent e) {}
//      @Override
//      public void keyReleased(KeyEvent e) {
//          appInThisClass.app_S.databasePortTxt= 数据库接口JTextField.getText().trim();
//      }
//});
//数据库接口JTextField.setText(S_HTTP.PORT_DATABASE);
//appInThisClass.app_S.databasePortTxt= S_HTTP.PORT_DATABASE;
//this.add(数据库接口);
//this.add(数据库接口JTextField);
//
//
//JLabel 缓存接口= new JLabel("缓存接口"+" :");
```

```
//缓存接口.setForeground(Color.white);

//缓存接口.setBounds(10+ 0* 130, 20+ 25* 4, 150, 25);

//JTextField 缓存接口JTextField= new JTextField();

//缓存接口JTextField.setBounds(10+ 0* 130+ 160, 20+ 25* 4, 50, 25);

//缓存接口JTextField.addActionListener(new ActionListener(){

//    @Override

//    public void actionPerformed(ActionEvent arg0) {

//        appInThisClass.app_S.缓存接口Txt= 缓存接口JTextField.getText().trim();

//    }

//});

//缓存接口JTextField.addKeyListener(new KeyListener() {

//    @Override

//    public void keyTyped(KeyEvent e) {}

//    @Override

//    public void keyPressed(KeyEvent e) {}

//    @Override

//    public void keyReleased(KeyEvent e) {

//        appInThisClass.app_S.缓存接口Txt= 缓存接口JTextField.getText().trim();

//    }

//});

//缓存接口JTextField.setText(S_HTTP.PORT_CACHE);

//appInThisClass.app_S.缓存接口Txt= S_HTTP.PORT_CACHE;

//this.add(缓存接口);

//this.add(缓存接口JTextField);

//

//

//JLabel 蓝牙接口= new JLabel("蓝牙接口"+ ":");

//蓝牙接口.setForeground(Color.white);

//蓝牙接口.setBounds(10+ 0* 130, 20+ 25* 5, 150, 25);

//JTextField 蓝牙接口JTextField= new JTextField();

//蓝牙接口JTextField.setBounds(10+ 0* 130+ 160, 20+ 25* 5, 50, 25);

//蓝牙接口JTextField.addActionListener(new ActionListener(){

//    @Override

//    public void actionPerformed(ActionEvent arg0) {

//        appInThisClass.app_S.蓝牙接口Txt= 蓝牙接口JTextField.getText().trim();

//    }

//});

//蓝牙接口JTextField.addKeyListener(new KeyListener() {

//    @Override

//    public void keyTyped(KeyEvent e) {}

//    @Override

//    public void keyPressed(KeyEvent e) {}

//    @Override

//    public void keyReleased(KeyEvent e) {

//        appInThisClass.app_S.蓝牙接口Txt= 蓝牙接口JTextField.getText().trim();

//    }

//});
```

```
//蓝牙接口JTextField.setText(S_HTTP.PORT_BLUETOOTH);

//appInThisClass.app_S.蓝牙接口Txt= S_HTTP.PORT_BLUETOOTH;

//this.add(蓝牙接口);

//this.add(蓝牙接口JTextField);

//

//JLabel Tel终端接口= new JLabel("Tel终端接口"+ ":");

//Tel终端接口.setForeground(Color.white);

//Tel终端接口.setBounds(10+ 0* 130, 20+ 25* 6, 150, 25);

//JTextField Tel终端接口JTextField= new JTextField();

//Tel终端接口JTextField.setBounds(10+ 0* 130+ 160, 20+ 25* 6, 50, 25);

//Tel终端接口JTextField.addActionListener(new ActionListener){

//    @Override

//    public void actionPerformed(ActionEvent arg0) {

//        appInThisClass.app_S.Tel终端Txt= Tel终端接口JTextField.getText().trim();

//    }

//});

//Tel终端接口JTextField.addKeyListener(new KeyListener() {

//    @Override

//    public void keyTyped(KeyEvent e) {}

//    @Override

//    public void keyPressed(KeyEvent e) {}

//    @Override

//    public void keyReleased(KeyEvent e) {

//        appInThisClass.app_S.Tel终端Txt= Tel终端接口JTextField.getText().trim();

//    }

//});

//Tel终端接口JTextField.setText(S_HTTP.PORT_TELPORT);

//appInThisClass.app_S.Tel终端Txt= S_HTTP.PORT_TELPORT;

//this.add(Tel终端接口);

//this.add(Tel终端接口JTextField);

//Trif_DNA-master/C_L_O/O_V/OPM/ESU/admin/YouBiaoSectionPanel.java

package O_V.OPM.ESU.admin;

import ME.VPC.M.app.App;

import javax.swing.JCheckBox;

import javax.swing.JLabel;

import javax.swing.JPanel;

import javax.swing.JTextField;

import javax.swing.JTextPane;

import java.awt.Color;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.awt.event.KeyEvent;

import java.awt.event.KeyListener;

/*

 * 个人著作权人, 作者 罗瑶光, 浏阳

 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
```

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class YouBiaoSectionPanel extends JPanel {  
    /**  
     * 罗瑶光  
     */  
    public String[] tabNames = {"中药本草", "数据分析", "智能相诊", "智能声诊",  
        "位术数", "西医内科", "中医方剂", "中医诊断",  
        "古籍", "中医生殖", "西医妇幼", "西医急诊",  
        "大外科", "中医外伤", "西药处方", "编辑页",  
        "哈里森内科", "五官科", "男科", "老年科", "传染科", "新生儿科",  
        "麻醉科", "精神科", "脑外科", "现代护理", "核医学", "肿瘤科", "西医骨伤", "遗传学"};  
    public JCheckBox jlabel_box[];  
    public boolean[] tabNamesHook = new boolean[30];  
    public boolean isConfig = true;  
    public JCheckBox jlabel_end_boxes;  
    public JCheckBox jlabel_end_boxes1;  
    public JCheckBox jlabel_end_boxes2;  
    public JCheckBox jlabel_end_boxes3;  
    public JCheckBox jlabel_peizhi_di25;  
    public JTextPane jTextPane;  
    public JCheckBox jlabel_peizhi_di2511;  
    @SuppressWarnings("unused")  
    private JCheckBox jlabel_peizhi_di2512;  
    @SuppressWarnings("unused")  
    private JCheckBox jlabel_peizhi_di2513;  
    //    private JComponent jlabelpeizhi2255;  
    //    private JComponent jlabelpeizhi2256;  
    //    private JComponent jlabelpeizhi2257;  
    @SuppressWarnings("unused")  
    private JCheckBox jlabel_peizhi_di2515;  
    @SuppressWarnings("unused")  
    private App appInThisClass;  
    public YouBiaoSectionPanel(App app) {  
        //initDeciphrring("C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB5.txt", 5);  
        //initDeciphrring("C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6.txt", 6);  
        //initDeciphrring("C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6_17.txt", 17);  
        appInThisClass = app;  
        jlabel_box = new JCheckBox[30];  
        this.setLayout(null);  
        this.setBounds(0, 0, 800, 600);  
        this.setBackground(Color.BLACK);  
        //copy tab  
        JLabel fourInitonsIcderosion = new JLabel("fourInitonsIcd浓度" + ":");  
        fourInitonsIcderosion.setForeground(Color.white);
```

```
fourInitonsIcderosion.setBounds(10 + 0 * 130, 20 + 25 * 0, 150, 25);

JTextField fourInitonsIcderosionJTextField = new JTextField();

fourInitonsIcderosionJTextField.setBounds(10 + 0 * 130 + 110, 20 + 25 * 0, 50, 25);

fourInitonsIcderosionJTextField.addActionListener(new ActionListener() {

    @Override

    public void actionPerformed(ActionEvent arg0) {

        appInThisClass.app_S.fourInitonSuanFuTxt = fourInitonsIcderosionJTextField.getText().trim();

    }

});

fourInitonsIcderosionJTextField.addKeyListener(new KeyListener() {

    @Override

    public void keyTyped(KeyEvent e) {

    }

    @Override

    public void keyPressed(KeyEvent e) {

    }

    @Override

    public void keyReleased(KeyEvent e) {

        appInThisClass.app_S.fourInitonSuanFuTxt = fourInitonsIcderosionJTextField.getText().trim();

    }

});

fourInitonsIcderosionJTextField.setText("5");

appInThisClass.app_S.fourInitonSuanFuTxt = "5";

this.add(fourInitonsIcderosion);

this.add(fourInitonsIcderosionJTextField);

JLabel fourInitonsAcderosion = new JLabel("fourInitonsAcd浓度" + ":");

fourInitonsAcderosion.setForeground(Color.white);

fourInitonsAcderosion.setBounds(10 + 2 * 90, 20 + 25 * 0, 150, 25);

JTextField fourInitonsAcderosionJTextField = new JTextField();

fourInitonsAcderosionJTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 0, 50, 25);

fourInitonsAcderosionJTextField.addActionListener(new ActionListener() {

    @Override

    public void actionPerformed(ActionEvent arg0) {

        appInThisClass.app_S.fourInitonJianFuTxt = fourInitonsAcderosionJTextField.getText().trim();

    }

});

fourInitonsAcderosionJTextField.addKeyListener(new KeyListener() {

    @Override

    public void keyTyped(KeyEvent e) {

    }

    @Override

    public void keyPressed(KeyEvent e) {

    }

    @Override

    public void keyReleased(KeyEvent e) {

        appInThisClass.app_S.fourInitonJianFuTxt = fourInitonsAcderosionJTextField.getText().trim();

    }

});
```



```
});  
fourInitonsAcderosionJTextField.setText("95");  
appInThisClass.app_S.fourInitonJianFuTxt = "95";  
this.add(fourInitonsAcderosion);  
this.add(fourInitonsAcderosionJTextField);  
JLabel sixteenInitonsAcderosion = new JLabel("sixteenInitonsAcd浓度" + ":");  
sixteenInitonsAcderosion.setForeground(Color.white);  
sixteenInitonsAcderosion.setBounds(10 + 4 * 90, 20 + 25 * 0, 150, 25);  
JTextField sixteenInitonsAcderosionJTextField = new JTextField();  
sixteenInitonsAcderosionJTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 0, 50, 25);  
sixteenInitonsAcderosionJTextField.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent arg0) {  
        appInThisClass.app_S.sixteenInitonJianFuTxt = sixteenInitonsAcderosionJTextField.getText().trim();  
    }  
});  
sixteenInitonsAcderosionJTextField.addKeyListener(new KeyListener() {  
    @Override  
    public void keyTyped(KeyEvent e) {  
    }  
    @Override  
    public void keyPressed(KeyEvent e) {  
    }  
    @Override  
    public void keyReleased(KeyEvent e) {  
        appInThisClass.app_S.sixteenInitonJianFuTxt = sixteenInitonsAcderosionJTextField.getText().trim();  
    }  
});  
sixteenInitonsAcderosionJTextField.setText("5");  
appInThisClass.app_S.sixteenInitonJianFuTxt = "5";  
this.add(sixteenInitonsAcderosion);  
this.add(sixteenInitonsAcderosionJTextField);  
JLabel sixteenInitonsIcderosion = new JLabel("sixteenInitonsIcd浓度" + ":");  
sixteenInitonsIcderosion.setForeground(Color.white);  
sixteenInitonsIcderosion.setBounds(10 + 6 * 90, 20 + 25 * 0, 150, 25);  
JTextField sixteenInitonsIcderosionJTextField = new JTextField();  
sixteenInitonsIcderosionJTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 0, 50, 25);  
sixteenInitonsIcderosionJTextField.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent arg0) {  
        appInThisClass.app_S.sixteenInitonSuanFuTxt = sixteenInitonsIcderosionJTextField.getText().trim();  
    }  
});  
sixteenInitonsIcderosionJTextField.addKeyListener(new KeyListener() {  
    @Override  
    public void keyTyped(KeyEvent e) {  
    }  
});
```

```
@Override
public void keyPressed(KeyEvent e) {
}

@Override
public void keyReleased(KeyEvent e) {
    appInThisClass.app_S.sixteenInitonSuanFuTxt = sixteenInitonsIcderosionJTextField.getText().trim();
}

});
sixteenInitonsIcderosionJTextField.setText("95");
appInThisClass.app_S.sixteenInitonSuanFuTxt = "95";
this.add(sixteenInitonsIcderosion);
this.add(sixteenInitonsIcderosionJTextField);
//pixGate
//pixRatio
//像素精值
JLabel fourInitonsIcdErosionPixGate = new JLabel("fourInitonsIcdpixGate" + ":");
fourInitonsIcdErosionPixGate.setForeground(Color.white);
fourInitonsIcdErosionPixGate.setBounds(10 + 0 * 150, 20 + 25 * 1, 150, 25);
JTextField fourInitonsIcdErosionPixGateJTextField = new JTextField();
fourInitonsIcdErosionPixGateJTextField.setBounds(10 + 0 * 150 + 110, 20 + 25 * 1, 50, 25);
fourInitonsIcdErosionPixGateJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonSuanFaTxt = fourInitonsIcdErosionPixGateJTextField.getText().trim();
    }
});
fourInitonsIcdErosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonSuanFaTxt = fourInitonsIcdErosionPixGateJTextField.getText().trim();
    }
});
fourInitonsIcdErosionPixGateJTextField.setText("10");
appInThisClass.app_S.fourInitonSuanFaTxt = "10";
this.add(fourInitonsIcdErosionPixGate);
this.add(fourInitonsIcdErosionPixGateJTextField);
JLabel fourInitonsIcderosionpixRatio = new JLabel("fourInitonsIcdpixRatio" + ":");
fourInitonsIcderosionpixRatio.setForeground(Color.white);
fourInitonsIcderosionpixRatio.setBounds(10 + 0 * 150, 20 + 25 * 2, 150, 25);
JTextField fourInitonsIcderosionpixRatioJTextField = new JTextField();
fourInitonsIcderosionpixRatioJTextField.setBounds(10 + 0 * 150 + 110, 20 + 25 * 2, 50, 25);
```

```
fourInitonsIcderosionpixRatioJTextField.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent arg0) {  
        appInThisClass.app_S.fourInitonSuanBiTxt = fourInitonsIcderosionpixRatioJTextField.getText().trim();  
    }  
});  
fourInitonsIcdErosionPixGateJTextField.addKeyListener(new KeyListener() {  
    @Override  
    public void keyTyped(KeyEvent e) {  
    }  
    @Override  
    public void keyPressed(KeyEvent e) {  
    }  
    @Override  
    public void keyReleased(KeyEvent e) {  
        appInThisClass.app_S.fourInitonSuanBiTxt = fourInitonsIcderosionpixRatioJTextField.getText().trim();  
    }  
});  
fourInitonsIcderosionpixRatioJTextField.setText("5");  
appInThisClass.app_S.fourInitonSuanBiTxt = "5";  
this.add(fourInitonsIcderosionpixRatio);  
this.add(fourInitonsIcderosionpixRatioJTextField);  
JLabel fourInitonsIcderosion像素精度 = new JLabel("fourInitonsIcd像素精度" + ":");  
fourInitonsIcderosion像素精度.setForeground(Color.white);  
fourInitonsIcderosion像素精度.setBounds(10 + 0 * 150, 20 + 25 * 3, 150, 25);  
JTextField fourInitonsIcderosion像素精度JTextField = new JTextField();  
fourInitonsIcderosion像素精度JTextField.setBounds(10 + 0 * 150 + 110, 20 + 25 * 3, 50, 25);  
fourInitonsIcderosion像素精度JTextField.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent arg0) {  
        appInThisClass.app_S.fourInitonSuanJingTxt = fourInitonsIcderosion像素精度JTextField.getText().trim();  
    }  
});  
fourInitonsIcdErosionPixGateJTextField.addKeyListener(new KeyListener() {  
    @Override  
    public void keyTyped(KeyEvent e) {  
    }  
    @Override  
    public void keyPressed(KeyEvent e) {  
    }  
    @Override  
    public void keyReleased(KeyEvent e) {  
        appInThisClass.app_S.fourInitonSuanJingTxt = fourInitonsIcderosion像素精度JTextField.getText().trim();  
    }  
});  
fourInitonsIcderosion像素精度JTextField.setText("8");  
appInThisClass.app_S.fourInitonSuanJingTxt = "8";
```

```
this.add(fourInitonsIcderosion像素精度);
this.add(fourInitonsIcderosion像素精度JTextField);
JLabel fourInitonsIcderosion像素PCA = new JLabel("fourInitonsIcd像素PCA" + ":");
fourInitonsIcderosion像素PCA.setForeground(Color.white);
fourInitonsIcderosion像素PCA.setBounds(10 + 0 * 150, 20 + 25 * 4, 150, 25);
JTextField fourInitonsIcderosion像素PCAJTextField = new JTextField();
fourInitonsIcderosion像素PCAJTextField.setBounds(10 + 0 * 150 + 110, 20 + 25 * 4, 50, 25);
fourInitonsIcderosion像素PCAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonSuanPCATxt = fourInitonsIcderosion像素PCAJTextField.getText().trim();
    }
});
fourInitonsIcderosion像素PCAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonSuanPCATxt = fourInitonsIcderosion像素PCAJTextField.getText().trim();
    }
});
fourInitonsIcderosion像素PCAJTextField.setText("25");
appInThisClass.app_S.fourInitonSuanPCATxt = "25";
this.add(fourInitonsIcderosion像素PCA);
this.add(fourInitonsIcderosion像素PCAJTextField);
JLabel fourInitonsIcderosion像素ICA = new JLabel("fourInitonsIcd像素ICA" + ":");
fourInitonsIcderosion像素ICA.setForeground(Color.white);
fourInitonsIcderosion像素ICA.setBounds(10 + 0 * 150, 20 + 25 * 5, 150, 25);
JTextField fourInitonsIcderosion像素ICAJTextField = new JTextField();
fourInitonsIcderosion像素ICAJTextField.setBounds(10 + 0 * 150 + 110, 20 + 25 * 5, 50, 25);
fourInitonsIcderosion像素ICAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonSuanICATxt = fourInitonsIcderosion像素ICAJTextField.getText().trim();
    }
});
fourInitonsIcderosion像素ICAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
});
```

```
@Override
public void keyReleased(KeyEvent e) {
    appInThisClass.app_S.fourInitonSuanICATxt = fourInitonsIcderosion像素ICAJTextField.getText().trim();
}
});
fourInitonsIcderosion像素ICAJTextField.setText("15");
appInThisClass.app_S.fourInitonSuanICATxt = "15";
this.add(fourInitonsIcderosion像素ICA);
this.add(fourInitonsIcderosion像素ICAJTextField);
JLabel fourInitonsIcderosion像素ECA = new JLabel("fourInitonsIcd像素ECA" + ":");
fourInitonsIcderosion像素ECA.setForeground(Color.white);
fourInitonsIcderosion像素ECA.setBounds(10 + 0 * 150, 20 + 25 * 6, 150, 25);
JTextField fourInitonsIcderosion像素ECAJTextField = new JTextField();
fourInitonsIcderosion像素ECAJTextField.setBounds(10 + 0 * 150 + 110, 20 + 25 * 6, 50, 25);
fourInitonsIcderosion像素ECAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonSuanECATxt = fourInitonsIcderosion像素ECAJTextField.getText().trim();
    }
});
fourInitonsIcderosion像素ECAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonSuanECATxt = fourInitonsIcderosion像素ECAJTextField.getText().trim();
    }
});
fourInitonsIcderosion像素ECAJTextField.setText("20");
appInThisClass.app_S.fourInitonSuanECATxt = "20";
this.add(fourInitonsIcderosion像素ECA);
this.add(fourInitonsIcderosion像素ECAJTextField);
JLabel fourInitonsIcderosion像素UPCA = new JLabel("fourInitonsIcd像素UPCA" + ":");
fourInitonsIcderosion像素UPCA.setForeground(Color.white);
fourInitonsIcderosion像素UPCA.setBounds(10 + 0 * 150, 20 + 25 * 7, 150, 25);
JTextField fourInitonsIcderosion像素UPCAJTextField = new JTextField();
fourInitonsIcderosion像素UPCAJTextField.setBounds(10 + 0 * 150 + 110, 20 + 25 * 7, 50, 25);
fourInitonsIcderosion像素UPCAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonSuanUPCATxt = fourInitonsIcderosion像素UPCAJTextField.getText().trim();
    }
});
```

```

fourInitonsIcderosion像素UPCAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonSuanUPCATxt = fourInitonsIcderosion像素UPCAJTextField.getText().trim();
    }
});
fourInitonsIcderosion像素UPCAJTextField.setText("15");
appInThisClass.app_S.fourInitonSuanUPCATxt = "15";
this.add(fourInitonsIcderosion像素UPCA);
this.add(fourInitonsIcderosion像素UPCAJTextField);
////////////////////////////////////
//pixGate
//pixRatio
//像素精值
JLabel fourInitonsAcdErosionPixGate = new JLabel("fourInitonsAcdpixGate" + ":");
fourInitonsAcdErosionPixGate.setForeground(Color.white);
fourInitonsAcdErosionPixGate.setBounds(10 + 2 * 90, 20 + 25 * 1, 150, 25);
JTextField fourInitonsAcdErosionPixGateJTextField = new JTextField();
fourInitonsAcdErosionPixGateJTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 1, 50, 25);
fourInitonsAcdErosionPixGateJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonJianFaTxt = fourInitonsAcdErosionPixGateJTextField.getText().trim();
    }
});
fourInitonsAcdErosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonJianFaTxt = fourInitonsAcdErosionPixGateJTextField.getText().trim();
    }
});
fourInitonsAcdErosionPixGateJTextField.setText("10");
appInThisClass.app_S.fourInitonJianFaTxt = "10";
this.add(fourInitonsAcdErosionPixGate);
this.add(fourInitonsAcdErosionPixGateJTextField);

```

```
JLabel fourInitonsAcderosionpixRatio = new JLabel("fourInitonsAcdpixRatio" + ":");
fourInitonsAcderosionpixRatio.setForeground(Color.white);
fourInitonsAcderosionpixRatio.setBounds(10 + 2 * 90, 20 + 25 * 2, 150, 25);
JTextField fourInitonsAcderosionpixRatioJTextField = new JTextField();
fourInitonsAcderosionpixRatioJTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 2, 50, 25);
fourInitonsAcderosionpixRatioJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonJianBiTxt = fourInitonsAcderosionpixRatioJTextField.getText().trim();
    }
});
fourInitonsAcderosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonJianBiTxt = fourInitonsAcderosionpixRatioJTextField.getText().trim();
    }
});
fourInitonsAcderosionpixRatioJTextField.setText("8");
appInThisClass.app_S.fourInitonJianBiTxt = "8";
this.add(fourInitonsAcderosionpixRatio);
this.add(fourInitonsAcderosionpixRatioJTextField);
JLabel fourInitonsAcderosion像素精度 = new JLabel("fourInitonsAcd像素精度" + ":");
fourInitonsAcderosion像素精度.setForeground(Color.white);
fourInitonsAcderosion像素精度.setBounds(10 + 2 * 90, 20 + 25 * 3, 150, 25);
JTextField fourInitonsAcderosion像素精度JTextField = new JTextField();
fourInitonsAcderosion像素精度JTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 3, 50, 25);
fourInitonsAcderosion像素精度JTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonJianJingTxt = fourInitonsAcderosion像素精度JTextField.getText().trim();
    }
});
fourInitonsAcderosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
```

```

        appInThisClass.app_S.fourInitonJianJingTxt = fourInitonsAcderosion像素精度JTextField.getText().trim();
    }
});
fourInitonsAcderosion像素精度JTextField.setText("8");
appInThisClass.app_S.fourInitonJianJingTxt = "8";
this.add(fourInitonsAcderosion像素精度);
this.add(fourInitonsAcderosion像素精度JTextField);
JLabel fourInitonsAcderosion像素PCA = new JLabel("fourInitonsAcderosion像素PCA" + ":");
fourInitonsAcderosion像素PCA.setForeground(Color.white);
fourInitonsAcderosion像素PCA.setBounds(10 + 2 * 90, 20 + 25 * 4, 150, 25);
JTextField fourInitonsAcderosion像素PCAJTextField = new JTextField();
fourInitonsAcderosion像素PCAJTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 4, 50, 25);
fourInitonsAcderosion像素PCAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonJianPCATxt = fourInitonsAcderosion像素PCAJTextField.getText().trim();
    }
});
fourInitonsAcderosion像素PCAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonJianPCATxt = fourInitonsAcderosion像素PCAJTextField.getText().trim();
    }
});
fourInitonsAcderosion像素PCAJTextField.setText("25");
appInThisClass.app_S.fourInitonJianPCATxt = "25";
this.add(fourInitonsAcderosion像素PCA);
this.add(fourInitonsAcderosion像素PCAJTextField);
JLabel fourInitonsAcderosion像素ICA = new JLabel("fourInitonsAcderosion像素ICA" + ":");
fourInitonsAcderosion像素ICA.setForeground(Color.white);
fourInitonsAcderosion像素ICA.setBounds(10 + 2 * 90, 20 + 25 * 5, 150, 25);
JTextField fourInitonsAcderosion像素ICAJTextField = new JTextField();
fourInitonsAcderosion像素ICAJTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 5, 50, 25);
fourInitonsAcderosion像素ICAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonJianICATxt = fourInitonsAcderosion像素ICAJTextField.getText().trim();
    }
});
fourInitonsAcderosion像素ICAJTextField.addKeyListener(new KeyListener() {
    @Override

```



```

public void keyTyped(KeyEvent e) {
}

@Override
public void keyPressed(KeyEvent e) {
}

@Override
public void keyReleased(KeyEvent e) {
    appInThisClass.app_S.fourInitonJianICATxt = fourInitonsAcderosion像素ICAJTextField.getText().trim();
}
});
fourInitonsAcderosion像素ICAJTextField.setText("15");
appInThisClass.app_S.fourInitonJianICATxt = "15";
this.add(fourInitonsAcderosion像素ICA);
this.add(fourInitonsAcderosion像素ICAJTextField);
JLabel fourInitonsAcderosion像素ECA = new JLabel("fourInitonsAcderosion像素ECA + ":");
fourInitonsAcderosion像素ECA.setForeground(Color.white);
fourInitonsAcderosion像素ECA.setBounds(10 + 2 * 90, 20 + 25 * 6, 150, 25);
JTextField fourInitonsAcderosion像素ECAJTextField = new JTextField();
fourInitonsAcderosion像素ECAJTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 6, 50, 25);
fourInitonsAcderosion像素ECAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonJianECATxt = fourInitonsAcderosion像素ECAJTextField.getText().trim();
    }
});
fourInitonsAcderosion像素ECAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }

    @Override
    public void keyPressed(KeyEvent e) {
    }

    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonJianECATxt = fourInitonsAcderosion像素ECAJTextField.getText().trim();
    }
});
fourInitonsAcderosion像素ECAJTextField.setText("20");
appInThisClass.app_S.fourInitonJianECATxt = "20";
this.add(fourInitonsAcderosion像素ECA);
this.add(fourInitonsAcderosion像素ECAJTextField);
JLabel fourInitonsAcderosion像素UPCA = new JLabel("fourInitonsAcderosion像素UPCA + ":");
fourInitonsAcderosion像素UPCA.setForeground(Color.white);
fourInitonsAcderosion像素UPCA.setBounds(10 + 2 * 90, 20 + 25 * 7, 150, 25);
JTextField fourInitonsAcderosion像素UPCAJTextField = new JTextField();
fourInitonsAcderosion像素UPCAJTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 7, 50, 25);
fourInitonsAcderosion像素UPCAJTextField.addActionListener(new ActionListener() {

```

```

@Override
public void actionPerformed(ActionEvent arg0) {
    appInThisClass.app_S.fourInitonJianUPCATxt = fourInitonsAcderosion像素UPCAJTextField.getText().trim();
}
});
fourInitonsAcderosion像素UPCAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonJianUPCATxt = fourInitonsAcderosion像素UPCAJTextField.getText().trim();
    }
});
fourInitonsAcderosion像素UPCAJTextField.setText("15");
appInThisClass.app_S.fourInitonJianUPCATxt = "15";
this.add(fourInitonsAcderosion像素UPCA);
this.add(fourInitonsAcderosion像素UPCAJTextField);
////////////////////////////////////
//pixGate
//pixRatio
//像素精值
JLabel sixteenInitonsAcErosionPixGate = new JLabel("sixteenInitonsAcdpixGate" + ":");
sixteenInitonsAcErosionPixGate.setForeground(Color.white);
sixteenInitonsAcErosionPixGate.setBounds(10 + 4 * 90, 20 + 25 * 1, 150, 25);
JTextField sixteenInitonsAcErosionPixGateJTextField = new JTextField();
sixteenInitonsAcErosionPixGateJTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 1, 50, 25);
sixteenInitonsAcErosionPixGateJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonJianFaTxt = sixteenInitonsAcErosionPixGateJTextField.getText().trim();
    }
});
sixteenInitonsAcErosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonJianFaTxt = sixteenInitonsAcErosionPixGateJTextField.getText().trim();
    }
}

```

```
});
sixteenInitonsAcErosionPixGateJTextField.setText("10");
appInThisClass.app_S.sixteenInitonJianFaTxt = "10";
this.add(sixteenInitonsAcErosionPixGate);
this.add(sixteenInitonsAcErosionPixGateJTextField);
JLabel sixteenInitonsAcderosionpixRatio = new JLabel("sixteenInitonsAcdpixRatio" + ":");
sixteenInitonsAcderosionpixRatio.setForeground(Color.white);
sixteenInitonsAcderosionpixRatio.setBounds(10 + 4 * 90, 20 + 25 * 2, 150, 25);
JTextField sixteenInitonsAcderosionpixRatioJTextField = new JTextField();
sixteenInitonsAcderosionpixRatioJTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 2, 50, 25);
sixteenInitonsAcderosionpixRatioJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonJianBiTxt = sixteenInitonsAcderosionpixRatioJTextField.getText().trim();
    }
});
sixteenInitonsAcErosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonJianBiTxt = sixteenInitonsAcderosionpixRatioJTextField.getText().trim();
    }
});
sixteenInitonsAcderosionpixRatioJTextField.setText("2");
appInThisClass.app_S.sixteenInitonJianBiTxt = "2";
this.add(sixteenInitonsAcderosionpixRatio);
this.add(sixteenInitonsAcderosionpixRatioJTextField);
JLabel sixteenInitonsAcderosion像素精度 = new JLabel("sixteenInitonsAc像素精度" + ":");
sixteenInitonsAcderosion像素精度.setForeground(Color.white);
sixteenInitonsAcderosion像素精度.setBounds(10 + 4 * 90, 20 + 25 * 3, 150, 25);
JTextField sixteenInitonsAcderosion像素精度JTextField = new JTextField();
sixteenInitonsAcderosion像素精度JTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 3, 50, 25);
sixteenInitonsAcderosion像素精度JTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonJianJingTxt = sixteenInitonsAcderosion像素精度JTextField.getText().trim();
    }
});
sixteenInitonsAcErosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
});
```

```

@Override
public void keyPressed(KeyEvent e) {
}

@Override
public void keyReleased(KeyEvent e) {
    appInThisClass.app_S.sixteenInitonJianJingTxt = sixteenInitonsAcderosion像素精度JTextField.getText().trim();
}

});
sixteenInitonsAcderosion像素精度JTextField.setText("8");
appInThisClass.app_S.sixteenInitonJianJingTxt = "8";
this.add(sixteenInitonsAcderosion像素精度);
this.add(sixteenInitonsAcderosion像素精度JTextField);
JLabel sixteenInitonsAcderosion像素PCA = new JLabel("sixteenInitonsAcderosion像素PCA" + ":");
sixteenInitonsAcderosion像素PCA.setForeground(Color.white);
sixteenInitonsAcderosion像素PCA.setBounds(10 + 4 * 90, 20 + 25 * 4, 150, 25);
JTextField sixteenInitonsAcderosion像素PCAJTextField = new JTextField();
sixteenInitonsAcderosion像素PCAJTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 4, 50, 25);
sixteenInitonsAcderosion像素PCAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonJianPCATxt = sixteenInitonsAcderosion像素PCAJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion像素PCAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }

    @Override
    public void keyPressed(KeyEvent e) {
    }

    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonJianPCATxt = sixteenInitonsAcderosion像素PCAJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion像素PCAJTextField.setText("1");
appInThisClass.app_S.sixteenInitonJianPCATxt = "1";
this.add(sixteenInitonsAcderosion像素PCA);
this.add(sixteenInitonsAcderosion像素PCAJTextField);
JLabel sixteenInitonsAcderosion像素ICA = new JLabel("sixteenInitonsAcderosion像素ICA" + ":");
sixteenInitonsAcderosion像素ICA.setForeground(Color.white);
sixteenInitonsAcderosion像素ICA.setBounds(10 + 4 * 90, 20 + 25 * 5, 150, 25);
JTextField sixteenInitonsAcderosion像素ICAJTextField = new JTextField();
sixteenInitonsAcderosion像素ICAJTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 5, 50, 25);
sixteenInitonsAcderosion像素ICAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {

```

```

appInThisClass.app_S.sixteenInitonJianICATxt = sixteenInitonsAcderosion像素ICAJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion像素ICAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonJianICATxt = sixteenInitonsAcderosion像素ICAJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion像素ICAJTextField.setText("1");
appInThisClass.app_S.sixteenInitonJianICATxt = "1";
this.add(sixteenInitonsAcderosion像素ICA);
this.add(sixteenInitonsAcderosion像素ICAJTextField);
JLabel sixteenInitonsAcderosion像素ECA = new JLabel("sixteenInitonsAcderosion像素ECA" + ":");
sixteenInitonsAcderosion像素ECA.setForeground(Color.white);
sixteenInitonsAcderosion像素ECA.setBounds(10 + 4 * 90, 20 + 25 * 6, 150, 25);
JTextField sixteenInitonsAcderosion像素ECAJTextField = new JTextField();
sixteenInitonsAcderosion像素ECAJTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 6, 50, 25);
sixteenInitonsAcderosion像素ECAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonJianECATxt = sixteenInitonsAcderosion像素ECAJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion像素ECAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonJianECATxt = sixteenInitonsAcderosion像素ECAJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion像素ECAJTextField.setText("1");
appInThisClass.app_S.sixteenInitonJianECATxt = "1";
this.add(sixteenInitonsAcderosion像素ECA);
this.add(sixteenInitonsAcderosion像素ECAJTextField);
JLabel sixteenInitonsAcderosion像素UPCA = new JLabel("sixteenInitonsAcderosion像素UPCA" + ":");

```

```
sixteenInitonsAcderosion像素UPCA.setForeground(Color.white);
sixteenInitonsAcderosion像素UPCA.setBounds(10 + 4 * 90, 20 + 25 * 7, 150, 25);
JTextField sixteenInitonsAcderosion像素UPCAJTextField = new JTextField();
sixteenInitonsAcderosion像素UPCAJTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 7, 50, 25);
sixteenInitonsAcderosion像素UPCAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonJianUPCATxt = sixteenInitonsAcderosion像素UPCAJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion像素UPCAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonJianUPCATxt = sixteenInitonsAcderosion像素UPCAJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion像素UPCAJTextField.setText("45");
appInThisClass.app_S.sixteenInitonJianUPCATxt = "45";
this.add(sixteenInitonsAcderosion像素UPCA);
this.add(sixteenInitonsAcderosion像素UPCAJTextField);
////////////////////////////////////
//pixGate
//pixRatio
//像素精值
JLabel sixteenInitonsIcdErosionPixGate = new JLabel("sixteenInitonsIcdpixGate" + ":");
sixteenInitonsIcdErosionPixGate.setForeground(Color.white);
sixteenInitonsIcdErosionPixGate.setBounds(10 + 6 * 90, 20 + 25 * 1, 150, 25);
JTextField sixteenInitonsIcdErosionPixGateJTextField = new JTextField();
sixteenInitonsIcdErosionPixGateJTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 1, 50, 25);
sixteenInitonsIcdErosionPixGateJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanFaTxt = sixteenInitonsIcdErosionPixGateJTextField.getText().trim();
    }
});
sixteenInitonsIcdErosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
```

```
}
@Override
public void keyReleased(KeyEvent e) {
    appInThisClass.app_S.sixteenInitonSuanFaTxt = sixteenInitonsIcdErosionPixGateJTextField.getText().trim();
}
});
sixteenInitonsIcdErosionPixGateJTextField.setText("10");
appInThisClass.app_S.sixteenInitonSuanFaTxt = "10";
this.add(sixteenInitonsIcdErosionPixGate);
this.add(sixteenInitonsIcdErosionPixGateJTextField);
JLabel sixteenInitonsIcderosionpixRatio = new JLabel("sixteenInitonsIcdpixRatio" + ":");
sixteenInitonsIcderosionpixRatio.setForeground(Color.white);
sixteenInitonsIcderosionpixRatio.setBounds(10 + 6 * 90, 20 + 25 * 2, 150, 25);
JTextField sixteenInitonsIcderosionpixRatioJTextField = new JTextField();
sixteenInitonsIcderosionpixRatioJTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 2, 50, 25);
sixteenInitonsIcderosionpixRatioJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanBiTxt = sixteenInitonsIcderosionpixRatioJTextField.getText().trim();
    }
});
sixteenInitonsIcdErosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonSuanBiTxt = sixteenInitonsIcderosionpixRatioJTextField.getText().trim();
    }
});
sixteenInitonsIcderosionpixRatioJTextField.setText("2");
appInThisClass.app_S.sixteenInitonSuanBiTxt = "2";
this.add(sixteenInitonsIcderosionpixRatio);
this.add(sixteenInitonsIcderosionpixRatioJTextField);
JLabel sixteenInitonsIcderosion像素精度 = new JLabel("sixteenInitonsIcd像素精度" + ":");
sixteenInitonsIcderosion像素精度.setForeground(Color.white);
sixteenInitonsIcderosion像素精度.setBounds(10 + 6 * 90, 20 + 25 * 3, 150, 25);
JTextField sixteenInitonsIcderosion像素精度JTextField = new JTextField();
sixteenInitonsIcderosion像素精度JTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 3, 50, 25);
sixteenInitonsIcderosion像素精度JTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanJingTxt = sixteenInitonsIcderosion像素精度JTextField.getText().trim();
    }
});
```

```
});
sixteenInitonsIcdErosionPixGateJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonSuanJingTxt = sixteenInitonsIcderosion像素精度JTextField.getText().trim();
    }
});
sixteenInitonsIcderosion像素精度JTextField.setText("8");
appInThisClass.app_S.sixteenInitonSuanJingTxt = "8";
this.add(sixteenInitonsIcderosion像素精度);
this.add(sixteenInitonsIcderosion像素精度JTextField);
JLabel sixteenInitonsIcderosion像素PCA = new JLabel("sixteenInitonsIcd像素PCA" + ":");
sixteenInitonsIcderosion像素PCA.setForeground(Color.white);
sixteenInitonsIcderosion像素PCA.setBounds(10 + 6 * 90, 20 + 25 * 4, 150, 25);
JTextField sixteenInitonsIcderosion像素PCAJTextField = new JTextField();
sixteenInitonsIcderosion像素PCAJTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 4, 50, 25);
sixteenInitonsIcderosion像素PCAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanPCATxt = sixteenInitonsIcderosion像素PCAJTextField.getText().trim();
    }
});
sixteenInitonsIcderosion像素PCAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonSuanPCATxt = sixteenInitonsIcderosion像素PCAJTextField.getText().trim();
    }
});
sixteenInitonsIcderosion像素PCAJTextField.setText("1");
appInThisClass.app_S.sixteenInitonSuanPCATxt = "1";
this.add(sixteenInitonsIcderosion像素PCA);
this.add(sixteenInitonsIcderosion像素PCAJTextField);
JLabel sixteenInitonsIcderosion像素ICA = new JLabel("sixteenInitonsIcd像素ICA" + ":");
sixteenInitonsIcderosion像素ICA.setForeground(Color.white);
sixteenInitonsIcderosion像素ICA.setBounds(10 + 6 * 90, 20 + 25 * 5, 150, 25);
```



```
JTextField sixteenInitonsIcderosion像素ICAJTextField = new JTextField();
sixteenInitonsIcderosion像素ICAJTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 5, 50, 25);
sixteenInitonsIcderosion像素ICAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanICATxt = sixteenInitonsIcderosion像素ICAJTextField.getText().trim();
    }
});
sixteenInitonsIcderosion像素ICAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonSuanICATxt = sixteenInitonsIcderosion像素ICAJTextField.getText().trim();
    }
});
sixteenInitonsIcderosion像素ICAJTextField.setText("1");
appInThisClass.app_S.sixteenInitonSuanICATxt = "1";
this.add(sixteenInitonsIcderosion像素ICA);
this.add(sixteenInitonsIcderosion像素ICAJTextField);
JLabel sixteenInitonsIcderosion像素ECA = new JLabel("sixteenInitonsIcd像素ECA" + ":");
sixteenInitonsIcderosion像素ECA.setForeground(Color.white);
sixteenInitonsIcderosion像素ECA.setBounds(10 + 6 * 90, 20 + 25 * 6, 150, 25);
JTextField sixteenInitonsIcderosion像素ECAJTextField = new JTextField();
sixteenInitonsIcderosion像素ECAJTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 6, 50, 25);
sixteenInitonsIcderosion像素ECAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanECATxt = sixteenInitonsIcderosion像素ECAJTextField.getText().trim();
    }
});
sixteenInitonsIcderosion像素ECAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonSuanECATxt = sixteenInitonsIcderosion像素ECAJTextField.getText().trim();
    }
});
```

```

sixteenInitonsIcderosion像素ECAJTextField.setText("1");
appInThisClass.app_S.sixteenInitonSuanECATxt = "1";
this.add(sixteenInitonsIcderosion像素ECA);
this.add(sixteenInitonsIcderosion像素ECAJTextField);
JLabel sixteenInitonsIcderosion像素UPCA = new JLabel("sixteenInitonsIcd像素UPCA" + ":");
sixteenInitonsIcderosion像素UPCA.setForeground(Color.white);
sixteenInitonsIcderosion像素UPCA.setBounds(10 + 6 * 90, 20 + 25 * 7, 150, 25);
JTextField sixteenInitonsIcderosion像素UPCAJTextField = new JTextField();
sixteenInitonsIcderosion像素UPCAJTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 7, 50, 25);
sixteenInitonsIcderosion像素UPCAJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanUPCATxt = sixteenInitonsIcderosion像素UPCAJTextField.getText().trim();
    }
});
sixteenInitonsIcderosion像素UPCAJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonSuanUPCATxt = sixteenInitonsIcderosion像素UPCAJTextField.getText().trim();
    }
});
sixteenInitonsIcderosion像素UPCAJTextField.setText("45");
appInThisClass.app_S.sixteenInitonSuanUPCATxt = "45";
this.add(sixteenInitonsIcderosion像素UPCA);
this.add(sixteenInitonsIcderosion像素UPCAJTextField);
//////////权重
JLabel sixteenInitonsIcderosion权重scale = new JLabel("sixteenInitonsIcd权重scale" + ":");
sixteenInitonsIcderosion权重scale.setForeground(Color.white);
sixteenInitonsIcderosion权重scale.setBounds(10 + 6 * 90, 20 + 25 * 9, 150, 25);
JTextField sixteenInitonsIcderosion权重scaleJTextField = new JTextField();
sixteenInitonsIcderosion权重scaleJTextField.setBounds(10 + 6 * 90 + 110, 20 + 25 * 9, 50, 25);
sixteenInitonsIcderosion权重scaleJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanScaleTxt = sixteenInitonsIcderosion权重scaleJTextField.getText().trim();
    }
});
sixteenInitonsIcderosion权重scaleJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }

```

```

@Override
public void keyPressed(KeyEvent e) {
}

@Override
public void keyReleased(KeyEvent e) {
    appInThisClass.app_S.sixteenInitonSuanScaleTxt = sixteenInitonsIcderosion权重scaleJTextField.getText().trim();
}

});
sixteenInitonsIcderosion权重scaleJTextField.setText("1");
appInThisClass.app_S.sixteenInitonSuanScaleTxt = "1";
this.add(sixteenInitonsIcderosion权重scale);
this.add(sixteenInitonsIcderosion权重scaleJTextField);
/////
JLabel sixteenInitonsAcderosion权重scale = new JLabel("sixteenInitonsAcderosion权重scale" + ":");
sixteenInitonsAcderosion权重scale.setForeground(Color.white);
sixteenInitonsAcderosion权重scale.setBounds(10 + 4 * 90, 20 + 25 * 9, 150, 25);
JTextField sixteenInitonsAcderosion权重scaleJTextField = new JTextField();
sixteenInitonsAcderosion权重scaleJTextField.setBounds(10 + 4 * 90 + 110, 20 + 25 * 9, 50, 25);
sixteenInitonsAcderosion权重scaleJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonJianScaleTxt = sixteenInitonsAcderosion权重scaleJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion权重scaleJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonJianScaleTxt = sixteenInitonsAcderosion权重scaleJTextField.getText().trim();
    }
});
sixteenInitonsAcderosion权重scaleJTextField.setText("1");
appInThisClass.app_S.sixteenInitonJianScaleTxt = "1";
this.add(sixteenInitonsAcderosion权重scale);
this.add(sixteenInitonsAcderosion权重scaleJTextField);
/////
JLabel fourInitonsIcderosion权重scale = new JLabel("fourInitonsIcd蚀权重scale" + ":");
fourInitonsIcderosion权重scale.setForeground(Color.white);
fourInitonsIcderosion权重scale.setBounds(10 + 2 * 90, 20 + 25 * 9, 150, 25);
JTextField fourInitonsIcderosion权重scaleJTextField = new JTextField();
fourInitonsIcderosion权重scaleJTextField.setBounds(10 + 2 * 90 + 110, 20 + 25 * 9, 50, 25);
fourInitonsIcderosion权重scaleJTextField.addActionListener(new ActionListener() {

```

```

@Override
public void actionPerformed(ActionEvent arg0) {
    appInThisClass.app_S.fourInitonSuanScaleTxt = fourInitonsIcderosion权重scaleJTextField.getText().trim();
}
});
fourInitonsIcderosion权重scaleJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonSuanScaleTxt = fourInitonsIcderosion权重scaleJTextField.getText().trim();
    }
});
fourInitonsIcderosion权重scaleJTextField.setText("0.001");
appInThisClass.app_S.fourInitonSuanScaleTxt = "0.001";
this.add(fourInitonsIcderosion权重scale);
this.add(fourInitonsIcderosion权重scaleJTextField);
JLabel fourInitonsAcderosion权重scale = new JLabel("fourInitonsAcderosion权重scale" + ":");
fourInitonsAcderosion权重scale.setForeground(Color.white);
fourInitonsAcderosion权重scale.setBounds(10 + 0 * 90, 20 + 25 * 9, 150, 25);
JTextField fourInitonsAcderosion权重scaleJTextField = new JTextField();
fourInitonsAcderosion权重scaleJTextField.setBounds(10 + 0 * 90 + 110, 20 + 25 * 9, 50, 25);
fourInitonsAcderosion权重scaleJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonJianScaleTxt = fourInitonsAcderosion权重scaleJTextField.getText().trim();
    }
});
fourInitonsAcderosion权重scaleJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonJianScaleTxt = fourInitonsAcderosion权重scaleJTextField.getText().trim();
    }
});
fourInitonsAcderosion权重scaleJTextField.setText("1");
appInThisClass.app_S.fourInitonJianScaleTxt = "1";
this.add(fourInitonsAcderosion权重scale);

```

```
this.add(fourInitonsAcderosion权重scaleJTextField);

//file
//shaohou

JLabel fourInitonsIcderosiondeciphtringAddress = new JLabel("fourInitonsIcderosiondeciphtringAddress" + ":");
fourInitonsIcderosiondeciphtringAddress.setForeground(Color.white);
fourInitonsIcderosiondeciphtringAddress.setBounds(10 + 0 * 150, 20 + 25 * 13, 150, 50);
JTextField fourInitonsIcderosiondeciphtringAddressJTextField = new JTextField();
fourInitonsIcderosiondeciphtringAddressJTextField.setBounds(10 + 0 * 150 + 160, 20 + 25 * 13, 550, 50);
fourInitonsIcderosiondeciphtringAddressJTextField.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonSuanTxt = fourInitonsIcderosiondeciphtringAddressJTextField.getText().trim();
    }
});

fourInitonsIcderosiondeciphtringAddressJTextField.addKeyListener(new KeyListener() {

    @Override
    public void keyTyped(KeyEvent e) {
    }

    @Override
    public void keyPressed(KeyEvent e) {
    }

    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.fourInitonSuanTxt = fourInitonsIcderosiondeciphtringAddressJTextField.getText().trim();
    }
});

fourInitonsIcderosiondeciphtringAddressJTextField.setText("C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6.txt");
appInThisClass.app_S.fourInitonSuanTxt = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6.txt";
this.add(fourInitonsIcderosiondeciphtringAddress);
this.add(fourInitonsIcderosiondeciphtringAddressJTextField);

//

JLabel fourInitonsAcderosionDeciphtringAddress = new JLabel("fourInitonsAcderosiondeciphtringAddress" + ":");
fourInitonsAcderosionDeciphtringAddress.setForeground(Color.white);
fourInitonsAcderosionDeciphtringAddress.setBounds(10 + 0 * 150, 20 + 25 * 16, 150, 50);
JTextField fourInitonsAcderosionDeciphtringAddressJTextField = new JTextField();
fourInitonsAcderosionDeciphtringAddressJTextField.setBounds(10 + 0 * 150 + 160, 20 + 25 * 16, 550, 50);
fourInitonsAcderosionDeciphtringAddressJTextField.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.fourInitonJianTxt = fourInitonsAcderosionDeciphtringAddressJTextField.getText().trim();
    }
});

fourInitonsAcderosionDeciphtringAddressJTextField.addKeyListener(new KeyListener() {

    @Override
    public void keyTyped(KeyEvent e) {
    }

    @Override
```

```

public void keyPressed(KeyEvent e) {
}
@Override
public void keyReleased(KeyEvent e) {
    appInThisClass.app_S.fourInitonJianTxt = fourInitonsAcdErosionDeciphtringAddressJTextField.getText().trim();
}
});
fourInitonsAcdErosionDeciphtringAddressJTextField.setText("C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB5.txt");
appInThisClass.app_S.fourInitonJianTxt = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB5.txt";
this.add(fourInitonsAcdErosionDeciphtringAddress);
this.add(fourInitonsAcdErosionDeciphtringAddressJTextField);
//
//sixteenInitonsIcderosion地址
JLabel sixteenInitonsIcdErosionDeciphtringAddress = new JLabel("sixteenInitonsIcderosiondeciphtringAddress" + ":");
sixteenInitonsIcdErosionDeciphtringAddress.setForeground(Color.white);
sixteenInitonsIcdErosionDeciphtringAddress.setBounds(10 + 0 * 150, 20 + 25 * 19, 150, 50);
JTextField sixteenInitonsIcdErosionDeciphtringAddressJTextField = new JTextField();
sixteenInitonsIcdErosionDeciphtringAddressJTextField.setBounds(10 + 0 * 150 + 160, 20 + 25 * 19, 550, 50);
sixteenInitonsIcdErosionDeciphtringAddressJTextField.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent arg0) {
        appInThisClass.app_S.sixteenInitonSuanTxt = sixteenInitonsIcdErosionDeciphtringAddressJTextField.getText().trim();
    }
});
sixteenInitonsIcdErosionDeciphtringAddressJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonSuanTxt = sixteenInitonsIcdErosionDeciphtringAddressJTextField.getText().trim();
    }
});
sixteenInitonsIcdErosionDeciphtringAddressJTextField.setText("C:/Users/yaogu/Desktop/project/开源门户/deciphering/
F_DB6_suan_17.txt");
appInThisClass.app_S.sixteenInitonSuanTxt = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6_suan_17.txt";
this.add(sixteenInitonsIcdErosionDeciphtringAddress);
this.add(sixteenInitonsIcdErosionDeciphtringAddressJTextField);
//sixteenInitonsAcderosion地址
JLabel sixteenInitonsAcdErosionDeciphtringAddress = new JLabel("sixteenInitonsAcderosiondeciphtringAddress" + ":");
sixteenInitonsAcdErosionDeciphtringAddress.setForeground(Color.white);
sixteenInitonsAcdErosionDeciphtringAddress.setBounds(10 + 0 * 150, 20 + 25 * 22, 150, 50);
JTextField sixteenInitonsAcdErosionDeciphtringAddressJTextField = new JTextField();
sixteenInitonsAcdErosionDeciphtringAddressJTextField.setBounds(10 + 0 * 150 + 160, 20 + 25 * 22, 550, 50);
sixteenInitonsAcdErosionDeciphtringAddressJTextField.addActionListener(new ActionListener() {

```

```

@Override
public void actionPerformed(ActionEvent arg0) {
    appInThisClass.app_S.sixteenInitonJianTxt = sixteenInitonsAcdErosionDeciphtringAddressJTextField.getText().trim();
}
});
sixteenInitonsAcdErosionDeciphtringAddressJTextField.addKeyListener(new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }
    @Override
    public void keyPressed(KeyEvent e) {
    }
    @Override
    public void keyReleased(KeyEvent e) {
        appInThisClass.app_S.sixteenInitonJianTxt = sixteenInitonsAcdErosionDeciphtringAddressJTextField.getText().trim();
    }
});
sixteenInitonsAcdErosionDeciphtringAddressJTextField.setText("C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6_jian_17.txt");
appInThisClass.app_S.sixteenInitonJianTxt = "C:/Users/yaogu/Desktop/project/开源门户/deciphering/F_DB6_jian_17.txt";
this.add(sixteenInitonsAcdErosionDeciphtringAddress);
this.add(sixteenInitonsAcdErosionDeciphtringAddressJTextField);
}
}

```

O_V.org.tinos.listen

```

//Trif_DNA-master/C_L_O/O_V/org/tinos/listen/Hear.java
package O_V.org.tinos.listen;

```

```

import U_A.PEU.P.movie.AdataFrame;
//import edu.cmu.sphinx.api.Configuration;
//import edu.cmu.sphinx.api.SpeechResult;
//import edu.cmu.sphinx.api.StreamSpeechRecognizer;
//import edu.cmu.sphinx.decoder.adaptation.Stats;
//import edu.cmu.sphinx.decoder.adaptation.Transform;
import javax.sound.sampled.AudioFormat;
import javax.sound.sampled.AudioInputStream;
import javax.sound.sampled.AudioSystem;
import javax.sound.sampled.DataLine;
import javax.sound.sampled.LineUnavailableException;
import javax.sound.sampled.TargetDataLine;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
//import soundProcessor.SoundPlay;
/*

```

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

**/

```
public class Hear {  
    public AudioInputStream audioInputStream;  
    public int big;  
    TargetDataLine line;  
    Thread thread;  
    AudioFormat format;  
    public int times= 0;  
    public boolean stoped= false;  
    public AdataFrame adataFrame;  
    public Hear(){  
        audioInputStream= null;  
        adataFrame= new AdataFrame();  
        AudioFormat.Encoding encoding= AudioFormat.Encoding.PCM_SIGNED;  
        float rate= 19200f;  
        int sampleSize= 8;  
        @SuppressWarnings("unused")  
        String signedString= "signed";  
        boolean bigEndian= true;  
        int channels= 1;  
        format= new AudioFormat(encoding, rate, sampleSize,  
channels, (sampleSize/ 8)* channels, rate, bigEndian);  
    }  
    @SuppressWarnings({ "unused", "resource" })  
    public void listen() {  
        DataLine.Info info= new DataLine.Info(TargetDataLine.class, format);  
        if (!AudioSystem.isLineSupported(info)) {  
            return;  
        }  
        try {  
            line= (TargetDataLine)AudioSystem.getLine(info);  
            line.open(format, line.getBufferSize());  
        } catch (LineUnavailableException e) {  
            e.printStackTrace();  
        }  
        try {  
            ByteArrayOutputStream out= new ByteArrayOutputStream();  
            //OutputStream out= new OutputStream();  
            int frameSizeInBytes= format.getFrameSize();  
            int bufferLengthInFrames= line.getBufferSize()/(8);  
            int bufferLengthInBytes= bufferLengthInFrames* frameSizeInBytes;
```



```

//20230106-S_logger.Log.logger.info("" + "bufferLengthInBytes : "+ bufferLengthInBytes + " **");

int numBytesRead= 0;

//byte[] data= new byte[bufferLengthInBytes];

byte[] data= new byte[2400];

line.start();

//audioInputStream= new AudioInputStream(line);

for(int i=0;i<100;i++) {

if((numBytesRead= line.read(data, 0, bufferLengthInBytes))== -1) {

    break;

}

if(numBytesRead<= 0) {

    break;

}

out.write(data, 0, numBytesRead);

}

byte[] audioBytes= out.toByteArray();

ByteArrayInputStream bais= new ByteArrayInputStream(audioBytes);

audioInputStream= new AudioInputStream(bais, format

, audioBytes.length/ frameSizeInBytes);

line.stop();

line.close();

line= null;

out.close();

stoped= true;

//print

//    DetaThread.sleepDeta(10);

//    SoundPlay soundPlay= new SoundPlay();

//    //20230106-S_logger.Log.logger.info("" + "speak");

//    soundPlay.Play(audioInputStream);

//Configuration configuration= new Configuration();

//configuration.setAcousticModelPath("resources:/edu/cmu/sphinx/models/en-us/en-us");

//configuration.setDictionaryPath("resources:/edu/cmu/sphinx/models/en-us/cmudict-en-us.dict");

//configuration.setLanguageModelPath("resources:/edu/cmu/sphinx/models/en-us/en-us.lm.bin");

//StreamSpeechRecognizer recognizer= new StreamSpeechRecognizer(configuration);

//to file

File file= new File("D:\\Users\\Lenovo\\eclipse-workspace\\DETA_YLJ1.0.0\\resources\\edu\\cmu\\sphinx\\

\\10001-90210-01803.wav");

InputStream stream= new FileInputStream(file);

stream.skip(44);

//read file

//Stats stats= recognizer.createStats(1);

//Transform transform= stats.createTransform();

// recognizer.setTransform(transform);

//recognizer.startRecognition(stream);

//recognizer.startRecognition(audioInputStream);

//SpeechResult result;

//while ((result= recognizer.getResult()) != null) {

//    System.out.format("Hypothesis: %s\\n", result.getHypothesis());

```

```

        //}

        //recognizer.stopRecognition();

    } catch (IOException e) {

        e.printStackTrace();

    }

}

public static void main(String[] argv) {

    Hear hear= new Hear();

    hear.listen();

}

}

//Trif_DNA-master/C_L_O/O_V/org/tinos/listen/HearTest.java
//Trif_DNA-master/C_L_O/O_V/org/tinos/listen/ReadToWav.java
package O_V.org.tinos.listen;

import U_A.PEU.P.movie.AdataFrame;

//import edu.cmu.sphinx.api.Configuration;

//import edu.cmu.sphinx.api.SpeechResult;

//import edu.cmu.sphinx.api.StreamSpeechRecognizer;

//import edu.cmu.sphinx.decoder.adaptation.Stats;

//import edu.cmu.sphinx.decoder.adaptation.Transform;

import javax.sound.sampled.AudioFileFormat;

import javax.sound.sampled.AudioFormat;

import javax.sound.sampled.AudioInputStream;

import javax.sound.sampled.AudioSystem;

import javax.sound.sampled.DataLine;

import javax.sound.sampled.LineUnavailableException;

import javax.sound.sampled.TargetDataLine;

import javax.sound.sampled.UnsupportedAudioFileException;

import java.io.ByteArrayInputStream;

import java.io.ByteArrayOutputStream;

import java.io.File;

import java.io.FileInputStream;

import java.io.IOException;

import java.io.InputStream;

//import soundProcessor.SoundPlay;

/*

* 个人著作权人, 作者 罗瑶光, 浏阳

* yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,

* (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)

* 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-

* 430181198505250014, G24402609, EB0581342

* 204925063, 389418686, F2406501, 0626136

* 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层

*/

public class ReadToWav {

    public AudioInputStream audioInputStream;

    public int big;

    TargetDataLine line;

```

```
Thread thread;

AudioFormat format;

public int times = 0;

public boolean stoped = false;

public AdataFrame adataFrame;

public void saveToFile(AudioFileFormat.Type type) {
    // audioInputStream.reset();
    File file = new File("D:\\temp\\temp.wav");
    try {
        if (AudioSystem.write(audioInputStream, type, file) == -1) {
            throw new IOException("error luo");
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

@SuppressWarnings("unused")
public ReadToWav() {
    audioInputStream = null;
    adataFrame = new AdataFrame();
    AudioFormat.Encoding encoding = AudioFormat.Encoding.PCM_SIGNED;
    float rate = 16000f;
    int sampleSize = 16;
    String signedString = "signed";
    boolean bigEndian = false;
    int channels = 1;

    format = new AudioFormat(encoding, rate, sampleSize, channels,
(sampleSize / 8) * channels, rate, bigEndian);
}

public String listen() {
    DataLine.Info info = new DataLine.Info(TargetDataLine.class, format);
    if (!AudioSystem.isLineSupported(info)) {
        return "";
    }
    try {
        line = (TargetDataLine) AudioSystem.getLine(info);
        line.open(format, line.getBufferSize());
    } catch (LineUnavailableException e) {
        e.printStackTrace();
    }
    try {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        // OutputStream out= new OutputStream();
        int frameSizeInBytes = format.getFrameSize();
        int bufferLengthInFrames = line.getBufferSize() / (8);
        int bufferLengthInBytes = bufferLengthInFrames * frameSizeInBytes;
```

```

        // 20230106-S_logger.Log.logger.info("'" + "bufferLengthInBytes : "
        // + bufferLengthInBytes + " *");
        int numBytesRead = 0;
        // byte[] data= new byte[bufferLengthInBytes];
        byte[] data = new byte[bufferLengthInBytes * 2 / 20];
        line.start();
        // audioInputStream= new AudioInputStream(line);
        for (int i = 0; i < 40; i++) {
    for (int j = 0; j < 20; j++) {
            if ((numBytesRead = line.read(data, 0, data.length)) > 0) {
                out.write(data, 0, numBytesRead);
            }
        }
        }
        byte[] audioBytes = out.toByteArray();
        ByteArrayInputStream bais = new ByteArrayInputStream(audioBytes);
        audioInputStream = new AudioInputStream(bais, format,
        audioBytes.length / frameSizeInBytes);
        line.stop();
        line.close();
        line = null;
        out.close();
        stoped = true;
        saveToFile(AudioFileFormat.Type.WAVE);
        // print
// Configuration configuration = new Configuration();
// configuration.setAcousticModelPath(
//     "resources:/edu/cmu/sphinx/models/en-us/en-us");
// configuration.setDictionaryPath(
//     "resources:/edu/cmu/sphinx/models/en-us/cmudict-en-us.dict");
// configuration.setLanguageModelPath(
//     "resources:/edu/cmu/sphinx/models/en-us/en-us.lm.bin");
// StreamSpeechRecognizer recognizer
//     = new StreamSpeechRecognizer(configuration);
// //to file
        File file = new File("D:\\temp\\temp.wav");
        audioInputStream = AudioSystem.getAudioInputStream(file);
        InputStream stream = new FileInputStream(file);
        stream.skip(44);
// Stats stats = recognizer.createStats(3);
// Transform transform = stats.createTransform();
// recognizer.setTransform(transform);
// recognizer.startRecognition(stream);
// SpeechResult result;
// String output = "";
// while ((result = recognizer.getResult()) != null) {
//     output += " " + result.getHypothesis();

```

```
//    }

//    recognizer.stopRecognition();

//    return output;

        } catch (IOException | UnsupportedAudioFileException e) {

            e.printStackTrace();

        }

        return "";

    }

    public static void main(String[] argv) throws LineUnavailableException {

        // 20230106-S_logger.Log.logger.info("" + new ReadToWav().listen());

    }

}

/Trif_DNA-master/C_L_O/O_V/org/tinos/listen/ReadToWavTest.java

O_V.OSA.shell

/Trif_DNA-master/C_L_O/O_V/OSA/shell/Const.java
package O_V.OSA.shell;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Const{

//    public final static String SET_ROOT= "setRoot";

//    public final static String BASE_NAME= "baseName";

//    public final static String TABLE_NAME= "tableName";

//    public final static String SELECT= "select";

//    public final static String CREATE= "create";

//    public final static String DELETE= "delete";

//    public final static String DROP= "drop";

//    public final static String INSERT= "insert";

//    public final static String UPDATE= "update";

//    public final static String CONDITION= S_ShellETL.SHELL_ETL_CONDITION;

//    public final static String LESS_THAN_AND_EQUAL_TO= "<=";

//    public final static String EQUAL_TO= "==" ;

//    public final static String LESS_THAN= "<";

//    public final static String GREAT_THAN= ">";

//    public final static String GREAT_THAN_AND_EQUAL_TO= ">=";

//    public final static String NOT_EQUAL_TO= "<>";

//    public final static String IN= "in";

//    public final static String NOT_IN= "!in";

//    public final static String EQUALS= "equal";

//    public final static String NOT_EQUALS= "!equal";

//    public final static String JOIN= "join";
```

```
//      public final static String RELATION= "relation";
//      public final static String AS= "|as|";
//      public final static String UP_TO= "|~|";
//      public final static String WITH_AGGREGATION= "aggregation";
//      public final static String GET_COLUMNS= "getColumns";
//      public final static String CHANGES_COLUMN_NAME= "changeColumnName";
//      public final static String COLUMN_NAME= "columnName";
//      public final static String COLUMN_VALUE= "columnValue";
//      public final static String SEMICOLON= ";";
//      public final static String COLON= ":";
}

//Trif_DNA-master/C_L_O/O_V/OSA/shell/Create.java
package O_V.OSA.shell;

/*
 * 个人著作权人, 作者 罗瑶光, 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */

public class Create{
//动机: 准备将下面的plsqli翻译成orm, 省的养疗经的QueryMode太固定, 上手修改麻烦。
//      String plsqli= "setRoot:C:/DetaDB";
//      plsqli+= "baseName:ZYY";
//      plsqli+= "tableName:zybc:create;" +
//          "columnName:pk.ID:string;" +
//          "columnName:uk:打分:string;" +
//          "columnName:uk:中药名称:string;" +
//          "columnName:uk:笔记原文:string;" +
//          "columnName:uk:功效:string;" +
//          "columnName:uk:风险规避:string;" +
//          "columnName:uk:用量:string;" +
//          "columnName:uk:性味:string;" +
//          "columnName:uk:经脉:string;" +
//          "columnName:uk:中医馆药理:string;" +
//          "columnName:uk:经解:string;" +
//          "columnName:uk:崇源:string;" +
//          "columnName:uk:愚按:string;" +
//          "columnName:uk:搭配:string;" +
//          "columnName:uk:常见药:string";
//这个函数用于确定Deta PLSQL的Root like: 'c:/dsdsd' etc
String PLSearch= "";
public String startAtRootDir(String rootAddress) {
    PLSearch= "setRoot:"+ rootAddress+ ";";
    return PLSearch;
}
}
```

```

    public String withBaseName(String baseName) {
        PLSearch+= "baseName:"+ baseName+ ";";
        return PLSearch;
    }

    public String withTableCreate(String tableName) {
        PLSearch+= "tableName:"+ tableName+ ":create;";
        return PLSearch;
    }

    public String withColumnName(String culumnName, String keyType, String dataFormat) {
        PLSearch+= "culumnName:"+ keyType+ ":"+ culumnName+ ":"+ dataFormat+ ";";
        return PLSearch;
    }
}

//Trif_DNA-master/C_L_O/O_V/OSA/shell/CreateTest.java
//Trif_DNA-master/C_L_O/O_V/OSA/shell/PL_XA_C.java
package O_V.OSA.shell;

import java.io.IOException;
import ME.VPC.M.app.App;
import S_A.pheromone.IMV_SQI;

//稍后将DMA文件与内存操作替换成 jtable表内存操作 罗瑶光
/*
 * 个人著作权人, 作者 罗瑶光. 浏阳
 * yaoguanguo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
 * (lyg.tin@gmail.com2018年回国后因国内G网屏蔽不再使用)
 * 15116110525-浏阳德塔软件开发有限公司-创始人-法人-总经理-永久非盈利-
 * 430181198505250014, G24402609, EB0581342
 * 204925063, 389418686, F2406501, 0626136
 * 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲一段路 208号 阳光家园别墅小区 第十栋别墅 第三层
 */
public interface PL_XA_C {
    public String getPLSearch();
    public void I_PLSearch(String pLSearch);
    public PL_XA_C withTableCreate(String tableName);
    public PL_XA_C withTableDelete(String tableName);
    public PL_XA_C withTableInsert(String tableName);
    public PL_XA_C withTableUpdate(String tableName);
    public PL_XA_C withTableSelect(String tableName);
    public PL_XA_C getCulums();
    public PL_XA_C startAtRootDir(String rootAddress);
    public PL_XA_C withBaseName(String baseName);
    public PL_XA_C withCondition(String conditionType);
    public PL_XA_C let(String leftSet);
    public PL_XA_C lessThanAndEqualTo(String compareSet);
    public PL_XA_C equalTo(String compareSet);
    public PL_XA_C lessThan(String compareSet);

```

```
public PL_XA_C greatThan(String compareSet);
public PL_XA_C greatThanAndEqualTo(String compareSet);
public PL_XA_C notEqualTo(String compareSet);
public PL_XA_C in(String compareSet);
public PL_XA_C notIn(String compareSet);
public PL_XA_C equals(String compareSet);
public PL_XA_C notEquals(String compareSet);
public PL_XA_C innerJoinWithTable(String baseName, String tableName);
public PL_XA_C withRelation(String relationType);
public PL_XA_C as(String compareSet);
public PL_XA_C upTo(String compareSet);
public PL_XA_C withAggregation(String aggregationType);
public PL_XA_C changeColumnName(String newColumnName, String oldColumnName);
public PL_XA_C withColumnName(String columnName, String dataType);
public PL_XA_C withColumnValue(String columnName, String columnValue);
public PL_XA_C checkErrors(String string);
public PL_XA_C fixErrors(String string);
public PL_XA_C finalE(boolean b, App NE) throws InterruptedException, IOException;
public IMV_SQI returnAsMap();
public PL_XA_C checkAndFixPlSearchGrammarErrors();
public PL_XA_C checkAndFixSystemEnvironmentErrors();
public PL_XA_C withTableDrop(String tabKey);
////根据tinshell002 文档的中文语法关键字 将PLORM 也中文化
//全部用拼音，先 之后矫正再 中文
public PL_XA_C ways(String ways);
public PL_XA_C conditions(String conditions);
public PL_XA_C colorTags(String yanSeLeiXing);
public PL_XA_C parsering(String luoJiLeiXing);
public PL_XA_C SortingString(String paiXuLeiXing);
public PL_XA_C scaleSearching(String searchContent);
public PL_XA_C execPLETL(String jieDianMing);
public PL_XA_C execPLTCP(String biaoLieMing);
public PL_XA_C tableConjunction(String zhiXingLuoJi);
public PL_XA_C tableCombination(String zhiXingLuoJi);
public PL_XA_C tableDelete(String zhiXingLuoJi);
public PL_XA_C webRequest(String duanKouHao);
public PL_XA_C getColumnName(String lieBiaoMing);
//。 。 。 。
//。 。
//。 。 。
//。
//目前用不到PLORM, 优先级稍后, 先继续设计更多 TINSHELL 命令
// 操作
// 条件为
// 获取表名
// 获取列名
// 进行分词
```